

Theory of Computation Notes

Contents

1	Sets, Logic, and Proof	1
1.1	Sets, Sequences, Functions, and Relations	1
1.2	Strings, Alphabets, and Languages	7
1.3	Logic, Quantifiers, and Boolean Connectives	13
1.4	Methods of Proof	19
1.5	Induction and the Well-Ordering Principle	25
2	Graph Theory for Computation	31
2.1	Graphs, Digraphs, and Degrees	31
2.2	Paths, Cycles, Connectivity, and Components	37
2.3	Trees and Spanning Trees	42
2.4	Bipartite Graphs, Matchings, and Coloring	48
2.5	Graphs as Computational Objects	54
3	Counting and Combinatorics	61
3.1	The Basic Counting Principles and Pigeonhole	61
3.2	Permutations, Combinations, and Binomial Coefficients	67
3.3	Inclusion-Exclusion	72
3.4	Recurrences and Generating Functions	78
3.5	Counting for Computation	85
4	Finite Automata and the Regular Languages	91
4.1	Deterministic Finite Automata	91
4.2	The Regular Operations and the Class of Regular Languages	96
4.3	Nondeterministic Finite Automata	102
4.4	Equivalence of NFAs and DFAs: the Subset Construction	108
4.5	Closure of the Regular Languages under the Regular Operations	115
5	Regular Expressions and Nonregularity	121
5.1	Regular Expressions	121
5.2	Kleene's Theorem: Equivalence of Regex and Finite Automata	126
5.3	The Pumping Lemma for Regular Languages	133
5.4	Proving Nonregularity	138
5.5	Myhill–Nerode, the Syntactic Monoid, and Minimization	143
6	Context-Free Grammars and Languages	151
6.1	Context-Free Grammars and Derivations	151
6.2	Designing Grammars and Ambiguity	156
6.3	Chomsky Normal Form	161
6.4	Regular Languages Are Context-Free; Closure Properties	167
6.5	Applications: Expressions, Balanced Strings, and Parsing	173

7	Pushdown Automata	180
7.1	Pushdown Automata: Model and Computation	180
7.2	Equivalence of PDAs and CFGs	185
7.3	The Pumping Lemma for Context-Free Languages	191
7.4	Proving Languages Non-Context-Free	197
7.5	Deterministic PDAs and the Language Hierarchy	202
8	Turing Machines	208
8.1	The Turing Machine Model	208
8.2	Designing Turing Machines	213
8.3	Robustness: Multitape, Nondeterministic, and Enumerators	218
8.4	The Church–Turing Thesis, Encodings, and the Universal TM	226
8.5	Decidable and Turing-Recognizable Languages	231
9	Decidability	238
9.1	Decidable Problems about Automata and Grammars	238
9.2	The Acceptance Problem and Recognizability	244
9.3	Diagonalization and the Uncountability of Languages	249
9.4	Undecidability of the Acceptance Problem	255
9.5	Turing-Unrecognizable Languages	260
10	Reducibility and Undecidability	266
10.1	Undecidability via Reductions	266
10.2	Mapping Reducibility	272
10.3	Proving Undecidability and Unrecognizability with $\leq_m$	277
10.4	Rice’s Theorem	284
10.5	The Post Correspondence Problem	289
11	Advanced Computability	296
11.1	The Recursion Theorem and Self-Reference	296
11.2	Applications of the Recursion Theorem	301
11.3	Turing Reducibility and Oracle Machines	306
11.4	The Arithmetic Hierarchy	312
11.5	Decidability of Logical Theories and Kolmogorov Complexity	318
12	Time Complexity	326
12.1	Measuring Time; Big-O and the Class TIME	326
12.2	The Class P	332
12.3	The Class NP and Verifiers	337
12.4	NP Problems and the P vs NP Question	343
12.5	Polynomial-Time Reducibility	349
13	NP-Completeness	355
13.1	NP-Completeness: Definition and Significance	355
13.2	The Cook–Levin Theorem	361
13.3	3SAT and Reductions to Graph Problems	368
13.4	More NP-Complete Problems	374
13.5	Coping with NP-Completeness	381
14	Space Complexity	388
14.1	Space, PSPACE, and Savitch’s Theorem	388
14.2	PSPACE-Completeness and TQBF	393
14.3	Games and Generalized Geography	400
14.4	The Classes L and NL	406
14.5	NL-Completeness and $NL = coNL$	412

15 Intractability	420
15.1 The Hierarchy Theorems	420
15.2 Relativization and the Limits of Diagonalization	426
15.3 Approximation Algorithms	433
15.4 Probabilistic Computation	439
15.5 Interactive Proofs and the Big Picture	445
16 Algebraic Methods	452
16.1 Monoids and Machines: Recognition by Finite Monoids	452
16.2 Group Actions and Counting: Burnside and Pólya	459
16.3 Generating Functions for Computation	467
16.4 Expander and Cayley Graphs	473
16.5 Synthesis: Where Algebra, Combinatorics, and Graphs Meet Computation	480
A Integrated Framework	488
A.1 The Integrated Concept Map	488
A.2 A Ten-Week Integrated Study Plan	488
A.3 Twenty Bridge Problems	492
A.4 Ten Capstone Projects	495
A.5 The Concept Map in Mermaid	497

Chapter 1

Sets, Logic, and the Language of Proof

1.1 Sets, Sequences, Functions, and Relations

Before we can say anything precise about machines, languages, or proofs, we need a small, shared vocabulary for the raw materials: *sets* (unordered collections), *sequences* (ordered lists), *functions* (deterministic input-output rules), and *relations* (yes/no links between objects). The one idea to carry forward is that almost every object in this book is built from these four by stacking them: an automaton, for instance, is a sequence (a tuple) whose entries are sets and a function between sets. This section pins down the definitions and notation we will reuse on every page after it.

Sets and their operations

Definition 1.1.1 (Set, membership, subset). A *set* is an unordered collection of distinct objects called its *elements* (or *members*). We write $x \in A$ when x is an element of A and $x \notin A$ otherwise. A set may be given by *listing*, as in $A = \{2, 3, 5, 7\}$, or by a *defining rule* $A = \{x \mid P(x)\}$, read “the set of all x such that $P(x)$ holds.” Two sets are *equal* exactly when they have the same elements; order and repetition in a listing are irrelevant, so $\{2, 3, 3\} = \{3, 2\}$. The set with no elements is the *empty set* $\emptyset$. We say A is a *subset* of B , written $A \subseteq B$, if every element of A is also an element of B ; if in addition $A \neq B$ we call A a *proper subset* and write $A \subsetneq B$.

Because equality of sets is “same elements,” the standard way to prove $A = B$ is to prove the two inclusions $A \subseteq B$ and $B \subseteq A$ separately – a habit worth forming now, since it reappears constantly when we argue that two machines recognize the same language.

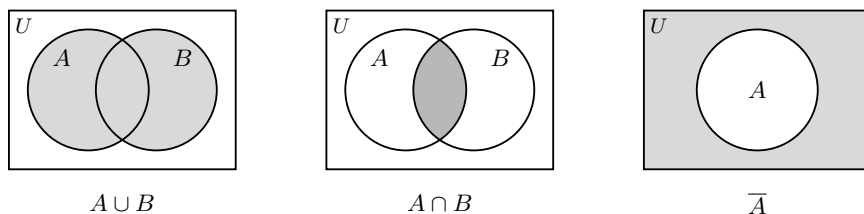
Definition 1.1.2 (Union, intersection, difference, complement). For sets A, B drawn from a fixed *universe* U (the set of all objects currently under discussion),

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}, \quad A \cap B = \{x \mid x \in A \text{ and } x \in B\},$$

$$A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}, \quad \overline{A} = U \setminus A = \{x \in U \mid x \notin A\}.$$

Sets A, B are *disjoint* when $A \cap B = \emptyset$. The number of elements of a finite set A is its *cardinality* $|A|$.

A *Venn diagram* draws each set as a region; overlap means shared elements. Three of these operations – union, intersection, and complement – are pictured below as shadings of those regions.



Definition 1.1.3 (Power set). The *power set* of A , written $\mathcal{P}(A)$, is the set of all subsets of A : $\mathcal{P}(A) = \{S \mid S \subseteq A\}$. If $|A| = n$ then $|\mathcal{P}(A)| = 2^n$, since each element is independently either in or out of a subset.

Proposition 1.1.4 (De Morgan's laws). For all $A, B \subseteq U$, $\overline{A \cup B} = \overline{A} \cap \overline{B}$ and $\overline{A \cap B} = \overline{A} \cup \overline{B}$.

Proof. We prove the first identity by mutual inclusion; the second is identical with the roles of “and”/“or” swapped. Take any $x \in \overline{A \cup B}$. By definition of complement, $x \notin A \cup B$, so x is in neither A nor B ; hence $x \in \overline{A}$ and $x \in \overline{B}$, that is $x \in \overline{A} \cap \overline{B}$. This shows $\overline{A \cup B} \subseteq \overline{A} \cap \overline{B}$. Conversely, if $x \in \overline{A} \cap \overline{B}$ then $x \notin A$ and $x \notin B$, so x belongs to neither, i.e. $x \notin A \cup B$, giving $x \in \overline{A \cup B}$. The two inclusions together yield equality. $\square$

Sequences, tuples, and the Cartesian product

Definition 1.1.5 (Sequence, tuple, Cartesian product). A *sequence* is an ordered list; we write it in parentheses, as $(7, 1, 7)$. Unlike a set, a sequence cares about *order* and *repetition*, so $(1, 2) \neq (2, 1)$ and $(7, 7) \neq (7)$. A finite sequence of length k is a *k-tuple*; a 2-tuple is an *ordered pair*. The *Cartesian product* of sets $A_1, \dots, A_k$ is the set of all *k-tuples* with one entry from each:

$$A_1 \times \dots \times A_k = \{(a_1, \dots, a_k) \mid a_i \in A_i \text{ for each } i\}.$$

We abbreviate the *k-fold* product of A with itself as A^k . If each A_i is finite then $|A_1 \times \dots \times A_k| = |A_1| \cdots |A_k|$.

Worked Example 1.1.6 (Listing a Cartesian product as a grid). With $A = \{1, 2, 3\}$ and $B = \{a, b\}$, list $A \times B$ and confirm its size.

Recognition. The phrase “all ordered pairs, first entry from A , second from B ” is the literal definition of a Cartesian product, so we enumerate row by row. The trigger is the word “pairs” together with two named source sets.

Solution. Lay the elements of A down the rows and the elements of B across the columns; each cell is one pair. There are $|A| \cdot |B| = 3 \cdot 2 = 6$ cells, matching Definition 1.1.5.

	a	b
1	(1, a)	(1, b)
2	(2, a)	(2, b)
3	(3, a)	(3, b)

Which idea, and why? A Cartesian product is exactly a two-axis grid of pairs, so reading it off a table guarantees we hit every combination once and the count is the product of the side lengths.

Check. The grid has 6 cells and no pair is repeated, agreeing with $|A \times B| = 6$. Swapping to $B \times A$ would give pairs like $(a, 1)$ – a different set, since order matters.

Functions

Definition 1.1.7 (Function, domain, codomain, range). A *function* $f: D \rightarrow C$ assigns to *each* element x of its *domain* D exactly one element $f(x)$ of its *codomain* C . The *range* (or *image*) of f is the set of values actually attained, $\text{ran}(f) = \{f(x) \mid x \in D\} \subseteq C$. We call f

- *injective* (one-to-one) if distinct inputs give distinct outputs: $x \neq y \implies f(x) \neq f(y)$;
- *surjective* (onto) if every codomain element is hit: $\text{ran}(f) = C$;
- *bijective* if it is both injective and surjective.

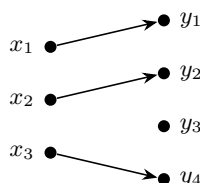
The two clauses “each x gets a value” and “exactly one value” are precisely the *determinism* we will demand of a transition function later: no input is left unhandled, and none has two answers. The picture of a function is a bundle of arrows leaving the domain dots, with the defining property that exactly one arrow leaves each left-hand dot.

Worked Example 1.1.8 (Reading injective, surjective, bijective off arrow diagrams). Classify each of the three maps $D \rightarrow C$ below as injective, surjective, both, or neither.

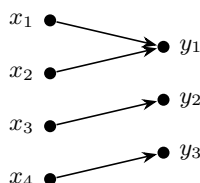
Recognition. The defining conditions are visual: injective means “no two arrows land on the same target,” surjective means “every target is hit,” and bijective is both. The trigger is being handed explicit input–output arrows, so we just inspect arrowheads.

Solution. Read each diagram against Definition 1.1.7.

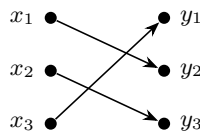
(a) injective, not onto



(b) onto, not injective



(c) bijective



In (a) every target receives at most one arrow (injective), but y_3 is unhit, so it is not onto. In (b) every target is hit (onto), but y_1 receives two arrows, so it is not injective. In (c) each target receives exactly one arrow, so the map is both – a bijection.

Which idea, and why? Injectivity is a statement about *collisions* at the codomain (at most one arrow per target) and surjectivity about *coverage* (at least one arrow per target); a bijection demands exactly one, which is why a bijection between finite sets forces $|D| = |C|$.

Check. In (c), $|D| = |C| = 3$, the equality a bijection requires. In (a), $|D| = 3 < 4 = |C|$, so onto was impossible from the start; in (b), $|D| = 4 > 3 = |C|$, so injectivity was impossible – the counts alone predicted both failures.

Worked Example 1.1.9 (A function given by a table: successor modulo 4). Let $f: \mathbb{Z}_4 \rightarrow \mathbb{Z}_4$ on $\mathbb{Z}_4 = \{0, 1, 2, 3\}$ be $f(n) = (n + 1) \bmod 4$. Tabulate f and decide whether it is a bijection.

Recognition. “Add one, then wrap” is a complete rule for every input in a finite domain, so a finite table captures the whole function. The trigger “ $f: \mathbb{Z}_4 \rightarrow \mathbb{Z}_4$ with an explicit formula” tells us to evaluate the formula at each of the four inputs.

Solution. Evaluate $(n + 1) \bmod 4$ at $n = 0, 1, 2, 3$:

n	0	1	2	3
$f(n)$	1	2	3	0

Every output value 0, 1, 2, 3 appears exactly once in the bottom row. Appearing *at least* once each is surjectivity; appearing *at most* once each is injectivity; both hold, so f is a bijection.

Which idea, and why? For a function on a *finite* set to itself, injective, surjective, and bijective all coincide – a single missing or repeated output value would break all three at once – so checking that the output row is a permutation of the input row settles every case.

Check. The bottom row 1, 2, 3, 0 is a rearrangement of 0, 1, 2, 3, confirming the permutation. The inverse map is $f^{-1}(n) = (n - 1) \bmod 4$, which exists precisely because f is a bijection.

Relations and equivalence relations

Definition 1.1.10 (Binary relation). A *binary relation* on a set A is any subset $R \subseteq A \times A$. We write $x R y$ to mean $(x, y) \in R$. A relation can be drawn as a *directed graph*: one node per element of A , and an arrow from x to y whenever $x R y$.

Definition 1.1.11 (Reflexive, symmetric, transitive; equivalence relation). A binary relation R on A is

- *reflexive* if $x R x$ for every $x \in A$ (every node has a self-loop);
- *symmetric* if $x R y$ implies $y R x$ (every arrow has a reverse arrow);

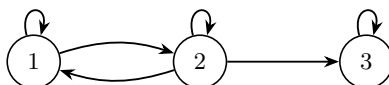
- *transitive* if $x R y$ and $y R z$ imply $x R z$ (every two-step path has a direct shortcut).

A relation that is reflexive, symmetric, and transitive is an *equivalence relation*.

Worked Example 1.1.12 (Spotting which of the three properties fail). On $A = \{1, 2, 3\}$ let $R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1), (2, 3)\}$. Draw its directed graph and determine which of reflexivity, symmetry, transitivity hold.

Recognition. The three properties are graph-shaped: self-loops (reflexive), paired arrows (symmetric), and shortcut arrows for two-step paths (transitive). The trigger is an explicitly listed relation, so we plot the arrows and look.

Solution. Plot a node per element and an arrow per listed pair. Self-loops at 1, 2, 3 come from $(1, 1), (2, 2), (3, 3)$; the double arrow $1 \leftrightarrow 2$ comes from $(1, 2)$ and $(2, 1)$; and a lone arrow $2 \rightarrow 3$ comes from $(2, 3)$.



Reflexive: yes – every node carries a self-loop. *Symmetric:* no – there is an arrow $2 \rightarrow 3$ but none $3 \rightarrow 2$. *Transitive:* no – $1 R 2$ and $2 R 3$ hold, yet the shortcut $1 R 3$ is missing. So R is reflexive only, and is *not* an equivalence relation.

Which idea, and why? Each property is a closure demand on the graph – a missing self-loop, a missing reverse arrow, or a missing shortcut breaks the corresponding property – so a single counterexample edge settles the question, no exhaustive search needed.

Check. Adding $(3, 2)$ would restore symmetry, but transitivity would still need $(1, 3)$ and $(3, 1)$; only when all such forced edges are present does the graph become an equivalence relation, splitting A into one block $\{1, 2, 3\}$.

Definition 1.1.13 (Equivalence class, partition). Let R be an equivalence relation on A . The *equivalence class* of x is $[x] = \{y \in A \mid x R y\}$. A *partition* of A is a collection of nonempty, pairwise disjoint subsets (*blocks*) whose union is A .

Theorem 1.1.14 (Equivalence relations and partitions correspond). *Let R be an equivalence relation on A . Then the distinct equivalence classes of R form a partition of A . Conversely, every partition of A arises from exactly one equivalence relation, namely “ x and y lie in the same block.”*

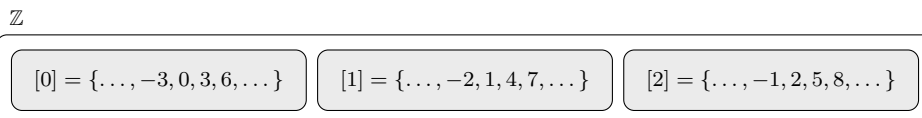
Proof. First we show the classes partition A . Each class is nonempty, since $x \in [x]$ by reflexivity, and their union is all of A because every x lies in its own class. It remains to show distinct classes are disjoint; equivalently, that if $[x] \cap [y] \neq \emptyset$ then $[x] = [y]$. Suppose $z \in [x] \cap [y]$, so $x R z$ and $y R z$. By symmetry $z R y$, and by transitivity $x R y$. Now take any $w \in [x]$, so $x R w$; symmetry gives $w R x$, and combined with $x R y$ transitivity gives $w R y$, i.e. (using symmetry again) $y R w$, so $w \in [y]$. Thus $[x] \subseteq [y]$, and the symmetric argument gives $[y] \subseteq [x]$, hence $[x] = [y]$.

Conversely, given a partition, define $x \sim y$ to mean “ x and y are in the same block.” This relation is reflexive (x shares its block with itself), symmetric (“same block” does not depend on order), and transitive (if x, y share a block and y, z share a block, then since blocks are disjoint these are the same block, so x, z share it). Its classes are exactly the blocks, and no other equivalence relation yields the same blocks, since the relation is forced by them. $\square$

Worked Example 1.1.15 (The classes of congruence modulo 3 partition $\mathbb{Z}$). Define $x \equiv y$ to mean $3 \mid (x - y)$ on the integers $\mathbb{Z}$. Verify it is an equivalence relation and exhibit its partition.

Recognition. “Differ by a multiple of 3” has the algebraic shape of a congruence, and congruences are the classic source of equivalence relations. The trigger “ $x - y$ is divisible by a fixed m ” points straight at reflexive/symmetric/transitive via simple divisibility facts.

Solution. *Reflexive:* $x - x = 0 = 3 \cdot 0$, divisible by 3. *Symmetric:* if $x - y = 3k$ then $y - x = 3(-k)$. *Transitive:* if $x - y = 3k$ and $y - z = 3\ell$ then $x - z = 3(k + \ell)$. So $\equiv$ is an equivalence relation. By Theorem 1.1.14 its classes partition $\mathbb{Z}$, and the class of x is determined by the remainder of x on division by 3, giving three blocks. We picture them as three disjoint cells covering $\mathbb{Z}$.



Which idea, and why? Congruence modulo m is an equivalence relation whose classes are the m residue classes; this is the very mechanism a counting-mod- m automaton will exploit later, where each class becomes one state.

Check. The three blocks are nonempty, pairwise disjoint, and every integer lands in exactly one according to its remainder mod 3 – the defining conditions of a partition. There are exactly $|\mathbb{Z}_3| = 3$ classes, as expected.

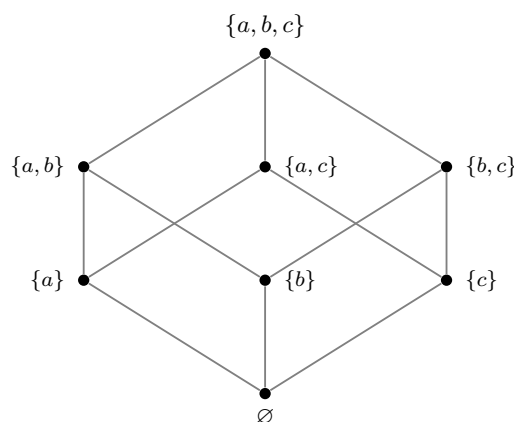
Ordering subsets: a Hasse diagram

Not every useful relation is an equivalence relation. The subset relation $\subseteq$ on a power set is reflexive and transitive but *antisymmetric* (if $A \subseteq B$ and $B \subseteq A$ then $A = B$), making it a *partial order*. We can draw a partial order compactly with a *Hasse diagram*: place smaller sets lower, larger sets higher, and draw an edge only for a *covering* step (one set immediately above another with nothing strictly between). The next example shows the power set of a three-element set, the same $2^3 = 8$ subsets counted in Definition 1.1.3.

Worked Example 1.1.16 (The Boolean lattice on $\{a, b, c\}$). Draw the Hasse diagram of $(\mathcal{P}(\{a, b, c\}), \subseteq)$.

Recognition. The objects are the subsets of a 3-element set ordered by inclusion; “ordered by inclusion, draw only covers” is the recipe for a Hasse diagram. The trigger is a partial order (reflexive, antisymmetric, transitive) on a small finite set.

Solution. List the 8 subsets by size: one empty set at the bottom, three singletons, three pairs, and the full set at the top. An edge joins S to T exactly when $T = S \cup \{x\}$ for a single new element x – a one-element growth step. This stacks into four levels.



Which idea, and why? For a finite partial order, a Hasse diagram shows *only* covering edges; all other comparabilities (such as $\emptyset \subseteq \{a, b\}$) are recovered as upward paths, which keeps the picture readable while losing no information.

Check. The diagram has 8 nodes ($= 2^3$, agreeing with $|\mathcal{P}(\{a, b, c\})|$) and is symmetric top-to-bottom: complementation $S \mapsto \bar{S}$ flips the picture upside down, sending each level k to level $3 - k$.

Method (How to read or build a function/relation diagram).

1. **Identify the kind of object.** A *function* $D \rightarrow C$ is one arrow out of every domain dot; a *relation* on A is any set of arrows among A 's dots. If you ever see a domain dot with zero or two outgoing arrows, it is not a function.
2. **For functions, test the two failure modes separately.** Injective fails iff two arrows share a target; surjective fails iff some codomain dot has no incoming arrow. Bijective means neither failure occurs.

3. **For relations, translate each property to a shape.** Reflexive = every dot has a self-loop; symmetric = every arrow is matched by its reverse; transitive = every length-two path has its shortcut. To disprove a property, exhibit one missing edge.
4. **If it is an equivalence relation, read off the partition.** The connected blocks of mutually related elements are the equivalence classes; check they are nonempty, disjoint, and cover A .
5. **If it is a partial order, draw the Hasse diagram.** Put minimal elements at the bottom, draw only covering edges, and recover all other comparabilities as upward paths.

Caution. Do not conflate the *codomain* C with the *range* $\text{ran}(f)$. The codomain is declared in advance as part of the function's type $f: D \rightarrow C$; the range is whatever values f actually produces. Surjectivity is precisely the statement that these two coincide. The same map can be onto one codomain and not onto a larger one, so “surjective” is only meaningful once the codomain is fixed.

Caution. “Symmetric” and “antisymmetric” are not opposites, and a relation can be neither. Symmetry says every arrow has a reverse; antisymmetry says no two *distinct* elements have arrows both ways (self-loops are still allowed). The relation $\{(1, 2), (2, 1), (2, 3)\}$ is neither symmetric (no reverse of $2 \rightarrow 3$) nor antisymmetric ($1 \rightarrow 2$ and $2 \rightarrow 1$), while $\{(1, 2), (2, 1)\}$ is symmetric but not antisymmetric. Equivalence relations are built from symmetry; partial orders are built from antisymmetry – mixing the two is a frequent slip when classifying a relation.

Caution. The empty set $\emptyset$ is a subset of *every* set, and it is an *element* of a power set, not the same thing as “no set at all.” In particular $\emptyset \in \mathcal{P}(A)$ always, so $\mathcal{P}(A)$ is never empty even when $A = \emptyset$: indeed $\mathcal{P}(\emptyset) = \{\emptyset\}$ has one element, not zero. Confusing $\emptyset$ (a set with no members) with $\{\emptyset\}$ (a set whose single member is the empty set) is a classic error that also breaks the count $|\mathcal{P}(A)| = 2^{|A|}$ at $|A| = 0$.

Exercises.

1. Let $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5\}$, with universe $U = \{1, 2, 3, 4, 5, 6\}$. Compute $A \cup B$, $A \cap B$, $A \setminus B$, $B \setminus A$, $\overline{A}$, and $\overline{A \cup B}$. Then verify $\overline{A \cup B} = \overline{A} \cap \overline{B}$ directly from your lists.
2. Let $A = \{x, y\}$ and $B = \{1, 2, 3\}$. List $A \times B$ and $B \times A$ explicitly, confirm both have $|A| \cdot |B|$ pairs, and explain in one sentence why $A \times B \neq B \times A$ as sets even though they have the same size. Draw the $A \times B$ grid.
3. List the power set $\mathcal{P}(\{p, q, r\})$ in full, grouped by size, and check that it has 2^3 members. Which of these eight subsets are subsets of $\{p, q\}$?
4. For a set A with $|A| = n$, prove that $|\mathcal{P}(A)| = 2^n$ by describing a bijection between subsets of A and length- n strings over $\{0, 1\}$ (the “indicator string” of a subset). Draw the arrow diagram of this bijection for $n = 2$, i.e. between $\mathcal{P}(\{a, b\})$ and $\{0, 1\}^2$.
5. Define $f: \{0, 1, 2, 3, 4\} \rightarrow \{0, 1, 2, 3, 4\}$ by $f(n) = (2n) \bmod 5$. Tabulate f , draw its arrow diagram, and decide whether it is injective, surjective, bijective. If it is a bijection, give f^{-1} as a table.
6. Give an explicit example of each, as an arrow diagram between small finite sets, or argue that none exists: (a) a function that is injective but not surjective; (b) one that is surjective but not injective; (c) a function $g: A \rightarrow A$ on a *finite* set A that is injective but not surjective. (For (c), state precisely why your answer is forced.)
7. Let $A = \{1, 2, 3\}$. Decide for each of the following relations on A which of reflexive, symmetric, transitive hold, drawing the directed graph of each: (a) $R_1 = \{(1, 1), (2, 2), (3, 3)\}$; (b) $R_2 = \{(1, 2), (2, 1), (1, 1), (2, 2)\}$; (c) $R_3 = \{(1, 2), (2, 3), (1, 3)\}$. Which, if any, are equivalence relations, and what partition does each equivalence relation induce?
8. On the set $L = \{a, ab, abc, b, bc, c\}$ of strings, define $x \preceq y$ to mean “ x is a prefix of y .” Argue that $\preceq$ is a partial order (reflexive, antisymmetric, transitive) and draw its Hasse diagram.
9. Define a relation $\sim$ on $\{0, 1\}^*$ by $x \sim y$ iff x and y have the same length. Prove $\sim$ is an equivalence relation, describe its equivalence classes, and say how many strings are in the class of a string of length n . (This “same length” partition reappears when we count strings in the next section.)

10. Let $f: A \rightarrow B$ be a function and let $S, T \subseteq A$. Prove that $f(S \cup T) = f(S) \cup f(T)$. Then show by a concrete two-point counterexample that $f(S \cap T) = f(S) \cap f(T)$ can *fail*, and identify which property of f (if assumed) would make it always hold.
11. *Find the bug.* A student writes: “The relation $R = \{(1, 1), (2, 2), (1, 2)\}$ on $\{1, 2\}$ is an equivalence relation because it is reflexive and transitive.” Draw the graph, state precisely which axiom fails, give the single pair that must be added to repair it, and name the partition of $\{1, 2\}$ that the repaired relation induces.
12. *Find the bug.* Someone argues: “Every relation that is symmetric and transitive is also reflexive: take any x , pick some y with $x R y$, then symmetry gives $y R x$ and transitivity gives $x R x$.” Exhibit a symmetric, transitive relation that is *not* reflexive, and pinpoint the exact unjustified step in the argument.

1.2 Strings, Alphabets, and Languages

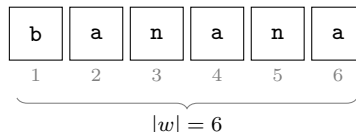
Everything a machine in this book reads, writes, or decides about is a *string*, and every problem we study is ultimately a *language* – a set of strings. This section pins down those two objects and the handful of operations on them (length, concatenation, reversal, the prefix and substring relations, and the language operations union, concatenation, and star). The one key idea is that strings are built by a single operation, concatenation, with the empty string ε as its neutral element; once that is clear, every later definition – regular expressions, the moves of an automaton, the contents of a Turing tape – is just structured talk about concatenation. We build up from a single symbol to alphabets, to strings, and finally to languages, computing small examples at each step.

Definition 1.2.1 (Alphabet and symbol). An *alphabet* is any nonempty finite set Σ . Its elements are called *symbols*. We write symbols in a typewriter font, e.g. 0, 1, a, b.

The two alphabets we use most are the binary alphabet $\Sigma = \{0, 1\}$ and small letter alphabets such as $\{a, b\}$. Finiteness and nonemptiness are both essential: a machine with infinitely many distinct input symbols could not be described in finite space, and an empty alphabet admits no nonempty input at all.

Definition 1.2.2 (String and its length). A *string* over Σ is a finite sequence of symbols from Σ , written by juxtaposition with no separators. If $w = w_1 w_2 \cdots w_n$ with each $w_i \in \Sigma$, the *length* of w is $|w| = n$, the number of symbol occurrences. The unique string of length 0 is the *empty string* ε . We write Σ^* for the set of *all* strings over Σ , and Σ^n for the set of strings of length exactly n , so that $\Sigma^0 = \{\varepsilon\}$.

A string is a sequence, so order and repetition both matter: over $\{a, b\}$ the strings **ab** and **ba** are different, and **aa** differs from **a**. This is exactly the opposite of a set, where $\{a, b\} = \{b, a\}$ and $\{a, a\} = \{a\}$. Picture a string as a row of labelled cells, one per symbol occurrence, read left to right.



The numbers under the cells stress that length counts *positions*, not *distinct symbols*: the string **banana** above has length 6 even though only three symbols (b, a, n) appear in it.

Definition 1.2.3 (Concatenation). If $x = x_1 \cdots x_m$ and $y = y_1 \cdots y_n$ are strings over Σ , their *concatenation* is $xy = x_1 \cdots x_m y_1 \cdots y_n$, the string formed by writing x and then y . We sometimes write it $x \cdot y$ when juxtaposition would be ambiguous. The *k-fold power* of x is $x^0 = \varepsilon$ and $x^k = x^{k-1}x$ for $k \geq 1$.

Proposition 1.2.4 (Concatenation is a monoid operation). *For all strings $x, y, z \in \Sigma^*$,*

$$(xy)z = x(yz), \quad \varepsilon x = x\varepsilon = x, \quad |xy| = |x| + |y|.$$

Proof. Write $x = x_1 \cdots x_m$, $y = y_1 \cdots y_n$, $z = z_1 \cdots z_p$. Both $(xy)z$ and $x(yz)$ are, by Definition 1.2.3 applied twice, the single sequence $x_1 \cdots x_m y_1 \cdots y_n z_1 \cdots z_p$ of length $m + n + p$; juxtaposition records no grouping, so the two are literally the same string. Prepending or appending the length-0 sequence ε adds no positions, giving $\varepsilon x = x\varepsilon = x$. Finally xy lists the m positions of x followed by the n positions of y , so it has $m + n$ positions, i.e. $|xy| = |x| + |y|$. $\square$

So Σ^* with concatenation is a monoid: an associative operation with identity ε . It is *not* commutative once $|\Sigma| \geq 2$ ($\mathbf{ab} \neq \mathbf{ba}$), which is why “concatenation order” is something to watch. The length map $w \mapsto |w|$ turns concatenation into addition – this is the workhorse identity behind almost every length argument later.

Definition 1.2.5 (Reversal). The *reversal* of $w = w_1 w_2 \cdots w_n$ is $w^{\mathcal{R}} = w_n \cdots w_2 w_1$, the same symbols in the opposite order. In particular $\varepsilon^{\mathcal{R}} = \varepsilon$.

Proposition 1.2.6 (Reversal flips a concatenation). For all $x, y \in \Sigma^*$, $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$.

Proof. Let $x = x_1 \cdots x_m$ and $y = y_1 \cdots y_n$. Then $xy = x_1 \cdots x_m y_1 \cdots y_n$, and reversing this length- $(m+n)$ sequence reads its symbols from the last position to the first: $y_n \cdots y_1 x_m \cdots x_1$. The first block $y_n \cdots y_1$ is $y^{\mathcal{R}}$ and the second block $x_m \cdots x_1$ is $x^{\mathcal{R}}$, so the whole is $y^{\mathcal{R}} x^{\mathcal{R}}$. $\square$

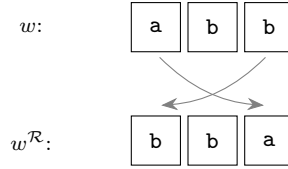
Definition 1.2.7 (Prefix, suffix, substring). Let $w \in \Sigma^*$. A string x is a *prefix* of w if $w = xz$ for some $z \in \Sigma^*$; it is a *suffix* if $w = zx$ for some z ; and it is a *substring* (or factor) if $w = uxv$ for some $u, v \in \Sigma^*$. A prefix (suffix, substring) is *proper* if it is not equal to w itself. We write $x \sqsubseteq w$ when x is a substring of w .

Every prefix and every suffix is a substring, but not conversely; and ε together with w are always among the (improper-or-proper) prefixes, suffixes, and substrings of w . We now compute with all of these.

Worked Example 1.2.8 (Length, powers, and reversal on a concrete string). Let $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ and $w = \mathbf{abb}$. Compute $|w|$, w^2 , w^0 , $w^{\mathcal{R}}$, and $(w^2)^{\mathcal{R}}$, and check $(w^2)^{\mathcal{R}} = (w^{\mathcal{R}})^2$ against Proposition 1.2.6.

Recognition. Each quantity is a *direct application* of a definition: length counts positions (Def. 1.2.2), a power is repeated concatenation (Def. 1.2.3), reversal flips the order (Def. 1.2.5). The trigger “evaluate these on a specific string” means: expand by the definitions and read off the answer.

Solution. $|w| = 3$ since w has three positions. The powers are $w^0 = \varepsilon$ and $w^2 = ww = \mathbf{abbabb} = \mathbf{abbabb}$, of length $6 = |w| + |w|$, matching Proposition 1.2.4. Reversal: $w^{\mathcal{R}} = \mathbf{bba}$. For $(w^2)^{\mathcal{R}}$ either reverse $\mathbf{abbabb}$ directly to get $\mathbf{bbabba}$, or use Proposition 1.2.6: $(ww)^{\mathcal{R}} = w^{\mathcal{R}} w^{\mathcal{R}} = \mathbf{bba bba} = \mathbf{bbabba}$.



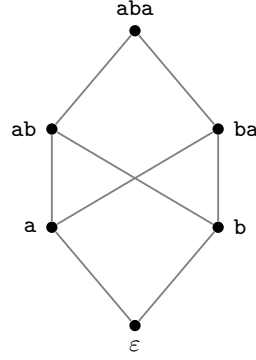
Which idea, and why? Reversal of a concatenation reverses the *pieces* and swaps their order (Prop. 1.2.6); on a single repeated block w the swap is invisible, so $(w^k)^{\mathcal{R}} = (w^{\mathcal{R}})^k$.

Check. Both routes give $\mathbf{bbabba}$, and indeed $(w^{\mathcal{R}})^2 = \mathbf{bba bba} = \mathbf{bbabba}$, confirming the identity.

Worked Example 1.2.9 (Listing prefixes, suffixes, and substrings; a substring poset). For $w = \mathbf{aba}$ over $\{\mathbf{a}, \mathbf{b}\}$, list all prefixes, all suffixes, and all substrings, and draw the substrings ordered by the substring relation $\sqsubseteq$.

Recognition. This is a *decomposition* task: a prefix is what sits to the left of a cut, a suffix what sits to the right, a substring any contiguous window (Def. 1.2.7). The trigger “list all” means slide a window over the positions and collect what you see, remembering that ε (the empty window) always qualifies.

Solution. Cutting $\mathbf{aba}$ after position 0, 1, 2, 3 gives the prefixes $\varepsilon, \mathbf{a}, \mathbf{ab}, \mathbf{aba}$. The parts to the right of the same cuts are the suffixes $\mathbf{aba}, \mathbf{ba}, \mathbf{a}, \varepsilon$. The substrings are all contiguous windows: the empty window ε ; length-1 windows $\mathbf{a}, \mathbf{b}$; length-2 windows $\mathbf{ab}, \mathbf{ba}$; and the whole string $\mathbf{aba}$. Note $\mathbf{a}$ occurs at two positions but is listed once – substrings form a *set*. Order them by “is a substring of”:



Which idea, and why? Prefixes and suffixes come from the $|w| + 1$ cut points; substrings come from all contiguous windows. The substring relation is a partial order (reflexive, antisymmetric, transitive), so the substrings of a fixed word form the Hasse diagram drawn above.

Check. There are 4 prefixes and 4 suffixes (always $|w| + 1$ of each), and the six substrings sit in the poset with ε at the bottom (a substring of everything) and **aba** at the top.

From strings to languages. A *language* is any set of strings over a fixed alphabet. The language is the thing a machine “decides”: membership in it is the yes/no question we ask of an input. Because languages are just sets, the ordinary set operations apply – but there are also two genuinely string-flavoured operations, *concatenation* and *star*, that have no analogue for plain sets.

Definition 1.2.10 (Language). A *language* over Σ is a subset $\mathcal{L} \subseteq \Sigma^*$. The two extreme languages are the *empty language* $\emptyset$ (no strings at all) and Σ^* itself (every string). The singleton-empty-string language $\{\varepsilon\}$ is a third, distinct, special case.

Caution. Keep $\emptyset$, $\{\varepsilon\}$, and ε apart. They are three different things: $\emptyset$ is a *language* with zero members; $\{\varepsilon\}$ is a *language* with exactly one member, the empty string; and ε is a *string*, not a language at all. We have $|\emptyset| = 0$ and $|\{\varepsilon\}| = 1$ as language sizes, while $|\varepsilon| = 0$ as a string length.

Definition 1.2.11 (Operations on languages). Let $A, B \subseteq \Sigma^*$ be languages. Their *union* $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$, *intersection* $A \cap B = \{w \mid w \in A \text{ and } w \in B\}$, and *complement* $\bar{A} = \Sigma^* \setminus A$ are the usual set operations. The string-aware operations are:

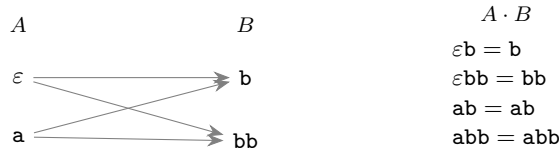
- *concatenation* $A \cdot B = \{xy \mid x \in A, y \in B\}$ (every left piece glued to every right piece);
- *star* $A^* = \{x_1x_2 \cdots x_k \mid k \geq 0, \text{ each } x_i \in A\}$, all finite concatenations of members of A , *including* the empty concatenation ($k = 0$), which is ε ;
- *plus* $A^+ = \{x_1 \cdots x_k \mid k \geq 1, \text{ each } x_i \in A\}$, the same but with $k \geq 1$;
- *reversal* $A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\}$.

Two relations to remember: $A^* = A^+ \cup \{\varepsilon\}$ always, and $A^+ = A^*$ exactly when $\varepsilon \in A$. Setting $A = \Sigma$ (the alphabet read as the language of its one-symbol strings) recovers the notation Σ^* from Definition 1.2.2: Σ^* really is the star of the alphabet.

Worked Example 1.2.12 (Computing a concatenation of two finite languages). Let $A = \{\varepsilon, a\}$ and $B = \{b, bb\}$ over $\{a, b\}$. Compute $A \cdot B$, then $B \cdot A$, and observe that they differ.

Recognition. Concatenation of languages is a *Cartesian-style pairing* (Def. 1.2.11): form every xy with $x \in A$ and $y \in B$. The trigger “concatenate two finite languages” means take all $|A| \cdot |B|$ glued pairs and then *deduplicate*, since the result is a set.

Solution. Glue each $x \in A$ to each $y \in B$:



So $A \cdot B = \{b, bb, ab, abb\}$. Going the other way, $B \cdot A = \{b\varepsilon, ba, bb\varepsilon, bba\} = \{b, ba, bb, bba\}$.

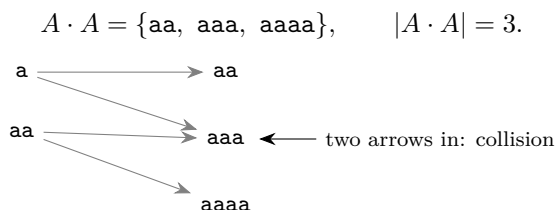
Which idea, and why? Language concatenation pairs *every* left string with *every* right string; because string concatenation is non-commutative, $A \cdot B \neq B \cdot A$ in general – here $ab \in A \cdot B$ but $ab \notin B \cdot A$.

Check. $|A \cdot B| = 4 = |A| \cdot |B|$ with no collisions; $A \cdot B$ and $B \cdot A$ share only $\{b, bb\}$, confirming order matters.

Worked Example 1.2.13 (When the product count drops: collisions in concatenation). Let $A = \{a, aa\}$ and $B = \{a, aa\}$ (so $B = A$). Compute $A \cdot A$ and explain why $|A \cdot A| < |A| \cdot |A|$.

Recognition. Same operation as before, but now the alphabet is a single letter, so different gluings can produce the *same* string – a *collision*. The trigger “count is smaller than the product” flags that the set quotient removes duplicates.

Solution. The four gluings are $a \cdot a = aa$, $a \cdot aa = aaa$, $aa \cdot a = aaa$, $aa \cdot aa = aaaa$. The middle two both equal aaa , so



Which idea, and why? The product rule $|A \cdot B| \leq |A| \cdot |B|$ is an *inequality*: equality holds only when the gluings are all distinct, which can fail over a small alphabet where $a \cdot aa$ and $aa \cdot a$ coincide.

Check. List the lengths: outputs have lengths 2, 3, 3, 4; the two length-3 outputs are the colliding pair, leaving three distinct strings.

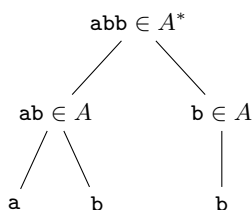
Worked Example 1.2.14 (Star of a finite language, layer by layer). Let $A = \{ab, b\}$ over $\{a, b\}$. Describe A^* layer by layer, and decide whether abb and aab are in A^* .

Recognition. Star is *all finite concatenations* of members of A (Def. 1.2.11). The trigger “is this string in A^* ?” becomes a *tiling* question: can the string be cut into a sequence of pieces each equal to ab or b ?

Solution. Build A^* in layers $A^0, A^1, A^2, \dots$, where A^k is the set of k -piece concatenations:

$$A^0 = \{\varepsilon\}, \quad A^1 = \{ab, b\}, \quad A^2 = \{abab, abb, bab, bb\}, \quad \dots$$

and $A^* = \bigcup_{k \geq 0} A^k$. A derivation tree makes the tiling of abb explicit:



So $abb = ab \cdot b$, a 2-piece tiling, hence $abb \in A^*$. But aab would have to start with a piece that is a prefix of aab ; neither ab nor b is, since aab begins aa . No tiling starts, so $aab \notin A^*$.

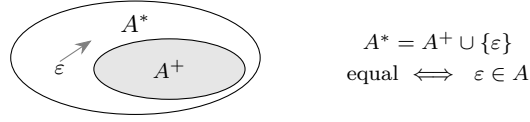
Which idea, and why? Membership in A^* is a left-to-right *exhaustive tiling*: peel off a prefix that lies in A , recurse on the rest, and accept iff some sequence of peels consumes the whole string and leaves ε .

Check. abb tiles as $(ab)(b)$ – accepted. aab has no first tile, so no tiling exists – rejected. And $\varepsilon \in A^*$ via the $k = 0$ empty tiling, even though $\varepsilon \notin A$.

Worked Example 1.2.15 (Star versus plus, and the empty-string subtlety). With $A = \{a\}$ and $C = \{\varepsilon, a\}$, compute A^*, A^+, C^*, C^+ and compare.

Recognition. This isolates the *only* difference between star and plus: whether the $k = 0$ (empty) concatenation is allowed (Def. 1.2.11). The trigger “does ε already belong to the base language?” decides whether star and plus coincide.

Solution. For $A = \{a\}$: the k -piece concatenations are a^k , so $A^* = \{\varepsilon, a, aa, \dots\} = \{a^k \mid k \geq 0\}$ and $A^+ = \{a^k \mid k \geq 1\}$; these differ only in ε . For $C = \{\varepsilon, a\}$: since $\varepsilon \in C$, gluing in extra copies of ε changes nothing, so already C^+ contains ε (take $k = 1$, piece ε), giving $C^* = C^+ = \{a^k \mid k \geq 0\} = A^*$.



Which idea, and why? A^* and A^+ differ at most by ε ; they are equal precisely when $\varepsilon \in A$, because then the $k = 1$ concatenation already supplies the empty string and the $k = 0$ case adds nothing new.

Check. $\varepsilon \in A^* \setminus A^+$ for $A = \{a\}$ (star and plus differ), while for $C = \{\varepsilon, a\}$ we have $\varepsilon \in C$, so $C^* = C^+$.

Worked Example 1.2.16 (Identities for the special languages $\emptyset$ and $\{\varepsilon\}$). Prove $A \cdot \emptyset = \emptyset$, $A \cdot \{\varepsilon\} = A$, and $\emptyset^* = \{\varepsilon\}$ for every language A .

Recognition. These are the *annihilator and identity laws* for language concatenation, the analogue of $x \cdot 0 = 0$ and $x \cdot 1 = x$. The trigger is the appearance of the two special languages $\emptyset$ and $\{\varepsilon\}$ as a concatenation factor.

Solution. By Definition 1.2.11, $A \cdot B = \{xy \mid x \in A, y \in B\}$.

- $A \cdot \emptyset$: there is *no* $y \in \emptyset$ to choose, so no pair xy can be formed; the set is empty, $A \cdot \emptyset = \emptyset$. (Even if $A = \emptyset$ as well, it is still $\emptyset$.)
- $A \cdot \{\varepsilon\}$: the only choice for y is ε , and $x\varepsilon = x$ by Proposition 1.2.4, so the products are exactly the strings of A : $A \cdot \{\varepsilon\} = A$.
- $\emptyset^*$: by definition star always includes the empty ($k = 0$) concatenation ε , and for $k \geq 1$ each piece would have to come from $\emptyset$, which is impossible; so the only member is ε , giving $\emptyset^* = \{\varepsilon\}$.

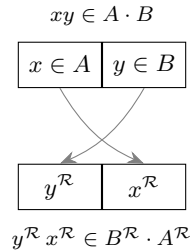
Which idea, and why? $\emptyset$ is the annihilator of concatenation (like 0 for $\times$) because it offers no right factor, while $\{\varepsilon\}$ is the identity (like 1) because its only factor leaves x unchanged; star never returns $\emptyset$ since the empty concatenation is always present.

Check. Contrast with union, where $\emptyset$ is the *identity*: $A \cup \emptyset = A$. So $\emptyset$ behaves like 0 for concatenation but like the additive identity for union – different roles, same symbol.

Worked Example 1.2.17 (Reversal distributes over union but flips a concatenation). For all languages A, B , show $(A \cup B)^{\mathcal{R}} = A^{\mathcal{R}} \cup B^{\mathcal{R}}$ and $(A \cdot B)^{\mathcal{R}} = B^{\mathcal{R}} \cdot A^{\mathcal{R}}$.

Recognition. Reversal of a language is reversal applied to each member (Def. 1.2.11); the question is how it interacts with $\cup$ and $\cdot$. The trigger is a structural identity “ $\mathcal{R}$ of an operation,” so we push $\mathcal{R}$ inward using the string-level law (Prop. 1.2.6).

Solution. For union, $w \in (A \cup B)^{\mathcal{R}}$ iff $w^{\mathcal{R}} \in A \cup B$ iff $w^{\mathcal{R}} \in A$ or $w^{\mathcal{R}} \in B$ iff $w \in A^{\mathcal{R}}$ or $w \in B^{\mathcal{R}}$ iff $w \in A^{\mathcal{R}} \cup B^{\mathcal{R}}$; the two sets have the same members. For concatenation, a typical member of $A \cdot B$ is xy with $x \in A, y \in B$, and by Proposition 1.2.6 its reversal is $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$ with $y^{\mathcal{R}} \in B^{\mathcal{R}}$ and $x^{\mathcal{R}} \in A^{\mathcal{R}}$; this is exactly a member of $B^{\mathcal{R}} \cdot A^{\mathcal{R}}$, and every member of $B^{\mathcal{R}} \cdot A^{\mathcal{R}}$ arises this way. Hence the order of the factors flips.



Which idea, and why? Reversal is a *set-of-images* operation, so it passes through union unchanged; against concatenation it inherits the string-level flip $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$, swapping the two language factors.

Check. Take $A = \{ab\}$, $B = \{cd\}$ over $\{a, b, c, d\}$. Then $A \cdot B = \{abcd\}$, whose reversal is $\{dcba\}$, and $B^{\mathcal{R}} \cdot A^{\mathcal{R}} = \{dc\} \cdot \{ba\} = \{dcba\}$ – they match.

Method (How to compute and reason about a language operation).

1. **Name the operation's defining set.** Write the right-hand set-builder from Definition 1.2.11: union/intersection compare membership; concatenation forms every glued pair xy ; star forms every finite list of pieces.
2. **For concatenation, pair exhaustively, then deduplicate.** Glue each $x \in A$ to each $y \in B$. Expect at most $|A| \cdot |B|$ strings; fewer if two gluings collide (common over a one-letter alphabet).
3. **For star, work in layers.** Compute $A^0 = \{\varepsilon\}$, $A^1 = A$, $A^2 = A \cdot A$, $\dots$; the answer is their union. To *test* membership of a single string, tile it left to right with pieces from A .
4. **Track ε deliberately.** Decide at each step whether ε is in the result: star always contains it; plus contains it iff $\varepsilon \in A$; concatenation contains it iff both factors contain ε .
5. **For an identity, prove two inclusions.** To show two languages are equal, show each is a subset of the other (" $w \in \text{LHS} \iff w \in \text{RHS}$ "), pushing reversal or concatenation through with Propositions 1.2.4 and 1.2.6.

Caution. Do not assume language concatenation is commutative or that the product count is exact. $A \cdot B$ and $B \cdot A$ usually differ (string concatenation is non-commutative), and $|A \cdot B|$ can be *strictly less* than $|A| \cdot |B|$ when distinct gluings collide to the same string. Always glue in a fixed order and then remove duplicates.

Caution. The star of a language *always* contains ε , regardless of whether $\varepsilon \in A$, because the empty ($k = 0$) concatenation is by definition a member of A^* . A frequent error is to write A^* for a language that should exclude ε ; use A^+ instead, and recall $A^+ = A^*$ only when ε is already in A .

Exercises.

1. Over $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ let $w = \mathbf{abba}$. Compute $|w|$, $w^{\mathcal{R}}$, w^2 , and list every prefix and every suffix of w . Which of $\mathbf{bb}$, $\mathbf{ba}$, $\mathbf{aba}$ are substrings of w ?
2. Let $\Sigma = \{0, 1\}$. Write out Σ^0 , Σ^1 , and Σ^2 explicitly, and say how many strings are in Σ^n in general. Then list the first six strings of Σ^* in shortlex order (by length, ties broken alphabetically).
3. For strings over $\{\mathbf{a}, \mathbf{b}\}$, decide whether each statement is true for *all* x, y , giving a one-line reason or a counterexample: (a) $|x^{\mathcal{R}}| = |x|$; (b) $xy = yx$; (c) $(x^{\mathcal{R}})^{\mathcal{R}} = x$; (d) if x is a prefix of y then $x^{\mathcal{R}}$ is a suffix of $y^{\mathcal{R}}$.
4. Let $A = \{\mathbf{a}, \mathbf{ba}\}$ and $B = \{\varepsilon, \mathbf{b}\}$ over $\{\mathbf{a}, \mathbf{b}\}$. Compute $A \cdot B$ and $B \cdot A$ as explicit sets, and confirm whether $|A \cdot B| = |A| \cdot |B|$ in each case. Draw the gluing diagram (each $x \in A$ joined to each $y \in B$) for $A \cdot B$.
5. Let $A = \{\mathbf{a}, \mathbf{aa}\}$ over $\{\mathbf{a}\}$. Compute $A^2 = A \cdot A$ and A^3 as sets, and explain in terms of *collisions* why $|A^3| < |A|^3$. (Hint: describe A^k by the lengths of strings it contains.)
6. Let $A = \{\mathbf{aa}, \mathbf{b}\}$ over $\{\mathbf{a}, \mathbf{b}\}$. Decide, with a left-to-right tiling argument, which of $\mathbf{aab}$, $\mathbf{aba}$, $\mathbf{baab}$, $\mathbf{aaab}$ lie in A^* . For each member, draw the derivation tree showing how it tiles into pieces of A .
7. Prove that for every language A one has $\emptyset \cdot A = \emptyset$ and $\{\varepsilon\} \cdot A = A$. State the analogous facts with the special language on the *right*, and say which (if any) change when the factors are swapped.
8. Prove or disprove, for all languages A, B, C over Σ : (a) $(A \cup B) \cdot C = (A \cdot C) \cup (B \cdot C)$; (b) $(A \cap B) \cdot C = (A \cdot C) \cap (B \cdot C)$. For the false one, give the smallest counterexample you can over $\{\mathbf{a}\}$.
9. Show that $(A^{\mathcal{R}})^{\mathcal{R}} = A$ and $(A^*)^{\mathcal{R}} = (A^{\mathcal{R}})^*$ for every language A . (For the second, use that reversal flips a concatenation (Proposition 1.2.6) on each k -piece product.)
10. Determine, with proof, exactly when $A^* = A^+$ for a language A , and exactly when $\varepsilon \in A \cdot B$. Give a single language A for which $A^* = A^+$ and a single one for which they differ.
11. *Find the bug.* A classmate claims " $A^* \cdot A^* = A^*$ holds for every language A , and also $A^+ \cdot A^+ = A^+$." One of these is always true and one can fail. Identify which is which, prove the true one, and give a concrete A over $\{\mathbf{a}\}$ where the false one breaks.

12. *Find the bug.* Someone writes: “Since $\varepsilon \in A^*$ always, and concatenating with ε changes nothing, we have $A \cdot A^* = A^*$ for every A .” Show the conclusion is false in general by testing $A = \{a\}$, and pinpoint the exact flaw in the reasoning. What is the correct relationship between $A \cdot A^*$ and A^+ ?

1.3 Logic, Quantifiers, and Boolean Connectives

Almost every statement we will prove about machines and languages is built from a handful of logical pieces: simple propositions glued together by *connectives* (and, or, not, implies) and ranged over by *quantifiers* (for all, there exists). The one key idea of this section is that the meaning of such a compound statement is completely determined by the truth values of its parts, so we can always compute it mechanically – by a truth table for the connectives, and by a fixed rule for pushing a negation through a quantifier. Mastering these moves now is what lets later sections say “ M accepts w iff $\hat{\delta}(q_0, w) \in F$ ” and then negate, specialize, or chain such statements without slipping.

Propositions and connectives

Definition 1.3.1 (Proposition and truth value). A *proposition* is a statement that is either *true* or *false* – never both, never neither. Its *truth value* is T (true) or F (false), often written 1 and 0. “3 is prime” is a proposition (value T); “ $x > 5$ ” is *not* a proposition on its own, because its value depends on x – it is a *predicate* (Definition 1.3.7).

Definition 1.3.2 (Boolean connectives). Let P, Q be propositions. The five basic *connectives* build new propositions whose value depends only on the values of P and Q :

- *negation* $\neg P$ (“not P ”) is true exactly when P is false;
- *conjunction* $P \wedge Q$ (“ P and Q ”) is true exactly when both are true;
- *disjunction* $P \vee Q$ (“ P or Q ,” inclusive) is true when at least one is true;
- *implication* $P \rightarrow Q$ (“if P then Q ”) is false *only* when P is true and Q is false; otherwise true;
- *biconditional* $P \leftrightarrow Q$ (“ P iff Q ”) is true exactly when P and Q have the same value.

Their meanings are fixed once and for all by the *truth table*

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

The implication row deserves a long stare: $P \rightarrow Q$ is true whenever P is false, no matter what Q is. A *vacuously true* implication like “if n is both even and odd, then $n = 0$ ” costs us nothing, and we will lean on this constantly (an empty machine accepts a string only “if” an impossible condition holds). We read the two halves of a biconditional as $P \rightarrow Q$ (“only if”) and $Q \rightarrow P$ (“if”); proving an “iff” means proving both arrows.

Definition 1.3.3 (Tautology, contradiction, logical equivalence). A compound proposition is a *tautology* if it is true under every assignment of values to its variables, and a *contradiction* if it is false under every assignment. Two compound propositions φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, if they have the same truth value under *every* assignment – equivalently, if $\varphi \leftrightarrow \psi$ is a tautology.

Proposition 1.3.4 (The equivalences we use most). *For all propositions P, Q, R :*

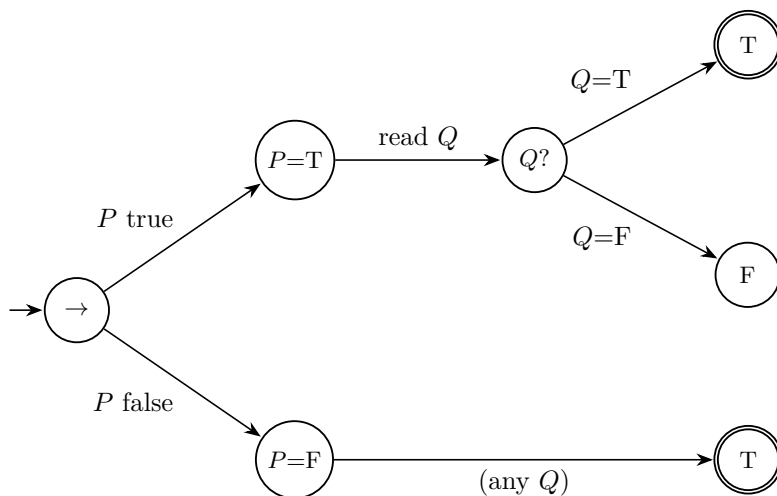
$$\begin{aligned}
 (\text{implication}) \quad & P \rightarrow Q \equiv \neg P \vee Q, \\
 (\text{contrapositive}) \quad & P \rightarrow Q \equiv \neg Q \rightarrow \neg P, \\
 (\text{De Morgan}) \quad & \neg(P \wedge Q) \equiv \neg P \vee \neg Q, \quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q, \\
 (\text{distributive}) \quad & P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R).
 \end{aligned}$$

Proof. Each is a claim that two columns of a truth table agree, so it suffices to fill in the table and compare. We do the contrapositive; the others are identical in spirit, and one De Morgan law is filled in as Worked Example 1.3.6 below.

P	Q	$\neg Q$	$\neg P$	$P \rightarrow Q$	$\neg Q \rightarrow \neg P$
T	T	F	F	T	T
T	F	T	F	F	F
F	T	F	T	T	T
F	F	T	T	T	T

The last two columns are identical, so $P \rightarrow Q \equiv \neg Q \rightarrow \neg P$. □

The single most useful way to *see* the meaning of a connective is to read it as a small decision procedure on the two input bits. The next figure draws $P \rightarrow Q$ this way: the only path that reaches “false” is P true followed by Q false.



Worked Example 1.3.5 (Build a truth table for a three-variable formula). Decide whether $\varphi = (P \rightarrow Q) \wedge (Q \rightarrow R) \rightarrow (P \rightarrow R)$ is a tautology.

Recognition. The formula mixes connectives over three variables and asks “true under *every* assignment?” The trigger “every assignment / is it a tautology” says: enumerate all $2^3 = 8$ rows of a truth table and check the final column. (This is the transitivity of implication, the backbone of chaining lemmas.)

Solution. List the eight assignments and evaluate inside-out. Write $A = P \rightarrow Q$, $B = Q \rightarrow R$, $C = P \rightarrow R$; the formula is $(A \wedge B) \rightarrow C$.

P	Q	R	A	B	C	$A \wedge B$	$(A \wedge B) \rightarrow C$
T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	T
T	F	T	F	T	T	F	T
T	F	F	F	T	F	F	T
F	T	T	T	T	T	T	T
F	T	F	T	F	T	F	T
F	F	T	T	T	T	T	T
F	F	F	T	T	T	T	T

Every entry in the last column is T, so φ is a tautology.

Which idea, and why? A claim of the form “ φ holds for every assignment” is exactly a tautology check, and with k variables the complete, decisive method is the 2^k -row truth table – here $2^3 = 8$ rows settle it with no cleverness required.

Check. The only way $(A \wedge B) \rightarrow C$ could be false is $A \wedge B$ true and C false; that needs $P \rightarrow Q$, $Q \rightarrow R$ both true yet $P \rightarrow R$ false, i.e. $P = T, R = F$. Then $P \rightarrow Q$ forces $Q = T$, but $Q \rightarrow R$ then forces $R = T$, a contradiction. No bad row exists, matching the table.

Worked Example 1.3.6 (Verify a De Morgan law by table). Show $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$.

Recognition. We must show two formulas agree on *every* assignment – that is a logical-equivalence claim (Definition 1.3.3). The trigger “ $\varphi \equiv \psi$ ” says: tabulate both and check the columns are identical.

Solution. Four rows suffice (two variables):

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
T	T	T	F	F	F	F
T	F	T	F	F	T	F
F	T	T	F	T	F	F
F	F	F	T	T	T	T

The columns $\neg(P \vee Q)$ and $\neg P \wedge \neg Q$ are identical, so the two are equivalent.

Which idea, and why? Negating a disjunction turns “or” into “and” and flips each operand; the table is the airtight justification, and the pattern is what we will use to negate quantifiers below.

Check. “It is not the case that $(P \text{ or } Q)$ ” should mean “neither P nor Q ,” which is exactly $\neg P$ and $\neg Q$ – the English matches the table.

Predicates and quantifiers

Definition 1.3.7 (Predicate). A *predicate* on a set D is a function $P: D \rightarrow \{T, F\}$; we write $P(x)$ for its value at $x \in D$ and call D the *domain of discourse*. A k -ary predicate takes k arguments, $P(x_1, \dots, x_k)$. Predicates over strings are everywhere in this book: e.g. $\text{Acc}_M(w) = \text{“}M \text{ accepts } w\text{”}$ a predicate on $w \in \Sigma^*$.

Definition 1.3.8 (Quantifiers). Let P be a predicate on a domain D .

- The *universal* statement $\forall x P(x)$ (“for all x , $P(x)$ ”) is true iff $P(x)$ is true for *every* $x \in D$.
- The *existential* statement $\exists x P(x)$ (“there exists x with $P(x)$ ”) is true iff $P(x)$ is true for *at least one* $x \in D$.

On a finite domain $D = \{x_1, \dots, x_n\}$ these unfold into connectives: $\forall x P(x) \equiv P(x_1) \wedge \dots \wedge P(x_n)$ and $\exists x P(x) \equiv P(x_1) \vee \dots \vee P(x_n)$.

The finite-domain unfolding is the bridge between this subsection and the last: $\forall$ is a big “and,” $\exists$ is a big “or.” Everything about negating quantifiers follows by pushing De Morgan through that unfolding.

Theorem 1.3.9 (Negation of quantifiers). *For any predicate P on any domain,*

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

Proof. Read the left equivalence directly: “not every x satisfies P ” means *some* x fails P , i.e. some x satisfies $\neg P$. The right is dual. To see it is forced rather than merely plausible, take a finite domain $\{x_1, \dots, x_n\}$ and unfold (Definition 1.3.8):

$$\neg \forall x P(x) \equiv \neg(P(x_1) \wedge \dots \wedge P(x_n)) \equiv \neg P(x_1) \vee \dots \vee \neg P(x_n) \equiv \exists x \neg P(x),$$

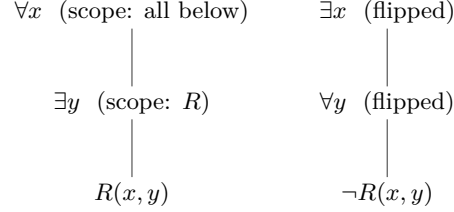
where the middle step is De Morgan (Proposition 1.3.4) applied $n - 1$ times. The existential case unfolds to a disjunction and negates to a conjunction the same way; the argument extends to infinite domains because the meaning of $\forall/\exists$ is exactly “all”/“some.” $\square$

Corollary 1.3.10 (Pushing negation through a chain). *To negate a string of quantifiers, swap each $\forall$ with $\exists$ and each $\exists$ with $\forall$, walking left to right, and negate the innermost predicate. For example,*

$$\neg \forall x \exists y \forall z R(x, y, z) \equiv \exists x \forall y \exists z \neg R(x, y, z).$$

Proof. Apply Theorem 1.3.9 once per quantifier, outermost first: each application moves the $\neg$ one quantifier inward and flips that quantifier’s type, until the $\neg$ lands on R . $\square$

A quantified statement is best read as a *nesting*: each quantifier owns everything to its right (its *scope*). The tree below shows the scope structure of $\forall x \exists y R(x, y)$ and where a negation lands after Corollary 1.3.10 turns it into $\exists x \forall y \neg R(x, y)$.



Worked Example 1.3.11 (Negate a real definition: “ M accepts every string”). The statement “ M accepts every string over Σ ” is $\forall w \in \Sigma^* \text{Acc}_M(w)$. Write its negation in plain words and in symbols.

Recognition. This asks to negate a *universal* statement over strings – exactly Theorem 1.3.9. The trigger “write the negation of ‘for all ...’” says: turn $\forall$ into $\exists$ and negate the body.

Solution. By Theorem 1.3.9,

$$\neg \forall w \text{Acc}_M(w) \equiv \exists w \neg \text{Acc}_M(w),$$

and $\neg \text{Acc}_M(w)$ is “ M rejects w .” In words: *M fails to accept every string iff there is some string M rejects.* So to refute “accepts everything” you need exactly *one* rejected string – a single counterexample.

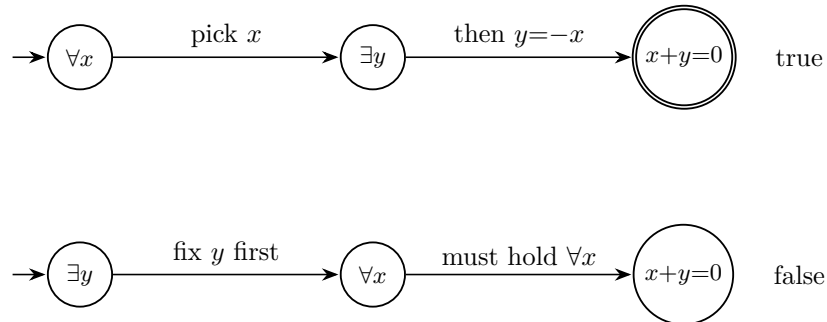
Which idea, and why? The negation of a universal is an existential with the body negated; the practical payoff is that disproving a “for all” claim is a search for one witness, never a re-proof over all inputs.

Check. If $L(M) = \Sigma^*$ then no witness w exists, the existential is false, and the original $\forall$ is true – the equivalence behaves correctly at the extreme.

Worked Example 1.3.12 (Order of quantifiers changes the meaning). On the domain $\mathbb{Z}$, compare $\forall x \exists y (x + y = 0)$ with $\exists y \forall x (x + y = 0)$.

Recognition. Two statements use the same predicate but swap the quantifier order. The trigger “ $\forall \exists$ vs. $\exists \forall$ ” says: read each as a nested game and ask who moves first, because the inner choice may legally depend on the outer one.

Solution. In $\forall x \exists y (x + y = 0)$ the value of y is chosen *after* x is fixed, so we may take $y = -x$; the statement is **true**. In $\exists y \forall x (x + y = 0)$ a single y must work for *all* x at once; no fixed y satisfies $x + y = 0$ for two different x , so the statement is **false**. Same atoms, opposite truth values.



Which idea, and why? In $\forall x \exists y$ the witness y may depend on x ; in $\exists y \forall x$ one witness must serve every x . Swapping the two quantifiers is therefore *not* a truth-preserving move, unlike swapping two adjacent $\forall$ s or two adjacent $\exists$ s.

Check. The order $\exists \forall$ is logically stronger: $\exists y \forall x P \rightarrow \forall x \exists y P$ always holds (the universal witness also works pointwise), but the converse fails here – consistent with $F \rightarrow T$ being true and $T \rightarrow F$ false.

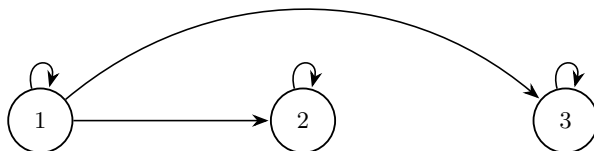
Predicates as relations, and a circuit view

A binary predicate $R(x, y)$ on a finite set is the same data as a directed graph: draw an arrow from x to y exactly when $R(x, y)$ is true. This is the picture that makes properties like reflexivity and symmetry visible at a glance, and it is the same drawing convention we use for state diagrams.

Worked Example 1.3.13 (Read a relation off its graph, then negate a property). On $D = \{1, 2, 3\}$ let R be “ x divides y ” ($x \mid y$). Draw R as a directed graph and decide whether $\forall x \exists y (x \neq y \wedge R(x, y))$ holds.

Recognition. A binary predicate on a small finite set is a directed graph (an arrow for each true pair), and the question is a $\forall\exists$ statement over that graph. The trigger “binary predicate + small finite domain” says: draw the arrows, then check the quantified claim node by node.

Solution. The true pairs of $x \mid y$ on $\{1, 2, 3\}$ are $(1, 1), (2, 2), (3, 3), (1, 2), (1, 3)$. Drawing them (self-loops are the reflexive pairs $x \mid x$):



Now test $\forall x \exists y (x \neq y \wedge x \mid y)$: $x=1$ has the witness $y=2$ ($1 \neq 2, 1 \mid 2$). But $x=2$ has *no* $y \neq 2$ in $\{1, 2, 3\}$ with $2 \mid y$ (only $2 \mid 2$, excluded by $x \neq y$). So the $\forall$ fails at $x=2$; the statement is **false**. By Theorem 1.3.9 its negation $\exists x \forall y (x = y \vee \neg R(x, y))$ holds, and $x=2$ is the witness.

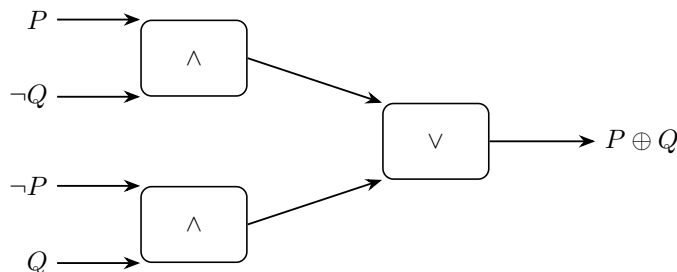
Which idea, and why? A binary relation on a finite domain is exactly a directed graph, so reflexivity = “a loop at every node” and a $\forall\exists$ claim = “every node has an outgoing arrow to a *different* node”; node 2 has only its self-loop, which both kills the claim and is the explicit witness for the negation.

Check. Count outgoing non-loop arrows: node 1 has two, nodes 2 and 3 have none. “Every node has a non-loop out-arrow” is therefore false, witnessed by 2 (or 3).

Worked Example 1.3.14 (A connective as a logic gate). Express the exclusive-or $P \oplus Q$ (true iff exactly one of P, Q is true) with $\wedge, \vee, \neg$, and draw it as a gate network.

Recognition. “Exactly one is true” is a custom connective; the trigger “define a new connective from the basic ones” says: write its truth table, read off the true rows, and assemble an $\vee$ -of- $\wedge$ s (a disjunctive normal form).

Solution. $P \oplus Q$ is true on the rows $(P, Q) = (T, F)$ and (F, T) , giving $P \oplus Q \equiv (P \wedge \neg Q) \vee (\neg P \wedge Q)$. As a network of gates, two AND gates each take one plain and one negated input, and their outputs feed a single OR:



Which idea, and why? Any connective is determined by its true rows, and “OR together one AND-clause per true row” (disjunctive normal form) realizes it from $\wedge, \vee, \neg$ alone – so $\{\wedge, \vee, \neg\}$ is a complete set of connectives, with XOR a derived gate.

Check. On $(P, Q) = (T, T)$ both AND-clauses are false (each needs a negated true input), so the OR is false – correct, since XOR is false when both agree.

Method (How to negate a quantified statement cleanly).

1. **Put it in prefix form:** all quantifiers up front, a quantifier-free body at the end, e.g. $\forall x \exists y \Phi(x, y)$.

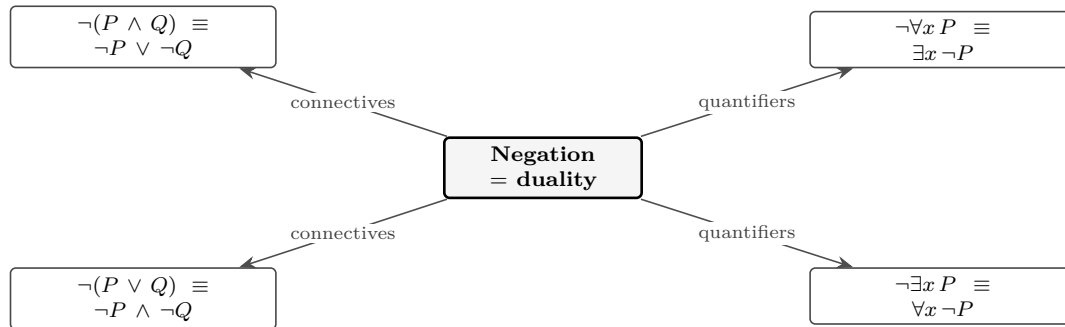
2. **Walk the quantifiers left to right**, flipping each: $\forall \leftrightarrow \exists$. Do not reorder them – order carries meaning (Worked Example 1.3.12).
3. **Negate the body** Φ using the connective laws (Proposition 1.3.4): $\neg(A \wedge B) = \neg A \vee \neg B$, $\neg(A \vee B) = \neg A \wedge \neg B$, $\neg(A \rightarrow B) = A \wedge \neg B$, and $\neg\neg A = A$.
4. **Translate back to English** and sanity-check against a tiny domain (one or two elements) where you can see both the statement and its negation by hand.

Caution. The negation of $P \rightarrow Q$ is *not* $P \rightarrow \neg Q$, and it is not $\neg P \rightarrow \neg Q$. Since $P \rightarrow Q \equiv \neg P \vee Q$, its negation is $\neg(\neg P \vee Q) \equiv P \wedge \neg Q$: “ P is true *and* Q is false.” Concretely, the negation of “if w is accepted then w ends in 1” is “ w is accepted *and* w does not end in 1,” a single conjunction you can exhibit, not another implication.

Caution. Do not swap adjacent quantifiers of *different* type. $\forall x \exists y R(x, y)$ and $\exists y \forall x R(x, y)$ are genuinely different statements (Worked Example 1.3.12); only the direction $\exists \forall \Rightarrow \forall \exists$ is always valid. Reordering two $\forall$ s with each other, or two $\exists$ s with each other, is harmless; mixing the types is where proofs silently go wrong.

Caution. “ P or Q ” in mathematics is *inclusive*: it is true when both hold. If you mean “exactly one,” you want exclusive-or $P \oplus Q$ (Worked Example 1.3.14), which is a different, non-basic connective. Reading a mathematical “or” as exclusive is a common source of off-by-one errors when you later count the strings a machine accepts.

The four negation laws of this section – two for connectives, two for quantifiers – are not four separate facts to memorize but one idea seen from four sides: *negation is duality*. Pushing a $\neg$ inward swaps each operator for its dual ($\wedge \leftrightarrow \vee$, $\forall \leftrightarrow \exists$) while negating what is inside, and the result is always logically equivalent to the original. The map below collects all four under that single principle.



Negation turns each connective or quantifier into its dual – $\wedge \leftrightarrow \vee$, $\forall \leftrightarrow \exists$ – while negating the inside; truth values are preserved, so each arrow is a logical equivalence, not just a rewrite. Check. The two left laws are exactly the De Morgan laws (one of them verified by table in Worked Example 1.3.6); the two right laws are Theorem 1.3.9, which the finite-domain unfolding reduces to those same De Morgan laws – so all four are one tautological equivalence wearing four faces.

Exercises.

1. Build the full truth table for $(P \vee Q) \rightarrow (P \wedge Q)$. Is it a tautology, a contradiction, or neither? In one sentence, describe in plain English the assignments on which it is false.
2. Use a truth table to decide whether $P \rightarrow (Q \rightarrow R)$ and $(P \wedge Q) \rightarrow R$ are logically equivalent. If they are, say in words why “ P implies that Q implies R ” should mean the same as “ P and Q together imply R .”
3. Verify the second De Morgan law $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ by truth table, then use it together with $P \rightarrow Q \equiv \neg P \vee Q$ to rewrite $\neg(P \rightarrow Q)$ as a conjunction with no arrow and no outer negation.
4. Show by truth table that $\{\neg, \rightarrow\}$ is a complete connective set by expressing both $P \vee Q$ and $P \wedge Q$ using only $\neg$ and $\rightarrow$. (Hint: start from $P \rightarrow Q \equiv \neg P \vee Q$.)
5. Let the domain be $\mathbb{Z}$. Write each statement in symbols, then write its negation in symbols and in plain English: (a) every integer has an additive inverse; (b) some integer is its own square; (c) for every x there is a y with $y > x$.

6. Negate the statement $\forall x \exists y \forall z (R(x, y) \wedge (R(y, z) \rightarrow R(x, z)))$, pushing the negation all the way to the predicates so that no $\neg$ stands in front of a quantifier or in front of a parenthesis.
7. On the domain $D = \{0, 1, 2, 3\}$ let $E(x)$ mean “ x is even” and $L(x, y)$ mean “ $x < y$.” Decide the truth value of each, justifying with a witness or a counterexample: (a) $\forall x (E(x) \vee \exists y L(x, y))$; (b) $\exists y \forall x L(x, y)$; (c) $\forall x \exists y (L(x, y) \wedge E(y))$.
8. Define a predicate over strings: for $w \in \{0, 1\}^*$ let $P(w)$ mean “ w contains at least one 1.” Express “ w is the all-zeros string (possibly empty)” using P and a negation, and express “ w has at least two 1s” using two quantifiers over positions of w .
9. On $D = \{1, 2, 3, 4\}$ let $R(x, y)$ mean “ $x + y$ is even.” Draw R as a directed graph (an arrow $x \rightarrow y$ whenever $R(x, y)$ is true; include self-loops). From your drawing, decide whether R is reflexive, whether it is symmetric, and whether $\forall x \exists y (x \neq y \wedge R(x, y))$ holds.
10. Give a connective expression for the “majority” function $\text{Maj}(P, Q, R)$, true iff at least two of P, Q, R are true, as an OR of AND-clauses (one clause per true row of its table). Then draw it as a gate network with four AND gates feeding one OR gate.
11. *Find the bug.* A student writes: “The negation of ‘for all x , if $x > 0$ then $x^2 > 0$ ’ is ‘for all x , if $x > 0$ then $x^2 \leq 0$.’” Identify every error (there are two: the quantifier and the body), and write the correct negation in symbols and in English.
12. *Find the bug.* Someone claims $\forall x \exists y (x \cdot y = 1)$ is true over $\mathbb{Z}$ “because you can always take $y = 1/x$.” Explain precisely why the argument fails, name a domain on which the statement *is* true with the same predicate, and state the correct truth value over $\mathbb{Z}$.
13. Prove that for any predicate P on any domain, $\exists y \forall x P(x, y)$ logically implies $\forall x \exists y P(x, y)$, but the converse fails. (For the failure, exhibit a small domain and a concrete P .) Explain in one sentence which quantifier order is the “stronger” statement and why.

1.4 Methods of Proof

A definition tells you what an object *is*; a theorem asserts that some objects have a property; a proof is the airtight argument that forces you to believe the theorem. This section is about the small number of *shapes* a proof can take. The one key idea is that the logical form of the statement you want to prove – an implication, an “if and only if,” an existence claim, a universal claim – dictates the skeleton of the argument, and once you can read that form off the statement, you know which of a handful of standard openings to write down. Everything later in this book (that a construction works, that two machines are equivalent, that some language is *not* regular) is one of these shapes dressed in the costume of automata, so it pays to see the shapes bare. We treat the five workhorses – direct proof, proof by contrapositive, proof by contradiction, proof by construction, and proof by cases – and pair each with a worked argument and, for all but construction, a diagram of its logical flow.

Reading a statement before proving it

Most proofs stall not on cleverness but on misreading the claim. Before writing anything, rewrite the statement so its logical form is explicit.

Definition 1.4.1 (The four common forms). Let P, Q be predicates (statements that may be true or false, possibly depending on a variable). The forms we meet constantly are:

- *Implication* “ $P \implies Q$ ”: if P holds then Q holds. By definition this is false in exactly one situation – P true while Q false.
- *Biconditional* “ $P \iff Q$ ”: both $P \implies Q$ and $Q \implies P$. Proving it means proving *two* implications, the forward and the reverse.
- *Universal* “for all x , $R(x)$ ”: R holds of every x in the intended domain. A single x with $R(x)$ false (a *counterexample*) refutes it.

- *Existential* “there exists x with $R(x)$ ”: at least one x makes R true. Producing one such x proves it.

The negations are worth memorizing, because contrapositive and contradiction both run on them: the negation of “ $P \implies Q$ ” is “ P and not Q ”; the negation of “for all x , $R(x)$ ” is “there exists x with not $R(x)$ ”; and the negation of “there exists x , $R(x)$ ” is “for all x , not $R(x)$.” A quantifier flips when you push a negation past it.

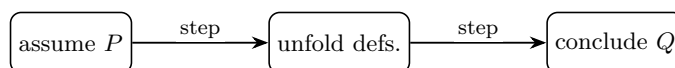
Definition 1.4.2 (Contrapositive and converse). The *contrapositive* of “ $P \implies Q$ ” is “(not Q) $\implies$ (not P)”; it is logically *equivalent* to the original, so proving one proves the other. The *converse* “ $Q \implies P$ ” is a *different* statement and may be true or false independently.

Confusing the contrapositive (always equivalent, a legitimate proof target) with the converse (a separate claim) is the single most common logical error, so keep them straight from the start.

The five methods

We state each method as a recipe, then prove a small theorem in that style. The accompanying diagram shows the *logical flow*: what you are allowed to assume and what you must reach.

1. Direct proof of an implication. To prove $P \implies Q$: assume P , and by a forward chain of definitions and prior results, deduce Q . The flow is a straight line.



Worked Example 1.4.3 (Direct proof: the product of two even integers is even). Prove: if m and n are even integers, then mn is even.

Recognition. The statement is a bare implication “ $P \implies Q$ ” with a friendly hypothesis (“even” has a clean algebraic meaning), and Q follows by substitution. That ease is the trigger for a *direct* proof: assume P and compute toward Q .

Solution. Assume m and n are even. By the definition of even, $m = 2a$ and $n = 2b$ for some integers a, b . Then

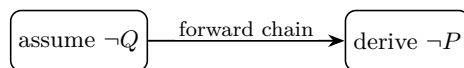
$$mn = (2a)(2b) = 2(2ab),$$

and $2ab$ is an integer, so mn is twice an integer, i.e. even. (We needed only one “even” here, so the result actually holds as soon as *one* factor is even; the worked claim is the weaker symmetric version.) ■

Which idea, and why? When the hypothesis hands you a usable structural fact (here, “ $= 2 \cdot \text{integer}$ ”) that you can substitute straight into the goal, a forward direct proof is shortest – no negation or case split is needed.

Check. Take $m = 4, n = 6$: $mn = 24 = 2 \cdot 12$, even. The algebra $24 = 2(2 \cdot 2 \cdot 3)$ matches the form $2(2ab)$ with $a = 2, b = 3$.

2. Proof by contrapositive. To prove $P \implies Q$, instead prove the equivalent (not Q) $\implies$ (not P). Use it when “not Q ” is a more concrete handle than “ P .”



equivalent to $P \implies Q$

Worked Example 1.4.4 (Contrapositive: if n^2 is even then n is even). Prove for every integer n : if n^2 is even, then n is even.

Recognition. A direct attack would start from “ n^2 even” and try to pry out a factor of 2 from n – awkward, because square roots do not respect parity nicely. The negation “ n is *odd*,” by contrast, is easy to compute with. A hypothesis that is hard to use but whose *opposite conclusion* is easy to manipulate is the trigger for the contrapositive.

Solution. The contrapositive is: if n is odd, then n^2 is odd. Assume n is odd, so $n = 2k + 1$ for some integer k . Then

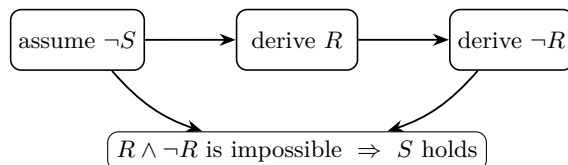
$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is odd (one more than twice an integer). This proves the contrapositive, hence the original. ■

Which idea, and why? The contrapositive is chosen because “ n odd $\Rightarrow n^2$ odd” is a direct computation, whereas the original direction offers no algebraic foothold; and the two are logically identical, so proving either settles the theorem.

Check. $n = 5$ (odd): $n^2 = 25$ odd – contrapositive instance holds. Original instance: $n^2 = 36$ even forces $n = \pm 6$, even. Both consistent.

3. Proof by contradiction. To prove a statement S , assume $\neg S$ and derive a contradiction (a statement and its own negation, or a known falsehood). Since $\neg S$ leads to the impossible, S must hold. The flow branches into an assumption that is then detonated.



Worked Example 1.4.5 (Contradiction: there is no largest prime). Prove: there are infinitely many primes (equivalently, no integer is the largest prime).

Recognition. The claim is an *existence-forever* statement (“infinitely many”); negating it gives a *finite* list of all primes – a concrete, manipulable object. Turning an unbounded claim into a finite hypothesis you can compute against is the classic trigger for contradiction.

Solution. Suppose, for contradiction, that there are only finitely many primes, say all of them are $p_1, p_2, \dots, p_k$. Consider

$$N = p_1 p_2 \cdots p_k + 1.$$

$N > 1$, so N has at least one prime divisor p . That p must be one of the p_i (they are all the primes). But p_i divides $p_1 \cdots p_k$, so p_i dividing N would force p_i to divide $N - p_1 \cdots p_k = 1$ – impossible, since no prime divides 1. We have a prime divisor that both is and is not on the list: a contradiction. Hence the primes are not finite. ■

Which idea, and why? Contradiction is the right tool when the statement denies existence of a bound or a pattern: assuming the bound (a finite prime list) gives a finite object to compute with, and the computed N collides with the assumption.

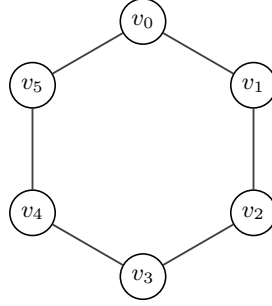
Check. With the (false) list $\{2, 3, 5\}$, $N = 2 \cdot 3 \cdot 5 + 1 = 31$, itself a prime not on the list – exactly the kind of escapee the argument guarantees.

4. Proof by construction. To prove “there exists an object with property R ,” build one explicitly and verify it has R . The proof *is* the recipe plus the check. This is the shape that pervades automata theory: “ $\mathcal{L}$ is regular” means “here is a machine, and it works.”

Worked Example 1.4.6 (Construction: a graph on any even $n \geq 4$ in which every vertex has degree 2). Prove: for every even integer $n \geq 4$ there is a graph with n vertices in which every vertex has degree exactly 2.

Recognition. “There is a graph such that...” is an *existence* claim about a buildable object, so we should display one and count degrees. A single cycle gives every vertex degree 2 for free, and the trigger “exhibit an object with a uniform local property” points straight at a regular, symmetric construction.

Solution. Take the vertices $v_0, v_1, \dots, v_{n-1}$ and join consecutive vertices around a ring, including the wrap-around edge $v_{n-1}v_0$. Formally the edge set is $\{\{v_i, v_{i+1}\} \mid 0 \leq i \leq n-2\} \cup \{\{v_{n-1}, v_0\}\}$. Each v_i touches exactly its two ring neighbours, so every vertex has degree 2. Here is the construction for $n = 6$:

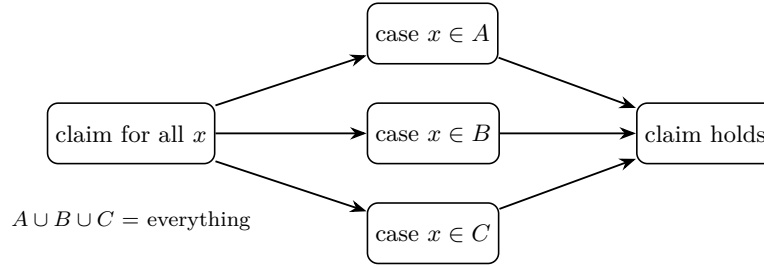


The wrap-around edge v_5v_0 is present, so the figure is a single closed loop and degree 2 is uniform. (For n even and ≥ 4 the same ring works; evenness is not even needed for degree 2, but the claim was stated for even n , which the ring certainly covers.) ■

Which idea, and why? An existence claim is settled by *exhibiting* a witness; the cycle on n vertices is the simplest object whose every vertex has the demanded degree, so building it and counting is the whole proof.

Check. For $n = 6$ the ring has 6 edges; the degree sum is $2 \cdot 6 = 12 = \sum_v \deg v = 6 \cdot 2$, consistent with the handshake count.

5. Proof by cases. When no single argument covers all inputs, split the domain into finitely many exhaustive cases and prove the claim in each. The split must *cover everything* (no gaps) and ideally not overlap. The control flow is a fan-out followed by a join.



Worked Example 1.4.7 (Cases: $n^2 + n$ is even for every integer n). Prove: for every integer n , the number $n^2 + n$ is even.

Recognition. The quantity behaves differently depending on the parity of n , and “every integer” splits cleanly into the two exhaustive cases *even* and *odd*. A property that hinges on a finite classification of the input (here, parity) is the trigger for a case split.

Solution. Note $n^2 + n = n(n + 1)$, the product of two consecutive integers. Split on the parity of n . *Case 1: n even.* Then $n = 2k$, so $n(n + 1) = 2k(n + 1)$ is twice an integer, hence even. *Case 2: n odd.* Then $n + 1$ is even, $n + 1 = 2j$, so $n(n + 1) = n \cdot 2j = 2(nj)$ is even. Every integer is even or odd and these are the only options, so the two cases are exhaustive and the claim holds for all n . ■

Which idea, and why? Cases are forced here because the algebra “which factor carries the 2” depends on the parity of n ; splitting on parity gives two exhaustive sub-claims, each a one-line direct argument.

Check. $n = 3$ (odd): $n^2 + n = 12$, even, with the 2 coming from $n + 1 = 4$. $n = 4$ (even): $n^2 + n = 20$, even, with the 2 coming from $n = 4$.

Equivalences and disproofs

Two further patterns appear constantly. A *biconditional* $P \iff Q$ is proved by establishing both implications separately; a *set equality* $A = B$ is the same shape, proving $A \subseteq B$ and $B \subseteq A$.

Worked Example 1.4.8 (A biconditional, proved in two directions). Prove: an integer n is divisible by 6 *iff* it is divisible by both 2 and 3.

Recognition. The connective is “iff,” i.e. a biconditional $P \iff Q$, which Definition 1.4.1 says must be split into a forward and a reverse implication. The “iff” itself is the trigger.

Solution. $(\Rightarrow)$ If $6 \mid n$ then $n = 6k = 2(3k) = 3(2k)$, so $2 \mid n$ and $3 \mid n$. $(\Leftarrow)$ Suppose $2 \mid n$ and $3 \mid n$. From $2 \mid n$, write $n = 2m$. Since $3 \mid n = 2m$ and 3 shares no factor with 2, we get $3 \mid m$, say $m = 3t$; then $n = 2 \cdot 3t = 6t$, so $6 \mid n$. Both directions hold, so the biconditional is proved. ■

Which idea, and why? A biconditional is two theorems in one wrapper; each arrow is proved by its own short direct argument, and only when *both* are in hand is the “iff” established.

Check. $n = 12$: divisible by 6 and by both 2, 3 – both sides true. $n = 9$: not divisible by 6, and indeed not by 2 – both sides false, consistent.

A *universal* statement is disproved by a single *counterexample*; you switch from proving to refuting, and one bad instance suffices.

Worked Example 1.4.9 (Disproof by counterexample). Decide whether this is true: “for every pair of primes p, q , the sum $p + q$ is even.”

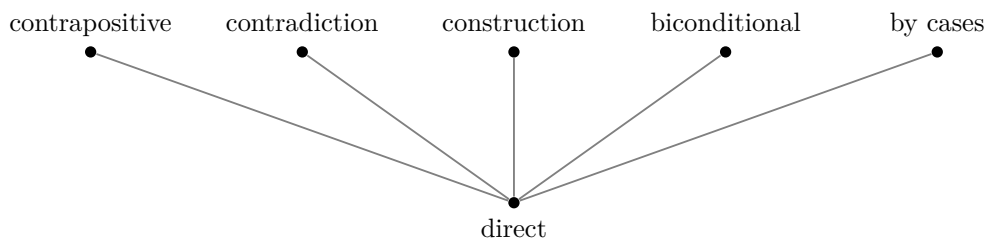
Recognition. The claim is universal (“for every pair”). To *refute* a universal it is enough to exhibit one pair where the conclusion fails – a counterexample. Suspecting an edge case (the only even prime) is the trigger.

Solution. The statement is false. Take $p = 2$ and $q = 3$, both prime; then $p + q = 5$, which is odd. This single pair violates the claim, so the universal statement is refuted. ■

Which idea, and why? One counterexample kills a “for all” claim, because the negation of “for all x , $R(x)$ ” is “there exists x with $\neg R(x)$,” and the witness $x = (2, 3)$ supplies exactly that.

Check. The claim is rescued by adding “both odd”: for odd primes p, q the sum is even (odd + odd), and the lone offender is the even prime 2.

The dependency between the methods is itself a small hierarchy: contrapositive and contradiction are re-expressions of a direct argument over a negated statement, while biconditional and case proofs are direct proofs assembled from smaller direct pieces. The Hasse-style picture below orders the methods by “builds on”:



The point of the picture is practical: when a fancier method stalls, drop to the direct argument underneath it and see what is really being claimed.

Method (How to pick a proof method from the statement’s form).

1. **Read off the logical form** (Definition 1.4.1): is it $P \Rightarrow Q$, $P \Leftrightarrow Q$, “for all,” or “there exists”? Rewrite it in those words before doing anything else.
2. **Existence claim?** Try a *construction*: build the object, then verify it has the property.
3. **Universal claim you suspect is false?** Hunt for a *counterexample*; one instance ends it.
4. **Implication $P \Rightarrow Q$?** Default to a *direct* proof. If P is awkward but “not Q ” is concrete, switch to the *contrapositive*. If the statement denies a bound or an existence (“no largest...”, “infinitely many...”), reach for *contradiction*.
5. **Biconditional $P \Leftrightarrow Q$ or set equality $A = B$?** Prove the two implications (two inclusions) separately.
6. **Behaviour depends on a finite classification?** Split into exhaustive *cases* and prove each; check that the cases leave no gap.
7. **Always end with a check** on a small instance, and confirm you proved the statement, not its converse.

Caution. Proving the *converse* by accident. To prove $P \implies Q$ you may assume P and must reach Q ; assuming Q and reaching P proves the unrelated statement $Q \implies P$. The contrapositive $(\neg Q) \implies (\neg P)$ is a legitimate substitute (it is equivalent), but the converse is not. Whenever you finish, reread the first and last lines and make sure the hypothesis you assumed is the one the theorem gave you.

Caution. A case split that is not exhaustive proves nothing. If you argue “ n even” and “ n divisible by 3” as your two cases, you have left out odd n not divisible by 3 – the cases do not cover the domain, so the conclusion is unjustified. Always state explicitly why your cases together exhaust every possibility (“every integer is even or odd, and these are the only options”).

Caution. In a proof by contradiction, the contradiction must be a genuine logical impossibility (R and $\neg R$, or a known falsehood like $3 \mid 1$), not merely something surprising or ugly. Deriving a strange-looking equation is not a contradiction unless it actually contradicts a fact you already hold. State clearly which two incompatible things you have derived.

Exercises.

- For each statement, name its logical form (implication, biconditional, universal, or existential) and write out its negation in plain words.
 - For every string $w \in \{0,1\}^*$, the reversal w^R has the same length as w .
 - There exists an integer that is both a perfect square and one more than a multiple of 3.
 - If a graph is a tree then it has no cycle.
- Give a *direct* proof: if a and b are odd integers, then $a + b$ is even. Then state the contrapositive of “if $a + b$ is odd then a and b are not both odd,” and say whether it is true.
- Prove by *contrapositive*: for every integer n , if n^2 is divisible by 3 then n is divisible by 3. (You may use the case split on the remainder of n modulo 3.)
- Prove by *contradiction* that $\sqrt{3}$ is irrational. State precisely which two incompatible facts you derive at the end.
- By *construction*, prove that for every $n \geq 1$ there is a string over $\{0,1\}$ of length $2n$ that is equal to its own reversal (a palindrome). Exhibit your string as a function of n and verify the palindrome property.
- Prove by *cases* that for every integer n , exactly one of n and $n + 1$ is even. Make the exhaustiveness of your cases explicit.
- Prove the biconditional: an integer n is even *iff* n^2 is even. Clearly separate the forward and reverse directions and say which proof technique you use for each.
- Disprove by *counterexample*: “for all integers a, b, c , if a divides bc then a divides b or a divides c .” Exhibit specific a, b, c and explain why they refute the claim.
- Find the error in the following “proof.” *Claim.* If $x^2 = y^2$ then $x = y$, for all real x, y . *Proof.* Assume $x^2 = y^2$. Taking square roots of both sides gives $x = y$, as required. Identify the false step and give a counterexample to the claim itself.
- Find the error in the following case proof. *Claim.* Every integer n satisfies $n^2 \geq n$. *Proof.* *Case* $n \geq 1$: multiplying $n \geq 1$ by the positive number n gives $n^2 \geq n$. *Case* $n = 0$: $0 \geq 0$. So $n^2 \geq n$ for all integers n . Explain which integers the cases fail to cover, and either repair the proof or give a value of n where the claim is in fact still true but uncovered.
- Let P be “ w ends in 0” and Q be “ w has even length,” for strings $w \in \{0,1\}^*$. Decide whether “ $P \implies Q$ ” is true. If true, prove it; if false, give the shortest counterexample w .
- Consider the statement “for every simple graph G on $n \geq 2$ vertices, some two vertices of G have the same degree.” Prove it by contradiction. (Hint: in a simple graph on n vertices each degree lies in $\{0, 1, \dots, n - 1\}$; argue that the values 0 and $n - 1$ cannot both occur, so only $n - 1$ distinct degree values are available for n vertices.)

1.5 Induction and the Well-Ordering Principle

Induction is the proof technique that lets a finite argument settle infinitely many cases at once: prove a statement for a smallest object, then prove that the truth of the statement for “smaller” objects forces it for the next one up, and the statement holds for every object in the family. The one key idea is that this works for *any* collection whose members can be built up from base cases by finitely many construction steps – not just the natural numbers but strings, trees, and the runs of an automaton. Because every machine we will study processes its input one step at a time and every language we recognize is defined by a recursive grammar, induction along the building-up order is the workhorse for proving machines correct; this section isolates the principle in its three guises (weak, strong, and structural) and ties each to the well-ordering principle that justifies them all.

The natural-number principles

Definition 1.5.1 (Well-ordering principle). The *well-ordering principle* states: every nonempty subset $S \subseteq \mathbb{N}$ has a *least element*, that is, an $m \in S$ with $m \leq n$ for all $n \in S$. (Here $\mathbb{N} = \{0, 1, 2, \dots\}$; the same holds for $\{1, 2, \dots\}$.)

We take well-ordering as the basic fact about $\mathbb{N}$ and derive induction from it. The point of doing so is that the two are logically equivalent, and seeing the equivalence explains *why* induction is allowed to conclude something about all of an infinite set from two finite proofs.

Theorem 1.5.2 (Principle of mathematical induction, weak form). *Let $P(n)$ be a statement about natural numbers $n \geq b$. Suppose*

- (Basis) $P(b)$ is true, and
- (Induction step) for every $k \geq b$, if $P(k)$ is true then $P(k+1)$ is true.

Then $P(n)$ is true for every $n \geq b$.

Proof. Suppose not. Then the set $S = \{n \geq b \mid P(n) \text{ is false}\}$ is nonempty, so by the well-ordering principle (Definition 1.5.1) it has a least element m . Since $P(b)$ holds, $m \neq b$, so $m > b$ and hence $m - 1 \geq b$. By minimality of m , the number $m - 1$ is *not* in S , so $P(m - 1)$ is true. But the induction step applied at $k = m - 1$ then makes $P(m)$ true, contradicting $m \in S$. So $S = \emptyset$ and $P(n)$ holds for all $n \geq b$. $\square$

The induction step never claims $P(k)$ outright; it claims only the *implication* $P(k) \Rightarrow P(k+1)$. The temporary assumption $P(k)$ inside that proof is the *induction hypothesis*. A useful picture is a row of dominoes: the basis topples the first, the induction step guarantees each domino topples its successor, and so the whole infinite row falls. The chain of implications below makes the metaphor exact.

$$\begin{array}{ccccccc}
 P(b) & \longrightarrow & P(b+1) & \longrightarrow & P(b+2) & \longrightarrow & P(b+3) & \longrightarrow & \dots \\
 \text{basis} & & & & \text{each arrow is one use of the induction step} & & & &
 \end{array}$$

Theorem 1.5.3 (Strong induction). *Let $P(n)$ be a statement about $n \geq b$. Suppose that for every $k \geq b$,*

$$(P(j) \text{ holds for all } b \leq j < k) \implies P(k).$$

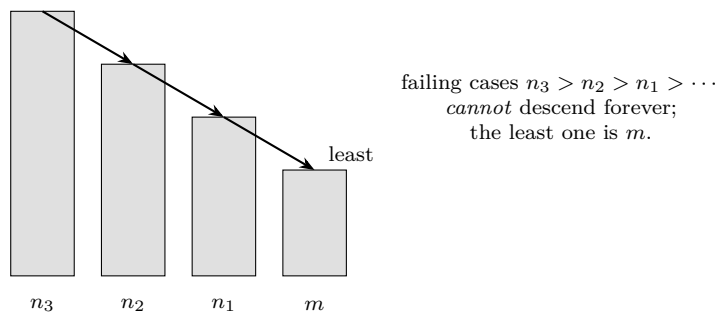
Then $P(n)$ holds for all $n \geq b$. (When $k = b$ the hypothesis is vacuous, so this single clause already includes the basis $P(b)$.)

Proof. Again suppose the set $S = \{n \geq b \mid P(n) \text{ false}\}$ is nonempty and let m be its least element. Every j with $b \leq j < m$ lies outside S by minimality, so $P(j)$ holds for all such j . The hypothesis at $k = m$ then yields $P(m)$, contradicting $m \in S$. $\square$

Strong induction differs from the weak form only in the strength of the hypothesis: to prove $P(k)$ you may assume $P(j)$ for *all* earlier j , not just the immediate predecessor. Logically the two are equivalent – each proves the other – but strong induction is the natural choice when reaching $P(k)$ needs a value of P

that is more than one step back, such as a recurrence that refers to $P(k-2)$, or a number that splits into two strictly smaller factors.

Both proofs above ran the same way: assume a counterexample, take the *least* one, and derive a contradiction from a still-smaller case. Well-ordering is exactly what forbids an infinite strictly-decreasing chain of naturals – any such chain would be a nonempty set with no least element. The schematic below shows a descent of failing cases halted at the least bad case m ; the induction step proves that even m cannot fail, so no bad case survives.



Worked Example 1.5.4 (A summation by weak induction). Prove that $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ for every $n \geq 1$.

Recognition. The claim is indexed by a single integer n and the value at n is obtained from the value at $n-1$ by adding one term. “Built from the previous case by one step” is the trigger for *weak* induction.

Solution. Let $P(n)$ be the displayed equation.

Basis ($n = 1$): the left side is 1 and the right side is $\frac{1 \cdot 2}{2} = 1$. So $P(1)$ holds.

Induction step: fix $k \geq 1$ and assume $P(k)$, i.e. $\sum_{i=1}^k i = \frac{k(k+1)}{2}$. Then

$$\sum_{i=1}^{k+1} i = \left(\sum_{i=1}^k i \right) + (k+1) \stackrel{P(k)}{=} \frac{k(k+1)}{2} + (k+1) = (k+1) \cdot \frac{k+2}{2} = \frac{(k+1)(k+2)}{2},$$

which is exactly $P(k+1)$. By Theorem 1.5.2, $P(n)$ holds for all $n \geq 1$.

Which idea, and why? Weak induction fits because the inductive step uses the single immediately preceding case $P(k)$ and nothing earlier; the new term $(k+1)$ is precisely the gap between consecutive sums.

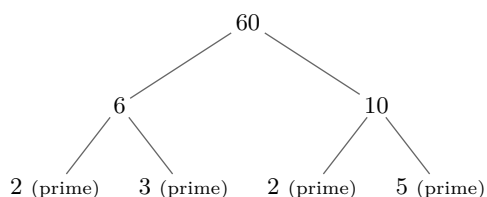
Check. At $n = 4$: $1 + 2 + 3 + 4 = 10$ and $\frac{4 \cdot 5}{2} = 10$.

Worked Example 1.5.5 (A statement that genuinely needs strong induction). Prove that every integer $n \geq 2$ is a product of one or more primes.

Recognition. Factoring n may split it as $n = ab$ with a, b both strictly between 1 and n ; the conclusion for n then rests on the conclusion for two numbers that can be far below $n-1$. “The case k depends on arbitrary smaller cases” is the trigger for *strong* induction.

Solution. Let $P(n)$ be “ n is a product of primes.”

Step (basis folded in): fix $k \geq 2$ and assume $P(j)$ for all j with $2 \leq j < k$. Two cases. If k is prime, then k is a one-factor product of primes and $P(k)$ holds outright (this covers $k = 2$, where the hypothesis is vacuous). If k is composite, write $k = ab$ with $2 \leq a, b < k$. By the induction hypothesis $P(a)$ and $P(b)$ both hold, so a and b are each products of primes; concatenating those two factorizations expresses k as a product of primes. Either way $P(k)$ holds. By Theorem 1.5.3, every $n \geq 2$ is a product of primes.



Which idea, and why? The split $k = ab$ can drop the problem to numbers nowhere near $k - 1$ (here $60 \rightarrow 6, 10$), so only the *strong* hypothesis – all smaller cases available at once – reaches them; the recursion tree bottoms out at primes, which are the base case.

Check. The tree for 60 multiplies back: $2 \cdot 3 \cdot 2 \cdot 5 = 60$, and every leaf is prime.

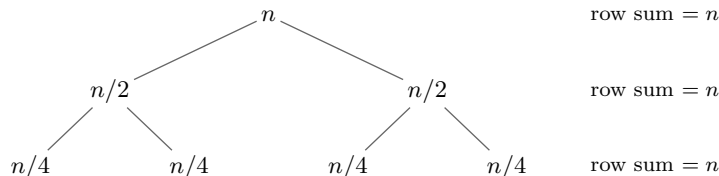
Worked Example 1.5.6 (A divide-and-conquer recurrence, by strong induction on a recursion tree). Let $T(1) = 0$ and, for n a power of 2 with $n \geq 2$, let $T(n) = 2T(n/2) + n$. Prove $T(n) = n \log_2 n$ for every $n = 2^k$, $k \geq 0$.

Recognition. The value $T(n)$ is defined from $T(n/2)$ – an index a *factor* of two below n , far from $n - 1$. A recurrence whose argument halves (or otherwise drops to a fraction of n) is the signature of *strong* induction, organized as a recursion tree where each node spawns two half-size children.

Solution. Induct on k where $n = 2^k$, with $P(k): T(2^k) = 2^k \cdot k$. *Basis* $k = 0$: $n = 1$ and $T(1) = 0 = 1 \cdot 0$. *Step*: assume $P(k)$, so $T(2^k) = 2^k k$; then with $n = 2^{k+1}$,

$$T(2^{k+1}) = 2T(2^k) + 2^{k+1} \stackrel{P(k)}{=} 2 \cdot 2^k k + 2^{k+1} = 2^{k+1} k + 2^{k+1} = 2^{k+1}(k + 1),$$

which is $P(k + 1)$ since $\log_2 2^{k+1} = k + 1$. The recursion tree below makes the total transparent: the $+n$ work at a node of size n sums to n across each level, and there are $\log_2 n$ levels of internal nodes, giving $n \log_2 n$.



Which idea, and why? Halving the argument places the recursive call at $n/2$, below $n - 1$ once $n \geq 4$, so weak induction on n cannot reach it; strong induction on n (equivalently, ordinary induction on $k = \log_2 n$) supplies $T(n/2)$ directly, and the recursion tree reads off the $\log_2 n$ equal rows that give $n \log_2 n$.

Check. $n = 4$: $T(4) = 2T(2) + 4 = 2(2T(1) + 2) + 4 = 2 \cdot 2 + 4 = 8 = 4 \log_2 4$. The tree for $n = 4$ has two levels of internal nodes, matching $\log_2 4 = 2$.

Caution. No basis, no induction. The induction step alone proves nothing: from “ $P(k) \Rightarrow P(k+1)$ ” for all k you can derive a true conclusion only after anchoring some actual case. The bogus “all horses are one color” argument has a perfectly valid step for $k \geq 2$ but silently uses an overlap of two sub-groups that fails at $k = 1$, so the step from $k = 1$ to $k = 2$ is the false link – the basis does not connect to the rest.

Structural induction

The constructions we care about – strings, well-formed expressions, parse trees, the runs of a machine – are defined *recursively*: a few base objects, plus rules that build new objects from old ones. For such families there is a tailored form of induction that mirrors the definition exactly.

Definition 1.5.7 (Inductively defined set). A set X is *defined inductively* by giving (i) one or more *base* elements that belong to X , and (ii) one or more *constructors*, each of which produces a new element of X from finitely many elements already known to be in X . The set X is the *smallest* collection closed under these rules: $x \in X$ iff x can be built from base elements by finitely many applications of the constructors.

Theorem 1.5.8 (Structural induction). *Let X be defined inductively as in Definition 1.5.7, and let P be a property of elements of X . If*

- *P holds of every base element, and*
- *whenever a constructor builds x from already-built pieces $x_1, \dots, x_r$, the truth of $P(x_1), \dots, P(x_r)$ forces $P(x)$,*

then P holds of every element of X .

Proof. Assign to each $x \in X$ its *construction length* $\ell(x)$, the least number of constructor applications needed to build x from base elements (base elements have $\ell = 0$). Then $\ell(x) \in \mathbb{N}$, and we prove $Q(n)$: “ $P(x)$ holds for every x with $\ell(x) \leq n$ ” by strong induction on n (Theorem 1.5.3). For the step, take any x with $\ell(x) = n$. If $n = 0$, x is a base element and $P(x)$ holds by the first hypothesis. Otherwise x is built by a constructor from pieces $x_1, \dots, x_r$, each of which has construction length $< n$; the induction hypothesis gives $P(x_1), \dots, P(x_r)$, and the second hypothesis then yields $P(x)$. So $Q(n)$ holds for all n , i.e. P holds throughout X . $\square$

Structural induction is thus strong induction in disguise, run along the construction order rather than the numeric order. In practice you never compute $\ell(x)$; you simply check P on the base elements and check that each constructor *preserves* P . The recursive definition of strings is the example we will lean on constantly.

Definition 1.5.9 (Strings, inductively). Fix an alphabet Σ . The set Σ^* of strings is defined inductively by: (base) the empty string $\varepsilon \in \Sigma^*$; (constructor) if $w \in \Sigma^*$ and $a \in \Sigma$ then the string wa (append a on the right) is in Σ^* . Length is defined along the same recursion by $|\varepsilon| = 0$ and $|wa| = |w| + 1$.

Worked Example 1.5.10 (Length adds over concatenation, by structural induction on strings). For all $x, y \in \Sigma^*$ prove $|xy| = |x| + |y|$.

Recognition. Strings are built by appending one symbol at a time (Definition 1.5.9), and the claim is about every pair of strings. “Property of all elements of a recursively defined set” is the trigger for *structural* induction; we induct on the second string y , since concatenation xy grows on the right exactly as y is built.

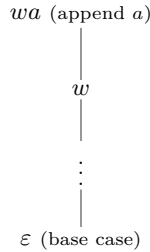
Solution. Fix x and let $P(y)$ be the equation $|xy| = |x| + |y|$.

Base ($y = \varepsilon$): $xy = x\varepsilon = x$, so $|xy| = |x| = |x| + 0 = |x| + |\varepsilon|$.

Constructor ($y = wa$ with $a \in \Sigma$, assuming $P(w)$): by definition of concatenation $x(wa) = (xw)a$, so using $|ua| = |u| + 1$ twice and the hypothesis $|xw| = |x| + |w|$,

$$|x(wa)| = |(xw)a| = |xw| + 1 \stackrel{P(w)}{=} (|x| + |w|) + 1 = |x| + (|w| + 1) = |x| + |wa|.$$

So $P(wa)$ holds. By Theorem 1.5.8, $P(y)$ holds for every $y \in \Sigma^*$.



Which idea, and why? Concatenation satisfies the recursion $x(wa) = (xw)a$ on its right argument, so structural induction on y lets each appended symbol contribute exactly +1 to both sides; the base $y = \varepsilon$ is the only non-recursive case, matching the single base clause of Σ^* .

Check. With $x = \mathbf{ab}$, $y = \mathbf{cde}$: $|xy| = |\mathbf{abcde}| = 5$ and $|x| + |y| = 2 + 3 = 5$.

Worked Example 1.5.11 (Reversal is an anti-homomorphism: $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$). Define reversal by $\varepsilon^{\mathcal{R}} = \varepsilon$ and $(wa)^{\mathcal{R}} = a w^{\mathcal{R}}$ for $a \in \Sigma$. Prove $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$ for all $x, y \in \Sigma^*$.

Recognition. Reversal is given recursively on a string’s right end, so a claim about all x, y asks for structural induction; we again induct on y because both $\mathcal{R}$ and concatenation unfold by recursion on the right ($(wa)^{\mathcal{R}} = a w^{\mathcal{R}}$ and $x(wa) = (xw)a$).

Solution. Fix x and let $P(y)$ be $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$.

Base ($y = \varepsilon$): $(x\varepsilon)^{\mathcal{R}} = x^{\mathcal{R}}$, while $\varepsilon^{\mathcal{R}} x^{\mathcal{R}} = \varepsilon x^{\mathcal{R}} = x^{\mathcal{R}}$. Equal.

Constructor ($y = wa$, assuming $P(w)$): using $x(wa) = (xw)a$ and the defining rule $(ua)^{\mathcal{R}} = a u^{\mathcal{R}}$,

$$(x(wa))^{\mathcal{R}} = ((xw)a)^{\mathcal{R}} = a (xw)^{\mathcal{R}} \stackrel{P(w)}{=} a w^{\mathcal{R}} x^{\mathcal{R}} = (wa)^{\mathcal{R}} x^{\mathcal{R}},$$

since $(wa)^{\mathcal{R}} = a w^{\mathcal{R}}$ by definition. So $P(wa)$ holds, and structural induction finishes.

Which idea, and why? The recursive clause $(wa)^{\mathcal{R}} = a w^{\mathcal{R}}$ peels the symbol off the right of y and plants it on the left of the result, which is exactly why the two factors swap order; induction on y tracks that one symbol per step.

Check. $x = ab, y = cd$: $(abcd)^{\mathcal{R}} = dcba$ and $cd^{\mathcal{R}} ab^{\mathcal{R}} = dc \cdot ba = dcba$.

Worked Example 1.5.12 (Counting the nodes of a full binary tree, structurally). Define *full binary trees* inductively: a single *leaf* is a full binary tree; and if T_L, T_R are full binary trees then the tree with a new root joined to T_L and T_R is a full binary tree. Prove that a full binary tree with ℓ leaves has exactly $\ell - 1$ internal (non-leaf) nodes.

Recognition. The trees are a recursively defined family with one base (a leaf) and one two-piece constructor (a root over T_L, T_R); a count over all such trees is structural induction on the tree, with the hypothesis used on *both* subtrees.

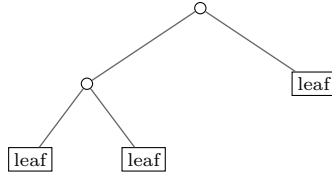
Solution. Let $i(T)$ and $\ell(T)$ be the numbers of internal nodes and leaves of T , and let $P(T)$ be “ $i(T) = \ell(T) - 1$.”

Base (a single leaf): $\ell = 1, i = 0$, and $0 = 1 - 1$. So P holds.

Constructor (T = root over T_L, T_R , assuming $P(T_L)$ and $P(T_R)$): a leaf of T is a leaf of T_L or of T_R , so $\ell(T) = \ell(T_L) + \ell(T_R)$; the internal nodes of T are the new root plus the internal nodes of each subtree, so $i(T) = 1 + i(T_L) + i(T_R)$. Then

$$i(T) = 1 + i(T_L) + i(T_R) \stackrel{\text{hyp}}{=} 1 + (\ell(T_L) - 1) + (\ell(T_R) - 1) = (\ell(T_L) + \ell(T_R)) - 1 = \ell(T) - 1.$$

So $P(T)$ holds, and Theorem 1.5.8 gives the claim for all full binary trees.



Which idea, and why? One base case and a constructor taking *two* prior objects mean structural induction must invoke the hypothesis on both T_L and T_R at once; the round nodes count toward i and the square leaves toward ℓ , and the bookkeeping $\ell = i + 1$ is preserved by adding one root over two valid subtrees.

Check. The drawn tree has 3 leaves and 2 round internal nodes, and $2 = 3 - 1$.

Method (How to run an induction proof).

1. **Name the variable you induct on** and state $P(\cdot)$ as a precise statement with that one variable free. For structural induction, name the recursively defined set and induct along its construction.
2. **Choose the form.** If $P(k)$ follows from $P(k-1)$ alone, use weak induction (Theorem 1.5.2). If it needs smaller cases that may be far back, or the object splits into smaller pieces, use strong (Theorem 1.5.3) or structural (Theorem 1.5.8) induction.
3. **Prove the basis** – every base case. List them explicitly; a recursively defined set with two base elements needs both.
4. **Prove the step.** Write down the induction hypothesis verbatim, then derive P of the next object using *only* the hypothesis and the definitions. Mark where you use the hypothesis (the $\stackrel{\text{hyp}}{=}$ tag above).
5. **Confirm coverage.** Check that every element of the family is reached: every $n \geq b$ for numeric induction, every object reachable by the constructors for structural induction.

Caution. **Inducting on the wrong end of a recursive definition.** Strings in Definition 1.5.9 are built by appending on the *right*. If you try to prove a fact about $^{\mathcal{R}}$ or concatenation by “peeling a symbol off the left,” you are using a different recursion than the one the definitions follow, and the algebra will not line up. Match the induction to the constructor: append-on-the-right definitions call for the constructor case $y = wa$, not $y = aw$.

Caution. A too-weak hypothesis. When a proof of $P(k)$ secretly uses $P(k-2)$ or $P(\lfloor k/2 \rfloor)$, weak induction's hypothesis (only $P(k-1)$) is not enough, and the step is invalid even though it “feels” inductive. Switch to strong induction so the earlier cases are available. Conversely, do not over-assume: the hypothesis covers cases *strictly* smaller than the one you are proving, never the case itself.

Exercises.

1. Prove by weak induction that $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$ for every $n \geq 1$. State the induction hypothesis explicitly and mark where you use it.
2. Prove that $\sum_{i=1}^n (2i-1) = n^2$ for all $n \geq 1$ (the sum of the first n odd numbers). Then give a one-sentence geometric reason, in terms of building an $n \times n$ square one L-shaped border at a time.
3. Prove by induction that $n^3 - n$ is divisible by 6 for every $n \geq 0$. (In the step, factor $(n+1)^3 - (n+1)$ so the induction hypothesis appears.)
4. Prove that $2^n > n^2$ for every integer $n \geq 5$. Identify the smallest base value for which the claim is true and explain why starting the basis any lower would break the induction.
5. Using only 4-cent and 7-cent stamps, prove that every postage amount of $n \geq 18$ cents can be formed exactly. Decide how many base cases your strong induction needs and justify the number in terms of how far back the inductive step reaches.
6. Define a sequence by $f_0 = 0$, $f_1 = 1$, and $f_n = f_{n-1} + f_{n-2}$ for $n \geq 2$. Prove by strong induction that $f_n < 2^n$ for all $n \geq 0$. Explain in one sentence why weak induction (using only f_{n-1}) is insufficient here.
7. The well-ordering principle. Prove, using a least-counterexample argument, that every integer $n \geq 2$ can be written as a product of primes. Then state precisely which step of your proof would fail if $\mathbb{N}$ were replaced by the nonnegative rationals $\mathbb{Q}_{\geq 0}$, which are *not* well-ordered.
8. Strings are defined inductively by “ $\varepsilon \in \Sigma^*$; and if $w \in \Sigma^*$, $a \in \Sigma$ then $wa \in \Sigma^*$,” with $|\varepsilon| = 0$ and $|wa| = |w| + 1$. Prove by structural induction on y that $|xy| = |x| + |y|$ for all $x, y \in \Sigma^*$. State the constructor case you induct on.
9. Define the number of occurrences of a symbol a in a string by $\#_a(\varepsilon) = 0$ and $\#_a(wb) = \#_a(w) + [b = a]$ (where $[b = a]$ is 1 if $b = a$ and 0 otherwise). Prove by structural induction that $\#_a(xy) = \#_a(x) + \#_a(y)$ for all strings x, y .
10. Reversal is defined by $\varepsilon^{\mathcal{R}} = \varepsilon$ and $(wa)^{\mathcal{R}} = a w^{\mathcal{R}}$. Prove by structural induction that $(w^{\mathcal{R}})^{\mathcal{R}} = w$ for every $w \in \Sigma^*$. (You may use, with a citation, the identity $(xy)^{\mathcal{R}} = y^{\mathcal{R}} x^{\mathcal{R}}$.)
11. A full binary tree is a leaf, or a root joined to two full binary trees. Prove by structural induction that a full binary tree with i internal nodes has exactly $2i + 1$ nodes in total. Draw a small tree with $i = 2$ and confirm your formula on it.
12. Let the height of a full binary tree be $h(\text{leaf}) = 0$ and $h(\text{root over } T_L, T_R) = 1 + \max\{h(T_L), h(T_R)\}$, and let $\ell(T)$ be its leaf count. Prove by structural induction that $\ell(T) \leq 2^{h(T)}$ for every full binary tree T .
13. *Find the bug.* A student “proves” that every finite set of lines in the plane, no two parallel, passes through a single common point: “Basis: one line trivially passes through a point on it. Step: given $k + 1$ such lines, by the hypothesis the first k meet at a point p and the last k meet at a point q ; since both groups share the middle $k - 1$ lines, $p = q$, so all $k + 1$ lines pass through p .” Identify the smallest k at which the step fails and explain why, in the spirit of the “all horses are the same color” fallacy.
14. *Find the bug.* Someone claims to prove $|xy| = |x| + |y|$ by inducting on x written as $x = a w$ (peel a symbol off the *left*), with base case $x = \varepsilon$. Explain why this induction does not match the recursive definition of Σ^* given above, and state the change needed to make the proof legitimate.

Chapter 2

Graph Theory for Computation

2.1 Graphs, Digraphs, and Degrees

A graph is the bare mathematics of “things and the pairs of them that are related”: a finite set of points together with a record of which points are joined. This is the data structure underneath almost everything later in the book – a finite automaton is a labelled graph, a computation is a walk through a graph of configurations, and many of our hardest problems (*HAMPATH*, *CLIQUE*, coloring) are questions about graphs. The one key idea of this section is that local information – how many edges touch each vertex – already constrains the global object rigidly, and the *handshake lemma* is our first proof of that. We make graphs and directed graphs precise, define adjacency and degree, prove the handshake lemma and its parity corollary, and then read those numbers off many small drawn examples.

Definition 2.1.1 (Simple graph). A (*simple, undirected*) graph is a pair $G = (V, E)$ where V is a finite set of *vertices* and E is a set of *edges*, each edge being a two-element subset $\{u, v\} \subseteq V$ with $u \neq v$. We write an edge $\{u, v\}$ as uv (so $uv = vu$). Two vertices u, v are *adjacent* if $uv \in E$, and then each is *incident* with the edge uv . The word *simple* encodes two bans: no edge from a vertex to itself (no *loops*, since $u \neq v$), and no two parallel copies of the same edge (since E is a *set*).

Because each edge is an unordered pair, a graph on $n = |V|$ vertices can have at most $\binom{n}{2} = \frac{n(n-1)}{2}$ edges; the graph attaining this maximum, in which every pair is joined, is the *complete graph* K_n . The next definition packages the local picture at a single vertex.

Definition 2.1.2 (Neighborhood and degree). Let $G = (V, E)$ and $v \in V$. The *neighborhood* of v is the set of vertices adjacent to it,

$$N(v) = \{u \in V \mid uv \in E\},$$

and the *degree* of v is $\deg(v) = |N(v)|$, the number of edges incident with v . A vertex of degree 0 is *isolated*; a vertex of degree 1 is a *leaf* (or pendant). The *degree sequence* of G is the list of all vertex degrees written in nonincreasing order.

In a simple graph $\deg(v)$ counts neighbors and incident edges interchangeably, since each incident edge supplies exactly one distinct neighbor. (Once loops or parallel edges are allowed the two counts diverge, which is one reason we insist on “simple” as the default.) The most useful fact about degrees is that, summed over the whole graph, they cannot be arbitrary.

Theorem 2.1.3 (Handshake lemma). *For every graph $G = (V, E)$,*

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Proof. Count, in two ways, the number of *incidences* – pairs (v, e) where v is a vertex, e is an edge, and v is an endpoint of e . Grouping the incidences by their vertex, each vertex v lies in exactly $\deg(v)$ incidences, so the total is $\sum_v \deg(v)$. Grouping instead by their edge, each edge $e = uv$ has exactly two endpoints and so contributes exactly two incidences, giving the total $2|E|$. The two counts are of the same set, hence equal. $\square$

Corollary 2.1.4 (Parity of odd-degree vertices). *In every graph the number of vertices of odd degree is even.*

Proof. Split the degree sum by parity. The vertices of even degree contribute an even total. By Theorem 2.1.3 the grand total $\sum_v \deg(v) = 2|E|$ is even, so the contribution of the odd-degree vertices is even as well. A sum of odd numbers is even only when there is an even number of them; hence the odd-degree vertices number evenly. $\square$

Definition 2.1.5 (Directed graph). A *directed graph* (or *digraph*) is a pair $G = (V, E)$ where $E \subseteq V \times V$ is a set of *ordered* pairs, the *arcs*. An arc (u, v) , drawn as an arrow $u \rightarrow v$, has *tail* u and *head* v . The *out-degree* $d^+(v)$ is the number of arcs with tail v , and the *in-degree* $d^-(v)$ is the number with head v . Here (u, v) and (v, u) are distinct arcs, and (unless we say otherwise) self-arcs (v, v) are allowed.

Digraphs are the right model whenever a relation is one-directional: a transition $\delta(q, a) = q'$ of a finite automaton is an arc $q \rightarrow q'$, and “ x depends on y ” or “ x reduces to y ” are arcs. The handshake count has a directed analogue, separated by orientation.

Proposition 2.1.6 (Directed handshake). *For every digraph $G = (V, E)$, $\sum_{v \in V} d^+(v) = \sum_{v \in V} d^-(v) = |E|$.*

Proof. Each arc (u, v) has exactly one tail, so summing out-degrees counts every arc once: $\sum_v d^+(v) = |E|$. Likewise each arc has exactly one head, so $\sum_v d^-(v) = |E|$. The two sums equal the same number $|E|$, hence each other. $\square$

Two ways to see a graph. A graph is drawn as a *diagram* – a dot per vertex, a line per edge, with no meaning attached to where the dots sit or whether the lines cross – or recorded combinatorially by listing V and E . The drawing exposes structure to the eye; the lists make “which pairs are joined” unambiguous. Only the *incidence pattern* is the graph; two very different-looking pictures can be the same graph, a fact we pin down with isomorphism.

Definition 2.1.7 (Subgraph and induced subgraph). A graph $H = (V_H, E_H)$ is a *subgraph* of $G = (V, E)$, written $H \subseteq G$, if $V_H \subseteq V$ and $E_H \subseteq E$ (every retained edge must still join two retained vertices). It is a *spanning* subgraph if $V_H = V$. For $X \subseteq V$, the *induced subgraph* $G[X]$ has vertex set X and *all* edges of G with both endpoints in X :

$$G[X] = (X, \{uv \in E \mid u, v \in X\}).$$

A general subgraph may drop both vertices and edges freely; an induced subgraph chooses only which vertices to keep and is then *forced* to keep every original edge among them. So $G[X]$ is the densest subgraph on the vertex set X , and a subgraph H on the same vertices is induced exactly when it omits no available edge.

Definition 2.1.8 (Isomorphism). Graphs $G = (V, E)$ and $G' = (V', E')$ are *isomorphic*, written $G \cong G'$, if there is a bijection $\varphi: V \rightarrow V'$ that preserves adjacency in both directions:

$$uv \in E \iff \varphi(u)\varphi(v) \in E' \quad \text{for all } u, v \in V.$$

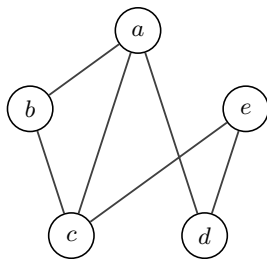
Such a φ is an *isomorphism*. Isomorphic graphs are “the same graph with relabelled vertices.”

An isomorphism must carry degrees to equal degrees – if $\varphi(u) = u'$ then u and u' have the same neighbors up to relabelling, so $\deg(u) = \deg(u')$. Hence the degree sequence is an *invariant*: graphs with different degree sequences cannot be isomorphic. (The converse fails; matching degree sequences do not guarantee isomorphism, as Worked Example 2.1.14 shows.)

Worked Example 2.1.9 (Reading degrees off a drawing, and checking the handshake lemma). For the graph G with $V = \{a, b, c, d, e\}$ and $E = \{ab, ac, ad, bc, ce, de\}$, list the degree of every vertex, give the degree sequence, and verify Theorem 2.1.3.

Recognition. This is direct bookkeeping: degree is the count of incident edges, and the handshake lemma is the cross-check that $\sum_v \deg(v) = 2|E|$. The trigger “verify the lemma” means compute both sides independently and compare.

Solution. Draw the five vertices and the six edges, then count the lines meeting each dot.



Counting incident edges: $\deg(a) = 3$ (to b, c, d), $\deg(b) = 2$ (to a, c), $\deg(c) = 3$ (to a, b, e), $\deg(d) = 2$ (to a, c), $\deg(e) = 2$ (to c, d). The degree sequence, sorted, is $(3, 3, 2, 2, 2)$.

Which idea, and why? Degree is purely local – count the lines at each dot – and the handshake lemma equates that local data, summed, with twice the (global) edge count; computing both sides is the verification itself.

Check. $\sum_v \deg(v) = 3 + 2 + 3 + 2 + 2 = 12$, and $2|E| = 2 \cdot 6 = 12$. They agree, and the two odd-degree vertices a, c are even in number, consistent with Corollary 2.1.4.

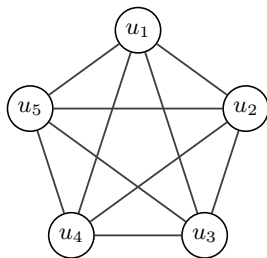
Worked Example 2.1.10 (Counting edges of K_n from degrees). Use the handshake lemma to find the number of edges of the complete graph K_n (every pair of n vertices joined), and confirm it on K_5 .

Recognition. A counting question where every degree is *known* by symmetry but the edge count is not yet. The trigger “all degrees equal, edges wanted” is exactly the situation the handshake lemma turns around: it lets you solve for $|E|$ from the degree sum.

Solution. In K_n each vertex is adjacent to the other $n - 1$ vertices, so $\deg(v) = n - 1$ for all v . The degree sum is therefore $n(n - 1)$, and Theorem 2.1.3 gives $2|E| = n(n - 1)$, hence

$$|E| = \frac{n(n - 1)}{2} = \binom{n}{2}.$$

For K_5 this predicts $\binom{5}{2} = 10$ edges, each vertex having degree 4.



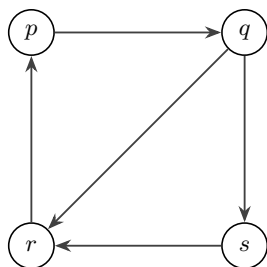
Which idea, and why? When every degree is forced by symmetry, the handshake lemma converts the easy degree sum into the wanted edge count – you never list the edges, you divide the degree sum by two.

Check. The drawing of K_5 has $\binom{5}{2} = 10$ edges, and reading any single vertex shows 4 lines leaving it, so $\deg = 4$ and $\sum_v \deg(v) = 5 \cdot 4 = 20 = 2 \cdot 10$.

Worked Example 2.1.11 (In-degree and out-degree in a digraph). Let the digraph D have vertices $\{p, q, r, s\}$ and arcs $p \rightarrow q$, $q \rightarrow r$, $r \rightarrow p$, $q \rightarrow s$, $s \rightarrow r$. Tabulate $d^+(v)$ and $d^-(v)$ for every vertex and verify Proposition 2.1.6.

Recognition. A digraph degree count: out-degree counts arrows *leaving*, in-degree counts arrows *arriving*, and they are tracked separately. The trigger “verify the directed handshake” means both sums must equal the number of arcs.

Solution. Draw each arc as an arrow from tail to head, then at each vertex count outgoing and incoming arrowheads.



v	p	q	r	s	sum
$d^+(v)$	1	2	1	1	5
$d^-(v)$	1	1	2	1	5

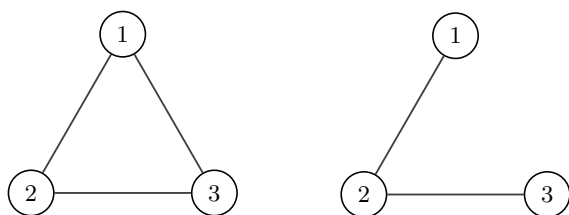
Which idea, and why? In a digraph the two degree notions are genuinely different per vertex (here q has $d^+ = 2, d^- = 1$), but each arc has exactly one tail and one head, so both rows must total the arc count.

Check. There are 5 arcs; both $\sum_v d^+(v)$ and $\sum_v d^-(v)$ equal $5 = |E|$, as the proposition requires. Note $d^+(q) \neq d^-(q)$, so the equality of the two sums does not hold vertex by vertex.

Worked Example 2.1.12 (Subgraph versus induced subgraph). Take G on $V = \{1, 2, 3, 4\}$ with $E = \{12, 23, 34, 41, 13\}$ (a 4-cycle plus the diagonal 13). For $X = \{1, 2, 3\}$, draw the induced subgraph $G[X]$, and separately exhibit a subgraph on X that is *not* induced.

Recognition. The trigger is the contrast “induced versus general.” Induced is determined the instant you pick X – keep every original edge inside X ; a non-induced subgraph on the same vertices must *drop* at least one available edge.

Solution. Inside $X = \{1, 2, 3\}$ the original edges are 12, 23, and 13 (the edges 34 and 41 leave X , so they are gone). The induced subgraph keeps all three, forming a triangle:



$G[\{1, 2, 3\}]$ (induced)

subgraph, *not* induced

The right picture has the same three vertices but omits the available edge 13, so it is a perfectly good subgraph yet not the induced one.

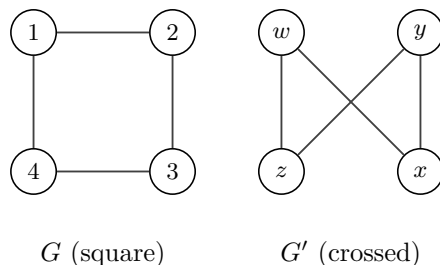
Which idea, and why? Picking X fixes the induced subgraph uniquely (all internal edges retained); any subgraph on X that discards an internal edge – here 13 – is non-induced by definition.

Check. $G[X]$ has 3 edges (a triangle, degree sequence $(2, 2, 2)$); the non-induced example has 2 edges (a path, degree sequence $(2, 1, 1)$). Different edge counts on the same vertices confirm they differ.

Worked Example 2.1.13 (Two drawings of one graph: building an isomorphism). Show that the 4-cycle drawn as a square is isomorphic to the same graph drawn as a “crossed” figure, by exhibiting an explicit vertex bijection.

Recognition. “Show isomorphic” is a *construction* task: produce a bijection and check it preserves edges both ways. The trigger is that the drawings look different but we suspect identical incidence structure, so a relabelling should match them.

Solution. Let G be the square with vertices 1, 2, 3, 4 and edges 12, 23, 34, 41. Let G' have vertices w, x, y, z and the *same cyclic* adjacency but drawn with two edges crossing: edges wx, xy, yz, zw . Define $\varphi(1) = w$, $\varphi(2) = x$, $\varphi(3) = y$, $\varphi(4) = z$.



Check each edge of G maps to an edge of G' : $12 \mapsto wx$, $23 \mapsto xy$, $34 \mapsto yz$, $41 \mapsto zw$ – all present. As φ is a bijection and both graphs have exactly four edges, the correspondence is onto the edge set as well, so adjacency is preserved in both directions.

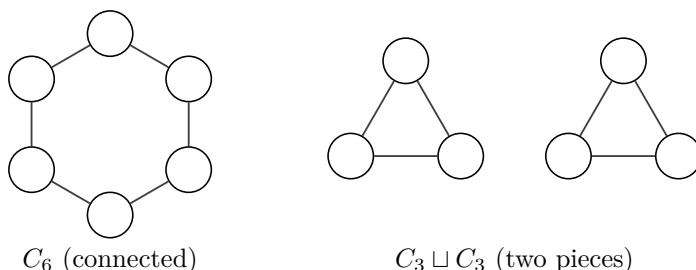
Which idea, and why? Isomorphism is proved by *building* the bijection and verifying uv is an edge iff $\varphi(u)\varphi(v)$ is; the misleading crossing in the drawing is irrelevant, since position and edge-crossings are not part of the graph.

Check. Both graphs have degree sequence $(2, 2, 2, 2)$ and four edges, the necessary invariants; and the explicit φ matches all four edges, which is sufficient.

Worked Example 2.1.14 (Same degree sequence, different graphs). Exhibit two graphs on six vertices that share the degree sequence $(2, 2, 2, 2, 2, 2)$ but are *not* isomorphic.

Recognition. The trigger is the warning after Definition 2.1.8: equal degree sequences are *necessary* but not *sufficient* for isomorphism. To refute sufficiency we need two graphs with identical degrees that differ in some other invariant.

Solution. Every vertex of degree 2 forces each graph to be a disjoint union of cycles. Two ways to use six degree-2 vertices: one 6-cycle C_6 , or two disjoint triangles $C_3 \sqcup C_3$.



Both have all degrees 2 and (by the handshake lemma) $|E| = \frac{12}{2} = 6$ edges, yet C_6 is connected – a single closed loop – while $C_3 \sqcup C_3$ splits into two components. An isomorphism preserves connectedness, so no bijection can match them.

Which idea, and why? To break “equal degrees $\Rightarrow$ isomorphic” we exhibit a counterexample distinguished by a *different* invariant – here the number of connected components (1 versus 2) – which the degree sequence cannot see.

Check. Degree sequences agree: $(2, 2, 2, 2, 2, 2)$ for each; edge counts agree: 6 each. They differ in component count, which suffices to deny isomorphism.

Method (How to compute degrees and apply the handshake lemma).

1. **Count locally.** For an undirected graph, $\deg(v)$ is the number of edges incident with v (equivalently, $|N(v)|$). In a drawing, count the line-ends meeting the dot; never count a far endpoint twice.
2. **Split direction for digraphs.** Track $d^+(v)$ (arrows out) and $d^-(v)$ (arrows in) separately. They need not be equal at a vertex.
3. **Sum and halve.** If you know all degrees, the edge count is $|E| = \frac{1}{2} \sum_v \deg(v)$ (undirected) or $|E| = \sum_v d^+(v) = \sum_v d^-(v)$ (directed). If you know $|E|$, the degree sum is $2|E|$ – a built-in check on any degree count.

4. **Use parity.** A claimed degree sequence with an odd number of odd entries is impossible (Corollary 2.1.4); reject it before drawing.
5. **Compare invariants, not pictures.** To decide isomorphism, first compare degree sequences (and edge counts, component counts). Matching invariants only suggest isomorphism – prove it by exhibiting an adjacency-preserving bijection; a single mismatch disproves it outright.

Caution. A proposed degree sequence must sum to an *even* number, because the sum equals $2|E|$. A sequence like $(3, 3, 2, 2)$ sums to 10 (even) and may be realizable, but $(3, 2, 2, 2)$ sums to 9 (odd) and *no* graph has it – there is no need to attempt a drawing. Equivalently, by Corollary 2.1.4, any list with an odd count of odd numbers is dead on arrival.

Caution. Equal degree sequences do *not* imply isomorphic graphs (Worked Example 2.1.14), and a misleading drawing does *not* imply non-isomorphic graphs. The graph is only the incidence pattern: where you place the dots, and whether the edges happen to cross, carry no information. To settle isomorphism you must either build an explicit adjacency-preserving bijection (to prove $\cong$) or point to an invariant that differs – degree sequence, edge count, number of components, presence of a triangle – to prove $\not\cong$. Eyeballing the two pictures is never a proof either way.

Exercises.

1. Let G have vertex set $\{a, b, c, d, e, f\}$ and edge set $\{ab, ac, bc, bd, de, df, ef\}$. Draw the diagram, list $\deg(v)$ for every vertex, write the degree sequence, and verify the handshake lemma by computing both $\sum_v \deg(v)$ and $2|E|$.
2. Draw the complete graph K_4 and the complete graph K_6 . For each, give the degree of every vertex and the number of edges, then check your edge counts against the formula $\binom{n}{2}$.
3. Decide, with a one-line reason in each case, which of the following are degree sequences of some simple graph: $(4, 3, 2, 2, 1)$, $(3, 3, 2, 1)$, $(5, 3, 2, 2, 2)$, $(2, 2, 2, 2)$. For every realizable one, draw a graph that achieves it.
4. Draw the digraph on $\{1, 2, 3, 4\}$ with arcs $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$, $1 \rightarrow 4$, $4 \rightarrow 3$, $3 \rightarrow 3$. Tabulate $d^+(v)$ and $d^-(v)$ for each vertex, and verify $\sum_v d^+(v) = \sum_v d^-(v) = |E|$. (Mind the self-arc $3 \rightarrow 3$.)
5. Prove the handshake lemma directly by induction on the number of edges $|E|$, deleting one edge at the inductive step. State carefully what removing an edge does to the two endpoints' degrees.
6. A party has n people, some pairs of whom shake hands. Model this as a graph and use Corollary 2.1.4 to prove that the number of people who shook hands an odd number of times is even. Then explain why a group of three people cannot each have shaken hands with exactly one other person in the group.
7. Let G be the graph on $\{1, 2, 3, 4, 5\}$ with edges $\{12, 23, 34, 45, 51, 13\}$. For $X = \{1, 2, 3, 4\}$, draw the induced subgraph $G[X]$ and give its degree sequence. Then draw one spanning subgraph of G that is *not* induced, and one subgraph on X that has strictly fewer edges than $G[X]$.
8. Show that the two graphs below are isomorphic by giving an explicit vertex bijection and checking every edge: G has vertices $\{1, 2, 3, 4, 5\}$ and edges $\{12, 23, 34, 45, 51\}$; G' has vertices $\{a, b, c, d, e\}$ and edges $\{ac, ce, eb, bd, da\}$. (Both are five-cycles, labelled differently.)
9. Exhibit two non-isomorphic simple graphs that both have degree sequence $(3, 2, 2, 2, 1)$, and name the invariant that distinguishes them. (Hint: consider whether a triangle is present.)
10. Prove that in any simple graph with at least two vertices there exist two distinct vertices of equal degree. (Hint: in a graph on n vertices each degree lies in $\{0, 1, \dots, n-1\}$; argue that the values 0 and $n-1$ cannot both occur, then use pigeonhole.)
11. *Find the bug.* A student claims the sequence $(5, 4, 3, 2, 1)$ is realized by some simple graph on five vertices and starts drawing. Without drawing, show the claim is impossible, and identify *two* separate reasons (one a counting/parity fact, one about the largest degree relative to the number of vertices).

12. *Find the bug.* Someone asserts that two graphs are isomorphic “because both pictures look like a square with one diagonal, and both have 5 edges.” Construct two graphs on four vertices that each have 5 edges but are *not* isomorphic – or explain why no such pair exists on four vertices – and state precisely why “same number of edges” alone never establishes isomorphism.
13. Prove that for every digraph, the vertices with $d^+(v) > d^-(v)$ contribute a total excess exactly balanced by the deficit at vertices with $d^+(v) < d^-(v)$; that is, $\sum_v (d^+(v) - d^-(v)) = 0$. Explain how this follows from Proposition 2.1.6.

2.2 Paths, Cycles, Connectivity, and Components

Section 2.1 gave us the static anatomy of a graph – vertices, edges, and degrees. This section makes graphs *move*: we follow sequences of edges and ask which vertices can reach which. The one organizing idea is *reachability* – “can I get from u to v by walking along edges?” – and almost everything here is a refinement of it: a walk is any such trip, a path is a trip that never repeats a vertex, connectivity says every pair is reachable, and a component is a maximal reachable chunk. These notions are not graph-theoretic trivia; reachability in a directed graph is exactly how a finite automaton decides whether a string drives it from the start state into an accept state, and the shortest-walk machinery below is the breadth-first search that later powers algorithms throughout the book.

Walks, trails, paths, and cycles

Definition 2.2.1 (Walk, trail, path). Let $G = (V, E)$ be a graph. A *walk* of length k from u to v is an alternating sequence

$$u = v_0, e_1, v_1, e_2, v_2, \dots, e_k, v_k = v,$$

where each $e_i = \{v_{i-1}, v_i\}$ is an edge of G ; its *length* is the number k of edges traversed (vertices and edges may repeat). A *trail* is a walk with no repeated *edge*. A *path* is a walk with no repeated *vertex* (hence no repeated edge either). A walk is *closed* if $v_0 = v_k$.

We usually abbreviate a walk by its vertex list $v_0 v_1 \dots v_k$, since on a simple graph the intervening edges are forced. The three notions tighten in sequence: every path is a trail, and every trail is a walk, but not conversely. The distinction matters because the constraints buy different things – “no repeated edge” (trail) is what Euler tours need, while “no repeated vertex” (path) is what makes a route genuinely simple and bounds its length by $|V| - 1$.

Definition 2.2.2 (Cycle). A *cycle* of length $k \geq 3$ is a closed walk $v_0 v_1 \dots v_{k-1} v_0$ in which $v_0, \dots, v_{k-1}$ are distinct (so it is a closed path apart from the forced return to the start). A graph with no cycles is called *acyclic*.

The lower bound $k \geq 3$ is deliberate for simple graphs: there are no loops (a length-1 “cycle”) and no repeated edges (a length-2 back-and-forth $v_0 v_1 v_0$ reuses the edge $\{v_0, v_1\}$), so the shortest genuine cycle is a triangle. The first basic fact is that walks can always be pruned down to paths – the workhorse lemma behind everything that follows.

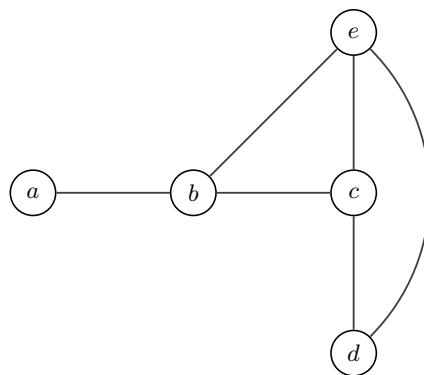
Lemma 2.2.3 (Walks contain paths). *If there is a walk from u to v in G , then there is a path from u to v in G .*

Proof. Among all walks from u to v , choose one of *minimum length*, say $W = v_0 v_1 \dots v_k$ with $v_0 = u$ and $v_k = v$. We claim W is a path. If not, some vertex repeats: $v_i = v_j$ with $i < j$. Deleting the segment strictly between the two visits gives the shorter walk $v_0 \dots v_i v_{j+1} \dots v_k$ from u to v (it is a walk because $v_i = v_j$, so v_i and v_{j+1} are adjacent), contradicting minimality. Hence no vertex repeats and W is a path. $\square$

Worked Example 2.2.4 (Classifying a route as walk, trail, or path). In the graph below, classify each route from a to e : $R_1 = abcbe$, $R_2 = abdcbe$, $R_3 = abcde$.

Recognition. Definition 2.2.1 is a checklist: “repeats a vertex?” separates path from non-path; “repeats an edge?” separates trail from non-trail. Read each route off the diagram.

Solution. Here is the graph; every listed step must be a real edge.



$R_1 = abcbe$ uses edges ab, bc, cb, be . The vertex b repeats, so it is *not* a path; the edge $\{b, c\}$ is used twice (as bc then cb), so it is *not even a trail*. It is a plain walk of length 4. $R_3 = abcde$ uses ab, bc, cd, de , all distinct edges and all five vertices distinct – a *path* of length 4. (The middle route R_2 was a deliberate trap: $\{a, b\}, \{b, d\}$ is not an edge list of this graph, since b and d are not adjacent, so R_2 is not a route at all.)

Which idea, and why? The vertex-repetition test is the sharper one: a repeated vertex already disqualifies “path,” and you only fall back to the edge-repetition test to decide between “trail” and “mere walk.”

Check. R_1 has length 4 on 5 symbols a, b, c, b, e – the duplicate b is visible in the list, confirming it is not a path. R_3 visits each of a, b, c, d, e once.

Connectivity and components

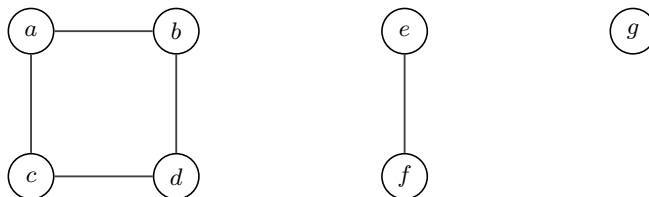
Definition 2.2.5 (Connected, component). Define a relation on V by $u \sim v$ iff there is a walk (equivalently, by Lemma 2.2.3, a path) from u to v . This $\sim$ is an equivalence relation; its equivalence classes are the *connected components* of G . A graph is *connected* if it has exactly one component, i.e. every pair of vertices is joined by a path.

That $\sim$ is an equivalence relation is the quiet engine of the whole subject, so we verify it. *Reflexive*: the length-0 walk v joins v to itself. *Symmetric*: reversing a walk from u to v gives one from v to u . *Transitive*: concatenating a u -to- v walk with a v -to- w walk yields a u -to- w walk. Because the classes of an equivalence relation *partition* the vertex set, every vertex lies in exactly one component, and there is never any edge between two different components – if there were, its endpoints would be joined and hence in the same class.

Worked Example 2.2.6 (Counting components). How many connected components does the following graph have, and what are they?

Recognition. “Component” = one $\sim$ -class = one maximal reachable island. Trace reachability outward from an unvisited vertex until it stops, mark that island, then start fresh from any still-unvisited vertex.

Solution.



Start at a : reachable set grows $\{a\} \rightarrow \{a, b, c\} \rightarrow \{a, b, c, d\}$ and then stops, since no edge leaves $\{a, b, c, d\}$. That is the first component. Now pick the unvisited e : reachability gives $\{e, f\}$ and stops. Finally g has no incident edge, so it is an *isolated vertex* and a component by itself. Three components: $\{a, b, c, d\}$, $\{e, f\}$, and $\{g\}$.

Which idea, and why? Components are the $\sim$ -classes, and $\sim$ is found by flood-fill: expand the reachable frontier from a seed until it closes off, which is precisely one maximal connected piece.

Check. The three classes are disjoint and cover all seven vertices ($4 + 2 + 1 = 7$), and no drawn edge crosses between them – both hallmarks of a partition into components.

The proposition below records how connectivity constrains the edge count; it is the bridge to trees in Section 2.3 and the reason “ $n - 1$ edges” will keep reappearing.

Proposition 2.2.7 (A connected graph has at least $n - 1$ edges). *If G is connected with $n = |V|$ vertices, then $|E| \geq n - 1$.*

Proof. Build G by adding its edges one at a time, starting from the edgeless graph on V , which has n components (one per vertex). Adding a single edge $\{x, y\}$ can lower the component count by *at most one*: it merges the component of x with the component of y if they differ, and changes nothing if they already coincide. To reach a connected graph (1 component) from n components, we must drop the count by $n - 1$, and since each edge drops it by at most 1, at least $n - 1$ edges are required. $\square$

Method (Finding the components of a graph).

1. Mark every vertex “unvisited” and set a component counter to 0.
2. While some vertex is unvisited, pick one as a *seed*, increment the counter, and run a search (below) from it; everything the search reaches is one component.
3. **The search.** Keep a frontier, initially just the seed. Repeatedly take a frontier vertex, mark it visited, and add its still-unvisited neighbours to the frontier. Stop when the frontier empties.
4. When all vertices are visited, the counter is the number of components, and the visited sets recorded per seed are the components themselves.

Distance and breadth-first search

If a path exists, we usually want the *shortest* one. Counting edges along shortest paths turns a graph into a metric space and is the first place graph search becomes quantitative.

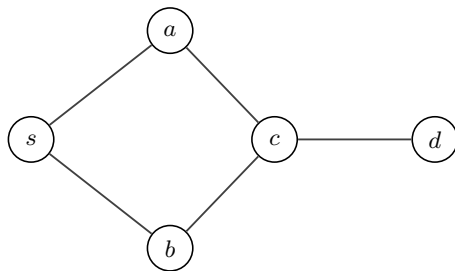
Definition 2.2.8 (Distance). The *distance* $d(u, v)$ between vertices u, v is the length of a shortest u – v walk, or ∞ if no walk exists (i.e. $u \not\sim v$). By Lemma 2.2.3 a shortest walk is a path, so $d(u, v)$ equals the length of a shortest *path*.

Breadth-first search (BFS) computes $d(s, \cdot)$ from a fixed source s by exploring the graph in concentric “layers”: layer 0 is $\{s\}$; layer $i + 1$ is every not-yet-seen vertex adjacent to a layer- i vertex. A vertex’s layer index is exactly its distance from s . The following example runs it.

Worked Example 2.2.9 (Breadth-first search and distances from a source). Run BFS from s on the graph below and read off $d(s, v)$ for every vertex.

Recognition. “Shortest path from one source” is the BFS trigger: peel the graph into distance layers $L_0, L_1, L_2, \dots$, never revisiting a vertex, and the layer a vertex lands in is its distance.

Solution.



$L_0 = \{s\}$. Neighbours of s not yet seen: a, b , so $L_1 = \{a, b\}$. New neighbours of L_1 : from a we reach c , from b we also reach c (recorded once), so $L_2 = \{c\}$. New neighbours of L_2 : from c we reach d , so $L_3 = \{d\}$. The frontier is now empty. Reading layers off as distances:

$$d(s, s) = 0, \quad d(s, a) = d(s, b) = 1, \quad d(s, c) = 2, \quad d(s, d) = 3.$$

Which idea, and why? BFS visits vertices in nondecreasing distance order, so the first time a vertex is seen is along a shortest path; its layer index is therefore its exact distance from s .

Check. A shortest s – d path is $sac d$ (length 3), matching $d(s, d) = 3$; there is no shorter route because d 's only neighbour c already sits at distance 2.

A useful structural fact, also from BFS layering, is that a shortest path never skips a layer: consecutive vertices on it differ in distance by exactly one. We state the cleaner triangle form, which is all later chapters need.

Proposition 2.2.10 (Triangle inequality for graph distance). *For all vertices u, v, w in the same component, $d(u, w) \leq d(u, v) + d(v, w)$.*

Proof. Take a shortest u – v path P (length $d(u, v)$) and a shortest v – w path Q (length $d(v, w)$). Concatenating P then Q is a u – w walk of length $d(u, v) + d(v, w)$. By Lemma 2.2.3 it contains a u – w path, whose length is at most the walk's length, and $d(u, w)$ is the minimum over all such paths, so $d(u, w) \leq d(u, v) + d(v, w)$. $\square$

Euler and Hamilton: traversing edges versus visiting vertices

Two of the oldest questions in graph theory ask for a route that hits everything exactly once – but “everything” can mean the *edges* or the *vertices*, and the two problems could not differ more in difficulty. They preview a theme that drives the whole book: superficially similar problems can sit on opposite sides of the tractable/intractable line.

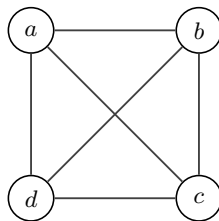
Definition 2.2.11 (Eulerian and Hamiltonian). An *Euler trail* is a trail using every edge of G exactly once; an *Euler circuit* is a closed such trail. A *Hamilton path* is a path visiting every vertex exactly once; a *Hamilton cycle* is a cycle doing so.

The Euler side has a clean, checkable criterion – Euler's theorem: a connected graph has an Euler circuit iff every vertex has *even* degree, and an Euler trail (open) iff exactly two vertices have odd degree. The reason is a local parity count: each time a circuit passes through a vertex it uses two incident edges (one in, one out), so traversing all edges forces every degree to be even. The Hamilton side has *no* such efficient test known; deciding whether a graph has a Hamilton path is (in its directed s -to- t form) the problem *HAMPATH*, which is NP-complete (Chapter 13). Keep the contrast in view: counting degrees is easy, but routing through all vertices is, as far as anyone knows, hard.

Worked Example 2.2.12 (Euler circuit by the even-degree test). Does the graph below have an Euler circuit, an open Euler trail, or neither? If a circuit exists, exhibit one.

Recognition. The Euler criterion is purely a degree-parity check: count odd-degree vertices. Zero of them means a closed Euler circuit; exactly two means an open Euler trail; anything else means neither.

Solution.



This is K_4 . Each vertex is joined to the other three, so $\deg(a) = \deg(b) = \deg(c) = \deg(d) = 3$, which is *odd*. With four odd-degree vertices (not zero and not two), the criterion says *neither* an Euler circuit nor an open Euler trail exists. No traversal can use all six edges without repeating one.

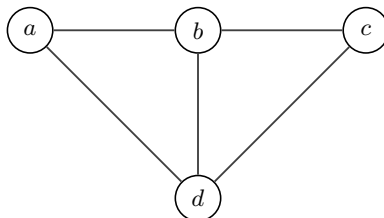
Which idea, and why? Euler's theorem reduces the global routing question to a local parity count; here four odd degrees instantly rule out any edge-exact traversal, so we need not search at all.

Check. Sum of degrees $= 4 \cdot 3 = 12 = 2|E|$, giving $|E| = 6$, the six edges of K_4 – and a graph with more than two odd-degree vertices can never be traced in one edge-disjoint sweep.

Worked Example 2.2.13 (An Euler trail that is not a circuit; and a Hamilton cycle). For the graph below, (i) find an open Euler trail, and (ii) find a Hamilton cycle.

Recognition. Part (i) is again the parity test – look for exactly two odd-degree vertices and start the trail at one of them. Part (ii) is the vertex-visiting question, with no shortcut criterion: we must exhibit a cycle through all vertices by hand.

Solution.



(i) *Euler trail.* Degrees: $\deg(a) = 2$ (ab, ad), $\deg(c) = 2$ (cb, cd), $\deg(b) = 3$ (ba, bc, bd), $\deg(d) = 3$ (da, dc, db). Exactly two odd-degree vertices, b and d , so an open Euler trail exists and must start at one of them. Starting at b : $b a d c b d$ uses ba, ad, dc, cb, bd – all five edges exactly once, ending at d . That is an Euler trail. (ii) *Hamilton cycle.* We need a cycle through all four vertices. Try $abcda$: the required edges are ab, bc, cd, da , and all four are present, returning to a . So $abcda$ is a Hamilton cycle.

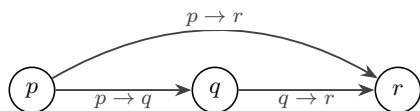
Which idea, and why? The two odd-degree vertices pin down where an Euler trail can begin and end; the Hamilton cycle, having no degree criterion, is produced by exhibiting one concrete vertex ordering whose consecutive pairs are all edges.

Check. The Euler trail $b a d c b d$ lists 6 vertices hence 5 edges, matching $|E| = 5$; the Hamilton cycle $abcda$ visits each of the four vertices once before closing.

Worked Example 2.2.14 (Reachability in a digraph: when direction breaks symmetry). In the directed graph below, decide for each pair of vertices u, v whether u can reach v and vice versa, and explain why connectivity in digraphs is subtler than in graphs.

Recognition. Directed reachability follows arrows *one way*, so the relation “ u reaches v ” is no longer symmetric. Test reachability separately in each direction by tracing forward along arrowheads only.

Solution.



Following arrows: from p we reach q (edge $p \rightarrow q$), r (edge $p \rightarrow r$, or via q). From q we reach r . But from r there are *no* outgoing edges, so r reaches nothing; and q cannot reach p , since both arrows at p point away from it. Thus p reaches everyone, yet no one reaches p . The underlying *undirected* graph (forget arrowheads) is connected, but directed reachability is one-way, so $\sim$ from Definition 2.2.5 is not symmetric here. This is why digraphs distinguish *strong* connectivity (every u reaches every v by a directed path) from *weak* connectivity (the underlying graph is connected).

Which idea, and why? Dropping symmetry is exactly the feature that makes a digraph model a computation: a finite automaton is a digraph whose directed reachability from the start state to an accept state decides membership, and you cannot run those transitions backward.

Check. The pair (p, r) is reachable in the order $p \rightsquigarrow r$ but not $r \rightsquigarrow p$, so the relation genuinely fails symmetry; the underlying undirected p – q – r with the chord pr is nonetheless one connected piece.

Caution. Path versus walk versus trail. The three are routinely conflated. A *walk* permits any repetition; a *trail* forbids repeating an *edge*; a *path* forbids repeating a *vertex*. When a result says “there is a path,” you may assume distinct vertices and length at most $n - 1$; when it says only “there is a walk,” you may not – prune it with Lemma 2.2.3 first.

Caution. An isolated vertex is its own component. Counting components, do not overlook degree-0 vertices: each is a singleton $\sim$ -class. A graph drawn as one “main blob” plus a few stray dots has more than one component, and “connected” fails the moment a single vertex is unreachable from the rest.

Caution. Even degree is the Euler test, not the Hamilton test. The clean degree-parity criterion governs *Euler* routes (every edge once). It says *nothing* about *Hamilton* routes (every vertex once): graphs with all-even degrees can lack a Hamilton cycle, and graphs with odd degrees can have one. There is no known efficient degree criterion for the Hamilton problem – that is the gist of its NP-completeness.

Exercises.

1. Consider the graph on vertices $\{a, b, c, d, e\}$ with edges $\{a, b\}, \{b, c\}, \{c, d\}, \{d, e\}, \{e, a\}, \{a, c\}$. Draw it, then for the route $abcac d$ decide whether it is a walk, a trail, or a path, and give its length. Find a genuine path from b to d and a closed walk that is not a cycle.
2. Prove that in any graph, if there is a trail from u to v then there is a path from u to v . (You may cite Lemma 2.2.3, but state precisely which hypothesis you use.)
3. Draw a connected graph on 6 vertices with exactly 5 edges, and a disconnected graph on 6 vertices with exactly 5 edges. What does Proposition 2.2.7 say must be true of the first, and why does it not contradict the existence of the second?
4. For the graph with vertices $\{1, 2, 3, 4, 5, 6, 7\}$ and edges $\{1, 2\}, \{2, 3\}, \{1, 3\}, \{4, 5\}, \{6, 7\}, \{5, 6\}$, list the connected components and state how many there are. Identify any isolated vertices.
5. Run breadth-first search from vertex s on a graph of your own drawing in which some vertex lies at distance exactly 3 from s . Label each vertex with its distance $d(s, \cdot)$, and exhibit one shortest path realizing the distance 3.
6. Prove that a graph G with at least one vertex is connected if and only if for every partition of V into two nonempty sets X and Y , at least one edge has one endpoint in X and the other in Y .
7. Determine, using only a degree count, whether each graph has an Euler circuit, an open Euler trail, or neither: (a) the cycle C_5 ; (b) the complete bipartite graph $K_{2,3}$; (c) the complete graph K_5 . State the degree of a typical vertex in each.
8. Design a connected graph on exactly 6 vertices that has an open Euler trail but no Euler circuit, and has a Hamilton cycle. Draw it, mark the two odd-degree vertices, and write out both the Euler trail and the Hamilton cycle as vertex sequences.
9. Prove the contrapositive of one direction of Euler’s criterion: if a connected graph has a vertex of odd degree, then it has no Euler circuit. (Hint: count how many edges incident to that vertex a closed trail must use, and argue by parity.)
10. In the directed graph on $\{p, q, r, s\}$ with arcs $p \rightarrow q, q \rightarrow r, r \rightarrow s, s \rightarrow q$, decide which vertices can reach which. Is the digraph strongly connected? Is its underlying undirected graph connected? Explain the difference in one sentence.
11. *Find the bug.* A student claims that “every connected graph on n vertices with exactly $n - 1$ edges contains a cycle.” Draw a small counterexample, and state the correct relationship between “ $n - 1$ edges” and “acyclic” that they should have written instead.
12. *Find the bug.* Someone asserts: “A graph has a Hamilton cycle if and only if every vertex has degree at least 2.” Give a graph in which every vertex has degree at least 2 yet no Hamilton cycle exists, and explain in one sentence why no simple degree condition can settle the Hamilton question.

2.3 Trees and Spanning Trees

A *tree* is the leanest possible connected graph: connected, but with not one edge to spare. This single tension – enough edges to hold the vertices together, but so few that removing any one disconnects the graph – forces a remarkable rigidity, and the one key idea of this section is that several superficially different descriptions of

that rigidity all pick out exactly the same graphs. We prove the equivalence, count the edges of a tree, locate its leaves, add a root to turn an undirected tree into the branching structure used for parse and computation trees later in the book, and finish with spanning trees – the tree skeletons hiding inside every connected graph.

Definition 2.3.1 (Forest, tree, leaf). A graph is *acyclic* if it contains no cycle. A *forest* is an acyclic graph; a *tree* is a connected forest, i.e. a connected acyclic graph. A vertex of degree 1 in a tree is a *leaf*; a vertex of degree ≥ 2 is an *internal* vertex. The connected components of a forest are exactly its constituent trees.

Throughout, $n = |V|$ denotes the number of vertices and $m = |E|$ the number of edges of the graph under discussion. The next two lemmas are the workhorses; they are easy on their own and combine into the characterization theorem.

Lemma 2.3.2 (Every tree on ≥ 2 vertices has a leaf). *A tree T with $n \geq 2$ vertices has at least two leaves.*

Proof. Take a *longest* path $P = v_0 v_1 \cdots v_k$ in T (a longest path exists because T is finite and, with $n \geq 2$ and T connected, has length $k \geq 1$). We claim its endpoint v_0 is a leaf. Suppose not: then v_0 has a neighbour u besides v_1 . If u lies off P , then $u v_0 v_1 \cdots v_k$ is a longer path, contradicting maximality. If instead $u = v_j$ for some $j \geq 2$, then $v_0 v_1 \cdots v_j v_0$ is a cycle, contradicting acyclicity. So v_0 has only the neighbour v_1 and is a leaf; the same argument at v_k gives a second leaf. $\square$

Lemma 2.3.3 (Edge count). *Every tree with n vertices has exactly $n - 1$ edges.*

Proof. Induct on n . For $n = 1$ the only tree is a single vertex with $0 = n - 1$ edges. For $n \geq 2$, by Lemma 2.3.2 pick a leaf ℓ . Deleting ℓ and its one incident edge yields a graph T' on $n - 1$ vertices: T' is still acyclic (deleting cannot create a cycle), and still connected (any path in T between two surviving vertices could only have used ℓ as an interior vertex, but ℓ has degree 1 and cannot be interior). So T' is a tree on $n - 1$ vertices, with $(n - 1) - 1$ edges by induction, and T has one more, namely $n - 1$. $\square$

We can now collect the standard descriptions of a tree and prove they coincide. This is the central result of the section; later we use whichever clause is most convenient.

Theorem 2.3.4 (Equivalent characterizations of a tree). *Let G be a graph with $n \geq 1$ vertices and m edges. The following are equivalent.*

- (i) G is a tree (connected and acyclic).
- (ii) Every two vertices of G are joined by exactly one path.
- (iii) G is connected and $m = n - 1$.
- (iv) G is acyclic and $m = n - 1$.
- (v) G is connected, but deleting any edge disconnects it (minimally connected).
- (vi) G is acyclic, but adding any edge between two non-adjacent vertices creates a cycle (maximally acyclic).

Proof. We show (i) $\Rightarrow$ (ii) $\Rightarrow$ (iii) $\Rightarrow$ (iv) $\Rightarrow$ (i), then (i) $\Leftrightarrow$ (v) and (i) $\Leftrightarrow$ (vi).

(i) $\Rightarrow$ (ii). Connectivity gives at least one path between any u, v . If two distinct u - v paths existed, their symmetric difference would contain a cycle, contradicting acyclicity; so the path is unique.

(ii) $\Rightarrow$ (iii). Unique paths imply connectivity. For the count, the unique-path property makes G acyclic (a cycle would give two paths between two of its vertices), so G is a tree and Lemma 2.3.3 gives $m = n - 1$.

(iii) $\Rightarrow$ (iv). Suppose G is connected with $m = n - 1$ but contains a cycle. Delete one edge of that cycle; the graph stays connected (the deleted edge's endpoints are still joined by the rest of the cycle). Repeat until acyclic; the result is a connected acyclic graph, i.e. a tree, on n vertices, which by Lemma 2.3.3 has $n - 1$ edges. But we deleted at least one edge from the original $n - 1$, leaving $\leq n - 2$, a contradiction. Hence G is acyclic with $m = n - 1$.

(iv) $\Rightarrow$ (i). Let G be acyclic with $m = n - 1$, and let its components be $T_1, \dots, T_c$ with n_i vertices each. Every component is connected and acyclic, hence a tree, so T_i has $n_i - 1$ edges. Summing, $m = \sum_i (n_i - 1) = n - c$. With $m = n - 1$ we get $c = 1$: G is connected, so it is a tree.

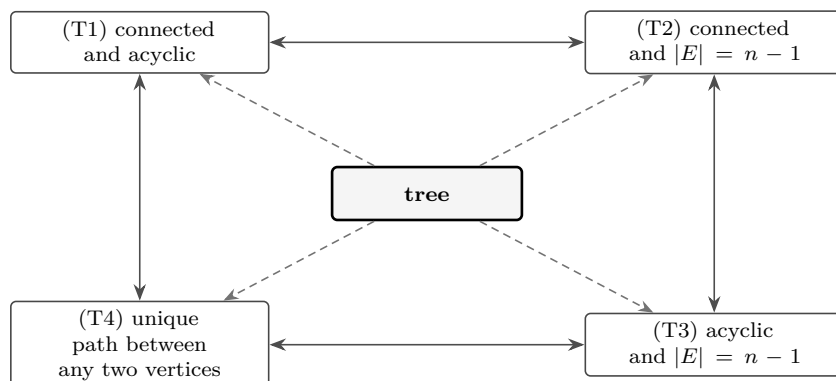
(i) $\Leftrightarrow$ (v). If G is a tree and $e = uv$ is an edge, then by (ii) e itself is the *only* u - v path; deleting it leaves no u - v path, so G disconnects. Conversely, if G is minimally connected it is connected, and acyclic: any edge lying on a cycle could be removed without disconnecting (its endpoints stay joined through the rest of the cycle), violating minimality.

(i) $\Leftrightarrow$ (vi). If G is a tree and u, v are non-adjacent, the unique u - v path together with the new edge uv forms a cycle, so G is maximally acyclic. Conversely a maximally acyclic graph is acyclic; it is connected, for if u, v were in different components the edge uv could be added without creating a cycle (no path between them existed), contradicting maximality. $\square$

Corollary 2.3.5 (Adding or removing one edge). *In a tree, removing any edge produces exactly two components, and adding any edge between non-adjacent vertices produces exactly one cycle.*

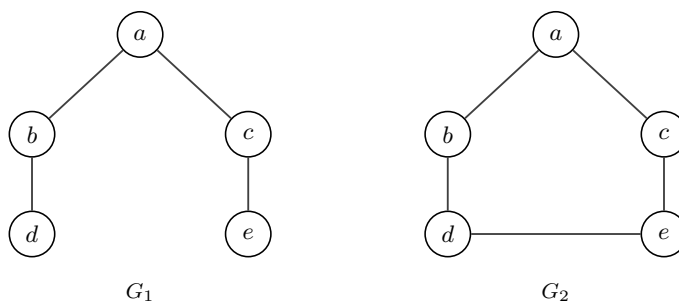
Proof. Removing edge uv disconnects G (clause (v)); the two components are the side reachable from u and the side reachable from v , and there are exactly two because the remaining graph has n vertices and $(n-1)-1 = n-2$ edges, forcing $c = n - (n-2) = 2$ by the count in (iv). Adding a non-edge creates a cycle (clause (vi)); it is unique because two cycles would share the new edge and their symmetric difference would contain a cycle already present in the tree. $\square$

It is worth seeing the four most-used clauses of Theorem 2.3.4 laid out as a closed ring, to make plain that no one of them is the “real” definition – each forces the other three.



Read any double arrow as “ $\Leftrightarrow$ ”: the ring says the four conditions are interchangeable, so each one alone defines the same class (the hub “tree”), and one may verify whichever clause is cheapest on the graph at hand. *Check.* On an n -vertex graph, any two of {connected, acyclic, $|E| = n - 1$ } force the third (Lemma 2.3.3 and clauses (iii), (iv) of Theorem 2.3.4), and a tree has a unique simple path between any two vertices (clause (ii)); hence T1–T4 are equivalent.

Worked Example 2.3.6 (Recognizing a tree from a drawing). Of the two graphs below on vertex set $\{a, b, c, d, e\}$, decide which is a tree.



Recognition. The fastest test pairs the edge count against connectivity (clause (iii)): a tree on n vertices has exactly $n - 1$ edges, so any deviation rules a tree out instantly, and the matching count plus visible connectivity rules one in.

Solution. Here $n = 5$, so a tree needs exactly 4 edges. G_1 has 4 edges and is clearly connected (every vertex reaches a), so G_1 is a tree. G_2 has 5 edges; since $5 \neq 4$, G_2 is not a tree – indeed the extra edge de closes the cycle $bac edb$.

Which idea, and why? Clause (iii) of Theorem 2.3.4 turns “is it a tree?” into a one-glance arithmetic check: count edges, compare with $n - 1$, then confirm connectivity (or, for the converse, acyclicity).

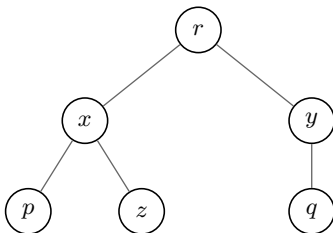
Check. In G_1 , deleting edge ab splits off exactly $\{b, d\}$ from $\{a, c, e\}$ — two components, as Corollary 2.3.5 predicts. In G_2 , the path from b to e is not unique (bde and $bace$), confirming via clause (ii) that G_2 is not a tree.

Rooted trees. Designating one vertex as the *root* orients a tree: every other vertex has a unique path back to the root (clause (ii)), and the first step of that path points at the vertex’s *parent*. This is the form trees take when they record a computation or a derivation.

Definition 2.3.7 (Rooted tree). A *rooted tree* is a tree T with a distinguished vertex r , the *root*. For a vertex $v \neq r$, the *parent* of v is the neighbour of v on the unique v – r path; the other neighbours of v are its *children*, and every neighbour of r is a child of r . A leaf of a rooted tree is a vertex with no children. The *depth* of v is the length of the v – r path; the *height* of T is the maximum depth.

Drawn with the root at top and children below their parent, a rooted tree is the picture we will use for grammar derivations and the branching runs of nondeterministic machines.

Worked Example 2.3.8 (Reading parent, depth, and leaves off a rooted tree). The rooted tree below has root r . Identify the children of x , the depth of z , and all leaves.



Recognition. “Children of x ” and “leaves” are read straight off the parent relation (one level down), and “depth” is the count of edges up to the root, so the trigger is simply: orient by the root, then read levels.

Solution. The children of x are the vertices one level below it, namely p and z . The vertex z sits two edges below r ($z \rightarrow x \rightarrow r$), so its depth is 2. The leaves are the childless vertices: p, z, q . (The root r is internal, not a leaf, and would remain so even if in the *underlying unrooted* tree r had degree 1 — rootedness, not degree, decides leaf-hood here.)

Which idea, and why? In a rooted tree the parent of v is its neighbour toward the root and a leaf is a vertex with no children, so every structural question reduces to walking the unique path to r (Definition 2.3.7).

Check. The tree has 6 vertices and 5 edges, matching $n - 1$ (Lemma 2.3.3); its height is 2, the largest depth attained (by p, z, q).

Spanning trees. A connected graph usually has extra edges beyond a tree’s $n - 1$; throwing away just enough to break every cycle, while keeping the graph connected, leaves a tree on the same vertex set.

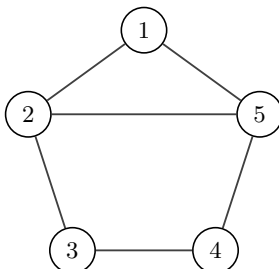
Definition 2.3.9 (Spanning tree). A *spanning tree* of a connected graph $G = (V, E)$ is a subgraph $T = (V, E_T)$ with $E_T \subseteq E$ that is a tree on the *full* vertex set V .

Proposition 2.3.10 (Existence of spanning trees). *Every connected graph has a spanning tree.*

Proof. If G is already acyclic it is its own spanning tree. Otherwise pick any cycle and delete one of its edges; by the argument in clause (iii) of Theorem 2.3.4 the graph stays connected and spans the same vertices. Each deletion removes an edge, so the process halts; it stops only when no cycle remains, leaving a connected acyclic spanning subgraph — a spanning tree. $\square$

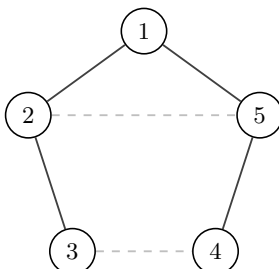
A spanning tree always uses exactly $n - 1$ of G 's edges (Lemma 2.3.3); the $m - (n - 1)$ discarded edges each close one of the graph's *independent cycles* when restored (Corollary 2.3.5). The graph traversals BFS and DFS each produce a spanning tree as a by-product: the tree edges are those that first discover a new vertex, and the *non-tree* edges are exactly the ones whose endpoints were already reached.

Worked Example 2.3.11 (Extracting a spanning tree by deleting cycle edges). The connected graph G below has $n = 5$ vertices and $m = 6$ edges. Find a spanning tree and identify the discarded edges.



Recognition. $m - (n - 1) = 6 - 4 = 2$, so exactly two edges are surplus; the trigger “connected with more than $n - 1$ edges” means cycles are present and we delete one edge per independent cycle (Proposition 2.3.10).

Solution. The outer pentagon $1\ 2\ 3\ 4\ 5\ 1$ is a 5-cycle and the chord 25 creates a second cycle $2\ 5\ 1\ 2$. Delete one edge from each cycle without disconnecting: drop the chord 25 , and drop edge 34 from the outer cycle. What remains, with edges $\{12, 23, 45, 51\}$, is connected (vertex 4 still hangs on through 5) and has $4 = n - 1$ edges, hence a spanning tree by clause (iii).



The two dashed edges 34 and 25 are the discarded ones; restoring either re-creates exactly one cycle.

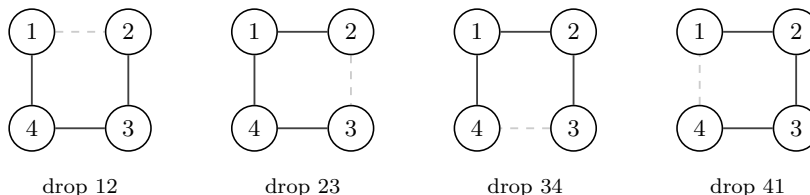
Which idea, and why? Proposition 2.3.10: kill cycles one edge at a time, never deleting an edge whose removal would disconnect the graph; the surviving $n - 1$ edges form the spanning tree.

Check. The spanning tree has 4 edges and is acyclic and connected. A *different* valid choice would keep the chord and drop two pentagon edges instead — spanning trees are generally not unique.

Worked Example 2.3.12 (Counting spanning trees of a small graph). How many distinct spanning trees does the 4-cycle C_4 on $\{1, 2, 3, 4\}$ (edges $12, 23, 34, 41$) have?

Recognition. C_4 is connected with $n = 4$ and $m = 4$, so $m - (n - 1) = 1$: exactly one edge must be removed to reach a tree. The trigger “one surplus edge” means each spanning tree is obtained by deleting a single edge, so we just count which deletions work.

Solution. A spanning tree needs $n - 1 = 3$ edges, so we delete exactly one of the four. Deleting any one edge of a cycle leaves a path on all four vertices, which is connected and acyclic — a spanning tree. There are 4 edges to delete, each giving a different tree (a different path), so C_4 has exactly 4 spanning trees.



Which idea, and why? When a connected graph has exactly one surplus edge ($m = n$), every spanning tree comes from deleting one cycle edge; counting spanning trees becomes counting the deletions that keep the graph connected.

Check. Each of the four pictures is a path on 4 vertices with $3 = n - 1$ edges, connected and acyclic. (For comparison, Cayley's formula gives $4^{4-2} = 16$ labelled spanning trees of the *complete* graph K_4 , far more than C_4 's four, because K_4 has many more edges to choose among.)

Method (How to decide whether a graph is a tree, and how to extract a spanning tree).

1. **Count first.** Compute n and m . If $m \neq n - 1$, the graph is not a tree — stop. This single comparison (clause (iii)/(iv) of Theorem 2.3.4) rejects most non-trees immediately.
2. **Then check one of connectivity or acyclicity.** With $m = n - 1$ in hand, you only need *one* more property: if the graph is connected it is a tree (iii); equivalently, if it is acyclic it is a tree (iv). Verify connectivity by a traversal from any vertex, or acyclicity by checking no traversal ever revisits an already-discovered vertex via a non-tree edge.
3. **To build a spanning tree of a connected G ,** run a graph search (BFS or DFS) from any start vertex; keep precisely the edges that first reach a new vertex and discard every edge whose far endpoint is already discovered. The kept edges number $n - 1$ and form a tree.
4. **To root a tree,** pick any vertex r ; each other vertex's parent is its neighbour on the unique path to r . Reading levels downward gives depths, heights, and leaves.

Caution. Do not treat “ $m = n - 1$ ” as sufficient by itself. A graph with $n - 1$ edges that is *disconnected* is not a tree — it must contain a cycle to compensate. For instance, on 4 vertices the disjoint union of a triangle and an isolated vertex has $3 = n - 1$ edges but is neither connected nor acyclic. The count $m = n - 1$ becomes decisive only *together with* connectivity or acyclicity, never alone.

Caution. “Leaf” means different things rooted and unrooted. In an unrooted tree a leaf is a degree-1 vertex (Definition 2.3.1); in a rooted tree a leaf is a vertex with *no children* (Definition 2.3.7). The root itself, unless it is the only vertex, is never a leaf, and a degree-1 vertex that happens to be chosen as the root is internal, not a leaf. Always state whether a tree is rooted before counting leaves.

Caution. A spanning tree must span *all* of V . A connected acyclic subgraph that misses even one vertex is a subtree, not a spanning tree, no matter how many edges it has. The defining requirement is $T = (V, E_T)$ on the original vertex set V (Definition 2.3.9), with only the edge set trimmed.

Exercises.

1. A graph G has 12 vertices. (a) If G is a tree, how many edges does it have? (b) If G is a forest with exactly 4 connected components, how many edges does it have? State the general formula for a forest with n vertices and c components and justify it from Lemma 2.3.3.
2. Draw all (unlabelled) trees on 5 vertices, up to isomorphism. For each, list its degree sequence and circle its leaves. (There are three such trees.)
3. Decide, with reasons, whether each graph on vertex set $\{a, b, c, d, e, f\}$ is a tree. Use the edge-count test before doing any further work.
 - (a) edges ab, bc, cd, de, ef ;
 - (b) edges ab, ac, ad, ae, af, bc ;
 - (c) edges ab, bc, ca, de, ef .
4. Prove that a tree with at least two vertices in which *every* internal vertex has degree exactly 3 has an even number of vertices. (Hint: combine Lemma 2.3.3 with the handshake sum $\sum_v \deg v = 2m$.)
5. Let T be a tree and let u, v be two non-adjacent vertices. Adding the edge uv creates a unique cycle C (Corollary 2.3.5). Prove that deleting *any* edge of C other than uv from the new graph yields a spanning tree of it. (This is the exchange step behind spanning-tree algorithms.)

6. For the connected graph G on $\{1, 2, 3, 4, 5, 6\}$ with edges 12, 23, 34, 45, 56, 61, 14, (a) compute n , m , and the number of surplus edges $m - (n - 1)$; (b) exhibit two *different* spanning trees by drawing the state of G with the discarded edges dashed; (c) explain why the chord 14 cannot appear together with all of 12, 23, 34 in a single spanning tree.
7. A rooted tree is drawn with root r and the following parent relation: the parent of a and b is r ; the parent of c and d is a ; the parent of e is c . (a) Draw the rooted tree with the root at top. (b) List all leaves. (c) Give the depth of each vertex and the height of the tree. (d) How many edges does it have, and does this match $n - 1$?
8. Prove that in any tree with $n \geq 2$ vertices, the number of leaves equals $2 + \sum_{v \text{ internal}} (\deg v - 2)$. Interpret the formula: a tree starts with two leaves and gains one extra leaf for each unit of internal degree above 2.
9. *Find the bug.* A classmate claims the following procedure always returns a spanning tree of a connected graph G : “repeatedly delete any edge whose two endpoints both have degree ≥ 2 , until no such edge remains.” Give a small connected graph on which this procedure either disconnects G or fails to remove all cycles, and explain precisely which deletion goes wrong.
10. Let G be connected with n vertices and m edges. Prove that G is a tree if and only if G has exactly one spanning tree (namely G itself). (One direction uses Proposition 2.3.10; the other uses Corollary 2.3.5.)
11. Show that every tree is *bipartite*. (Hint: root the tree and 2-colour each vertex by the parity of its depth; argue every edge joins an even-depth vertex to an odd-depth one. This previews the bipartite material of the next section.)
12. *Find the bug.* Below is an attempted “proof” that every connected graph with $m = n - 1$ edges is acyclic. Locate the unjustified step. “Suppose G is connected with $m = n - 1$ but has a cycle C . Delete an edge of C ; the graph stays connected, so it is still a tree, so it has $n - 1$ edges. But we deleted an edge, so it has $n - 2$ edges — contradiction.” Which sentence asserts something not yet established at that point in the argument, and how does Theorem 2.3.4 repair it?

2.4 Bipartite Graphs, Matchings, and Coloring

Three closely linked questions organize this section: *when can a graph’s vertices be split into two “sides” with all edges crossing between them* (bipartiteness), *when can we pair up vertices so no two pairs share a vertex* (matching), and *how few colors suffice so that adjacent vertices differ* (coloring). The single idea tying them together is a partition of the vertices into *independent* classes – bipartiteness is a partition into two such classes, coloring is a partition into k of them, and matching is the dual notion of pairing across a bipartition. These problems also mark the boundary between the easy and the hard in this book: bipartiteness and bipartite matching are solvable by fast graph search, whereas deciding the chromatic number is one of the first NP-complete problems we will meet. We build the theory on the small graphs we drew in the previous sections.

Bipartite graphs and 2-colorability

Definition 2.4.1 (Bipartite graph; complete bipartite graph). A graph $G = (V, E)$ is *bipartite* if its vertex set can be partitioned into two disjoint sets U and W (the *parts*) so that every edge joins a vertex of U to a vertex of W ; no edge lies inside U or inside W . We write the bipartition as $G = (U \cup W, E)$. If in addition every $u \in U$ is joined to every $w \in W$, the graph is the *complete bipartite graph* $K_{m,n}$, where $m = |U|$ and $n = |W|$.

A *proper 2-coloring* of G assigns each vertex one of two colors so that adjacent vertices get different colors. The two color classes are exactly the two parts of a bipartition, so the next statement is almost a restatement of the definition; its content is the third, checkable characterization in terms of cycles.

Theorem 2.4.2 (Characterization of bipartite graphs). *For a graph G the following are equivalent:*

- (i) G is bipartite;

(ii) G has a proper 2-coloring;

(iii) G contains no cycle of odd length.

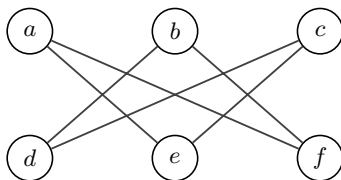
Proof. (i) $\Leftrightarrow$ (ii) is immediate: color U red and W blue. An edge inside a part would join two same-colored vertices, and a monochromatic edge would put both endpoints in one part; so a valid bipartition and a proper 2-coloring are the same data.

(i) $\Rightarrow$ (iii). Suppose $G = (U \cup W, E)$ is bipartite and let $v_0 v_1 \cdots v_{k-1} v_0$ be a cycle. Walking along it, every edge switches sides, so the vertices alternate $U, W, U, W, \dots$. To return to the starting vertex's side after k steps the number of switches k must be even. Hence every cycle has even length.

(iii) $\Rightarrow$ (i). Assume G has no odd cycle; we 2-color it. Color one component at a time, so assume G is connected. Pick a root r , run a breadth-first search, and color each vertex x by the parity of its distance $d(r, x)$: even distance goes to U , odd distance to W . We claim no edge is monochromatic. If an edge xy joined two vertices of equal distance-parity, then combining a shortest $r-x$ path, the edge xy , and a shortest $y-r$ path yields a closed walk of odd length (two paths of equal parity plus one edge), and a closed walk of odd length always contains an odd cycle – contradicting (iii). So every edge crosses between the parts, and G is bipartite. $\square$

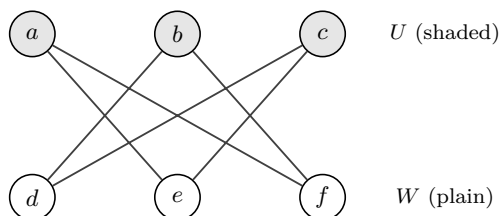
The equivalence (ii) $\Leftrightarrow$ (iii) is what makes bipartiteness *efficiently testable*: the BFS 2-coloring either succeeds (producing the parts) or hits a monochromatic edge (exposing an odd closed walk), in time proportional to the size of the graph. We return to this algorithmic view in Section 2.5.

Worked Example 2.4.3 (Color a graph to read off its bipartition). Decide whether the graph below is bipartite, and if so exhibit the two parts.



Recognition. The task is exactly Theorem 2.4.2(ii): try to 2-color. Start anywhere, force the opposite color across each edge, and watch for a clash. The trigger “partition into two sides with all edges crossing” is the same thing as a proper 2-coloring.

Solution. Root the coloring at a and color it U (shaded). Its neighbors e, f are forced to W (plain) via the edges ae, af . Each plain vertex then forces its other neighbors back to U : from e the edge ce forces $c \in U$, from f the edge bf forces $b \in U$. Finally b forces $d \in W$ via bd , and the last edge dc checks out ($d \in W, c \in U$). Redrawing with U on top and W on the bottom makes the bipartition explicit.



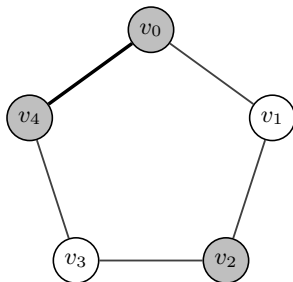
Which idea, and why? A successful greedy 2-coloring *is* a bipartition; the color classes are the parts, so finding the coloring and finding the partition are one and the same step.

Check. The bipartition is $U = \{a, b, c\}$, $W = \{d, e, f\}$. The six edges ae, bd, bf, ce, af, dc each have one shaded (U) and one plain (W) endpoint, and neither part contains an internal edge – so the partition is valid and the graph is bipartite.

Worked Example 2.4.4 (An odd cycle blocks bipartiteness). Show that the 5-cycle C_5 is not bipartite, and locate the obstruction.

Recognition. By Theorem 2.4.2(iii) the only obstruction to bipartiteness is an odd cycle, and C_5 is an odd cycle. The trigger is the cycle's odd length 5.

Solution. Attempt the forced 2-coloring around the rim $v_0v_1v_2v_3v_4v_0$. Color v_0 red; then v_1 blue, v_2 red, v_3 blue, v_4 red. Now the closing edge v_4v_0 joins red to red – a monochromatic edge. The alternation cannot close up because 5 is odd.



shaded = red, plain = blue;
bold edge v_4v_0 is the forced clash

Which idea, and why? Odd length is the exact certificate of non-bipartiteness; the forced alternation uses up the two colors and the final wrap-around edge must repeat a color, which is the monochromatic edge the BFS test would report.

Check. Any 2-coloring of C_5 has $\lceil 5/2 \rceil = 3$ vertices of one color among the five, and three vertices on a 5-cycle always include an adjacent pair; so a clash is unavoidable.

Matchings

Definition 2.4.5 (Matching, maximum, perfect). A *matching* in a graph G is a set $M \subseteq E$ of edges no two of which share a vertex. A vertex incident to an edge of M is *matched* (or *saturated*); otherwise it is *unmatched*. A matching is *maximum* if no matching of G has more edges, and it is *perfect* if it saturates every vertex. The *matching number* $\nu(G)$ is the number of edges in a maximum matching.

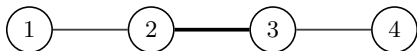
A perfect matching pairs up all the vertices, so it can exist only when $|V|$ is even, and then $\nu(G) = |V|/2$. “Maximum” (the largest possible) must not be confused with “maximal” (one that cannot be extended by adding an edge): a maximal matching can be far from maximum, as the next example shows.

Worked Example 2.4.6 (Maximal is not maximum). In the path P_4 on vertices 1 2 3 4, exhibit a maximal matching that is not maximum.

Recognition. The pitfall is the maximal/maximum distinction drawn after Definition 2.4.5. The trigger is a single edge that, once chosen, blocks both of the others from being added.

Solution. The middle edge $\{2, 3\}$ is a matching of size 1. Its endpoints 2 and 3 are now used, so neither $\{1, 2\}$ nor $\{3, 4\}$ can be added: the matching is *maximal*. But the two end edges $\{1, 2\}$ and $\{3, 4\}$ together form a matching of size 2, which is *maximum* (and perfect).

maximal, size 1 (bold $\{2, 3\}$)



maximum, size 2 (bold ends)

Which idea, and why? Maximality is a *local* stopping condition (no edge can be added), maximality of size is a *global* optimum; a greedy first edge can land at a local stop that is globally too small.

Check. P_4 has 4 vertices, so any perfect matching has 2 edges; the bottom matching saturates all four vertices and the top saturates only two, confirming $\nu(P_4) = 2$.

Hall's theorem

The central existence question for bipartite graphs is: when can we match *every* vertex of one side? For a set S of vertices write $N(S)$ for its *neighborhood*, the set of all vertices adjacent to at least one vertex of S . If some k -set $S \subseteq U$ has fewer than k neighbors, its k vertices must all be matched into $N(S)$, which is impossible – so $|N(S)| \geq |S|$ is clearly *necessary*. Hall's theorem says it is also *sufficient*.

Theorem 2.4.7 (Hall's marriage theorem). *Let $G = (U \cup W, E)$ be bipartite. There is a matching saturating every vertex of U if and only if Hall's condition holds:*

$$|N(S)| \geq |S| \quad \text{for every } S \subseteq U.$$

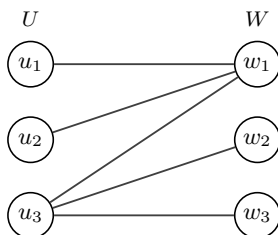
Proof. ($\Rightarrow$) If a matching M saturates U , then each $s \in S$ is matched to a distinct partner in W , and those partners lie in $N(S)$; distinct s give distinct partners, so $|N(S)| \geq |S|$.

($\Leftarrow$) We induct on $|U|$. If $|U| \leq 1$ the condition $|N(\{u\})| \geq 1$ gives a neighbor to match u to. For the step, distinguish two cases.

Case 1: every nonempty proper $S \subsetneq U$ has slack, $|N(S)| \geq |S| + 1$. Pick any $u \in U$ and any neighbor $w \in N(\{u\})$, match them, and delete both. For any $S \subseteq U \setminus \{u\}$, deleting w removes at most one neighbor, so in the smaller graph $|N(S)| \geq (|S| + 1) - 1 = |S|$. Hall's condition survives, the induction hypothesis matches $U \setminus \{u\}$, and adding the edge uw saturates all of U .

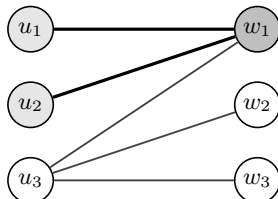
Case 2: some nonempty proper $T \subsetneq U$ is tight, $|N(T)| = |T|$. The bipartite graph on $T \cup N(T)$ satisfies Hall's condition (any $S \subseteq T$ has its neighbors inside $N(T)$), so by induction it has a matching saturating T . Remove T and $N(T)$ entirely. In the remaining graph on $(U \setminus T) \cup (W \setminus N(T))$, take any $S \subseteq U \setminus T$. Back in the full graph $|N(S \cup T)| \geq |S \cup T| = |S| + |T|$, while $N(S \cup T) \subseteq N(S)' \cup N(T)$ where $N(S)'$ is the neighborhood in the remaining graph; since $|N(T)| = |T|$, subtracting gives $|N(S)'| \geq |S|$. Hall's condition holds again, so induction matches $U \setminus T$. The two matchings use disjoint vertices and together saturate U . $\square$

Worked Example 2.4.8 (Apply Hall's condition, and find the violating set). For the bipartite graph below with parts $U = \{u_1, u_2, u_3\}$ and $W = \{w_1, w_2, w_3\}$, decide whether U can be saturated.



Recognition. The question “can U be saturated” is exactly Theorem 2.4.7. The trigger is the crowding visible in the picture: u_1 and u_2 each touch only w_1 .

Solution. Test the suspicious set $S = \{u_1, u_2\}$. Both vertices have w_1 as their only neighbor, so $N(S) = \{w_1\}$ and $|N(S)| = 1 < 2 = |S|$. Hall's condition fails on S , so no matching can saturate U : the two vertices u_1, u_2 are competing for the single vertex w_1 , and one of them must go unmatched.



$$S = \{u_1, u_2\} \text{ (shaded), } N(S) = \{w_1\}: \\ |N(S)| = 1 < 2 = |S|$$

Which idea, and why? Hall's theorem reduces a global existence question to a local count over subsets; one deficient set S (more vertices than collective neighbors) is a self-contained certificate that no saturating matching exists.

Check. A matching saturating U would need three distinct partners in W for u_1, u_2, u_3 , but u_1, u_2 force two of those partners to be w_1 – a contradiction, confirming the verdict.

Corollary 2.4.9 (Regular bipartite graphs have perfect matchings). *Every k -regular bipartite graph (every vertex of degree k) with $k \geq 1$ has a perfect matching.*

Proof. Let $G = (U \cup W, E)$ be k -regular. Counting edges from each side, $k|U| = |E| = k|W|$, so $|U| = |W|$; a matching saturating U is then automatically perfect. To verify Hall's condition, fix $S \subseteq U$. The edges leaving S number exactly $k|S|$, and every one ends in $N(S)$; but each vertex of $N(S)$ absorbs at most k edges, so $k|S| \leq k|N(S)|$, i.e. $|N(S)| \geq |S|$. By Theorem 2.4.7 a saturating – hence perfect – matching exists. $\square$

Graph coloring and the chromatic number

Definition 2.4.10 (Proper coloring; chromatic number). A *proper k -coloring* of $G = (V, E)$ is a function $c: V \rightarrow \{1, \dots, k\}$ with $c(x) \neq c(y)$ whenever $xy \in E$. The graph is *k -colorable* if such a coloring exists. The *chromatic number* $\chi(G)$ is the least k for which G is k -colorable.

Each color class $c^{-1}(i)$ is an *independent set* – a set of pairwise non-adjacent vertices – so a proper k -coloring is the same as a partition of V into k independent sets. In this language, $\chi(G) \leq 2$ exactly says G is bipartite (Theorem 2.4.2), and $\chi(G) = 1$ exactly says G has no edges. Two easy bounds anchor the rest.

Proposition 2.4.11 (Clique and greedy bounds). *Let $\omega(G)$ be the size of the largest complete subgraph (clique) in G and let Δ be the maximum degree. Then*

$$\omega(G) \leq \chi(G) \leq \Delta + 1.$$

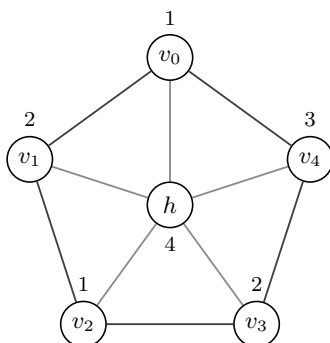
Proof. Lower bound. The vertices of a clique are pairwise adjacent, so a proper coloring must give them all distinct colors; a clique on ω vertices already forces ω colors.

Upper bound. Order the vertices $v_1, \dots, v_n$ arbitrarily and color greedily: give v_i the smallest color in $\{1, \dots, \Delta + 1\}$ not used by its already-colored neighbors. When v_i is reached it has at most Δ neighbors, hence at most Δ forbidden colors, so among the $\Delta + 1$ available colors at least one is free. The process never gets stuck, producing a proper $(\Delta + 1)$ -coloring. $\square$

Worked Example 2.4.12 (Chromatic number of a wheel). Compute $\chi(W_5)$, the wheel formed by a 5-cycle plus a hub joined to all five rim vertices.

Recognition. We need a coloring (upper bound) together with a clique or parity obstruction (lower bound). The trigger is the odd rim: the rim is C_5 , which Theorem 2.4.2 already tells us is not 2-colorable.

Solution. The rim C_5 is an odd cycle, so coloring it alone needs 3 colors – alternate 1, 2, 1, 2 around four rim vertices and give the fifth a third color 3. The hub is adjacent to all rim vertices, so it cannot reuse any rim color; a fourth color 4 on the hub completes a proper 4-coloring, giving $\chi(W_5) \leq 4$. For the lower bound, take any rim edge $\{v_i, v_{i+1}\}$ together with the hub h : these three vertices are mutually adjacent, a triangle (clique of size 3), forcing 3 colors. That alone gives only $\chi(W_5) \geq 3$ (W_5 has no K_4); but the odd rim by itself needs 3 colors, and the hub, adjacent to every rim vertex, can reuse none of them, so no proper coloring uses fewer than 4, and $\chi(W_5) = 4$.



Which idea, and why? The clique bound $\omega \leq \chi$ gives only 3 (the hub-plus-edge triangle); the odd rim together with the hub adjacent to all of it supplies the true lower bound 4, and the explicit greedy-style coloring supplies the matching upper bound; together they pin χ exactly.

Check. The displayed colors are all distinct across every drawn edge: rim colors alternate 1, 2, 1, 2, 3 with no equal neighbors, and the hub's 4 differs from all of 1, 2, 3, so the 4-coloring is proper and $\chi(W_5) = 4$.

Remark. Computing $\chi(G)$ in general is hard: the decision problem “is $\chi(G) \leq k$?” is NP-complete already for $k = 3$ (a classical result; Chapter 12 shows that 3-coloring is in NP). So Proposition 2.4.11 is typically the best we get cheaply – the two bounds can be far apart, and closing the gap is exactly the intractable part. The contrast with bipartiteness ($k = 2$, decidable by one BFS) is the first concrete glimpse of the easy/hard frontier that the rest of the book maps out.

Method (How to find the chromatic number of a small graph).

1. **Get a lower bound.** Find the largest clique you can; it forces that many colors. A single edge gives 2, a triangle gives 3. An odd cycle of length at least 5 (no clique larger than 2) still forces 3 by Theorem 2.4.2, so check parity of cycles too.
2. **Get an upper bound by coloring.** Order the vertices (high-degree first often helps) and color greedily with the smallest legal color. The number of colors used is an upper bound; by Proposition 2.4.11 you never need more than $\Delta + 1$.
3. **Squeeze.** If the lower and upper bounds meet, that common value *is* $\chi(G)$. If they differ, try to recolor more cleverly to lower the upper bound, or find a larger clique / odd structure to raise the lower bound.
4. **Sanity-check the coloring.** Walk every edge and confirm its endpoints differ. One monochromatic edge means the coloring is not proper.

Caution. Do not read “ G has no triangle” as “ $\chi(G) \leq 2$.” The clique bound is only a *lower* bound; triangle-free graphs can need many colors. The cleanest small witness is the odd cycle C_5 : it has no triangle ($\omega = 2$) yet $\chi(C_5) = 3$, because its odd length blocks 2-coloring by Theorem 2.4.2. Bipartiteness, not triangle-freeness, is the condition equivalent to 2-colorability.

Caution. Keep *maximal* and *maximum* distinct for matchings (and the analogous *maximal* vs *maximum* for independent sets). A maximal matching merely cannot be enlarged by adding one edge; a maximum matching has the most edges of any matching. Greedily grabbing edges yields a maximal matching, which may be only half the size of the maximum – so “I cannot add another edge” is *not* a proof of optimality.

Caution. When applying Hall’s theorem, remember the condition must hold for *every* subset $S \subseteq U$, not just for single vertices or for U itself. A graph can have “enough neighbors” for each individual vertex and for the whole side, yet fail on a middle-sized set (as when two vertices share their only neighbor). Finding one deficient set S with $|N(S)| < |S|$ is what certifies that no saturating matching exists.

Exercises.

1. Decide whether each graph is bipartite by attempting a 2-coloring; if it is, draw the two parts, and if it is not, exhibit an odd cycle. (a) The cube graph Q_3 (vertices are the 3-bit strings, edges join strings differing in one bit). (b) The complete graph K_4 . (c) The “bowtie”: two triangles sharing a single common vertex.
2. Draw the complete bipartite graph $K_{3,3}$. Count its edges and verify the edge bound $|E| \leq \lfloor n^2/4 \rfloor$ for $n = 6$ vertices. Then state, with a one-line reason, the value of $\chi(K_{3,3})$.
3. Prove that a graph is bipartite if and only if every one of its connected components is bipartite. (Use the part of Theorem 2.4.2 relating bipartiteness to odd cycles.)
4. For the bipartite graph with $U = \{u_1, u_2, u_3, u_4\}$, $W = \{w_1, w_2, w_3, w_4\}$ and edges $u_1w_1, u_1w_2, u_2w_2, u_2w_3, u_3w_3, u_3w_4, u_4w_4, u_4w_1$, draw the graph, exhibit a perfect matching, and verify Hall’s condition for at least three different subsets $S \subseteq U$ including one of size 3.
5. Build a bipartite graph with $|U| = |W| = 4$ that has *no* matching saturating U , and point to the specific subset $S \subseteq U$ that violates Hall’s condition. Make your counterexample as small (fewest edges removed from $K_{4,4}$) as you can.

6. In the path P_7 on vertices 1 2 3 4 5 6 7, list a maximal matching that is *not* maximum, and a maximum matching. State $\nu(P_7)$ and explain in one sentence why P_7 has no perfect matching.
7. Prove directly from Definition 2.4.5 that in any graph G with no isolated vertices, a maximum matching M leaves an *independent* set of vertices unmatched. (Hint: if two unmatched vertices were adjacent, you could enlarge M .)
8. Use Corollary 2.4.9 to show that a 2-regular bipartite graph is a disjoint union of even cycles, and exhibit a perfect matching in one such graph on 8 vertices by drawing it and highlighting the matched edges.
9. Find $\chi(G)$ for each graph and justify both the lower and the upper bound: (a) the complete graph K_5 ; (b) the cycle C_6 ; (c) the Petersen-style “two pentagons” graph consisting of an outer C_5 , an inner C_5 , and spokes joining corresponding vertices. Draw your colorings.
10. Run the greedy coloring of Proposition 2.4.11 on C_6 in the vertex order $v_0, v_3, v_1, v_4, v_2, v_5$ and again in the order $v_0, v_1, v_2, v_3, v_4, v_5$. Report the number of colors each order uses, and explain why one order beats the other even though $\chi(C_6)$ is the same in both.
11. *Find the bug.* A student claims “a graph is 2-colorable if and only if it has no triangle.” Give the smallest graph that refutes this claim, state its chromatic number, and name the correct condition that is equivalent to 2-colorability.
12. *Find the bug.* Someone asserts that the bipartite graph $U = \{u_1, u_2, u_3\}$, $W = \{w_1, w_2, w_3\}$ with edges $u_1w_1, u_2w_2, u_3w_1, u_3w_2$ has a perfect matching “because every u_i has a neighbor.” Draw the graph, find the subset $S \subseteq U$ that breaks Hall’s condition, and explain precisely why per-vertex neighbors are not enough.

2.5 Graphs as Computational Objects

Every machine in this book is, when you look at it, a graph. A finite automaton is a labelled digraph whose vertices are states and whose arcs are transitions; a Turing machine’s space of configurations is a digraph; a grammar’s derivations form trees. This closing section turns the relationship around and asks the computational questions: *how* do we hand a graph to an algorithm, and *which* questions about graphs are easy or hard? The single idea tying the chapter to the rest of the book is that “running a machine” and “searching a graph for a path” are the same activity, so the data structures and the complexity of graph problems are the foundation on which decidability and the classes P, NP, and friends are built.

Encoding a graph as a string

A decision problem about graphs – “is G connected?”, “does G have a Hamiltonian path?” – is a *language*: the set of (encodings of) graphs with the property. To talk about machines reading graphs we must first fix how a graph becomes a finite string over a finite alphabet. We write $\langle G \rangle$ for such an encoding.

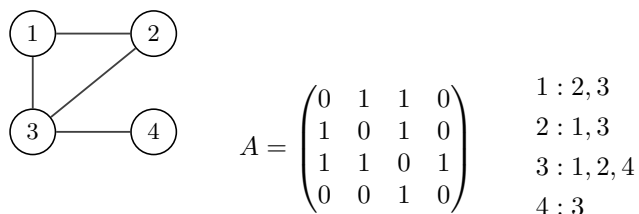
Definition 2.5.1 (Encoding of a graph). Let $G = (V, E)$ be a finite graph with vertices $V = \{v_1, \dots, v_n\}$. An *encoding* $\langle G \rangle$ is any string over a fixed finite alphabet that determines G up to isomorphism and from which n , the adjacency of any two named vertices, and the degree of any vertex can be recovered by an algorithm. Two standard choices are the *adjacency-list* encoding – the list of vertices, each followed by its neighbours – and the *adjacency-matrix* encoding – the $n \times n$ matrix A with $A_{ij} = 1$ iff $v_i v_j \in E$, written out row by row.

The exact format does not matter, only that translating between two reasonable encodings can be done in polynomial time; this is what lets us speak of “the” complexity of a graph problem without naming an encoding. The matrix uses $\Theta(n^2)$ bits regardless of how many edges there are, while the list uses $\Theta(n + m)$ entries (vertex names of $O(\log n)$ bits each) for m edges, which is far smaller for *sparse* graphs (few edges) and is the encoding behind near-linear graph algorithms.

Worked Example 2.5.2 (Encoding one small graph two ways). Encode the graph G on vertices $\{1, 2, 3, 4\}$ with edges $\{12, 13, 23, 34\}$ as both an adjacency matrix and an adjacency list.

Recognition. Two representations of the *same* edge set are wanted; the trigger is “write down $\langle G \rangle$,” which means choose a format and read the adjacencies off the picture.

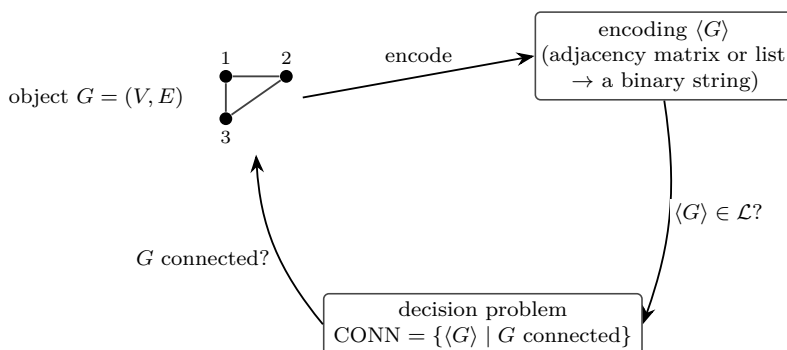
Solution. Draw G , then record for each unordered pair whether it is an edge.



Which idea, and why? An undirected simple graph is determined by its symmetric 0/1 adjacency relation, so both the matrix (the relation in full) and the list (the relation as neighbour sets) carry exactly G ; they differ only in size and in which queries are fast.

Check. The matrix is symmetric with zero diagonal, as it must be for a simple undirected graph; summing row 3 gives degree 3, matching the three neighbours $\{1, 2, 4\}$ in the list, and the four off-diagonal 1s above the diagonal match the four edges.

The encoding is the hinge that lets a machine touch a graph at all, so it is worth seeing the three faces of a single graph side by side – the object, its string, and the yes/no question a machine actually decides.



The single graph G appears as an object (top left), as the string $\langle G \rangle$ the encoder produces (right), and as the question of whether that string lies in the language CONN (bottom); because the encoding is a fixed scheme, invertible on its image, the loop closes and “ G has the property” becomes exactly “ $\langle G \rangle$ is in the language.” *Check.* The map $G \mapsto \langle G \rangle$ is computable and invertible on its image (one can recover G up to isomorphism from $\langle G \rangle$, Definition 2.5.1), so for any property P the set $\mathcal{L}_P = \{\langle G \rangle \mid G \text{ has } P\}$ is a language with $G \in P \iff \langle G \rangle \in \mathcal{L}_P$.

Automata and transition systems are labelled digraphs

The state diagrams of Chapters 4 and 5 are not just *drawn* as graphs; they *are* digraphs with edge labels from Σ . Making this explicit lets us reuse graph algorithms on automata, and conversely lets us picture any step-by-step process as a graph to be searched.

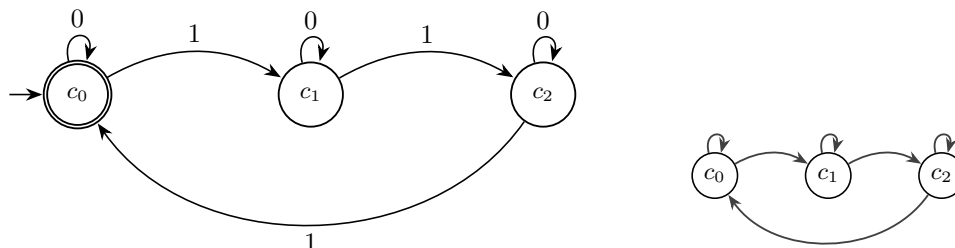
Definition 2.5.3 (Transition system / labelled digraph of a machine). The *underlying labelled digraph* of a DFA $M = (Q, \Sigma, \delta, q_0, F)$ has vertex set Q and, for each $q \in Q$ and $a \in \Sigma$, a directed arc $q \rightarrow \delta(q, a)$ labelled a . A string $w = a_1 a_2 \cdots a_k$ is accepted iff there is a directed walk from q_0 to some $f \in F$ whose edge labels, read in order, spell w . In this language, *running the machine on w is following the unique w -labelled walk out of q_0 .*

For a DFA the walk is unique (determinism), so acceptance is a reachability question along one labelled walk; for an NFA several walks may spell w , and acceptance asks whether *some* walk ending at an accept vertex spells w . Dropping the labels entirely leaves a plain digraph in which the only question is which states are reachable at all – exactly the data needed to decide whether a machine has any useful states.

Worked Example 2.5.4 (The same object as automaton and as digraph). Take the 3-state machine over $\{0, 1\}$ that counts the number of 1s read modulo 3, accepting when the count is $\equiv 0$. Show it as a labelled state diagram and strip it to its underlying digraph.

Recognition. A machine is asked to be viewed two ways; the trigger is Definition 2.5.3: keep the labels for the automaton, erase them for the bare reachability picture.

Solution. With states c_0, c_1, c_2 for the residue of the 1-count, a 1 advances the residue and a 0 loops in place. Start and accept at c_0 .



Which idea, and why? The state diagram *is* the labelled digraph of the machine; erasing the symbol labels leaves the directed reachability structure, the object graph-search algorithms actually consume.

Check. Every vertex of the stripped digraph has out-degree 2 – one arc per alphabet symbol – as a DFA over a two-symbol alphabet must, and the 1-arcs form one directed 3-cycle, the residue cycle.

Reachability is graph search

The most basic algorithmic question on a digraph is *reachability*: given a source s and a target t , is there a directed path from s to t ? Breadth-first search (BFS) answers it by exploring vertices in waves of increasing distance from s ; depth-first search (DFS) by following each path to its end before backtracking. We package the decision problem as a language.

Definition 2.5.5 (The reachability language).

$$PATH = \{\langle G, s, t \rangle \mid G \text{ is a digraph with a directed } s\text{-to-}t \text{ path}\}.$$

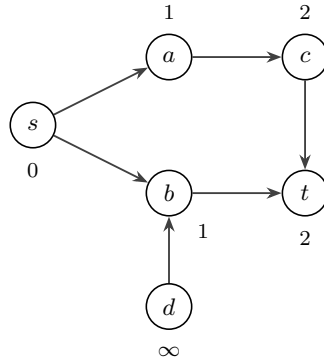
Theorem 2.5.6 (Reachability is decidable in polynomial time). $PATH \in P$.

Proof. On input $\langle G, s, t \rangle$ with G on n vertices, run BFS from s : maintain a set R of reached vertices, initially $\{s\}$, and a queue. Repeatedly dequeue a vertex u and, for each arc $u \rightarrow v$, if $v \notin R$ add v to R and enqueue it. Each vertex is enqueued at most once and each arc is examined at most once, so the loop runs in $O(n + m)$ steps, polynomial in the input length. Accept iff $t \in R$ when the queue empties. Correctness: a simple induction on distance shows $v \in R$ exactly when v is reachable from s , because a vertex enters R only via an arc from an already-reached vertex, and every reachable vertex is found at the wave matching its distance. $\square$

Worked Example 2.5.7 (BFS layers and a reachability verdict). In the digraph below decide whether t is reachable from s , and label each reachable vertex with its BFS distance from s .

Recognition. A source-to-target reachability question; the trigger is Definition 2.5.5, so run BFS and read off the distance layers.

Solution. Wave 0 is $\{s\}$. Wave 1 is the out-neighbours of s , namely a and b . Wave 2 is the new out-neighbours of those, namely c (from a) and t (from b). The vertex d has no incoming arc (its only arc, $d \rightarrow b$, points away from it), so it is never reached.



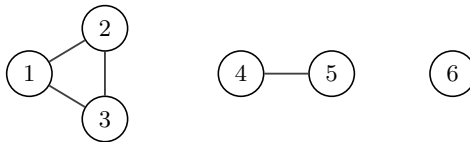
Which idea, and why? BFS settles reachability and shortest-path distance together: a vertex’s BFS wave number is its distance from s , and any vertex never enqueued (here d) is exactly an unreachable one.

Check. Two directed s -to- t paths exist, sbt and $sact$, of lengths 2 and 3; BFS reports the shorter distance 2 at t , and the verdict is “reachable.” Vertex d has out-degree 1 into b but no incoming arc, so nothing reaches it – distance ∞ .

Worked Example 2.5.8 (Connected components by repeated search). Given the undirected graph below, list its connected components by launching a search from each not-yet-visited vertex.

Recognition. “Group the vertices that can reach one another” is the *components* question; the trigger is that reachability in an undirected graph is an equivalence relation, whose classes are the components, each found by one search.

Solution. Search from 1: it reaches 2, 3, closing the first component $\{1, 2, 3\}$. The next unvisited vertex is 4; it reaches 5, giving $\{4, 5\}$. Vertex 6 is isolated, forming $\{6\}$ alone.



Which idea, and why? Components are the reachability classes of an undirected graph; one search per unvisited vertex visits exactly one component, so the number of searches launched equals the number of components.

Check. Three searches were launched (from 1, 4, 6), giving three components $\{1, 2, 3\}$, $\{4, 5\}$, $\{6\}$ that partition all six vertices and overlap nowhere, as a partition into equivalence classes must.

Why graph problems pervade complexity

Reachability is easy. Most interesting graph properties are not known to be – and the boundary between the two is where complexity theory lives. Two problems will recur throughout the rest of the book as the canonical “hard” graph questions.

Definition 2.5.9 (Two hard graph languages).

$HAMPATH = \{\langle G, s, t \rangle \mid G \text{ has a directed } s\text{-to-}t \text{ path visiting every vertex exactly once}\},$

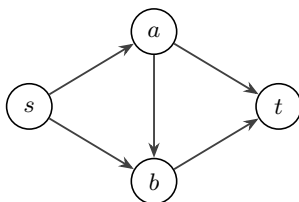
$CLIQUE = \{\langle G, k \rangle \mid G \text{ has } k \text{ pairwise-adjacent vertices}\}.$

Both sit in NP: a claimed Hamiltonian path or a claimed k -clique is a short certificate that a verifier checks in polynomial time. Neither is known to be in P; both are NP-complete, so a polynomial algorithm for either would put $P = NP$. The contrast with *PATH* is the whole drama of the subject: asking for *a* path is easy, asking for a path that hits *every vertex once* appears to be exponentially harder, even though the two questions differ by a single innocuous clause.

Worked Example 2.5.10 (One graph, three questions of three difficulties). For the digraph below, contrast the questions “is t reachable from s ?” (*PATH*), “is there a Hamiltonian s -to- t path?” (*HAMPATH*), and “does the underlying undirected graph have a triangle?” (*CLIQUE* with $k = 3$).

Recognition. The same object feeds an easy, a hard, and a moderate question; the trigger is to match each phrasing to its language and recall where that language sits.

Solution. Reachability: BFS from s reaches a, b, t , so $\langle G, s, t \rangle \in \text{PATH}$, decided in linear time. Hamiltonicity: a path from s to t through *all four* vertices must be s, a, b, t – and those arcs are present – so this instance is a yes, but verifying it required exhibiting the order, the kind of certificate NP trades in. Triangle: the underlying undirected graph has the 3-clique $\{s, a, b\}$, so the $k = 3$ *CLIQUE* instance is also a yes.



Which idea, and why? Reachability, Hamiltonicity, and clique are three *different* languages over the same encoded graph; which one you ask, not how the graph is drawn, decides whether you face a P question or an NP-complete one.

Check. The Hamiltonian path $sab t$ visits all four vertices once and uses only present arcs; the triangle sab has all three of its underlying edges sa, sb, ab ; and the easy reachability verdict agrees, since that very path witnesses t reachable from s .

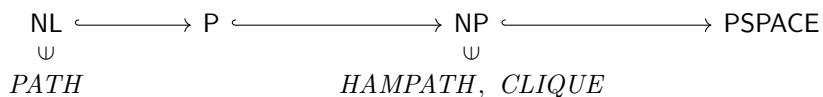
A map of the classes ahead

The classes these graph problems live in form a hierarchy, drawn here both as the chain of inclusions we will justify in later chapters and as a containment diagram. Nothing below is proved here; the picture is the road map.

Worked Example 2.5.11 (The inclusion chain as a commuting diagram). Record the standard containments $\text{NL} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE}$, placing the graph problems of this section at their classes.

Recognition. A request to organize complexity classes by inclusion; the trigger “ $X \subseteq Y$ for a chain of classes” is exactly what a containment diagram displays.

Solution. Reachability *PATH* is in fact in NL (a path can be guessed one vertex at a time in logarithmic space), hence in P. The hard graph languages *HAMPATH* and *CLIQUE* sit in NP. All of these are inside PSPACE.



Which idea, and why? Each arrow $\hookrightarrow$ is a known class inclusion, and the vertical $\in$ marks place a representative language inside its class; the diagram is the inclusion chain made into a picture, with *PATH* on the easy end and the NP-complete problems further right.

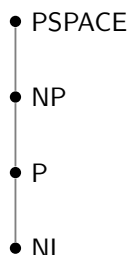
Check. The chain has no gaps: every class is contained in the next, so any language in the leftmost class is in all of them; $\text{PATH} \in \text{NL} \subseteq \text{PSPACE}$ and $\text{HAMPATH} \in \text{NP} \subseteq \text{PSPACE}$ are both consistent with the picture.

Inclusions are best read as a poset: each class sits above the ones it contains. The Hasse diagram makes the “smallest class containing” relationships visible at a glance.

Worked Example 2.5.12 (The class inclusions as a Hasse diagram). Draw the inclusion order on $\{\text{NL}, \text{P}, \text{NP}, \text{PSPACE}\}$ as a Hasse diagram, lower meaning smaller.

Recognition. An ordering by containment is a partial order, and the trigger “draw the order, smaller below” is the definition of a Hasse diagram: edges only for covering inclusions.

Solution. The known chain is total here, $NL \subseteq P \subseteq NP \subseteq PSPACE$, so the Hasse diagram is a single vertical ladder, each class covering the one beneath it.



Which idea, and why? A Hasse diagram draws only *covering* pairs; because the four classes are known to form a chain, every class covers exactly the next one down and the diagram is one straight ladder.

Check. Reading upward recovers the full order by transitivity: NL sits below all three others, and PSPACE above all three, matching $NL \subseteq P \subseteq NP \subseteq PSPACE$. No edge skips a level, as Hasse diagrams forbid drawing implied (non-covering) inclusions.

Method (How to turn a graph question into a language and place it).

1. **Name the inputs and the yes-condition.** Decide what is given (a graph, plus maybe a source/target, a budget k) and the exact property that makes an instance a “yes.”
2. **Write the language.** Form $\mathcal{L} = \{\langle \text{inputs} \rangle \mid \text{yes-condition}\}$, fixing an encoding $\langle \cdot \rangle$ via an adjacency list or matrix (Definition 2.5.1).
3. **Look for a search.** If a yes-instance is witnessed by something a BFS/DFS finds – reachability, a component, a shortest path, a cycle – the problem is in P (often near linear).
4. **Look for a short certificate.** If a yes-instance has a polynomial-size witness (a tour, a clique, a colouring) checkable in polynomial time, the problem is in NP; if no search seems to find it, suspect NP-hardness.
5. **Mind the encoding size.** State bounds in n and m and remember the matrix is $\Theta(n^2)$ bits while the list is $\Theta(n + m)$ entries; a “linear” algorithm is linear in the encoding you actually chose.

Caution. Do not conflate *reachable* with *Hamiltonian*. Reachability asks only for some directed path from s to t and is decided in polynomial time by one BFS. Requiring the path to visit every vertex exactly once is the *HAMPATH* condition, an apparently exponentially harder question and one of the standard NP-complete problems. The two phrasings differ by a single clause, but they sit on opposite sides of the P-versus-NP divide.

Caution. The choice of encoding changes the *size* of the input but not the *problem*, yet it can change whether a bound looks linear or quadratic. An algorithm that scans the whole adjacency matrix runs in $\Theta(n^2)$ time even on a graph with only $n - 1$ edges, where an adjacency-list scan finishes in $O(n)$. When you claim a running time, say which encoding it is measured against, and never call a matrix-scanning algorithm “linear in the number of edges.”

Caution. A drawing is not an encoding. Two pictures that look different may encode isomorphic graphs, and a machine reads $\langle G \rangle$, not your layout: vertex positions, edge curvature, and colours carry no information unless the encoding records them. Conversely, deciding whether two encodings describe the *same* graph up to renaming vertices is the graph-isomorphism problem, which is not known to be in P – so “obviously the same graph” is a claim a machine cannot make for free.

Exercises.

1. Let G be the undirected graph on $\{1, 2, 3, 4, 5\}$ with edges $\{12, 23, 34, 45, 51, 13\}$. Write down both the adjacency matrix and the adjacency list of G , and draw G . Which encoding is shorter here, and by how many symbols (counting each entry once)?

2. Give an adjacency-matrix encoding of a *directed* graph on $\{a, b, c\}$ with arcs $a \rightarrow b$, $b \rightarrow c$, and $c \rightarrow a$. Explain precisely why this matrix is *not* symmetric, in contrast with the matrix of an undirected graph.
3. Draw the underlying labelled digraph of the DFA over $\{0, 1\}$ with states q_0, q_1 , start q_0 , accept $\{q_1\}$, and transitions $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_1$, $\delta(q_1, 1) = q_0$. Then erase the labels and state the out-degree of every vertex in the stripped digraph; explain why that out-degree is forced.
4. For the digraph with vertices $\{s, a, b, c, t\}$ and arcs $s \rightarrow a$, $a \rightarrow b$, $b \rightarrow c$, $s \rightarrow c$, $c \rightarrow t$, run BFS from s and annotate each vertex with its distance from s . Is t reachable, and what is its distance?
5. Build a digraph on at least five vertices in which some vertex t is *not* reachable from a designated source s , yet s is reachable from t . Draw it and mark s and t .
6. Prove that in an undirected graph the relation “ u and v lie in the same connected component” is an equivalence relation (reflexive, symmetric, transitive). Conclude that the components partition the vertex set.
7. Show that *PATH* for *undirected* graphs is in P by describing a search that decides it, and give its running time in terms of n and m . State explicitly where your argument uses that the graph is undirected.
8. A student claims that because *PATH* $\in P$, the language *HAMPATH* is also in P , “since a Hamiltonian path is just a path.” Identify the flaw in this reasoning, and name the property of a Hamiltonian path that *PATH*’s BFS argument does not establish.
9. Draw a digraph on six vertices that has a directed path from s to t but *no* Hamiltonian path from s to t . Briefly justify both claims by pointing to the relevant vertices/arcs.
10. Exhibit a short certificate for the *CLIQUE* instance $\langle G, 3 \rangle$ where G is the undirected graph on $\{1, 2, 3, 4\}$ with edges $\{12, 13, 23, 34\}$, and describe in one sentence how a polynomial-time verifier checks it. Then explain why $k = 4$ is a *no*-instance for the same G .
11. (Find the bug.) A classmate “proves” *HAMPATH* $\in P$ as follows: “Run BFS from s ; if it reaches t , output yes, since BFS finds a path through the graph.” Pinpoint exactly which step is wrong and what the BFS-found path fails to guarantee about visiting every vertex once.
12. (Find the bug.) Another classmate encodes an undirected graph by an adjacency matrix A but fills in only the entries *above* the diagonal, leaving the lower triangle and the diagonal blank to “save space.” Explain why a machine reading this matrix can still recover G , but why the convention must be stated as part of the encoding; then give one query (e.g. “is $v_i v_j$ an edge for $i > j$?”) that becomes ambiguous if the convention is omitted.
13. Order the four languages *PATH*, *HAMPATH*, *CLIQUE*, and “ G is connected” by the smallest standard complexity class each is known to lie in, and draw the resulting placement against the chain $NL \subseteq P \subseteq NP \subseteq PSPACE$ as a small Hasse-style diagram.

Chapter 3

Counting and Combinatorics

3.1 The Basic Counting Principles and Pigeonhole

Counting is the art of deciding *how many* objects of some kind exist without listing them all, and almost every such count rests on two rules: when a choice splits into separate cases you *add*, and when a choice is made in independent stages you *multiply*. The one key idea of this section is that the size of a finite set is a structural fact you can compute from how the set is built – as a disjoint union, as a product of stages, or as a relabelling (bijection) of a set you already understand. We make the two rules precise, use bijections to transport a count from one set to another, and then turn the tables: the *pigeonhole principle* reads the rules backwards to force two objects to collide, which is exactly the shape of argument we will reuse later to show that machines and codes must repeat themselves.

Throughout, $|S|$ denotes the number of elements of a finite set S , and we write $[n] = \{1, 2, \dots, n\}$ for the standard n -element set, with $[0] = \emptyset$. We write $S \sqcup T$ for a union of sets that we are asserting to be *disjoint* ($S \cap T = \emptyset$), and $S \times T = \{(s, t) \mid s \in S, t \in T\}$ for their Cartesian product.

Lemma 3.1.1 (The Sum and Product Rules). *Let S and T be finite sets.*

- (a) (Sum Rule) *If $S \cap T = \emptyset$, then $|S \sqcup T| = |S| + |T|$.*
- (b) (Product Rule) *For any finite S, T , $|S \times T| = |S| \cdot |T|$.*

Proof. (a) Write $S = \{s_1, \dots, s_m\}$ and $T = \{t_1, \dots, t_n\}$. Disjointness says no s_i equals any t_j , so listing $s_1, \dots, s_m, t_1, \dots, t_n$ names $m + n$ *distinct* elements, and these are exactly the elements of $S \sqcup T$; hence $|S \sqcup T| = m + n$.

(b) Fix S with $|S| = m$ and induct on $n = |T|$. If $n = 0$ then $T = \emptyset$ and $S \times T = \emptyset$, so both sides are 0. If $n \geq 1$, pick $t \in T$ and set $T' = T \setminus \{t\}$. Then $S \times T = (S \times T') \sqcup (S \times \{t\})$, a disjoint union because the second coordinates differ. The slice $S \times \{t\} = \{(s_1, t), \dots, (s_m, t)\}$ has m elements, so by part (a) and the induction hypothesis $|S \times T| = m(n - 1) + m = mn$. $\square$

In words: “*or*” (a choice between disjoint alternatives) becomes addition, and “*and*” (a sequence of independent choices) becomes multiplication. The product rule does not actually require the second stage to be *the same* set every time; it only requires the *number* of options at the second stage to be the same no matter what the first stage chose.

Proposition 3.1.2 (Generalized Product Rule). *Suppose an object is built in k successive stages, where stage 1 has n_1 choices and, for each $i \geq 2$, stage i offers exactly n_i choices regardless of the earlier choices. Then the number of objects is $n_1 n_2 \cdots n_k$. In particular, the number of length- k strings over an alphabet Σ of size $|\Sigma| = s$ is s^k , so $|\Sigma^k| = s^k$.*

Proof. Induct on k . The case $k = 1$ is immediate and $k = 2$ is Lemma 3.1.1(b). For the step, group the objects by their first $k - 1$ stages: there are $n_1 \cdots n_{k-1}$ such prefixes by induction, and each extends in exactly n_k ways, so Lemma 3.1.1(b) applied to (prefixes) $\times$ (stage- k choices) gives $(n_1 \cdots n_{k-1}) n_k$. For strings, each of the k positions is an independent stage with s choices. $\square$

A picture makes the product rule vivid: a *choice tree* branches once per stage, and the objects counted are exactly its leaves. Counting an object is the same as counting a root-to-leaf path.

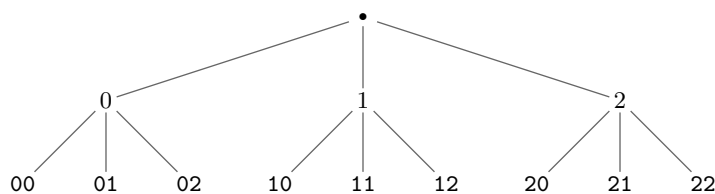
Definition 3.1.3 (Bijection and equinumerosity). A function $f: S \rightarrow T$ is a *bijection* if it is injective (distinct inputs give distinct outputs) and surjective (every $t \in T$ is $f(s)$ for some s); equivalently, f has a two-sided inverse $f^{-1}: T \rightarrow S$. If a bijection $S \rightarrow T$ exists between finite sets we say S and T are *equinumerous*, and then $|S| = |T|$.

The slogan of *bijjective counting* is: to count S , find a bijection from S to a set T whose size you already know, and read off $|S| = |T|$. The cleanest way to prove a candidate f is a bijection is to exhibit its inverse f^{-1} explicitly and check $f^{-1} \circ f = \text{id}_S$ and $f \circ f^{-1} = \text{id}_T$; this is more useful than checking injectivity and surjectivity separately, because the inverse map is itself a counting recipe you can reuse.

Worked Example 3.1.4 (The product rule as a choice tree: $\{0, 1, 2\}$ -strings of length 2). Count the strings of length 2 over the alphabet $\Sigma = \{0, 1, 2\}$, and exhibit the count as the leaves of a choice tree.

Recognition. A string is built position by position, and each position is an independent stage with the same number of options (3). The trigger “independent successive choices, fixed count per stage” is exactly the product rule.

Solution. Position one has 3 choices and position two has 3 choices, so by Proposition 3.1.2 there are $3 \cdot 3 = 9$ strings. The choice tree branches three ways at the root (first symbol) and three ways at each child (second symbol); its 9 leaves are the 9 strings.



Which idea, and why? Each input position is an independent stage with the same option count, so the total is the product of the per-stage counts; the tree shows why – every first choice fans out into the *same* number of second choices.

Check. The leaves are all 9 length-2 strings, each exactly once, matching $3^2 = 9$.

Worked Example 3.1.5 (Subsets as binary strings: a bijection that counts $2^{[n]}$). Let $2^{[n]}$ be the set of all subsets of $[n]$. Prove $|2^{[n]}| = 2^n$ by a bijection.

Recognition. We do not yet know $|2^{[n]}|$, but we *do* know $|\{0, 1\}^n| = 2^n$ by the product rule. The trigger “count an unknown set by relabelling it as a known one” is bijective counting (Definition 3.1.3).

Solution. Send a subset $A \subseteq [n]$ to its *indicator string* $\chi(A) = b_1 b_2 \cdots b_n$, where $b_i = 1$ if $i \in A$ and $b_i = 0$ if $i \notin A$. The inverse reads a string $b_1 \cdots b_n$ back to the set $\{i \mid b_i = 1\}$, and the two maps undo each other, so χ is a bijection $2^{[n]} \rightarrow \{0, 1\}^n$. Hence $|2^{[n]}| = |\{0, 1\}^n| = 2^n$. The diagram shows the case $n = 3$, with each subset wired to its 3-bit string.

$A \subseteq [3]$	$\chi(A)$
$\emptyset$	000
$\{1\}$	100
$\{2\}$	010
$\{3\}$	001
$\{1, 2\}$	110
$\{1, 3\}$	101
$\{2, 3\}$	011
$\{1, 2, 3\}$	111

Which idea, and why? A bijection transports a count: the bit string records, independently for each element, the one yes/no decision “is i in A ?”, so subsets and binary strings are the same data in two costumes and must be equal in number.

Check. For $n = 3$ both columns have $8 = 2^3$ entries, and the arrows pair the two columns one-to-one.

Worked Example 3.1.6 (A combinatorial interpretation: tilings counted by Fibonacci numbers). Let $\mathcal{T}_n$ be the set of ways to tile a $1 \times n$ strip using 1×1 squares and 1×2 dominoes. Show $|\mathcal{T}_n| = F_{n+1}$, where $F_1 = F_2 = 1$ and $F_m = F_{m-1} + F_{m-2}$.

Recognition. We want a count $t_n = |\mathcal{T}_n|$ for every n . The trigger is a *last-piece* case split: classify each tiling by its rightmost tile, which is a disjoint union (Sum Rule) and yields a recurrence.

Solution. There is one empty tiling ($t_0 = 1$) and one tiling of a single cell ($t_1 = 1$). For $n \geq 2$, split $\mathcal{T}_n$ by the last tile: tilings ending in a square biject with $\mathcal{T}_{n-1}$ (remove the square), and tilings ending in a domino biject with $\mathcal{T}_{n-2}$ (remove the domino). By the Sum Rule, $t_n = t_{n-1} + t_{n-2}$. So t_n and F_{n+1} satisfy the same recurrence and the same two initial values $t_0 = 1 = F_1$, $t_1 = 1 = F_2$; hence $t_n = F_{n+1}$ for all n . Below are the $t_3 = F_4 = 3$ tilings of a 1×3 strip (a shaded cell is half of a domino).



Which idea, and why? Classifying by the last tile is a disjoint-union (Sum Rule) decomposition, and each class bijects to a smaller tiling set, which manufactures the Fibonacci recurrence from pure counting.

Check. Directly $\mathcal{T}_3 = \{\text{SSS}, \text{DS}, \text{SD}\}$ has 3 elements, and indeed $F_4 = F_3 + F_2 = 2 + 1 = 3$.

Counting from above tells us how many objects there are; the *pigeonhole principle* runs the same arithmetic from below, turning a size inequality into a forced coincidence.

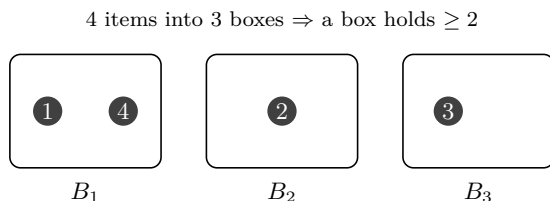
Theorem 3.1.7 (Pigeonhole Principle). *Let $f: S \rightarrow T$ be a function between finite sets with $|S| > |T|$. Then f is not injective: there exist distinct $x \neq y$ in S with $f(x) = f(y)$. Equivalently, if n objects are placed into m boxes and $n > m$, some box receives at least two objects.*

Proof. Suppose for contradiction f were injective. Then f is a bijection from S onto its image $f(S) \subseteq T$, so $|S| = |f(S)| \leq |T|$, contradicting $|S| > |T|$. Hence two distinct elements share an image. $\square$

Theorem 3.1.8 (Strong Pigeonhole Principle). *If n objects are placed into m boxes, some box contains at least $\lceil n/m \rceil$ objects. More generally, if the box sizes sum to n , the largest box has at least $\lceil n/m \rceil$ objects and the smallest has at most $\lfloor n/m \rfloor$.*

Proof. If every box held at most $\lceil n/m \rceil - 1$ objects, the total would be at most $m(\lceil n/m \rceil - 1) < m \cdot (n/m) = n$, since $\lceil n/m \rceil < (n/m) + 1$. That total is strictly less than n , contradicting that all n objects were placed. The bound on the smallest box is the same argument applied to “at least” in place of “at most.” $\square$

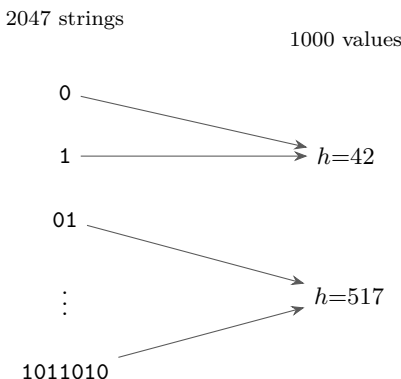
The basic principle (Theorem 3.1.7) is the case $m + 1$ objects in m boxes of Theorem 3.1.8: then $\lceil (m + 1)/m \rceil = 2$. The diagram below is the entire idea: four items, three boxes, so one box is forced to hold two.



Worked Example 3.1.9 (Two binary strings of bounded length must collide modulo a hash). Suppose a hash function h assigns to every string in $\{0, 1\}^{\leq 10}$ (strings of length at most 10) a value in $\{0, 1, \dots, 999\}$. Show that two *different* strings get the same hash.

Recognition. We are mapping a large set into a small set of buckets and want a forced collision. The trigger “more inputs than possible outputs” is the pigeonhole principle with strings as pigeons and hash values as holes.

Solution. Count the domain: $|\{0, 1\}^{\leq 10}| = \sum_{k=0}^{10} 2^k = 2^{11} - 1 = 2047$ strings (Sum Rule over the lengths, each block counted by the product rule). The codomain has only 1000 values. Since $2047 > 1000$, the map h from strings to values cannot be injective by Theorem 3.1.7, so two distinct strings $x \neq y$ satisfy $h(x) = h(y)$.



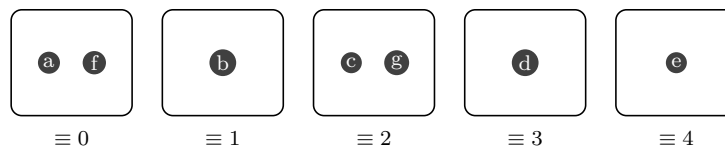
Which idea, and why? The string set strictly outnumbers the value set, so by pigeonhole the assignment must send two distinct strings to one value – no property of h beyond “it is a function” is used.

Check. If instead the codomain had ≥ 2047 values the argument would fail, exactly as it should: a collision is forced only when outputs are scarcer than inputs.

Worked Example 3.1.10 (Strong form: a repeated remainder in any long enough integer list). Show that among any 7 integers, some two leave the same remainder on division by 5, and in fact one remainder class contains at least $\lceil 7/5 \rceil = 2$ of them.

Recognition. “Some two share a value” over a fixed finite set of values ($\{0, 1, 2, 3, 4\}$ of remainders) is pigeonhole; the quantitative “how many at least” is the strong form (Theorem 3.1.8).

Solution. Map each integer to its remainder mod 5, a function from a 7-element set into the 5-element set $\{0, 1, 2, 3, 4\}$. Since $7 > 5$, two integers share a remainder; the strong form sharpens this to a box (remainder class) of size $\geq \lceil 7/5 \rceil = 2$. The boxes-and-items picture has 7 items distributed over 5 classes, so at least one class is doubly occupied.



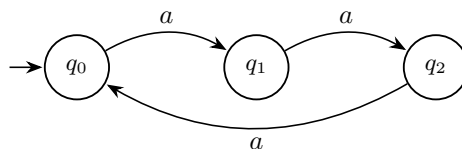
Which idea, and why? With more integers than remainder classes, pigeonhole forces a shared class, and dividing 7 items as evenly as possible over 5 boxes still leaves one box with $\lceil 7/5 \rceil = 2$.

Check. Five integers, one per class, can avoid any repeat; the sixth integer is the first that *must* collide, confirming the threshold is $5 + 1 = 6$ for a guaranteed pair.

Worked Example 3.1.11 (A pumping-style consequence: a long run through few states repeats a state). A machine has k states. Show that on any input of length $\geq k$ the sequence of states it visits must enter some state twice.

Recognition. Reading a length- n input visits $n + 1$ states (counting the start), all drawn from a set of size k . “More visits than available states” is pigeonhole again, now with the time steps as pigeons.

Solution. Running on an input of length $n \geq k$ produces the state sequence $p_0, p_1, \dots, p_n$ with $n + 1 \geq k + 1$ entries, each p_i lying in the k -element state set. Since $n + 1 > k$, by Theorem 3.1.7 two time indices $i < j$ have $p_i = p_j$: the run revisits a state. The diagram threads the $k + 1$ visits of a length- k input through only k state-boxes, so two arrows land in the same box.



visits on input **aaa**: $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0$
(4 visits, 3 states $\Rightarrow q_0$ recurs)

Which idea, and why? The visited-state count exceeds the number of states, so pigeonhole forces a repeat; this single observation is the engine behind later arguments that long accepted strings can be “pumped.”

Check. With $k = 3$ states, an input of length 3 already gives 4 visits, and the run q_0, q_1, q_2, q_0 revisits q_0 exactly as predicted.

Method (How to set up a counting or pigeonhole argument).

1. **Decide “and” vs. “or.”** If the object is built in independent stages, multiply the per-stage counts (Product Rule); if it splits into disjoint cases, add the case counts (Sum Rule). Phrase each stage so its option count does not depend on earlier stages.
2. **Reach for a bijection** when the set is awkward but “looks like” a set you can count (strings, subsets, paths). Define the map, then define its inverse and check the two compose to the identity – the inverse is your proof and your decoding recipe.
3. **For a forced coincidence, name the pigeons and the holes.** The pigeons are the objects you have many of; the holes are the finitely many values/classes they map to. Show $\# \text{pigeons} > \# \text{holes}$ and invoke pigeonhole; for a quantitative “at least r in one hole,” compare against $\lceil n/m \rceil$ via the strong form.
4. **Sanity-check the threshold.** Construct an extremal configuration with no collision (m pigeons, one per hole); the very next pigeon is the one pigeonhole forces to collide. If your “one more” does not force the conclusion, you have miscounted a box.

Caution. The Sum Rule needs *disjoint* cases. If you split a set into cases that overlap and just add the case sizes, you double-count the overlap. “Strings that start with 0 *or* end with 0” is not $|\text{start } 0| + |\text{end } 0|$, because strings doing both are counted twice; correcting this overlap is the business of inclusion–exclusion (Section 3.3). When in doubt, force disjointness by a case split like “ends with 0” versus “ends with 1.”

Caution. Pigeonhole guarantees that *some* box is overloaded, but says nothing about *which* one, and nothing about exact counts beyond the $\lceil n/m \rceil$ floor. Do not over-read it: from “13 people, so two share a birth month” you may not conclude anything about a *particular* month, only that *at least one* month is shared. Likewise the strong form gives a lower bound on the fullest box, not the size of every box – the boxes can be wildly unequal.

Caution. A bijection proof is only complete once the map is shown to be *onto as well as one-to-one*. Defining $f: S \rightarrow T$ and checking it is injective shows $|S| \leq |T|$, not equality; you still owe either surjectivity or, better, an explicit inverse. Forgetting this is the same error as claiming two finite sets are equinumerous from a one-way matching that misses some elements of T .

Exercises.

1. A password is exactly 4 characters long, each character a lowercase letter (26 options) or a digit (10 options). Using the product rule, count the passwords. Then count those that use only letters, and use the sum rule to count the passwords that contain *at least one* digit (count the complement).
2. Over the alphabet $\Sigma = \{a, b, c\}$, count the strings of length ≤ 3 (that is, lengths 0, 1, 2, 3). Show the length-by-length split as a sum and give the total. Then draw the choice tree for the length-2 strings only and confirm its leaf count.
3. Let $S = \{1, 2, 3, 4\}$. Exhibit an explicit bijection between the 2-element subsets $\binom{S}{2}$ and the edges of a complete graph on 4 labelled vertices, and use it to count both. State the inverse map.
4. Prove by a bijection that the number of binary strings of length n with an *even* number of 1s equals the number with an *odd* number of 1s, for $n \geq 1$. (Hint: flip the last bit; show this map is its own inverse and reverses parity.)
5. Define $f: \{0, 1\}^n \rightarrow 2^{[n]}$ and its inverse so that f is a bijection, and use it to re-derive $|2^{[n]}| = 2^n$. Then state, in one sentence, what data the bijection of Worked Example 3.1.5 (subsets to indicator strings) and this one have in common.
6. How many integers must you pick from $\{1, 2, \dots, 100\}$ to guarantee that two of them differ by a multiple of 9? Identify the pigeons, the holes, and the exact threshold, and exhibit a collision-free choice of one fewer integer.
7. Among any 26 people, show that at least three were born in the same month. State which form of the pigeonhole principle you use and compute the guaranteed lower bound $\lceil n/m \rceil$ explicitly.
8. A machine has 5 states. Argue that on every input of length ≥ 5 the run visits some state at least twice, and explain in one sentence why this is the basic-pigeonhole case $m + 1$ pigeons into m holes. What is the longest input length for which a repeat is *not* yet forced?
9. Show that any sequence of 10 distinct integers contains two whose difference is divisible by 9, and separately that any sequence of 11 distinct integers contains a value and a later value with the same last decimal digit. Identify the holes in each case.
10. Prove the *strong* pigeonhole bound directly: if n items are placed in m boxes, then *some* box has at least $\lceil n/m \rceil$ items. (The claim that in *every* placement no box has more than $\lceil n/m \rceil$ items is false in general.) Then give a placement of $n = 7$ items into $m = 3$ boxes meeting this bound with equality in the fullest box.
11. *Find the bug.* A student counts the strings over $\{0, 1\}$ of length 3 that “start with 0 or end with 0” as $|\text{start } 0| + |\text{end } 0| = 4 + 4 = 8$, then is puzzled that there are only 8 such strings total but their list has repeats. Explain exactly which strings are double-counted and give the correct total.
12. *Find the bug.* Someone claims to prove $||[n]| = |[n + 1]|$ by the injection $g: [n] \rightarrow [n + 1]$, $g(i) = i$, asserting “an injection between finite sets forces equal cardinality.” Pinpoint the false step in one sentence and state the correct conclusion an injection $S \rightarrow T$ gives.
13. Let $a_1, \dots, a_n$ be any integers. Prove that some nonempty consecutive block $a_{i+1} + \dots + a_j$ is divisible by n . (Hint: consider the $n + 1$ prefix sums $0, a_1, a_1 + a_2, \dots$ modulo n and apply pigeonhole.)

3.2 Permutations, Combinations, and Binomial Coefficients

The previous section gave us the sum and product rules; this section turns them into the two workhorse counts that pervade all of enumeration: the number of *ordered* selections (permutations) and the number of *unordered* selections (combinations). The one key idea is that an unordered selection is an ordered one with the internal orderings divided out, so $k!$ orderings of the same k -set collapse to one subset; that single quotient turns $n^{\underline{k}}$ into $\binom{n}{k}$ and unlocks the binomial coefficients, Pascal's triangle, and the algebra of $(x+y)^n$. We make the falling factorial precise, count permutations and combinations, prove the binomial theorem and Pascal's identities, and – crucially – learn to read a counting identity as two ways of counting one set.

Definition 3.2.1 (Falling factorial). For an integer n and an integer $k \geq 0$, the *falling factorial* $n^{\underline{k}}$ is the product of k consecutive integers descending from n :

$$n^{\underline{k}} = n(n-1)(n-2)\cdots(n-k+1) = \prod_{i=0}^{k-1} (n-i).$$

By convention $n^{\underline{0}} = 1$ (an empty product). When $0 \leq k \leq n$ this is $n^{\underline{k}} = n!/(n-k)!$, but the product form makes sense for every n , whereas the quotient does not.

The number k is simply the count of factors. For $n \geq 0$ and $k > n$ one of the factors is 0, so $n^{\underline{k}} = 0$ – exactly right, since you cannot line up more distinct items than you have.

Definition 3.2.2 (Permutation, k -permutation). Let S be a set with $|S| = n$. A *permutation* of S is an ordering $\pi = \pi_1\pi_2\cdots\pi_n$ of all its elements. A *k -permutation* of S is a sequence $\pi = \pi_1\cdots\pi_k$ of k *distinct* elements of S ; the number k is its length. We write $P(S, k)$ for the set of all k -permutations, and $P(S) = P(S, n)$ for the full permutations.

Theorem 3.2.3 (Counting permutations). For $n, k \geq 0$, the number of k -permutations of an n -element set is

$$|P([n], k)| = n^{\underline{k}}, \quad \text{and in particular} \quad |P([n])| = n!.$$

Proof. Build a k -permutation $\pi_1\cdots\pi_k$ one slot at a time. There are n choices for π_1 . Having fixed π_1 , the entry π_2 must avoid it, leaving $n-1$ choices; and crucially the *number* of remaining choices is $n-1$ no matter which value was used for π_1 . Continuing, slot i has $n-(i-1)$ available values, again independent of the earlier choices. By the product rule the total is $n(n-1)\cdots(n-k+1) = n^{\underline{k}}$. Setting $k = n$ gives $n^{\underline{n}} = n!$. $\square$

Definition 3.2.4 (Combination, binomial coefficient). For a set S , let $\binom{S}{k} = \{T \subseteq S \mid |T| = k\}$ be the family of its k -element subsets. For $n \geq 0$ the *binomial coefficient* is the number of such subsets of an n -set,

$$\binom{n}{k} = \left| \binom{[n]}{k} \right|.$$

We extend the definition by $\binom{n}{k} = 0$ for $k < 0$.

Theorem 3.2.5 (Combinations from permutations). For $n, k \geq 0$,

$$\binom{n}{k} = \frac{n^{\underline{k}}}{k!}, \quad \text{and for } 0 \leq k \leq n, \quad \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Proof. We count the set $P([n], k)$ of k -permutations in a second way. Every k -permutation is obtained by first choosing *which* k elements appear (a k -subset, $\binom{n}{k}$ choices) and then choosing an *order* for those k elements ($k!$ choices, by Theorem 3.2.3). Distinct (subset, order) pairs give distinct sequences and every sequence arises exactly once, so the product rule gives $|P([n], k)| = \binom{n}{k} \cdot k!$. By Theorem 3.2.3 the left side is $n^{\underline{k}}$, hence $\binom{n}{k} = n^{\underline{k}}/k!$. For $0 \leq k \leq n$ substitute $n^{\underline{k}} = n!/(n-k)!$. $\square$

The quotient form $\frac{n!}{k!(n-k)!}$ exposes the *symmetry* $\binom{n}{k} = \binom{n}{n-k}$: choosing the k elements to include is the same act as choosing the $n-k$ to leave out. This is the first of several identities we will both compute and *interpret*.

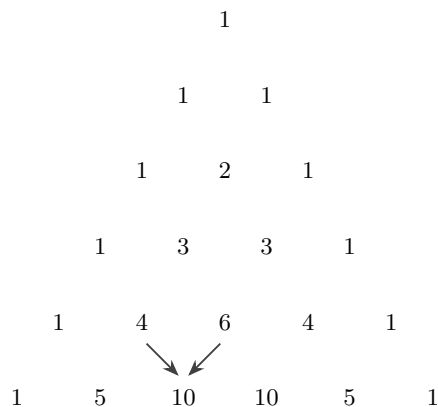
Theorem 3.2.6 (Pascal's recurrence and symmetry). *For all $n \geq 1$ and all integers k ,*

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{Pascal's rule}),$$

with initial condition $\binom{0}{k} = \delta_{k,0}$, and $\binom{n}{k} = \binom{n}{n-k}$ for $0 \leq k \leq n$.

Proof. We give a *combinatorial proof* of Pascal's rule: both sides count $\binom{[n]}{k}$, the k -subsets of $[n]$. Split those subsets by whether they contain the element n . A subset that *contains* n is determined by its other $k-1$ elements, chosen from $[n-1]$: there are $\binom{n-1}{k-1}$ of these. A subset that *omits* n is a k -subset of $[n-1]$: there are $\binom{n-1}{k}$ of these. The two cases are disjoint and exhaust $\binom{[n]}{k}$, so the sum rule gives the identity. The initial condition holds because the only 0-element set is $\emptyset$. Symmetry was argued above via complementation. $\square$

These two facts *are* Pascal's triangle: row n holds the numbers $\binom{n}{0}, \dots, \binom{n}{n}$, each interior entry is the sum of the two above it, and the rows read the same forwards and backwards.



The arrows show one instance of Pascal's rule: $\binom{5}{2} = 10$ is the sum $\binom{4}{1} + \binom{4}{2} = 4 + 6$ sitting just above it.

Theorem 3.2.7 (Binomial theorem). *For any commuting x, y and any integer $n \geq 0$,*

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}.$$

Proof. Write $(x + y)^n = \underbrace{(x + y)(x + y) \cdots (x + y)}_{n \text{ factors}}$ and expand by choosing, from each factor, either the x term or the y term. A single choice across all n factors yields one monomial; if we picked x from exactly k of the factors then that monomial is $x^k y^{n-k}$. The number of ways to pick *which* k of the n factors contribute an x is $\binom{n}{k}$, so the monomial $x^k y^{n-k}$ occurs $\binom{n}{k}$ times. Summing over k collects all 2^n choices into the stated sum. $\square$

Two specializations are worth keeping at your fingertips. Putting $x = y = 1$ gives $\sum_k \binom{n}{k} = 2^n$ (every subset of $[n]$, counted by size). Putting $x = -1, y = 1$ gives $\sum_k (-1)^k \binom{n}{k} = 0$ for $n \geq 1$: the even-sized and odd-sized subsets of a nonempty set are equinumerous.

Worked Example 3.2.8 (Reading n^k off a description). A passcode is an ordered list of 4 *distinct* digits drawn from $\{0, 1, \dots, 9\}$. How many passcodes are there?

Recognition. The words “ordered list” and “distinct” are the trigger: we are counting k -permutations of an n -set, with $n = 10$ and $k = 4$. Order matters and repetition is banned, so this is n^k , not $\binom{n}{k}$ and not n^k .

Solution. By Theorem 3.2.3 the count is

$$10^4 = 10 \cdot 9 \cdot 8 \cdot 7 = 5040.$$

The slot picture below makes the descending factors visible: each slot has one fewer choice than the last because the chosen digit is now unavailable.

$$\begin{array}{cccc} \square & \square & \square & \square \\ 10 & 9 & 8 & 7 \end{array} = 10^4 = 5040$$

Which idea, and why? Ordered selection of distinct items is the falling factorial $n^{\underline{k}}$: the product of k descending choice-counts, one per slot.

Check. Contrast with allowing repeats ($10^4 = 10000$ words) and with ignoring order ($\binom{10}{4} = 210$ subsets); the distinct-and-ordered count 5040 lies strictly between, as it must, since $\binom{n}{k} \leq n^{\underline{k}} \leq n^k$.

Worked Example 3.2.9 (The same objects, counted with and without order). From a club of 9 people, (a) how many ways to choose a committee of 3, and (b) how many ways to choose a president, then a vice-president, then a treasurer (three distinct offices)?

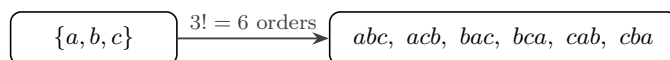
Recognition. The single distinguishing question is “does order matter?” A committee is an unordered subset (use $\binom{n}{k}$); a slate of three labelled offices is an ordered selection of distinct people (use $n^{\underline{k}}$). The two differ by the factor $k!$, which is exactly the theme of Theorem 3.2.5.

Solution. For (a),

$$\binom{9}{3} = \frac{9^{\underline{3}}}{3!} = \frac{9 \cdot 8 \cdot 7}{6} = 84.$$

For (b), the offices are distinguishable, so order matters:

$$9^{\underline{3}} = 9 \cdot 8 \cdot 7 = 504 = 3! \cdot \binom{9}{3}.$$



Which idea, and why? Each k -subset spreads into $k!$ ordered slates, so the ordered count is $k!$ times the unordered count; equivalently $\binom{n}{k} = n^{\underline{k}}/k!$.

Check. The picture shows one 3-subset $\{a, b, c\}$ fanning out into its $3! = 6$ orderings; multiply the 84 committees by 6 to recover 504 slates.

Worked Example 3.2.10 (Pascal’s rule by counting one set two ways). Verify $\binom{5}{2} = \binom{4}{1} + \binom{4}{2}$ by a direct bijective argument, not by arithmetic.

Recognition. The shape “ $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ ” is Pascal’s rule (Theorem 3.2.6); the cleanest justification splits the k -subsets by whether they contain a distinguished element. Here $n = 5$, $k = 2$, and we single out the element 5.

Solution. The 2-subsets of $[5] = \{1, 2, 3, 4, 5\}$ split into those that *use* 5 and those that *avoid* it:

$$\underbrace{\{15\}, \{25\}, \{35\}, \{45\}}_{\text{contain 5: } \binom{4}{1}=4} \uplus \underbrace{\{12\}, \{13\}, \{14\}, \{23\}, \{24\}, \{34\}}_{\text{avoid 5: } \binom{4}{2}=6}.$$

Removing 5 from a subset of the first kind leaves a 1-subset of $[4]$ (a bijection onto $\binom{[4]}{1}$); a subset of the second kind *is* a 2-subset of $[4]$. So the count is $4 + 6 = 10 = \binom{5}{2}$.

$$\begin{array}{ccccccc} & & 1 & & 3 & & 3 & & 1 \\ & & & & & & & & & & \\ & 1 & & 4 & & 6 & & 4 & & 1 \\ & & & \swarrow & & \searrow & & & & & \\ 1 & & 5 & & 10 & & 10 & & 5 & & 1 \end{array} \quad 4 + 6 = 10$$

Which idea, and why? Conditioning on a single element (5 in or out) partitions the k -subsets into two disjoint families counted by $\binom{n-1}{k-1}$ and $\binom{n-1}{k}$; the sum rule then *is* Pascal’s rule.

Check. The triangle shows 10 in row 5 resting on the 4 and 6 of row 4, the same split the set computation produced.

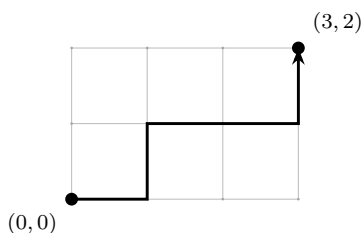
Worked Example 3.2.11 (Lattice paths and a closed form for $\binom{m+n}{m}$). How many monotone lattice paths go from $(0, 0)$ to $(3, 2)$ using only unit steps east ($E = [1, 0]$) and north ($N = [0, 1]$)?

Recognition. Each such path is a sequence of moves, and the only freedom is *which* of the total steps are east. “Choose which positions get E ” is a subset choice, so the count is a binomial coefficient – this is the standard lattice-path model of $\binom{m+n}{m}$.

Solution. A path from $(0, 0)$ to $(3, 2)$ has exactly 3 east and 2 north steps, $m + n = 5$ steps in all. The path is determined by choosing which 3 of the 5 step-positions are E (the rest are N), so the number of paths is

$$\binom{3+2}{3} = \binom{5}{3} = 10.$$

One such path, $ENEEN$, is drawn below on the grid.



Which idea, and why? A monotone E/N path to (m, n) is a word with m E 's and n N 's; choosing the positions of the E 's among the $m + n$ steps gives $\binom{m+n}{m}$ paths.

Check. Symmetry $\binom{5}{3} = \binom{5}{2}$ matches the obvious fact that choosing the E positions and choosing the N positions are the same act; both give 10.

Worked Example 3.2.12 (Expanding a power with the binomial theorem). Expand $(2a - b)^4$ in full.

Recognition. A fixed nonnegative integer power of a two-term sum is exactly the binomial theorem (Theorem 3.2.7). The trigger is “ $(\cdot + \cdot)^n$ with n a small constant”; take $x = 2a$, $y = -b$, $n = 4$, and read the coefficients off row 4 of Pascal's triangle.

Solution. Row 4 is 1, 4, 6, 4, 1. With $x = 2a$ and $y = -b$,

$$(2a - b)^4 = \sum_{k=0}^4 \binom{4}{k} (2a)^k (-b)^{4-k}.$$

Computing term by term (note $(-b)^{4-k}$ alternates sign):

$$(2a)^4 - 4(2a)^3b + 6(2a)^2b^2 - 4(2a)b^3 + b^4 = 16a^4 - 32a^3b + 24a^2b^2 - 8ab^3 + b^4.$$

Which idea, and why? Each term is $\binom{n}{k} x^k y^{n-k}$; the binomial coefficient counts the ways to choose which k of the n factors donate the x , and the sign comes entirely from the powers of $y = -b$.

Check. Set $a = b = 1$: the polynomial gives $(2 - 1)^4 = 1$, and the coefficients sum $16 - 32 + 24 - 8 + 1 = 1$. Set $a = 1, b = 0$: $16 = 16a^4$, the leading term. Both pass.

Worked Example 3.2.13 (A summation identity by counting subsets). Show that $\sum_{k=0}^n \binom{n}{k} = 2^n$ by counting one set in two ways.

Recognition. An identity “(sum of binomials) = (a clean closed form)” is begging for a combinatorial proof: find a set whose size is obviously the right side, then partition it so the parts are obviously the left side. Here the right side 2^n shouts “number of subsets of $[n]$.”

Solution. Let $\mathcal{U} = 2^{[n]}$ be the family of *all* subsets of $[n]$; we count $|\mathcal{U}|$ twice. *First count:* build a subset by deciding independently, for each of the n elements, in or out – two choices each, so by the product rule $|\mathcal{U}| = 2^n$. *Second count:* sort the subsets by size. The subsets of size exactly k number $\binom{n}{k}$ by definition, and every subset has exactly one size $k \in \{0, 1, \dots, n\}$, so the sum rule gives $|\mathcal{U}| = \sum_{k=0}^n \binom{n}{k}$. Equating the two counts proves the identity.

$$1 \quad 3 \quad 3 \quad 1 \quad 1 + 3 + 3 + 1 = 2^3 = 8$$

Which idea, and why? Counting the same set $2^{[n]}$ by “decide each element” versus “sort by size” yields $2^n = \sum_k \binom{n}{k}$; the equality of two valid counts of one set is the entire proof.

Check. Row 3 of Pascal’s triangle is 1, 3, 3, 1, and indeed $1 + 3 + 3 + 1 = 8 = 2^3$.

Method (How to decide which count a word problem wants).

1. **Order, or no order?** If rearranging the chosen items gives a *different* outcome (offices, rankings, sequences, words), order matters. If rearranging gives the *same* outcome (committees, hands, subsets), it does not.
2. **Repetition allowed?** Distinct items only, or may an item recur?
3. **Pick the formula.** Ordered + distinct $\Rightarrow n^k$. Ordered + repeats allowed $\Rightarrow n^k$. Unordered + distinct $\Rightarrow \binom{n}{k}$. (Unordered + repeats is the multiset count $\binom{n+k-1}{k}$.)
4. **For an identity, count a set twice.** To prove “ $A = B$,” exhibit one finite set whose cardinality is plainly A from one viewpoint and plainly B from another (condition on an element, sort by size, or set up a bijection). Equal counts of one set force $A = B$.
5. **Sanity-check at extremes.** Test $k = 0$, $k = n$, $n = 0$, or plug $x = y = 1$ into a binomial identity; a formula that fails an endpoint is wrong.

Caution. Do not confuse n^k with $\binom{n}{k}$. They count different things: n^k counts *ordered* distinct selections and $\binom{n}{k} = n^k/k!$ counts *unordered* ones. Whenever an answer feels “too big by a factor of $k!$,” you have almost certainly left the selections ordered when the problem wanted subsets. The reverse error – dividing by $k!$ for genuinely labelled slots (distinct offices, ranked positions) – undercounts by the same factor.

Caution. In the binomial theorem, mind *which* variable carries the exponent k . The term is $\binom{n}{k} x^k y^{n-k}$, so the exponents of x and y always sum to n . When a term itself is negative or scaled – as with $(2a - b)^n$ – substitute the *whole* term ($x = 2a$, $y = -b$) and let the sign and the constant ride along inside the powers; forgetting to raise the coefficient (2^k here) or mishandling the alternating sign of $(-b)^{n-k}$ is the most common expansion slip.

Exercises.

1. Evaluate by hand, showing the falling-factorial product before simplifying: 8^3 , $\binom{8}{3}$, and $\binom{8}{5}$. Explain in one sentence why the last two are equal.
2. A relay team runs 4 legs, each by a different runner chosen from a squad of 7. How many ordered line-ups are possible? How many *sets* of 4 runners could form a team if the legs were not yet assigned? Which of 7^4 , $\binom{7}{4}$, 7^4 did you use for each, and why?
3. Over the alphabet $\{a, b, c, d\}$, count separately: (a) the 3-letter words with no repeated letter; (b) the 3-letter words with repetition allowed; (c) the 3-element subsets. Verify that your three answers stand in the ratio predicted by Theorems 3.2.3 and 3.2.5.
4. How many monotone lattice paths run from $(0, 0)$ to $(4, 3)$ using only east and north unit steps? Draw the grid and one such path, and explain which set of step-positions your binomial coefficient is choosing.
5. Write out row 6 of Pascal’s triangle from row 5 = $(1, 5, 10, 10, 5, 1)$ using only Pascal’s rule (no factorials). Then check one interior entry against the formula $\binom{6}{k} = 6!/(k!(6-k)!)$.
6. Use the binomial theorem to expand $(x + 3)^4$ and $(1 - 2t)^3$ in full. For each, confirm your answer by substituting a convenient value (e.g. $x = 1$ or $t = 1$) into both the original power and your expansion.
7. Give a combinatorial proof (count one set two ways, do *not* manipulate factorials) of the absorption identity

$$k \binom{n}{k} = n \binom{n-1}{k-1} \quad (1 \leq k \leq n).$$

Hint: count pairs (T, x) where T is a k -subset of $[n]$ and $x \in T$ is a marked element.

8. Prove that $\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$ for every $n \geq 1$, in two ways: (a) by substituting suitable values into the binomial theorem; (b) in one sentence, by interpreting it as “the nonempty set $[n]$ has as many even subsets as odd subsets.”

9. Give a combinatorial proof of *Vandermonde’s identity*

$$\binom{m+n}{r} = \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k},$$

by counting the r -subsets of a set split into an m -part and an n -part. State clearly what the index k records.

10. Using the lattice-path model, give a combinatorial proof that the number of monotone paths from $(0,0)$ to (m,n) equals the number from $(0,0)$ to (n,m) , and identify which binomial-coefficient symmetry this is.
11. *Find the bug.* A student claims that the number of ways to seat 5 distinguishable people in a row is $\binom{5}{5} = 1$, “since we are using all 5 people.” Identify the conceptual error, name the quantity actually being computed, and give the correct count.
12. *Find the bug.* To expand $(2x+y)^3$ a student writes $2x^3 + 3 \cdot 2x^2y + 3 \cdot 2xy^2 + y^3$. Pinpoint exactly which binomial-theorem ingredient was mishandled, and give the correct expansion.

3.3 Inclusion-Exclusion

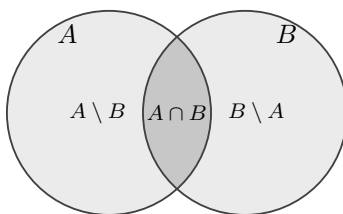
The sum rule counts a union of *disjoint* sets by adding their sizes, but the moment the sets overlap that simple addition double-counts the shared elements. Inclusion-exclusion is the exact correction: add the sizes of the sets, subtract the sizes of the pairwise overlaps, add back the triple overlaps, and so on with alternating signs. The one key idea is that an element lying in exactly m of the sets contributes a net of zero to the alternating sum (because $\binom{m}{0} - \binom{m}{1} + \cdots = 0$ for $m \geq 1$), so only the elements in *none* of the sets survive – which is precisely what we usually want to count. This “sieve” is the workhorse behind counting derangements, surjections, and Euler’s totient, and it returns later as the cleanest way to count strings or functions that avoid a list of forbidden patterns.

The two-set case and the complement viewpoint

Everything starts from the corrected sum rule for two sets. If A and B are finite,

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

because $|A| + |B|$ counts each element of $A \cap B$ twice, so we subtract the overlap once to restore an honest count. The shaded region in the picture below is $A \cup B$; the doubly-shaded lens is the $A \cap B$ that must be removed.



(shade A, B ; the lens $A \cap B$ is subtracted once)

It is almost always more convenient to count the elements lying in *none* of the chosen subsets. Fix a finite universe S and subsets $S_1, \dots, S_n \subseteq S$; we want $|S \setminus (S_1 \cup \cdots \cup S_n)|$. Rearranging the two-set identity into this complement form,

$$|S \setminus (S_1 \cup S_2)| = |S| - |S_1| - |S_2| + |S_1 \cap S_2|,$$

and the pattern of alternating signs continues for every n .

Definition 3.3.1 (Intersection over an index set). For $I \subseteq \{1, \dots, n\}$ write $S_I = \bigcap_{i \in I} S_i$, with the convention $S_\emptyset = S$ (the empty intersection is the whole universe). Thus $S_{\{i\}} = S_i$, $S_{\{i,j\}} = S_i \cap S_j$, and $|S_I|$ depends only on which subsets are intersected.

Theorem 3.3.2 (Principle of Inclusion-Exclusion). *Let S be a finite set with subsets $S_1, \dots, S_n$. Then*

$$\left| S \setminus \bigcup_{i=1}^n S_i \right| = \sum_{I \subseteq \{1, \dots, n\}} (-1)^{|I|} |S_I| = |S| - \sum_i |S_i| + \sum_{i < j} |S_i \cap S_j| - \dots + (-1)^n |S_1 \cap \dots \cap S_n|.$$

Equivalently, for the union itself,

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{\emptyset \neq I \subseteq \{1, \dots, n\}} (-1)^{|I|+1} |S_I|.$$

Proof. We count the total contribution of a single element $x \in S$ to the right-hand sum $\sum_I (-1)^{|I|} |S_I|$ and show it is 1 when x lies in no S_i and 0 otherwise. Say x belongs to exactly m of the subsets. The element x is counted in $|S_I|$ precisely when I indexes a sub-collection of those m subsets, i.e. when I is one of the $\binom{m}{k}$ size- k subsets of the m indices. Summing its signed contributions,

$$\sum_{I: x \in S_I} (-1)^{|I|} = \sum_{k=0}^m (-1)^k \binom{m}{k}.$$

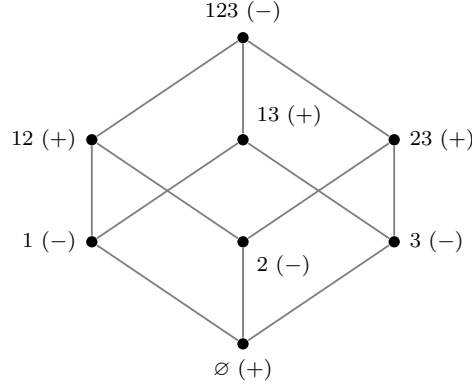
If $m = 0$ the only term is $k = 0$, giving 1. If $m \geq 1$ the alternating sum $\sum_k (-1)^k \binom{m}{k} = (1 - 1)^m = 0$ by the binomial theorem. Hence each $x \notin \bigcup_i S_i$ contributes 1 and each $x \in \bigcup_i S_i$ contributes 0, so the total equals $|S \setminus \bigcup_i S_i|$. The union form follows by writing $|\bigcup_i S_i| = |S| - |S \setminus \bigcup_i S_i|$ and isolating the $I = \emptyset$ term $|S_\emptyset| = |S|$. $\square$

The proof's accounting is worth seeing as a ledger. Fix one element x lying in exactly $t \geq 1$ of the subsets; the table below tallies how the inclusion-exclusion terms count x , level by level, and why the running total lands on a single net count.

term	how many contain x	signed count of x
singletons	$\binom{t}{1}$	$+\binom{t}{1}$
pairs	$\binom{t}{2}$	$-\binom{t}{2}$
triples	$\binom{t}{3}$	$+\binom{t}{3}$
$\vdots$	$\vdots$	$\vdots$
t -fold	$\binom{t}{t}$	$(-1)^{t+1} \binom{t}{t}$
net		$\sum_{k=1}^t (-1)^{k+1} \binom{t}{k} = 1$

Read the bottom line as the cancellation: the signed counts of x across all levels sum to $\sum_{k=1}^t (-1)^{k+1} \binom{t}{k} = 1 - \sum_{k=0}^t (-1)^k \binom{t}{k} = 1 - (1 - 1)^t = 1$, so inclusion-exclusion overcounts x and then corrects perfectly, leaving exactly one net count. *Check.* The binomial theorem gives $\sum_{k=0}^t (-1)^k \binom{t}{k} = (1 - 1)^t = 0$ for every $t \geq 1$, which is precisely what forces the alternating tally above down to the single count 1.

The signed bookkeeping is the whole story: counting a 2^n -fold collection of intersections by the sign of the index-set size. The lattice of index sets makes the cancellation visible for $n = 3$.



The sign attached to each index set I is $(-1)^{|I|}$: the bottom (empty intersection S) is $+$, the singletons are $-$, the pairs are $+$, and the top triple is $-$. An element in exactly $m \geq 1$ subsets is counted with these signs over the 2^m nodes at or below the node of its m indices, and the column sums of $\binom{m}{k}$ with alternating signs cancel.

Application 1: derangements

Definition 3.3.3 (Derangement). A *derangement* of $\{1, \dots, n\}$ is a permutation π with no fixed point, i.e. $\pi(i) \neq i$ for every i . The number of derangements of an n -element set is D_n .

Theorem 3.3.4 (Derangement formula). For $n \geq 0$,

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!} = n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \dots + \frac{(-1)^n}{n!} \right),$$

and $D_n/n! \rightarrow e^{-1}$ as $n \rightarrow \infty$.

Proof. Let S be all $n!$ permutations of $\{1, \dots, n\}$, and let S_i be the permutations that *do* fix i (so $\pi(i) = i$). A derangement fixes nothing, so $D_n = |S \setminus \bigcup_i S_i|$, exactly the quantity Theorem 3.3.2 computes. For a fixed index set I of size k , a permutation in S_I pins down $\pi(i) = i$ for the k indices in I and permutes the remaining $n - k$ symbols freely, so $|S_I| = (n - k)!$. There are $\binom{n}{k}$ index sets of size k , each contributing the same $(n - k)!$, so

$$D_n = \sum_{k=0}^n (-1)^k \binom{n}{k} (n - k)! = \sum_{k=0}^n (-1)^k \frac{n!}{k! (n - k)!} (n - k)! = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

The final sum is the degree- n truncation of the Taylor series $e^{-1} = \sum_{k \geq 0} (-1)^k / k!$, so $D_n/n! \rightarrow e^{-1}$. $\square$

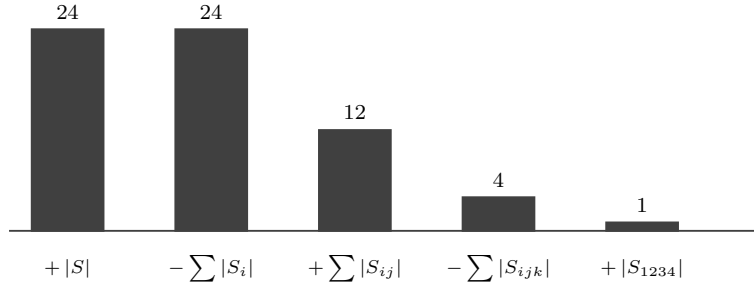
Worked Example 3.3.5 (Computing D_4 and the “no one gets their own hat” probability). Four guests check identical-looking hats; the hats are returned in a random order. In how many of the $4!$ orderings does *no* guest get their own hat back, and what fraction is that?

Recognition. “No element is fixed” over a finite ground set with one forbidden condition per index is the derangement setup; the trigger is “count permutations avoiding all n fixed-point conditions,” which is $S \setminus \bigcup S_i$.

Solution. Apply Theorem 3.3.4 with $n = 4$:

$$D_4 = 4! \left(1 - \frac{1}{1} + \frac{1}{2} - \frac{1}{6} + \frac{1}{24} \right) = 24 \cdot \frac{9}{24} = 9.$$

The sieve diagram tallies the same number directly from the intersection sizes $|S| = 24$, $\sum |S_i| = \binom{4}{1} 3! = 24$, $\sum |S_i \cap S_j| = \binom{4}{2} 2! = 12$, $\sum |S_{ijk}| = \binom{4}{3} 1! = 4$, $|S_{1234}| = \binom{4}{4} 0! = 1$:



The alternating combination is $24 - 24 + 12 - 4 + 1 = 9$, matching the formula. The probability that no guest gets their own hat is $D_4/4! = 9/24 = 3/8 = 0.375$.

Which idea, and why? Each “guest i keeps hat i ” is one forbidden condition; counting permutations that escape all of them is exactly $S \setminus \bigcup S_i$, and the constant intersection size $(n - k)!$ collapses the sieve to the truncated e^{-1} series.

Check. The exact answer $9/24 = 0.375$ is already close to $e^{-1} = 0.3679\dots$; the next term would change it by only $1/5! \approx 0.008$, illustrating how fast the truncation converges.

Application 2: surjections (onto functions)

Theorem 3.3.6 (Counting surjections). *The number of surjective (onto) functions from an n -element set onto an m -element set is*

$$\text{Surj}(n, m) = \sum_{k=0}^m (-1)^k \binom{m}{k} (m - k)^n.$$

Proof. Let S be all m^n functions $f: \{1, \dots, n\} \rightarrow \{1, \dots, m\}$ (each of the n inputs has m independent choices). For target value j let $S_j = \{f \in S \mid j \notin \text{im } f\}$, the functions that *miss* value j . A function is onto iff it misses no value, so $\text{Surj}(n, m) = |S \setminus \bigcup_j S_j|$. For an index set I of size k , the functions in S_I are exactly those whose image avoids all k values in I , i.e. functions into the remaining $m - k$ values; there are $(m - k)^n$ of them. With $\binom{m}{k}$ index sets of each size, Theorem 3.3.2 gives $\sum_{k=0}^m (-1)^k \binom{m}{k} (m - k)^n$. $\square$

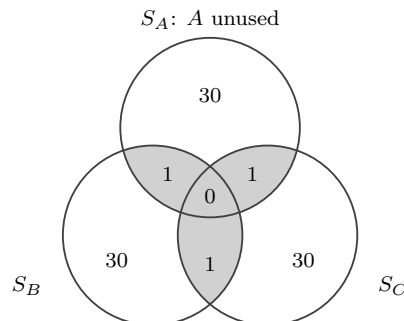
Worked Example 3.3.7 (How many ways to distribute 5 distinct jobs to 3 servers, each used?). Five distinct jobs are each assigned to one of three servers $\{A, B, C\}$. How many assignments use *every* server at least once?

Recognition. An assignment is a function (jobs $\rightarrow$ servers); “every server used” means onto. The trigger “each target value must appear” is the surjection-counting sieve with one “value j is missed” condition per server.

Solution. Take $n = 5$ jobs, $m = 3$ servers in Theorem 3.3.6:

$$\text{Surj}(5, 3) = \sum_{k=0}^3 (-1)^k \binom{3}{k} (3 - k)^5 = 3^5 - \binom{3}{1} 2^5 + \binom{3}{2} 1^5 - \binom{3}{3} 0^5.$$

Evaluating term by term: $3^5 = 243$; $\binom{3}{1} 2^5 = 3 \cdot 32 = 96$; $\binom{3}{2} 1^5 = 3 \cdot 1 = 3$; the last term is $1 \cdot 0^5 = 0$. The Venn picture below shows the three “server unused” sets being sieved out of the 243 unrestricted assignments:



Each single circle S_A, S_B, S_C holds $2^5 = 32$ functions (into the other two servers), 30 of them outside the lenses; each pairwise lens holds $1^5 = 1$ (into the single remaining server); the central triple intersection holds $0^5 = 0$ (no server left). So

$$\text{Surj}(5, 3) = 243 - 3 \cdot 32 + 3 \cdot 1 - 0 = 243 - 96 + 3 = 150.$$

Which idea, and why? “Onto” is the negation of “some value is missed,” one missed-value condition per target element; the size $(m - k)^n$ of a k -fold intersection is just “functions into the surviving $m - k$ values.”

Check. A cross-check is $\text{Surj}(n, m) = m! S(n, m)$, where $S(n, m)$ is the Stirling number of the second kind; with $S(5, 3) = 25$ this gives $3! \cdot 25 = 6 \cdot 25 = 150$. The two counts agree.

Application 3: Euler’s totient

Theorem 3.3.8 (Euler’s totient via inclusion-exclusion). *For $N \geq 1$ with distinct prime divisors $p_1, \dots, p_r$, the number of integers in $\{1, \dots, N\}$ coprime to N is*

$$\varphi(N) = N \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right).$$

Proof. Let $S = \{1, \dots, N\}$ and let S_i be the multiples of p_i in S . An integer is coprime to N iff it is divisible by none of $p_1, \dots, p_r$, so $\varphi(N) = |S \setminus \bigcup_i S_i|$. For an index set I , an element lies in S_I iff it is divisible by every p_i with $i \in I$, i.e. by the product $\prod_{i \in I} p_i$ (the primes are distinct, so this is their lcm); the count of such multiples up to N is $|S_I| = N / \prod_{i \in I} p_i$. Theorem 3.3.2 then gives

$$\varphi(N) = \sum_{I \subseteq \{1, \dots, r\}} (-1)^{|I|} \frac{N}{\prod_{i \in I} p_i} = N \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right),$$

where the last equality is the expansion of the product: choosing the $-1/p_i$ factor for $i \in I$ and the 1 factor otherwise reproduces each signed term exactly once. $\square$

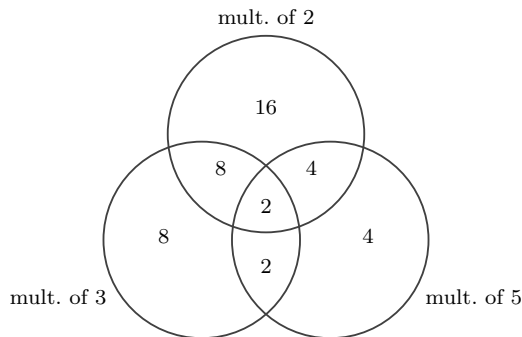
Worked Example 3.3.9 ($\varphi(60)$ by sieving out multiples of 2, 3, 5). Count the integers in $\{1, \dots, 60\}$ that share no common factor with 60 beyond 1.

Recognition. “Coprime to N ” means divisible by none of N ’s prime factors; the trigger “avoid every divisor in a fixed list” is the totient sieve with one “divisible by p_i ” condition per prime.

Solution. Factor $60 = 2^2 \cdot 3 \cdot 5$, so the distinct primes are $p_1 = 2, p_2 = 3, p_3 = 5$. Set $S_p = \{\text{multiples of } p \text{ up to } 60\}$ for $p \in \{2, 3, 5\}$: $|S_2| = 30$, $|S_3| = 20$, $|S_5| = 12$; $|S_2 \cap S_3| = 60/6 = 10$, $|S_2 \cap S_5| = 60/10 = 6$, $|S_3 \cap S_5| = 60/15 = 4$; $|S_2 \cap S_3 \cap S_5| = 60/30 = 2$. Inclusion-exclusion gives the count divisible by some prime:

$$\left| \bigcup_p S_p \right| = (30 + 20 + 12) - (10 + 6 + 4) + 2 = 62 - 20 + 2 = 44,$$

so $\varphi(60) = 60 - 44 = 16$. The three-set Venn diagram records every region exactly:



regions sum to 44; the 16 outside all circles are coprime to 60

Which idea, and why? Coprimality to N is “divisible by no prime factor of N ,” one condition per distinct prime; because the primes are distinct, a k -fold intersection counts multiples of their product, and the signed sum factors into $N \prod (1 - 1/p_i)$.

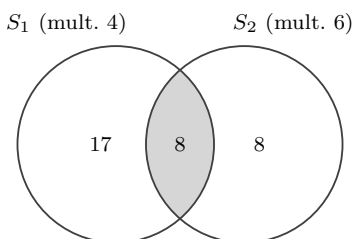
Check. The product formula confirms it directly: $\varphi(60) = 60 \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{4}{5} = 60 \cdot \frac{8}{30} = 16$.

Worked Example 3.3.10 (When the intersections are not constant: a two-divisor count). How many integers in $\{1, \dots, 100\}$ are divisible by neither 4 nor 6?

Recognition. Two forbidden conditions (“divisible by 4,” “divisible by 6”), so this is the two-set complement form. The trigger is that the divisors are *not* coprime, so the intersection counts multiples of their lcm, not their product.

Solution. Let $S = \{1, \dots, 100\}$, $S_1 = \{\text{multiples of 4}\}$, $S_2 = \{\text{multiples of 6}\}$. Then $|S_1| = \lfloor 100/4 \rfloor = 25$ and $|S_2| = \lfloor 100/6 \rfloor = 16$. The overlap is multiples of $\text{lcm}(4, 6) = 12$, not of 24: $|S_1 \cap S_2| = \lfloor 100/12 \rfloor = 8$. So

$$|S \setminus (S_1 \cup S_2)| = 100 - 25 - 16 + 8 = 67.$$



lens is mult. of $\text{lcm}(4, 6) = 12$, so $8 - \text{not } \lfloor 100/24 \rfloor = 4$

Which idea, and why? For two non-coprime divisors the pairwise overlap counts multiples of the *least common multiple*; using the product $4 \cdot 6 = 24$ would undercount the lens and deflate the answer.

Check. Inside-the-circles count is $25 + 16 - 8 = 33$ divisible by 4 or 6; $100 - 33 = 67$ matches. Using 24 wrongly would give $100 - 25 - 16 + 4 = 63$, a detectable error of 4.

Method (Setting up an inclusion-exclusion count).

1. **Name the universe S .** Choose the set of *all* objects with no restrictions, whose total size $|S|$ is easy to write down.
2. **Turn each restriction into a “bad” subset S_i .** Make S_i the objects that *violate* the i th requirement, so that the thing you want is $S \setminus \bigcup_i S_i$ (the objects violating nothing).
3. **Compute $|S_I|$ for a generic index set.** Find $|S_i|$, then $|S_i \cap S_j|$, and so on; the construction is well chosen exactly when these intersection sizes are easy and usually depend only on $|I| = k$.
4. **Assemble the alternating sum.** Add $(-1)^{|I|} |S_I|$ over all index sets, grouping equal-size terms as $\sum_k (-1)^k (\text{number of size-}k \text{ sets}) \cdot (\text{common } |S_I|)$.
5. **Sanity-check small cases or a known formula** (e.g. a Stirling-number or product-formula cross-check, or a hand count for tiny parameters).

Caution. Inclusion-exclusion only works cleanly when you sieve out conditions you can *intersect*. Do *not* define S_i to be “the objects with exactly i violations” – those sets are disjoint, so the alternating sum collapses and computing each $|S_i|$ is as hard as the original problem. The right S_i is “the objects that violate condition i (and possibly others),” so that overlaps occur and the intersections $S_i \cap S_j$ become easy.

Caution. When forming a k -fold intersection of divisibility (or pattern) conditions, intersect the *constraints*, not the numbers. The integers divisible by both 4 and 6 are the multiples of $\text{lcm}(4, 6) = 12$, not of $4 \cdot 6 = 24$. Only when the divisors are pairwise coprime (as in the totient proof) does the product equal the lcm and the shortcut $|S_I| = N / \prod p_i$ apply.

Exercises.

1. Of 200 students, 90 take algorithms, 80 take databases, and 35 take both. Using the two-set inclusion-exclusion identity, how many take at least one of the two courses, and how many take neither? Draw the two-circle Venn diagram and label all four regions with their counts.

2. Count the integers in $\{1, \dots, 300\}$ that are divisible by at least one of 2, 3, 5. Then count those divisible by none of them. Show the seven region counts on a three-set Venn diagram and verify that they sum to your union total.
3. Compute $\varphi(N)$ for $N = 84$ and $N = 1000$ using the product formula $\varphi(N) = N \prod_{p|N} (1 - 1/p)$, and confirm each by writing out the inclusion-exclusion alternating sum over the prime divisors.
4. How many functions from a 4-element set onto a 2-element set are there? Compute it both directly (a short argument) and from the surjection formula $\sum_k (-1)^k \binom{m}{k} (m-k)^n$, and check that the two agree.
5. Count the surjections from a 6-element set onto a 4-element set. Lay out the alternating sum term by term, and cross-check against $m! S(n, m)$ given that $S(6, 4) = 65$.
6. Prove the recurrence $D_n = (n-1)(D_{n-1} + D_{n-2})$ for the derangement numbers, where $D_0 = 1$ and $D_1 = 0$. Use it to tabulate $D_2, \dots, D_6$, and confirm $D_4 = 9$ and $D_6 = 265$ against the closed-form $D_n = n! \sum_{k=0}^n (-1)^k / k!$.
7. How many permutations of $\{1, 2, 3, 4, 5\}$ fix *exactly* two of the five points? (Hint: choose which two are fixed, then derange the rest.) Generalize to a formula for the number of permutations of $\{1, \dots, n\}$ with exactly j fixed points in terms of $\binom{n}{j}$ and a derangement number.
8. Count the binary strings of length 8 over $\{0, 1\}$ that contain at least one occurrence of each of the patterns 00 and 11. Set up the universe and the two “missing pattern” subsets, and explain why inclusion-exclusion applies; you may leave the two single-pattern counts as named quantities if you cannot evaluate them in closed form.
9. How many ways can 5 distinct gifts be placed into 3 distinct boxes so that no box is empty? State the answer using the surjection count, then re-derive the same number directly with inclusion-exclusion on the “box i is empty” conditions.
10. Let $S = \{1, \dots, n\}$ and let $S_1, \dots, S_n$ be subsets. Prove the *union* form of inclusion-exclusion, $|\bigcup_i S_i| = \sum_{\emptyset \neq I} (-1)^{|I|+1} |S_I|$, directly from the complement form, and state precisely where the convention $S_\emptyset = S$ is used.
11. *Find the bug.* A student counts the integers in $\{1, \dots, 120\}$ divisible by none of 4, 6, 9 as $120 - \lfloor 120/4 \rfloor - \lfloor 120/6 \rfloor - \lfloor 120/9 \rfloor + \lfloor 120/24 \rfloor + \lfloor 120/36 \rfloor + \lfloor 120/54 \rfloor - \lfloor 120/216 \rfloor$. Identify which intersection terms are wrong, give the correct value of each pairwise and triple intersection (using least common multiples), and compute the right answer.
12. *Find the bug.* To count derangements of 5 objects, someone defines S_i to be “the permutations with exactly i fixed points” and writes $D_5 = |S| - |S_1| + |S_2| - \dots$. Explain why these S_i are the wrong subsets for an inclusion-exclusion sieve, what goes wrong when you try to compute the terms, and how to redefine the subsets so the principle applies.

3.4 Recurrences and Generating Functions

A *recurrence* defines a sequence by saying how each term is built from earlier ones, together with a few boundary values; it is the natural way a recursive algorithm’s cost, or the number of structures of size n , gets specified. The trouble is that a recurrence tells you each term only in terms of its predecessors, so reading off the n th term means grinding through all the ones before it. The one key idea of this section is to pack the whole sequence into a single algebraic object – its *ordinary generating function* – so that the recurrence becomes one equation we can solve in closed form, and the answer to “what is a_n ?” becomes “what is the coefficient of x^n ?” This turns a step-by-step unfolding into routine algebra, and it is the bridge between the counting of the previous sections and the running-time analyses we will need later.

Definition 3.4.1 (Ordinary generating function). Let $a_0, a_1, a_2, \dots$ be a sequence of numbers. Its *ordinary generating function* (ogf) is the formal power series

$$f(x) = \sum_{n \geq 0} a_n x^n = a_0 + a_1 x + a_2 x^2 + \dots$$

We write $[x^n]f(x) = a_n$ for the operation of *extracting* the coefficient of x^n . The series is *formal*: x is a bookkeeping placeholder and we never substitute a numerical value for it, so questions of convergence do not arise – the powers of x are just labelled slots holding the terms of the sequence.

The point of the word *formal* is that two power series are added and multiplied exactly like polynomials, coefficient by coefficient. Addition is termwise, and the product $(\sum_n a_n x^n)(\sum_n b_n x^n) = \sum_n c_n x^n$ has $c_n = \sum_{k=0}^n a_k b_{n-k}$, the *Cauchy product*. We will lean on one identity constantly.

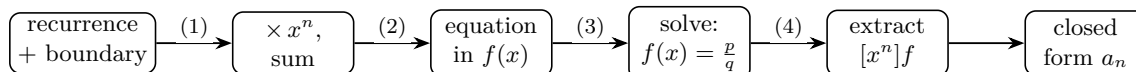
Lemma 3.4.2 (Geometric series). *In the algebra of formal power series, for any constant α ,*

$$\frac{1}{1 - \alpha x} = \sum_{n \geq 0} \alpha^n x^n.$$

Proof. We must check that $1 - \alpha x$ and $\sum_{n \geq 0} \alpha^n x^n$ multiply to 1. By the Cauchy product, the coefficient of x^0 in $(1 - \alpha x) \sum_{n \geq 0} \alpha^n x^n$ is 1, and for $n \geq 1$ the coefficient of x^n is $\alpha^n - \alpha \cdot \alpha^{n-1} = 0$. Every coefficient beyond the constant term cancels, so the product is the series 1, which is exactly what “being the inverse of $1 - \alpha x$ ” means. $\square$

The reason this lemma is the workhorse is the bridge it builds: a sequence whose ogf is $1/(1 - \alpha x)$ is exactly the geometric sequence $a_n = \alpha^n$. Solving a recurrence will repeatedly produce a generating function that is a ratio of polynomials, and our whole job in the last step will be to break that ratio into pieces of the form $1/(1 - \alpha x)$ and read the exponentials straight off.

The four-step recipe. Every linear recurrence in this section is solved the same way: (1) multiply the recurrence by x^n and sum over all indices where it is valid; (2) rewrite each sum in terms of $f(x) = \sum_n a_n x^n$, using the boundary values to account for the missing low-order terms; (3) solve the resulting equation algebraically for $f(x)$; (4) expand $f(x)$ back into a power series and extract $[x^n]f(x)$. We carry it out on increasingly substantial examples.



Worked Example 3.4.3 (A first-order recurrence: $a_n = 2a_{n-1}$, $a_0 = 3$). Solve the recurrence $a_0 = 3$ and $a_n = 2a_{n-1}$ for $n \geq 1$ in closed form using its generating function.

Recognition. The recurrence is *linear, first order, constant coefficient*: each term is a fixed multiple of the one before. The trigger “ a_n equals a constant times a_{n-1} ” tells us the ogf will be a single geometric factor, so Lemma 3.4.2 will finish the job.

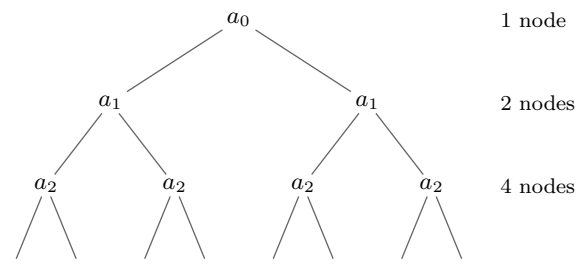
Solution. Multiply $a_n = 2a_{n-1}$ by x^n and sum over $n \geq 1$ (the range where the rule holds):

$$\sum_{n \geq 1} a_n x^n = 2 \sum_{n \geq 1} a_{n-1} x^n.$$

The left side is $f(x) - a_0 = f(x) - 3$. On the right, pull out one x and reindex $m = n - 1$: $2 \sum_{n \geq 1} a_{n-1} x^n = 2x \sum_{m \geq 0} a_m x^m = 2x f(x)$. Hence

$$f(x) - 3 = 2x f(x) \implies f(x)(1 - 2x) = 3 \implies f(x) = \frac{3}{1 - 2x}.$$

By Lemma 3.4.2 with $\alpha = 2$, $f(x) = 3 \sum_{n \geq 0} 2^n x^n$, so $a_n = 3 \cdot 2^n$. The doubling is exactly what a balanced recursive split of a size- n problem produces at depth n , which is why this shape recurs throughout runtime analysis:



Which idea, and why? A first-order constant-coefficient recurrence yields an ogf of the form $c/(1-\alpha x)$, whose single geometric term gives $a_n = c\alpha^n$ at once; no partial fractions are needed because there is only one factor in the denominator.

Check. Directly: $a_1 = 2 \cdot 3 = 6 = 3 \cdot 2^1$, $a_2 = 2 \cdot 6 = 12 = 3 \cdot 2^2$. The formula $a_n = 3 \cdot 2^n$ agrees and also gives $a_0 = 3 \cdot 2^0 = 3$, matching the boundary value.

Worked Example 3.4.4 (An inhomogeneous recurrence: $a_n = 2a_{n-1} + 1$, $a_0 = 0$). This is the cost recurrence for moving a tower of n disks (move $n-1$, move one, move $n-1$). Solve $a_0 = 0$, $a_n = 2a_{n-1} + 1$ for $n \geq 1$.

Recognition. Same first-order shape, but with a constant “+1” driving term. The trigger “a fixed *added* constant” tells us the ogf picks up an extra $\frac{x}{1-x} = \sum_{n \geq 1} x^n$ piece (the all-ones sequence, summed where the recurrence holds), and the denominator will now have two factors, $(1-2x)$ and $(1-x)$.

Solution. Multiply by x^n and sum over $n \geq 1$:

$$\sum_{n \geq 1} a_n x^n = 2 \sum_{n \geq 1} a_{n-1} x^n + \sum_{n \geq 1} x^n.$$

The left side is $f(x) - a_0 = f(x)$. The first right-hand sum is $2xf(x)$ as before; the second is $\sum_{n \geq 1} x^n = \frac{x}{1-x}$. So $f(x) = 2xf(x) + \frac{x}{1-x}$, giving

$$f(x)(1-2x) = \frac{x}{1-x} \implies f(x) = \frac{x}{(1-x)(1-2x)}.$$

Split by partial fractions: $\frac{x}{(1-x)(1-2x)} = \frac{A}{1-x} + \frac{B}{1-2x}$ forces $x = A(1-2x) + B(1-x)$. Setting $x = 1$ gives $1 = A(-1)$, so $A = -1$; setting $x = \frac{1}{2}$ gives $\frac{1}{2} = B(\frac{1}{2})$, so $B = 1$. Thus

$$f(x) = \frac{1}{1-2x} - \frac{1}{1-x} = \sum_{n \geq 0} 2^n x^n - \sum_{n \geq 0} x^n, \quad a_n = 2^n - 1.$$

Which idea, and why? An additive constant +1 in a linear recurrence contributes $\frac{x}{1-x} = \sum_{n \geq 1} x^n$, the constant summed over $n \geq 1$; the denominator here factors as a product of distinct $(1-\alpha_i x)$, and partial fractions split it into one geometric term per factor.

Check. $a_1 = 2 \cdot 0 + 1 = 1 = 2^1 - 1$; $a_2 = 2 \cdot 1 + 1 = 3 = 2^2 - 1$; $a_3 = 2 \cdot 3 + 1 = 7 = 2^3 - 1$. The closed form $2^n - 1$ matches, and $a_0 = 2^0 - 1 = 0$ recovers the boundary value.

Worked Example 3.4.5 (The Fibonacci recurrence and its closed form). Solve $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$, and read off the exponential growth rate.

Recognition. A *second-order* linear recurrence: each term depends on the two before it. The trigger “order two” tells us the denominator of the ogf will be a quadratic, $1-x-x^2$, whose two roots give two geometric terms – and here the formula is genuinely impossible to guess from the integers $0, 1, 1, 2, 3, 5, 8, \dots$, which is exactly when generating functions earn their keep.

Solution. Multiply by x^n and sum over $n \geq 2$. With $f(x) = \sum_{n \geq 0} F_n x^n$,

$$\underbrace{\sum_{n \geq 2} F_n x^n}_{f(x) - F_0 - F_1 x = f(x) - x} = \underbrace{\sum_{n \geq 2} F_{n-1} x^n}_{x(f(x) - F_0) = xf(x)} + \underbrace{\sum_{n \geq 2} F_{n-2} x^n}_{x^2 f(x)}.$$

So $f(x) - x = xf(x) + x^2 f(x)$, hence

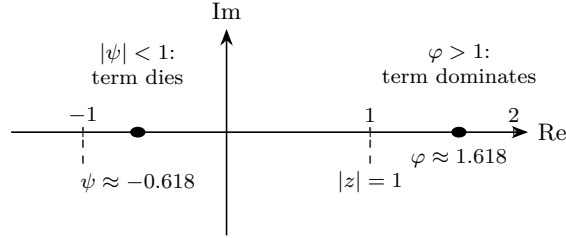
$$f(x) = \frac{x}{1-x-x^2}.$$

Factor the denominator. Its reciprocal roots are the solutions of $r^2 - r - 1 = 0$, namely $\varphi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$ (the golden ratio and its conjugate), and one checks $1-x-x^2 = (1-\varphi x)(1-\psi x)$. Partial fractions

$\frac{x}{(1-\varphi x)(1-\psi x)} = \frac{A}{1-\varphi x} + \frac{B}{1-\psi x}$ give, after clearing, $A = \frac{1}{\sqrt{5}}$ and $B = -\frac{1}{\sqrt{5}}$ (using $\varphi - \psi = \sqrt{5}$). Expanding each term geometrically,

$$F_n = [x^n]f(x) = \frac{1}{\sqrt{5}} \varphi^n - \frac{1}{\sqrt{5}} \psi^n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right].$$

This is *Binet's formula*. Since $|\psi| = \frac{\sqrt{5}-1}{2} \approx 0.618 < 1$, the second term shrinks to 0, so F_n is the nearest integer to $\varphi^n/\sqrt{5}$ and grows like $\Theta(\varphi^n)$. The reciprocal-root structure is worth a picture:



Which idea, and why? For an order- d linear recurrence the ogf is $p(x)/q(x)$ with $\deg q = d$; the roots of q supply d geometric terms, and the reciprocal root of largest modulus – here φ – sets the asymptotic growth rate.

Check. $F_2 = \frac{1}{\sqrt{5}}(\varphi^2 - \psi^2) = \frac{1}{\sqrt{5}}(\varphi - \psi)(\varphi + \psi) = \frac{1}{\sqrt{5}} \cdot \sqrt{5} \cdot 1 = 1$, matching $F_2 = F_1 + F_0 = 1$. Numerically $\varphi^{10}/\sqrt{5} \approx 55.0036$, and indeed $F_{10} = 55$.

Theorem 3.4.6 (Generating functions of linear recurrences). *A sequence $a_0, a_1, a_2, \dots$ satisfies a homogeneous linear recurrence with constant coefficients,*

$$a_n + c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_d a_{n-d} = 0 \quad (n \geq d), \quad c_d \neq 0,$$

if and only if its ordinary generating function is a rational function

$$f(x) = \frac{p(x)}{q(x)}, \quad q(x) = 1 + c_1 x + c_2 x^2 + \dots + c_d x^d, \quad \deg p < d.$$

Proof. ($\Rightarrow$) Multiply the recurrence by x^n and sum over $n \geq d$. Grouping the sums for each shift a_{n-k} gives $x^k(f(x) - (\text{its first } d-k \text{ terms}))$, so the total is $q(x)f(x)$ minus a polynomial collecting all the subtracted low-order terms. Because every subtracted term has degree at most $d-1$, that polynomial – call it $p(x)$ – has $\deg p < d$, and the summed recurrence reads $q(x)f(x) - p(x) = 0$, i.e. $f(x) = p(x)/q(x)$.

($\Leftarrow$) Conversely, suppose $f(x) = p(x)/q(x)$ with q as displayed and $\deg p < d$. Clearing denominators, $q(x)f(x) = p(x)$. Comparing coefficients of x^n for any $n \geq d$: the left side contributes $a_n + c_1 a_{n-1} + \dots + c_d a_{n-d}$, while the right side contributes 0 because $\deg p < d \leq n$. That equation is exactly the recurrence, valid for all $n \geq d$. $\square$

Corollary 3.4.7 (Shape of the closed form). *With $q(x) = \prod_{i=1}^k (1 - r_i x)^{d_i}$ in lowest terms (distinct nonzero reciprocal roots r_i , multiplicities d_i), the solution has the form*

$$a_n = \sum_{i=1}^k P_i(n) r_i^n,$$

where each P_i is a polynomial of degree less than d_i . In particular, when all roots are simple ($d_i = 1$) the closed form is a plain linear combination $a_n = \sum_i A_i r_i^n$ of exponentials.

Proof. Partial-fraction $p(x)/q(x)$ into terms $A_{i,j}/(1 - r_i x)^j$ for $1 \leq j \leq d_i$. By the generalized geometric expansion $[x^n](1 - rx)^{-j} = \binom{n+j-1}{j-1} r^n$, and $\binom{n+j-1}{j-1}$ is a polynomial in n of degree $j-1 < d_i$. Summing these contributions over j collects, for each i , a polynomial $P_i(n)$ of degree below d_i multiplying r_i^n . $\square$

Worked Example 3.4.8 (A repeated root: $a_n = 4a_{n-1} - 4a_{n-2}$, $a_0 = 1$, $a_1 = 4$). Solve the recurrence and observe what a double root does to the closed form.

Recognition. Order two again, but the trigger here is that the denominator $1 - 4x + 4x^2 = (1 - 2x)^2$ is a *perfect square*: a repeated reciprocal root $r = 2$ of multiplicity 2. Corollary 3.4.7 predicts not a constant but a *degree-one polynomial* in n times 2^n .

Solution. Following the recipe, $\sum_{n \geq 2} a_n x^n = 4x \sum_{n \geq 2} a_{n-1} x^{n-1} - 4x^2 \sum_{n \geq 2} a_{n-2} x^{n-2}$. The left side is $f(x) - 1 - 4x$; the right side is $4x(f(x) - 1) - 4x^2 f(x)$. Solving,

$$f(x) - 1 - 4x = 4xf(x) - 4x - 4x^2 f(x) \implies f(x)(1 - 4x + 4x^2) = 1 \implies f(x) = \frac{1}{(1 - 2x)^2}.$$

By the generalized expansion $[x^n](1 - 2x)^{-2} = \binom{n+1}{1} 2^n = (n+1)2^n$, so

$$a_n = (n+1)2^n.$$

Which idea, and why? A reciprocal root of multiplicity m supplies a factor $(1 - rx)^{-m}$, whose coefficients $\binom{n+m-1}{m-1} r^n$ are a degree- $(m-1)$ polynomial in n times r^n ; a double root therefore yields a *linear* prefactor, not a constant.

Check. The recurrence gives $a_2 = 4 \cdot 4 - 4 \cdot 1 = 12$, and the formula gives $(2+1)2^2 = 12$; $a_3 = 4 \cdot 12 - 4 \cdot 4 = 32 = (3+1)2^3 = 32$. Boundaries: $(0+1)2^0 = 1 = a_0$, $(1+1)2^1 = 4 = a_1$.

Worked Example 3.4.9 (A nonlinear recurrence: the Catalan numbers). The number C_n of full binary trees with n internal nodes satisfies $C_0 = 1$ and the convolution recurrence $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$ for $n \geq 1$. Find its generating function.

Recognition. The right side is a *Cauchy product* of the sequence with itself – the hallmark of a “split into a left part and a right part” decomposition. The trigger “ a_n is a convolution $\sum a_i a_{n-1-i}$ ” tells us the ogf satisfies a *quadratic* equation $c(x) = 1 + x c(x)^2$, not a linear one, so the answer will be algebraic rather than rational.

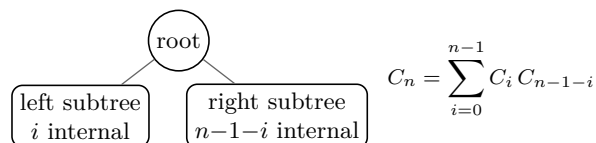
Solution. Let $c(x) = \sum_{n \geq 0} C_n x^n$. A full binary tree is either a leaf (the $n = 0$ case, giving the 1) or a root with a left and a right subtree whose internal-node counts add to $n - 1$; that is precisely the convolution. Multiplying the recurrence by x^n and summing over $n \geq 1$, the right-hand convolution sums to $x c(x)^2$, so

$$c(x) = 1 + x c(x)^2.$$

Solve the quadratic $x c^2 - c + 1 = 0$ for $c = c(x)$ by the quadratic formula: $c(x) = \frac{1 \pm \sqrt{1 - 4x}}{2x}$. Choose the sign that keeps $c(x)$ a genuine power series: at $x = 0$ we need $c(0) = C_0 = 1$, and only the minus sign gives the $\frac{0}{0} \rightarrow 1$ behaviour (the plus sign blows up like $1/x$). Hence

$$c(x) = \frac{1 - \sqrt{1 - 4x}}{2x}, \quad \text{and expanding,} \quad C_n = \frac{1}{n+1} \binom{2n}{n}.$$

The decomposition that produces the convolution is the structural content of the recurrence:



Which idea, and why? A self-convolution recurrence $a_n = \sum a_i a_{n-1-i}$ translates to a *quadratic* functional equation in the ogf because the Cauchy product of f with itself is f^2 ; solving it gives an algebraic (square-root) generating function rather than a rational one.

Check. Expanding $c(x) = 1 + x + 2x^2 + 5x^3 + 14x^4 + \dots$ matches $C_0, \dots, C_4 = 1, 1, 2, 5, 14$, and the closed form gives $C_3 = \frac{1}{4} \binom{6}{3} = \frac{20}{4} = 5$.

Worked Example 3.4.10 (Reading asymptotics off the dominant pole). A divide-and-conquer count satisfies $a_0 = 1$ and $a_n = 3a_{n-1} + 2^n$ for $n \geq 1$. Find a_n and its growth rate without fully solving, then confirm.

Recognition. The driving term 2^n is itself geometric, with ogf $\frac{1}{1-2x}$. The trigger “constant coefficient recurrence plus a geometric forcing term” tells us the ogf is rational with denominator $(1-3x)(1-2x)$; the *larger* reciprocal root – equivalently the pole closest to the origin, at $x = \frac{1}{3}$ – controls the asymptotics.

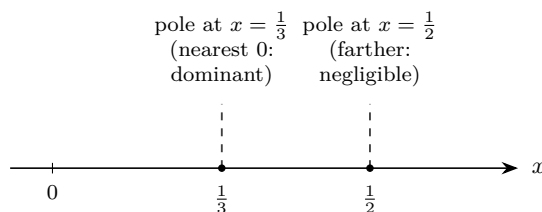
Solution. Multiply by x^n , sum over $n \geq 1$. The left side is $f(x) - 1$; the $3a_{n-1}$ part gives $3xf(x)$; the forcing part gives $\sum_{n \geq 1} 2^n x^n = \frac{1}{1-2x} - 1 = \frac{2x}{1-2x}$. So

$$f(x) - 1 = 3xf(x) + \frac{2x}{1-2x} \implies f(x)(1-3x) = 1 + \frac{2x}{1-2x} = \frac{1}{1-2x} \implies f(x) = \frac{1}{(1-3x)(1-2x)}.$$

Partial fractions: $\frac{1}{(1-3x)(1-2x)} = \frac{3}{1-3x} - \frac{2}{1-2x}$ (clear denominators and match), so

$$a_n = 3 \cdot 3^n - 2 \cdot 2^n = 3^{n+1} - 2^{n+1}.$$

Of the two reciprocal roots $r = 3$ and $r = 2$, the larger is 3, so $a_n = \Theta(3^n)$: the term from the pole nearest the origin (at $x = \frac{1}{3}$) dominates.



Poles of $f(x) = \frac{1}{(1-3x)(1-2x)}$ on the real axis. The pole nearest the origin, at $x = \frac{1}{3}$, sets the rate $(1/\frac{1}{3})^n = 3^n$.

Which idea, and why? For a rational ogf, the singularity (pole) nearest the origin governs the growth: a pole at $x = 1/r$ contributes a term growing like r^n , and the smallest such $1/r$ – the largest r – wins; here $x = \frac{1}{3}$ beats $x = \frac{1}{2}$, giving $\Theta(3^n)$.

Check. $a_1 = 3 \cdot 1 + 2^1 = 5 = 3^2 - 2^2$; $a_2 = 3 \cdot 5 + 2^2 = 19 = 3^3 - 2^3 = 27 - 8$; and $3^{n+1} - 2^{n+1}$ gives $a_0 = 3 - 2 = 1$, the boundary value.

Method (How to solve a recurrence with its generating function).

1. **Set up.** Write $f(x) = \sum_{n \geq 0} a_n x^n$. Multiply the recurrence by x^n and sum over the smallest range of n for which it is valid (typically $n \geq d$ for an order- d rule).
2. **Translate each piece.** A shift a_{n-k} becomes $x^k f(x)$ after reindexing; a truncated sum $\sum_{n \geq d} a_n x^n$ becomes $f(x)$ minus its first d terms, which you supply from the boundary conditions. A geometric forcing term β^n becomes $\frac{1}{1-\beta x}$ (minus its missing low terms).
3. **Solve algebraically.** Collect all $f(x)$ terms on one side and divide to get a closed expression for $f(x)$. For a *linear* recurrence this is a rational function $p(x)/q(x)$ with $\deg p < \deg q$; for a self-convolution it is a quadratic in $f(x)$ – use the quadratic formula and pick the root with the correct constant term.
4. **Extract coefficients.** Factor $q(x) = \prod_i (1 - r_i x)^{d_i}$, do a partial-fraction split, and expand each $1/(1 - r_i x)^j$ via $[x^n](1 - r_i x)^{-j} = \binom{n+j-1}{j-1} r_i^n$. Sum to read off a_n .
5. **Check and read growth.** Verify the closed form on the boundary values and one or two computed terms. For asymptotics, the largest reciprocal root r_i (the pole nearest the origin) gives $a_n = \Theta(n^{d_i-1} r_i^n)$, provided p/q is in lowest terms and no other reciprocal root has the same modulus.

Caution. When translating a truncated sum, do not forget to subtract the right number of low-order terms. $\sum_{n \geq 2} a_n x^n$ is $f(x) - a_0 - a_1 x$, *not* $f(x)$; and after reindexing a shifted sum, re-examine which boundary terms survive. A single dropped a_0 or $a_1 x$ silently corrupts the numerator $p(x)$ and produces a wrong closed form that still “looks” rational.

Caution. The expansion $\frac{1}{1-\alpha x} = \sum_{n \geq 0} \alpha^n x^n$ requires the constant term of the denominator to be 1. If your denominator comes out as, say, $2-3x$, factor the constant out first: $\frac{1}{2-3x} = \frac{1}{2} \cdot \frac{1}{1-\frac{3}{2}x} = \frac{1}{2} \sum_{n \geq 0} (\frac{3}{2})^n x^n$. Applying the geometric formula to $2-3x$ as if it were $1-3x$ is a common and silent error.

Caution. For a *repeated* root you must include the higher-power partial fractions. A factor $(1-2x)^2$ in the denominator needs *both* $\frac{A}{1-2x}$ and $\frac{B}{(1-2x)^2}$; writing only the first throws away the $n \cdot 2^n$ part of the answer. By Corollary 3.4.7, a multiplicity- m root carries a degree- $(m-1)$ polynomial prefactor, so it contributes m partial-fraction terms, not one.

Exercises.

- Let $a_0 = 5$ and $a_n = 4a_{n-1}$ for $n \geq 1$. Find the generating function $f(x) = \sum_{n \geq 0} a_n x^n$ in closed form, and read off a formula for a_n .
- A sequence satisfies $b_0 = 1$ and $b_n = b_{n-1} + 3$ for $n \geq 1$. Using generating functions, show $f(x) = \frac{1+2x}{(1-x)^2}$ and deduce $b_n = 3n + 1$.
- Solve $a_0 = 2$, $a_1 = 5$, $a_n = 5a_{n-1} - 6a_{n-2}$ for $n \geq 2$. Factor the denominator of the generating function, do the partial-fraction split, and give the closed form a_n as a combination of two geometric terms.
- (Repeated root.) Solve $a_0 = 1$, $a_1 = 6$, $a_n = 6a_{n-1} - 9a_{n-2}$ for $n \geq 2$. Show the generating function is $\frac{1}{(1-3x)^2}$ and conclude $a_n = (n+1)3^n$. Explain, in one sentence, why the answer has a factor linear in n .
- Find the ordinary generating function for the sequence $a_n = n$ (that is, $0, 1, 2, 3, \dots$) by starting from $\frac{1}{1-x} = \sum_{n \geq 0} x^n$ and differentiating-then-multiplying, or by solving the recurrence $a_0 = 0$, $a_n = a_{n-1} + 1$. Confirm both routes give $\frac{x}{(1-x)^2}$.
- Let T_n count the binary strings of length n over $\{0, 1\}$ that contain no two consecutive 1s. Set up a recurrence for T_n with boundary values, solve it with generating functions, and identify the resulting growth rate. (You should recognize a familiar denominator.)
- Determine the asymptotic growth rate, in Θ -notation, of the sequence whose generating function is $\frac{1+x}{(1-2x)(1-5x)}$, *without* computing the full closed form. State which pole controls the answer and why.
- Prove that if $f(x)$ is the ogf of $(a_n)_{n \geq 0}$, then $\frac{f(x)}{1-x}$ is the ogf of the sequence of partial sums $s_n = \sum_{k=0}^n a_k$. (Hint: use the Cauchy product with the all-ones series.) Then use this to find the ogf of the sequence $s_n = \sum_{k=0}^n 2^k$.
- (Convolution / Catalan-style.) A sequence satisfies $g_0 = 1$ and $g_n = \sum_{i=0}^{n-1} g_i g_{n-1-i}$ for $n \geq 1$. Write the functional equation its generating function $g(x)$ must satisfy, solve for $g(x)$ with the correct branch of the square root, and state $g_0, \dots, g_4$.
- Solve the inhomogeneous recurrence $a_0 = 0$, $a_n = 2a_{n-1} + n$ for $n \geq 1$ using generating functions. (You will need the ogf of the sequence n from Exercise 5 as the forcing term; expect a denominator with both $(1-2x)$ and $(1-x)^2$.)
- (Find the bug.) A student wants to solve $a_0 = 1$, $a_n = 2a_{n-1}$ ($n \geq 1$). They write $\sum_{n \geq 1} a_n x^n = 2 \sum_{n \geq 1} a_{n-1} x^n$, then claim the left side equals $f(x)$ and conclude $f(x) = 2x f(x)$, so that $f(x) = 0$. Identify the exact step where they mishandled a boundary term, correct it, and give the right closed form.
- (Find the bug.) To solve $a_0 = 1$, $a_1 = 1$, $a_n = 4a_{n-2}$ ($n \geq 2$), a student factors the generating-function denominator as $1-4x^2 = (1-2x)(1-2x)$ and writes the answer as a single term $A \cdot 2^n$. Explain what is wrong with the factorization, give the correct factorization of $1-4x^2$, and write the corrected closed form for a_n .

3.5 Counting for Computation

Every machine we will ever build is a finite object: it can be written down with finitely many symbols. Every language, by contrast, is a set of strings, and there are vastly more of those. The one key idea of this section is that simply *counting* the two kinds of objects – machines on one side, languages on the other – forces deep limitations on computation, with no clever construction required. We first count strings and languages exactly, then run a cardinality comparison to conclude that most languages are not regular and indeed most are undecidable; we then use the same counting style to show that hard Boolean functions must exist, and we close with a counting tool, Burnside’s lemma, that counts configurations up to symmetry.

Counting strings and languages

Fix a finite alphabet Σ with $|\Sigma| = s \geq 1$. The most basic count is the number of strings of a given length.

Proposition 3.5.1 (Strings of a fixed length). *For every $n \geq 0$ the number of strings in Σ^* of length exactly n is s^n . Consequently the number of strings of length at most n is $\sum_{k=0}^n s^k = \frac{s^{n+1}-1}{s-1}$ when $s \geq 2$, and $n+1$ when $s = 1$.*

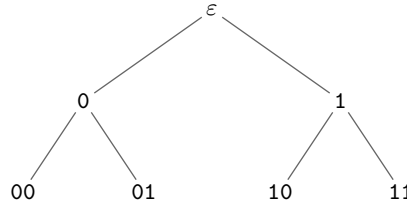
Proof. A length- n string is an ordered choice of n symbols, each from a set of size s , so the product rule gives s^n . There is exactly one string of length 0 (the empty string ε), matching $s^0 = 1$. Summing the geometric series $s^0 + s^1 + \dots + s^n$ gives the stated closed form. $\square$

The strings over Σ form an infinite set, but a *very* structured one: they fan out level by level as a complete s -ary tree rooted at ε , each node having one child per alphabet symbol.

Worked Example 3.5.2 (Counting and listing short binary strings as a tree). Over $\Sigma = \{0, 1\}$ ($s = 2$), how many strings have length at most 2, and how are they organized?

Recognition. This is a direct application of Proposition 3.5.1: the count is a sum of powers of s . The trigger “strings of length up to n , branching by one symbol at a time” says to draw the prefix tree, where depth = length and each level k holds the s^k strings of that length.

Solution. By the proposition the counts by length are $s^0 = 1$, $s^1 = 2$, $s^2 = 4$, totalling $1 + 2 + 4 = 7$. Reading the complete binary tree from the root downward lists them: ε at depth 0; 0, 1 at depth 1; 00, 01, 10, 11 at depth 2.



Which idea, and why? Each symbol read adds one level and multiplies the node count by s , so the number at depth k is s^k and the cumulative count is the geometric sum – exactly Proposition 3.5.1.

Check. The leaves at depth 2 number $2^2 = 4$, and $1 + 2 + 4 = 7$ nodes total, as the formula $\frac{2^3-1}{2-1} = 7$ predicts.

A language over Σ is any subset $\mathcal{L} \subseteq \Sigma^*$. For a *finite* universe the subsets are easy to count.

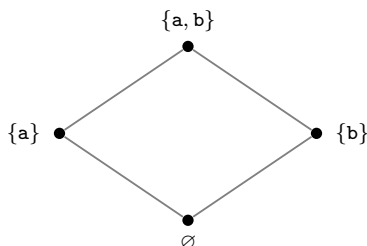
Proposition 3.5.3 (Languages over a bounded length). *Let $\Sigma_{\leq n} = \{w \in \Sigma^* \mid |w| \leq n\}$, which has $N = \frac{s^{n+1}-1}{s-1}$ elements (for $s \geq 2$). The number of languages contained in $\Sigma_{\leq n}$ is $2^N = |\mathcal{P}(\Sigma_{\leq n})|$.*

Proof. A language $\mathcal{L} \subseteq \Sigma_{\leq n}$ is determined by deciding, independently for each of the N strings, whether it is in or out – a binary choice per string. By the product rule there are 2^N such decisions, i.e. 2^N subsets, which is exactly the size of the power set $\mathcal{P}(\Sigma_{\leq n})$. $\square$

Worked Example 3.5.4 (The lattice of all languages over a tiny universe). Take the two-string universe $U = \{a, b\}$ (think of it as all length-1 strings over $\{a, b\}$). List every language $\mathcal{L} \subseteq U$ and arrange them by inclusion.

Recognition. “All subsets of a finite set, ordered by $\subseteq$ ” is the Boolean lattice. By Proposition 3.5.3 with $N = |U| = 2$ there are $2^2 = 4$ of them, and inclusion makes them a square (a Hasse diagram). The trigger is “count and organize *all* subsets,” which is the power-set construction.

Solution. The four languages are $\emptyset$, $\{a\}$, $\{b\}$, and $\{a, b\} = U$. Each single-letter language sits above $\emptyset$ and below U ; the two singletons are incomparable.



Which idea, and why? Counting languages over a fixed finite universe U is counting subsets of U : there are $2^{|U|}$ of them, and the inclusion order is the Boolean lattice $\mathcal{P}(U)$ – here a 4-element square.

Check. The four nodes match $2^N = 2^2 = 4$ from Proposition 3.5.3; each covering edge adds exactly one string to the lower language.

There are more languages than machines

Now let the universe be *all* of Σ^* . The set Σ^* is infinite but *countable*: list its strings in shortlex order (by length, ties broken alphabetically), $\varepsilon, 0, 1, 00, 01, \dots$, and this enumeration hits every string exactly once. Every machine we study – DFA, NFA, regular expression, Turing machine – can be written as a finite string over a fixed finite description alphabet, so the machines of any one type also form a countable set. Languages, however, are subsets of a countably infinite set, and there are uncountably many of those.

Theorem 3.5.5 (A diagonal language). *The set $\mathcal{P}(\Sigma^*)$ of all languages over Σ is uncountable. Hence for any countable list $M_1, M_2, M_3, \dots$ of machines there is a language recognized by none of them.*

Proof. Suppose for contradiction that the languages could be listed as $\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3, \dots$. Enumerate the strings as $w_1, w_2, w_3, \dots$ in shortlex order and define a new language by flipping the diagonal:

$$D = \{w_i \in \Sigma^* \mid w_i \notin \mathcal{L}_i\}.$$

For every index i , the language D disagrees with $\mathcal{L}_i$ on the string w_i : by construction $w_i \in D \iff w_i \notin \mathcal{L}_i$. So $D \neq \mathcal{L}_i$ for every i , contradicting that the list was complete. Thus no enumeration of all languages exists; $\mathcal{P}(\Sigma^*)$ is uncountable. The second statement follows by taking $\mathcal{L}_i = L(M_i)$: the language D equals no $L(M_i)$, so no machine on the list recognizes it. $\square$

This is a pure counting result – a *cardinality squeeze* – and it instantly bounds what any finite-description model can do. The countable side (machines, hence the languages they recognize) can never exhaust the uncountable side (all languages).

Corollary 3.5.6 (Most languages are non-regular, and most are undecidable). *There exist non-regular languages; in fact “almost all” languages are non-regular. The same counting argument shows there exist languages recognized by no Turing machine, so undecidable – in fact unrecognizable – languages exist, and “almost all” languages are undecidable.*

Proof. There are only countably many DFAs (each is a finite tuple over a fixed finite alphabet), so the regular languages – the languages they recognize – form a countable set. By Theorem 3.5.5 the set of all languages is uncountable. A countable subset cannot be all of an uncountable set, so some (indeed uncountably many) languages are non-regular. Replacing “DFA” by “Turing machine,” which is again a finite object describable by a finite string, the recognizable languages are likewise countable, while all languages are uncountable; hence unrecognizable – and a fortiori undecidable – languages exist and are the overwhelming majority. $\square$

Worked Example 3.5.7 (Why the diagonal language escapes every machine, pictured). Make the diagonal construction concrete: show how D is built so that it differs from each listed language in at least one place.

Recognition. This is the bookkeeping behind Theorem 3.5.5. Lay the data out as an infinite 0/1 table whose (i, j) entry records whether $w_j \in \mathcal{L}_i$; the trigger “build a row unequal to every existing row” means walk the main diagonal and flip each bit.

Solution. Index languages down the side and strings across the top; entry 1 means “in.” The new language D is read off the boxed diagonal with every bit flipped, so its i -th coordinate differs from $\mathcal{L}_i$ ’s i -th coordinate.

	w_1	w_2	w_3	w_4	$\dots$
$\mathcal{L}_1$	0	1	1	0	$\dots$
$\mathcal{L}_2$	1	1	0	1	$\dots$
$\mathcal{L}_3$	0	0	0	1	$\dots$
$\mathcal{L}_4$	1	1	0	1	$\dots$
D	1	0	1	0	$\dots$

Here D ’s entries 1, 0, 1, 0, $\dots$ are the boxed diagonal 0, 1, 0, 1, $\dots$ flipped. Because D disagrees with $\mathcal{L}_i$ in column w_i , it equals no row.

Which idea, and why? Diagonalization: to defeat a countable list, build one new object that is engineered to differ from the i -th listed object at the i -th coordinate, so it cannot appear anywhere on the list.

Check. Reading the diagonal: $\mathcal{L}_1$ has 0 at w_1 , so D has 1; $\mathcal{L}_2$ has 1 at w_2 , so D has 0; and so on – $D \neq \mathcal{L}_i$ for each shown i .

The relationship between the classes is a chain of strict inclusions: the regular languages sit inside the decidable ones, inside the recognizable ones, inside all languages. Counting alone forces only the last step to be strict, and that step leaves uncountably many languages behind; the first two steps compare countable classes, and their strictness needs explicit witnesses (a non-regular decidable language, and a recognizable undecidable one).

$$\begin{array}{ccccccc} \text{REGULAR} & \hookrightarrow & \text{DECIDABLE} & \hookrightarrow & \text{RECOGNIZABLE} & \hookrightarrow & \mathcal{P}(\Sigma^*) \\ & & \text{countable} \subsetneq \text{countable} \subsetneq \text{countable} \subsetneq \text{uncountable} & & & & \end{array}$$

Method (How to run a counting (cardinality) lower-bound argument).

1. **Name the two sides.** Identify the set of *solvers* (machines, circuits, formulas, descriptions of bounded size) and the set of *problems* (languages, functions) you want to solve.
2. **Count or bound the solvers from above.** Show the solvers are describable by finite strings of bounded length, so there are at most $2^{(\text{size})}$ of them – or simply countably many if the size is unbounded.
3. **Count the problems from below.** Show the problems are at least as numerous: 2^N for a universe of size N , or uncountably many for an infinite universe.
4. **Compare.** If the problems outnumber the solvers, some problem has no solver – without ever exhibiting one. For an exact threshold, find the largest size at which solvers still outnumber problems.
5. **Sharpen to “almost all” if needed.** A countable set is a vanishing fraction of an uncountable one, so the hard problems are not merely present but typical.

Counting functions versus counting circuits

The cardinality squeeze also works *quantitatively*: by counting Boolean functions against the machines that could compute them, we prove hard functions exist long before anyone exhibits one. A Boolean function on n inputs is a map $f: \{0, 1\}^n \rightarrow \{0, 1\}$.

Proposition 3.5.8 (Counting Boolean functions). *There are exactly 2^{2^n} Boolean functions of n variables.*

Proof. Such a function is fixed by its truth table: a value in $\{0, 1\}$ for each of the 2^n input rows. By the product rule the number of truth tables is $2^{(2^n)} = 2^{2^n}$. $\square$

Worked Example 3.5.9 (Few small circuits, but doubly-exponentially many functions). A Boolean circuit over the gates $\{\wedge, \vee, \neg\}$ with g gates and n inputs can be encoded: each gate names its type (one of 3 choices) and its inputs (each an earlier gate or one of the n variables). Compare the number of such circuits to the number of functions on n variables and conclude that some function needs many gates.

Recognition. Counting solvers (circuits, by their finite description) against problems (Boolean functions, counted in Proposition 3.5.8). The trigger is “encode each circuit as a short string, then compare $2^{(\text{description length})}$ to 2^{2^n} .”

Solution. Each of the g gates is specified by its type and its two input sources, each source chosen from at most $g + n$ candidates. So a circuit is described by at most $g \cdot \log_2(3(g + n)^2)$ bits, and the number of circuits with g gates is at most $(3(g + n)^2)^g$. If this is smaller than 2^{2^n} , some function is computed by no such circuit. Taking $g = 2^n/(10n)$, the bound $(3(g + n)^2)^g$ is far below 2^{2^n} , so a function on n variables exists that needs more than $2^n/(10n)$ gates.

n	#functions = 2^{2^n}	#circuits with $g = \lfloor 2^n/(10n) \rfloor$ gates (at most)
10	$2^{1,024}$	$\approx 2^{102}$ ($g = 10$)
15	$2^{32,768}$	$\approx 2^{3,774}$ ($g = 218$)
20	$2^{1,048,576}$	$\approx 2^{137,905}$ ($g = 5,242$)

Which idea, and why? Counting argument for lower bounds (Shannon’s method): the solvers are described by $\approx g \log g$ bits, giving $\approx 2^{g \log g}$ of them, while the targets number 2^{2^n} ; when targets exceed solvers, a hard target must exist, with no explicit construction.

Check. For $n = 5$ there are $2^{32} \approx 4.3 \times 10^9$ functions; allowing only, say, $g = 6$ gates gives at most $(3 \cdot 11^2)^6 = 363^6 \approx 2.3 \times 10^{15}$ labelled descriptions, more than 2^{32} , so at this size the count proves nothing. As g shrinks the circuit count drops below 2^{2^n} : at $g = 4$ it is $(3 \cdot 9^2)^4 = 243^4 \approx 3.5 \times 10^9 < 2^{32}$, forcing a function of 5 variables that needs more than 4 gates.

Counting up to symmetry: Burnside’s lemma

A final counting tool answers “how many configurations are there if we regard two as the same when one is a symmetry of the other?” – exactly the question behind counting circuits up to relabelling, graphs up to isomorphism, or states up to renaming. Let a finite group G act on a finite set X : each $g \in G$ permutes X , the identity acts trivially, and composing group elements composes the permutations. Two elements of X are equivalent when some g sends one to the other; the equivalence classes are the *orbits*, and the orbits are what we want to count. For $g \in G$ write $X^g = \{x \in X \mid g \cdot x = x\}$ for its set of fixed points.

Theorem 3.5.10 (Burnside’s lemma). *If a finite group G acts on a finite set X , the number of orbits equals the average number of fixed points:*

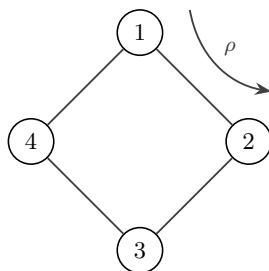
$$\#\text{orbits} = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$

Proof. Count in two ways the pairs (g, x) with $g \cdot x = x$. Summing over g first gives $\sum_{g \in G} |X^g|$. Summing over x first gives $\sum_{x \in X} |G_x|$, where $G_x = \{g \mid g \cdot x = x\}$ is the stabilizer of x . By the orbit–stabilizer relation $|G_x| = |G| / |\mathcal{O}_x|$, where $\mathcal{O}_x$ is the orbit of x ; and within a single orbit $\mathcal{O}$ each of its $|\mathcal{O}|$ members contributes $|G| / |\mathcal{O}|$, so the orbit contributes $|G|$ to the sum. Summing over the orbits gives $\sum_{x \in X} |G_x| = \#\text{orbits} \cdot |G|$. Equating the two counts and dividing by $|G|$ yields the formula. $\square$

Worked Example 3.5.11 (Two-colored necklaces up to rotation). Count the colorings of 4 beads arranged in a cycle using 2 colors, where two colorings are the same if one is a rotation of the other.

Recognition. “Count colorings up to rotation” is a group action: X is all $2^4 = 16$ colorings and $G = \{\rho^0, \rho^1, \rho^2, \rho^3\}$ is the rotation group of the 4-cycle. The trigger “same under a symmetry group” calls for Burnside’s lemma; the work is counting fixed points X^g of each rotation.

Solution. Label the bead positions 1, 2, 3, 4 around the cycle; rotation ρ sends position i to $i + 1 \pmod{4}$.



A coloring is fixed by g exactly when it is constant on each cycle of g acting on the positions, so $|X^g| = 2^{c(g)}$ where $c(g)$ is the number of cycles.

rotation g	cycle structure on $\{1, 2, 3, 4\}$	$ X^g = 2^{c(g)}$
ρ^0 (identity)	$(1)(2)(3)(4)$, $c = 4$	$2^4 = 16$
ρ^1	$(1\ 2\ 3\ 4)$, $c = 1$	$2^1 = 2$
ρ^2	$(1\ 3)(2\ 4)$, $c = 2$	$2^2 = 4$
ρ^3	$(1\ 4\ 3\ 2)$, $c = 1$	$2^1 = 2$

By Burnside's lemma the number of distinct necklaces is

$$\frac{1}{|G|} \sum_{g \in G} |X^g| = \frac{1}{4} (16 + 2 + 4 + 2) = \frac{24}{4} = 6.$$

Which idea, and why? Counting configurations up to a symmetry group is precisely counting orbits, which Burnside's lemma evaluates as the average fixed-point count; a coloring is fixed by a rotation iff it is constant on that rotation's cycles, giving $|X^g| = 2^{c(g)}$.

Check. Direct enumeration agrees: the 6 necklaces are all-color- A ; all-color- B ; one B (the single B can sit anywhere, all rotations of each other); three B 's (symmetric to one B); two adjacent B 's; and two opposite B 's – six classes in all.

Method (How to count configurations up to symmetry with Burnside).

1. **Identify the ground set X** of all configurations *before* quotienting (e.g. all r^n colorings, all labelled graphs on n vertices, all assignments).
2. **Identify the symmetry group G** that you regard as “the same” (rotations, reflections, vertex relabellings) and confirm it acts on X .
3. **For each $g \in G$, count fixed points $|X^g|$.** For colorings this is $r^{c(g)}$, where $c(g)$ is the number of cycles of g on the positions; group the g by cycle type.
4. **Average.** The number of orbits is $\frac{1}{|G|} \sum_g |X^g|$.
5. **Sanity-check** small cases by hand, and check that the average comes out to a whole number.

Caution. “Countable” is not “small.” The regular languages, the decidable languages, and the recognizable languages are each *infinite* – there are infinitely many regular languages – yet they are all countable, hence a vanishing sliver of the uncountably many languages. The counting argument of Theorem 3.5.5 only guarantees that hard languages *exist*; it names none. Producing a specific non-regular or undecidable language still requires a separate construction (a pumping argument, or a diagonal/reduction argument), which later sections supply.

Caution. In Burnside's lemma you must average $|X^g|$ over *all* $g \in G$, including the identity, whose fixed-point count is the full $|X|$ and is usually the largest term. Forgetting the identity, or dividing by the wrong group order, throws the count off. Likewise, $|X^g| = r^{c(g)}$ counts cycles of the permutation *on the positions being colored*, not the colors; mixing up the two is the classic slip.

Exercises.

1. Over $\Sigma = \{0, 1, 2\}$, how many strings have length exactly 4? How many have length at most 4? Give both as numbers, and state the general formula you used.

2. Draw the complete prefix tree of all strings of length at most 2 over $\Sigma = \{a, b, c\}$. How many leaves does it have at depth 2, and how many nodes total? Confirm both against s^n and the geometric sum.
3. Let $U = \{00, 01, 10\}$ be a three-string universe. List all languages $\mathcal{L} \subseteq U$, count them, and draw the Hasse diagram of $(\mathcal{P}(U), \subseteq)$. How many of these languages contain the string 01?
4. Prove that the set Σ^* of all strings over a finite alphabet Σ with $|\Sigma| \geq 1$ is countable by giving an explicit shortlex enumeration and explaining why every string appears exactly once.
5. Adapt the diagonal argument of the section to show directly that the set of all *infinite* binary sequences is uncountable. Set up the 0/1 table, describe the flipped diagonal sequence, and state precisely why it differs from every listed sequence.
6. Explain in your own words why the regular languages over $\{0, 1\}$ form a countable set, citing the fact that each DFA is a finite tuple. Then conclude in one sentence that some language over $\{0, 1\}$ is non-regular, *without* exhibiting one.
7. How many Boolean functions are there of $n = 1, 2$, and 3 variables? For $n = 2$, write out the truth tables of two specific functions and confirm that flipping any single output bit gives a different function, consistent with the count 2^{2^2} .
8. A circuit-style counting argument: suppose each “program” is a binary string of length at most L . Show there are at most $2^{L+1} - 1$ such programs. If you want a distinct program for each of the 2^{2^n} Boolean functions on n variables, derive a lower bound on L in terms of n , and interpret it as “most functions need long programs.”
9. Use Burnside’s lemma to count the distinct 2-colorings of the vertices of an equilateral triangle (positions 1, 2, 3 in a cycle) up to rotation only. List the three rotations, give each rotation’s cycle structure and fixed-point count $2^{c(g)}$, and average. Verify your answer by listing the orbits directly.
10. Redo the previous exercise allowing both rotations *and* reflections, so G is the full symmetry group of the triangle ($|G| = 6$). Tabulate $|X^g|$ for all six symmetries and apply Burnside’s lemma. Does allowing reflections change the count? Explain why or why not in this case.
11. Count the distinct 3-colorings of a 4-bead necklace up to rotation (group of size 4). Set up the Burnside table with $|X^g| = 3^{c(g)}$ and evaluate $\frac{1}{4}(3^4 + 3^1 + 3^2 + 3^1)$. Then state the general formula for r colors and check it gives a whole number for $r = 2$ and $r = 3$.
12. *Find the bug.* A student computes the number of 2-colored 4-bead necklaces up to rotation as $\frac{1}{4}(2^1 + 2^2 + 2^1) = 2$, claiming there are only two. Identify exactly what is wrong with the calculation, fix it, and give the correct count.
13. *Find the bug.* Another student argues: “There are only finitely many regular languages, because each DFA has finitely many states; therefore the regular languages are countable.” The conclusion (countable) is true, but one clause of the reasoning is false. Pinpoint the false clause, explain why it is false, and supply a correct one-line justification that the regular languages are countable.

Chapter 4

Finite Automata and the Regular Languages

4.1 Deterministic Finite Automata

A deterministic finite automaton (DFA) is the simplest computing device we will study: a machine with finitely many states and no memory beyond “which state am I in right now.” It reads an input string once, left to right, one symbol at a time, and each symbol forces a single, fully determined move to a next state. The one key idea of this section is that a finite set of states can encode exactly the finite amount of information a machine needs to remember about the part of the input it has already seen; everything else in the theory of regular languages is built on top of this. We make the model precise, show two equivalent ways to present a machine (the state diagram and the transition table), define what it means for a machine to accept a string, and then design several DFAs from scratch.

Definition 4.1.1 (Deterministic finite automaton). A *deterministic finite automaton* is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set, the *states*;
- Σ is a finite set, the *input alphabet*;
- $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*;
- $q_0 \in Q$ is the *start state*;
- $F \subseteq Q$ is the set of *accept* (or *final*) states.

The word *deterministic* refers to δ being a *function*: for every state q and every symbol a there is exactly one value $\delta(q, a)$, so the machine never has a choice and never gets stuck. The word *finite* refers to Q . Note carefully that δ is defined on *single symbols*; to run the machine on a whole string we extend it.

Definition 4.1.2 (Extended transition function). Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. The *extended transition function* $\hat{\delta}: Q \times \Sigma^* \rightarrow Q$ is defined by recursion on the length of the string:

$$\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a) \quad \text{for } w \in \Sigma^*, a \in \Sigma.$$

Thus $\hat{\delta}(q, w)$ is the state the machine is in after starting in q and reading all of w .

In words: feeding the empty string changes nothing, and feeding w followed by one more symbol a means “run w from q , then take the single a -step from wherever you landed.” We write $\hat{\delta}$ with a hat to keep it visually distinct from δ , but on length-one strings the two agree: $\hat{\delta}(q, a) = \delta(q, a)$.

Definition 4.1.3 (Acceptance and the language of a machine). A DFA $M = (Q, \Sigma, \delta, q_0, F)$ *accepts* a string $w \in \Sigma^*$ if $\hat{\delta}(q_0, w) \in F$, that is, if running w from the start state ends in an accept state; otherwise M *rejects* w . The *language recognized by* M is

$$L(M) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \}.$$

We say M recognizes the language $L(M)$.

Every machine recognizes exactly one language (possibly $\emptyset$, possibly all of Σ^*), but a single language is usually recognized by many different machines. The class of languages reachable this way is the central object of the chapter.

Definition 4.1.4 (Regular language). A language $\mathcal{L} \subseteq \Sigma^*$ is *regular* if $\mathcal{L} = L(M)$ for some DFA M .

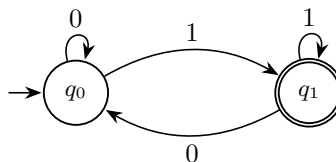
Two presentations of a machine. A DFA can be drawn as a *state diagram* – a directed graph whose nodes are the states and whose labelled edges are the transitions – or written as a *transition table* listing $\delta(q, a)$ for every state q and symbol a . The two carry identical information. The diagram makes the dynamics visible; the table makes δ unambiguous and is easy to fill in. We will always give the diagram and often the table beside it.

Worked Example 4.1.5 (Strings over $\{0, 1\}$ that end in 1). Design a DFA over $\Sigma = \{0, 1\}$ recognizing $\mathcal{L} = \{w \mid w \text{ ends in the symbol } 1\}$.

Recognition. This is a *suffix* condition: whether w is accepted depends only on its *last* symbol. The trigger “depends only on the last symbol” tells us the machine needs to remember just one bit – what the most recent symbol was.

Solution. Use two states: q_0 “last symbol was 0 (or no symbol yet)” and q_1 “last symbol was 1.” On reading 0 go to q_0 ; on reading 1 go to q_1 ; this holds from either state. Accept exactly when the last symbol was 1, so $F = \{q_1\}$. The empty string has no last symbol and must be rejected, which is automatic because $q_0 \notin F$.

$$\delta(q_0, 0) = q_0, \quad \delta(q_0, 1) = q_1, \quad \delta(q_1, 0) = q_0, \quad \delta(q_1, 1) = q_1.$$



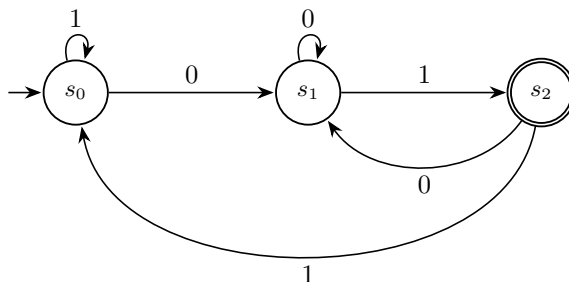
Which idea, and why? A last-symbol (suffix-of-length-one) condition needs only the most recent symbol remembered, so two states – one per possible last symbol – suffice.

Check. Trace 010: $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \notin F$, rejected (correct, it ends in 0). Trace 011: $q_0 \rightarrow q_0 \rightarrow q_1 \rightarrow q_1 \in F$, accepted.

Worked Example 4.1.6 (A fixed two-symbol suffix 01). Design a DFA over $\{0, 1\}$ recognizing $\mathcal{L} = \{w \mid w \text{ ends in } 01\}$.

Recognition. Again a suffix condition, but now of length two. The machine must remember enough of the recent past to know whether the last two symbols were 0 then 1. The trigger is “ends in a fixed string of length k ,” which calls for states tracking *how much of that suffix has just been matched*.

Solution. Track progress toward the pattern 01: state s_0 = “no useful progress,” s_1 = “the last symbol was 0,” s_2 = “the last two symbols were 01.” From s_0 , a 0 starts a match ($\rightarrow s_1$) and a 1 does nothing ($\rightarrow s_0$). From s_1 , another 0 keeps us at “last symbol 0” ($\rightarrow s_1$) and a 1 completes 01 ($\rightarrow s_2$). From s_2 (we have just seen 01), a following 0 makes the new last symbol 0 ($\rightarrow s_1$) and a 1 breaks the suffix back to “no progress” ($\rightarrow s_0$). Accept at s_2 .



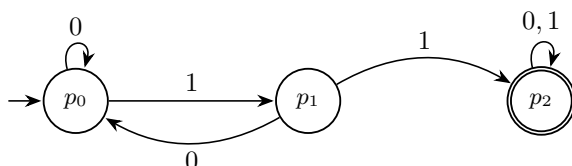
Which idea, and why? For an “ends in the fixed string u ” condition, the states record the longest suffix of the input seen so far that is a prefix of u ; here $u = 01$ has prefixes $\varepsilon, 0, 01$, giving the three states.

Check. Trace 1001: $s_0 \xrightarrow{1} s_0 \xrightarrow{0} s_1 \xrightarrow{0} s_1 \xrightarrow{1} s_2 \in F$, accepted. Trace 010: $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_1 \notin F$, rejected (it ends in 0).

Worked Example 4.1.7 (Contains the substring 11). Design a DFA over $\{0, 1\}$ recognizing $\mathcal{L} = \{w \mid w \text{ contains } 11 \text{ somewhere}\}$.

Recognition. This is a *substring* (factor) condition, not a suffix condition: once 11 appears *anywhere*, the string is accepted no matter what follows. The trigger “contains u somewhere” calls for states that track partial matches of u *plus a permanent “found-it” state that, once entered, is never left.*

Solution. Use p_0 = “the previous symbol was not 1 (no pending match),” p_1 = “the previous symbol was 1 (one 1 pending),” and p_2 = “11 already seen.” From p_0 : a 1 starts a pending match ($\rightarrow p_1$), a 0 stays ($\rightarrow p_0$). From p_1 : another 1 completes 11 ($\rightarrow p_2$), a 0 resets the pending match ($\rightarrow p_0$). State p_2 is a *trap*: both symbols loop back to p_2 , because acceptance can no longer be lost. Accept at p_2 .



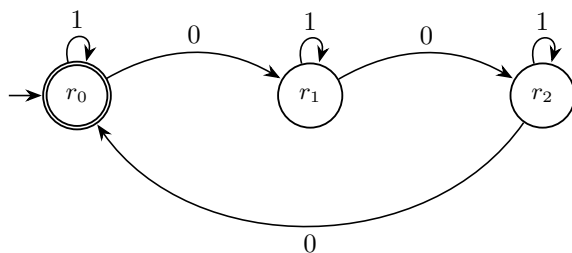
Which idea, and why? “Contains u ” differs from “ends in u ” by the trap: after u is found the answer is locked to accept, so the accept state absorbs every further symbol rather than tracking the suffix.

Check. Trace 0110: $p_0 \xrightarrow{0} p_0 \xrightarrow{1} p_1 \xrightarrow{1} p_2 \xrightarrow{0} p_2 \in F$, accepted. Trace 1010: $p_0 \rightarrow p_1 \rightarrow p_0 \rightarrow p_1 \rightarrow p_0 \notin F$, rejected (no two adjacent 1s).

Worked Example 4.1.8 (A count condition modulo k : number of 0s divisible by 3). Design a DFA over $\{0, 1\}$ recognizing $\mathcal{L} = \{w \mid \text{the number of 0s in } w \text{ is a multiple of } 3\}$.

Recognition. A *counting-mod- k* condition: only the count of 0s *modulo 3* matters, never the exact count. The trigger “some quantity, taken mod k ” means the states should be the k residue classes $0, 1, \dots, k-1$, and a relevant symbol advances the residue by one.

Solution. Use three states r_0, r_1, r_2 for “number of 0s so far $\equiv 0, 1, 2 \pmod{3}$.” Reading a 0 advances the residue ($r_0 \rightarrow r_1 \rightarrow r_2 \rightarrow r_0$); reading a 1 does not change the count, so it loops. Start at r_0 (zero 0s seen, and 0 is a multiple of 3); accept at r_0 . The empty string has zero 0s and is accepted, which is correct since 0 is a multiple of 3.



Which idea, and why? A “count of a is $\equiv 0 \pmod{k}$ ” condition is recognized by a k -state cycle of residues, with the relevant symbol stepping around the cycle and all other symbols looping in place.

Check. Trace 0100 (three 0s): $r_0 \xrightarrow{0} r_1 \xrightarrow{1} r_1 \xrightarrow{0} r_2 \xrightarrow{0} r_0 \in F$, accepted. Trace 00 (two 0s): $r_0 \rightarrow r_1 \rightarrow r_2 \notin F$, rejected.

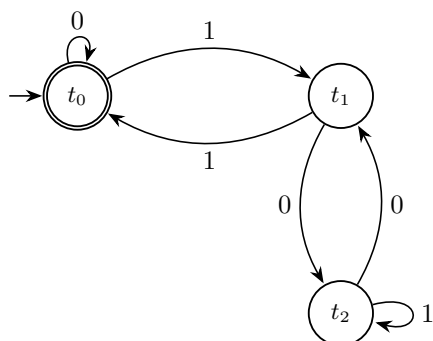
Worked Example 4.1.9 (Divisibility of a binary value by 3). Read a string $w \in \{0, 1\}^*$ as the binary numeral for an integer n (most significant bit first). Design a DFA recognizing $\mathcal{L} = \{w \mid n \equiv 0 \pmod{3}\}$.

Recognition. Another mod-3 condition, but the residue now changes by an *arithmetic* rule rather than by a simple count. The trigger is the schoolbook fact that appending a bit b to a binary numeral of value n produces value $2n + b$. So the states are residues mod 3, and each symbol updates the residue by $r \mapsto (2r + b) \bmod 3$.

Solution. Let the states be t_0, t_1, t_2 meaning “value read so far $\equiv 0, 1, 2 \pmod{3}$.” Using $r \mapsto (2r + b) \bmod 3$:

δ	0 ($b=0$)	1 ($b=1$)
t_0	t_0	t_1
t_1	t_2	t_0
t_2	t_1	t_2

(For example, from t_1 a 0 gives $2 \cdot 1 + 0 = 2 \equiv t_2$, and a 1 gives $2 \cdot 1 + 1 = 3 \equiv 0 \equiv t_0$.) Start at t_0 (the empty prefix has value 0) and accept at t_0 .



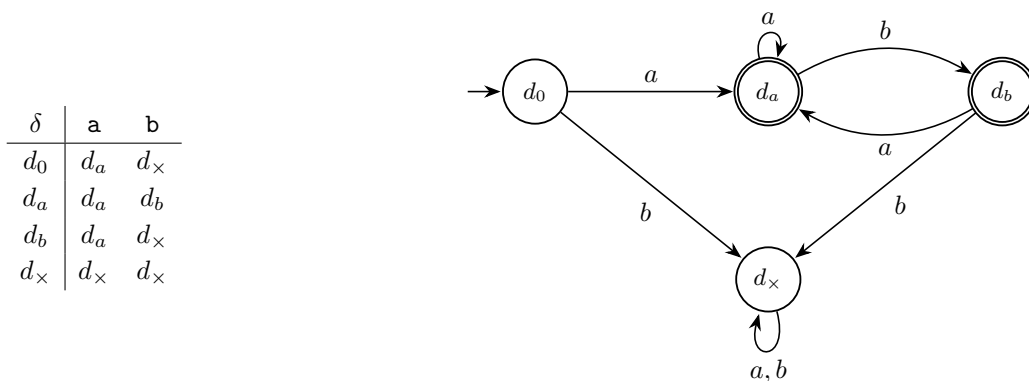
Which idea, and why? When each input symbol updates a numeric quantity by a fixed arithmetic rule and only the value mod k matters, take the k residues as states and read each transition straight off the update rule $r \mapsto (2r + b) \bmod k$.

Check. The string 110 is binary 6, a multiple of 3: $t_0 \xrightarrow{1} t_1 \xrightarrow{1} t_0 \xrightarrow{0} t_0 \in F$, accepted. The string 101 is binary 5: $t_0 \rightarrow t_1 \rightarrow t_2 \rightarrow t_2 \notin F$, rejected.

Worked Example 4.1.10 (Reading a transition table, and a machine that needs a dead state). Over $\Sigma = \{a, b\}$, let $\mathcal{L} = \{w \mid w \text{ starts with } \mathbf{a} \text{ and has no } \mathbf{bb}\}$. Give the transition table and diagram.

Recognition. The condition combines a *prefix* requirement (first symbol is **a**) with a *forbidden-substring* requirement (no **bb**). A forbidden pattern, once violated, can never be repaired – that is the trigger for a permanent *dead* (reject-trap) state.

Solution. State d_0 = start, nothing read; d_a = “valid so far, last symbol **a**”; d_b = “valid so far, last symbol **b**”; $d_\times$ = dead (either the first symbol was **b**, or a **bb** occurred). From d_0 : **a** $\rightarrow d_a$, but **b** $\rightarrow d_\times$ (bad start). From d_a : **a** $\rightarrow d_a$, **b** $\rightarrow d_b$. From d_b : **a** $\rightarrow d_a$, but **b** $\rightarrow d_\times$ (that is **bb**). The dead state loops on everything. Accept the two “valid so far” states d_a, d_b .



δ	a	b
d_0	d_a	$d_\times$
d_a	d_a	d_b
d_b	d_a	$d_\times$
$d_\times$	$d_\times$	$d_\times$

Which idea, and why? Because δ must be *total* (defined for every state-symbol pair), a condition that can be permanently violated forces an explicit non-accepting trap; deleting $d_\times$ and its edges would leave δ undefined and the object would not be a DFA.

Check. Trace **abab**: $d_0 \xrightarrow{a} d_a \xrightarrow{b} d_b \xrightarrow{a} d_a \xrightarrow{b} d_b \in F$, accepted. Trace **abb**: $d_0 \rightarrow d_a \rightarrow d_b \rightarrow d_\times \notin F$, rejected; and **b...** dies immediately.

Method (How to design a DFA from a description of its language).

1. **Name the finite memory.** Ask: “As I scan the input left to right, what is the least I must remember about the part already read to decide acceptance later?” That finite list of distinguishable situations becomes your set of states. Typical memories: the last symbol (or last few), how much of a target pattern is matched, a count taken modulo k , a running value modulo k , or a one-way “something good/bad has happened” flag.
2. **Pick the start state** – the situation *before any symbol is read* (usually “nothing seen yet”).
3. **Fill in δ totally.** For each state q and each symbol a , ask “if I am in situation q and now read a , what is the new situation?” Make sure *every* pair (q, a) gets exactly one target; add a dead state if some inputs make acceptance impossible forever.
4. **Choose F** – exactly the states whose “situation” means the string read so far should be accepted. Decide deliberately whether the empty string is in or out, since that is whether $q_0 \in F$.
5. **Test** on a few strings: at least one that should be accepted, one that should be rejected, and the empty string. Trace each through $\hat{\delta}$.

Caution. The most common error is an *incomplete* transition function. A DFA’s δ must give a next state for *every* state and *every* alphabet symbol; if your diagram has a state with no outgoing edge for some symbol, you have not drawn a DFA. The usual fix is a *dead state* (also called a trap or sink): a single non-accepting state that loops to itself on every symbol and collects all the “can never accept now” transitions. Forgetting it is easy when the language is phrased positively (“starts with **a**”), because the failing inputs (“starts with **b**”) have no natural home.

Caution. Do not confuse the *accept states* F (a chosen subset of Q) with the states the machine happens to pass through while reading a string. Acceptance is decided *only* by the *final* state $\hat{\delta}(q_0, w)$, not by whether the run visited an accept state somewhere in the middle. A run may pass through several accept states and still reject, if it ends outside F .

Exercises.

1. Let $M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$ with $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_2$, $\delta(q_1, 1) = q_1$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_2$. Draw the state diagram, then compute $\hat{\delta}(q_0, w)$ for $w = 1101$, $w = 0010$, and $w = \varepsilon$. Which of these does M accept?
2. In one English sentence each, describe the language recognized by the machine of the previous exercise and the language recognized by the same machine after you change the accept set to $\{q_0\}$.
3. Design a DFA over $\{0, 1\}$ recognizing the strings that *begin* with 10. Draw the state diagram and be sure δ is total (include a dead state if needed).
4. Design a DFA over $\{0, 1\}$ recognizing the strings whose length is even. Then design one recognizing the strings whose length is a multiple of 3. Draw both diagrams.
5. Design a DFA over $\{0, 1\}$ recognizing the strings that contain at least two 1s. Draw the diagram, and state in words what each state “remembers.”
6. Design a DFA over $\{a, b\}$ recognizing the strings that contain neither **aa** nor **bb** as a substring (i.e. no two equal symbols are ever adjacent). Draw the diagram and identify the dead state.
7. Read a string over $\{0, 1\}$ as a binary number, least significant bit *last* (ordinary most-significant-first order). Design a DFA recognizing the strings whose value is even, and separately one recognizing the strings whose value is $\equiv 2 \pmod{4}$. Draw both.

8. Design a DFA over $\{0, 1\}$ recognizing the strings that contain the substring 010. Draw the diagram, mark the trap-style accept state, and trace 00100 and 0110.
9. Let $\mathcal{L} = \{w \in \{0, 1\}^* \mid w \text{ has an even number of 0s and ends in 1}\}$. Design a DFA for $\mathcal{L}$ by combining the “even number of 0s” memory with the “last symbol” memory; how many states does your machine use, and why?
10. *Find the bug.* A student claims the following machine recognizes “strings over $\{0, 1\}$ ending in 0.” It has states u_0 (start, accepting) and u_1 , with $\delta(u_0, 0) = u_0$, $\delta(u_0, 1) = u_1$, $\delta(u_1, 0) = u_0$, and $\delta(u_1, 1) = u_1$. Draw the machine, find a string it classifies incorrectly, and fix the error by changing exactly one part of the 5-tuple.
11. *Find the bug.* Someone draws a three-state machine over $\{0, 1\}$ intended to accept exactly the strings with no 1 at all, but they leave state c_2 with no outgoing edge on 0. Explain precisely why the drawing fails to be a DFA, and give the smallest repair.
12. Prove that for any DFA M and any strings $x, y \in \Sigma^*$, $\widehat{\delta}(q, xy) = \widehat{\delta}(\widehat{\delta}(q, x), y)$ for every state q . (Use induction on the length of y and Definition 4.1.2.) Explain in one sentence why this justifies splitting a long trace at any point you like.

4.2 The Regular Operations and the Class of Regular Languages

A language is just a set of strings, so the ordinary set operations – union, intersection, complement – already apply to it. What makes the theory of regular languages move is a second family of operations built from the way strings are glued together: *concatenation* and *Kleene star*, together with union. The central promise of this chapter is that the class REGULAR of regular languages is *closed* under all of these operations; this section defines the operations, computes them on small examples, and gives the first closure proof – the product construction, which settles union and intersection in one stroke. The remaining regular operations need the machinery of Section 4.3, so we record them now as a goal.

The operations on languages

Definition 4.2.1 (The regular operations). Let $A, B \subseteq \Sigma^*$ be languages over a common alphabet Σ . Define

$$\begin{aligned} A \cup B &= \{w \mid w \in A \text{ or } w \in B\} && (\text{union}), \\ A \circ B &= AB = \{xy \mid x \in A \text{ and } y \in B\} && (\text{concatenation}), \\ A^* &= \{x_1x_2 \cdots x_k \mid k \geq 0 \text{ and each } x_i \in A\} && (\text{Kleene star}). \end{aligned}$$

The three operations $\cup$, $\circ$, * are the **regular operations**. Two convenient relatives are

$$A^+ = \{x_1x_2 \cdots x_k \mid k \geq 1, x_i \in A\} = A \cdot A^*, \quad A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\},$$

where $w^{\mathcal{R}}$ is the string w written backwards.

Two points of the definition deserve emphasis because they are the source of most beginner errors. First, in A^* the index k is allowed to be 0: the empty product of zero strings is the empty string ε , so $\varepsilon \in A^*$ *always*, even when $A = \emptyset$. Thus $\emptyset^* = \{\varepsilon\}$ and $\{\varepsilon\}^* = \{\varepsilon\}$. The plus version A^+ forbids $k = 0$, so $\varepsilon \in A^+$ iff $\varepsilon \in A$ already. Second, concatenation is *not* commutative: AB and BA are generally different sets, because $xy \neq yx$ as strings.

Worked Example 4.2.2 (Computing union, concatenation, and star by hand). Let $\Sigma = \{a, b\}$, $A = \{a, bb\}$, and $B = \{\varepsilon, b\}$. Compute $A \cup B$, AB , BA , and the first few strings of A^* .

Recognition. These are finite languages, so each operation is a finite set computation: list every combination demanded by Definition 4.2.1.

Solution. Union just pools the members: $A \cup B = \{\varepsilon, a, b, bb\}$. For AB glue each $x \in A$ in front of each $y \in B$:

$$AB = \{a \cdot \varepsilon, a \cdot b, bb \cdot \varepsilon, bb \cdot b\} = \{a, ab, bb, bbb\}.$$

The opposite order BA glues B in front of A :

$$BA = \{\varepsilon \cdot a, \varepsilon \cdot bb, b \cdot a, b \cdot bb\} = \{a, bb, ba, bbb\}.$$

These differ: $ab \in AB$ but $ab \notin BA$, while $ba \in BA$ but $ba \notin AB$. For A^* take products of members of A : the empty product gives ε ; length-one products give a, bb ; length-two products give $aa, abb, bba, bbbb$; and so on. Hence

$$A^* = \{\varepsilon, a, bb, aa, abb, bba, bbbb, \dots\}.$$

Which idea, and why? Concatenation is a Cartesian-style pairing of the two languages, so $|AB| \leq |A| \cdot |B|$ and order matters; star is the union of all finite self-concatenations, so it is infinite as soon as A contains a nonempty string.

Check. $|A| = |B| = 2$, so AB has at most 4 strings – and indeed exactly 4, since no two products collided. $\varepsilon \in A^*$ even though $\varepsilon \notin A$.

Worked Example 4.2.3 (Star of the empty set versus star of a singleton). Compute $\emptyset^*$, $\{\varepsilon\}^*$, $\emptyset^+$, and $\{0\}^*$.

Recognition. Apply the definition of $*$ and $+$ literally, paying attention to whether $k = 0$ is permitted.

Solution. $\emptyset^*$: the only product available is the empty product (there are no members of $\emptyset$ to use), so $\emptyset^* = \{\varepsilon\}$. Forbidding $k = 0$ removes even that: $\emptyset^+ = \emptyset$. For the singleton, $\{\varepsilon\}^*$ concatenates copies of ε , all of which are ε , so $\{\varepsilon\}^* = \{\varepsilon\}$. Finally $\{0\}^* = \{\varepsilon, 0, 00, 000, \dots\}$, the set of all strings of zeros including the empty one.

Which idea, and why? Star always contains ε via the empty product; it is the unique “free” string that every star produces regardless of the base language.

Check. $\emptyset^* \neq \emptyset$ – the single most common slip is to write $\emptyset^* = \emptyset$. The empty product saves it.

The goal: closure of REGULAR

Recall from Section 4.1 that a language is **regular** iff some DFA recognizes it, and **REGULAR** denotes the class of all regular languages. The organizing theorem of the chapter is the following; we state it now and discharge the pieces across this section and Section 4.5.

Theorem 4.2.4 (Closure of REGULAR under the regular operations). *If A and B are regular languages over Σ , then so are $A \cup B$, $A \cap B$, $\bar{A} = \Sigma^* \setminus A$, $A \circ B$, and A^* .*

Two of these – union and intersection – can be proved directly with DFAs by running two machines side by side, and that is the subject of the rest of this section. Complement is an easy DFA trick we record below. Concatenation and star resist a clean DFA argument (a DFA cannot “guess” where to split a string), so they wait for the nondeterministic machines of Section 4.3 and the constructions of Section 4.5.

Proposition 4.2.5 (Complement). *If A is regular then $\bar{A}$ is regular.*

Proof. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA with $L(M) = A$. Because M is deterministic and total, for every string w the run $\widehat{\delta}(q_0, w)$ lands in exactly one state, and $w \in A$ iff that state lies in F . Swap accepting and non-accepting roles: let $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$. Then M' accepts w iff M rejects w , so $L(M') = \bar{A}$. $\square$

Caution. Complementation by swapping accept states works *only* because a DFA is total and deterministic – every string has exactly one run, ending in exactly one state. The same swap on an *NFA* does *not* compute the complement, since a string may have both an accepting and a rejecting run; flipping the accept set then accepts strings that the original also accepted. Determinize first (Section 4.4), *then* swap.

The product construction

To recognize $A \cap B$ or $A \cup B$ we want a single DFA that simulates two given DFAs at once. The idea is to remember a *pair* of states, one for each machine, and update both coordinates on every input symbol.

Theorem 4.2.6 (Product / Cartesian construction). Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ be DFAs over the same alphabet Σ . Define the product DFA $M = (Q, \Sigma, \delta, s, F)$ by

$$Q = Q_1 \times Q_2, \quad s = (s_1, s_2), \quad \delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a)),$$

and choose the accept set according to the operation:

$$F_{\cap} = F_1 \times F_2 \quad (\text{for } A \cap B), \quad F_{\cup} = \{(p, q) \mid p \in F_1 \text{ or } q \in F_2\} \quad (\text{for } A \cup B).$$

With $A = L(M_1)$ and $B = L(M_2)$, the machine M satisfies $L(M) = A \cap B$ when $F = F_{\cap}$ and $L(M) = A \cup B$ when $F = F_{\cup}$. Consequently REGULAR is closed under union and intersection.

Proof. We first check that M tracks both runs simultaneously. Claim: for every $w \in \Sigma^*$,

$$\widehat{\delta}((s_1, s_2), w) = (\widehat{\delta}_1(s_1, w), \widehat{\delta}_2(s_2, w)).$$

Induct on $|w|$. For $w = \varepsilon$ both sides are (s_1, s_2) . If the claim holds for w and $a \in \Sigma$, then by the definition of $\widehat{\delta}$ and of δ ,

$$\widehat{\delta}(s, wa) = \delta(\widehat{\delta}(s, w), a) = \delta((\widehat{\delta}_1(s_1, w), \widehat{\delta}_2(s_2, w)), a) = (\widehat{\delta}_1(s_1, wa), \widehat{\delta}_2(s_2, wa)),$$

completing the induction. Now $w \in L(M)$ iff the pair $(\widehat{\delta}_1(s_1, w), \widehat{\delta}_2(s_2, w))$ lies in F . With $F = F_{\cap}$ this says $\widehat{\delta}_1(s_1, w) \in F_1$ and $\widehat{\delta}_2(s_2, w) \in F_2$, i.e. $w \in A$ and $w \in B$, i.e. $w \in A \cap B$. With $F = F_{\cup}$ it says $w \in A$ or $w \in B$, i.e. $w \in A \cup B$. Both target languages are recognized by a DFA, hence regular. $\square$

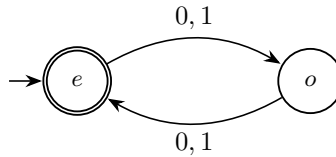
Method (Building the product DFA of M_1 and M_2).

1. Form the state set $Q_1 \times Q_2$. Write each state as a pair (p, q) ; there are $|Q_1| \cdot |Q_2|$ of them (some may turn out unreachable – drop those).
2. The start state is (s_1, s_2) .
3. On symbol a , send (p, q) to $(\delta_1(p, a), \delta_2(q, a))$: advance *both* coordinates by the same letter.
4. For *intersection* accept a pair iff *both* coordinates accept ($F_1 \times F_2$); for *union* accept iff *at least one* coordinate accepts.
5. Only the accept set differs between $\cap$ and $\cup$ – states and transitions are identical, so you can read both languages off the same diagram.

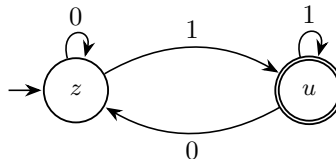
Worked Example 4.2.7 (Intersection: even length AND ending in 1). Let $A = \{w \mid w \text{ has even length}\}$ and $B = \{w \mid w \text{ ends in } 1\}$, both over $\Sigma = \{0, 1\}$. Build a DFA for $A \cap B$.

Recognition. Both A and B are individually easy two-state DFAs; their intersection is a job for the product construction with accept set $F_1 \times F_2$.

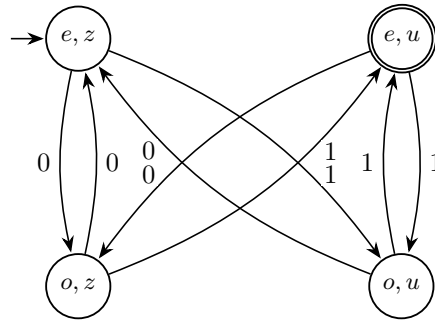
Solution. Machine M_1 for “even length” has states e (even, start, accept) and o (odd), toggling on every symbol:



Machine M_2 for “ends in 1” has states z (last symbol 0, or no symbol yet – start) and u (last symbol 1, accept):



The product has four states $(e, z), (e, u), (o, z), (o, u)$. Read δ off the rule “toggle the first coordinate; in the second, 1 goes to u and 0 goes to z .” For *intersection* the only accept state is (e, u) (even length AND ends in 1):



Which idea, and why? Intersection of two regular languages is the product DFA with $F_1 \times F_2$: a string is accepted exactly when both simulated machines accept, which is what the pair landing in $F_1 \times F_2$ records.

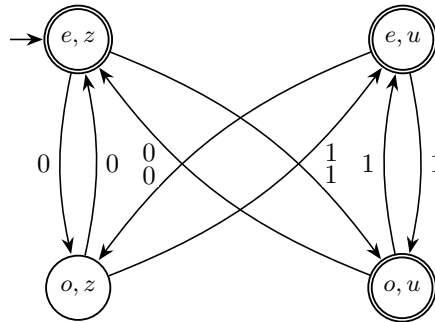
Check. Trace 01: start $(e, z) \xrightarrow{0} (o, z) \xrightarrow{1} (e, u)$, which is accepting – and 01 has even length and ends in 1. Trace 1: $(e, z) \xrightarrow{1} (o, u)$, not accepting – 1 ends in 1 but has odd length. Correct on both.

The strength of the construction is that the *same* state set and transitions also recognize the union – only the double circles move. We redraw the product once more with the union accept set to make the contrast concrete: every pair whose first or second coordinate accepts, i.e. every pair except (o, z) .

Worked Example 4.2.8 (Same product, union accept set: even length OR ends in 1). Using the two machines of the previous example, build a DFA for $A \cup B$.

Recognition. Same product states and transitions; switch to the union accept set $F_\cup$.

Solution. Accept a pair iff its first coordinate is e or its second is u . The first coordinate accepts at e ; the second accepts at u . So the accepting pairs are $(e, z), (e, u), (o, u)$ – all except (o, z) (odd length and not ending in 1):



Which idea, and why? Union of two regular languages is the *same* product DFA with accept set $F_\cup = \{(p, q) \mid p \in F_1 \text{ or } q \in F_2\}$; only the double circles move.

Check. The single non-accepting state is (o, z) . Trace 10: $(e, z) \xrightarrow{1} (o, u) \xrightarrow{0} (e, z)$, accepting – 10 has even length (in A), so it belongs to $A \cup B$. Trace 0: $(e, z) \xrightarrow{0} (o, z)$, rejecting – 0 has odd length and does not end in 1, so it is in neither A nor B . Correct.

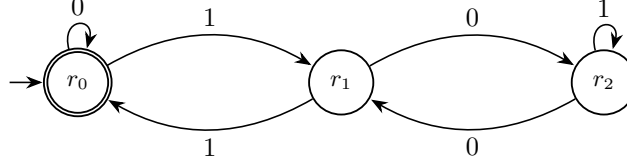
Worked Example 4.2.9 (Intersection where the product is genuinely needed: $\equiv 0 \pmod 3$ AND contains 11). Over $\Sigma = \{0, 1\}$ let

$$A = \{w \mid \text{the binary value of } w \text{ is divisible by } 3\} \quad \text{and} \quad B = \{w \mid w \text{ contains } 11 \text{ as a substring}\}.$$

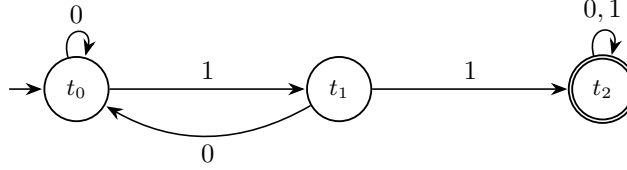
Sketch the product DFA for $A \cap B$.

Recognition. Neither language is a sub-case of the other, and there is no shortcut: build both DFAs and take the product with $F_1 \times F_2$.

Solution. M_1 tracks the running remainder mod 3 of the value read so far. Reading a bit b updates value $v \mapsto 2v + b$, hence remainder $r \mapsto (2r + b) \bmod 3$. States r_0, r_1, r_2 with start and accept r_0 :



M_2 looks for 11: state t_0 (no progress), t_1 (just saw a 1), t_2 (seen 11, accept and stay):



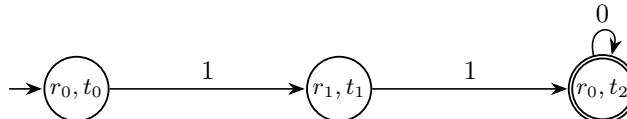
The product has $3 \times 3 = 9$ states (r_i, t_j) , all of them reachable from the start (r_0, t_0) . On input a the rule is $(r_i, t_j) \mapsto (\delta_1(r_i, a), \delta_2(t_j, a))$. For intersection the accept set is $F_1 \times F_2 = \{r_0\} \times \{t_2\} = \{(r_0, t_2)\}$: divisible by 3 *and* containing 11. Nine states is too many for an uncluttered diagram, so we read δ off the two component machines into a transition table (the start row is marked $\rightarrow$, the accept row *):

	state (r_i, t_j)	on 0	on 1
$\rightarrow$	(r_0, t_0)	(r_0, t_0)	(r_1, t_1)
	(r_0, t_1)	(r_0, t_0)	(r_1, t_2)
*	(r_0, t_2)	(r_0, t_2)	(r_1, t_2)
	(r_1, t_0)	(r_2, t_0)	(r_0, t_1)
	(r_1, t_1)	(r_2, t_0)	(r_0, t_2)
	(r_1, t_2)	(r_2, t_2)	(r_0, t_2)
	(r_2, t_0)	(r_1, t_0)	(r_2, t_1)
	(r_2, t_1)	(r_1, t_0)	(r_2, t_2)
	(r_2, t_2)	(r_1, t_2)	(r_2, t_2)

Each entry just pairs the next r -state (from M_1) with the next t -state (from M_2); the only accepting row is (r_0, t_2) . The portion of this machine that a run actually visits is small, so a trace is the practical way to use it. *Which idea, and why?* Two unrelated conditions with no containment force the product: the single accept pair (r_0, t_2) encodes “remainder 0 *and* the 11-search has succeeded.”

Check. Trace 110 (value 6, divisible by 3, contains 11): $(r_0, t_0) \xrightarrow{1} (r_1, t_1) \xrightarrow{1} (r_0, t_2) \xrightarrow{0} (r_0, t_2)$, ending accepting – correct. Trace 11 (value 3, contains 11): $(r_0, t_0) \xrightarrow{1} (r_1, t_1) \xrightarrow{1} (r_0, t_2)$, accepting – value 3 is divisible by 3, correct. Trace 1 (value 1): $(r_0, t_0) \xrightarrow{1} (r_1, t_1)$, not accepting – not divisible by 3 and no 11, correct.

The table makes the product look like nine disconnected rows, but the product is a single machine whose states are pairs (r_i, t_j) that step in lockstep. Drawing even the small corner reached by the run on 110 makes the lockstep visible:



Each arrow advances *both* coordinates by the same letter – the r -part by M_1 's rule and the t -part by M_2 's – and the only accepting pair drawn, (r_0, t_2) , is the one in $F_1 \times F_2$.

The same lockstep is the whole content of the product construction, and it is what makes the recognized language come out as a Boolean combination of $L(A)$ and $L(B)$. Writing A, B for the two input DFAs, the construction and its effect on languages commute:

$$\begin{array}{ccc}
 (A, B) & \xrightarrow{\text{product, state } (q,r)} & A \times B \\
 \downarrow (L(A), L(B)) & & \downarrow L(\cdot) \\
 (L(A), L(B)) & \xrightarrow{\cap / \cup} & L(A) \cap L(B) / L(A) \cup L(B)
 \end{array}$$

The product runs A and B in lockstep – state (q, r) tracks both – so with accept set $F_A \times F_B$ it recognizes $L(A) \cap L(B)$, and with accept set $(F_A \times Q_B) \cup (Q_A \times F_B)$ it recognizes $L(A) \cup L(B)$. *Check.* Correct because the product's run on w reaches (q, r) iff A reaches q and B reaches r on w (the lockstep invariant of Theorem 4.2.6); so it accepts exactly the strings in *both* languages when the accept set is $F_A \times F_B$, and the strings in *either* language when it is $(F_A \times Q_B) \cup (Q_A \times F_B)$.

Why concatenation and star resist a DFA argument

It is tempting to try a product-style trick for $A \circ B$ as well, but there is a real obstacle. To accept xy with $x \in A$ and $y \in B$, a machine must decide *where* to stop reading the A -part and start the B -part – and a DFA reading left to right gets no second look. Concretely, on the string **abab** a machine for $A \circ B$ might have to choose between the prefix **ab** and the prefix **a** as the A -part; a deterministic device cannot pursue both splits at once. The clean fix is to let the machine *guess* the split point and explore all guesses in parallel – exactly the power of a nondeterministic finite automaton, introduced in Section 4.3. We therefore postpone the proofs that REGULAR is closed under $\circ$ and $*$ (Theorem 4.2.4) to Section 4.5, where the ε -glue constructions make them short.

Caution. Concatenation order. $A \circ B$ puts an A -string *first*, then a B -string. Writing $A \circ B$ when you mean “a B -string followed by an A -string” inverts the language. When in doubt, name the parts: xy with $x \in A, y \in B$.

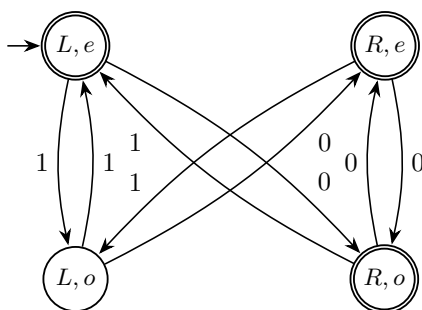
Caution. Star always admits ε . Because the empty product is allowed, $\varepsilon \in A^*$ for *every* A , including $A = \emptyset$. If you want “one or more,” use $A^+ = AA^*$, which contains ε only if A already does. A frequent error is to claim a star-language excludes the empty string; it never does on its own.

Caution. The product needs a common alphabet and total DFAs. The construction in Theorem 4.2.6 multiplies transition functions symbol by symbol, so both machines must read the *same* Σ and be total (a defined transition on every symbol from every state). If a source machine is missing transitions, complete it with a dead state first (Section 4.1), or the product transition δ is undefined on some pair.

Exercises.

1. Let $\Sigma = \{a, b\}$, $A = \{a, ab\}$, and $B = \{b, ba\}$. Write out, as explicit finite sets, each of: $A \cup B$, $A \circ B$, $B \circ A$, and $A \cap B$. Confirm in writing whether $A \circ B = B \circ A$.
2. For each language below, decide whether ε is a member, and justify in one line: (a) $\emptyset^*$; (b) $\emptyset^+$; (c) $\{ab\}^*$; (d) $\{\varepsilon, a\}^+$; (e) $(\{0\} \cup \emptyset)^*$.
3. Let $A = \{0\}$ and $B = \{1\}$ over $\Sigma = \{0, 1\}$. Describe in words the languages $(A \cup B)^*$ and $A^* \circ B^*$, then give a single string that belongs to the first but not the second. (This shows the two are different.)
4. Design a DFA over $\Sigma = \{0, 1\}$ for the language of all strings that have *odd* length *and* contain at least one 0. Build it as a product of a length-parity DFA and a “contains a 0” DFA; draw both component machines and the product, and mark the accept states for the intersection.

5. Using the same two component machines as in the previous exercise, draw the product DFA for the *union* (odd length *or* contains a 0). State exactly which product states are non-accepting, and explain why in one sentence.
6. Let A be the set of binary strings whose value is $\equiv 1 \pmod{2}$ (i.e. ending in 1) and B the set of binary strings of length divisible by 3. Give the state sets Q_1 and Q_2 of natural DFAs for A and B , state how many states the product DFA has, and identify the start state and the accept set $F_\cap$ for $A \cap B$ as a set of pairs.
7. Prove that REGULAR is closed under *set difference*: if A and B are regular, then $A \setminus B = \{w \mid w \in A, w \notin B\}$ is regular. (Hint: $A \setminus B = A \cap \overline{B}$; combine two closure results already in this section.)
8. Prove the *symmetric difference* $A \triangle B = (A \setminus B) \cup (B \setminus A)$ of two regular languages is regular. Then describe the accept set of a single product DFA that recognizes $A \triangle B$ directly (no detour through complement), in terms of F_1 and F_2 .
9. Decide, with a one-line proof or counterexample, whether each statement holds for *all* languages $A, B \subseteq \Sigma^*$: (a) $(A \cup B)^* = A^* \cup B^*$; (b) $(A \circ B)^* = A^* \circ B^*$; (c) $A^* \circ A^* = A^*$; (d) $(A \circ B)^\mathcal{R} = B^\mathcal{R} \circ A^\mathcal{R}$.
10. *Find the bug.* A student claims the following DFA recognizes $A \cap B$, where $A = \{w \mid w \text{ ends in } 0\}$ and $B = \{w \mid w \text{ has even length}\}$, over $\Sigma = \{0, 1\}$. The student took the product of a correct 2-state DFA for A and a correct 2-state DFA for B but used the accept set for *union* instead of intersection:



Here the first coordinate is L (last symbol not 0, or empty) versus R (last symbol 0, the accepting half of A), and the second is e/o for even/odd length. Identify every accept state that should not be accepting for $A \cap B$, give the corrected accept set, and exhibit a string this faulty machine accepts but that is not in $A \cap B$.

11. *Find the bug.* Someone proposes to recognize $A \cup B$ by running a single DFA that is the product of M_1 (for A) and M_2 (for B) but with accept set $F_1 \times F_2$. Show by a small concrete example (give A , B , and one string) that this machine does *not* recognize $A \cup B$, and name which language it actually recognizes.
12. Let A be regular over $\Sigma = \{a, b\}$. Prove that the language $\text{prefix}(A) = \{x \mid xy \in A \text{ for some } y \in \Sigma^*\}$ of all prefixes of strings in A is regular. (Hint: start from a DFA for A ; make a state accepting iff some string drives it into an original accept state. You may describe the new accept set without drawing the machine.)

4.3 Nondeterministic Finite Automata

A deterministic machine has exactly one move available at every step, so reading a string traces out a single path through its states. We now relax that requirement. A *nondeterministic* finite automaton may have several arrows for one symbol, may have none, and may even change state on the empty string. The payoff is design economy: an NFA can often capture a pattern with a handful of states where the corresponding DFA needs many more, because nondeterminism lets the machine “guess” which possibility will work out and accept as long as *some* guess succeeds. This section defines NFAs, gives a precise meaning to acceptance via the set of states a machine can occupy, and works several small examples; the next section proves that this added freedom buys no new languages.

The formal definition

Definition 4.3.1. A *nondeterministic finite automaton* (NFA) is a five-tuple $N = (Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of *states*, Σ is a finite *alphabet*, $q_0 \in Q$ is the *start state*, $F \subseteq Q$ is the set of *accept states*, and

$$\delta: Q \times (\Sigma \cup \{\varepsilon\}) \longrightarrow \mathcal{P}(Q)$$

is the *transition function*. For a state q and a symbol $a \in \Sigma \cup \{\varepsilon\}$, the value $\delta(q, a)$ is the *set* of states reachable from q by reading a .

Two features distinguish this from the DFA definition. First, the codomain is the power set $\mathcal{P}(Q)$ rather than Q : a single input symbol may lead to zero, one, or many successor states. When $\delta(q, a) = \emptyset$ the computation along that branch simply dies. Second, the domain includes the empty string ε , so $\delta(q, \varepsilon)$ lists the states reachable from q *without consuming any input*. Every DFA is the special case in which every set $\delta(q, a)$ is a singleton and $\delta(q, \varepsilon) = \emptyset$ for all q .

Definition 4.3.2. For a set $S \subseteq Q$, the ε -closure $E(S)$ is the set of all states reachable from some state of S by following zero or more ε -arrows. Formally, $E(S)$ is the smallest set with $S \subseteq E(S)$ and $\delta(q, \varepsilon) \subseteq E(S)$ whenever $q \in E(S)$.

The ε -closure is what lets us treat an NFA's ε -arrows as “free” moves: before and after reading each real symbol, the machine may silently slide along any chain of ε -arrows.

What it means to accept

Running an NFA on a string is best pictured as exploring *all* branches at once. At every moment the machine occupies a *set* of states – every state some branch could currently be in. We make this precise with an extended transition function on sets.

Definition 4.3.3. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. Define $\hat{\delta}: \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$ by

$$\hat{\delta}(S, \varepsilon) = E(S), \quad \hat{\delta}(S, wa) = E\left(\bigcup_{q \in \hat{\delta}(S, w)} \delta(q, a)\right)$$

for $w \in \Sigma^*$ and $a \in \Sigma$. The NFA *accepts* w if $\hat{\delta}(E(\{q_0\}), w) \cap F \neq \emptyset$, i.e. if at least one branch ends in an accept state. The *language* $L(N)$ is the set of strings N accepts.

In words: start from the ε -closure of the start state; to read a symbol a , fire a from every currently active state, collect all the targets, then take the ε -closure of that collection. Accept if the final set of active states contains any accept state. There is no requirement that every branch accept; one surviving accepting branch is enough.

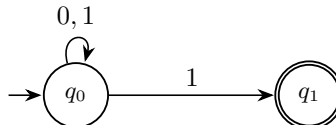
Remark. The same string may be accepted by one branch and rejected by another, and that is fine – acceptance is existential. It is also fine for a branch to die (reach $\emptyset$) partway through; it just stops contributing. The string is rejected only when *no* branch is left in an accept state after the whole input is read.

Reading an NFA by tracking active states

Worked Example 4.3.4 (A two-state NFA for “ends in 1” over $\{0, 1\}$). Build a small NFA for $\mathcal{L} = \{w \mid w \text{ ends in } 1\}$ and trace it on 011.

Recognition. The pattern fixes only the *last* symbol, so a guess-the-end strategy fits: loop on everything, then guess that the current 1 is the final one.

Solution. Take states $\{q_0, q_1\}$ with q_0 the start and q_1 accepting. Let q_0 loop on 0 and 1 (stay put while the string runs), and add a 1-arrow $q_0 \rightarrow q_1$. State q_1 has no outgoing arrows, so a branch that lands there must immediately reach the end of input to survive in an accept state.



Trace on 011. Active set starts at $\{q_0\}$. Read 0: from q_0 only q_0 , so $\{q_0\}$. Read 1: q_0 goes to q_0 (loop) and to q_1 , so $\{q_0, q_1\}$. Read 1: q_0 goes to $\{q_0, q_1\}$ and q_1 goes nowhere, so $\{q_0, q_1\}$. The final set $\{q_0, q_1\}$ meets $F = \{q_1\}$, so 011 is accepted.

Which idea, and why? Guess-the-suffix: nondeterminism lets the loop keep one branch alive while another branch commits to “this 1 is the last symbol,” so only two states are needed.

Check. On 010 the run is $\{q_0\} \rightarrow \{q_0\} \rightarrow \{q_0, q_1\} \rightarrow \{q_0\}$; the final set misses q_1 , so 010 is rejected, as it should be.

Method (Reading an NFA: track the set of active states). To decide whether N accepts $w = a_1 a_2 \cdots a_n$ without drawing the whole computation tree:

1. Start with $S_0 = E(\{q_0\})$ – the start state together with everything reachable by ε -arrows.
2. For $i = 1, \dots, n$: form $S_i = E(\bigcup_{q \in S_{i-1}} \delta(q, a_i))$. Fire a_i from each active state, union the targets, then add the ε -closure.
3. If any S_i becomes $\emptyset$, every branch has died; the input is rejected.
4. Accept iff $S_n \cap F \neq \emptyset$.

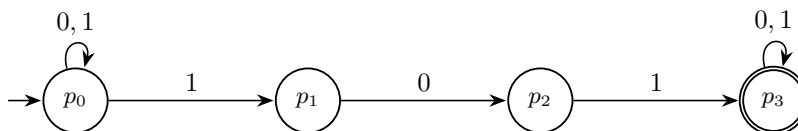
This is exactly the bookkeeping the subset construction of the next section automates.

Nondeterminism as economy: a substring guess

Worked Example 4.3.5 (Strings over $\{0, 1\}$ containing 101). Design an NFA recognizing $\mathcal{L} = \{w \mid 101 \text{ is a substring of } w\}$.

Recognition. We must spot a fixed block 101 somewhere inside w . Rather than remember how much of the block has matched at every position (a DFA bookkeeping task), guess *when* the block begins and march straight through it.

Solution. Use four states. The start state p_0 loops on 0, 1, idling until the machine guesses that 101 starts here. A 1 then moves to p_1 , a 0 to p_2 , a 1 to the accept state p_3 , which loops on 0, 1 to absorb the rest of the string.



A branch that guesses the start position wrongly simply dies when the next symbol does not match (e.g. at p_1 a 1 has no arrow), while the looping branch at p_0 stays alive to try again at the next position.

Which idea, and why? Substring search by guessing the start: an NFA need not track partial matches deterministically because a wrong guess kills only its own branch, leaving the idling branch free to retry.

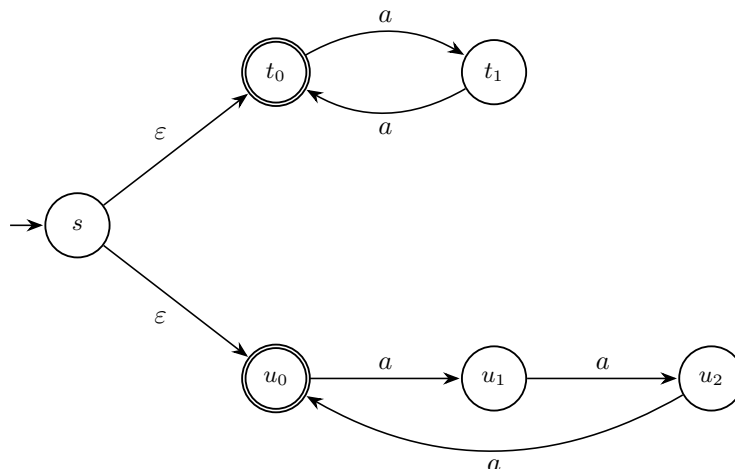
Check. On 0101 one branch idles through 0, then matches 1, 0, 1 to reach p_3 ; the string is accepted. On 1001 no position spells 101, so no branch reaches p_3 (the final active set is $\{p_0, p_1\}$) – rejected.

Using ε -arrows: a union of two patterns

Worked Example 4.3.6 (Multiples of two OR multiples of three (unary)). Over the one-symbol alphabet $\{a\}$, let $\mathcal{L} = \{a^n \mid n \text{ is a multiple of 2 or of 3}\}$. Build an NFA with an ε -arrow that guesses which clause to satisfy.

Recognition. The language is a *union* of two simple periodic languages. Nondeterminism handles union directly: from a fresh start, take an ε -arrow into whichever counter the input will satisfy.

Solution. Make one start state s . Add an ε -arrow from s into a 2-cycle ($t_0 \rightarrow t_1 \rightarrow t_0$, accept t_0) and another ε -arrow from s into a 3-cycle ($u_0 \rightarrow u_1 \rightarrow u_2 \rightarrow u_0$, accept u_0). The empty string is in $\mathcal{L}$ (it has length 0, a multiple of 2), and indeed $E(\{s\}) = \{s, t_0, u_0\}$ already contains accept states.



Which idea, and why? ε -arrows implement union: a single ε -branch sends the input into the 2-counter and another into the 3-counter, and acceptance by either branch suffices.

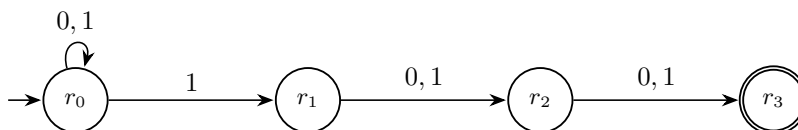
Check. Read **aaaa** ($n = 4$). The bottom (3-cycle) branch lands on u_1 , not accepting; the top (2-cycle) branch returns to t_0 , accepting. Some branch accepts, so **aaaa** $\in \mathcal{L}$. For $n = 5$ the top branch sits on t_1 and the bottom on u_2 ; no branch accepts, so **aaaaa** $\notin \mathcal{L}$ – correct, since 5 is a multiple of neither.

When a fixed-length tail forces a small guess

Worked Example 4.3.7 (The k th symbol from the end is 1). For a fixed $k = 3$, recognize $\mathcal{L} = \{w \in \{0, 1\}^* \mid |w| \geq 3 \text{ and the 3rd symbol from the end is } 1\}$.

Recognition. “Counts from the end” is the signature of guess-the-end nondeterminism: idle, then guess that exactly 3 symbols remain and verify the window.

Solution. Loop at the start state r_0 on 0, 1. On a 1, guess that this is the 3rd-from-last symbol and step into a chain $r_1 \rightarrow r_2 \rightarrow r_3$ that consumes exactly two more symbols (any of 0, 1), landing on the accept state r_3 precisely at end of input.



Only a guess made exactly three symbols before the end keeps a branch alive at r_3 when the input ends; every other guess either dies early or stalls before reaching r_3 .

Which idea, and why? Guess-from-the-end: nondeterminism replaces an exponential-in- k memory of the last k symbols with a $k+1$ -state chain entered on a guess.

Check. On 0100 the branch that guesses at the lone 1 walks r_1, r_2, r_3 over 1, 0, 0 and ends on r_3 : accepted (the 3rd-from-last symbol is indeed 1). On 0010 the matching window puts a 0 third from the end and no surviving branch reaches r_3 : rejected.

The computation tree

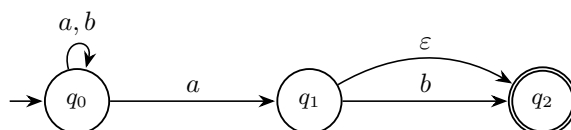
Acceptance is existential over branches, and the branches form a tree: the root is the start configuration, and a node for “in state q , input left x ” has one child per state in $\delta(q, a)$ (reading the first symbol a of x) plus children for each ε -move. The string is accepted iff some leaf reaching end-of-input sits in an accept state. The next picture is the tree for the “ends in 1” machine of Worked Example 4.3.4 on input 011 – read top to bottom, each level consuming one symbol.

Caution. Acceptance is existential, not universal. The NFA accepts when *some* branch ends in an accept state. A common error is to reject a string because one branch died or one branch ended in a non-accept state. As long as a single surviving branch is accepting, the string is in the language.

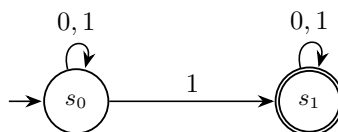
Caution. Do not forget the ε -closure. When tracking active states, you must apply $E(\cdot)$ both to the initial set *and* after every symbol. Skipping the closure after a symbol drops exactly the states reachable by free moves and can make you reject an accepted string.

Exercises.

1. Draw an NFA over $\{0, 1\}$, using at most three states, that recognizes $\{w \mid w \text{ ends in } 00\}$. Then trace the active-state set on 1000 and on 1001, and report which is accepted.
2. Give an NFA with an ε -arrow recognizing the union $\{a^n \mid n \text{ is even}\} \cup \{a^n \mid n \text{ is a multiple of } 3\}$ over $\{a\}$. State the ε -closure of the start state and explain why ε is accepted.
3. Design an NFA over $\{0, 1\}$ for the language of strings that contain 01 as a substring *or* contain 10 as a substring. Use a single start state that loops on the alphabet and two guessed branches.
4. Over $\{0, 1\}$, build an NFA with at most four states recognizing $\{w \mid \text{the 2nd symbol from the end is } 0\}$. Draw the state diagram and trace it on 110.
5. For the NFA below over $\{a, b\}$, list the active-state set after reading each prefix of **aba**, and state whether **aba** is accepted.



6. Write the formal five-tuple $(Q, \Sigma, \delta, q_0, F)$ for the “contains 101” NFA from this section, giving δ as an explicit table over $Q \times (\Sigma \cup \{\varepsilon\})$ with power-set values.
7. *Find the bug.* A student claims the NFA below recognizes $\{w \in \{0, 1\}^* \mid w \text{ ends in } 1\}$. Exhibit a string the machine accepts that does *not* end in 1 (or one it rejects that does), and describe the smallest fix.



8. Draw the computation tree of the NFA from Exercise 5 on input **ab**. Mark each leaf as accepting, rejecting, or dead, and use the tree to confirm your acceptance decision.
9. Over $\{a, b\}$, give an NFA recognizing $\{w \mid w \text{ contains at least two } a\text{'s}\}$ using exactly three states. Then argue informally why no NFA with fewer than three states can recognize it.
10. Let N have start state p and ε -arrows $p \rightarrow q$, $q \rightarrow r$, with r accepting and no other arrows. Compute $E(\{p\})$ and state $L(N)$ over $\{a, b\}$.
11. Design an NFA over $\{0, 1\}$ for $\{w \mid |w| \text{ is even}\} \cup \{w \mid w \text{ contains } 11\}$. Identify which clause each branch of your machine verifies, and trace 011.
12. (Reversal by reflection.) Take the five-state “ends in **aa** or **bb**” NFA of this section. Reverse every arrow, swap start and accept roles by adding a new start state with ε -arrows to the old accept states and making the old start state accepting. Draw the result and describe, in one sentence, the language it now recognizes.

4.4 Equivalence of NFAs and DFAs: the Subset Construction

Nondeterminism looks like extra power: an NFA may follow several branches at once and jump on ε without reading input. The surprise of this section is that the power is an illusion. Every language an NFA recognizes is already recognized by some DFA, so the two models accept exactly the same class of languages. The bridge is the *subset construction*: run the deterministic machine on the *set* of states the NFA could currently occupy, treating that set as a single state. The set of active states is itself a finite object (there are only finitely many subsets), so it can serve as the memory of a DFA.

The theorem and its construction

Definition 4.4.1. Two automata are *equivalent* when they recognize the same language. We compare any mix of DFAs and NFAs by this single criterion: M_1 and M_2 are equivalent iff $L(M_1) = L(M_2)$.

Recall from the previous section that for an NFA $N = (Q, \Sigma, \delta, q_0, F)$ the transition function has type $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$, and that the ε -closure of a set $R \subseteq Q$ is

$$E(R) = \{q \mid q \text{ is reachable from some state of } R \text{ by following zero or more } \varepsilon\text{-arrows}\}.$$

The closure always contains R itself (zero ε -steps), and it is the key to making the conversion handle the ε -transitions correctly.

Theorem 4.4.2 (Subset construction). *For every NFA N there is a DFA M with $L(M) = L(N)$. Hence nondeterminism adds no recognizing power to finite automata.*

Proof. Let $N = (Q, \Sigma, \delta, q_0, F)$. Define a DFA $M = (Q', \Sigma, \delta', q'_0, F')$ whose states are the subsets of Q :

$$Q' = \mathcal{P}(Q), \quad q'_0 = E(\{q_0\}), \quad F' = \{R \subseteq Q \mid R \cap F \neq \emptyset\}.$$

A subset R is accepting exactly when it contains at least one of N 's accept states. The transition function reads a symbol from every state in R , unions the results, and then takes the ε -closure:

$$\delta'(R, a) = E\left(\bigcup_{q \in R} \delta(q, a)\right) \quad \text{for } a \in \Sigma.$$

Note δ' consumes only *real* symbols; the ε -arrows are folded silently into $E(\cdot)$. The start state already absorbs the ε -moves available before any input is read.

Correctness. By induction on $|w|$ we show that after M reads w , the subset it occupies is precisely the set of states N could be in after reading w , i.e. $\widehat{\delta}'(q'_0, w) = E(\{q \mid N \text{ can reach } q \text{ on } w\})$. The base case $w = \varepsilon$ holds because $q'_0 = E(\{q_0\})$ is by definition the set of states reachable before any symbol. For the step, suppose the claim holds for w and read one more symbol a . The states N can occupy after wa are exactly those reached by taking an a -arrow from a current state and then any ε -arrows – which is precisely $E(\bigcup_{q \in R} \delta(q, a)) = \delta'(R, a)$ for the current subset R . So the invariant is preserved. Finally M accepts w iff its final subset meets F' , iff N could end in an accept state, iff N accepts w . Thus $L(M) = L(N)$. $\square$

Corollary 4.4.3. *A language is regular if and only if some NFA recognizes it.*

Proof. If a DFA recognizes $\mathcal{L}$ then $\mathcal{L}$ is regular by definition; and a DFA *is* an NFA (one with no ε -arrows and exactly one target per symbol), so every regular language is recognized by an NFA. Conversely, if an NFA recognizes $\mathcal{L}$, Theorem 4.4.2 produces an equivalent DFA, so $\mathcal{L}$ is regular. $\square$

The corollary is the practical payoff: from now on we may design the *small, intuitive* NFA for a language and rest assured a DFA exists. We never have to draw the (possibly huge) DFA unless we want to run it.

Reachable subsets only

The construction lists all $2^{|Q|}$ subsets, but often most are unreachable from the start subset and may be dropped. In practice we build the DFA *lazily*: begin with $q'_0 = E(\{q_0\})$, and from each subset already discovered compute $\delta'(R, a)$ for every symbol a , adding any new subset to a worklist. We stop when no new subset appears. The reachable part is usually far smaller than $2^{|Q|}$, and the empty set $\emptyset$ – reached when N has died on every branch – is the dead state of the DFA.

Method (Subset construction, run by hand (the worklist algorithm)).

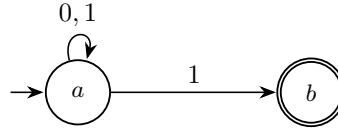
1. Compute the start subset $q'_0 = E(\{q_0\})$ (start state plus everything reachable by ε -arrows). Put it on the worklist.
2. Take an unprocessed subset R from the worklist. For each input symbol $a \in \Sigma$:
 - (a) form $S = \bigcup_{q \in R} \delta(q, a)$ – the a -targets from every state in R ;
 - (b) take $\delta'(R, a) = E(S)$ – add states reachable from S by ε -arrows.
3. If $\delta'(R, a)$ is a subset you have not seen, add it to the worklist (this includes $\emptyset$, the dead state).
4. Repeat until the worklist is empty.
5. Mark a subset accepting iff it contains at least one accept state of N . The start subset is q'_0 .

Worked conversions

Worked Example 4.4.4 (Convert a two-state NFA with one nondeterministic branch). Convert to a DFA the NFA N_1 over $\Sigma = \{0, 1\}$ with states $\{a, b\}$, start a , accept $\{b\}$, and arrows $\delta(a, 0) = \{a\}$, $\delta(a, 1) = \{a, b\}$, $\delta(b, 0) = \delta(b, 1) = \emptyset$. (Intuitively N_1 “guesses” that the current 1 is the last symbol.) There are no ε -arrows.

Recognition. The transition function returns a *set* ($\delta(a, 1) = \{a, b\}$ has two targets), so the machine is nondeterministic and we apply the subset construction. With no ε -arrows, $E(R) = R$, which simplifies every step.

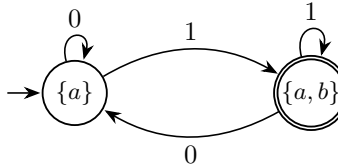
Solution. Here is N_1 :



Start subset: $q'_0 = E(\{a\}) = \{a\}$. Run the worklist.

R	$\delta'(R, 0)$	$\delta'(R, 1)$
$\{a\}$	$\{a\}$	$\{a, b\}$
$\{a, b\}$	$\{a\}$	$\{a, b\}$

From $\{a\}$ on 1 we get $\delta(a, 1) = \{a, b\}$, a new subset; from $\{a, b\}$ on 0 we get $\delta(a, 0) \cup \delta(b, 0) = \{a\} \cup \emptyset = \{a\}$. No new subsets appear, so the reachable DFA has exactly two states. The subset $\{a, b\}$ contains the accept state b , so it is accepting.



Which idea, and why? The subset construction: each DFA state records exactly which NFA states are simultaneously alive, so the deterministic machine simulates all of N_1 's guesses in parallel.

Check. N_1 accepts strings that end in 1. Trace 101 in the DFA: $\{a\} \xrightarrow{1} \{a, b\} \xrightarrow{0} \{a\} \xrightarrow{1} \{a, b\}$, accepting. Trace 10: $\{a\} \xrightarrow{1} \{a, b\} \xrightarrow{0} \{a\}$, rejecting. Correct.

The worked example above shows the construction running on an individual machine; the structural point is that it changes the machine *model* – from an NFA over state set Q to a DFA over $\mathcal{P}(Q)$ – but leaves the recognized language untouched. That preservation is exactly the commuting of the following square.

$$\begin{array}{ccc}
 N & \xrightarrow{\text{subset construction}} & M = \text{subset}(N) \\
 \downarrow L(\cdot) & & \downarrow L(\cdot) \\
 L(N) & \xlongequal{\hspace{1cm}} & L(M)
 \end{array}$$

The subset construction changes the state space from Q to $\mathcal{P}(Q)$, but the recognized language is unchanged: $L(N) = L(M)$. *Check.* The square commutes because, by the correctness invariant proved in Theorem 4.4.2, the subset DFA after reading w sits in the state $E(\{q \mid N \text{ can reach } q \text{ on } w\})$ – the ε -closure of the reachable set – and this subset meets F iff N accepts w . Hence M accepts exactly the strings N accepts, i.e. $L(M) = L(N)$: the two downward $L(\cdot)$ legs land on the same language.

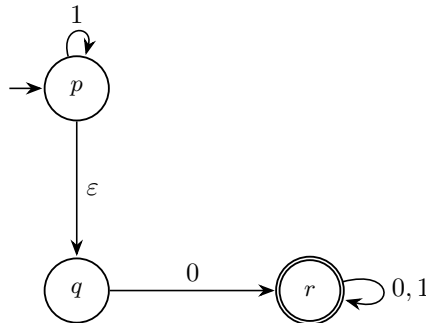
Worked Example 4.4.5 (An NFA with an ε -arrow: closures matter). Convert N_2 over $\Sigma = \{0, 1\}$ with states $\{p, q, r\}$, start p , accept $\{r\}$, and

$$\delta(p, \varepsilon) = \{q\}, \quad \delta(p, 1) = \{p\}, \quad \delta(q, 0) = \{r\}, \quad \delta(r, 0) = \{r\}, \quad \delta(r, 1) = \{r\},$$

all other transitions empty. (It recognizes strings of the form $1^*0\Sigma^*$: any number of 1s, then a 0, then anything.)

Recognition. There is an ε -arrow $p \rightarrow q$, so the start subset and every target must be passed through the ε -closure $E(\cdot)$. This is the case the closure was invented for.

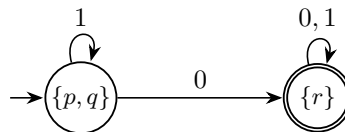
Solution. Here is N_2 :



Start subset: $q'_0 = E(\{p\}) = \{p, q\}$ (the ε -arrow pulls q in). Run the worklist; remember to close after each symbol.

R	$\delta'(R, 0)$	$\delta'(R, 1)$
$\{p, q\}$	$\{r\}$	$\{p, q\}$
$\{r\}$	$\{r\}$	$\{r\}$

From $\{p, q\}$ on 0: $\delta(p, 0) \cup \delta(q, 0) = \emptyset \cup \{r\} = \{r\}$, then $E(\{r\}) = \{r\}$. From $\{p, q\}$ on 1: $\delta(p, 1) \cup \delta(q, 1) = \{p\} \cup \emptyset = \{p\}$, then $E(\{p\}) = \{p, q\}$ – the closure re-adds q . The subset $\{r\}$ is accepting (contains r); $\{p, q\}$ is not.



Which idea, and why? The ε -closure is taken *both* for the start subset and after every symbol read, so the DFA never forgets the silent ε -jump from p to q .

Check. On 110: $\{p, q\} \xrightarrow{1} \{p, q\} \xrightarrow{1} \{p, q\} \xrightarrow{0} \{r\}$, accept. On 11: stays at $\{p, q\}$, reject – correct, since 1^* with no 0 is not in the language.

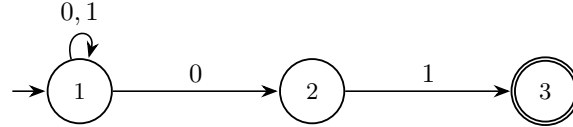
Worked Example 4.4.6 (A guessing NFA for “ends in 01”). Convert N_3 over $\Sigma = \{0, 1\}$, states $\{1, 2, 3\}$, start 1, accept $\{3\}$, with

$$\delta(1, 0) = \{1, 2\}, \delta(1, 1) = \{1\}, \delta(2, 1) = \{3\}, \delta(2, 0) = \emptyset, \delta(3, 0) = \delta(3, 1) = \emptyset.$$

(State 1 loops on every symbol; on a 0 it *also* guesses “this is the start of the final 01” and forks to 2; from 2 a 1 confirms the guess at 3.) No ε -arrows.

Recognition. $\delta(1, 0) = \{1, 2\}$ branches, so this is an NFA; the deterministic machine must track whether a “pending” 0 has just been seen, which is what the subsets will encode.

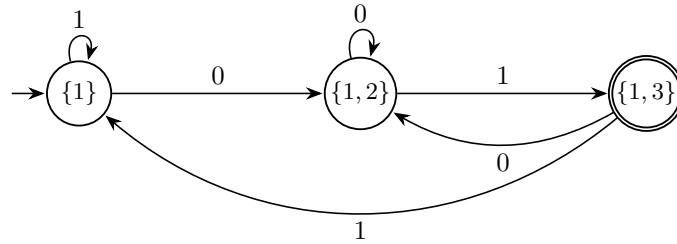
Solution. Here is N_3 :



Worklist (no closures needed):

R	$\delta'(R, 0)$	$\delta'(R, 1)$
$\{1\}$	$\{1, 2\}$	$\{1\}$
$\{1, 2\}$	$\{1, 2\}$	$\{1, 3\}$
$\{1, 3\}$	$\{1, 2\}$	$\{1\}$

From $\{1, 2\}$ on 1: $\delta(1, 1) \cup \delta(2, 1) = \{1\} \cup \{3\} = \{1, 3\}$. From $\{1, 3\}$ on 0: $\delta(1, 0) \cup \delta(3, 0) = \{1, 2\} \cup \emptyset = \{1, 2\}$. Three reachable subsets; the only accepting one is $\{1, 3\}$.



Which idea, and why? Each reachable subset records the relevant suffix state – “2 alive” means a 0 is pending, “3 alive” means 01 just completed – so the DFA needs one state per distinct “how close are we to ending in 01” situation.

Check. The DFA accepts iff its final subset contains 3, i.e. iff the last two symbols were 01. Trace 1001 (ends in 01): $\{1\} \xrightarrow{1} \{1\} \xrightarrow{0} \{1, 2\} \xrightarrow{0} \{1, 2\} \xrightarrow{1} \{1, 3\}$, accept. Trace 010 (ends in 10): $\{1\} \xrightarrow{0} \{1, 2\} \xrightarrow{1} \{1, 3\} \xrightarrow{0} \{1, 2\}$, reject. Correct.

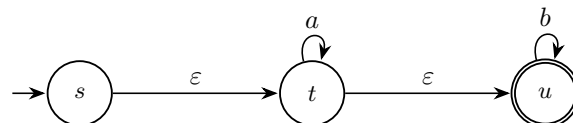
Worked Example 4.4.7 (Two ε -arrows feeding one start: a fuller closure). Convert N_4 over $\Sigma = \{a, b\}$ with states $\{s, t, u\}$, start s , accept $\{u\}$, and

$$\delta(s, \varepsilon) = \{t\}, \quad \delta(t, a) = \{t\}, \quad \delta(t, \varepsilon) = \{u\}, \quad \delta(u, b) = \{u\}.$$

(It recognizes a^*b^* : the start can silently slide to t then to u .)

Recognition. Two ε -arrows in a chain ($s \rightarrow t \rightarrow u$) mean the start closure is a three-element set; this stresses the closure step the most.

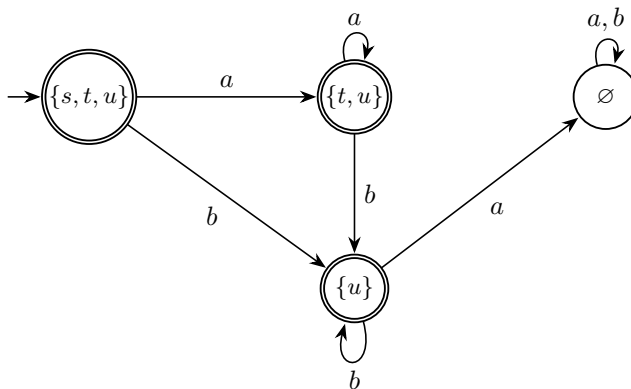
Solution. Here is N_4 :



Start subset: $E(\{s\}) = \{s, t, u\}$ – the ε -chain pulls in both t and u . Worklist:

R	$\delta'(R, a)$	$\delta'(R, b)$
$\{s, t, u\}$	$\{t, u\}$	$\{u\}$
$\{t, u\}$	$\{t, u\}$	$\{u\}$
$\{u\}$	$\emptyset$	$\{u\}$
$\emptyset$	$\emptyset$	$\emptyset$

From $\{s, t, u\}$ on a : only t has an a -arrow, giving $\{t\}$, then $E(\{t\}) = \{t, u\}$. On b : only u moves, $\{u\}$, closed to itself. From $\{u\}$ on a : no a -arrow, so $\emptyset$ – the dead state, which loops to itself on everything. All subsets containing u are accepting.



Which idea, and why? Because the start state's ε -closure already contains the accept state u , the DFA's start state is itself accepting – so the empty string is accepted, exactly as a^*b^* demands.

Check. $aabb \in a^*b^*$: $\{s, t, u\} \xrightarrow{a} \{t, u\} \xrightarrow{a} \{t, u\} \xrightarrow{b} \{u\} \xrightarrow{b} \{u\}$, accept. $ba \notin a^*b^*$: $\{s, t, u\} \xrightarrow{b} \{u\} \xrightarrow{a} \emptyset$, reject. Correct.

Worked Example 4.4.8 (Reading off the transition table without drawing N first). Given the NFA table below over $\Sigma = \{0, 1\}$ (start x , accept $\{z\}$), produce the DFA's transition table. Empty cells are $\emptyset$.

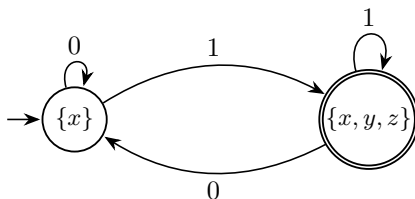
δ	0	1	ε
x	$\{x\}$	$\{x, y\}$	$\emptyset$
y	$\emptyset$	$\emptyset$	$\{z\}$
z	$\emptyset$	$\emptyset$	$\emptyset$

Recognition. We are handed δ directly; the subset construction is purely mechanical from a table, so no diagram of N is required to start. The single ε -arrow $y \rightarrow z$ is the wrinkle.

Solution. Since $y \rightarrow z$ is the only ε -arrow, $E(\{y\}) = \{y, z\}$, while $E(\{x\}) = \{x\}$ and $E(\{z\}) = \{z\}$. Start subset $q'_0 = E(\{x\}) = \{x\}$. Worklist, closing after every symbol:

R	$\delta'(R, 0)$	$\delta'(R, 1)$
$\{x\}$	$\{x\}$	$\{x, y, z\}$
$\{x, y, z\}$	$\{x\}$	$\{x, y, z\}$

From $\{x\}$ on 1: $\delta(x, 1) = \{x, y\}$, then $E(\{x, y\}) = \{x, y, z\}$ – the ε -arrow pulls in z , so the union is *not* simply $\{x, y\}$. From $\{x, y, z\}$ on 0: $\delta(x, 0) \cup \delta(y, 0) \cup \delta(z, 0) = \{x\} \cup \emptyset \cup \emptyset = \{x\}$. Only two reachable subsets; $\{x, y, z\}$ contains z , so it is accepting.



Which idea, and why? The construction depends only on δ , the start state, and F – so a transition table is enough input; the picture is a convenience, not a prerequisite. The mandatory closure after each symbol is what adds z to the 1-target.

Check. The NFA can be in z exactly when the last symbol read was a 1 (guess that 1, slide $y \rightarrow z$ by ε); z has no outgoing arrows, so a following 0 sends the DFA back to $\{x\}$. So $L(N) = \{w \mid w \text{ ends in } 1\}$. Trace 001: $\{x\} \xrightarrow{0} \{x\} \xrightarrow{0} \{x\} \xrightarrow{1} \{x, y, z\}$, accept. Trace 00: stays at $\{x\}$, reject. Correct.

How big can the DFA get?

The number of states never exceeds $2^{|Q|}$, and this bound is essentially tight: there are languages whose smallest NFA has $n + 1$ states but whose smallest DFA has 2^n states. The standard witness is “the n -th symbol from the end is 1.”

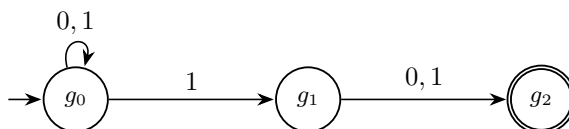
Worked Example 4.4.9 (The 2^n blow-up, illustrated for $n = 2$). The language

$$\mathcal{L}_2 = \{w \mid \text{the second symbol from the end of } w \text{ is } 1\}$$

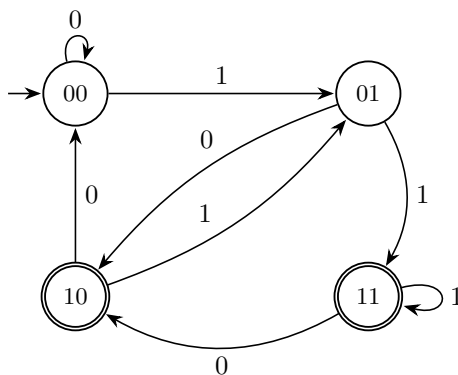
over $\Sigma = \{0, 1\}$ has a tiny 3-state NFA but needs $2^2 = 4$ DFA states.

Recognition. This is the witness for the exponential gap; we exhibit both machines to see the gap concretely at $n = 2$.

Solution. The NFA guesses where the second-from-last symbol is:



The subset construction yields four reachable subsets – $\{g_0\}$, $\{g_0, g_1\}$, $\{g_0, g_2\}$, $\{g_0, g_1, g_2\}$ – which is no accident: the DFA must remember the *last two symbols read*, and there are $2^2 = 4$ such two-bit memories. Renaming each subset by the two bits it remembers gives the clean DFA:



A state labelled b_1b_0 means “the previous symbol was b_1 , the current symbol was b_0 ”; the state is accepting iff $b_1 = 1$ (the second-from-last symbol). On reading c , b_1b_0 moves to b_0c .

Which idea, and why? The DFA must distinguish every possible value of the last n symbols, because any two strings differing there can be completed to one accepted and one rejected string – so at least 2^n states are forced.

Check. 010: second-from-last is 1, so accept. Trace: $00 \xrightarrow{0} 00 \xrightarrow{1} 01 \xrightarrow{0} 10$, and 10 is accepting ($b_1 = 1$). Correct.

Remark. The exponential is a *worst case*, not the typical case. Most NFAs that arise in practice convert to DFAs of modest size, because only a handful of subsets are reachable. Always build lazily from the start subset; never enumerate all $2^{|Q|}$ subsets blindly.

Caution. Forgetting the ε -closure is the most common error. You must close *twice*: once when forming the start subset $E(\{q_0\})$, and again after *every* symbol, as the outer $E(\cdot)$ in $\delta'(R, a) = E(\bigcup_{q \in R} \delta(q, a))$. If you only union the a -targets and skip the closure, you drop states the NFA could silently reach, and your DFA will wrongly reject some strings.

Caution. The empty set $\emptyset$ is a legitimate DFA state – the dead state – not an omission. It appears whenever every branch of the NFA dies (no arrow on the symbol read). A DFA must have a transition for *every* state on *every* symbol, so $\emptyset$ loops to itself on all of Σ and is never accepting (it contains no accept state). Leaving it out makes the transition function partial, and the result is not a DFA.

Caution. Do not confuse “the set of states currently active” with “the set of all states.” A DFA *state* is one particular subset of Q ; the DFA’s *state set* Q' is the collection of those subsets. Writing the start state as the whole of Q (instead of $E(\{q_0\})$) is a classic slip – the machine should start where the NFA starts, not everywhere at once.

Exercises.

1. Convert the following NFA over $\Sigma = \{0, 1\}$ to an equivalent DFA by the subset construction. States $\{a, b, c\}$, start a , accept $\{c\}$; transitions $\delta(a, 0) = \{a, b\}$, $\delta(a, 1) = \{a\}$, $\delta(b, 1) = \{c\}$, all other cells empty. List every reachable subset, give the DFA transition table, and draw its state diagram. Identify the dead state if one appears.
2. Convert the NFA over $\Sigma = \{0, 1\}$ with states $\{p, q\}$, start p , accept $\{q\}$, and $\delta(p, 0) = \{p, q\}$, $\delta(p, 1) = \{p\}$, $\delta(q, 1) = \{q\}$ (other cells empty) to a DFA. Then describe $L(N)$ in plain English and confirm your description by tracing 010 and 11.
3. An NFA over $\Sigma = \{a, b\}$ has states $\{1, 2, 3\}$, start 1, accept $\{3\}$, and the ε -arrows $\delta(1, \varepsilon) = \{2\}$ and $\delta(2, \varepsilon) = \{3\}$, together with $\delta(1, a) = \{1\}$, $\delta(2, b) = \{2\}$, $\delta(3, a) = \{3\}$. Compute the ε -closure of every state, then run the subset construction. What is the start subset, and why is it accepting?
4. Convert to a DFA the NFA over $\Sigma = \{0, 1\}$ that recognizes “the string contains 010 as a substring”: states $\{s_0, s_1, s_2, s_3\}$, start s_0 , accept $\{s_3\}$, with $\delta(s_0, 0) = \{s_0, s_1\}$, $\delta(s_0, 1) = \{s_0\}$, $\delta(s_1, 1) = \{s_2\}$, $\delta(s_2, 0) = \{s_3\}$, $\delta(s_3, 0) = \delta(s_3, 1) = \{s_3\}$. How many reachable subsets are there?
5. Draw the smallest NFA you can for “the second-from-last symbol is 0” over $\Sigma = \{0, 1\}$, then convert it to a DFA. Compare the number of states in your NFA and DFA, and explain why the DFA needs at least four states.
6. Give an NFA with 4 states (including one ε -arrow) whose subset construction produces a DFA with exactly 2 reachable states. Show both machines. (Hint: make almost every branch lead back to the same set of active states.)
7. Consider the NFA over $\Sigma = \{0, 1\}$ with states $\{u, v, w\}$, start u , accept $\{w\}$, and $\delta(u, 0) = \{u\}$, $\delta(u, 1) = \{u, v\}$, $\delta(v, \varepsilon) = \{w\}$, $\delta(w, 0) = \{w\}$, $\delta(w, 1) = \{w\}$. Run the subset construction, draw the DFA, and prove that the DFA’s start state is *not* accepting while it has at least one accepting state.
8. *Find the bug.* A student converts an NFA with an ε -arrow $\delta(q_0, \varepsilon) = \{q_1\}$ and writes the DFA start state as $\{q_0\}$, then proceeds. Explain precisely which step was skipped, what the start state should have been, and give a concrete string on which the student’s DFA could give the wrong answer.
9. *Find the bug.* Another student runs the subset construction on a 2-state NFA, gets the subsets $\{a\}$, $\{a, b\}$, and stops – declaring the DFA complete with two states – but the student’s table has no entry for the symbol on which $\{a, b\}$ loses all targets. What state is missing, what are its outgoing transitions, and is it accepting?
10. Prove that if N has n states then the DFA produced by the subset construction has at most 2^n states, and exhibit (with a state diagram) a 2-state NFA whose subset construction reaches all $2^2 = 4$ subsets, including $\emptyset$.
11. Using Corollary 4.4.3, prove that the class of regular languages is closed under *reversal*: if $\mathcal{L}$ is regular then so is $\mathcal{L}^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in \mathcal{L}\}$. (Sketch: reverse every arrow of a DFA for $\mathcal{L}$, swap start and accept roles

using ε -arrows, obtain an NFA, then invoke the theorem of this section.) State clearly where the subset construction is used.

12. Let N be an NFA in which the start state is also its only accept state and has no incoming arrows. Argue that in the DFA produced by the subset construction, a subset R is accepting iff it contains that start state, and use this to decide whether $\varepsilon \in L(N)$ directly from the DFA's start state.

4.5 Closure of the Regular Languages under the Regular Operations

Back in Section 4.2 we listed the three regular operations – union, concatenation, and star – and promised that the class of regular languages is *closed* under each of them: feed in regular languages, get a regular language back out. The product construction already settled union (and intersection) by gluing two DFAs together state by state, but it has no comfortable analogue for concatenation or star. The clean tool is nondeterminism. An NFA may sprout extra start branches, jump on ε without reading input, and loop back to its own beginning, so the three constructions below are little more than a few ε -arrows soldered between copies of machines we already have. Because every NFA has an equivalent DFA (Section 4.4), proving closure with NFAs proves it for regular languages.

Throughout, fix an alphabet Σ and let $A = L(N_A)$ and $B = L(N_B)$ be regular languages, recognized by NFAs $N_A = (Q_A, \Sigma, \delta_A, q_A, F_A)$ and $N_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$. We assume $Q_A \cap Q_B = \emptyset$ (rename states if not); this lets us drop both machines into one diagram without their states colliding.

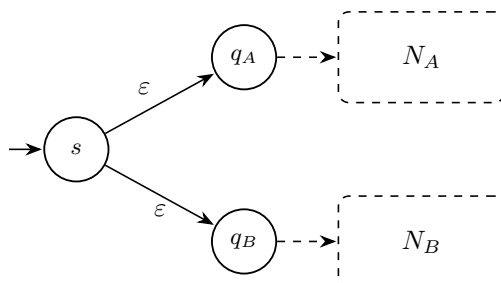
Closure under union

Theorem 4.5.1. *If A and B are regular, then $A \cup B$ is regular.*

Proof. Build an NFA N for $A \cup B$ as follows. Add one fresh start state s , and from s draw two ε -arrows, one to q_A and one to q_B . Keep every state and every transition of N_A and N_B unchanged. Formally $N = (Q, \Sigma, \delta, s, F)$ with

$$Q = \{s\} \cup Q_A \cup Q_B, \quad F = F_A \cup F_B,$$

and δ agreeing with δ_A on Q_A , with δ_B on Q_B , with $\delta(s, \varepsilon) = \{q_A, q_B\}$, and $\delta(s, a) = \emptyset$ for every $a \in \Sigma$. On input w , the very first move is a free ε -step that commits the computation to either N_A or N_B ; thereafter N behaves exactly like the machine it entered. Hence some branch accepts w iff N_A accepts w or N_B accepts w , i.e. iff $w \in A \cup B$. So $L(N) = A \cup B$, and by the equivalence of NFAs and DFAs this language is regular. $\square$

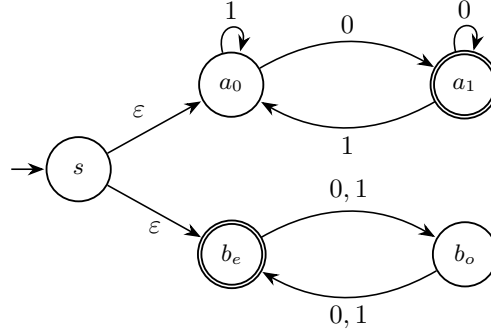


The dashed boxes stand for the unchanged bodies of N_A and N_B , with their old start states q_A, q_B now reached only through s . The new machine accepts when either body accepts.

Worked Example 4.5.2 (Union of “ends in 0” and “has even length”). Let $A = \{w \in \{0, 1\}^* \mid w \text{ ends in } 0\}$ and $B = \{w \mid |w| \text{ is even}\}$. Build an NFA for $A \cup B$.

Recognition. The task is a union of two regular languages, so apply the parallel-start construction: a fresh start state ε -branches to a machine for each language.

Solution. A two-state DFA N_A recognizes A (start a_0 non-accepting, a_1 accepting; read 0 to land in a_1 , read 1 to fall back to a_0). A two-state DFA N_B recognizes B (start b_e accepting for even length, b_o non-accepting, each input symbol toggling between them). Now add a start state s with ε -arrows to a_0 and b_e :



Which idea, and why? Union calls for the new-start-state-with-two- ε -branches gadget: the single free move at the start chooses which sublanguage to test, and we change nothing inside either machine.

Check. Trace 010. The top branch ends in 0, so N_A reaches the accepting a_1 ; the bottom branch sees length 3 (odd), so N_B ends in b_o , rejecting. One accepting branch suffices, so 010 is accepted – correct, since it ends in 0. Trace 11: it does not end in 0 (top rejects) but has even length (bottom accepts in b_e), so it is accepted too.

Closure under concatenation

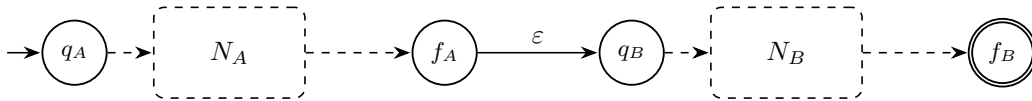
Recall $A \cdot B = \{xy \mid x \in A, y \in B\}$: a string in the concatenation splits somewhere into an A -part followed by a B -part. The trouble is that we do not know *where* the split falls. Nondeterminism guesses it: at every moment N_A is in an accepting state, the machine may take a free ε -jump to the start of N_B and try to finish the rest of the input in B .

Theorem 4.5.3. *If A and B are regular, then $A \cdot B$ is regular.*

Proof. Build $N = (Q, \Sigma, \delta, q_A, F_B)$ with $Q = Q_A \cup Q_B$. The start state is q_A (the old start of N_A) and the accept set is F_B alone (the old accepts of N_B). Keep all transitions of N_A and N_B , and add an ε -arrow from every accepting state of N_A to the start state q_B :

$$\delta(q, \varepsilon) \supseteq \{q_B\} \quad \text{for each } q \in F_A.$$

A computation reads some prefix while inside N_A ; whenever it reaches an F_A state it may ε -cross into N_B and read the remaining suffix there. It accepts iff the input factors as $w = xy$ with N_A accepting x and N_B accepting y , that is iff $w \in A \cdot B$. Note the old F_A states are *not* accepting in N : landing in A is only halfway. Hence $L(N) = A \cdot B$, which is therefore regular. $\square$

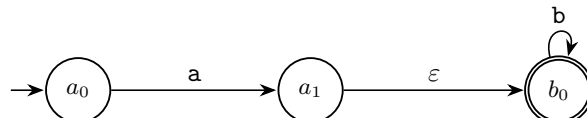


Here f_A marks a typical old-accept of N_A (no longer accepting in N) and f_B a typical accept of N_B (the only accepts that survive). Every F_A state gets such an ε -arrow into q_B .

Worked Example 4.5.4 (Concatenating “a single **a**” with “any number of **bs**”). Over $\Sigma = \{\mathbf{a}, \mathbf{b}\}$ let $A = \{\mathbf{a}\}$ and $B = \{\mathbf{b}\}^* = \{\varepsilon, \mathbf{b}, \mathbf{bb}, \dots\}$. Build an NFA for $A \cdot B = \{\mathbf{ab}^n \mid n \geq 0\}$.

Recognition. A concatenation of two regular languages: link the accepts of the first machine to the start of the second with ε , and accept only in the second.

Solution. Machine N_A : start a_0 , read **a** to the accepting a_1 (anything else is a dead end, which we leave implicit). Machine N_B : a single accepting start b_0 with a self-loop on **b**. Splice them by ε -joining the old accept a_1 to b_0 , demote a_1 to non-accepting, and keep b_0 accepting:



Which idea, and why? Concatenation glues accept-of- A to start-of- B with ε : the free jump guesses that the A -part has ended and the B -part begins, and only B -accepts count.

Check. On **abb**: read **a** ($a_0 \rightarrow a_1$), take the ε -jump to b_0 , then loop on each **b**, finishing in the accepting b_0 – accepted. On **ba**: the first symbol is **b**, but a_0 has no **b**-move, so the only branch dies – rejected, correctly, since **ba** $\notin A \cdot B$. On **a**: read **a**, ε -jump, accept (the $n = 0$ case).

Caution. Do *not* leave the old F_A states accepting after the splice. If a_1 above stayed a double circle, the machine would accept the bare string **a** as well as accept having merely reached the boundary – which happens to be fine here, but in general it would wrongly accept strings whose B -part is empty when $\varepsilon \notin B$. Only the accepts of the *second* machine may remain accepting. Likewise, the start state of the combined machine is the start of N_A , never N_B 's.

Closure under star

The star $A^* = \{x_1x_2 \cdots x_k \mid k \geq 0, \text{ each } x_i \in A\}$ is the set of all finite concatenations of A -strings, *including* the empty concatenation ε (the $k = 0$ case). The construction reuses the concatenation trick – loop the accepts back to the start so the machine can restart A as often as it likes – plus a wrinkle to get ε for free.

Theorem 4.5.5. *If A is regular, then A^* is regular.*

Proof. Take $N_A = (Q_A, \Sigma, \delta_A, q_A, F_A)$ for A . Build $N = (Q, \Sigma, \delta, s, F)$ where

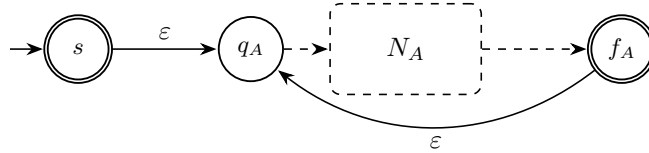
$$Q = \{s\} \cup Q_A, \quad F = \{s\} \cup F_A,$$

with a brand-new start state s that is *also* accepting. Add an ε -arrow from s to the old start q_A , and an ε -arrow from every accept of N_A back to q_A :

$$\delta(s, \varepsilon) = \{q_A\}, \quad \delta(q, \varepsilon) \supseteq \{q_A\} \text{ for each } q \in F_A,$$

all other transitions inherited from N_A . The new accepting start s gives ε with no input read ($k = 0$). For nonempty input, the machine runs N_A once; each time it reaches an accept it may ε -loop back to q_A to begin another block. Thus it accepts iff the input is a concatenation of zero or more strings of A , i.e. iff $w \in A^*$. $\square$

Remark. The new start state s is essential. A tempting shortcut just makes the old start q_A accepting and adds the back- ε -edges. But if q_A has an incoming arrow from inside the machine, marking it accepting can let through strings that are *not* in A^* – the loop would create a spurious accept midway through reading a block. The separate s , reachable only at the very start, supplies the ε case cleanly without corrupting the loop.

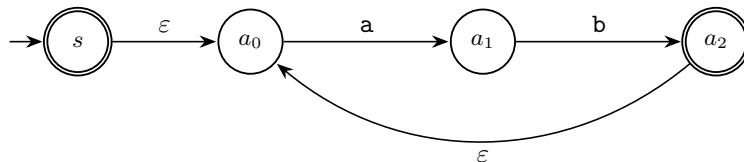


The ε -edge from the new s launches one run of N_A (or, by stopping in s , accepts ε); the curved ε -edge from each old accept f_A back to q_A lets the machine begin a fresh block as many times as it wants.

Worked Example 4.5.6 (Star of “exactly **ab**”). Over $\Sigma = \{a, b\}$ let $A = \{ab\}$. Build an NFA for $A^* = \{(\mathbf{ab})^n \mid n \geq 0\} = \{\varepsilon, \mathbf{ab}, \mathbf{abab}, \dots\}$.

Recognition. A star of a regular language: introduce a new accepting start state with an ε -arrow into the machine, and loop each accept back to the machine’s start.

Solution. Machine N_A for $\{ab\}$: $a_0 \xrightarrow{a} a_1 \xrightarrow{b} a_2$, with a_2 accepting. Add the new accepting start s with $s \xrightarrow{\varepsilon} a_0$, and an ε -arrow $a_2 \rightarrow a_0$ to restart:



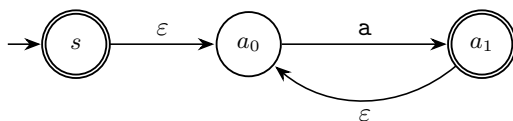
Which idea, and why? Star needs the fresh accepting start (for the ε case) plus an ε -loop from each accept back to the start (to repeat the block), exactly the gadget above.

Check. The empty string is accepted by stopping in s . On **abab**: $s \rightarrow a_0$ (ε), read **ab** to a_2 , ε -loop to a_0 , read **ab** again to a_2 – accept. On **aba**: after the second **a** the machine sits in a_1 , no further input, a_1 not accepting – rejected, correct since **aba** $\notin A^*$.

Worked Example 4.5.7 (Two readings of the empty string). Why must the star machine accept ε , and where exactly does that happen?

Recognition. This is a definitional check on A^* : by definition the empty concatenation ($k = 0$ blocks) lies in A^* regardless of whether $\varepsilon \in A$.

Solution. Consider $A = \{a\}$, which does *not* contain ε . Still $\varepsilon \in A^*$. In the construction the accepting start s is reached with no input consumed, so ε is accepted there – and *only* there for this A , since the path into a_0 then onward requires reading at least one **a** before any accept. Contrast the naive “just mark q_A accepting” approach: it also accepts ε , but it can additionally mis-accept partial blocks once back-edges feed into q_A .



Which idea, and why? The ε membership of A^* is structural, not inherited from A ; the fresh accepting s is the one state that supplies it without disturbing the loop.

Check. Strings accepted: ε (stop in s), **a**, **aa**, **aaa**, ... – exactly $\{a\}^*$. The string **b** dies immediately at a_0 .

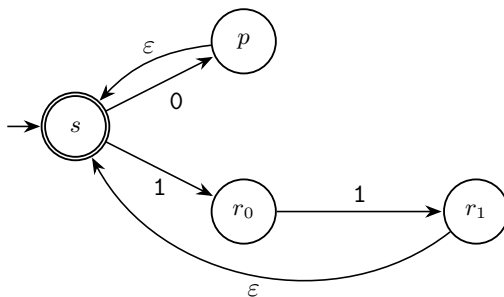
Putting the three together; intersection and complement

The three constructions interlock: any expression built from finite languages by union, concatenation, and star describes a regular language, because each operation preserves regularity and the single-string (finite) languages are trivially regular. The next worked example chains all three.

Worked Example 4.5.8 (Building $\mathcal{L} = (0 \cup 11)^*$ piece by piece). Over $\Sigma = \{0, 1\}$, recognize all strings that decompose into blocks each equal to 0 or to 11.

Recognition. The language is described by an expression in the regular operations, so build it bottom-up: singletons, then a union, then a star.

Solution. Start with one-state-per-symbol pieces for $\{0\}$ and $\{11\}$. Their union (parallel ε -start) recognizes $\{0, 11\}$; starring that (new accepting start, back-edges) gives the language. Collapsing the chained ε -arrows, one clean NFA is:



After reading a complete block (0 into p , or 11 into r_1) an ε -arrow returns to the accepting start s , ready for the next block; s accepting handles ε and marks every block boundary.

Which idea, and why? Composition of the closures: union supplies the two block shapes and star supplies “repeat any number of times,” so the machine is the union gadget wired into the star gadget.

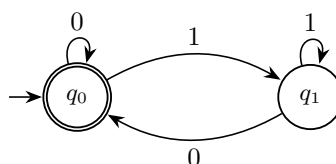
Check. 0110 parses as 0,11,0: $s \rightarrow p \rightarrow s \rightarrow r_0 \rightarrow r_1 \rightarrow s \rightarrow p \rightarrow s$, ending accepting. 10 jams – after 1 we are in r_0 needing another 1, but get 0; no branch survives, so 10 is rejected, as it must be.

In fact this is the whole story: REGULAR is the *smallest* class of languages that contains every finite language and is closed under union, concatenation, and star. (One half is the closures above; the other half – that everything regular is reachable this way – is the content of regular expressions, taken up next.) The constructions of Section 4.2 add the Boolean operations: regular languages are closed under *intersection* (run two DFAs in lockstep, accept when both accept) and under *complement* (run one DFA, swap accepting and non-accepting states). Together these make REGULAR closed under union, intersection, complement, concatenation, and star; Exercise 11 below adds reversal.

Worked Example 4.5.9 (Complement by swapping accept states). Let $\mathcal{L}$ be the language of strings over $\{0, 1\}$ that end in 1, recognized by a 2-state DFA. Build a DFA for the complement $\bar{\mathcal{L}}$, the strings that do not end in 1.

Recognition. Complement of a language given by a DFA: flip accepting and non-accepting states. This needs a DFA (totally defined, deterministic), not an NFA.

Solution. The DFA for $\mathcal{L}$: start q_0 (non-accepting), q_1 accepting; reading 1 goes to q_1 , reading 0 goes to q_0 . To complement, swap roles – make q_0 accepting and q_1 non-accepting, changing nothing else:



Which idea, and why? Complement = swap F and $Q \setminus F$ in a DFA, where exactly one state is active per string, so flipping accept-status flips the answer on every input.

Check. 10 now ends in q_0 (accepting) – correct, it does not end in 1. 01 ends in q_1 (now non-accepting) – correct. The empty string sits in the accepting q_0 , and indeed ε does not end in 1.

Caution. Swapping accept states computes the complement *only for a DFA*. For an NFA the trick fails: an NFA may have both an accepting and a non-accepting branch on the same input, so flipping accept-status does *not* flip the accept/reject verdict. To complement a language given by an NFA, first convert to a DFA (subset construction, Section 4.4), then swap. A second trap: the DFA must be *total* – every state needs an outgoing edge for every symbol – or a missing edge silently becomes an implicit reject that swapping will not catch. Add the dead state first if needed.

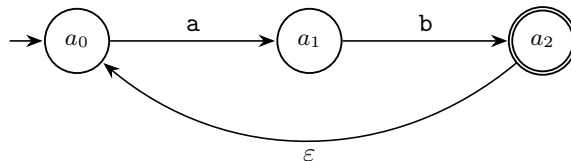
Method (Proving a class of languages is closed under an operation by an automaton construction). Goal: show that if the inputs are regular then the output is regular.

1. **Take machines for the inputs.** Assume NFAs (or DFAs) for the given languages, with disjoint state sets.
2. **Pick the gadget by the operation.** Union: new ε -branching start. Concatenation: ε from accepts-of-first to start-of-second; accept only in the second. Star: new accepting start, ε in and ε back from every accept. Intersection/complement: product DFA / swap accepts (use DFAs).
3. **Set start and accept sets carefully.** Decide which states accept in the new machine – this is where most errors hide (demote F_A for concatenation; add s to F for star).
4. **Argue both directions.** Show every accepted string is in the target language and every target string is accepted, usually by reading off what each ε -jump commits the computation to.
5. **Invoke equivalence.** An NFA recognizing the result certifies the result is regular (Section 4.4). Done.

Exercises.

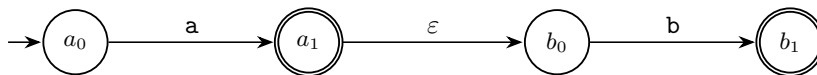
1. Let $A = \{w \in \{0, 1\}^* \mid w \text{ starts with } 0\}$ and $B = \{w \mid w \text{ contains the substring } 11\}$. Draw an NFA for $A \cup B$ using the parallel-start construction. Label the new start state and both ε -arrows.

- Over $\Sigma = \{a, b\}$ let $A = \{a\}^*$ and $B = \{b\}$. Draw an NFA for $A \cdot B = \{a^n b \mid n \geq 0\}$ by ε -linking the accept of a machine for A to the start of a machine for B . Which states accept in your final machine?
- Let $A = \{01, 10\}$. Draw an NFA for A^* using the new-accepting-start construction. Then trace your machine on 0110 and on 011, stating which is accepted.
- Give an NFA for $(\{0\} \cdot \{1\}^*) \cup \{1\}$ over $\{0, 1\}$ by composing the union and concatenation and star gadgets bottom-up. Draw each intermediate machine, then the final one.
- (Find the bug.) A student claims the NFA below recognizes $\{ab\}^*$. It does not, because it gets one short string wrong. Identify that string, say whether the machine over- or under-accepts it, and redraw the machine correctly.



(Hint: the start state is not accepting – what required member of $\{ab\}^*$ does that exclude?)

- (Find the bug.) The following NFA is offered as the concatenation $\{a\} \cdot \{b\} = \{ab\}$. Trace it on a and explain why it wrongly accepts that string; then fix it by changing exactly one accept marking.



- Prove that if A is regular then so is $A^+ = \{x_1 \cdots x_k \mid k \geq 1, x_i \in A\}$. (Modify the star construction so that ε is accepted only when $\varepsilon \in A$.) Draw the resulting NFA schematic.
- Prove directly, by an automaton construction, that the regular languages are closed under *intersection*. Then draw the product DFA for $\{w \mid |w| \text{ even}\} \cap \{w \mid w \text{ ends in } 1\}$ over $\{0, 1\}$, labelling each state with its pair of component states.
- Let $\mathcal{L}$ be recognized by the DFA with states $\{p, q, r\}$, start p , accept $\{q\}$, and transitions $\delta(p, 0) = q$, $\delta(p, 1) = r$, $\delta(q, 0) = q$, $\delta(q, 1) = r$, $\delta(r, 0) = r$, $\delta(r, 1) = r$. Draw a DFA for $\overline{\mathcal{L}}$ and describe $\overline{\mathcal{L}}$ in words. (First confirm the DFA is total.)
- Use the closure theorems to argue that $\{w \in \{0, 1\}^* \mid w \text{ starts with } 0 \text{ and ends with } 1\}$ is regular, by writing it as a concatenation of three regular languages and naming each.
- Show the regular languages are closed under *reversal*: if A is regular so is $A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\}$. (Sketch: reverse every arrow of an NFA for A , add a new start with ε -arrows to the old accepts, and make the old start the sole accept.) Draw the construction schematically.
- Let $A = \{0\}^*$ and $B = \{1\}^*$. Describe $A \cdot B$ in words, draw an NFA for it via the concatenation gadget, and then argue whether $A \cdot B$ equals $\{0, 1\}^*$ – give a string that decides it.

Chapter 5

Regular Expressions, Nonregularity, and the Algebra of Regular Languages

5.1 Regular Expressions

A regular expression is a compact piece of notation that *names* a language by building it up from single symbols using the three regular operations – union, concatenation, and star. Where a finite automaton is a *machine* that decides membership symbol by symbol, a regular expression is an *algebraic term* whose value is a set of strings; the one key idea of this section is that a finite syntax, parsed exactly like an arithmetic expression, suffices to describe a large and useful family of languages. We pin down the syntax (what counts as a regular expression) and the semantics (which language each expression denotes), fix the precedence that lets us drop most parentheses, and then design and read many expressions. That every regular expression denotes a language recognized by some automaton – and conversely – is the content of the next section; here we learn to speak the language fluently first.

Definition 5.1.1 (Regular expression). Fix an alphabet Σ . The *regular expressions* over Σ are the strings of symbols generated by the following inductive rules. R is a regular expression if R is

- (1) a for some symbol $a \in \Sigma$ (an *atom*);
- (2) ε , denoting the empty string;
- (3) $\emptyset$, denoting the empty language;
- (4) $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions (union);
- (5) $(R_1 \cdot R_2)$, where R_1 and R_2 are regular expressions (concatenation);
- (6) (R_1^*) , where R_1 is a regular expression (star).

Cases (1)–(3) are the *base* cases (no smaller expression inside); cases (4)–(6) are the *inductive* cases, each built from strictly smaller expressions. There is no circularity: the parts R_1, R_2 are always shorter than the whole, so the definition is a legitimate inductive (recursive) definition, and every regular expression is a finite term. The symbols $\cup, \cdot, *$, the parentheses, ε and $\emptyset$ are *meta-symbols* of the notation; they are not themselves members of Σ (we assume Σ contains none of them).

Definition 5.1.2 (The language of a regular expression). To each regular expression R we assign a language $L(R) \subseteq \Sigma^*$, defined by recursion on the structure of R exactly mirroring Definition 5.1.1:

$$\begin{aligned} L(a) &= \{a\} \quad (a \in \Sigma), & L(\varepsilon) &= \{\varepsilon\}, & L(\emptyset) &= \emptyset, \\ L(R_1 \cup R_2) &= L(R_1) \cup L(R_2), & L(R_1 \cdot R_2) &= L(R_1) \cdot L(R_2), & L(R_1^*) &= (L(R_1))^*, \end{aligned}$$

where on the right $\cup, \cdot, *$ are the set operations on languages (union; concatenation $A \cdot B = \{xy \mid x \in A, y \in B\}$; star $A^* = \{x_1 \cdots x_k \mid k \geq 0, x_i \in A\}$). We say R *describes* the language $L(R)$.

The two $*$ on the last line live in different worlds: the left one is a piece of *syntax* (a meta-symbol you can type), the right one is the *star operation* on a set. Reading “ $L(\cdot)$ ” as “the value of,” Definition 5.1.2 is just the evaluation rule for the term, like saying the value of $(5 + 3) \times 4$ is 32.

Precedence and abbreviations. Writing every parenthesis is unbearable, so we adopt the standard precedence, highest binding first:

$$* \text{ (star)} > \cdot \text{ (concatenation)} > \cup \text{ (union)}.$$

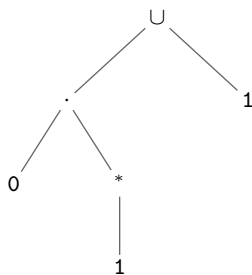
Thus 01^* means $0 \cdot (1^*)$, not $(01)^*$; and $0 \cup 10^*$ means $0 \cup (1 \cdot (0^*))$. The concatenation dot is almost always dropped, so $R_1 R_2$ abbreviates $R_1 \cdot R_2$. We write R^+ for RR^* (one or more copies, so $R^+ \cup \varepsilon = R^*$) and R^k for the k -fold concatenation $R \cdot \dots \cdot R$. Finally Σ , used as an expression, abbreviates $(a_1 \cup a_2 \cup \dots \cup a_m)$ where $\Sigma = \{a_1, \dots, a_m\}$, so $L(\Sigma)$ is all length-one strings and $L(\Sigma^*) = \Sigma^*$.

Remark. Just as parsing $2 + 3 \times 4$ produces an expression *tree* with $\times$ below $+$, parsing a regular expression produces a tree whose leaves are atoms ($a, \varepsilon, \emptyset$) and whose internal nodes are the operators, arranged by precedence. Evaluating the tree bottom-up by Definition 5.1.2 computes $L(R)$. We draw such trees below to make precedence visible.

Worked Example 5.1.3 (Parsing a regular expression: the operator tree of $01^* \cup 1$). Draw the parse tree of $R = 01^* \cup 1$ over $\Sigma = \{0, 1\}$ and read off $L(R)$.

Recognition. The task is to make the implicit grouping explicit. The trigger is the presence of all three operators with no parentheses, so precedence ($*$ then $\cdot$ then $\cup$) alone decides the structure.

Solution. Star binds tightest: 1^* groups first. Concatenation is next, so $0 \cdot 1^* = 01^*$ is one factor. Union binds loosest and sits at the root, joining 01^* with the bare 1. The fully parenthesized form is $((0)(1^*)) \cup (1)$.



Evaluating bottom-up: $L(1) = \{1\}$, so $L(1^*) = \{\varepsilon, 1, 11, \dots\}$ (any run of 1s); concatenating 0 in front gives $\{0, 01, 011, \dots\}$ (a 0 then any number of 1s); the root unions in the single string 1. Hence $L(R) = \{1\} \cup \{01^k \mid k \geq 0\}$.

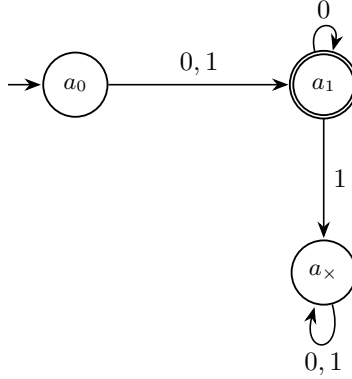
Which idea, and why? Precedence is exactly the rule for shaping an expression tree: $*$ is deepest, $\cdot$ above it, $\cup$ at the root, so the unparenthesized string has one unambiguous reading.

Check. The string 0111 is 01^3 , in the left branch; 1 is the lone right-branch string; but 10 is in neither, so it is excluded – matching the description “either the single 1, or a 0 followed by any number of 1s.”

Worked Example 5.1.4 (From expression to machine: an automaton for $(0 \cup 1)0^*$). Exhibit the language of $R = (0 \cup 1)0^*$ and give a DFA that recognizes it, so the expression and a machine sit side by side.

Recognition. An expression and an automaton are two notations for the *same* language; the trigger “ $(\cdot)$ then 0^* ” says: first read exactly one symbol (a 0 or a 1), then read any number of 0s and nothing else.

Solution. By Definition 5.1.2, $L(0 \cup 1) = \{0, 1\}$ and $L(0^*) = \{\varepsilon, 0, 00, \dots\}$, so $L(R) = \{x0^k \mid x \in \{0, 1\}, k \geq 0\}$: a first symbol followed by a block of 0s. A machine needs: a_0 “nothing read yet,” a_1 “first symbol read; only 0s allowed now” (accept), and a dead state $a_\times$ entered if a 1 arrives after the first symbol. From a_0 either symbol goes to a_1 ; from a_1 a 0 stays (more trailing zeros) and a 1 dies; the empty string is *not* in $L(R)$ since at least one symbol must be read, so $a_0 \notin F$.



(There is no loop at a_0 : both symbols leave it on the very first move, which is correct because the language forces at least one symbol to be read before anything can be accepted.)

Which idea, and why? Reading a regular expression left to right as “do this, then this, then star this” often suggests the states directly: one state per stage of the expression, with star realized as a self-loop.

Check. 1000 is 10^3 : $a_0 \xrightarrow{1} a_1 \xrightarrow{0} a_1 \xrightarrow{0} a_1 \xrightarrow{0} a_1 \in F$, accepted. 010 has a 1 after the first symbol: $a_0 \xrightarrow{0} a_1 \xrightarrow{1} a_x \xrightarrow{0} a_x \notin F$, rejected.

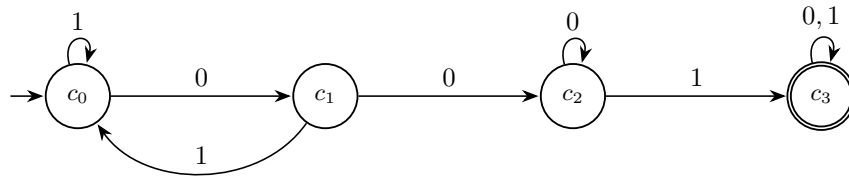
Worked Example 5.1.5 (Designing an expression: “contains the substring 001”). Over $\Sigma = \{0, 1\}$ write a regular expression for $\mathcal{L} = \{w \mid w \text{ contains } 001 \text{ somewhere}\}$.

Recognition. A *contains-a-fixed-substring* condition. The trigger “ u appears somewhere” is matched by the template “anything, then u , then anything,” because the two “anything” blocks absorb whatever surrounds the required occurrence.

Solution. Use Σ^* for “any string” on each side and the literal pattern in the middle:

$$R = \Sigma^* 001 \Sigma^* = (0 \cup 1)^* 001 (0 \cup 1)^*.$$

The left Σ^* matches everything before the first occurrence of 001, the middle spells the pattern exactly, and the right Σ^* matches the remainder.



The diagram is the standard substring-search DFA: c_0, c_1, c_2 track how much of 001 has just been matched, and the accept state c_3 traps once the pattern is found. The expression and this machine denote the same $\mathcal{L}$.

Which idea, and why? For “contains u ,” flank the literal u with Σ^* on both sides; the stars let the required occurrence sit anywhere in the string.

Check. 11001 ends in 001, so it matches R with the left star eating 11; tracing the machine, $c_0 \xrightarrow{1} c_0 \xrightarrow{1} c_0 \xrightarrow{0} c_1 \xrightarrow{0} c_2 \xrightarrow{1} c_3$, accepted. 0101 has no 001 and is rejected.

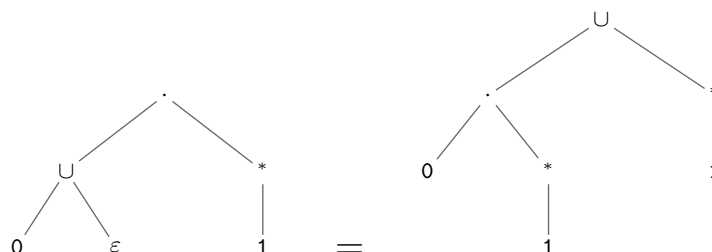
Worked Example 5.1.6 (An algebraic identity: $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$). Show that the two regular expressions $(0 \cup \varepsilon)1^*$ and $01^* \cup 1^*$ describe the *same* language.

Recognition. This is a *language equality* between expressions, the regex analogue of an arithmetic identity. The trigger is the factor $(0 \cup \varepsilon)$ multiplying 1^* : concatenation distributes over union, so we can “multiply out.”

Solution. Concatenation distributes over union on the right: $(A \cup B) \cdot C = (A \cdot C) \cup (B \cdot C)$. With $A = \{0\}$, $B = \{\varepsilon\}$, $C = L(1^*)$,

$$L((0 \cup \varepsilon)1^*) = (\{0\} \cup \{\varepsilon\}) \cdot L(1^*) = \{0\}L(1^*) \cup \{\varepsilon\}L(1^*) = L(01^*) \cup L(1^*),$$

using $\{\varepsilon\} \cdot C = C$. The right-hand side is exactly $L(01^* \cup 1^*)$, so the two expressions are equal.



The left tree applies $\cdot$ to a $\cup$; distributing pushes the $\cdot$ below the $\cup$, producing the right tree – a structural picture of the same identity.

Which idea, and why? Concatenation distributes over union, so a product of a sum can be expanded term by term; the empty-string factor ε acts as a multiplicative identity, $\varepsilon R = R$.

Check. Both sides contain ε (take ε from $0 \cup \varepsilon$ and ε from 1^* ; or take the 1^* summand with zero 1s) and both contain 011 (left: 0 then 11; right: from the 01^* summand). Neither contains 10 .

Worked Example 5.1.7 (Reading subtle cases: $\emptyset$ versus ε , and $\emptyset^*$). Decide the languages of $R_1 = 1^*\emptyset$, $R_2 = \emptyset^*$, and $R_3 = 0 \cup \varepsilon$, and explain why the first differs sharply from intuition.

Recognition. These are *edge cases* of the semantics. The trigger is the appearance of the two special atoms $\emptyset$ and ε , which behave like a multiplicative zero and a multiplicative one, respectively – but only if you keep them straight.

Solution. Apply Definition 5.1.2 literally.

- $L(R_1) = L(1^*) \cdot L(\emptyset) = L(1^*) \cdot \emptyset = \emptyset$, because concatenating *any* language with the empty language yields the empty language (there is no string $y \in \emptyset$ to append). So $1^*\emptyset$ denotes nothing at all, not “some 1s.”
- $L(R_2) = (L(\emptyset))^* = \emptyset^* = \{\varepsilon\}$: the star of the empty language is $\{\varepsilon\}$, since the star always includes the empty concatenation ($k = 0$), even when there are no one-or-more concatenations available.
- $L(R_3) = \{0\} \cup \{\varepsilon\} = \{0, \varepsilon\}$: either a single 0 or nothing.

expression	language	in words
$1^*\emptyset$	$\emptyset$	no strings at all
$\emptyset^*$	$\{\varepsilon\}$	exactly the empty string
$0 \cup \varepsilon$	$\{\varepsilon, 0\}$	optional single 0

Which idea, and why? The empty language $\emptyset$ annihilates under concatenation ($A\emptyset = \emptyset$) but *not* under star ($\emptyset^* = \{\varepsilon\}$); the empty string ε is the unit of concatenation ($A\varepsilon = A$). Mixing them up is the most common semantic slip.

Check. If $1^*\emptyset$ meant “runs of 1s,” then 11 would be in it – but to build 11 you would need a string from $L(\emptyset)$ to append, and there is none, confirming $L(1^*\emptyset) = \emptyset$.

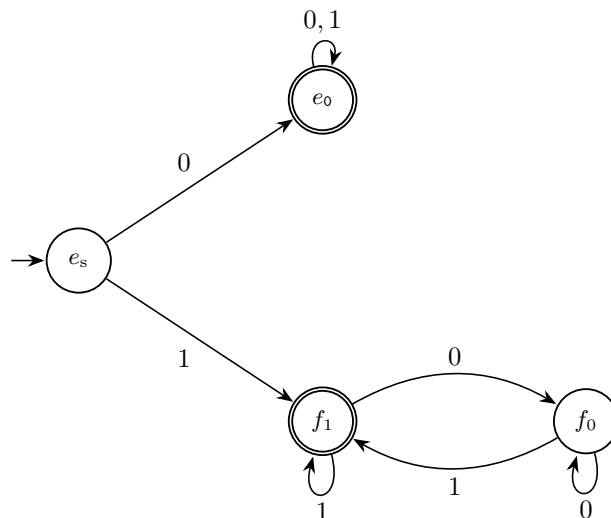
Worked Example 5.1.8 (Designing with Σ : “starts with 0 or ends with 1”). Over $\Sigma = \{0, 1\}$ write a regular expression for $\mathcal{L} = \{w \mid w \text{ starts with } 0 \text{ or } w \text{ ends with } 1\}$.

Recognition. An “or” of two positional conditions. The trigger word “or” points at union at the top level, with each side a positional pattern built from a fixed symbol and a Σ^* wildcard for the unconstrained part.

Solution. “Starts with 0” is $0\Sigma^*$ (a 0 then anything). “Ends with 1” is Σ^*1 (anything then a 1). Union them:

$$R = 0\Sigma^* \cup \Sigma^*1 = 0(0 \cup 1)^* \cup (0 \cup 1)^*1.$$

A small automaton makes the “or” concrete: track whether the first symbol was 0, and, when it was not, what the current last symbol is.



Here e_0 means “first symbol was 0,” which is permanently accepting because the start condition is locked in. States f_1 (accepting) and f_0 apply when the first symbol was 1: then acceptance depends entirely on whether the input currently ends in 1, so f_1 is accepting and f_0 is not.

Which idea, and why? Translate “ P or Q ” into $R_P \cup R_Q$, building each clause from a literal anchor (0 at the front, 1 at the back) padded by Σ^* for the free part.

Check. 0110 starts with 0: it matches the left summand $0\Sigma^*$. 101 ends with 1: it matches the right summand Σ^*1 . 10 does neither (starts 1, ends 0) and is excluded.

Method (How to design a regular expression for a target language).

1. **Find the top-level operator from the connective in the description.** “ P or Q ” $\rightarrow$ a union $R_P \cup R_Q$; “ P then Q ” or “ P followed by Q ” $\rightarrow$ a concatenation $R_P R_Q$; “any number of blocks of the form P ” $\rightarrow$ a star $(R_P)^*$.
2. **Use Σ^* as a wildcard** for parts of the string you do not constrain, and a literal spelling for parts you do. “Contains u ” $\rightarrow \Sigma^* u \Sigma^*$; “starts with u ” $\rightarrow u \Sigma^*$; “ends with u ” $\rightarrow \Sigma^* u$.
3. **Encode “optional” with $\cup \varepsilon$** and “one or more” with $^+$: an optional sign is $(+ \cup - \cup \varepsilon)$, a nonempty block of 0s is 0^+ .
4. **Watch precedence; parenthesize anything you star or union.** A star binds only the atom or group immediately to its left, so write $(01)^*$, never 01^* , when you mean repetitions of the block 01.
5. **Test by parsing a few strings.** Check at least one string that should match, one that should not, and the empty string; expand the expression’s tree if a grouping is unclear.

Caution. Do not confuse ε and $\emptyset$. The expression ε denotes $\{\varepsilon\}$, a language with one (empty) string; $\emptyset$ denotes the empty language, with *no* strings. They behave oppositely under the operations: $R\varepsilon = R$ and $R \cup \emptyset = R$ (each is a “do-nothing”), but $R\emptyset = \emptyset$ (concatenating $\emptyset$ destroys everything) while $R \cup \varepsilon$ may differ from R (it can add the empty string). A handy mnemonic: ε is the multiplicative unit, $\emptyset$ is the multiplicative zero of concatenation – yet $\emptyset^* = \{\varepsilon\}$, the one place the “zero” springs back to life.

Caution. Precedence errors are the usual bug. Because star binds tighter than concatenation, $\mathbf{ab}^*$ means $\mathbf{a(b^*)}$ – an $\mathbf{a}$ followed by any number of $\mathbf{bs}$ – *not* repetitions of the block $\mathbf{ab}$, which must be written $(\mathbf{ab})^*$. Likewise $\mathbf{a \cup bc}$ means $\mathbf{a \cup (bc)} = \{\mathbf{a, bc}\}$, not $(\mathbf{a \cup b})\mathbf{c}$. When in doubt, parenthesize, or draw the parse tree and evaluate it bottom-up.

Caution. Remember that R^* *always* contains ε (the empty concatenation, $k = 0$), even when R itself cannot produce the empty string. So $(01)^*$ accepts $\varepsilon, 01, 0101, \dots$; if you need “one or more” copies and want to exclude ε , use $R^+ = RR^*$ instead. Forgetting this silently lets the empty string into a language you meant to be nonempty.

Exercises.

- Over $\Sigma = \{0, 1\}$, describe in one English sentence the language denoted by each expression, and give two strings that belong and two that do not: (a) 10^*1 ; (b) $(0 \cup 1)00(0 \cup 1)^*$; (c) $0^* \cup 1^*$; (d) $(01)^*(10)^*$.
- Insert all parentheses according to the precedence rules ($*$, then $\cdot$, then $\cup$) into each expression, then draw its parse tree: (a) $ab \cup c^*$; (b) $a^*b \cup b^*a$; (c) $(a \cup b)c^*a$.
- Write a regular expression over $\{0, 1\}$ for the strings that contain at least two 1s. Then write one for the strings that contain *exactly* two 1s.
- Write a regular expression over $\{0, 1\}$ for the strings of even length, and separately for the strings whose length is a multiple of three. (Use Σ as shorthand for $0 \cup 1$.)
- Write a regular expression over $\{a, b\}$ for the strings that start and end with the same symbol. (Treat the one-symbol strings a and b as the boundary cases, and decide whether the empty string should be included.)
- Give a regular expression over $\{0, 1\}$ for the strings in which every 0 is immediately followed by at least one 1. Trace your expression on 0110 and on 0101, and say which (if either) it accepts.
- For each pair, decide whether the two expressions denote the same language; prove equality by an algebraic identity (distribution, $\varepsilon R = R$, $R \cup \emptyset = R$, etc.) or exhibit a string in one but not the other: (a) $(a \cup b)^*$ and a^*b^* ; (b) $(a^*)^*$ and a^* ; (c) $a(b \cup c)$ and $ab \cup ac$.
- Evaluate each expression precisely, paying attention to ε and $\emptyset$: (a) $\emptyset \cup 1^*$; (b) $(0 \cup \varepsilon)(1 \cup \varepsilon)$; (c) $\emptyset 0^*$; (d) $(\emptyset^*)^*$. Give the language as a set where it is finite.
- Write a regular expression for decimal numerals (over the digit alphabet $D = \{0, 1, \dots, 9\}$ plus the symbols $+$, $-$, $.$) that allows an optional leading sign and either an integer part, a fractional part, or both, so that 72, 3.14, +7., and -.01 are all matched. Explain which subexpression handles the optional sign.
- Find the bug.* A student writes 01^* and claims it denotes “any number of copies of the block 01,” so that 0101 matches. Explain, using precedence, exactly what 01^* really denotes, give a string the student’s reading would wrongly accept, and give the correct expression for “any number of copies of 01.”
- Find the bug.* Someone claims $1^*\emptyset$ denotes “all strings of 1s,” citing the 1^* factor. Using Definition 5.1.2 of $L(\cdot)$, compute $L(1^*\emptyset)$ exactly and explain in one sentence why the $\emptyset$ factor overrides the 1^* .
- Prove that for all regular expressions R and S , the expressions $(R \cup S)^*$ and $(R^*S^*)^*$ denote the same language. (Argue both inclusions on the level of strings: every string is a concatenation of pieces, each from $L(R)$ or $L(S)$.) Then give a single expression showing that $(R^*)^+ = R^*$.

5.2 Kleene’s Theorem: Equivalence of Regex and Finite Automata

Section 5.1 gave two different ways to name a regular language: build a machine that recognizes it, or write a regular expression that describes it. This section proves that those two descriptions are interchangeable – a language is recognized by a finite automaton if and only if some regular expression denotes it. This is *Kleene’s theorem*, and it is the bridge that makes the whole algebra of regular languages usable: anything you can prove about machines transfers to expressions and back. The proof is constructive in both directions and the constructions are the real payload, so we develop each as an algorithm you can run by hand. Going from an expression to a machine, we glue tiny automata together with ε -transitions (the *Thompson construction*); going from a machine back to an expression, we rip states out one at a time, paying for each removed state with a regular expression on the surviving edges (*state elimination* on a *GNFA*).

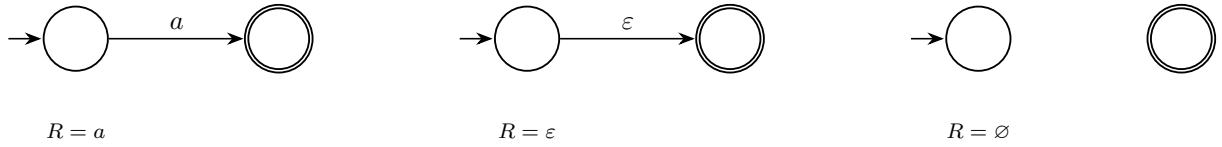
Theorem 5.2.1 (Kleene). *A language $\mathcal{L} \subseteq \Sigma^*$ is regular if and only if some regular expression describes it. Equivalently, the languages recognized by finite automata are exactly the languages denoted by regular expressions.*

We prove the two directions as separate lemmas, each carrying its own construction. Throughout, recall that a regular expression R is built from the atoms $a \in \Sigma$, ε , and $\emptyset$ by the operations union $R_1 \cup R_2$, concatenation $R_1 R_2$, and star R^* , and that $L(R)$ denotes the language it describes. Recall also (Corollary 4.4.3) that a language is regular iff some NFA recognizes it; we are free to target NFAs, which keeps the constructions small.

From regular expression to NFA

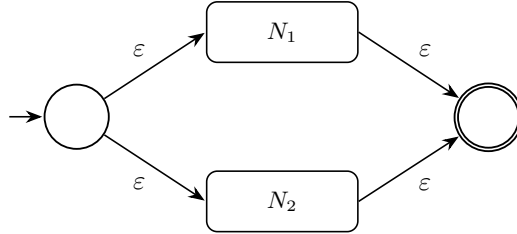
Lemma 5.2.2 (Thompson construction). *If R is a regular expression, then there is an NFA N with $L(N) = L(R)$. Moreover N can be built with a single start state, a single accept state distinct from the start, and at most $2|R|$ states, where $|R|$ is the number of symbols and operators in R .*

Proof. We induct on the structure of R . The three *atoms* are recognized by the two-state machines below. For the empty-language atom $\emptyset$ we use a start and an accept state with no edge between them, so nothing is accepted.

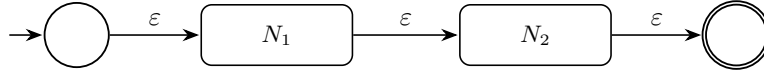


For the *compound* cases, assume by induction that R_1 and R_2 already have NFAs N_1, N_2 , each drawn as a box with one entry (its start state) and one exit (its single accept state). We wire the boxes together with fresh states and ε -edges; in each new machine the fresh start has no incoming edge and the fresh accept has no outgoing edge, preserving the one-in/one-out shape so the induction continues.

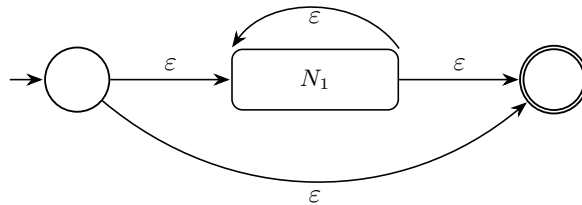
Union $R_1 \cup R_2$: a new start branches by ε into both boxes, and both boxes feed by ε into a new accept. A run guesses which alternative to take.



Concatenation $R_1 R_2$: the accept of N_1 is linked by ε to the start of N_2 . A new start feeds N_1 's start by ε , and N_2 's accept feeds a new accept by ε . A run must finish N_1 before N_2 can begin, so it accepts exactly the strings xy with $x \in L(R_1)$, $y \in L(R_2)$.



Star R^ :* a new start reaches N_1 's start by ε and also jumps by ε straight to a new accept; N_1 's accept feeds the new accept by ε and loops back by ε to N_1 's start. The direct ε -edge from the new start to the new accept puts ε in the language; the back-edge lets a run reread the body any number of times.



Each construction recognizes the right language by the closure proofs of Section 4.5 (union, concatenation, star), and the structural induction bottoms out at the atoms. Counting states: each atom uses at most two, and each operator adds at most two fresh states, giving at most $2|R|$ in total. $\square$

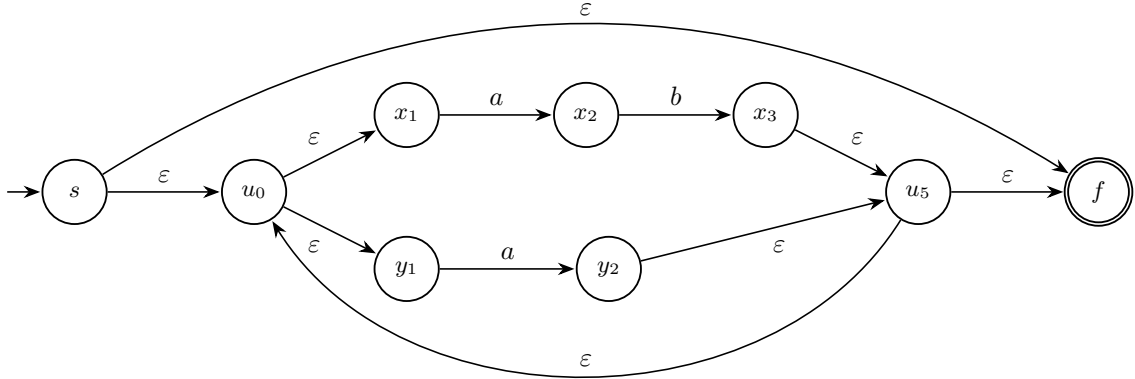
Method (How to build an NFA from a regular expression (Thompson)).

1. **Parse** R by precedence (star binds tightest, then concatenation, then union), giving a tree whose leaves are atoms $a, \varepsilon, \emptyset$ and whose internal nodes are $\cup, \cdot, ^*$.
2. **Base.** Draw the two-state atom machine for each leaf: an a -edge for a , an ε -edge for ε , no edge for $\emptyset$.
3. **Combine bottom-up.** At each internal node glue the child machines with the union / concatenation / star gadget above, always adding *fresh* start/accept states so every subexpression keeps one entry and one exit.
4. **Stop** at the root: the machine for the whole tree is your NFA. Optionally collapse obvious ε -chains afterward, but the raw output is already correct.

Worked Example 5.2.3 (Build an NFA for $(ab \cup a)^*$). Convert $R = (ab \cup a)^*$ over $\Sigma = \{a, b\}$ to an NFA by the Thompson construction.

Recognition. The outermost operator is the star, whose operand is a union; the left branch of the union is a concatenation ab . So the parse tree is $[(a \cdot b) \cup a]^*$, and we build inside-out. The trigger is “a regex with nested operators”: handle each node with its gadget.

Solution. The atoms a and b are single-edge machines. Concatenate to get a machine for ab ; union that with a second copy of the a -machine; finally apply the star gadget. Writing the inner union machine compactly (states u_0 start, u_5 accept) and then adding the star’s new start/accept s, f gives:



The top path $x_1 \rightarrow x_2 \rightarrow x_3$ reads ab ; the bottom path $y_1 \rightarrow y_2$ reads a ; the ε from s straight to f supplies the empty string; the back-edge $u_5 \rightarrow u_0$ repeats the body.

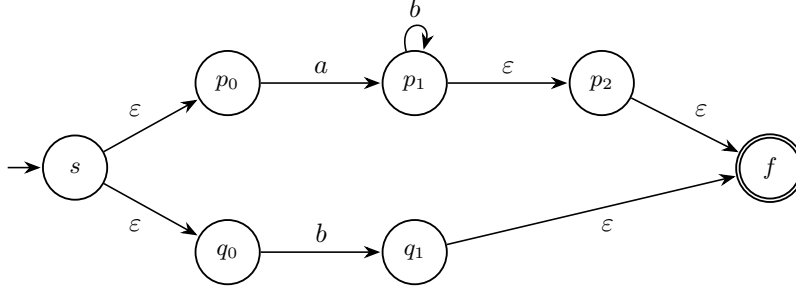
Which idea, and why? Each operator of R maps to exactly one Thompson gadget (atom, union, concatenation, star), so building the parse tree bottom-up and substituting gadgets is guaranteed to yield an NFA for $L(R)$.

Check. The string aab should be accepted, as $a \cdot ab$: $s \xrightarrow{\varepsilon} u_0 \xrightarrow{\varepsilon} y_1 \xrightarrow{a} y_2 \xrightarrow{\varepsilon} u_5 \xrightarrow{\varepsilon} u_0 \xrightarrow{\varepsilon} x_1 \xrightarrow{a} x_2 \xrightarrow{b} x_3 \xrightarrow{\varepsilon} u_5 \xrightarrow{\varepsilon} f$, accept. The empty string is accepted via $s \xrightarrow{\varepsilon} f$. The string ba is rejected: no run can read a leading b .

Worked Example 5.2.4 (Build an NFA for $ab^* \cup b$). Convert $R = ab^* \cup b$ to an NFA, watching the precedence.

Recognition. The trigger is precedence: star binds tighter than concatenation, which binds tighter than union, so R parses as $(a \cdot (b^*)) \cup b$, *not* as $a \cdot (b^* \cup b)$ and *not* as $(ab)^* \dots$. Getting the tree right is the whole game.

Solution. Build b^* first (a star gadget around the b -atom), concatenate an a -atom in front to get ab^* , then union with a bare b -atom.



The top branch reads one **a** then any number of **b**'s (the self-loop **b** on p_1 is the star); the bottom branch reads a single **b**.

Which idea, and why? Precedence fixes the parse tree before any drawing happens; the star applies only to **b**, so the loop sits on the **b**-machine and the leading **a** stays outside it.

Check. **a** is accepted (zero **b**'s): $s \xrightarrow{\varepsilon} p_0 \xrightarrow{a} p_1 \xrightarrow{\varepsilon} p_2 \xrightarrow{\varepsilon} f$. So is **abb** and the single **b**. But **ba** is rejected, confirming the union is **ab*** or **b**, never **ba**.

From DFA to regular expression: GNFA's and state elimination

The reverse direction needs a machine flexible enough to carry whole regular expressions on its edges while we dismantle it. That machine is the *generalized NFA*.

Definition 5.2.5 (Generalized NFA). A *generalized nondeterministic finite automaton* (GNFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, q_{\text{acc}})$ in which each transition arrow is labelled by a *regular expression* rather than a single symbol, and which is kept in *normal form*:

- q_0 has an arrow to every other state and no incoming arrow;
- $q_{\text{acc}} \neq q_0$ has an arrow from every other state and no outgoing arrow;
- between every other ordered pair of states (including a state to itself) there is exactly one arrow.

Formally $\delta: (Q \setminus \{q_{\text{acc}}\}) \times (Q \setminus \{q_0\}) \rightarrow \mathcal{R}$, where $\mathcal{R}$ is the set of regular expressions over Σ . The GNFA accepts w if w can be split as $w = w_1 w_2 \cdots w_k$ and there are states $q_0 = r_0, r_1, \dots, r_k = q_{\text{acc}}$ with $w_i \in L(\delta(r_{i-1}, r_i))$ for each i .

Normal form is a convenience, not a restriction: “no arrow” is recorded as the label $\emptyset$ (which matches nothing), and parallel arrows are merged by unioning their labels.

Lemma 5.2.6 (State elimination). *Every DFA M has a regular expression R with $L(R) = L(M)$.*

Proof. First turn M into a GNFA G in normal form: add a new start q_0 with an ε -arrow to M 's old start, add a new accept q_{acc} with ε -arrows from all of M 's old accept states, replace any multi-symbol label such as a, b by the union $a \cup b$, and fill every missing arrow with the label $\emptyset$. Now G has some number $k \geq 2$ of states and recognizes $L(M)$.

If $k > 2$, pick any state q_{rip} other than q_0, q_{acc} and *eliminate* it. For every surviving pair (q_i, q_j) – including $q_i = q_j$ – update the label on the $q_i \rightarrow q_j$ arrow so it also accounts for paths that detoured through q_{rip} . If the four relevant old labels are

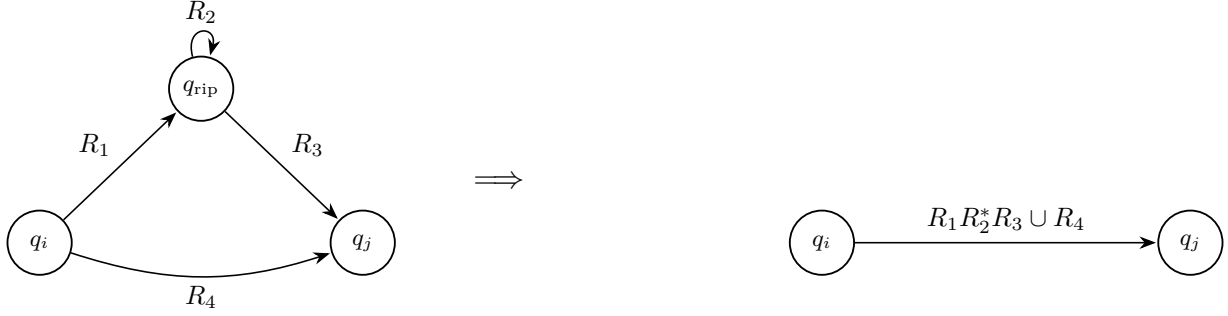
$$R_1 = \delta(q_i, q_{\text{rip}}), \quad R_2 = \delta(q_{\text{rip}}, q_{\text{rip}}), \quad R_3 = \delta(q_{\text{rip}}, q_j), \quad R_4 = \delta(q_i, q_j),$$

then the new label is

$$\delta'(q_i, q_j) = R_1 R_2^* R_3 \cup R_4.$$

This reads “go straight from q_i to q_j (R_4), or step into q_{rip} (R_1), loop there any number of times (R_2^*), and step out to q_j (R_3).” The detoured strings are exactly the ones lost when q_{rip} disappears, and the new label restores them alongside the direct ones, so the new GNFA on $k - 1$ states recognizes the same language. Repeat until $k = 2$. The lone surviving arrow from q_0 to q_{acc} is labelled by a regular expression R describing precisely the accepted strings, and $L(R) = L(M)$. $\square$

The heart of the lemma is the rewiring rule, worth a picture of its own.



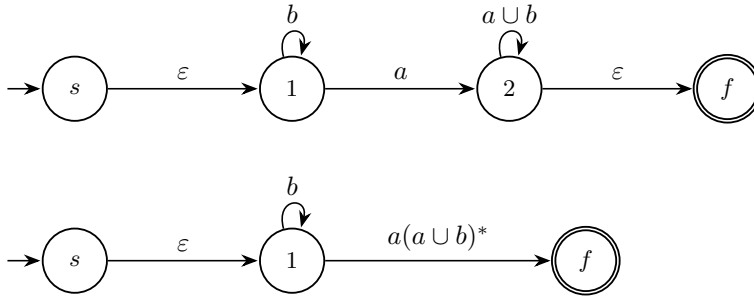
Method (How to convert a DFA to a regular expression (state elimination)).

1. **Normalize.** Add a fresh q_0 (ε to the old start) and a fresh q_{acc} (ε from each old accept). Merge parallel edges by union; treat absent edges as $\emptyset$ (you may leave them undrawn).
2. **Choose a victim.** Pick any internal state $q_{rip} \neq q_0, q_{acc}$. A state with few in/out edges keeps the algebra small.
3. **Rewire.** For every ordered pair (q_i, q_j) of survivors, replace $\delta(q_i, q_j)$ by $R_1 R_2^* R_3 \cup R_4$ with R_1, R_2, R_3, R_4 as above. Do this for self-pairs too (that is how loops are inherited).
4. **Simplify** each new label using $\emptyset R = \emptyset$, $\varepsilon R = R$, $R \cup \emptyset = R$, $\emptyset^* = \varepsilon$, so the expressions do not explode.
5. **Repeat** until only q_0, q_{acc} remain. The single edge's label is your regular expression.

Worked Example 5.2.7 (Convert a two-state DFA to a regular expression). Let M over $\{a, b\}$ have states 1 (start) and 2 (accept), with $\delta(1, a) = 2$, $\delta(1, b) = 1$, and $\delta(2, a) = \delta(2, b) = 2$. Find a regular expression for $L(M)$.

Recognition. The trigger is “DFA $\rightarrow$ regex,” so run state elimination. Two real states plus the two fresh endpoints gives a four-state GNFA; two eliminations finish it.

Solution. Normalize: add $s \xrightarrow{\varepsilon} 1$ and $2 \xrightarrow{\varepsilon} f$ (writing f for q_{acc}). State 1 self-loops on b ; the $1 \rightarrow 2$ edge is a ; state 2 self-loops on $a \cup b$. Now eliminate state 2. For the pair $(1, f)$ take $q_i = 1$, $q_{rip} = 2$, $q_j = f$, so $R_1 = a$ (edge $1 \rightarrow 2$), $R_2 = a \cup b$ (loop at 2), $R_3 = \varepsilon$ (edge $2 \rightarrow f$), and $R_4 = \emptyset$ (no direct $1 \rightarrow f$). The new label is $R_1 R_2^* R_3 \cup R_4 = a(a \cup b)^* \varepsilon \cup \emptyset = a(a \cup b)^*$.



Now eliminate state 1: for the pair (s, f) we have $R_1 = \varepsilon$ (edge $s \rightarrow 1$), $R_2 = b$ (loop at 1), $R_3 = a(a \cup b)^*$ (edge $1 \rightarrow f$), and $R_4 = \emptyset$. The final label is $\varepsilon b^* a(a \cup b)^* \cup \emptyset = b^* a(a \cup b)^*$.

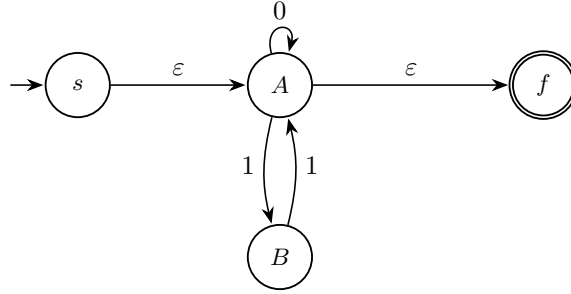
Which idea, and why? State elimination is a systematic DFA-to-regex method; each removed state contributes an $R_1 R_2^* R_3$ term that re-creates the paths through it, so the surviving endpoint edge accumulates exactly the accepted language.

Check. The expression $b^* a(a \cup b)^*$ says: some b 's, then the first a , then anything. That matches “contains at least one a ,” which is exactly what M accepts (you reach accept state 2 the moment you read an a and never leave). The string bba matches; bb does not.

Worked Example 5.2.8 (Convert a three-state cyclic DFA). Let M over $\{0, 1\}$ have states A, B, C with start A and accept $\{A\}$, and transitions $\delta(A, 0) = A$, $\delta(A, 1) = B$, $\delta(B, 1) = A$, $\delta(B, 0) = C$, $\delta(C, 0) = C$, $\delta(C, 1) = C$. Give a regular expression for $L(M)$.

Recognition. State C is a non-accepting trap (both symbols loop and it is never accepting), so it can never contribute to the surviving start-accept edge. The trigger “a trap state” tells us to eliminate C first; its outgoing label to the accept side is $\emptyset$, which annihilates its term.

Solution. Normalize with $s \xrightarrow{\varepsilon} A$ and $A \xrightarrow{\varepsilon} f$ (since A is the only accept). Eliminating C : every edge *into* C followed by C ’s loop and *out* of C contributes $R_1 R_2^* R_3$, but C has no edge to any surviving state ($R_3 = \emptyset$), so each such term is $\emptyset$ and vanishes; we simply delete C and its incoming $B \xrightarrow{0} C$ edge.



What remains (after dropping C) is the loop 0 at A and the two-step cycle $A \xrightarrow{1} B \xrightarrow{1} A$. Eliminate B : the only nontrivial pair is (A, A) , with $R_1 = 1$ ($A \rightarrow B$), $R_2 = \emptyset$ (no loop at B), $R_3 = 1$ ($B \rightarrow A$), and $R_4 = 0$ (the existing A -loop). The new A -self-loop label is $R_1 R_2^* R_3 \cup R_4 = 1 \emptyset^* 1 \cup 0 = 1 \emptyset^* 1 \cup 0 = 11 \cup 0$. Finally eliminate A : for (s, f) we have $R_1 = \varepsilon$, $R_2 = 11 \cup 0$, $R_3 = \varepsilon$, $R_4 = \emptyset$, giving

$$R = (0 \cup 11)^*.$$

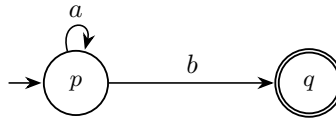
Which idea, and why? Eliminating the trap C first costs nothing because its out-label to the rest is $\emptyset$; then the residual A, B cycle collapses to a single self-loop $11 \cup 0$, which the star wraps into the answer.

Check. $(0 \cup 11)^*$ describes strings made of blocks 0 and 11 – equivalently, strings of 0 ’s and 1 ’s in which the 1 ’s come in even runs and you never hit the trap. Trace 0110 : $A \xrightarrow{0} A \xrightarrow{1} B \xrightarrow{1} A \xrightarrow{0} A \in F$, accepted, and indeed $0110 = 0 \cdot 11 \cdot 0$. Trace 10 : $A \xrightarrow{1} B \xrightarrow{0} C \notin F$, rejected, and 10 is not a block string.

Worked Example 5.2.9 (Both directions at once: a small round trip). Take the regular expression $R = \mathbf{a^*b}$. Build an NFA with Thompson, then read a regular expression back off it by state elimination, and confirm you recover the same language.

Recognition. This is a sanity loop: the trigger is “check that the two constructions are inverse on languages.” We do $\text{regex} \rightarrow \text{NFA}$, then $\text{NFA} \rightarrow \text{regex}$, and compare languages (not necessarily the literal expression).

Solution. Thompson on $\mathbf{a^*b}$: a star gadget over the $\mathbf{a}$ -atom, concatenated with the $\mathbf{b}$ -atom. Collapsing trivial ε -chains, an equivalent small NFA is p (start) with a self-loop $\mathbf{a}$, a $\mathbf{b}$ -edge $p \rightarrow q$, and q accepting.



Now state-eliminate. Add $s \xrightarrow{\varepsilon} p$ and $q \xrightarrow{\varepsilon} f$. Eliminate q (label out is ε): the (p, f) edge becomes $\mathbf{b} \cdot \varepsilon = \mathbf{b}$. Eliminate p : for (s, f) take $R_1 = \varepsilon$, $R_2 = \mathbf{a}$ (the loop), $R_3 = \mathbf{b}$, $R_4 = \emptyset$, giving $\varepsilon \mathbf{a^*b} \cup \emptyset = \mathbf{a^*b}$.

Which idea, and why? Thompson and state elimination are inverse *at the level of languages*: applying one then the other returns a regex describing the same language, even when the literal expression differs by simplification.

Check. We recovered $\mathbf{a^*b}$ verbatim. Either description accepts $\mathbf{b, ab, aab, \dots}$ (any number of $\mathbf{a}$ ’s then one $\mathbf{b}$) and rejects $\mathbf{a, ba,}$ and ε .

Caution. In the star gadget, do not simply mark the body's original start state as accepting and stop there. You must add a *new* start state with an ε -edge into the body and an ε -edge straight to the new accept. Re-using the old start as the accept can pull in extra strings: a back-edge from an accept state to an interior start may let a partial pass through the body be accepted, so the machine would recognize more than $L(R)^*$. The fresh start's direct ε -edge adds exactly ε and nothing else.

Caution. When eliminating a state, you must update *every* ordered pair of survivors, including each state's pair with *itself*. Forgetting the self-pairs is the most common error: a path that leaves q_i , loops through q_{rip} , and returns to q_i becomes a new self-loop on q_i , and if you skip it you silently delete those strings from the language. Likewise keep the order $R_1 R_2^* R_3 \cup R_4$ – the detour term is concatenated *in reading order* (in, loop, out), and it is unioned with, never multiplied by, the direct term R_4 .

Caution. Mind the difference between $\emptyset$ and ε while simplifying labels. The annihilator is $\emptyset$ (so $\emptyset R = R\emptyset = \emptyset$ and $R \cup \emptyset = R$), while the identity for concatenation is ε (so $\varepsilon R = R\varepsilon = R$) and $\emptyset^* = \varepsilon$. Writing ε where a missing edge belongs (it should be $\emptyset$) creates phantom transitions that accept strings the original machine rejects.

Exercises.

1. Using the Thompson construction, build an NFA for the regular expression $\mathbf{a(ab)^*}$. Draw every intermediate gadget (atoms, the inner concatenation, the star, the outer concatenation) and label all ε -edges.
2. Build an NFA for $(0 \cup 1)^* 00$ by Thompson. Then trace your machine on 100 and on 010, listing the set of active states after each symbol, and say which is accepted.
3. Parse each expression by precedence and state its parse tree in words, *without* drawing a machine: (a) $\mathbf{a \cup bc^*}$; (b) $\mathbf{ab \cup c^*}$; (c) $\mathbf{(a \cup b)c^*}$; (d) $\mathbf{ab^*c}$. Then show that no two of the four denote the same language, by exhibiting for each pair a string in one but not the other.
4. Convert the two-state DFA over $\{0, 1\}$ with start 1 (accepting) and second state 2, where $\delta(1, 0) = 2$, $\delta(1, 1) = 1$, $\delta(2, 0) = 2$, $\delta(2, 1) = 1$, to a regular expression by state elimination. Show the GNFA after each elimination.
5. Convert the three-state DFA over $\{a, b\}$ with states q_0 (start), q_1 (accept), q_2 , and transitions $\delta(q_0, \mathbf{a}) = q_1$, $\delta(q_0, \mathbf{b}) = q_0$, $\delta(q_1, \mathbf{a}) = q_1$, $\delta(q_1, \mathbf{b}) = q_2$, $\delta(q_2, \mathbf{a}) = q_1$, $\delta(q_2, \mathbf{b}) = q_0$, to a regular expression. Eliminate q_2 first and show your $R_1 R_2^* R_3 \cup R_4$ computation for the (q_1, q_0) pair.
6. Give a GNFA in normal form, with exactly four states, whose start-to-accept regular expression after full elimination is $(01)^*$. Then eliminate down to two states to confirm.
7. Prove that the order in which you remove the internal states of a GNFA does not change the *language* of the final expression (though it may change the literal expression). One or two sentences citing the correctness argument of Lemma 5.2.6 suffice.
8. Show directly, without state elimination, that $\mathbf{a^* \cup b^*}$ and $\mathbf{(a^* b^*)^*}$ denote *different* languages by exhibiting a string in one but not the other.
9. *Find the bug.* A student applies the star gadget to $R = \mathbf{a^* b}$ by taking the two-state NFA for $\mathbf{a^* b}$ from the round-trip example (start p with an $\mathbf{a}$ -loop, and a $\mathbf{b}$ -edge to the accept q) and just making its start state accepting and adding an ε back-edge from accept to start. Draw their machine, find a string it accepts that is not in $L((\mathbf{a^* b})^*)$, and state the correct fix.
10. *Find the bug.* Eliminating a state q_{rip} , a student writes the new $q_i \rightarrow q_j$ label as $R_4 \cup R_1 R_3$, dropping the R_2^* factor. Give a concrete tiny GNFA (with a self-loop at q_{rip}) on which this wrong rule changes the recognized language, and exhibit a specific string it now mishandles.
11. Let $\mathcal{L}$ be recognized by the NFA over $\{a, b\}$ with start state r_0 , a $\mathbf{b}$ self-loop on r_0 , an $\mathbf{a}$ -edge $r_0 \rightarrow r_1$, a $\mathbf{b}$ -edge $r_1 \rightarrow r_1$, and r_1 accepting. Convert it to a regular expression by state elimination, then describe $\mathcal{L}$ in one plain-English sentence and confirm your sentence against the expression.
12. Use Kleene's theorem to argue, in a short paragraph, that the class of languages denoted by regular expressions is closed under intersection and complement, even though those operations are not among the

three operators allowed in a regular expression. (Hint: convert to a DFA, apply the machine-level closure, convert back.)

5.3 The Pumping Lemma for Regular Languages

Every finite-state machine has a fixed, finite memory, and that single fact has a sharp consequence: on a long enough input the machine must reuse a state, and the loop of input read between the two visits to that state can be traversed any number of times without the machine noticing. The pumping lemma packages this observation into a clean combinatorial property that *every* regular language must have. The one key idea is the pigeonhole principle applied to the sequence of states a DFA passes through: more steps than states forces a repeat, and a repeat forces a pumpable loop. This section states the lemma, proves it from the repeated-state picture, and explains carefully what the elusive pumping length p is and how the decomposition $s = xyz$ must respect it; in the next section we turn the lemma into a tool for proving languages nonregular.

Theorem 5.3.1 (Pumping lemma for regular languages). *If $\mathcal{L}$ is a regular language, then there is an integer $p \geq 1$, called a pumping length for $\mathcal{L}$, such that every string $s \in \mathcal{L}$ with $|s| \geq p$ can be written as $s = xyz$ with*

- (1) $xy^iz \in \mathcal{L}$ for every $i \geq 0$;
- (2) $|y| > 0$; and
- (3) $|xy| \leq p$.

Here y^i denotes i copies of y concatenated, with $y^0 = \varepsilon$.

Condition (1) is the heart of the statement: the middle block y may be deleted ($i = 0$), kept as is ($i = 1$), or repeated any number of times, and the result stays in the language. Condition (2) forbids the empty loop $y = \varepsilon$, without which the lemma would be vacuous (any string could be cut with an empty middle). Condition (3) confines x and y to the first p symbols, which pins the loop down to an early part of s – a technical restriction that does real work in the harder nonregularity proofs. Either of x or z may be empty, but y may not.

Remark. The lemma is a one-way (necessary) condition: *regular* $\Rightarrow$ *pumpable*. It does *not* say that a pumpable language is regular – some nonregular languages satisfy the pumping property too. We use it only in its contrapositive form, *not pumpable* $\Rightarrow$ *not regular*, which is the subject of Section 5.4.

Why a repeat must happen. The proof rests entirely on counting. Run a DFA on a string of length n and record the states it visits: it starts in one state, then makes one move per input symbol, so it occupies $n + 1$ states in all (counting the start). If the machine has only p states and $n \geq p$, then $n + 1 \geq p + 1$ states are listed but only p distinct values are available, so two of the listed states must coincide. That is the pigeonhole principle: $p + 1$ pigeons in p holes force a shared hole. The repeated state is where the loop lives.

Proof. Since $\mathcal{L}$ is regular, fix a DFA $M = (Q, \Sigma, \delta, q_0, F)$ with $L(M) = \mathcal{L}$, and let $p = |Q|$ be its number of states. Take any $s = s_1s_2 \cdots s_n \in \mathcal{L}$ with $n = |s| \geq p$. Let

$$r_1, r_2, \dots, r_{n+1}$$

be the run of M on s : that is, $r_1 = q_0$ and $r_{k+1} = \delta(r_k, s_k)$ for $1 \leq k \leq n$. Because $s \in \mathcal{L}$, the final state $r_{n+1} \in F$.

This sequence has $n + 1 \geq p + 1$ entries, all drawn from the p -element set Q . By the pigeonhole principle, among the first $p + 1$ entries $r_1, \dots, r_{p+1}$ two must be equal; pick the earliest such repetition and call its positions $j < \ell$, so $r_j = r_\ell$ with $\ell \leq p + 1$. Now cut s at these positions:

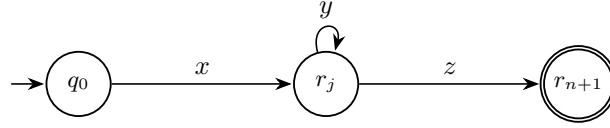
$$x = s_1 \cdots s_{j-1}, \quad y = s_j \cdots s_{\ell-1}, \quad z = s_\ell \cdots s_n.$$

By construction x drives M from $r_1 = q_0$ to r_j ; the block y drives it from r_j back to $r_\ell = r_j$ (a loop at that state); and z drives it from r_j to $r_{n+1} \in F$. We verify the three conditions.

(1) Reading y returns M to the same state r_j , so reading y any number of times also returns to r_j . Hence for every $i \geq 0$ the string xy^iz drives M from q_0 to r_j (via x), loops at r_j exactly i times (via the i copies of y), then proceeds via z to the accept state r_{n+1} . Thus $\delta(q_0, xy^iz) \in F$ and $xy^iz \in \mathcal{L}$.

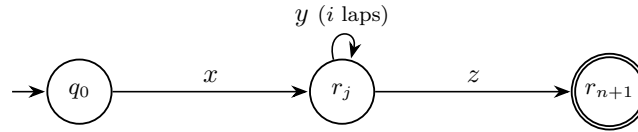
- (2) The positions satisfy $j < \ell$, so $y = s_j \cdots s_{\ell-1}$ has at least one symbol and $|y| = \ell - j > 0$.
(3) Since $\ell \leq p + 1$ we have $|xy| = \ell - 1 \leq p$. □

The decomposition is exactly the loop in the run. The following picture shows the three state-stretches the proof carves out: x to the first visit of the repeated state r_j , then the loop y back to it, then z on to an accept state.



Read the diagram as a compressed run: each labelled edge stands for a whole block of input, and the self-loop at r_j is the part that may be traversed $0, 1, 2, \dots$ times. Pumping “up” adds laps of the loop ($i \geq 2$); pumping “down” to $i = 0$ cuts the loop out entirely, short-circuiting straight from r_j along z .

The correctness of pumping is the statement that this loop preserves *membership*: every lap count i leaves the verdict unchanged, so $xy^iz \in \mathcal{L}$ for all $i \geq 0$.



$$xy^iz \in \mathcal{L} \text{ for all } i \geq 0$$

side conditions: $|xy| \leq p, |y| \geq 1$

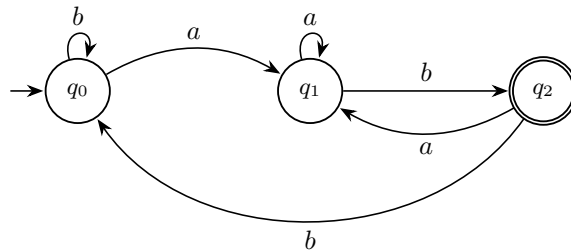
A long accepted string forces a repeated state, hence a loop y ; traversing the loop i times shows xy^iz is accepted for every $i \geq 0$. This is a *necessary* condition on regular languages, used to *disprove* regularity by its contrapositive – it is *not* sufficient (some nonregular languages pump too).

Check. By pigeonhole a run of length $\geq p$ repeats a state within its first p steps, giving $|xy| \leq p$ and $|y| \geq 1$; the loop at r_j may be taken any number of times, so xy^iz reaches the same accept state r_{n+1} for every $i \geq 0$.

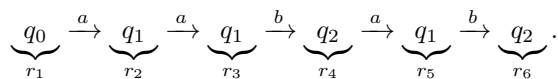
Worked Example 5.3.2 (Watching a concrete machine repeat a state). Let M be the three-state DFA below over $\Sigma = \{a, b\}$, with $L(M) = \{w \mid w \text{ ends in } \mathbf{ab}\}$. Run it on $s = \mathbf{aabab}$ (length $5 > 3 = |Q|$), find the repeated state guaranteed by the proof, and read off a pumping decomposition.

Recognition. The machine has $p = 3$ states and the input has length 5, so the run lists 6 states from a pool of 3: the trigger “more steps than states” guarantees a repeat, and the proof tells us to take the *first* one.

Solution. The machine:



Run $s = \mathbf{aabab}$, recording the state *before* each symbol:



The first repetition is $r_2 = r_3 = q_1$ (positions $j = 2, \ell = 3$). Cutting there gives $x = \mathbf{a}$ (drives $q_0 \rightarrow q_1$), $y = \mathbf{a}$ (loops $q_1 \rightarrow q_1$), $z = \mathbf{bab}$ (drives $q_1 \rightarrow q_2 \in F$). Note $|xy| = 2 \leq p$ and $|y| = 1 > 0$, as promised.

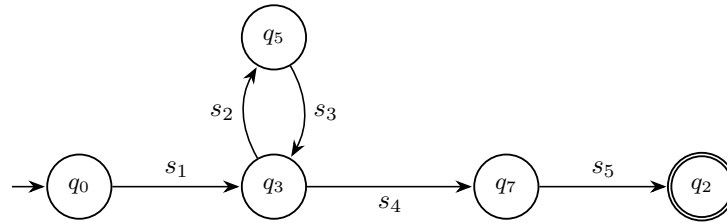
Which idea, and why? The pumping decomposition is nothing more than the first state-repeat in the run: the input between the two visits is exactly the loop y , which is why deleting or duplicating it cannot change the accept/reject verdict.

Check. Pump up: one more lap gives $xy^2z = \mathbf{aaabab}$, run $q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2 \in F$, still ends in $\mathbf{ab}$, accepted. Pump down: $xy^0z = xz = \mathbf{abab}$, run $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_2 \in F$, accepted. Every lap-count lands in $L(M)$.

Worked Example 5.3.3 (Reading the three pieces off the loop picture). Suppose a DFA's run on some accepted string s has the state sequence $q_0, q_3, q_5, q_3, q_7, q_2$ (so $|s| = 5$). Identify x, y, z and describe all strings the lemma certifies are in the language.

Recognition. The proof says: list the run, find the *first* repeated state, and cut at its two positions. The repeat here is q_3 at positions 2 and 4.

Solution. Write the run as a path and mark the loop:



The first repeat is q_3 (positions $j = 2, \ell = 4$), so

$$x = s_1, \quad y = s_2s_3, \quad z = s_4s_5.$$

The loop $q_3 \xrightarrow{s_2} q_5 \xrightarrow{s_3} q_3$ is the block $y = s_2s_3$. The lemma certifies $s_1(s_2s_3)^i s_4s_5 \in \mathcal{L}$ for all $i \geq 0$: that is, $s_1s_4s_5$ (with the loop removed), the original $s_1s_2s_3s_4s_5$, then $s_1s_2s_3s_2s_3s_4s_5$, and so on.

Which idea, and why? We cut at the *earliest* repetition precisely so that condition (3) holds; here the earliest repeat q_3 occurs by position 4, keeping x and y inside the prefix the lemma allows.

Check. Condition (2): $|y| = 2 > 0$. Condition (3): $|xy| = 3$, which is $\leq p$ as long as the machine has at least 3 states – and it must, since the run already shows 5 distinct states q_0, q_3, q_5, q_7, q_2 , so $p \geq 5$.

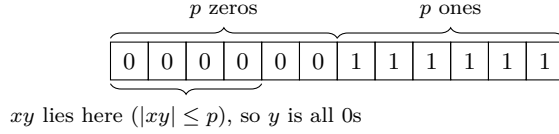
Method (How to apply the pumping lemma).

1. **Get p .** Assume the language is regular and let p be a pumping length from Theorem 5.3.1. You do *not* get to choose p ; treat it as an unknown positive integer and let the adversary fix it.
2. **Choose one string s in the language with $|s| \geq p$,** written in terms of p . Pick s to expose the language's "essence" – the feature a finite memory cannot track. A string whose length grows with p (e.g. 0^p1^p) is usually right.
3. **Invoke the decomposition.** The lemma hands you *some* split $s = xyz$ with $|y| > 0$ and $|xy| \leq p$. You do not pick x, y, z ; you must handle every legal split.
4. **Use condition (3) to pin y down.** Because $|xy| \leq p$, the loop y sits inside the first p symbols of s . If those symbols are all of one kind (say all 0), then y is all 0 – this single observation collapses the case analysis.
5. **Pump to a contradiction.** Choose an i (often $i = 2$ to pump up, or $i = 0$ to pump down) for which xy^iz leaves the language. Since the lemma claimed xy^iz stays, you have a contradiction, so the language is not regular.

Worked Example 5.3.4 (The template applied: $\{0^n1^n \mid n \geq 0\}$ is not pumpable). Show that $\mathcal{L} = \{0^n1^n \mid n \geq 0\}$ fails the pumping property, so by the contrapositive it is not regular.

Recognition. The language demands an exact match between two unbounded counts ($\#0 = \#1$), which a finite memory cannot maintain. The string 0^p1^p puts that balance front and centre.

Solution. Assume $\mathcal{L}$ is regular and let p be a pumping length. Take $s = 0^p 1^p$, which is in $\mathcal{L}$ and has length $2p \geq p$. The lemma gives $s = xyz$ with $|y| > 0$ and $|xy| \leq p$.



Legend: each square cell is one input symbol of $s = 0^p 1^p$; the top braces count the two blocks, the bottom brace marks the region $|xy| \leq p$ in which condition (3) traps the loop y . By condition (3), xy lies within the first p symbols, which are all 0; hence $y = 0^k$ for some k with $1 \leq k \leq p$ (condition (2) gives $k \geq 1$). Pump up to $i = 2$:

$$xy^2z = 0^{p+k} 1^p.$$

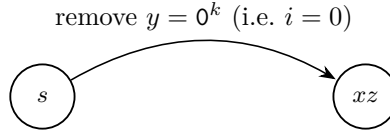
This string has $p + k > p$ zeros but only p ones, so $\#0 \neq \#1$ and $xy^2z \notin \mathcal{L}$. That contradicts condition (1). *Which idea, and why?* Condition (3) is the lever: it forces the loop into the all-0 prefix, so any number of laps changes the zero-count alone and breaks the 0/1 balance the language requires.

Check. Pumping down works identically: $xy^0z = 0^{p-k} 1^p$ has fewer zeros than ones, also outside $\mathcal{L}$. Either direction contradicts the lemma.

Worked Example 5.3.5 (Pumping down: when only $i = 0$ exposes the flaw). Show that $\mathcal{L} = \{0^i 1^j \mid i > j \geq 0\}$ fails the pumping property. Notice that pumping *up* fails to help, so the loop must be removed instead.

Recognition. The language needs strictly more 0s than 1s. A string with a razor-thin margin, just one extra 0, is fragile: removing even one 0 should break it.

Solution. Assume $\mathcal{L}$ regular with pumping length p , and take $s = 0^{p+1} 1^p$, which lies in $\mathcal{L}$ (it has $p + 1 > p$ zeros) and has length $2p + 1 \geq p$. Decompose $s = xyz$ with $|xy| \leq p$, so again xy is inside the leading zeros and $y = 0^k$, $1 \leq k \leq p$.



Pumping *up* to $i = 2$ gives $0^{p+1+k} 1^p$, which still has more 0s than 1s – it stays in $\mathcal{L}$, so no contradiction. Instead pump *down* to $i = 0$:

$$xy^0z = xz = 0^{p+1-k} 1^p.$$

Since $k \geq 1$, the zero-count is $p + 1 - k \leq p$, so xz has at most as many 0s as 1s and fails the requirement $i > j$. Hence $xz \notin \mathcal{L}$, contradicting condition (1).

Which idea, and why? When a language tolerates extra copies of the loop but not its absence, the right move is $i = 0$: “pumping down” deletes y and trips the strict-inequality margin that one 0 was guarding.

Check. If by accident $p + 1 - k = p$ (i.e. $k = 1$) we still get $\#0 = p \not> p = \#1$, so the string is excluded; the contradiction holds for every legal k .

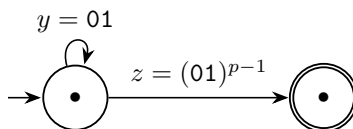
Worked Example 5.3.6 (Why a bad choice of s wastes the lemma). Using $\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ has equally many 0s and 1s}\}$, explain why the candidate $s = (01)^p$ is a poor choice even though it is in $\mathcal{L}$.

Recognition. The lemma only bites if *no* legal split survives pumping. A string that the adversary can split into a harmless loop gives you nothing.

Solution. The string $s = (01)^p = 0101 \dots 01$ has p zeros and p ones, so $s \in \mathcal{L}$ and $|s| = 2p \geq p$. But the adversary may take $x = \varepsilon$, $y = 01$, $z = (01)^{p-1}$ (valid: $|y| = 2 > 0$, $|xy| = 2 \leq p$). Then for *every* i ,

$$xy^i z = (01)^i (01)^{p-1} = (01)^{p-1+i},$$

which has equal 0s and 1s and therefore stays in $\mathcal{L}$.



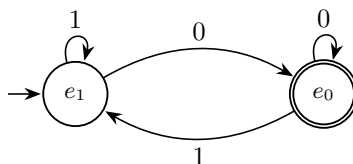
No value of i leaves $\mathcal{L}$, so this s produces no contradiction – not because $\mathcal{L}$ is regular (it is not), but because the choice of s was wrong. The fix is the string $0^p 1^p$ of the earlier example, whose only legal loops are all-0. *Which idea, and why?* The prover, not the adversary, chooses s ; the choice must be a string *every* legal split breaks. A string with a benign internal period (like 01 repeated) hands the adversary a survivable loop, so it cannot force a contradiction.

Check. Contrast the two: $0^p 1^p$ forces y into a single symbol-block by condition (3); $(01)^p$ lets y straddle a full 01 period that pumps cleanly. Same language, very different leverage.

Worked Example 5.3.7 (Pumpable but not all-pumpable: where the lemma is silent). Show that $\mathcal{L} = \{w0^n \mid n \geq 1, w \in \{0,1\}^*\}$ – which is just $\{u \mid u \text{ ends in } 0\}$ – is regular and trace the loop the lemma supplies, to emphasize that the lemma *succeeds* on regular languages.

Recognition. This is a last-symbol condition, recognized by a tiny DFA, so the pumping property must hold. Tracing it shows the lemma is a true statement about regular languages, not a magic nonregularity detector.

Solution. A two-state DFA recognizes “ends in 0”:



It has $p = 2$ states, so take any $s \in \mathcal{L}$ with $|s| \geq 2$, say $s = 100$. Its run is $e_1 \xrightarrow{1} e_1 \xrightarrow{0} e_0 \xrightarrow{0} e_0$, i.e. e_1, e_1, e_0, e_0 , whose first repeat is e_1 at positions 1, 2. So $x = \varepsilon$, $y = 1$, $z = 00$. Pumping gives $1^i 00$ for all $i \geq 0$, every one of which ends in 0 and is accepted.

Which idea, and why? For a regular language the lemma is simply true, and the loop it returns is an honest self-loop in some recognizing DFA; the property is necessary for regularity, so it can never fail on a regular language.

Check. The decomposition obeys all three conditions: $|y| = 1 > 0$ and $|xy| = 1 \leq 2 = p$, and indeed $xy^0 z = 00 \in \mathcal{L}$, $xy^2 z = 1100 \in \mathcal{L}$.

Caution. You do *not* choose the pumping length p , and you do *not* choose the split x, y, z . Both are handed to you adversarially: p is some fixed-but-unknown positive integer, and the decomposition is whatever the lemma supplies, subject only to $|y| > 0$ and $|xy| \leq p$. A correct argument fixes the *string* s (your first free choice) and then defeats *every* legal split. The two choices you actually make are s and the pumping exponent i .

Caution. Condition (3), $|xy| \leq p$, is easy to forget and often essential. Without it the adversary could place the loop y anywhere in s , and many useful candidate strings would then admit a harmless loop straddling a balanced period. Always state where the first p symbols of your s sit, and use condition (3) to conclude that y lies entirely within that homogeneous block.

Caution. The lemma is a necessary condition, not a characterization. Showing that a language *is* pumpable proves nothing about regularity – there exist nonregular languages with the pumping property. The lemma earns its keep only through its contrapositive: exhibiting one unpumpable string proves nonregularity. Never write “ $\mathcal{L}$ can be pumped, therefore $\mathcal{L}$ is regular.”

Exercises.

1. State the pumping lemma in full, then explain in one sentence each why conditions (2) $|y| > 0$ and (3) $|xy| \leq p$ are present – in particular, what goes wrong with the lemma if each is dropped.
2. A DFA M over $\{a, b\}$ has 4 states. Without knowing its transitions, what is the smallest length n guaranteeing that the run of M on *every* input of length n must revisit some state? Justify your answer with the pigeonhole count.

3. Consider the DFA over $\{0, 1\}$ with states q_0, q_1, q_2 , start q_0 , accept $\{q_2\}$, and transitions $\delta(q_0, 0) = q_1$, $\delta(q_0, 1) = q_0$, $\delta(q_1, 0) = q_2$, $\delta(q_1, 1) = q_0$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_2$. Draw its state diagram. Then run it on $s = 10010$, list the full state sequence $r_1, \dots, r_6$, identify the *first* repeated state, and write down the corresponding x, y, z . Verify $|y| > 0$ and $|xy| \leq p$.
4. For the decomposition you found in the previous exercise, write out xy^0z , xy^2z , and xy^3z explicitly, and confirm by tracing that each is accepted by the machine.
5. A run of some DFA on an accepted string of length 6 produces the state sequence

$$q_0, q_4, q_1, q_4, q_1, q_9, q_3.$$

Identify the first repeated state, give the resulting x, y, z (as positions s_k of the input), and describe in words the infinite family of strings the lemma certifies belong to the language.

6. Let $\mathcal{L} = \{0^n 1^{2n} \mid n \geq 0\}$. Using the Method, choose a string s of length $\geq p$, explain why condition (3) forces y to be all 0s, and pick an exponent i that produces a string outside $\mathcal{L}$. Conclude that $\mathcal{L}$ fails the pumping property.
7. Let $\mathcal{L} = \{0^i 1^j \mid i < j\}$. Show this language fails the pumping property, and explain why *pumping up* ($i \geq 2$) – not pumping down – is the move that produces the contradiction. (Compare with the strictly-more-0s language from the section.)
8. A classmate proposes the string $s = 0^p$ to prove that $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ is nonregular. Explain why $s = 0^p$ is not even a member of $\mathcal{L}$ (for $p \geq 1$), so the proof never starts, and state the correct choice of s .
9. *Find the bug.* Here is a flawed “proof” that the regular language $\mathcal{L} = \{w \mid |w| \text{ is even}\}$ is *not* regular: “Let p be a pumping length and take $s = 0^{2p}$. The lemma splits $s = xyz$. Pump to $i = 2$, getting xy^2z with $2p + |y|$ symbols. Choosing $|y| = 1$ makes the length odd, so $xy^2z \notin \mathcal{L}$ – a contradiction.” Identify the precise error in this argument. (Hint: who chooses $|y|$?)
10. Prove directly from Theorem 5.3.1 that every *finite* language $\mathcal{L}$ is pumpable, by exhibiting a pumping length p for which the “if $|s| \geq p$ ” clause is vacuous. What is the smallest such p in terms of the longest string of $\mathcal{L}$?
11. Suppose $\mathcal{L}$ is regular and p is a pumping length for it. Show that $p + 5$ is also a valid pumping length. More generally, argue that any integer $q \geq p$ is a pumping length for $\mathcal{L}$.
12. Let $\mathcal{L} = \{1^n \mid n \text{ is a multiple of } 3\}$, a regular language. Take the smallest string of length ≥ 3 that the lemma applies to, give a valid decomposition x, y, z coming from a recognizing DFA’s loop, and check that $xy^i z \in \mathcal{L}$ for all $i \geq 0$. (This is the lemma *confirming* regularity, not refuting it.)

5.4 Proving Nonregularity

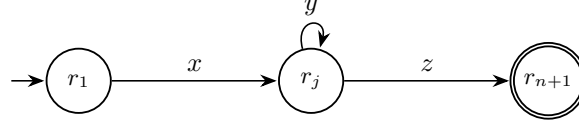
Section 5.3 proved that every regular language has a *pumping length* p : a threshold above which every accepted string contains a loop that can be repeated or deleted without leaving the language. That theorem is interesting in its own right, but its real job is negative. Read in contrapositive, it says that a language with *no* such threshold cannot be regular, and this is by far the most common way to prove that a given language lies outside REGULAR. The key idea of this section is to turn the lemma into a concrete *adversary game*: we play against an opponent who supplies the pumping length and the loop decomposition, and we win – proving nonregularity – by always finding a pump that escapes the language. We then collect the standard nonregular languages, see closure properties used as a labor-saving shortcut, and pin down exactly which steps of the game are ours to choose and which belong to the adversary.

We restate the tool in the form we will use it. It was established in Section 5.3.

Theorem 5.4.1 (Pumping lemma, restated). *If $\mathcal{L}$ is regular, then there is an integer $p \geq 1$ (a pumping length) such that every $s \in \mathcal{L}$ with $|s| \geq p$ can be written $s = xyz$ with*

$$(i) \ xy^i z \in \mathcal{L} \text{ for every } i \geq 0, \quad (ii) \ |y| > 0, \quad (iii) \ |xy| \leq p.$$

The decomposition comes from a state that repeats during the run on s : x is read on the way to the repeated state, y is the loop that returns to it, and z finishes the string. Because the loop can be traversed any number of times (including zero), every xy^iz is also accepted. The picture from Section 5.3 is worth keeping in view, since the whole method is just its contrapositive.



Reading the quantifiers as a game. Theorem 5.4.1 is a nest of alternating quantifiers, and getting the alternation right is the whole skill. Spelled out, “ $\mathcal{L}$ regular” implies

$$\exists p \forall s \in \mathcal{L} (|s| \geq p) \exists x, y, z (s = xyz, |y| > 0, |xy| \leq p) \forall i \geq 0 : xy^iz \in \mathcal{L}.$$

To *refute* regularity we negate this. The negation flips every quantifier, which turns the statement into a two-player game in which the lemma’s existential choices belong to an adversary and its universal choices belong to us:

- **Adversary** names a pumping length p (we may not assume any value for it).
- **We** choose a single string $s \in \mathcal{L}$ with $|s| \geq p$, free to use p in its definition.
- **Adversary** splits $s = xyz$ subject only to $|y| > 0$ and $|xy| \leq p$; we must defeat *every* legal split.
- **We** pick a power $i \geq 0$ and show $xy^iz \notin \mathcal{L}$.

If we have a winning strategy – a recipe that beats every p and every legal split – then the language fails the lemma, so it is not regular. The two moves that are ours, the choice of s and the choice of i , are where all the creativity lives.

Method (The pumping-lemma adversary game (how to prove $\mathcal{L} \notin \text{REGULAR}$)).

1. **Assume regular.** Suppose $\mathcal{L}$ is regular and let p be the pumping length the lemma supplies. Treat p as an unknown constant.
2. **Pick a hard string $s \in \mathcal{L}$ depending on p .** Choose s long enough ($|s| \geq p$) and shaped so that condition (iii), $|xy| \leq p$, traps y inside a controlled region. The classic choice puts a long run of one symbol in the first p positions (e.g. $0^p 1^p$), forcing y to be pure.
3. **Use $|xy| \leq p$ to constrain y .** Since xy occupies the first p characters, y lies within them. Deduce what y can possibly be; often y must be a block of a single symbol.
4. **Pump and escape.** Choose i so that $xy^iz \notin \mathcal{L}$. Pumping *up* ($i = 2$) usually breaks a counting equality by adding to one side; pumping *down* ($i = 0$) is the move when adding stays inside the language but deleting does not.
5. **Conclude.** Every legal split fails, contradicting the lemma. Hence $\mathcal{L}$ is not regular.

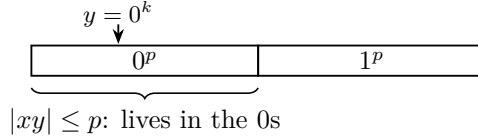
Worked Example 5.4.2 (The canonical example $0^n 1^n$). Prove that $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Recognition. The defining condition is a numeric equality between two unbounded blocks ($\#0\text{s} = \#1\text{s}$). A finite-state device cannot store an unbounded count, which is the textbook signal that the pumping lemma applies. The trigger is “equal numbers of two symbols, each unbounded.”

Solution. Assume $\mathcal{L}$ regular with pumping length p , and play the game. We choose $s = 0^p 1^p \in \mathcal{L}$, which has length $2p \geq p$. The adversary writes $s = xyz$ with $|y| > 0$ and $|xy| \leq p$. The first p characters of s are all 0s, so xy sits entirely inside the 0-block; hence $y = 0^k$ for some $k \geq 1$. Pump up with $i = 2$:

$$xy^2z = 0^{p+k} 1^p, \quad k \geq 1,$$

which has more 0s than 1s and so is not in $\mathcal{L}$. This defeats every legal split, so $\mathcal{L}$ fails the lemma and is not regular.



Which idea, and why? Putting all the 0s first and invoking $|xy| \leq p$ pins y to a block of 0s; one pump-up then unbalances the counts. This is the model the other examples imitate.

Check. The split $x = \varepsilon$, $y = 0^p 1^p$, $z = \varepsilon$ violates $|xy| \leq p$, so the adversary is barred from it; were it allowed, it would need a separate argument (it still fails at $i = 2$, since $(0^p 1^p)^2 \notin \mathcal{L}$) – condition (iii) is what lets one argument cover every legal split.

Worked Example 5.4.3 (Equal counts, anywhere: $C = \{w \mid \#_0(w) = \#_1(w)\}$). Prove that $C = \{w \in \{0, 1\}^* \mid w \text{ has equally many 0s and 1s}\}$ is not regular, and then reprove it in one line.

Recognition. Again an unbounded equality, but now the symbols may be interleaved, so a careless choice of s can backfire. The trigger is the same counting equality; the lesson is that the *string we pick* must exhibit the difficulty, not hide it.

Solution. Assume C regular with pumping length p . The naive instinct is to test $s = (01)^p$, but that string *can* be pumped: with $x = \varepsilon$, $y = 01$, $z = (01)^{p-1}$ every $xy^i z$ keeps the counts equal. A bad s proves nothing. Instead reuse the hard string $s = 0^p 1^p \in C$. By $|xy| \leq p$ the loop is $y = 0^k$ with $k \geq 1$, so $xy^2 z = 0^{p+k} 1^p$ has unequal counts and is not in C . Contradiction.

One-line reproof via closure. If C were regular, then $C \cap L(0^* 1^*)$ would be regular, because REGULAR is closed under intersection (Section 4.2) and $0^* 1^*$ is regular. But that intersection is exactly $\{0^n 1^n \mid n \geq 0\}$, which the previous example showed is nonregular – contradiction.

$$C \in \text{REGULAR} \xrightarrow{\cap L(0^* 1^*)} C \cap L(0^* 1^*) \in \text{REGULAR} = \{0^n 1^n\} \in \text{REGULAR} \Rightarrow \text{contradiction}$$

Which idea, and why? When a new language intersects with a simple regular set to give a language already known to be nonregular, closure under intersection transfers the nonregularity for free – no fresh pumping argument needed.

Check. The closure route depends on $0^* 1^*$ being regular and on the intersection being precisely $0^n 1^n$: a string is in both iff its 0s all precede its 1s *and* the counts match.

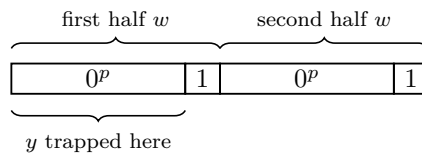
Worked Example 5.4.4 (Duplicated strings: $F = \{ww \mid w \in \{0, 1\}^*\}$). Prove that $F = \{ww \mid w \in \{0, 1\}^*\}$ is not regular.

Recognition. Membership asks whether the first half equals the second half – an unbounded *comparison*, not just a count. The trigger “two halves must match” again exceeds finite memory, and condition (iii) will be essential to localize y .

Solution. Assume F regular with pumping length p . A tempting but useless choice is $s = 0^p 0^p = 0^{2p}$: it is in F (take $w = 0^p$) but it pumps happily, since any 0^{2p+k} with k even still has matching halves. We need a string whose structure a pump can disturb. Take

$$s = 0^p 1 0^p 1 \in F \quad (\text{here } w = 0^p 1).$$

By $|xy| \leq p$, the loop y lies in the leading 0^p block, so $y = 0^k$, $k \geq 1$. Pump up: $xy^2 z = 0^{p+k} 1 0^p 1$. For this to be in F it must split into two equal halves, but its two “1”-marked blocks now have different numbers of leading 0s ($p+k$ versus p), so no halving makes the halves agree. Hence $xy^2 z \notin F$, a contradiction.



Which idea, and why? Inserting separator 1s makes the two halves *visible*; pumping the first 0-block changes only the first half, breaking the halves-match condition. The marker turns a vague comparison into a count the pump can attack.

Check. The bad choice 0^{2p} fails precisely because deleting or adding 0s there keeps both halves all-zero; the markers are what prevent that escape.

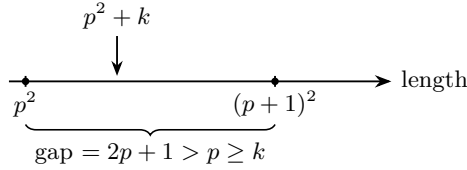
Worked Example 5.4.5 (A unary language: perfect squares $D = \{1^{n^2} \mid n \geq 0\}$). Prove that $D = \{1^m \mid m \text{ is a perfect square}\}$ is not regular.

Recognition. Here the alphabet is unary, so “counting” arguments by symbol type are unavailable; the obstruction is the *spacing* of the lengths. The trigger is a length set whose gaps grow without bound, so a fixed-size pump cannot always land on a valid length.

Solution. Assume D regular with pumping length p . Choose $s = 1^{p^2} \in D$, length $p^2 \geq p$. Any split $s = xyz$ has $|y| = k$ with $1 \leq k \leq p$ (by $|y| \leq |xy| \leq p$). Pump up to $xy^2z = 1^{p^2+k}$. Now compare its length with the nearest squares:

$$p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2.$$

So $p^2 + k$ lies strictly between the consecutive squares p^2 and $(p+1)^2$ and cannot itself be a square. Therefore $xy^2z \notin D$, a contradiction.



Which idea, and why? For a unary language the pump adds a bounded amount $k \leq p$, but the gap between consecutive squares near p^2 is $2p + 1 > p$; a bounded step cannot bridge a widening gap, so the pumped length is never a square.

Check. With $p = 3$, $s = 1^9$; pumping by $k \leq 3$ gives length 10, 11, or 12, all strictly between $9 = 3^2$ and $16 = 4^2$ – none a square.

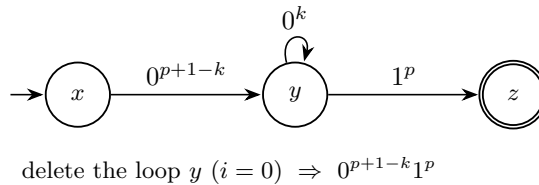
Worked Example 5.4.6 (When you must pump down: $E = \{0^i 1^j \mid i > j\}$). Prove that $E = \{0^i 1^j \mid i > j \geq 0\}$ is not regular.

Recognition. A strict inequality between two unbounded counts. The instinct is to pump up, but adding 0s only makes $i > j$ *more* true, so up-pumping fails to escape. The trigger “adding keeps me inside” tells us to try $i = 0$, i.e. pump *down*.

Solution. Assume E regular with pumping length p . Choose the tight string $s = 0^{p+1}1^p \in E$ (one more 0 than 1). By $|xy| \leq p$ the loop lies in the 0-block, so $y = 0^k$, $k \geq 1$. Pumping up gives $0^{p+1+k}1^p$, still in E – no help. So pump *down* to $i = 0$:

$$xy^0z = xz = 0^{p+1-k}1^p.$$

Since $k \geq 1$, the 0-count drops to $p+1-k \leq p$, so it is no longer strictly greater than the 1-count p . Hence $xz \notin E$, a contradiction.



Which idea, and why? When pumping up stays inside the language because it reinforces the inequality, the escape is to pump down to $i = 0$; deleting the 0-loop erases the surplus 0 and violates $i > j$.

Check. The surplus here is exactly one 0; removing even a single 0 (any $k \geq 1$) destroys $i > j$, which is why the tight string $0^{p+1}1^p$ is the right choice rather than $0^{2p}1^p$.

Worked Example 5.4.7 (Closure as a shortcut: $\{0^n 1^m \mid n \neq m\}$). Prove that $\mathcal{L} = \{0^n 1^m \mid n \neq m, n, m \geq 0\}$ is not regular, with as little pumping as possible.

Recognition. A direct pumping argument here is fiddly, because $n \neq m$ has many escape routes. The trigger is that $\mathcal{L}$ relates to the known-nonregular $0^n 1^n$ through Boolean operations, so closure properties can do the work.

Solution. Suppose $\mathcal{L}$ is regular. The class REGULAR is closed under complement and intersection (Section 4.2). The set $0^* 1^*$ is regular. Consider

$$\overline{\mathcal{L}} \cap L(0^* 1^*).$$

A string of the form $0^n 1^m$ is in $\overline{\mathcal{L}}$ exactly when it is *not* of the shape “0s then 1s with $n \neq m$ ” – but inside $0^* 1^*$ every string already has that shape, so it survives iff $n = m$. Thus the displayed language equals $\{0^n 1^n \mid n \geq 0\}$. If $\mathcal{L}$ were regular this would be regular too, contradicting the canonical example.

$$\mathcal{L} \in \text{REGULAR} \xrightarrow{\text{complement}} \overline{\mathcal{L}} \in \text{REGULAR} \xrightarrow{\cap 0^* 1^*} \{0^n 1^n\} \in \text{REGULAR} \implies \text{contradiction}$$

Which idea, and why? Build the target $0^n 1^n$ out of $\mathcal{L}$ using only operations under which REGULAR is closed (here complement then intersect with $0^* 1^*$); regularity of $\mathcal{L}$ would then force regularity of $0^n 1^n$, which is false.

Check. Inside $0^* 1^*$, the complement of “ $n \neq m$ ” is “ $n = m$,” so the construction lands exactly on $0^n 1^n$ and nothing else.

Method (Proving nonregularity by closure (no pumping)).

1. **Find a known nonregular target** T (typically $0^n 1^n$, ww , or perfect squares).
2. **Express T from your language $\mathcal{L}$** using only operations that preserve REGULAR: union, intersection, complement, concatenation, star, reversal, and intersection with a fixed regular set. A frequent move is $\mathcal{L} \cap R$ for a simple regular R that “filters” $\mathcal{L}$ down to a clean shape.
3. **Argue by contradiction.** If $\mathcal{L} \in \text{REGULAR}$, applying those closed operations would make $T \in \text{REGULAR}$. Since $T \notin \text{REGULAR}$, conclude $\mathcal{L} \notin \text{REGULAR}$.
4. **Caveat.** This works only when the operations are genuinely closure operations. Closure does *not* run backwards: $T \notin \text{REGULAR}$ alone tells you nothing about a language that merely *contains* or is *contained in* T .

Caution. The single most common error is letting the *adversary* pick the string s , or picking an s that happens to be pumpable. *You* choose s , and you must choose one that *cannot* be pumped; a pumpable witness (like $(01)^p$ for the equal-counts language, or 0^{2p} for ww) proves nothing at all. Aim for a string that displays the “essence” of the difficulty, usually with a long pure run in the first p positions.

Caution. Do not forget that you must defeat *every* legal split $s = xyz$, not one convenient one. Condition (iii), $|xy| \leq p$, is your friend here: it forces y into the first p symbols, so a single argument often covers all splits at once. Dropping condition (iii) – treating y as if it could sit anywhere – is what makes proofs like the equal-counts and ww examples appear to fail.

Caution. The pumping lemma is a one-way implication: regular $\Rightarrow$ pumpable. Passing the pumping test does *not* prove a language regular. There are nonregular languages that satisfy the pumping condition for some p , so “I found a p that works” is never a proof of regularity – only a failed attempt at a nonregularity proof.

Exercises.

1. Use the pumping-lemma adversary game to prove that $\mathcal{L} = \{0^n 1^{2n} \mid n \geq 0\}$ is not regular. State your choice of s , explain how $|xy| \leq p$ constrains y , and give the pumping power i you use.
2. Prove that $\mathcal{L} = \{a^n b^m c^n \mid n, m \geq 0\}$ is not regular. Choose a string that traps the loop in the leading a -block, and say why pumping there breaks membership.
3. Prove that the language of *balanced* parenthesis strings over $\{(,)\}$ (every prefix has at least as many $($ as $)$, and the totals are equal) is not regular.

4. Show that $\mathcal{L} = \{1^n \mid n \text{ is prime}\}$ is not regular. (Hint: pick $s = 1^q$ for a prime $q \geq p + 2$, let $|y| = k$, and find a value of i for which the length $q + (i - 1)k$ is composite; trying $i = q + 1$ makes the length divisible by a factor you can name.)
5. Prove that $\mathcal{L} = \{ww^R \mid w \in \{0, 1\}^*\}$ (even-length palindromes) is not regular. Choose a string whose middle the loop cannot reach.
6. Using closure properties and a single known nonregular language, give a one-paragraph proof that $\mathcal{L} = \{w \in \{0, 1\}^* \mid \#_0(w) > \#_1(w)\}$ is not regular. Name every closure property you use.
7. Prove that $\mathcal{L} = \{0^i 1^j \mid i \neq j\}$ is not regular in two different ways: (a) directly with the pumping lemma, deciding whether to pump up or down; and (b) by intersecting $\mathcal{L}$ with a fixed regular set. Which proof is shorter, and why?
8. Let $\mathcal{L} = \{0^m 1^n \mid m \leq n\}$. Determine, with proof, whether pumping *up* or pumping *down* yields a contradiction, and write the proof out using the move that works.
9. *Find the bug.* A student writes: “ $\mathcal{L} = \{0^n 1^n\}$ is regular, because I can take $p = 1$, $s = 01$, $x = 0$, $y = 1$, $z = \varepsilon$; then $xy^i z = 01^i \in \mathcal{L}$ only for $i = 1$, but the lemma just needs *some* split that pumps, so it is satisfied.” Identify every misuse of the pumping lemma in this argument (direction of the implication, who chooses what, and the meaning of condition (i)).
10. *Find the bug.* Another student proves $\{ww \mid w \in \{0, 1\}^*\}$ nonregular by choosing $s = 0^{2^p}$ and pumping. Explain why this choice cannot yield a contradiction, and supply a string that can.
11. Show that regular languages *are* closed under the operation

$$\text{half}(\mathcal{L}) = \{x \mid xy \in \mathcal{L} \text{ for some } y \text{ with } |x| = |y|\},$$

even though its definition matches two lengths. (Hint: from a DFA for $\mathcal{L}$, build a DFA whose state after reading x records the state reached on x together with the set of states from which an accept state can be reached in exactly $|x|$ steps.)

12. Decide whether each language is regular; prove your answer. (a) $\{0^n 1^m \mid n + m \leq 100\}$; (b) $\{0^n 1^m \mid n + m \text{ is even}\}$; (c) $\{0^n 1^m \mid nm \equiv 0 \pmod{2}\}$; (d) $\{0^n 1^m \mid n = m\}$. Exactly one of the four is nonregular – find it and prove it, and give DFAs or regular expressions for the other three.

5.5 Myhill–Nerode, the Syntactic Monoid, and Minimization

The pumping lemma told us a language can fail to be regular; it never told us how *much* memory a regular language truly needs, nor whether a given DFA is wasteful. This section answers both at once. The single idea is that a language $\mathcal{L}$ induces, all by itself, an equivalence relation on Σ^* – two strings are equivalent when no suffix can ever tell them apart – and the number of classes of that relation is exactly the number of states the smallest DFA for $\mathcal{L}$ must have. Finitely many classes means regular; infinitely many means nonregular. Reading the relation algebraically, it is a right-congruence on the free monoid Σ^* , its two-sided counterpart has classes forming a quotient monoid, and “regular” becomes “recognized by a finite monoid.” This is the bridge from automata to the algebra of Dummit & Foote: index of a subgroup becomes index of a congruence, and the quotient group becomes the syntactic monoid.

The Myhill–Nerode relation

Definition 5.5.1 (Distinguishing extension; Myhill–Nerode relation). Fix a language $\mathcal{L} \subseteq \Sigma^*$. A string $z \in \Sigma^*$ *distinguishes* x from y if exactly one of xz, yz lies in $\mathcal{L}$. Define $x \sim_{\mathcal{L}} y$ to mean that *no* string distinguishes them:

$$x \sim_{\mathcal{L}} y \iff (\forall z \in \Sigma^*) [xz \in \mathcal{L} \iff yz \in \mathcal{L}].$$

We call $\sim_{\mathcal{L}}$ the *Myhill–Nerode relation* of $\mathcal{L}$, and write $[x]$ for the class of x . The number of classes, $|\Sigma^* / \sim_{\mathcal{L}}|$, is the *index* of $\mathcal{L}$.

The relation $\sim_{\mathcal{L}}$ is an equivalence relation: reflexivity and symmetry are immediate, and transitivity holds because if no z separates x from y and none separates y from w , then none separates x from w . Two further properties make it the right tool.

Proposition 5.5.2 ($\sim_{\mathcal{L}}$ is a right-congruence that refines membership). *For every $x, y \in \Sigma^*$ and every $a \in \Sigma$:*

1. (right-congruence) *if $x \sim_{\mathcal{L}} y$ then $xa \sim_{\mathcal{L}} ya$, and more generally $xu \sim_{\mathcal{L}} yu$ for all $u \in \Sigma^*$;*
2. (saturation) *if $x \sim_{\mathcal{L}} y$ then $x \in \mathcal{L} \iff y \in \mathcal{L}$, so each class lies entirely inside $\mathcal{L}$ or entirely outside it.*

Proof. (1) Suppose $x \sim_{\mathcal{L}} y$ and let $a \in \Sigma$. For any z , the string az is itself a candidate suffix for the pair (x, y) , so $x(az) \in \mathcal{L} \iff y(az) \in \mathcal{L}$; regrouping, $(xa)z \in \mathcal{L} \iff (ya)z \in \mathcal{L}$. As z was arbitrary, $xa \sim_{\mathcal{L}} ya$. Iterating one symbol at a time gives $xu \sim_{\mathcal{L}} yu$ for every u . (2) Take $z = \varepsilon$ in Definition 5.5.1: $x\varepsilon = x$ and $y\varepsilon = y$, so $x \in \mathcal{L} \iff y \in \mathcal{L}$. $\square$

Property (1) is exactly the statement that $\sim_{\mathcal{L}}$ is compatible with *appending on the right*, which is how a machine consumes input. It is the language-side shadow of a congruence in the sense of Dummit & Foote: a partition of a monoid that respects the operation. We make that algebraic reading precise in the last part of the section.

The Myhill–Nerode theorem

Theorem 5.5.3 (Myhill–Nerode). *A language $\mathcal{L} \subseteq \Sigma^*$ is regular if and only if its Myhill–Nerode relation $\sim_{\mathcal{L}}$ has finitely many classes. When $\mathcal{L}$ is regular, the index of $\mathcal{L}$ equals the number of states of the smallest DFA recognizing $\mathcal{L}$, and that minimal DFA is unique up to renaming states.*

Proof. ($\Rightarrow$) Let $M = (Q, \Sigma, \delta, q_0, F)$ be any DFA with $L(M) = \mathcal{L}$. Define a map sending a string x to the state it reaches, $x \mapsto \widehat{\delta}(q_0, x)$. If two strings reach the same state, then from that state every continuation z has the same fate, so $\widehat{\delta}(q_0, x) = \widehat{\delta}(q_0, y)$ forces $x \sim_{\mathcal{L}} y$. Hence each $\sim_{\mathcal{L}}$ -class is a union of preimages and there are at most $|Q|$ classes: $|\Sigma^*/\sim_{\mathcal{L}}| \leq |Q| < \infty$. In particular the index is finite, and it is $\leq$ the size of *every* DFA for $\mathcal{L}$.

($\Leftarrow$) Suppose the index is finite. We build a DFA directly from the classes – the *quotient automaton* $M_{\mathcal{L}}$:

$$Q = \Sigma^*/\sim_{\mathcal{L}}, \quad q_0 = [\varepsilon], \quad \delta([x], a) = [xa], \quad F = \{[x] \mid x \in \mathcal{L}\}.$$

The transition δ is well defined precisely by the right-congruence property (Proposition 5.5.2(1)): if $[x] = [y]$ then $[xa] = [ya]$, so the rule does not depend on the representative. The accept set is well defined by saturation (Proposition 5.5.2(2)). A trivial induction gives $\widehat{\delta}([\varepsilon], w) = [w]$, so $M_{\mathcal{L}}$ accepts w iff $[w] \in F$ iff $w \in \mathcal{L}$. Thus $M_{\mathcal{L}}$ recognizes $\mathcal{L}$ with exactly (index-many) states, and $\mathcal{L}$ is regular.

(Minimality and uniqueness.) The first half showed every DFA for $\mathcal{L}$ has at least index-many states, and $M_{\mathcal{L}}$ meets that bound, so $M_{\mathcal{L}}$ is smallest. For uniqueness, in any minimal DFA the state-reaching map $x \mapsto \widehat{\delta}(q_0, x)$ must be a surjection onto the reachable states whose fibres are exactly the $\sim_{\mathcal{L}}$ -classes (no class may be split over two states, or we could merge them and shrink; every state must be reachable, or we could delete it). The induced bijection between classes and states is an isomorphism of DFAs onto $M_{\mathcal{L}}$, so the minimal machine is unique up to renaming. $\square$

Remark. The forward direction is the practical nonregularity test of the previous section in sharper form: to prove $\mathcal{L}$ nonregular, exhibit an *infinite* set of pairwise distinguishable strings $x_0, x_1, x_2, \dots$ (each pair separated by some suffix). No two can share a state in any DFA, so no finite DFA exists. This often beats the pumping lemma because it needs no game against an adversary p .

The two directions of the theorem say the same thing structurally: *minimization is literally a quotient construction*. Collapsing Σ^* by the right-congruence $\sim_{\mathcal{L}}$ produces the classes $[w]$, and those classes *are* the states of the minimal DFA – the bijection is the content of the theorem.

$$\begin{array}{ccc} \Sigma^* & \xrightarrow{\sim_{\mathcal{L}}} & \Sigma^*/\sim_{\mathcal{L}} \xrightarrow[\substack{\cong \\ [w] \mapsto \widehat{\delta}([\varepsilon], w)}}{Q_{\min} \text{ (minimal-DFA states)}} \\ w \longmapsto & & [w] \longmapsto \text{state reached on } w \end{array}$$

The quotient map $w \mapsto [w]$ is a surjection (every class is some $[w]$); the second arrow is a bijection onto $Q_{\min}$ because two strings reach the same minimal state iff no suffix distinguishes them. The language $\mathcal{L}$ is the invariant carried along: each class lies entirely inside or outside $\mathcal{L}$ (Proposition 5.5.2(2)), so “accepting” is well defined on $Q_{\min}$. Minimization is a quotient: the minimal DFA’s states are exactly the Myhill–Nerode classes $\Sigma^*/\sim_{\mathcal{L}}$, and two strings collapse to one state iff no suffix distinguishes them. The number of classes is *finite exactly when* $\mathcal{L}$ is regular – finiteness of the index is the regularity criterion.

Check. $\sim_{\mathcal{L}}$ is a right-congruence (Proposition 5.5.2(1)), so the quotient automaton on $\Sigma^*/\sim_{\mathcal{L}}$ is well defined; $\mathcal{L}$ is regular iff $\sim_{\mathcal{L}}$ has finite index, and that quotient automaton is the minimal DFA.

Worked examples: counting classes

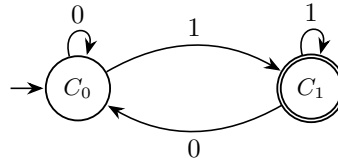
Worked Example 5.5.4 (The index of “ends in 1”). Over $\Sigma = \{0, 1\}$ let $\mathcal{L} = \{w \mid w \text{ ends in } 1\}$. Find the $\sim_{\mathcal{L}}$ -classes and the minimal DFA.

Recognition. We are asked for the Myhill–Nerode classes, so group strings by their *future behaviour*: $x \sim_{\mathcal{L}} y$ exactly when appending any suffix lands both in or both out of $\mathcal{L}$.

Solution. Membership of xz depends only on the last symbol of xz , which (for nonempty z) depends only on z , and for $z = \varepsilon$ on the last symbol of x . So the only thing about x that matters downstream is its last symbol: “ends in 1” versus “ends in 0 or is empty.” That gives two classes,

$$C_1 = \{x \mid x \text{ ends in } 1\}, \quad C_0 = \{x \mid x = \varepsilon \text{ or } x \text{ ends in } 0\}.$$

They are distinguished by $z = \varepsilon$ (members of C_1 are in $\mathcal{L}$, members of C_0 are not). Reading the quotient automaton off the classes – $\delta([x], a) = [xa]$ – gives the two-state minimal DFA:



Which idea, and why? The Myhill–Nerode theorem says the classes *are* the states of the minimal DFA; two classes here, so the minimal machine has exactly two states and no smaller DFA exists.

Check. The two strings 0 and 1 are distinguished by $z = \varepsilon$ ($1 \in \mathcal{L}$, $0 \notin \mathcal{L}$), so they need different states – two is forced.

Worked Example 5.5.5 (An infinite index: $0^n 1^n$ is not regular). Let $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ over $\Sigma = \{0, 1\}$. Use Myhill–Nerode to show $\mathcal{L}$ is nonregular.

Recognition. Nonregularity via the index: find infinitely many pairwise-distinguishable strings, so $\sim_{\mathcal{L}}$ has infinite index and Theorem 5.5.3 forbids a DFA.

Solution. Consider the family $0^0, 0^1, 0^2, \dots$, i.e. $x_i = 0^i$. For $i \neq j$, the suffix $z = 1^i$ distinguishes them: $x_i z = 0^i 1^i \in \mathcal{L}$ but $x_j z = 0^j 1^i \notin \mathcal{L}$ (unequal block counts). Hence $x_i \not\sim_{\mathcal{L}} x_j$ whenever $i \neq j$, so the strings $\{0^i \mid i \geq 0\}$ fall into infinitely many distinct classes. The matrix below tabulates, for small i and the test suffix $z = 1^j$, whether $x_i z \in \mathcal{L}$ (■) or not (×); any two rows differ in some column, so every pair is split:

$x_i \setminus z$	1^1	1^2	1^3	1^4
0^1	■	×	×	×
0^2	×	■	×	×
0^3	×	×	■	×
0^4	×	×	×	■

Which idea, and why? An infinite set of pairwise-distinguishable strings forces infinite index; by Myhill–Nerode infinite index means no finite DFA, hence nonregular – no pumping game needed.

Check. A distinguishing column for the pair $(0^2, 0^3)$ is $z = 1^2$: $0^2 1^2 \in \mathcal{L}$ while $0^3 1^2 \notin \mathcal{L}$. The diagonal of ■’s shows all rows are different.

DFA minimization

The quotient automaton $M_{\mathcal{L}}$ is built from $\mathcal{L}$ abstractly, but in practice we start with a concrete DFA and want to *shrink* it. Two states are redundant when they have the same future, and we merge them. This is the same equivalence as $\sim_{\mathcal{L}}$, now read on states.

Definition 5.5.6 (Indistinguishable states). States p, q of a DFA are *distinguishable* if some string z leads from one to an accept state and from the other to a non-accept state: exactly one of $\widehat{\delta}(p, z), \widehat{\delta}(q, z)$ is in F . Otherwise p, q are *indistinguishable*, written $p \equiv q$.

Method (The table-filling algorithm (find all distinguishable pairs)).

1. **Discard unreachable states** (states no input can reach from q_0); they never affect $\mathcal{L}$.
2. **Mark the obvious pairs.** For every pair $\{p, q\}$ with one accepting and one not, mark it distinguishable (witness $z = \varepsilon$).
3. **Propagate.** Repeatedly scan unmarked pairs $\{p, q\}$. If for some symbol a the pair $\{\delta(p, a), \delta(q, a)\}$ is already marked, mark $\{p, q\}$ (witness: a followed by the witness for the target pair).
4. **Stop** when a full scan marks nothing new. Every unmarked pair is indistinguishable; merge each $\equiv$ -class into one state.

Worked Example 5.5.7 (Minimizing a four-state DFA with two redundant states). Minimize the DFA M over $\Sigma = \{0, 1\}$ with states $\{A, B, C, D\}$, start A , accepts $\{C, D\}$, and

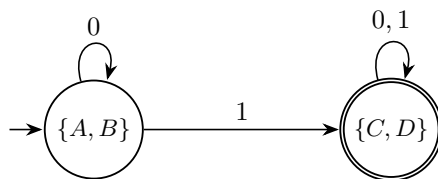
δ	0	1
A	B	C
B	A	D
C	C	C
D	D	D

Recognition. Minimization: run table-filling to find indistinguishable pairs, then collapse each class to one state. The trigger is “two accept states that behave identically forever.”

Solution. All four states are reachable (A start; $0 \rightarrow B$, $1 \rightarrow C$, $01 \rightarrow D$). Both C and D are sink-accepts: every symbol loops them and they are accepting, so no suffix ever leaves F from either – they are indistinguishable, $C \equiv D$. The non-accepts A, B differ from C, D at $z = \varepsilon$. Is $A \equiv B$? On 1, $A \rightarrow C$ (accept) but $B \rightarrow D$ (accept), no split; on 0, $A \rightarrow B$, $B \rightarrow A$, still unsplit; every continuation keeps both in the non-accept region until it first reads a 1, after which both sit in the merged accept sink. So $A \equiv B$ as well. The table-filling grid (lower triangle; $\times$ = marked distinguishable, blank = indistinguishable) is:

	A	B	C
B			
C	$\times$	$\times$	
D	$\times$	$\times$	

Merging the classes $\{A, B\}$ and $\{C, D\}$ yields a two-state minimal DFA with start $\{A, B\}$ (non-accepting) and sink $\{C, D\}$ (accepting); a 1 moves to the accept sink, a 0 stays:



Which idea, and why? Two states are merged exactly when no suffix tells them apart; table-filling marks every distinguishable pair, so whatever stays unmarked is safe to collapse, and the result has index-many states – it is minimal.

Check. The recognized language is “contains at least one 1.” From start $\{A, B\}$, every 0 loops, the first 1 enters the accept sink and stays – exactly the strings with a 1. The four-state original recognized the same language with two wasted states.

The transition monoid and the syntactic monoid

So far each string acted by *moving a start vector to a state*. A richer view lets each string act on *all* states at once: a string w induces the function $\rho_w: Q \rightarrow Q$, $\rho_w(q) = \hat{\delta}(q, w)$, telling where every state goes under w . These functions multiply by composition, and reading w then v composes their actions.

Definition 5.5.8 (Transition monoid of a DFA). For a DFA $M = (Q, \Sigma, \delta, q_0, F)$, the *transition monoid* $T(M)$ is the set of functions $\{\rho_w \mid w \in \Sigma^*\}$ under composition, with identity $\rho_\varepsilon = \text{id}_Q$. The map $w \mapsto \rho_w$ is a monoid homomorphism $\Sigma^* \rightarrow T(M)$, and $T(M)$ is finite (at most $|Q|^{|Q|}$ elements).

This is the automata-theoretic analogue of a group acting on a set in Dummit & Foote: the free monoid Σ^* acts on Q , and $T(M)$ is the image of that action – the monoid of “moves the machine can actually perform.” Stripping the choice of machine away gives an invariant of the *language* alone.

Definition 5.5.9 (Syntactic congruence and syntactic monoid). For a language $\mathcal{L} \subseteq \Sigma^*$, declare $u \approx_{\mathcal{L}} v$ when u and v are interchangeable in *every* context: for all $x, y \in \Sigma^*$,

$$xuy \in \mathcal{L} \iff xvy \in \mathcal{L}.$$

This $\approx_{\mathcal{L}}$ is the *syntactic congruence*: an equivalence relation respecting multiplication on *both* sides ($u \approx_{\mathcal{L}} v \Rightarrow xuy \approx_{\mathcal{L}} xvy$). The quotient $M(\mathcal{L}) = \Sigma^* / \approx_{\mathcal{L}}$ is a monoid under $[u][v] = [uv]$, the *syntactic monoid* of $\mathcal{L}$.

Theorem 5.5.10 (Algebraic characterization of regularity). *A language $\mathcal{L}$ is regular if and only if its syntactic monoid $M(\mathcal{L})$ is finite. Equivalently, $\mathcal{L}$ is regular iff some homomorphism $\varphi: \Sigma^* \rightarrow M$ onto a finite monoid M recognizes $\mathcal{L}$, meaning $\mathcal{L} = \varphi^{-1}(S)$ for some $S \subseteq M$.*

Proof. ($\Rightarrow$) If $\mathcal{L}$ is regular, take its minimal DFA $M_{\mathcal{L}}$; its transition monoid $T(M_{\mathcal{L}})$ is finite, and one checks $u \approx_{\mathcal{L}} v$ iff $\rho_u = \rho_v$ in $T(M_{\mathcal{L}})$ (two strings act the same on every state exactly when they are interchangeable in every context). So $M(\mathcal{L}) \cong T(M_{\mathcal{L}})$ is finite, and $\varphi(w) = \rho_w$ with $S = \{m \mid m([\varepsilon]) \in F\}$ recognizes $\mathcal{L}$. ($\Leftarrow$) If $\varphi: \Sigma^* \rightarrow M$ onto a finite M recognizes $\mathcal{L} = \varphi^{-1}(S)$, build a DFA with state set M , start $\varphi(\varepsilon)$, transition $\delta(m, a) = m \cdot \varphi(a)$, and accept set S . It reaches $\varphi(w)$ on input w , so it accepts exactly $\varphi^{-1}(S) = \mathcal{L}$; finitely many states means $\mathcal{L}$ is regular. $\square$

This is the cleanest statement of regularity: “recognizable by a finite monoid.” It mirrors the group-theory slogan that a finite-index subgroup yields a finite quotient – here a finite-index congruence on Σ^* yields the finite quotient monoid $M(\mathcal{L})$.

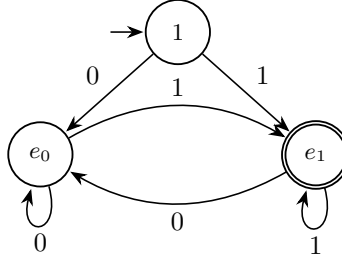
Worked Example 5.5.11 (The syntactic monoid of “ends in 1”). Compute $M(\mathcal{L})$ for $\mathcal{L} = \{w \mid w \text{ ends in } 1\}$ over $\Sigma = \{0, 1\}$.

Recognition. We want the syntactic monoid, so quotient Σ^* by $\approx_{\mathcal{L}}$: lump strings that are interchangeable in every two-sided context, then read off the multiplication.

Solution. What a string contributes in context $x \square y$ is captured by its action on the two-state minimal DFA (states C_0, C_1 from the earlier example). Each string acts as one of: $\rho_\varepsilon = \text{id}$ (identity); $\rho_0 =$ “go to C_0 ” (a constant map to the non-accept state); $\rho_1 =$ “go to C_1 ” (constant to accept). There are no others: any string ending in 0 acts like ρ_0 , any ending in 1 like ρ_1 , and only ε acts as the identity. So $M(\mathcal{L}) = \{1, e_0, e_1\}$ with $1 = [\varepsilon]$, $e_0 = [0]$, $e_1 = [1]$, and the Cayley table (row $\cdot$ column, where the result is determined by the *last* factor’s symbol):

$\cdot$	1	e_0	e_1
1	1	e_0	e_1
e_0	e_0	e_0	e_1
e_1	e_1	e_0	e_1

The accepting set is $S = \{e_1\}$ (a word is in $\mathcal{L}$ iff its class is e_1), and indeed e_0, e_1 are idempotent constant maps ($e_0e_0 = e_0, e_1e_1 = e_1$) – the rightmost symbol wins.



Which idea, and why? The syntactic monoid is the transition monoid of the minimal DFA; for this language each string collapses to “last symbol seen” (plus the empty word), giving three elements with the right factor dominating.

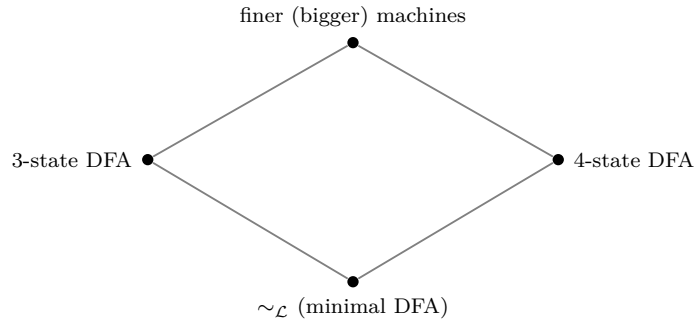
Check. $e_0 \cdot e_1 = e_1$ matches $0 \cdot 1 = 01$, which ends in 1 (class e_1). $e_1 \cdot e_0 = e_0$ matches 10 , ending in 0 (class e_0). The diagram (the right Cayley graph: nodes = monoid elements, edges = right-multiply by 0 or 1) returns the same table.

The algebra thread: congruence, index, quotient

The three relations of this section are one idea at three altitudes. Listing them beside their Dummit & Foote counterparts:

automata object	relation on Σ^*	D&F analogue
states of minimal DFA	$\sim_{\mathcal{L}}$ (right-congruence)	cosets Hg of a subgroup
transition / syntactic monoid	$\approx_{\mathcal{L}}$ (two-sided congruence)	quotient G/N by a normal subgroup
index of $\mathcal{L}$	$ \Sigma^* / \sim_{\mathcal{L}} $	$[G : H]$ index

A subgroup $H \leq G$ cuts G into cosets and the count is the index $[G : H]$; here $\mathcal{L}$ cuts Σ^* into Myhill–Nerode classes and the count is the index of $\mathcal{L}$. A *normal* subgroup gives a congruence respecting multiplication on both sides, hence a quotient *group*; the syntactic congruence respects multiplication on both sides, hence a quotient *monoid*. “Finite index” is the regularity condition in both worlds. The lattice of congruences finer than $\sim_{\mathcal{L}}$ orders the DFAs that recognize $\mathcal{L}$, smallest at the bottom; its Hasse diagram has $\sim_{\mathcal{L}}$ (the minimal machine) as least element:



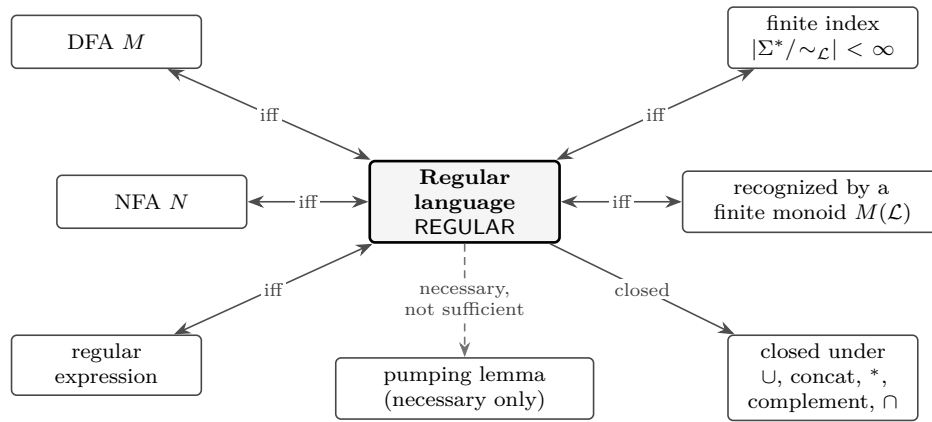
Every DFA for $\mathcal{L}$ sits above the minimal one: its state-classes form a *refinement* of $\sim_{\mathcal{L}}$ that merges fewer strings, and quotienting by indistinguishability walks back down to the bottom. The whole hierarchy of recognizers – from automaton to finite-index congruence to finite monoid – collapses to the same finiteness condition:

$$\mathcal{L} \text{ regular} \iff M \text{ finite} \iff |\Sigma^* / \sim_{\mathcal{L}}| < \infty \iff M(\mathcal{L}) \text{ finite}$$

Caution. The two congruences are *not* interchangeable. The Myhill–Nerode relation $\sim_{\mathcal{L}}$ respects appending on the right only and its classes count *states* (the index). The syntactic congruence $\approx_{\mathcal{L}}$ respects multiplication on *both* sides and its classes form the *monoid*. Always $|M(\mathcal{L})| \geq |\Sigma^* / \sim_{\mathcal{L}}|$, and the two can differ: the index counts how many futures a prefix can have, while the syntactic monoid counts how many context-actions a factor can have. Use $\sim_{\mathcal{L}}$ to find the minimal DFA and to prove nonregularity by infinite index; use $\approx_{\mathcal{L}}$ for the algebraic “finite monoid” picture.

Caution. When minimizing, *delete unreachable states first*. The table-filling algorithm only merges states that are present, and an unreachable accept state can masquerade as a class of its own, leaving the output larger than the true minimal DFA. Reachability and indistinguishability are two separate prunings: unreachable states are removed, indistinguishable reachable states are merged. Skipping the first step is the most common way to report a “minimal” DFA that is not minimal.

Stepping back, the whole chapter describes *one* class of languages seen from several sides. The equivalent characterizations all pin down the same **REGULAR**; the pumping lemma is a one-way test that sits outside the equivalence.



Five equivalent faces of one class: a language is regular iff given by any of DFA / NFA / regex / finite Myhill–Nerode index / finite-monoid recognition; the pumping lemma is only a one-way test, drawn off to the side with a dashed edge because passing it does *not* imply regularity.

Check. Kleene’s theorem gives regex = DFA = NFA; Myhill–Nerode gives finite index; Theorem 5.5.10 gives finite-monoid recognition – all five characterize **REGULAR**. The pumping condition is implied by regularity but does not imply it, so it is not a face of the equivalence.

Exercises.

1. For $\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ ends in } 00\}$, list the Myhill–Nerode classes of $\sim_{\mathcal{L}}$ by describing the “relevant memory” each one records, give a distinguishing suffix for each pair of classes, and draw the minimal DFA. How many states does it have?
2. Over $\Sigma = \{0,1\}$ let $\mathcal{L} = \{w \mid w \text{ contains the substring } 101\}$. Show that $\varepsilon, 1, 10, 101$ lie in four different $\sim_{\mathcal{L}}$ -classes by exhibiting a separating suffix for each pair, and conclude the minimal DFA has at least four states.
3. Use the Myhill–Nerode theorem (infinite index) to prove that $\mathcal{L} = \{0^n 1^m \mid n > m \geq 0\}$ is not regular. Give an explicit infinite family of pairwise-distinguishable strings together with the suffix that separates each pair.
4. Prove from Definition 5.5.1 that $\sim_{\mathcal{L}}$ is transitive, and prove the right-congruence property $x \sim_{\mathcal{L}} y \Rightarrow xa \sim_{\mathcal{L}} ya$ directly (do not cite Proposition 5.5.2). State in one sentence why right-congruence is what makes the quotient automaton’s transition function well defined.

5. Minimize the DFA over $\{0, 1\}$ with states $\{A, B, C, D, E\}$, start A , accept set $\{D, E\}$, and transitions

δ	0	1
A	B	C
B	A	D
C	E	A
D	D	D
E	E	E

Run the table-filling algorithm, show the completed lower-triangular grid of distinguishable pairs, merge the indistinguishable classes, and draw the minimal DFA.

6. A DFA has states $\{p, q, r\}$, start p , accept set $\{r\}$, and an unreachable state r (no transition from p or q ever enters r). A student runs table-filling and reports a two-state “minimal” DFA. Explain precisely what step was skipped, and give the correct minimal machine.
7. Let $\mathcal{L} = \{w \in \{0, 1\}^* \mid \text{the number of 1s in } w \text{ is even}\}$. Compute the syntactic monoid $M(\mathcal{L})$: identify its elements, write the Cayley table, say which element(s) form the accepting set S , and identify which standard two-element monoid $M(\mathcal{L})$ is.
8. Prove that the index of $\mathcal{L}$ (the number of $\sim_{\mathcal{L}}$ -classes) is at most the number of elements of the syntactic monoid $M(\mathcal{L})$, i.e. $|\Sigma^* / \sim_{\mathcal{L}}| \leq |M(\mathcal{L})|$. (Show that $u \approx_{\mathcal{L}} v$ implies $u \sim_{\mathcal{L}} v$, so $\approx_{\mathcal{L}}$ refines $\sim_{\mathcal{L}}$.) Give a language for which the inequality is strict.
9. *Find the bug.* A classmate claims that for every language $\mathcal{L}$, the strings x and y are $\sim_{\mathcal{L}}$ -equivalent whenever $x \in \mathcal{L} \iff y \in \mathcal{L}$ (same membership). Give a concrete $\mathcal{L}$ and a pair x, y with the same membership but $x \not\sim_{\mathcal{L}} y$, and state in one sentence which half of Definition 5.5.1 the claim ignores.
10. *Find the bug.* To minimize a DFA, someone merges two accept states C, D “because they are both accepting,” without checking suffixes. Construct a small DFA with two accept states that are *distinguishable*, show the suffix that separates them, and explain why merging them changes the recognized language.
11. Let $\mathcal{L}$ be recognized by a DFA with n reachable states. Prove that $\mathcal{L}$ is regular iff $\overline{\mathcal{L}} = \Sigma^* \setminus \mathcal{L}$ is regular, and show that $\mathcal{L}$ and $\overline{\mathcal{L}}$ have the *same* Myhill–Nerode relation, hence the same minimal-DFA size. (Hint: a suffix z distinguishes x, y for $\mathcal{L}$ iff it distinguishes them for $\overline{\mathcal{L}}$.)
12. Two DFAs M_1, M_2 are given over $\{0, 1\}$, each with no unreachable states. Describe an algorithm, using indistinguishability, that decides whether $L(M_1) = L(M_2)$. (Hint: form the disjoint union, run table-filling, and check whether the two start states fall into the same class.) Explain why your algorithm always terminates.

Chapter 6

Context-Free Grammars and Languages

6.1 Context-Free Grammars and Derivations

A *context-free grammar* is a finite set of substitution rules that *generates* a language: starting from one designated symbol you rewrite, again and again, until only ordinary alphabet symbols remain, and the strings you can reach this way form the language. The one key idea is that a rule may refer to itself – a variable can reappear on the right of its own rule – and this self-reference is exactly what lets a finite list of rules describe an infinite, *recursively structured* language, including languages like $\{0^n 1^n \mid n \geq 0\}$ that no DFA can recognize. Where finite automata *accept* strings handed to them, grammars *produce* strings; this section makes the production precise (the 4-tuple, the yields and derives relations, leftmost derivations, and parse trees) and works many examples.

Definition 6.1.1 (Context-free grammar). A *context-free grammar* (CFG) is a 4-tuple $G = (V, \Sigma, R, S)$ where

- V is a finite set, the *variables* (or nonterminals);
- Σ is a finite set disjoint from V , the *terminals* (the output alphabet);
- R is a finite set of *rules* (productions), each of the form $A \rightarrow \alpha$ with $A \in V$ and $\alpha \in (V \cup \Sigma)^*$ a string of variables and terminals;
- $S \in V$ is the *start variable*.

The name *context-free* refers to the left side of every rule being a *single* variable A : wherever A occurs, it may be replaced by α no matter what surrounds it – the substitution does not depend on the context. By convention we write variables as capital letters and terminals as lowercase letters, digits, or punctuation, and we abbreviate several rules sharing a left side with a vertical bar, so $A \rightarrow \alpha \mid \beta \mid \gamma$ means the three rules $A \rightarrow \alpha$, $A \rightarrow \beta$, $A \rightarrow \gamma$. When a grammar is given by its rules alone, the variable on the left of the first rule is the start variable, and the terminals are whatever symbols never appear on a left side.

Definition 6.1.2 (Yields, derives, and the language generated). Let $G = (V, \Sigma, R, S)$. For strings $u, v, w \in (V \cup \Sigma)^*$, if $A \rightarrow w$ is a rule of G we say uAv *yields* uwv , written $uAv \Rightarrow uwv$: one occurrence of A is replaced by w . We say u *derives* v , written $u \Rightarrow^* v$, if $u = v$ or there is a finite chain $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow v$. The *language generated* by G is

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \},$$

the set of *terminal* strings derivable from the start variable. A language is *context-free* (a CFL) if it equals $L(G)$ for some CFG G .

Two points deserve care. First, $\Rightarrow$ is one rewriting step on a string that may still contain variables; such an intermediate string is called a *sentential form*. Second, $L(G)$ collects only those sentential forms that are

all terminals: a derivation that stalls with a variable still present has not produced a string of the language. The relation $\Rightarrow^*$ is the reflexive-transitive closure of $\Rightarrow$, so $S \Rightarrow^* S$ holds trivially in zero steps.

Definition 6.1.3 (Leftmost derivation and parse tree). A derivation $S \Rightarrow u_1 \Rightarrow \dots \Rightarrow w$ is *leftmost* if at every step the variable replaced is the leftmost variable of the current sentential form. A *parse tree* (or derivation tree) for w records the derivation as a rooted, ordered tree: the root is S ; each internal node is a variable A whose children, read left to right, are the symbols of the right side α of the rule $A \rightarrow \alpha$ applied at that node; and the leaves, read left to right, spell w . (A node for a rule $A \rightarrow \varepsilon$ has a single leaf labelled ε .)

A parse tree throws away the *order* in which independent variables were expanded and keeps only the *nesting structure*. Many derivations – differing only in which variable you happened to expand first – collapse to the same tree; among them exactly one is leftmost (and exactly one is rightmost). The tree, not the linear derivation, is what carries meaning, which is why parsers build trees and why “how many essentially different parse trees does w have” (the subject of Section 6.2) is the right question to ask about structure.

Worked Example 6.1.4 (Matched powers: $\{0^n 1^n \mid n \geq 0\}$). Give a CFG generating $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ and derive 0011.

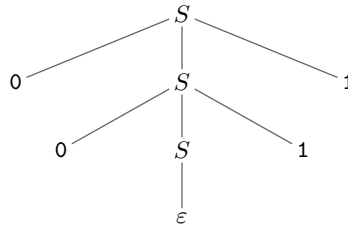
Recognition. The two blocks are *linked*: every 0 on the left must be matched by a 1 on the right, and the count is unbounded. The trigger “equal, unbounded counts wrapped around a center” calls for a single recursive rule $S \rightarrow 0S1$ that adds one symbol to *each* end per step, plus a base case $S \rightarrow \varepsilon$ to stop.

Solution. Take $G = (\{S\}, \{0, 1\}, R, S)$ with

$$S \rightarrow 0S1 \mid \varepsilon.$$

Each use of $S \rightarrow 0S1$ wraps one matched pair around what remains; the ε -rule closes the recursion. A leftmost derivation of 0011 (here S is always the only variable, so the derivation is automatically leftmost):

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 00\varepsilon11 = 0011.$$



Which idea, and why? Two substrings whose lengths must stay equal under an unbounded count are generated by a self-embedding rule $S \rightarrow xSy$ that grows both ends together, with a base rule to terminate; the nesting of the tree records the pairing.

Check. The leaves read left to right give $00\varepsilon11 = 0011$. The empty string comes from the single step $S \Rightarrow \varepsilon$, so $\varepsilon \in \mathcal{L}$, as required by $n = 0$.

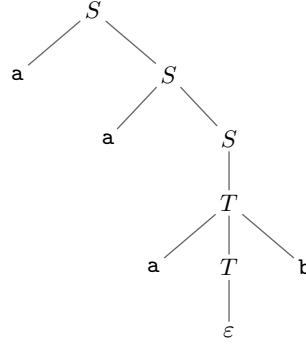
Worked Example 6.1.5 (Balanced brackets). Let $\Sigma = \{(\,,\,)\}$. Give a CFG for the language $\mathcal{L}$ of *balanced* bracket strings (every (matched by a later), never closing more than has been opened), and exhibit a parse tree for $((\,))(\,)$.

Recognition. Balance is recursive in two independent ways: a balanced string may be *another balanced string wrapped in a pair*, and two balanced strings written one after the other are again balanced. The trigger “nest or juxtapose” asks for one rule of each shape plus an empty base.

Solution. Take $G = (\{S\}, \{(\,,\,)\}, R, S)$ with

$$S \rightarrow (S) \mid SS \mid \varepsilon.$$

The rule $S \rightarrow (S)$ nests, $S \rightarrow SS$ concatenates, and $S \rightarrow \varepsilon$ is the empty balanced string. A parse tree for $((\,))(\,)$ splits it as $((\,)) \cdot (\,)$ at the top via $S \rightarrow SS$:



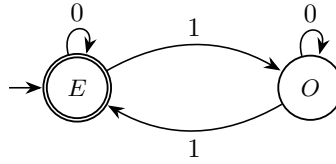
Which idea, and why? When a language is a matched core plus an unmatched surplus, give each part its own variable – T for the recursively matched $\mathbf{a^j b^j}$, S for the free leading $\mathbf{as}$ – and let the start variable chain the surplus before delegating to the core.

Check. The leaves spell $\mathbf{a a a \varepsilon b} = \mathbf{aaab}$ with $i = 3 \geq j = 1$. Setting both regimes to their base cases ($S \Rightarrow T \Rightarrow \varepsilon$) gives $\varepsilon \in \mathcal{L}$, the case $i = j = 0$.

Worked Example 6.1.8 (From a DFA to a grammar: even number of 1s). The language $\mathcal{L} = \{w \in \{0, 1\}^* \mid w \text{ has an even number of 1s}\}$ is regular. Convert a two-state DFA for it into a CFG, illustrating that every regular language is context-free.

Recognition. The mechanical recipe “one variable per state, one rule per transition, an ε -rule at each accept state” turns any DFA into a grammar. The trigger is that the language is presented as a machine and we want a generator: read the rules straight off the transition function.

Solution. The DFA has states E (even, start, accept) and O (odd), with 0 looping and 1 toggling:



Introduce a variable for each state (E, O), a rule $X \rightarrow aY$ for each transition $\delta(X, a) = Y$, and a rule $X \rightarrow \varepsilon$ for each accept state. Since E is the only accept state:

$$E \rightarrow 0E \mid 1O \mid \varepsilon, \quad O \rightarrow 0O \mid 1E.$$

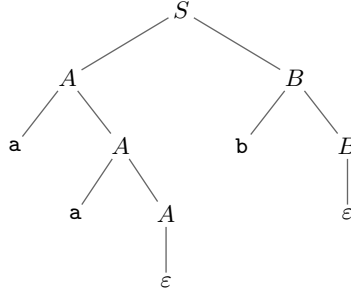
A leftmost derivation of 1010 traces the run $E \rightarrow O \rightarrow O \rightarrow E \rightarrow E$:

$$E \Rightarrow 1O \Rightarrow 10O \Rightarrow 101E \Rightarrow 1010E \Rightarrow 1010\varepsilon = 1010.$$

Which idea, and why? A DFA $\rightarrow$ CFG conversion encodes “current state” in the single remaining variable: each terminal-then-variable rule consumes one input symbol and moves to the next state, and an ε -rule at an accept state ends the string exactly when the machine would accept.

Check. The string 1010 has two 1s (even), is accepted by the DFA (ends in E), and is derived above; the derivation can stop only at E , so the generated strings are precisely the inputs ending the run in the accept state.

Worked Example 6.1.9 (Reading the tree to recover the derivation). A parse tree is given below for a grammar with rules $S \rightarrow AB$, $A \rightarrow \mathbf{a}A \mid \varepsilon$, $B \rightarrow \mathbf{b}B \mid \varepsilon$. Recover the leftmost derivation and the generated string, and name the language.



Recognition. Given the tree, the leftmost derivation is read by a left-to-right, top-down walk: expand the leftmost unfinished variable using the rule shown at its node. The trigger “tree given, derivation wanted” is the inverse of building a tree from a derivation.

Solution. Walking the tree leftmost-first, S uses $S \rightarrow AB$; the left A uses $A \rightarrow \mathbf{a}A$ twice then $A \rightarrow \varepsilon$; then B uses $B \rightarrow \mathbf{b}B$ once then $B \rightarrow \varepsilon$:

$$S \Rightarrow AB \Rightarrow \mathbf{a}AB \Rightarrow \mathbf{a}\mathbf{a}AB \Rightarrow \mathbf{a}\mathbf{a}B \Rightarrow \mathbf{a}\mathbf{a}\mathbf{b}B \Rightarrow \mathbf{a}\mathbf{a}\mathbf{b} = \mathbf{aab}.$$

Which idea, and why? A parse tree determines its leftmost derivation uniquely: the rule at each internal node is fixed, and “leftmost” fixes the order, so reading the tree top-down and left-to-right reconstructs the one leftmost derivation.

Check. The frontier (leaves left to right, dropping ε) is $\mathbf{aab} = \mathbf{aab}$. Since A generates any block $\mathbf{a}^i$ and B any block $\mathbf{b}^j$ independently, the language is $\{\mathbf{a}^i\mathbf{b}^j \mid i, j \geq 0\} = L(\mathbf{a}^*\mathbf{b}^*)$, which is regular.

Method (How to design a context-free grammar from a language description).

1. **Look for linkage.** Ask whether two parts of the string must agree in an unbounded way (equal counts, mirror symmetry, matched brackets). A linked pair $x \cdots y$ is generated by a self-embedding rule $A \rightarrow xAy$ that grows both ends together; a base rule $A \rightarrow \varepsilon$ (or a terminal) stops the recursion.
2. **Break a union into pieces.** If $\mathcal{L} = \mathcal{L}_1 \cup \cdots \cup \mathcal{L}_k$, build a grammar G_i with start variable S_i for each piece, then add $S \rightarrow S_1 \mid \cdots \mid S_k$ over the combined (renamed-apart) rules.
3. **Use one variable per independent “part.”** Substrings that vary independently (e.g. a free $\mathbf{a}^i$ prefix and a matched core) get their own variables, chained by the start variable.
4. **If the language is regular, read it off a DFA.** One variable per state, a rule $X \rightarrow aY$ for each transition $\delta(X, a) = Y$, and $X \rightarrow \varepsilon$ at every accept state.
5. **Test by deriving.** Derive two or three target strings and confirm the grammar refuses an obvious non-member; check whether ε should be in the language and that exactly the intended base cases produce it.

Caution. A derivation is finished only when *no variable remains*. A sentential form such as $00S11$ is an intermediate stage, not a member of $L(G)$; you must keep rewriting until every symbol is a terminal (possibly using an ε -rule to erase a variable). A string with a leftover variable has not been generated.

Caution. Do not confuse the grammar’s *terminal* alphabet Σ with its *variable* alphabet V : they must be disjoint, and only terminals appear in the final strings. A symbol that occurs on the left of some rule is a variable; the others are terminals. Writing $S \rightarrow 0S1$ uses the terminals $0, 1$ and the variable S – the start symbol legitimately reappears on the right, which is the whole point of recursion, but 0 and 1 may never sit on a left side.

Caution. Ambiguity is judged by *parse trees* (equivalently, leftmost derivations), *not* by raw derivation order. Two derivations that differ only in whether you expanded a left or a right variable first are the *same* parse and do not make a grammar ambiguous; only genuinely different trees do. We return to this distinction in Section 6.2.

Exercises.

1. Let G have rules $S \rightarrow \mathbf{a}S\mathbf{a} \mid \mathbf{b}S\mathbf{b} \mid \varepsilon$ over $\Sigma = \{\mathbf{a}, \mathbf{b}\}$. Give a leftmost derivation of $\mathbf{abba}$, draw its parse tree, and describe $L(G)$ in one sentence.

2. For the grammar $S \rightarrow 0S1 \mid \varepsilon$, write out the full derivation of 000111 and draw the parse tree. Then explain in one sentence why no DFA recognizes $L(G)$.
3. Design a CFG generating $\{a^n b^{2n} \mid n \geq 0\}$ (twice as many bs as as). Derive aabbbb and draw its parse tree.
4. Design a CFG for $\mathcal{L} = \{w \in \{0,1\}^* \mid w = w^R\}$, the palindromes over $\{0,1\}$. Be careful to allow both even- and odd-length palindromes, and derive 01010.
5. Using the union technique, design a CFG for $\{0^n 1^n \mid n \geq 0\} \cup \{1^n 0^n \mid n \geq 0\}$. Identify the start variable and the two sub-grammar start variables.
6. Convert the following DFA into a CFG by the one-variable-per-state method. The DFA over $\{0,1\}$ has states p (start, accepting) and q , with $\delta(p,0) = p$, $\delta(p,1) = q$, $\delta(q,0) = q$, $\delta(q,1) = p$. Write the rules and derive 0110. What language is this?
7. The grammar $E \rightarrow E + E \mid E * E \mid a$ generates $a + a * a$. Draw two *different* parse trees for $a + a * a$ and give the corresponding two leftmost derivations. (You need not yet say which grouping is “correct”; just show the grammar admits both.)
8. Consider $S \rightarrow SS \mid (S) \mid \varepsilon$. Give two distinct leftmost derivations of $()()()$ that produce *different* parse trees, thereby exhibiting that this grammar is ambiguous.
9. Let G have rules $S \rightarrow aT$, $T \rightarrow bT \mid aS \mid \varepsilon$. List all strings in $L(G)$ of length at most 3, and give a one-sentence description of $L(G)$.
10. *Find the bug.* A student writes $S \rightarrow aSb \mid ab$ and claims it generates $\{a^n b^n \mid n \geq 0\}$. Find a string in the intended language that the grammar fails to generate, and repair the grammar with a one-rule change.
11. *Find the bug.* Someone proposes $S \rightarrow 0A$, $A \rightarrow 1A$ for $\{01^n \mid n \geq 0\}$. Derive as far as you can from S and explain precisely why $L(G) = \emptyset$. Add the single rule that fixes it.
12. Prove that if $\mathcal{L}_1$ and $\mathcal{L}_2$ are context-free, then $\mathcal{L}_1 \cup \mathcal{L}_2$ is context-free. (Take grammars $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ with disjoint variable sets, and exhibit a grammar for the union; justify in one sentence why $S \xRightarrow{*} w$ in your grammar iff $S_1 \xRightarrow{*} w$ in G_1 or $S_2 \xRightarrow{*} w$ in G_2 .)

6.2 Designing Grammars and Ambiguity

Section 6.1 gave us the machinery of context-free grammars; this section turns to the craft of *writing* one for a language handed to you, and to the hazard that lurks once you have. The one key idea is that a grammar’s rules should mirror the *recursive shape* of the strings you want: a piece that repeats becomes a variable that calls itself, two pieces that must match in length become a single rule $R \rightarrow uRv$ that grows them in lockstep, and a language built by “ A or B ” becomes one grammar that offers both through a fresh start variable. The hazard is *ambiguity*: a grammar may generate a string by two genuinely different parse trees, and since the parse tree is where meaning lives, that is usually a defect we must design away.

Four techniques for designing a grammar

Most grammars are assembled from a small set of moves. We state them as a method first, then drill each one in a worked example.

Method (How to design a CFG for a target language).

1. **Split into a union.** If $\mathcal{L} = \mathcal{L}_1 \cup \dots \cup \mathcal{L}_k$ and each piece is easier on its own, build a grammar with start variable S_i for each $\mathcal{L}_i$, take the disjoint union of all the rules, and add one top rule $S \rightarrow S_1 \mid S_2 \mid \dots \mid S_k$.
2. **Borrow a finite automaton.** If the piece is *regular*, build a DFA for it and read off a grammar mechanically: a variable R_i per state q_i , a rule $R_i \rightarrow aR_j$ for each transition $\delta(q_i, a) = q_j$, and $R_i \rightarrow \varepsilon$ whenever q_i is accepting.

3. **Link matched parts with $R \rightarrow uRv$.** When two stretches of a string must correspond (equal counts, mirror order), generate them together: a rule $R \rightarrow uRv$ lays down one copy of u on the left for every v on the right, so they can never drift out of step.
4. **Recurse for nested structure.** When a structure can reappear inside itself (a parenthesized expression inside an expression), put the variable that generates the structure at the spot where it may recur.

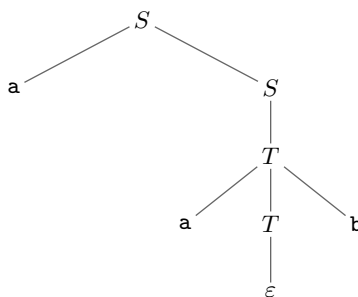
Worked Example 6.2.1 (Union of grammars: an unequal-blocks language). Build a CFG for $\mathcal{L} = \{a^m b^n \mid m \geq n \geq 0\}$ over $\Sigma = \{a, b\}$.

Recognition. The condition $m \geq n$ says: every **b** is matched by an **a**, and there may be *extra* **as** on the left. That is two jobs glued together – a matched $a^n b^n$ core, with a free run of **as** prepended – so technique 3 (linked parts) handles the core and technique 1 (union/concatenation) attaches the surplus.

Solution. Generate the matched core with $T \rightarrow aTb \mid \varepsilon$, which produces exactly $\{a^n b^n\}$. Prepend any number of extra **as** with a right-recursive helper folded into the start variable:

$$S \rightarrow aS \mid T, \quad T \rightarrow aTb \mid \varepsilon.$$

From S you may pump out leading **as** ($S \rightarrow aS$) as long as you like, then drop into the matched part ($S \rightarrow T$). The leading **as** contribute the surplus $m - n$, and T supplies the balanced $a^n b^n$.



Which idea, and why? A “ $\geq$ ” between two block lengths is a matched core ($R \rightarrow uRv$) plus a free surplus; split the surplus off into its own recursive rule and combine, rather than trying to count in one rule.

Check. The tree above derives **aab** ($m = 2, n = 1$): the top $S \rightarrow aS$ lays down the extra **a**, then $S \rightarrow T$ and $T \rightarrow aTb$ with $T \rightarrow \varepsilon$ gives the matched **ab**. The string **abb** ($m = 1, n = 2$) cannot be derived: T always adds an **a** and a **b** together, and the only other **a**-source is the *leading* block, so **bs** can never outnumber **as**.

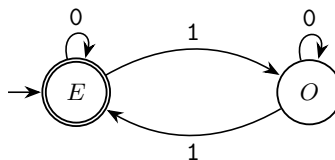
Worked Example 6.2.2 (From a DFA to a grammar). The language

$$\mathcal{L} = \{w \in \{0, 1\}^* \mid w \text{ has an even number of } 1\text{s}\}$$

is regular. Convert a DFA for it into an equivalent CFG.

Recognition. “Regular” is the trigger for technique 2: do not invent grammar rules by hand, build the two-state parity DFA and transcribe it. Each state becomes a variable; each arrow becomes a rule; each accept state earns an ε -rule.

Solution. The DFA has states E (even number of 1s so far, the start *and* accept state) and O (odd). Reading 0 keeps the parity; reading 1 flips it.



Transcribe: a variable R_E for E and R_O for O ; an arrow $\delta(q, a) = q'$ becomes $R_q \rightarrow a R_{q'}$; the accept state E contributes $R_E \rightarrow \varepsilon$. The start variable is R_E :

$$R_E \rightarrow 0 R_E \mid 1 R_O \mid \varepsilon, \quad R_O \rightarrow 0 R_O \mid 1 R_E.$$

A derivation walks a path through the DFA: each substitution emits one input symbol and moves to the variable for the next state, and the run halts (via $R_E \rightarrow \varepsilon$) only in the accepting state E .

Which idea, and why? For a regular piece, the DFA-to-CFG transcription is automatic and never ambiguous about the rules, so use it instead of guessing – the grammar’s variables *are* the machine’s states.

Check. Derive 101 (two 1s, even): $R_E \Rightarrow 1R_O \Rightarrow 10R_O \Rightarrow 101R_E \Rightarrow 101$, using $R_E \rightarrow \varepsilon$ at the end – accepted. The string 1 has only the derivation $R_E \Rightarrow 1R_O$, and R_O has no ε -rule, so it cannot terminate: rejected, as it should be (one 1 is odd).

Worked Example 6.2.3 (Linked parts: twice as many bs as as). Build a CFG for $\mathcal{L} = \{w \in \{a, b\}^* \mid \#_b(w) = 2\#_a(w)\}$, where $\#_x(w)$ is the number of x s in w and the letters may appear in any order.

Recognition. The counts are linked ($\#_b = 2\#_a$) but the *order* is free, so this is technique 3 with a twist: each **a** must be accompanied somewhere by two bs, and the rule must be free to interleave them. A single recursive variable that, per use, emits one **a** and two bs in every relative position keeps the ratio exact.

Solution. Let S generate the empty string or, recursively, insert one **a** and two bs around and between existing material:

$$S \rightarrow \varepsilon \mid S a S b S b S \mid S b S a S b S \mid S b S b S a S.$$

Every non- ε alternative contributes exactly one **a** and two bs, so after any number of expansions the ratio $\#_b = 2\#_a$ is preserved; the three alternatives place the **a** first, second, or third among its two partner bs, and the interleaved S ’s let other groups nest in any gap, which together reach every legal ordering.

Which idea, and why? A fixed ratio between symbol *counts* with arbitrary order is enforced by a recursive rule whose every productive alternative emits one full “unit” (here **abb** in each order), with S sprinkled between symbols to allow interleaving.

Check. **bab**: $S \Rightarrow S b S a S b S \xRightarrow{*} \text{bab}$ (each $S \rightarrow \varepsilon$) – one **a**, two bs, accepted. **ab** has $\#_a = 1, \#_b = 1 \neq 2$ and no alternative emits a lone **ab**, so it is rejected.

Ambiguity

A derivation records *which* rules were applied and *in what order*; a parse tree records only the rules and their nesting, discarding the order of unrelated replacements. Meaning lives in the parse tree, so the right notion of “the grammar produced this string in two essentially different ways” must be phrased in trees, not raw derivations. To pin the order down we agree to always expand the leftmost variable.

Definition 6.2.4 (Leftmost derivation). A derivation $S \Rightarrow \cdots \Rightarrow w$ is a *leftmost derivation* if at every step the variable replaced is the leftmost variable in the current string. Each parse tree corresponds to exactly one leftmost derivation, so leftmost derivations are faithful names for parse trees.

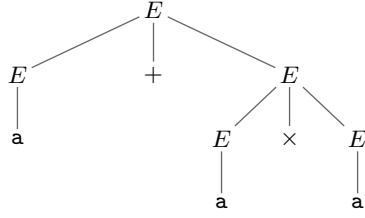
Definition 6.2.5 (Ambiguity). A grammar G *derives a string w ambiguously* if w has two or more distinct parse trees in G (equivalently, two or more distinct leftmost derivations). G is *ambiguous* if it derives some string ambiguously. A context-free language is *inherently ambiguous* if *every* CFG generating it is ambiguous.

Remark. Ambiguity is a property of a *grammar*, while inherent ambiguity is a property of a *language*. A language can have an ambiguous grammar and an unambiguous one at the same time; it is inherently ambiguous only if no unambiguous grammar exists for it at all. Most languages you will meet are not inherently ambiguous, and the work is to find the unambiguous grammar.

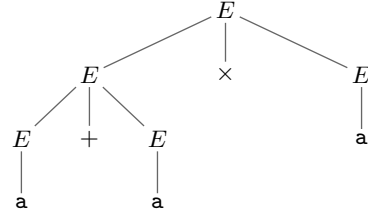
Worked Example 6.2.6 (An ambiguous expression grammar). The grammar G_{amb} with the single variable E and rules $E \rightarrow E + E \mid E \times E \mid (E) \mid a$ generates the language of $+, \times$ -expressions over the atom **a**. Show it is ambiguous.

Recognition. The trigger is a binary operator written with the expression variable *itself* as the recursive variable on both sides ($E \rightarrow E \text{ op } E$): nothing in the rule says which operator binds tighter, so a string with two unparenthesized operators can be grouped two ways. To prove ambiguity, exhibit one string with two parse trees.

Solution. Take $w = a + a \times a$. One parse applies $E \rightarrow E + E$ at the root (grouping as **a**+(**a**×**a**)); the other applies $E \rightarrow E \times E$ at the root (grouping as (**a**+**a**)×**a**). Both yield the same string, with different trees.



+ at the root: $a+(a \times a)$



$\times$ at the root: $(a+a) \times a$

Which idea, and why? An operator rule of the form $E \rightarrow E \text{ op } E$ encodes *no* precedence or associativity, so any string with two unparenthesized operators splits into two trees; this self-on-both-sides shape is the canonical source of ambiguity.

Check. The two trees are distinct (different root expansion), both have frontier $a+a \times a$, so the string has two leftmost derivations and G_{amb} is ambiguous. Numerically the trees even *disagree*: one means $a + (a \cdot a)$, the other $(a + a) \cdot a$.

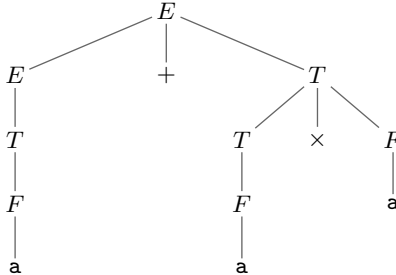
Worked Example 6.2.7 (Disambiguating by encoding precedence). Rewrite the grammar of the previous example into an *unambiguous* grammar for the same language, making $\times$ bind tighter than $+$ and both associate to the left.

Recognition. The fix for a precedence ambiguity is technique 4 applied with discipline: introduce one variable *layer* per precedence level, lowest-binding at the top, so the tree shape is forced. The trigger is “I want a unique grouping,” which means each operator must live at exactly one level.

Solution. Use three variables: E (sums) at the loosest level, T (products) tighter, and F (atoms / parenthesized subexpressions) tightest. Left recursion ($E \rightarrow E+T$) forces left associativity; demoting to the next variable forces $\times$ inside $+$.

$$E \rightarrow E+T \mid T, \quad T \rightarrow T \times F \mid F, \quad F \rightarrow (E) \mid a.$$

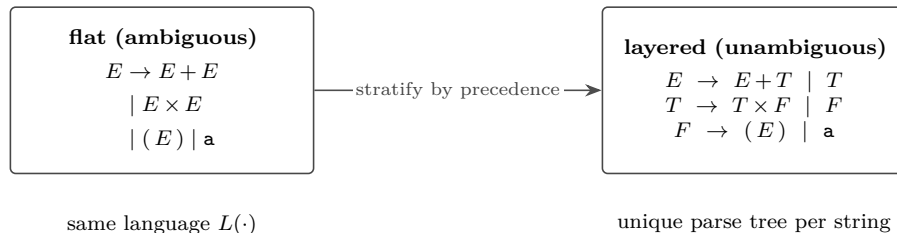
Now $a+a \times a$ has *one* tree: the root must be $E \rightarrow E+T$ (only E produces a top-level $+$), the $\times$ is trapped inside the T on the right, and the left associativity of $+$ is fixed by the recursion sitting on the left.



Which idea, and why? Layered variables (one per precedence level, looser binding higher in the tree, plus left- or right-recursion for associativity) force a unique grouping, which is exactly what removes the ambiguity while keeping the same language.

Check. Without parentheses there is no way to put $\times$ above $+$ in this grammar: $\times$ is introduced only by $T \rightarrow T \times F$, and T sits strictly below the E -level $+$. So the string has a single parse tree, matching the standard reading $a + (a \times a)$.

The whole disambiguation is a single move: stratify the one variable E into precedence layers E, T, F , leaving the generated language fixed while collapsing the parse trees to one per string.



Same language, fewer parse trees: splitting E into $E/T/F$ precedence layers generates the same expressions but forces a unique leftmost derivation per string. *Check.* The layered grammar generates the same arithmetic-expression language and is unambiguous (precedence and associativity now fix the parse).

Caution. This stratification removes ambiguity for *this* grammar, but not every context-free grammar can be disambiguated: some context-free languages are *inherently ambiguous*, meaning every CFG generating them is ambiguous. Layering by precedence works here because the expression language is not inherently ambiguous.

Worked Example 6.2.8 (An ambiguous grammar that is easy to miss). Show that G with $S \rightarrow SS \mid a$ is ambiguous, and give an unambiguous grammar for the same language $\{a^n \mid n \geq 1\}$.

Recognition. The rule $S \rightarrow SS$ is associative-looking but unbracketed: a string of three or more a s can be split into the two S s at more than one place. The trigger is a “concatenate two copies of myself” rule with no anchor fixing where the split goes.

Solution. The string aaa parses as $(aa)a$ or $a(aa)$:



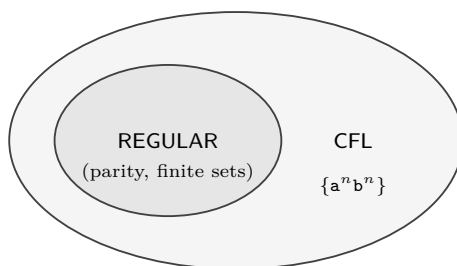
The cure is to make the recursion grow the string *one a at a time*, which fixes a unique shape: $S \rightarrow aS \mid a$. Now aaa has the single right-leaning tree $S \Rightarrow aS \Rightarrow aaS \Rightarrow aaa$.

Which idea, and why? A balanced self-concatenation rule $S \rightarrow SS$ admits many split points and is almost always ambiguous; replacing it with linear recursion ($S \rightarrow aS$) makes the parse tree unique without changing the language.

Check. Both trees above have frontier aaa but different internal structure, so G is ambiguous. In $S \rightarrow aS \mid a$ every string a^n has exactly one derivation (peel one a at a time, finish with $S \rightarrow a$), so it is unambiguous.

Where the regular and context-free languages sit

Technique 2 (DFA-to-CFG) is more than a trick: it shows every regular language is context-free, so $\text{REGULAR} \subseteq \text{CFL}$. The containment is strict, since $\{a^n b^n \mid n \geq 0\}$ is context-free (Worked Example 6.2.1, with $m = n$) but not regular.



Caution. Do not test ambiguity by hunting for two *derivations* – every string with two or more variables has many derivations that differ only in the *order* of unrelated replacements, and those do *not* count. The honest test is two distinct *parse trees*, equivalently two distinct *leftmost* derivations. Always fix the order (leftmost) before declaring a grammar ambiguous or unambiguous.

Caution. “Ambiguous grammar” and “inherently ambiguous language” are different claims. Finding one ambiguous grammar for a language proves nothing about the language: you must show *no* unambiguous grammar exists to call the language inherently ambiguous, and that is a much harder statement (it cannot be

settled by exhibiting a single bad grammar). Conversely, finding one unambiguous grammar settles that the language is *not* inherently ambiguous.

Caution. When you split a language into a union and merge grammars with $S \rightarrow S_1 \mid \cdots \mid S_k$, keep the variable sets of the pieces *disjoint*. If two pieces accidentally share a variable name, their rules fuse and the merged grammar can generate strings in neither original piece. Rename first, then combine.

Exercises.

1. Design a CFG for $\mathcal{L} = \{a^m b^n \mid n \geq m \geq 0\}$ (at least as many bs as as). Draw the parse tree your grammar gives for `abbb`.
2. Give a CFG for $\mathcal{L} = \{w \in \{0,1\}^* \mid w = w^R\}$, the palindromes over $\{0,1\}$, including the even- and odd-length cases. Show a derivation of `01010`.
3. The language $\mathcal{L} = \{w \in \{a,b\}^* \mid w \text{ starts and ends with the same symbol}\}$ is regular. Build a DFA for it, then transcribe the DFA into a CFG using the DFA-to-CFG technique (a variable per state, an ε -rule per accept state).
4. Design a CFG for $\mathcal{L} = \{a^i b^j c^k \mid i = j + k, i, j, k \geq 0\}$. (Hint: the `a` block must match the `bs` and the `cs` together; lay the `as` down in two linked phases.)
5. Using the union technique, write a single CFG for $\mathcal{L} = \{a^n b^n \mid n \geq 0\} \cup \{a^n b^{2n} \mid n \geq 0\}$. Be careful to keep the two pieces' variables disjoint, and identify the start variable.
6. Show that the grammar $S \rightarrow aSb \mid abS \mid \varepsilon$ is ambiguous by exhibiting a string with two distinct parse trees. Draw both trees.
7. Consider $S \rightarrow \text{if } C \text{ then } S \mid \text{if } C \text{ then } S \text{ else } S \mid s$, with C and s terminals. Show this “dangling-else” grammar is ambiguous by giving two parse trees for one string, and describe in one sentence which grouping each tree assigns.
8. Give an unambiguous CFG for the language of balanced strings of one bracket type, $\mathcal{L} = \{w \in \{(\,,\,)\}^* \mid w \text{ is properly nested}\}$, and argue informally why your grammar assigns each string a unique parse tree.
9. For the layered expression grammar $E \rightarrow E+T \mid T, T \rightarrow T \times F \mid F, F \rightarrow (E) \mid a$, draw the unique parse tree for $(a + a) \times a$ and confirm it groups the $+$ before the $\times$.
10. *Find the bug.* A student claims the grammar $S \rightarrow aSa \mid bSb \mid \varepsilon$ generates exactly the palindromes over $\{a,b\}$. State precisely which strings it actually generates and which palindromes it misses, then repair the grammar.
11. Prove that the regular language $\mathcal{L} = \{w \in \{0,1\}^* \mid w \text{ contains } 00\}$ is context-free by giving a CFG for it (you may convert a DFA). Then state, with a one-line reason, whether your grammar is ambiguous.
12. Let $\mathcal{L} = \{a^i b^j \mid i \neq j, i, j \geq 0\}$. Design a CFG for $\mathcal{L}$ by splitting into the cases $i > j$ and $i < j$, building a grammar for each, and combining them with a fresh start variable.

6.3 Chomsky Normal Form

A grammar can be written in countless equivalent ways, and most of those ways are awkward to compute with: a single rule may spawn a whole intermediate string, an ε -rule may make a variable silently vanish, and a chain of rules $A \rightarrow B \rightarrow C$ may rename a variable without producing anything. *Chomsky normal form* (CNF) is a stripped-down shape in which every rule does exactly one of two simple things – emit a single terminal, or branch into exactly two variables – so that a derivation becomes a binary tree of known size. The one key idea of this section is that *every* context-free language has such a grammar, and the conversion is a finite, mechanical sequence of clean-up stages. This normal form is what makes the parsing and membership algorithms of later sections possible, because it pins down both the branching factor and the length of every derivation.

Definition 6.3.1 (Chomsky normal form). A context-free grammar $G = (V, \Sigma, R, S)$ is in *Chomsky normal form* if every rule has one of the three shapes

$$A \rightarrow BC, \quad A \rightarrow a, \quad S \rightarrow \varepsilon,$$

where $a \in \Sigma$ is a terminal, $A, B, C \in V$ are variables, the two variables B, C on the right-hand side of a binary rule are *not* the start variable S , and the only ε -rule allowed is $S \rightarrow \varepsilon$ (used precisely when $\varepsilon \in L(G)$).

The three restrictions work together. Forbidding S on the right of a binary rule guarantees the start variable is never reentered, so the lone permitted ε -rule cannot interfere with the rest of the grammar. Banning every other ε -rule and every *unit* rule $A \rightarrow B$ removes the two ways a derivation step can fail to make visible progress. What remains is rigid: a terminal rule adds one symbol of output, and a binary rule adds one internal branch. The next theorem says this costs us no expressive power.

Theorem 6.3.2 (Chomsky). *Every context-free language is generated by a grammar in Chomsky normal form.*

Proof. Let $G = (V, \Sigma, R, S)$ be any CFG for the language. We transform G in four stages, at each step preserving the generated language, until only the rule shapes of Definition 6.3.1 survive.

Stage 1 (new start variable). Add a fresh variable S_0 and the rule $S_0 \rightarrow S$, and make S_0 the start variable. Now the start variable never appears on the right-hand side of any rule, which we will need so that the eventual rule $S_0 \rightarrow \varepsilon$ stands alone.

Stage 2 (eliminate ε -rules). Repeatedly pick an ε -rule $A \rightarrow \varepsilon$ with $A \neq S_0$ and delete it. To keep the language unchanged, for every rule that contains A on its right side, add new rules covering all ways of leaving each occurrence of A out. For instance $R \rightarrow uAv$ also gives $R \rightarrow uv$, and $R \rightarrow uAvAw$ gives $R \rightarrow uvAw$, $R \rightarrow uAvw$, and $R \rightarrow uvw$. If a rule $R \rightarrow A$ (or more generally $R \rightarrow A \cdots A$) is present, add $R \rightarrow \varepsilon$ – unless that ε -rule was already removed once, so the process terminates. Repeat until no ε -rule remains except possibly $S_0 \rightarrow \varepsilon$.

Stage 3 (eliminate unit rules). Repeatedly pick a unit rule $A \rightarrow B$ (with B a variable) and delete it. For every rule $B \rightarrow u$, add $A \rightarrow u$, unless $A \rightarrow u$ is itself a unit rule that was already removed (this guards against the cycles $A \rightarrow B \rightarrow A$). Repeat until no unit rule remains. After this stage every right-hand side (apart from that of $S_0 \rightarrow \varepsilon$) is either a single terminal or a string of length ≥ 2 .

Stage 4 (shorten and purify). Two clean-ups remain. First, in any rule whose right-hand side has length ≥ 2 , replace each terminal a that appears there by a new variable T_a and add the rule $T_a \rightarrow a$. Now every long right-hand side is a string of variables only. Second, replace each rule $A \rightarrow B_1 B_2 \cdots B_k$ with $k \geq 3$ by the cascade

$$A \rightarrow B_1 C_1, \quad C_1 \rightarrow B_2 C_2, \quad \dots, \quad C_{k-2} \rightarrow B_{k-1} B_k,$$

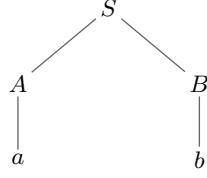
introducing fresh variables $C_1, \dots, C_{k-2}$. Every rule now has shape $A \rightarrow BC$, $A \rightarrow a$, or $S_0 \rightarrow \varepsilon$, which is exactly Definition 6.3.1. Each stage preserves the language, so the final grammar generates the original language and is in Chomsky normal form. $\square$

Why a binary right-hand side matters. In a CNF grammar every internal node of a parse tree has exactly two children (from a rule $A \rightarrow BC$), except that every leaf carrying a terminal hangs from a node with a single child (from $A \rightarrow a$). The parse tree of a nonempty string is therefore a binary tree, and one can show every derivation of a length- n string ($n \geq 1$) uses exactly $2n - 1$ steps. That fixed, predictable size is precisely what membership algorithms exploit.

Worked Example 6.3.3 (Reading off the binary shape of a CNF parse tree). The grammar $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$ is already in Chomsky normal form. Derive **ab** and draw its parse tree, then confirm the step count $2n - 1$.

Recognition. Every right-hand side is either two variables or one terminal, and no ε - or unit rule is present, so Definition 6.3.1 is met and the tree must be binary at its internal nodes. The trigger “rules are only $A \rightarrow BC$ and $A \rightarrow a$ ” is what guarantees the clean shape.

Solution. The leftmost derivation is $S \Rightarrow AB \Rightarrow aB \Rightarrow ab$, three steps, and $2n - 1 = 2 \cdot 2 - 1 = 3$. The branching step $S \rightarrow AB$ creates the two children A, B ; each terminal then hangs below its variable.



Which idea, and why? Because the only branching rule is $A \rightarrow BC$, every variable node with two children comes from one binary rule and every terminal leaf comes from one rule $A \rightarrow a$; counting gives n terminal steps and $n - 1$ branchings, i.e. $2n - 1$ steps.

Check. The string has length $n = 2$; the derivation used $3 = 2n - 1$ steps and the tree has one binary internal node (S) and two terminal-emitting nodes (A, B), as predicted.

Worked Example 6.3.4 (Stage 2: eliminating an ε -rule). Remove the ε -rule from

$$S \rightarrow XaX, \quad X \rightarrow b \mid \varepsilon,$$

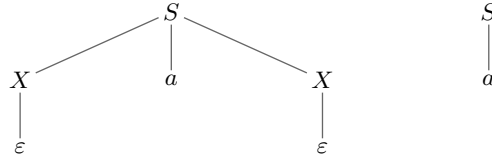
keeping the language fixed.

Recognition. A nullable variable X (one with $X \Rightarrow^* \varepsilon$) appears twice on the right of $S \rightarrow XaX$. The trigger “a variable can derive ε ” means we must add, for every occurrence, the rules that simply omit that occurrence.

Solution. Delete $X \rightarrow \varepsilon$. In $S \rightarrow XaX$ the variable X occurs twice, so we add all four ways of keeping or dropping each occurrence: keep both (XaX), drop the first (aX), drop the second (Xa), drop both (a). The result is

$$S \rightarrow XaX \mid aX \mid Xa \mid a, \quad X \rightarrow b.$$

No ε -rule remains, and X now only produces **b**. The two derivations below show that the old string **a** (using $X \Rightarrow^* \varepsilon$ twice) is still produced, now by the new rule $S \rightarrow a$ on the right.



Which idea, and why? Eliminating $A \rightarrow \varepsilon$ requires adding, to every rule that mentions A , the variants that delete each subset of A ’s occurrences; two occurrences of X give $2^2 = 4$ variants of $S \rightarrow XaX$.

Check. The string **ba** is produced before by $S \rightarrow XaX \Rightarrow baX \Rightarrow ba$ and after by the new $S \rightarrow Xa \Rightarrow ba$; the languages agree on it. The empty string was never in L (every S -rule emits the terminal **a**), so no $S_0 \rightarrow \varepsilon$ is needed.

Worked Example 6.3.5 (Stage 3: eliminating unit rules along a chain). Remove the unit rules from

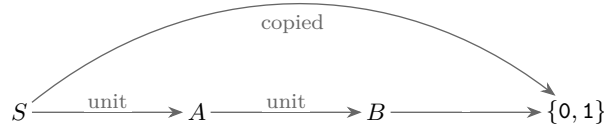
$$S \rightarrow A \mid 1S0, \quad A \rightarrow B, \quad B \rightarrow 0 \mid 1.$$

Recognition. The chain $S \rightarrow A \rightarrow B$ is a sequence of renamings that produce no output on their own. The trigger “a rule whose whole right side is a single variable” marks each link to be short-circuited by copying through what the target can do.

Solution. Process the chain from the bottom. Remove $A \rightarrow B$: since $B \rightarrow 0$ and $B \rightarrow 1$, add $A \rightarrow 0$ and $A \rightarrow 1$, giving $A \rightarrow 0 \mid 1$. Now remove $S \rightarrow A$: since $A \rightarrow 0$ and $A \rightarrow 1$, add $S \rightarrow 0 \mid 1$. The unit-free grammar is

$$S \rightarrow 1S0 \mid 0 \mid 1, \quad A \rightarrow 0 \mid 1, \quad B \rightarrow 0 \mid 1$$

(the variables A and B are now unreachable and may be dropped). The diagram traces how the productions of B are copied up the chain to S .



Legend: a *unit* edge is a unit rule $A \rightarrow B$ (one variable rewritten to another, no output); *copied* marks the new productions handed to S – the non-unit bodies of B copied straight up the chain when each unit rule is eliminated.

Which idea, and why? A unit rule $A \rightarrow B$ is removed by giving A every non-unit production of B ; doing this up a chain $S \rightarrow A \rightarrow B$ pushes B 's terminal productions all the way to S .

Check. The bare string 1 came before from $S \Rightarrow A \Rightarrow B \Rightarrow 1$ (three unit/terminal steps) and comes after directly from the new rule $S \rightarrow 1$. The string 110 came before from $S \Rightarrow 1S0 \Rightarrow 1A0 \Rightarrow 1B0 \Rightarrow 110$ and after from $S \Rightarrow 1S0 \Rightarrow 110$ via $S \rightarrow 1$; the language is unchanged.

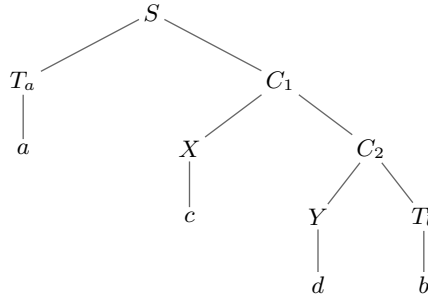
Worked Example 6.3.6 (Stage 4: shortening a long rule and purifying terminals). Put the single rule $S \rightarrow \mathbf{a}XY\mathbf{b}$ (with $X \rightarrow \mathbf{c}$, $Y \rightarrow \mathbf{d}$) into CNF shape.

Recognition. The right-hand side has length $4 > 2$ and mixes terminals ($\mathbf{a}, \mathbf{b}$) with variables (X, Y). The trigger “a long or terminal-bearing right side” calls for two clean-ups: replace stray terminals by fresh variables, then cascade the long rule into binary pieces.

Solution. First purify: introduce $T_a \rightarrow \mathbf{a}$ and $T_b \rightarrow \mathbf{b}$ and rewrite the rule as $S \rightarrow T_aXYT_b$, now all variables. Then cascade the length-4 right side into binary rules with fresh variables C_1, C_2 :

$$S \rightarrow T_a C_1, \quad C_1 \rightarrow X C_2, \quad C_2 \rightarrow Y T_b, \quad T_a \rightarrow \mathbf{a}, \quad T_b \rightarrow \mathbf{b}, \quad X \rightarrow \mathbf{c}, \quad Y \rightarrow \mathbf{d}.$$

Every rule is now $A \rightarrow BC$ or $A \rightarrow a$. The cascade is a right-leaning binary spine, shown below for the string $\mathbf{acdb}$.



Which idea, and why? A right-hand side of $k \geq 3$ variables becomes $k - 1$ binary rules by repeatedly pairing the first symbol with a fresh variable that carries the rest; isolated terminals in long rules are first hidden behind variables $T_a \rightarrow \mathbf{a}$.

Check. Reading the leaves left to right gives $\mathbf{a} \mathbf{c} \mathbf{d} \mathbf{b} = \mathbf{acdb}$, the same string the original rule produced; the tree has only binary branchings and single-terminal leaves.

Worked Example 6.3.7 (A full conversion to Chomsky normal form). Convert to CNF:

$$S \rightarrow 0S1 \mid T, \quad T \rightarrow TT \mid \varepsilon.$$

Recognition. All four obstacles are present: S is the start variable and appears on a right side, $T \rightarrow \varepsilon$ is an ε -rule, $S \rightarrow T$ is a unit rule, and $S \rightarrow 0S1$ is long and mixes terminals with a variable. The trigger is simply “a general CFG,” so we run all four stages in order.

Solution. Stage 1. Add $S_0 \rightarrow S$:

$$S_0 \rightarrow S, \quad S \rightarrow 0S1 \mid T, \quad T \rightarrow TT \mid \varepsilon.$$

Stage 2. The nullable variables are T (directly), S (via $S \rightarrow T$), and S_0 (via $S_0 \rightarrow S$). Remove $T \rightarrow \varepsilon$: in $\overline{T} \rightarrow \overline{TT}$, dropping one occurrence gives $T \rightarrow T$, and dropping both would re-create the just-deleted $T \rightarrow \varepsilon$, so we skip it; in $S \rightarrow T$, dropping gives $S \rightarrow \varepsilon$. Now S is nullable, so remove $S \rightarrow \varepsilon$: in $S \rightarrow 0S1$ dropping the S gives the crucial base rule $S \rightarrow 01$, and in $S_0 \rightarrow S$ dropping gives $S_0 \rightarrow \varepsilon$. After Stage 2:

$$S_0 \rightarrow S \mid \varepsilon, \quad S \rightarrow 0S1 \mid 01 \mid T, \quad T \rightarrow TT \mid T.$$

Stage 3. Remove the unit rules. $T \rightarrow T$ is a self-loop, just deleted. For $S \rightarrow T$, copy T 's only (non-unit) production: add $S \rightarrow TT$. For $S_0 \rightarrow S$, copy S 's non-unit productions: add $S_0 \rightarrow 0S1 \mid 01 \mid TT$. Now:

$$S_0 \rightarrow 0S1 \mid 01 \mid TT \mid \varepsilon, \quad S \rightarrow 0S1 \mid 01 \mid TT, \quad T \rightarrow TT.$$

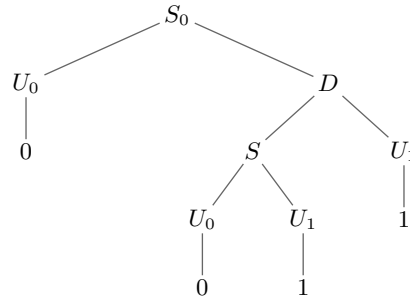
But $T \rightarrow TT$ is the only rule for T , and T has no terminal production, so T generates *nothing*; every rule mentioning T is useless and is discarded. This leaves

$$S_0 \rightarrow 0S1 \mid 01 \mid \varepsilon, \quad S \rightarrow 0S1 \mid 01.$$

Stage 4. Purify terminals with $U_0 \rightarrow 0$, $U_1 \rightarrow 1$; the length-2 rule 01 becomes U_0U_1 , and the length-3 rule $0S1$ cascades into binaries:

$$S_0 \rightarrow U_0D \mid U_0U_1 \mid \varepsilon, \quad S \rightarrow U_0D \mid U_0U_1, \quad D \rightarrow SU_1, \quad U_0 \rightarrow 0, \quad U_1 \rightarrow 1.$$

This is in Chomsky normal form. (Indeed $L = \{0^n 1^n \mid n \geq 0\}$: the rule U_0U_1 is the base case $n = 1$, the recursion U_0D with $D \rightarrow SU_1$ adds an outer $0 \cdots 1$, and $S_0 \rightarrow \varepsilon$ supplies $n = 0$.) The parse tree for 0011 shows the characteristic binary spine.



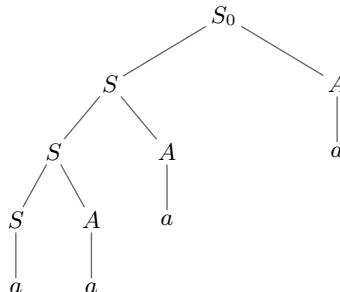
Which idea, and why? Run the four stages strictly in the order new-start, ε -removal, unit-removal, shorten/purify; intermediate stages may *create* new unit or ε rules, which is why each stage repeats to a fixed point and useless variables like T are pruned along the way.

Check. The leftmost derivation $S_0 \Rightarrow U_0D \Rightarrow 0D \Rightarrow 0SU_1 \Rightarrow 0U_0U_1U_1 \Rightarrow 00U_1U_1 \Rightarrow 001U_1 \Rightarrow 0011$ uses $7 = 2n - 1$ steps for the length- $n = 4$ string, and the depicted tree reads 0011 off its leaves.

Worked Example 6.3.8 (Counting derivation steps via the $2n - 1$ law). A CNF grammar G has $L(G) = \{a^n \mid n \geq 1\}$ through the rules $S_0 \rightarrow SA \mid a$, $S \rightarrow SA \mid a$, $A \rightarrow a$ (start variable S_0). How many steps does the unique leftmost derivation of $aaaa$ take, and does the law $2n - 1$ hold?

Recognition. The grammar is in CNF (only $A \rightarrow BC$ and $A \rightarrow a$ rules), so the $2n - 1$ step law applies directly; the trigger is “count derivation steps in a CNF grammar.”

Solution. For $aaaa$ we have $n = 4$. The branchings ($S_0 \rightarrow SA$ once, then $S \rightarrow SA$ twice) are applied $n - 1 = 3$ times to build a left spine of four leaf-positions, and then $n = 4$ terminal rules ($S \rightarrow a$ once, $A \rightarrow a$ three times) finish each leaf, for $3 + 4 = 7 = 2 \cdot 4 - 1$ steps.



Which idea, and why? In any CNF grammar a length- n string needs exactly $n - 1$ binary (branching) rule applications to create n leaf slots plus n terminal rules to fill them, totalling $2n - 1$ – a fixed count that no choice of grammar can change.

Check. The tree has $n - 1 = 3$ internal binary nodes and $n = 4$ terminal-emitting leaves; $3 + 4 = 7 = 2n - 1$, matching the law.

Method (How to convert a CFG to Chomsky normal form).

1. **Add a new start variable.** Introduce S_0 with the single rule $S_0 \rightarrow S$ so the start variable can never recur on a right-hand side.
2. **Remove ε -rules.** Find the nullable variables (those deriving ε). Delete each rule $A \rightarrow \varepsilon$ (with $A \neq S_0$) and, for every rule containing A , add the variants that omit each subset of the A -occurrences. Keep $S_0 \rightarrow \varepsilon$ only if $\varepsilon \in L$. Repeat to a fixed point.
3. **Remove unit rules.** Delete each unit rule $A \rightarrow B$ and give A every production of B , never re-adding a unit rule already removed (this stops cycles). Repeat until none remain.
4. **Purify terminals.** In every right-hand side of length ≥ 2 , replace each terminal a by a fresh variable T_a and add $T_a \rightarrow a$.
5. **Binarize.** Replace each rule $A \rightarrow B_1 \cdots B_k$ with $k \geq 3$ by a cascade of $k - 1$ binary rules through fresh variables. Optionally prune any useless variables.
6. **Sanity-check.** Confirm every rule is $A \rightarrow BC$, $A \rightarrow a$, or $S_0 \rightarrow \varepsilon$, and that a short string still derives correctly.

Caution. Do the stages *in order*, and re-run each to a fixed point. Removing an ε -rule frequently *creates* new unit rules (e.g. $T \rightarrow TT$ collapses to $T \rightarrow T$), and removing one unit rule may expose another. If you stop after a single pass you will be left with forbidden rule shapes that look, at a glance, already converted.

Caution. When you remove $A \rightarrow \varepsilon$, you must add the deletion-variants to *every* rule mentioning A , and for a rule with two A 's you need all four combinations, not just “ A present” versus “ A absent.” Omitting a variant silently shrinks the language. The one combination you do *not* re-add is an ε -rule you have already deleted on a previous pass – adding it back would loop forever.

Caution. Terminals are allowed on the right-hand side only in a length-one rule $A \rightarrow a$. A rule like $A \rightarrow aB$ is *not* in CNF even though it is short and looks harmless; it must become $A \rightarrow T_a B$ with $T_a \rightarrow a$. Likewise, the rule $S_0 \rightarrow \varepsilon$ is the sole legal ε -rule, and the start variable S_0 may never appear on a right-hand side – check both before declaring a grammar converted.

Exercises.

1. For each grammar, state whether it is in Chomsky normal form; if not, name every rule that violates Definition 6.3.1 and say which condition it breaks.
 - (a) $S \rightarrow AB \mid a, A \rightarrow a, B \rightarrow b$.
 - (b) $S \rightarrow aB \mid AB, A \rightarrow a, B \rightarrow b$.
 - (c) $S \rightarrow AB, A \rightarrow BC \mid a, B \rightarrow b, C \rightarrow \varepsilon$.
 - (d) $S \rightarrow SS \mid S \mid a$.
2. Carry out only **Stage 2** (ε -rule elimination) on $S \rightarrow AbA \mid B, A \rightarrow \varepsilon \mid a, B \rightarrow b$. List the nullable variables first, then give the resulting rules.
3. Carry out only **Stage 3** (unit-rule elimination) on $S \rightarrow A \mid aSb, A \rightarrow B \mid c, B \rightarrow b \mid SS$. Identify the unit rules, remove them in a valid order, and give the resulting (unit-free) grammar.
4. Convert the grammar $S \rightarrow 0S0 \mid 1S1 \mid \varepsilon$ for even-length palindromes over $\{0, 1\}$ to Chomsky normal form. Show all four stages.
5. Convert $S \rightarrow ASB \mid \varepsilon, A \rightarrow aAS \mid a, B \rightarrow SbS \mid A \mid bb$ to CNF. (Take your time with the nullable variables.)

6. A CNF grammar generates a string w with $|w| = 6$. How many rule applications does any derivation of w use? Justify your answer from the binary shape of the parse tree, and state how many of those applications are binary rules $A \rightarrow BC$ versus terminal rules $A \rightarrow a$.
7. Build a CNF grammar generating $\{a^n b^n \mid n \geq 1\}$ directly (without converting an existing grammar), then draw the parse tree for **aabb** and confirm it is binary at every internal node.
8. Give a CNF grammar for $\{w \in \{a, b\}^* \mid |w| \text{ is even and } |w| \geq 2\}$, and explain why your grammar needs (or does not need) the rule $S_0 \rightarrow \varepsilon$.
9. *Find the bug.* A student claims the following grammar is the CNF conversion of $S \rightarrow 0S0 \mid \varepsilon$:

$$S_0 \rightarrow U_0 S \mid \varepsilon, \quad S \rightarrow U_0 S \mid U_0 U_0, \quad U_0 \rightarrow 0.$$

Show that this grammar generates a string that is *not* in the original language, identify which conversion stage was done wrong, and give a correct CNF grammar.

10. *Find the bug.* During Stage 2 a student removes $A \rightarrow \varepsilon$ from $S \rightarrow AA \mid b, A \rightarrow \varepsilon \mid c$ and writes the new rules $S \rightarrow AA \mid A \mid b$. Explain what is missing, write the correct set of S -rules, and trace a short string the student's grammar fails to generate.
11. Prove that if G is in Chomsky normal form then $\varepsilon \in L(G)$ if and only if G contains the rule $S_0 \rightarrow \varepsilon$ (where S_0 is the start variable). Use the fact that no other rule can erase all output.
12. Let G be a CNF grammar with v variables. Argue that if G generates any string at all, it generates one of length at most 2^{v-1} . (Hint: in a parse tree of a shortest such string no variable repeats on any root-to-leaf path, so the tree has height at most v ; its last level (rules $A \rightarrow a$) does not branch, so bound the number of leaves of a binary tree of height $v - 1$.)

6.4 Regular Languages Are Context-Free; Closure Properties

The previous sections built grammars by hand and put them into normal form; this section places the context-free languages inside the larger landscape of language classes. Two facts do most of the work. First, every regular language is context-free, so grammars are at least as powerful as finite automata – we prove this by turning any finite automaton mechanically into a grammar. Second, the class CFL is closed under the three regular operations (union, concatenation, star) but, unlike the regular languages, it is *not* closed under intersection or complement; we give explicit counterexamples. The one idea tying these together is that a grammar's start variable can be wired to the start variables of other grammars, exactly the way an ε -arrow wires automata together in Chapter 4.

Every regular language is context-free

Theorem 6.4.1 (Regular implies context-free). *Every regular language is context-free. Equivalently, $\text{REGULAR} \subseteq \text{CFL}$.*

Proof. Let $\mathcal{L}$ be regular, so $\mathcal{L} = L(M)$ for some DFA $M = (Q, \Sigma, \delta, q_0, F)$. We build a CFG $G = (V, \Sigma, R, S)$ that simulates the run of M inside its derivations. Take one variable for each state: $V = \{R_q \mid q \in Q\}$, and let the start variable be $S = R_{q_0}$, matching the start state. The rules encode the transition function and the accepting condition:

- for every transition $\delta(q, a) = q'$, add the rule $R_q \rightarrow a R_{q'}$;
- for every accept state $q \in F$, add the rule $R_q \rightarrow \varepsilon$.

The invariant is that $R_q \Rightarrow^* w R_{q'}$ (a derivation consuming the terminals of w and leaving the single variable $R_{q'}$) exactly when $\hat{\delta}(q, w) = q'$. This is immediate by induction on $|w|$: the empty string gives $R_q \Rightarrow^* R_q$ and $\hat{\delta}(q, \varepsilon) = q$; and reading one more symbol a applies the rule $R_{q'} \rightarrow a R_{q''}$ for $\delta(q', a) = q''$, which is the recursive clause of $\hat{\delta}$. A *complete* derivation $S \Rightarrow^* w$ must end by erasing the trailing variable with an ε -rule, and an ε -rule exists only at accept states. Hence $S = R_{q_0} \Rightarrow^* w$ iff $\hat{\delta}(q_0, w) \in F$, i.e. iff $w \in L(M)$. Therefore $L(G) = L(M) = \mathcal{L}$, and $\mathcal{L}$ is context-free. $\square$

Every grammar produced this way is *right-linear*: each rule has at most one variable, and it sits at the right end. Right-linear grammars generate precisely the languages that finite automata recognize, and the proof of Theorem 6.4.1 gives one direction of this equivalence.

The construction is best seen as a structure square: passing a DFA M to its right-linear grammar G_M commutes with taking the recognized/generated language, and the resulting inclusion $\text{REGULAR} \hookrightarrow \text{CFL}$ is proper.

$$\begin{array}{ccc}
 & q \xrightarrow{a} q' \rightsquigarrow R_q \rightarrow a R_{q'} & \\
 M & \xrightarrow{q \in F \rightsquigarrow R_q \rightarrow \varepsilon} & G_M \\
 \downarrow L(\cdot) & & \downarrow L(\cdot) \\
 L(M) & \xlongequal{\hspace{1cm}} & L(G_M)
 \end{array}$$

$$\text{REGULAR} \xhookrightarrow{\text{proper}} \text{CFL}$$

Every regular language is context-free: each DFA transition $q \xrightarrow{a} q'$ becomes a rule $R_q \rightarrow a R_{q'}$ (and $R_q \rightarrow \varepsilon$ at accept states), so $L(G_M) = L(M)$; the inclusion $\text{REGULAR} \subseteq \text{CFL}$ is *proper*, e.g. $\{0^n 1^n\}$ is context-free but not regular. *Check.* The right-linear grammar's leftmost derivations mirror the DFA run step for step, so $L(G_M) = L(M)$; properness is witnessed by $\{0^n 1^n\}$, a member of $\text{CFL} \setminus \text{REGULAR}$.

Corollary 6.4.2. *The containment $\text{REGULAR} \subseteq \text{CFL}$ is proper: there are context-free languages that are not regular, for instance $\{0^n 1^n \mid n \geq 0\}$.*

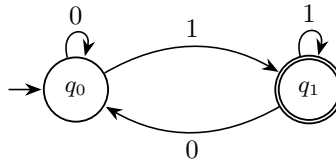
Proof. The grammar $S \rightarrow 0S1 \mid \varepsilon$ generates $\{0^n 1^n \mid n \geq 0\}$, so the language is context-free. It is not regular: a finite automaton has no way to remember an unbounded count of 0s, as the pumping lemma for regular languages makes precise. Thus CFL contains a language outside REGULAR , and the inclusion is strict. $\square$

Worked Example 6.4.3 (Convert a two-state DFA to a grammar). Let M over $\Sigma = \{0, 1\}$ have states q_0 (start) and q_1 (accept), with $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_0$, and self-loops $\delta(q_0, 0) = q_0$, $\delta(q_1, 1) = q_1$. This is the “ends in 1” machine. Build an equivalent CFG.

Recognition. The task is “regular language $\rightarrow$ grammar,” and we already have a DFA, so Theorem 6.4.1 applies directly: one variable per state, one rule per arrow, an ε -rule at each accept state.

Solution. Use variables R_0, R_1 for q_0, q_1 and start variable R_0 . Read the four arrows off the diagram and add an ε -rule at q_1 :

$$R_0 \rightarrow 0R_0 \mid 1R_1, \quad R_1 \rightarrow 1R_1 \mid 0R_0 \mid \varepsilon.$$



Which idea, and why? Each rule mirrors one DFA arrow $R_q \rightarrow a R_{q'}$ and each ε -rule marks an accept state, so a leftmost derivation is literally a transcript of the machine's run.

Check. Derive 01 : $R_0 \Rightarrow 0R_0 \Rightarrow 01R_1 \Rightarrow 01$, using $R_1 \rightarrow \varepsilon$ at the end – and indeed 01 ends in 1, so M accepts it. The string 10 forces $R_0 \Rightarrow 1R_1 \Rightarrow 10R_0$, and R_0 has no ε -rule, so no terminal string 10 is reachable: correctly rejected.

The same recipe handles an NFA, including ε -moves, with one extra rule shape.

Proposition 6.4.4 (NFA to CFG). *Every language recognized by an NFA is context-free.*

Proof. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA, so $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$. Use variables R_q as before with start variable R_{q_0} . For each state q , symbol $x \in \Sigma \cup \{\varepsilon\}$, and each $q' \in \delta(q, x)$ add the rule $R_q \rightarrow xR_{q'}$ (when $x = \varepsilon$ this is the rule $R_q \rightarrow R_{q'}$, a unit rule that takes a silent ε -move). For each $q \in F$ add $R_q \rightarrow \varepsilon$. A leftmost derivation now chooses, at each step, one of possibly several rules for R_q – exactly the nondeterministic choice the NFA makes among the branches of $\delta(q, x)$. So $S \Rightarrow^* w$ iff some accepting branch of N reads w , which is the definition of $w \in L(N)$. $\square$

Because every NFA also has an equivalent DFA, Proposition 6.4.4 adds no languages beyond Theorem 6.4.1; it is convenient because we often have an NFA in hand and want to skip the subset construction.

Method (How to turn a finite automaton into a grammar).

1. **One variable per state.** Introduce R_q for each state q ; the start variable is R_{q_0} , the variable for the start state.
2. **One rule per arrow.** For each transition on symbol a from q to q' , write $R_q \rightarrow aR_{q'}$. For an ε -move (NFA only) write the unit rule $R_q \rightarrow R_{q'}$.
3. **One ε -rule per accept state.** For each $q \in F$, write $R_q \rightarrow \varepsilon$. This is the only way a derivation can finish, so it is what stops the run in an accepting state.
4. **Check.** The start variable plays the start state; reading the grammar's rules off the diagram should let you “walk” any accepted string as a leftmost derivation.

Closure under the regular operations

The grammar constructions for union, concatenation, and star are the cleanest closure proofs in the whole theory: each adds one new start variable and a single rule scheme.

Theorem 6.4.5 (Closure of CFL under the regular operations). *The class of context-free languages is closed under union, concatenation, and Kleene star. That is, if A and B are context-free, then so are $A \cup B$, $A \cdot B$, and A^* .*

Proof. Let $A = L(G_A)$ and $B = L(G_B)$ with $G_A = (V_A, \Sigma, R_A, S_A)$ and $G_B = (V_B, \Sigma, R_B, S_B)$. Rename variables if necessary so that $V_A \cap V_B = \emptyset$; then the rule sets can be combined without interference.

Union. Add a fresh start variable S and the rule $S \rightarrow S_A \mid S_B$, keeping all rules of G_A and G_B . A derivation first commits to one of the two branches, then derives entirely within that grammar, so the generated language is $L(G_A) \cup L(G_B) = A \cup B$.

Concatenation. Add a fresh start variable S and the rule $S \rightarrow S_A S_B$. Any derivation expands S_A into a string of A and S_B into a string of B , side by side, so the language is $\{xy \mid x \in A, y \in B\} = A \cdot B$.

Star. Add a fresh start variable S and the rules $S \rightarrow SS_A \mid \varepsilon$. The ε -rule yields the empty string (so $\varepsilon \in A^*$ always), and the recursive rule appends one more copy of an A -string on each use, giving every finite concatenation $x_1 x_2 \cdots x_k$ of strings $x_i \in A$. Hence the language is A^* . $\square$

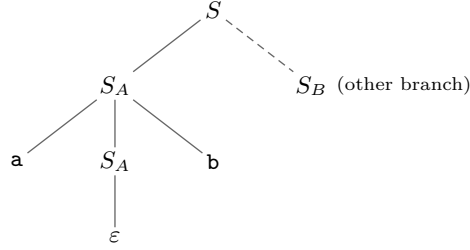
Worked Example 6.4.6 (Glue two grammars for a union). Let $A = \{a^n b^n \mid n \geq 0\}$ and $B = \{b^n a^n \mid n \geq 0\}$. Build a CFG for $A \cup B$ and show its top-level branching as a parse tree.

Recognition. “Union of two CFLs” is the trigger for the new-start-variable rule $S \rightarrow S_A \mid S_B$ from Theorem 6.4.5.

Solution. Grammars for the pieces are $S_A \rightarrow aS_A b \mid \varepsilon$ and $S_B \rightarrow bS_B a \mid \varepsilon$. Combine them under a fresh start variable:

$$S \rightarrow S_A \mid S_B, \quad S_A \rightarrow aS_A b \mid \varepsilon, \quad S_B \rightarrow bS_B a \mid \varepsilon.$$

The very first step of any derivation picks the branch; the chosen sub-grammar does the rest.



Which idea, and why? The single rule $S \rightarrow S_A \mid S_B$ makes the union: a derivation is forced to choose one sub-grammar at the root and can never mix their rules, because their variable sets are disjoint.

Check. The string $\mathbf{aabb} \in A$ comes from $S \Rightarrow S_A \Rightarrow \mathbf{a}S_A\mathbf{b} \Rightarrow \mathbf{aa}S_A\mathbf{bb} \Rightarrow \mathbf{aabb}$, while $\mathbf{ba} \in B$ comes from $S \Rightarrow S_B \Rightarrow \mathbf{b}S_B\mathbf{a} \Rightarrow \mathbf{ba}$. Both lie in $A \cup B$; no derivation produces, say, $\mathbf{abba}$ from a single branch.

Worked Example 6.4.7 (Concatenate and then star a language). Let $A = \{0^n 1^n \mid n \geq 1\}$ with grammar $S_A \rightarrow 0S_A 1 \mid 01$. Give CFGs for $A \cdot A$ and for A^* .

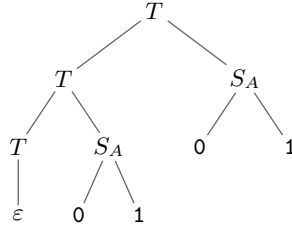
Recognition. Two closure operations stacked: first “concatenation” ($S \rightarrow S_A S_A$), then “star” ($S \rightarrow S S_A \mid \varepsilon$), both from Theorem 6.4.5.

Solution. For the concatenation, one fresh start variable suffices:

$$S \rightarrow S_A S_A, \quad S_A \rightarrow 0S_A 1 \mid 01.$$

For the star, the recursive start variable appends copies of an A -string and the ε -rule allows zero copies:

$$T \rightarrow T S_A \mid \varepsilon, \quad S_A \rightarrow 0S_A 1 \mid 01.$$



Which idea, and why? The star rule $T \rightarrow T S_A \mid \varepsilon$ is left-recursive: each use of the recursive rule appends one fresh copy of an A -string, and the ε -rule terminates the chain, generating every finite concatenation $x_1 \cdots x_k$ with $x_i \in A$ (including the empty one).

Check. The tree shown derives $T \Rightarrow^* 0101 = 0101 \in A^*$ (two copies). Taking the ε -rule at the root gives the empty string, so $\varepsilon \in A^*$, as required even though $\varepsilon \notin A$.

CFL is not closed under intersection or complement

Here the regular and context-free worlds part ways. Finite automata could be run “in parallel” to intersect two regular languages (the product construction of Section 4.2). Grammars have no such product: there is no way to force a derivation to obey two grammars at once, and in fact no construction can exist, because the closure simply fails.

Theorem 6.4.8 (CFL is not closed under intersection). *There are context-free languages A and B with $A \cap B$ not context-free.*

Proof. Let

$$A = \{\mathbf{a}^n \mathbf{b}^n \mathbf{c}^m \mid n, m \geq 0\}, \quad B = \{\mathbf{a}^m \mathbf{b}^n \mathbf{c}^n \mid n, m \geq 0\}.$$

A is context-free: the grammar $S \rightarrow X C$, $X \rightarrow \mathbf{a} X \mathbf{b} \mid \varepsilon$, $C \rightarrow \mathbf{c} C \mid \varepsilon$ matches the $\mathbf{a}$ s to the $\mathbf{b}$ s and tacks on any number of $\mathbf{c}$ s. By the symmetric grammar $S \rightarrow Z Y$, $Z \rightarrow \mathbf{a} Z \mid \varepsilon$, $Y \rightarrow \mathbf{b} Y \mathbf{c} \mid \varepsilon$, B is context-free too. Their intersection forces *both* matchings at once:

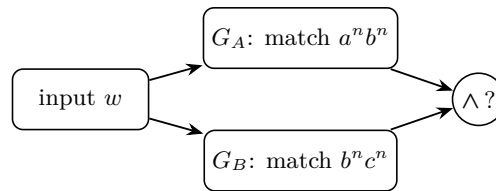
$$A \cap B = \{\mathbf{a}^n \mathbf{b}^n \mathbf{c}^n \mid n \geq 0\}.$$

This language is the standard example of a non-context-free language (the pumping lemma for CFLs, proved in the next chapter, rules it out: pumping any short factor breaks the three-way balance). Since A and B are context-free but $A \cap B$ is not, CFL is not closed under intersection. $\square$

Worked Example 6.4.9 (Why the parallel trick fails for grammars). Display the two grammars for A and B above and explain, with a schematic, why you cannot “run them together” the way the DFA product runs two automata together.

Recognition. The trigger is “intersection of CFLs”; Theorem 6.4.8 says no construction can work, so the task is to see *where* the automata product loses its footing.

Solution. For finite automata, intersection used a product machine that tracks one state from each machine simultaneously; the state set $Q_A \times Q_B$ is finite, so the combined device is still a DFA. The analogue for grammars would be a single derivation that is simultaneously a derivation in G_A and in G_B , but a derivation has only one start variable and one stack of pending variables – there is no room to carry two independent matchings.



no single grammar enforces both at once

Which idea, and why? A DFA has bounded ($|Q|$) memory, so the product state set stays finite; a grammar’s “memory” is an unbounded stack of pending variables, and one stack cannot simultaneously balance **a** against **b** and **b** against **c**.

Check. If a product construction existed it would make $\{a^n b^n c^n\}$ context-free; but that language is provably not context-free, so no such construction can exist.

Theorem 6.4.10 (CFL is not closed under complement). *There is a context-free language whose complement is not context-free.*

Proof. This follows from Theorem 6.4.8 by De Morgan’s law. The class CFL is closed under union (Theorem 6.4.5). If CFL were also closed under complement, then for any context-free A, B we could form

$$A \cap B = \overline{\overline{A} \cup \overline{B}},$$

which would make $A \cap B$ context-free, contradicting Theorem 6.4.8. Hence CFL is not closed under complement. (Concretely, one can show the complement of $\{a^n b^n c^n \mid n \geq 0\}$ is context-free, while the language itself is not, giving a direct witness.) $\square$

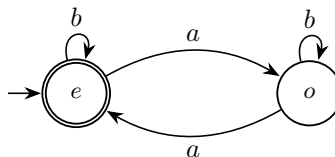
Corollary 6.4.11. *CFL is not closed under set difference, and not under intersection with itself; but the intersection of a context-free language with a regular language is always context-free.*

Proof. If CFL were closed under set difference, then $\Sigma^* \setminus A = \overline{A}$ would be context-free for every context-free A (since Σ^* is regular, hence context-free), contradicting Theorem 6.4.10. The positive statement – CFL is closed under intersection with a regular language – is proved by running a PDA for A in product with a DFA for the regular language; the DFA contributes only finitely many extra control states, so the result is still recognized by a PDA, hence context-free. $\square$

Worked Example 6.4.12 (Use the regular-intersection rule to simplify a language). Show that $\mathcal{L} = \{a^n b^n \mid n \geq 0, n \text{ even}\}$ is context-free, without designing a grammar for it from scratch.

Recognition. The set $\{a^n b^n\}$ is a familiar CFL, and “ n even” is a regular constraint; this is the “CFL $\cap$ regular” situation of Corollary 6.4.11.

Solution. Write $\mathcal{L} = C \cap D$ where $C = \{a^n b^n \mid n \geq 0\}$ is context-free (grammar $S \rightarrow aSb \mid \varepsilon$) and $D = \{w \mid w \text{ has an even number of } a\text{'s}\}$ is regular – recognized by the two-state parity automaton below.



By Corollary 6.4.11, $C \cap D$ is context-free, so $\mathcal{L}$ is context-free.

Which idea, and why? Intersecting a CFL with a regular language stays inside CFL, so a regular side-condition (here a parity test) can always be imposed on a context-free language for free.

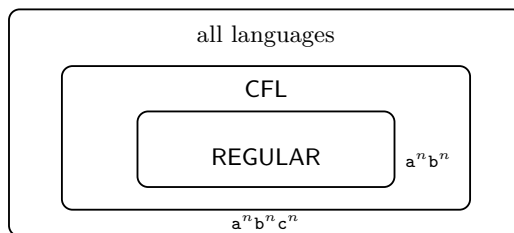
Check. $aabb \in \mathcal{L}$ (two a s, even). The automaton on $aabb$ runs $e \rightarrow o \rightarrow e \rightarrow e \rightarrow e$, ending in the accept state e , and $aabb \in C$; so it survives the intersection. The string ab has one a (odd) and is correctly excluded.

The closure picture

It is worth keeping the contrast with the regular languages in one image: both classes are closed under union, concatenation, and star, but only the regular languages are closed under intersection and complement.

operation	union	concat.	star	intersection	complement
REGULAR closed?	yes	yes	yes	yes	yes
CFL closed?	yes	yes	yes	no	no

The same information as a class diagram: **REGULAR** sits properly inside **CFL**, and the “hard” languages that separate them are the ones with a single unbounded matching ($a^n b^n$) or two competing matchings ($a^n b^n c^n$, which escapes CFL entirely).



Caution. Closure under union does *not* give closure under intersection “by symmetry.” The union proof adds a rule $S \rightarrow S_A \mid S_B$ that lets a derivation pick *one* grammar; there is no rule that forces a derivation to satisfy *both* grammars, and Theorem 6.4.8 shows no such rule could exist. Treating intersection as “just the dual of union” is the classic error here.

Caution. When you intersect a CFL with another language, check whether the second language is *regular*. If it is, the result is still context-free (Corollary 6.4.11); if it is only known to be context-free, you may have left CFL. The phrase “with the extra condition that...” usually hides a regular side-constraint, which is safe – but confirm it really is regular before relying on closure.

Caution. In the star construction $S \rightarrow SS_A \mid \varepsilon$, do not forget the ε -rule. Without it the recursion can never bottom out, and the grammar generates the empty language instead of A^* . The ε -rule is also what puts ε into A^* even when $\varepsilon \notin A$.

Exercises.

1. Convert the following DFA over $\Sigma = \{a, b\}$ into an equivalent right-linear CFG using the state-to-variable recipe. States p_0 (start), p_1 , p_2 (accept); $\delta(p_0, a) = p_1$, $\delta(p_0, b) = p_0$, $\delta(p_1, a) = p_1$, $\delta(p_1, b) = p_2$, $\delta(p_2, a) = p_1$, $\delta(p_2, b) = p_0$. Give the rule set and the start variable, then derive the string abb if it is accepted.

2. Draw a DFA recognizing $\{w \in \{0,1\}^* \mid w \text{ has an even number of 1s}\}$, then mechanically write the equivalent CFG. State which rule of your grammar corresponds to which arrow.
3. An NFA N over $\{a, b\}$ has start state s_0 , an ε -move $s_0 \rightarrow s_1$, transitions $s_1 \xrightarrow{a} s_1$, $s_1 \xrightarrow{b} s_2$, and accept state s_2 . Write the CFG produced by the NFA-to-CFG construction, being explicit about the unit rule coming from the ε -move.
4. Let $A = \{a^n \mid n \geq 0\}$ and $B = \{b^m \mid m \geq 1\}$. Give CFGs for A , B , $A \cup B$, $A \cdot B$, and A^* , in each case using exactly the closure construction from the section (a fresh start variable plus one rule scheme). Identify which language each of your start variables generates.
5. Prove that the class of *regular* languages is a proper subclass of CFL by exhibiting one CFL that is not regular and citing why a finite automaton cannot recognize it. (You need not reprove the pumping lemma; just name the obstruction.)
6. Using the union and star constructions, build a single CFG for $(\{a^n b^n \mid n \geq 0\} \cup \{b^n a^n \mid n \geq 0\})^*$. Show a parse tree for the string $abba$ (two copies, one from each side of the union).
7. *Find the bug.* A student claims the grammar below puts $A \cdot B$ into CFL, where A has grammar $S_A \rightarrow aS_A \mid \varepsilon$ and B has grammar $S_B \rightarrow bS_B \mid \varepsilon$:

$$S \rightarrow S_A \mid S_B, \quad S_A \rightarrow aS_A \mid \varepsilon, \quad S_B \rightarrow bS_B \mid \varepsilon.$$

State what language this grammar actually generates, why it is not $A \cdot B$, and give the corrected start rule.

8. Show that $\{a^n b^m \mid n, m \geq 0, n+m \text{ is even}\}$ is context-free by writing it as the intersection of a context-free language with a regular language, and naming the closure result that justifies the conclusion. Draw the small DFA for the regular factor.
9. Give two context-free languages A, B over $\{a, b, c\}$ whose intersection is not context-free, and identify $A \cap B$ explicitly. Briefly say which language-class property your example shows fails.
10. Suppose L is context-free and R is regular. Decide, with a one-line justification for each, whether the following are guaranteed context-free: (a) $L \cap R$; (b) $L \cup R$; (c) $L \cap \bar{R}$; (d) $\bar{L}$. (Note that $\bar{R}$ is regular.)
11. A classmate argues: “CFL is closed under union and under complement, because the regular languages are, and every regular language is context-free.” Pinpoint the logical flaw in this argument and state which one of the two closure claims is actually false for CFL.
12. Let $A = \{0^n 1^n \mid n \geq 0\}$. Decide whether $A \cap A$, $A \cup \bar{A}$, and A^* are context-free. For each, either give the construction or name the theorem that settles it.

6.5 Applications: Expressions, Balanced Strings, and Parsing

The machinery of the previous sections pays off when we point it at the structures programs are actually made of. The one idea that carries this section is that a grammar does not merely accept or reject a string – its parse tree *is* the meaning: the way the tree groups symbols records the intended precedence, nesting, and order of operations. We work two running applications – arithmetic expressions and balanced brackets – and then turn the question around: given a string, how do we *recover* a parse tree mechanically? We meet two parsers, the top-down recursive-descent method that mirrors the grammar directly, and the bottom-up CYK table that decides membership for any grammar in Chomsky normal form.

Grammars that encode precedence

Definition 6.5.1 (Expression grammar). Over the terminals $\{a, +, *, (,)\}$ (where a stands for any atomic operand), the *standard expression grammar* G_E has variables $\{E, T, F\}$, start variable E , and rules

$$E \rightarrow E+T \mid T, \quad T \rightarrow T*F \mid F, \quad F \rightarrow (E) \mid a.$$

Here E is an *expression* (a sum of terms), T a *term* (a product of factors), and F a *factor* (an atom or a parenthesized expression).

The three-layer shape is not decoration: it is exactly what forces $*$ to bind tighter than $+$ and makes both operators group to the left. A factor can never directly contain a $+$ unless that $+$ is wrapped in parentheses, so, when no parentheses intervene, addition always sits *above* multiplication in any parse tree. We make this precise and then read it off a picture.

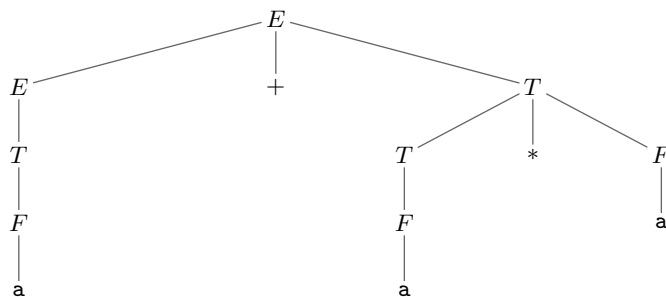
Proposition 6.5.2 (Precedence and associativity in G_E). *In every parse tree of G_E , each $*$ is applied before any $+$ that shares an operand with it, and equal-precedence operators associate to the left. Consequently G_E is unambiguous: no string has two parse trees.*

Proof. A $+$ is introduced only by the rule $E \rightarrow E+T$, whose left child is again an E and whose right child is a T ; a $*$ is introduced only by $T \rightarrow T*F$. Since no rule lets a T derive a string containing an unparenthesized $+$, the operands of any $*$ are themselves $+$ -free factors – the multiplication is resolved before the surrounding addition. Left recursion ($E \rightarrow E+T$, $T \rightarrow T*F$) puts the already-built part on the left and a single fresh operand on the right, so a chain $a+a+a$ must group as $(a+a)+a$; a right grouping would require a rule $E \rightarrow T+E$, which G_E does not have. Uniqueness follows because at each node the variable (E , T , or F) and the position of the rightmost lowest-precedence top-level operator determine the rule applied, leaving no choice. $\square$

Worked Example 6.5.3 (Reading precedence off the tree for $a + a * a$). Exhibit the parse tree of $a + a * a$ in G_E and explain why it computes the multiplication first.

Recognition. The string mixes $+$ and $*$, so the trigger is “which operator owns the root?” In G_E only $E \rightarrow E+T$ can sit at the top of a string that contains a top-level $+$, so the $+$ must be the root split.

Solution. Split at the top-level $+$: the left part a is an E (via $E \rightarrow T \rightarrow F \rightarrow a$), the right part $a * a$ is a single T (via $T \rightarrow T*F$). The $*$ therefore lives *below* the $+$ in the tree, so it is evaluated first.



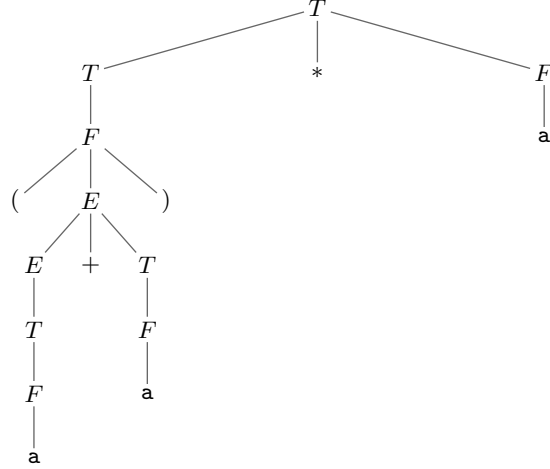
Which idea, and why? The lowest-precedence top-level operator becomes the root split; because $+$ is introduced only at the E layer and $*$ only at the deeper T layer, the tree shape forces $*$ to be the inner (earlier) operation.

Check. Erasing the variables and reading the leaves left to right gives $a+a*a = a + a * a$, the original string; the subtree over $a * a$ is a complete T , confirming the grouping $a+(a*a)$.

Worked Example 6.5.4 (Forcing the other grouping with parentheses: $(a + a) * a$). Show that $(a + a) * a$ parses with the $+$ *below* the $*$, reversing the previous grouping.

Recognition. Parentheses appear, and the only rule that produces them is $F \rightarrow (E)$. The trigger “a parenthesized block” tells us an entire expression hides inside one factor.

Solution. Now the whole string is a single term $T \rightarrow T*F$: the left T is the factor $(a+a)$ and the right F is a . Inside the parentheses an $E \rightarrow E+T$ rebuilds the sum, so the $+$ sits two layers under the $*$.



Which idea, and why? Parentheses are generated only at the factor layer, so wrapping a sum demotes its $+$ beneath every operator outside the parentheses; the tree shape, not a separate “precedence table,” is what records the override.

Check. Leaves read $(a+a)*a = (a+a)*a$. Compared with the previous example the $+$ and $*$ have swapped depths, exactly the reversal of grouping we wanted.

Caution. The “flat” grammar $E \rightarrow E+E \mid E*E \mid (E) \mid a$ generates the same language as G_E but is *ambiguous*: $a + a * a$ has one tree with $+$ at the root and another with $*$ at the root, so it fixes no precedence. Generating the right strings is not enough for an application; a compiler needs each string to have a *single* tree, which is why the layered G_E is the grammar of record.

Balanced strings

Definition 6.5.5 (The balanced-bracket language). Let $\mathcal{L}_{\text{bal}} \subseteq \{ (,) \}^*$ be the set of strings of parentheses that are *balanced*: every prefix has at least as many $($ as $)$, and the totals are equal. The grammar G_B with start variable S and rules

$$S \rightarrow (S)S \mid \varepsilon$$

generates $\mathcal{L}_{\text{bal}}$.

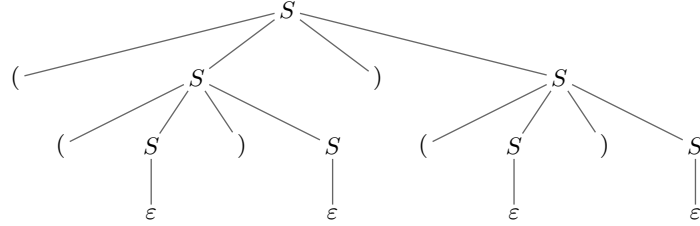
Proposition 6.5.6. $L(G_B) = \mathcal{L}_{\text{bal}}$.

Proof. ($\subseteq$) Every string S derives is balanced, by induction on the derivation length: ε is balanced, and if x, y are balanced then so is $(x)y$, since wrapping a balanced block keeps every prefix non-negative and preserves equal totals. ($\supseteq$) Let w be balanced and nonempty. Its first symbol is $($ (a leading $)$ would make a prefix go negative). Scan to the *first* place the running count returns to 0; that closing $)$ matches the opening $($, splitting $w = (x)y$ with x and y both balanced and shorter than w . By induction $S \xRightarrow{*} x$ and $S \xRightarrow{*} y$, so $S \rightarrow (S)S \xRightarrow{*} (x)y = w$. $\square$

Worked Example 6.5.7 (Parsing a nested-then-adjacent string $(())()$). Give a parse tree for $(())()$ in G_B .

Recognition. The rule $S \rightarrow (S)S$ has *two* recursive slots: the inner S (what sits between a matched pair) and the trailing S (what comes after). The trigger “a closed block followed by more text” maps onto exactly that left/right split.

Solution. The first $($ matches the fourth symbol $)$ (running count first returns to 0 there), giving $x = ()$ inside and $y = ()$ after. Recurse on each: $()$ is $(\varepsilon)\varepsilon$.



Which idea, and why? Each application of $S \rightarrow (S)S$ corresponds to one matched pair plus everything to its right, so the tree's right spine walks the outermost pairs left to right while inner subtrees hold their contents – the bracket nesting and the tree nesting coincide.

Check. Reading leaves and dropping the ε 's gives $((()))() = (())()$. The running count 1, 2, 1, 0, 1, 0 never dips below 0 and ends at 0, confirming balance.

A taste of recursive-descent parsing

A grammar whose rules are written without left recursion can be turned into a parser almost mechanically: give each variable a procedure that reads input and tries to match that variable's right-hand sides. Left recursion ($E \rightarrow E+T$) would loop forever – the procedure for E would call itself before consuming anything – so we first rewrite G_E into the equivalent *right-recursive* form

$$E \rightarrow T E', \quad E' \rightarrow +T E' \mid \varepsilon, \quad T \rightarrow F T', \quad T' \rightarrow *F T' \mid \varepsilon, \quad F \rightarrow (E) \mid a,$$

which generates the same language but lets a parser decide each step by looking at a single upcoming symbol.

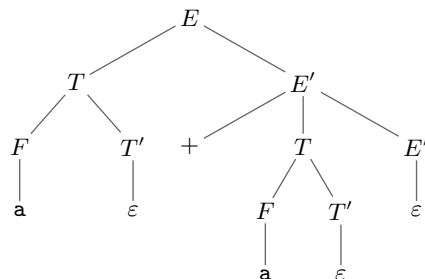
Method (Recursive-descent parsing (predictive, one-symbol lookahead)).

1. **Remove left recursion** so no procedure can call itself without first consuming input.
2. **One procedure per variable.** Procedure V is responsible for matching some string derivable from V starting at the current input position.
3. **Choose a rule by the next symbol.** For variable V with rules $V \rightarrow \alpha_1 \mid \dots \mid \alpha_k$, peek at the current input symbol and pick the unique α_i that can begin with it; if an ε -rule is present, use it when no other alternative's first symbol matches.
4. **Expand the chosen rule left to right.** For each symbol of α_i : if it is a terminal, check it equals the input and advance; if it is a variable, call that variable's procedure.
5. **Accept** iff the whole input is consumed and every call returned successfully.

Worked Example 6.5.8 (Tracing a predictive parse of $a + a$). Trace the recursive-descent parser on $a + a$ using the right-recursive grammar above, and read off the parse tree it builds.

Recognition. Each variable's rule is selected by one lookahead symbol: E, T, F are entered on a or $($, while E' and T' either fire on their operator or vanish via ε . The trigger is “the next symbol uniquely names the rule,” which is precisely what makes the grammar parseable top-down.

Solution. With the cursor before a : E calls T , which calls F ; lookahead a chooses $F \rightarrow a$ and consumes it. Back in T , the call to T' sees $+$ (not $*$), so $T' \rightarrow \varepsilon$. Back in E , E' sees $+$, fires $E' \rightarrow +T E'$: consume $+$, call T on the second a (again $T \rightarrow F T'$ with $T' \rightarrow \varepsilon$), then the inner E' sees end-of-input and takes ε . Input fully consumed: accept.



Which idea, and why? Because the left-factored grammar lets one lookahead symbol pick each rule, the parser never backtracks; the recursion stack at its deepest point is exactly the path from the root to the current leaf of the tree being built.

Check. The leaves spell $\mathbf{a}\varepsilon + \mathbf{a}\varepsilon\varepsilon = \mathbf{a} + \mathbf{a}$, and every call returned at end-of-input, so the parse succeeds.

Caution. Recursive descent of this simple, no-backtracking kind needs the grammar to be *left-factored* and *free of left recursion*; otherwise the one-symbol lookahead cannot pick a rule. Feeding the left-recursive G_E directly to the method makes the procedure for E call E immediately and spin forever. Always transform first.

The CYK membership test

Recursive descent is fast but fussy about the grammar. The Cocke–Younger–Kasami (CYK) algorithm trades that away: it decides $w \in L(G)$ for *any* grammar in Chomsky normal form, by filling a table from the bottom up. Recall that in CNF every rule is $A \rightarrow BC$ or $A \rightarrow a$ (with the lone exception $S \rightarrow \varepsilon$). The algorithm rests on a simple recursion on substrings.

Theorem 6.5.9 (CYK). *Let G be in Chomsky normal form and $w = a_1a_2 \cdots a_n$ with $n \geq 1$. For $1 \leq i \leq j \leq n$ let $X_{i,j} = \{A \mid A \xRightarrow{*} a_i \cdots a_j\}$ be the variables generating the substring from position i to j . Then*

$$X_{i,i} = \{A \mid A \rightarrow a_i \text{ is a rule}\}, \quad X_{i,j} = \bigcup_{k=i}^{j-1} \{A \mid A \rightarrow BC, B \in X_{i,k}, C \in X_{k+1,j}\} \quad (i < j),$$

and $w \in L(G)$ iff the start variable $S \in X_{1,n}$ (together with the convention that $\varepsilon \in L(G)$ iff $S \rightarrow \varepsilon$ is a rule).

Proof. A length-one substring a_i is generated by A iff $A \rightarrow a_i$ is a rule, since the only CNF rules producing a single terminal are the $A \rightarrow a$ rules; that is the base case. For $j > i$, a CNF derivation $A \xRightarrow{*} a_i \cdots a_j$ must begin with a rule $A \rightarrow BC$ (no terminal rule can start a length- ≥ 2 derivation), after which B derives a nonempty prefix $a_i \cdots a_k$ and C the nonempty suffix $a_{k+1} \cdots a_j$ for some split point k with $i \leq k < j$. Conversely any such $B \in X_{i,k}$ and $C \in X_{k+1,j}$ assemble a derivation for A . Taking the union over all split points gives the displayed recursion. Setting $i = 1, j = n$ and asking whether $S \in X_{1,n}$ is exactly asking whether $S \xRightarrow{*} w$. $\square$

Each $X_{i,j}$ depends only on entries for strictly shorter substrings, so we fill the table by increasing substring length $\ell = j - i + 1$. The result is an $O(n^3 \cdot |G|)$ membership test – polynomial, in contrast to the unbounded search a naive derivation hunt would do.

Method (Running CYK on a CNF grammar).

1. **Index the input** $w = a_1 \cdots a_n$ and make a triangular table whose cell (i, j) ($i \leq j$) will hold $X_{i,j}$.
2. **Length-1 row.** For each i , put into cell (i, i) every variable A with a rule $A \rightarrow a_i$.
3. **Longer substrings.** For $\ell = 2, 3, \dots, n$ and each start i (with $j = i + \ell - 1$), scan every split k from i to $j - 1$: whenever $B \in X_{i,k}$ and $C \in X_{k+1,j}$ and $A \rightarrow BC$ is a rule, add A to cell (i, j) .
4. **Decide.** Accept iff the start variable lies in cell $(1, n)$.

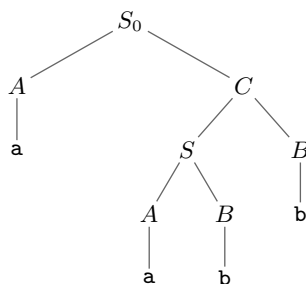
Worked Example 6.5.10 (A full CYK chart for $\mathbf{aabb}$). Let G be the CNF grammar with start variable S_0 and rules $S_0 \rightarrow AB \mid AC, S \rightarrow AB \mid AC, C \rightarrow SB, A \rightarrow \mathbf{a}, B \rightarrow \mathbf{b}$. (One checks $L(G) = \{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\}$.) Run CYK on $w = \mathbf{aabb}$.

Recognition. The grammar is in CNF and the question is pure membership, so the trigger is “decide $w \in L(G)$ for a CNF grammar” – the CYK table. We build it by substring length.

Solution. Index $a_1a_2a_3a_4 = \mathbf{aabb}$. Length 1: cell (i, i) gets A over each $\mathbf{a}$ and B over each $\mathbf{b}$. Then climb. The filled triangle (cell (i, j) at row i , column j ; cell (i, j) holds $X_{i,j}$, the variables generating $a_i \cdots a_j$):

$i \backslash j$	1 (a)	2 (a)	3 (b)	4 (b)
1	$\{A\}$	$\emptyset$	$\emptyset$	$\{S_0, S\}$
2		$\{A\}$	$\{S_0, S\}$	$\{C\}$
3			$\{B\}$	$\emptyset$
4				$\{B\}$

Working the nontrivial cells in order of length. Length 2: cell (1, 2) over **aa** has only $A \in X_{1,1}$ and $A \in X_{2,2}$, and no rule has right-hand side AA , so $\emptyset$; cell (2, 3) over **ab** takes the split $k = 2$ with $A \in X_{2,2}$, $B \in X_{3,3}$, rules $S_0 \rightarrow AB$ and $S \rightarrow AB$, giving $\{S_0, S\}$; cell (3, 4) over **bb** has only B, B and no BB -rule, so $\emptyset$. Length 3: cell (1, 3) over **aab** tries $k = 1$ ($A \in X_{1,1}$, $X_{2,3} = \{S_0, S\}$, but $S_0 \rightarrow AC$ and $S \rightarrow AC$ need C , not S_0 or S) and $k = 2$ ($X_{1,2} = \emptyset$), so $\emptyset$ – consistent with **aab** $\notin L(G)$; cell (2, 4) over **abb** tries $k = 2$ ($X_{3,4} = \emptyset$) and $k = 3$ ($X_{2,3} = \{S_0, S\}$, $X_{4,4} = \{B\}$, rule $C \rightarrow SB$), giving $\{C\}$. Length 4: cell (1, 4) over **aabb** tries $k = 2, 3$ (both involve an empty sub-cell) and $k = 1$ ($A \in X_{1,1}$, $X_{2,4} = \{C\}$, rules $S_0 \rightarrow AC$ and $S \rightarrow AC$), giving $\{S_0, S\}$. Since $S_0 \in X_{1,4}$, accept.



Which idea, and why? CYK fills cell (i, j) from all ways to split $a_i \cdots a_j$ into two shorter blocks the grammar already knows how to make; the parse tree above is recovered by reading which split and which rule populated (1, 4), then recursing into the chosen sub-cells.

Check. The displayed tree's leaves spell **aabb** and $S_0 \in X_{1,4}$, so the algorithm and the tree agree. By contrast, running the same fill on **abb** leaves $S_0 \notin X_{1,3}$, correctly rejecting an unbalanced string.

Caution. CYK as stated requires the grammar to be in Chomsky normal form: the split into exactly two nonempty halves assumes binary rules. Feeding it a grammar with a rule like $A \rightarrow BCD$ or with stray ε -rules breaks the recursion, because a substring could then arise from three pieces or from an empty piece. Convert to CNF first (Section 6.3); only then is filling the triangular table valid. Also remember the ε corner case – the table starts at length 1, so membership of the empty string is decided separately by whether $S \rightarrow \varepsilon$ is a rule.

Exercises.

- Using the standard expression grammar G_E of Definition 6.5.1, draw the parse tree of **a * a + a**. Mark which operation is performed first and explain, by pointing at the tree, why the grammar groups it as **(a*a)+a** rather than **a*(a+a)**.
- In G_E , draw the parse tree of **a + a + a** and use it to argue that $+$ associates to the *left*. Then describe in one sentence what single rule change would make it associate to the right instead.
- Extend G_E with a subtraction operator $-$ at the same precedence as $+$ and a division operator $/$ at the same precedence as $*$. Write the augmented rules, then draw the parse tree of **a - a * a** and confirm the multiplication binds tighter.
- Show that the string **a + a * a** has two distinct parse trees in the flat grammar $E \rightarrow E+E \mid E * E \mid (E) \mid a$. Draw both trees and state, for each, the order in which it evaluates the operators.
- Give a context-free grammar for the language of *balanced strings over two bracket types* $\{(,), [,]\}$, in which $($ matches only $)$ and $[$ matches only $]$ and brackets are properly nested. Then draw a parse tree for $([])[]$.

6. For the grammar G_B ($S \rightarrow (S)S \mid \varepsilon$) of Definition 6.5.5, draw the parse tree of $((()))$. Verify balance by listing the running open-bracket count after each symbol and checking it never goes negative and ends at 0.
7. Rewrite the grammar $S \rightarrow S;S \mid \mathbf{s} \mid \varepsilon$ (a toy “statement list”) in a form free of left recursion suitable for recursive-descent parsing, and explain in one sentence why the original form would send the descent parser into an infinite loop.
8. Apply the recursive-descent method to the right-recursive expression grammar ($E \rightarrow TE'$, $E' \rightarrow +TE' \mid \varepsilon$, $T \rightarrow FT'$, $T' \rightarrow *FT' \mid \varepsilon$, $F \rightarrow (E) \mid \mathbf{a}$). Trace the parser on $\mathbf{a} * (\mathbf{a} + \mathbf{a})$, listing the sequence of rules chosen and the lookahead symbol that selects each one.
9. Convert the grammar $S \rightarrow 0S1 \mid 01$ to Chomsky normal form, then run CYK on the input 0011 . Show the completed triangular table and state whether the string is accepted.
10. *Find the bug.* A student writes the grammar $E \rightarrow T \mid E+T$, $T \rightarrow F \mid F*T$, $F \rightarrow \mathbf{a} \mid (E)$ and claims it gives the same precedence *and* associativity as G_E . Identify the one place where the associativity differs from G_E , draw the parse tree of $\mathbf{a} * \mathbf{a} * \mathbf{a}$ in each grammar, and say which way each groups.
11. *Find the bug.* Someone runs CYK on $\mathbf{aba}$ using the grammar $S \rightarrow \mathbf{aSa} \mid \mathbf{b}$ “in Chomsky normal form.” Explain precisely why CYK cannot be applied to this grammar as written, identify every rule that violates CNF, and describe the repair needed before the table can be filled.
12. Prove that for any grammar G in Chomsky normal form and any nonempty w of length n , the CYK table has exactly $n(n+1)/2$ cells, and the value in cell (i, j) depends only on cells (i, k) and $(k+1, j)$ with $k < j$. Conclude that filling the table by increasing substring length never reads a cell before it has been computed.

Chapter 7

Pushdown Automata and Non-Context-Free Languages

7.1 Pushdown Automata: Model and Computation

A pushdown automaton (PDA) is a finite automaton given one new piece of hardware: a stack, an unbounded last-in first-out memory that it may push onto and pop from as it reads the input. The finite control still has only finitely many states, so the genuinely new power comes entirely from the stack; the one key idea of this section is that a stack lets the machine remember an *unbounded* amount of information about what it has read, but only in a strictly nested, last-written-first-read order. That single device is exactly enough to count, to match brackets, and to compare a string against its reversal – tasks a finite automaton provably cannot do – yet not enough to do everything. We make the model precise, fix the diagram notation $a, b \rightarrow c$ for a transition, define what it means for a PDA to accept by following an accepting branch of its nondeterministic computation, and then build several PDAs from scratch.

Definition 7.1.1 (Pushdown automaton). A *pushdown automaton* is a 6-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where

- Q is a finite set, the *states*;
- Σ is a finite set, the *input alphabet*;
- Γ is a finite set, the *stack alphabet*;
- $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the *transition function*, where $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ and $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$;
- $q_0 \in Q$ is the *start state*;
- $F \subseteq Q$ is the set of *accept states*.

Two features distinguish this from a DFA. First, the machine has *two* alphabets: Σ for the input it reads and Γ for the symbols it stores; these need not be related, and the stack alphabet usually carries a few extra bookkeeping symbols. Second, δ returns a *set* of moves, so the model is *nondeterministic*: in a given situation the machine may have several legal next moves, or none. The domain factor Γ_ε on the input side means an ε in the top-of-stack slot lets the machine move without popping; the codomain factor Γ_ε means an ε in the pushed slot lets it move without pushing. Likewise an ε input lets it move without consuming an input symbol.

Reading a transition. A single move is written on the diagram as a label

$$a, b \rightarrow c$$

on an edge from state p to state q , with the meaning: *when reading a from the input and finding b on top of the stack, the machine may move to q , popping b and pushing c* . Formally this is the statement $(q, c) \in \delta(p, a, b)$. Any of a, b, c may be ε :

- $a = \varepsilon$: take the move without consuming an input symbol (a “free” move);

- $b = \varepsilon$: take the move *without* popping (the stack top is ignored and untouched);
- $c = \varepsilon$: take the move *without* pushing (so if $b \neq \varepsilon$, the net effect is a pop).

Thus a pure push is $a, \varepsilon \rightarrow c$, a pure pop is $a, c \rightarrow \varepsilon$, and a replace is $a, b \rightarrow c$.

Remark. The definition gives no way to test directly whether the stack is empty or whether the input has ended. The standard trick handles both. Push a private bottom marker $\$ \in \Gamma$ first; seeing $\$$ on top again means the stack is otherwise empty. And because acceptance is checked only when the input has been fully read (next definition), reaching an accept state *after* the last symbol has the effect of testing for end-of-input.

Definition 7.1.2 (Computation and acceptance). Let $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be a PDA and $w \in \Sigma^*$. The machine *accepts* w if w can be written as $w = y_1 y_2 \cdots y_m$ with each $y_i \in \Sigma_\varepsilon$, and there exist states $r_0, r_1, \dots, r_m \in Q$ and stack strings $s_0, s_1, \dots, s_m \in \Gamma^*$ such that

- (1) $r_0 = q_0$ and $s_0 = \varepsilon$ (start in the start state with an empty stack);
- (2) for each $i = 0, \dots, m-1$ there are $a, b \in \Gamma_\varepsilon$ and $t \in \Gamma^*$ with $s_i = at$, $s_{i+1} = bt$, and $(r_{i+1}, b) \in \delta(r_i, y_{i+1}, a)$ (each step obeys δ , touching only the top of the stack);
- (3) $r_m \in F$ (the last state is an accept state).

The *language recognized by* M is $L(M) = \{w \in \Sigma^* \mid M \text{ accepts } w\}$. (Theorem 7.2.1 will show that a language is context-free exactly when some PDA recognizes it.)

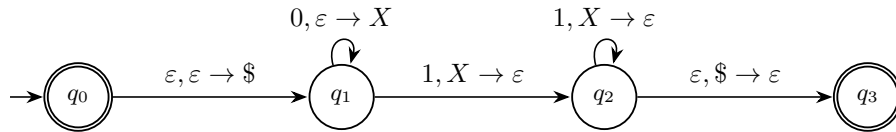
Writing the top of the stack on the *left*, condition (2) says each step pops a and pushes b while leaving the deeper contents t untouched. Because δ may offer several moves, M 's computation branches into a tree; M accepts as soon as *one* branch reads all of w and ends in F . A branch that gets stuck (no legal move) or ends outside F simply fails, and failing branches never veto an accepting one. This “some branch accepts” rule is what makes nondeterminism a genuine asset here, not just convenience.

Remark. Unlike finite automata, deterministic and nondeterministic PDAs are *not* equivalent: some context-free languages are recognized only by nondeterministic PDAs. We use the nondeterministic model throughout because it is the one that matches context-free grammars exactly (Section 7.2). The deterministic restriction is taken up in Section 7.5.

Worked Example 7.1.3 (Counting then matching: the language $\{0^n 1^n \mid n \geq 0\}$). Build a PDA recognizing $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ over $\Sigma = \{0, 1\}$.

Recognition. No finite automaton can recognize this – it would have to remember an unbounded count of 0s. The trigger is “count one block, then match it against a second block of equal size”: push a marker for every symbol in the first block, pop one for every symbol in the second.

Solution. Use a private bottom marker $\$$ to detect the empty stack. Step one: in the start state q_0 , push $\$$ for free and go to q_1 . Step two: in q_1 , for each 0 push a stack symbol X . Step three: on the first 1, move to q_2 and begin popping one X per 1. Step four: when $\$$ surfaces (every X popped, so the counts matched), pop it for free and go to the accept state q_3 . Make q_0 accepting as well, so that $n = 0$ (the empty string) is accepted; no move ever returns to q_0 .



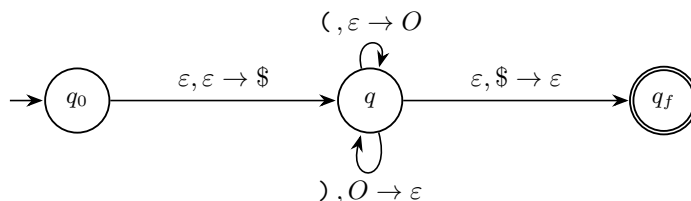
Which idea, and why? An “equal counts of two consecutive blocks” condition is the canonical stack job: push during the first block and pop during the second, with a bottom marker to certify that the push and pop totals agreed exactly.

Check. On 0011: q_0 pushes $\$$ ($\rightarrow q_1$); two 0s push XX (stack $XX\$$); first 1 pops one X ($\rightarrow q_2$); second 1 pops the other; $\$$ on top, free pop to $q_3 \in F$ – accepted. On 001: after the single 1 the stack is $X\$$ and the input is exhausted in $q_2 \notin F$ – rejected. The empty string is accepted in q_0 before any move.

Worked Example 7.1.4 (Balanced brackets: the Dyck language on $\{(,)\}$). Let $\Sigma = \{(,)\}$ and let $\mathcal{L}$ be the strings of *properly nested* parentheses (every prefix has at least as many $($ as $)$, and the totals are equal). Build a PDA for $\mathcal{L}$.

Recognition. A bracket-matching condition. Each (opens an obligation that some later) must close, in nested order – exactly last-in first-out, the natural stack discipline. The trigger “nested matching” says: push on open, pop on close, and accept only with an empty stack.

Solution. One working state q suffices besides the bookkeeping. Start state q_0 pushes the bottom marker $\$$ and moves to q . In q : reading (pushes an O ; reading) pops an O – and if no O is available (a) with nothing open), no move applies and the branch dies, correctly rejecting. When the input ends with the stack back to $\$$, a free pop to the accept state q_f certifies balance.



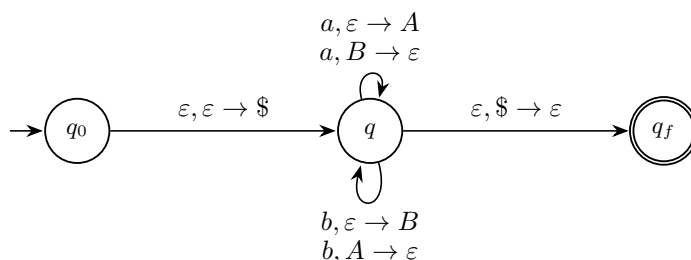
Which idea, and why? Nested bracket matching is the defining use of a stack: each opener pushes a token and each closer pops the most recent token, so a closer with an empty stack (an unmatched) has no legal move and rejects, while leftover tokens at end-of-input block the final free pop.

Check. On ((): push O , push O (stack $OO\$$), pop O , pop O (stack $\$$), free pop to q_f – accepted. On ()): after () the stack is $\$$, then the stray) finds no O to pop – the branch dies, rejected. On ((: input ends with $OO\$$ on the stack in $q \notin F$ – rejected.

Worked Example 7.1.5 (Comparing counts in any order: equal numbers of **a** and **b**). Over $\Sigma = \{a, b\}$, let $\mathcal{L} = \{w \mid w \text{ has equally many as and bs}\}$ (the as and bs may be interleaved arbitrarily). Build a PDA for $\mathcal{L}$.

Recognition. Now order does not matter, only the running *difference* between the two counts. The trigger is “net count of one symbol against another”; a stack can hold a tally whose sign tells which symbol is currently ahead, by keeping at most one kind of token at a time.

Solution. Keep on the stack a unary tally of the current surplus, using token **A** for “as ahead” and token **B** for “bs ahead.” Reading **a**: if a **B** is on top, a **b** was waiting, so cancel it (pop **B**); otherwise push an **A**. Symmetrically for **b**. (The diagram also lets **a** push an **A** onto a **B**; such extra branches are harmless, since every move on **a** raises the number of **As** minus the number of **Bs** on the stack by one and every move on **b** lowers it by one, so $\$$ can resurface only when the counts agree.) Accept exactly when the surplus is zero, i.e. the marker $\$$ is back on top.



Which idea, and why? When two symbol counts must agree but may interleave, store the *signed difference* as a unary stack tally: one token type per sign, pushing to grow the surplus and popping to cancel against the opposite symbol, so an empty tally at the end means the counts matched.

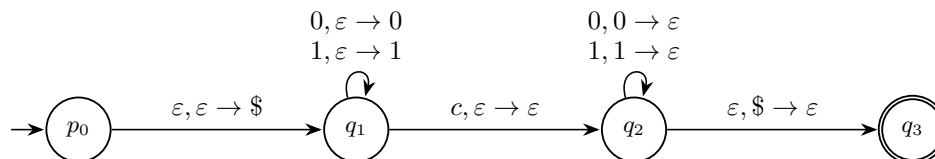
Check. On **abba**: **a** pushes **A** (stack $A\$$); **b** sees **A** on top, pops it (stack $\$$); **b** pushes **B** (stack $B\$$); **a** sees **B**, pops it (stack $\$$); free pop to q_f – accepted. On **aab**: stack ends $A\$$ (one surplus **a**) in $q \notin F$ – rejected.

Worked Example 7.1.6 (Reversal with a center marker: $\{wcw^{\mathcal{R}} \mid w \in \{0, 1\}^*\}$). Over $\Sigma = \{0, 1, c\}$, build a PDA for $\mathcal{L} = \{wcw^{\mathcal{R}} \mid w \in \{0, 1\}^*\}$, where $w^{\mathcal{R}}$ is w reversed and c marks the exact center.

Recognition. Matching a string against its reversal is a stack’s other signature task: push the first half, then pop it against the second half. Because a stack reverses the order it returns symbols in, the top of the stack

after the first half is exactly the symbol that the mirror image must match first. The explicit center marker c tells the machine *when* to switch from pushing to popping, so no guessing is needed.

Solution. Phase one (state q_1): push every input symbol (0 pushes 0, 1 pushes 1) onto the stack. On reading c , switch to phase two (state q_2) without touching the stack. Phase two: for each input symbol, pop the matching symbol – 0 must meet a 0 on top, 1 must meet a 1. A mismatch finds no legal move and the branch dies. When the bottom marker $\$$ surfaces, the halves matched symbol for symbol; free-pop it to the accept state q_3 .



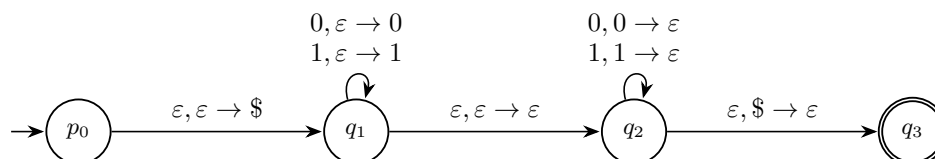
Which idea, and why? To test that the second half is the reverse of the first, push the first half and then pop it: a stack returns symbols in reversed order, so popping while reading the mirror half lines up w against w^R automatically, and the center marker removes any need to guess the midpoint.

Check. On 01c10: push 0, push 1 (stack 10\$); read c ($\rightarrow q_2$); read 1, pop matching 1; read 0, pop matching 0 (stack \$); free pop to q_3 – accepted. On 01c11: after c , the first input 1 pops the top 1 fine, but the second input 1 now meets 0 on top – no move, rejected.

Worked Example 7.1.7 (Where nondeterminism is essential: even-length palindromes $\{ww^R\}$). Over $\Sigma = \{0, 1\}$, build a PDA for $\mathcal{L} = \{ww^R \mid w \in \{0, 1\}^*\}$ – the even-length palindromes, now *without* a center marker.

Recognition. This is the previous example with the center hidden. The machine cannot see where w ends and w^R begins, so it must *guess* the midpoint. The trigger “the split point is not marked” is the classic cue for nondeterminism: branch on every position, and let the one correct guess carry an accepting computation.

Solution. Phase one (state q_1): push each symbol read, just as before. At *any* moment, take a free ϵ -move to phase two (state q_2) – this is the nondeterministic guess that the center has been reached. Phase two: pop a matching symbol for each remaining input symbol. If the guessed center was right and the string is a palindrome, this branch empties the stack to $\$$ exactly as the input runs out; free-pop to accept in q_3 . Wrong guesses die or end off-marker and are harmless.



Which idea, and why? When the place to switch from storing to matching is not visible in the input, replace the marked switch by a free ϵ -move that may fire at any position: nondeterminism explores every candidate midpoint at once, and acceptance needs only the single branch that guessed correctly.

Check. On 0110: the branch that guesses the center after the first two symbols pushes 0, 1 (stack 10\$), ϵ -jumps to q_2 , then pops 1 against input 1 and 0 against input 0, reaching $\$$ as input ends – accepted. On 0100: *no* split point lets both halves match (it is not a palindrome), so every branch either dies on a mismatch or ends off- $\$$ – rejected.

Method (How to design a PDA from a description of its language).

1. **Decide what the stack must remember.** The finite control handles only a bounded flag (which phase you are in, the last few symbols); push onto the stack precisely the *unbounded* quantity – a count, a saved prefix, a signed surplus – that the finite memory cannot hold.
2. **Install a bottom marker.** From the start state, take a free move $\epsilon, \epsilon \rightarrow \$$ to push a private symbol $\$$. Seeing $\$$ on top later is your only legal test for “stack otherwise empty.”

3. **Push during the storing phase.** For each input symbol that grows the stored quantity, push one stack token. Use distinct token types only when you must distinguish what was stored (e.g. 0 vs 1 for a reversal, or a sign for a surplus).
4. **Pop during the matching phase.** For each input symbol that should consume stored information, pop the token it must match. A required token that is absent gives *no legal move*; let the branch die – that is how a PDA rejects.
5. **Switch phases.** If the input marks the switch point, switch on that symbol. If it does not, switch with a free ε -move that may fire anywhere – this is the nondeterministic guess.
6. **Accept on an empty stack at end of input.** Take a free $\varepsilon, \$ \rightarrow \varepsilon$ move into an accept state once $\$$ resurfaces. Trace one string that should be accepted, one that should be rejected for the wrong counts, and the empty string.

Caution. The label $a, b \rightarrow c$ is a *pop-then-push*, not a free read. Writing $a, \varepsilon \rightarrow c$ pushes c without inspecting the stack; writing $a, c \rightarrow \varepsilon$ requires a c on top and pops it. Confusing $a, \varepsilon \rightarrow c$ (always legal, pushes) with $a, c \rightarrow \varepsilon$ (legal only when c is on top, pops) is the most common modeling error – the first never tests the stack, the second is your only way to test it.

Caution. A PDA does not reject by going to a “dead state.” It rejects a branch simply by having *no applicable move*: if the top of the stack is wrong for every outgoing transition, that branch halts and fails. Do not add explicit trap states as you would for a DFA; with nondeterminism, a dead branch is the normal and correct way to discard a bad guess, and adding a catch-all transition can accidentally make the machine accept strings it should reject.

Caution. Acceptance requires *both* an accept state *and* the input fully read; the empty stack is not automatically required by the definition unless you build the machine to test for it. If you mean “accept only with empty stack,” you must route through a free $\varepsilon, \$ \rightarrow \varepsilon$ move to the accept state. Conversely, ending in an accept state with symbols still stacked above $\$$ is fine by the formal definition unless your design forbids it – so decide deliberately whether leftover stack contents should block acceptance.

Exercises.

1. Consider the PDA $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{A, \$\}, \delta, q_0, \{q_0, q_2\})$ whose moves are: q_0 on $\varepsilon, \varepsilon \rightarrow \$$ goes to q_1 ; q_1 on $a, \varepsilon \rightarrow A$ loops; q_1 on $b, A \rightarrow \varepsilon$ goes to q_2 ; q_2 on $b, A \rightarrow \varepsilon$ loops; q_2 on $\varepsilon, \$ \rightarrow \varepsilon$ goes to q_0 . Draw its state diagram, then trace its computation on **aabb** and on **aab**. Which does M accept, and what language does M recognize?
2. Design a PDA over $\Sigma = \{0, 1\}$ recognizing $\{0^n 1^{2n} \mid n \geq 0\}$ (twice as many 1s as 0s). Draw the state diagram and explain in one sentence how many tokens you push per 0 and why.
3. Design a PDA over $\{a, b\}$ recognizing $\{a^n b^m \mid m \geq n \geq 0\}$ (at least as many bs as as, all as before all bs). Draw the diagram and say what the machine does with the “extra” bs after the stack is exhausted.
4. Design a PDA over $\{(,)\}$ recognizing the strings of brackets in which the number of (equals the number of) but *nesting need not be proper* (e.g.)(is accepted). Draw the diagram and contrast it in one sentence with the Dyck-language PDA of the section.
5. Design a PDA over $\{0, 1, \#\}$ recognizing $\{x \# x^R \mid x \in \{0, 1\}^*\}$, where $\#$ marks the exact center. Draw the diagram, then trace **10#01**.
6. Design a PDA over $\{a, b, c\}$ recognizing $\{a^i b^j c^k \mid i = j + k, i, j, k \geq 0\}$ (the number of as equals the number of bs and cs combined). Draw the diagram and identify which block is pushed and which blocks pop.
7. Let M be the PDA over $\{a, b\}$ with one working state q (besides the bottom-marker setup), q on $a, \varepsilon \rightarrow A$ looping, q on $b, A \rightarrow \varepsilon$ looping, and a free $\varepsilon, \$ \rightarrow \varepsilon$ move to the accept state. Describe $L(M)$ precisely in words, and decide whether **abab** and **aabb** are accepted.
8. Design a PDA recognizing $\{w \in \{a, b\}^* \mid w \text{ has more as than bs}\}$ (strictly more). Draw the diagram and say what must change in the equal-counts PDA of the section to make the surplus condition strict.

9. Give a PDA over $\{0, 1\}$ for the *odd*-length palindromes $\{w c w^R \mid w \in \{0, 1\}^*, c \in \{0, 1\}\}$ with the unmarked center symbol c . Draw the diagram and explain how the single middle symbol is consumed without pushing or popping.
10. Argue in two or three sentences why the language $\{0^n 1^n \mid n \geq 0\}$ of the section is recognized by a PDA whose every reachable configuration holds at most one kind of stack token besides $\$,$ and why this means a stack *depth* but not a richer data structure is what the language demands.
11. *Find the bug.* A student's PDA for the balanced-bracket (Dyck) language uses a single state q that is both the start state and an accept state, with q on $(, \varepsilon \rightarrow O$ looping and q on $), O \rightarrow \varepsilon$ looping, and *no* bottom marker $\$$. Find a string this machine accepts that it should reject, and fix the machine by adding exactly the marker mechanism from the section.
12. *Find the bug.* Someone builds a PDA for $\{0^n 1^n \mid n \geq 1\}$ in which the push transition is written $0, \varepsilon \rightarrow X$ but the pop transition is mistakenly written $1, \varepsilon \rightarrow \varepsilon$ (popping nothing). Explain precisely why this machine accepts strings outside the language, name one such string, and give the corrected pop transition.
13. Prove that the class of context-free languages is closed under *union*: given PDAs M_1 and M_2 recognizing $\mathcal{L}_1$ and $\mathcal{L}_2$, describe a PDA recognizing $\mathcal{L}_1 \cup \mathcal{L}_2$ using a single new start state and free ε -moves, and explain in one sentence why nondeterminism makes this construction correct.

7.2 Equivalence of PDAs and CFGs

Chapter 6 and Section 7.1 gave us two ways to talk about the context-free languages: as the strings *generated* by a context-free grammar, and as the strings *recognized* by a pushdown automaton. This section proves that these two descriptions pick out exactly the same class of languages. The one key idea is that a stack is precisely the right amount of memory to carry out a derivation: a PDA can keep the unexpanded tail of a sentential form on its stack and replace its top variable by the right-hand side of a rule, while a grammar can be built whose variables track “what happens to the stack” as the PDA runs between two states. Having both descriptions is what makes the theory usable – to show a language is context-free you may exhibit *either* a grammar *or* a PDA, whichever is more convenient.

Theorem 7.2.1 (Equivalence of PDAs and CFGs). *A language is context-free if and only if some pushdown automaton recognizes it.*

The two directions are quite different in flavour, so we prove them as separate lemmas. The forward direction (grammar to machine) is a direct simulation and is the one you will use constantly. The reverse direction (machine to grammar) is the clever one; we develop it carefully because the variables it introduces are easy to misread.

$$\text{CFG } G \xrightleftharpoons[\text{Lemma 7.2.5}]{\text{Lemma 7.2.2}} \text{PDA } P$$

From a grammar to a pushdown automaton

Lemma 7.2.2 (Every CFL is recognized by a PDA). *If $\mathcal{L}$ is generated by a context-free grammar $G = (V, \Sigma, R, S)$, then some pushdown automaton P recognizes $\mathcal{L}$.*

Proof. The idea is to let P carry an intermediate string of the derivation on its stack and check, by nondeterministic guessing, that the input is one of the strings G can derive. We keep on the stack only the suffix of the current sentential form from its leftmost variable onward; any terminals to the left of that variable have already been matched against the input and discarded.

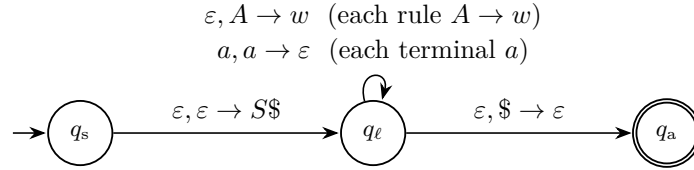
P has three core states q_s (start), q_ℓ (the main loop), and q_a (accept), plus auxiliary states used only to push a multi-symbol right-hand side one symbol at a time. Its stack alphabet is $\Gamma = V \cup \Sigma \cup \{\$, \}$, where $\$$ is a fresh bottom-of-stack marker. The transitions are:

1. **Initialize.** $\delta(q_s, \varepsilon, \varepsilon) = \{(q_\ell, S\$)\}$: push the start variable on top of the marker, then enter the loop.

2. **Expand a variable.** For every rule $A \rightarrow w$ in R , let $\delta(q_\ell, \varepsilon, A)$ contain (q_ℓ, w) : if a variable A is on top, pop it and push the string w (leftmost symbol of w ending up on top), reading nothing from the input.
3. **Match a terminal.** For every terminal $a \in \Sigma$, let $\delta(q_\ell, a, a) = \{(q_\ell, \varepsilon)\}$: if a terminal a is on top *and* the next input symbol is a , pop it and consume that input symbol.
4. **Accept.** $\delta(q_\ell, \varepsilon, \$) = \{(q_a, \varepsilon)\}$: if only the marker remains, the whole sentential form has been matched, so move to the accept state.

A string w is accepted iff some sequence of expansions turns S into w while every pushed terminal is matched in order against w – that is, iff $S \Rightarrow^* w$ in G . Hence $L(P) = L(G) = \mathcal{L}$. $\square$

The whole machine looks the same for every grammar; only the expansion edges in step 2 change. We draw it once, with the rule set of G abbreviated on the self-loop.



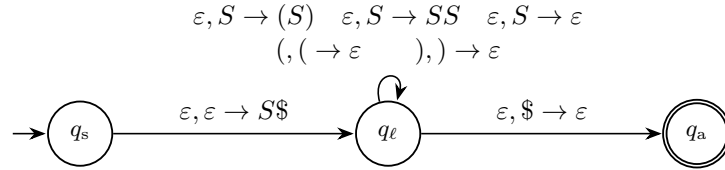
Worked Example 7.2.3 (Build a PDA from a balanced-bracket grammar). Let G have start variable S and rules

$$S \rightarrow (S) \mid SS \mid \varepsilon,$$

which generates the balanced strings over $\{(\,,\,)\}$. Construct a PDA recognizing $L(G)$ and trace it on $((\,))$.

Recognition. This is the forward direction of Theorem 7.2.1: a grammar is given, and we want a machine. The trigger is “convert a CFG to a PDA,” which is exactly Lemma 7.2.2 – run the universal construction with G ’s rules on the expansion loop.

Solution. Take $\Gamma = \{S, (,), \$\}$. Initialize with $S\$$, loop while expanding S by one of its three rules or matching a bracket, and accept when only $\$$ is left. The expansion edges are $\varepsilon, S \rightarrow (S)$, $\varepsilon, S \rightarrow SS$, and $\varepsilon, S \rightarrow \varepsilon$; the matching edges are $(, (\rightarrow \varepsilon$ and $),) \rightarrow \varepsilon$.



Which idea, and why? The forward direction is a uniform simulation: the PDA stack holds the unmatched suffix of a sentential form, an expansion edge per grammar rule replaces a top variable, and a match edge per terminal cancels a top terminal against the input.

Check. Trace $((\,))$, writing the stack top-first (the marker $\$$ is at the bottom).

read so far	move	stack
ε	init	$S\$$
ε	$S \rightarrow (S)$	$(S)\$$
$($	match $($	$S)\$$
$(($	$S \rightarrow (S)$	$(S))\$$
$(($	match $($	$S))\$$
$(($	$S \rightarrow \varepsilon$	$))\$$
$(($	match $)$	$)\$$
$(($	match $)$	$\$$
$(($	$\$ \rightarrow \varepsilon$	(accept)

The input is exhausted exactly as the stack reduces to $\$$, so $((\,))$ is accepted, as it should be.

From a pushdown automaton to a grammar

The reverse direction is the substantial one. We are given a PDA and must *generate* its language with a grammar. The trick is to introduce, for each ordered pair of states p, q , a variable A_{pq} whose job is to derive exactly the strings that drive the machine from p to q while leaving the stack as it found it, never touching what was already on it.

Definition 7.2.4 (Generating variables A_{pq}). Let $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{acc}\})$ be a PDA. For states $p, q \in Q$, the variable A_{pq} is intended to generate every string x that can take P from state p with an empty stack to state q with an empty stack.

To make the bookkeeping clean we first put P into a normal form: a single accept state, an empty stack at the moment of accepting, and every move either a pure *push* (read a symbol or ε , push one stack symbol) or a pure *pop* (read a symbol or ε , pop one stack symbol) – never both at once and never neither. Any PDA can be massaged into this shape by adding a few states, so assume P already has it.

Lemma 7.2.5 (Every PDA language is context-free). *If a pushdown automaton P recognizes $\mathcal{L}$, then some context-free grammar generates $\mathcal{L}$.*

Proof. Build G with variables $\{A_{pq} \mid p, q \in Q\}$ and start variable $A_{q_0 q_{acc}}$. The rules come in three families, each capturing one shape of computation that takes P from p to q on an empty stack:

1. **(matched ends)** For all $p, q, r, s \in Q$, all $u \in \Gamma$, and all $a, b \in \Sigma \cup \{\varepsilon\}$, if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε) , add

$$A_{pq} \rightarrow a A_{rs} b.$$

Here the very first move pushes u and the very last move pops the *same* u , so the stack is empty only at the two ends.

2. **(split at empty stack)** For all $p, q, r \in Q$, add

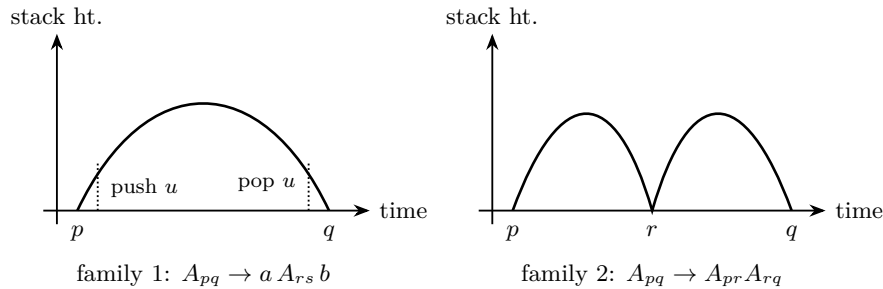
$$A_{pq} \rightarrow A_{pr} A_{rq},$$

for the case where the stack also becomes empty at some intermediate state r .

3. **(do nothing)** For all $p \in Q$, add $A_{pp} \rightarrow \varepsilon$, recording that the empty string takes p to itself.

One proves by induction on the length of a derivation (respectively a computation) that $A_{pq} \Rightarrow^* x$ holds *iff* x drives P from p to q with empty stacks at both ends. Specializing to $p = q_0$ and $q = q_{acc}$ – and using that P empties its stack before accepting – gives $A_{q_0 q_{acc}} \Rightarrow^* x$ *iff* P accepts x . Hence $L(G) = L(P) = \mathcal{L}$. $\square$

The two non-trivial rule families correspond to two pictures of how the stack height rises and falls during a computation from p to q . The figure below plots stack height against time. On the left, the height returns to zero only at the ends, so the first and last moves are a matched push/pop pair (family 1). On the right, the height touches zero at an interior state r , splitting the run into two independent empty-to-empty pieces (family 2).



Worked Example 7.2.6 (Apply the A_{pq} construction to a tiny PDA). Let P have states $\{p, q, f\}$ with p the start state and f the only accept state, input alphabet $\{a, b\}$, stack alphabet $\{u\}$, and exactly two

moves: a push $(q, u) \in \delta(p, a, \varepsilon)$ and a pop $(f, \varepsilon) \in \delta(q, b, u)$. Thus P pushes one u on an a and pops it on a b , accepting ab . Produce a grammar for $L(P)$.

Recognition. This is the reverse direction: a machine is given, a grammar is wanted. The trigger “convert a PDA to a CFG” invokes Lemma 7.2.5, so we instantiate the three rule families using only the two transitions P actually has.

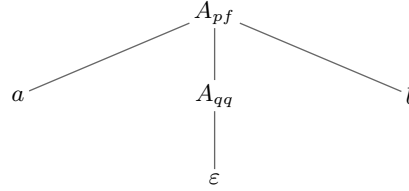
Solution. The variables are the nine A_{xy} with $x, y \in \{p, q, f\}$, and the start variable is A_{pf} . *Family 1* needs a push and a matching pop. The only push is $(q, u) \in \delta(p, a, \varepsilon)$ (so $r = q$, the pushed symbol u , input letter a); the only pop of u is $(f, \varepsilon) \in \delta(q, b, u)$ (so $s = q$, input letter b). With p on the left and the final state f on the right, this gives the single matched-ends rule

$$A_{pf} \rightarrow a A_{qq} b.$$

Family 3 gives $A_{pp} \rightarrow \varepsilon$, $A_{qq} \rightarrow \varepsilon$, and $A_{ff} \rightarrow \varepsilon$. *Family 2* rules $A_{pf} \rightarrow A_{pr} A_{rf}$ exist but every one of them contains a variable with no terminating derivation except through the ε -rules, so after substitution they add nothing new. Keeping only what is reachable and productive from A_{pf} :

$$A_{pf} \rightarrow a A_{qq} b, \quad A_{qq} \rightarrow \varepsilon.$$

Substituting $A_{qq} \Rightarrow \varepsilon$ yields the lone string ab , exactly $L(P)$.



Which idea, and why? Family 1 mirrors a matched push/pop pair: the input letter a read while pushing u becomes the left terminal, the letter b read while popping that same u becomes the right terminal, and the inner variable A_{rs} generates whatever happened in between.

Check. P on ab : from p read a and push u to reach q ; read b and pop u , reaching f with empty stack – accept. The grammar derives $A_{pf} \Rightarrow a A_{qq} b \Rightarrow ab$. P accepts, and the grammar generates, exactly ab .

Worked Example 7.2.7 (The matched-ends rule on a counting PDA: deriving $a^n b^n$). A PDA P for $\mathcal{L} = \{a^n b^n \mid n \geq 1\}$ pushes a marker for each a and pops one for each b . After normalizing it has a push state p and a pop state q with a self-push $(p, u) \in \delta(p, a, \varepsilon)$, a hand-off pop $(q, \varepsilon) \in \delta(p, b, u)$, and a self-pop $(q, \varepsilon) \in \delta(q, b, u)$; to empty its stack before accepting, it also pushes a bottom marker $\$$ on a free move from its start state s to p and pops it on a free move from q to its accept state f . Read off the recursive grammar rule for the a/b matching.

Recognition. Again the reverse direction, but now we focus on the *recursion* that family 1 creates when the inner pair of states equals the outer pair. The trigger is “one push at the start matched by one pop at the very end with the same computation nested inside,” which is exactly the self-referential matched-ends rule.

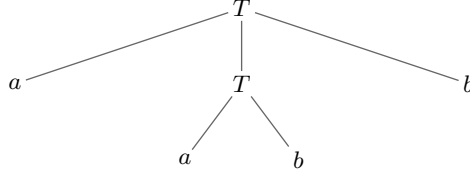
Solution. The self-push $(p, u) \in \delta(p, a, \varepsilon)$ (read a , push u , go $p \rightarrow p$) paired with the self-pop $(q, \varepsilon) \in \delta(q, b, u)$ (read b , pop u , go $q \rightarrow q$) gives a matched-ends rule with outer states p, q and inner states p, q :

$$A_{pq} \rightarrow a A_{pq} b.$$

A base case comes from the hand-off $(q, \varepsilon) \in \delta(p, b, u)$ together with a push: pairing the self-push at p with this pop gives $A_{pq} \rightarrow a A_{pp} b$, and $A_{pp} \rightarrow \varepsilon$ closes it off, so effectively $A_{pq} \rightarrow ab$ for the shortest case. Writing $T = A_{pq}$, the productive grammar is

$$T \rightarrow a T b \mid ab,$$

the textbook grammar for $a^n b^n$ with $n \geq 1$. Each application of $T \rightarrow a T b$ wraps one a on the left and one b on the right, mirroring one push now matched by one pop later. The marker push and pop pair up into the only other family-1 rule, $A_{sf} \rightarrow A_{pq}$ (with $a = b = \varepsilon$), so the start variable A_{sf} generates exactly what T does.



Which idea, and why? A matched-ends rule whose inner state pair coincides with its outer pair is *recursive* ($A_{pq} \rightarrow aA_{pq}b$); this recursion is exactly how the grammar records that an unbounded number of pushes must be balanced by the same number of later pops.

Check. The tree above derives $T \Rightarrow aTb \Rightarrow aabb = \mathbf{aabb}$ when the inner T uses $T \rightarrow ab$, matching P 's run on $\mathbf{aabb}$ between the two marker moves: push, push, pop, pop, ending in q with only $\$$ left.

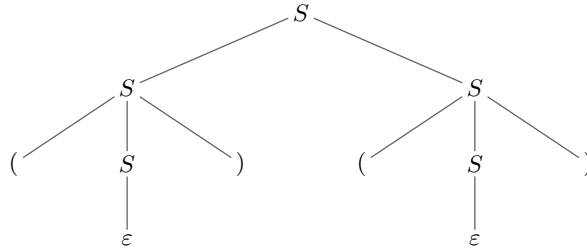
Worked Example 7.2.8 (A leftmost derivation read as a parse tree). For the bracket grammar $S \rightarrow (S) \mid SS \mid \varepsilon$ of the first worked example, give a leftmost derivation of $()()$ and draw its parse tree.

Recognition. A grammar is given and we want the *structure* it assigns to a string. The trigger “exhibit a derivation / parse tree” calls for choosing rules whose yield is the target string, recorded as a tree whose leaves spell the string left to right.

Solution. Use SS first to make room for two bracket pairs, then expand each S :

$$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()().$$

The corresponding parse tree has the start symbol at the root, the rule $S \rightarrow SS$ branching it into two S subtrees, each of which expands by $S \rightarrow (S)$ with an inner $S \rightarrow \varepsilon$.



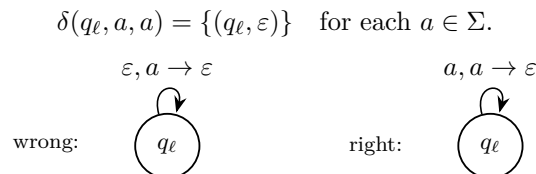
Which idea, and why? A parse tree displays *which* rules were applied and how their yields nest, ignoring the order of expansion; reading its leaves left to right recovers the generated string $()()$.

Check. The leaves left to right are $(, \varepsilon,), (, \varepsilon,)$, which concatenate to $()()$ as required, and every internal node matches a rule of G .

Worked Example 7.2.9 (Find the bug: an over-eager terminal-match edge). A student converts the grammar $S \rightarrow \mathbf{a}S \mid \mathbf{b}$ to a PDA but writes the terminal-matching family as $\delta(q_\ell, \varepsilon, a) = \{(q_\ell, \varepsilon)\}$ for each terminal a – popping a terminal from the stack on an ε -move instead of on a real input symbol. Why is the resulting machine wrong, and what is the fix?

Recognition. This tests the precise form of the match edge in Lemma 7.2.2. The trigger is a match transition that reads ε rather than the terminal itself, decoupling the stack from the input.

Solution. With $\delta(q_\ell, \varepsilon, a) = \{(q_\ell, \varepsilon)\}$ the PDA may discard a terminal sitting on the stack *without consuming any input*. So after pushing the right-hand side of a rule, it can silently delete the terminals instead of checking them against w . Since the match edges were the only moves that read input, the machine now never consumes an input symbol at all: it accepts exactly the empty string, which is not in $L(S) = \{\mathbf{a}^n\mathbf{b} \mid n \geq 0\}$, and rejects every string that is, such as $\mathbf{b}$ and $\mathbf{ab}$. The fix is the correct match edge, which reads the terminal from the input as it pops it:



Which idea, and why? The match family of the CFG-to-PDA construction must couple the stack to the input: a terminal is popped *only* while reading that same terminal, so the input letters are verified in order against the derived string.

Check. With the corrected edge, the terminal **b** on top can be popped only when the next input symbol is **b**, so **aab** is accepted (**a** matches, **a** matches, **b** matches, marker pops) while **aba** jams at the trailing **a** with no **a** on the stack to match.

Method (How to convert between CFGs and PDAs).

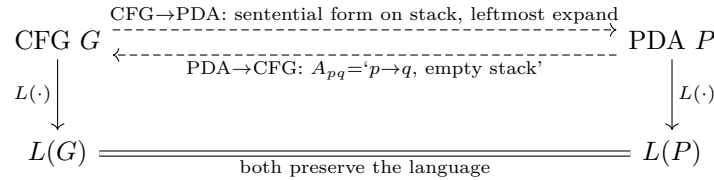
1. **CFG $\rightarrow$ PDA (Lemma 7.2.2).** Use the fixed three-state skeleton $q_s \rightarrow q_\ell \rightarrow q_a$. Initialize with $\varepsilon, \varepsilon \rightarrow S\$$. On the loop at q_ℓ , add one expansion edge $\varepsilon, A \rightarrow w$ for each rule $A \rightarrow w$, and one match edge $a, a \rightarrow \varepsilon$ for each terminal a . Accept with $\varepsilon, \$ \rightarrow \varepsilon$. Push a multi-symbol right-hand side so that its *leftmost* symbol ends on top.
2. **PDA $\rightarrow$ CFG (Lemma 7.2.5).** First normalize: one accept state, empty stack at acceptance, every move a pure push or a pure pop. Make a variable A_{pq} for each ordered pair of states. Add (1) $A_{pq} \rightarrow aA_{rs}b$ whenever a push $(r, u) \in \delta(p, a, \varepsilon)$ is matched by a pop $(q, \varepsilon) \in \delta(s, b, u)$; (2) $A_{pq} \rightarrow A_{pr}A_{rq}$ for every middle state r ; (3) $A_{pp} \rightarrow \varepsilon$. Start variable $A_{q_0 q_{acc}}$.
3. **Prune.** Delete variables not reachable from the start variable and variables that derive no terminal string; the surviving rules are the working grammar.
4. **Sanity-check** both ways: trace one accepted and one rejected string on the machine, and derive the accepted one in the grammar.

Caution. In the CFG-to-PDA construction, mind the *push order*. When you push a right-hand side $w = w_1w_2 \cdots w_k$ in step 2, the symbol that must be processed first (w_1) has to end up on *top* of the stack. A real PDA pushes one symbol per move, so you implement $\varepsilon, A \rightarrow w_1w_2 \cdots w_k$ through a short chain of auxiliary states that pushes w_k first and w_1 last. Pushing in the wrong order matches the reversal of the right-hand side and breaks the machine.

Caution. The variable A_{pq} generates the strings that take the PDA from p to q *starting and ending with an empty stack*, not merely “from p to q .” This empty-to-empty condition is what makes the inductive proof work, and it is why the construction first forces the machine to empty its stack before accepting. Reading A_{pq} as “any path from p to q ” will make families 1 and 2 look arbitrary; reading it as “a self-contained stack episode from p to q ” makes both rules obvious.

Caution. Do not skip the normal form before the PDA-to-CFG construction. If a transition both pushes and pops (or does neither), there is no clean “first push, last pop” to anchor family 1, and the matched-ends rule has nothing to read off. Splitting every such transition in two through a new intermediate state – a pure pop followed by a pure push, or, for a move that does neither, a push of a dummy symbol followed by its pop – is cheap and is what licenses the entire rule set.

Putting the two lemmas back to back, the content of Theorem 7.2.1 is that each construction is *language-preserving*: the grammar and the machine it is turned into (and back) all name one and the same set of strings.



Takeaway. A language is context-free iff some PDA recognizes it: the dashed top edge (CFG $\rightarrow$ PDA) keeps the current sentential form on the stack and expands its leftmost variable, the dashed return edge (PDA $\rightarrow$ CFG) introduces a variable A_{pq} for “go from state p to state q , starting and ending with an empty stack,” and each map commutes with $L(\cdot)$, so the square closes as $L(G) = L(P)$ – membership in CFL is the preserved invariant.

Check. The CFG $\rightarrow$ PDA construction simulates leftmost derivations on the stack, so P accepts w exactly when $S \Rightarrow^* w$ in G ; the PDA $\rightarrow$ CFG construction’s A_{pq} generate exactly the strings that drive P from p

(empty stack) to q (empty stack), so $A_{q_0 q_{acc}} \Rightarrow^* w$ exactly when P accepts w . Hence $L(G) = L(P)$ in both directions.

Caution. This square equates *nondeterministic* PDAs with CFL. It does *not* say $\text{DPDA} = \text{CFL}$: deterministic PDAs recognize only the strict subclass $\text{DCFL} \subsetneq \text{CFL}$ (Section 7.5), so the top and return arrows above must be read as constructions for the *nondeterministic* model – the $\text{CFG} \rightarrow \text{PDA}$ direction in particular relies on the freedom to guess which rule to expand.

Exercises.

1. Let G have start variable S and rules $S \rightarrow 0S1 \mid T$, $T \rightarrow 0T \mid \varepsilon$. Using the construction of Lemma 7.2.2, draw the three-state PDA recognizing $L(G)$, labelling every expansion and match edge on the loop at q_ℓ .
2. For the PDA you built in the previous exercise, trace the stack contents move by move on the input 0011, writing the stack top-first at each step, and confirm that the run ends with only the marker $\$$ on the stack.
3. Convert the grammar $S \rightarrow \mathbf{aSa} \mid \mathbf{bSb} \mid \varepsilon$ (even-length palindromes over $\{\mathbf{a}, \mathbf{b}\}$ with a fixed centre) into a PDA. Then give a one-sentence informal description of how the resulting machine uses its stack.
4. Take the PDA P with start state s , accept state f , stack alphabet $\{u, \$\}$, the marker moves $(p, \$) \in \delta(s, \varepsilon, \varepsilon)$ and $(f, \varepsilon) \in \delta(q, \varepsilon, \$)$, and the two moves $(p, u) \in \delta(p, a, \varepsilon)$ (self-push at p) and $(q, \varepsilon) \in \delta(p, b, u)$ (read $\mathbf{b}$, pop, go to q), plus $(q, \varepsilon) \in \delta(q, b, u)$ (self-pop at q). Carry out the construction of Lemma 7.2.5: list the variables A_{pq} , write down every family-1, family-2, and family-3 rule, then prune to a clean grammar and identify the language.
5. In the construction of Lemma 7.2.5, explain in two or three sentences how a matched-ends rule whose inner state pair equals its outer state pair (i.e. $A_{pq} \rightarrow a A_{pq} b$) lets the grammar generate arbitrarily long strings, and connect this to the stack-height picture rising and falling once.
6. Give a leftmost derivation and the parse tree of the string $\mathbf{aabb}$ in the grammar $T \rightarrow \mathbf{aTb} \mid TT \mid \varepsilon$. Then give a *different* leftmost derivation of $\mathbf{aabb}$ in the same grammar, and conclude that this grammar is ambiguous.
7. Prove that the class of context-free languages is closed under union by working with grammars: given CFGs $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ with disjoint variable sets, exhibit a CFG for $L(G_1) \cup L(G_2)$ and justify in one sentence why it generates exactly that union.
8. Using Theorem 7.2.1 in the direction that is convenient, prove that every regular language is context-free. (Hint: a DFA is a stack-ignoring PDA; or convert the DFA's transition function directly into grammar rules $R_i \rightarrow aR_j$ with $R_i \rightarrow \varepsilon$ at accept states.)
9. *Find the bug.* A classmate converts $S \rightarrow \mathbf{abS} \mid \varepsilon$ to a PDA but, when implementing the expansion edge for $S \rightarrow \mathbf{abS}$, pushes the right-hand side so that $\mathbf{a}$ ends up on the *bottom* of the three pushed symbols and S on top. Explain why their machine accepts the wrong language, name the order in which the three symbols must actually be pushed, and state which string their machine wrongly accepts or rejects.
10. *Find the bug.* Someone carrying out Lemma 7.2.5 skips the normalization step and keeps a single transition $(r, u) \in \delta(p, a, v)$ that pops v and pushes u in one move ($v \neq \varepsilon$). Explain precisely why no family-1 rule can be read off this transition, and describe the two-step replacement (through a new state) that repairs it.
11. Let P be a PDA with k states. Counting only the variables A_{pq} , how many variables does the grammar of Lemma 7.2.5 have before pruning? Give the exact count as a function of k , and explain in one sentence why family-2 contributes $O(k^3)$ rules.
12. Argue that for *every* context-free grammar the CFG-to-PDA construction of Lemma 7.2.2 produces a PDA with the same three core states q_s, q_ℓ, q_a (plus auxiliary push states), and explain why nondeterminism is essential to its correctness even when the original grammar is unambiguous.

7.3 The Pumping Lemma for Context-Free Languages

Pushdown automata gave us machines for the context-free languages; this section gives us the tool that draws the *boundary* of that class from the other side. The one key idea is that a context-free grammar

with finitely many variables cannot generate an arbitrarily long string without some variable repeating on a single root-to-leaf path of the parse tree, and that repetition is a lever: the subtree between the two copies of the repeated variable can be cloned or deleted, inflating or shrinking the string while keeping it in the language. Made precise, this is the *pumping lemma for context-free languages*, the exact analogue of the regular pumping lemma of Section 5.3, but pumping *two* pieces at once instead of one. We state it, prove it by surgery on parse trees, and then use it to convict several languages of *not* being context-free.

From tall trees to repeated variables. The proof rests on a counting fact about trees. Recall that a parse tree for a grammar G has the start variable at its root, variables at internal nodes, and terminals at its leaves; reading the leaves left to right spells the generated string. If every rule of G has at most b symbols on its right-hand side, then every internal node has at most b children, so a tree of height h has at most b^h leaves. Contrapositively, a long string forces a tall tree, and a tall tree forces a long root-to-leaf path, on which only finitely many variable names are available — so one must recur.

Lemma 7.3.1 (Tall trees repeat a variable). *Let G have variable set V and let every rule have right-hand side of length at most $b \geq 2$. If a parse tree of G yields a string of length at least $b^{|V|} + 1$, then some root-to-leaf path contains at least $|V| + 1$ variable nodes, and hence (by the pigeonhole principle) some variable R of G occurs at least twice on that path.*

Proof. A tree of height h has at most b^h leaves, so a tree with more than $b^{|V|}$ leaves has height at least $|V| + 1$; its longest root-to-leaf path then has at least $|V| + 1$ edges and therefore at least $|V| + 2$ nodes. The bottom node is a leaf (a terminal or ε) and the remaining $\geq |V| + 1$ nodes on the path are variables. Since G has only $|V|$ distinct variables, two of these $|V| + 1$ variable nodes carry the same name R . $\square$

Theorem 7.3.2 (Pumping lemma for context-free languages). *For every context-free language A there is a number $p \geq 1$, the pumping length, such that every string $s \in A$ with $|s| \geq p$ can be written as*

$$s = uvxyz$$

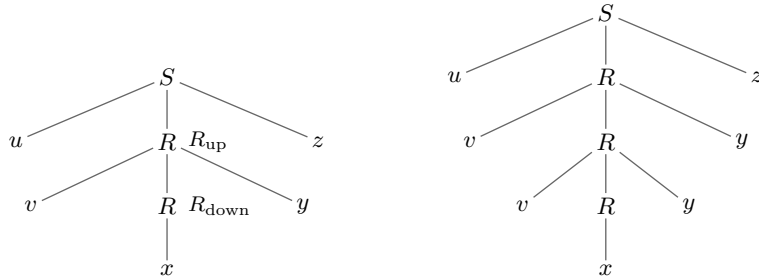
with

- (1) $uv^i xy^i z \in A$ for every $i \geq 0$;
- (2) $|vy| > 0$ (that is, v and y are not both empty); and
- (3) $|vxy| \leq p$.

Proof. Let G be a CFG generating A , let $b \geq 2$ bound the length of every right-hand side, and let V be the variable set. Put $p = b^{|V|+1}$.

Take any $s \in A$ with $|s| \geq p$ and let τ be a parse tree for s with the fewest nodes among all parse trees for s . Since $p \geq 2b^{|V|} \geq b^{|V|} + 1$, by Lemma 7.3.1 some variable repeats on a longest root-to-leaf path of τ ; among the lowest $|V| + 1$ variables on that path, pick two occurrences of one name R , the higher call R_{up} and the lower call R_{down} .

Each occurrence of R heads a subtree that derives a contiguous block of s . The subtree under R_{up} derives a block we name vxy , and inside it the subtree under R_{down} derives the shorter block x . Everything to the left of the R_{up} block is u and everything to its right is z , so indeed $s = uvxyz$ and $R \Rightarrow^* vxy$, $R \Rightarrow^* x$ (figure on the left below).



Condition 1. Because R_{up} and R_{down} are the *same* variable, either subtree is a legal replacement for the other. Splicing the larger R_{up} subtree in place of the smaller R_{down} subtree adds one more copy of v on the left and y on the right; doing it $i - 1$ times yields a parse tree for $uv^i xy^i z$ (the right-hand figure shows $i = 2$). Splicing the smaller subtree in place of the larger deletes both, giving a parse tree for $uxz = uv^0 xy^0 z$. Hence $uv^i xy^i z \in A$ for every $i \geq 0$.

Condition 2. If v and y were both empty, then replacing the R_{up} subtree by the R_{down} subtree would produce a parse tree for the *same* string s but with strictly fewer nodes, contradicting the minimality of τ . So $|vy| > 0$.

Condition 3. We chose R among the lowest $|V| + 1$ variables on a longest path, so the subtree under R_{up} has height at most $|V| + 1$. A tree of height $|V| + 1$ yields a string of length at most $b^{|V|+1} = p$, so $|vxy| \leq p$. $\square$

Remark. Two pieces, not one. The regular pumping lemma splits $s = xyz$ and pumps the *single* middle block y . The context-free lemma splits $s = uvxyz$ and pumps the *pair* v, y in lockstep, the same number of times on each side. That paired pumping is exactly what a parse tree's nested self-embedding $R \Rightarrow^* vRy$ produces: each extra level adds a v on the left and a y on the right at once.

How the lemma is used. The pumping lemma is a one-directional implication: *context-free* $\Rightarrow$ *pumpable*. To prove a language L is *not* context-free we use the contrapositive. Assume L is context-free, obtain its pumping length p (a fixed but unknown number), choose one clever string $s \in L$ with $|s| \geq p$, and then show that *no* split $s = uvxyz$ obeying conditions 2 and 3 can satisfy condition 1: for some pumping index i the string $uv^i xy^i z$ falls out of L . That single forced failure contradicts the lemma, so the assumption was wrong. This is an adversary game: the adversary picks p and the split, and you must defeat *every* legal split.

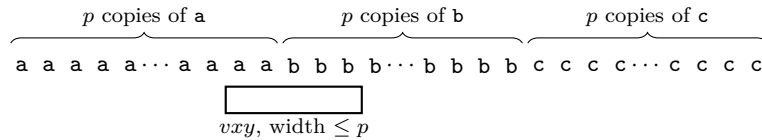
Worked Example 7.3.3 ($\{a^n b^n c^n \mid n \geq 0\}$ is not context-free). Show that $L = \{a^n b^n c^n \mid n \geq 0\}$ is not a CFL.

Recognition. Equal counts of *three* symbols is the canonical non-CFL: a stack can match two blocks but not three. The trigger is “a CFG/PDA matches at most one pair of blocks,” which says paired pumping will unbalance the triple. So apply Theorem 7.3.2.

Solution. Assume L is context-free with pumping length p , and take

$$s = a^p b^p c^p \in L, \quad |s| = 3p \geq p.$$

Let $s = uvxyz$ be any split satisfying conditions 2 and 3. By condition 3, $|vxy| \leq p$, so the window vxy spans at most p consecutive symbols and therefore cannot reach from the a -block all the way into the c -block: vxy touches at most two of the three letters. Pump up to $i = 2$, giving $uv^2 xy^2 z$. By condition 2 at least one extra symbol is inserted, and those extra symbols come from at most two of the three letters. Whichever letter vxy misses keeps its count at p , while at least one other count grows past p . The three counts are now unequal, so $uv^2 xy^2 z \notin L$, contradicting condition 1.



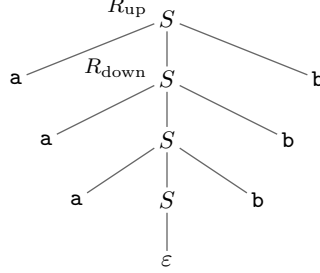
Which idea, and why? Condition 3 confines vxy to a window of width $\leq p$, which cannot cover all three blocks at once; pumping then changes at most two counts and leaves the third behind, so equality of all three counts is impossible to maintain.

Check. Concretely, if vxy lands inside the ab region with $v = a^k$, $y = b^m$ and $k + m > 0$, then $uv^2 xy^2 z = a^{p+k} b^{p+m} c^p$, which has $p + k$ or $p + m$ exceeding p (the c -count), so it is not in L .

Worked Example 7.3.4 (A grammar's own parse tree, pumped). Take G with the rules $S \rightarrow aSb \mid \varepsilon$, generating $\mathcal{L} = \{a^n b^n \mid n \geq 0\}$ (a CFL). Exhibit the repeated-variable surgery on the string $a^3 b^3$ and read off a pumping split.

Recognition. Here we are *not* disproving anything — $\mathcal{L}$ is context-free — we are watching the proof mechanism on a concrete tree to see what u, v, x, y, z literally are. The trigger is the self-embedding rule $S \rightarrow \mathbf{a}S\mathbf{b}$, which is exactly the $R \Rightarrow^* vRy$ pattern.

Solution. The parse tree of $\mathbf{a}^3\mathbf{b}^3$ is a chain of three $S \rightarrow \mathbf{a}S\mathbf{b}$ steps capped by $S \rightarrow \varepsilon$. Pick the top two S 's as R_{up} and R_{down} .



R_{up} is the root S ; its subtree yields the whole string, so $vxy = \mathbf{a}^3\mathbf{b}^3$. R_{down} is the next S down; its subtree yields $\mathbf{a}^2\mathbf{b}^2$, so $x = \mathbf{a}^2\mathbf{b}^2$. The single rule application between them contributes one $\mathbf{a}$ on the left and one $\mathbf{b}$ on the right, so reading leaves gives the clean split

$$u = \varepsilon, \quad v = \mathbf{a}, \quad x = \mathbf{a}^2\mathbf{b}^2, \quad y = \mathbf{b}, \quad z = \varepsilon.$$

Which idea, and why? Cloning the R_{up} subtree onto R_{down} once adds a $v = \mathbf{a}$ and a $y = \mathbf{b}$, giving $\mathbf{a}^4\mathbf{b}^4$; deleting down to the R_{down} subtree gives $uxz = \mathbf{a}^2\mathbf{b}^2$. Both lie in $\mathcal{L}$, illustrating condition 1 on a language that *is* context-free.

Check. $|vy| = 2 > 0$ (condition 2). Here $|vxy| = 6$ is the whole string; the proof instead takes the two *lowest* S 's, giving $u = \mathbf{a}^2$, $v = \mathbf{a}$, $x = \varepsilon$, $y = \mathbf{b}$, $z = \mathbf{b}^2$ with $|vxy| = 2$ for every n (condition 3). Pumping preserves the matched form $\mathbf{a}^m\mathbf{b}^m$, as it must.

Worked Example 7.3.5 ($\{ww \mid w \in \{0, 1\}^*\}$ is not context-free). Show that $D = \{ww \mid w \in \{0, 1\}^*\}$ is not a CFL.

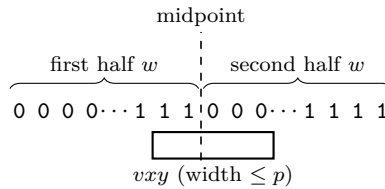
Recognition. “A string repeated” forces an *order* constraint across the midpoint, not just a count. Counts alone won’t catch it (a string like $0^p1^p0^p1^p$ is needed, not the naive 0^p1^p). The trigger is “copy language,” and the move is to use condition 3 to pin vxy near the middle. Apply Theorem 7.3.2.

Solution. Assume D is context-free with pumping length p , and choose

$$s = 0^p1^p0^p1^p \in D \quad (\text{here } w = 0^p1^p), \quad |s| = 4p.$$

Let $s = uvxyz$ satisfy conditions 2 and 3. Because $|vxy| \leq p$, the window vxy cannot reach from the first half into the second half by more than p symbols; we check the cases.

If vxy lies entirely within the first half 0^p1^p , then pumping up to $i = 2$ lengthens the first half while the second half is untouched, so the two halves no longer match in length, and even after relabeling the midpoint the string is not of the form ww . Symmetrically if vxy lies in the second half. If vxy straddles the exact midpoint, pump *down* to $i = 0$: uxz has the form $0^p1^j0^k1^p$ with j, k not both equal to p , which cannot be split into two equal halves. Every case contradicts condition 1.



Which idea, and why? The copy constraint is positional, so a length-mismatch *or* a symbol in the wrong position breaks it; condition 3 keeps vxy short enough that it cannot edit both halves symmetrically, and a single direction of pumping always destroys the ww shape.

Check. The weaker candidate $s = 0^p 10^p 1$ fails: it splits as $u = 0^{p-1}, v = 0, x = 1, y = 0, z = 0^{p-1} 1$ and pumps happily ($uv^i xy^j z = 0^{p-1+i} 10^{p-1+i} 1 \in D$), so it proves nothing — the four-block choice is what makes condition 3 bite.

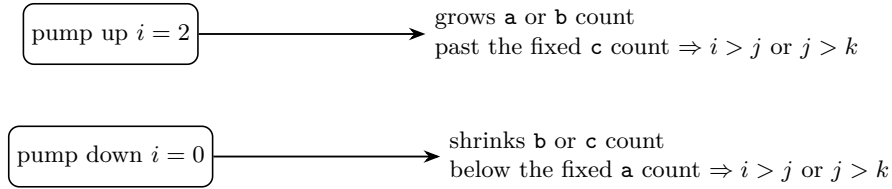
Worked Example 7.3.6 (Pumping down: $\{a^i b^j c^k \mid i \leq j \leq k\}$ is not context-free). Show that $C = \{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$ is not a CFL.

Recognition. The counts need not be equal, only nondecreasing, so pumping *up* alone may stay inside C . The trigger “an inequality $i \leq j \leq k$ ” tells us to pump in *whichever direction* breaks the relevant inequality — sometimes down. Apply Theorem 7.3.2, using both $i = 2$ and $i = 0$.

Solution. Assume C is context-free with pumping length p , and take $s = a^p b^p c^p \in C$. Fix a split $s = uvxyz$ with conditions 2,3. By condition 3, vxy touches at most two of the three letters, so one letter is absent from both v and y . Split on which:

- **a absent from v, y :** pump *down* ($i = 0$). Then uxz keeps p a’s but loses some b’s or c’s, giving $i = p > j$ or $j = p > k$; not in C .
- **c absent from v, y :** pump *up* ($i = 2$). Then $uv^2 xy^2 z$ gains a’s or b’s while the c-count stays p , forcing $j > k$ or $i > j$; not in C .
- **b absent from v, y :** then v, y are made of a’s and c’s only. If any a appears, pump up ($i > j$ eventually since j is fixed at p). If only c’s appear, pump down to drop c’s below p , breaking $j \leq k$. Either way, out of C .

Every case contradicts condition 1.



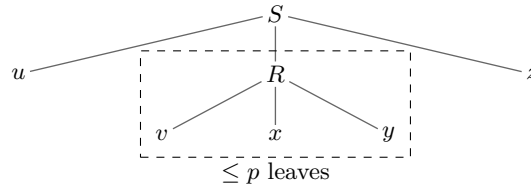
Which idea, and why? For an order condition like $i \leq j \leq k$, pumping up breaks an *upper* inequality and pumping down breaks a *lower* one; choosing the direction by which letter the window misses guarantees one of the inequalities fails.

Check. Equal-count examples ($a^p b^p c^p$) lie on the boundary of C , so *up* can stay inside (pump only c’s) and so can *down* (pump only a’s) — which is exactly why a single direction is not enough here and the case split is needed.

Worked Example 7.3.7 (Pumping $\{a^n b^n c^n \mid n \geq 1\}$ over a tree). Reprove non-context-freeness of $L = \{a^n b^n c^n \mid n \geq 1\}$ by exhibiting the shape of any candidate parse tree and where the window must sit, to see the geometry behind the algebra of the first worked example.

Recognition. The opener’s language without ε ; the point is the *picture*, not a new result. The trigger is “visualize condition 3 as a bounded subtree near a longest path.” We use Theorem 7.3.2 geometrically.

Solution. Suppose a CFG generated L and consider the minimal parse tree τ of $a^p b^p c^p$. The repeated variable R sits near the bottom of a longest path, so its subtree — the one yielding vxy — spans a *contiguous* leaf interval of width $\leq p$. A width- $\leq p$ leaf interval inside $a^p b^p c^p$ overlaps at most two of the three letter-blocks.



The boxed R -subtree controls a contiguous block of at most p symbols; cloning it adds copies of v and y taken from at most two blocks, so the third block’s count never moves, and the triple splits apart exactly as in the algebraic version.

Which idea, and why? Condition 3 is the statement “the pumped material lives under one bounded subtree,” and a bounded subtree spans a short, contiguous, hence at-most-two-letter window — the visual reason a three-way balance cannot survive paired pumping.

Check. If instead one tried to pump material spanning all three blocks, the window would have width $\geq p + 2 > p$, violating condition 3; so no such split exists, consistent with the algebra.

Method (How to use the CFL pumping lemma to prove a language is not context-free).

1. **Set up the contradiction.** Assume L is context-free and let p be its pumping length (you do *not* get to choose p ; treat it as a fixed unknown).
2. **Choose $s \in L$ with $|s| \geq p$ wisely.** Pick a string in which condition 3 will force the window vxy into a position you can exploit. Pad blocks with exponent p ; for copy/order languages use four or more blocks so the window cannot straddle freely.
3. **Quantify over all splits.** For an *arbitrary* $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$, use the width bound to limit where v and y can lie (which letters they contain, which boundary they straddle).
4. **Pick a pumping index that breaks membership.** Usually $i = 2$ (pump up) or $i = 0$ (pump down). For inequalities, choose the direction that violates the relevant inequality; you may need a case split on which letter the window misses.
5. **Conclude.** Some $uv^i xy^i z \notin L$ for every legal split, contradicting condition 1. Hence L is not context-free.

Caution. The pumping lemma is necessary, not sufficient. “ L is pumpable” does *not* prove L is context-free — there are non-context-free languages that satisfy the pumping conditions. The lemma only runs one way: context-free $\Rightarrow$ pumpable, used in contrapositive to *refute* context-freeness. Never cite it to *certify* a language as a CFL.

Caution. You must defeat *every* split $s = uvxyz$, but you get to choose the single string s and the single pumping index i . A common error is to fix a convenient split (say $v = a^p$) and pump only that one — the adversary is free to choose a different split, so a proof that handles only one split is no proof. Use conditions 2 and 3 to constrain *all* splits at once, then exhibit one bad i for each surviving case.

Caution. Do not forget to pump *down*. For languages defined by inequalities (like $i \leq j \leq k$) or by “at least” conditions, pumping up may keep the string inside L ; the contradiction often comes only from $i = 0$. Always ask whether uxz (the $i = 0$ case) leaves L before concluding the string is pumpable.

Exercises.

1. State the pumping length p that the proof of Theorem 7.3.2 produces from a grammar with $|V| = 4$ variables and longest right-hand side $b = 3$. Then explain in one sentence why every string of that length forces a repeated variable on some root-to-leaf path.
2. Use the pumping lemma to prove that $\{a^n b^n c^n d^n \mid n \geq 0\}$ is not context-free. Identify the string s you choose and the pumping index i you use, and explain why the window vxy cannot reach all four blocks.
3. Let $L = \{0^m 1^m 0^m \mid m \geq 1\}$. Prove L is not a CFL. (Hint: use condition 3 to confine vxy to at most two of the three blocks, then pump up.)
4. Show that $\{w\#w \mid w \in \{a, b\}^*\}$ is not context-free, where $\#$ is a separator symbol not in $\{a, b\}$. Pick a string of the form $a^p b^p \# a^p b^p$ and argue with condition 3.
5. Consider $C = \{a^i b^j \mid j = i^2, i \geq 0\}$ (the count of b 's is the square of the count of a 's). Prove C is not context-free by pumping up and comparing the new a -count against the new b -count. (Hint: between consecutive squares the gap grows, so a bounded additive change to both counts cannot keep j equal to i^2 .)
6. Prove that $\{a^n \mid n \text{ is prime}\}$ is not context-free. (This language is over a one-letter alphabet, so u, v, x, y, z are all blocks of a 's; pump up to land between two primes.)
7. Decide whether each language is context-free, and justify briefly: (a) $\{a^n b^m \mid n \leq m\}$; (b) $\{a^n b^n c^m \mid n, m \geq 0\}$; (c) $\{a^n b^m c^n d^m \mid n, m \geq 0\}$. Use the pumping lemma for any you claim is not context-free.

8. A student claims to prove $\{a^n b^n c^n\}$ is not a CFL as follows: “Choose $s = a^p b^p c^p$ and the split $u = \varepsilon$, $v = a^p$, $x = \varepsilon$, $y = b^p$, $z = c^p$; pumping to $i = 2$ gives $a^{2p} b^{2p} c^p \notin L$, contradiction.” Find the bug in this argument and explain what a correct proof must instead do.
9. Another student writes: “ $\{a^i b^j c^k \mid i \leq j \leq k\}$ is not a CFL because choosing $s = a^p b^p c^p$ and pumping *up* the a ’s gives more a ’s than b ’s.” Show that pumping *up* need not leave the language (the adversary may place v and y in the c -block), and state which pumping direction actually produces a contradiction in this case.
10. Pumping down. Let $L = \{a^n b^m \mid n \geq m \geq 0\}$. A classmate believes L is not context-free. Decide whether it is, and if it is, give a CFG generating it; if it is not, give a pumping proof, being explicit about whether you pump up or down.
11. Prove that the class of context-free languages is *not* closed under intersection by combining two CFLs whose intersection is the non-CFL $\{a^n b^n c^n \mid n \geq 0\}$. Exhibit the two grammars (or describe the two languages) and state which earlier worked example certifies that their intersection is not context-free.
12. True or false, with a one-line reason each: (a) if L is pumpable in the sense of Theorem 7.3.2, then L is context-free; (b) every finite language is context-free; (c) if $L_1 \subseteq L_2$ and L_2 is context-free, then L_1 is context-free; (d) the pumping lemma can show a language is regular.

7.4 Proving Languages Non-Context-Free

The pumping lemma of the previous section is a tool with one job: to certify that a given language is *not* context-free. The lemma says “if A is a CFL, then every long string of A can be cut into five pieces $uvxyz$ that pump.” Reading that implication backwards gives the whole method of this section – if we can exhibit one long string in A that *cannot* be pumped, the assumption “ A is a CFL” collapses. The one key idea is to treat the proof as a game against an adversary who is forced to hand us a decomposition $uvxyz$, after which we choose a power i that drives the pumped string out of A ; our job is to win no matter which legal decomposition the adversary picks. We assemble the standard playbook, then refute a gallery of classic languages: $\{a^n b^n c^n\}$, the inequality chain $\{a^i b^j c^k \mid i \leq j \leq k\}$, the copy language $\{ww\}$, and a worked variant using the marker $\#$.

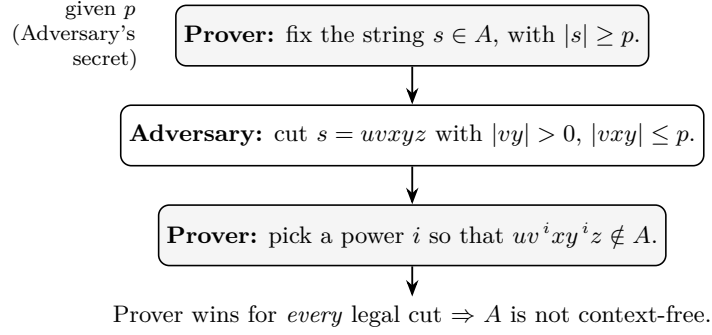
The adversary game

It pays to make the logical shape of these proofs explicit before doing any. The pumping lemma is a “ $\exists \forall \exists$ ” statement, and a refutation by contradiction flips every quantifier.

Theorem 7.4.1 (Pumping lemma, contrapositive form). *Let $A \subseteq \Sigma^*$. Suppose that for every integer $p \geq 1$ there exists a string $s \in A$ with $|s| \geq p$ such that for every division $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$, there exists an integer $i \geq 0$ with $uv^i xy^i z \notin A$. Then A is not context-free.*

Proof. This is the contrapositive of the pumping lemma. If A were context-free it would have some pumping length p , and the lemma would furnish, for the string s we chose at that p , a single division $s = uvxyz$ meeting $|vy| > 0$ and $|vxy| \leq p$ for which *all* pumped strings $uv^i xy^i z$ stay in A . That directly contradicts the hypothesis of the theorem. Hence A is not context-free. $\square$

The nesting of quantifiers tells us exactly who chooses what, and in which order. Picture two players, “Prover” (us, showing A is not a CFL) and “Adversary” (defending that A is a CFL).



Two features of the game decide every proof in this section. First, *Prover moves before the adversary names the cut but after p is fixed*, so the chosen string may mention p (e.g. $a^p b^p c^p$) but nothing finer. Second, the constraint $|vxy| \leq p$ is a *short window*: the entire pumped material vxy fits inside a block of length p , so it cannot reach across long stretches of the string. The art of choosing s is to make that window too narrow to span the boundaries that the language cares about.

Method (How to prove a language A is not context-free).

1. **Assume A is context-free** and let p be a pumping length (treat p as an unknown but fixed positive integer; never assign it a value).
2. **Choose one string** $s \in A$ with $|s| \geq p$, depending on p . Pick s so that the length- p window vxy is forced to lie inside a region where pumping must break membership. Good first guesses concentrate the “hard” boundary in the middle (e.g. $a^p b^p c^p$).
3. **Take an arbitrary legal cut** $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$. Do not assume any particular shape for v and y – you must beat *all* of them.
4. **Classify the cut** into a small number of cases according to where the window vxy lands (which symbol blocks v and y touch). Use $|vxy| \leq p$ to rule out impossible spans.
5. **Pick i to win each case.** “Pump up” ($i = 2$) to create a surplus, or “pump down” ($i = 0$) to create a deficit; show $uv^i xy^i z \notin A$ in every case.
6. **Conclude** the contradiction: s cannot be pumped, contradicting the lemma, so A is not context-free.

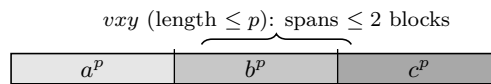
The canonical non-CFL: $a^n b^n c^n$

A pushdown automaton has one stack, enough to match *one* pair of nested counts but not two independent ones. The language $\{a^n b^n c^n\}$ asks a PDA to verify $\#a = \#b$ and $\#b = \#c$ simultaneously, and intuition says the single stack cannot. The pumping lemma turns that intuition into a proof.

Worked Example 7.4.2 ($\{a^n b^n c^n\}$ is not context-free). Let $B = \{a^n b^n c^n \mid n \geq 0\}$ over $\Sigma = \{a, b, c\}$. Show B is not a CFL.

Recognition. The language pins down two independent equalities of counts ($\#a = \#b$ and $\#b = \#c$). Pumping perturbs at most two contiguous spots v and y , and the window $|vxy| \leq p$ keeps those spots close together – the trigger that they cannot simultaneously touch all three letter blocks. So pumping must unbalance some pair of counts.

Solution. Assume B is a CFL with pumping length p , and take $s = a^p b^p c^p \in B$, $|s| = 3p \geq p$. Let $s = uvxyz$ be any legal cut. Picture s as three blocks and the window vxy as a sliding bar of length $\leq p$:



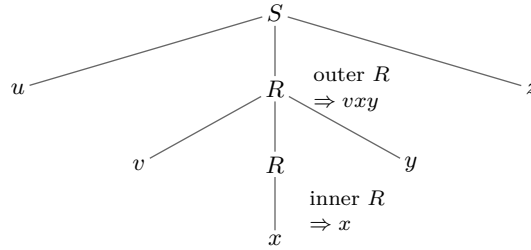
Because $|vxy| \leq p$, the window cannot stretch from the a -block all the way into the c -block (that distance exceeds p). So at least one of the three letters is *untouched* by both v and y . Pump up to $i = 2$, giving

$s' = uv^2xy^2z$, which is longer than s by $|vy| > 0$. Since $|vy| > 0$, the count of some touched letter strictly increases, while the count of the untouched letter stays at p . Hence in s' the three letter-counts are not all equal – and if v or y contained two different letters, s' would even fail the order $a^*b^*c^*$. Either way $s' \notin B$. *Which idea, and why?* The short window $|vxy| \leq p$ cannot reach across all three equal blocks, so one count is frozen while another grows; choose s as three same-length blocks precisely so the window is too narrow to keep them balanced.

Check. Concretely, if $v = a^j$ and $y = b^k$ with $j + k > 0$, then s' has $p + j$ a 's, $p + k$ b 's, and p c 's; the c -count p no longer matches whichever of a, b grew, so $s' \notin B$.

Why a single cut, not the grammar, is the engine: parse-tree surgery

It is worth recalling *where* the decomposition $uvxyz$ comes from, because the gallery proofs treat it as a black box. The lemma's guarantee is born from a repeated variable on a long root-to-leaf path of a parse tree. The diagram below shows the surgery that produces uv^ixy^iz : the same variable R occurs twice, the outer copy yields vxy and the inner copy yields x , and splicing one subtree in place of the other repeats v and y in lockstep.



Reusing the outer subtree at the inner R repeats v, y : yields uv^2xy^2z . Reusing the inner subtree at the outer R deletes them: yields $uxz = uv^0xy^0z$.

The picture also explains the two technical conditions we lean on. The minimal-tree choice forces $|vy| > 0$ (otherwise a smaller tree would already generate s), and choosing the repeat among the lowest variables on the longest path forces $|vxy| \leq p$. We never reconstruct the grammar in a refutation – we only use these two numeric facts about whatever cut the adversary offers.

Pumping down: $\{a^ib^jc^k \mid i \leq j \leq k\}$

When a language tolerates *inequalities* of counts, pumping up may keep us inside the language, so we must sometimes “pump down” to $i = 0$ to manufacture a deficit. This is the standard upgrade over the previous proof.

Worked Example 7.4.3 ($\{a^ib^jc^k \mid 0 \leq i \leq j \leq k\}$ is not context-free). Let $C = \{a^ib^jc^k \mid 0 \leq i \leq j \leq k\}$. Show C is not a CFL.

Recognition. Now equality is relaxed to $i \leq j \leq k$, so merely adding symbols ($i = 2$) often stays legal; the trigger is that we must *remove* symbols ($i = 0$) to break an ordering constraint. We split on which letter the window misses.

Solution. Assume C is a CFL with pumping length p and take $s = a^pb^pc^p \in C$ (here $i = j = k = p$ satisfies $i \leq j \leq k$), $|s| = 3p$. Let $s = uvxyz$ be any legal cut. As before, $|vxy| \leq p$ forces vxy to touch at most two adjacent letter blocks, so at least one of a, b, c is missing from both v and y . We win by choosing i per case.

missing letter	move	why $w^i x y^i z \notin C$
a (window in b, c)	pump down $i = 0$	a -count stays p ; some b or c drops below p , so $i > j$ or $j > k$.
b (window in a only or c only)	pump up if a grows: $i = 2$ pump down if in c : $i = 0$	more a 's than b 's $\Rightarrow i > j$. fewer c 's than b 's $\Rightarrow j > k$.
c (window in a, b)	pump up $i = 2$	some a or b exceeds p while c stays p , so $i > k$ or $j > k$.

In the first row, pumping down drops $m > 0$ symbols, so $j < p = i$ would violate $i \leq j$ if some lost symbols were b 's, and if they were all c 's then $j = p > p - m = k$ violates $j \leq k$; either way the inequality chain breaks. The middle row handles the case where b is the absent letter: if the window sits in the a -block, pumping up makes $\#a > \#b$; if it sits in the c -block, pumping down makes $\#b > \#c$. The last row is the $a^n b^n c^n$ argument with c frozen.

Which idea, and why? An ordering constraint $i \leq j \leq k$ is broken in *both* directions: a surplus ($i = 2$) violates the upper end and a deficit ($i = 0$) violates the lower end, so we keep both pump directions in the toolkit and select by which block the window misses.

Check. Suppose $v = b^2$, $y = \varepsilon$ (legal: $|vxy| \leq p$; the window lies in the b -block, so a is untouched and this is the “ a missing” row). Pump down: $uxz = a^p b^{p-2} c^p$, which has $j = p - 2 < p = k$, fine for $j \leq k$, but $i = p > p - 2 = j$ violates $i \leq j$. Not in C .

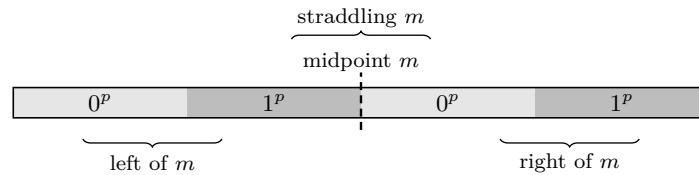
The window must straddle the middle: $\{ww\}$

The copy language exposes the deepest use of $|vxy| \leq p$. Here a naive choice of s can be pumped, so the choice of string itself is part of the proof.

Worked Example 7.4.4 ($\{ww \mid w \in \{0,1\}^*\}$ is not context-free). Let $D = \{ww \mid w \in \{0,1\}^*\}$. Show D is not a CFL.

Recognition. A bad first guess such as $s = 0^p 10^p 1$ can be pumped (pumping the last 0 of the first half together with the first 0 of the second keeps the halves equal), so the trigger is to pick s with four alternating blocks, $s = 0^p 1^p 0^p 1^p$, where any short window is trapped in one half and breaks the copy structure.

Solution. Assume D is a CFL with pumping length p and set $s = 0^p 1^p 0^p 1^p \in D$ (take $w = 0^p 1^p$), $|s| = 4p$. Let $s = uvxyz$ be a legal cut, so $|vxy| \leq p$. Mark the midpoint m after the first $2p$ symbols. The window vxy , of length $\leq p$, lies in one of three zones relative to m :



Window entirely left of m . Pump up to $i = 2$. The first half grows while the second half is unchanged, so the new midpoint shifts right and the second symbol-block of the (now longer) string no longer matches the first half; concretely uv^2xy^2z has its $1^p 0^p$ boundary displaced, so the two halves differ. **Window entirely right of m .** Symmetric: pump up and the second half grows, breaking equality. **Window straddles m .** Then vxy lies within $1^p 0^p$ around the center (since $|vxy| \leq p$, it cannot reach the outer 0^p or 1^p). Pump down to $i = 0$: $uxz = 0^p 1^{p-a} 0^{p-b} 1^p$ with $a + b = |vy| > 0$ and $a, b < p$ (deleting a whole middle run would force $|vxy| > p$), so the result has the shape $0^p 1^j 0^k 1^p$ with $j, k \geq 1$ and $(j, k) \neq (p, p)$. Splitting any such string in half cannot give two equal halves (if $uxz = tt$, then t begins with 0 and ends with 1, so tt has twice as many runs as t ; comparing with the runs of uxz forces $t = 0^p 1^j = 0^k 1^p$, i.e. $j = k = p$), so $uxz \notin D$.

Which idea, and why? The third condition $|vxy| \leq p$ is the workhorse: it confines the window to one half (pump up to desynchronize) or to the central $1^p 0^p$ (pump down to misalign), and the four-block string $0^p 1^p 0^p 1^p$ is engineered so no window can sit harmlessly in a uniform run.

Check. The discarded candidate $0^p 10^{p+1} 1$ fails: cutting $v = 0$ at the end of the first 0^p , $x = 1$, and $y = 0$ at the start of the second 0^p pumps to $0^{p+1} 10^{p+1} 1 \in D$ for $i = 2$ (and to $0^{p-1+i} 10^{p-1+i} 1 \in D$ for every i) – it stays in D , which is why we needed the sharper string.

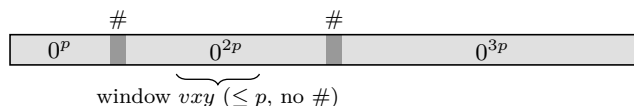
Intersecting with a regular language; a #-marker variant

Sometimes the cleanest refutation first *intersects* the target with a regular language to expose a non-CFL core, using the closure fact that CFL is closed under intersection with regular languages. The next example uses a marker $\#$ so the window's position is even easier to pin down.

Worked Example 7.4.5 ($\{0^n \# 0^{2n} \# 0^{3n} \mid n \geq 0\}$ is not context-free). Let $E = \{0^n \# 0^{2n} \# 0^{3n} \mid n \geq 0\}$ over $\Sigma = \{0, \#\}$. Show E is not a CFL.

Recognition. Three blocks whose lengths are locked in ratio 1 : 2 : 3 by two equalities ($\#_2 = 2\#_1$ and $\#_3 = 3\#_1$); the markers fix the block boundaries, and a short window touches at most two of the three 0-runs – the trigger that one ratio must break.

Solution. Assume E is a CFL with pumping length p and take $s = 0^p \# 0^{2p} \# 0^{3p} \in E$, $|s| = 6p + 2$. Let $s = uvxyz$ be a legal cut. First, neither v nor y can contain a $\#$: if a pumped string changed the number of $\#$'s away from exactly two, it would leave the format $0^* \# 0^* \# 0^*$ entirely, so $uv^2xy^2z \notin E$ immediately. Hence v and y consist of 0's drawn from the runs, and $|vxy| \leq p$ forces them into at most two *adjacent* runs. So one of the three runs is untouched; let its length-relation be the one that must fail. Pump up to $i = 2$: each touched run gains $|v|$ or $|y|$ 0's while the untouched run keeps its length, so at least one of the two ratios $\#_2 = 2\#_1$, $\#_3 = 3\#_1$ is destroyed (the ratios cannot all be preserved unless every run grows in exact proportion, impossible when one run is frozen and another grows). Thus $uv^2xy^2z \notin E$.



Which idea, and why? Markers freeze the block boundaries so the window cannot blur them; with one run untouched and another pumped, a fixed-ratio constraint like 2 : 1 or 3 : 1 cannot survive, exactly as equal-count constraints could not in $a^n b^n c^n$.

Check. If $v = 0^2$ inside the first run and $y = \varepsilon$, then $uv^2xy^2z = 0^{p+2} \# 0^{2p} \# 0^{3p}$: the middle run should be twice the first, i.e. $2(p+2) = 2p+4 \neq 2p$. Not in E .

Pitfalls

Caution. You do *not* get to choose the decomposition $uvxyz$. The adversary chooses it, and your proof must defeat *every* legal choice. A “proof” that picks a convenient v and y (say $v = a$, $y = b$) and pumps those proves nothing – the lemma only fails if *no* valid cut survives pumping. Always quantify: “let $s = uvxyz$ be *any* division with $|vy| > 0$ and $|vxy| \leq p$.”

Caution. Conversely, you *do* get to choose the string s , and the choice is the whole game. Picking a weak s (like $0^p 10^{p+1}$ for $\{ww\}$, or a string shorter than p) can leave a harmless place for the window and let the language survive pumping. Choose s so that the length- p window is trapped against a boundary the language cares about; when one direction of pumping seems to stay inside A , try the other direction ($i = 0$ vs. $i = 2$).

Caution. Treat p as an opaque positive integer, never a specific number. Writing “let $p = 5$ ” invalidates the proof, because the pumping lemma only promises that *some* p exists; a refutation must work for whatever p the lemma hands you. Likewise, never assume p is even, prime, or larger than some constant of your choosing.

Caution. The pumping lemma is a one-way test. Passing it (every long string pumps) does *not* prove a language is context-free – there exist non-CFLs that satisfy the pumping condition. Use the lemma only to certify *non-membership* in CFL; to certify membership, exhibit a grammar or a PDA.

Exercises.

1. Use the pumping lemma to show that $L_1 = \{a^n b^{2n} c^{3n} \mid n \geq 0\}$ is not context-free. State the string s you choose in terms of p , and say which power i wins.

2. Show that $L_2 = \{a^n b^m c^n d^m \mid n, m \geq 0\}$ is not context-free. (Hint: a short window cannot touch both an a -block and the matching c -block at once.)
3. Prove that $L_3 = \{w\#w \mid w \in \{0,1\}^*\}$ is not context-free. Explain why the marker $\#$ makes choosing s easier than in the markerless copy language $\{ww\}$.
4. Consider $L_4 = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$. Decide whether L_4 is context-free; if not, prove it with the pumping lemma, and if so, give a CFG or a PDA.
5. Let $L_5 = \{0^i 1^j \mid j = i^2\}$. Show L_5 is not context-free. (Hint: pump and compare the gap between consecutive squares against the amount a single pump can add.)
6. A classmate claims to prove $\{a^n b^n c^n\}$ is not a CFL as follows: “Take $s = a^p b^p c^p$ and let $v = a$, $y = c$. Then $uv^2xy^2z = a^{p+1}b^p c^{p+1} \notin B$, done.” Identify the exact flaw in this argument and explain what a correct proof must additionally do.
7. Show that the context-free languages are not closed under intersection by exhibiting two CFLs whose intersection equals $\{a^n b^n c^n \mid n \geq 0\}$. (You proved that intersection is not a CFL in the section; you only need to display the two CFLs and verify the intersection.)
8. Use the closure of CFL under intersection with a regular language to prove that the language L_8 of all $w \in \{a,b,c\}^*$ with $\#_a(w) = \#_b(w) = \#_c(w)$ is not context-free. (Intersect with a suitable regular language to reduce to a gallery example.)
9. Let $L_9 = \{0^n 1^n 0^n \mid n \geq 0\}$. Prove L_9 is not context-free, and draw the three-block-with-window picture you use to justify that the length- p window misses one block.
10. Determine whether $L_{10} = \{a^i b^j \mid i \leq j \leq 2i\}$ is context-free. Justify your answer (either a grammar/PDA or a pumping-lemma refutation, with the chosen s and the winning i).
11. Show that $L_{11} = \{a^n b^n \mid n \geq 0\} \cup \{a^n b^m c^m \mid n, m \geq 0\}$ is context-free by giving a grammar, then explain in one sentence why this does not contradict the fact that $\{a^n b^n c^n\}$ is non-context-free.
12. For each of the following, state which pump direction ($i = 0$ or $i = 2$) breaks membership and why, given a legal cut whose window lies in the named block: (a) $\{a^n b^n c^n\}$, window in the b -block; (b) $\{a^i b^j c^k \mid i \leq j \leq k\}$, window in the c -block; (c) $\{0^p 1^p 0^p 1^p\}$ -type strings of ww , window straddling the midpoint.

7.5 Deterministic PDAs and the Language Hierarchy

A pushdown automaton becomes a far more practical device once we forbid it to guess. The nondeterministic PDA of the previous sections may explore many stack computations in parallel, and that freedom is exactly what makes it as expressive as a context-free grammar; but a real parser inside a compiler cannot afford to branch. This section studies the *deterministic* pushdown automaton (DPDA), the machine that has at most one legal move in every situation. The single key idea is that determinism here is a genuine restriction – unlike finite automata, where determinism costs nothing, a DPDA is strictly weaker than a PDA – and the languages it recognizes, the *deterministic context-free languages* (DCFLs), sit in their own layer between the regular languages and the full context-free languages. We make the model precise, prove the key closure property that separates DCFLs from CFLs (complementation), and assemble the whole picture into the chain $\text{REGULAR} \subsetneq \text{DCFL} \subsetneq \text{CFL}$.

Definition 7.5.1 (Deterministic pushdown automaton). A *deterministic pushdown automaton* is a 6-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where Q, Σ, Γ, F are finite sets and

$$\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow (Q \times \Gamma_\varepsilon) \cup \{\emptyset\},$$

writing $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ and $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$, subject to the *determinism condition*: for every $q \in Q$, $a \in \Sigma$, and $x \in \Gamma$, exactly one of the four values

$$\delta(q, a, x), \quad \delta(q, a, \varepsilon), \quad \delta(q, \varepsilon, x), \quad \delta(q, \varepsilon, \varepsilon)$$

is different from $\emptyset$. As before $q_0 \in Q$ is the start state and $F \subseteq Q$ the set of accept states.

The output is either a single move (r, y) – “enter state r and replace the read stack symbol by y ” – or the empty set $\emptyset$, meaning “no move of this exact form.” Reading the four slots: $\delta(q, a, x)$ both consumes the input symbol a and pops x ; $\delta(q, a, \varepsilon)$ consumes a but touches nothing on the stack; $\delta(q, \varepsilon, x)$ pops x without consuming input; and $\delta(q, \varepsilon, \varepsilon)$ does neither. The determinism condition forces these never to compete: in any concrete situation (state q , next input a , stack top x) the machine has *at most one* applicable move. We allow ε -moves – a DPDA may legitimately push or pop without reading, as the DPDA for $\{0^n 1^n\}$ below does with its bottom marker; what is forbidden is having both an ε -move and a symbol-reading move enabled at once, since that would reintroduce a choice.

Definition 7.5.2 (Deterministic context-free language). A DPDA accepts $w \in \Sigma^*$ if, after reading all of w , it is in an accept state. A language $A \subseteq \Sigma^*$ is a *deterministic context-free language* (DCFL) if $A = L(P)$ for some DPDA P . Write DCFL for the class of all such languages.

Two ways a DPDA can reject deserve attention, because they have no analogue for DFAs. The machine rejects w if it reads all of w and halts outside F – but also if it *fails to read all of w* at all. This second failure happens by *hanging* (the stack is empty and the only specified move needs to pop a symbol, so the machine is stuck) or by *looping* (an endless run of ε -input moves that never advances the head). For finite automata neither can occur.

Lemma 7.5.3 (Read-to-the-end normal form). *Every DPDA has an equivalent DPDA that reads its entire input string on every run.*

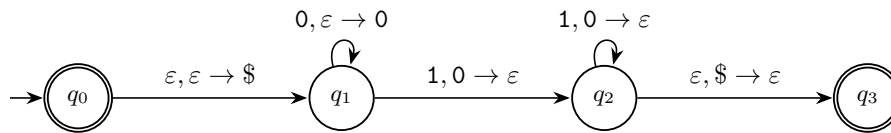
Proof idea. Hanging is cured by planting a fresh bottom marker $\$$ on the stack before anything else; if $\$$ is ever exposed, the machine knows the original stack is empty, and instead of getting stuck it scans the rest of the input and rejects (or accepts, if it had entered an accept state after reading the last input symbol). Looping is cured by detecting the finitely many *looping situations* – pairs (q, x) from which the machine, started in state q with x on top, never reads another input symbol – and rewiring each so that it instead reads the remaining input and then halts with the verdict it was destined for. Both fixes add only finitely many states, and neither changes which strings are accepted. $\square$

From now on we silently assume DPDAs read to the end; it makes their accept/reject behavior a clean total function of the input, exactly as for DFAs.

Worked Example 7.5.4 (A DPDA for $\{0^n 1^n \mid n \geq 0\}$). Build a deterministic PDA recognizing $A = \{0^n 1^n \mid n \geq 0\}$ over $\Sigma = \{0, 1\}$.

Recognition. This is the canonical context-free, non-regular language: count the 0s, then match them against the 1s. The trigger “push one marker per 0, pop one per 1, accept on a clean stack” is a counting job, and crucially there is *never a choice* about what to do, so a DPDA suffices.

Solution. Use a bottom marker $\$$ to detect an empty stack. In q_0 push $\$$ and go to the counting state q_1 . In q_1 each 0 pushes a 0. The first 1 switches to the matching state q_2 , popping one 0. In q_2 each further 1 pops a 0. When $\$$ resurfaces under an ε -move, pop it and accept in q_3 . Every state/input/stack-top triple has a unique move, so determinism holds (a triple with no drawn move goes to an omitted non-accepting dead state that reads the rest of the input).



The start state q_0 is also accepting, which accepts the empty input (giving $n = 0$); no move returns to q_0 .

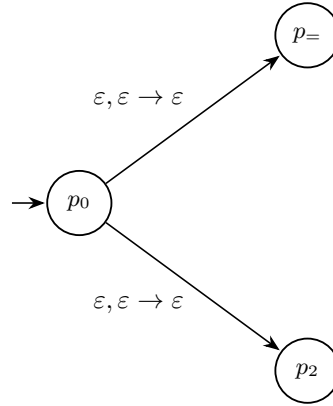
Which idea, and why? When the matching rule is forced – push on every 0, pop on every 1, switch exactly once – no guessing is ever required, so the language is not merely context-free but deterministic context-free.

Check. On 0011: push $\$$, push 0, push 0, then pop on each 1, exposing $\$$, pop it, accept. On 001 the stack still holds one 0 over $\$$ at end of input, so $\$$ is never exposed and the string is rejected.

Worked Example 7.5.5 (Why $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$ resists determinism). Consider $B = \{0^n 1^n \mid n \geq 1\} \cup \{0^n 1^{2n} \mid n \geq 1\}$. It is context-free. Explain informally why no DPDA seems able to recognize it, so that a PDA's nondeterminism is doing real work.

Recognition. The trigger is a *commitment* problem: after reading the block of 0s and starting to read 1s, the machine must already be popping at a rate (one 0 per 1, or one 0 per two 1s) that it cannot know is right until it sees how many 1s arrive.

Solution. A PDA simply *guesses* which disjunct it is in and runs the matching counter for that guess; some branch accepts iff $w \in B$. A deterministic machine has no branch to spare. Having pushed n markers for the 0s, it reaches the 1s with one stack and one state, and from that single configuration it must simultaneously be prepared to accept after n ones and after $2n$ ones – two different pop schedules. There is no way for a single deterministic pop policy to verify both possible endings, and a careful argument (an exercise below) turns this intuition into a proof. So B is a CFL that is not a DCFL.



The two outgoing ε -edges from p_0 are precisely the *forbidden* competition of Definition 7.5.1: a DPDA may not have two enabled moves in one situation. A PDA may, and that is the whole difference here.

Which idea, and why? A single accepting computation does not have to commit early, but a deterministic machine does; when the correct “rate” depends on data not yet seen, nondeterminism is essential.

Check. Both 0011 ($n = 2$, the $=$ disjunct) and 001111 ($n = 2$, the $2n$ disjunct) are in B ; a PDA accepts each on a different branch from the same prefix 00.

Theorem 7.5.6 (DCFLs are closed under complement). *If A is a DCFL then so is its complement $\bar{A} = \Sigma^* \setminus A$.*

Proof. Take a DPDA P for A in read-to-the-end normal form (Lemma 7.5.3). For a DFA one would simply swap accept and non-accept states; the obstruction for a DPDA is that between two consecutive input symbols the machine may pass through several states by ε -moves, touching both accepting and non-accepting ones. Swapping naively would then mis-handle a string that was accepted only because the run *visited* an accept state mid-burst. The fix is to pin acceptance to a single well-defined instant per symbol. Call q a *reading state* if its only enabled move consumes an input symbol (if it instead pops first, split that step into a pop followed by a read, creating a new reading state that carries the accepting status). Make only reading states eligible to be accept states: P passes through *exactly one* reading state per input symbol, right before reading the next one, and (remembering in its finite control whether an accept state was visited since the last read) we make that reading state accepting exactly when one was. Now swapping the accepting/non-accepting label *among the reading states only* flips the verdict on every string, so the modified DPDA recognizes $\bar{A}$. $\square$

This one theorem does a surprising amount of work, because the analogous statement is *false* for the full class CFL (we saw a non-context-free complement of a CFL in Theorem 6.4.10). It follows at once that DCFL is a *proper* subclass of CFL.

Corollary 7.5.7 (There is a CFL that is not a DCFL). $\text{DCFL} \subsetneq \text{CFL}$. *Concretely, $C = \{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$ is context-free but not deterministic context-free.*

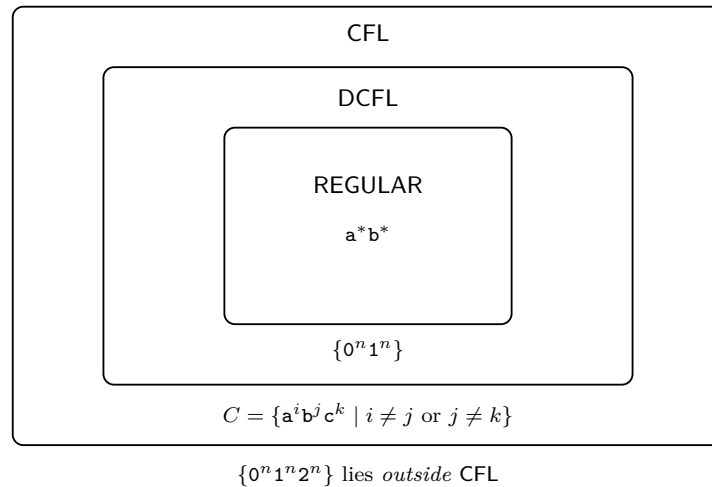
Proof. C is a CFL (it is the union of the CFL $\{a^i b^j c^k \mid i \neq j\}$ and the CFL $\{a^i b^j c^k \mid j \neq k\}$, and CFLs are closed under union). If C were a DCFL, then by Theorem 7.5.6 its complement would be a DCFL, hence a CFL. Intersecting that complement with the regular language $a^* b^* c^*$ (CFLs are closed under intersection with regular languages) yields exactly $\{a^n b^n c^n \mid n \geq 0\}$, which the pumping lemma of the previous section shows is *not* context-free – a contradiction. So C is a CFL outside DCFL. $\square$

Remark. The complement closure is the *main* pleasant closure property DCFLs have that CFLs lack; in the other direction DCFLs are *worse* behaved. The class DCFL is *not* closed under union, intersection, concatenation, star, or reversal. The language B of Worked Example 7.5.5 is a union of two DCFLs that is not a DCFL, which already kills closure under union (and, via De Morgan with Theorem 7.5.6, under intersection too).

Worked Example 7.5.8 (Reading off a containment from a closure property). Show that the DCFLs are sandwiched strictly: $\text{REGULAR} \subsetneq \text{DCFL} \subsetneq \text{CFL}$.

Recognition. The trigger is “three nested classes, prove each containment proper.” Containments come from “every machine of the weaker kind is a machine of the stronger kind”; properness comes from exhibiting one separating language at each level.

Solution. A DFA is a DPDA that never touches its stack, so $\text{REGULAR} \subseteq \text{DCFL}$. A DPDA is in particular a PDA, so $\text{DCFL} \subseteq \text{CFL}$. For the strict gaps: $\{0^n 1^n\}$ is a DCFL (Worked Example 7.5.4) but not regular, separating REGULAR from DCFL ; and C of Corollary 7.5.7 is a CFL but not a DCFL, separating DCFL from CFL .



Which idea, and why? Each inclusion is “weaker machine is a special case of stronger machine”; each strictness is one witness language living in the bigger class but provably not the smaller, supplied by the pumping lemma (for REGULAR) and by the complement trick (for DCFL).

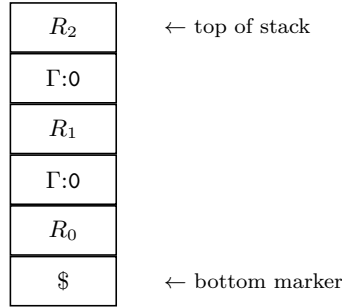
Check. The witnesses are distinct and each is verified by a theorem already in hand, so both $\subseteq$ are genuinely $\subsetneq$.

Worked Example 7.5.9 (Endmarkers make DPDA design easier without adding power). For a language A let $A\vdash = \{w\vdash \mid w \in A\}$ tag every string with a fresh endmarker $\vdash \notin \Sigma$. Argue that A is a DCFL iff $A\vdash$ is a DCFL, and explain why one direction is the useful one in practice.

Recognition. The trigger is “the endmarker tells the machine when the input is over.” A DPDA cannot peek ahead, so knowing it has reached the last symbol – normally invisible – lets it perform a final stack cleanup deterministically.

Solution. $(\Rightarrow)$ Given a DPDA P for A , build $P\vdash$ that simulates P and, upon reading $\vdash$, accepts iff P was in an accept state just before; it then reads nothing further. This is routine. $(\Leftarrow)$ Given a DPDA P for $A\vdash$, build P' for A that, before reading each real symbol, works out whether P *would* accept were the next symbol $\vdash$ – but it cannot afford to pop the whole stack each time, so it stores alongside each stack symbol a

set $R \in \mathcal{P}(Q)$ of states from which P accepts-on- $\dashv$ with the current stack. Thus $\Gamma' = \Gamma \cup \mathcal{P}(Q)$, interleaving real stack data with this lookahead summary, and the test becomes reading the top set R in $O(1)$.



each $R_i \in \mathcal{P}(Q)$ summarizes
“accept-on- $\dashv$ from here”

Which idea, and why? Both directions are simulations, so neither enlarges the class; the gain is purely in convenience – with an endmarker the machine gets an explicit “input ends now” signal that a deterministic device otherwise lacks.

Check. The map $w \mapsto w\dashv$ is a bijection between A and $A\dashv$, so the two languages carry the same membership information; only the moment of decision moves.

Method (How to tell whether a context-free language is likely a DCFL).

1. **Look for a forced single strategy.** Try to design a PDA in which, at every step, the next move is dictated by (state, next input symbol, stack top) with no genuine alternative. If you can, you have a DPDA and the language is a DCFL. Matching-counter languages like $\{0^n 1^n\}$ and balanced parentheses pass this test.
2. **Hunt for an unresolvable guess.** If the machine must decide *early* between two futures whose correctness depends on input not yet read (the two pop-rates of B above; the fold point of $\{ww^R\}$), suspect the language is *not* a DCFL.
3. **Use complement as a lever.** If $\bar{A}$ is provably not context-free, then A cannot be a DCFL, by Theorem 7.5.6. This is the cleanest non-DCFL test and is how Corollary 7.5.7 works.
4. **Allow an endmarker while designing.** If a deterministic design is awkward only because the machine cannot tell when the input ends, design it on $A\dashv$ instead; the equivalence guarantees A itself is a DCFL.

Caution. Do not assume that “deterministic” costs nothing just because it was free for finite automata. For DFAs vs. NFAs the subset construction makes determinism free; for pushdown machines it is a strict loss of power, $\text{DCFL} \subsetneq \text{CFL}$. The reason the subset trick fails is that you cannot bound the stack contents of the parallel branches by a finite set of subsets the way you bound NFA state sets.

Caution. The determinism condition of Definition 7.5.1 is about *competition*, not about forbidding ε -moves. A DPDA is allowed – often required – to make ε -input and ε -stack moves. What it may not do is have, in one concrete situation (q, a, x) , both a move that reads a and a move that ignores a via ε . Checking a candidate DPDA means checking that for every (q, a, x) exactly one of the four δ -slots fires, not that ε never appears.

Caution. Closure properties of CFL and of DCFL are different and easy to swap. CFL is closed under union, concatenation, and star but *not* under complement or intersection. DCFL is closed under complement but *not* under union, intersection, concatenation, star, or reversal. Quoting the wrong list is the most common error in this material; when in doubt, rederive the one you need from a machine construction or a complement argument.

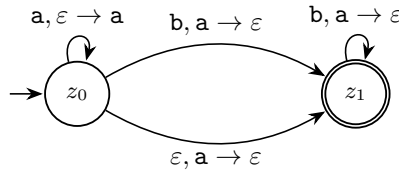
Exercises.

1. Design a DPDA over $\Sigma = \{0, 1\}$ for the balanced-bracket language D consisting of all strings w that have equally many 0s as 1s and in which every prefix has at least as many 0s as 1s (read 0 as “open”, 1 as “close”). Draw the state diagram with stack labels of the form $a, x \rightarrow y$, and trace your machine on 001011.

2. Build a DPDA recognizing $\{a^n b^m \mid m \geq n \geq 1\}$. Explain in one sentence why the matching rule never forces a choice, so the machine is genuinely deterministic.
3. Give a DPDA for $\{wcw^R \mid w \in \{a, b\}^*\}$, where the center marker c appears exactly once. Draw the diagram, and contrast in one sentence with $\{ww^R\}$ (no center marker), which is *not* a DCFL.
4. Prove that every regular language is a DCFL by describing how to turn an arbitrary DFA $M = (Q, \Sigma, \delta, q_0, F)$ into a DPDA that ignores its stack. State precisely which δ -slot of Definition 7.5.1 each DFA transition uses.
5. (Complement test.) Using closure of DCFL under complement, show that $E = \{a^i b^j \mid i \neq j\}$ is a DCFL by giving a DPDA for its complement $\bar{E}$ (the strings $a^n b^n$ together with all strings outside $a^* b^*$) and citing the right theorem. Then explain why the same “complement” move does *not* prove $\{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$ is a DCFL.
6. Carry out the argument sketched in Worked Example 7.5.5: prove that $B = \{0^n 1^n \mid n \geq 1\} \cup \{0^n 1^{2n} \mid n \geq 1\}$ is not a DCFL. (Hint: suppose a DPDA P recognized B . By determinism, its run on $0^n 1^{2n}$ passes through its accepting run on $0^n 1^n$; so a PDA that simulates P and, at an accept state, may switch to a copy of P reading 2s in place of 1s accepts, among its strings ending in 2, exactly $\{0^n 1^n 2^n \mid n \geq 1\}$, which is not context-free.)
7. Decide, with brief justification, whether each language is (a) regular, (b) DCFL but not regular, (c) CFL but not DCFL, or (d) not context-free:

$$L_1 = \{0^m 1^n \mid m \neq n\}, \quad L_2 = \{ww \mid w \in \{0, 1\}^*\}, \quad L_3 = \{0^n 1^n 0^n \mid n \geq 0\}.$$

8. The following claimed DPDA over $\Sigma = \{a, b\}$ is *not* deterministic. Find the violation of the determinism condition (Definition 7.5.1) and name the state, input symbol, and stack top at which two moves compete.



9. State the closure properties of DCFL under (i) complement, (ii) union, (iii) intersection, (iv) intersection with a regular language. For each that fails, give a one-line reason or a counterexample built from languages in this section.
10. Show that DCFL is not closed under reversal by arguing that the reversal of some DCFL is not a DCFL. (Hint: $B' = \{c 1^n 0^n \mid n \geq 1\} \cup \{d 1^{2n} 0^n \mid n \geq 1\}$ is a DCFL, since a DPDA reading B' left-to-right learns the pop rate from the first symbol and has no early commitment, whereas B'^R announces the rate only at the very end; the lookahead sets of Worked Example 7.5.9 would turn a DPDA for B'^R into one for the witness B of Worked Example 7.5.5.)
11. Explain, in three or four sentences, why the subset construction that converts an NFA to a DFA does *not* convert a PDA to a DPDA. Identify exactly which finiteness assumption breaks.
12. Endmarkers. Let $A = \{a^n b^n \mid n \geq 0\}$ and $A\vdash = \{w\vdash \mid w \in A\}$. Design a DPDA for $A\vdash$ that exploits the endmarker to perform its final stack check, and point out the single move that the endmarked design makes cleaner than the un-endmarked DPDA of Worked Example 7.5.4.

Chapter 8

Turing Machines and the Church–Turing Thesis

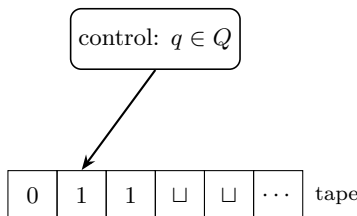
8.1 The Turing Machine Model

A Turing machine is the model we reach for once finite automata and pushdown automata run out of power. It keeps the finite-control idea – a machine sits in one of finitely many states – but it trades the one-way input for a single, one-way-infinite *tape* that the machine may both read and write, with a head that moves left or right one cell at a time. The one key idea of this section is that unrestricted read/write access to unlimited memory is exactly what was missing: with it the control can store, revisit, and overwrite scratch work, and the resulting model captures every algorithm we will ever care to run. We make the model precise as a 7-tuple, define a *configuration* and the one-step *yields* relation that drives a computation, pin down the three possible fates of a run (accept, reject, loop), and define the language a machine recognizes.

Definition 8.1.1 (Turing machine). A *Turing machine* is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ where Q, Σ, Γ are finite sets and

- Q is the set of *states*;
- Σ is the *input alphabet*, which does *not* contain the blank symbol $\sqcup$;
- Γ is the *tape alphabet*, with $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$;
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the *transition function*;
- $q_0 \in Q$ is the *start state*;
- $q_{\text{acc}} \in Q$ is the *accept state*;
- $q_{\text{rej}} \in Q$ is the *reject state*, with $q_{\text{rej}} \neq q_{\text{acc}}$.

The whole model lives in the shape of δ . Reading $\delta(q, a) = (r, b, L)$ as an instruction: “in state q scanning the symbol a , overwrite that cell with b , enter state r , and move the head one cell left.” Every step does three things at once – change state, write a symbol, move one cell. Two conventions matter. First, the tape starts holding the input $w = w_1 \cdots w_n \in \Sigma^*$ on its leftmost n cells and blanks $\sqcup$ everywhere after; because $\sqcup \notin \Sigma$, the first blank marks where the input ends. Second, if δ ever calls for a left move while the head is already on the leftmost cell, the head simply stays put – it cannot fall off the left end.



The finite control points a read/write head at one cell of a one-way-infinite tape; here the head reads 1 in state q .

Definition 8.1.2 (Configuration). A *configuration* of M records everything that can change during a computation: the current state, the tape contents, and the head position. We write it as a string uqv with $u, v \in \Gamma^*$ and $q \in Q$, meaning the tape holds uv (followed by blanks), the machine is in state q , and the head scans the *first* symbol of v . The *start configuration* on input w is $q_0 w$. A configuration is *accepting* if its state is q_{acc} and *rejecting* if its state is q_{rej} ; these two are the *halting* configurations.

For instance, $01q_310$ says the tape reads 0110 (then blanks), the state is q_3 , and the head sits on the third cell – the first 1 of the suffix 10 .

Definition 8.1.3 (The yields relation). Configuration C_1 *yields* C_2 , written $C_1 \vdash C_2$, if the machine can legally pass from C_1 to C_2 in one step of δ . Concretely, for $a, b, c \in \Gamma$, $u, v \in \Gamma^*$, and states q_i, q_j :

$$u a q_i b v \vdash u q_j a c v \quad \text{if } \delta(q_i, b) = (q_j, c, L), \quad u a q_i b v \vdash u a c q_j v \quad \text{if } \delta(q_i, b) = (q_j, c, R).$$

At the left end, $q_i b v \vdash q_j c v$ for a left-moving step (the head stays), and $q_i b v \vdash c q_j v$ for a right-moving step. A configuration $u q_i$ with the head at the right end is read as $u q_i \sqcup$, so the cases above still apply. We write $C \vdash^* C'$ if C' is reachable from C in zero or more steps.

Halting configurations have no successors: once the state is q_{acc} or q_{rej} the machine stops, so we never apply δ to them. Everywhere else δ is total, so a non-halting configuration yields exactly one successor – the machine is deterministic.

Definition 8.1.4 (Acceptance, the language, and recognizability). M *accepts* w if there is a finite sequence of configurations $C_1, C_2, \dots, C_k$ with C_1 the start configuration on w , each $C_i \vdash C_{i+1}$, and C_k accepting. The *language recognized by M* is

$$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \}.$$

A language is *Turing-recognizable* if some Turing machine recognizes it.

Started on w , a machine has exactly three fates: it *accepts* (reaches q_{acc}), it *rejects* (reaches q_{rej}), or it *loops* – runs forever without ever halting. Rejecting and looping both count as “does not accept,” but they are very different in practice: a rejecting run delivers a definite “no,” while a looping run delivers nothing at all. A machine that halts on *every* input – one that never loops – is called a *decider*, and the language it recognizes is then said to be *decidable*. Decidability is the subject of a later section; here we only need the model and its three outcomes.

Caution. “Recognize” is weaker than “decide.” If M recognizes $\mathcal{L}$, then on a string $w \in \mathcal{L}$ it is *guaranteed* to halt and accept, but on a string $w \notin \mathcal{L}$ it is allowed to either reject *or loop forever*. So observing that a machine has not yet halted tells you nothing: the input might still be accepted later, or the machine might never stop. Only a decider promises a yes/no answer on every input.

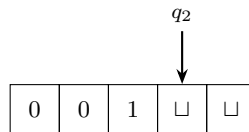
Worked Example 8.1.5 (Reading a computation off the yields relation). Let M have the rules $\delta(q_1, 0) = (q_1, 0, R)$, $\delta(q_1, 1) = (q_2, 1, R)$, and $\delta(q_2, \sqcup) = (q_{acc}, \sqcup, R)$. Trace M on input 001 as a chain of configurations.

Recognition. A trace is just repeated application of the *yields* relation from Definition 8.1.3: at each step locate the scanned symbol (the first symbol after the state in uqv), look up δ , then rewrite the configuration.

Solution. Start at $q_1 001$. Reading 0 in q_1 : write 0, stay in q_1 , move right. Reading the next 0: same. Reading 1 in q_1 : write 1, go to q_2 , move right. Now the head is on the first blank; $\delta(q_2, \sqcup)$ sends us to q_{acc} .

$$q_1 001 \vdash 0 q_1 01 \vdash 00 q_1 1 \vdash 001 q_2 \vdash 001 \sqcup q_{acc}.$$

The last configuration is accepting, so M accepts 001 . (As a tape picture, the head walks rightward across 001 and halts on the blank that follows it.)



The configuration $001q_2$: tape 001 , head on the first blank, state q_2 , one step before accepting.

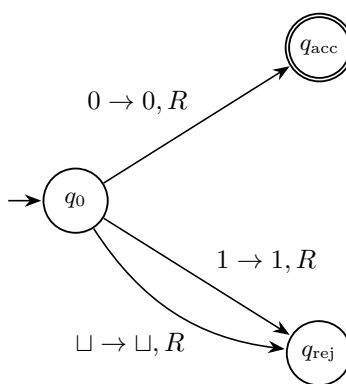
Which idea, and why? Tracing a deterministic TM is mechanical because each non-halting configuration yields *exactly one* successor; you never choose, you only apply δ to the scanned symbol.

Check. The chain has $C_1 = q_1 001$ (the start configuration), every adjacent pair is a single δ -step, and the final state is q_{acc} – the three conditions of Definition 8.1.4.

Worked Example 8.1.6 (A control diagram for a one-pass TM: strings beginning with 0). Design (as a control state diagram) a Turing machine deciding $\mathcal{L} = \{w \in \{0, 1\}^* \mid w \text{ begins with } 0\}$, and explain why it is a decider.

Recognition. The condition depends only on the *first* symbol, so the machine needs no writing and no back-and-forth: read one cell and halt. The trigger “decide from a single leftmost symbol” tells us to branch immediately on the start state’s read.

Solution. From q_0 , reading 0 go straight to q_{acc} ; reading 1 go straight to q_{rej} ; reading $\sqcup$ (the empty input has no first symbol) go to q_{rej} . No other transitions are reachable, so the head never moves more than once and the machine always halts. We label each control edge $a \rightarrow b, D$ meaning “read a , write b , move D ”; here nothing is overwritten, so we write $a \rightarrow a, R$.



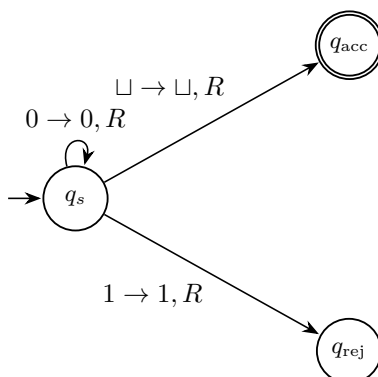
Which idea, and why? When acceptance is fixed by a bounded prefix of the input, a TM can decide it by reading that prefix and jumping to a halting state; because every branch reaches q_{acc} or q_{rej} after finitely many steps, the machine halts on all inputs and is a decider.

Check. On 011: $q_0 011 \vdash 0 q_{acc} 11$ accepted. On 101: $q_0 101 \vdash 1 q_{rej} 01$ rejected. On ε : the head reads $\sqcup$ and rejects. All three halt.

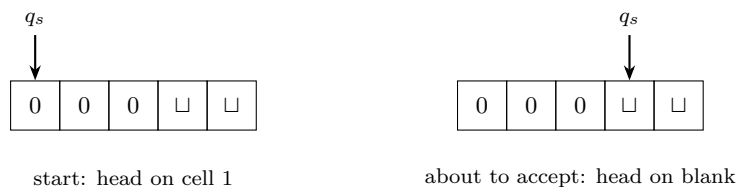
Worked Example 8.1.7 (A one-sweep decider: $\mathcal{L} = \{0^*\}$). Build a Turing machine over $\Sigma = \{0, 1\}$ deciding $\mathcal{L} = \{w \mid w \text{ is all } 0\text{s}\}$ (including ε), and show two tape snapshots of its run on 000.

Recognition. A scan-and-verify task: walk left to right checking that no 1 ever appears, then halt. The trigger “check a property of every cell, then stop” calls for a single rightward sweep that rejects on the first violating symbol and accepts at the trailing blank.

Solution. State q_s sweeps right; no left-end marker is needed, since the head never has to return home. In q_s , reading 0 writes 0 and moves right; reading 1 goes to q_{rej} ; reading $\sqcup$ (end of input, no 1 ever seen) goes to q_{acc} .



On 000 the head sweeps across the three 0s and halts on the blank:



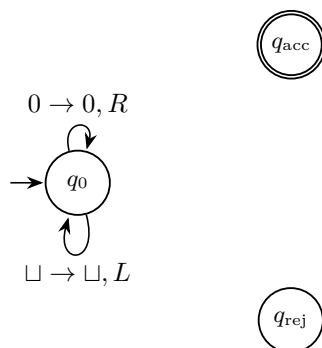
Which idea, and why? A “hold for every cell” property is decided by one rightward sweep with a self-loop on the good symbols, an edge to q_{rej} on any bad symbol, and an edge to q_{acc} at the terminating blank – the sweep halts after $\leq n+1$ steps, so the machine is a decider.

Check. On 000 the loop fires three times then the blank sends it to q_{acc} . On 010 the second cell holds 1, sending it to q_{rej} . On ε the head reads $\sqcup$ at once and accepts (the empty string is all 0s, vacuously).

Worked Example 8.1.8 (A recognizer that loops: why looping is a genuine outcome). Over $\Sigma = \{0\}$, consider the machine M with $\delta(q_0, 0) = (q_0, 0, R)$, $\delta(q_0, \sqcup) = (q_0, \sqcup, L)$, and no transition into a halting state. What is $L(M)$, and how does this illustrate the third outcome?

Recognition. No edge ever reaches q_{acc} , so no input can be accepted; the trigger “never enters q_{acc} ” forces $L(M) = \emptyset$. The interesting part is *how* the machine fails to accept – by rejecting, or by looping.

Solution. Start on any input 0^n . In q_0 the head runs right over all the 0s, hits the blank, and the rule $\delta(q_0, \sqcup) = (q_0, \sqcup, L)$ steps it back left; from there it reads 0 or $\sqcup$ again and keeps oscillating. It never reaches a halting state, so it *loops* on every input. Hence $L(M) = \emptyset$, but the machine witnesses this by looping, not by rejecting.



The halting states q_{acc}, q_{rej} exist but are unreachable; every run cycles in q_0 forever.

Which idea, and why? $L(M) = \emptyset$ is forced by “ q_{acc} unreachable,” and the empty language is also recognized by a machine that simply rejects; the example shows recognizers may *loop* rather than reject, which is exactly why recognizability is weaker than decidability.

Check. $\emptyset$ is trivially Turing-recognizable (and decidable) – replace the back-and-forth rule by $\delta(q_0, \sqcup) = (q_{rej}, \sqcup, R)$ and the same language is now *decided*, halting on every input. Same language, different fate on non-members.

Method (How to specify and run a Turing machine).

1. **Fix the alphabets.** List Σ (input symbols) and $\Gamma \supseteq \Sigma \cup \{\sqcup\}$, adding any *scratch* symbols you need – a left-end marker $\triangleright$, a “crossed-off” mark, or dotted versions of symbols for matching. Remember $\sqcup \notin \Sigma$.
2. **Choose the states as a plan.** Each state should stand for one phase of the algorithm (“sweeping right looking for a 1,” “returning to the left end”). Always include q_0, q_{acc}, q_{rej} .
3. **Write δ as instructions $a \rightarrow b, D$.** For every reachable (state, scanned-symbol) pair decide the triple (next state, symbol to write, L or R). Send hopeless cases to q_{rej} and successful completion to q_{acc} .
4. **Run by configurations.** Write the start configuration $q_0 w$ and apply the *yields* relation step by step, rewriting $u q v$ each time. Stop at the first halting state.

5. **Decide vs. recognize.** If *every* branch is guaranteed to reach a halting state (e.g. each phase makes measurable progress and there are finitely many phases), you have a decider; otherwise you have only a recognizer and must flag where it can loop.

Caution. Do not let the head run off the left end. The model pins the head in place when a left move is requested at the leftmost cell, so a naive “move left until you fall off” loop simply stalls there forever instead of signalling “reached home.” To find the left end reliably, mark it on the first step (write a $\triangleright$ over cell one, or use a blank as in many textbook machines) and scan left until you see the marker. Relying on the off-the-end behaviour as an end-detector is possible but subtle, and forgetting it is a classic source of infinite loops.

Caution. The blank symbol $\sqcup$ is a tape symbol, never an input symbol: $\sqcup \in \Gamma$ but $\sqcup \notin \Sigma$. This is what lets the first blank mark the end of the input. If you mistakenly put $\sqcup$ in Σ , an input string could contain blanks and the machine would have no way to tell input from scratch space – and your object would violate Definition 8.1.1.

Exercises.

- Let M have transition rules $\delta(q_1, \mathbf{a}) = (q_1, \mathbf{a}, R)$, $\delta(q_1, \mathbf{b}) = (q_2, \mathbf{b}, R)$, $\delta(q_2, \mathbf{a}) = (q_2, \mathbf{a}, R)$, $\delta(q_2, \mathbf{b}) = (q_2, \mathbf{b}, R)$, and $\delta(q_2, \sqcup) = (q_{\text{acc}}, \sqcup, R)$, with all unlisted pairs going to q_{rej} . Write the full chain of configurations on input $\mathbf{aab}$ and on input $\mathbf{ba}$. Which is accepted?
- Using Definition 8.1.3, give the configuration C_2 with $C_1 \vdash C_2$ in each case: (a) $C_1 = 10 q_3 01$ and $\delta(q_3, 0) = (q_5, 1, R)$; (b) $C_1 = 10 q_3 01$ and $\delta(q_3, 0) = (q_5, 1, L)$; (c) $C_1 = q_3 01$ and $\delta(q_3, 0) = (q_5, 1, L)$ (mind the left end).
- Draw the control state diagram of a Turing machine over $\Sigma = \{0, 1\}$ that *decides* the language of strings ending in 0. (Hint: sweep to the right end, then check the last non-blank symbol; you may use a marker symbol.) Argue in one sentence that your machine halts on all inputs.
- Give a Turing machine deciding $\mathcal{L} = \{w \in \{a, b\}^* \mid w \text{ contains at least one } \mathbf{a}\}$. Draw the control diagram, then exhibit two tape snapshots of its run on $\mathbf{bba}$.
- Specify (states, Σ , Γ , and the transition rules) a Turing machine that decides $\mathcal{L} = \{\varepsilon\}$ – it accepts the empty input and rejects every nonempty input. Trace it on ε and on 0.
- A machine M over $\Sigma = \{0\}$ has $\delta(q_0, 0) = (q_0, 0, R)$ and $\delta(q_0, \sqcup) = (q_0, \sqcup, R)$, with no edge into a halting state. Describe $L(M)$ and the fate of every run, and state whether M is a recognizer, a decider, or neither.
- Explain why a configuration written $u q_{\text{acc}} v$ never yields another configuration. Then explain why, for a non-halting state q , the configuration $u q v$ yields *exactly one* successor.
- Let M be any Turing machine. Prove that for every input w , the run of M on w ends in at most one of the three outcomes accept, reject, loop, and that exactly one occurs. (Use that δ is a function and that halting configurations have no successors.)
- Recognize vs. decide.* Suppose M recognizes $\mathcal{L}$ and is known to halt on every string *in* $\mathcal{L}$ within $|w|^2$ steps, but may loop on strings not in $\mathcal{L}$. Is $\mathcal{L}$ necessarily decidable from this information alone? Argue yes or no in two or three sentences.
- Find the bug.* A student says: “Put $\sqcup$ into Σ so the machine can take blank inputs.” Explain precisely what breaks in Definition 8.1.1 and in the convention that the input occupies the leftmost cells, and state the smallest fix.
- Find the bug.* A drawn machine over $\Sigma = \{0, 1\}$ is supposed to decide “the input is nonempty,” but its only rule out of q_0 is $\delta(q_0, 0) = (q_{\text{acc}}, 0, R)$; there is no rule for reading 1 or $\sqcup$ in q_0 . Describe the fate of the input 1 and of the input ε under the convention that a missing transition goes to q_{rej} , decide whether the machine is correct, and give the smallest repair.
- Show that the head-stays-at-the-left-end rule is genuinely needed by giving a short instruction sequence that, without it, would try to move left off cell one. Then describe how a left-end marker $\triangleright$ lets the machine detect home without relying on that rule.

8.2 Designing Turing Machines

Section 8.1 fixed the formal machinery: a Turing machine is a 7-tuple, a configuration records the whole state of the computation, and the yields relation drives it forward one step. Writing every transition of that machinery by hand is exhausting and unreadable for any machine bigger than a toy. The one key idea of this section is that we almost never do: we describe a Turing machine at the *implementation level* – prose that says how the head sweeps the tape and what marks it leaves behind – and trust (rightly, by the Church–Turing thesis of Section 8.4) that the prose could be expanded into a formal δ . We build that fluency by designing several machines whose languages are provably beyond the reach of finite automata, drawing the tape at key moments so the strategy is visible.

Three levels of description. A Turing machine can be specified at three levels of detail. A *formal description* lists Q , Σ , Γ , and every value of δ ; it is exact but unwieldy. An *implementation-level description* uses English to say how the head moves and what it writes, without naming individual states. A *high-level description* states the algorithm and ignores the tape entirely. Section 8.1 worked at the formal level; from here on we work mostly at the implementation level, because it is precise enough to pin down the language while staying short enough to read.

Definition 8.2.1 (Implementation-level description). An *implementation-level description* of a Turing machine M is a finite list of stages, each a finite sequence of primitive head actions – read the current symbol, write a symbol, move one cell left or right, and branch on what was read – that together determine, for every configuration, the next configuration. It is a faithful description of a Turing machine provided each stage keeps only a bounded amount of information “in mind” at once, so that the bookkeeping can be folded into a finite set of control states.

The proviso in Definition 8.2.1 is the one real constraint. A stage may say “scan right to the next 1” (a finite control can do that by looping in one state), but it may *not* say “remember the entire left half of the tape” (no finite control holds unbounded information). Everything unbounded must live on the tape, recorded by writing symbols. This is exactly why a Turing machine surpasses a finite automaton: the tape is the unbounded memory the states lack.

Definition 8.2.2 (Marking a tape symbol). To *mark* a tape symbol a is to overwrite it with a distinct symbol $\dot{a} \in \Gamma$ that the machine treats as “ a , already processed.” Including a marked copy $\dot{a}$ of each symbol a we wish to track enlarges Γ but not Σ , and lets the head recover the original a later by reading off the dot. *Crossing off* is the special case in which the mark $\times$ carries no need to recover the original.

Marking is the workhorse technique of Turing-machine design. A machine that must pair up, count, or compare parts of its input does so by marking matched symbols and sweeping back and forth across the unmarked remainder; the marks are the only memory the finite control could not hold on its own.

Proposition 8.2.3 (Decidable beats finite-state). *The language $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ is decidable, yet recognized by no finite automaton.*

Proof. That no DFA recognizes $\mathcal{L}$ is the standard pumping argument: a k -state DFA reading $0^k 1^k$ must repeat a state inside the block of 0s, and pumping that loop changes the number of 0s without changing the number of 1s, producing an accepted string outside $\mathcal{L}$. That $\mathcal{L}$ is decidable is Worked Example 8.2.4 below, which exhibits a Turing machine that halts on every input and accepts exactly $\mathcal{L}$. $\square$

We now design machines. Each worked example gives an implementation-level description, a sample run shown as tape snapshots, and – for the first machine – a control state diagram in the δ -as-arrows style of Section 8.1, so the connection between prose and formal object stays concrete.

Worked Example 8.2.4 (Matching two blocks: $\{0^n 1^n \mid n \geq 0\}$). Design a Turing machine $M_{=}$ deciding $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ over $\Sigma = \{0, 1\}$.

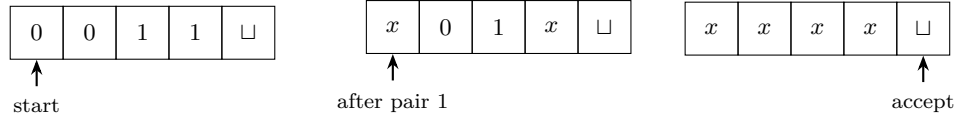
Recognition. The condition pairs each 0 with a 1, so the natural strategy is *cross-off matching*: repeatedly delete one 0 and one 1 and check that they run out together. The trigger “equal counts of two symbols appearing in fixed blocks” calls for marking matched symbols and sweeping between the blocks.

Solution. Use the tape alphabet $\Gamma = \{0, 1, x, \sqcup\}$, marking a matched symbol by overwriting it with x .

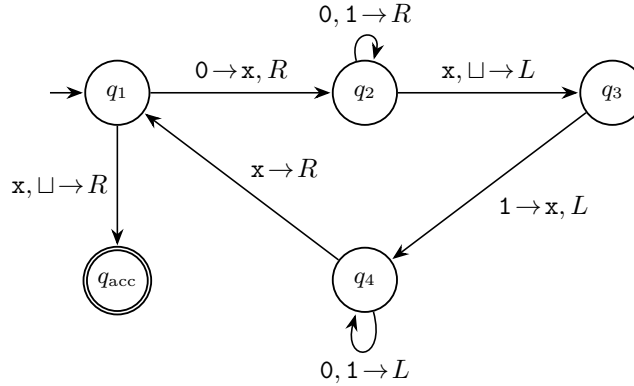
$M_ =$ “On input w :

1. If the current symbol is $\sqcup$, the input is empty; *accept*.
2. Cross off the leftmost 0 (write x). If instead the leftmost symbol is 1, *reject* (a 1 precedes every 0).
3. Scan right to the last uncrossed symbol (just before the next x or $\sqcup$) and cross it off if it is a 1. If it is a 0, or no uncrossed symbol is left, there are unmatched 0s; *reject*.
4. Scan left back to the leftmost uncrossed symbol.
5. If an uncrossed 0 remains, go to stage 2. If none remains, scan right: if any 1 is still uncrossed, *reject*; otherwise *accept*.”

Here are three snapshots of $M_ =$ on input 0011 – start, after the first matched pair, and at acceptance.



A control state diagram realizing stages 2–4 (the matching loop) makes the head motion explicit. Label $0 \rightarrow R$ means “read 0, leave it, move right”; $0 \rightarrow x, R$ means “read 0, write x , move right”; $\sqcup$ denotes the blank; the reject state and its incoming edges are suppressed, as is standard.



Which idea, and why? Pairing two equal-size blocks is a cross-off-matching job: mark one symbol from each block per pass and let the tape, not the control, hold the running count, since n is unbounded and a finite control cannot.

Check. On 001 the machine crosses the first 0 and the 1, returns left, crosses the second 0, scans right, finds no uncrossed 1, and rejects – correct, since $001 \notin \mathcal{L}$. On 0011 it matches both pairs and accepts; on the empty string it accepts at stage 1.

Worked Example 8.2.5 (Comparing two halves: $\{w\#w \mid w \in \{a, b\}^*\}$). Design a Turing machine $M_\#$ deciding $B = \{w\#w \mid w \in \{a, b\}^*\}$ over $\Sigma = \{a, b, \#\}$.

Recognition. The two halves must be *identical*, position by position, so we compare the i th symbol on the left with the i th on the right for every i . The trigger “check two stretches of tape agree symbol-by-symbol” is the *zig-zag*: mark a symbol on the left, cross to its partner on the right, check, and repeat.

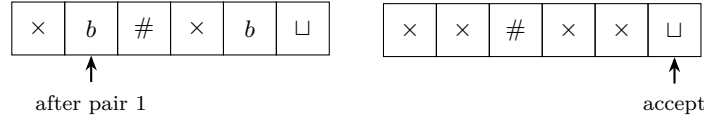
Solution. Mark with $\times$. After confirming there is exactly one $\#$ (a left-to-right scan), the machine zig-zags.

$M_\#$ = “On input w :

1. Remember the leftmost uncrossed symbol s to the left of $\#$, then cross it off. If the symbol at the head is $\#$, all left symbols are crossed; go to stage 4.
2. Scan right past the $\#$ to the leftmost uncrossed symbol on the right. If it is not s , or if a $\sqcup$ is reached first, *reject*; otherwise cross it off.

3. Scan left to the #, then to the leftmost uncrossed symbol; go to stage 1.
4. Scan right past the #: if any uncrossed symbol remains on the right, the right half is longer; *reject*. Otherwise *accept*.”

Two snapshots on input **ab#ab**: after matching the first **a** on each side, and at acceptance.



The phrase “remember the symbol s ” in stage 1 is legitimate because $s \in \{a, b\}$ is one of *finitely many* symbols; the control simply has a separate copy of stages 2–3 for $s = a$ and for $s = b$. It could *not* remember the symbol’s *position*, which is unbounded – so it relocates the position each pass by scanning to the # and counting off crossed cells, using the tape rather than the control.

Which idea, and why? Position-by-position equality of two tape stretches is a zig-zag with marking: one symbol of finite-alphabet memory may ride in the control, but the unbounded position must be re-found on the tape each pass.

Check. On **ab#ab** both pairs match and stage 4 finds nothing uncrossed: *accept*. On **ab#aa** the second pair mismatches (**b** vs. **a**) and stage 2 rejects. On **a#ab** the left empties after one pass but stage 4 finds an uncrossed **b**: *reject*.

Worked Example 8.2.6 (A multiplicative count: $\{a^i b^j \mid j = 2i, i \geq 1\}$). Design a Turing machine M_2 deciding $D = \{a^i b^j \mid j = 2i, i \geq 1\}$.

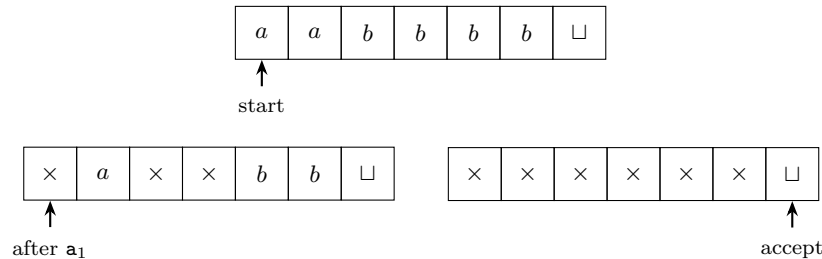
Recognition. Each **a** must account for exactly two **bs**, so cross off *one a* and *two bs* per pass. The trigger “one of these for every k of those” is again cross-off matching, but with a fixed multiplicity ($k = 2$) per pass.

Solution. Mark with $\times$ on both letters.

M_2 = “On input w :

1. Verify by one left-to-right scan that w matches $a^+ b^+$ (some **a**s, then some **b**s, nothing else); otherwise *reject*. Return to the left end.
2. Cross off the leftmost uncrossed **a**. If none remains, go to stage 4.
3. Scan right to the first uncrossed **b** and cross it; scan on to the next uncrossed **b** and cross it. If either **b** is missing (a $\sqcup$ is met first), *reject*. Return to the left end and go to stage 2.
4. All **a**s are crossed. Scan the **b** region: if any **b** is uncrossed, *reject*; otherwise *accept*.”

Snapshots on input **aabbbb**: start, after the first **a** has consumed two **bs**, and at acceptance.



Which idea, and why? A fixed multiplicative relation $j = ki$ is cross-off matching with k marks of one kind per single mark of the other; the format pre-check in stage 1 ensures the blocks are in order so the counting is well defined.

Check. On **abb** ($i = 1, j = 2$) the single **a** crosses both **bs** and stage 4 accepts. On **aabb** the first **a** crosses both **bs**, then the second **a** finds no **b** left and stage 3 rejects – correct, since $j = 2 \neq 4$. On **abbb** stage 4 finds an uncrossed **b**: *reject*.

Worked Example 8.2.7 (Modifying the tape: binary successor). Design a Turing machine M_{+1} that, given a binary numeral $w \in \{0,1\}^+$ (most-significant bit first), *rewrites the tape* to hold the binary numeral for the value of w plus one, then halts. (This machine computes a function rather than deciding a language, illustrating that Turing machines transform tapes, not just accept or reject.)

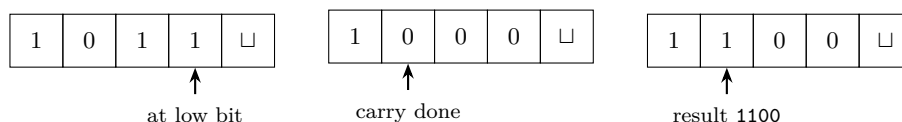
Recognition. Adding one in binary flips the trailing run of 1s to 0s and turns the first 0 to its left into a 1; if there is no such 0 (the numeral is all 1s) a new leading 1 must appear. The trigger “schoolbook carry from the right” says: work from the *least* significant end, so first run the head to the right end.

Solution.

M_{+1} = “On input w :

1. Scan right to the first $\sqcup$, noting in the control whether any 0 was seen, then move one cell left, onto the least-significant bit.
2. While the current symbol is 1: write 0 and move left (propagating the carry).
3. The current symbol is now 0: write 1. If the input was all 1s (as noted in stage 1), this 0 is the first cell, flipped by the carry – the head stays there, since there is no cell to its left – so also write 0 on the first blank, lengthening the numeral by one digit. *Halt.*”

Snapshots on input 1011 (value 11, successor 12 = 1100): the head arrives at the right end, then the carry propagates left across the trailing 11, and finally the first 0 becomes 1.



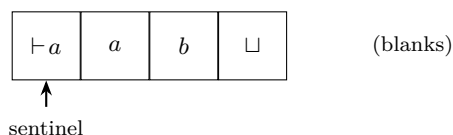
Which idea, and why? Computing a function “in place” uses the head’s read–write power directly: the carry of binary addition is a right-to-left rewrite, so the head seeks the least-significant bit first and flips digits until the carry is absorbed.

Check. On 111 (value 7) the carry flips all three to 0, the first cell included, so stage 3 writes 1 there and 0 on the first blank, giving 1000 = 8. On 100 the low bit is 0, immediately rewritten to 1: 101 = 5, the successor of 4.

Worked Example 8.2.8 (Finding the left end with a sentinel). Many designs (like M_2 above) say “return to the left end.” Show how a machine *finds* the left end of its tape, given that nothing on the tape marks cell 1 as the end.

Recognition. The right end of the input is marked for free – the first $\sqcup$ past the last input symbol – but the left end has no terminator. The trigger “need a left landmark” calls for planting a *sentinel*: at start-up, mark the leftmost cell with a special symbol the machine can later scan back to.

Solution. At the very first step, before any other work, overwrite the leftmost input symbol a with a marked copy $\vdash a$ that means “ a , and this is cell 1.” Thereafter “return to the left end” means “scan left until the head reads a $\vdash$ -marked symbol, then stop.” The sentinel is never erased, so the landmark persists for the whole computation.



An alternative, which adds only one special symbol to Γ , exploits the rule that a left move off the tape’s left end leaves the head in place (Section 8.1): write the special symbol, try to move left, and if the special symbol is still under the head the move failed, so the head was already at cell 1; then restore the original symbol.

Which idea, and why? The tape has no left terminator, so “go to the left end” must be *implemented*: either plant a permanent sentinel at start-up, or detect the boundary using the stay-put rule for left moves off the end.

Check. With a sentinel, a leftward scan from anywhere halts exactly at cell 1 and never runs past it, because $\vdash a$ is the unique $\vdash$ -marked cell. The stay-put method needs only one extra symbol but costs a write-test-restore at each boundary check.

Method (How to design a Turing machine from a language description).

1. **Choose a level.** For all but the tiniest machines, aim for an implementation-level description: stages of head motion and marking, not individual states.
2. **Decide what must be remembered, and put it on the tape.** Anything unbounded – a count, a position, a matched prefix – lives as marks on the tape. Only finite, alphabet-sized facts (“the symbol I just saw,” “am I past the # yet”) may ride in the control.
3. **Pick a sweeping pattern.** The standard patterns are: *cross-off matching* (pair symbols across blocks, one mark per pass), *zig-zag comparison* (check two stretches agree symbol-by-symbol), *single-pass scanning* (recognize a regular shape like a^+b^+ as a finite automaton would), and *in-place rewriting* (transform the tape, e.g. a carry).
4. **Handle the ends.** The right end is the first blank; for the left end, plant a sentinel or use the stay-put rule. Pre-check the input’s gross format with one scan if the later stages assume it.
5. **Account for halting.** Make sure every pass strictly reduces something (uncrossed symbols, digits to the right) so the machine cannot loop; a decider must halt on *every* input, accepted or not.
6. **Test** on a shortest accepted string, a near-miss that must be rejected, and the empty string. Trace the tape, not just the states.

Caution. Do not let the finite control “remember” anything unbounded. A stage that says “recall how many as you have seen” is illegal once that number can grow without bound: no finite set of states can store an arbitrary integer. The count must be recorded on the tape (here, by the crossed-off as themselves). Remembering one *symbol* from a fixed alphabet is fine; remembering one unbounded *number* or *position* is not.

Caution. A decider must halt on every input. It is easy to write a matching loop that returns to stage 2 forever when the counts do not line up – for instance, sweeping for a 1 that is not there and never noticing. Always give each failure its own explicit *reject*: “if a $\sqcup$ is reached first, reject.” A machine that merely *recognizes* a language may loop on rejected inputs, but a *decider* may not, and the languages in this section are all decidable, so every machine here must terminate on every input.

Caution. Crossing off is not the same as deleting. A Turing machine cannot remove a cell and close the gap in one step; it can only overwrite a symbol in place. “Cross off the leftmost 0” means “write x over it,” leaving the cell occupied. Designs that silently assume the tape shrinks (so that later symbols slide left) are wrong: any compaction must be programmed explicitly as a shifting subroutine, one cell at a time.

Exercises.

1. Give an implementation-level description of a Turing machine that decides $\{a^n b^n c^n \mid n \geq 0\}$. Explain why one cross-off pass must mark one symbol of each of the three kinds, and state explicitly what happens if the three counts fail to run out together.
2. Design a Turing machine deciding $\{w \in \{0,1\}^* \mid w \text{ has the same number of 0s as 1s}\}$. Unlike $0^n 1^n$, the symbols here may be interleaved; describe how each pass should locate “the leftmost uncrossed 0” and “the leftmost uncrossed 1” regardless of order.
3. Design a Turing machine deciding $\{w \# w^R \mid w \in \{a,b\}^*\}$ (the right block is the *reversal* of the left). Draw two tape snapshots of your machine running on $ab \# ba$, marking the head position, and say how your zig-zag differs from the one for $w \# w$ in Worked Example 8.2.5.
4. For the binary-successor machine M_{+1} of Worked Example 8.2.7, draw the full sequence of tape snapshots (head position marked at each step) for the input 0111. State the resulting numeral and confirm it is the successor.

5. Give an implementation-level description of a Turing machine that, on input $w \in \{a, b\}^*$, *halts with the tape holding w^R* (the reversal of w). You may use extra tape alphabet symbols. Explain why this needs in-place rewriting rather than mere marking.
6. Design a Turing machine deciding $\{0^i 1^j \mid i \leq j\}$ (at least as many 1s as 0s). Describe the cross-off pattern and exactly which leftover condition after the matching loop should lead to *accept* versus *reject*.
7. A student proposes the following “machine” for $\{0^n 1^n\}$: “Count the 0s, remembering the total n in the control; then count the 1s and accept iff the second count equals n .” Identify the precise flaw and explain why no Turing machine can be built from this description as written.
8. The element-distinctness idea compares every pair of blocks. Give an implementation-level description of a Turing machine deciding

$$\{\#x_1\#x_2\#\cdots\#x_\ell \mid \text{each } x_i \in \{0, 1\}^* \text{ and the } x_i \text{ are pairwise distinct}\},$$

and explain which two nested marks the machine maintains and why each must be re-found on the tape rather than held in the control.

9. Here is a buggy machine for $\{w\#w\}$ over $\{a, b\}$. “Cross off the leftmost uncrossed left symbol; scan right to the leftmost uncrossed right symbol and cross it off whatever it is; repeat; accept when all symbols are crossed.” Exhibit a string *not* in the language that this machine wrongly accepts, and state the one stage that must be repaired.
10. Design a Turing machine deciding $\{a^i b^j a^k \mid k = i + j, i, j \geq 0\}$. Describe a strategy that crosses off both an earlier **a** and a **b** against the trailing **a**s, and explain how the machine knows when the first **a**-block and the **b**-block are both exhausted.
11. Sketch (implementation level) a Turing machine that decides whether its input $w \in \{0, 1\}^*$ is a palindrome, i.e. $w = w^R$. Give a tape snapshot on 0110 at the moment the two outer symbols have just been matched, and explain why the machine compares the *outermost* uncrossed pair on each pass rather than adjacent symbols.
12. For each of three sweeping patterns named in the Method (cross-off matching, zig-zag comparison, in-place rewriting), give one short language or task from this section’s exercises that it solves, and in one sentence justify why that pattern fits.

8.3 Robustness: Multitape, Nondeterministic, and Enumerators

The Turing machine of the previous two sections looks fragile: one tape, one head, a head that must move left or right on every step and may never pause. It is natural to suspect that handing the machine more tapes, or the freedom to branch nondeterministically, would let it recognize languages the bare model cannot. The one key idea of this section is that it does not: each of these apparently stronger machines can be *simulated* by a basic one-tape deterministic Turing machine, so they all recognize exactly the same class of languages. This stability under reasonable changes of definition is called *robustness*, and it is the evidence that the Turing machine captures something model-independent about computation rather than an accident of one particular wiring. Every equivalence below is proved the same way – by exhibiting a simulation – which is the proof technique we will lean on for the rest of the book.

Definition 8.3.1 (Simulation and equivalence of machines). A machine B *simulates* a machine A if B can, on every input w , reproduce the accept/reject behaviour of A on w : B accepts w exactly when A accepts w , rejects when A rejects, and runs forever when A runs forever. Two machine *models* are *equivalent* when every machine of one model has an equivalent machine of the other, where “equivalent” means recognizing the same language. To show two models equivalent it suffices to show each can simulate the other.

Before the two big variants, a tiny example shows the whole method in miniature. Our basic TM forces the head to move (L or R) on every step. Suppose we relax this and allow a third option S (“stay put”), so $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$. Does the stay-put machine recognize anything new?

Proposition 8.3.2 (Stay-put adds no power). *Every Turing machine with the stay-put option has an equivalent ordinary Turing machine whose head moves on every step.*

Proof. Replace each stay-put instruction. Wherever the transition function says $\delta(q, a) = (r, b, S)$ – write b , stay, go to r – substitute two ordinary moves through a fresh intermediate state q' : from q on a , write b , move right, and enter q' ; from q' on *every* tape symbol c , write c back unchanged, move left, and enter r . The head ends one cell right and then one cell left, i.e. back where it started, having written b and reached r – exactly the effect of the stay-put step, using only L/R moves. Doing this for every stay-put instruction yields an equivalent ordinary TM. $\square$

The replacement is local: one new state and one “move back” rule per stay-put transition. The machine’s behaviour on the input is identical step-for-step at the cells that matter. This is the template – find a gadget that mimics the extra feature using only basic moves – and the remaining results are larger instances of the same idea.

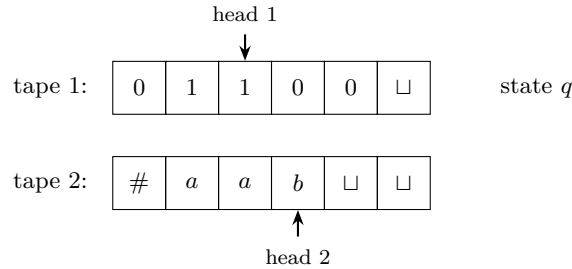
Multitape Turing machines

Definition 8.3.3 (Multitape Turing machine). A k -tape Turing machine ($k \geq 1$) is a Turing machine with k one-way-infinite tapes, each with its own independent read/write head. The input is written on tape 1; tapes $2, \dots, k$ start blank. The transition function reads all k heads, writes all k heads, and moves each head independently:

$$\delta: Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k.$$

A single instruction $\delta(q, a_1, \dots, a_k) = (r, b_1, \dots, b_k, D_1, \dots, D_k)$ means: in state q with head i scanning a_i , go to state r , write b_i under head i , and move head i in direction $D_i \in \{L, R, S\}$, for every i at once. Acceptance is as before: the machine halts in q_{acc} .

A second work tape is a genuine convenience: it gives the machine scratch space to copy, compare, and mark without destroying its input. Picture a two-tape machine partway through a computation – the heads roam independently over the two tapes.



A 2-tape machine: tape 1 holds the input, tape 2 is scratch; the two heads move independently.

Theorem 8.3.4 (Multitape collapses to one tape). *Every multitape Turing machine has an equivalent single-tape Turing machine.*

Proof. Let M have k tapes. We build a one-tape machine S that stores all k tape contents on its single tape, separated by a delimiter $\#$ (a new symbol). To remember where each of M ’s k heads sits, S uses a second copy of each tape symbol with a dot, $\dot{a}$, marking the scanned cell. Thus S ’s tape alphabet is $\Gamma \cup \dot{\Gamma} \cup \{\#\}$. The layout for input $w = w_1 \dots w_n$ on a $k = 3$ machine is

$$\# \dot{w}_1 w_2 \dots w_n \# \dot{} \# \dot{} \#$$

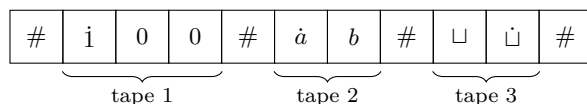
– tape 1’s contents with head on w_1 , then two blank tapes each scanning a single blank.

S runs as follows. To simulate one step of M : scan left to right from the first $\#$ to the last, reading the dotted symbol in each block to learn the k symbols under M ’s heads, and remember this k -tuple in S ’s finite control. Now S knows δ_M on that tuple, so it makes a second left-to-right pass, updating each block: in block i it overwrites the dotted symbol with the new symbol M writes, and shifts the dot one cell left, right,

or not at all to record head i 's move. If a head moves right off the right end of its block – i.e. onto the block's $\#$ – that head has reached new blank tape; S writes a blank there and shifts the entire remaining tape one cell right to make room, then continues. S accepts/rejects exactly when M does.

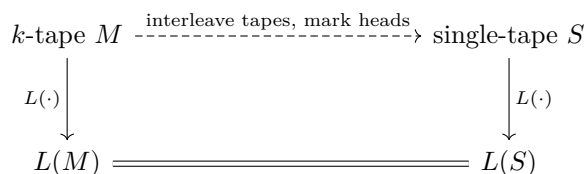
Each block faithfully holds one of M 's tapes, and the dot faithfully holds one head position, so after every simulated step S 's tape encodes M 's full configuration. Hence S accepts w iff M does, and $L(S) = L(M)$. $\square$

The encoding – k blocks separated by $\#$, each with one dotted “virtual head” – is the whole trick. Here is S 's single tape representing a snapshot of a 3-tape machine.



One tape of S encoding three tapes of M ; a dotted symbol $\dot{x}$ marks each virtual head. Here head 1 scans 1, head 2 scans a, head 3 scans a blank.

The encoding above shows the layout on S 's single tape; the square below states the invariant it buys – not the tape contents but the *language*.



Extra tapes add no recognizing power: a k -tape machine is simulated by a single-tape machine that stores the k tapes on one tape with delimiters and marked head positions, recognizing the same language.

Check (Theorem 8.3.4). The single-tape simulator S faithfully reproduces each multitape step, so $L(S) = L(M)$ and the square commutes. The price is paid in *time*, not recognizability: each step of M costs S a sweep across all k blocks, so a $T(n)$ -time multitape machine (with $T(n) \geq n$) becomes an $O(T(n)^2)$ -time single-tape machine – a polynomial slowdown that changes the running time but not the language recognized.

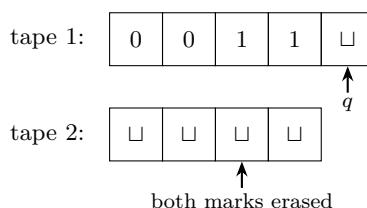
Corollary 8.3.5. *A language is Turing-recognizable if and only if some multitape Turing machine recognizes it.*

Proof. A one-tape TM is the special case $k = 1$ of a multitape TM, giving one direction. Conversely a k -tape recognizer is converted to a one-tape recognizer by Theorem 8.3.4. The same construction preserves halting, so the corollary also holds with “recognizable” and “recognizes” replaced by “decidable” and “decides.” $\square$

Worked Example 8.3.6 (Using a second tape to recognize $\{0^n 1^n \mid n \geq 0\}$ in one pass). Sketch a 2-tape TM for $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ and show why the extra tape makes it easy, then note what the single-tape simulation costs.

Recognition. The condition “equal numbers of 0s and 1s, all 0s first” is a *counting/matching* task. With a scratch tape the machine can record the 0s as tally marks and then cancel one mark per 1 – no back-and-forth shuttling on the input. The trigger “match a left block against a right block” calls for a second tape used as a counter.

Solution. Tape 1 holds the input; tape 2 is scratch. In phase 1, scan tape 1 left to right: while reading 0s, write one mark X on tape 2 per 0 (heads move right together). When the first 1 appears, move tape 2's head back to the leftmost mark. In phase 2, for each 1 on tape 1, erase one mark on tape 2. Accept iff the input ends exactly when the marks run out (and no 0 ever follows a 1); reject otherwise. After reading the input 0011:



The two marks written in phase 1 are both erased by the two 1s in phase 2, and the input ends together with the marks, so 0011 is accepted.

Which idea, and why? A second tape turns “compare two equal-length blocks” into “produce a counter, then consume it,” which is a single left-to-right pass over the input; Theorem 8.3.4 guarantees a one-tape machine exists, at the cost of the head shuttling that the second tape let us avoid.

Check. 0011: two marks made, two erased, input exhausted – accept. 001: two marks made, one erased, one mark left – reject. 0101: a 0 follows a 1 – reject in phase 2. Correct.

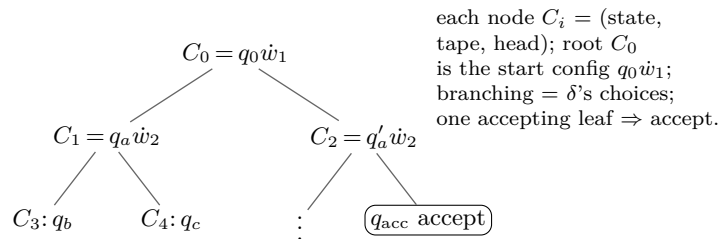
Nondeterministic Turing machines

Definition 8.3.7 (Nondeterministic Turing machine). A *nondeterministic Turing machine* (NTM) has a transition function that returns a *set* of choices:

$$\delta: Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

From a configuration the machine may proceed to any of the listed successor configurations. Its computation on input w is a tree: the root is the start configuration, and the children of a node are its successors under δ . The NTM *accepts* w if *some* branch of the tree reaches an accepting configuration; it rejects w if *every* branch halts in a rejecting configuration.

Acceptance is existential: one accepting branch among possibly infinitely many is enough. Picture the tree – each node a configuration, each downward edge one nondeterministic choice.



The computation tree of an NTM on w . The input is accepted because some branch (here through C_2) reaches an accepting configuration.

Theorem 8.3.8 (Nondeterminism collapses to determinism). *Every nondeterministic Turing machine N has an equivalent deterministic Turing machine D .*

Proof. D searches N 's computation tree for an accepting configuration. The search must be *breadth-first*: explore all branches to depth 1, then all to depth 2, and so on. Depth-first search would be fatal – D could dive down a single infinite branch and never discover an accepting configuration on a sibling branch.

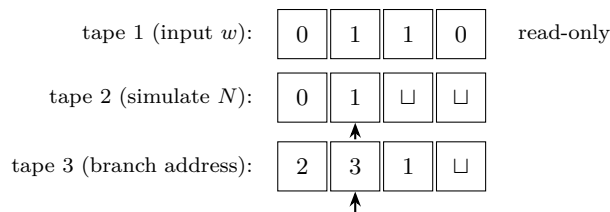
We give D three tapes (legitimate by Theorem 8.3.4):

- *Tape 1* holds the input w and is never altered;
- *Tape 2* is a work tape on which D simulates N along one branch;
- *Tape 3* holds an *address*: a string over $\{1, \dots, b\}$, where b is the largest number of choices δ_N ever offers. The address $d_1d_2 \dots d_m$ names the branch “at step j , take the d_j -th choice.”

D repeats: (1) copy w from tape 1 to tape 2; (2) simulate N on tape 2, at step j consulting symbol d_j on tape 3 to pick the choice – if d_j exceeds the number of choices available, or the address ends, abandon this branch; (3) if this branch accepts, D accepts; (4) otherwise replace tape 3 with the next address in shortlex order (the order that lists all shorter addresses before any longer one) and repeat from (1).

Listing addresses in shortlex order visits the tree in breadth-first order, so D reaches every node at finite depth after finitely many rounds. If some branch of N accepts at depth m , the corresponding address is tried after finitely many rounds and D accepts. If no branch accepts, D never accepts – looping if N has an infinite non-accepting branch, exactly mirroring N . Thus $L(D) = L(N)$. $\square$

Here is the three-tape simulator at a glance.



D simulating N : tape 1 preserves the input, tape 2 runs one branch, tape 3's address 231 selects which choice to take at each step.

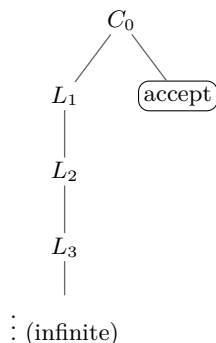
Corollary 8.3.9. *A language is Turing-recognizable if and only if some NTM recognizes it. It is decidable iff some NTM decides it (an NTM decides a language if it recognizes it and every branch halts on every input).*

Proof. Every deterministic TM is a (degenerate) NTM, giving one direction. Conversely Theorem 8.3.8 converts an NTM recognizer to a DTM. For deciders, modify D to also detect when *all* branches of N have halted at the current depth and no deeper nodes exist; if N halts on every branch of every input, D then halts too, rejecting if no branch accepted. $\square$

Worked Example 8.3.10 (Why depth-first search breaks the NTM simulation). Exhibit a computation tree on which depth-first search of D fails but breadth-first succeeds, and read off the moral.

Recognition. The question is about *search order*, not the language. The trigger is “an infinite branch sitting to the left of an accepting branch”: depth-first commits to the left branch forever. This is exactly the failure Theorem 8.3.8 avoids.

Solution. Consider an NTM whose root has two choices. The first choice begins an infinite chain of configurations $L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \dots$ that never accepts; the second choice accepts after one step.



Depth-first search follows the leftmost child first: it walks $C_0 \rightarrow L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \dots$ and never backtracks to the right child, so it never finds the accepting leaf – D would loop forever on an input N accepts. Breadth-first search visits both depth-1 nodes (L_1 and the accepting leaf) before going deeper, so it finds the accepting leaf in the second round and D accepts.

Which idea, and why? The simulator must enumerate branches by *increasing depth* (shortlex over addresses), because acceptance can hide on a short branch while a sibling branch is infinite; breadth-first guarantees every finite-depth node is reached after finitely many rounds.

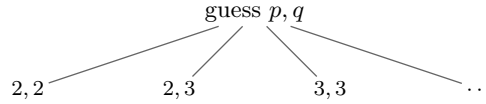
Check. With breadth-first order the addresses tried are $\varepsilon, 1, 2, \dots$; address 2 (root's second choice) reaches the accepting leaf at depth 1 and is tried in the second round. Accept, as required.

Worked Example 8.3.11 (An NTM that guesses a factor, and its deterministic cost). Let $\mathcal{L} = \{1^n \mid n \text{ is composite}\}$ (unary composite numbers). Give an NTM that recognizes $\mathcal{L}$ by guessing and explain what its determinization does.

Recognition. “ n is composite” means $n = p \cdot q$ with $p, q \geq 2$. Verifying a *given* factorization is easy; *finding* one is the hard part. The trigger “there exists a witness that is cheap to check” is precisely what nondeterminism is for: guess the witness, then verify deterministically.

Solution. On input 1^n : (1) nondeterministically write some 1^p with $p \geq 2$ on a work tape and some 1^q with $q \geq 2$ on another (each guess a branch of the tree); (2) compute $p \cdot q$ by repeated copying and compare its

length to n ; (3) accept on this branch iff $p \cdot q = n$. Some branch accepts iff n has a factorization into two factors ≥ 2 , i.e. iff n is composite. The tree's branching at step (1) is the guessing:



For $n = 9$ the branch $p = q = 3$ verifies $3 \cdot 3 = 9$ and accepts. Determinizing by Theorem 8.3.8 replaces the guess with a breadth-first search over all pairs (p, q) – effectively trial division – which still recognizes $\mathcal{L}$, just without the guessing shortcut.

Which idea, and why? Nondeterminism = “guess a witness, then verify deterministically”; the determinization (Theorem 8.3.8) turns the single guessed branch into an exhaustive search over all candidate witnesses, preserving the recognized language.

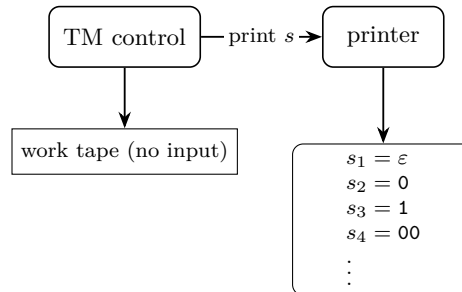
Check. $n = 9$: branch $(3, 3)$ accepts, so $1^9 \in \mathcal{L}$ (correct, $9 = 3 \cdot 3$). $n = 7$: no pair $p, q \geq 2$ multiplies to 7, no branch accepts, $1^7 \notin \mathcal{L}$ (correct, 7 is prime).

Enumerators

Definition 8.3.12 (Enumerator). An *enumerator* E is a Turing machine with a write-only *printer* attached and no input (its work tape starts blank). Whenever E enters a special *print* state it sends the current contents of a designated output region to the printer, appending that string to an ever-growing list. The *language enumerated by* E is

$$L(E) = \{ s \in \Sigma^* \mid E \text{ eventually prints } s \}.$$

E may run forever, may print strings in any order, and may print repeats; only the set of strings that ever appear matters.



An enumerator: a Turing machine that, instead of accepting an input, prints an ever-growing list. Its language is the set of strings that ever appear.

Theorem 8.3.13 (Enumerators recognize exactly the recognizable languages). *A language A is Turing-recognizable if and only if some enumerator enumerates it.*

Proof. ($\Leftarrow$) Suppose an enumerator E enumerates A . Build a recognizer M : on input w , run E and compare each string it prints against w ; accept as soon as a printed string equals w . If $w \in A$ then E prints w eventually and M accepts; if $w \notin A$, w is never printed and M runs forever – which is allowed for a recognizer. So $L(M) = A$.

($\Rightarrow$) Suppose a TM M recognizes A . Fix an enumeration $s_1, s_2, s_3, \dots$ of *all* strings in Σ^* (e.g. in shortlex order). Build an enumerator E :

For $i = 1, 2, 3, \dots$: run M for i steps on each of $s_1, \dots, s_i$ (a finite batch); whenever one of these bounded runs accepts s_j , print s_j .

This *dovetailing* avoids getting stuck: M might loop on some $s_j \notin A$, but E only ever runs M for a bounded number of steps in each round, so no single divergent run can stall the others. If $w \in A$, then M accepts $w = s_j$ within some number of steps t ; once $i \geq \max(j, t)$ the round- i batch accepts w and E prints it. If $w \notin A$, M never accepts w , so E never prints it. Hence E enumerates exactly A . $\square$

The forward direction is where the name *recursively enumerable* – an older synonym for Turing-recognizable – comes from: a language is recognizable exactly when its members can be mechanically listed. The dovetailing schedule (round i : run each of the first i strings for i steps) is the reusable gadget; it interleaves infinitely many possibly-divergent computations so each makes progress.

Worked Example 8.3.14 (Enumerate $\{0^n 1^n \mid n \geq 0\}$ in order; recognizing it back). Describe an enumerator for $\mathcal{L} = \{0^n 1^n \mid n \geq 0\}$ and trace its first outputs, then say what the recognizer-from-enumerator does.

Recognition. This language has an obvious generation rule – “print $0^n 1^n$ for $n = 0, 1, 2, \dots$ ” – so we can enumerate it *directly* without dovetailing a recognizer. The trigger “I can list the members by a formula” means an enumerator is the natural description.

Solution. The enumerator E ignores all input and loops: for $n = 0, 1, 2, \dots$, write n copies of 0 followed by n copies of 1 on the work tape and print. Its output list begins

$\varepsilon, 01, 0011, 000111, 00001111, \dots$

printing ($n = 2$):

0	0	1	1
---	---	---	---

 $\rightarrow$ list

By Theorem 8.3.13 ($\Leftarrow$), a recognizer M for $\mathcal{L}$ runs E and, on input w , accepts the moment E prints a string equal to w . Because E prints in strictly increasing length, M even *halts and rejects* once E has printed a string longer than w without a match – so here $\mathcal{L}$ is in fact decidable.

Which idea, and why? When a language’s members follow a generation rule, an enumerator is built directly from that rule; the equivalence (Theorem 8.3.13) then hands back a recognizer for free, and ordered output upgrades recognition to a decision.

Check. Is 0011 enumerated? Yes – it is the $n = 2$ output. Is 0111? It never appears (the rule always pairs equal counts), so M rejects it. Correct.

Method (How to prove a TM variant is no more powerful than the basic model).

1. **State the equivalence as a simulation.** You must show each model can simulate the other (Definition 8.3.1). The easy direction is usually “basic is a special case of the variant” (set $k = 1$, use a singleton choice set, etc.).
2. **Design the encoding.** Decide how the basic machine will store the variant’s extra data on its own tape(s): blocks separated by $\#$ for several tapes; dotted symbols for several heads; an address string for a branch of a nondeterministic tree.
3. **Simulate one step.** Describe how the basic machine reads the relevant data, looks up the variant’s transition, and updates the encoding. Keep the invariant “after each simulated step, my tape encodes the variant’s full configuration.”
4. **Get the search order right.** If the variant branches (nondeterminism) or runs many computations (enumeration), schedule them *breadth-first* / by *dovetailing* so no infinite branch starves a finite one.
5. **Match accept/reject/loop.** Verify the simulator accepts exactly when the variant accepts, and conclude the languages are equal; for deciders, check halting is preserved.

Caution. Do not simulate a nondeterministic machine by depth-first search. Diving down one branch can lead into an infinite non-accepting computation while an accepting configuration waits one step down a sibling branch – the simulator would loop forever on an input the NTM accepts. The fix is breadth-first search (equivalently, shortlex enumeration of branch addresses), which reaches every finite-depth configuration after finitely many rounds.

Caution. “Recognizes” and “decides” are not the same after determinization. Theorem 8.3.8 gives a deterministic *recognizer*: if every branch of the NTM rejects but one branch is infinite, the simulator loops rather than rejecting. You get a *decider* only when the NTM halts on *every* branch of *every* input (Corollary 8.3.9). Claiming nondeterminism preserves decidability without that halting condition is a common slip.

Caution. An enumerator's order and repetition carry no meaning. The language $L(E)$ is the *set* of strings ever printed – printing the same string twice, or printing in a scrambled order, changes nothing. Do not assume a string is absent just because it has not appeared yet; E may print it arbitrarily later. Only when E prints in strictly increasing order can you conclude a not-yet-seen short string is absent (and then the language is decidable, not merely recognizable).

Exercises.

1. **Stay-put gadget.** A Turing machine M has a stay-put instruction $\delta(q, a) = (r, b, S)$. Write out the two ordinary (L/R -only) instructions that replace it, naming the fresh intermediate state, and explain why the replacement leaves the head and tape in exactly the same configuration as the original step.
2. **A four-direction head.** Suppose we extended the basic TM so that on each step the head may move L , R , jump two cells right (RR), or stay (S). Argue that this variant is equivalent to the basic model by describing, for each of RR and S , how to simulate it with ordinary single-cell moves. (You need not write the full transition table; describe the gadgets.)
3. **Read the encoding.** A single-tape machine S simulates a 3-tape machine M as in Theorem 8.3.4, using $\#$ as the block delimiter and a dot for each virtual head. Decode the snapshot

$\# 0 \dot{1} 0 \# \sqcup \# a \dot{b} \#$

into the contents of M 's three tapes and the symbol under each of M 's three heads.

4. **Design a 2-tape machine.** Give an implementation-level description of a 2-tape Turing machine that decides $\mathcal{L} = \{w\#w \mid w \in \{0, 1\}^*\}$ (two equal halves separated by $\#$), using the second tape as scratch. State precisely what each tape holds during the comparison, and say in one sentence why a one-tape machine for the same language must do more head shuttling.
5. **Count the simulation cost.** A 2-tape TM runs for t steps, using at most t cells on each tape. Explain why the single-tape simulator of Theorem 8.3.4 may use as many as $O(t^2)$ steps to simulate those t steps. (Identify the operation in the simulation whose cost grows with the tape length.)
6. **Draw a computation tree.** A nondeterministic TM N on some input has start configuration C_0 ; from C_0 it has two choices leading to C_1 and C_2 ; C_1 rejects (no children); C_2 has three choices leading to C_3, C_4, C_5 ; C_4 is accepting. Draw the computation tree, mark the accepting node, and state whether N accepts the input and why.
7. **Search order matters.** Construct (in words) a computation tree of a nondeterministic TM on which a left-to-right *depth-first* simulator never accepts, even though the NTM does accept the input. Then state which symbol-by-symbol branch-address order the simulator should use instead, and why it succeeds.
8. **Guess-and-check NTM.** Let $\mathcal{L}$ be the language of all $w \in \{0, 1\}^*$ that contain a palindromic substring of length ≥ 3 (a substring equal to its own reverse). Design a nondeterministic Turing machine recognizing $\mathcal{L}$. Describe the guessing step and the deterministic verification step separately, and say what the determinization of Theorem 8.3.8 does to your guess.
9. **Decider vs. recognizer.** An NTM N recognizes a language A , and on every input *every* branch of N halts. Explain why the deterministic machine produced by Theorem 8.3.8 can be modified into a *decider* for A , and pinpoint exactly the step of the breadth-first simulation where the modification detects “all branches have halted.”
10. **Enumerate a regular language.** Give an enumerator that prints exactly the strings of $\mathcal{L} = \{w \in \{0, 1\}^* \mid w \text{ contains the substring } 11\}$, and list the first five strings it outputs if it generates strings in shortlex order. Does your enumerator print in increasing length? What does that let you conclude about decidability of $\mathcal{L}$?
11. **Dovetailing.** A Turing machine M recognizes a language A , but M loops on some strings not in A . Explain why the enumerator “for $j = 1, 2, 3, \dots$: run M to completion on s_j , and print s_j if it accepts” *fails* to enumerate A , and how the dovetailing schedule of Theorem 8.3.13 (round i : run each of $s_1, \dots, s_i$ for i steps) repairs it.

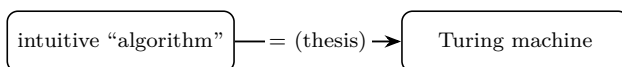
12. *Find the bug.* A classmate claims the following enumerator E enumerates a language A recognized by a TM M : “For $j = 1, 2, 3, \dots$: run M on s_j until it halts; if it accepts, print s_j ; then go to $j + 1$.” Identify the bug that makes E incorrect for some M , give a concrete situation where E fails to print a string of A , and state the one change that fixes it.
13. **Ordered enumeration implies decidable.** Prove: if an enumerator E prints the strings of an infinite language A in strictly increasing shortlex order (no repeats), then A is decidable. (Describe the decider and argue it always halts.)

8.4 The Church–Turing Thesis, Encodings, and the Universal TM

Sections 8.1 through 8.3 built the Turing machine and then showed how astonishingly *robust* it is: multitape, nondeterministic, and enumerator variants all recognize the same class of languages. This section draws the philosophical conclusion of that robustness – the *Church–Turing thesis*, which identifies “algorithm” with “Turing machine” – and then cashes it out technically. The one key idea is that a Turing machine takes a *string* as input, so to feed it a graph, a polynomial, an automaton, or even another Turing machine, we must first *encode* that object as a string $\langle O \rangle$. Encoding lets a single fixed machine – the *universal* Turing machine – read a description $\langle M, w \rangle$ and simulate M on w , which is the seed of everything in the next chapter.

The Church–Turing thesis

Every reasonable model of unrestricted computation proposed since 1936 – Turing machines, Church’s λ -calculus, general recursive functions, register machines, cellular automata, every general-purpose programming language – has turned out to recognize exactly the Turing-recognizable languages, no more and no less. There is no theorem that *all conceivable* models must agree, because “algorithm” was, before 1936, only an informal idea. What we have instead is a sweeping empirical convergence, elevated to a working definition.



The Church–Turing thesis equates the informal notion of an effective procedure with what a Turing machine can do.

Definition 8.4.1 (Church–Turing thesis). The *Church–Turing thesis* asserts that a problem can be solved by *some* effective mechanical procedure (an “algorithm” in the intuitive sense) if and only if it can be solved by a Turing machine. It is not a mathematical theorem – one side of the claimed equivalence, the informal notion, is not formally defined – but a thesis about the adequacy of the formal model. We adopt it: from now on “there is an algorithm for X ” *means* “some Turing machine decides X ,” and “ X is computable” means “some Turing machine computes it.”

The thesis is what gives the undecidability results of Chapter 9 their bite. To prove that *no algorithm whatsoever* can test a polynomial for integer roots (Hilbert’s tenth problem, settled negatively by Matiyasevich in 1970), it suffices – by the thesis – to prove that no Turing machine can; and the latter is a precise mathematical statement we can attack. Without a formal model, “no algorithm exists” could never be more than a confession of failure to find one.

Remark. The thesis runs in both directions and both are used. “Algorithm $\Rightarrow$ TM” lets us describe machines in plain English (an *implementation-level* or *high-level* description) and trust that a formal 7-tuple exists; we did exactly this in Section 8.2. “TM $\Rightarrow$ algorithm” lets us treat anything a Turing machine does as a legitimate effective procedure when we reason informally.

Encoding objects as strings

A Turing machine reads a string and nothing else. Yet the problems we care about – is this graph connected? does this automaton accept the empty string? does this machine halt? – are about *objects*: graphs, automata, machines. The bridge is a fixed, decodable *encoding* of objects as strings.

Definition 8.4.2 (Encoding). Let O be a finite discrete object (a number, a tuple, a graph, an automaton, a Turing machine, a list of such objects, ...). An *encoding* of O is a string $\langle O \rangle \in \Sigma^*$ over some fixed finite alphabet, chosen so that the map $O \mapsto \langle O \rangle$ is injective and *decodable*: there is an algorithm that, given $\langle O \rangle$, recovers O , and an algorithm that, given an arbitrary string, decides whether it is a legal encoding. For several objects we write $\langle O_1, O_2, \dots, O_k \rangle$ for the encoding of the whole list as one string.

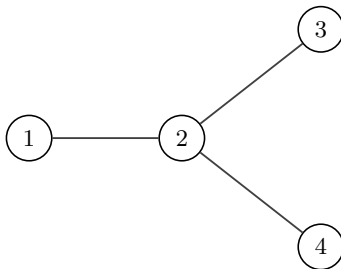
Three points make encodings harmless. First, the *particular* encoding does not matter: any two reasonable encodings can be translated into each other by a Turing machine, so decidability and recognizability are independent of the choice. Second, a machine that expects $\langle O \rangle$ *implicitly first checks* that its input is a well-formed encoding and rejects junk, so we never have to worry about ill-formed inputs. Third, encoding is cheap: it never turns a decidable problem into an undecidable one, because decoding is itself an algorithm.

Worked Example 8.4.3 (Encoding a finite graph as a string). Fix the alphabet $\Sigma = \{0, 1, \#, (,), ,\}$ (commas between list items written “#”). Encode the undirected graph G with vertices $\{1, 2, 3, 4\}$ and edges $\{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$ as a string $\langle G \rangle$, and describe how a machine recovers G .

Recognition. An object-to-string task. The trigger is Definition 8.4.2: pick *any* systematic, decodable layout – a vertex list followed by an edge list – and write it down in binary.

Solution. List the vertices, then the edges as unordered pairs. Writing each number in binary ($1=1, 2=10, 3=11, 4=100$) and using “#” to separate fields:

$$\langle G \rangle = \underbrace{1\#10\#11\#100}_{\text{vertices}} \#\# \underbrace{(1,10)\#(10,11)\#(10,100)}_{\text{edges}}.$$



The graph G encoded by $\langle G \rangle$ above: a star centred at vertex 2.

Which idea, and why? Definition 8.4.2 only asks for an injective, decodable layout; a vertex-list-then-edge-list format is the canonical choice, and binary numerals plus a separator make every field unambiguous.

Check. To decode, a machine scans to “##”, reads the vertices left of it and the edge pairs right of it, verifies every endpoint occurs in the vertex list, and rejects otherwise. Here all of 1, 2, 3, 4 appear, so $\langle G \rangle$ is well-formed and recovers G uniquely.

Worked Example 8.4.4 (Encoding a DFA, and the well-formedness check). Show how to encode an arbitrary DFA $M = (Q, \Sigma, \delta, q_0, F)$ as a string $\langle M \rangle$, and explain what the implicit input check does on a bad string.

Recognition. Again Definition 8.4.2: a DFA is just five finite pieces of data, so encode each piece and concatenate. The trigger is “the object is a finite tuple of finite sets and a finite table.”

Solution. Number the states $q_1, \dots, q_n$ and the alphabet symbols $a_1, \dots, a_m$. Encode M as the list

$$\langle M \rangle = \langle n, m, \delta, (\text{index of } q_0), F \rangle,$$

where the transition function δ is written as the table of triples (i, j, k) meaning $\delta(q_i, a_j) = q_k$, and F as the list of accepting-state indices. Each number is in binary; fields and rows are separated by reserved symbols.

$$\langle M \rangle = \boxed{n} \quad \boxed{m} \quad \boxed{\delta \text{ table}} \quad \boxed{\text{start}} \quad \boxed{F}$$

Layout of $\langle M \rangle$: a fixed sequence of fields, each a decodable binary block.

Which idea, and why? An object built from finitely many finite components is encoded component by component; the only requirement (Definition 8.4.2) is that the layout be parseable back into those components.

Check. On input x , the machine first parses the fields. If x has the wrong number of fields, or names a transition target q_k with $k > n$, or lists an accepting index $> n$, the parse fails and the machine rejects x as “not a valid $\langle M \rangle$.” Only well-formed strings reach the actual computation – this is the “implicitly tests and rejects” clause of Definition 8.4.2.

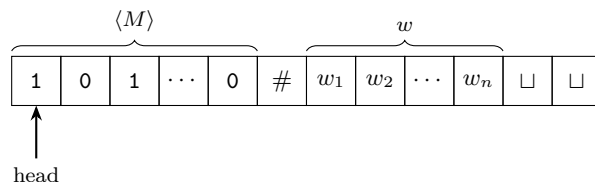
Encoding a machine together with its input: $\langle M, w \rangle$

The single most important encoded object is a *pair*: a machine M and an input string w for it. Because a Turing machine is a finite object, $\langle M \rangle$ is a perfectly ordinary string; pairing it with w gives $\langle M, w \rangle$. This is the input on which all of computability theory turns, because it lets one machine take *another machine* as data.

Worked Example 8.4.5 (Reading the pair $\langle M, w \rangle$ off a tape). Describe the contents of the input tape when a machine is started on $\langle M, w \rangle$, and identify where M ends and w begins.

Recognition. A tape-layout task: $\langle M, w \rangle$ is one string, the concatenation of the encoding of M , a reserved separator, and w itself. The trigger is “a pair encoding $\langle \cdot, \cdot \rangle$.”

Solution. Place $\langle M \rangle$ on the tape, then a separator symbol $\#$ that occurs nowhere inside $\langle M \rangle$, then the literal input string w . The first blank marks the right end. A machine finds the boundary by scanning to the (uniquely reserved) $\#$.



The tape holding $\langle M, w \rangle$: the description of M , a reserved separator $\#$, then the input w , then blanks $\square$.

Which idea, and why? A pair is encoded by concatenating the two encodings with a separator that cannot appear inside the first component, so the boundary is found by a single scan – exactly the decodability demanded by Definition 8.4.2.

Check. The separator $\#$ is reserved, so it occurs in $\langle M, w \rangle$ only at the boundary; scanning left to right, the first $\#$ ends $\langle M \rangle$ and everything after it (up to the first blank) is w . The decoding is unambiguous.

The universal Turing machine

Encodings let us state Turing’s most striking construction. A single fixed machine U can simulate *every* Turing machine: hand U the string $\langle M, w \rangle$ and it does to w whatever M would do. The machine M is no longer hard-wired into the control; it is *data* on the tape. This is precisely the principle of the stored-program computer, a decade before such computers were built.

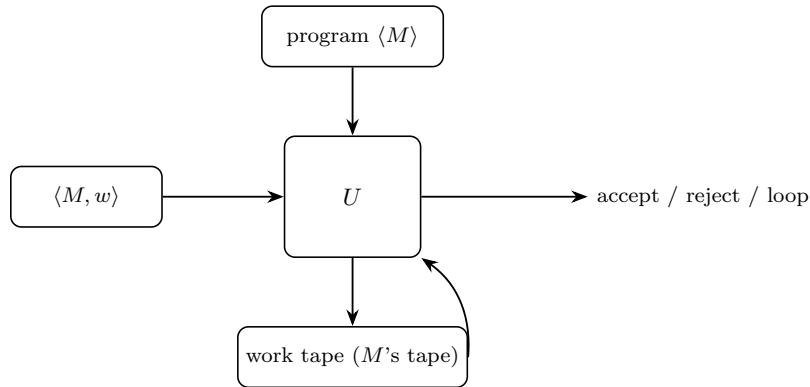
Theorem 8.4.6 (Universal Turing machine). *There is a Turing machine U , the universal Turing machine, such that for every Turing machine M and every input string w ,*

$$U \text{ on input } \langle M, w \rangle \text{ accepts} \iff M \text{ on input } w \text{ accepts},$$

and likewise U rejects $\langle M, w \rangle$ iff M rejects w , and U loops on $\langle M, w \rangle$ iff M loops on w . Consequently $L(U) = \{\langle M, w \rangle \mid M \text{ is a TM that accepts } w\}$.

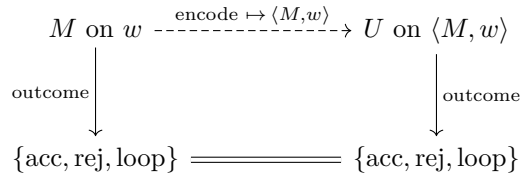
Proof. U keeps three blocks of information, which by Theorem 8.3.4 we may put on three tapes: tape 1 holds the (never-altered) description $\langle M \rangle$; tape 2 holds the simulated contents of M ’s tape; tape 3 holds the current simulated state of M . On input $\langle M, w \rangle$, U first parses $\langle M \rangle$ and verifies it is a legal machine encoding (rejecting if not), copies w onto tape 2 as M ’s starting tape, and writes M ’s start state $\langle q_0 \rangle$ on tape 3.

To simulate one step of M , U reads the symbol under M 's virtual head from tape 2 and the current state from tape 3, then scans the transition table inside $\langle M \rangle$ on tape 1 for the matching rule $\delta(q, a) = (r, b, D)$. Finding it, U writes b on tape 2, updates tape 3 to $\langle r \rangle$, and moves the virtual head on tape 2 one cell in direction D . If the new state on tape 3 is M 's accept state, U accepts; if it is M 's reject state, U rejects; otherwise U simulates the next step. Each of M 's steps is reproduced exactly, so U accepts, rejects, or loops precisely as M does on w . $\square$



Schematic of the universal machine U : it reads the program $\langle M \rangle$ and runs it step by step on a work tape initialised to w , echoing M 's outcome.

The schematic shows *how* U runs; the square below states *what* it preserves – the only thing a recognizer can be judged by, the outcome over $\{\text{accept, reject, loop}\}$.



The universal machine U decodes $\langle M, w \rangle$ and steps through M 's computation on w , halting exactly as M does: U accepts/rejects/loops iff M does. Simulation preserves the outcome.

Check. U maintains M 's tape, state, and head on its own tapes and applies M 's transition each step, so $U(\langle M, w \rangle)$ accepts iff $M(w)$ accepts (and likewise rejects/loops): the square commutes on the nose.

Corollary 8.4.7 (A universal recognizer). *The language $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$ is Turing-recognizable.*

Proof. By Theorem 8.4.6, U recognizes exactly A_{TM} : it accepts $\langle M, w \rangle$ iff M accepts w , and otherwise (when M rejects or loops) U does not accept. So U is a recognizer for A_{TM} . $\square$

Remark. U recognizes A_{TM} but does not *decide* it: when M loops on w , the simulation loops too, so U never halts to say “no.” That gap – recognizable but apparently not decidable – is exactly the crack that Chapter 9 pries open to produce the first undecidable language. The universal machine is therefore both the foundation of the stored-program idea and the doorway to the limits of computation.

Worked Example 8.4.8 (Tracing the universal machine on a tiny example). Let M be a one-state-plus-halts DFA-like Turing machine over $\Sigma = \{\mathbf{a}\}$ that, on reading $\mathbf{a}$, immediately moves right and accepts, and that accepts at once on a blank. Trace what U does on $\langle M, \mathbf{a} \rangle$ at the level of “which step of M is being simulated.”

Recognition. A simulation trace: apply the proof of Theorem 8.4.6 step by step, watching the three blocks of U 's memory. The trigger is “what does U do, concretely.”

Solution. Parsing succeeds: $\langle M \rangle$ is a legal machine. U copies $\mathbf{a}$ to its work tape and writes M 's start state to the state block.

U 's phase	state block	work tape (head underlined)	action
init	q_0	$\underline{\mathbf{a}} \sqcup$	copied w , set q_0
sim step 1	q_0	$\underline{\mathbf{a}} \sqcup$	rule $\delta(q_0, \mathbf{a}) = (q_{\text{acc}}, \mathbf{a}, R)$
after step 1	q_{acc}	$\mathbf{a} \sqcup$	state is accept
halt	q_{acc}	$\mathbf{a} \sqcup$	U accepts

Because the simulated state became M 's accept state, U accepts $\langle M, \mathbf{a} \rangle$ – which is correct, since M accepts $\mathbf{a}$. *Which idea, and why?* Theorem 8.4.6 says U reproduces M 's computation one step at a time and copies M 's verdict; tracing U is therefore just tracing M while bookkeeping the three memory blocks.

Check. M accepts $\mathbf{a}$ in one step, and U accepted after simulating exactly one step. Had M instead looped on $\mathbf{a}$, the table would never reach an accept/reject state and U would loop too, as Theorem 8.4.6 requires.

Method (How to set up and use an encoding $\langle O \rangle$).

1. **List the components.** Break O into its finitely many finite parts (vertices and edges; states, alphabet, transition table, start, accepts; a machine and an input).
2. **Choose a flat layout.** Write each part as a binary block and separate fields and records with reserved symbols that cannot occur inside a block. Encode a list of objects by concatenating their encodings with a separator.
3. **Check decodability.** Make sure an algorithm can parse the string back into O and can reject any string that is not a legal encoding. You never need the “best” encoding, only a decodable one – all reasonable choices are interchangeable.
4. **Use it as input.** Feed $\langle O \rangle$ (or $\langle O_1, \dots, O_k \rangle$) to a machine; assume the machine first verifies well-formedness and rejects junk, then operates on the recovered object. For machine-on-machine problems, the key object is the pair $\langle M, w \rangle$, run by the universal machine U .

Caution. Do not confuse a machine M with its encoding $\langle M \rangle$. M is a tuple of sets and functions; $\langle M \rangle$ is a *string*. Only strings can sit on a tape, so whenever a machine takes “another machine” as input it really takes the string $\langle M \rangle$. Writing “run M on N ” is shorthand for “run M on the string $\langle N \rangle$.”

Caution. The universal machine U *recognizes* A_{TM} ; it does not *decide* it. When the simulated M loops on w , U loops as well and never reports “reject.” Treating U as a decider – as if it could always answer “does M accept w ?” – is the single most common error, and assuming it leads straight to a contradiction (Chapter 9). U can confirm acceptance, never refute it.

Caution. The particular encoding is irrelevant, but it must still be *decodable*. An “encoding” with no separators, in which the boundary between $\langle M \rangle$ and w cannot be located, is not a legal encoding at all – the map must be invertible by an algorithm. Sloppiness here (running fields together with no delimiter) breaks the well-formedness check that every encoded-input machine relies on.

Exercises.

1. Fix the alphabet of Worked Example 8.4.3 on graph encodings. Write down $\langle H \rangle$ for the directed graph H with vertices $\{1, 2, 3\}$ and directed edges $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$, using ordered pairs in the edge list. Then describe, in one or two sentences, the algorithm that decodes your string back into H .
2. Let $\Sigma = \{0, 1\}$ and consider two different reasonable encodings of natural numbers: (a) ordinary binary, and (b) the unary scheme $n \mapsto 1^n$. Give an algorithm that converts an input in encoding (a) into the corresponding string in encoding (b). What does the existence of such conversion algorithms (in both directions) tell you about whether a language of numbers is decidable under (a) versus under (b)?
3. Design (as an implementation-level description, no formal 7-tuple) a Turing machine that, given $\langle G \rangle$ for an undirected graph G in the format of the section, decides whether G has an *isolated* vertex (a vertex appearing in the vertex list but in no edge). Then appeal to the Church–Turing thesis to justify the single phrase “and we can clearly do this.”

4. Explain precisely why the universal machine U recognizes $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$ but is not known to *decide* it. Identify the exact step in the simulation where a decision can fail to be reached, and say what input behaviour of M causes it.
5. A student proposes encoding a pair $\langle M, w \rangle$ by simply concatenating $\langle M \rangle$ and w with *no* separator, arguing “a string is a string.” Exhibit a concrete way this can go wrong: give (informally) two different pairs $(M_1, w_1) \neq (M_2, w_2)$ whose proposed encodings are the same string, and state which clause of the definition of an encoding is violated.
6. Suppose someone hands you a Turing machine V and claims it is universal. Describe a finite set of test inputs $\langle M_i, w_i \rangle$ you could run on V to *increase your confidence* that V is universal, and explain why no finite set of tests could ever *prove* it. (Relate your answer to the difference between recognizing and deciding.)
7. Let $\mathcal{L} = \{\langle M \rangle \mid M \text{ is a Turing machine that, started on the blank tape, halts within 100 steps}\}$. Argue that $\mathcal{L}$ is decidable by describing a machine that decides it. Where does your machine use the universal-simulation idea, and why does the 100-step bound rescue it from the looping problem that afflicts U on A_{TM} ?
8. Give an encoding $\langle P \rangle$ of a single-variable integer polynomial $p(x) = c_k x^k + \cdots + c_1 x + c_0$ as a string over a finite alphabet, allowing negative coefficients. Then describe a Turing machine that, on input $\langle P \rangle$ and a binary integer $\langle n \rangle$, computes $p(n)$ and halts. (You may use high-level steps; cite the Church–Turing thesis for the arithmetic.)
9. “Find the bug.” A classmate writes: “The universal machine U decides A_{TM} , because on input $\langle M, w \rangle$ it simulates M on w ; if the simulation accepts, U accepts, and if the simulation does not accept, U rejects.” Pinpoint the false step and explain what actually happens in the case the classmate has overlooked.
10. “Find the bug.” Another classmate claims: “Since there are uncountably many languages but the encoding $\langle M \rangle$ shows there are only countably many Turing machines, encoding objects as strings must lose information.” Explain why the conclusion does not follow: distinguish the claim that *each individual object* has a faithful (injective, decodable) encoding from any claim about *all languages* being encodable, and identify the genuine fact that the counting argument really establishes.
11. Argue, using the Church–Turing thesis, that the class of decidable languages is closed under *decodable re-encoding*: if $\mathcal{L}$ over alphabet Σ is decidable and $f: \Sigma^* \rightarrow \Delta^*$ is a computable, algorithmically invertible encoding, then $f(\mathcal{L}) = \{f(w) \mid w \in \mathcal{L}\}$ is decidable. Sketch the decider for $f(\mathcal{L})$.
12. Let U be the universal machine of Theorem 8.4.6. Consider feeding U the encoding of *itself* paired with some input, i.e. a string of the form $\langle U, x \rangle$. Explain why this is a perfectly legal input that U simply runs (there is no paradox *yet*), and describe in words what U does step by step. (The paradox appears only in Chapter 9, when a machine is made to act on its *own* description in a self-contradictory way – note where the present innocent self-application stops short of that.)

8.5 Decidable and Turing-Recognizable Languages

Once we have a precise machine model, the natural next question is not “what can one particular Turing machine do” but “which *languages* can Turing machines pin down, and how exactly do they pin them down.” The answer splits into two classes that look almost the same but differ on one crucial point: whether the machine is guaranteed to give an answer. A *recognizer* promises to accept every string in its language, but on a string outside the language it may reject or it may run forever; a *decider* additionally promises to halt on every input, so it always returns a definite yes or no. The one key idea of this section is that this single difference – “always halts” versus “halts when it accepts” – carves the Turing-acceptable languages into two nested classes with very different closure behaviour, and it is the formal home of the word *algorithm*. Everything in the rest of the book about what computers can and cannot do is phrased in terms of these two classes.

Definition 8.5.1 (Recognizing and deciding). Let M be a Turing machine. The *language recognized* by M is $L(M) = \{w \in \Sigma^* \mid M \text{ accepts } w\}$. On an input w , exactly one of three outcomes occurs: M *accepts* (halts in q_{acc}), *rejects* (halts in q_{rej}), or *loops* (never halts). A machine M is a *decider* if it halts on *every* input – it never loops – so on every w it either accepts or rejects. A decider M *decides* the language $L(M)$.

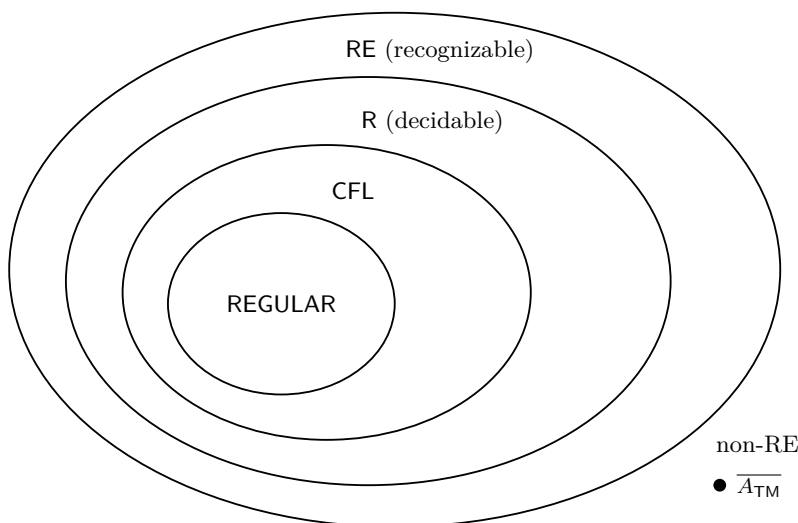
Definition 8.5.2 (The two classes). A language $\mathcal{L}$ is *Turing-recognizable* if some Turing machine recognizes it, and *decidable* (equivalently *Turing-decidable*) if some Turing machine decides it. Write RE for the class of Turing-recognizable languages and R for the class of decidable languages. (Older texts call RE the *recursively enumerable* languages and R the *recursive* languages.)

The distinction is entirely about the *reject* side. A recognizer may answer “yes” by halting in q_{acc} , but it is allowed to express “no” by never halting at all – and from the outside an observer cannot tell a machine that will eventually halt from one that will loop forever. A decider removes that ambiguity: every “no” is delivered as an actual halt in q_{rej} .

Proposition 8.5.3 (Deciders are recognizers). *Every decidable language is Turing-recognizable; that is, $R \subseteq \text{RE}$.*

Proof. If M decides $\mathcal{L}$, then in particular M recognizes $\mathcal{L}$: a decider is a Turing machine, and “decides $\mathcal{L}$ ” means it halts on every input and accepts exactly the strings of $\mathcal{L}$, which already entails “accepts exactly the strings of $\mathcal{L}$.” Demanding the extra halting guarantee can only shrink the class, never enlarge it, so $R \subseteq \text{RE}$. $\square$

The inclusion is proper – there are Turing-recognizable languages that no Turing machine decides – but proving it requires the diagonalization argument of the next chapter. For now we treat $R \subsetneq \text{RE}$ as the picture to keep in mind, and we record the landscape of all the language classes built so far.



The language classes nested so far: every regular language is context-free, every context-free language is decidable, every decidable language is recognizable, and some languages (e.g. $\overline{A_{\text{TM}}}$, Chapter 9) are not even recognizable. Each inclusion is proper.

Decidable languages are closed under the Boolean operations. The decider’s halting guarantee makes the easy closure proofs go through without a hitch: if a machine always stops, we can run it, read off its verdict, and act on the verdict. This is exactly what fails for bare recognizers, where “run it and read the verdict” may never return.

Theorem 8.5.4 (Closure of R). *The class of decidable languages is closed under complement, union, and intersection.*

Proof. Let $\mathcal{L}_1, \mathcal{L}_2$ be decided by deciders M_1, M_2 .

Complement. Build M' that on input w runs $M_1(w)$ – which is guaranteed to halt – and then *swaps* the verdict: accept if M_1 rejected, reject if M_1 accepted. Then M' halts on every input and accepts exactly $\overline{\mathcal{L}_1}$, so $\overline{\mathcal{L}_1}$ is decidable.

Union. Build $M_\cup$ that on input w runs $M_1(w)$ to completion; if it accepts, accept. Otherwise run $M_2(w)$ to completion; accept iff it accepts. Both runs halt, so $M_\cup$ halts and accepts w iff $w \in \mathcal{L}_1 \cup \mathcal{L}_2$.

Intersection. Same skeleton, but accept only if *both* $M_1(w)$ and $M_2(w)$ accept; again both runs halt, so $M_\cap$ is a decider for $\mathcal{L}_1 \cap \mathcal{L}_2$. (Alternatively, $\mathcal{L}_1 \cap \mathcal{L}_2 = \overline{\overline{\mathcal{L}_1} \cup \overline{\mathcal{L}_2}}$ and use the two cases above.) $\square$

The recognizable class is closed under union and intersection by the same “run them” idea, but the running must be done *carefully*, because a recognizer may loop and we must not get stuck on one machine while the other would have accepted.

Theorem 8.5.5 (Closure of RE under union and intersection). *The class of Turing-recognizable languages is closed under union and intersection.*

Proof. Let $\mathcal{L}_1, \mathcal{L}_2$ be recognized by M_1, M_2 .

Intersection. Build $M_\cap$ that on input w runs $M_1(w)$; if it ever accepts, then runs $M_2(w)$; if that also accepts, accept. If $w \in \mathcal{L}_1 \cap \mathcal{L}_2$ both sub-runs accept and $M_\cap$ accepts; if w is missing from either language the relevant sub-run fails to accept (it rejects or loops) and $M_\cap$ never accepts. So $L(M_\cap) = \mathcal{L}_1 \cap \mathcal{L}_2$.

Union. Here sequential running is unsafe: if $M_1(w)$ loops we never reach M_2 , even though M_2 might accept. The fix is to *interleave* – simulate one step of $M_1(w)$, then one step of $M_2(w)$, then the next step of each, and so on. Accept as soon as *either* simulation accepts. If $w \in \mathcal{L}_1 \cup \mathcal{L}_2$ then some accepting computation is finite, so it is reached after finitely many interleaved steps and $M_\cup$ accepts. If w is in neither language, neither simulation ever accepts and $M_\cup$ never accepts. Hence $L(M_\cup) = \mathcal{L}_1 \cup \mathcal{L}_2$. $\square$

Remark. The asymmetry is the whole point. R is closed under *complement*; RE is *not* (we prove this in Chapter 9). The reason the complement proof breaks for recognizers is that swapping “accept” and “reject” does nothing about *looping*: a recognizer that loops on w still loops after we relabel its halting verdicts, so the swap fails to turn “does not accept” into “accepts.” Only a decider, which never loops, can be safely flipped.

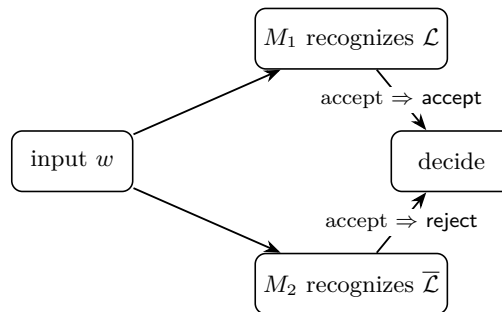
The relationship between the two classes is captured exactly by one elegant theorem, which also explains the name “co-recognizable” for the complement side.

Theorem 8.5.6 (Decidable = recognizable and co-recognizable). *A language $\mathcal{L}$ is decidable if and only if both $\mathcal{L}$ and its complement $\overline{\mathcal{L}}$ are Turing-recognizable.*

Proof. ($\Rightarrow$) If $\mathcal{L}$ is decidable then $\mathcal{L} \in \text{RE}$ by Proposition 8.5.3, and $\overline{\mathcal{L}}$ is decidable by Theorem 8.5.4, hence also in RE.

($\Leftarrow$) Suppose M_1 recognizes $\mathcal{L}$ and M_2 recognizes $\overline{\mathcal{L}}$. Build a decider M that on input w runs $M_1(w)$ and $M_2(w)$ *in parallel*, interleaving one step of each. Every string w lies in $\mathcal{L}$ or in $\overline{\mathcal{L}}$ (never neither, never both), so exactly one of the two machines is guaranteed to eventually accept w . As soon as one does, M halts: if M_1 accepted, M accepts; if M_2 accepted, M rejects. Because one of the two *must* accept after finitely many steps, M halts on every input, so it is a decider, and it accepts exactly $\mathcal{L}$. $\square$

The parallel-simulation device in this proof – run two machines a step at a time and watch for the first to halt – is worth a picture, because the same trick reappears constantly.

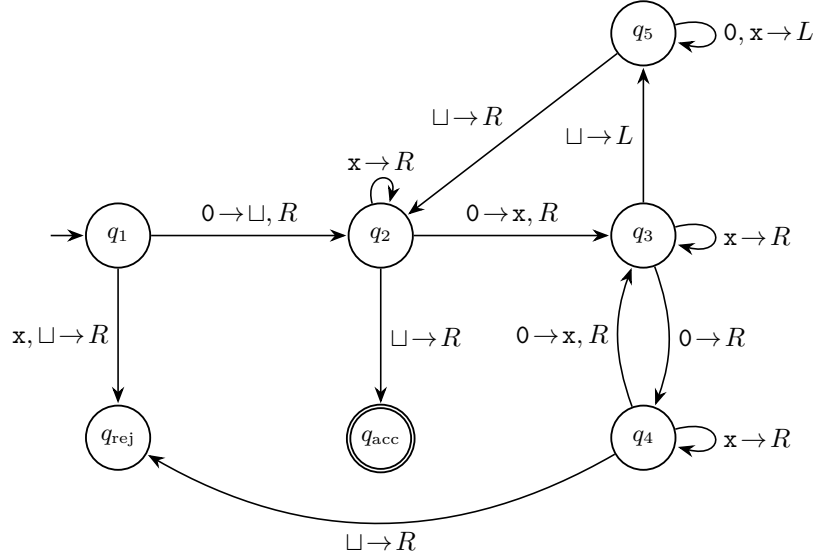


Parallel simulation: feed w to both recognizers a step at a time. Exactly one halts in “accept”; its identity decides w .

Worked Example 8.5.7 (A trivial decider: the power-of-two tape sweep). Sketch a decider for $A = \{0^n \mid n \text{ is a power of } 2\}$ and confirm it never loops.

Recognition. “Decider” means we must guarantee halting, not merely accepting. The trigger is that the machine must *make progress* on every input so that no computation runs forever.

Solution. Repeatedly sweep across the tape crossing off every other 0 and remembering the parity of the count seen. After a sweep: if no 0 remains (the input was empty), reject; if a single 0 remains, accept; if more than one remains and the count was odd, reject; otherwise return to the left end and sweep again.



The whole machine: q_1 overwrites the first 0 with $\sqcup$ (it still counts as a 0, and marks the left end); q_2, q_3, q_4 make one sweep, crossing off every other 0 (q_2 : one 0 seen, q_3 : an even number, q_4 : an odd number > 1); q_5 returns to the left end. A sweep that meets a lone 0 accepts; an odd count > 1 rejects.

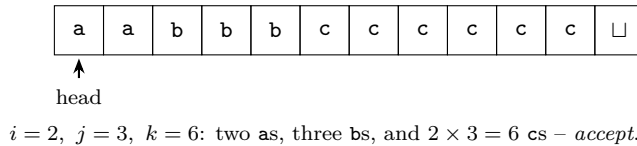
Which idea, and why? Each sweep halves the number of 0s, so that count strictly shrinks and the number of sweeps is at most $1 + \log_2$ of the input length; a strictly decreasing quantity bounded below forces halting on *every* input, which is exactly the decider requirement.

Check. On 0000 ($n = 4 = 2^2$): sweep one crosses two, leaving two 0s (even); sweep two crosses one, leaving one 0; accept. On 000 ($n = 3$): sweep one leaves two but the count 3 is odd > 1 ; reject. Both halt.

Worked Example 8.5.8 (Multiplication as a decider: $C = \{a^i b^j c^k \mid ij = k, i, j, k \geq 1\}$). Argue informally that C is decidable by describing a machine that always halts.

Recognition. The language encodes an arithmetic identity, $i \cdot j = k$, which a TM can *check* by crossing-off bookkeeping. Because the input is finite and each crossing-off makes progress, the process terminates – the trigger for “decidable, not merely recognizable.”

Solution. First verify the shape $a^+ b^+ c^+$ in one left-to-right pass; reject if it fails. Then, for each a , cross it off and run through the b block once, crossing off one c per b (shuttling between the blocks); restore the b s and repeat with the next a . Accept iff every c is crossed off exactly when the a s run out.



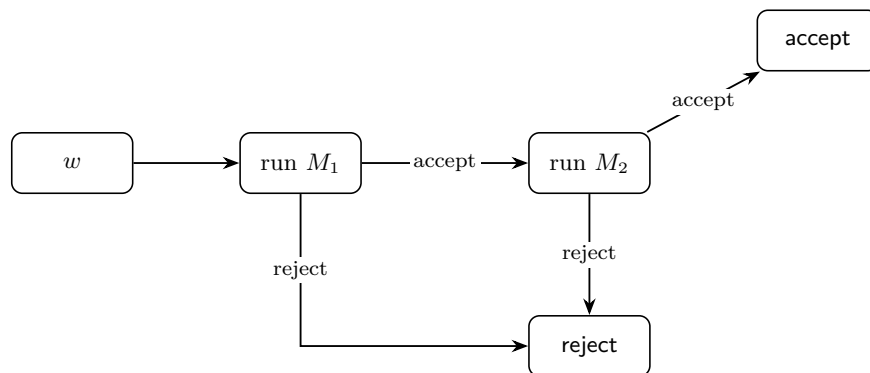
Which idea, and why? The membership test is a finite computation on a finite tape whose “work remaining” (uncrossed a s, then uncrossed c s) strictly decreases, so the machine halts on every input; a halting yes/no test is precisely a decider, putting $C \in R$.

Check. On aabbbcccccc ($i = 2, j = 3, k = 6$) the crossing-off ends with all c s gone as the last a is consumed: accept. On aabbbcccccc ($k = 5 \neq 6$) the c s run out one short: reject. Both halt.

Worked Example 8.5.9 (Closure in action: building a decider for an intersection). Given deciders for $\mathcal{L}_1 = \{w \mid w \text{ has even length}\}$ and $\mathcal{L}_2 = \{w \mid w \text{ ends in } 1\}$, design a decider for $\mathcal{L}_1 \cap \mathcal{L}_2$ and trace it.

Recognition. This is Theorem 8.5.4 (intersection) made concrete. The trigger is “combine two deciders”: because each halts, we may run one, then the other, and accept only on a double yes.

Solution. The combined decider $M_\cap$ runs $M_1(w)$ to its (guaranteed) halt; if it rejects, reject. Otherwise run $M_2(w)$ to its halt and copy its verdict. Schematically:



Sequential composition of two deciders: both halt, so the combination halts and accepts iff both accept.

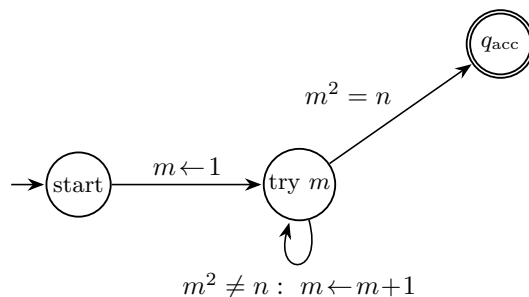
Which idea, and why? When every component machine is a *decider*, sequential “run then run” is safe – no run can stall the next – so it realises intersection directly; with bare recognizers we would have needed the interleaving of Theorem 8.5.5 instead.

Check. On $w = 0110$: length 4 is even (M_1 accepts) and it ends in 0 (M_2 rejects), so $M_\cap$ rejects – correct, since $0110 \notin \mathcal{L}_2$. On $w = 011$: length 3 is odd, M_1 rejects, $M_\cap$ rejects without ever running M_2 .

Worked Example 8.5.10 (A recognizer that is provably *not* a decider as written). Consider M that on input 0^n searches for a positive integer m with $m^2 = n$ by trying $m = 1, 2, 3, \dots$ and halting-accept when $m^2 = n$. What language does M recognize, and why is M (as described) not a decider?

Recognition. The machine has an unbounded search with no built-in stop, so it may run forever – the trigger that separates a recognizer from a decider.

Solution. M accepts 0^n exactly when n is a perfect square, so $L(M) = \{0^n \mid n = m^2 \text{ for some } m \geq 1\}$. But on input 0^3 the search tries $m = 1$ ($1 \neq 3$), $m = 2$ ($4 \neq 3$), $m = 3$ ($9 \neq 3$), $\dots$ forever: the loop never finds a match and never stops, so M *loops* on 0^3 .



The self-loop “ $m \leftarrow m + 1$ ” has no exit when n is not a square, so the machine loops – it recognizes but does not decide.

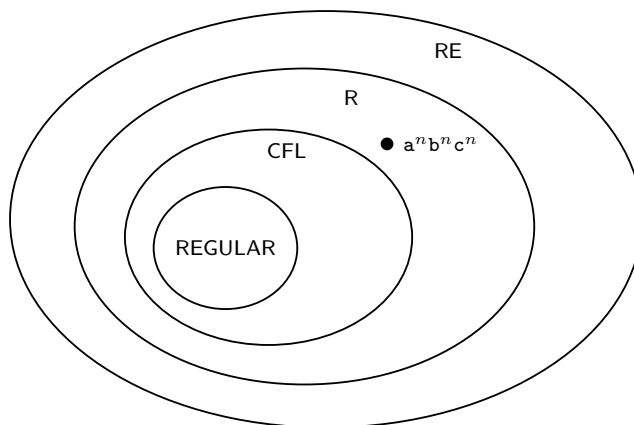
Which idea, and why? Recognizers are allowed to express “no” by never halting; this M does exactly that on non-squares, so it witnesses $\{0^{m^2}\} \in \text{RE}$ but not (via this machine) membership in R .

Check. The language *is* in fact decidable – bound the search by $m \leq n$ and reject once $m^2 > n$ – which shows the defect is in the *machine*, not the language. Distinguishing the two is the heart of the section.

Worked Example 8.5.11 (Reading the class hierarchy off a single language). Place $\mathcal{L} = \{a^n b^n c^n \mid n \geq 0\}$ into the smallest class containing it, and justify each non-membership and membership.

Recognition. We are asked to locate a language in the $\text{REGULAR} \subseteq \text{CFL} \subseteq \text{R} \subseteq \text{RE}$ tower. The trigger is to test the cheap classes first and stop at the first that works.

Solution. The language is *not* regular and *not* context-free (by the standard pumping arguments for $a^n b^n c^n$), but it *is* decidable: a TM checks the shape $a^* b^* c^*$, then crosses off one a , one b , one c per pass, accepting iff all three blocks empty simultaneously. This always halts, so $\mathcal{L} \in \text{R}$.



$a^n b^n c^n$ sits in $\text{R} \setminus \text{CFL}$: decidable but not context-free.

Which idea, and why? Membership in the tower is decided bottom-up; the first class whose tools *succeed* (here the cross-off decider for R , after the pumping lemmas rule out REGULAR and CFL) is the answer, and it is automatically a member of every larger class above it.

Check. $aabbcc \in \mathcal{L}$ ($n = 2$) and the cross-off run empties all three blocks together; $aabbbcc \notin \mathcal{L}$ and the run leaves a leftover b , so the decider rejects. Both halt.

The notion of an algorithm. We now have the vocabulary to say precisely what an *algorithm* is. Intuitively an algorithm is a finite, unambiguous procedure that always terminates with the right answer. The Church–Turing thesis identifies this intuitive notion with a single formal object: an algorithm is exactly a Turing machine that halts on all inputs – a *decider*. Thus “ $\mathcal{L}$ is decidable” and “there is an algorithm deciding membership in $\mathcal{L}$ ” say the same thing, and “ $\mathcal{L}$ is undecidable” means *no* algorithm can solve it, no matter how clever. This is what let Hilbert’s tenth problem (is there an algorithm testing a polynomial for an integer root?) finally receive a definitive *no*: the set of solvable polynomials is recognizable but provably not decidable.

Method (How to show a language is decidable (versus only recognizable)).

1. **Describe the test procedure** at the implementation level: what does the machine scan, mark, and compare? State it as “On input w : ... accept/reject.”
2. **Check coverage.** Make sure the procedure reaches an *accept* on every $w \in \mathcal{L}$ and a *reject* on every $w \notin \mathcal{L}$ – not a loop. A reject must be an explicit halt.
3. **Find a terminating measure.** Exhibit a quantity that strictly decreases each pass and is bounded below (crossed-off symbols, remaining blocks, a search bound $m \leq n$). A bounded, strictly decreasing measure proves the machine halts on *all* inputs – the decider condition.
4. **If you can only guarantee acceptance** (an unbounded search with no stopping bound), you have shown $\mathcal{L} \in \text{RE}$ only. To upgrade to R , either bound the search or show $\overline{\mathcal{L}}$ is also recognizable and invoke Theorem 8.5.6.

Caution. “Recognizes” and “decides” are not synonyms. A machine *recognizes* $\mathcal{L}$ if it accepts exactly the strings of $\mathcal{L}$ – but on strings outside $\mathcal{L}$ it is free to loop. It *decides* $\mathcal{L}$ only if it additionally halts on *every* input. When asked to show a language is *decidable*, exhibiting a machine that merely accepts the right strings is not enough; you must argue it never loops.

Caution. Do not assume the recognizable languages are closed under complement – they are not. The complement trick (swap q_{acc} and q_{rej}) only works for *deciders*, because swapping two halting states does nothing to a computation that never halts. A recognizer that loops on w still loops after the swap, so it fails to accept w in the complement. Closure under complement is a hallmark of R, not of RE.

Caution. A language being recognized by a *badly designed* machine that loops does not make the language undecidable. Decidability is a property of the *language* – it asks whether *some* machine decides it – so a perfect-square test that loops on non-squares still recognizes a decidable language (bound the search to fix it). Never conclude “undecidable” from a single looping machine.

Exercises.

1. State, in one precise sentence each, the difference between a Turing machine that *recognizes* a language and one that *decides* it. Then give the three possible outcomes of running a Turing machine on an input, and say which outcome a decider is forbidden to produce.
2. For each language below, decide whether the most natural machine you can describe is a *decider* or only a *recognizer*, and justify your choice by naming a terminating measure (if a decider) or the unbounded search (if only a recognizer): (a) $\{0^n 1^n \mid n \geq 0\}$; (b) $\{0^n \mid n \text{ is prime}\}$; (c) $\{w\#w \mid w \in \{0, 1\}^*\}$.
3. Prove that the decidable languages are closed under *set difference*: if $\mathcal{L}_1$ and $\mathcal{L}_2$ are decidable, so is $\mathcal{L}_1 \setminus \mathcal{L}_2$. (Describe the combined decider and argue it always halts.)
4. Let $\mathcal{L}$ be decided by a decider M with k states. Describe a decider for $\overline{\mathcal{L}}$ and explain why exactly one small modification of M suffices. Then explain in one sentence why the identical modification fails to produce a recognizer for $\overline{\mathcal{L}}$ when M is only a recognizer.
5. Suppose M_1 recognizes $\mathcal{L}_1$ and M_2 recognizes $\mathcal{L}_2$, and you are told (wrongly) to recognize $\mathcal{L}_1 \cup \mathcal{L}_2$ by “run $M_1(w)$ to completion; if it accepts, accept; otherwise run $M_2(w)$ and copy its verdict.” Exhibit a concrete w , $\mathcal{L}_1$, $\mathcal{L}_2$ for which this procedure gives the wrong answer, and state the one-word fix.
6. Without quoting Theorem 8.5.6, prove directly: if both $\mathcal{L}$ and $\overline{\mathcal{L}}$ are Turing-recognizable, then $\mathcal{L}$ is decidable. Draw the parallel-simulation schematic (two recognizer boxes feeding one “decide” box) and label which box’s acceptance produces **accept** and which produces **reject**.
7. Decide whether each statement is true or false, with a one-line reason: (a) Every decidable language is Turing-recognizable. (b) Every Turing-recognizable language is decidable. (c) If $\mathcal{L}$ is decidable then $\overline{\mathcal{L}}$ is decidable. (d) If $\mathcal{L}$ is Turing-recognizable then $\overline{\mathcal{L}}$ is Turing-recognizable.
8. Give the implementation-level description of a decider for $D = \{a^i b^j \mid j = 2i, i \geq 1\}$, then draw a tape picture of a single accepting input of your choice with the head shown at its start position. Identify the strictly decreasing measure that proves your machine halts on every input.
9. A classmate claims “ $\{0^n \mid n = m^3 \text{ for some } m \geq 0\}$ is undecidable, because the obvious machine – try $m = 0, 1, 2, \dots$ and accept when $m^3 = n$ – loops on inputs that are not perfect cubes.” Find the bug in this reasoning and describe a machine that actually *decides* the language.
10. Place each language in the smallest class of the tower $\text{REGULAR} \subseteq \text{CFL} \subseteq \text{R} \subseteq \text{RE}$ and justify the boundary you cross: (a) $\{w \in \{0, 1\}^* \mid w = w^{\text{R}}\}$ (palindromes); (b) $\{a^n b^n c^n \mid n \geq 1\}$; (c) $\{w \mid w \text{ has an even number of 1s}\}$.
11. Prove that if $\mathcal{L}$ is decidable then so is the language $\mathcal{L}' = \{w \mid w \in \mathcal{L} \text{ and } w^{\text{R}} \in \mathcal{L}\}$. (You may assume w^{R} can be computed on the tape by a halting procedure.)
12. *Find the bug.* A student proposes the following “decider” for $\mathcal{L}_1 \cap \mathcal{L}_2$, where M_1, M_2 are deciders: “On input w , interleave one step of $M_1(w)$ and one step of $M_2(w)$; accept as soon as either one accepts.” The machine halts on every input, yet it does not decide $\mathcal{L}_1 \cap \mathcal{L}_2$. Explain what language it actually accepts, and correct the acceptance condition.

Chapter 9

Decidability

9.1 Decidable Problems about Automata and Grammars

A *decision problem* asks a yes/no question about an input, and we encode it as a language: the set of (suitably encoded) inputs whose answer is “yes.” To say the problem is *decidable* is to say some Turing machine *halts on every input* and answers correctly – it never loops. The one key idea of this section is that many natural questions we can ask about a finite automaton, a regular expression, or a context-free grammar – “does it accept this string?”, “is its language empty?”, “do these two machines agree?” – have such an always-halting decider, because each of these objects is a *finite* description that an algorithm can read, simulate, and exhaust. This sets the stage for the next sections, where the same questions asked about *Turing machines* will turn out to have no decider at all.

Throughout, $\langle B \rangle$ denotes a string encoding of an object B (a DFA, NFA, regular expression, or grammar), and $\langle B, w \rangle$ a string encoding of a pair. A decider may begin by checking that its input has the right form and rejecting if not; we suppress that boilerplate and describe only the interesting work.

Definition 9.1.1 (Decider and decidable language). A Turing machine M is a *decider* if it halts – entering its accept or reject state – on *every* input. A language $\mathcal{L}$ is *decidable* (equivalently, *recursive*) if some decider M recognizes it, i.e. M accepts every $w \in \mathcal{L}$ and rejects every $w \notin \mathcal{L}$.

The crucial word is *halts*. A machine that accepts the strings of $\mathcal{L}$ but *loops* on some strings outside $\mathcal{L}$ only *recognizes* $\mathcal{L}$; to *decide* it, the machine must reach a verdict on every input in finite time. Every decidable language is recognizable, but – as later sections show – not conversely.

Definition 9.1.2 (Acceptance and emptiness languages). We package the section’s problems as languages:

$$\begin{aligned} A_{\text{DFA}} &= \{ \langle B, w \rangle \mid B \text{ is a DFA and } B \text{ accepts } w \}, \\ A_{\text{NFA}} &= \{ \langle B, w \rangle \mid B \text{ is an NFA and } B \text{ accepts } w \}, \\ A_{\text{REG}} &= \{ \langle R, w \rangle \mid R \text{ is a regular expression and } R \text{ generates } w \}, \\ E_{\text{DFA}} &= \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}, \\ EQ_{\text{DFA}} &= \{ \langle A, B \rangle \mid A, B \text{ are DFAs and } L(A) = L(B) \}, \\ A_{\text{CFG}} &= \{ \langle G, w \rangle \mid G \text{ is a CFG and } G \text{ generates } w \}, \\ E_{\text{CFG}} &= \{ \langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset \}. \end{aligned}$$

Deciding the language is exactly deciding the corresponding yes/no problem: e.g. $\langle B, w \rangle \in A_{\text{DFA}}$ iff the DFA B accepts the string w .

The recurring strategy is *simulate and exhaust*. Because the object is finite and the relevant search space can be bounded, a TM can carry out the simulation or search to completion and then announce a verdict. Watch for one design move repeated below: a harder problem is solved by *converting* its input into the input of an easier problem already solved, then calling that easier decider as a *subroutine*.

Acceptance for finite automata and regular expressions

Theorem 9.1.3. A_{DFA} is decidable.

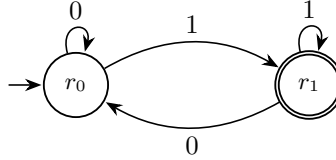
Proof. We build a decider M_1 . On input $\langle B, w \rangle$, where $B = (Q, \Sigma, \delta, q_0, F)$ is a DFA and $w = a_1 a_2 \cdots a_n$, the machine simulates B on w by bookkeeping on its tape: it stores the “current state,” initialized to q_0 , and scans w left to right, replacing the current state q by $\delta(q, a_i)$ as it reads each a_i . After the last symbol it checks whether the current state lies in F ; if so it accepts, otherwise it rejects. The simulation performs exactly n lookups in the (finite, explicitly listed) table δ and then one membership test, so M_1 always halts. It accepts $\langle B, w \rangle$ precisely when B ends in an accept state, i.e. precisely when B accepts w . Hence M_1 decides A_{DFA} . $\square$

The decider is nothing more than the DFA’s run, performed by a TM that reads B off its tape. The reason it *halts* is that a DFA makes exactly one move per input symbol and the input is finite – there is no opportunity to loop.

Worked Example 9.1.4 (Tracing the A_{DFA} decider on a concrete instance). Let B be the DFA over $\Sigma = \{0, 1\}$ that accepts strings ending in 1 (states r_0, r_1 , start r_0 , accept $\{r_1\}$). Decide whether $\langle B, 010 \rangle \in A_{\text{DFA}}$ and whether $\langle B, 011 \rangle \in A_{\text{DFA}}$.

Recognition. Membership in A_{DFA} is decided by *simulating B on w and reading off the final state*; the trigger is “does this specific DFA accept this specific string,” a single finite run.

Solution. Here is B :



The decider keeps a “current state” cell on its tape and updates it once per symbol:

$$010 : r_0 \xrightarrow{0} r_0 \xrightarrow{1} r_1 \xrightarrow{0} r_0 \notin F, \quad 011 : r_0 \xrightarrow{0} r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_1 \in F.$$

So $\langle B, 010 \rangle \notin A_{\text{DFA}}$ (reject) and $\langle B, 011 \rangle \in A_{\text{DFA}}$ (accept).

Which idea, and why? A_{DFA} ’s decider is the direct simulation of Theorem 9.1.3: one table lookup per symbol, then a single accept-set test – finitely many steps, so it always halts.

Check. 010 ends in 0, correctly rejected; 011 ends in 1, correctly accepted. The number of simulation steps equals the string length, 3 in each case.

For NFAs and regular expressions we do not write a fresh simulator. Instead we *convert* the input into a form the previous decider already handles – the subroutine pattern.

Theorem 9.1.5. A_{NFA} is decidable.

Proof. We build a decider M_2 that uses M_1 from Theorem 9.1.3 as a subroutine. On input $\langle B, w \rangle$ with B an NFA, M_2 first converts B to an equivalent DFA C by the subset construction (a finite, mechanical procedure: its states are the reachable subsets of B ’s states, computed with ε -closures). It then runs M_1 on $\langle C, w \rangle$ and copies M_1 ’s answer. The conversion halts (it produces at most $2^{|Q|}$ states), and M_1 halts, so M_2 halts. Since C recognizes the same language as B , C accepts w iff B does; thus M_2 accepts $\langle B, w \rangle$ iff B accepts w , and M_2 decides A_{NFA} . $\square$

Theorem 9.1.6. A_{REG} is decidable.

Proof. A decider M_3 converts the regular expression R into an equivalent NFA A (the standard inductive construction: a constant in one or two states, with union, concatenation, and star glued by ε -arrows). It then runs M_2 from Theorem 9.1.5 on $\langle A, w \rangle$ and copies the answer. Both the conversion and M_2 halt, and A generates exactly the strings R does, so M_3 decides A_{REG} . $\square$

Worked Example 9.1.7 (The decider chain $\text{REX} \rightarrow \text{NFA} \rightarrow \text{DFA}$ as a schematic). Show how A_{REX} , A_{NFA} , and A_{DFA} stack: a single instance $\langle R, w \rangle$ flows through two conversions before a base simulation answers it.

Recognition. Each theorem solved a harder acceptance problem by *converting to the next-easier form and calling its decider*; drawing the call chain makes the “conversion as subroutine” pattern visible.

Solution. The instance is reshaped, never re-solved from scratch; only the bottom box does real simulation.



Which idea, and why? Theorems 9.1.3–9.1.6 form a tower: A_{REX} calls A_{NFA} , which calls A_{DFA} . Each layer only *translates* its input downward, so the whole chain halts because the base case does.

Check. Every arrow strictly simplifies the machine type (regular expression, then NFA, then DFA); the chain has finite depth 3, so no step can loop. The verdict produced by M_1 is passed back up unchanged.

Remark. Theorems 9.1.3–9.1.6 say that for *decidability* purposes it makes no difference whether the input is presented as a DFA, an NFA, or a regular expression: each form can be algorithmically converted to any other, so a decider for one yields a decider for all. We will exploit this freely, choosing whichever representation is most convenient.

Emptiness and equivalence for DFAs

Acceptance asks about *one* string. Emptiness asks about *all* strings at once: does the DFA accept anything? We cannot try every string – there are infinitely many – so we switch from simulation to a finite *graph* search on the state diagram.

Theorem 9.1.8. E_{DFA} is decidable.

Proof. A DFA accepts *some* string iff there is a directed path of transition arrows from the start state to *some* accept state. (A path spelling out a string w is exactly a computation of the DFA on w ending in that accept state.) So emptiness is a reachability question on a finite graph, which a marking algorithm settles. Decider T , on input $\langle A \rangle$:

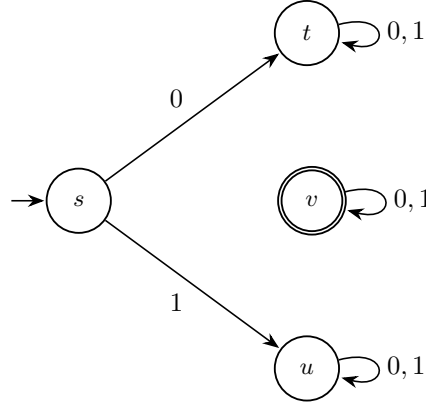
1. mark the start state q_0 ;
2. repeat until a pass marks nothing new: mark any state that has an incoming transition arrow from an already-marked state;
3. if no accept state ever got marked, *accept* (the language is empty); otherwise *reject*.

There are finitely many states, and each round either marks a new state or stops, so at most $|Q|$ rounds occur and T halts. The marked states are exactly those reachable from q_0 ; hence an accept state is marked iff some accepting path exists iff $L(A) \neq \emptyset$. So T accepts $\langle A \rangle$ iff $L(A) = \emptyset$, and T decides E_{DFA} . $\square$

Worked Example 9.1.9 (Running the emptiness marking algorithm). Decide $\langle A \rangle \in E_{\text{DFA}}$ for the DFA A below over $\Sigma = \{0, 1\}$: states s, t, u, v , start s , the only accept state v , with $\delta(s, 0) = t$, $\delta(s, 1) = u$, $\delta(t, 0) = \delta(t, 1) = t$, $\delta(u, 0) = \delta(u, 1) = u$, and v unreachable (it loops to itself on both symbols).

Recognition. Emptiness is *reachability of an accept state from the start*; the trigger is “does the machine accept any string at all,” answered by marking reachable states, not by trying strings.

Solution. Draw A and run the marks. The accept state v sits off to the side with no arrow into it from the reachable part.



Marking: round 0 marks $\{s\}$; round 1 adds $\{t, u\}$ (both have an arrow from s); round 2 adds nothing new (t, u only point at themselves, and nothing points at v). Marked set is $\{s, t, u\}$. The accept state v is *not* marked, so T accepts: $L(A) = \emptyset$.

Which idea, and why? E_{DFA} 's decider is the reachability marking of Theorem 9.1.8: every accept state stays unmarked exactly when no input can drive the machine into acceptance.

Check. From s you can only ever reach t or u , both non-accepting traps; no string leads to v . So indeed the language is empty, matching the accept verdict.

Equivalence – do two DFAs recognize the *same* language? – is reduced to emptiness by a clever choice of combined machine.

Theorem 9.1.10. EQ_{DFA} is decidable.

Proof. Two languages are equal iff their *symmetric difference* is empty. For DFAs A and B , the symmetric difference

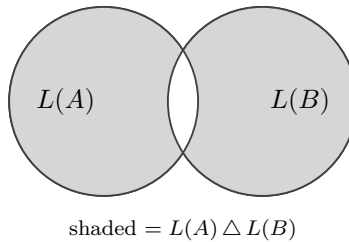
$$L(A) \triangle L(B) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

contains exactly the strings on which the two machines *disagree*; it is empty iff $L(A) = L(B)$. The regular languages are closed under complement (swap accept and non-accept states) and under union and intersection (the product construction running both machines in lockstep), and these constructions are mechanical, so we can build a DFA C with $L(C) = L(A) \triangle L(B)$. Decider F , on input $\langle A, B \rangle$: construct C , then run the decider T from Theorem 9.1.8 on $\langle C \rangle$; if T accepts (so $L(C) = \emptyset$), accept; otherwise reject. Both the construction and T halt, so F halts, and F accepts iff $L(A) \triangle L(B) = \emptyset$ iff $L(A) = L(B)$. Thus F decides EQ_{DFA} . $\square$

Worked Example 9.1.11 (Why symmetric difference, and what it measures). Two DFAs A, B are claimed equivalent. Explain, using a Venn picture, why testing emptiness of $L(A) \triangle L(B)$ settles the claim, and identify what a string in the shaded region would prove.

Recognition. Equivalence ($L(A) = L(B)$) is reduced to emptiness via the symmetric difference; the trigger is “two sets are equal iff neither contains a point the other misses.”

Solution. The symmetric difference is the two crescents – in one language but not the other. It is empty exactly when the two circles coincide.



Which idea, and why? EQ_{DFA} reduces to E_{DFA} : build a DFA for the shaded set using closure under intersection, union and complement, then ask the emptiness decider whether the shaded set is empty.

Check. A string x in the left crescent lies in $L(A)$ but not $L(B)$ – a witness that the machines *disagree*, hence are not equivalent. No such witness exists iff the crescents are empty iff the circles coincide.

Acceptance and emptiness for context-free grammars

Grammars are not machines, but their derivations form a search space we can bound. The danger is loops: trying derivations blindly can run forever. The fix is to put the grammar in a normal form that caps derivation length.

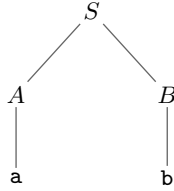
Theorem 9.1.12. A_{CFG} is decidable.

Proof. The naive idea – enumerate all derivations of G and watch for one yielding w – can loop forever when G does not generate w , since infinitely many derivations may exist; that gives only a recognizer. To get a decider we *bound the search*. Convert G to an equivalent grammar in Chomsky normal form (every rule is $A \rightarrow BC$ or $A \rightarrow a$, plus possibly $S \rightarrow \varepsilon$). In such a grammar, any derivation of a string of length $n \geq 1$ uses exactly $2n - 1$ rule applications (each internal node of the parse tree is a binary branch and each leaf comes from a terminal rule). So decider S , on input $\langle G, w \rangle$ with $n = |w|$: convert G to Chomsky normal form; if $n \geq 1$, list all derivations of length $2n - 1$ (if $n = 0$, list all one-step derivations, to catch $S \rightarrow \varepsilon$); accept if any yields w , else reject. There are only finitely many derivations of that fixed length, so the search terminates and S halts. It accepts iff some valid derivation produces w , i.e. iff G generates w . Thus S decides A_{CFG} . $\square$

Worked Example 9.1.13 (Bounding the parse search by tree shape). Let G (in Chomsky normal form) have rules $S \rightarrow AB \mid BC$, $A \rightarrow a$, $B \rightarrow b$, $C \rightarrow c$. Decide $\langle G, ab \rangle \in A_{CFG}$, and confirm the $2n - 1$ derivation-length bound.

Recognition. A_{CFG} is decided by *searching only derivations of length $2n - 1$ in Chomsky normal form*; the trigger is “does this grammar generate this string,” made finite by the length cap.

Solution. Here $n = |ab| = 2$, so we examine derivations of length $2n - 1 = 3$. A derivation of ab has the binary parse tree



which uses the three rules $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$ – exactly $3 = 2n - 1$ applications. This derivation yields ab , so S accepts: $\langle G, ab \rangle \in A_{CFG}$.

Which idea, and why? Chomsky normal form forces a binary parse tree, so a length- n string has exactly $n - 1$ internal $A \rightarrow BC$ steps plus n terminal $A \rightarrow a$ steps, i.e. $2n - 1$ rules – the bound that makes the search finite.

Check. The alternative start rule $S \rightarrow BC$ would force the first terminal to be b , generating strings like bc ; it cannot yield ab . With only finitely many length-3 derivations to inspect, the decider halts after a bounded search.

Theorem 9.1.14. E_{CFG} is decidable.

Proof. We cannot try all strings w (infinitely many). Instead we ask which variables are *generating* – able to derive *some* string of terminals – and check whether the start variable is among them. This is a marking algorithm on the grammar. Decider R , on input $\langle G \rangle$:

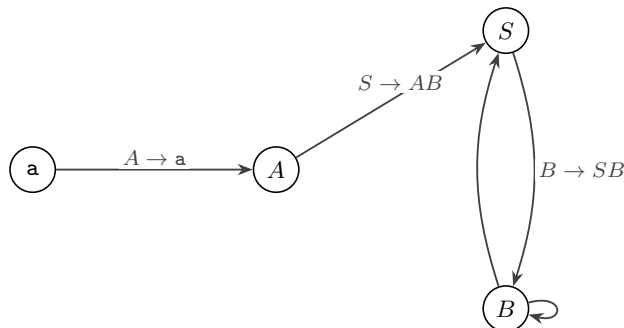
1. mark every terminal symbol of G ;
2. repeat until a pass marks nothing new: mark a variable A whenever G has a rule $A \rightarrow U_1 U_2 \cdots U_k$ with every U_i already marked (the rule $A \rightarrow \varepsilon$ marks A immediately, as the right side is empty);
3. if the start variable is *not* marked, accept (the language is empty); otherwise reject.

Each round marks at least one new symbol or halts, and there are finitely many symbols, so R halts. By induction on the number of rounds, a variable is marked iff it can derive a terminal string; hence the start variable is marked iff $L(G) \neq \emptyset$. So R accepts iff $L(G) = \emptyset$, deciding E_{CFG} . $\square$

Worked Example 9.1.15 (Marking generating variables). Decide $\langle G \rangle \in E_{CFG}$ for G with rules $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow SB$. (There is no terminal-only rule for B .)

Recognition. E_{CFG} is decided by *bottom-up marking of generating variables*; the trigger is “can the start variable ever reach a terminal string,” a fixpoint computation, not a string search.

Solution. Work upward from terminals. The dependency picture shows what each variable needs:



Marking: mark terminal a ; then A (rule $A \rightarrow a$, right side marked). Now S needs both A and B marked, but B has *only* the rule $B \rightarrow SB$, which needs B already marked – a circular dependency that never bottoms out in terminals. So B is never marked, hence S is never marked. The start variable S is unmarked, so R accepts: $L(G) = \emptyset$.

Which idea, and why? The generating-variable marking of Theorem 9.1.14: a variable is marked only if some rule lets it derive marked (terminal-producing) symbols; B ’s sole rule is self-referential, so it produces nothing.

Check. Any attempted derivation from S must expand B , and every expansion of B reintroduces a B , so no terminal string is ever reached. The language is empty, matching the accept verdict.

Remark. E_{CFG} is decidable, yet *equivalence* of context-free grammars, EQ_{CFG} , is *not*. The DFA trick fails: context-free languages are not closed under complement or intersection, so there is no grammar for the symmetric difference to feed to the emptiness decider. The closure properties that powered Theorem 9.1.10 simply are not available here – a first hint that the grammar world is more delicate than the automaton world.

Method (How to show a problem about automata or grammars is decidable).

1. **Encode the problem as a language** of the form $\{\langle \text{object} \rangle \mid \text{property holds}\}$ (or $\langle \text{object}, w \rangle$). Deciding the language is deciding the problem.
2. **Pick the algorithmic shape.** “Does it accept this one string?” is a *simulation* (run the machine / bound the parse search). “Is the language empty / infinite / equal?” is a *reachability or marking* search on a finite structure, or a *reduction* to one already solved.
3. **Reuse a decider as a subroutine.** Convert your input to a form some earlier decider handles (NFA $\rightarrow$ DFA, regex $\rightarrow$ NFA, “equal?” $\rightarrow$ “empty?” via symmetric difference, “empty?” $\rightarrow$ reachability), then call that decider and relay its answer.
4. **Argue halting.** Bound the work: finitely many states/variables to mark, a fixed derivation length $2n - 1$, or finitely many conversion steps. A decider must halt on *every* input – name the finite quantity that forces termination.
5. **Argue correctness.** Show the decider accepts exactly the yes-instances: the simulation’s verdict matches acceptance, or the marking reaches a symbol iff the property holds.

Caution. Do not confuse *deciding* with merely *recognizing*. For A_{CFG} , the naive “enumerate every derivation and watch for w ” machine is a recognizer, not a decider: when G does *not* generate w , it can search forever and never reject. The Chomsky-normal-form length bound $2n - 1$ is precisely what converts the search from unbounded to finite, turning the recognizer into a decider. Whenever an algorithm “searches until it finds,” ask what happens when there is nothing to find.

Caution. For E_{DFA} and E_{CFG} , the decider *accepts* when the language is *empty* and rejects otherwise – the polarity is inverted relative to acceptance problems. It is a common slip to wire the final step backwards. Anchor it to the definition: $\langle A \rangle \in E_{\text{DFA}}$ means $L(A) = \emptyset$, so the decider must *accept* exactly the machines whose accept states are all unreachable.

Caution. The reduction in Theorem 9.1.10 relies on the regular languages being closed under complement, union, and intersection. Do not transplant it to context-free grammars: CFLs are not closed under complement or intersection, and EQ_{CFG} is in fact undecidable. “ E_X is decidable” does *not* imply “ EQ_X is decidable” – the bridge between them is the closure properties, and that bridge exists only for the regular world.

Exercises.

1. Describe, at the implementation level, a decider for $A_{\text{DFA}} = \{\langle D, w \rangle \mid D \text{ is a DFA and } w \in L(D)\}$. Explain why the simulation is guaranteed to halt, and state its running idea in one sentence.
2. Give a decider for E_{TM} -style emptiness on DFAs, $E_{\text{DFA}} = \{\langle D \rangle \mid L(D) = \emptyset\}$, by reducing the question to reachability in a graph. Which graph, which two vertices, and which standard algorithm? (This is the place graph search from the foundations pays off.)
3. Using the previous two ideas, build a decider for $EQ_{\text{DFA}} = \{\langle D_1, D_2 \rangle \mid L(D_1) = L(D_2)\}$. (Hint: form a DFA for the symmetric difference $L(D_1) \triangle L(D_2)$ and test it for emptiness.)
4. Show that

$$A_{\text{NFA}} = \{\langle N, w \rangle \mid N \text{ is an NFA accepting } w\} \quad \text{and} \\ A_{\text{REG}} = \{\langle R, w \rangle \mid R \text{ is a regular expression and } w \in L(R)\}$$

are decidable, by reduction to A_{DFA} . What conversion does each reduction perform, and why does it terminate?

5. Describe a decider for $A_{\text{CFG}} = \{\langle G, w \rangle \mid G \text{ is a CFG generating } w\}$. Why is it not enough to “try all derivations”? State the bound on derivation length once G is in Chomsky normal form, for $|w| = n \geq 1$.
6. Give a decider for $E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$ by marking the variables that can generate a terminal string, bottom-up. Run your procedure on $S \rightarrow AB \mid a$, $A \rightarrow BB$, $B \rightarrow b$ and report whether $L(G)$ is empty.
7. Prove that the class of decidable languages is closed under union, intersection, and complement, by describing the deciders explicitly. Where does “the machine always halts” get used?
8. Prove the class of decidable languages is closed under concatenation and Kleene star. (Care: a recognizer would “guess” a split, but a decider must not loop. Explain how to make the search finite.)
9. Devise a decider that, given a DFA D , reports whether $L(D)$ is *finite* or *infinite*. (Hint: an accepted string of length $\geq |Q|$ forces a reusable cycle. Which cycle, reachable from where, leading to what?)
10. *Spot the flaw.* A student claims E_{CFG} can be decided “by checking whether the start variable appears on the left of any rule.” Give a grammar on which this test gives the wrong answer, and explain what it misses.
11. *Spot the flaw.* Another student decides $EQ_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid L(G_1) = L(G_2)\}$ “the same way as EQ_{DFA} , via symmetric difference.” Identify the closure properties that fail for context-free languages and break this plan.

9.2 The Acceptance Problem and Recognizability

In the previous section every problem we attacked turned out *decidable*: each machine simulated a finite automaton or a grammar that was guaranteed to finish, so the deciding Turing machine always halted. The acceptance problem for Turing machines themselves is different, and the difference is the whole point of this chapter. Let $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$. The one key idea of this section is the gap between two ways a machine can “solve” a problem: a *recognizer* must accept exactly the strings in the language but may run forever on the rest, while a *decider* must halt on *every* input. We build the universal Turing machine and use it to show A_{TM} is recognizable; the question of whether A_{TM} is also decidable is deferred to Section 9.4, where the answer is no.

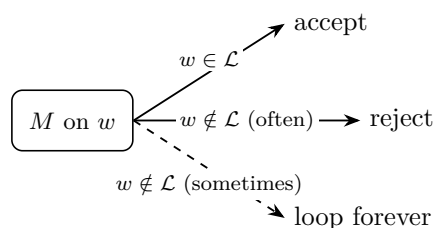
Recognizers versus deciders

Definition 9.2.1 (Recognizing and deciding). Let M be a Turing machine. On input w , M has three possible fates: it may halt and *accept*, halt and *reject*, or *loop* (run forever without halting). The *language recognized by M* is $L(M) = \{w \mid M \text{ accepts } w\}$. We say:

- M *recognizes* a language $\mathcal{L}$ if $L(M) = \mathcal{L}$ (so M accepts every string of $\mathcal{L}$ and accepts no other string, but may loop on non-members);
- M *decides* a language $\mathcal{L}$ if M recognizes $\mathcal{L}$ and halts on every input (a machine that always halts is a *decider*, and it then *rejects* every non-member rather than looping).

A language is *Turing-recognizable* (or *recognizable*) if some TM recognizes it, and *decidable* if some TM decides it.

The branching of fates is worth drawing once, because the missing third branch is exactly what separates a recognizer from a decider.



A *decider* is a machine for which the dashed branch never happens: every input lands on accept or reject. A *recognizer* is allowed the dashed branch, but only for strings *outside* the language – it must never loop on, or wrongly reject, a member. Deciding is strictly harder, so every decidable language is recognizable; the converse is the central question we resolve negatively later.

Remark. A recognizer’s silence is ambiguous. If you run a recognizer M on w and it has not halted after a million steps, you cannot conclude $w \notin L(M)$: maybe M accepts at step 1 000 001, maybe it never halts. A decider removes this ambiguity by promising to answer in finite time on *every* input. “Decidable” is the formal stand-in for “solvable by an algorithm,” because an algorithm is expected to terminate.

The universal Turing machine

To recognize A_{TM} we need a single machine that, handed the description of *any* TM M together with an input w , faithfully carries out M ’s computation on w . Such a machine exists; it is the *universal* Turing machine, the theoretical ancestor of the stored-program computer.

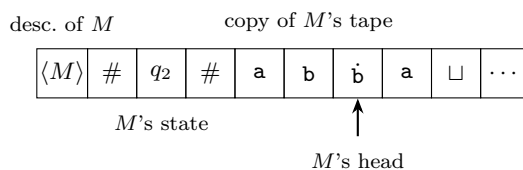
Theorem 9.2.2 (The universal machine). *There is a Turing machine U that, on input $\langle M, w \rangle$ where M is a TM and w a string, simulates M on w : U accepts $\langle M, w \rangle$ if M accepts w , rejects $\langle M, w \rangle$ if M rejects w , and loops if M loops on w . Consequently $L(U) = A_{\text{TM}}$.*

Proof. The input $\langle M, w \rangle$ is a finite string encoding M ’s 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ together with the input w . The machine U keeps three blocks of information on its tape: a verbatim copy of M ’s transition table δ , the current state of M , and the current contents of M ’s tape with a marker for M ’s head position. U first checks that the input is a well-formed encoding of a TM and a string; if not, it rejects. It then writes M ’s start state q_0 and lays out w as M ’s initial tape. To simulate one step of M , U reads the marked symbol under M ’s head and M ’s current state, scans the stored copy of δ for the matching rule, and updates the recorded state, the marked tape symbol, and the head marker accordingly. U repeats this until M enters its accept state (then U accepts) or its reject state (then U rejects). If M never halts on w , U simulates forever and never halts either. Thus U accepts $\langle M, w \rangle$ exactly when M accepts w , so $L(U) = A_{\text{TM}}$. $\square$

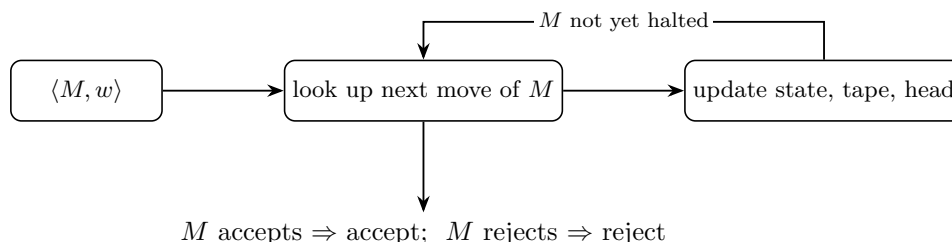
Corollary 9.2.3. A_{TM} is Turing-recognizable.

Proof. The universal machine U of Theorem 9.2.2 satisfies $L(U) = A_{\text{TM}}$, so U is a recognizer for A_{TM} . $\square$

Two pictures fix the mechanics. The first shows what sits on U 's tape: the description of the simulated machine, a copy of its tape, and the head marker. The second shows U as a black box that loops one step of M at a time.



The dotted symbol $\dot{b}$ marks the cell currently under M 's simulated head; q_2 records that M is presently in state q_2 . One simulated step of M rewrites the dotted cell, moves the dot one square, and updates the stored state – all by consulting the copy of M 's transition table to the left.



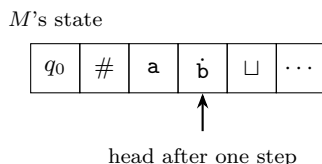
Worked Example 9.2.4 (Running U on a concrete $\langle M, w \rangle$). Let M be a TM that, on input over $\{a, b\}$, sweeps right to the first blank, accepting if the last symbol read was b and rejecting otherwise. Trace what U does on $\langle M, ab \rangle$.

Recognition. We are asked to simulate the universal machine. U never reasons about what M “means”; it mechanically replays M 's steps from the stored table. The trigger is that the input has the two-component form $\langle M, w \rangle$ – description plus string – which is exactly U 's input format.

Solution. U writes M 's start configuration: tape ab , head on the first cell, state q_0 . It then consults M 's table once per step.

$$q_0 \underline{ab} \vdash a \underline{q_0} b \vdash ab \underline{q_0} \sqcup \vdash a \underline{q_1} b \vdash \text{accept (last symbol was } b).$$

At each $\vdash$, U reads the dotted cell and the recorded state, finds the matching rule in the stored copy of δ , and rewrites its working area. When the simulated M enters its accept state, U accepts $\langle M, ab \rangle$.



Which idea, and why? $\langle M, w \rangle \in A_{\text{TM}}$ iff M accepts w , and U decides “accept” purely by finishing the simulation; we never analyse M – we run it.

Check. On $\langle M, aa \rangle$ the same simulation ends with the last symbol a , so M rejects and U rejects. This M always halts, so here U always halts too – the looping danger only appears when the simulated machine itself can loop.

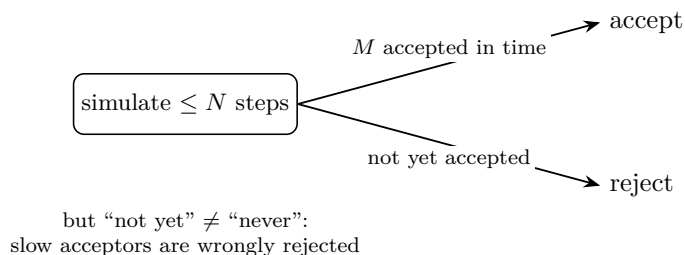
Why the obvious decider fails

The universal machine recognizes A_{TM} , but it is *not* a decider: if M loops on w , then U loops on $\langle M, w \rangle$, never reaching accept or reject. One might hope to repair this – add a watchdog that gives up after “enough” steps and rejects. The next example shows why no fixed bound, and indeed no step-counting trick at all, turns U into a decider.

Worked Example 9.2.5 (The watchdog that rejects after N steps is wrong). Fix a bound N and define U_N : simulate M on w for at most N steps; if M accepts within N steps, accept; otherwise reject. Show U_N does not decide A_{TM} .

Recognition. This is a bounded-simulation idea. The trigger to distrust it: deciding A_{TM} would require detecting non-halting, and a *fixed* cutoff cannot tell “will never halt” from “has not halted yet.”

Solution. U_N always halts, so it is a decider for *some* language – just not A_{TM} . Take a machine M^* and input w^* such that M^* accepts w^* but only after $N + 1$ steps (any slow-but-accepting computation works; pad with N harmless moves). Then $\langle M^*, w^* \rangle \in A_{\text{TM}}$, yet U_N rejects it because acceptance came one step too late. So U_N ’s language is not A_{TM} . The schematic shows the fatal mislabelling.



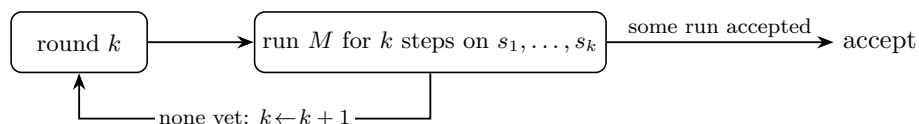
Which idea, and why? A decider must be correct on *every* input; a step bound N confuses “halts after step N ” with “never halts,” so it errs on every machine that accepts just past the cutoff. No single N works for all inputs.

Check. Worse, letting N grow with the input does not help either: to know how long to wait you would already need to know whether M halts, which is the very question. This foreshadows the undecidability proof of Section 9.4.

Worked Example 9.2.6 (Recognizable but the recognizer must be allowed to loop). Consider $\mathcal{L} = \{\langle M \rangle \mid M \text{ is a TM that accepts at least one string}\}$. Sketch a recognizer and explain why no obvious modification makes it a decider.

Recognition. “There exists a string that M accepts” is an *existential* condition over an infinite search space. The trigger is the unbounded “ $\exists$ ”: we can confirm a witness but never finish ruling all witnesses out.

Solution. Enumerate all strings $s_1, s_2, s_3, \dots$ and run M on them. The naive version – run M on s_1 to completion, then s_2 , and so on – can get stuck if M loops on some early s_i . Fix this with *dovetailing*: in round k , run M for k steps on each of $s_1, \dots, s_k$. If any run accepts, accept. This visits every (string, step-count) pair, so if M accepts *any* string the search finds it and halts.

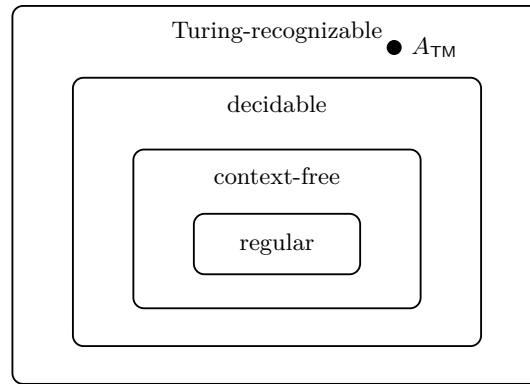


Which idea, and why? An “ $\exists$ witness” property is recognizable by a dovetailed search: finding a witness halts the machine, so members are accepted; but if no witness exists the search runs forever, which is allowed for a recognizer and fatal for a decider.

Check. If M accepts some s_i in t steps, dovetailing reaches round $k = \max(i, t)$ and accepts – so every member is accepted. If M accepts nothing, the search never halts: the machine recognizes $\mathcal{L}$ but does not decide it, and in fact $\mathcal{L}$ is undecidable.

Where recognizable and decidable sit among the classes

The classes we have met nest. Every regular language is context-free, every context-free language is decidable (Section 9.1), and every decidable language is recognizable (a decider is in particular a recognizer). All three inclusions are strict, and A_{TM} is the witness that the outermost one is strict: it is recognizable (Corollary 9.2.3) but, as we will prove, not decidable.



The dot for A_{TM} sits inside “recognizable” but outside “decidable.” Section 9.3 will show the outermost box does not fill the whole universe of languages – there are *uncountably* many languages but only countably many machines, so some languages are not even recognizable, and Section 9.5 will name one.

Method (Telling “recognizable” from “decidable” for a language $\mathcal{L}$).

1. **Write $\mathcal{L}$ as a membership test.** Phrase “ $x \in \mathcal{L}$ ” as a condition you could check by running machines or searching strings.
2. **For recognizability, accept on success and never falsely accept.** If a single finite computation or a found witness certifies membership, build a searcher that accepts the moment it succeeds. Use *dovetailing* when several possibly-looping simulations must be advanced together. Looping on non-members is permitted.
3. **For decidability, you must also be able to reject in finite time.** Ask: is there a computable bound after which “not yet” safely becomes “never”? A finite search space, a step bound that provably suffices, or a structural shortcut (as for $A_{\text{DFA}}, E_{\text{DFA}}$ -style problems in Section 9.1) gives decidability; an unbounded “ $\exists$ over all strings/all step counts” with no such bound usually does not.
4. **Suspect undecidability when only non-halting separates the answers.** If the only obstacle to a decider is that a simulation might loop and you cannot detect looping in advance, that is the signature of A_{TM} -style undecidability (Section 9.4).

Caution. Do not confuse *recognizes* with *decides*. A machine recognizes $\mathcal{L}$ as long as it accepts exactly the members; what it does on non-members (reject *or* loop) is unconstrained. A decider additionally must halt – and therefore reject – on every non-member. Saying “ U solves the acceptance problem” is wrong: U recognizes A_{TM} but does not decide it, because it inherits M ’s looping.

Caution. A bounded simulation “run M for N steps, then reject” always halts, so it is tempting to call it a decider for A_{TM} . It is a decider, but for the *wrong* language: the set of $\langle M, w \rangle$ on which M accepts *within* N steps. That language is decidable and genuinely smaller than A_{TM} . The lesson: halting on every input is necessary for a decider but not sufficient – it must also be *correct* on every input.

Caution. When recognizing an “ $\exists$ string with property P ” language, do not run M to completion on s_1 , then s_2 , and so on. If M loops on s_1 you never reach s_2 , and you might miss a string M *does* accept. Always dovetail: advance every simulation a little each round so no single looping run can starve the others.

Exercises.

1. Classify each statement as true or false and justify in one sentence. (a) Every decidable language is Turing-recognizable. (b) Every Turing-recognizable language is decidable. (c) A machine that halts on every input but accepts the wrong set of strings still decides *some* language. (d) If M loops on w then $\langle M, w \rangle \notin A_{\text{TM}}$.
2. On input $\langle M, w \rangle$ the universal machine U has three possible behaviours. State exactly which behaviour of U corresponds to each of: $\langle M, w \rangle \in A_{\text{TM}}$; M halts and rejects w ; M loops on w . Which of the three makes U fail to be a decider?

3. Let M be a one-tape TM over $\{0, 1\}$ that scans right to the first blank and accepts iff it read an even number of 1s. Draw a tape picture (in the house style, with a dotted head cell and a recorded state) showing the configuration of U 's simulation of M on input 101 *after two simulated steps*, and state whether U ultimately accepts $\langle M, 101 \rangle$.
4. Give the precise reason the bounded simulator $U_N = \text{"simulate } M \text{ on } w \text{ for } N \text{ steps; accept if } M \text{ accepted, else reject"}$ is a decider but not a decider *for* A_{TM} . Describe in set notation the language U_N actually decides.
5. A classmate proposes this decider for A_{TM} : "run U on $\langle M, w \rangle$; if it has not halted after $\langle M, w \rangle$'s length squared steps, reject." Find a concrete family of inputs on which this machine gives the wrong answer, and explain why no polynomial (or any fixed computable) step budget rescues the idea.
6. Let $\mathcal{L} = \{\langle M \rangle \mid M \text{ accepts the string } 00\}$. Show $\mathcal{L}$ is Turing-recognizable by describing a recognizer built from U . Does your recognizer halt on every input? Identify exactly the inputs on which it may loop.
7. Explain why the following "recognizer" for $\{\langle M \rangle \mid M \text{ accepts some string}\}$ is buggy: "for $i = 1, 2, 3, \dots$, run M to completion on s_i ; accept if it accepts." Give a single M on which it fails to accept even though M accepts some string, and repair it.
8. Define $\mathcal{L} = \{\langle M, w \rangle \mid M \text{ halts on } w \text{ within } 2|w| \text{ steps}\}$. Argue that $\mathcal{L}$ is decidable, and contrast this with A_{TM} : which feature of $\mathcal{L}$'s definition supplies the halting guarantee that A_{TM} lacks?
9. Suppose $\mathcal{L}_1$ and $\mathcal{L}_2$ are both Turing-recognizable, recognized by M_1 and M_2 . Describe a machine that recognizes $\mathcal{L}_1 \cup \mathcal{L}_2$. Why must you run M_1 and M_2 by dovetailing rather than " M_1 first, then M_2 "?
10. Suppose $\mathcal{L}_1$ and $\mathcal{L}_2$ are Turing-recognizable, recognized by M_1 and M_2 . Show $\mathcal{L}_1 \cap \mathcal{L}_2$ is recognizable. Here you may run M_1 to completion before M_2 ; explain why sequential running is safe for intersection but not for union.
11. True or false, with justification: if a language $\mathcal{L}$ has a recognizer that halts on every input *of even length* (and may loop on odd-length inputs), then $\mathcal{L}$ is decidable. (Hint: think about what a decider must do on the odd-length non-members.)
12. Let M be a TM and consider the recognizer R that on input w runs M on w and copies its verdict. Prove that if M is a decider then $L(R)$ is decidable, but if M merely recognizes its language then R need not be a decider. Which exact property of M transfers to R , and which does not?

9.3 Diagonalization and the Uncountability of Languages

The previous section gave a recognizer for A_{TM} but no decider, and hinted that none can exist. Before we prove that, we need a tool for comparing the sizes of infinite collections, because the whole obstruction is a counting mismatch: there are far more languages than there are machines. The one key idea here is Cantor's notion that two sets have the same size when their elements can be paired off one-for-one, together with the *diagonal argument* that shows some pairings are impossible. From this we get, with almost no extra work, a purely numerical proof that *some languages are recognized by no Turing machine at all* – there simply are not enough machines to go around. This section builds that machinery; the next two cash it in on A_{TM} .

Sizing sets by pairing

Definition 9.3.1 (One-to-one, onto, correspondence). Let $f: A \rightarrow B$ be a function. We call f *one-to-one* (injective) if distinct inputs get distinct outputs: $a \neq a' \Rightarrow f(a) \neq f(a')$; *onto* (surjective) if every $b \in B$ is hit, i.e. for each $b \in B$ there is some $a \in A$ with $f(a) = b$; and a *correspondence* (bijection) if it is both one-to-one and onto. A correspondence pairs each element of A with exactly one element of B and vice versa.

Definition 9.3.2 (Same size, countable, uncountable). Sets A and B have the *same size* if some correspondence $f: A \rightarrow B$ exists. A set A is *countable* if it is finite or has the same size as $\mathbb{N} = \{1, 2, 3, \dots\}$; otherwise A is *uncountable*. A correspondence with $\mathbb{N}$ is the same thing as an *enumeration* $a_1, a_2, a_3, \dots$ that lists every

element of A exactly once. (In this section $\mathbb{N}$ starts at 1, to match the enumeration indices; elsewhere in the book $\mathbb{N} = \{0, 1, 2, \dots\}$.)

The slogan “countable = listable” is worth fixing in mind: A is countable exactly when you can arrange all of its elements in a single (possibly endless) list $a_1, a_2, \dots$ with nothing left out and nothing repeated. Finiteness is not required, and – as the next examples show – a proper subset or a seemingly “bigger” set can still be the same size as $\mathbb{N}$.

Worked Example 9.3.3 (The even numbers are the same size as all of $\mathbb{N}$). Let $E = \{2, 4, 6, \dots\}$ be the positive even numbers. Show $|E| = |\mathbb{N}|$, even though $E \subsetneq \mathbb{N}$.

Recognition. “Same size” is asked for, so by Definition 9.3.2 we must *exhibit a correspondence* $\mathbb{N} \rightarrow E$, not argue from inclusion. The trigger is the word *size*: ignore the fact that E omits the odd numbers and just build a pairing.

Solution. Take $f(n) = 2n$. It is one-to-one ($2n = 2n' \Rightarrow n = n'$) and onto (any even $2k$ is $f(k)$), so it is a correspondence. The pairing, displayed as a two-row list:

n	1	2	3	4	5	...
$f(n) = 2n$	2	4	6	8	10	...

Which idea, and why? Size is defined by pairing, not by the subset relation; producing an explicit bijection $n \mapsto 2n$ settles it and the “ E is missing half of $\mathbb{N}$ ” intuition is simply not how infinite size works.

Check. Every even number appears once in the bottom row ($2k$ sits under k), and no row entry repeats, so the list enumerates E with no gaps or duplicates.

Worked Example 9.3.4 (The positive rationals are countable). Let $\mathbb{Q}^+ = \{p/q \mid p, q \in \mathbb{N}\}$. Show $\mathbb{Q}^+$ is countable.

Recognition. We must list $\mathbb{Q}^+$ as $r_1, r_2, \dots$ hitting every fraction once. The naive “list row by row” fails because each row $\frac{i}{1}, \frac{i}{2}, \dots$ is already infinite, so we never reach row 2. The trigger “a doubly-indexed family $\frac{i}{j}$ ” calls for the *diagonal sweep*: walk the finite anti-diagonals $i + j = \text{const}$ in turn.

Solution. Place $\frac{i}{j}$ in row i , column j of an infinite grid, then read it along the anti-diagonals (arrows), skipping any fraction already listed in lowest terms (e.g. $\frac{2}{2}$ repeats $\frac{1}{1}$). Each anti-diagonal is finite, so every entry is reached after finitely many steps.

$\frac{1}{1}$	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	...
$\frac{2}{1}$	$\frac{2}{2}$	$\frac{2}{3}$	$\frac{2}{4}$	...
$\frac{3}{1}$	$\frac{3}{2}$	$\frac{3}{3}$	$\frac{3}{4}$	...
$\frac{4}{1}$	$\frac{4}{2}$	$\frac{4}{3}$	$\frac{4}{4}$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	

The list begins $\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{3}{1}, \frac{1}{3}, \frac{1}{4}, \dots$ (with $\frac{2}{2}$ skipped as a repeat of $\frac{1}{1}$). Pairing the k th list entry with $k \in \mathbb{N}$ gives the required correspondence.

Which idea, and why? A set indexed by *two* natural-number coordinates is countable because the grid splits into finite anti-diagonals; sweeping them one by one threads all the entries onto a single list, which is exactly an enumeration.

Check. Any specific fraction $\frac{p}{q}$ lives on anti-diagonal $p + q$, the $(p + q - 1)$ th diagonal, which is reached after finitely many earlier diagonals – so $\frac{p}{q}$ has a finite position in the list.

An uncountable set: Cantor’s diagonal argument

Not every infinite set is countable. The proof technique – *diagonalization* – is the engine behind every undecidability result in this chapter, so we present it carefully on its classic target, the real numbers.

Theorem 9.3.5 (Cantor). *The set $\mathbb{R}$ of real numbers is uncountable.*

Proof. It suffices to show the interval $(0, 1)$ is uncountable. Suppose, for contradiction, that some correspondence $f: \mathbb{N} \rightarrow (0, 1)$ exists, so every real in $(0, 1)$ appears as some $f(n)$. Write each $f(n)$ by its infinite decimal expansion $f(n) = 0.d_{n1}d_{n2}d_{n3}\dots$, choosing the expansion that does *not* end in all 9’s so each number has one canonical row. Lay the list out as a table and look down the diagonal digits $d_{11}, d_{22}, d_{33}, \dots$

n	$f(n) = 0.d_{n1}d_{n2}d_{n3}d_{n4}\dots$
1	0. 4 159...
2	0.3 3 33...
3	0.27 1 8...
4	0.905 0 ...
$\vdots$	$\vdots$

Define a new number $x = 0.e_1e_2e_3\dots$ by changing every boxed diagonal digit: set

$$e_k = \begin{cases} 5 & \text{if } d_{kk} \neq 5, \\ 4 & \text{if } d_{kk} = 5. \end{cases}$$

Avoiding the digits 0 and 9 guarantees x has a unique decimal form (no “.4999... = .5000...” ambiguity) and that $x \in (0, 1)$. By construction $e_k \neq d_{kk}$ for every k , so x differs from $f(k)$ in its k th digit, hence $x \neq f(k)$ for *all* k . Thus $x \in (0, 1)$ is missed by f , contradicting that f is onto. No correspondence $\mathbb{N} \rightarrow (0, 1)$ exists, so $(0, 1)$ – and therefore $\mathbb{R}$ – is uncountable. $\square$

Worked Example 9.3.6 (Running the diagonal flip on a concrete table). Using the four rows tabulated in the proof, write out the first four digits of the diagonal number x .

Recognition. This is a direct application of the rule “read the diagonal, then flip each digit.” The trigger is that we are handed an explicit table, so we just harvest $d_{11}, d_{22}, d_{33}, d_{44}$ and apply the case split.

Solution. The boxed diagonal digits are $d_{11} = 4$, $d_{22} = 3$, $d_{33} = 1$, $d_{44} = 0$. None equals 5, so each becomes 5:

k	1	2	3	4
d_{kk}	4	3	1	0
e_k	5	5	5	5

so $x = 0.5555\dots$ in these first four places. It differs from $f(1)$ in digit 1 (since $5 \neq 4$), from $f(2)$ in digit 2, and so on – the defining property of the diagonal number.

Which idea, and why? The diagonal number is built digit-by-digit to disagree with row k at position k ; that single guaranteed mismatch per row is what forces x off the entire list.

Check. If $f(2)$ had instead been $0.x5xx\dots$ (diagonal digit 5), the rule would set $e_2 = 4$, again $\neq 5$ – the flip is engineered so a mismatch is unavoidable whatever d_{kk} is.

Counting the machines, counting the languages

We now transplant the diagonal argument from real numbers to languages. Two facts about Σ^* do the bookkeeping.

Lemma 9.3.7. *For any finite alphabet Σ , the set Σ^* of all strings over Σ is countable.*

Proof. List the strings in *shortlex* order: first by length, and within each length in alphabetical (dictionary) order. There are only finitely many strings of each length ℓ (at most $|\Sigma|^\ell$ of them), so this rule produces a single list $s_1, s_2, s_3, \dots$ reaching every string after finitely many steps. That is an enumeration of Σ^* . $\square$

Lemma 9.3.8. *The set of all Turing machines is countable.*

Proof. Fix a finite alphabet in which machine descriptions are written. Each Turing machine M has a finite encoding $\langle M \rangle$, a string over that alphabet. By Lemma 9.3.7 the set of all such strings is countable; delete the strings that are not legal machine encodings and the remaining list enumerates all Turing machines. Hence the set of Turing machines is countable. $\square$

Theorem 9.3.9. *For any alphabet Σ with $\Sigma \neq \emptyset$, the set of all languages over Σ is uncountable.*

Proof. Fix the enumeration $\Sigma^* = \{s_1, s_2, s_3, \dots\}$ from Lemma 9.3.7. To each language $\mathcal{L} \subseteq \Sigma^*$ attach its *characteristic sequence* $\chi_{\mathcal{L}}$, the infinite binary string whose i th bit is

$$\chi_{\mathcal{L}}(i) = \begin{cases} 1 & s_i \in \mathcal{L}, \\ 0 & s_i \notin \mathcal{L}. \end{cases}$$

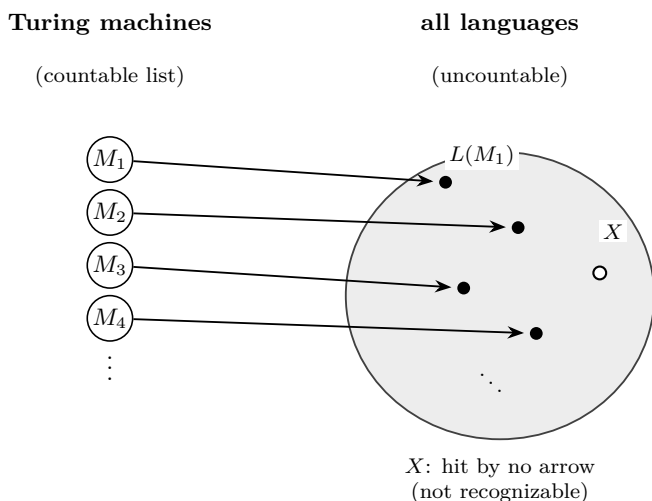
The map $\mathcal{L} \mapsto \chi_{\mathcal{L}}$ is a correspondence between languages and infinite binary sequences: different languages disagree on some s_i and so give different bits there (one-to-one), and any binary sequence is the characteristic sequence of the language it describes (onto). So it is enough to show the set B of infinite binary sequences is uncountable.

Suppose for contradiction that B were countable, with enumeration $b_1, b_2, b_3, \dots$. Write $b_k = b_{k1}b_{k2}b_{k3} \dots$ and define a new sequence $c = c_1c_2c_3 \dots$ by flipping the diagonal, $c_k = 1 - b_{kk}$. Then c differs from b_k in position k for every k , so c equals no b_k – yet $c \in B$. This contradicts the assumption that the list contained every sequence. Hence B , and with it the set of languages, is uncountable. $\square$

Corollary 9.3.10. *Some languages over $\{0, 1\}$ are not Turing-recognizable.*

Proof. By Lemma 9.3.8 the Turing machines form a countable set, and each machine recognizes *one* language, so the set of Turing-recognizable languages is countable (it is the image of a countable set). By Theorem 9.3.9 the set of all languages is uncountable. A countable set cannot equal an uncountable one, so some language is recognized by no Turing machine. $\square$

This is a pure *counting* argument: it proves unrecognizable languages exist without naming a single one. The mismatch is stark – countably many machines, uncountably many targets:



The arrows map each machine to the one language it recognizes; their images fill only a countable part of the uncountable blob, so points like X are left uncovered.

The same counting mismatch can be made fully explicit with a diagonal flip on the *machines themselves*. List the (countably many) Turing machines $M_1, M_2, M_3, \dots$ as rows and the strings $s_1, s_2, s_3, \dots$ as columns; the entry in row i , column j records whether M_i accepts s_j (1) or not (0, i.e. rejects or loops). Boxing the diagonal and flipping each boxed bit defines a language D that disagrees with row i at string s_i .

" M_i accepts s_j ?"	s_1	s_2	s_3	s_4	$\dots$
M_1	1	0	1	1	$\dots$
M_2	1	1	0	0	$\dots$
M_3	0	0	0	1	$\dots$
M_4	1	0	1	0	$\dots$
$\vdots$					$\ddots$
$\chi_D = 1 - d_{ii}$	0	0	1	1	$\dots$

The boxed diagonal reads $d_{11}d_{22}d_{33}d_{44} = 1, 1, 0, 0$; flipping gives $\chi_D = 0, 0, 1, 1, \dots$, so $s_1 \notin D$, $s_2 \notin D$, $s_3 \in D$, $s_4 \in D$. By construction χ_D differs from row i at column i , hence $D \neq L(M_i)$ for *every* i : no machine on the list recognizes D , and the list was *all* machines.

Takeaway. List machines as rows and strings as columns; flipping the diagonal builds a language D that disagrees with machine M_i on string s_i , so D is recognized by no machine – there are more languages than machines.

Worked Example 9.3.11 (Reading off a characteristic sequence, and diagonalizing a list of languages). Fix $\Sigma = \{0, 1\}$ with the shortlex enumeration $s_1 = \varepsilon$, $s_2 = 0$, $s_3 = 1$, $s_4 = 00, \dots$. Let $\mathcal{L}$ be “strings of even length” (so $\varepsilon, 00, 01, \dots \in \mathcal{L}$). Write the first few bits of $\chi_{\mathcal{L}}$. Then, given a list of three languages, build a language differing from all of them by the diagonal trick.

Recognition. Two sub-tasks. First, $\chi_{\mathcal{L}}(i) = 1$ exactly when $s_i \in \mathcal{L}$ – a membership lookup. Second, “a language off every entry of a list” is precisely Theorem 9.3.9’s diagonal flip applied to the rows.

Solution. Characteristic sequence. Check each s_i for even length: ε (len 0, even, 1), 0 (len 1, 0), 1 (len 1, 0), 00 (len 2, 1), 01 (1), so $\chi_{\mathcal{L}} = 10011\dots$ *Diagonalization.* Suppose three languages have the characteristic-sequence rows below; box the diagonal bits and flip them to get a sequence c , hence a language C off all three rows.

	s_1	s_2	s_3	s_4	$\dots$
$\mathcal{L}_1$	1	0	1	1	$\dots$
$\mathcal{L}_2$	0	0	0	1	$\dots$
$\mathcal{L}_3$	1	1	1	0	$\dots$
$c = \chi_C$	0	1	0	$\dots$	

The diagonal bits are 1, 0, 1; flipping gives $c = 0, 1, 0, \dots$, i.e. $\varepsilon \notin C$, $0 \in C$, $1 \notin C$. Then $C \neq \mathcal{L}_1$ (they disagree on s_1), $C \neq \mathcal{L}_2$ (on s_2), $C \neq \mathcal{L}_3$ (on s_3).

Which idea, and why? A language is named by its characteristic sequence, so “find a language unlike every one on a list” becomes “find a binary sequence unlike every row” – exactly Cantor’s diagonal flip, which guarantees one disagreement per row.

Check. If the list were extended to a fourth row $\mathcal{L}_4$, we would simply flip b_{44} too; the construction never runs out, which is why no countable list of languages can be complete.

Worked Example 9.3.12 (Why “countable union of countable sets” keeps machines countable). Argue that adding more tape symbols or states does not make the set of Turing machines uncountable: the machines with at most k states and k tape symbols form a countable family for each k .

Recognition. The trigger is “is this still countable after taking a union over all k ?” We use the fact that a countable union of countable (indeed finite) sets is countable, mirroring the anti-diagonal sweep of Worked Example 9.3.4 for $\mathbb{Q}^+$.

Solution. For each fixed k there are only *finitely* many Turing machines with at most k states and k tape symbols, because there are finitely many possible transition tables on a finite state/symbol set. Call this finite set T_k . Every Turing machine lies in some T_k (take k as the larger of its state and symbol counts), so the set of all machines is $\bigcup_{k \geq 1} T_k$, a countable union of finite sets. Enumerate it by listing T_1 , then the new machines of T_2 , then T_3 , and so on – a single list, hence countable.

Which idea, and why? Countability is robust under countable unions: indexing the machines by a size parameter k turns “all machines” into a $\bigcup_k T_k$ of finite slabs, which an interleaved list enumerates – the same finite-slab idea that made $\mathbb{Q}^+$ countable.

Check. This is consistent with Lemma 9.3.8: both routes (encode-as-strings, or slab-by-size) put the machines in bijection with a subset of $\mathbb{N}$.

Method (How to prove a set is countable, or uncountable).

1. **To show A is countable**, produce an enumeration $a_1, a_2, \dots$ of all of A with no omissions. Reliable patterns: (i) *shortlex*, if $A \subseteq \Sigma^*$ (length, then alphabetical); (ii) the *anti-diagonal sweep*, if elements are indexed by a pair (or finite tuple) of naturals; (iii) *encode into strings*, if each element has a finite description – then apply (i). A countable union of countable sets is again countable.
2. **To show A is uncountable**, assume a complete list $a_1, a_2, \dots$ exists and *diagonalize*: build an object x that differs from a_k in “coordinate k ” for every k , so x is on no row. For sequences/sets, flip the k th bit of the k th row; for decimals, change the k th digit (avoiding 0 and 9). The resulting $x \in A$ contradicts completeness.
3. **To conclude a counting impossibility** (e.g. “not all X are recognizable”), pair “countable supply” against “uncountable demand”: if the objects you can build form a countable set but the targets form an uncountable set, some target is unbuildable.

Caution. “ A is infinite” does *not* mean “ A is uncountable.” Many infinite sets – $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, Σ^* , the set of all Turing machines – are countable. Uncountability is the stronger claim that even an *infinite* list misses something, and it must be earned by a diagonal argument, never assumed from sheer size.

Caution. The diagonal construction must produce an element that genuinely lies in the set you are listing. In Theorem 9.3.5 we avoid the digits 0 and 9 precisely so the new number has a *unique* decimal representation and cannot secretly coincide with a listed number via the identity $0.49 = 0.50$. Skip that safeguard and the “contradiction” evaporates, because x might equal some $f(k)$ after all.

Caution. Do not confuse a *language* (a set of strings, possibly infinite) with its *characteristic sequence* (one infinite binary string). The correspondence $\mathcal{L} \mapsto \chi_{\mathcal{L}}$ of Theorem 9.3.9 is what lets us diagonalize over languages at all; without fixing a single enumeration $s_1, s_2, \dots$ of Σ^* first, the i th bit “ $s_i \in \mathcal{L}?$ ” is not even well defined.

Exercises.

1. For each function decide whether it is one-to-one, onto, or a correspondence from $\mathbb{N}$ to $\mathbb{N}$, and justify: (a) $f(n) = n + 1$; (b) $g(n) = \lceil n/2 \rceil$; (c) $h(n) = 2n - 1$. For any that fail to be onto, name an element of the codomain that is missed.
2. Exhibit an explicit correspondence between $\mathbb{N}$ and the set $\mathbb{Z}$ of all integers (positive, negative, and zero). Write out the first six pairs of your bijection as a two-row table, and give a closed formula for $f(n)$.
3. Let $\Sigma = \{a, b\}$. List the first eight strings of Σ^* in shortlex order, and state which position s_i the string ba occupies. Then give a general rule for the position of an arbitrary length-2 string.
4. Show that the set $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$ of ordered triples of natural numbers is countable. (Hint: triples are indexed by three coordinates; reduce to the two-coordinate case twice, or sweep the planes $i + j + k = \text{const.}$)
5. Prove that the set of all *finite* subsets of $\mathbb{N}$ is countable. Then explain in one sentence why this does *not* contradict the uncountability of the set of *all* subsets of $\mathbb{N}$.
6. Carry out Cantor’s diagonal argument on the concrete list below to produce a real number $x \in (0, 1)$ not on the list. Use the flip rule “5 unless the diagonal digit is 5, in which case 4,” and write the first four digits of x .

$$f(1) = 0.2718 \dots, \quad f(2) = 0.1414 \dots, \quad f(3) = 0.5000 \dots, \quad f(4) = 0.3162 \dots$$

7. Given the membership table below for three languages over $\{0, 1\}$ (column s_i is the shortlex enumeration), build the characteristic sequence of a language C that differs from all three. Identify, for each $j \in \{1, 2, 3\}$, a string on which C and $\mathcal{L}_j$ disagree.

	s_1	s_2	s_3	s_4
$\mathcal{L}_1$	0	1	1	0
$\mathcal{L}_2$	1	1	0	0
$\mathcal{L}_3$	0	0	0	1

8. Write down the characteristic sequence (first five bits) of each of these languages over $\{0, 1\}$ using the shortlex enumeration $s_1 = \varepsilon, s_2 = 0, s_3 = 1, s_4 = 00, s_5 = 01$: (a) $\emptyset$; (b) Σ^* ; (c) the strings ending in 1; (d) the strings of length exactly 1.
9. Prove that the set of all *deciders* (Turing machines that halt on every input) is countable, and conclude that some language over $\{0, 1\}$ is not decidable – *without* naming such a language. State clearly which set you are calling countable and which uncountable.
10. *Spot the flaw.* A student “proves” that Σ^* over $\{0, 1\}$ is uncountable as follows: “List all strings $s_1, s_2, \dots$. Build x by flipping the k th bit of s_k . Then x is a string on no row, so the list is incomplete.” Explain precisely why this argument is wrong, and identify the step that fails. (Compare with Lemma 9.3.7 on the countability of Σ^* .)
11. *Spot the flaw.* Another student argues that there are only countably many languages, “because each language can be described by an English sentence, and there are only countably many finite English sentences.” Pinpoint the false step and connect it to Theorem 9.3.9.
12. Let $\mathcal{L}$ be a countable set of languages over $\{0, 1\}$, say $\mathcal{L} = \{\mathcal{L}_1, \mathcal{L}_2, \dots\}$. Prove there is a language C with $C \neq \mathcal{L}_k$ for every k . (This is the diagonal argument stated for an arbitrary countable family.) Then state what this implies about whether *any* fixed countable list of languages can equal the set of all languages.

9.4 Undecidability of the Acceptance Problem

Section 9.3 showed, by counting, that *some* language must escape every Turing machine: there are uncountably many languages but only countably many machines. That argument is non-constructive – it names no escapee. This section delivers a concrete one. We prove that the acceptance problem $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is *undecidable*: no algorithm, on input the code of a machine M and a string w , can always halt and correctly report whether M accepts w . The one key idea is to turn the diagonal argument inward – if a decider for A_{TM} existed we could feed it a machine engineered to contradict its own verdict, exactly the self-reference $D(\langle D \rangle)$ that has no consistent value. The same machine, read once more, also kills the halting problem.

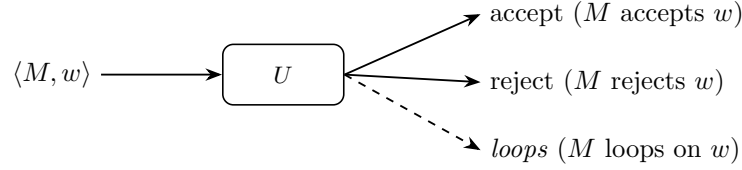
The acceptance problem and its recognizer

Recall from Section 9.2 that A_{TM} is Turing-recognizable: the universal machine U simulates M on w and copies M ’s verdict. We restate it because the gap between U and a *decider* is the whole point of this section.

Theorem 9.4.1 (A_{TM} is recognizable). *The language A_{TM} is Turing-recognizable.*

Proof. Let U be the universal Turing machine: on input $\langle M, w \rangle$, U simulates the run of M on w step by step. If that run ever enters M ’s accept state, U accepts; if it ever enters M ’s reject state, U rejects. If M accepts w then U accepts $\langle M, w \rangle$, and if M does not accept w then U does not accept $\langle M, w \rangle$ (it rejects when M rejects, and loops when M loops). Hence $L(U) = A_{\text{TM}}$, so A_{TM} is recognizable. $\square$

The phrase “loops when M loops” is the crux: U recognizes A_{TM} but does not decide it, because on a pair $\langle M, w \rangle$ for which M runs forever, U also runs forever and never reports “reject.” A decider may not loop – it must halt on every input. The diagram below contrasts the recognizer we have with the decider we are about to forbid.



The recognizer U for A_{TM} . The dashed branch – looping – is the behaviour a decider is not allowed to have.

The diagonal machine

Theorem 9.4.2 (Undecidability of the acceptance problem). A_{TM} is undecidable: no Turing machine decides A_{TM} .

Proof. Suppose, for contradiction, that some TM H decides A_{TM} . Then H halts on every input and

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w, \\ \text{reject} & \text{if } M \text{ does not accept } w. \end{cases}$$

Using H as a subroutine, build a new machine D that takes a single machine code $\langle M \rangle$, asks H what M does on its *own* description, and then deliberately does the opposite:

- D = “On input $\langle M \rangle$, where M is a TM:
1. Run H on input $\langle M, \langle M \rangle \rangle$.
 2. If H accepts, *reject*; if H rejects, *accept*.”

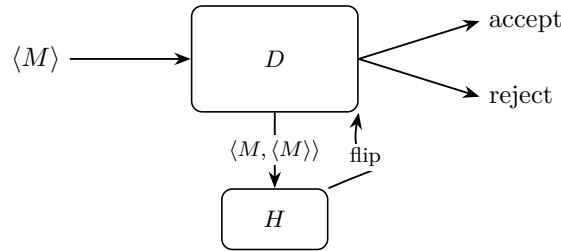
Because H always halts, so does D ; thus D is a genuine machine with

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ does not accept } \langle M \rangle, \\ \text{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$

Now run D on its own description $\langle D \rangle$. Substituting $M = D$ into the line above gives

$$D(\langle D \rangle) = \begin{cases} \text{accept} & \text{if } D \text{ does not accept } \langle D \rangle, \\ \text{reject} & \text{if } D \text{ accepts } \langle D \rangle. \end{cases}$$

So D accepts $\langle D \rangle$ exactly when D does not accept $\langle D \rangle$. This is a flat contradiction, so D cannot exist; since D was built from H by a finite, legal construction, the fault lies with H . No decider H for A_{TM} exists. $\square$



D feeds $\langle M, \langle M \rangle \rangle$ to the assumed decider H , then outputs the *opposite*. Running D on $\langle D \rangle$ closes the loop and contradicts itself.

Remark. The construction is not a trick of notation. Running a program on its own source is routine: a C compiler can compile its own source, an interpreter can interpret a copy of itself. What Theorem 9.4.2 shows is that *deciding* a self-referential question about that run is impossible, not that self-reference itself is ill-formed.

Where the diagonal hides

The name “diagonalization” is justified by tabulating behaviour. List all Turing machines down the rows, $M_1, M_2, M_3, \dots$, and all their codes across the columns, $\langle M_1 \rangle, \langle M_2 \rangle, \dots$. Put acc in cell (i, j) when M_i accepts $\langle M_j \rangle$ and leave it blank otherwise. If H decided A_{TM} , it would compute every entry of this table on demand.

Worked Example 9.4.3 (Reading the acceptance table H would fill in). Write the first rows and columns of the table whose (i, j) entry is the value of H on $\langle M_i, \langle M_j \rangle \rangle$, and locate the diagonal.

Recognition. This is the “ H computes the whole acceptance grid” picture. The trigger is that D inspects only entry (i, i) – machine M_i on its own code – so we should mark that diagonal.

Solution. Each entry is a definite accept/reject because H halts. A made-up but consistent grid:

H	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$
M_1	acc	rej	acc	rej
M_2	acc	acc	acc	acc
M_3	rej	rej	rej	rej
M_4	acc	acc	rej	rej

The boxed cells form the diagonal: entry (i, i) is whether M_i accepts its own code $\langle M_i \rangle$. Reading down it gives the sequence acc, acc, rej, rej.

Which idea, and why? Self-reference uses only the diagonal: D ’s behaviour on $\langle M_i \rangle$ depends on cell (i, i) alone, so that single diagonal entry is all that matters for the contradiction.

Check. Row M_2 says M_2 accepts $\langle M_3 \rangle$; that is an off-diagonal fact and plays no role. Only the boxed $(2, 2) = \text{acc}$ feeds into $D(\langle M_2 \rangle)$.

Worked Example 9.4.4 (Adding D to the table forces the impossible cell). The machine D is itself a Turing machine, so it appears as some row in the list. Append its row and column and find the entry that cannot be filled.

Recognition. Now we use “ D flips the diagonal.” Since D exists (granting H), it must occupy a row, and its own diagonal cell is the site of the contradiction.

Solution. By construction $D(\langle M_i \rangle)$ is the opposite of the boxed diagonal entry (i, i) above: where the diagonal had acc, acc, rej, rej, row D holds rej, rej, acc, acc. Appending D ’s row and the column $\langle D \rangle$:

H	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\langle D \rangle$
M_1	acc	rej	acc	rej	acc
M_2	acc	acc	acc	acc	acc
M_3	rej	rej	rej	rej	acc
M_4	acc	acc	rej	rej	rej
D	rej	rej	acc	acc	?

Every column for D ’s row is the negation of that column’s diagonal entry. The boxed cell is the entry for D on $\langle D \rangle$: it must be the opposite of itself, since it both lies on the diagonal (so D flips it) and *is* that diagonal value. No accept/reject fits, which is the contradiction.

Which idea, and why? D ’s row is the negated diagonal; placing D in the table makes cell $(D, \langle D \rangle)$ both equal to and opposite to the diagonal value there – the single self-referential cell that cannot be consistently filled.

Check. Off-diagonal entries of D ’s row are fine: $D(\langle M_1 \rangle) = \text{rej}$ negates the diagonal $(1, 1) = \text{acc}$, no conflict. Only $(D, \langle D \rangle)$ breaks.

The halting problem

The same machine D shows that we cannot even decide whether a computation *stops*, let alone what it decides. Define the halting problem

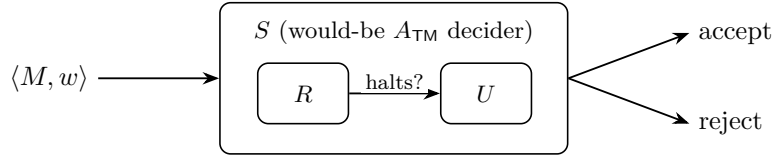
$$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts (accepts or rejects) on input } w \}.$$

Theorem 9.4.5 (Undecidability of the halting problem). $HALT_{TM}$ is undecidable.

Proof. Suppose a TM R decides $HALT_{TM}$. We build a decider S for A_{TM} , contradicting Theorem 9.4.2:

- $S =$ “On input $\langle M, w \rangle$:
1. Run R on $\langle M, w \rangle$.
 2. If R rejects (so M loops on w), *reject*.
 3. If R accepts (so M halts on w), simulate M on w with U until it halts; *accept* iff M accepts.”

Step 3’s simulation is safe precisely because step 2 has already certified that M halts on w , so U cannot loop here. Thus S halts on every input and accepts $\langle M, w \rangle$ iff M accepts w ; that is, S decides A_{TM} . Since no such S exists, no decider R for $HALT_{TM}$ exists. $\square$



If R decided $HALT_{TM}$, the box S would decide A_{TM} : ask R whether M halts, and only then let U finish the simulation safely.

Worked Example 9.4.6 (A naive halting test that is wrong). A student proposes deciding $HALT_{TM}$ by running M on w for a fixed budget of 10^6 steps: accept if M halts within the budget, reject otherwise. Explain why this does not decide $HALT_{TM}$.

Recognition. This is the classic “timeout in place of a decision” error. The trigger is that no step bound, whether fixed or computed from the input, can certify non-halting.

Solution. The proposed machine T is a perfectly good *decider* – it always halts – but it decides the wrong language. Take a machine M_0 that halts on w_0 only after $10^6 + 1$ steps. Then M_0 does halt on w_0 , so $\langle M_0, w_0 \rangle \in HALT_{TM}$, yet T rejects because the budget ran out. So T rejects a string in $HALT_{TM}$: $L(T) \neq HALT_{TM}$. No constant – and no computable function of the input – works, because to pick a correct bound for $\langle M, w \rangle$ one would already need to know whether M halts.

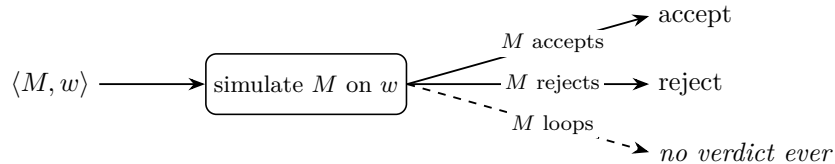
Which idea, and why? Timeouts decide “halts within k steps,” a different, decidable language; deciding “halts eventually” would require a halting oracle, which is exactly what Theorem 9.4.5 forbids.

Check. The bounded language $\{\langle M, w \rangle \mid M \text{ halts on } w \text{ in } \leq 10^6 \text{ steps}\}$ is decidable (just simulate 10^6 steps), confirming the issue is the unbounded “eventually,” not the simulation.

Worked Example 9.4.7 (Why you cannot decide A_{TM} by simulating until a verdict). Pin down exactly which step of “run M on w and report what happens” fails to give a decider for A_{TM} .

Recognition. “ U recognizes but does not decide.” The trigger is a missing reject branch on the looping case.

Solution. Simulating M on w has three possible fates: M accepts (then accept), M rejects (then reject), or M loops (then the simulation loops too, and *never* reaches a reject). The first two fates are handled; the third is the failure. To convert U into a decider we would need to detect, in finite time, that M will loop on w – the very capability the diagonal machine D shows is impossible.



Which idea, and why? A decider must close the dashed branch with a finite “reject,” but doing so amounts to deciding $HALT_{TM}$, so by Theorem 9.4.5 the branch cannot be closed.

Check. If you cap the simulation to force a verdict, you decide a bounded variant (previous example), not A_{TM} itself.

These two undecidable languages anchor a small map of the classes. Decidable languages sit inside the recognizable ones, and the strictness of that inclusion is now witnessed concretely.

$$\begin{array}{ccc}
\text{decidable} & \longleftrightarrow & \text{Turing-recognizable} \\
\notin & & \in \\
\text{HALT}_{\text{TM}}, A_{\text{TM}} & & A_{\text{TM}}
\end{array}$$

A_{TM} is recognizable but not decidable; HALT_{TM} likewise. Section 9.5 shows the *complement* $\overline{A_{\text{TM}}}$ is not even recognizable.

Method (How to prove a language $\mathcal{L}$ is undecidable by reduction from A_{TM}).

1. **Assume a decider.** Suppose a TM decides $\mathcal{L}$; you will derive a contradiction by building a decider for a language already known to be undecidable (usually A_{TM}).
2. **Build the harness.** Describe a machine S that, on input $\langle M, w \rangle$, calls the assumed $\mathcal{L}$ -decider on some instance you construct from $\langle M, w \rangle$.
3. **Make every branch halt.** Ensure S never falls into an uncontrolled simulation: only simulate M on w after a halting guarantee, or arrange that the constructed instance's membership in $\mathcal{L}$ mirrors “ M accepts w ” exactly.
4. **Read off the contradiction.** Conclude S decides A_{TM} , contradicting Theorem 9.4.2; therefore no decider for $\mathcal{L}$ exists.

The HALT_{TM} proof above is the template: R -for- HALT_{TM} was turned into S -for- A_{TM} , with the halting check inserted precisely to tame the dangerous simulation step.

Caution. “Undecidable” does not mean “unrecognizable.” A_{TM} is recognizable – the universal machine U accepts exactly its members – it merely fails to halt on some non-members. Reserve “not recognizable” for languages, like $\overline{A_{\text{TM}}}$ in Section 9.5, that no machine accepts even when allowed to loop on rejects.

Caution. The proof of Theorem 9.4.2 does not exhibit one fixed input on which every candidate H goes wrong; it shows that the *machine* H cannot exist at all. Do not look for “the hard instance” of A_{TM} . The hardness is global: each candidate decider H is defeated on the tailored input $\langle D, \langle D \rangle \rangle$, where D is built from that very decider, and D changes when H changes.

Caution. When you reduce, keep the direction straight. To prove HALT_{TM} undecidable we built an A_{TM} -decider *out of* an assumed HALT_{TM} -decider – we used the new problem to solve the old one. Building a HALT_{TM} -decider out of an A_{TM} -decider would prove nothing about HALT_{TM} . The known-hard language (A_{TM}) is the thing you must end up deciding, never the thing you assume.

Exercises.

1. Reconstruct the undecidability proof for A_{TM} . Suppose H decides A_{TM} , and define D on input $\langle M \rangle$ to run H on $\langle M, \langle M \rangle \rangle$ and then output the *opposite* answer. Fill in the table of what H , and then D , say when D is run on $\langle D \rangle$, and state the contradiction in one line.
2. Explain precisely why the “obvious” machine – simulate M on w and accept/reject as M does – *recognizes* A_{TM} but does not *decide* it. On which inputs does it fail to halt?
3. Prove that if A_{TM} were decidable, then *every* Turing-recognizable language would be decidable. (Hint: a recognizer plus a decider for A_{TM} gives a decider.)
4. Show that the halting problem $\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ halts on } w \}$ is undecidable, assuming A_{TM} is. Describe a decider for A_{TM} that you could build from a decider for HALT_{TM} , and why that is impossible.
5. Define $A_{\text{TM}}^{\leq t} = \{ \langle M, w, 1^t \rangle \mid M \text{ accepts } w \text{ within } t \text{ steps} \}$. Prove this language *is* decidable. Why does the bounded version escape the undecidability of A_{TM} ?
6. The diagonal table. Index all Turing machines $M_1, M_2, \dots$ and let entry (i, j) record whether M_i accepts $\langle M_j \rangle$. Describe the language D read off the flipped diagonal, and explain why D equals $L(M_i)$ for no i .
7. Prove that A_{TM} is Turing-recognizable by describing the universal Turing machine that recognizes it. Where exactly can it loop?
8. Let B be any decidable language and $C = A_{\text{TM}} \cup B$. Is C necessarily recognizable? Necessarily decidable? Justify each answer.

9. *Spot the flaw.* A student argues A_{TM} is decidable: “Given $\langle M, w \rangle$, build M ’s computation tree; if it is finite, read off the answer; if infinite, reject.” Identify the non-effective step.
10. *Spot the flaw.* Another student claims the self-referential machine D “cannot be built, because a machine cannot read its own description.” Explain briefly why D is constructible (you may cite the existence of an encoding $\langle \cdot \rangle$ and the universal machine).
11. Show that undecidability is not about “large” languages: exhibit a decidable language and an undecidable language that are both infinite and both co-infinite, and explain why size alone tells you nothing about decidability.

9.5 Turing-Unrecognizable Languages

The previous section produced a concrete undecidable language, A_{TM} , yet A_{TM} is still *recognizable*: a universal machine accepts every $\langle M, w \rangle$ with M accepting w , merely looping on some inputs it should reject. The job of this section is to climb one rung higher and exhibit a language that no Turing machine recognizes *at all*. The single idea that gets us there is a symmetry between a language and its complement: deciding a language means recognizing the “yes” instances *and* recognizing the “no” instances, so a language sits in the decidable class exactly when both it and its complement are recognizable. Reading that equivalence backwards turns the undecidability of A_{TM} – already proved – into the *unrecognizability* of its complement, and it draws the final boundary of the class map: decidable strictly inside recognizable strictly inside all languages.

Definition 9.5.1 (Complement and co-recognizability). For $\mathcal{L} \subseteq \Sigma^*$ the *complement* is $\bar{\mathcal{L}} = \Sigma^* \setminus \mathcal{L}$, the set of strings *not* in $\mathcal{L}$. A language is *Turing-recognizable* if some TM accepts exactly its strings (and on the others rejects or loops). A language is *co-Turing-recognizable* if its complement $\bar{\mathcal{L}}$ is Turing-recognizable. We write REC for the class of recognizable languages and DEC for the decidable ones (the classes written RE and R in Definition 8.5.2).

Recognizable and co-recognizable are genuinely different requirements. A recognizer for $\mathcal{L}$ is allowed to loop on strings *outside* $\mathcal{L}$; a recognizer for $\bar{\mathcal{L}}$ is allowed to loop on strings *inside* $\mathcal{L}$. Having both at once is exactly what removes every excuse to loop, and that is the content of the next theorem.

Theorem 9.5.2 (Decidable = recognizable + co-recognizable). *A language $\mathcal{L}$ is decidable if and only if both $\mathcal{L}$ and $\bar{\mathcal{L}}$ are Turing-recognizable. Equivalently, $\text{DEC} = \text{REC} \cap \text{coREC}$.*

Proof. ($\Rightarrow$) Suppose $\mathcal{L}$ is decidable, say by a TM D that halts on every input. Then D recognizes $\mathcal{L}$, so $\mathcal{L} \in \text{REC}$. Build D' by swapping D ’s accept and reject outcomes; D' still halts on every input and now accepts exactly $\bar{\mathcal{L}}$, so $\bar{\mathcal{L}}$ is decidable and in particular recognizable. Hence both $\mathcal{L}$ and $\bar{\mathcal{L}}$ are recognizable.

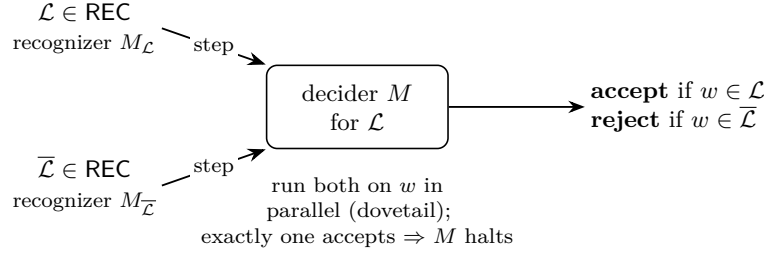
($\Leftarrow$) Suppose M_1 recognizes $\mathcal{L}$ and M_2 recognizes $\bar{\mathcal{L}}$. Define a two-tape machine M :
 $M =$ “On input w :

1. Simulate M_1 and M_2 on w *in parallel*, advancing one step of each per round.
2. If M_1 accepts, **accept**; if M_2 accepts, **reject**.”

Every string w lies in exactly one of $\mathcal{L}$, $\bar{\mathcal{L}}$. If $w \in \mathcal{L}$ then M_1 accepts w in finitely many steps, so the parallel simulation reaches that point and M accepts. If $w \in \bar{\mathcal{L}}$ then M_2 accepts w in finitely many steps, so M rejects. In either case one of the two simulations halts with acceptance, so M halts on *every* input: it is a decider. It accepts exactly $\mathcal{L}$, so $\mathcal{L}$ is decidable. $\square$

The parallel simulation is essential. Running M_1 *to completion* first would never return on a string outside $\mathcal{L}$ (where M_1 may loop forever), and we would never get to try M_2 . Dovetailing the two – one step of each, turn by turn – guarantees that whichever recognizer is destined to accept gets its turn in finite time.

The same equivalence can be read as a structure diagram, tracking the *objects* (the language $\mathcal{L}$, its complement $\bar{\mathcal{L}}$, the two recognizers, and the decider they assemble into) rather than the step-by-step dynamics. The two recognizers feed one combined machine; on any w exactly one of them is bound to accept, which is precisely the invariant “decidable.”



Check. Every w lies in exactly one of $\mathcal{L}, \bar{\mathcal{L}}$, so exactly one recognizer accepts w in finite time and M always halts with the right verdict; conversely a decider for $\mathcal{L}$ yields recognizers for both sides by swapping accept/reject. Hence $\text{DEC} = \text{REC} \cap \text{coREC}$.

Takeaway. $\mathcal{L}$ is decidable iff both $\mathcal{L}$ and its complement are recognizable: dovetail the two recognizers on w ; exactly one accepts, so the combined machine always halts with the right answer.

Corollary 9.5.3 ($\overline{A_{\text{TM}}}$ is unrecognizable). *The complement $\overline{A_{\text{TM}}}$ of the acceptance problem is not Turing-recognizable.*

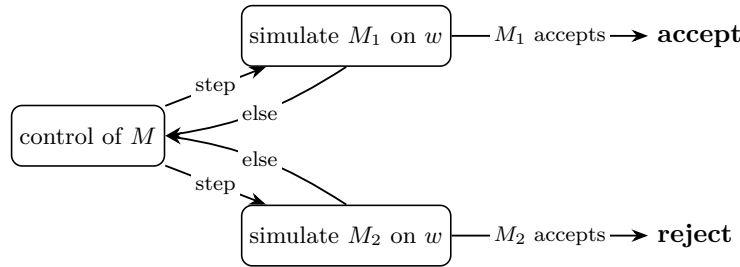
Proof. A_{TM} is Turing-recognizable (the universal machine recognizes it) but, by the previous section, *not* decidable. If $\overline{A_{\text{TM}}}$ were also recognizable, then A_{TM} would be both recognizable and co-recognizable, hence decidable by Theorem 9.5.2 – contradicting its undecidability. So $\overline{A_{\text{TM}}}$ is not recognizable. $\square$

This is our first natural problem provably outside REC. Note the shape of the argument: we never built a diagonal machine against $\overline{A_{\text{TM}}}$ directly; we *reused* the undecidability of A_{TM} and the new equivalence. That reuse is the lever, and the worked examples below drill it in several directions.

Worked Example 9.5.4 (The parallel decider as a two-tape machine). Suppose $\mathcal{L}$ and $\bar{\mathcal{L}}$ are both recognizable, by M_1 and M_2 . Draw the machine M that decides $\mathcal{L}$ and explain why it never loops.

Recognition. This is the construction of Theorem 9.5.2: “both directions recognizable” triggers the *parallel/dovetailing* decider. The picture to draw is a two-tape simulation, one tape per recognizer.

Solution. M keeps tape 1 simulating M_1 on w and tape 2 simulating M_2 on w , alternating a single step of each. The control watches both for an accept.



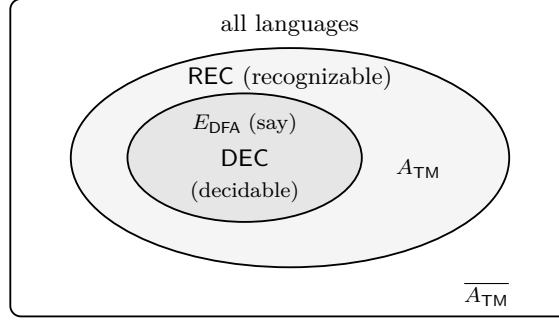
Which idea, and why? Because every w is in $\mathcal{L}$ or in $\bar{\mathcal{L}}$, exactly one of M_1, M_2 is guaranteed to accept in finite time; dovetailing reaches that accept, so M halts on all inputs and is a decider.

Check. If $w \in \mathcal{L}$, the M_1 -branch fires after finitely many rounds and M accepts; the possibly-looping M_2 never gets the chance to matter. Symmetric for $w \in \bar{\mathcal{L}}$. No input survives both branches, so there is no infinite loop.

Worked Example 9.5.5 (Pinpointing A_{TM} and its complement on the class map). Place DEC, REC, and the three languages A_{TM} (an undecidable-but-recognizable problem), $\overline{A_{\text{TM}}}$, and E_{DFA} (some decidable language) on a map of language classes.

Recognition. “Where does a language live?” is answered by the two coordinates of Theorem 9.5.2: *is it recognizable?* and *is its complement recognizable?* A nested-region (Venn) map is the right picture.

Solution. Decidable languages form the innermost region; recognizable languages a strictly larger one; all languages the outermost. A_{TM} is recognizable but not decidable, so it sits in the recognizable ring outside DEC. Its complement $\overline{A_{\text{TM}}}$ is not recognizable (Corollary 9.5.3), so it sits entirely outside REC.



Which idea, and why? The two recognizability coordinates partition the map: $\text{DEC} = \text{REC} \cap \text{coREC}$ is the overlap, A_{TM} has only the first coordinate, and $\overline{A_{\text{TM}}}$ has only the second, landing it outside REC altogether.

Check. A language in DEC must have both coordinates, consistent with its position inside both rings. $\overline{A_{\text{TM}}}$ outside REC certifies that the recognizable ring is *strictly* inside “all languages.”

Worked Example 9.5.6 (The four recognizability quadrants). Classify every language by the pair (is $\mathcal{L}$ recognizable?, is $\overline{\mathcal{L}}$ recognizable?). Which quadrants are occupied, and by what?

Recognition. Theorem 9.5.2 makes the pair of yes/no answers the natural classifier, so a 2×2 table is the diagram. The “yes/yes” cell is forced to be exactly DEC .

Solution. Tabulate the four combinations. “Yes/yes” is decidable; the two “yes/no” cells are the recognizable-but-undecidable languages and their complements; “no/no” holds languages so hard that neither side can even be recognized.

	$\overline{\mathcal{L}} \in \text{REC}$	$\overline{\mathcal{L}} \notin \text{REC}$
$\mathcal{L} \in \text{REC}$	$\mathcal{L}$ decidable (E_{DFA})	$\mathcal{L} = A_{\text{TM}}$
$\mathcal{L} \notin \text{REC}$	$\mathcal{L} = \overline{A_{\text{TM}}}$	e.g. EQ_{TM}

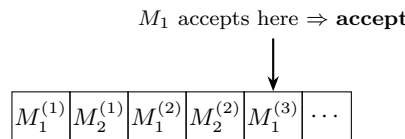
Which idea, and why? The yes/yes cell is $\text{REC} \cap \text{coREC} = \text{DEC}$ by Theorem 9.5.2; A_{TM} and $\overline{A_{\text{TM}}}$ are reflections across the diagonal of this table, and the “no/no” corner is where the truly unrecognizable-both-ways languages live.

Check. A language and its complement always sit in mirror-image cells (swap the two coordinates), so the table is symmetric under transpose with $\mathcal{L} \leftrightarrow \overline{\mathcal{L}}$; A_{TM} in the top-right corresponds to $\overline{A_{\text{TM}}}$ in the bottom-left, as drawn.

Worked Example 9.5.7 (Reading the dovetailing schedule off a tape). Show the alternation schedule M uses when dovetailing M_1 and M_2 , and locate the step at which a string $w \in \mathcal{L}$ gets accepted.

Recognition. “How does parallel simulation actually run” is a *scheduling* question; the clean picture is a tape-like strip of rounds, each round running one step of M_1 then one step of M_2 .

Solution. Lay the global step counter as a strip. Odd cells advance M_1 , even cells advance M_2 . If M_1 accepts w after k of its own steps, that happens at global round k , i.e. tape cell $2k - 1$.



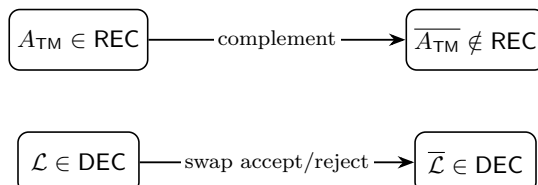
Which idea, and why? Interleaving guarantees both simulations make unbounded progress, so the recognizer that is destined to accept reaches its accepting step at a *finite* cell; running either machine alone first could stall forever on a non-accepting loop.

Check. The cell index of M_1 ’s k -th step is $2k - 1$, always finite, so acceptance for $w \in \mathcal{L}$ occurs in bounded time – the decider never reads past a finite prefix of this strip.

Worked Example 9.5.8 (Closure of DEC under complement, and the recognizable asymmetry). Prove DEC is closed under complement. Then show REC is *not* closed under complement.

Recognition. Two contrasting facts share the same lever. “DEC closed under complement” is the swap-outcomes trick; “REC not closed” is a single *witness*: a recognizable language whose complement is not recognizable.

Solution. If $\mathcal{L} \in \text{DEC}$ via decider D , swapping accept/reject in D yields a decider for $\overline{\mathcal{L}}$, so $\overline{\mathcal{L}} \in \text{DEC}$. For REC: take $\mathcal{L} = A_{\text{TM}} \in \text{REC}$. Its complement $\overline{A_{\text{TM}}} \notin \text{REC}$ by Corollary 9.5.3. So a single language witnesses that complementing can leave REC.



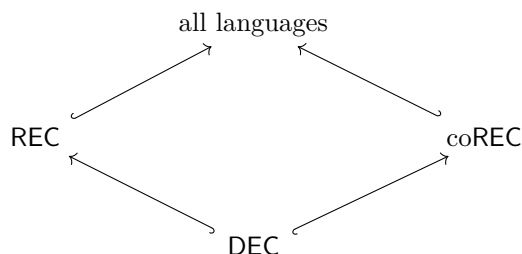
Which idea, and why? Inside DEC the decider halts, so flipping its answer is legal and stays in DEC; inside REC the recognizer may loop, so there is no answer to flip, and A_{TM} is the concrete counterexample.

Check. If REC *were* closed under complement, then $\text{REC} = \text{coREC}$ and Theorem 9.5.2 would give $\text{REC} = \text{DEC}$, collapsing the strict inclusion – impossible since $A_{\text{TM}} \in \text{REC} \setminus \text{DEC}$.

Worked Example 9.5.9 (A class map as an inclusion diagram). Express the relationships among DEC, REC, coREC, and “all languages” as a diagram of strict inclusions, and mark where A_{TM} and $\overline{A_{\text{TM}}}$ enter.

Recognition. When the content is *set inclusions* rather than machine dynamics, a node-and-arrow inclusion (lattice) diagram communicates it best. The intersection $\text{REC} \cap \text{coREC} = \text{DEC}$ is the meet at the bottom.

Solution. Put “all languages” at the top, REC and coREC as two incomparable classes below it, and DEC at their meet. Each arrow is a strict “ $\subsetneq$ ”, witnessed by A_{TM} (in REC, not in DEC) and $\overline{A_{\text{TM}}}$ (in coREC, not in DEC).



Which idea, and why? Each hooked arrow is a strict inclusion; $A_{\text{TM}} \in \text{REC} \setminus \text{DEC}$ makes the left inclusion strict and $\overline{A_{\text{TM}}} \in \text{coREC} \setminus \text{DEC}$ makes the right one strict, while $\text{DEC} = \text{REC} \cap \text{coREC}$ is the diagram’s bottom meet.

Check. Reading bottom-up: anything decidable is both recognizable and co-recognizable (both upward arrows from DEC); the converse meet equals DEC exactly, which is Theorem 9.5.2.

Method (How to prove a specific language is unrecognizable).

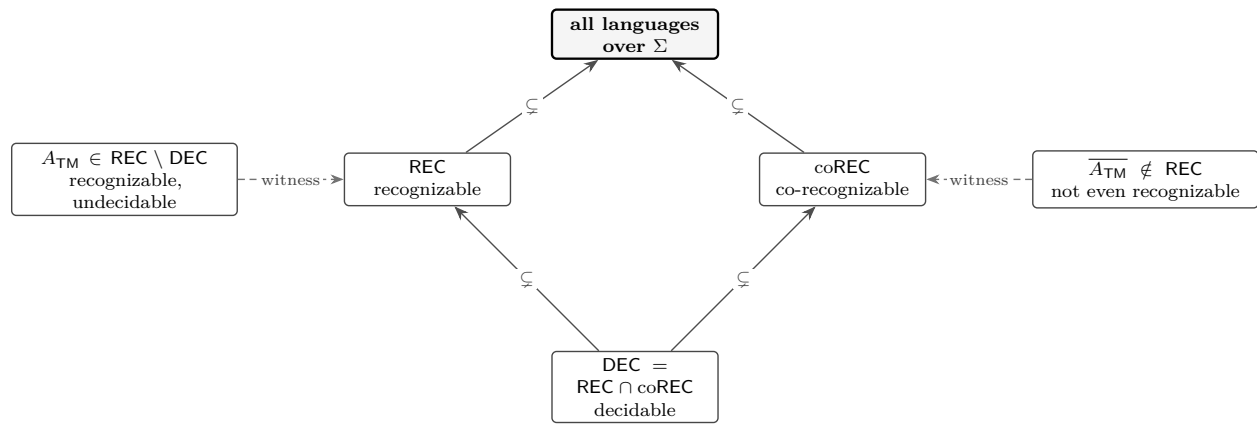
1. **Aim at the complement.** To show $\mathcal{L} \notin \text{REC}$, it is almost always easier to study $\overline{\mathcal{L}}$ and invoke Theorem 9.5.2.
2. **Find the recognizable partner.** Check whether $\overline{\mathcal{L}}$ is recognizable (often the universal machine or a simple search recognizes it). If $\overline{\mathcal{L}} \in \text{REC}$, then $\mathcal{L} \in \text{REC}$ *would* make $\mathcal{L}$ decidable.
3. **Cite an undecidability fact.** If $\overline{\mathcal{L}}$ is known to be undecidable (e.g. it *is* A_{TM} , or reduces from it), then $\overline{\mathcal{L}}$ cannot be both recognizable and co-recognizable, so the side that is *not* recognizable is your target.
4. **Name the contradiction.** Write: “if $\mathcal{L}$ were recognizable, then with $\overline{\mathcal{L}} \in \text{REC}$ already, $\overline{\mathcal{L}}$ would be decidable, contradicting its undecidability; hence $\mathcal{L} \notin \text{REC}$.”

Caution. Recognizable does not mean its complement is recognizable. For undecidable languages the two cannot both hold: $A_{\text{Tm}} \in \text{REC}$ while $\overline{A_{\text{Tm}}} \notin \text{REC}$. Only *decidable* languages are guaranteed to have a recognizable complement.

Caution. You cannot complement a recognizer by swapping accept and reject. That trick works for a *decider* (it always halts, so flipping a definite answer is fine). A recognizer may loop on the very inputs you wanted to accept after flipping, so the swapped machine accepts nothing on those inputs – it does not recognize the complement.

Caution. Do not run the two recognizers sequentially. If the parallel decider in Theorem 9.5.2 ran M_1 fully and only then M_2 , it would hang forever on any input where M_1 loops, never reaching M_2 . Dovetailing (one step of each per round) is what makes the construction halt on every input.

To close the chapter, a single labelled map gathers the classes and their witnesses. It is not the nested-region picture above but the *relationship* map: which class includes which (and whether the inclusion is proper), where the equality $\text{DEC} = \text{REC} \cap \text{coREC}$ sits, and which example pins down each strict boundary.



All four upward arrows are *proper* inclusions; A_{Tm} witnesses $\text{DEC} \subsetneq \text{REC}$ and $\overline{A_{\text{Tm}}}$ witnesses that REC is properly inside “all languages.”

Takeaway. The decidability landscape: decidable = recognizable *and* co-recognizable; A_{Tm} is recognizable but undecidable, and its complement is not even recognizable.

Exercises.

1. Prove the central characterization: a language $\mathcal{L}$ is decidable if and only if both $\mathcal{L}$ and its complement $\overline{\mathcal{L}}$ are Turing-recognizable. Describe the decider you build from the two recognizers, and name the technique that keeps it from looping.
2. Using the previous result and the undecidability of A_{Tm} , prove that $\overline{A_{\text{Tm}}}$ is *not* Turing-recognizable. State the one-line argument.
3. *Dovetailing.* Explain how to run two machines R_1, R_2 “in parallel” on the same input on a single Turing machine, so that if either accepts you find out. Why is round-robin simulation necessary rather than running R_1 to completion first?
4. Classify each language as decidable, recognizable-but-undecidable, co-recognizable-but-undecidable, or neither recognizable nor co-recognizable, with a one-line reason: (a) A_{DFA} ; (b) A_{Tm} ; (c) $\overline{A_{\text{Tm}}}$; (d) $\{\langle M \rangle \mid M \text{ halts on the blank tape}\}$.
5. Prove that the class of Turing-recognizable languages is closed under union and intersection, but give a reason (citing exercise 1) to expect it is *not* closed under complement.
6. Show there exist uncountably many Turing-unrecognizable languages over $\{0,1\}$. (Hint: there are uncountably many languages and only countably many recognizers; bound the recognizable ones.)
7. Suppose $\mathcal{L}$ is recognizable and $\mathcal{L} \leq_m \overline{\mathcal{L}}$ via a computable function f (i.e. $w \in \mathcal{L} \iff f(w) \in \overline{\mathcal{L}}$; see Definition 10.2.2). What can you conclude about $\mathcal{L}$? Prove it.

8. Let $\mathcal{L}$ be recognized by M and suppose additionally that M halts on *at least* every string *not* in $\mathcal{L}$. Show $\mathcal{L}$ is decidable.
9. *Spot the flaw.* A student says: “The complement of a recognizable language is recognizable, since I can just swap the accept and reject states of its recognizer.” Explain why swapping does not work for Turing machines (contrast with DFAs), and what goes wrong on looping inputs.
10. *Spot the flaw.* Another student concludes from “ A_{TM} is recognizable” that “ A_{TM} is co-recognizable too, by symmetry.” Pinpoint the false appeal to symmetry and connect it to exercise 2.
11. Give the map of the four classes – decidable inside (recognizable $\cap$ co-recognizable), with A_{TM} and $\overline{A_{\text{TM}}}$ placed in the proper regions – and write one sentence explaining why the decidable languages are exactly the overlap.

Chapter 10

Reducibility and Undecidability

10.1 Undecidability via Reductions

In the previous chapter we proved A_{TM} undecidable, and we did it the hard way – by diagonalization. This section develops the tool that does all the rest of the work: *reduction*. The one key idea is that to prove a new problem B undecidable we never argue from scratch; we instead show that an already-known undecidable problem A would become decidable if B were, and read off the contradiction. Every proof here has the same skeleton – assume a decider R for the target B , then build from it a decider S for A_{TM} (or another problem already known undecidable) – and the creative part is only the little machine S feeds to R . We make the pattern precise, run it four times (on HALT_{TM} , E_{TM} , $\text{REGULAR}_{\text{TM}}$, and EQ_{TM}), and then sharpen it with the *computation-history* technique, which lets a reduction inspect a TM’s entire run as a single string.

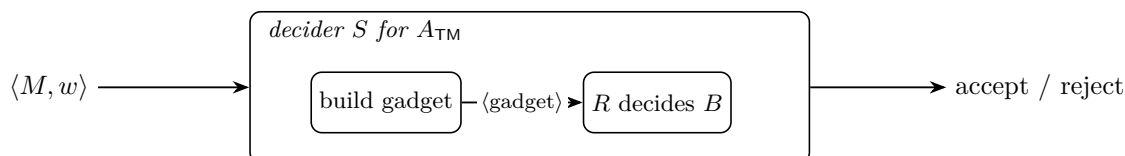
Definition 10.1.1 (Informal reduction). A problem A *reduces* to a problem B when any decider for B can be used as a subroutine to build a decider for A . Intuitively, A is “no harder than” B : a solution to B delivers a solution to A . (A formal and stricter version, mapping reducibility $A \leq_m B$, is the subject of the next section; here we use the relation only to transport undecidability.)

The whole method rests on the contrapositive of that sentence, which we state as the working principle of the chapter.

Proposition 10.1.2 (The reduction principle). *Suppose A reduces to B in the sense of Definition 10.1.1. If A is undecidable, then B is undecidable.*

Proof. We argue by contraposition. If B were decidable, some TM R would decide it; but a reduction of A to B turns R into a decider S for A , making A decidable. Hence “ B decidable” forces “ A decidable,” and contrapositively “ A undecidable” forces “ B undecidable.” $\square$

So nearly every undecidability proof in this section reduces $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM that accepts } w\}$ (known undecidable) to the target language. The shape never changes; only the gadget machine does. The schematic below is the picture to keep in mind: the assumed decider R for B sits *inside* the decider S we are building for A_{TM} .



Universal layout of a reduction $A_{\text{TM}} \rightarrow B$: S transforms $\langle M, w \rangle$ into the encoding of a gadget machine, hands it to the hypothetical decider R for B , and turns R ’s verdict into a verdict about whether M accepts w .

The halting problem

Theorem 10.1.3 (The halting problem is undecidable). *The language*

$$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on } w\}$$

is undecidable.

Proof. Assume for contradiction that a TM R decides $HALT_{TM}$. We build a decider S for A_{TM} :

$S =$ “On input $\langle M, w \rangle$:

1. Run R on $\langle M, w \rangle$.
2. If R rejects (so M loops on w), **reject**.
3. If R accepts (so M halts on w), simulate M on w until it halts.
4. If that simulation accepted, **accept**; if it rejected, **reject**.”

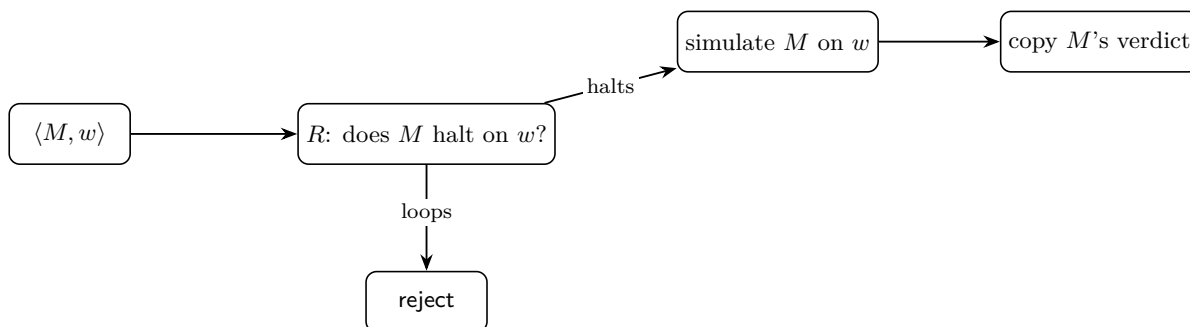
S always halts: step 1 halts because R is a decider, and the simulation in step 3 is reached only when R has *guaranteed* that M halts on w , so it cannot run forever. And S is correct: M accepts w exactly when M halts on w and that halt is in the accept state, which is precisely when S accepts. Thus S decides A_{TM} , contradicting the undecidability of A_{TM} . Therefore no such R exists and $HALT_{TM}$ is undecidable. $\square$

The single insight that makes this work is that knowing “ M halts” removes the only obstacle to simulating M : the risk of an infinite loop. We never could decide A_{TM} by raw simulation because of looping; the halting oracle deletes that risk.

Worked Example 10.1.4 (Why naive simulation fails but the halting oracle rescues it). Spell out, as a flow diagram, exactly where the danger of looping sits in deciding A_{TM} and how the $HALT_{TM}$ subroutine removes it.

Recognition. The target is $HALT_{TM}$ and the source is A_{TM} ; the trigger is “I could simulate M on w if only I were promised it stops.” Whenever a would-be decider stalls solely because of nontermination, a halting oracle is the natural fix.

Solution. The doomed plan is to simulate and hope. The diagram instead inserts R first, so simulation is entered only along the “halts” edge.



Which idea, and why? This is the reduction $A_{TM} \rightarrow HALT_{TM}$: the only reason raw simulation cannot decide A_{TM} is looping, so a decider for the *halting* question is exactly the missing subroutine, and the contradiction it produces proves $HALT_{TM}$ undecidable.

Check. Follow a looping pair $\langle M, w \rangle$: R reports “loops,” the right branch never runs, and S rejects – correct, since a looping M does not accept w . Follow an accepting pair: R reports “halts,” the simulation reaches accept, and S accepts.

Emptiness of a TM’s language

Theorem 10.1.5 (Emptiness is undecidable). *The language*

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$$

is undecidable.

Proof. Assume a TM R decides E_{TM} . The naive idea – run R on $\langle M \rangle$ itself – fails, because R rejecting $\langle M \rangle$ tells us only that M accepts *something*, not that it accepts our specific w . The fix is to feed R a *modified* machine whose language is empty unless M accepts w . Given $\langle M, w \rangle$, define the gadget

$M_w =$ “On input x :

1. If $x \neq w$, reject.
2. If $x = w$, run M on w and accept iff M accepts.”

The string w is hard-coded into M_w , and its description is computable from $\langle M, w \rangle$ by adding a few states that compare the input against w symbol by symbol. By design, M_w can accept only the single string w , and it accepts w iff M does. Hence

$$L(M_w) = \begin{cases} \{w\} & \text{if } M \text{ accepts } w, \\ \emptyset & \text{if not.} \end{cases}$$

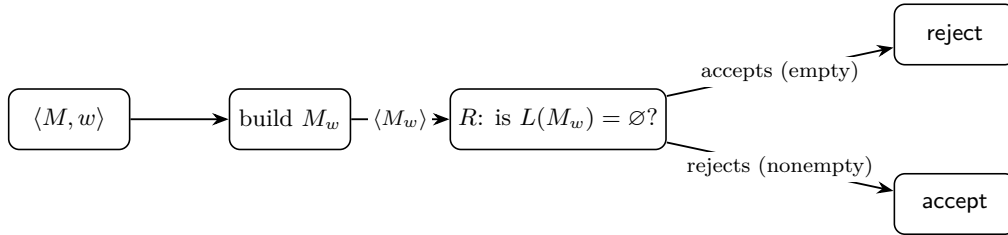
Now define $S =$ “On input $\langle M, w \rangle$: construct M_w ; run R on $\langle M_w \rangle$; if R accepts, **reject**, and if R rejects, **accept**.” Then S accepts $\langle M, w \rangle$ iff $L(M_w) \neq \emptyset$ iff M accepts w , so S decides A_{TM} – a contradiction. Therefore E_{TM} is undecidable. $\square$

Caution. The gadget M_w is *never executed* by the reduction; only its *description* is computed and handed to R . It is tempting to ask “but what if M_w loops?” – irrelevant, because we never run it. S runs only R (a decider) and the description-building routine (a finite text edit), so S always halts. Confusing “build the description of” with “run” is the single most common error in these proofs.

Worked Example 10.1.6 (The emptiness reduction, drawn as a data flow). Show how a single pair $\langle M, w \rangle$ travels through the E_{TM} reduction, and mark the polarity flip at the end.

Recognition. Source A_{TM} , target E_{TM} ; the trigger is “the target asks about the *whole* language, so engineer a machine whose whole language collapses to the one fact we care about.”

Solution. The reduction is a straight pipeline with one twist: R ’s answer is inverted, because “language is empty” is the *opposite* of “ M accepts w .”



Which idea, and why? We reduce A_{TM} to E_{TM} by a *language-engineering* gadget: M_w filters its input down to w , so its emptiness is a perfect proxy for “ M does not accept w ,” and the proxy is read off by inverting R .

Check. If M accepts w then $L(M_w) = \{w\} \neq \emptyset$, R rejects, S accepts – correct. If M does not accept w then $L(M_w) = \emptyset$, R accepts, S rejects – correct.

Regularity of a TM’s language

Theorem 10.1.7 (Regularity is undecidable). *The language*

$$\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is regular}\}$$

is undecidable.

Proof. Assume a TM R decides $\text{REGULAR}_{\text{TM}}$. Given $\langle M, w \rangle$ we build a gadget whose language is forced to be regular exactly when M accepts w . The trick is to pin its language to a known *nonregular* set $\{0^n 1^n \mid n \geq 0\}$ in the failing case and to all of Σ^* (regular) in the succeeding case. Define

$M_2 =$ “On input x :

1. If x has the form $0^n 1^n$ for some $n \geq 0$, **accept**.

2. Otherwise, run M on w and accept iff M accepts w .”

Trace the two cases. If M accepts w , then step 2 accepts *every* string that fails step 1, so $L(M_2) = \Sigma^*$, which is regular. If M does not accept w , step 2 never accepts, so $L(M_2) = \{0^n 1^n \mid n \geq 0\}$, which is not regular. Hence $L(M_2)$ is regular iff M accepts w . Now $S =$ “On input $\langle M, w \rangle$: construct M_2 ; run R on $\langle M_2 \rangle$; accept iff R accepts” decides A_{TM} , a contradiction. So $\text{REGULAR}_{\text{TM}}$ is undecidable. $\square$

Worked Example 10.1.8 (Reading the two cases of the regularity gadget). Tabulate $L(M_2)$ in each case and confirm that “regular” lines up with “ M accepts w .”

Recognition. Target $\text{REGULAR}_{\text{TM}}$, source A_{TM} ; the trigger is “the property is a *class membership* (regular vs. not), so build a machine that lands in the easy class only on a yes-instance.”

Solution. The gadget accepts the nonregular template language for free and tops it up to everything exactly when M accepts w .

case	$L(M_2)$	regular?
M accepts w	Σ^*	yes
M does not accept w	$\{0^n 1^n \mid n \geq 0\}$	no

Which idea, and why? To attack a property that is “membership in a language class,” build a gadget whose language is forced into the class on yes-instances and out of it on no-instances; here “regular” toggles with “ M accepts w ,” so R ’s verdict is A_{TM} ’s verdict directly (no inversion).

Check. The string $0011 = 0^2 1^2$ is accepted by M_2 in *both* cases (it fits the template), so it alone cannot tell the cases apart – the difference shows only on strings *outside* the template, e.g. 10 is accepted iff M accepts w .

Equivalence of two TMs, by reducing from emptiness

Not every reduction must start from A_{TM} . Once E_{TM} is known undecidable, it can be the source, and for equivalence it is the convenient one: an emptiness question is a special equivalence question.

Theorem 10.1.9 (Equivalence is undecidable). *The language*

$$EQ_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs and } L(M_1) = L(M_2)\}$$

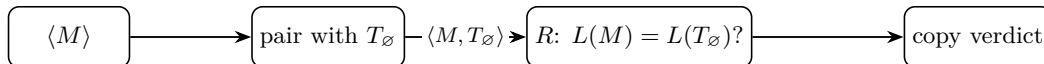
is undecidable.

Proof. We reduce E_{TM} to EQ_{TM} . Let $T_\emptyset$ be a fixed TM that rejects every input, so $L(T_\emptyset) = \emptyset$. Assume R decides EQ_{TM} and set $S =$ “On input $\langle M \rangle$: run R on $\langle M, T_\emptyset \rangle$ and copy its answer.” Then S accepts iff $L(M) = L(T_\emptyset) = \emptyset$, i.e. iff $\langle M \rangle \in E_{\text{TM}}$. So S decides E_{TM} , which is undecidable by Theorem 10.1.5; hence EQ_{TM} is undecidable. $\square$

Worked Example 10.1.10 (Emptiness is the special case “equal to the empty machine”). Explain why fixing one argument of EQ_{TM} to $T_\emptyset$ yields exactly E_{TM} , and draw the one-box reduction.

Recognition. Source E_{TM} , target EQ_{TM} ; the trigger is “the target subsumes the source by holding one input fixed.” When a binary problem specializes to a known-hard unary problem, reduce by freezing an argument.

Solution. No gadget construction is needed at all – the reduction just pairs M with the constant empty machine.



Which idea, and why? $E_{\text{TM}} \leq_m EQ_{\text{TM}}$ by argument-freezing: $L(M) = \emptyset$ is literally “ $L(M) = L(T_\emptyset)$,” so equivalence with the fixed empty machine is emptiness, and the source’s hardness transfers with no inversion and no inner simulation.

Check. If $L(M) = \emptyset$ the two machines agree (both accept nothing), R accepts, S accepts; if $L(M) \neq \emptyset$ they differ on some string, R rejects, S rejects – matching E_{TM} exactly.

Reductions via computation histories

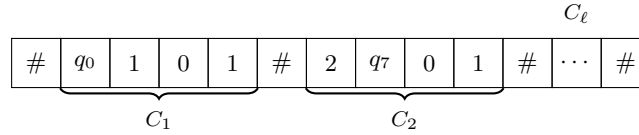
The gadgets above only had to *run* M on w inside themselves. Some targets cannot do that – they need to *verify* a computation without simulating it, by examining the whole run laid out as one string. That object is the computation history.

Definition 10.1.11 (Computation history). Let M be a TM and w an input. An *accepting computation history* for M on w is a finite sequence of configurations $C_1, C_2, \dots, C_\ell$ in which C_1 is the start configuration q_0w , each C_{i+1} follows from C_i by one step of M (written $C_i \vdash C_{i+1}$), and C_ℓ is an accepting configuration. A *rejecting* computation history is defined identically with C_ℓ rejecting. If M does not halt on w , no such (finite) history exists.

We write a history as one string, the configurations separated by a delimiter $\#$:

$$\# C_1 \# C_2 \# \dots \# C_\ell \#.$$

The crucial feature is *local checkability*: a string is an accepting history iff it starts right ($C_1 = q_0w$), ends right (C_ℓ accepting), and every adjacent pair C_i, C_{i+1} differs only in the few cells around the head, exactly as M 's transition function dictates. None of these checks needs to run M ; each is a finite, mechanical comparison of neighboring blocks.



A computation history as a single tape string: $\#$ -delimited configurations, each a snapshot uqv . Adjacent blocks must agree except near the head, where M 's transition rule decides the change.

This local structure is exactly what a memory-bounded checker can verify, so computation histories let us reduce A_{TM} to problems about *simpler* machines. The classic target is the emptiness problem for a linear bounded automaton (LBA), a TM whose head may never leave the input region.

Theorem 10.1.12 (LBA emptiness is undecidable). *The language*

$$E_{\text{LBA}} = \{\langle B \rangle \mid B \text{ is an LBA and } L(B) = \emptyset\}$$

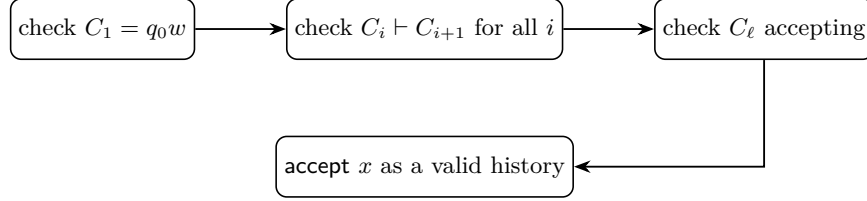
is undecidable.

Proof sketch. Reduce A_{TM} to E_{LBA} . From $\langle M, w \rangle$ build an LBA B whose input is a candidate string x and which accepts x iff x is an *accepting computation history* for M on w . To check this, B verifies the three local conditions of Definition 10.1.11: that x begins with $\#q_0w\#$, ends with a $\#$ -block containing the accept state, and that each consecutive pair of blocks is consistent with M 's transition function. Every check only zig-zags between adjacent blocks of the *already written* input, so B never needs space beyond its input – it is a genuine LBA. Now $L(B)$ contains a string iff M has an accepting history on w , i.e. iff M accepts w . So a decider R for E_{LBA} gives a decider for A_{TM} via “build B ; run R on $\langle B \rangle$; accept iff R rejects,” a contradiction. $\square$

Worked Example 10.1.13 (Why the checker fits inside an LBA). The reduction of Theorem 10.1.12 hinges on the verifier B using no extra tape. Say precisely which work B does and why it stays within the input region.

Recognition. The target is an LBA problem, so the trigger is “the gadget must *verify*, not *simulate*, and verification of a history is local.” Whenever a problem forbids unbounded memory, look for a property checkable by comparing neighboring chunks in place.

Solution. B makes three passes over its input $x = \#C_1\#\dots\#C_\ell\#$, never writing outside it.



All three checks read only the given input x ; the head zig-zags between adjacent blocks, marking positions with dots, so no cell beyond x is ever needed.

Which idea, and why? Computation-history reduction: $A_{\text{TM}} \rightarrow E_{\text{LBA}}$ works because an accepting run is a *string* whose validity is locally checkable, and local checking needs only the input region – exactly the resource an LBA has. Inverting R ’s answer (empty $\Rightarrow$ no accepting run) reads off “ M does not accept w .”

Check. If M accepts w , its unique accepting run is one string B accepts, so $L(B) \neq \emptyset$; R rejects $\langle B \rangle$ and S accepts $\langle M, w \rangle$. If M does not accept w , no accepting history exists, $L(B) = \emptyset$, R accepts, S rejects.

Method (How to prove a TM-property language B undecidable).

1. **Pick the source.** Default to A_{TM} . If B is an *equivalence* or two-machine problem, E_{TM} is often easier; if B forbids unbounded memory, plan a computation-history reduction.
2. **Design one gadget machine** from the source input. The gadget should make the answer to B *toggle* with the source’s answer. Common gadgets: “accept only w , and only if M does” (for E_{TM}); “accept a fixed nonregular set, plus everything iff M accepts w ” (for class membership); “accept exactly the accepting histories of M on w ” (for memory-bounded targets).
3. **Build, do not run.** Your decider S only *computes the description* of the gadget (a finite edit of $\langle M \rangle$ and w) and then calls the assumed decider R for B . S itself never simulates the gadget, so S always halts.
4. **Fix the polarity.** Decide whether the yes-instance of B matches “ M accepts w ” or its negation, and accept or invert R ’s answer accordingly. (E_{TM} and E_{LBA} invert; $\text{REGULAR}_{\text{TM}}$ does not.)
5. **Read off the contradiction.** S now decides the (undecidable) source, so R cannot exist and B is undecidable.

Caution. Polarity errors are the most frequent slip. “Accept iff R accepts” is correct only when B ’s yes-instances coincide with “ M accepts w .” For emptiness-style targets the coincidence is with “ M does *not* accept w ,” so you must *invert* R ’s answer. Always re-derive the truth table of the gadget’s language against the two cases of M on w before wiring the final step.

Caution. A reduction transports undecidability in one direction only. Showing “ A reduces to B ” with A undecidable proves B undecidable – never the reverse. Do not conclude anything about A from B ’s undecidability, and do not reduce *from* the problem you are trying to indict: to prove B undecidable you reduce a known-undecidable source *to* B , putting the assumed decider for B inside the decider you build for the source.

Exercises.

1. State, in one sentence each, what it means for problem A to reduce to problem B , and why a reduction of an undecidable A to B proves B undecidable. Then explain why the implication does *not* run the other way.
2. Let $\text{ACCEPT2} = \{\langle M \rangle \mid M \text{ is a TM that accepts at least two distinct strings}\}$. Give a reduction from A_{TM} proving ACCEPT2 undecidable. Describe the gadget machine M'_2 you feed to the assumed decider and state $L(M'_2)$ in each of the two cases.
3. Consider $\text{ACCEPTS-}\varepsilon = \{\langle M \rangle \mid M \text{ is a TM and } M \text{ accepts the empty string } \varepsilon\}$. Prove it undecidable by reducing A_{TM} to it. (Hint: build a gadget that ignores its own input and runs M on w .)
4. In the proof that E_{TM} is undecidable, the gadget M_w first checks whether its input equals w . Explain in two or three sentences why the reduction would *break* if instead M_w simply ran M on its own input x (ignoring w). What would $L(M_w)$ then fail to capture?

5. *Polarity drill.* For each target language below, say whether a reduction from A_{TM} that toggles the gadget's property with " M accepts w " should *copy* or *invert* the assumed decider's answer, and justify in one line: (a) E_{TM} ; (b) $\text{REGULAR}_{\text{TM}}$; (c) $\text{NONEMPTY}_{\text{TM}} = \{\langle M \rangle \mid L(M) \neq \emptyset\}$.
6. Let $\text{CFL}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is a context-free language}\}$. Adapt the regularity proof to show CFL_{TM} undecidable. Which fixed language plays the role that $\{0^n 1^n \mid n \geq 0\}$ played there, and what must you know about it? Explain.
7. Prove that HALT_{TM} and E_{TM} are each undecidable using the *other* as the known-hard source instead of A_{TM} , or explain precisely what goes wrong and why A_{TM} is the natural source. (Pick one of the two and argue it carefully.)
8. Show that EQ_{TM} remains undecidable when we additionally require the two input machines to share the same input alphabet. Does fixing the alphabet weaken the reduction of Theorem 10.1.9? Explain.
9. Write out a complete computation history, as a single $\#$ -delimited string, for a small TM of your own design: take M over $\Sigma = \{a\}$ that on input aa moves right twice and then accepts, and exhibit $\#C_1\#C_2\#\cdots\#C_\ell\#$. Mark which adjacent pair encodes each transition step.
10. *Find the bug.* A student "proves" A_{TM} decidable as follows: "Given $\langle M, w \rangle$, build the gadget M_w from the E_{TM} proof, then *run* M_w on input w ; if it accepts, accept, else reject." Identify the fatal flaw, naming the exact step where the argument stops being an algorithm.
11. *Find the bug.* Someone claims: " $\text{REGULAR}_{\text{TM}}$ is decidable because every TM either halts or loops, and in each case its language is regular." Pinpoint the false statement and give a single TM whose language is not regular, refuting it.
12. Let $\text{INFINITE}_{\text{TM}} = \{\langle M \rangle \mid L(M) \text{ is infinite}\}$. Give a reduction from A_{TM} proving it undecidable. Design the gadget so that $L(M_2)$ is infinite iff M accepts w , and state $L(M_2)$ in both cases.
13. Using the computation-history idea, sketch (in words, no full construction needed) why one can build, from $\langle M, w \rangle$, a machine that accepts a *nonempty* language iff M accepts w . Then explain in one sentence how this single idea underlies both the E_{TM} reduction and the E_{LBA} reduction, despite their different gadgets.

10.2 Mapping Reducibility

In the previous section we proved undecidability by an informal slogan: "a decider for B would give a decider for A , but A is undecidable." That argument hides a precise object – a single computable function that turns each question about A into an equivalent question about B – and the one key idea of this section is to bring that function into the open. Once the reduction is a concrete map f with the property $w \in A \iff f(w) \in B$, the undecidability transfer becomes a one-line corollary, the relation $A \leq_m B$ becomes transitive, and – crucially for the next chapter – it also transfers *recognizability*, which the informal version could not. We make f precise, prove its transfer theorems, and read off several reductions as explicit functions.

Computable functions. A Turing machine need not only answer yes/no; started with a string on its tape it can halt with a different string written there, and that output string is the value of a function of the input. This is the notion of computation we need to define a reduction.

Definition 10.2.1 (Computable function). A function $f: \Sigma^* \rightarrow \Sigma^*$ is *computable* if some Turing machine F , on every input w , halts with exactly $f(w)$ on its tape (and nothing else). We call F a *transducer* for f .

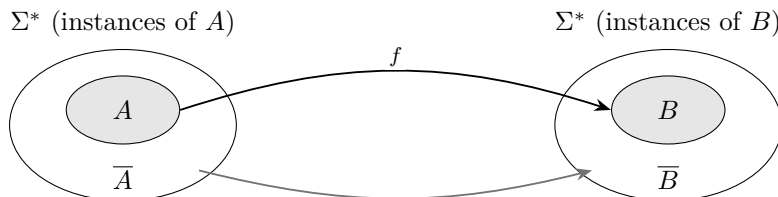
The familiar arithmetic operations are computable once integers are written as strings: a TM can read $\langle m, n \rangle$ and leave $m + n$ on the tape. More important for us, the *input and output can be machine descriptions*. A transducer may read $\langle M \rangle$ and write $\langle M' \rangle$, where M' is built from M by a fixed editing rule – adding states, rewiring transitions, hard-wiring a string into the program. The transducer never *runs* M or M' ; it only manipulates their descriptions as text, the way a compiler rewrites source code without executing it. This "machine in, edited machine out" pattern is the engine of every reduction below.

Definition 10.2.2 (Mapping reducibility). A language A is *mapping reducible* to a language B , written $A \leq_m B$, if there is a computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that for every $w \in \Sigma^*$,

$$w \in A \iff f(w) \in B.$$

The function f is called the *reduction* from A to B .

Read the biconditional carefully: f must send *every* member of A into B and every non-member of A outside B . It is a single function handling both directions at once; a map that only guarantees “ $w \in A \implies f(w) \in B$ ” is not a mapping reduction. The picture below shows f as an arrow between the two universes of strings, carrying the inside of A to inside B and the outside to outside.



A mapping reduction converts membership testing in A into membership testing in B : to decide whether $w \in A$, compute $f(w)$ and ask whether $f(w) \in B$. The next theorem is exactly this sentence turned into a machine.

Theorem 10.2.3 (Decidability transfers downward). *If $A \leq_m B$ and B is decidable, then A is decidable.*

Proof. Let M be a decider for B and let f be the reduction, computed by a transducer F . Build a TM N :

N = “on input w : compute $f(w)$ using F ; run M on $f(w)$ and copy its answer.”

Both F and M halt on every input, so N halts on every input – it is a decider. And N accepts w iff M accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$, the last step by Definition 10.2.2. Hence N decides A . $\square$

Corollary 10.2.4 (Undecidability transfers upward). *If $A \leq_m B$ and A is undecidable, then B is undecidable.*

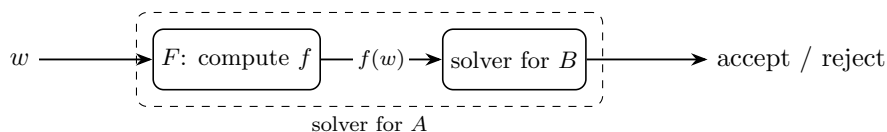
Proof. Contrapositive of Theorem 10.2.3: if B were decidable, then A would be decidable. $\square$

Corollary 10.2.4 is the workhorse. To prove a target B undecidable we exhibit a mapping reduction $A \leq_m B$ from a known-undecidable A (usually A_{TM}). The same transducer, read with recognizers in place of deciders, transfers recognizability; we record this now and use it heavily in Section 10.3.

Theorem 10.2.5 (Recognizability transfers downward). *If $A \leq_m B$ and B is Turing-recognizable, then A is Turing-recognizable. Equivalently (contrapositive): if $A \leq_m B$ and A is not recognizable, then B is not recognizable.*

Proof. Repeat the proof of Theorem 10.2.3 with M a *recognizer* for B . The machine N computes $f(w)$ (which always halts) and then runs the recognizer M on $f(w)$. Now N accepts w iff M accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$; on inputs $w \notin A$ the machine N may loop, exactly as a recognizer is permitted to. So N recognizes A . $\square$

The schematic for both transfer theorems is the same composed machine: the transducer feeds the solver for B , and the verdict comes straight back.



A symmetry to watch. The reduction f also reduces the complements: $A \leq_m B$ says $w \in A \iff f(w) \in B$, and negating both sides gives $w \in \overline{A} \iff f(w) \in \overline{B}$, so the *same* f witnesses $\overline{A} \leq_m \overline{B}$. This is why mapping reducibility, unlike the informal Turing-style reduction, is sensitive to complementation: it ties the two complements together. The classic application reduces $\overline{A_{TM}}$ (which is not recognizable) to a target to prove the target unrecognizable.

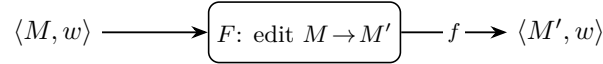
Worked Example 10.2.6 ($A_{TM} \leq_m HALT_{TM}$ as an explicit function). Exhibit a computable f with $\langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle) \in HALT_{TM}$, recovering Theorem 10.1.3 as a mapping reduction.

Recognition. The informal proof of Section 10.1 turned a halting decider into an acceptance decider; to package it as a *map* we must output a single instance $\langle M', w \rangle$ of $HALT_{TM}$ whose halting behaviour mirrors M 's *accepting* behaviour. The trigger is “make rejecting look like not halting,” so that halting on w becomes the same event as accepting w .

Solution. The transducer edits M into M' that loops wherever M rejects:

$M' = \text{“on input } x: \text{ run } M \text{ on } x; \text{ if } M \text{ accepts, accept; if } M \text{ rejects, enter an infinite loop.”}$

Set $f(\langle M, w \rangle) = \langle M', w \rangle$; malformed inputs map to a fixed string outside $HALT_{TM}$. The map f is computable – building M' only redirects M 's reject transitions into a new looping state, pure text editing.



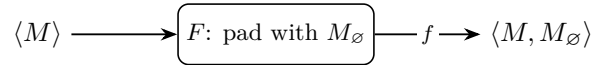
Which idea, and why? A mapping reduction is the construction-only half of the informal argument: emit the edited instance and stop, never running it. Here M' halts on w exactly when M accepts w , so f satisfies the biconditional of Definition 10.2.2.

Check. If $\langle M, w \rangle \in A_{TM}$ then M accepts w , so M' accepts (hence halts) on w , giving $\langle M', w \rangle \in HALT_{TM}$. If $\langle M, w \rangle \notin A_{TM}$ then M rejects or loops on w ; either way M' fails to halt on w , so $\langle M', w \rangle \notin HALT_{TM}$. Both directions hold.

Worked Example 10.2.7 ($E_{TM} \leq_m EQ_{TM}$ as an explicit function). Recall $E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$ and $EQ_{TM} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$. Give a mapping reduction $E_{TM} \leq_m EQ_{TM}$.

Recognition. “ $L(M)$ is empty” is the special case of “ $L(M_1) = L(M_2)$ ” in which the second machine is pinned to the empty language. The trigger “one input becomes a pair, with one slot fixed” says: pad $\langle M \rangle$ with a fixed companion that rejects everything.

Solution. Let $M_\emptyset$ be the fixed TM that rejects every input, so $L(M_\emptyset) = \emptyset$. Define $f(\langle M \rangle) = \langle M, M_\emptyset \rangle$, a pure padding step, hence computable.



Which idea, and why? To realize problem A as a special case of problem B , the reduction fixes the “free” parameters of B to constants – here the second machine – so that B asks exactly the A -question.

Check. $\langle M \rangle \in E_{TM} \iff L(M) = \emptyset \iff L(M) = L(M_\emptyset) \iff \langle M, M_\emptyset \rangle \in EQ_{TM}$. The biconditional holds, so EQ_{TM} inherits undecidability from E_{TM} by Corollary 10.2.4.

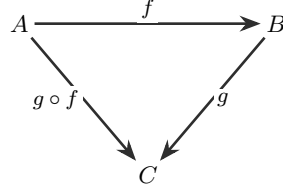
Theorem 10.2.8 (Transitivity). *If $A \leq_m B$ and $B \leq_m C$, then $A \leq_m C$.*

Proof. Let f reduce A to B and g reduce B to C , computed by transducers F and G . The composition $h = g \circ f$ is computed by running F then G , so h is computable. For every w ,

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C \iff h(w) \in C,$$

using the defining biconditionals of f and g in turn. Thus h reduces A to C . □

Transitivity is a directed triangle: the two given reductions are the legs, and their composite is the hypotenuse, every arrow pointing from the no-harder source into the at-least-as-hard target.



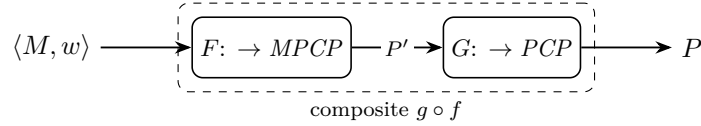
membership preserved along the chain

Mapping reductions compose: if f reduces A to B and g reduces B to C , then $g(f(\cdot))$ reduces A to C , so hardness travels along directed paths. *Check.* $g \circ f$ is computable and $w \in A \iff f(w) \in B \iff g(f(w)) \in C$, so $A \leq_m C$.

Worked Example 10.2.9 (Chaining reductions with transitivity). Suppose you have already built $A_{\text{TM}} \leq_m \text{MPCP}$ and $\text{MPCP} \leq_m \text{PCP}$. Conclude $A_{\text{TM}} \leq_m \text{PCP}$ and read off what it proves.

Recognition. Two reductions sharing the middle problem MPCP are exactly the hypotheses of Theorem 10.2.8; the trigger “the output type of one reduction is the input type of the next” lets us compose their transducers.

Solution. Compose the maps: feed $\langle M, w \rangle$ through the first transducer to get an MPCP instance, then through the second to get a PCP instance. The composite is one computable function $A_{\text{TM}} \leq_m \text{PCP}$.



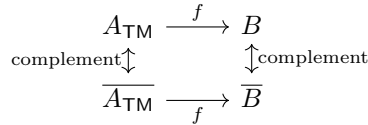
Which idea, and why? Transitivity (Theorem 10.2.8) lets undecidability hop across a chain: since A_{TM} is undecidable and $A_{\text{TM}} \leq_m \text{PCP}$, Corollary 10.2.4 makes PCP undecidable, with no need to reduce A_{TM} to PCP directly.

Check. Membership is preserved at each link: $\langle M, w \rangle \in A_{\text{TM}} \iff P' \in \text{MPCP} \iff P \in \text{PCP}$. The end-to-end biconditional is the conjunction of the two.

Worked Example 10.2.10 (Using the complement symmetry to prove unrecognizability). You are told A_{TM} is not Turing-recognizable. Show that any target B with $A_{\text{TM}} \leq_m B$ is not co-recognizable, i.e. $\overline{B}$ is not recognizable.

Recognition. The cue is “not recognizable,” which Theorem 10.2.5 handles – but only through a reduction *into* the non-recognizable language. We must route $\overline{A_{\text{TM}}}$ to $\overline{B}$, and the complement symmetry of mapping reductions does exactly that.

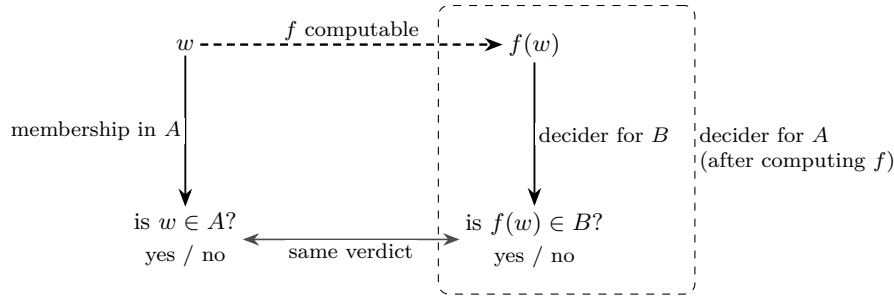
Solution. From $A_{\text{TM}} \leq_m B$ the same function gives $\overline{A_{\text{TM}}} \leq_m \overline{B}$. Now apply the contrapositive half of Theorem 10.2.5 with $A = \overline{A_{\text{TM}}}$ (not recognizable) and the target $\overline{B}$.



Which idea, and why? Because $w \in A \iff f(w) \in B$ negates to $w \in \overline{A} \iff f(w) \in \overline{B}$, one reduction simultaneously relates the languages and their complements; unrecognizability of $\overline{A_{\text{TM}}}$ then flows to $\overline{B}$ along the bottom arrow.

Check. If $\overline{B}$ were recognizable, Theorem 10.2.5 applied to $\overline{A_{\text{TM}}} \leq_m \overline{B}$ would make $\overline{A_{\text{TM}}}$ recognizable – false. So $\overline{B}$ is not recognizable.

Where mapping reducibility sits. The transfer theorems place $\leq_m$ as a one-way “difficulty” order: an arrow $A \leq_m B$ certifies A is no harder than B for decidability and recognizability alike. The cleanest way to see why is the single square the whole definition draws. The top edge is the computable reduction $w \mapsto f(w)$; the two vertical edges are the membership tests “is $w \in A$?” and “is $f(w) \in B$?”. Definition 10.2.2 says exactly that this square *commutes*: the two routes from a string to a yes/no verdict always agree.



A mapping reduction $A \leq_m B$ is the square that commutes iff $(w \in A) \iff (f(w) \in B)$. Feeding $f(w)$ to a decider for B (the dashed box: first compute f , then decide B) decides A – so B decidable and $A \leq_m B$ give A decidable; contrapositively, A undecidable forces B undecidable. *Check.* f is computable and $w \in A \iff f(w) \in B$, so “decide B on $f(w)$ ” decides A ; the contrapositive is how undecidability spreads from the known-hard A to the target B . The same square, read with recognizers instead of deciders, collects all four transfer implications under the standing hypothesis $A \leq_m B$:

$$B \text{ decidable} \implies A \text{ decidable} \quad (\text{contra}) \quad A \text{ undecid.} \implies B \text{ undecid.}$$

$$B \text{ recogniz.} \implies A \text{ recogniz.} \quad (\text{contra}) \quad A \text{ unrecog.} \implies B \text{ unrecog.}$$

all four under the standing hypothesis $A \leq_m B$.

Method (How to present a reduction as a mapping reduction $A \leq_m B$).

1. **Fix the instance types.** Write down what an instance of A looks like (e.g. $\langle M, w \rangle$) and what an instance of B looks like (e.g. $\langle M' \rangle$ or a pair). Your f maps the first kind of string to the second.
2. **Describe the transducer F .** Give the editing rule that builds the output *description* from the input description. Use only operations a TM can do to text: add states, hard-wire a string, splice two machines, redirect transitions. Never “run” the input machine inside F .
3. **State f explicitly**, including a default output (some fixed string outside B , or inside B if malformed strings belong to A , as for $\overline{A_{\text{TM}}}$) for malformed inputs.
4. **Prove the biconditional** $w \in A \iff f(w) \in B$, both directions. This is where the construction earns its keep; the $\implies$ direction alone is not enough.
5. **Invoke a transfer theorem.** For undecidability cite Corollary 10.2.4; for unrecognizability arrange the reduction so the non-recognizable language is on the *left* and cite Theorem 10.2.5.

Caution. A mapping reduction is a *total, halting* function that produces an instance – it is not a decider and must never run the machine it builds. Writing “ $f(\langle M, w \rangle)$: run M on w ; if it accepts, output ...” is a category error: f would then fail to halt whenever M loops, so it would not be a computable function at all. The transducer only *edits* descriptions; all running happens later, inside the assumed solver for B .

Caution. The defining condition is a *biconditional*. A function with $w \in A \implies f(w) \in B$ but not the converse is useless: it could map some $w \notin A$ into B , and then a decider for B would wrongly accept w . Always check the contrapositive direction $w \notin A \implies f(w) \notin B$ separately.

Caution. Direction matters for unrecognizability. Theorem 10.2.5 pushes non-recognizability from the *source* A to the *target* B along $A \leq_m B$. To prove B unrecognizable you must reduce a known non-recognizable language (typically $\overline{A_{\text{TM}}}$) to B , i.e. show $\overline{A_{\text{TM}}} \leq_m B$ – not $B \leq_m \overline{A_{\text{TM}}}$. Reducing in the wrong direction proves nothing.

Exercises.

1. Let $f: \Sigma^* \rightarrow \Sigma^*$ be defined by $f(w) = ww$ (the string concatenated with itself). Prove that f is a computable function by describing a transducer for it at the level of “copy, then append.”
2. Let $A = \{0^n \mid n \geq 0\}$ and $B = \{w \mid w \text{ contains the symbol } 1\}$ over $\Sigma = \{0, 1\}$. Give an explicit computable f witnessing $A \leq_m B$, and verify the biconditional $w \in A \iff f(w) \in B$ in both directions. (Hint: every member of A is itself a non-member of B and vice versa, so f cannot be the identity.)

3. Show directly from Definition 10.2.2 that every language A satisfies $A \leq_m A$ (reflexivity), and combine this with Theorem 10.2.8 to argue that $\leq_m$ is a preorder.
4. Suppose $A \leq_m B$ via a reduction f , and that A is undecidable. A classmate concludes “therefore f is not computable.” Explain precisely why this conclusion is wrong.
5. Prove that if $A \leq_m B$ and B is decidable, then $\overline{A}$ is also decidable. Do it two ways: (a) directly from Theorem 10.2.3, and (b) using the complement symmetry $\overline{A} \leq_m \overline{B}$.
6. Let C be any decidable language with $C \neq \emptyset$ and $C \neq \Sigma^*$. Show that every decidable language A satisfies $A \leq_m C$. (Hint: a transducer may run a decider for A to completion, then output a fixed element of C or of $\overline{C}$.) Why does the same argument fail when A is merely recognizable?
7. Recall $E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$. Construct an explicit mapping reduction $A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$: give the transducer that, from $\langle M, w \rangle$, outputs a machine $\langle M_1 \rangle$ with $L(M_1) \neq \emptyset \iff M$ accepts w . Verify the biconditional.
8. Find the bug in this purported mapping reduction $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$:

$$f(\langle M, w \rangle) = \begin{cases} \langle M, w \rangle & \text{if } M \text{ accepts } w, \\ \langle M_{\text{loop}} \rangle & \text{otherwise,} \end{cases}$$

where M_{loop} loops on every input. State which clause of Definition 10.2.1 or 10.2.2 is violated, and why the alleged reduction is not even a function a TM can compute.

9. Let $A \leq_m B$ via f and suppose additionally that f is a bijection whose inverse is also computable. Prove that then $B \leq_m A$ as well, so A and B are “equally hard” for both transfer theorems. Give the reduction in the reverse direction explicitly.
10. True or false, with justification: if $A \leq_m B$ and B is recognizable, then A is decidable. If false, state the strongest valid conclusion about A that the hypotheses support.
11. Suppose f reduces A to B and g reduces B to A (so each problem reduces to the other). Does it follow that A is decidable? Does it follow that A and B have the same decidability status? Justify each answer.
12. Let $J = \{0x \mid x \in A_{\text{TM}}\} \cup \{1y \mid y \in \overline{A_{\text{TM}}}\}$. Exhibit a mapping reduction $A_{\text{TM}} \leq_m J$ and a mapping reduction $\overline{A_{\text{TM}}} \leq_m J$, and explain in one sentence what these two reductions together say about whether J is recognizable or co-recognizable.

10.3 Proving Undecidability and Unrecognizability with $\leq_m$

Section 10.2 turned the informal “reduce A to B ” slogan into a precise relation $A \leq_m B$: a single computable function f with $w \in A \iff f(w) \in B$, together with the two transfer theorems — if B is decidable so is A , and if B is Turing-recognizable so is A . This section is the practical payoff. We turn those theorems into two reusable *templates* — one for proving a language undecidable, one for proving it *unrecognizable* — and then run them on a parade of concrete problems about Turing machines. The single key idea is contrapositive bookkeeping: to push hardness *onto* B you exhibit a reduction *from* a known-hard language *into* B , and the direction of the arrow is everything.

The two templates

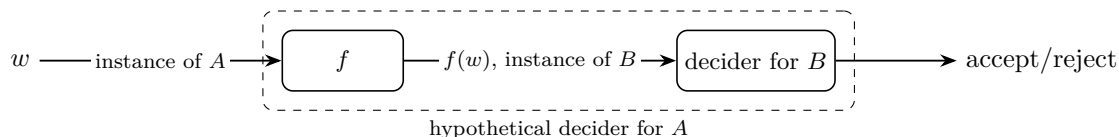
Recall the master tool from Section 10.2 in the form we actually use it. Its proof is the contrapositive of the transfer theorems, so we restate it as the working lemma of this section.

Lemma 10.3.1 (Hardness transfer). *Let $A \leq_m B$. Then:*

1. *if A is undecidable, then B is undecidable;*
2. *if A is not Turing-recognizable, then B is not Turing-recognizable.*

Proof. For (1): if B were decidable then $A \leq_m B$ would make A decidable, by the decidability-transfer theorem of Section 10.2 — contradiction. For (2): if B were recognizable then $A \leq_m B$ would make A recognizable, by the recognizability-transfer theorem — contradiction. In both cases the reduction carries the easiness of B backward onto A , so the hardness of A must already live in B . $\square$

The lemma reads off the page as a schematic. A reduction f is a machine that rewrites an instance w of A into an instance $f(w)$ of B so that membership is preserved in both directions; a hypothetical decider for B then composes with f to decide A .



So the recipe never builds a decider — it assumes one for B , wraps it in f , and derives the forbidden decider for A . Two choices of the source language A do almost all the work.

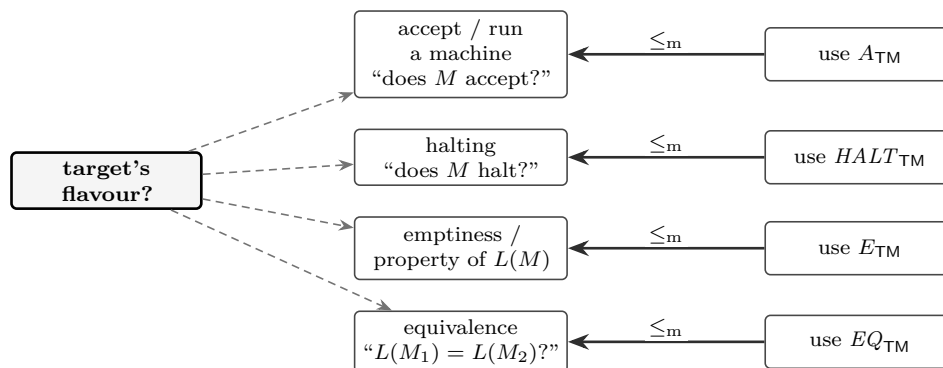
Method (Template U — proving B is undecidable).

1. **Pick a known-undecidable source**, usually $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$.
2. **Describe a computable f** that maps $\langle M, w \rangle$ to an instance of B . Almost always f *constructs the description of a new machine M'* (or a pair of machines, or a grammar) that bakes “ M accepts w ” into its behaviour, and outputs that description.
3. **Prove the biconditional** $\langle M, w \rangle \in A_{\text{TM}} \iff f(\langle M, w \rangle) \in B$, i.e. M accepts w exactly when the constructed object lands in B . Argue both directions.
4. **Confirm f is computable**: a TM can write down M' from $\langle M, w \rangle$ by copying M and adding finitely many fixed states. It does *not* run M' .
5. **Conclude** $A_{\text{TM}} \leq_m B$, so by Lemma 10.3.1(1) B is undecidable.

Method (Template N — proving B is not Turing-recognizable).

1. **Pick the canonical non-recognizable source** $\overline{A_{\text{TM}}}$, which is not Turing-recognizable (it is co-recognizable but not recognizable).
2. **Give a computable f** with $w \in \overline{A_{\text{TM}}} \iff f(w) \in B$. By the complementation identity below it is equivalent, and usually easier, to build f with $\langle M, w \rangle \in A_{\text{TM}} \iff f(\langle M, w \rangle) \in \overline{B}$ — that is, “ M accepts w ” should make $f(\langle M, w \rangle)$ land *outside* B .
3. **Conclude** $\overline{A_{\text{TM}}} \leq_m B$, so by Lemma 10.3.1(2) B is not recognizable.

Before running the templates, one choice has to be made every time: *which* known-undecidable problem to reduce from. The target’s flavour decides it – match the shape of the question B asks to the seed that already asks it.



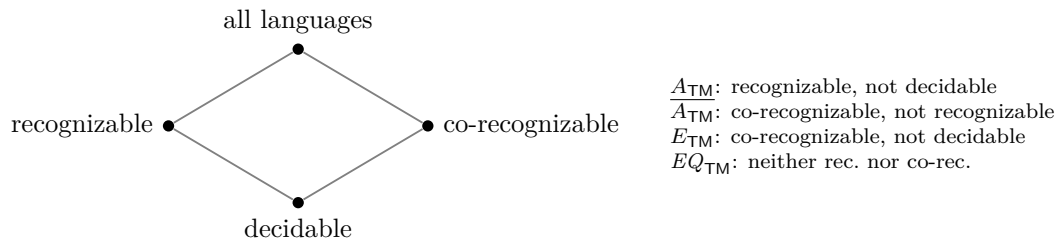
Picking the source of an undecidability reduction: match the target’s shape – “does M accept?” use A_{TM} , “does M halt?” use HALT_{TM} , “is $L(M)$ empty / has property P ?” use E_{TM} . The arrow always points from the known-hard seed *into* the new target. *Check*. These are the standard undecidable seeds, and a reduction seed $\leq_m$ target proves the target undecidable (direction: known-hard $\rightarrow$ target).

The bridge that makes Template N tick is a one-line fact about complements.

Proposition 10.3.2 (Reductions ignore complementation). *For all languages, $A \leq_m B$ if and only if $\overline{A} \leq_m \overline{B}$, via the same reduction function f .*

Proof. A function f witnesses $A \leq_m B$ iff $w \in A \iff f(w) \in B$ for all w . Negating both sides of the biconditional gives $w \notin A \iff f(w) \notin B$, i.e. $w \in \overline{A} \iff f(w) \in \overline{B}$, which is exactly the statement that the same f witnesses $\overline{A} \leq_m \overline{B}$. $\square$

In practice this means: to show $\overline{A_{\text{TM}}} \leq_m B$ it suffices to produce a reduction $A_{\text{TM}} \leq_m \overline{B}$, because the two come from one function. We exploit this constantly. The recognizability landscape these templates populate is the following.



Template U lands a language above the “decidable” node (undecidable); Template N lands it strictly outside the “recognizable” wing. A language proved hard by *both* an A_{TM} reduction and an $\overline{A_{\text{TM}}}$ reduction sits at the very top, in “neither.”

Template U in action

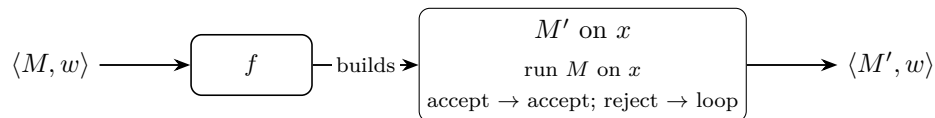
Worked Example 10.3.3 (The halting problem HALT_{TM} is undecidable). Let $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$. Show HALT_{TM} is undecidable by a mapping reduction $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$.

Recognition. A Template U task: source A_{TM} , target HALT_{TM} . The trigger is “ M accepts” versus “ M halts”: accepting is halting-and-saying-yes, so we must convert M into a machine whose halting alone already encodes acceptance.

Solution. Define $f(\langle M, w \rangle) = \langle M', w \rangle$ where M' is built from M as follows.

$M' =$ “On input x : run M on x ; if M accepts, *accept*; if M rejects, *enter a deliberate loop*.”

Thus M' halts on w iff M *accepts* w (a rejection of M is turned into looping, so the only way for M' to halt is via an acceptance). The construction is a fixed edit of M ’s transition table — redirect the reject state into a 1-state self-loop — so f is computable and does not run M' .



Which idea, and why? Template U with the standard “swallow the reject” gadget: redirecting M ’s rejection into a loop makes *halting* of M' coincide with *acceptance* of M , which is exactly the $\text{HALT}_{\text{TM}}/A_{\text{TM}}$ correspondence the reduction needs.

Check. If M accepts w , then M' accepts w , hence halts: $\langle M', w \rangle \in \text{HALT}_{\text{TM}}$. If M rejects or loops on w , then M' loops on w , so $\langle M', w \rangle \notin \text{HALT}_{\text{TM}}$. Therefore $\langle M, w \rangle \in A_{\text{TM}} \iff f(\langle M, w \rangle) \in \text{HALT}_{\text{TM}}$, and HALT_{TM} is undecidable.

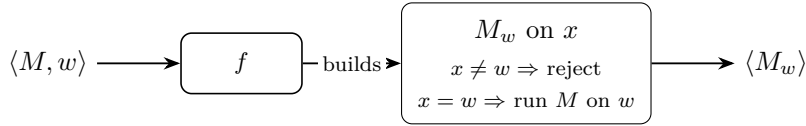
Worked Example 10.3.4 (The emptiness problem E_{TM} is undecidable). Let $E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$. Show $A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$, hence E_{TM} is undecidable.

Recognition. Template U again, but the target output is a *single* machine description $\langle M_w \rangle$, and acceptance of M on w must control whether $L(M_w)$ is empty. The trigger is the slogan “filter the input down to the one string w ”: force the constructed machine to ignore every input except w .

Solution. Define $f(\langle M, w \rangle) = \langle M_w \rangle$ where

$M_w =$ “On input x : if $x \neq w$, reject; if $x = w$, run M on w and accept iff M does.”

The string w is hard-coded into M_w , so the only input M_w can possibly accept is w itself. Hence $L(M_w) = \{w\}$ if M accepts w , and $L(M_w) = \emptyset$ if M does not. Writing M_w from $\langle M, w \rangle$ needs only a fixed “compare x with w ” front end, so f is computable.



Which idea, and why? The “filter to w ” gadget converts the acceptance question into an emptiness question: because M_w accepts nothing but w , its language is nonempty *exactly* when M accepts w , giving $A_{\text{TM}} \leq_m \overline{E_{\text{TM}}}$.

Check. M accepts $w \Rightarrow L(M_w) = \{w\} \neq \emptyset \Rightarrow \langle M_w \rangle \in \overline{E_{\text{TM}}}$. M does not accept $w \Rightarrow L(M_w) = \emptyset \Rightarrow \langle M_w \rangle \in E_{\text{TM}}$. So $\langle M, w \rangle \in A_{\text{TM}} \iff \langle M_w \rangle \in \overline{E_{\text{TM}}}$, and E_{TM} is undecidable.

Remark. The previous reduction lands in $\overline{E_{\text{TM}}}$, not E_{TM} — the constructed machine is *nonempty* exactly when M accepts. That is fine for *undecidability*, because a language is decidable iff its complement is, so E_{TM} is undecidable iff $\overline{E_{\text{TM}}}$ is. It matters enormously for *recognizability*, where complementing is not free; we return to this in the unrecognizability examples.

Worked Example 10.3.5 (Reducing through a non-regular target: $\text{REGULAR}_{\text{TM}}$). Let $\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid L(M) \text{ is regular}\}$. Show $A_{\text{TM}} \leq_m \text{REGULAR}_{\text{TM}}$.

Recognition. Template U where the property is “the language is regular.” The trigger is to make the constructed machine’s language flip between a *non-regular* language and a regular one according to whether M accepts w — a regular/non-regular switch driven by acceptance.

Solution. Use the witness non-regular language $\{0^n 1^n \mid n \geq 0\}$. Define $f(\langle M, w \rangle) = \langle M_2 \rangle$ where

$M_2 =$ “On input x : if x has the form $0^n 1^n$, accept; otherwise run M on w and accept iff M accepts w .”

If M does *not* accept w , the “otherwise” branch never accepts, so $L(M_2) = \{0^n 1^n\}$, which is *not* regular. If M *does* accept w , the “otherwise” branch accepts every x , so $L(M_2) = \Sigma^*$, which *is* regular. Either way M_2 is built mechanically from M, w , so f is computable.

$M \text{ accepts } w \longrightarrow M_2 \longrightarrow L(M_2) = \Sigma^* \text{ (regular)}$

$M \text{ rejects/loops on } w \longrightarrow M_2 \longrightarrow L(M_2) = \{0^n 1^n\} \text{ (not regular)}$

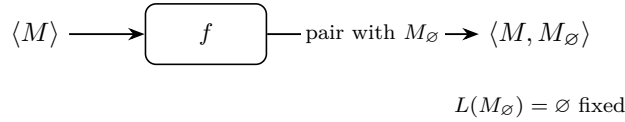
Which idea, and why? The reduction installs a regular/non-regular toggle: M_2 always accepts the witness non-regular language, and acceptance of M floods in the rest of Σ^* , tipping the language from non-regular to regular; thus $\langle M_2 \rangle \in \text{REGULAR}_{\text{TM}} \iff M \text{ accepts } w$.

Check. $\langle M, w \rangle \in A_{\text{TM}} \iff L(M_2) = \Sigma^* \iff L(M_2) \text{ is regular} \iff \langle M_2 \rangle \in \text{REGULAR}_{\text{TM}}$. So $A_{\text{TM}} \leq_m \text{REGULAR}_{\text{TM}}$ and the problem is undecidable. The same skeleton works for “is context-free,” “is decidable,” “is finite,” by swapping in an appropriate witness language — a foretaste of Rice’s theorem in Section 10.4.

Worked Example 10.3.6 (Reducing from a different source: EQ_{TM} via E_{TM}). Let $EQ_{TM} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$. Show $E_{TM} \leq_m EQ_{TM}$.

Recognition. Template U, but the convenient source is now E_{TM} (already undecidable), not A_{TM} . Emptiness is the special case of equivalence in which the second machine is pinned to recognize $\emptyset$. The trigger is “one of the two languages is a fixed constant.”

Solution. Fix once and for all a machine $M_\emptyset$ that rejects every input, so $L(M_\emptyset) = \emptyset$. Define $f(\langle M \rangle) = \langle M, M_\emptyset \rangle$ — output the pair consisting of M and the constant empty machine. This is a trivial, clearly computable rewrite (append a fixed description).



Which idea, and why? Casting emptiness as a special case of equivalence: $L(M) = L(M_\emptyset)$ says exactly $L(M) = \emptyset$, so testing the pair for equivalence answers the emptiness question, giving $E_{TM} \leq_m EQ_{TM}$.

Check. $\langle M \rangle \in E_{TM} \iff L(M) = \emptyset \iff L(M) = L(M_\emptyset) \iff \langle M, M_\emptyset \rangle \in EQ_{TM}$. Hence EQ_{TM} is undecidable. Choosing E_{TM} over A_{TM} as the source made the reduction a one-liner — a reminder that the source is yours to pick.

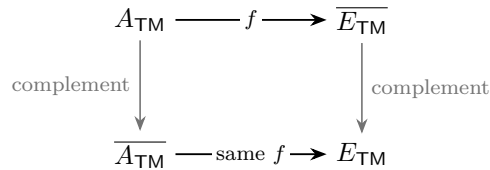
Template N: from undecidable to unrecognizable

Undecidability says no machine *decides* the language; unrecognizability says no machine even *recognizes* it (no machine that halts-and-accepts on exactly its members). The strictly stronger conclusion comes from reducing the non-recognizable source $\overline{A_{TM}}$.

Worked Example 10.3.7 ($\overline{A_{TM}} \leq_m E_{TM}$, so E_{TM} is not recognizable). Show that E_{TM} is not Turing-recognizable.

Recognition. Template N: source $\overline{A_{TM}}$, target E_{TM} . By Proposition 10.3.2 it is enough to give a reduction $\overline{A_{TM}} \leq_m \overline{E_{TM}}$ — and we already built one in the E_{TM} example above. The trigger is “I want M -accepts to send the output *into* $\overline{E_{TM}}$,” i.e. to a nonempty machine.

Solution. Reuse $f(\langle M, w \rangle) = \langle M_w \rangle$ with the “filter to w ” machine M_w , whose language is $\{w\}$ if M accepts w and $\emptyset$ otherwise. We showed $\langle M, w \rangle \in \overline{A_{TM}} \iff \langle M_w \rangle \in \overline{E_{TM}}$, i.e. f witnesses $\overline{A_{TM}} \leq_m \overline{E_{TM}}$. By Proposition 10.3.2 the *same* f witnesses $\overline{A_{TM}} \leq_m E_{TM}$.



Which idea, and why? Proposition 10.3.2 lets a single function serve both rows: a reduction $A_{TM} \leq_m \overline{E_{TM}}$ is a reduction $\overline{A_{TM}} \leq_m E_{TM}$, and since $\overline{A_{TM}}$ is not recognizable, neither is E_{TM} .

Check. $\overline{A_{TM}}$ is not Turing-recognizable (a standard fact: A_{TM} is recognizable but not co-recognizable). Lemma 10.3.1(2) with $\overline{A_{TM}} \leq_m E_{TM}$ gives: E_{TM} is not Turing-recognizable. Combined with the earlier remark that E_{TM} is co-recognizable, E_{TM} sits as a non-recognizable, co-recognizable, undecidable language.

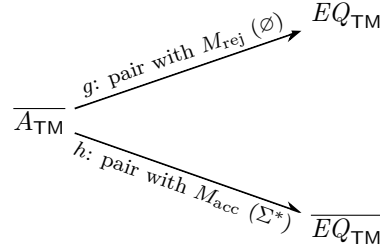
Worked Example 10.3.8 (EQ_{TM} is neither recognizable nor co-recognizable). Show that both EQ_{TM} and $\overline{EQ_{TM}}$ fail to be Turing-recognizable.

Recognition. This needs Template N *twice*: one reduction $\overline{A_{TM}} \leq_m EQ_{TM}$ to kill recognizability, and one $\overline{A_{TM}} \leq_m \overline{EQ_{TM}}$ to kill co-recognizability. The trigger is “land at the very top of the lattice: not recognizable and not co-recognizable.”

Solution. Build two machines from $\langle M, w \rangle$. For each, M_{run} denotes the machine “on any input, run M on w ; if it accepts, accept,” so $L(M_{\text{run}}) = \Sigma^*$ if M accepts w and $\emptyset$ otherwise.

Reduction 1 ($\overline{A_{TM}} \leq_m EQ_{TM}$). Let $g(\langle M, w \rangle) = \langle M_{rej}, M_{run} \rangle$ where M_{rej} rejects everything ($L(M_{rej}) = \emptyset$). Then the pair is equivalent iff $L(M_{run}) = \emptyset$ iff M does *not* accept w . So $\langle M, w \rangle \in \overline{A_{TM}} \iff g(\langle M, w \rangle) \in EQ_{TM}$.

Reduction 2 ($\overline{A_{TM}} \leq_m \overline{EQ_{TM}}$). Let $h(\langle M, w \rangle) = \langle M_{acc}, M_{run} \rangle$ where M_{acc} accepts everything ($L(M_{acc}) = \Sigma^*$). Then the pair is equivalent iff $L(M_{run}) = \Sigma^*$ iff M *does* accept w ; so the pair is in $\overline{EQ_{TM}}$ iff M does not accept w , giving $\langle M, w \rangle \in \overline{A_{TM}} \iff h(\langle M, w \rangle) \in \overline{EQ_{TM}}$.



Which idea, and why? Two Template N reductions from the same non-recognizable source, differing only in the fixed partner machine (M_{rej} vs. M_{acc}): pairing M_{run} with $\emptyset$ tests emptiness, pairing it with Σ^* tests universality, and the two together pin both EQ_{TM} and its complement outside the recognizable languages.

Check. Reduction 1 plus Lemma 10.3.1(2): EQ_{TM} not recognizable. Reduction 2 plus the same lemma: $\overline{EQ_{TM}}$ not recognizable, i.e. EQ_{TM} not co-recognizable. So EQ_{TM} occupies the “neither” corner of the lattice.

Worked Example 10.3.9 (Choosing the right source: a recognizability split for A_{TM} vs. $\overline{A_{TM}}$). Suppose you must show some language B is undecidable *but you also know B is recognizable*. Which source can you possibly use, A_{TM} or $\overline{A_{TM}}$? Explain the constraint.

Recognition. A meta-example about *source choice*. The trigger is matching the recognizability of the source to the recognizability you are allowed to conclude: a reduction can only transfer a property the source actually has.

Solution. If B is recognizable, then B *cannot* be the target of a reduction from $\overline{A_{TM}}$: such a reduction would force B to be non-recognizable by Lemma 10.3.1(2), contradicting recognizability of B . So a recognizable-but-undecidable B must be proved hard from A_{TM} (which transfers undecidability but never non-recognizability), never from $\overline{A_{TM}}$.

A_{TM} (recognizable) — OK: gives undecidable $\rightarrow$ recognizable B

$\overline{A_{TM}}$ (not recognizable) - - - - - forbidden - - - - - $\rightarrow$ recognizable B

Which idea, and why? A reduction transfers exactly the source’s hardness: $\overline{A_{TM}} \leq_m B$ would make B non-recognizable, so for a known-recognizable target the only admissible source is the recognizable one, A_{TM} .

Check. $HALT_{TM}$ and A_{TM} itself are recognizable and undecidable; both are proved hard from A_{TM} , and no $\overline{A_{TM}}$ reduction into them can exist. Conversely $\overline{A_{TM}} \leq_m E_{TM}$ succeeds precisely because E_{TM} is *not* recognizable.

Caution. The arrow direction is the whole ballgame. To prove B is hard you reduce a known-hard language *into* B (hard $\leq_m B$), never B into something hard. Writing “ $B \leq_m A_{TM}$ ” proves nothing about B ’s hardness — in fact every recognizable language reduces to A_{TM} , so that arrow is satisfied by easy languages too. Always read $A \leq_m B$ as “ A is no harder than B ,” and put the known-hard problem on the *left*.

Caution. Do not confuse undecidability with unrecognizability, and do not let an A_{TM} reduction masquerade as proof of the stronger fact. A reduction $A_{TM} \leq_m B$ shows that B is *undecidable* (and not co-recognizable), but because A_{TM} is itself recognizable, it can never certify that B is non-recognizable. For the stronger conclusion you must reduce from a non-recognizable source — typically $\overline{A_{TM}}$ — which by Proposition 10.3.2 usually amounts to building a reduction $A_{TM} \leq_m \overline{B}$ instead of $A_{TM} \leq_m B$.

Caution. The constructed machine $(M', M_w, M_2, M_{\text{run}}, \dots)$ is *never run* by the reduction — the reduction only *writes its description*. Confusing “construct $\langle M_w \rangle$ ” with “simulate M_w ” breaks computability: simulating could loop forever, but writing down a finite description that hard-codes w and bolts a few states onto M always halts. A reduction is a total computable function; it must finish on every input.

Exercises.

1. Let $ACCEPTS\text{-}\varepsilon_{\text{TM}} = \{\langle M \rangle \mid M \text{ accepts the empty string } \varepsilon\}$. Prove that this language is undecidable by giving an explicit reduction $A_{\text{TM}} \leq_m ACCEPTS\text{-}\varepsilon_{\text{TM}}$. Describe the machine your reduction constructs from $\langle M, w \rangle$, and verify the biconditional in both directions.
2. Let $ONE_{\text{TM}} = \{\langle M \rangle \mid |L(M)| = 1\}$, the machines whose language contains exactly one string. Show ONE_{TM} is undecidable by adapting the “filter to w ” construction. What is $L(M_w)$ in each of the two cases, and why does exactly one of them have size 1?
3. Show that $A_{\text{TM}} \leq_m HALT_{\text{TM}}$ and $HALT_{\text{TM}} \leq_m A_{\text{TM}}$ both hold, so A_{TM} and $HALT_{\text{TM}}$ are mapping-equivalent. (For the second direction, build from $\langle M, w \rangle$ a machine that on w runs M and routes *every* halt, accepting or rejecting, to an *accept*.)
4. Decide whether each statement is true, and justify in one or two sentences.
 - (a) If $A \leq_m B$ and B is decidable, then A is decidable.
 - (b) If $A \leq_m B$ and A is undecidable, then B is undecidable.
 - (c) If $A \leq_m B$ and A is decidable, then B is decidable.
 - (d) If $A \leq_m B$ and B is regular, then A is regular.
5. Using Proposition 10.3.2 (that $A \leq_m B \iff \overline{A} \leq_m \overline{B}$ via the same f), prove that the class of Turing-recognizable languages is *not* closed under the operation $A \mapsto \overline{A}$, by exhibiting a recognizable language whose complement is unrecognizable. Name the language and cite the relevant reduction.
6. Let $INFINITE_{\text{TM}} = \{\langle M \rangle \mid L(M) \text{ is infinite}\}$. Give a reduction $A_{\text{TM}} \leq_m INFINITE_{\text{TM}}$ that constructs a machine whose language is infinite exactly when M accepts w , and finite (indeed empty) otherwise. State your witness infinite language.
7. *Find the bug.* A student writes: “To prove E_{TM} is undecidable, reduce E_{TM} to A_{TM} : given $\langle M \rangle$, output $\langle M, \varepsilon \rangle$; then a decider for A_{TM} decides E_{TM} .” Identify the precise error in this argument, and explain why it cannot establish any hardness of E_{TM} .
8. *Find the bug.* Someone claims to show A_{TM} is not Turing-recognizable by giving a correct reduction $A_{\text{TM}} \leq_m HALT_{\text{TM}}$ and then asserting “so A_{TM} inherits non-recognizability from $HALT_{\text{TM}}$.” Two things are wrong. State both: which language is actually recognizable here, and which source a Template N proof requires.
9. Let B be a language with $A_{\text{TM}} \leq_m B$ and also $\overline{A_{\text{TM}}} \leq_m B$. Locate B in the recognizability lattice (decidable / recognizable / co-recognizable / neither) as tightly as the hypotheses allow, and justify each exclusion using Lemma 10.3.1.
10. Let $REJECTS\text{-}\varepsilon_{\text{TM}} = \{\langle M \rangle \mid M \text{ rejects or loops on } \varepsilon\}$. Show this language is not Turing-recognizable by reducing from $\overline{A_{\text{TM}}}$. (Hint: build M_w so that M_w on ε mirrors whether M accepts w , then apply Proposition 10.3.2.)
11. Let $EQ_{\text{TM}}^{\emptyset} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) = \emptyset\}$. Prove this restricted equivalence problem is undecidable by a reduction from E_{TM} , and explain in one sentence why your reduction outputs a pair both of whose components are tied to the same M .
12. Suppose you are told only that $A \leq_m B$ and that A is not Turing-recognizable but B is co-Turing-recognizable. Derive everything you can about $\overline{A}$, $\overline{B}$, and whether B can be decidable. Make each deduction explicit and cite the transfer direction it uses.

10.4 Rice's Theorem

The reductions of the previous sections each attacked one specific Turing-machine problem ($HALT_{TM}$, E_{TM} , $REGULAR_{TM}$, EQ_{TM}) with its own ad hoc gadget. Rice's theorem packages the ones about a single machine's language (E_{TM} , $REGULAR_{TM}$) into a single sweeping statement: *any* question that asks “does the language recognized by this machine have such-and-such property?” is undecidable, provided the property is not utterly trivial and depends only on the language. The one key idea is that the reduction from A_{TM} used over and over – splice the simulation of M on w in front of a fixed machine T , so that the spliced machine's *language* flips between $L(T)$ and $\emptyset$ exactly according to whether M accepts w – works no matter which language property we are testing. This section makes “property of the language” precise, proves the theorem, and then harvests undecidability of half a dozen problems for free.

Definition 10.4.1 (Language property; semantic vs. syntactic). A *language property* is a set P of Turing-machine encodings that respects language equality: whenever $L(M_1) = L(M_2)$, we have $\langle M_1 \rangle \in P \iff \langle M_2 \rangle \in P$. Equivalently, P is determined by a collection $\mathcal{C}$ of languages, with $\langle M \rangle \in P \iff L(M) \in \mathcal{C}$. A property is *semantic* if it is a language property in this sense, and *syntactic* if it can depend on *how* the machine is written (number of states, whether it ever moves left, etc.) rather than only on the set of strings it accepts.

Definition 10.4.2 (Trivial vs. nontrivial property). A language property P is *trivial* if either every Turing machine is in P (so P is all encodings) or no Turing machine is in P (so $P = \emptyset$). Otherwise P is *nontrivial*: there is at least one machine T with $\langle T \rangle \in P$ and at least one machine with its encoding outside P .

The two qualifiers are exactly the loopholes a property must avoid for Rice's theorem to apply. “Trivial” properties are decidable for a silly reason – a decider can answer “yes” (or always “no”) without reading its input. “Syntactic” properties dodge the theorem because they can inspect the text of $\langle M \rangle$, which is given explicitly; only properties of the abstract *language* are forced to be undecidable.

Theorem 10.4.3 (Rice). *Let P be any nontrivial language property. Then the language $P = \{\langle M \rangle \mid M \text{ is a TM and } \langle M \rangle \in P\}$ is undecidable.*

Proof. We reduce A_{TM} to P (or to its complement), so a decider for P would decide A_{TM} , which is impossible. Fix a machine $T_\emptyset$ that rejects every input, so $L(T_\emptyset) = \emptyset$. By swapping P for its complement $\bar{P}$ if necessary – legitimate because a language is decidable iff its complement is, and $\bar{P}$ is also a nontrivial language property – we may assume $\langle T_\emptyset \rangle \notin P$, i.e. the empty language does *not* have the property. Since P is nontrivial there is some machine T with $\langle T \rangle \in P$; note $L(T) \neq \emptyset$ because $\emptyset$ lacks the property.

Suppose, for contradiction, that a TM R_P decides P . We build a TM S deciding A_{TM} . On input $\langle M, w \rangle$, machine S constructs the description of a new machine M_w :

M_w = “On input x : (1) simulate M on w ; if it halts and rejects, reject. (2) If M accepted, simulate T on x and do as T does.”

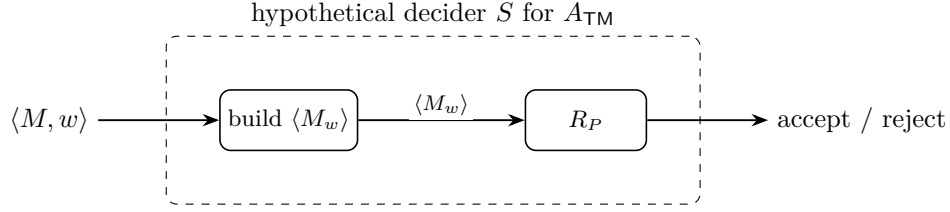
Then S runs R_P on $\langle M_w \rangle$ and copies its answer. The construction of $\langle M_w \rangle$ from $\langle M, w \rangle$ and the fixed $\langle T \rangle$ is a purely textual edit – bolt the “simulate M on w ” preamble onto a copy of T – so it is computable, and crucially M_w is never *run* by S ; only its description is handed to R_P .

Now read off $L(M_w)$. If M accepts w , stage (1) always passes (it halts and does not reject), so M_w proceeds to stage (2) on every input x and behaves exactly like T ; hence $L(M_w) = L(T)$, and $\langle M_w \rangle \in P$. If M does not accept w – it rejects or loops – then stage (1) never passes (it either rejects, or never finishes), so M_w accepts nothing; $L(M_w) = \emptyset = L(T_\emptyset)$, and $\langle M_w \rangle \notin P$. Summarizing,

$$\langle M, w \rangle \in A_{TM} \iff L(M_w) = L(T) \iff \langle M_w \rangle \in P,$$

so R_P 's verdict on $\langle M_w \rangle$ is exactly the A_{TM} answer. Thus S decides A_{TM} , contradicting its undecidability. Therefore no such R_P exists and P is undecidable. $\square$

The engine of the proof is one reduction schematic, reused for every property at once. The decider R_P we assume sits inside the box; the only moving part the reduction supplies is the splice that turns $\langle M, w \rangle$ into $\langle M_w \rangle$.



Remark. The single hypothesis that does all the work is $L(M_w) \in \{L(T), \emptyset\}$: the spliced machine’s language is pinned to one of two fixed languages, one having the property and one not. Everything about P beyond “ $\emptyset$ is out, some $L(T)$ is in” is irrelevant, which is why one proof covers infinitely many problems.

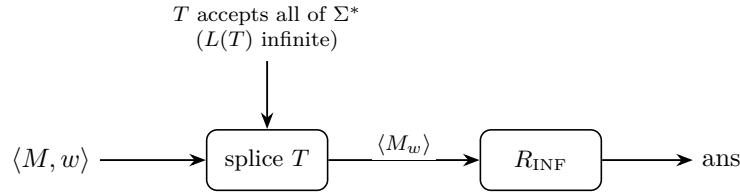
Method (How to prove a problem undecidable by Rice’s theorem).

1. **Phrase the problem as a set of machine encodings** $P = \{\langle M \rangle \mid L(M) \text{ satisfies } \dots\}$. Check that the dots mention *only* the language $L(M)$, never the number of states, the running time, the tape moves, or any other feature of the text $\langle M \rangle$. If a syntactic feature sneaks in, Rice’s theorem does not apply.
2. **Verify semantic invariance.** Confirm that if $L(M_1) = L(M_2)$ then both or neither machine is in P . This is automatic once the condition is stated purely about $L(M)$, but say it.
3. **Verify nontriviality.** Exhibit one concrete machine whose language *has* the property and one whose language *lacks* it. Two witnesses, both needed.
4. **Invoke the theorem.** Steps 2 and 3 are exactly the two hypotheses, so Rice’s theorem declares P undecidable. No reduction needs to be written by hand.

Worked Example 10.4.4 ($INFINITE_{TM}$: the language is infinite). Show that $INFINITE_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is infinite}\}$ is undecidable.

Recognition. The condition “ $L(M)$ is infinite” names a feature of the *set* $L(M)$, with no mention of how M is built. The trigger “property phrased about $L(M)$ alone” says reach for Theorem 10.4.3; we only owe semantic invariance and two witnesses.

Solution. Semantic: if $L(M_1) = L(M_2)$ then the two sets are literally equal, so one is infinite iff the other is – the property respects language equality. Nontrivial: a machine accepting all of Σ^* has an infinite language (in P); a machine that rejects everything has $L = \emptyset$, which is finite (out of P). Both witnesses exist, so Rice’s theorem applies and the problem is undecidable.



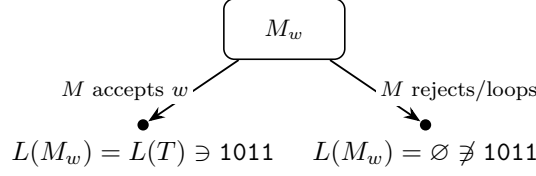
Which idea, and why? “ $L(M)$ is infinite” is a nontrivial language property, so the generic A_{TM} -splice of Theorem 10.4.3 – using a fixed T with infinite language – proves undecidability without a bespoke construction.

Check. Take T to recognize Σ^* . Then $L(M_w) = \Sigma^*$ (infinite) when M accepts w , and $L(M_w) = \emptyset$ (finite) otherwise; a decider for $INFINITE_{TM}$ would read off the A_{TM} answer.

Worked Example 10.4.5 (A fixed-string membership property: $1011 \in L(M)$). Show that $L_{1011} = \{\langle M \rangle \mid 1011 \in L(M)\}$ is undecidable.

Recognition. “1011 is one of the strings M accepts” is a statement about which strings lie in the set $L(M)$, so it is a language property. The trigger is the same as before, only the collection $\mathcal{C}$ is now “all languages containing 1011.”

Solution. Semantic: if $L(M_1) = L(M_2)$ then $1011 \in L(M_1) \iff 1011 \in L(M_2)$, since it is membership in the same set. Nontrivial: the machine recognizing Σ^* accepts 1011 (in P); the machine recognizing $\emptyset$ does not (out of P). Rice’s theorem applies. Concretely the splice uses a witness T with $1011 \in L(T)$, e.g. $L(T) = \Sigma^*$.



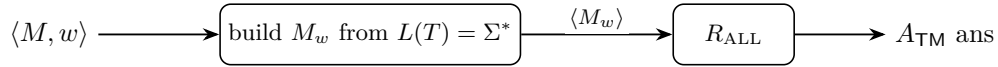
Which idea, and why? Membership of a fixed string is a nontrivial language property; the two language outcomes of the splice differ on whether 1011 is present, which is precisely what the decider would detect.

Check. With $L(T) = \Sigma^* \ni 1011$: if M accepts w then $1011 \in L(M_w) = \Sigma^*$, and if not then $L(M_w) = \emptyset$ omits 1011 – the property tracks A_{TM} .

Worked Example 10.4.6 (ALL_{TM} : the language is everything). Show that $ALL_{\text{TM}} = \{\langle M \rangle \mid L(M) = \Sigma^*\}$ is undecidable.

Recognition. “ $L(M)$ equals Σ^* ” compares the set $L(M)$ to a fixed set, again purely about the language. The trigger “stated about $L(M)$ only” points to Rice once more.

Solution. Semantic: $L(M_1) = L(M_2)$ forces $L(M_1) = \Sigma^* \iff L(M_2) = \Sigma^*$. Nontrivial: a machine recognizing Σ^* is in P ; a machine recognizing $\emptyset$ is out. Here we must choose the *witness with the property* and complement appropriately: T recognizing Σ^* has the property, $T_\emptyset$ recognizing $\emptyset$ lacks it, and since $\langle T_\emptyset \rangle \notin P$ the proof of Theorem 10.4.3 runs verbatim with this T .



Which idea, and why? Equality of the language to a fixed nontrivial target is a language property with the empty language excluded, so the standard splice with T recognizing Σ^* settles it.

Check. $L(M_w) = \Sigma^*$ exactly when M accepts w , and $L(M_w) = \emptyset \neq \Sigma^*$ otherwise; an ALL_{TM} decider would therefore decide A_{TM} .

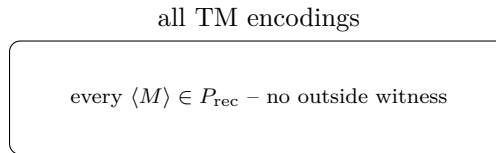
Worked Example 10.4.7 (Why triviality kills the theorem). Examine

$$P_{\text{rec}} = \{\langle M \rangle \mid L(M) \text{ is Turing-recognizable}\}$$

and $P_{\emptyset\text{-or-not-}\emptyset} = \{\langle M \rangle \mid L(M) = \emptyset \text{ or } L(M) \neq \emptyset\}$. Are these undecidable by Rice’s theorem?

Recognition. Both are stated about $L(M)$, so they are language properties – but Rice’s theorem needs the *second* hypothesis, nontriviality. The trigger is to test for two witnesses, one in and one out; if you cannot produce both, the theorem is silent.

Solution. Every $L(M)$ is by definition Turing-recognizable (it is recognized by M), so P_{rec} contains *every* encoding: it is trivial, and in fact *decidable* – a decider can accept any well-formed $\langle M \rangle$ outright. The second property is the tautology “ $\emptyset$ or not $\emptyset$,” true of every language, so it too is all of the encodings and decidable. Neither admits a machine *lacking* the property, so the nontriviality witness is missing and Rice’s theorem does not apply (correctly, since these are decidable).



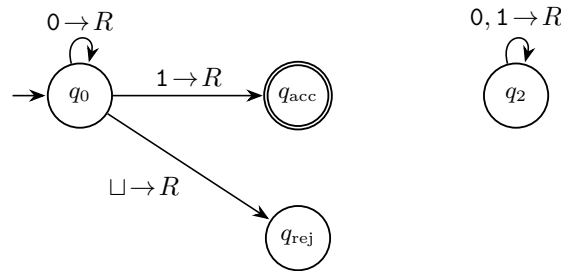
Which idea, and why? A trivial property labels every machine the same way, so a constant decider settles it; Rice’s theorem assumes a genuine split of machines into haves and have-nots, which a trivial property never provides.

Check. A would-be reduction would need a witness T with $\langle T \rangle \notin P_{\text{rec}}$, i.e. a machine whose language is *not* recognizable – impossible, since $L(T)$ is always recognized by T . The construction has no foothold.

Worked Example 10.4.8 (Why syntactic properties escape: a state count). Let $Q_5 = \{\langle M \rangle \mid M \text{ has at most 5 states}\}$. Show this is *decidable*, hence not subject to Rice’s theorem, and pinpoint which hypothesis fails.

Recognition. “Has at most 5 states” reads a number off the encoding $\langle M \rangle$; it is a feature of the *text*, not of $L(M)$. The trigger “the condition inspects how M is written” flags a *syntactic* property, outside the scope of Theorem 10.4.3.

Solution. Q_5 is decidable: a decider parses $\langle M \rangle$, counts the listed states, and accepts iff the count is ≤ 5 – no simulation, always halts. It is not a language property: two machines can recognize the same language with different state counts (pad any machine with unreachable states), so $L(M_1) = L(M_2)$ does *not* force $\langle M_1 \rangle \in Q_5 \iff \langle M_2 \rangle \in Q_5$. Semantic invariance fails, so Rice’s theorem says nothing, and indeed the problem is decidable.



same language as a 3-state machine, but 4 states drawn: count is syntactic

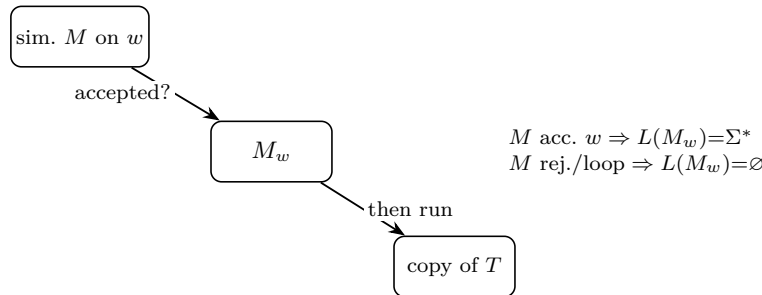
Which idea, and why? State count is read directly from $\langle M \rangle$ and is not invariant under language equality; because it violates the semantic hypothesis, Rice’s theorem – which speaks only of language properties – has nothing to say, and the count is plainly computable.

Check. The drawn machine and its 3-state reduct (delete the unreachable trap q_2) recognize the same language yet sit on opposite sides of, say, Q_3 ; a language property could never separate them, confirming Q_5 is syntactic.

Worked Example 10.4.9 (Reducing a fresh problem with the splice by hand). Show $E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$ is undecidable directly from Rice, and also exhibit the explicit A_{TM} -splice for the *complement* property, to see the construction concretely.

Recognition. Emptiness is a language property: “ $L(M) = \emptyset$.” Nontrivial, since some machines accept nothing and some accept something. The twist is that here $\emptyset$ itself *has* the property, so the proof’s normalization (assume $\emptyset \notin P$) targets $\overline{E_{\text{TM}}}$.

Solution. By Rice’s theorem E_{TM} is undecidable: it is semantic and nontrivial (witness $T_\emptyset$ in P ; a machine recognizing Σ^* out of P). To see the splice, prove the equivalent complement $NONEMPTY_{\text{TM}}$ undecidable, where $\emptyset \notin P$ holds outright. Pick T with $L(T) \neq \emptyset$, say $L(T) = \Sigma^*$, and form M_w as in Theorem 10.4.3: $L(M_w) = \Sigma^*$ (nonempty) iff M accepts w , else $L(M_w) = \emptyset$.



Which idea, and why? Emptiness is nontrivial and semantic, so Rice applies; the $\emptyset$ -in- P orientation is exactly the case the proof handles by passing to the complement, illustrating why that step is present.

Check. A decider for $NONEMPTY_{TM}$ on $\langle M_w \rangle$ returns “nonempty” precisely when M accepts w ; negating gives a decider for E_{TM} , both impossible.

Caution. Rice’s theorem is about the *language* $L(M)$, never about the machine’s behavior on a single input (beyond whether it accepts) or its runtime. “ M halts on the blank tape,” “ M uses at most 100 tape cells on 01,” and “ M has fewer than 7 states” are *not* language properties: two machines with identical languages can disagree on all three. Applying Rice to such a property is a category error – some of these problems are undecidable and some are decidable, but *never because of Rice*. Always check Definition 10.4.1 first: does $L(M_1) = L(M_2)$ force the same verdict?

Caution. Do not forget the nontriviality check. Properties like “ $L(M)$ is recognizable” (always true) or “ $L(M) \subseteq \Sigma^*$ ” (always true) are language properties but *trivial*, so they are decidable and Rice does not apply. You must name *both* a machine with the property and a machine without it; producing only one witness – a beginner’s slip – proves nothing, and may even be impossible because the property is secretly trivial.

Caution. Rice’s theorem tells you a property is undecidable; it does *not* tell you whether the property or its complement is Turing-recognizable. $INFINITE_{TM}$ and E_{TM} are both undecidable by Rice, yet they sit at different levels: E_{TM} is co-recognizable while $INFINITE_{TM}$ is neither recognizable nor co-recognizable. Recognizability is a finer question that the splice argument is blind to – settle it with the separate mapping reductions of the previous section, not with Rice.

Exercises.

- For each property below, decide whether it is a *language property* in the sense of Definition 10.4.1. Justify each answer by either confirming invariance under $L(M_1) = L(M_2)$ or exhibiting two machines with equal languages but opposite verdicts.
 - $L(M)$ contains at least one string of length 7.
 - M ever writes a blank symbol when run on input 0.
 - $L(M)$ is a regular language.
 - M halts within 50 steps on every input of length 3.
- Use Rice’s theorem to prove that $FINITE_{TM} = \{\langle M \rangle \mid L(M) \text{ is finite}\}$ is undecidable. Name your two nontriviality witnesses explicitly and verify semantic invariance.
- Let $CONTAINS-AB = \{\langle M \rangle \mid ab \in L(M)\}$. Prove it is undecidable by Rice’s theorem, and draw the A_{TM} -splice reduction diagram, labelling the two possible values of $L(M_w)$ and which one contains ab .
- Show that $PALINDROME_{TM} = \{\langle M \rangle \mid L(M) \text{ consists only of palindromes}\}$ is undecidable. Give the witness machine T used in the splice and state $L(T)$.
- The property $REGULAR_{TM}$ (the language of M is regular) was shown undecidable in Section 10.1 by a hand-built reduction. Re-derive its undecidability in three lines using Rice’s theorem instead. Then explain in one sentence what the earlier proof gives that the Rice proof does not.
- Consider the property $P = \{\langle M \rangle \mid L(M) = \overline{L(M')}\}$ for some fixed TM M' , where M' is a particular machine recognizing $\{0^n 1^n \mid n \geq 0\}$. Is P a language property? Is it nontrivial? Conclude whether Rice’s theorem applies, and if so, what it tells you.
- Find the bug.* A student writes: “By Rice’s theorem, the problem $BLANK-HALT = \{\langle M \rangle \mid M \text{ halts on the empty input}\}$ is undecidable, because halting is a nontrivial property of machines.” The conclusion (undecidability) is correct, but the *justification* is wrong. Identify precisely which hypothesis of Rice’s theorem fails for $BLANK-HALT$, and give two machines with the same language that disagree on whether they halt on the empty input.
- Prove that $TWO_{TM} = \{\langle M \rangle \mid |L(M)| = 2\}$ (the language has exactly two strings) is undecidable. Identify the witness T and confirm that the empty-language orientation in the proof of Theorem 10.4.3 is handled correctly here.
- Both conditions in Rice’s theorem are necessary. **(a)** Give a *trivial* language property that is decidable, showing nontriviality cannot be dropped. **(b)** Give a *syntactic* (non-semantic) property that is decidable, showing the language-property condition cannot be dropped.

10. Let P be a nontrivial language property such that the empty language $\emptyset$ has property P . Walk through the proof of Theorem 10.4.3 for this case and state precisely where and why the argument passes to the complement $\bar{P}$. What machine plays the role of the witness T , and what is required of $L(T)$?
11. A property P satisfies: there exists a single TM T_0 such that $\langle M \rangle \in P \iff L(M) = L(T_0)$. Show P is a nontrivial language property and conclude it is undecidable. Then explain why the special case $L(T_0) = \Sigma^*$ recovers ALL_{TM} .
12. *Find the bug.* Consider this attempted reduction proving $INFINITE_{TM}$ undecidable: “On input $\langle M, w \rangle$, build M_w that on input x first simulates T (a machine with infinite language) on x and accepts if T does, and then, separately, simulates M on w .” Explain why this M_w has $L(M_w)$ infinite regardless of whether M accepts w , so the reduction fails. State the correct ordering of the two simulations and why order matters.

10.5 The Post Correspondence Problem

Undecidability is not a disease of self-referential machine problems alone. In this section we exhibit a problem about plain strings – no Turing machines mentioned in its statement – that is nevertheless undecidable: the *Post Correspondence Problem* (PCP). The one key idea is that a match in a carefully built tile puzzle can be forced to spell out an accepting computation history of a Turing machine, so deciding whether the puzzle has a match would decide whether the machine accepts. We first state PCP as a concrete domino game, express it as a language, and then reduce A_{TM} to it through the computation-history technique developed earlier in the chapter.

The puzzle, and the language

A *domino* is an ordered pair of nonempty strings over a fixed alphabet, drawn as a fraction with a top string and a bottom string:

$$\left[\frac{t}{b} \right], \quad t, b \in \Sigma^*.$$

An *instance* of the puzzle is a finite collection of dominos, with repetitions of any domino allowed when you play. A *match* is a finite list of dominos from the collection – $\left[\frac{t_{i_1}}{b_{i_1}} \right] \left[\frac{t_{i_2}}{b_{i_2}} \right] \cdots \left[\frac{t_{i_\ell}}{b_{i_\ell}} \right]$ – such that reading all the top strings left to right gives exactly the same string as reading all the bottom strings:

$$t_{i_1} t_{i_2} \cdots t_{i_\ell} = b_{i_1} b_{i_2} \cdots b_{i_\ell}.$$

Definition 10.5.1 (Post Correspondence Problem). An instance of the *Post Correspondence Problem* is a finite collection of dominos $P = \left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\}$ with each $t_i, b_i \in \Sigma^*$. A *match* for P is a sequence of indices $i_1, i_2, \dots, i_\ell$ (with $\ell \geq 1$, repetitions allowed) for which $t_{i_1} \cdots t_{i_\ell} = b_{i_1} \cdots b_{i_\ell}$. The decision problem is

$$PCP = \{ \langle P \rangle \mid P \text{ is an instance of PCP that has a match} \}.$$

Two features of the rules deserve emphasis up front. A domino may be reused as often as you like, so the same index may appear many times in a match. And the *order* of the dominos is yours to choose; the constraint is only that the concatenated top equal the concatenated bottom.

Worked Example 10.5.2 (Finding a match by hand). Decide whether the instance

$$P = \left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}$$

has a match, and exhibit one if it does.

Recognition. This is a small instance, so we search directly. The trigger for “search by hand” is a collection with only a few dominos and short strings: track the running top/bottom difference and try to drive it to zero.

Solution. Number the dominos 1, 2, 3, 4 as listed. A match must start with a domino whose top and bottom agree on their first symbol, otherwise the two concatenations already disagree at position one. Only domino $2 = \begin{bmatrix} a \\ ab \end{bmatrix}$ qualifies (top a, bottom a). After playing it the bottom is one symbol ahead – it has an extra trailing **b** that the top still owes. We keep playing so that each new top symbol pays off the bottom’s lead. Working it through, the list 2, 1, 3, 2, 4 matches:

top	a	b	c	a	a	a	b	c
bottom	a	b	c	a	a	a	b	c

Reading the top of 2, 1, 3, 2, 4 gives $a \cdot b \cdot ca \cdot a \cdot abc = abcaaaabc$, and reading the bottom gives $ab \cdot ca \cdot a \cdot ab \cdot c = abcaaaabc$. The two agree, so this is a match and $\langle P \rangle \in PCP$.

Which idea, and why? A match must agree symbol-by-symbol from the very first position, so the search is anchored: only a domino whose top and bottom share a first symbol can begin a match, and from there the running “surplus” of one side must be repaid by the other.

Check. Both readings are the eight-symbol string **abcaaaabc**; in particular the lengths agree ($8 = 8$), a necessary condition we verify before trusting the match.

Worked Example 10.5.3 (An instance with no possible match). Show that $P = \left\{ \begin{bmatrix} abc \\ ab \end{bmatrix}, \begin{bmatrix} ca \\ a \end{bmatrix}, \begin{bmatrix} acc \\ ba \end{bmatrix} \right\}$ has no match.

Recognition. When every top is *strictly longer* than its bottom, no concatenation can balance: a length count rules out a match before any search. The trigger is a uniform length inequality across all dominos.

Solution. For each domino, $|t_i| > |b_i|$: lengths $3 > 2$, $2 > 1$, $3 > 2$. For any nonempty list $i_1, \dots, i_\ell$,

$$|t_{i_1} \cdots t_{i_\ell}| = \sum_j |t_{i_j}| > \sum_j |b_{i_j}| = |b_{i_1} \cdots b_{i_\ell}|,$$

so the top string is strictly longer than the bottom and the two can never be equal. Hence $\langle P \rangle \notin PCP$.

Which idea, and why? Equal strings have equal length, so the per-domino length comparison gives a free necessary condition: if every top exceeds (or every top falls short of) its bottom, the totals can never meet and no match exists.

Check. The shortest candidate list, a single domino, already fails: **abc** vs **ab**, **ca** vs **a**, **acc** vs **ba** – none is an equality.

The modified problem and computation histories

The proof reduces A_{TM} to PCP, but it is cleaner to first hit an intermediate target where the match is *required to begin with a designated first domino*. Recall from the computation-history method that for a deterministic TM M on input w , an *accepting computation history* is the sequence of configurations $C_1, C_2, \dots, C_t$ where C_1 is the start configuration on w , each C_{i+1} follows from C_i by one step of M , and C_t is accepting. Writing configurations with a # separator,

$$\# C_1 \# C_2 \# \cdots \# C_t \#,$$

is a single string that exists exactly when M accepts w . The whole construction below is an engine for making a domino match *spell out this string* on its bottom row.

Definition 10.5.4 (Modified PCP). $MPCP = \{ \langle P \rangle \mid P \text{ has a match that uses the first listed domino as its first tile} \}$.

Lemma 10.5.5 (MPCP reduces to PCP). $MPCP \leq_m PCP$; consequently if PCP were decidable, so would MPCP be.

Proof. Fix a fresh symbol $*$ not in the alphabet. For a string $u = u_1 u_2 \cdots u_n$ write $*u = *u_1 u_2 \cdots *u_n$ (a $*$ before each symbol), $u* = u_1 *u_2 \cdots u_n *$ (a $*$ after each symbol), and $*u* = *u_1 *u_2 \cdots *u_n *$ (both). Given an MPCP instance $P' = \left[\begin{smallmatrix} t_1 \\ b_1 \end{smallmatrix} \right], \dots, \left[\begin{smallmatrix} t_k \\ b_k \end{smallmatrix} \right]$ with designated first domino $\left[\begin{smallmatrix} t_1 \\ b_1 \end{smallmatrix} \right]$, output the PCP instance

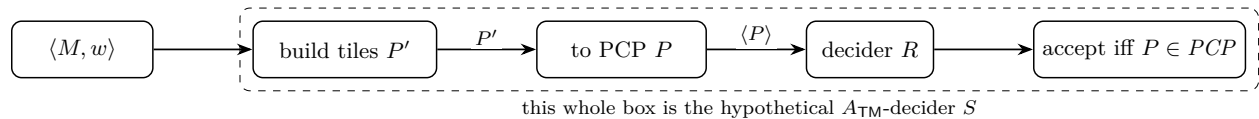
$$P: \left[\begin{smallmatrix} *t_1 \\ *b_1* \end{smallmatrix} \right], \left[\begin{smallmatrix} *t_1 \\ b_1* \end{smallmatrix} \right], \left[\begin{smallmatrix} *t_2 \\ b_2* \end{smallmatrix} \right], \dots, \left[\begin{smallmatrix} *t_k \\ b_k* \end{smallmatrix} \right], \left[\begin{smallmatrix} *\diamond \\ \diamond \end{smallmatrix} \right],$$

where $\diamond$ is another fresh end symbol. The interleaved $*$'s force structure. The only domino whose top and bottom begin with the same symbol is the first, $\left[\begin{smallmatrix} *t_1 \\ *b_1* \end{smallmatrix} \right]$, so every match must start there – exactly the MPCP restriction, now built into the tiles. Stripping the $*$'s and the final $\diamond$ domino, matches of P correspond precisely to matches of P' that start with tile 1. The map $\langle P' \rangle \mapsto \langle P \rangle$ is computable, so it is a mapping reduction. $\square$

Theorem 10.5.6 (Post). *PCP is undecidable.*

Proof sketch via $A_{\text{TM}} \leq_m \text{MPCP} \leq_m \text{PCP}$. By Lemma 10.5.5 it suffices to reduce A_{TM} to MPCP. Given $\langle M, w \rangle$ we build an MPCP instance P' whose matches are exactly the accepting computation histories of M on w . We may assume M never moves left off the tape, and (if $w = \varepsilon$) that the blank $\sqcup$ stands in for the input. The dominos come in seven groups, displayed and explained in the worked examples below; their combined effect is that the *only* way to extend a partial match is to write down M 's next configuration on the bottom while the top copies the previous one, so a completed match forces a correct, accepting simulation. Thus $\langle M, w \rangle \in A_{\text{TM}}$ iff $P' \in \text{MPCP}$. The construction is computable, so $A_{\text{TM}} \leq_m \text{MPCP}$, and chaining with Lemma 10.5.5 gives $A_{\text{TM}} \leq_m \text{PCP}$. Since A_{TM} is undecidable, so is PCP. $\square$

The reduction is summarized by the schematic below: a would-be PCP decider R is turned into a decider S for A_{TM} by interposing the tile-builder.



The tiles, group by group

We now show how each group of dominos does its job. Throughout, $\#$ separates configurations and the bottom row always runs one configuration *ahead* of the top; that lead is the engine that forces each next step. We illustrate with a tiny machine fragment: tape alphabet $\Gamma = \{0, 1, 2, \sqcup\}$, input $w = 010$, start state q_0 , and the transition $\delta(q_0, 0) = (q_7, 2, R)$ (read 0 in q_0 : write 2, move right, enter q_7).

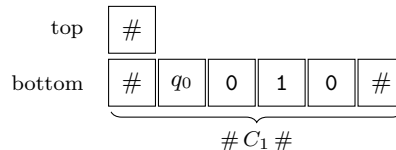
Worked Example 10.5.7 (Group 1: the start tile (and why MPCP forces the opening)). State the first domino and trace why the match must begin with the start configuration on the bottom.

Recognition. The match must produce $\#C_1\# \cdots$ on the bottom, and MPCP pins the first tile. The trigger is “establish the initial configuration”: one tile carries the entire start configuration on its bottom and only $\#$ on top.

Solution. Put as tile 1

$$\left[\begin{array}{c} \# \\ \# q_0 w_1 w_2 \cdots w_n \# \end{array} \right] = \left[\begin{array}{c} \# \\ \# q_0 010 \# \end{array} \right] \quad (\text{our example}).$$

Because the instance is MPCP, the match opens here. The bottom now reads $\#q_0 010\#$ while the top reads only $\#$: the top is “behind” by exactly C_1 followed by a $\#$, which the later groups must let it catch up to.



Which idea, and why? One designated start tile, with the full start configuration on its bottom and a lone separator on top, seeds the match so the bottom is exactly one configuration ahead from the outset.

Check. The bottom string is $\#q_0 010\#$, which is precisely $\#C_1\#$ for the start configuration $C_1 = q_0 010$; the top owes everything after the first $\#$.

Worked Example 10.5.10 (Groups 6–7: letting the top catch up after accepting). Give the dominos that close the match once the bottom reaches an accepting configuration.

Recognition. So far the bottom is permanently one configuration ahead of the top; a match needs the two to become *equal*, which can only happen once the simulation halts in q_{acc} . The trigger is “the accept state has appeared – now shrink the gap to zero.”

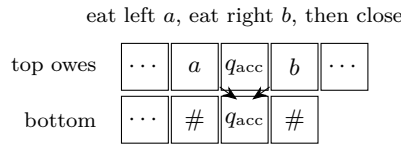
Solution. Once q_{acc} is on the bottom, add “eating” tiles that delete a tape symbol adjacent to q_{acc} : for every tape symbol a ,

$$\text{group 6: } \left[\frac{a q_{\text{acc}}}{q_{\text{acc}}} \right], \left[\frac{q_{\text{acc}} a}{q_{\text{acc}}} \right].$$

Each such tile lets the top write two symbols where the bottom writes one, so the top gains ground. When only q_{acc} remains between two $\#$ ’s, the final tile (group 7)

$$\left[\frac{q_{\text{acc}} \# \#}{\#} \right]$$

closes the match: the top supplies the last $q_{\text{acc}}\#$ it owed plus a closing $\#$, and the bottom adds a single $\#$, so top and bottom become identical.



Which idea, and why? Equality of top and bottom is impossible while the machine is mid-run because the bottom stays one configuration ahead; only the accept-state eating tiles let the top write more than the bottom, closing the unique surplus and completing the match exactly when M has accepted.

Check. If M never reaches q_{acc} , the eating tiles are never usable, the bottom stays strictly ahead forever, and no finite match exists – matching “ M does not accept w .”

Corollary 10.5.11 (Undecidability spreads beyond machine problems). *There exist undecidable problems whose statements mention no Turing machine at all. In particular, PCP is such a problem, and any problem B with $\text{PCP} \leq_m B$ is undecidable as well.*

Proof. PCP is undecidable by Theorem 10.5.6 and its definition refers only to finite collections of strings. For the second claim, if B were decidable then $\text{PCP} \leq_m B$ would make PCP decidable, contradicting the theorem. $\square$

Worked Example 10.5.12 (Using PCP as a source: an ambiguity-style argument). Explain, at the level of the reduction skeleton, how an undecidability proof for some string- or grammar-problem B can be built from PCP rather than from A_{TM} directly.

Recognition. We already have one undecidable string problem; the trigger “new problem also about strings” suggests reducing PCP to it instead of rebuilding the computation-history machinery.

Solution. Exhibit a computable map taking a PCP instance P to an object $f(P)$ of problem B such that P has a match iff $f(P) \in B$. Typically f encodes “the two readings of a domino list” as two derivations or two strings inside B ’s world, so that a coincidence in B corresponds exactly to a match in P . Then $\text{PCP} \leq_m B$, and undecidability of B follows from Corollary 10.5.11 without ever mentioning a tape.

$$A_{\text{TM}} \xrightarrow{\leq_m} \text{MPCP} \xrightarrow{\leq_m} \text{PCP} \xrightarrow{\leq_m} B$$

Which idea, and why? Once one problem in a domain is known undecidable, fresh proofs reduce *from* it, chaining mapping reductions; transitivity of $\leq_m$ then carries undecidability all the way down the chain to B .

Check. The chain is sound because $\leq_m$ is transitive (Section 10.2): each arrow is a computable map preserving membership, so their composition is too.

Method (How to prove a string/grammar problem B undecidable via PCP).

1. **Pick the direction.** Reduce *from* PCP to B : build a computable f with $P \in PCP \iff f(P) \in B$. Never reduce the other way – that proves nothing about B .
2. **Encode the two readings.** Represent the top concatenation and the bottom concatenation of a domino list as two parallel objects in B (e.g. two grammar derivations, two paths, two strings), indexed by the same chosen sequence $i_1, \dots, i_\ell$.
3. **Make “match” equal “yes-instance.”** Arrange the encoding so that the two readings coincide exactly when $f(P)$ is a positive instance of B (an ambiguity, an intersection, an equal-string event, etc.).
4. **Check computability.** Confirm f is produced by an always-halting procedure – it only rewrites the finite list P , so this is routine.
5. **Conclude.** $PCP \leq_m B$, hence B is undecidable by Corollary 10.5.11.

Caution. Do not reduce *from* a *trivially easy* restriction of PCP and claim victory. For instance, PCP becomes *decidable* when the alphabet has a single symbol (then a match is just a numeric balance condition you can solve by arithmetic). Undecidability is a property of the *general* problem; choosing a special case with extra structure can make it decidable, so a reduction must start from the unrestricted problem (or from a restriction already proved undecidable).

Caution. Keep the reduction arrow pointing the right way. To show B undecidable you build $PCP \leq_m B$ (or $A_{TM} \leq_m B$): a yes-instance of the *known-hard* problem must map to a yes-instance of B . Writing $B \leq_m PCP$ instead shows only that B is no harder than PCP, which says nothing about whether B is decidable. Each arrow’s tail is the problem you already know is undecidable.

Caution. The match condition is about *strings*, not multisets or lengths alone. Equal total length is necessary but not sufficient: $\begin{bmatrix} ab \\ ba \end{bmatrix}$ used alone has equal lengths yet $ab \neq ba$. Always confirm the two concatenations agree symbol-by-symbol, and remember the order of tiles in the list is what you get to choose.

Exercises.

1. For the instance $P = \left\{ \begin{bmatrix} 1 \\ 111 \end{bmatrix}, \begin{bmatrix} 10111 \\ 10 \end{bmatrix}, \begin{bmatrix} 10 \\ 0 \end{bmatrix} \right\}$, find a match. (Hint: which domino must come first, and which must come last? Track how far ahead one side runs.)
2. Decide whether $P = \left\{ \begin{bmatrix} ab \\ abab \end{bmatrix}, \begin{bmatrix} b \\ a \end{bmatrix}, \begin{bmatrix} aba \\ b \end{bmatrix}, \begin{bmatrix} aa \\ a \end{bmatrix} \right\}$ has a match. If it does, give one and verify both readings; if not, give a one-line reason.
3. Prove that if an instance P has *no* domino whose top and bottom share a common first symbol, then P has no match. State the analogous fact about last symbols.
4. Show that the single-domino instances are easy: describe exactly which dominos $\begin{bmatrix} t \\ b \end{bmatrix}$ (used alone, possibly repeated) admit a match, and prove your characterization.
5. Give a PCP instance over the one-symbol alphabet $\{a\}$ that has a match, and one that does not. Then argue in two or three sentences why PCP restricted to a one-symbol alphabet is decidable.
6. In the $A_{TM} \leq_m MPCP$ construction, the start tile is $\begin{bmatrix} \# \\ \# q_0 w_1 \dots w_n \# \end{bmatrix}$. Explain in your own words why, immediately after this tile, the bottom string is exactly one configuration (plus a separator) ahead of the top, and why that “lead” never shrinks until the accept state appears.
7. Write down the group-2 and group-3 dominos generated by the transitions $\delta(q_3, a) = (q_4, b, R)$ and $\delta(q_4, b) = (q_2, a, L)$ over tape alphabet $\{a, b, \sqcup\}$. (Remember group 3 produces one tile per possible left-neighbor symbol.)
8. *Find the bug.* A student claims the instance $P = \left\{ \begin{bmatrix} a \\ aa \end{bmatrix}, \begin{bmatrix} aa \\ a \end{bmatrix} \right\}$ has no match “because the lengths can never balance.” Find a match, and identify the exact flaw in the length argument.
9. *Find the bug.* Someone reduces a problem B to PCP, writing $B \leq_m PCP$, and concludes that B is undecidable. Explain precisely why this conclusion does not follow, and state which direction the reduction would need to go to prove B undecidable.

10. Prove that PCP is *Turing-recognizable*: describe a procedure that halts and accepts on exactly the instances that have a match. Why does this not contradict undecidability?
11. Let $PCP_{\leq 2}$ be PCP restricted to instances with at most two dominos. Without solving it, explain why the general undecidability proof of Theorem 10.5.6 says *nothing* about whether $PCP_{\leq 2}$ is decidable, and contrast this with the unrestricted problem.
12. Suppose f is a computable function with $P \in PCP \iff f(P) \in B$ for every PCP instance P . Using transitivity of $\leq_m$ and the chain $A_{TM} \leq_m MPCP \leq_m PCP \leq_m B$, write a clean two-sentence proof that B is undecidable. Identify which single fact about $\leq_m$ each sentence uses.

Chapter 11

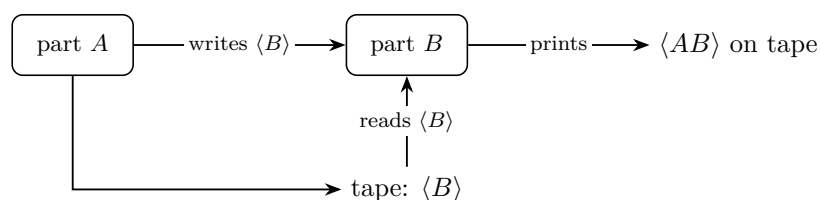
Advanced Computability: Recursion and Degrees

11.1 The Recursion Theorem and Self-Reference

A machine that prints its own description sounds impossible: it would have to “contain” a copy of itself, which seems to demand that it be larger than itself. The recursion theorem dissolves this paradox and turns it into a tool. Its one key idea is that any Turing machine can be built so that it has access to its *own* description $\langle M \rangle$ as if that description were handed to it for free; once a machine can talk about itself, self-reproduction is no harder than copying, and “do the opposite of what a decider says about me” becomes a one-line diagonal argument. This section makes the construction precise, proves the theorem, and uses it to write the smallest interesting self-referential machines.

The paradox, and why it is not one

The folklore argument against self-reproduction runs as follows. A machine A that builds a machine B must contain the blueprint of B , so A is strictly more complicated than B . If a machine could build a copy of *itself*, it would have to be strictly more complicated than itself, which is absurd. The flaw is the hidden assumption that the only way to “contain” a blueprint is to store it literally. A short program can *generate* a long output, and in particular a program can generate *its own text* without storing that text inside itself. The trick is to split the work between a part that holds data and a part that reads that data back off the tape and reassembles the whole.



The two-part trick: A leaves $\langle B \rangle$ on the tape; B reads it back, recomputes $\langle A \rangle$, and prints $\langle AB \rangle$.

Printing a fixed string, then printing yourself

Everything rests on one easy computable function: from a string w we can build, mechanically, the *code* of a machine that prints w .

Lemma 11.1.1 (The print-this gadget). *There is a computable function $q: \Sigma^* \rightarrow \Sigma^*$ such that for every w , the string $q(w) = \langle P_w \rangle$ is the description of a Turing machine P_w that, on any input, erases its input, writes w on the tape, and halts.*

Proof. Given w , build the description of P_w literally: P_w has w “baked in” as a fixed sequence of write-and-move instructions. The machine Q below carries this out and is plainly an algorithm.

$Q =$ “On input w : construct the machine $P_w =$ “erase the input, write w , halt,” then output $\langle P_w \rangle$.”

Since Q halts on every input with the desired description, $q(w) = \langle P_w \rangle$ is computable. $\square$

We write $\langle P_w \rangle$ for the code of the printer of w . The point of Lemma 11.1.1 is that the map “ $w \mapsto$ a machine that outputs w ” is itself something a machine can perform, so a machine can manufacture printers on the fly.

Theorem 11.1.2 (A self-printing machine exists). *There is a Turing machine **SELF** that, on any input, halts with its own description $\langle \text{SELF} \rangle$ on the tape and nothing else.*

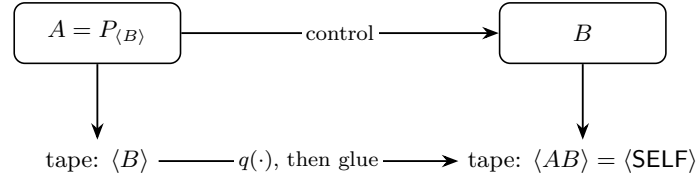
Proof. Write $\text{SELF} = AB$ as two procedures run in order. The job of A is to print $\langle B \rangle$; the job of B is to recover $\langle A \rangle$ from what is on the tape, glue the two halves together, and print $\langle AB \rangle$.

Part A. Let $A = P_{\langle B \rangle}$, the printer of the string $\langle B \rangle$, obtained from Lemma 11.1.1 as $A =$ the machine with code $q(\langle B \rangle)$. When A runs it leaves exactly $\langle B \rangle$ on the tape. (We can write down A only after we know B ; that is fine, because B does not mention A .)

Part B. We must *not* set $B = P_{\langle A \rangle}$: that would define B in terms of A , which is defined in terms of B – a circular definition. Instead B *computes* $\langle A \rangle$ from the tape:

- $B =$ “On input $\langle M \rangle$ (a fragment of a machine), found on the tape:
1. compute $q(\langle M \rangle)$, the code of the printer of $\langle M \rangle$;
 2. combine that code with $\langle M \rangle$ into a single machine description;
 3. print the resulting description and halt.”

Now run $\text{SELF} = AB$. First A runs and leaves $\langle B \rangle$ on the tape. Then B starts, reads $\langle B \rangle$ from the tape, computes $q(\langle B \rangle) = \langle P_{\langle B \rangle} \rangle = \langle A \rangle$ (this is exactly the code of part A!), combines it with $\langle B \rangle$ to form $\langle AB \rangle = \langle \text{SELF} \rangle$, and prints it. The output is the machine’s own description, with no circularity: B never refers to A by name, only to whatever string it finds on the tape. $\square$



Schematic of **SELF**. A writes $\langle B \rangle$; B applies q to recover $\langle A \rangle$, glues, and prints $\langle AB \rangle$.

The recursion theorem

SELF only prints itself. The recursion theorem upgrades “obtain your own description” into a reusable subroutine: a machine may grab $\langle M \rangle$ and then *compute with it* however it likes.

Theorem 11.1.3 (Recursion theorem). *Let T compute a function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Then there is a Turing machine R computing the function $r: \Sigma^* \rightarrow \Sigma^*$ given by*

$$r(w) = t(\langle R \rangle, w) \quad \text{for every } w \in \Sigma^*.$$

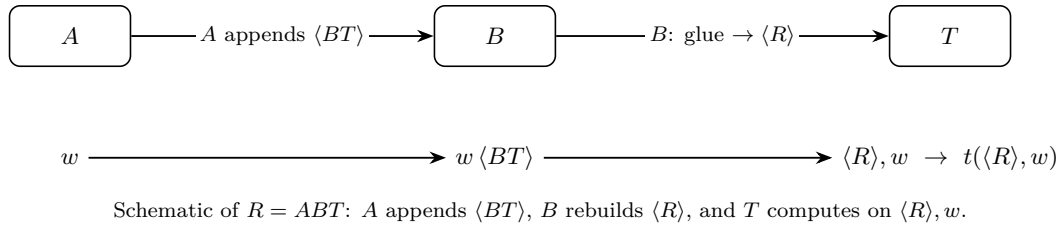
That is, R behaves exactly like T would on the pair $\langle R \rangle, w$, with R ’s own description supplied automatically as the first argument.

Proof. Build $R = ABT$ in three parts, where T is the given machine. We modify the printer so that it writes *after* any string already on the tape, preserving the input w .

Part A. Let $A = P_{\langle BT \rangle}$, the printer of $\langle BT \rangle$; after A runs the tape holds $w \langle BT \rangle$ (the original input w followed by $\langle BT \rangle$).

Part B. B examines the tape, applies q to the portion $\langle BT \rangle$ to obtain $q(\langle BT \rangle) = \langle A \rangle$, and concatenates to form $\langle ABT \rangle = \langle R \rangle$. It then arranges the tape to hold the pair $\langle R \rangle, w$ and passes control to T .

Part T. T now runs on $\langle R \rangle, w$ and computes $t(\langle R \rangle, w)$. Hence R on input w produces $t(\langle R \rangle, w) = r(w)$, as claimed. $\square$



In practice we never draw A , B , and T explicitly. The theorem licenses a single phrase in any algorithm: “*obtain own description* $\langle M \rangle$ *via the recursion theorem.*” After that line, $\langle M \rangle$ is just another value the machine has computed and may print, measure, or simulate.

Remark. The recursion theorem does *not* give the machine magical knowledge of its future behavior – only of its own *syntax* $\langle M \rangle$. Whatever the machine then does with $\langle M \rangle$ (simulate it, count its states, run a purported decider on it) is ordinary computation that may, of course, fail to halt.

Worked examples

Worked Example 11.1.4 (SELF written in one line with the recursion theorem). Reconstruct the self-printing machine of Theorem 11.1.2 using only the “obtain own description” phrase, with no explicit A/B split.

Recognition. The task is “a machine that outputs its own code.” The trigger phrase “its own” signals the recursion theorem: design T to do the easy job assuming the code is handed to it, then let the theorem supply the code.

Solution. Take T to be the trivial machine that, given a pair, prints its first component:

$$T = \text{“On input } \langle M, w \rangle: \text{ print } \langle M \rangle \text{ and halt.”}$$

By Theorem 11.1.3 there is a machine R with $r(w) = t(\langle R \rangle, w) = \langle R \rangle$ for every w . So R ignores its input and prints $\langle R \rangle$ – it is exactly SELF. Phrased directly:

$$\text{SELF} = \text{“On any input: obtain own description } \langle \text{SELF} \rangle; \text{ print it and halt.”}$$

After it runs, the tape holds precisely $\langle \text{SELF} \rangle$:



Which idea, and why? The recursion theorem turns “refer to myself” into “read my code as a value,” so the job reduces to the trivial T that echoes its first argument.

Check. The first argument of t is always $\langle R \rangle$, and t prints exactly that argument, so the output is $\langle R \rangle$ regardless of w – a genuine self-printer (a *quine*).

Worked Example 11.1.5 (A quine in plain language, with no self-referential “this”). Write an English instruction that causes the reader to reproduce the very sentence read, without using a word like “this sentence.”

Recognition. “Reproduce itself” is again the recursion-theorem pattern. The clause “the second one in quotes” plays the role of q : quoting a string is exactly building a printer for it.

Solution. Use the A/B split with quoting as q :

Print out two copies of the following, the second one in quotes:
 “Print out two copies of the following, the second one in quotes:”

The quoted line is the *data* ($\langle B \rangle$ on the tape); the unquoted line is the *program* (B) that, reading the data, reprints it twice – once as program, once quoted as data – yielding the whole sentence again.

Which idea, and why? Quoting is the q of Lemma 11.1.1: it converts a string into something that, when run by the reader, prints that string; the unquoted copy then reassembles program plus data.

Check. Executing the two lines reproduces both lines in the same order, so the output equals the input sentence – a self-reproducing instruction with no literal self-reference.

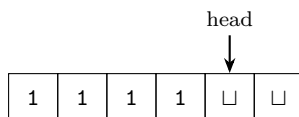
Worked Example 11.1.6 (A machine that announces its own state count). Design a machine G that ignores its input and writes in unary the number of states in its own description $\langle G \rangle$.

Recognition. “Its own description” is the trigger. Once the recursion theorem hands G the string $\langle G \rangle$, counting states is ordinary string processing.

Solution.

G = “On any input: (1) obtain $\langle G \rangle$; (2) scan $\langle G \rangle$ and count its state symbols k ; (3) write 1^k and halt.”

There is no circularity: $\langle G \rangle$ is a fixed string the moment G is built, and step (2) is a finite scan. The output tape after G runs, for a machine of, say, four states:



Which idea, and why? Anything computable *from* a machine’s text is computable *by* that machine, because the recursion theorem makes the text available as input – here, the state count.

Check. The number written is fixed once $\langle G \rangle$ is fixed; it does not depend on the input, so G writes the same unary numeral on every run.

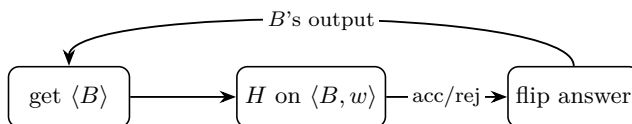
Worked Example 11.1.7 (A new, diagonal proof that A_{TM} is undecidable). Use the recursion theorem to show $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$ is undecidable, with no explicit diagonalization.

Recognition. We want a contradiction from a supposed decider H for A_{TM} . The phrase “a machine that asks H about *itself*” is the recursion-theorem signal.

Solution. Suppose H decides A_{TM} . Build

B = “On input w : (1) obtain $\langle B \rangle$; (2) run H on $\langle B, w \rangle$; (3) do the *opposite* of H ’s answer.”

Then for every w : B accepts w iff H says “ B rejects w ” iff (since H is correct) B rejects w – a contradiction. So no such H exists.



Which idea, and why? The recursion theorem replaces Cantor’s diagonal table: instead of arranging machines in a grid and walking the diagonal, B confronts the decider with its own description directly.

Check. The contradiction has no escape clause: H is assumed total, so step (2) always halts, and step (3) always disagrees with it. Thus A_{TM} has no decider.

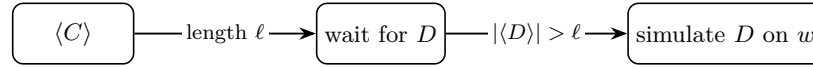
Worked Example 11.1.8 (MIN_{TM} is not Turing-recognizable). Let $MIN_{\text{TM}} = \{\langle M \rangle \mid M \text{ is minimal}\}$, where M is *minimal* if no equivalent machine has a shorter description. Show MIN_{TM} is not recognizable.

Recognition. “Equivalent but shorter” compares a machine’s behavior with its own length; building a machine that compares against *its own* description is the recursion-theorem cue.

Solution. Suppose an enumerator E lists MIN_{TM} . Build

C = “On input w : (1) obtain $\langle C \rangle$; (2) run E until some D appears with $|\langle D \rangle| > |\langle C \rangle|$; (3) simulate D on w .”

Because MIN_{TM} is infinite, E eventually lists a machine D longer than C , so step (2) halts. Then C and D are equivalent (step 3 makes C run D), yet $|\langle C \rangle| < |\langle D \rangle|$, so D is *not* minimal. But D was on E 's list of minimal machines – contradiction.



Which idea, and why? Self-reference lets a *short* machine C adopt the behavior of a *longer* “minimal” machine D , exposing D as non-minimal – impossible if E 's list were correct.

Check. The argument needs only that MIN_{TM} is infinite (so a longer D appears) and that C knows its own length; both hold, so no enumerator exists.

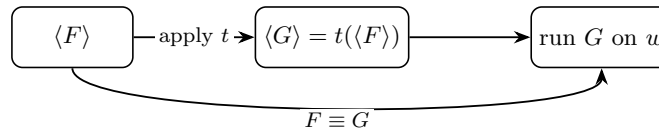
Worked Example 11.1.9 (A fixed point of any transformation of programs). Let $t: \Sigma^* \rightarrow \Sigma^*$ be computable (think: a compiler optimizer, a code obfuscator, or “add a useless extra state”). Show there is a machine F with $t(\langle F \rangle)$ describing a machine *equivalent* to F – a program whose behavior is unchanged by t .

Recognition. “Behavior unchanged by the transformation” is a fixed-point statement; the recursion theorem is the fixed-point engine. The trigger is that t rewrites *descriptions* of machines.

Solution. Define

$F = \text{“On input } w: (1) \text{ obtain } \langle F \rangle; (2) \text{ compute } t(\langle F \rangle) = \langle G \rangle; (3) \text{ simulate } G \text{ on } w.”}$

Then F runs exactly as G does, so $L(F) = L(G)$; that is, $\langle F \rangle$ and $t(\langle F \rangle) = \langle G \rangle$ describe equivalent machines. (We read any non-machine string $t(\langle F \rangle)$ as a machine that rejects at once, so $t(\langle F \rangle)$ always denotes some machine.)



Which idea, and why? The recursion theorem gives F access to $\langle F \rangle$, so F can *become* whatever t turns it into; running $t(\langle F \rangle)$ makes F and its t -image behave identically.

Check. There is no contradiction here – the conclusion is positive: *every* computable transformation of code has a semantic fixed point. Even “flip accept and reject states” has one: some machine is unaffected, because it neither accepts nor halts where flipping would matter.

Method and pitfalls

Method (How to use the recursion theorem when designing a machine).

1. **Spot the self-reference.** If the specification mentions the machine’s *own* description, code, length, or behavior, the recursion theorem applies. Phrases like “prints itself,” “asks a decider about itself,” or “a fixed point” are the giveaways.
2. **Pretend the code is free.** Write the algorithm as if its first step were the line “obtain own description $\langle M \rangle$.” From that point treat $\langle M \rangle$ as an ordinary input string.
3. **Do the real work as plain computation.** Whatever follows – printing $\langle M \rangle$, scanning it, running another machine on $\langle M, w \rangle$ – is ordinary, possibly non-halting, computation. Design it without worrying about circularity.
4. **Discharge the phrase.** The construction $R = ABT$ implements “obtain own description” once and for all; you may cite Theorem 11.1.3 and never expand A , B , T again.
5. **Read off the consequence.** For an impossibility proof, derive a contradiction from a supposed decider/enumerator; for a possibility result (quine, fixed point), conclude that the machine exists.

Caution. Do not define one part in terms of the other. Setting part $B = P_{\langle A \rangle}$ in the SELF construction is circular, since $A = P_{\langle B \rangle}$ already mentions B . The whole device exists to *avoid* this: B recomputes $\langle A \rangle$ from the tape contents $\langle B \rangle$ by applying q , never referring to A by name. “Obtain own description” is legitimate precisely because the recursion theorem reduces it to this non-circular tape trick.

Caution. The recursion theorem gives a machine its *syntax*, not its *semantics*. A machine can read its own description $\langle M \rangle$, but it cannot, in general, decide what $\langle M \rangle$ *does* – that would solve the halting problem. So “obtain own description” is free, but “determine whether I halt on w ” is not; the latter still requires running a (possibly diverging) simulation.

Caution. In the A_{TM} proof, the contradiction depends on H being a *total decider*. If H merely *recognizes* A_{TM} (may loop on rejects), then step (2) of B might not halt, and “do the opposite” never executes – no contradiction follows. This is exactly why the same argument shows A_{TM} is undecidable but does *not* show it is unrecognizable (indeed it is recognizable).

Exercises.

1. State the function q of Lemma 11.1.1 in your own words, then explain in two sentences why q is computable but the map $\langle M \rangle \mapsto L(M)$ (a machine to its language) is not.
2. Using the recursion theorem, design a machine that ignores its input and writes the number of 0s appearing in its own description $\langle M \rangle$, in unary. Identify the single step that is “free” by the theorem and the steps that are ordinary computation.
3. Describe a machine W that, on any input, obtains $\langle W \rangle$ and then loops forever. Is $\langle W \rangle$ a member of $\{\langle M \rangle \mid M \text{ halts on the empty input}\}$? Justify.
4. Give an English sentence, in the quoting style of the chapter, that prints *three* copies of itself rather than one. Indicate which part plays the role of the data $\langle B \rangle$ and which plays the program B .
5. Explain precisely where the proof that A_{TM} is undecidable would break if the assumed machine H were only a *recognizer* for A_{TM} rather than a decider. Which step might fail to halt?
6. Let t be the computable transformation that, given $\langle M \rangle$, outputs the description of a machine that runs M but swaps its accept and reject states. Use Theorem 11.1.3 to argue that t has a fixed point F with $L(F) = L(t(F))$, and describe in one sentence what such an F must do on every input.
7. *Find the bug.* A student defines SELF by “part A prints $\langle B \rangle$ and part B is defined as $P_{\langle A \rangle}$, the printer of A ’s code.” Explain why this definition is circular and is *not* what the construction in Theorem 11.1.2 does. What does B actually do instead?
8. Show that the language $L_{\text{self}} = \{\langle M \rangle \mid M \text{ prints } \langle M \rangle \text{ on every input}\}$ is nonempty by exhibiting one member, and explain why the recursion theorem guarantees at least one such machine exists.
9. Sketch a machine V (a “virus”) that, on any input, obtains $\langle V \rangle$, appends $\langle V \rangle$ to the output, and then performs some other task τ on its input. Argue that V exists for every computable τ , and say which line of V the recursion theorem provides.
10. Let t be *any* computable function on machine descriptions. Prove that there is a machine F such that F and the machine described by $t(\langle F \rangle)$ recognize the same language. State clearly the convention you use when $t(\langle F \rangle)$ is not a valid machine encoding.
11. *True or false, with justification.* “By the recursion theorem, every machine can decide whether it halts on its own description $\langle M \rangle$, since it can obtain $\langle M \rangle$ and inspect it.” Decide and explain, distinguishing a machine’s syntax from its semantics.
12. Using the recursion theorem, give a short proof that $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$ is undecidable. Model your machine on the A_{TM} argument and state exactly what contradiction arises.

11.2 Applications of the Recursion Theorem

The recursion theorem says that a Turing machine can be built to obtain its own description and then compute with it; the previous section produced that description and used it only to print a copy of the machine. The one key idea of this section is that the self-reference is a *programming primitive*: when we design a machine M we may freely write the instruction “obtain own description $\langle M \rangle$ ” and then use $\langle M \rangle$ like any other value – count its states, feed it to another machine, or simulate it. Three payoffs follow almost

mechanically. We re-prove that A_{TM} is undecidable in two lines, we show that the set of size-optimal machines is not even recognizable, and we prove a fixed-point theorem stating that every computable transformation of programs leaves some program's behavior unchanged. Throughout, the move is the same: assume a decider or enumerator exists, then build a self-referential machine that contradicts it.

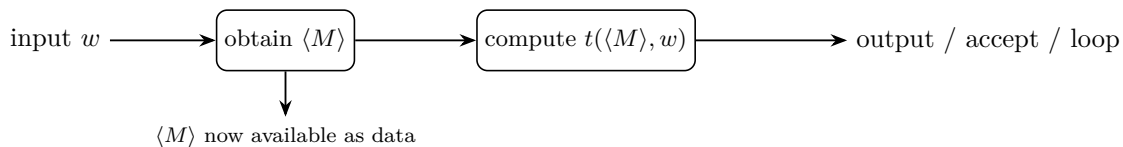
The tool, restated for use. We will lean on one statement, proved in the previous section, and quote it in the form we actually apply.

Theorem 11.2.1 (Recursion theorem, design form). *Let T compute a function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Then there is a Turing machine R such that for every input w ,*

$$R(w) = t(\langle R \rangle, w),$$

that is, R runs exactly as T would run if handed R 's own description as an extra argument. Equivalently: in describing a machine M we may include the step “obtain own description $\langle M \rangle$,” and a genuine M realizing that description exists.

The equivalence between the formula $R(w) = t(\langle R \rangle, w)$ and the informal “obtain own description” license is the entire content. Whenever a construction below writes a step like “obtain $\langle M \rangle$,” read it as: package everything after that step as the machine T taking $(\langle M \rangle, w)$, and let the recursion theorem manufacture the M that supplies its own description automatically.



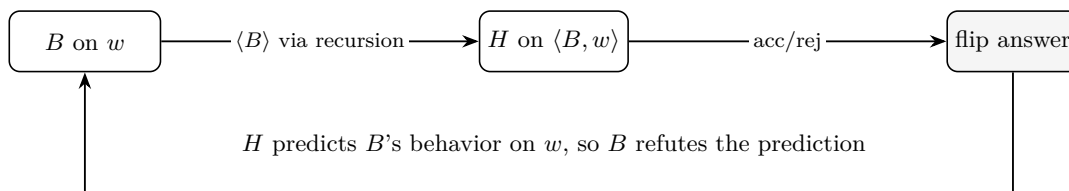
First payoff: a two-line undecidability proof. The diagonal proof that A_{TM} is undecidable builds a machine that simulates the assumed decider and flips its answer, then runs that machine on its own encoding. The recursion theorem hides the “runs on its own encoding” bookkeeping inside step 1, so only the flip remains.

Theorem 11.2.2. $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is undecidable.

Proof. Suppose, for contradiction, that some TM H decides A_{TM} : H halts on every input and accepts $\langle M, w \rangle$ iff M accepts w . Build a machine B :

- $B =$ “On input w : 1. obtain own description $\langle B \rangle$ (recursion theorem);
 2. run H on $\langle B, w \rangle$;
 3. if H accepts, *reject*; if H rejects, *accept*.”

By the recursion theorem such a B exists. Now fix any w ; since H halts, so does B , and we ask whether B accepts w . If B accepts w , then by correctness of H the run of H on $\langle B, w \rangle$ in step 2 accepts – but then step 3 makes B reject w , a contradiction. If B rejects w , then B does *not* accept w , so by correctness H rejects $\langle B, w \rangle$; yet then step 3 makes B accept, again a contradiction. Either way B does the opposite of what H predicts about B , so H does not decide A_{TM} . $\square$



Caution. The classic diagonal argument needed a separate machine D that, on input $\langle M \rangle$, simulated H on $\langle M, \langle M \rangle \rangle$ and then asked what happens at $D(\langle D \rangle)$. The recursion-theorem proof seems to skip that step, but it does not eliminate the self-application – it *absorbs* it into the phrase “obtain own description.” Do not read the short proof as avoiding self-reference; it merely delegates the construction of the self-referential object to Theorem 11.2.1.

Second payoff: optimal machines are unrecognizable. Call $\langle M \rangle$ *minimal* if no Turing machine equivalent to M (one recognizing the same language) has a strictly shorter description, where the *length* $|\langle M \rangle|$ is the number of symbols in the description. These are the size-optimal programs.

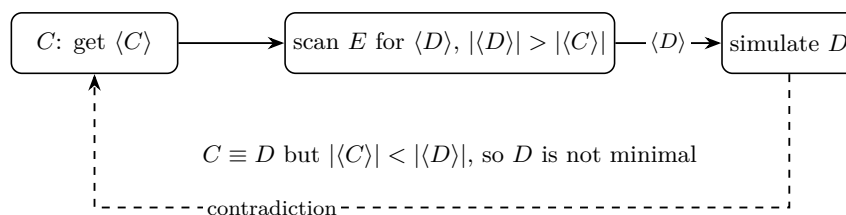
Definition 11.2.3. $MIN_{TM} = \{\langle M \rangle \mid M \text{ is a TM and no equivalent TM has a shorter description}\}.$

Theorem 11.2.4. MIN_{TM} is not Turing-recognizable.

Proof. Recall that a language is Turing-recognizable iff some enumerator lists exactly its members. Suppose for contradiction that an enumerator E prints exactly the strings in MIN_{TM} , in some order. There are arbitrarily long minimal machines – for instance a machine recognizing a sufficiently complicated finite language cannot be compressed below a length growing with the language – so MIN_{TM} is infinite and E ’s list contains arbitrarily long descriptions. Build:

$C =$ “On input w :
 1. obtain own description $\langle C \rangle$ (recursion theorem);
 2. run E until it prints some $\langle D \rangle$ with $|\langle D \rangle| > |\langle C \rangle|$;
 3. simulate D on w and do as D does.”

Because MIN_{TM} is infinite, step 2 must eventually find such a D , so C halts step 2 on every input and then behaves identically to D ; hence C and D are equivalent. But $|\langle C \rangle| < |\langle D \rangle|$, so D has a strictly shorter equivalent machine, namely C – contradicting $D \in MIN_{TM}$, which is why E printed it. The enumerator cannot exist, so MIN_{TM} is not recognizable. $\square$



Worked Example 11.2.5 (Why the proof refutes an enumerator rather than a decider). Explain why the proof of Theorem 11.2.4 attacks *recognizability* rather than mere *decidability*, and what would change if we only assumed a decider for MIN_{TM} .

Recognition. The trigger is the phrase “run E until...”: the construction consumes an *enumerator*, which is exactly the witness for recognizability. Whenever a proof’s self-referential machine *searches a list* rather than *queries a yes/no box*, it is refuting an enumerator, hence recognizability.

Solution. The machine C needs an actual description $\langle D \rangle$ longer than itself that it can then *simulate*. The enumerator supplies one: it streams the members out, so C waits for one longer than $|\langle C \rangle|$. A decider for MIN_{TM} , which only answers “is $\langle M \rangle$ minimal?”, would supply one too, but only by being turned into an enumerator: test the strings longer than $|\langle C \rangle|$ in order until it accepts one. Since every decider yields an enumerator in this way, assuming only an enumerator is the *weaker* hypothesis, and refuting it gives the unrecognizability conclusion, which is strictly stronger than undecidability (non-recognizable implies undecidable) – so we get the sharper result for free.

Which idea, and why? Refuting an enumerator yields “not Turing-recognizable”; refuting a decider yields only “undecidable.” Choose the assumed object to match the strongest conclusion the self-referential machine can contradict.

Check. If instead we had assumed a decider R , the same C works with step 2 replaced by “test the strings longer than $|\langle C \rangle|$ in order until R accepts one”; the contradiction arises exactly as before, but it refutes only a decider, so it shows only that MIN_{TM} is undecidable. (Merely asking R whether $\langle C \rangle$ itself is minimal is not enough: that gives C no *longer equivalent* machine to clone.)

Third payoff: a fixed-point theorem. A *fixed point* of a transformation is an input the transformation leaves unchanged. Here the transformation t rewrites program descriptions – think “add a useless state,” “swap the accept and reject states,” or “optimize the machine.” One might hope to always change a program’s behavior by rewriting its code; the recursion theorem forbids it.

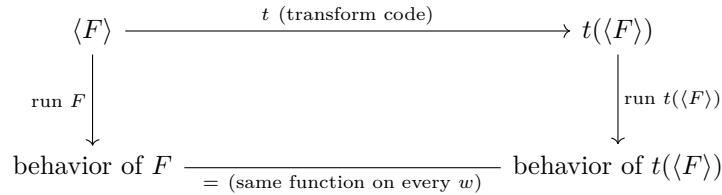
Theorem 11.2.6 (Fixed-point theorem). *Let $t: \Sigma^* \rightarrow \Sigma^*$ be computable. Then there is a Turing machine F such that $t(\langle F \rangle)$ describes a machine equivalent to F (it recognizes the same language and, in fact, agrees with F on every input). We adopt the convention that a string that is not a valid encoding describes the machine that rejects at once.*

Proof. Define

- $F =$ “On input w : 1. obtain own description $\langle F \rangle$ (recursion theorem);
 2. compute $t(\langle F \rangle)$, calling the result $\langle G \rangle$;
 3. simulate G on w and do as G does.”

By the recursion theorem F exists. On every input w , machine F does exactly what G does, where $\langle G \rangle = t(\langle F \rangle)$. Hence F and the machine $G = t(\langle F \rangle)$ accept the same inputs, reject the same inputs, and loop on the same inputs: $t(\langle F \rangle)$ describes a machine equivalent to F . So $\langle F \rangle$ is the desired fixed point of t , up to behavioral equivalence. $\square$

The construction reads off as a square on *machine descriptions*: the top arrow is the code transformation t , the two downward arrows are “run the described machine,” and the fixed point F is built so that both runs land on one and the same computed function.



Takeaway. The recursion theorem makes self-reference legal: for any computable transformation t of code, some machine F behaves exactly like $t(\langle F \rangle)$, so a program may use its own description as data. The invariant preserved around the square is the *computed behavior* – the bottom edge is an equality of functions, not of strings.

Check. Correct (Kleene’s recursion / fixed-point theorem): there is a computable construction giving F with $\langle F \rangle$ available to it, so F and $t(\langle F \rangle)$ compute the same function. The square commutes up to behavioral equivalence even though $t(\langle F \rangle) \neq \langle F \rangle$ as strings.

Remark. “Fixed point” here is up to *behavioral equivalence*, not string equality. We are not claiming $t(\langle F \rangle) = \langle F \rangle$ as strings – the transformation really may change the code. What cannot change is what the code *does*: F is engineered to run whatever code t hands back, so its behavior chases $t(\langle F \rangle)$ and the two coincide.

Worked Example 11.2.7 (No total computable “behavior-changer” on programs). Show there is no computable t that, for *every* TM M , outputs $t(\langle M \rangle)$ describing a machine that is *not* equivalent to M (a machine that always changes behavior).

Recognition. The task asks for a transformation with *no* fixed point. The trigger “produce a program guaranteed to behave differently from its input” is precisely the negation of the fixed-point theorem, so the theorem is the tool.

Solution. Suppose such a t were computable. By Theorem 11.2.6 there is an F with $t(\langle F \rangle)$ equivalent to F . That contradicts the assumption that $t(\langle M \rangle)$ is inequivalent to M for every M , in particular for $M = F$. So no such t exists.

Which idea, and why? The fixed-point theorem is exactly the statement “every computable program transformation has a behavioral fixed point,” so any demand for a universally behavior-changing transformation is refuted by exhibiting that fixed point.

Check. A concrete attempt: let t flip a machine’s accept and reject decisions (complement its language among halting inputs). One feels this must change behavior, yet the theorem guarantees some F whose “complemented” code still does what F does – possible because F may loop on the very inputs where flipping would have mattered.

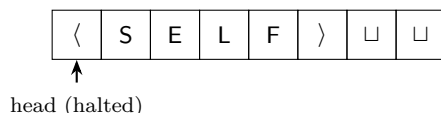
Worked Example 11.2.8 (A program that prints its own source, via the design pattern). Use the “obtain own description” primitive to specify **SELF**, a machine that on every input halts with $\langle \text{SELF} \rangle$ written on its tape.

Recognition. The trigger is “output a copy of yourself.” Self-output is the canonical use of step 1; everything after step 1 is an ordinary program acting on the data $\langle M \rangle$.

Solution. Specify

SELF = “On any input: 1. obtain own description $\langle \text{SELF} \rangle$; 2. print $\langle \text{SELF} \rangle$ and halt.”

To see this is legitimate under Theorem 11.2.1, let T be the machine $T(\langle M \rangle, w) = \langle M \rangle$ (print the first component, ignore w). The recursion theorem delivers R with $R(w) = T(\langle R \rangle, w) = \langle R \rangle$, and this R is **SELF**. The tape after halting holds the encoding and nothing else:



Which idea, and why? “Print own description” is the recursion theorem with $t(\langle M \rangle, w) = \langle M \rangle$; the self-reference that a naive quine struggles with is supplied wholesale by step 1.

Check. Replacing step 2 by “count the states of $\langle \text{SELF} \rangle$ and print that number” is equally valid – once $\langle M \rangle$ is data, any computation on it is allowed; the print-a-copy version is just the simplest.

Worked Example 11.2.9 (Computer viruses as applied self-reference). Sketch why a self-replicating program (a virus) is an instance of the recursion theorem, and identify the part that plays the role of step 1.

Recognition. The trigger is “a program that writes a copy of itself elsewhere.” Any self-replication is self-output plus a side effect, so it is the **SELF** pattern with extra work appended.

Solution. Model the virus as

V = “On any input: 1. obtain own description $\langle V \rangle$; 2. write $\langle V \rangle$
into a fresh host; 3. run the payload.”

Step 1 is the recursion-theorem primitive: it furnishes $\langle V \rangle$ without the program needing a literal self-pointer. Steps 2–3 are an ordinary program over the data $\langle V \rangle$. Thus the theorem certifies that genuine self-replicators exist; the “machine cannot be more complex than itself” paradox dissolves because V never *builds* a richer machine, it merely *copies a description it was handed*.

Which idea, and why? Self-replication is **SELF** with a propagation payload bolted on after step 2; the existence of the self-copy is exactly Theorem 11.2.1.

Check. Remove step 1 and hard-code a string s to copy: then the copy is s , not V , unless s happens to equal $\langle V \rangle$ – and arranging that by hand is the quine puzzle the recursion theorem solves once and for all.

Method (How to use the recursion theorem to prove a negative result).

1. **Name the object you want to rule out** – a decider H , an enumerator E , or a behavior-changing transformation t – and assume it exists.
2. **Design a self-referential machine** M whose first step is “obtain own description $\langle M \rangle$.” After step 1, write a perfectly ordinary algorithm that uses $\langle M \rangle$ as input data together with the assumed object.
3. **Make M contradict the assumption about itself.** Have M query or search the assumed object about $\langle M \rangle$, then act so that M ’s actual behavior is inconsistent with what the object reports.
4. **Invoke Theorem 11.2.1** to certify that M exists, then read off the contradiction. Match the assumed object to the conclusion: contradict a decider for “undecidable,” an enumerator for “not recognizable.”

Caution. A computable transformation t in the fixed-point theorem must be *total*: it has to return a description for every input string. If a candidate “transformation” is only partial – undefined or looping on some encodings – it is not a function $\Sigma^* \rightarrow \Sigma^*$ and the theorem does not apply, so do not invoke a fixed point for it.

Caution. When designing the self-referential M , every later step must be effective *given* $\langle M \rangle$ as ordinary data. Writing “2. decide whether $\langle M \rangle$ halts on w ” is illegal, because that step is not computable – the recursion theorem hands you the *description* $\langle M \rangle$, never an oracle for M ’s behavior. The primitive supplies code, not a crystal ball.

Exercises.

1. Using the recursion theorem, give a machine $COUNT$ that on every input halts with the number of states in its own description, written in binary, on the tape. Identify the function $t(\langle M \rangle, w)$ you are feeding to Theorem 11.2.1.
2. Let $TWICE$ be the machine that obtains its own description $\langle TWICE \rangle$ and prints it *twice*, separated by a blank. Describe $TWICE$ in the “obtain own description” style, and state precisely what string ends up on the tape.
3. Reprove that $HALT_{TM} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$ is undecidable using the recursion theorem. Assume a decider H for $HALT_{TM}$, build a self-referential machine B that consults H about $\langle B, w \rangle$, and derive a contradiction. State carefully what B does in each of the two cases.
4. Find the bug. A student claims the following proves A_{TM} undecidable: “Assume H decides A_{TM} . Build B that obtains $\langle B \rangle$, then *decides whether* B *accepts* w directly and outputs the opposite.” Explain exactly why this construction is not a legal use of the recursion theorem, citing which step fails to be computable.
5. Let t be the computable transformation that, given $\langle M \rangle$, outputs the description of a machine that on input w runs M on w and, if M accepts, loops forever instead. By the fixed-point theorem there is an F with $t(\langle F \rangle)$ equivalent to F . Describe the language $L(F)$ that is forced by this fixed point, and explain why it is consistent that “flip accept to loop” did not change F ’s behavior.
6. Show that there is a Turing machine G whose language is $\{\langle G \rangle\}$ – it accepts exactly its own description and rejects everything else. Use the recursion theorem and explain why no special quine trickery is needed.
7. True or false, with justification: there is a computable function f such that for every TM M , $f(\langle M \rangle)$ is the description of a machine recognizing a language *different* from $L(M)$. Relate your answer to Theorem 11.2.6.
8. Prove that MIN_{TM} is infinite, as the proof of Theorem 11.2.4 assumed. Argue that for each length n there is at least one minimal machine of description length $\geq n$, and conclude the set is unbounded in length.
9. Let $E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$. Using the recursion theorem, build a machine M_0 such that $\langle M_0 \rangle \in E_{TM}$ even though M_0 ’s code explicitly contains an accepting state. Explain how a machine with an accept state can still recognize the empty language, and why self-reference makes the construction clean.
10. Find the bug. A proposed proof that MIN_{TM} is undecidable runs: “Assume a decider R . Build C that obtains $\langle C \rangle$, asks R whether $\langle C \rangle$ is minimal, and if R says yes, makes itself non-minimal by padding.” Explain why padding C after the fact does not produce a contradiction, and what genuine ingredient (present in the unrecognizability proof) is missing here.
11. Combine two applications: assuming a decider H for A_{TM} , use the recursion theorem to build a machine M that obtains $\langle M \rangle$, asks H whether M accepts the string 0, and then *accepts* 0 exactly when H says it does not, while looping on all other inputs. State the contradiction and the condition the language $L(M)$ would have to satisfy.
12. The fixed-point theorem yields one fixed point. Show that in fact *infinitely many* pairwise inequivalent machines F satisfy “ $t(\langle F \rangle)$ is equivalent to F ” for a suitable t , or give a t for which this fails. (Hint: consider t equal to the identity transformation, and count behaviorally distinct machines.)

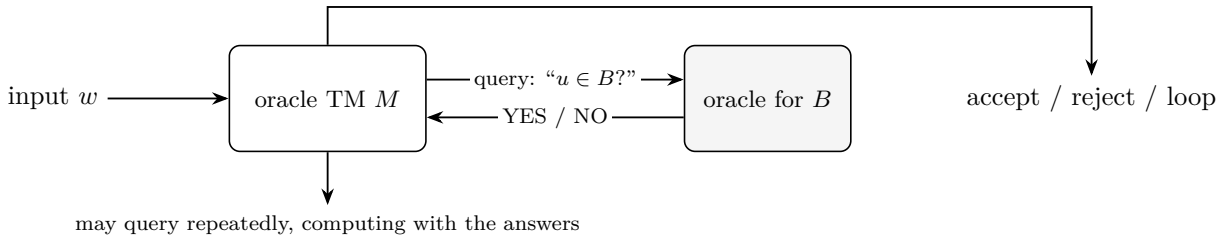
11.3 Turing Reducibility and Oracle Machines

Mapping reducibility, from Chapter 10, captures one way to use a solution to B as a solution to A : transform the question about A into a single question about B . But that is too rigid – it forbids consulting B more

than once, and it cannot flip an answer, so a problem and its complement need not be mapping-reducible to each other even when each trivially solves the other. The one key idea of this section is to free the reduction: give a Turing machine an *oracle*, a black box that answers membership queries about B for free, and let the machine query it as often as it likes and compute with the answers however it likes. The resulting notion, *Turing reducibility* $A \leq_T B$, is the most general sense in which “a solver for B yields a solver for A ,” it subsumes mapping reducibility, and it repairs the asymmetry: a language and its complement are always Turing-equivalent. This connects directly to the previous section’s undecidability results – A_{TM} reappears here as the canonical oracle, powerful enough to decide many of the problems we have so far proved undecidable, yet (as the arithmetic hierarchy will show) still not powerful enough to decide everything.

Definition 11.3.1 (Oracle and oracle Turing machine). An *oracle* for a language $B \subseteq \Sigma^*$ is an external device that, given any string w , instantly and correctly reports whether $w \in B$. An *oracle Turing machine* M^B is an ordinary Turing machine augmented with a distinguished *query tape* and three special states $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$: whenever M enters q_{query} with a string w on the query tape, in one step it moves to q_{yes} if $w \in B$ and to q_{no} if $w \notin B$. Apart from the oracle, M^B computes exactly like a standard Turing machine.

We make no assumption about *how* the oracle knows the answer; B may be wildly undecidable. The oracle is a hypothetical, a way to ask “if membership in B were free, what else could I compute?” The superscript records which language the box answers for: the same program text M becomes a different machine M^B for each choice of B .



Definition 11.3.2 (Decidable relative to B ; Turing reducibility). A language A is *decidable relative to B* if some oracle Turing machine M^B is a decider for A – it halts on every input and accepts exactly the strings of A . In that case we say A is *Turing reducible* to B and write

$$A \leq_T B.$$

If additionally $B \leq_T A$, we call A and B *Turing equivalent*, written $A \equiv_T B$.

The relation $\leq_T$ is a preorder: it is reflexive ($A \leq_T A$ via the machine that just queries its own oracle) and transitive (Worked Example 11.3.7). Turing equivalence $\equiv_T$ is the induced equivalence relation, and its classes are the *Turing degrees* – the precise sense in which two problems are “equally hard” for an unrestricted algorithm with a black box.

Theorem 11.3.3 (Turing reducibility respects decidability). *If $A \leq_T B$ and B is decidable, then A is decidable.*

Proof. Let M^B be an oracle decider for A , and let D be an ordinary Turing machine that decides B . Build an ordinary machine M' that simulates M^B step for step, except that each time M^B would enter q_{query} on a query string u , the machine M' pauses the simulation, runs D on u to completion, and resumes the simulation in q_{yes} or q_{no} according to D ’s answer. Since D halts on every u and M^B halts on every input, M' halts on every input and accepts exactly when M^B does, i.e. exactly on A . Thus M' decides A without any oracle. $\square$

This is the property that makes $\leq_T$ a genuine notion of reducibility: solving the harder problem B really does solve A . Its contrapositive is the form we use to prove negative results: if A is undecidable and $A \leq_T B$, then B is undecidable.

Theorem 11.3.4 (Turing reducibility generalizes mapping reducibility). *If $A \leq_m B$ then $A \leq_T B$.*

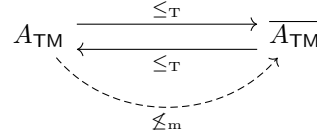
Proof. Let f be a computable function with $w \in A \iff f(w) \in B$ for all w . The oracle machine “on input w : compute $f(w)$, write it on the query tape, query the oracle, and accept iff the answer is YES” is an oracle decider for A using oracle B . Hence $A \leq_T B$. $\square$

The converse fails, and the failure is exactly what motivated the whole section. A mapping reduction is a *single* query with *no* post-processing of the answer; Turing reducibility allows many queries and arbitrary post-processing, including negation. The next example pins down a pair that separates the two notions.

Worked Example 11.3.5 (A_{TM} and $\overline{A_{\text{TM}}}$ are Turing equivalent but not mapping reducible). Show $A_{\text{TM}} \equiv_T \overline{A_{\text{TM}}}$, yet $A_{\text{TM}} \not\leq_m \overline{A_{\text{TM}}}$.

Recognition. The phrase “a problem and its complement” is the trigger. Complementation is free for an oracle machine (just swap the accept and reject reactions to the oracle’s answer) but illegal for a mapping reduction, which can never flip a yes/no answer.

Solution. For the Turing equivalence, note that an oracle for $\overline{A_{\text{TM}}}$ answers “ $u \in \overline{A_{\text{TM}}}$?” which is the exact negation of “ $u \in A_{\text{TM}}$?”. The machine “on input x : query the oracle about x ; accept iff the oracle says NO” decides A_{TM} relative to $\overline{A_{\text{TM}}}$, giving $A_{\text{TM}} \leq_T \overline{A_{\text{TM}}}$; swapping “NO” to “YES” gives the reverse, so the two are Turing equivalent. For the mapping non-reduction, recall from the earlier chapters that A_{TM} is Turing-recognizable but $\overline{A_{\text{TM}}}$ is not. If we had $A_{\text{TM}} \leq_m \overline{A_{\text{TM}}}$ then, since mapping reducibility transfers recognizability in the form “ $A \leq_m C$ and C recognizable $\Rightarrow A$ recognizable” applied to the *complements* ($\overline{A_{\text{TM}}} \leq_m A_{\text{TM}}$ would make $\overline{A_{\text{TM}}}$ recognizable), we reach a contradiction. Concretely: $A \leq_m B$ always implies $\overline{A} \leq_m \overline{B}$ via the same f ; so $A_{\text{TM}} \leq_m \overline{A_{\text{TM}}}$ would give $\overline{A_{\text{TM}}} \leq_m A_{\text{TM}}$, and as A_{TM} is recognizable this would force $\overline{A_{\text{TM}}}$ recognizable – false.



Which idea, and why? Turing reducibility may negate the oracle’s answer, so any language is Turing equivalent to its complement; mapping reducibility cannot negate, so it distinguishes a recognizable language from its non-recognizable complement.

Check. The asymmetry is one-directional in the mapping world only: $A_{\text{TM}} \leq_m A_{\text{TM}}$ and $\overline{A_{\text{TM}}} \leq_m \overline{A_{\text{TM}}}$ trivially, but no $\leq_m$ arrow crosses between them, whereas $\leq_T$ arrows cross both ways.

Worked Example 11.3.6 (Emptiness testing reduces to acceptance: $E_{\text{TM}} \leq_T A_{\text{TM}}$). Recall $E_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$. Build an oracle machine that decides E_{TM} given an oracle for A_{TM} .

Recognition. The trigger is “ $L(M) = \emptyset$,” a statement quantified over *all* inputs: M accepts *no* string. An A_{TM} oracle answers about *one* machine on *one* string, so we engineer a single machine whose acceptance of a single fixed input encodes “ M accepts *something*.”

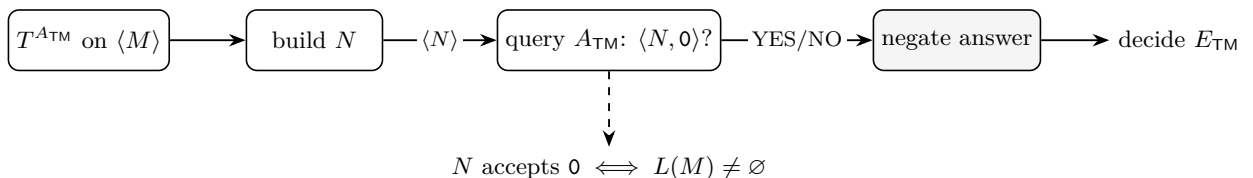
Solution. Given $\langle M \rangle$, build a helper machine

$N =$ “On any input: run M in parallel on $\varepsilon, 0, 1, 00, \dots$ (all of Σ^*); if M accepts any string, accept.”

Then $L(N) = \Sigma^*$ if M accepts at least one string, and $L(N) = \emptyset$ otherwise. In particular N accepts the fixed input 0 iff $L(M) \neq \emptyset$. So define the oracle machine

$T^{A_{\text{TM}}} =$ “On input $\langle M \rangle$:
 1. construct $\langle N \rangle$ as above;
 2. query the A_{TM} oracle about $\langle N, 0 \rangle$;
 3. if the oracle says NO, *accept*; if YES, *reject*.”

The oracle answers YES exactly when N accepts 0, i.e. when $L(M) \neq \emptyset$; so $T^{A_{\text{TM}}}$ accepts exactly when $L(M) = \emptyset$, deciding E_{TM} . It always halts because the oracle replies in one step. Hence $E_{\text{TM}} \leq_T A_{\text{TM}}$.



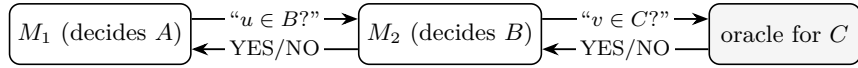
Which idea, and why? An A_{TM} oracle decides “does machine X accept string y ?”; to test a *universally* quantified property (“accepts nothing”) we compress it into one such question by building a machine whose acceptance of a single input mirrors the property, then read the oracle and negate.

Check. If M accepts, say, 11, then N ’s parallel search reaches 11, N accepts 0, the oracle says YES, and $T^{A_{\text{TM}}}$ rejects $\langle M \rangle$ – correct, since $L(M) \neq \emptyset$. If M accepts nothing, N accepts nothing, the oracle says NO, and $T^{A_{\text{TM}}}$ accepts – correct.

Worked Example 11.3.7 (Transitivity of Turing reducibility). Show that if $A \leq_T B$ and $B \leq_T C$ then $A \leq_T C$.

Recognition. The trigger is “chain two reductions.” We have a machine that decides A using a B -oracle and a machine that decides B using a C -oracle; the only B -specific resource in the first machine is its oracle queries, which the second machine can answer.

Solution. Let M_1^B decide A and let M_2^C decide B . Construct an oracle machine M^C that simulates M_1 but, whenever M_1 would query its B -oracle on a string u , instead runs the decider M_2^C on u (using the available C -oracle for M_2 ’s own queries) and feeds M_1 the result $u \in B$ as the oracle answer. Both M_1^B and M_2^C are deciders, so every simulated query terminates and the whole simulation halts; M^C accepts exactly when M_1^B does, i.e. exactly on A . Hence $A \leq_T C$.



Which idea, and why? An oracle query is just a subroutine call; replacing the B -oracle by a decider for B that itself runs against C composes the two reductions, exactly as substituting one program into another.

Check. The construction needs M_2 to be a *decider* (it must answer every query M_1 poses); a mere *recognizer* for B could loop on a non-member u and stall M_1 ’s simulation, which is why transitivity is stated for $\leq_T$ (deciders) rather than for relative recognizability.

The Turing jump. Turing reducibility lets us compare problems, but it also lets us manufacture a strictly harder one from any starting point. Relativize the halting/acceptance problem to an oracle: this is the *jump* operation, the engine that will drive the arithmetic hierarchy in the next section.

Definition 11.3.8 (The Turing jump). For a language B , the *jump* of B is

$$B' = \{ \langle M, w \rangle \mid M^B \text{ is an oracle TM and } M^B \text{ accepts } w \}.$$

Thus B' is the acceptance problem A_{TM} for machines equipped with a B -oracle. Writing $\emptyset$ for the empty oracle (which an ordinary TM uses), $\emptyset'$ is Turing equivalent to the ordinary A_{TM} .

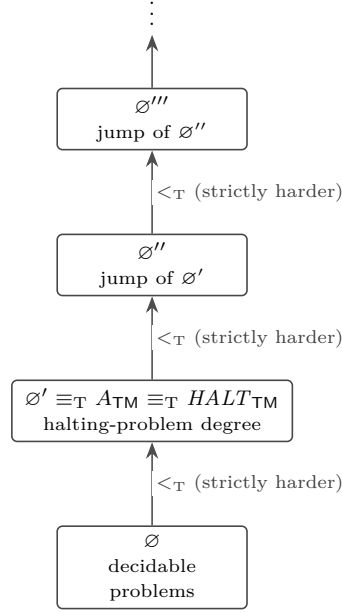
Theorem 11.3.9 (The jump strictly increases degree). *For every language B : $B \leq_T B'$, but $B' \not\leq_T B$. Hence $B <_T B'$ strictly.*

Proof. ($B \leq_T B'$.) To decide “ $x \in B$ ” with a B' -oracle, fix the trivial oracle machine Q that on input x immediately queries its own oracle about x and accepts iff the answer is YES; then Q^B accepts x iff $x \in B$, so $\langle Q, x \rangle \in B' \iff x \in B$. The machine “on input x : query the B' -oracle about $\langle Q, x \rangle$ and copy its answer” decides B . Hence $B \leq_T B'$.

($B' \not\leq_T B$.) Relativize the undecidability of A_{TM} . Suppose some oracle machine R^B decided B' . Build D^B : “on input $\langle M \rangle$, run R^B on $\langle M, \langle M \rangle \rangle$; if it accepts, reject; if it rejects, accept.” Then D^B halts on every input (because R^B does), so $\langle D, \langle D \rangle \rangle \in B' \iff D^B \text{ accepts } \langle D \rangle$. But by construction D^B accepts $\langle D \rangle$ iff R^B rejects $\langle D, \langle D \rangle \rangle$ iff $\langle D, \langle D \rangle \rangle \notin B'$ – a contradiction. So no such R^B exists and $B' \not\leq_T B$. $\square$

$$\emptyset \xrightarrow{<_T} \emptyset' (\equiv_T A_{\text{TM}}) \xrightarrow{<_T} \emptyset'' \xrightarrow{<_T} \dots$$

Read as a Hasse-like tower of Turing degrees, each level strictly harder than the one below it, the same chain stacks vertically; the order is *strict* at every step (no two levels are Turing equivalent), and $\emptyset'$ is the degree of the halting problem.



Takeaway. The Turing jump strictly climbs: $\emptyset <_T \emptyset' <_T \emptyset'' <_T \dots$, where $\emptyset'$ is the degree of the halting problem; each jump is strictly harder under Turing reducibility. The invariant is *relative computability*, and it increases strictly at every level.

Check. Correct: $A <_T A'$ always (by Theorem 11.3.9, A' is A -recognizable but not A -decidable), so the jumps form a strictly increasing tower; and $\emptyset' \equiv_T A_{\text{TM}} \equiv_T \text{HALT}_{\text{TM}}$ (Worked Example 11.3.10). The order is strict at every step: $A \leq_T A'$ but $A' \not\leq_T A$.

The jump produces an infinite strictly increasing tower $\emptyset <_T \emptyset' <_T \emptyset'' <_T \dots$ of ever-harder problems: an oracle for A_{TM} does not let you decide *everything*, because its own jump $\emptyset''$ sits strictly above it. This is the negative half of Section 11.4's hierarchy, seen here purely through reducibility.

Worked Example 11.3.10 ($\text{HALT}_{\text{TM}} \equiv_T A_{\text{TM}}$). Show the halting problem $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$ is Turing equivalent to A_{TM} .

Recognition. “Equivalent” asks for reductions both ways. Halting and accepting differ only by the reject/loop distinction, which an oracle machine can probe by running and asking.

Solution. ($A_{\text{TM}} \leq_T \text{HALT}_{\text{TM}}$.) “On input $\langle M, w \rangle$: query the HALT_{TM} -oracle about $\langle M, w \rangle$. If NO, M loops on w , so reject. If YES, M halts on w ; now *simulate* M on w to its guaranteed halt and accept iff it accepts.” ($\text{HALT}_{\text{TM}} \leq_T A_{\text{TM}}$.) From $\langle M, w \rangle$ build M_1 that runs M on w and accepts if M accepts, and build M_2 that runs M on w and accepts if M rejects. Then M halts on w iff M accepts or rejects w , i.e. iff $\langle M_1, w \rangle \in A_{\text{TM}}$ or $\langle M_2, w \rangle \in A_{\text{TM}}$; query the A_{TM} -oracle on both and accept iff either says YES. Both directions halt, so $\text{HALT}_{\text{TM}} \equiv_T A_{\text{TM}}$.

Which idea, and why? Two languages that “leak” into each other through a constant number of oracle queries plus ordinary computation are Turing equivalent; here one query (sometimes followed by a guaranteed-halting simulation) suffices each way.

Check. The $A_{\text{TM}} \leq_T \text{HALT}_{\text{TM}}$ direction crucially uses the oracle’s YES answer to *license* the otherwise-dangerous simulation of M : knowing M halts on w makes “simulate to the end” an effective step rather than a possible infinite loop.

Method (How to prove $A \leq_T B$ (build an oracle decider)).

1. **Assume the oracle for B is free.** You may, at any point and as often as you like, write a string u and learn “ $u \in B?$ ” in one step.
2. **Describe an algorithm for A that calls the oracle.** On input x , do ordinary computation, form one or more query strings, consult the oracle, and *compute with the answers* – you may negate them, branch on them, or use them to license a further simulation.

3. **Engineer the query.** If the property of x you need is quantified over all inputs (e.g. “ M accepts nothing”), build an auxiliary machine whose acceptance of a *single* fixed string encodes that property, and query the oracle about *it*.
4. **Guarantee halting.** Every oracle reply is instantaneous, so the machine halts provided its non-oracle computation halts; make sure any simulation you run is bounded or is licensed by a prior oracle answer (as in $A_{\text{TM}} \leq_T \text{HALT}_{\text{TM}}$).
5. **Conclude.** The machine is an oracle decider for A using B , so $A \leq_T B$. To get $\equiv_T$, repeat for $B \leq_T A$.

Caution. Turing reducibility is *not* a complexity statement. Calling the oracle is free and unbounded, so $A \leq_T B$ says nothing about running time or about how many queries are used; it only says “a decider for B would give a decider for A .” Do not confuse $\leq_T$ with the polynomial-time reductions $\leq_p$ used for NP-completeness, where the number and cost of the reduction’s steps are the whole point.

Caution. Mapping reducibility $\leq_m$ is the special case “one query, answer copied verbatim.” Because $\leq_T$ may negate the oracle’s answer, a $\leq_T$ reduction can cross between a language and its complement, but a $\leq_m$ reduction cannot. Never argue “ $A \leq_T B$, therefore $A \leq_m B$ ”: the implication runs only the other way (Theorem 11.3.4), and the separating witness $A_{\text{TM}}, \overline{A_{\text{TM}}}$ shows the converse genuinely fails.

Caution. An oracle hands back a single bit, *not* a computation. “Query the oracle about $\langle M, w \rangle$ and then read off M ’s output” is illegal: the box only tells you membership (yes/no), so if you need M ’s actual behavior you must *simulate* M yourself – and that simulation may loop unless a prior oracle answer guarantees it halts.

Exercises.

1. **Warm-up reductions.** For each pair, state whether $A \leq_T B$ is obvious from a single direct construction, and if so give the one-line oracle program: (a) $A = A_{\text{TM}}, B = \text{HALT}_{\text{TM}}$; (b) $A = \overline{E_{\text{TM}}}, B = E_{\text{TM}}$; (c) $A = \emptyset$ (the empty language), B arbitrary; (d) $A = \Sigma^*$, B arbitrary.
2. **Draw the schematic.** Sketch the oracle-machine schematic (input arrow, machine box, two-way arrows to the oracle box labelled “query” and “YES/NO,” output arrow) for a machine $T^{A_{\text{TM}}}$ that decides E_{TM} . Annotate which arrow carries $\langle N, 0 \rangle$.
3. **Complement is free.** Prove from the definitions that for every language B , $B \equiv_T \overline{B}$. Identify the single line of the oracle program that does the work, and explain why the analogous claim for $\leq_m$ is false in general.
4. **Reflexive and transitive.** Show directly that $\leq_T$ is reflexive ($A \leq_T A$ for all A) and recall the transitivity construction. Then explain why $\equiv_T$ is an equivalence relation, so the Turing degrees are well defined.
5. **$\leq_m$ but trace the queries.** Let $A \leq_m B$ via computable f . Exactly how many oracle queries does the oracle decider for A built in Theorem 11.3.4 make on any input, and does it ever negate the oracle’s answer? Contrast this with the $E_{\text{TM}} \leq_T A_{\text{TM}}$ decider of the text.
6. **Acceptance reduces to emptiness.** Show $A_{\text{TM}} \leq_T E_{\text{TM}}$ by building, from $\langle M, w \rangle$, a machine M_w with $L(M_w) \neq \emptyset$ iff M accepts w , and querying the E_{TM} oracle about $\langle M_w \rangle$. State whether your decider negates the oracle’s answer.
7. **Equal-language testing.** Recall $EQ_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$. Show $EQ_{\text{TM}} \leq_T (A_{\text{TM}})'$, the jump of A_{TM} (Definition 11.3.8). (Hint: with an A_{TM} oracle a machine can test whether M_1 and M_2 agree on any one string, so it can search for a string in the symmetric difference $L(M_1) \triangle L(M_2)$; ask the jump whether that search succeeds. An A_{TM} oracle alone does not suffice: EQ_{TM} lies in Π_2 but not in Δ_2 , Section 11.4.)
8. **Find the bug.** A student claims to show $\overline{A_{\text{TM}}} \leq_m A_{\text{TM}}$ as follows: “On input $\langle M, w \rangle$, output $\langle M, w \rangle$ unchanged; the A_{TM} -oracle answers, and we flip its bit, so we have decided $\overline{A_{\text{TM}}}$.” Pinpoint the precise step that violates the definition of a mapping reduction, and name the reducibility under which the construction *does* work.
9. **An oracle that is still not enough.** Using the jump, argue that an oracle for A_{TM} cannot decide every language. Specifically, exhibit (by name) a problem that is undecidable *relative to* A_{TM} , and justify it from Theorem 11.3.9.

10. **Two queries, not one.** Give a property $P(\langle M \rangle)$ of Turing machines that is decidable relative to A_{TM} using exactly *two* oracle queries whose answers you combine with a logical connective (e.g. “ M accepts 0 but rejects/loops on 1”). Write the oracle program and state which connective you use.
11. **Recognition, not decision, breaks transitivity.** Suppose $A \leq_T B$, but B is accepted relative to C only by an oracle machine that is a *recognizer* (it may loop on some non-members). Explain with a concrete failure scenario why the transitivity argument of Worked Example 11.3.7 can break down, identifying the simulated query that may never return.
12. **Jump arithmetic.** Let $B = \emptyset$. Order the three languages B, B', B'' by $\leq_T$, prove no two are Turing equivalent, and identify which one is Turing equivalent to the ordinary acceptance problem A_{TM} . State which separations you are invoking Theorem 11.3.9 for.

11.4 The Arithmetic Hierarchy

Turing reducibility (Section 11.3) lets an oracle for one set decide another, and the jump B' packages “the halting problem relative to B ” into a single set strictly harder than B . This section organizes *all* the sets we have met – decidable, recognizable, their complements, and harder sets still – into one ruler called the *arithmetic hierarchy*. The one key idea is that the difficulty of a set is measured by the *logical complexity of a formula that defines it*: how many times one must alternate “there exists” and “for all” over the natural numbers, on top of a decidable core. Counting those alternations places a set on a definite rung Σ_n or Π_n , recovers “recognizable” and “co-recognizable” as the very first rung, and shows precisely why each new oracle buys genuinely new computational power.

Throughout, we identify strings with natural numbers under a fixed computable bijection, so a “set” $A \subseteq \mathbb{N}$, a language $A \subseteq \Sigma^*$, and a unary relation are interchangeable; $\bar{A}$ denotes the complement. A relation $R(x_1, \dots, x_k) \subseteq \mathbb{N}^k$ is *decidable* if some TM halts on every input and accepts exactly the tuples in R .

Definition 11.4.1 (Arithmetical formula and bounded vs. unbounded quantifiers). An *arithmetical formula* is built from decidable relations on $\mathbb{N}$ using the Boolean connectives $\wedge, \vee, \neg$ and the quantifiers $\exists x$ and $\forall x$ ranging over $\mathbb{N}$. A quantifier is *bounded* if it has the form $\exists x \leq t$ or $\forall x \leq t$ for a term t ; otherwise it is *unbounded*. A set $A \subseteq \mathbb{N}$ is *arithmetical* if $A = \{x \mid \varphi(x)\}$ for some arithmetical formula φ with one free variable x .

The point of bounded quantifiers is that they cost nothing: a relation defined from a decidable relation using only bounded quantifiers and connectives is again decidable, because a machine can simply loop x from 0 to t and test each case. Only *unbounded* alternations add power, so the hierarchy counts those.

Definition 11.4.2 (The classes $\Sigma_n, \Pi_n, \Delta_n$). Let $\Sigma_0 = \Pi_0 = \Delta_0$ be the class of decidable sets. For $n \geq 1$:

- $A \in \Sigma_n$ if $A = \{x \mid \exists y_1 \forall y_2 \exists y_3 \cdots Qy_n R(x, \vec{y})\}$ for some decidable relation R , where the n unbounded quantifiers strictly *alternate* and the block *begins* with $\exists$ (so Q is $\exists$ if n is odd, $\forall$ if n is even);
- $A \in \Pi_n$ if $A = \{x \mid \forall y_1 \exists y_2 \forall y_3 \cdots Qy_n R(x, \vec{y})\}$ with n strictly alternating unbounded quantifiers *beginning* with $\forall$;
- $\Delta_n = \Sigma_n \cap \Pi_n$.

Adjacent like quantifiers may always be merged into one using a computable pairing $\langle \cdot, \cdot \rangle$, so “ n quantifiers” really means “ n alternating blocks.”

A useful slogan: Σ_n formulas *lead with* $\exists$, Π_n formulas lead with $\forall$, and the subscript is the number of alternating blocks. Negation swaps the two: $A \in \Sigma_n \iff \bar{A} \in \Pi_n$, since pushing $\neg$ through the prefix flips every quantifier and reverses the leading symbol while preserving the count.

Proposition 11.4.3 (Containments that build the hierarchy). *For every $n \geq 0$, $\Sigma_n \cup \Pi_n \subseteq \Sigma_{n+1} \cap \Pi_{n+1} = \Delta_{n+1}$.*

Proof. Take $A \in \Sigma_n$, so $A = \{x \mid \exists y_1 \cdots Qy_n R\}$. Prepend a dummy unbounded $\forall y_0$ that does not occur in R : the formula $\forall y_0 \exists y_1 \cdots Qy_n R$ has $n+1$ alternating blocks and leads with $\forall$, so $A \in \Pi_{n+1}$. Instead append, after Qy_n , a dummy unbounded quantifier $Q'y_{n+1}$ of the kind opposite to Q (for $n = 0$, a dummy $\exists y_1$), again not occurring in R : the formula $\exists y_1 \cdots Qy_n Q'y_{n+1} R$ has $n+1$ alternating blocks and leads with $\exists$, so

$A \in \Sigma_{n+1}$. (A dummy of the *same* kind as its neighbor would merely merge into that block.) Neither dummy changes the defined set, since R ignores it. Hence $\Sigma_n \subseteq \Sigma_{n+1}$ and $\Sigma_n \subseteq \Pi_{n+1}$; symmetrically for Π_n . The two together give $\Sigma_n \cup \Pi_n \subseteq \Delta_{n+1}$. $\square$

Theorem 11.4.4 (The bottom two rungs are exactly the familiar classes). Δ_1 is the class of decidable sets, Σ_1 is the class of Turing-recognizable sets, and Π_1 is the class of co-recognizable sets.

Proof. $\Sigma_1 = \text{recognizable}$. If A is recognized by a TM M , then $x \in A$ iff M accepts x in some number of steps, i.e. $x \in A \iff \exists t [M \text{ accepts } x \text{ within } t \text{ steps}]$. The bracketed relation is decidable (simulate M on x for exactly t steps and check whether it has halted accepting), so $A \in \Sigma_1$. Conversely if $A = \{x \mid \exists y R(x, y)\}$ with R decidable, the machine that dovetails over $y = 0, 1, 2, \dots$ and accepts as soon as some $R(x, y)$ holds recognizes A .

$\Pi_1 = \text{co-recognizable}$. Immediate by complementation: $A \in \Pi_1 \iff \bar{A} \in \Sigma_1 \iff \bar{A} \text{ recognizable} \iff A \text{ co-recognizable}$.

$\Delta_1 = \text{decidable}$. If A is decidable it is both recognizable and co-recognizable, so $A \in \Sigma_1 \cap \Pi_1 = \Delta_1$. Conversely if $A \in \Delta_1$ then A and $\bar{A}$ are both recognizable; running both recognizers in parallel, exactly one accepts, which decides A – this is the recognizable-plus-co-recognizable-equals-decidable theorem of basic computability. $\square$

So the picture from earlier chapters is the first floor of an infinite tower. The next results locate our standard undecidable problems on it.

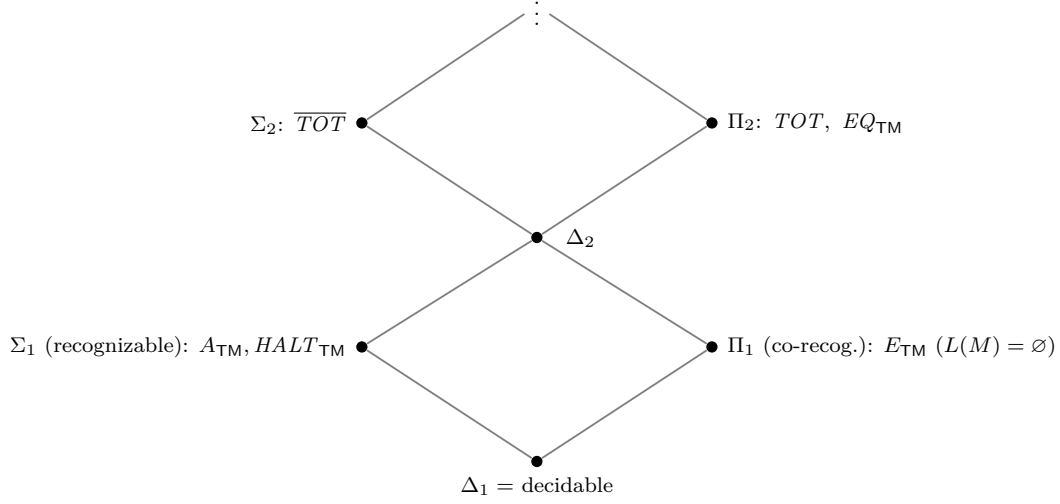
Theorem 11.4.5 (Where A_{TM} and friends live). $A_{\text{TM}} \in \Sigma_1$ and A_{TM} is Σ_1 -complete (every Σ_1 set mapping-reduces to it). Its complement $\bar{A}_{\text{TM}}$ is Π_1 -complete. The totality problem $TOT = \{\langle M \rangle \mid M \text{ halts on every input}\}$ is Π_2 -complete, and the cofiniteness problem $COF = \{\langle M \rangle \mid L(M) \text{ is cofinite}\}$ is Σ_3 -complete.

Proof sketch of the upper bounds. $\langle M, w \rangle \in A_{\text{TM}} \iff \exists t [M \text{ accepts } w \text{ in } t \text{ steps}]$, a Σ_1 definition. For totality, $\langle M \rangle \in TOT \iff \forall w \exists t [M \text{ halts on } w \text{ in } t \text{ steps}]$: a $\forall \exists$ prefix over a decidable matrix, hence Π_2 . For cofiniteness, $L(M)$ is cofinite iff $\exists b \forall w > b \exists t [M \text{ accepts } w \text{ in } t \text{ steps}]$; the inner “ $\forall w > b$ ” is unbounded, so the prefix is $\exists \forall \exists$, giving Σ_3 . Completeness (the lower bounds) is harder and uses the relativized halting problem; we record it but prove only the placements here. $\square$

Theorem 11.4.6 (Post: the hierarchy is strict). For every $n \geq 1$, $\Sigma_n \neq \Pi_n$, and consequently $\Delta_n \subsetneq \Sigma_n \subsetneq \Sigma_{n+1}$ and $\Delta_n \subsetneq \Pi_n \subsetneq \Pi_{n+1}$. In particular no rung collapses: each new alternation defines sets unreachable below.

Idea. At level 1 this is the classical fact $A_{\text{TM}} \in \Sigma_1 \setminus \Pi_1$ (it is recognizable but not co-recognizable). At level n the same diagonal argument is run relative to the $(n-1)$ -fold jump $\mathcal{O}^{(n-1)}$: the relativized halting set sits in $\Sigma_n \setminus \Pi_n$ by the identical undecidability proof carried out with that oracle. Strictness of $\Sigma_n \subsetneq \Sigma_{n+1}$ then follows because $\Pi_n \subseteq \Sigma_{n+1}$ while $\Pi_n \not\subseteq \Sigma_n$. $\square$

The shape of the tower. The classes interleave: for $n \geq 1$, each Δ_{n+1} contains both Σ_n and Π_n , which are incomparable to each other, and sits strictly below the incomparable pair Σ_{n+1}, Π_{n+1} . The Hasse diagram below shows the bottom of this lattice, with set inclusion going upward; a representative complete (or characteristic) problem is attached at each level so the abstract rungs have concrete inhabitants.



Inclusion goes upward; all displayed inclusions are *strict* (Post's theorem: $\Sigma_n \neq \Pi_n$ at every level), so each labelled problem lies on its rung and on no lower one.

Worked Example 11.4.7 (Reading the prefix of a definition off a problem). Classify $INF = \{\langle M \rangle \mid L(M) \text{ is infinite}\}$ in the arithmetic hierarchy by writing an arithmetical formula for it and counting alternating blocks.

Recognition. The task is “place a set on the hierarchy.” The trigger is that we are handed an English condition (“the accepted language is infinite”) and must turn it into a quantifier prefix over a decidable core, then read off Σ_n or Π_n from the prefix.

Solution. A language is infinite iff for every bound b there is some string longer than b that is accepted. “Accepted” is itself a Σ_1 event (accepts in some number of steps), so:

$$\langle M \rangle \in INF \iff \forall b \exists w \exists t [|w| > b \wedge M \text{ accepts } w \text{ in } t \text{ steps}].$$

The matrix in brackets is decidable. The prefix is $\forall \exists \exists$; merge the two adjacent $\exists$ blocks into one via pairing, leaving $\forall \exists$ – two alternating blocks beginning with $\forall$. Hence $INF \in \Pi_2$.

$$\begin{array}{c} \overbrace{\forall \exists : \text{two blocks} \Rightarrow \Pi_2} \\ \forall b \quad \exists w \quad \exists t \quad [\text{decidable}] \\ \underbrace{\hspace{1.5cm}} \\ \text{merge} \rightarrow \text{one } \exists \end{array}$$

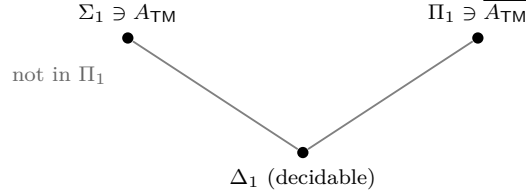
Which idea, and why? The leading unbounded quantifier names the class ($\forall \Rightarrow \Pi$), and the number of *alternating* blocks after merging like quantifiers is the subscript; adjacent existentials count once.

Check. The matrix really is decidable: given b, w, t we simulate M on w for exactly t steps and compare $|w|$ to b – a finite computation. And $INF \in \Pi_2$ is consistent with INF being undecidable, since Π_2 properly contains the decidable sets.

Worked Example 11.4.8 (Why A_{TM} sits in Σ_1 but not in Π_1). Show $A_{TM} \in \Sigma_1$ from its definition, then explain on the hierarchy diagram why it cannot be Π_1 .

Recognition. This is a “locate exactly” task: produce the cheapest prefix (upper bound), then invoke a known separation for the matching lower bound. The trigger is that A_{TM} is the standard recognizable-but-not-decidable set, so it should land on the very first Σ rung and stay off the Π side.

Solution. The single-existential definition $\langle M, w \rangle \in A_{TM} \iff \exists t [M \text{ accepts } w \text{ in } t \text{ steps}]$ has prefix $\exists$ over a decidable matrix, so $A_{TM} \in \Sigma_1$. If A_{TM} were also in Π_1 it would be in Δ_1 , i.e. decidable by Theorem 11.4.4 – but A_{TM} is undecidable. So $A_{TM} \in \Sigma_1 \setminus \Pi_1$, the left vertex one level up from Δ_1 .



Which idea, and why? A set with a one-existential definition is at worst Σ_1 ; if it were also Π_1 it would be decidable, so a known-undecidable set with that definition lands *exactly* on $\Sigma_1 \setminus \Delta_1$.

Check. Complementing flips the picture: $\overline{A_{TM}} = \{\langle M, w \rangle \mid \forall t [M \text{ does not accept } w \text{ within } t \text{ steps}]\}$ has a $\forall$ prefix, so it sits on the Π_1 vertex – the mirror image, exactly as the diagram shows.

Worked Example 11.4.9 (Counting alternations: the totality problem is Π_2). Place $TOT = \{\langle M \rangle \mid M \text{ halts on all inputs}\}$ precisely, and contrast it with A_{TM} to see why one extra alternation is genuinely needed.

Recognition. “Halts on all inputs” begins with an unbounded universal (“for all inputs”), and inside it hides an existential (“there is a halting time”). The trigger – a $\forall$ over inputs wrapping an $\exists$ over time that cannot be bounded – signals a true $\forall\exists$, i.e. level 2.

Solution. Write

$$\langle M \rangle \in TOT \iff \forall w \exists t [M \text{ halts on } w \text{ within } t \text{ steps}].$$

The matrix is decidable. The prefix $\forall\exists$ is two alternating blocks leading with $\forall$, so $TOT \in \Pi_2$. The inner $\exists t$ cannot be merged outward through the $\forall w$ (they alternate), and it cannot be bounded, because no computable bound on the halting time holds for all inputs – that is exactly why TOT does not drop to Π_1 .

$$A_{TM} : \exists t [\dots] \xrightarrow{\text{add } \forall w} \forall w \exists t [\dots] \iff TOT \in \Pi_2$$

Which idea, and why? Each unbounded quantifier that genuinely alternates with its neighbor raises the rung by one; a universal over an unbounded domain wrapping a recognizable (Σ_1) event yields exactly Π_2 .

Check. TOT is neither recognizable nor co-recognizable, so it lies outside $\Sigma_1 \cup \Pi_1$, consistent with being a fresh level-2 set; and its complement “ M fails to halt on some input” $= \exists w \forall t [\dots]$ is Σ_2 , the expected mirror.

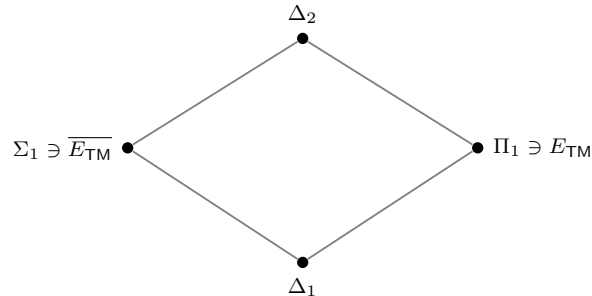
Worked Example 11.4.10 (A Δ_2 set: the emptiness problem E_{TM} from two sides). Show $E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$ lies in Π_1 , and that $\overline{E_{TM}}$ lies in Σ_1 , so E_{TM} sits cleanly on the Π_1 vertex (hence in Δ_2).

Recognition. Two complementary phrasings – “no string is ever accepted” (a $\forall$) versus “some string is accepted” (an $\exists$) – are the trigger to bound a set from *both* sides and pin it to a single rung rather than overcounting.

Solution. “ $L(M)$ is empty” says no input is accepted at any time:

$$\langle M \rangle \in E_{TM} \iff \forall w \forall t [M \text{ does not accept } w \text{ in } t \text{ steps}].$$

The two adjacent $\forall$ blocks merge to one, so the prefix is a single $\forall$ over a decidable matrix: $E_{TM} \in \Pi_1$. Dually $\overline{E_{TM}} = \{\langle M \rangle \mid \exists w \exists t [M \text{ accepts } w \text{ in } t \text{ steps}]\} \in \Sigma_1$ – indeed $\overline{E_{TM}}$ is recognizable. So E_{TM} is co-recognizable but not recognizable: the Π_1 vertex.



Which idea, and why? When a set has a pure- $\forall$ definition *and* its complement has a pure- $\exists$ one, both bounds are level 1, so the set is Π_1 (and automatically Δ_2) – there is no need to climb higher.

Check. E_{TM} is the classic co-recognizable, undecidable set: a machine cannot confirm emptiness in finite time, but it *can* confirm non-emptiness by dovetailing until some string is accepted – matching “ $\overline{E_{\text{TM}}} \in \Sigma_1$, $E_{\text{TM}} \in \Pi_1$.”

Worked Example 11.4.11 (The jump climbs exactly one rung). Let $\varnothing' = A_{\text{TM}}$ (the halting problem, identified with the jump of the empty oracle). Argue from the hierarchy that the second jump $\varnothing''$ sits at level 2 and not below, illustrating the slogan “each jump = one alternation.”

Recognition. The phrase “the jump” from Section 11.3 is the trigger: the jump B' is the halting problem relative to B , a Σ_1 -relative-to- B set, and relativizing a Σ_1 definition over a Σ_1 oracle stacks a fresh $\exists\forall$ alternation – one rung up.

Solution. $\varnothing'$ is Σ_1 -complete by Theorem 11.4.5. Its own jump $\varnothing''$ is the halting problem relative to $\varnothing'$, so writing “ $M^{\varnothing'}$ accepts” unfolds the oracle queries into a universal check over computations consistent with $\varnothing'$ membership, landing one block higher: $\varnothing''$ is Σ_2 -complete. By Post’s theorem (Theorem 11.4.6’s engine), $\Sigma_2 \neq \Pi_2$, so $\varnothing'' \notin \Pi_2$, hence $\varnothing''$ is strictly above level 1. The tower of jumps $\varnothing <_{\text{T}} \varnothing' <_{\text{T}} \varnothing'' <_{\text{T}} \dots$ marches up one rung per jump.

$$\begin{array}{ccccc} \varnothing & \xrightarrow{(\cdot)'} & \varnothing' = A_{\text{TM}} & \xrightarrow{(\cdot)'} & \varnothing'' & \xrightarrow{(\cdot)'} & \dots \\ \cap & & \cap & & \cap & & \\ \Delta_1 & & \Sigma_1 & & \Sigma_2 & & \end{array}$$

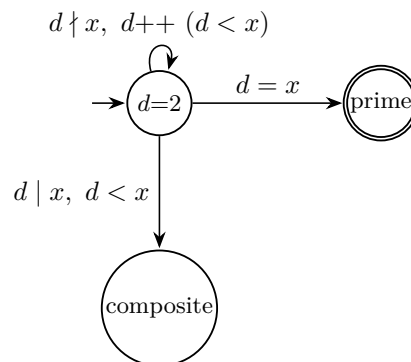
Which idea, and why? Relativizing the Σ_1 halting definition over a Σ_n -complete oracle adds one alternating block, so the jump is exactly Σ_{n+1} -complete – the jump and the hierarchy step in lockstep.

Check. The chain of strict reductions $\varnothing <_{\text{T}} \varnothing' <_{\text{T}} \varnothing''$ matches strictness of $\Delta_1 \subsetneq \Sigma_1 \subsetneq \Sigma_2$; if any jump failed to climb, a rung would collapse, contradicting Post’s theorem.

Worked Example 11.4.12 (Bounded quantifiers are free: a tempting prefix that overcounts). The set $\text{PRIME} = \{x \mid x \text{ is prime}\}$ has the natural definition “ $x \geq 2$ and $\forall d [1 < d < x \Rightarrow d \nmid x]$.” Its prefix shows a $\forall$, so a hasty reading would put it in Π_1 . Show it is actually decidable (Δ_0).

Recognition. The trigger is a quantifier whose range is *bounded* by a term in x (here “ $d < x$ ”). Bounded quantifiers never raise the level, so seeing one is the cue to fold it into the decidable matrix rather than count it as an alternation.

Solution. The universal is bounded: $\forall d < x$. A machine can test primality by trial division – loop d from 2 to $x - 1$ and check $d \nmid x$ – a guaranteed-halting computation. So PRIME is decidable, i.e. in $\Delta_0 = \Sigma_0 = \Pi_0$, the base of the tower, far below Π_1 .



Which idea, and why? A quantifier bounded by a term in the input ranges over a finite, computable set, so it collapses into the decidable matrix and does not count toward the Σ_n/Π_n index; only *unbounded* quantifiers move you up the hierarchy.

Check. The little decider always halts (the counter d strictly increases and is capped at x), confirming $\text{PRIME} \in \Delta_0$; the visible $\forall$ was a bounded one and never mattered.

Method (How to place a set on the arithmetic hierarchy).

1. **Write a defining formula** $x \in A \iff (\text{quantifier prefix})[\text{matrix}]$. Choose the matrix to be a *decidable* relation – typically “ M accepts/halts on $\dots$ within t steps,” which is always decidable once the step count t is one of the quantified variables.
2. **Strip the bounded quantifiers.** Any $\exists x \leq t$ or $\forall x \leq t$ folds into the matrix (a finite loop), keeping it decidable. Do not count these.
3. **Merge adjacent like quantifiers** using pairing, so the prefix strictly alternates. Now the prefix is a clean string of $\exists/\forall$ blocks.
4. **Read off the upper bound.** Leading $\exists \Rightarrow \Sigma_n$, leading $\forall \Rightarrow \Pi_n$, where n is the number of alternating blocks.
5. **Get a matching lower bound** (for completeness/exactness) by reducing a known complete set for that level to A . A defining formula for the *complement* gives a second *upper* bound instead, via $A \in \Sigma_n \iff \bar{A} \in \Pi_n$: if both A and $\bar{A}$ are in Σ_n , then $A \in \Delta_n$.

Caution. Do not count *bounded* quantifiers as alternations. A definition like $\forall i \leq n \exists j \leq n [\dots]$ with both bounds computable is still decidable; only quantifiers ranging over *all* of $\mathbb{N}$ (no term bound) push a set up a rung. The fastest way to misclassify a set is to read its English phrasing literally – “for every divisor,” “there is a witness below x ” – and mistake a bounded quantifier for an unbounded one.

Caution. The subscript counts *alternating blocks*, not the raw number of quantifier symbols. A prefix $\exists\exists\exists$ is still Σ_1 (merge the three into one $\exists$ by pairing), and $\forall\forall\exists$ is Π_2 , not Π_3 . Always merge runs of like quantifiers *before* reading the index, and always check which symbol *leads*: $\exists$ first means Σ , $\forall$ first means Π , regardless of what trails.

Caution. $\Delta_n = \Sigma_n \cap \Pi_n$ is not the same as “decidable” except at $n = 1$. A set in Δ_2 (such as E_{TM} , or any set Turing-reducible to A_{TM}) is generally *undecidable*; it is merely decidable *relative to the oracle* $\emptyset'$. Reserve “decidable” for Δ_1 alone, and read higher Δ_n as “decidable with the level- $(n-1)$ jump as an oracle.”

Exercises.

1. For each set, write a defining arithmetical formula with a decidable matrix, then give the tightest class Σ_n or Π_n you can read off the prefix (state your merging of like quantifiers):
 - (a) $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$;
 - (b) $\{\langle M \rangle \mid M \text{ accepts at least one even-length string}\}$;
 - (c) $\{\langle M \rangle \mid M \text{ rejects every input}\}$.
2. Show directly from Definition 11.4.2 that Σ_n is closed under finite unions and finite intersections. (Hint: rename the quantified variables apart, then merge like blocks.)
3. Prove the negation rule $A \in \Sigma_n \iff \bar{A} \in \Pi_n$ by pushing $\neg$ through a Σ_n prefix one quantifier at a time, and use it to conclude Δ_n is closed under complement.
4. Classify $\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid L(M) \text{ is a regular language}\}$ as low in the hierarchy as you can by writing a quantifier prefix over a decidable matrix. (You do not need to prove completeness; aim for a correct upper bound and explain each alternation.)
5. Draw the Hasse diagram of the six classes $\Delta_1, \Sigma_1, \Pi_1, \Delta_2, \Sigma_2, \Pi_2$ under inclusion, with edges only for *covering* relations (no edge that is implied by transitivity). Mark on it where A_{TM} , E_{TM} , and TOT live.
6. Let $\text{EQ}_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid L(M_1) = L(M_2)\}$. Give an arithmetical formula for it and show it is in Π_2 . (Hint: two languages are equal iff every string is in both or in neither; express “string w is accepted” as a Σ_1 event and watch the alternation.)
7. *Find the bug.* A student claims $\text{COF} = \{\langle M \rangle \mid L(M) \text{ is cofinite}\}$ is in Π_2 via “ $\langle M \rangle \in \text{COF} \iff \forall w \exists t [M \text{ accepts } w \text{ in } t \text{ steps}]$.” Explain precisely what this formula actually defines, why it is the wrong condition for cofiniteness, and what alternation the correct formula needs.

8. True or false, with a one-line justification each:
 - (a) every decidable set is in Π_5 ;
 - (b) $\Sigma_2 \cap \Pi_2 = \Delta_2$ contains only decidable sets;
 - (c) if $A \in \Sigma_1$ and $B \in \Pi_1$ then $A \cap B \in \Delta_2$.
9. Prove $A \leq_m B \Rightarrow (B \in \Sigma_n \Rightarrow A \in \Sigma_n)$, that is, that Σ_n is closed downward under mapping reducibility. Then explain why this makes a Σ_n -complete set a single “hardest” representative of the whole rung.
10. Show that the bounded-quantifier set $\{\langle x, y \rangle \mid \exists z \leq x \forall u \leq y R(x, y, z, u)\}$ is decidable whenever R is decidable, by describing a halting decider. Conclude that no stack of *bounded* quantifiers ever leaves Δ_0 .
11. Let B be any set with $\emptyset' \leq_T B \leq_T \emptyset''$. Argue that B cannot be decidable, and explain in one or two sentences how such a B can still be “decidable relative to” an oracle – naming which jump suffices.
12. *Find the bug.* A proof sketch claims “ $TOT \in \Pi_1$ because halting is just $\forall w [M \text{ halts on } w]$, a single universal.” Identify the hidden unbounded quantifier that this collapses away, and state the correct level with the corrected prefix.

11.5 Decidability of Logical Theories and Kolmogorov Complexity

This section samples two corners of computability where the abstract machinery of Chapters 9–10 pays off in concrete, surprising ways. First we treat a mathematical theory as a language of true sentences and ask whether a machine can decide membership: the arithmetic of addition alone, $\text{Th}(\mathbb{N}, +)$, turns out to be decidable, while adding multiplication to get $\text{Th}(\mathbb{N}, +, \times)$ makes the theory undecidable – so “is this arithmetic statement true?” has no algorithm. Second we ask how much *information* a single binary string carries, and answer with the shortest program that prints it; this descriptive (Kolmogorov) complexity gives a clean notion of randomness as incompressibility. The single idea linking the two halves is that a string can describe a computation, and the length or decidability of such descriptions is itself a computability question.

Theories, models, and the decision problem for truth

We fix a tiny logical language. Sentences are built from variables $x, y, z, \dots$, the Boolean connectives $\wedge, \vee, \neg$, the quantifiers $\forall, \exists$, parentheses, and a finite list of relation symbols $R_1, \dots, R_k$. An *atomic formula* is $R_i(x_1, \dots, x_j)$ where j is the fixed arity of R_i . A *formula* is an atomic formula, a Boolean combination of smaller formulas, or a quantification $\exists x [\varphi]$ or $\forall x [\varphi]$. A variable not bound by any quantifier is *free*; a formula with no free variables is a *sentence*. Throughout we assume *prenex* form, all quantifiers pushed to the front.

Definition 11.5.1 (Model and theory). A *model* is a pair $\mathcal{M} = (U; P_1, \dots, P_k)$ consisting of a *universe* U together with a relation $P_i \subseteq U^{j_i}$ assigned to each relation symbol R_i of arity j_i . A sentence φ in the language of $\mathcal{M}$ is either true or false in $\mathcal{M}$. The *theory* of $\mathcal{M}$ is the set of true sentences

$$\text{Th}(\mathcal{M}) = \{ \varphi \mid \varphi \text{ is a sentence true in } \mathcal{M} \}.$$

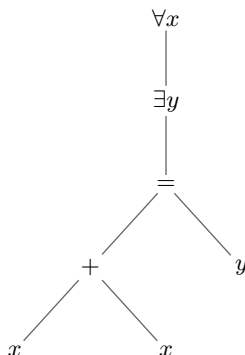
When the intended relation is standard we write it infix: $(\mathbb{N}, +)$ is the model with universe $\mathbb{N} = \{0, 1, 2, \dots\}$ and the ternary relation $\text{PLUS}(a, b, c)$ that holds exactly when $a + b = c$; $(\mathbb{N}, +, \times)$ adds the relation $\text{TIMES}(a, b, c)$ holding when $a \cdot b = c$. Constants like 0, 1 and functions like $+$ are coded as relations, so this loses nothing. Asking whether $\text{Th}(\mathcal{M})$ is decidable is asking: *is there an algorithm that, given a sentence, says whether it is true in $\mathcal{M}$?*

Worked Example 11.5.2 (Reading a sentence, and a formula that is not a sentence). Decide which of the following are sentences in the language of $(\mathbb{N}, +)$, and whether the sentences are true: (a) $\forall x \exists y [x + x = y]$; (b) $\exists y \forall x [x + x = y]$; (c) $\exists y [x + x = y]$.

Recognition. The trigger is the definition: a *sentence* has no free variable. Scan each formula for a variable that no quantifier binds; that decides “sentence or not” before truth even enters.

Solution. In (c) the variable x occurs but only y is quantified, so x is free: (c) is a formula, not a sentence, and has no truth value on its own. Both (a) and (b) are sentences. For (a): for every x the number $y = 2x$

exists, so it is *true*. For (b): it claims a single y works for all x simultaneously, i.e. one number equals $2x$ for every x , which is impossible – *false*. The two differ only in quantifier order, yet one is true and one false.



Which idea, and why? Sentencehood is purely syntactic – count free variables – and is independent of truth; only after a formula is closed (a sentence) does the model decide true/false, and quantifier *order* can flip that verdict.

Check. Swapping the leading quantifiers $\forall x \exists y$ to $\exists y \forall x$ in the tree changes nothing about the leaves yet inverts the truth value – the dependence of y on x is exactly what a decision procedure must track.

Th($\mathbb{N}, +$) is decidable: automata that recognize true tuples

The pleasant surprise is that pure additive arithmetic can be decided by the finite automata of Chapter 4. The trick is to feed several numbers to one automaton *in parallel*, one bit of each per input symbol, and to model each quantifier by an automaton operation: $\exists$ becomes nondeterministic guessing (projection), $\forall$ becomes complement–guess–complement.

Definition 11.5.3 (Parallel encoding of a tuple). For $i \geq 1$, let $\Sigma_i = \{0, 1\}^i$, the set of length- i columns of bits. A string over Σ_i encodes an i -tuple of binary integers by reading *across the rows*: row r spells the r th number. Pad shorter numbers with leading 0s so all rows have equal length.

Worked Example 11.5.4 (Encoding a single addition as one input string). Exhibit the Σ_3 -string that presents the triple $(a, b, c) = (5, 3, 8)$ to an automaton checking $a + b = c$, and confirm it is a valid addition.

Recognition. The trigger is “feed numbers in parallel”: stack the three binary numerals as rows of a grid and read the columns left to right. Each column is one symbol of Σ_3 .

Solution. Write $5 = 0101$, $3 = 0011$, $8 = 1000$ (four bits each, most significant first). Stacking gives the grid below; its four columns are the four input symbols.

four input symbols of Σ_3 , read left to right

$a=5$	0	1	0	1
$b=3$	0	0	1	1
$c=8$	1	0	0	0

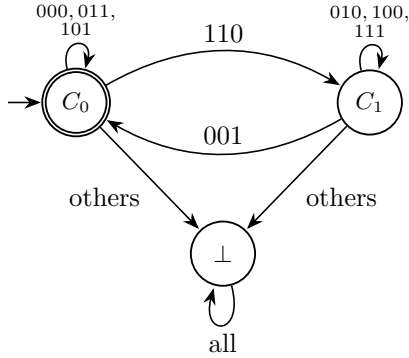
Which idea, and why? An i -ary atomic relation on numbers becomes a *regular* condition on the parallel encoding, because checking it left to right needs only a bounded carry/comparison state – exactly what a DFA can hold.

Check. Column-wise this is binary addition of $0101 + 0011$: rightmost column $1 + 1 = 10$ writes 0 carry 1; next $0 + 1 + 1 = 10$ writes 0 carry 1; next $1 + 0 + 1 = 10$ writes 0 carry 1; leftmost $0 + 0 + 1 = 1$. The sum row reads $1000 = 8$, matching c .

Worked Example 11.5.5 (A DFA that recognizes the addition relation). Build a finite automaton over $\Sigma_3 = \{0, 1\}^3$ (columns (a -bit, b -bit, s -bit)) accepting exactly the column-strings, read *least significant bit first*, for which $a + b = s$.

Recognition. The trigger is “binary addition with carry”: scanning from the low end, the only memory needed between columns is the current carry bit, which is finite – so a two-state DFA (plus a reject trap) does it.

Solution. States C_0 (carry 0, start and accept) and C_1 (carry 1); a dead state $\perp$. On a column (a, b, s) from state C_γ compute $a + b + \gamma$; the low bit must equal s and the high bit becomes the next carry. If the low bit $\neq s$, go to $\perp$. At end of input the carry must be 0, so only C_0 accepts.



Which idea, and why? Addition is a finite-state task because the carry is the *only* information that must cross a column boundary; this is the base case from which the whole decision procedure for $\text{Th}(\mathbb{N}, +)$ is assembled.

Check. Take $1 + 1 = 2$: low-first columns are $(1, 1, 0)$ then $(0, 0, 1)$. From C_0 , symbol 110 goes to C_1 (writes 0, carry 1); from C_1 , symbol 001 goes to C_0 (writes 1, carry 0); end in C_0 , accept. Good: $01 + 01 = 10$ low-first reads $1+1=0$ carry, then $0+0+1 = 1$.

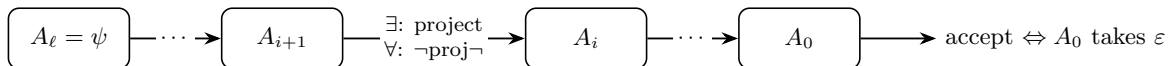
Theorem 11.5.6 (Decidability of additive arithmetic). $\text{Th}(\mathbb{N}, +)$ is decidable.

Proof. Given a sentence $\varphi = Q_1x_1 Q_2x_2 \cdots Q_\ell x_\ell [\psi]$ in prenex form with quantifier-free matrix ψ , define for $0 \leq i \leq \ell$ the formula $\varphi_i = Q_{i+1}x_{i+1} \cdots Q_\ell x_\ell [\psi]$, so $\varphi_\ell = \psi$ and $\varphi_0 = \varphi$; here φ_i has i free variables. We build automata A_i over Σ_i so that A_i accepts exactly the encodings of i -tuples $(a_1, \dots, a_i)$ for which $\varphi_i(a_1, \dots, a_i)$ is true, working from A_ℓ down to A_0 .

Base A_ℓ . The matrix ψ is a Boolean combination of atomic additions. Each single addition is regular (the carry automaton A_+ of Worked Example 11.5.5 above is the prototype, used reversed since here the columns are read most significant bit first, which is what the ε -moves below assume), and the regular languages are closed under reversal, union, intersection, and complement, so the Boolean combination ψ is recognized by some A_ℓ over Σ_ℓ .

Step $A_i \leftarrow A_{i+1}$. If $\varphi_i = \exists x_{i+1} \varphi_{i+1}$, obtain A_i by *projection*: run A_{i+1} but, at each input column for the first i numbers, nondeterministically *guess* the missing $(i+1)$ st bit and feed the augmented column to A_{i+1} (with ε -moves at the start to guess leading bits of x_{i+1}). Then A_i accepts $(a_1, \dots, a_i)$ iff some a_{i+1} makes A_{i+1} accept, which is exactly $\exists x_{i+1}$. If $\varphi_i = \forall x_{i+1} \varphi_{i+1}$, rewrite it as $\neg \exists x_{i+1} \neg \varphi_{i+1}$: complement A_{i+1} , project, complement again.

Finish. A_0 is over $\Sigma_0 = \{[]\}$ and accepts iff $\varphi_0 = \varphi$ is true. Run A_0 on the empty string; accept the sentence φ iff A_0 does. Every step is an effective automaton construction, so the whole procedure is an algorithm. $\square$



Remark. The same idea decides relatives such as $\text{Th}(\mathbb{N}, <)$, and $\text{Th}(\mathbb{R}, +, \times)$ is decidable too (real-closed fields, by Tarski, via quantifier elimination rather than automata). The line is crossed when *both* $+$ and $\times$ act on $\mathbb{N}$.

$\text{Th}(\mathbb{N}, +, \times)$ is undecidable, and Gödel's incompleteness

Multiplication breaks the automaton method, because once both operations are available the language can encode arbitrary computation.

Theorem 11.5.7 (Church). $\text{Th}(\mathbb{N}, +, \times)$ is undecidable.

Proof sketch. We reduce A_{TM} to $\text{Th}(\mathbb{N}, +, \times)$. The engine is a lemma: from $\langle M, w \rangle$ one can *effectively* build a formula $\varphi_{M,w}(x)$ with one free variable x such that, using $+$ and $\times$ to extract and verify digits, $\varphi_{M,w}(n)$ holds iff the integer n codes an accepting computation history of M on w . Then

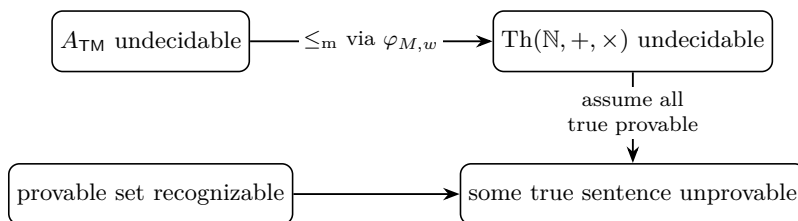
$$M \text{ accepts } w \iff \exists x \varphi_{M,w}(x) \in \text{Th}(\mathbb{N}, +, \times).$$

The map $\langle M, w \rangle \mapsto \exists x \varphi_{M,w}(x)$ is computable, so it is a mapping reduction $A_{\text{Tm}} \leq_m \text{Th}(\mathbb{N}, +, \times)$. Since A_{Tm} is undecidable, so is $\text{Th}(\mathbb{N}, +, \times)$. $\square$

A short chain of consequences gives a computability-flavoured Gödel incompleteness theorem. Assume a proof system that is *checkable* (the set $\{\langle \varphi, \pi \rangle \mid \pi \text{ proves } \varphi\}$ is decidable) and *sound* (provable $\Rightarrow$ true).

Theorem 11.5.8 (Incompleteness, computability form). *In any checkable, sound proof system for $\text{Th}(\mathbb{N}, +, \times)$, some true sentence is not provable.*

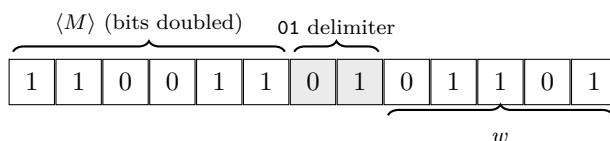
Proof. The provable sentences are recognizable: search all candidate proofs π and accept φ when a valid one appears (the checker is the decider needed inside). Suppose, for contradiction, that *every* true sentence were provable. Build a decider D for truth: on input φ , dovetail the recognizer on φ and on $\neg\varphi$. Exactly one of $\varphi, \neg\varphi$ is true, hence (by assumption) provable, hence accepted; soundness makes the verdict correct. So D decides $\text{Th}(\mathbb{N}, +, \times)$, contradicting Theorem 11.5.7. Therefore some true sentence is unprovable. $\square$



Remark. Using the recursion theorem (Section 11.1) one can exhibit an *explicit* true-but-unprovable sentence ψ that asserts “no proof of me exists.” If ψ were provable it would be false (it would have a proof), violating soundness; so ψ is unprovable and therefore true. This is the self-reference at the heart of Gödel’s original argument.

Descriptive (Kolmogorov) complexity

We now measure the information in a single binary string by the length of the shortest program that prints it. We use the encoding $\langle M, w \rangle = \langle M \rangle w$ – the binary code of a TM M followed by an input w – with a prefix-free coding of $\langle M \rangle$ so the boundary between M and w is recoverable. A standard trick doubles each bit of $\langle M \rangle$ and ends it with 01.



Definition 11.5.9 (Minimal description and descriptive complexity). For a binary string x , a *description* of x is a string $\langle M, w \rangle$ such that M run on input w halts with x on its tape. The *minimal description* $d(x)$ is the shortest such string (lexicographically first among shortest). The *descriptive complexity* of x is

$$K(x) = |d(x)|.$$

A string x is *incompressible by c* if $K(x) > |x| - c$, and simply *incompressible* if $K(x) \geq |x|$.

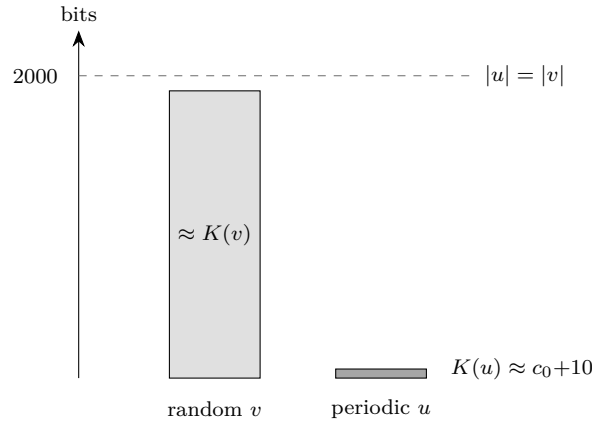
Worked Example 11.5.10 (Computing complexity bounds for a structured and a random-looking string). Compare $K(u)$ and $K(v)$ for $u = (01)^{1000}$ (the block 01 repeated 1000 times, length 2000) and a length-2000 string v produced by fair coin tosses.

Recognition. The trigger is the contrast between *pattern* and *no pattern*: a short program plus a repeat count describes u , whereas a coin-toss v typically has no description shorter than itself.

Solution. For u : a fixed TM M_{rep} takes a block b and a count n (in binary) and prints b repeated n times. The description $\langle M_{\text{rep}} \rangle 01 (1000)_2$ has length $O(1) + |(1000)_2| = O(1) + 10$ bits, so

$$K(u) \leq c_0 + 10 \ll 2000 = |u|.$$

Thus u is wildly compressible. For v : by Theorem 11.5.13 below, some length-2000 string is incompressible, and a fair random v is incompressible by, say, 20 with overwhelming probability, giving $K(v) > 1980$.



Which idea, and why? Complexity is an *upper bound by exhibiting a program*: any concrete description of x caps $K(x)$ from above, so a short generator (block + count) proves u compressible, while a counting argument – not a construction – forces some v to be incompressible.

Check. The bound on $K(u)$ used only the description's length, never its optimality, which is exactly how Theorem 11.5.11 works; and $K(v) > 1980$ is consistent with the universal upper bound $K(v) \leq 2000 + c$ from that same theorem.

Theorem 11.5.11 (Universal upper bound). *There is a constant c with $K(x) \leq |x| + c$ for every string x .*

Proof. Let M_{id} be the TM that halts immediately, leaving its input untouched, so it computes the identity. Then $\langle M_{\text{id}} \rangle x$ is a description of x , of length $|\langle M_{\text{id}} \rangle| + |x|$. Taking $c = |\langle M_{\text{id}} \rangle|$ (a fixed number independent of x) gives $K(x) \leq |x| + c$, since the minimal description can only be shorter. $\square$

Theorem 11.5.12 (Doubling is almost free). *There is a constant c with $K(xx) \leq K(x) + c$ for every x .*

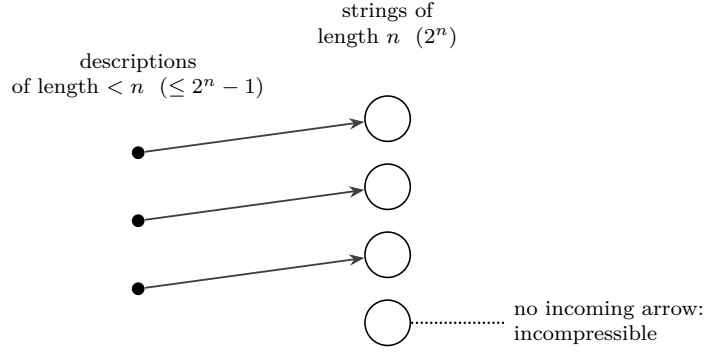
Proof. Let M_2 be the TM that, on input $\langle N, w \rangle$, runs N on w to get an output s and then prints ss . Then $\langle M_2 \rangle d(x)$ is a description of xx : it runs the minimal description of x and doubles the result. Its length is $|\langle M_2 \rangle| + |d(x)| = c + K(x)$ with $c = |\langle M_2 \rangle|$. $\square$

Theorem 11.5.13 (Incompressible strings exist). *For every length n there is an incompressible string of length n .*

Proof. There are 2^n strings of length n . A description shorter than n has length in $\{0, 1, \dots, n-1\}$, so the number of such descriptions is at most

$$\sum_{i=0}^{n-1} 2^i = 2^n - 1 < 2^n.$$

Each description produces at most one string, so the $2^n - 1$ short descriptions cover at most $2^n - 1$ strings. Hence at least one length- n string has no description of length $< n$; that string x has $K(x) \geq n = |x|$ and is incompressible. $\square$



Corollary 11.5.14 (Most strings are nearly incompressible). *For $c \leq n + 1$, at least $2^n - 2^{n-c+1} + 1$ of the 2^n strings of length n are incompressible by c .*

Proof. A string compressible by c has a description of length $\leq n - c$. Descriptions of length $\leq n - c$ number at most $\sum_{i=0}^{n-c} 2^i = 2^{n-c+1} - 1$, so at most that many length- n strings are c -compressible. The rest, at least $2^n - (2^{n-c+1} - 1)$, are incompressible by c . $\square$

Theorem 11.5.15 (K is not computable). *The function $x \mapsto K(x)$ is not computable, and the set of incompressible strings is undecidable.*

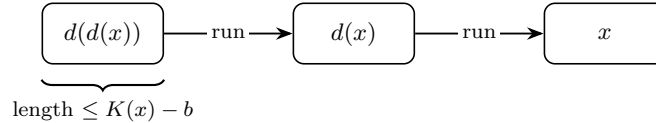
Proof idea. If K were computable, the machine “output the lexicographically first x of length n with $K(x) \geq n$ ” would, on input n , print an incompressible string x_n of length n . But that machine plus the short input n is a description of x_n of length $O(1) + \log_2 n \ll n$, so $K(x_n) \ll n$ – contradicting $K(x_n) \geq n$. (This is the Berry-paradox argument: “the first string with no short description” just described it.) The same argument needs only a decider for incompressibility (to test “ $K(x) \geq n$ ” for each x of length n), so that set is undecidable too. $\square$

Theorem 11.5.16 (Minimal descriptions are themselves incompressible). *There is a constant b such that for every x , the string $d(x)$ is incompressible by b .*

Proof. Let M be the TM that, on input $\langle R, y \rangle$, runs R on y ; if the output has the form $\langle S, z \rangle$ it then runs S on z and halts with that result. Put $b = |\langle M \rangle| + 1$. Suppose, for contradiction, that $d(x)$ were b -compressible, i.e. $|d(d(x))| \leq |d(x)| - b$. Then $\langle M \rangle d(d(x))$ describes x : running $d(d(x))$ outputs $d(x)$, which M then runs to output x . Its length is

$$|\langle M \rangle| + |d(d(x))| \leq (b - 1) + (|d(x)| - b) = |d(x)| - 1 < K(x),$$

contradicting minimality of $d(x)$. Hence $d(x)$ is incompressible by b . $\square$



prepend $\langle M \rangle$: a description of x of length $< K(x)$ – impossible

Method (How to bound $K(x)$ in a worked problem).

1. **Upper bound = build a generator.** To show $K(x) \leq f(n)$, exhibit one TM M and input w with $\langle M, w \rangle$ of length $f(n)$ that prints x . You never need the optimal description – any description caps K from above.
2. **Reuse standard machines.** Identity ($\Rightarrow K(x) \leq |x| + c$), “repeat a block n times,” “run then double,” and “run two sub-descriptions” are off-the-shelf; count their fixed code as a constant.

3. **Lower bound = counting, not construction.** To show many strings have large K , compare the number of length- n strings (2^n) with the number of short descriptions ($< 2^n$). Pigeonhole forces incompressibles; you cannot point to one effectively.
4. **Never claim to compute K .** If your argument needs the *exact* value of $K(x)$ on demand, it is wrong: K is not computable (Theorem 11.5.15).

Caution. K is a property of the *string*, not of any one program you have in mind. A description you happen to know gives only an *upper* bound $K(x) \leq (\text{its length})$; a cleverer machine might do better, so you can never conclude x is incompressible just because *your* program is long. Lower bounds on K come from counting, never from inspecting a single description.

Caution. The decidable/undecidable split for theories hinges on *which operations* the model has, not on how complicated the sentences look. $\text{Th}(\mathbb{N}, +)$ is decidable even for sentences with many alternating quantifiers, while a single $\exists x \varphi_{M,w}(x)$ in $\text{Th}(\mathbb{N}, +, \times)$ already encodes the halting problem. Do not assume “arithmetic” is one monolithic decidable (or undecidable) thing – adding $\times$ is exactly what crosses the line.

Caution. Incompleteness (Theorem 11.5.8) is not a defect of one proof system you can patch by adding axioms. The argument applies to *every* checkable, sound system for arithmetic: any such system leaves some true sentence unproved. Reading it as “we just need better axioms” misses that the gap reappears in the stronger system.

Exercises.

1. Classify each string as a *sentence*, a *formula but not a sentence*, or *ill-formed*, in the language of $(\mathbb{N}, +)$: (a) $\forall x \exists y [y + y = x]$; (b) $\exists x [x + y = y]$; (c) $\forall x \forall y [x + y]$. For each genuine sentence, state whether it is true in $(\mathbb{N}, +)$ and justify in one line.
2. Show that the relation “ $x \leq y$ ” is expressible in the language of $(\mathbb{N}, +)$ by giving a formula with free variables x, y that holds exactly when $x \leq y$. Then express “ $x = 0$ ” and “ $x = 1$ ” by formulas of $(\mathbb{N}, +)$ using only $+$ and equality. Finally, using your $x \leq y$, write one true and one false two-quantifier sentence and justify each truth value in a line.
3. In the parallel-encoding scheme of Definition 11.5.3, write out the Σ_3 -string (as a sequence of column symbols, most significant bit first) that presents the triple $(a, b, c) = (6, 5, 11)$, and verify column by column that $a + b = c$.
4. Draw a finite automaton over the column alphabet $\{0, 1\}^2$ that, reading two numbers a, b *least significant bit first*, accepts exactly when $a = b$. State what each state remembers, and trace your machine on $(a, b) = (5, 5)$ and on $(5, 7)$.
5. Using the projection idea from the proof of Theorem 11.5.6, describe (in words, with a small diagram if helpful) how to turn an automaton recognizing the truth set of $\varphi(x, y)$ into one recognizing the truth set of $\exists y \varphi(x, y)$. Why does the same construction, sandwiched between two complementations, handle $\forall y$?
6. Explain in two or three sentences why the decision procedure of Theorem 11.5.6 breaks down if the model is $(\mathbb{N}, +, \times)$ instead of $(\mathbb{N}, +)$. Point to the exact step of the construction that fails and say why multiplication is the culprit.
7. Give a mapping reduction showing that if $\text{Th}(\mathbb{N}, +, \times)$ were decidable, then HALT_{TM} would be decidable. (Use the lemma that from $\langle M, w \rangle$ one can build $\varphi_{M,w}$ with $\exists x \varphi_{M,w}(x)$ true iff M accepts w , and recall $\text{HALT}_{\text{TM}} \leq_m A_{\text{TM}}$ machinery.)
8. Let p be the string of the first n bits of a fixed computable real (say the binary expansion of $1/3$). Show $K(p) \leq 2 \log_2 n + c$ for some constant c , and contrast this with the existence of length- n strings q having $K(q) \geq n$. (This subsumes the case $p = 0^n$: the length- n prefixes of any computable infinite string have descriptions of logarithmic length.)
9. Using Corollary 11.5.14, compute a lower bound on the fraction of length- n strings that are incompressible by $c = 5$, and show this fraction exceeds 0.93 for all sufficiently large n . Interpret the result as “almost every string is essentially random.”

10. Prove that there is a constant c such that $K(xy) \leq 2K(x) + K(y) + c$ for all strings x, y . (Build a TM that reads two self-delimiting sub-descriptions – double the bits of $d(x)$, mark the end with 01, then append $d(y)$ – runs both, and concatenates the outputs.)
11. *Find the bug.* A student argues: “The string $z = d(x)$ is a description, and I can write the short program ‘print z ’, so $K(x)$ is small for every x ; therefore no string is incompressible.” Identify the precise error in this reasoning and explain why it does not refute Theorem 11.5.13.
12. *Find the bug.* Another student claims to compute K : “On input x of length n , try all programs of length $0, 1, \dots, n + c$ in order; the first that prints x gives $K(x)$.” Explain exactly why this procedure does not halt in general, and connect the failure to Theorem 11.5.15.

Chapter 12

Time Complexity: P and NP

12.1 Measuring Time; Big-O and the Class TIME

Until now a language was either decidable or it was not; how *long* the decider took never entered the discussion. Complexity theory restores that missing dimension. The one key idea of this section is that we measure a decider by its *worst-case running time as a function of input length*, and then – so that the measure does not drown in irrelevant detail – we read that function only *asymptotically*, through big- O notation. We make “running time” precise for a Turing machine, set up the asymptotic vocabulary (O , o , Ω , Θ), bundle equally-fast languages into the time classes $\text{TIME}(t(n))$, and close by asking how much the answer depends on the machine model – the question that motivates the class P in the next section.

Running time

Definition 12.1.1 (Running time). Let M be a deterministic Turing machine that halts on every input. The *running time* (or *time complexity*) of M is the function $f: \mathbb{N} \rightarrow \mathbb{N}$ where $f(n)$ is the *maximum* number of steps M takes on any input of length n . We then say M runs in time $f(n)$, and that M is an $f(n)$ time machine. By convention n denotes the length of the input.

Two choices are baked into this definition. First, time is a function of the input *length* n alone, not of the particular input – if the input is a graph we do not split the count into nodes and edges, we fold everything into the single number $n = |w|$. Second, we take the *worst case*: $f(n)$ is the longest the machine could run over *all* 2^n or more inputs of length n . (*Average-case* analysis, which averages over inputs of length n , is a different and often harder enterprise; we do not pursue it here.) A single step is one move of the machine: read the scanned cell, write a cell, shift the head one square, change state.

Caution. Running time counts *steps*, not *lines of pseudocode*. A high-level stage such as “scan the tape” is *not* one step; on a length- n tape it costs $\Theta(n)$ head moves. When you analyze an algorithm you must translate each stage into the number of machine steps it really consumes, then add up over all stages.

Asymptotic notation

Exact step counts like $7n^2 + 3n + 12$ are fragile – they change if we tweak the tape alphabet or the head-motion bookkeeping – and they obscure the one thing that survives such tweaks: the *growth rate*. Asymptotic notation keeps only that.

Definition 12.1.2 (Big- O and small- o). Let $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ (the nonnegative reals). We write

$$f(n) = O(g(n)) \quad \text{if there exist constants } c > 0 \text{ and } n_0 \in \mathbb{N} \text{ with } f(n) \leq c g(n) \text{ for all } n \geq n_0,$$

and we call g an *asymptotic upper bound* for f . We write

$$f(n) = o(g(n)) \quad \text{if} \quad \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0,$$

equivalently: for *every* constant $c > 0$ there is an n_0 with $f(n) < cg(n)$ for all $n \geq n_0$.

Read $f = O(g)$ as “ f grows no faster than g , up to a constant factor” and $f = o(g)$ as “ f grows *strictly* slower than g ” – the relationship between $\leq$ and $<$. The companion lower bounds are $f(n) = \Omega(g(n))$, meaning $g(n) = O(f(n))$ (so $f \geq cg$ eventually), and $f(n) = \Theta(g(n))$, meaning $f = O(g)$ and $f = \Omega(g)$ at once (f and g grow at the same rate). The constant c and threshold n_0 may be anything we like, as long as they are *fixed* once and for all and the inequality then holds for every $n \geq n_0$.

Remark. Inside $O(\cdot)$ the base of a logarithm vanishes: $\log_a n = (\log_b n)/(\log_b a)$ differs from $\log_b n$ by the constant factor $1/(\log_b a)$, which O absorbs. So we write $O(\log n)$ with no base. Likewise $n^{O(1)}$ means “ n^c for some constant c ” (the polynomial bounds), and $2^{O(n)}$ means “ 2^{cn} for some constant c ” (an exponential bound).

Worked Example 12.1.3 (Reading the highest-order term off a polynomial). Show that $f(n) = 4n^3 + 30n^2 + 100n + 7$ satisfies $f(n) = O(n^3)$, that $f(n) = O(n^4)$ as well, but that $f(n) \neq O(n^2)$.

Recognition. A polynomial’s growth is set by its *leading term*; everything else is lower order. The trigger “bound a polynomial” says: drop the lower terms and the leading coefficient, then the leading power is your g . Producing explicit c, n_0 then *proves* the bound from Definition 12.1.2.

Solution. The leading term is $4n^3$, so the candidate is $g(n) = n^3$. For $n \geq n_0 = 10$ each lower term is dominated by n^3 : indeed $30n^2 \leq 3n^3$, $100n \leq n^3$, and $7 \leq n^3$ once $n \geq 10$. Hence

$$4n^3 + 30n^2 + 100n + 7 \leq 4n^3 + 3n^3 + n^3 + n^3 = 9n^3 \quad (n \geq 10),$$

so $c = 9$, $n_0 = 10$ witness $f(n) = O(n^3)$. Since $n^3 \leq n^4$ for $n \geq 1$, the same c gives $f(n) = O(n^4)$ too – a *loose* but valid upper bound. Finally $f(n) \neq O(n^2)$: for any proposed c we have $f(n)/n^2 \geq 4n \rightarrow \infty$, so $f(n) \leq cn^2$ fails for all large n no matter how c, n_0 are chosen.

Which idea, and why? Big- O records only the leading power: keep the top term, discard its coefficient. A higher power (n^4) is always a legal – if wasteful – upper bound, while a lower power (n^2) can never bound a genuinely cubic function because the ratio diverges.

Check. At $n = 10$: $f = 4000 + 3000 + 1000 + 7 = 8007 \leq 9 \cdot 1000 = 9000$. The bound holds with room to spare, and tightens as n grows.

The next picture is the reason the whole theory bothers with the distinction. On a log vertical axis the polynomial and exponential curves separate violently.

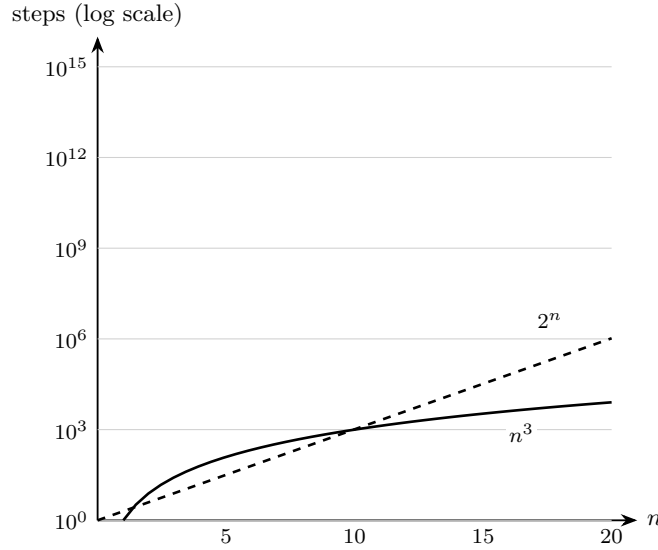
Worked Example 12.1.4 (Why polynomial vs. exponential is the line that matters). Compare the growth of a typical polynomial n^3 against a typical exponential 2^n , and explain why complexity theory draws its main boundary exactly between them.

Recognition. The trigger is the words “polynomial” and “exponential.” For *any* fixed power k , $n^k = o(2^n)$, so every polynomial is eventually crushed by every exponential. The gap is not a constant factor that O could hide – it is unbounded.

Solution. Tabulate the two functions, and the chasm is immediate.

n	n^3	2^n
10	10^3	$\approx 1.0 \times 10^3$
20	8.0×10^3	$\approx 1.0 \times 10^6$
40	6.4×10^4	$\approx 1.1 \times 10^{12}$
100	10^6	$\approx 1.3 \times 10^{30}$

At $n = 100$ the polynomial sits near a million – a fraction of a second of computing – while the exponential would outlast the age of the universe even at 10^{12} steps per second. Plotting both on a base-10 logarithmic axis (so a straight line means a fixed growth rate) makes the divergence visible: the polynomial bends *toward* the horizontal, the exponential stays a rising straight line.



Which idea, and why? All reasonable deterministic machines simulate one another with only a *polynomial* slowdown, so a polynomial-vs-polynomial difference is invisible to the model; an exponential is not. Drawing the boundary here makes “feasible” a property of the *problem*, independent of the machine.

Check. The crossover is early: 2^n already passes n^3 near $n = 10$ ($2^{10} = 1024$ vs $10^3 = 1000$) and never looks back, exactly as $n^3 = o(2^n)$ predicts.

Worked Example 12.1.5 (A small- o hierarchy). Verify the chain $\sqrt{n} = o(n) = o(n \log n) = o(n^2) = o(2^n)$ using Definition 12.1.2, and explain why $f(n) \neq o(f(n))$ for any f .

Recognition. Small- o is a limit-of-ratio test (Definition 12.1.2). The trigger “strictly slower” means we must drive each ratio f/g to 0, not merely bound it.

Solution. Take the consecutive ratios and send $n \rightarrow \infty$:

$$\frac{\sqrt{n}}{n} = \frac{1}{\sqrt{n}} \rightarrow 0, \quad \frac{n}{n \log n} = \frac{1}{\log n} \rightarrow 0, \quad \frac{n \log n}{n^2} = \frac{\log n}{n} \rightarrow 0, \quad \frac{n^2}{2^n} \rightarrow 0,$$

the last because 2^n outgrows every polynomial. Each limit is 0, so each “ $= o$ ” holds, and o is transitive, giving the full chain. But $f(n)/f(n) = 1 \not\rightarrow 0$, so f is never $o(f)$: a function does not grow strictly slower than itself. (Contrast $f(n) = O(f(n))$, which always holds with $c = 1$.)

Which idea, and why? o is the strict cousin of O : it demands the ratio vanish, not merely stay bounded, which is exactly why the reflexive case $f = o(f)$ fails while $f = O(f)$ succeeds.

Check. Sanity on $n \log n = o(n^2)$ at $n = 2^{10}$: $\log n = 10$ while $n = 1024$, so the ratio is $10/1024 \approx 0.01$, already tiny and still shrinking.

Analyzing a Turing machine, and the time classes

To turn step-counting into a working tool we analyze a concrete decider. Take the non-regular language

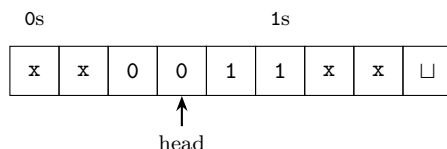
$$A = \{0^k 1^k \mid k \geq 0\},$$

the strings of some number of 0s followed by *equally many* 1s. A natural single-tape decider repeatedly crosses off one 0 and one 1 from the ends, checking they run out together.

Worked Example 12.1.6 (Time analysis of a single-tape decider for $\{0^k 1^k\}$). Analyze the running time of the machine M_1 that decides A by: (i) one left-to-right scan rejecting if a 0 follows a 1; then (ii) repeatedly scanning the tape and crossing off the leftmost remaining 0 together with the rightmost remaining 1, until symbols run out.

Recognition. The trigger is “loop whose body is a tape scan.” Cost = (number of iterations) $\times$ (cost per iteration); a full scan of an n -cell tape is $\Theta(n)$ steps.

Solution. Picture the tape after a few cross-offs (x marks a crossed cell), with the head mid-scan:



Stage (i) is one scan plus repositioning: $2n = O(n)$ steps. *Stages (ii)*: each pass crosses off exactly one 0 and one 1, i.e. removes 2 live symbols, so at most $n/2$ passes occur; each pass is a scan costing $O(n)$. The loop therefore costs $(n/2) \cdot O(n) = O(n^2)$. A final scan to confirm the tape is empty adds $O(n)$. Total:

$$O(n) + O(n^2) + O(n) = O(n^2).$$

Which idea, and why? When the loop body is a linear scan and the loop runs a linear number of times, the product is quadratic. The leading $O(n^2)$ swallows the two linear stages, by the arithmetic $O(n^2) + O(n) = O(n^2)$.

Check. On 000111 ($n = 6$): three cross-off passes, each touching ≤ 6 cells, plus the two framing scans – on the order of 6^2 steps, matching $O(n^2)$.

Definition 12.1.7 (Time complexity class). For a function $t: \mathbb{N} \rightarrow \mathbb{R}^+$, the class

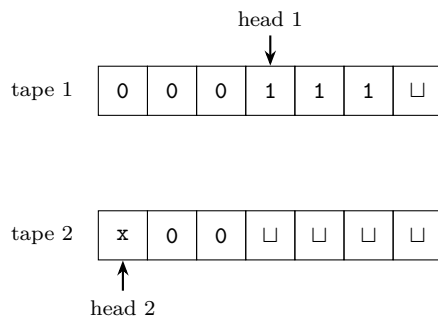
$$\text{TIME}(t(n)) = \{\mathcal{L} \mid \mathcal{L} \text{ is decided by some } O(t(n)) \text{ time deterministic single-tape TM}\}.$$

The previous example shows $A \in \text{TIME}(n^2)$. Sharper bookkeeping (cross off *half* the remaining symbols per pass, by parity) decides A in $O(n \log n)$ time, so in fact $A \in \text{TIME}(n \log n)$ – the best a single tape can do for this language. The next result shows a *second tape* does even better.

Worked Example 12.1.8 (A second tape buys linear time for $\{0^k 1^k\}$). Show that A is decided in time $O(n)$ by a *two-tape* machine M_3 that copies the block of 0s onto tape 2, then deletes one tape-2 0 per 1 read on tape 1.

Recognition. Two heads move independently, so a “count and match” that cost a nested loop on one tape becomes a single synchronized pass. The trigger is “a second tape lets us remember a block verbatim instead of re-scanning for it.”

Solution. Tape 1 holds the input and is read once left to right; tape 2 accumulates a copy of the 0-block, which is then consumed one cell per 1:



Stage 1, a single scan, verifies the form 0^*1^* in $O(n)$. Stage 2 copies the 0s to tape 2 as head 1 advances: $O(n)$. Stage 3 reads the 1s on tape 1, crossing off one tape-2 0 each; mismatched counts are caught when one tape runs out first – $O(n)$. Every stage is a single coordinated pass, so the total is $O(n)$, which is optimal since merely reading the input takes n steps.

Which idea, and why? Extra tapes are extra free memory: the inner re-scan that made the one-tape machine quadratic disappears because tape 2 stores the 0-block to be matched directly against the 1s.

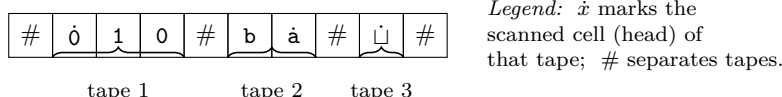
Check. On 000111: copy 3 zeros (3 steps), match against 3 ones (3 steps), framing scans $O(n)$ – linear, versus the one-tape machine’s $\sim n^2$.

How much does the model matter?

We have now decided the *same* language A in $O(n \log n)$ on one tape but $O(n)$ on two: time complexity is *model-dependent*. This is the sharpest break from computability theory, where the Church–Turing thesis made all reasonable models equivalent. Here the model changes the running time. The good news is that for the models we care about the change is only *polynomial*.

Theorem 12.1.9 (Multitape simulation costs at most a square). *Let $t(n) \geq n$. Every $t(n)$ time multitape Turing machine has an equivalent single-tape Turing machine running in time $O(t(n)^2)$.*

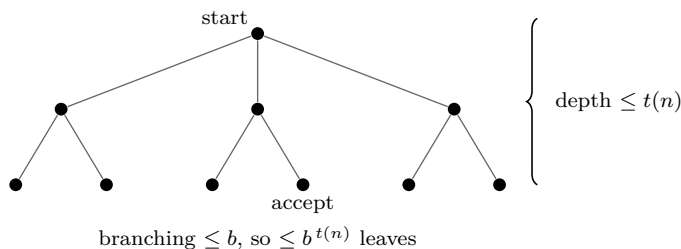
Proof. Let M be a k -tape machine running in time $t(n)$; build a single-tape S simulating it. S stores the contents of all k tapes consecutively on its one tape, separated by a delimiter, with each head position marked by a dotted symbol:



To simulate *one* step of M , machine S sweeps once across the active region to read the k marked symbols, then sweeps back to write the new symbols and move the dots, occasionally shifting a block right to make room when a head runs off its end. Each marked tape of M has active length at most $t(n)$ (it cannot visit more than $t(n)$ cells in $t(n)$ steps), so the active region of S has length at most $k \cdot t(n) = O(t(n))$, and one sweep costs $O(t(n))$. Thus simulating one M -step costs $O(t(n))$, and M makes $t(n)$ steps, so the simulation costs $t(n) \cdot O(t(n)) = O(t(n)^2)$, plus $O(n)$ to lay out the initial format. Since $t(n) \geq n$, the total is $O(t(n)^2)$. $\square$

Theorem 12.1.10 (Nondeterministic simulation costs at most an exponential). *Let $t(n) \geq n$. Every $t(n)$ time nondeterministic single-tape Turing machine has an equivalent deterministic single-tape Turing machine running in time $2^{O(t(n))}$.*

Proof. Let N be a nondeterministic decider running in time $t(n)$, meaning *every* branch of its computation halts within $t(n)$ steps. Its computation on an input is a tree: the root is the start configuration, and a node's children are the configurations reachable by N 's nondeterministic choices. Let b be the largest number of choices at any step. Then the tree has depth $\leq t(n)$ and branching $\leq b$, so it has at most $b^{t(n)}$ leaves and $O(b^{t(n)})$ nodes total.



A deterministic D accepts iff some leaf accepts, so D searches the tree breadth-first, visiting every node. Reaching one node from the root and replaying its choices costs $O(t(n))$, and there are $O(b^{t(n)})$ nodes, giving

$$O(t(n)) \cdot O(b^{t(n)}) = 2^{O(t(n))}$$

on a multitape D . Converting that to a single tape squares the time (Theorem 12.1.9), and $(2^{O(t(n))})^2 = 2^{O(t(n))}$ is unchanged. $\square$

These two theorems are the whole reason the next section can speak of “polynomial time” without naming a machine: deterministic models differ *polynomially* (Theorem 12.1.9), so any polynomial bound is robust across them, whereas the only known deterministic simulation of nondeterminism is *exponential* (Theorem 12.1.10) – the gap that the P versus NP question is about.

Worked Example 12.1.11 (Pushing a bound through the simulation theorems). A language $\mathcal{L}$ is decided by a 3-tape machine in time $O(n^2)$. What single-tape bound follows? And if instead $\mathcal{L}$ were decided by a *nondeterministic* single-tape machine in time $O(n)$, what deterministic bound follows?

Recognition. The trigger is “convert between models,” which is exactly what Theorems 12.1.9 and 12.1.10 are for: feed $t(n)$ in, read the cost out.

Solution. For the multitape case, Theorem 12.1.9 with $t(n) = n^2$ gives a single-tape bound of $O((n^2)^2) = O(n^4)$ – still polynomial. For the nondeterministic case, Theorem 12.1.10 with $t(n) = n$ gives a deterministic bound of $2^{O(n)}$ – exponential. The contrast is the point: crossing from multitape to single tape stays polynomial, but crossing from nondeterministic to deterministic explodes.

Which idea, and why? Each theorem is a transfer rule on the time bound: multitape-to-single squares it (polynomial in, polynomial out), while nondeterministic-to-deterministic exponentiates it.

Check. Both source bounds (n^2 and n) respect $t(n) \geq n$ (the hypothesis), and both rules reduce to the source bound when no conversion is needed: a single-tape deterministic machine is already its own simulator at cost $O(t(n))$.

Method (How to analyze the running time of a TM algorithm).

1. **Fix the size parameter.** Decide what n is – the length of the input string. Express every count in terms of n (e.g. a graph on m nodes has encoding length $n = \Theta(m^2)$ as an adjacency matrix, so a bound polynomial in m is polynomial in n).
2. **Bound the number of stages/iterations.** For each loop, find how many times its body runs as a function of n (here: “each pass removes two symbols, so $\leq n/2$ passes”).
3. **Bound the cost of one stage in TM steps.** Translate the high-level body into head moves: a full tape scan is $O(n)$, not $O(1)$.
4. **Multiply and add.** Cost = $\sum_{\text{stages}} (\text{count}) \times (\text{per-stage cost})$. Collapse with O arithmetic: lower-order terms vanish, constant factors vanish.
5. **Name the class.** Read off the leading term $t(n)$ and conclude $\mathcal{L} \in \text{TIME}(t(n))$. If the model was multitape or nondeterministic, apply Theorem 12.1.9 or 12.1.10 to get a single-tape deterministic bound.

Caution. Big- O is an *upper* bound only. Writing $f(n) = O(n^3)$ does *not* claim f grows like n^3 ; a linear f also satisfies $f(n) = O(n^3)$. To assert a matching lower bound you need Ω , and to assert the rate *exactly* you need Θ . Saying “the algorithm is $O(n^2)$, therefore it cannot be faster” confuses an upper bound with a lower bound – the very mistake that hides the $n \log n$ and linear deciders for A above.

Caution. The equals sign in $f(n) = O(g(n))$ is one-directional shorthand for “ f is one of the functions bounded by g .” It does *not* mean $O(g(n)) = f(n)$, and you may not cancel it across an equation: from $n = O(n^2)$ and $n^2 = O(n^2)$ you cannot conclude $n = n^2$. Read every $O(\cdot)$ as “some function that grows no faster than this,” never as a specific value.

Exercises.

1. Using Definition 12.1.2 directly, find explicit constants c and n_0 proving $3n^2 + 50n + 200 = O(n^2)$. Then show that this same function is *not* $O(n)$ by arguing that the ratio $f(n)/n$ is unbounded.
2. For each pair (f, g) below decide whether $f = O(g)$, $f = o(g)$, $f = \Theta(g)$, or $g = o(f)$ – list every relation that holds:

$(a) f = n^2, g = n^2 + n; \quad (b) f = n \log n, g = n^2;$
 $(c) f = 2^n, g = 3^n; \quad (d) f = \log_2 n, g = \log_{10} n.$
3. Order the following functions from slowest-growing to fastest-growing, and justify each neighbouring “ $= o$ ” by a ratio limit: $\sqrt{n}$, n , $n \log n$, n^2 , $n^2 \log n$, 2^n , $n!$.
4. Decide whether each statement is true or false; give a one-line reason. (a) If $f(n) = O(g(n))$ then $g(n) = \Omega(f(n))$. (b) If $f(n) = o(g(n))$ then $f(n) = O(g(n))$. (c) $2^{n+1} = O(2^n)$. (d) $2^{2n} = O(2^n)$.
5. A single-tape Turing machine decides a language $\mathcal{L}$ in time $O(n^3)$. A classmate concludes “so $\mathcal{L}$ requires at least cubic time.” Explain precisely what is wrong with this inference, and state what kind of bound *would* support the claim “ $\mathcal{L}$ requires at least cubic time.”

6. Consider the language $B = \{w\#w \mid w \in \{0,1\}^*\}$ over the alphabet $\{0,1,\#\}$. Describe a *single-tape* decider for B that repeatedly matches the symbol just left of $\#$ against the corresponding symbol to its right by crossing both off, and draw one tape snapshot mid-computation (use $\mathbf{x}$ for a crossed cell, an arrow for the head). Argue informally that your machine runs in time $O(n^2)$.
7. For the language B of the previous exercise, describe a *two-tape* machine that decides B in time $O(n)$, and explain in one sentence what the second tape is used for.
8. Let $C = \{0^k \mid k \text{ is a power of } 2\}$. The natural single-tape decider repeatedly halves the number of 0s (by crossing off every other one), rejecting on the empty input or if an odd count > 1 ever appears, and accepting when exactly one remains. Bound the number of halving passes as a function of n , bound the cost of one pass, and conclude an $O(n \log n)$ running time.
9. Find the bug. A student claims this machine decides $A = \{0^k 1^k\}$ in linear time on *one* tape: “Stage 1: in a single left-to-right scan, count the 0s in a counter and the 1s in a second counter; Stage 2: accept iff the counters are equal.” Identify the flaw in the *time* claim (think about what “counter” means for a finite-state control on a single tape) and state the correct single-tape complexity of A .
10. Suppose $\mathcal{L}_1 \in \text{TIME}(n^2)$ and $\mathcal{L}_2 \in \text{TIME}(n^3)$ via single-tape deciders M_1, M_2 . Show that the language $\mathcal{L}_1 \cup \mathcal{L}_2$ is in $\text{TIME}(n^3)$ by describing a machine that runs M_1 then M_2 on the same input, and bounding its running time. Where does your bound use the fact that $O(n^2) + O(n^3) = O(n^3)$?
11. A language $\mathcal{L}$ is decided by a 5-tape Turing machine in time $O(n \log n)$. Using the simulation theorem of this section, give the best single-tape deterministic time bound you can for $\mathcal{L}$, and state explicitly which theorem and which value of $t(n)$ you used.
12. A nondeterministic single-tape Turing machine decides $\mathcal{L}$ in time $O(n^2)$. Give the deterministic single-tape time bound guaranteed by the simulation theorem, and explain in one sentence why this bound is exponential rather than polynomial in n .

12.2 The Class P

The previous section measured running time and sorted languages into the classes $\text{TIME}(t(n))$. The trouble with any single class $\text{TIME}(n^2)$ is that it is fragile: a language sitting in $\text{TIME}(n^2)$ on a two-tape machine may need $\text{TIME}(n^4)$ on a one-tape machine, so the exact exponent depends on bookkeeping we would rather forget. The one key idea of this section is to coarsen the scale until the bookkeeping disappears: collect *all* polynomial time bounds into one class, P . That single act buys robustness – P is the same class on every reasonable deterministic model – and a working definition of “feasible.” We make P precise, prove the closure-under-composition fact that makes polynomial-time reasoning painless, and then certify membership for three concrete problems: reachability in a graph, testing two numbers for common factors, and deciding any context-free language.

Definition 12.2.1 (The class P). P is the class of languages decidable in polynomial time on a deterministic single-tape Turing machine:

$$P = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

Equivalently, $\mathcal{L} \in P$ iff some deterministic decider runs in time $O(n^k)$ for some fixed constant k independent of the input.

Two facts make P the right object. First, it is *model-invariant*: by the simulation theorems of the previous section, a single-tape machine simulates a k -tape machine with at most a squaring of the time, and squaring a polynomial leaves a polynomial, so P does not change if we switch tape models, add registers, or pass to any reasonable random-access model. Second, it matches the rough boundary of *practical* solvability: an n^3 algorithm on an input of size 1000 does about 10^9 steps (a moment on a real machine), whereas a 2^n algorithm does more steps than there are atoms in the observable universe.

Definition 12.2.2 (Polynomially bounded; the encoding convention). A function $p: \mathbb{N} \rightarrow \mathbb{N}$ is *polynomially bounded* if $p(n) = O(n^k)$ for some constant k . Throughout, $\langle X \rangle$ denotes a *reasonable* encoding of an object X as a string – one for which encoding and decoding are themselves polynomial and for which numbers are written in base $b \geq 2$ (never in unary). Under any reasonable encoding the size of a graph is polynomial in its number of nodes, so “polynomial in the number of nodes” and “polynomial in the input length” coincide.

The unary caveat matters. The integer n in binary occupies $\lceil \log_2(n+1) \rceil$ symbols, so an algorithm that loops n times runs in time *exponential* in that input length. Writing n in unary as 1^n would hide the blow-up by inflating the input; banning unary keeps the length honest.

Why P is closed under composition

The reason we can chain polynomial-time steps freely – run one polynomial algorithm, feed its output to a second – is the elementary algebraic fact that polynomials are closed under composition and addition. We isolate it because every later running-time argument leans on it.

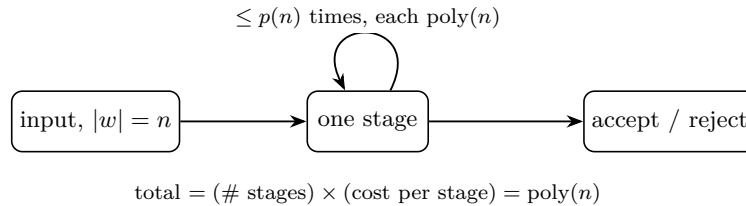
Proposition 12.2.3 (Composition of polynomial bounds). *Let p and q be polynomially bounded. Then $n \mapsto p(n) + q(n)$, $n \mapsto p(n) \cdot q(n)$, and $n \mapsto p(q(n))$ are all polynomially bounded.*

Proof. Write $p(n) = O(n^a)$ and $q(n) = O(n^b)$, so for some constants c, d and a threshold n_0 we have $p(n) \leq cn^a$ and $q(n) \leq dn^b$ for all $n \geq n_0$. Then $p(n) + q(n) \leq cn^a + dn^b \leq (c + d)n^{\max(a,b)}$, which is $O(n^{\max(a,b)})$. The product satisfies $p(n)q(n) \leq cdn^{a+b} = O(n^{a+b})$. For the composition, first enlarge c so that $p(m) \leq cm^a + c$ for *every* m (only the finitely many $m < n_0$ need this, as $q(n)$ may be small); then $p(q(n)) \leq c(q(n))^a + c \leq c(dn^b)^a + c = cd^a n^{ab} + c = O(n^{ab})$. Each bound is a fixed power of n , hence polynomial. $\square$

Corollary 12.2.4 (The stage-counting principle). *Suppose an algorithm runs for at most $p(n)$ stages on inputs of length n , where p is polynomially bounded, and suppose each stage can be carried out in at most $q(m)$ steps on a string of length m , where q is polynomially bounded and no string ever produced has length exceeding some polynomial $r(n)$. Then the algorithm runs in polynomial time.*

Proof. Total time is at most $p(n) \cdot q(r(n))$ (we may take q nondecreasing, replacing $q(m)$ by $\max_{i \leq m} q(i)$, which is still polynomially bounded). By Proposition 12.2.3, $q(r(n))$ is polynomially bounded and the product of two polynomially bounded functions is polynomially bounded, so the total is $O(n^k)$ for some k . $\square$

Corollary 12.2.4 is the workhorse. To prove a language is in P we now only ever do two things: bound the *number of stages* by a polynomial, and check that *each stage* is a polynomial-time operation (a scan, a comparison, an arithmetic step, a table lookup). We never again count individual head motions. The following schematic is the shape of every such argument.



Worked Example 12.2.5 (Reachability in a directed graph: $\text{PATH} \in \text{P}$). Let $\text{PATH} = \{\langle G, s, t \rangle \mid G \text{ is a digraph with a directed path from } s \text{ to } t\}$. Show $\text{PATH} \in \text{P}$.

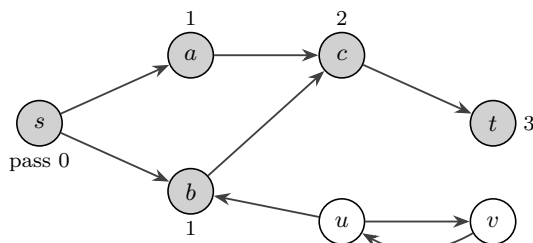
Recognition. The task is a yes/no question about a graph, and the obvious brute force – enumerate candidate paths – is exponential, so the trigger is “a search problem where naive enumeration explodes but a *marking/propagation* sweep stays polynomial.” That points to breadth-first reachability rather than path enumeration.

Solution. A path, if one exists, can be taken simple, so it visits at most m nodes (where m is the number of nodes); there are about m^m candidate node-sequences of that length, which is exponential – brute force is out. Instead *mark* reachable nodes outward from s :

$M =$ “On input $\langle G, s, t \rangle$:

1. Mark s .
2. Repeat until no new node gets marked:
3. scan every edge (a, b) ; if a is marked and b is not, mark b .
4. If t is marked, *accept*; else *reject*.”

Consider the seven-node digraph below; nodes are shaded in the order they are marked starting from s . Each outer pass marks at least one new node, so after at most $m - 1$ passes the marked set stops growing.



Which idea, and why? Reachability is a fixed-point of the rule “a neighbour of a marked node is marked”; iterating it marks every reachable node and the marked set can grow at most m times, which converts an exponential search into a polynomial sweep.

Check (stage count). Stage 3 scans all $\leq m^2$ edges, a polynomial-time operation, and the loop runs at most m times because each non-final pass marks a new node; by Corollary 12.2.4 the total is $O(m^3)$, polynomial in the input. In the picture t is marked on pass 3, while u, v are never reached, so M accepts – correct.

Worked Example 12.2.6 (Common factors without factoring: $RELPRIME \in P$). Call x, y *relatively prime* if $\gcd(x, y) = 1$. Let $RELPRIME = \{\langle x, y \rangle \mid \gcd(x, y) = 1\}$ with x, y written in binary. Show $RELPRIME \in P$.

Recognition. The naive method – try every divisor up to $\min(x, y)$ – runs a number of steps proportional to the *value* of the input, which is exponential in its *length*. The trigger “cost proportional to the numeric value, not its digit length” flags that brute force is exponential and we need an algorithm whose progress is geometric in the value.

Solution. Use the Euclidean algorithm, whose key invariant is that one remainder step shrinks the larger operand by at least half. Run

$E =$ “On input $\langle x, y \rangle$ (binary):

1. Repeat until $y = 0$: $x \leftarrow x \bmod y$; swap x and y .
2. Output x .”

Then $\gcd(x, y)$ is the output, and $\langle x, y \rangle \in RELPRIME$ iff that output is 1. Why each iteration halves: after $x \leftarrow x \bmod y$ we have $x < y$; the swap restores $x > y$; so at the next remainder step either $x/2 \geq y$, giving $x \bmod y < y \leq x/2$, or $x/2 < y$, giving $x \bmod y = x - y < x/2$. Either way the new x is below half the old. The run on $\langle 48, 30 \rangle$:

step	x	y	$x \bmod y$
0	48	30	18
1	30	18	12
2	18	12	6
3	12	6	0
4	6	0	— output 6

Which idea, and why? Replacing trial division by the Euclidean recurrence turns a value-proportional (hence length-exponential) search into one whose operand halves each round, so the round count is the *bit length*, not the value – this is the standard “measure progress against the value, count stages against the length” move.

Check (stage count). Each operand is halved every two iterations, so the loop runs $O(\log x + \log y) = O(n)$ times, where n is the input length; each iteration is one binary division, a polynomial-time operation. By Corollary 12.2.4, E is polynomial. Here $\gcd(48, 30) = 6 \neq 1$, so $\langle 48, 30 \rangle \notin RELPRIME$, whereas $\gcd(35, 24)$ would run out to 1.

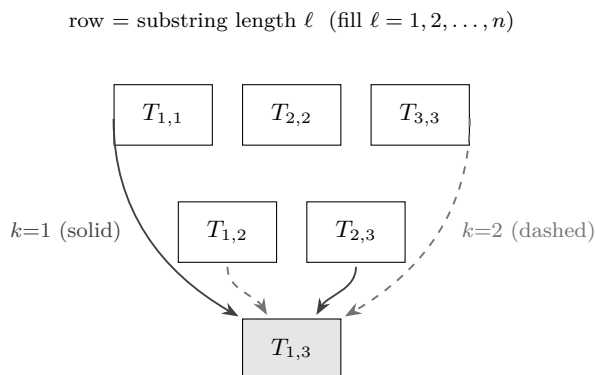
Worked Example 12.2.7 (Every context-free language is in P (the CYK table)). Let $\mathcal{L}$ be context-free, generated by a grammar G in Chomsky normal form. Show $\mathcal{L} \in P$ by deciding $w \in \mathcal{L}$ for $w = w_1 w_2 \cdots w_n$.

Recognition. Membership “does G derive w ?” has exponentially many candidate derivations, but the subproblems – “which variables generate substring $w_i \cdots w_j$?” – overlap heavily. The trigger “overlapping subproblems indexed by intervals” is exactly the cue for *dynamic programming*: tabulate by substring length.

Solution. Build a table whose cell (i, j) holds the set $T_{i,j}$ of variables deriving the substring $w_i \cdots w_j$. Fill by increasing length $\ell = j - i + 1$. For length 1, put A into $T_{i,i}$ whenever $A \rightarrow w_i$ is a rule. For $\ell \geq 2$, a CNF rule is $A \rightarrow BC$, so any derivation of $w_i \cdots w_j$ splits at some position k with $i \leq k < j$: B derives the left piece $w_i \cdots w_k$ and C the right piece $w_{k+1} \cdots w_j$. Hence

$$T_{i,j} = \{A \mid \exists k, A \rightarrow BC, B \in T_{i,k}, C \in T_{k+1,j}\}.$$

Accept iff the start variable $S \in T_{1,n}$ (for $w = \varepsilon$, accept iff $S \rightarrow \varepsilon$ is a rule). The dependency is upward: a longer interval is filled purely from strictly shorter ones already in the table, as the arrows show for $T_{1,3}$ splitting at $k = 1$ and $k = 2$.



Which idea, and why? Trying all derivations is exponential, but the interval subproblems overlap, so dynamic programming – one table cell per substring, each filled once from shorter substrings – collapses the work to a polynomial number of constant-cost combinations.

Check (stage count). The table has $O(n^2)$ cells; filling one cell tries $O(n)$ split points k and a fixed number of rules per split, so each cell costs $O(n)$. Total $O(n^3)$ cell-updates, each a polynomial-time set operation; by Corollary 12.2.4 the decider is polynomial, so $\mathcal{L} \in P$ and therefore $CFL \subseteq P$.

Theorem 12.2.8 (Three membership results). $PATH \in P$, $RELPRIME \in P$, and every context-free language is in P; in particular $CFL \subseteq P$.

Proof. Each of the three worked examples exhibits a deterministic decider together with a stage count that, via Corollary 12.2.4, is polynomial: $O(m^3)$ for $PATH$ (marking sweep), $O(n)$ rounds of binary division for $RELPRIME$ (Euclid), and $O(n^3)$ table updates for CNF membership (CYK). Polynomial running time is exactly Definition 12.2.1, so all three languages lie in P, and since the third argument applies to an arbitrary CFL we get $CFL \subseteq P$. $\square$

A useful way to picture where P sits is the inclusion chain it anchors. The marking and table algorithms above put genuinely useful problems inside P; the class of all decidable languages is far larger, and P is contained in NP (Section 12.3) and in PSPACE.

$$\text{CFL} \longleftrightarrow \text{P} \longleftrightarrow \text{NP} \longleftrightarrow \text{PSPACE} \longleftrightarrow \text{DECIDABLE}$$

Method (How to certify a language is in P).

1. **Describe a deterministic algorithm in numbered stages.** Work at the high level (scans, comparisons, arithmetic, table lookups); never mention tape head motions.
2. **Bound the number of stages by a polynomial in n .** Find the quantity each loop pass drives toward termination (a newly marked node, a halved operand, a filled table cell) and read off how many passes that allows.
3. **Check each stage is itself polynomial time.** A single scan, one arithmetic step on numbers of polynomial length, or one set/table operation each qualifies.
4. **Verify nothing produced grows super-polynomially.** Intermediate strings, numbers, and tables must stay polynomial in length so step 3 stays honest.
5. **Conclude via Corollary 12.2.4.** Polynomially many stages, each polynomial, with bounded intermediate sizes, multiply to a polynomial total – the language is in P.

Caution. Counting *loop iterations* is not the same as counting *input length*. An algorithm that loops “ x times” on a number x given in binary is *exponential*, because x can be as large as 2^n for input length n . Trial-division primality and the brute-force factoring search both fall into this trap. The fix is to measure progress against the input length: Euclid’s algorithm loops $O(n)$ times precisely because each round halves a value, so the round count tracks the *bit length*, not the value.

Caution. Encoding matters, and unary is the classic cheat. A problem can look polynomial only because its input was padded: if a number x is written in unary as 1^x , an algorithm running in time proportional to x is “polynomial in the (inflated) input length,” but the same algorithm is exponential under the honest binary encoding. Always assume base $b \geq 2$ for numbers, and check that your claimed polynomial bound is in the length of a *reasonable* encoding, not a wasteful one.

Exercises.

1. Let $f(n) = 4n^3 \log_2 n + 17n^3 + 200n + 9$. Find the smallest integer k with $f(n) = O(n^k)$, and explain in one sentence why $f(n) \neq O(n^3)$ but $f(n) = o(n^4)$. (The log factor is the point.)
2. Suppose algorithm A runs in time $O(n^2)$ and produces an output string of length $O(n^2)$, and algorithm B runs in time $O(m^3)$ on inputs of length m . You run A , then feed its output to B . Give a polynomial bound on the total running time as a function of n , and name the result that guarantees the composite is still polynomial.
3. Run the marking algorithm for *PATH* on the digraph with nodes $\{s, a, b, c, t\}$ and edges $s \rightarrow a, a \rightarrow b, b \rightarrow a, s \rightarrow c, c \rightarrow t$ removed (i.e. *without* the edge $c \rightarrow t$). Draw the digraph, list which nodes are marked on each pass starting from s , and state whether the algorithm accepts $\langle G, s, t \rangle$.
4. Design a polynomial-time algorithm in numbered stages deciding

$$\text{EVEN-PATH} = \{\langle G, s, t \rangle \mid G \text{ has a directed } s\text{-to-}t \text{ walk of even length}\}.$$

(Hint: make two copies of every node, one “even” and one “odd,” and let every original edge flip the parity.) Bound the number of stages.

5. Trace the Euclidean algorithm on $\langle 84, 30 \rangle$. Tabulate x , y , and $x \bmod y$ at every iteration as in the worked example, give $\text{gcd}(84, 30)$, and decide whether $\langle 84, 30 \rangle \in \text{RELPRIME}$.
6. Prove that if $\mathcal{L}_1 \in \text{P}$ and $\mathcal{L}_2 \in \text{P}$ over the same alphabet, then $\mathcal{L}_1 \cap \mathcal{L}_2 \in \text{P}$ and $\mathcal{L}_1 \cup \mathcal{L}_2 \in \text{P}$. (Run both deciders in sequence; use the composition result to bound the total time.)
7. Prove that P is closed under complement: if $\mathcal{L} \in \text{P}$ then $\overline{\mathcal{L}} \in \text{P}$. Explain why the same swap-the-accept-and-reject argument that works for P is *not* obviously available for NP.
8. Let G be the Chomsky-normal-form grammar with rules $S \rightarrow AB \mid BC, A \rightarrow BA \mid \mathbf{a}, B \rightarrow CC \mid \mathbf{b}, C \rightarrow AB \mid \mathbf{a}$. Build the CYK table for $w = \mathbf{baaba}$ and decide whether $w \in L(G)$. Show the table cells by substring length and indicate which split positions k populate the top cell $T_{1,5}$.

9. For the CYK decider, suppose the input grammar has v variables and r rules. Express the running time as a function of n , v , and r , and argue that for a *fixed* grammar the bound is $O(n^3)$. Where in the analysis does the assumption that v and r are constants get used?
10. *Find the bug.* A student claims the following decides

$$COMPOSITE = \{\langle x \rangle \mid x = pq \text{ for integers } p, q > 1\}$$

in polynomial time: “On input $\langle x \rangle$ with x in binary, for $d = 2, 3, \dots, x-1$ test whether d divides x ; accept if some d does.” Each division is fast, so the student says the algorithm is polynomial. Identify the flaw precisely (relate the loop count to the input length $n = |\langle x \rangle|$) and state what the true running time is.

11. *Find the bug.* Someone argues “ $UNARY-FACTOR = \{1^x \mid x \text{ is composite}\}$ is in P, because trial division up to x takes about x steps and the input has length x , so it is linear in the input.” Is the conclusion that $UNARY-FACTOR \in P$ correct? Explain what is and is not legitimate about the argument, and why the same reasoning fails for the binary encoding $\{\langle x \rangle \mid x \text{ is composite}\}$.
12. Define $TRIANGLE = \{\langle G \rangle \mid G \text{ is an undirected graph containing three mutually adjacent nodes}\}$. Give a polynomial-time algorithm in numbered stages, bound its number of stages as a function of the node count m , and verify each stage is polynomial time. Why does this *not* immediately put the general *CLIQUE* problem in P?

12.3 The Class NP and Verifiers

The class P of the previous section captured problems we can *solve* quickly. This section captures a different and, on the surface, weaker demand: problems whose “yes” answers we can *check* quickly when someone hands us a short hint. The one key idea is that for a great many natural problems – finding a route, a clique, a satisfying assignment – searching for a solution looks expensive, yet *recognizing* a proposed solution is easy. Packaging that asymmetry precisely gives the class NP. We define it two ways, by a polynomial-time *verifier* fed a *certificate* and by a *nondeterministic* polynomial-time Turing machine, prove the two definitions coincide, and read several problems through both lenses.

Definition 12.3.1 (Verifier and certificate). A *verifier* for a language $\mathcal{L} \subseteq \Sigma^*$ is a deterministic algorithm V such that

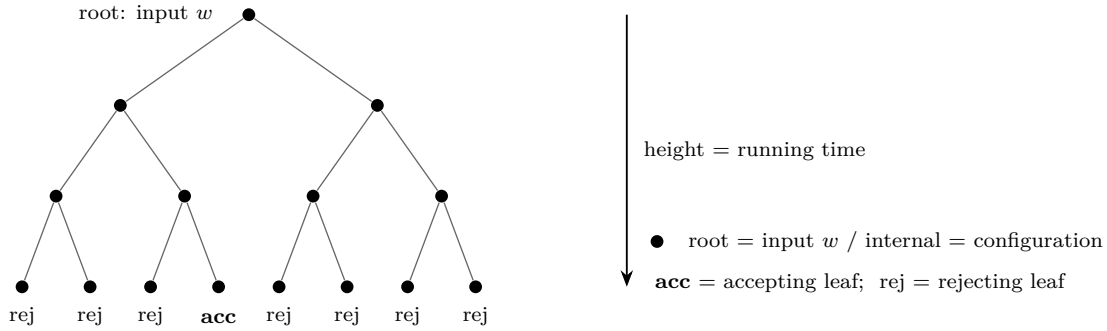
$$\mathcal{L} = \{w \in \Sigma^* \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}.$$

The auxiliary string c is a *certificate* (or *proof*, or *witness*) of w ’s membership. We measure the running time of V as a function of $|w|$ alone. The verifier V is a *polynomial-time verifier* if it runs in time $O(|w|^k)$ for some constant k , and $\mathcal{L}$ is *polynomially verifiable* if it has one.

Two consequences of measuring time in $|w|$ are worth stating at once. First, a polynomial-time verifier can read only polynomially many symbols of c , so without loss of generality every *useful* certificate has length $|c| = O(|w|^k)$: a certificate longer than the time bound cannot be examined and may as well be truncated. Second, the quantifier is one-sided. For $w \in \mathcal{L}$ *some* certificate makes V accept; for $w \notin \mathcal{L}$ *no* string c – however cleverly chosen – can make V accept. The verifier never has to detect a forged proof by working hard; a correct verifier simply has no accepting certificate to find when the answer is “no.”

Definition 12.3.2 (The class NP). NP is the class of languages that have polynomial-time verifiers.

The name NP abbreviates *nondeterministic polynomial time*, which looks unrelated to verifiers until we recall how a nondeterministic Turing machine (NTM) computes. On an input the machine may have several legal moves at a step; its possible runs form a tree, with the input at the root and each branch a complete computation. We declared earlier (the running time of a nondeterministic decider) that an NTM *accepts* w if *some* branch ends in an accept state, and that its running time on inputs of length n is the height of the tallest branch over all length- n inputs. The next figure fixes that picture.



The whole tree may have exponentially many branches, yet *one* accepting branch is enough. That single lucky branch is exactly the data a verifier would package into a certificate, which is why the two notions of NP turn out to be the same.

Definition 12.3.3 (Nondeterministic time and NTIME). For $t: \mathbb{N} \rightarrow \mathbb{R}^+$, a nondeterministic decider N runs in time $t(n)$ if on every input of length n every branch of its computation halts within $t(n)$ steps. The class $\text{NTIME}(t(n))$ is the collection of languages decided by some $O(t(n))$ -time NTM.

Theorem 12.3.4 (Verifier $\equiv$ nondeterministic machine). A language $\mathcal{L}$ is in NP if and only if some nondeterministic Turing machine decides $\mathcal{L}$ in polynomial time. Consequently $\text{NP} = \bigcup_{k \geq 1} \text{NTIME}(n^k)$.

Proof. ($\Rightarrow$) Suppose $\mathcal{L} \in \text{NP}$, witnessed by a verifier V running in time n^k on inputs of the form $\langle w, c \rangle$ (measuring in $n = |w|$). Build an NTM N :

N on input w of length n : (1) nondeterministically write a string c of length at most n^k , one symbol at a time by branching over Σ (or stopping) at each of up to n^k positions; (2) run V on $\langle w, c \rangle$; (3) accept iff V accepts.

Stage 1 has a branch for each candidate c ; stage 2 runs in n^k steps. If $w \in \mathcal{L}$ then some certificate c exists and the branch that guesses it accepts, so N accepts. If $w \notin \mathcal{L}$ then no c makes V accept, so every branch rejects and N rejects. Every branch halts within $O(n^k)$ steps, so N is a polynomial-time NTM for $\mathcal{L}$.

($\Leftarrow$) Suppose a polynomial-time NTM N decides $\mathcal{L}$ within n^k steps per branch. At each step N has at most b legal choices, where b depends only on N 's transition function. Encode a branch as the string c of the choice indices (1 through b) taken at each of its $\leq n^k$ steps; then $|c| = O(n^k)$. Build a verifier:

V on input $\langle w, c \rangle$: simulate N on w , at step i following the i -th symbol of c to pick the move; accept iff this single branch of N accepts.

Simulating one fixed branch is deterministic and takes $O(n^k)$ steps. If $w \in \mathcal{L}$, some branch of N accepts, and its choice-sequence is a certificate making V accept. If $w \notin \mathcal{L}$, no branch accepts, so no c makes V accept. Hence V is a polynomial-time verifier and $\mathcal{L} \in \text{NP}$. The class identity $\text{NP} = \bigcup_k \text{NTIME}(n^k)$ is the same statement re-expressed through Definition 12.3.3. $\square$

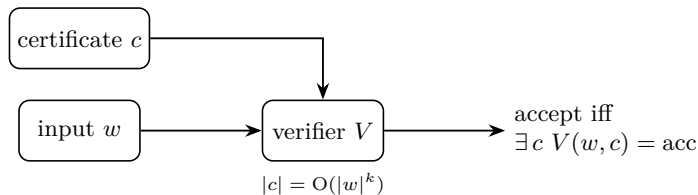
Remark. The two directions trade the same object back and forth: the accepting *branch* of the NTM is the verifier's *certificate*, and guessing the certificate is the NTM's *nondeterminism*. Keep this dictionary handy – “guess a witness, then check it” is the universal recipe for placing a problem in NP, and you may use whichever half is more convenient.

Proposition 12.3.5 ($\text{P} \subseteq \text{NP}$). Every polynomially decidable language is polynomially verifiable.

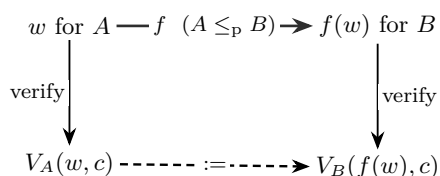
Proof. Let $\mathcal{L} \in \text{P}$ via a polynomial-time decider D . Define a verifier V that on $\langle w, c \rangle$ ignores c entirely and outputs $D(w)$. Then V runs in polynomial time in $|w|$, and $w \in \mathcal{L}$ iff V accepts $\langle w, c \rangle$ for some (indeed every) c , e.g. $c = \varepsilon$. So $\mathcal{L} \in \text{NP}$. $\square$

The two halves of the verifier story – the *characterization* of NP and its *stability* under polynomial reductions – are worth seeing as a single picture. The top row is the definition; the bottom row is the certificate-transport square that makes NP-membership flow backward along a reduction $A \leq_p B$.

characterization: $w \in \mathcal{L} \iff \exists c \, V(w, c) = \text{accept}$



transport: same certificate c , reused



Takeaway. $\mathcal{L} \in \text{NP}$ iff a polynomial-time verifier V accepts $\langle w, c \rangle$ for *some* short certificate c exactly when $w \in \mathcal{L}$; and a polynomial reduction $A \leq_p B$ pulls a certificate for $f(w) \in B$ back to certify $w \in A$, by setting $V_A(w, c) := V_B(f(w), c)$, so NP-membership transfers backward along $\leq_p$ (the invariant carried across the square is certificate validity). *Check.* If $B \in \text{NP}$ via V_B and $A \leq_p B$ via f , then $V_A(w, c) = V_B(f(w), c)$ computes $f(w)$ in polynomial time and runs V_B , so it is a polynomial-time verifier for A ; hence $A \in \text{NP}$. The arrow points *from* the source A *into* the target B , and tractability/membership flows the opposite way.

The certificate is the design choice. To show a problem is in NP you must answer two questions: *what is the witness*, and *how is it checked in polynomial time*. A good certificate is short (polynomial in $|w|$) and makes verification a matter of routine checking, never re-solving the problem. The worked examples below are organized around exactly this pair of questions.

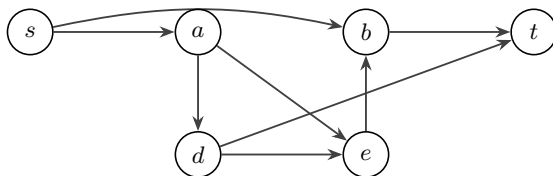
Worked Example 12.3.6 (*HAMPATH* is in NP). Let

$$\text{HAMPATH} = \{ \langle G, s, t \rangle \mid G \text{ is a digraph with a Hamiltonian path from } s \text{ to } t \},$$

where a *Hamiltonian* path visits every node exactly once. Show $\text{HAMPATH} \in \text{NP}$.

Recognition. We do not know a fast way to *find* such a path, but a proposed path is trivial to *check*: walk it and confirm each step is an edge and each node appears once. “Easy to check a candidate” is the trigger for the verifier definition.

Solution. Certificate. A sequence $c = (v_1, v_2, \dots, v_m)$ of the m nodes of G , the proposed order of the visit. *Verifier V on $\langle \langle G, s, t \rangle, c \rangle$:* (i) check $v_1 = s$ and $v_m = t$; (ii) check the v_i are a permutation of all m nodes (no repeats, none missing); (iii) check each consecutive pair (v_i, v_{i+1}) is a directed edge of G ; accept iff all pass. Each check scans the list and the edge set once, so V runs in time polynomial in $|\langle G, s, t \rangle|$, and the certificate has length $O(m)$.



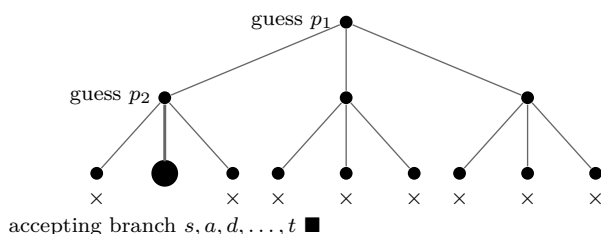
Which idea, and why? A problem joins NP by exhibiting a short witness plus a polynomial check; here the witness is the visiting order and the check is “permutation + every consecutive pair is an edge.”

Check. The certificate $c = (s, a, d, e, b, t)$ uses every node once, and each step $s \rightarrow a \rightarrow d \rightarrow e \rightarrow b \rightarrow t$ is a drawn edge, so V accepts: $\langle G, s, t \rangle \in \text{HAMPATH}$. A bogus certificate like (s, b, t) fails step (ii) because it omits a, d, e .

Worked Example 12.3.7 (The same problem through a nondeterministic machine). Reprove $\text{HAMPATH} \in \text{NP}$ using Theorem 12.3.4, by giving a polynomial-time NTM.

Recognition. Theorem 12.3.4 lets us replace “guess a certificate, then verify” by “branch over the guess, then verify on each branch.” The trigger is simply that we already *have* a certificate-checker; nondeterminism supplies the guess.

Solution. NTM N on $\langle G, s, t \rangle$: (1) nondeterministically write a list $p_1, \dots, p_m$, each p_i chosen in $\{1, \dots, m\}$ – this is the branching; (2) reject the branch if any value repeats; (3) reject unless p_1 names s and p_m names t ; (4) reject unless every (p_i, p_{i+1}) is an edge; otherwise accept. Each branch does $O(m^2)$ work, so N runs in nondeterministic polynomial time.



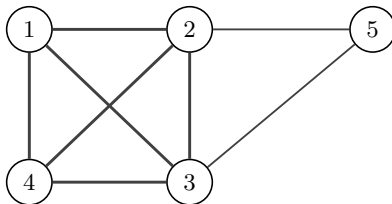
Which idea, and why? By Theorem 12.3.4 a polynomial-time NTM and a polynomial-time verifier are interchangeable; the accepting branch (the bold leaf) *is* the certificate of the previous example.

Check. The bold branch guesses the same order s, a, d, e, b, t that served as the certificate; all other branches fail the repeat/edge tests and die. One accepting branch suffices, matching Definition 8.3.7.

Worked Example 12.3.8 (CLIQUE is in NP). A k -clique in an undirected graph is a set of k vertices that are pairwise adjacent. Let $\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ has a } k\text{-clique}\}$. Show $\text{CLIQUE} \in \text{NP}$.

Recognition. Finding a large clique is the expensive part; checking that a *given* set of vertices is a clique only requires confirming each pair is joined. “Verify a proposed object” again points to a certificate.

Solution. Certificate. A list c of k vertices. *Verifier:* confirm c has exactly k distinct vertices of G , then confirm all $\binom{k}{2}$ pairs are edges of G ; accept iff both hold. There are $O(k^2)$ pairs, each looked up in the edge set, so verification is polynomial in $|\langle G, k \rangle|$.



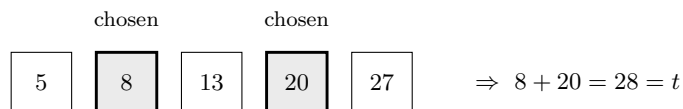
Which idea, and why? Membership in NP follows from a short witness (k vertices) plus a polynomial check (all pairs adjacent); the witness need not be unique or found efficiently.

Check. For $\langle G, 4 \rangle$ the certificate $c = \{1, 2, 3, 4\}$ is a 4-clique: the bold edges show all six pairs among 1, 2, 3, 4 are present, so the verifier accepts. For $\langle G, 3 \rangle$ the bogus certificate $\{1, 2, 5\}$ fails the pair test because 1–5 is not an edge, even though it passes the count test.

Worked Example 12.3.9 (SUBSET-SUM is in NP). Given a multiset $S = \{x_1, \dots, x_k\}$ of positive integers and a target t , let $\text{SUBSET-SUM} = \{\langle S, t \rangle \mid \text{some sub-multiset of } S \text{ sums to } t\}$. Show it is in NP.

Recognition. Choosing which numbers to add looks like brute-force search over 2^k subsets, but adding up a *specified* subset and comparing to t is one pass. The trigger is once more “check a proposed solution.”

Solution. Certificate. A sub-multiset $c \subseteq S$ (say, the chosen indices). *Verifier:* confirm every element of c is drawn from S (respecting multiplicities), add the elements of c , and accept iff the sum equals t . Addition of numbers written in binary is polynomial in the input length, and there are at most k of them, so the check is polynomial.



Which idea, and why? The witness is the subset itself; verification is a single bounded sum, never a search, so the problem sits in **NP** regardless of how hard finding the subset may be.

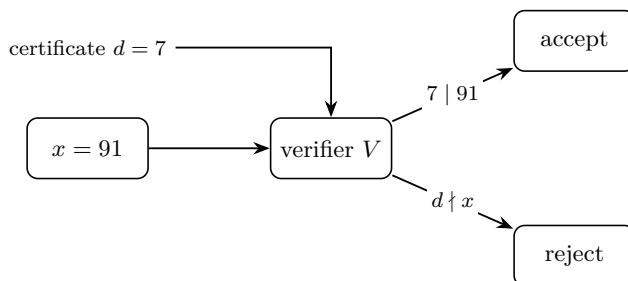
Check. For $S = \{5, 8, 13, 20, 27\}$ and $t = 28$, the certificate $c = \{8, 20\}$ sums to 28, so the verifier accepts. The empty subset (sum 0) and $\{5, 8\}$ (sum 13) are certificates that fail, but their failure does not matter: one accepting certificate puts $\langle S, 28 \rangle$ in the language.

Worked Example 12.3.10 (*COMPOSITES* is in **NP** (and the asymmetry of complements)). Let $COMPOSITES = \{x \mid x = pq \text{ for integers } p, q > 1\}$, with x written in binary. Show $COMPOSITES \in \mathbf{NP}$, and explain why its complement is not obviously in **NP**.

Recognition. A factor is a one-line proof of compositeness, and checking a factor is just a division. The trigger is the existence of a short, checkable witness.

Solution. Certificate. A single integer d with $1 < d < x$. *Verifier:* accept iff $1 < d < x$ and d divides x evenly. Division of binary integers is polynomial in $|x|$ (the number of bits), so this is a polynomial-time verifier and $COMPOSITES \in \mathbf{NP}$.

For the complement $\overline{COMPOSITES}$ (the primes, plus 0, 1), what would a certificate of *primality* be? “Try a divisor” proves compositeness, not primality; to be sure x is prime by exhibiting witnesses one must rule out *all* possible factors, and no *obvious* short certificate does that. This is the general phenomenon: a polynomial verifier checks a one-sided “yes” witness, so $\mathcal{L} \in \mathbf{NP}$ gives no automatic certificate for $\overline{\mathcal{L}}$. The class of complements of **NP** languages is named **coNP**, and whether $\mathbf{coNP} = \mathbf{NP}$ is open.



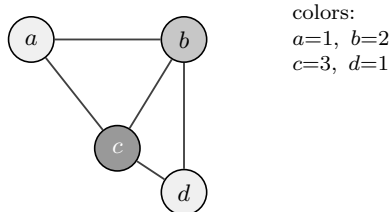
Which idea, and why? Membership uses a one-sided witness: a factor certifies “composite,” but a short witness certifying “prime” is far from obvious (one does exist, by Pratt’s theorem, and primality is even in **P**); in general no symmetric witness need exist, which is exactly why **NP** and **coNP** need not coincide.

Check. For $x = 91$ the certificate $d = 7$ gives $91 = 7 \cdot 13$, so V accepts and $91 \in COMPOSITES$. For $x = 97$ no d with $1 < d < 97$ divides it, so no certificate exists and $97 \notin COMPOSITES$ – correctly, since 97 is prime.

Worked Example 12.3.11 (A language that is verifiable but where the brute force is exponential). Let $3COL = \{\langle G \rangle \mid \text{the vertices of } G \text{ can be 3-colored so adjacent vertices differ}\}$. Show $3COL \in \mathbf{NP}$ and contrast the witness size with the search size.

Recognition. A proposed coloring is checked edge by edge; the trigger is the gap between $3^{|V|}$ possible colorings to search and the single coloring to check.

Solution. Certificate. A function $c: V \rightarrow \{1, 2, 3\}$ assigning a color to each vertex, written as the list of colors. *Verifier:* for each edge $\{u, v\}$ of G confirm $c(u) \neq c(v)$; accept iff every edge passes. There are $|E|$ edges, each a constant-time comparison, so verification is $O(|E|)$, polynomial in $|\langle G \rangle|$, while the certificate has length $O(|V|)$.



Which idea, and why? The defining trade-off of NP in miniature: $3^{|V|}$ colorings to search but a single coloring to verify in $O(|E|)$, so a short witness plus a fast check places 3COL in NP.

Check. The shown assignment gives $a=1, b=2, c=3, d=1$; the five edges ab, ac, bc, bd, cd all join differently-colored endpoints (note a and d share color 1 but are *not* adjacent), so V accepts. Recoloring $d=2$ would break edge bd and that certificate would be rejected.

Method (How to show a language $\mathcal{L}$ is in NP).

1. **Name the certificate.** Decide what object would *convince* a skeptic that $w \in \mathcal{L}$: a path, a subset, a coloring, an assignment, a factor. It must be expressible as a string.
2. **Bound its length.** Confirm $|c| = O(|w|^k)$. If the natural witness is too long to read in polynomial time, it cannot be used; find a shorter one or reconsider.
3. **Give the verifier.** Describe a deterministic algorithm V on $\langle w, c \rangle$ that *checks* the witness – never re-solves the problem – and argue it runs in time polynomial in $|w|$.
4. **Verify the biconditional.** Show $w \in \mathcal{L} \Rightarrow$ some c makes V accept, and $w \notin \mathcal{L} \Rightarrow$ no c makes V accept. Both directions are required.
5. **(Optional) Translate.** By Theorem 12.3.4 you may instead give a polynomial-time NTM that guesses the witness and runs V ; choose whichever is clearer.

Caution. The certificate must be *checkable* in polynomial time, not merely short. Proposing “the certificate is the answer to the question” is circular: if checking the witness secretly requires re-deciding $\mathcal{L}$, you have shown nothing. A valid witness reduces the problem to routine verification (walk the path, add the subset, compare adjacent colors), with no hidden search inside the verifier.

Caution. NP is defined by a *one-sided* guarantee. A polynomial-time verifier certifies the “yes” instances via an accepting certificate; it says nothing about how to certify “no” instances. So $\mathcal{L} \in \text{NP}$ does *not* give $\bar{\mathcal{L}} \in \text{NP}$ for free – that would require a short witness of *non-membership*, which for problems like *HAMPATH* is not known to exist. Do not assume NP is closed under complement; the question “NP = coNP?” is open.

Caution. Measuring the verifier’s time in $|w|$ rather than in $|\langle w, c \rangle|$ is essential. If we allowed the certificate to be arbitrarily long, a “verifier” could smuggle an exponential brute-force answer into c and run in time polynomial in $|c|$, trivializing the class. Always bound certificate length by a polynomial in $|w|$ and charge the verifier’s clock against $|w|$ alone.

Exercises.

1. Let $\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ is an undirected graph containing a triangle}\}$, where a triangle is three pairwise-adjacent vertices. Describe a certificate and a polynomial-time verifier proving $\text{TRIANGLE} \in \text{NP}$. (Then convince yourself *TRIANGLE* is actually in P by sketching a brute-force-over-triples algorithm and bounding its running time.)
2. Let $\text{VERTEX-COVER} = \{\langle G, k \rangle \mid G \text{ has a set of } \leq k \text{ vertices touching every edge}\}$. Give a certificate, describe the verifier, and bound the certificate length and the verification time as polynomials in $|\langle G, k \rangle|$.
3. Prove that NP is closed under union: if $\mathcal{L}_1, \mathcal{L}_2 \in \text{NP}$ then $\mathcal{L}_1 \cup \mathcal{L}_2 \in \text{NP}$. (Describe how a certificate for $w \in \mathcal{L}_1 \cup \mathcal{L}_2$ encodes which language w is being certified for, then run the appropriate verifier.)
4. Prove that NP is closed under intersection. What must a single certificate for $w \in \mathcal{L}_1 \cap \mathcal{L}_2$ contain, and how does the verifier use it?

5. Consider the language $HALF-CLIQUE = \{\langle G \rangle \mid G \text{ on } m \text{ vertices has a clique of size } \geq m/2\}$. Give a verifier showing it is in NP. State the certificate and bound its length.
6. For the verifier-to-NTM direction of Theorem 12.3.4, suppose a verifier V runs in time n^3 on inputs of length $n = |w|$. Exactly how long may the certificate be, and over how many positions does the constructed NTM branch in stage 1? Explain why each branch still halts in polynomial time.
7. Draw the nondeterministic computation tree (down to depth 3) of a polynomial-time NTM for $SUBSET-SUM$ on the instance $S = \{2, 4, 7\}$, $t = 6$, where at step i the machine branches on whether to include x_i . Mark every accepting leaf.
8. *True, false, or open, with justification.* (a) If $\mathcal{L} \in P$ then $\mathcal{L} \in NP$. (b) If $\mathcal{L} \in NP$ then $\bar{\mathcal{L}} \in NP$. (c) Every language in NP is decidable. (d) $NP \subseteq EXPTIME$.
9. A student claims the following is a verifier proving $\overline{HAMPATH} \in NP$: “the certificate c lists, for the graph G , all $m!$ orderings of the vertices, and the verifier checks that none of them is a Hamiltonian path from s to t .” Identify the fatal flaw in this argument, pointing precisely to which clause of Definition 12.3.1 it violates.
10. *Find the bug.* Someone offers this “proof” that $CLIQUE \in P$: “On input $\langle G, k \rangle$, nondeterministically guess k vertices and check they are pairwise adjacent; this runs in polynomial time, so $CLIQUE \in P$.” Explain exactly which word makes the conclusion P wrong, and state the correct class the argument actually establishes.
11. Let

$$DOUBLE-SAT = \{\langle \phi \rangle \mid \text{the Boolean formula } \phi \text{ has at least two distinct satisfying assignments}\}.$$

Describe a certificate and a polynomial-time verifier showing $DOUBLE-SAT \in NP$. What goes wrong if you try to use the single string “yes” as the certificate?

12. Show that if $\mathcal{L} \in NP$ then there is a polynomial p and a language $\mathcal{L}' \in P$ such that $w \in \mathcal{L}$ iff $\exists c[|c| \leq p(|w|) \text{ and } \langle w, c \rangle \in \mathcal{L}']$. (This is the “ $\exists$ over a polynomially bounded certificate, then a polynomial check” normal form for NP; read it straight off Definition 12.3.1.)

12.4 NP Problems and the P vs NP Question

The class NP was defined in the previous section by an abstract property – a short certificate that a polynomial-time verifier can check. This section fills that frame with the concrete problems that make NP matter: the satisfiability of Boolean formulas, and a family of graph and arithmetic problems (cliques, Hamiltonian paths, subset sums, vertex covers). The one idea running through all of them is a sharp asymmetry: *finding* a solution seems to demand searching an exponentially large space, while *checking* a proposed solution is trivially fast. Whether that asymmetry is real or only apparent is the P versus NP question, the central open problem of the field, which we frame precisely at the end.

Definition 12.4.1 (Boolean formula, satisfiability, SAT). A *Boolean variable* ranges over $\{0, 1\}$ (read as FALSE/TRUE). A *Boolean formula* is built from variables with the operations $\wedge$ (and), $\vee$ (or), and $\neg$ (not, also written with an overline). An *assignment* fixes a value in $\{0, 1\}$ for each variable; the formula is *satisfiable* if some assignment makes it evaluate to 1. The satisfiability problem is the language

$$SAT = \{\langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula}\}.$$

A *literal* is a variable x or its negation $\bar{x}$. A *clause* is an $\vee$ of literals, and a formula is in *conjunctive normal form* (a *cnf-formula*) if it is an $\wedge$ of clauses; it is a *3cnf-formula* if every clause has exactly three literals. The restriction of SAT to 3cnf-formulas is the language 3SAT. These restricted forms look weaker but are not: 3SAT is already as hard as the general problem, a fact we exploit in later sections.

Definition 12.4.2 (The graph and arithmetic problems). For an undirected graph G , a k -*clique* is a set of k vertices that are pairwise adjacent, and a *vertex cover of size k* is a set of k vertices that touches every edge

(each edge has at least one endpoint in the set). For a directed graph G with vertices s, t , a *Hamiltonian path from s to t* is a directed path that starts at s , ends at t , and visits every vertex exactly once. We define

$$\begin{aligned} \text{CLIQUE} &= \{\langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}, \\ \text{VERTEX-COVER} &= \{\langle G, k \rangle \mid G \text{ is an undirected graph with a vertex cover of size } k\}, \\ \text{HAMPATH} &= \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a Hamiltonian } s\text{-}t \text{ path}\}, \\ \text{SUBSET-SUM} &= \{\langle S, t \rangle \mid S \text{ is a multiset of integers with a subset summing to } t\}. \end{aligned}$$

The unifying theorem is that all five problems sit in **NP**. In each case the certificate is the “solution object” itself – a satisfying assignment, the k clique vertices, the cover vertices, the path as an ordering, the chosen subset – and the verifier merely confirms that the object has the claimed property.

Theorem 12.4.3 (Membership in **NP**). *SAT*, *3SAT*, *CLIQUE*, *VERTEX-COVER*, *HAMPATH*, and *SUBSET-SUM* all belong to **NP**.

Proof. We exhibit a polynomial-time verifier for each; by Definition 12.3.2 of the previous section a language with such a verifier is in **NP**. Throughout, n denotes the length of the encoded input.

SAT (and *3SAT*). The certificate is an assignment a to the variables of φ , a string of at most n bits. The verifier substitutes a into φ and evaluates the formula by a single bottom-up pass over its parse structure; this takes time linear in the size of φ , hence polynomial in n . It accepts iff φ evaluates to 1.

CLIQUE. The certificate is a list c of k vertices. The verifier checks that the k vertices are distinct and that all $\binom{k}{2}$ pairs are edges of G , by scanning the edge list; this is $O(n^2)$ work. It accepts iff every pair is present.

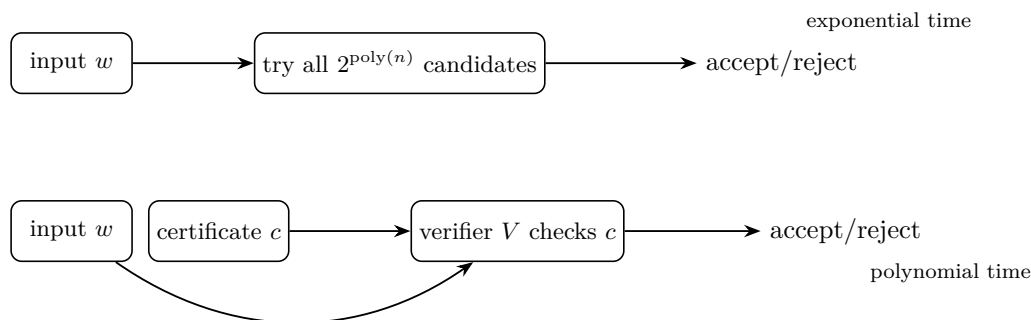
VERTEX-COVER. The certificate is a list c of k vertices. The verifier checks that c consists of k distinct vertices of G and then scans every edge of G , confirming that each edge has at least one endpoint in c ; this is again polynomial. It accepts iff no edge is missed.

HAMPATH. The certificate is a sequence $p_1, \dots, p_m$ of the m vertices. The verifier checks that the p_i are a permutation of all vertices (no repeats, all present), that $p_1 = s$ and $p_m = t$, and that each consecutive pair (p_i, p_{i+1}) is a directed edge. Each check is a polynomial scan.

SUBSET-SUM. The certificate is the chosen subcollection c . The verifier checks that c is drawn from S (respecting multiplicities) and that the numbers in c add up to exactly t . Addition of polynomially many integers, each of polynomial bit-length, is polynomial.

In every case the certificate has length polynomial in n and the check runs in polynomial time, so each language is in **NP**. $\square$

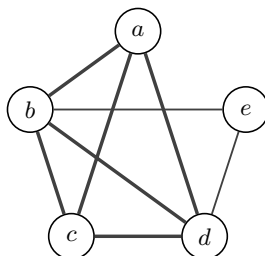
The certificate-versus-brute-force gap is the whole story. For each problem there is an obvious exponential-time decider that tries all candidate solutions – all 2^v assignments, all $\binom{m}{k}$ vertex subsets, all $m!$ vertex orderings, all $2^{|S|}$ subcollections – and a polynomial-time *verifier* that, handed one good candidate, confirms it fast. The schematic below contrasts the two; the verifier is the same machine, but it receives the certificate instead of discovering it.



Worked Example 12.4.4 (Verifying a 4-clique). For the undirected graph G drawn below, decide whether $\langle G, 4 \rangle \in \text{CLIQUE}$ and exhibit a certificate if it is.

Recognition. This asks for membership in *CLIQUE*, where the certificate is a set of $k = 4$ pairwise-adjacent vertices. The trigger is the word “clique”: we hunt for four mutually connected vertices, then *verify* by checking all six pairs – exactly what the verifier of Theorem 12.4.3 does.

Solution. The vertices $\{a, b, c, d\}$ are pairwise joined (all six edges ab, ac, ad, bc, bd, cd are present, drawn heavy below), so they form a 4-clique. Vertex e misses a and c , so it cannot extend the clique, but it is not needed.



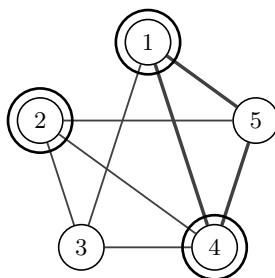
Which idea, and why? Membership in *CLIQUE* is certified by naming the clique itself; correctness is settled by checking all $\binom{4}{2} = 6$ pairs, which is the verifier’s polynomial-time test, not a search.

Check. The certificate $c = \{a, b, c, d\}$ has 4 vertices and all 6 pairs are edges, so the verifier accepts $\langle\langle G, 4 \rangle, c\rangle$. There is no 5-clique, since e is adjacent to only b, d .

Worked Example 12.4.5 (Clique and vertex cover on one graph). Take G on vertices $\{1, 2, 3, 4, 5\}$ with all edges *except* $\{1, 2\}$ and $\{3, 5\}$. Decide whether $\langle G, 3 \rangle \in \text{CLIQUE}$ and whether $\langle G, 3 \rangle \in \text{VERTEX-COVER}$.

Recognition. Two problems from Definition 12.4.2 on one graph: a clique is a fully-connected vertex set; a vertex cover is a set of vertices touching every edge. The trigger words “clique” and “cover” tell us to produce one certificate of each kind and verify it by a scan.

Solution. The eight edges of G are 13, 14, 15, 23, 24, 25, 34, 45 (the two non-edges $\{1, 2\}$ and $\{3, 5\}$ are omitted). For a 3-clique, the triple $\{1, 4, 5\}$ works: 14, 15, 45 are all edges (heavy below). For a vertex cover of size 3, take $\{1, 2, 4\}$: vertex 1 covers 13, 14, 15, vertex 2 covers 23, 24, 25, and vertex 4 covers the remaining 34, 45 (cover vertices ringed).



The triple $\{3, 4, 5\}$ is *not* a clique, because $\{3, 5\}$ is a non-edge; that is why we chose $\{1, 4, 5\}$ instead.

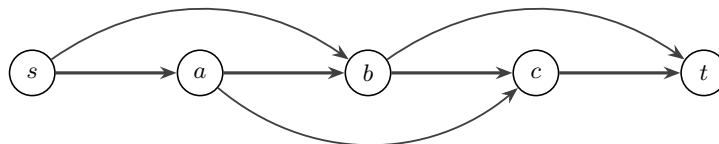
Which idea, and why? Clique looks for a fully-connected subset; vertex cover looks for a hitting set for the edges. Each certificate is checked by re-scanning the edge list, and that polynomial re-scan – not a search – is all NP membership requires.

Check. Clique $\{1, 4, 5\}$: edges 14, 15, 45 all present, so the verifier accepts. Cover $\{1, 2, 4\}$: each of the eight edges has an endpoint in $\{1, 2, 4\}$ (the only pair outside the cover is $\{3, 5\}$, and that pair is a non-edge), so the verifier accepts. Both $\langle G, 3 \rangle \in \text{CLIQUE}$ and $\langle G, 3 \rangle \in \text{VERTEX-COVER}$.

Worked Example 12.4.6 (A Hamiltonian path in a directed graph). For the directed graph below with source s and sink t , decide whether $\langle G, s, t \rangle \in \text{HAMPATH}$.

Recognition. Membership in *HAMPATH*: the certificate is an ordering of *all* vertices forming a directed s -to- t path. The trigger “visits every vertex exactly once” tells us to look for a permutation, then verify consecutive edges – never to enumerate all $m!$ orderings.

Solution. The graph has vertices s, a, b, c, t . Tracing forward edges, the ordering $s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$ uses each vertex once and every step is a directed edge.



The thin shortcut edges $s \rightarrow b$, $a \rightarrow c$, $b \rightarrow t$ tempt a path that skips a vertex; such a path is *not* Hamiltonian. Only the heavy spine $s a b c t$ touches all five.

Which idea, and why? *HAMPATH* is certified by the vertex ordering; the verifier checks it is a permutation and that each consecutive pair is an edge – a polynomial scan, not the $m!$ brute-force search.

Check. Order (s, a, b, c, t) : distinct, all five vertices, s first and t last, and edges sa, ab, bc, ct all present. The verifier accepts. The shortcut $s \rightarrow b$ would orphan a , so it cannot begin a Hamiltonian path.

Worked Example 12.4.7 (A subset-sum certificate). Decide whether $\langle S, t \rangle \in \text{SUBSET-SUM}$ for $S = \{3, 7, 8, 12, 20\}$ and target $t = 27$.

Recognition. An arithmetic membership test: the certificate is a subcollection of S adding to t . The trigger “a subset summing to t ” tells us to find such a subset and then verify by addition, never to enumerate all 2^5 subsets.

Solution. Searching for addends of 27 from $\{3, 7, 8, 12, 20\}$: $7 + 8 + 12 = 27$, and also $20 + 7 = 27$. Either is a valid certificate. We display the chosen subset against the full multiset.

3	7	8	12	20
---	---	---	----	----

$\downarrow \quad \downarrow \quad \downarrow$

$$7 + 8 + 12 = 27 = t$$

Which idea, and why? *SUBSET-SUM* membership is certified by the summing subset; the verifier adds the chosen numbers and compares to t , which is fast arithmetic, not a search over $2^{|S|}$ subsets.

Check. The certificate $c = \{7, 8, 12\}$ is drawn from S and $7 + 8 + 12 = 27 = t$, so the verifier accepts. The alternative $c' = \{7, 20\}$ also gives 27; both are legitimate proofs.

Worked Example 12.4.8 (Satisfying a 3cnf-formula). Decide whether $\langle \varphi \rangle \in \text{3SAT}$ for $\varphi = (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3)$.

Recognition. Membership in *3SAT*: the certificate is a truth assignment satisfying every clause. The trigger is the cnf shape – an $\wedge$ of clauses – so we need each clause to contain at least one true literal. Verification is a single evaluation pass.

Solution. Try $x_1 = 1$. Then clause 1 is satisfied (by x_1) and clause 3 is satisfied (by x_1). For clause $2 = (\overline{x_1} \vee x_2 \vee x_3)$, since $x_1 = 1$ kills $\overline{x_1}$, set $x_2 = 1$, which satisfies it. Leave $x_3 = 0$. So $a = (x_1, x_2, x_3) = (1, 1, 0)$ satisfies φ .

clause	$x_1 = 1$	$x_2 = 1$	$x_3 = 0$	true literal
$x_1 \vee x_2 \vee \overline{x_3}$	1	1	$\overline{0} = 1$	x_1
$\overline{x_1} \vee x_2 \vee x_3$	$\overline{1} = 0$	1	0	x_2
$x_1 \vee \overline{x_2} \vee x_3$	1	$\overline{1} = 0$	0	x_1

Which idea, and why? *3SAT* membership is certified by an assignment; the verifier substitutes and evaluates the formula in one pass, accepting iff every clause has a true literal – polynomial, not a sweep over all 2^3 assignments.

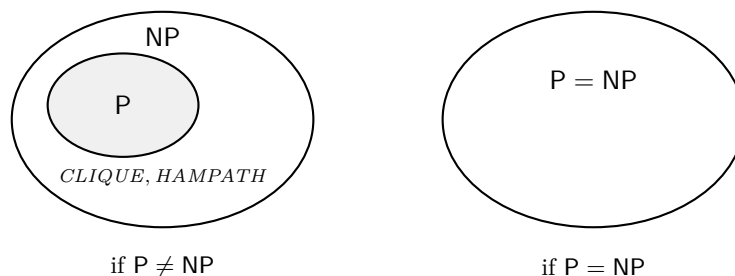
Check. Under $(1, 1, 0)$ each row above shows a literal evaluating to 1, so all three clauses are true and φ is satisfied. The verifier accepts $\langle \langle \varphi \rangle, a \rangle$.

We can now state the question that organizes the rest of the chapter. By definition every polynomial-time decider is, trivially, a verifier that ignores its certificate, so $P \subseteq NP$. The reverse containment is unknown.

Theorem 12.4.9 ($P \subseteq NP$). *Every language in P is in NP .*

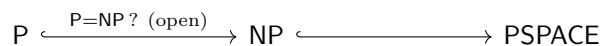
Proof. Let $\mathcal{L} \in P$, decided by a polynomial-time machine M . Define a verifier V that on input $\langle w, c \rangle$ ignores c and simulates M on w , accepting iff M accepts. Then $w \in \mathcal{L}$ iff V accepts $\langle w, \varepsilon \rangle$ (the empty certificate suffices), and V runs in polynomial time in $|w|$. Hence $\mathcal{L} \in NP$. $\square$

Definition 12.4.10 (The P versus NP question). The question is whether $P = NP$: whether every language whose membership can be *verified* in polynomial time can also be *decided* in polynomial time. It is open; the two logically possible pictures are drawn below.



In the left picture, problems such as *CLIQUE* and *HAMPATH* live in the unshaded gap $NP \setminus P$: verifiable but not decidable in polynomial time. In the right picture there is no gap, and every fast verifier could be replaced by a fast decider. No one knows which holds, though the best deterministic algorithms known for such problems run in exponential time, and the containment $NP \subseteq EXPTIME$ is proved while no smaller deterministic time bound is known.

It helps to place NP in the chain of containments that *are* proved, while marking exactly where the open question lives.



Takeaway. The known inclusions are $P \subseteq NP \subseteq PSPACE$, all proved; whether $P = NP$ is the central *open* question, and no strict inclusion among these three classes is currently known (it is unknown whether $NP \subseteq PSPACE$ is strict, just as it is unknown whether $P \subseteq NP$ is). The hook arrows record the proved containments; the label flags the one edge whose strictness is the famous open problem. *Check.* $P \subseteq NP$ is Theorem 12.4.9; $NP \subseteq PSPACE$ holds because a polynomial-time verifier can be run over all polynomially-long certificates in polynomial space. No arrow in the chain is asserted to be strict.

Method (How to show a problem is in NP via a certificate).

1. **Name the solution object.** State what a “yes” instance *has* – a satisfying assignment, a clique, an ordering, a subset. That object is your certificate c .
2. **Bound its size.** Argue $|c|$ is polynomial in the input length n (e.g. at most the number of variables, vertices, or numbers). If a candidate can be exponentially long, this approach fails and the problem may not obviously be in NP .
3. **Describe the check.** Give an algorithm V that, on $\langle w, c \rangle$, confirms c is a genuine solution for w : a scan, an evaluation, a sum. Verify each step is polynomial in n .
4. **State the equivalence.** Confirm $w \in \mathcal{L}$ iff some valid c exists, so V accepts $\langle w, c \rangle$ for that c . Then $\mathcal{L} \in NP$.

Caution. Do not confuse *verifying* a solution with *finding* one. Putting a problem in NP requires only a fast checker for a *given* certificate; it says nothing about how hard the certificate is to discover. Writing “the verifier searches all subsets for one summing to t ” describes an exponential-time machine, not a polynomial-time verifier. The verifier is *handed* the subset and only adds it up.

Caution. The certificate must have length polynomial in the input. A would-be certificate that is itself exponentially long cannot be read, let alone checked, within a polynomial time bound. This is why the complement problems – “ G has *no* k -clique,” “ G has *no* Hamiltonian path” – are not obviously in **NP**: there is no evident short proof of *non*-existence. Such complements populate the class **coNP**, and whether **coNP** = **NP** is itself open.

Caution. $P \subseteq NP$ is the *easy* containment and is fully proved (Theorem 12.4.9); the open question is the reverse. Saying “*CLIQUE* is in **NP**, so it is probably not in **P**” is a non-sequitur: membership in **NP** alone is no evidence of hardness, since $P \subseteq NP$ means easy problems are in **NP** too. The evidence of hardness comes later, from **NP**-completeness, not from mere **NP** membership.

Exercises.

1. For the formula $\varphi = (\overline{x_1} \vee x_2) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3})$, find a satisfying assignment and verify it clause by clause, or argue no assignment works. State explicitly the certificate the *SAT* verifier would receive.
2. Draw an undirected graph on six vertices that contains a 3-clique but no 4-clique. Mark a 3-clique with heavy edges, and explain in one sentence why your graph has no 4-clique.
3. Let G be the undirected 5-cycle 1 2 3 4 5 1. Find the smallest vertex cover of G and give a certificate for membership of $\langle G, k \rangle$ in *VERTEX-COVER* for that k . Why is no cover of size 2 possible?
4. Give a polynomial-time verifier for the language

$$HALF-CLIQUE = \{\langle G \rangle \mid G \text{ has a clique on at least half its vertices}\}.$$

State the certificate, bound its length, and describe the check; conclude that *HALF-CLIQUE* $\in$ **NP**.

5. Construct a directed graph on five vertices s, a, b, c, t that has a Hamiltonian path from s to t but is *not* fully connected (omit at least three of the possible directed edges). Exhibit the Hamiltonian path as an ordering and verify each consecutive edge.
6. For $S = \{2, 5, 6, 9, 14\}$ and target $t = 22$, decide whether $\langle S, t \rangle \in SUBSET-SUM$. If yes, give a certificate and verify the sum; if no, argue briefly that no subset works.
7. Define a polynomial-time verifier for

$$DOUBLE-SAT = \{\langle \varphi \rangle \mid \varphi \text{ has at least two satisfying assignments}\}.$$

What is the certificate here, and why is its length polynomial? Conclude *DOUBLE-SAT* $\in$ **NP**.

8. Prove that **P** is closed under union: if $A, B \in P$ then $A \cup B \in P$. Then explain in one sentence why the same kind of argument, run on verifiers, shows **NP** is closed under union.
9. *Find the bug.* A student claims to prove *SUBSET-SUM* $\in$ **NP** as follows: “The verifier, on input $\langle S, t \rangle$, lists all subsets of S , and accepts iff one of them sums to t .” Identify precisely why this fails to be a polynomial-time verifier, and rewrite the verifier so the argument is correct.
10. *Find the bug.* Someone argues: “*HAMPATH* (graphs with *no* Hamiltonian s - t path) is in **NP**, because the verifier can be handed the graph and simply check that no Hamiltonian path exists.” Explain why the proposed certificate does not meet the definition of a verifier, and name the class to which *HAMPATH* naturally belongs.
11. Show that *CLIQUE* $\in$ **NP** by giving a *nondeterministic* polynomial-time Turing machine that decides it, rather than a verifier. Describe what each stage guesses and checks, and argue every branch runs in polynomial time.
12. Suppose someone exhibits a polynomial-time algorithm that, given $\langle G, k \rangle$, decides *CLIQUE*. Using only the relationships proved in this section (no **NP**-completeness), state exactly what you may and may not conclude about **P** and **NP**, and justify each part in one sentence.

12.5 Polynomial-Time Reducibility

The previous sections measured the cost of solving a problem outright. This one measures the cost of *converting* one problem into another. The single idea is that if we can transform every instance of problem A into an instance of problem B using only a polynomial amount of work, in such a way that yes-instances map to yes-instances and no-instances to no-instances, then B is “at least as hard” as A up to a polynomial: any fast algorithm for B becomes a fast algorithm for A . This is the efficient refinement of the mapping reductions that were used to compare decidability in Chapter 10, and it is the exact tool that, in the next section, lets a single problem stand in for the entire class NP.

Polynomial-time computable functions and the relation $\leq_p$

A reduction is built from a function that rewrites strings. To count it as efficient, we demand that the rewriting itself run in polynomial time.

Definition 12.5.1 (Polynomial-time computable function). A function $f: \Sigma^* \rightarrow \Sigma^*$ is *polynomial-time computable* if some polynomial-time Turing machine, started on any input w , halts with exactly $f(w)$ written on its tape (and nothing else). Equivalently, there is a constant c and a TM computing f that on every input of length n halts within $O(n^c)$ steps.

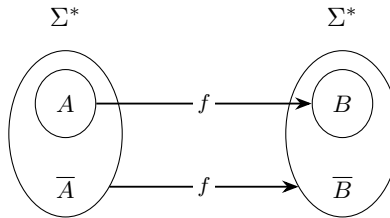
Because the machine runs for at most $O(n^c)$ steps it can write at most that many symbols, so $|f(w)|$ is polynomially bounded in $|w|$: a polynomial-time reduction can never blow the instance up to super-polynomial size. Keep this length bound in mind; it is what makes the composition argument below go through.

Definition 12.5.2 (Polynomial-time reducibility). A language A is *polynomial-time (mapping) reducible* to a language B , written $A \leq_p B$, if there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that for every string w ,

$$w \in A \iff f(w) \in B.$$

Such an f is called a *polynomial-time reduction* of A to B .

The biconditional is the whole content: f must preserve membership in *both* directions, sending every member of A into B and every non-member of A outside B . The function need not be onto, need not be injective, and need not be invertible; it only has to route the yes/no answer correctly. Read $A \leq_p B$ as “ A is no harder than B , to within a polynomial.”



The picture is the entire definition in one glance: f carries A into B and its complement into B 's complement. A point of A may not hit *every* point of B , but it must land somewhere inside B ; that is all the biconditional asks.

Theorem 12.5.3 (Reductions transfer tractability). *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Proof. Let f be a polynomial-time reduction of A to B , computable in time $O(n^a)$, and let M_B be a polynomial-time decider for B running in time $O(m^b)$ on inputs of length m . Define a machine N for A :

N = “On input w : (1) compute $x = f(w)$; (2) run M_B on x and copy its answer.”

Correctness is immediate from the reduction: N accepts w iff M_B accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$. For the running time, let $n = |w|$. Stage (1) costs $O(n^a)$ and, as noted after Definition 12.5.1, produces x of length $|x| \leq cn^a$ for some constant c . Stage (2) then costs $O(|x|^b) = O((cn^a)^b) = O(n^{ab})$. The total $O(n^a) + O(n^{ab})$ is polynomial because a sum and a composition of polynomials is again a polynomial. Hence $A \in P$. $\square$

The contrapositive is how the theorem is most often used: if $A \leq_p B$ and $A \notin P$, then $B \notin P$. A reduction from a known-hard problem certifies hardness of the target.

Proposition 12.5.4 (Transitivity). *If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.*

Proof. Let f reduce A to B in time $O(n^a)$ and g reduce B to C in time $O(m^b)$. Take the composition $h(w) = g(f(w))$. It is a correct reduction:

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C,$$

the first step because f reduces A to B , the second because g reduces B to C . It is polynomial-time computable by the same accounting as in Theorem 12.5.3: computing $f(w)$ takes $O(n^a)$ and yields a string of length $O(n^a)$, on which computing g takes $O((n^a)^b) = O(n^{ab})$. So h runs in polynomial time and $A \leq_p C$. $\square$

Remark. $\leq_p$ is reflexive (f the identity) and transitive (Proposition 12.5.4), so it is a preorder on languages. It is *not* antisymmetric: two different problems can each reduce to the other, and indeed all problems in P other than $\emptyset$ and Σ^* reduce to one another, collapsing into a single “easy” equivalence class. The interesting structure lives above P .

Proposition 12.5.5 (Closure of P and NP downward under $\leq_p$). *If $A \leq_p B$ and $B \in NP$, then $A \in NP$.*

Proof. Replace “decider M_B ” in the proof of Theorem 12.5.3 by a polynomial-time *verifier* V_B for B . Given a certificate u for $f(w) \in B$, the machine “compute $f(w)$, then run V_B on $\langle f(w), u \rangle$ ” is a polynomial-time verifier for A , with the same certificate u . The length of u is polynomial in $|f(w)|$, hence polynomial in $|w|$. So $A \in NP$. $\square$

Worked reductions

Worked Example 12.5.6 (A reduction in the wrong direction proves nothing). You are told $HAMPATH \leq_p B$ for some language B , and separately that $B \in P$. What, if anything, follows about $HAMPATH$? What if instead you only knew $B \leq_p HAMPATH$?

Recognition. This tests the *direction* of Theorem 12.5.3: tractability flows *backward* along $\leq_p$ (from the target B to the source A), while hardness flows *forward*.

Solution. From $HAMPATH \leq_p B$ and $B \in P$, Theorem 12.5.3 gives $HAMPATH \in P$ at once – a genuine and strong conclusion. But from $B \leq_p HAMPATH$ alone (with $B \in P$) *nothing* follows about $HAMPATH$: an easy problem reducing to a hard one is no surprise and carries no information. The arrow must point from the problem you care about *into* the problem you already understand.

$$HAMPATH \xrightarrow{\leq_p} B \in P$$

so $HAMPATH \in P$ 

Which idea, and why? Theorem 12.5.3 reads “ $A \leq_p B$ and B easy $\Rightarrow A$ easy.” The known-easy language must be the *target* B , so a useful reduction sends the unknown problem into the understood one, never the reverse.

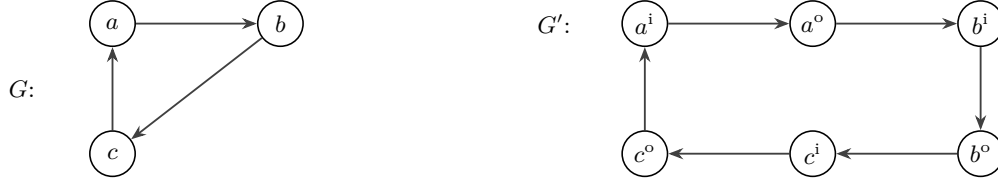
Check. Take the trivial reduction of any P -language into $HAMPATH$ (map yes instances to one fixed Hamiltonian graph, no instances to one fixed non-Hamiltonian graph). It exists regardless of whether $HAMPATH$ is easy or hard, confirming it tells us nothing.

Worked Example 12.5.7 ($\leq_p$ between two graph problems: general cycle to even cycle). Over directed graphs, let $EVENCYC = \{\langle G \rangle \mid G \text{ has a directed cycle of even length}\}$ and $CYC = \{\langle G \rangle \mid G \text{ has a directed cycle}\}$. Show $CYC \leq_p EVENCYC$.

Recognition. We must *rewrite* an instance G into a new instance G' so that “ G has a cycle” becomes exactly “ G' has an even cycle.” The device is to split each vertex so that every single step of G corresponds to two steps of G' , doubling cycle lengths.

Solution. Build G' on two copies of the vertex set, an *in*-layer and an *out*-layer. For each edge $u \rightarrow v$ of G , put an edge $u^{\text{out}} \rightarrow v^{\text{in}}$ in G' ; for each vertex w , put an edge $w^{\text{in}} \rightarrow w^{\text{out}}$. A walk in G' alternates out $\rightarrow$ in (a

real G -edge) and in $\rightarrow$ out (the internal link), so any cycle of G' has even length and projects to a G -cycle traversed once. Conversely a G -cycle of length ℓ lifts to a G' -cycle of length 2ℓ . Thus G' has an even cycle iff it has *any* cycle iff G has *some* cycle, so $f(\langle G \rangle) = \langle G' \rangle$ reduces *CYC* to *EVENCYC*; this bipartite-doubling layout shows the rewriting style.



Which idea, and why? A reduction is a polynomial-time *instance rewriting*, not an algorithm for either problem; here the gadget (split each vertex into in/out, route old edges through it) mechanically translates the structural property “has a cycle of a controlled length” from one graph to another, and the layered construction is plainly polynomial in $|G|$.

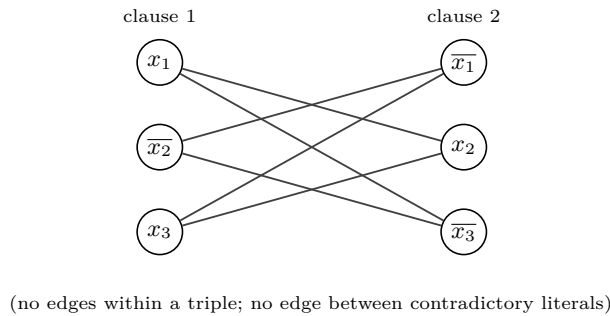
Check. The triangle $a \rightarrow b \rightarrow c \rightarrow a$ in G (length 3) lifts to a 6-edge cycle in G' , even as required, and G' has a cycle exactly when G does. Building G' touches each vertex and edge a constant number of times: linear time, certainly polynomial.

Worked Example 12.5.8 (The headline reduction: $3SAT \leq_p CLIQUE$). Recall $3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$ and $CLIQUE = \{\langle G, k \rangle \mid G \text{ has a } k\text{-clique}\}$. Exhibit a polynomial-time reduction f from $3SAT$ to $CLIQUE$.

Recognition. We need a single rewriting $\langle \phi \rangle \mapsto \langle G, k \rangle$ with “ ϕ satisfiable” $\iff$ “ G has a k -clique.” The trigger is that a satisfying assignment picks *one true literal per clause*, and “one choice per group, pairwise compatible” is exactly what a clique across groups encodes.

Solution. Let $\phi = (a_1 \vee b_1 \vee c_1) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$ have k clauses. Build G with $3k$ nodes, grouped into k *triples*, one triple per clause, each node labelled by its literal. Put an edge between two nodes *unless* (i) they lie in the same triple, or (ii) they carry contradictory labels x and $\bar{x}$. Set the target k equal to the number of clauses, and output $f(\langle \phi \rangle) = \langle G, k \rangle$.

If ϕ is satisfiable: fix a satisfying assignment and, in each clause, pick one node whose literal is true. These k nodes lie in distinct triples (so (i) never blocks them) and no two are contradictory (both being true forbids x and $\bar{x}$, so (ii) never blocks them); hence they are pairwise adjacent – a k -clique. *If G has a k -clique:* its k nodes must lie in distinct triples (within-triple pairs are non-adjacent), one per clause. Set each chosen literal true; this is consistent because no two chosen literals contradict (contradictory pairs are non-adjacent, so cannot both be in the clique). Every clause then has a true literal, so ϕ is satisfied.



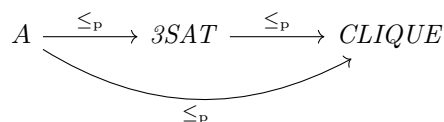
Which idea, and why? $3SAT \leq_p CLIQUE$ uses the *gadget* method: triples mimic clauses (forcing one choice each by leaving them edge-free internally) and the contradiction-edges mimic logical consistency, so a global clique *is* a satisfying assignment. The construction is polynomial – $3k$ nodes and at most $\binom{3k}{2}$ edges, decided by a constant-time label check per pair.

Check. The two-clause sketch shows a 2-clique, e.g. x_1 (clause 1) with x_2 (clause 2): they are in different triples and not contradictory, so the edge is present, and the assignment $x_1 = x_2 = \text{true}$ satisfies both clauses. Picking the contradictory x_1 and $\bar{x}_1$ is impossible – no edge joins them – mirroring that a variable cannot be both true and false.

Worked Example 12.5.9 (Composing reductions to import hardness). Suppose later we prove $3SAT$ is NP-hard (every NP problem reduces to it). Using only the reduction of the previous example, argue $CLIQUE$ is NP-hard as well.

Recognition. This is Proposition 12.5.4 (transitivity) in action: chain “everything $\leq_p 3SAT$ ” with “ $3SAT \leq_p CLIQUE$.”

Solution. Let A be any language in NP. By assumption $A \leq_p 3SAT$, and by the previous example $3SAT \leq_p CLIQUE$. Transitivity gives $A \leq_p CLIQUE$. Since A was an arbitrary NP language, *every* NP problem reduces to $CLIQUE$, which is the definition of NP-hardness.



Which idea, and why? Because $\leq_p$ composes (Proposition 12.5.4), the two arrows in the diagram concatenate into the curved one; hardness, once established for $3SAT$, propagates along every outgoing reduction.

Check. The same chain run with $B \in P$ instead of an arbitrary A gives nothing new, since $P \subseteq NP$ already; the force of the argument is that the source A ranges over *all* of NP.

The road to completeness

Transitivity is exactly the lever that makes one hard problem enough. If B is so hard that every NP problem reduces to it, and if $B \leq_p C$ for some $C \in NP$, then C inherits that universal hardness. This motivates the central definition of the next section.

Definition 12.5.10 (NP-hard, NP-complete). A language B is **NP-hard** if $A \leq_p B$ for every $A \in NP$. It is **NP-complete** if, in addition, $B \in NP$.

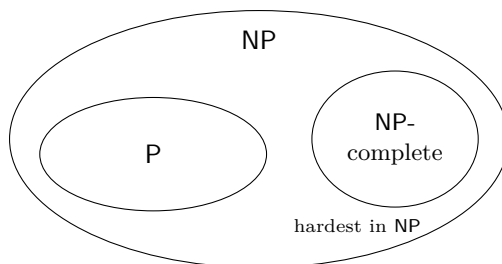
Theorem 12.5.11 (Why one complete problem decides everything). *If B is NP-complete and $B \in P$, then $P = NP$.*

Proof. $P \subseteq NP$ always. For the reverse, take any $A \in NP$. Since B is NP-hard, $A \leq_p B$; since $B \in P$, Theorem 12.5.3 yields $A \in P$. Hence $NP \subseteq P$, so the two classes coincide. $\square$

Corollary 12.5.12 (Spreading completeness by a single reduction). *If B is NP-complete, $C \in NP$, and $B \leq_p C$, then C is NP-complete.*

Proof. $C \in NP$ by hypothesis. For hardness, let $A \in NP$ be arbitrary. As B is NP-hard, $A \leq_p B$; with $B \leq_p C$ and transitivity (Proposition 12.5.4), $A \leq_p C$. So every NP problem reduces to C , making C NP-complete. $\square$

So the program is clear: establish *one* NP-complete problem from scratch (this is the Cook–Levin theorem, which shows SAT is complete), and thereafter manufacture new complete problems cheaply – each by a single polynomial-time reduction from a problem already known complete, exactly as Corollary 12.5.12 licenses. The map below shows the logical landscape these reductions carve out.



The diagram is drawn for the world most researchers expect, $P \neq NP$: then P and the NP -complete problems are *disjoint* regions of NP , and no complete problem has a polynomial algorithm. Should even one complete problem turn out to be in P , Theorem 12.5.11 would slam the whole picture flat – P would swallow NP and the two ovals would merge.

Method (How to build a polynomial-time reduction $A \leq_p B$).

1. **Fix the direction.** Decide which way the arrow points. To prove B *hard*, reduce a known-hard A *into* B ($A \leq_p B$). To prove A *easy* from B 's easiness, again reduce A *into* B . The language being proved hard, or the one known to be easy, is always on the right.
2. **Design the map f .** Describe, for a generic instance w of A , the instance $f(w)$ of B you produce. Reductions between structured problems usually assemble $f(w)$ from small *gadgets* – one per variable, clause, vertex, or constraint of w .
3. **Prove the biconditional.** Show both $w \in A \Rightarrow f(w) \in B$ and $f(w) \in B \Rightarrow w \in A$. The forward direction turns a solution of w into a solution of $f(w)$; the backward direction reads a solution of $f(w)$ back as a solution of w .
4. **Bound the running time.** Argue f is computable in polynomial time – count the gadgets and check each is built in polynomial time, so $|f(w)|$ stays polynomial in $|w|$.
5. **Conclude.** Invoke Theorem 12.5.3 (tractability transfer) or Definition 12.5.10 (completeness), whichever the goal demanded.

Caution. The single most common error is reducing in the *wrong direction*. To show a problem B is hard you must reduce a hard problem *to* B , i.e. produce $A \leq_p B$ with A already known hard. Writing $B \leq_p A$ instead proves only that B is no harder than something hard – a vacuous statement. Whenever you set up a hardness reduction, name out loud which language is the known-hard one and confirm the arrow starts there.

Caution. A reduction must satisfy the biconditional in *both* directions. It is tempting to verify only that yes-instances of A map to yes-instances of B and forget the converse. But if some no-instance of A also maps into B , the map is not a reduction – a fast decider for B would then accept strings outside A . Always check the backward implication $f(w) \in B \Rightarrow w \in A$ explicitly; in gadget reductions this is the step that reads a B -solution back as an A -solution.

Caution. “Polynomial-time” constrains the *reduction*, not a search. The function f may not, even implicitly, solve a hard A or B along the way – e.g. it cannot test satisfiability of ϕ while building the graph. It only rewrites the instance; all the genuine computation is deferred to the decider for B . If your description of f secretly requires deciding a hard problem, it is not a polynomial-time reduction.

Exercises.

1. **Direction check.** You learn that $TSP \leq_p B$ and that $B \in P$, where TSP is the (decision version of the) travelling-salesman problem. State exactly what this lets you conclude about TSP , and what (if anything) it lets you conclude about B from the additional fact that TSP is believed not to be in P .
2. **A reduction by trivial rewriting.** Let

$$A = \{1^n \mid n \text{ is even}\}, \quad B = \{w \in \{0, 1\}^* \mid w \text{ has an even number of 0s}\}.$$

Give an explicit polynomial-time computable f with $w \in A \iff f(w) \in B$, and verify the biconditional in both directions. (Hint: rewrite each 1 as a 0, and send any input containing a 0 to the string 0.)

3. **Length bounds.** Definition 12.5.1 forces $|f(w)|$ to be polynomially bounded in $|w|$. Explain why, and exhibit a function g that is computable but is *not* polynomial-time computable solely because its output is too long, namely $g(1^n) = 1^{2^n}$.
4. **Transitivity, used.** Suppose $A \leq_p B$ via a reduction running in time $O(n^2)$ and $B \leq_p C$ via one running in time $O(m^3)$. Write down the composed reduction $A \leq_p C$ and give an explicit polynomial upper bound on its running time as a function of $|w|$.

5. **Closure of NP downward.** Prove directly from the verifier definition that if $A \leq_p B$ and $B \in \text{NP}$, then $A \in \text{NP}$. Be explicit about what certificate your verifier for A uses and why its length stays polynomial.
6. **Reduction is not symmetric.** Give two languages A, B with $A \leq_p B$ for which it is *not* known whether $B \leq_p A$, and explain in one sentence why exhibiting $B \leq_p A$ would be a major result. (Pick $A \in \text{P}$ and B a problem believed to be outside P .)
7. **Self-reduction within P.** Prove that if $A, B \in \text{P}$ and $B \neq \emptyset$ and $B \neq \Sigma^*$, then $A \leq_p B$. (Hint: decide A inside the reduction and map to a fixed yes- or no-instance of B .) Why does this argument collapse all nontrivial P languages into one $\leq_p$ -equivalence class, and why does it fail when $B \notin \text{P}$?
8. **A graph gadget reduction.** Define $\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ contains a triangle}\}$ and CLIQUE as in the text. Show $\text{TRIANGLE} \leq_p \text{CLIQUE}$ by a one-line map, and draw a small graph G together with the instance $\langle G, k \rangle$ your reduction produces.
9. *Find the bug.* A student claims to prove CLIQUE is NP-hard by giving a polynomial-time reduction $\text{CLIQUE} \leq_p 3\text{SAT}$ (CLIQUE instances rewritten as formulas). Identify the flaw in the student's plan, and state precisely which reduction would actually establish NP-hardness of CLIQUE .
10. **3SAT to CLIQUE, by hand.** Apply the construction of the text to the formula $\phi = (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3)$. Draw the graph G (two triples of three labelled nodes), draw exactly the edges the rule prescribes, state the target k , and exhibit a k -clique together with the satisfying assignment it encodes.
11. **Spreading completeness.** Assume 3SAT is NP-complete. You are given a problem $D \in \text{NP}$ and a polynomial-time reduction $3\text{SAT} \leq_p D$. Prove D is NP-complete, citing the exact result from the section you use at each step.
12. **Contrapositive of the collapse.** Using Theorem 12.5.11, prove that if $\text{P} \neq \text{NP}$ then no NP-complete language lies in P . Then explain why this makes finding a polynomial algorithm for *any* single NP-complete problem (such as SAT or CLIQUE) equivalent to proving $\text{P} = \text{NP}$.

Chapter 13

NP-Completeness

13.1 NP-Completeness: Definition and Significance

The previous chapter settled what P and NP are and left the great open question $P \stackrel{?}{=} NP$ hanging. This section builds the lever that turns that one question into thousands of equivalent ones. The single key idea is *polynomial-time mapping reducibility*: a fast translator f that carries every yes-instance of problem A to a yes-instance of problem B and every no-instance to a no-instance, so that an efficient decider for B becomes an efficient decider for A . Stacking such translators, we will isolate the *hardest* problems in NP – the NP-complete ones – and prove that a single fast algorithm for any one of them would collapse P onto all of NP.

Polynomial-time mapping reducibility

Definition 13.1.1 (Polynomial-time computable function). A function $f: \Sigma^* \rightarrow \Sigma^*$ is *polynomial-time computable* if some deterministic Turing machine M runs in time $n^{O(1)}$ and, started on any input w , halts with exactly $f(w)$ written on its tape and nothing else.

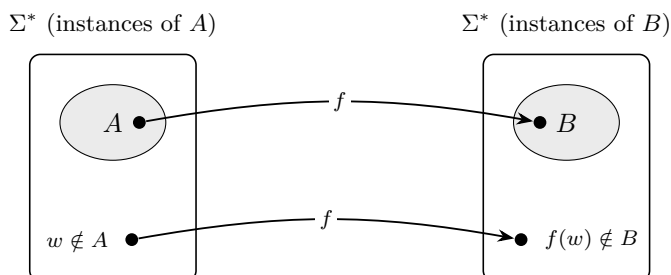
Because M halts within a polynomial number of steps, the output $f(w)$ has only polynomial length: a machine cannot write more cells than it takes steps. This bound is used constantly and is worth stating once.

Definition 13.1.2 (Polynomial-time mapping reducibility). Language A is *polynomial-time mapping reducible* to language B , written $A \leq_p B$, if there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that for every string w ,

$$w \in A \iff f(w) \in B.$$

The function f is the *reduction* of A to B . We call f *correct* when this biconditional holds and *efficient* when f is polynomial-time computable.

Read $A \leq_p B$ as “ A is no harder than B , up to polynomial overhead.” The reduction does not solve either problem; it only *translates* an instance of A into an equivalent instance of B . Both directions of the biconditional matter: $w \in A \Rightarrow f(w) \in B$ keeps the translator from turning a yes into a no, and $w \notin A \Rightarrow f(w) \notin B$ keeps it from manufacturing a spurious yes. The picture is a single arrow from the A -world to the B -world that respects membership.



Proposition 13.1.3 (Transitivity of $\leq_p$). *If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.*

Proof. Let f reduce A to B in time $\leq p(n)$ and let g reduce B to C in time $\leq q(n)$, with p, q polynomials. Define the composition $h(w) = g(f(w))$. *Correctness:* for every w ,

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C,$$

the first step because f reduces A to B , the second because g reduces B to C ; hence $w \in A \iff h(w) \in C$. *Efficiency:* on input w of length n , computing $f(w)$ costs $\leq p(n)$ steps, and since the head writes at most one cell per step, $|f(w)| \leq p(n)$. Running g on $f(w)$ then costs $\leq q(p(n))$ steps. The total $p(n) + q(p(n))$ is a polynomial in n , because a polynomial of a polynomial is a polynomial. So h is a polynomial-time reduction of A to C , i.e. $A \leq_p C$. $\square$

Theorem 13.1.4 (Reductions transport membership in P). *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Proof. Let f be a polynomial-time reduction of A to B , computable in time $\leq p(n)$, and let M_B decide B in time $\leq q(n)$. Build a machine M_A :

$M_A =$ “On input w : (1) compute $f(w)$; (2) run M_B on $f(w)$ and copy its answer.”

Correctness: M_A accepts w iff M_B accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$, the last step by the reduction. *Time:* stage (1) takes $\leq p(n)$ steps and produces an output of length $|f(w)| \leq p(n)$; stage (2) then takes $\leq q(p(n))$ steps. The total $p(n) + q(p(n))$ is polynomial, so M_A decides A in polynomial time and $A \in P$. $\square$

Corollary 13.1.5 (Contrapositive: hardness transports the other way). *If $A \leq_p B$ and $A \notin P$, then $B \notin P$. Consequently, if $A \leq_p B$ and no polynomial-time algorithm exists for A , none exists for B either.*

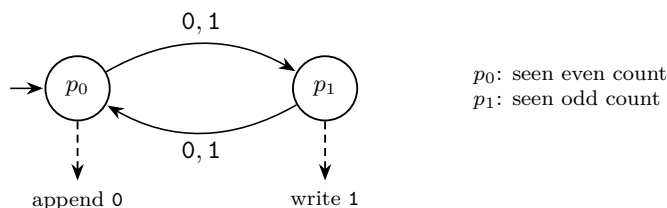
Proof. This is the contrapositive of Theorem 13.1.4: were $B \in P$, the theorem would force $A \in P$, contradicting $A \notin P$. $\square$

The corollary is the whole reason reductions point the way they do. To argue that a *new* problem B is hard, we reduce a *known*-hard problem A to B . The hardness of A then flows across the arrow and lands on B . Reducing in the wrong direction proves nothing about B ; it only re-proves that the easy thing is easy.

Worked Example 13.1.6 (Verifying a reduction is poly-time and correct: even length to a fixed target). Let $A = \{w \in \{0,1\}^* \mid |w| \text{ is even}\}$ and $B = \{w \in \{0,1\}^* \mid w \text{ ends in } 0\}$. Show $A \leq_p B$ by exhibiting a reduction f , and verify both that f is polynomial-time computable and that it is correct.

Recognition. The trigger is “show $X \leq_p Y$ ”: build one map f so that membership is preserved both ways, then check the two obligations of Definition 13.1.2 – speed and the biconditional.

Solution. Define $f(w) = w0$ if $|w|$ is even and $f(w) = 1$ if $|w|$ is odd. (The odd case just needs *some* fixed non-member of B ; the string 1 does not end in 0 , so it works.) A two-pass machine computes f : pass one counts the length parity by toggling a control bit, pass two either appends a 0 or overwrites the tape with 1 . This is $O(n)$ time, hence polynomial.



Correctness ($w \in A \iff f(w) \in B$): if $|w|$ is even then $f(w) = w0$, which ends in 0 , so $f(w) \in B$; if $|w|$ is odd then $f(w) = 1 \notin B$. Thus $f(w) \in B$ exactly when $|w|$ is even, i.e. exactly when $w \in A$.

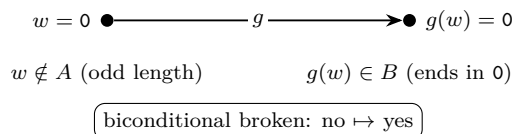
Which idea, and why? A reduction is a membership-preserving translator, so we must hit *both* the yes-case (map into B) and the no-case (map out of B); a single f doing both, computed in polynomial time, is exactly Definition 13.1.2.

Check. $w = 01$ ($|w| = 2$, even): $f(w) = 010 \in B$. $w = 0$ ($|w| = 1$, odd): $f(w) = 1 \notin B$. Both agree with membership in A .

Worked Example 13.1.7 (A subtle wrong reduction: getting the no-case wrong). A student proposes to show $A \leq_P B$ (same A, B as above) with the map $g(w) = w$. Decide whether g is a valid reduction.

Recognition. The trigger is “is this a valid reduction?” – test the biconditional on a concrete string, hunting especially for a *no*-instance of A that g pushes *into* B .

Solution. The yes-direction is fine on some strings but the no-direction fails. Take $w = 0$. Then $|w| = 1$ is odd, so $w \notin A$. But $g(w) = 0$ ends in 0, so $g(w) \in B$. That violates $w \notin A \Rightarrow g(w) \notin B$. The map preserves the *string* but not its *membership status*.



Which idea, and why? Validity is a property of *every* input, not the convenient ones; one no-instance of A landing inside B destroys the reduction, by Definition 13.1.2.

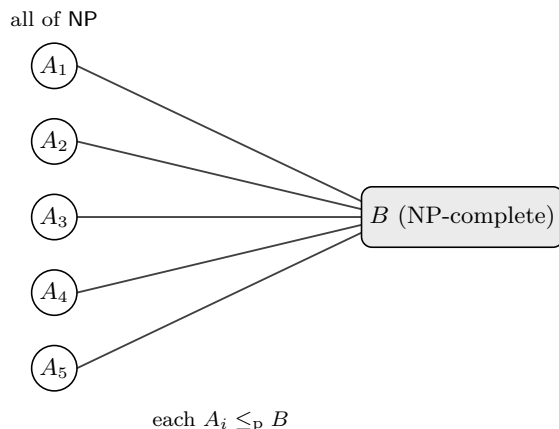
Check. The earlier f avoids exactly this trap by sending all odd-length strings to the fixed non-member 1, sealing the no-case.

NP-hardness and NP-completeness

Definition 13.1.8 (NP-hard, NP-complete). A language B is *NP-hard* if every language $A \in \text{NP}$ satisfies $A \leq_P B$. A language B is *NP-complete* if

- (i) $B \in \text{NP}$, and
- (ii) B is NP-hard.

NP-hardness is a lower bound on difficulty: B is at least as hard as everything in NP. NP-completeness adds that B is itself a member of NP – so it is a *hardest* problem *inside* the class, not merely an upper bound on the class from outside. A useful mental image is a funnel: every language of NP reduces *into* an NP-complete B .



Theorem 13.1.9 (The keystone collapse). *If some NP-complete language B lies in P, then $P = \text{NP}$.*

Proof. We always have $P \subseteq \text{NP}$, so we need only the reverse inclusion. Let A be any language in NP. Because B is NP-complete it is NP-hard, so $A \leq_P B$ by Definition 13.1.8. Given $B \in P$ as well, Theorem 13.1.4 yields $A \in P$. As $A \in \text{NP}$ was arbitrary, $\text{NP} \subseteq P$, and therefore $P = \text{NP}$. □

Corollary 13.1.10 (Two equivalent forms). (a) *If B is NP-hard and $B \in P$, then $P = \text{NP}$.* (b) *If $P \neq \text{NP}$, then no NP-complete language is in P.*

Proof. (a) is the proof of Theorem 13.1.9 with only NP-hardness used – membership of B in NP was never invoked. (b) is the contrapositive of Theorem 13.1.9. □

Part (a) is why NP-completeness is treated as strong evidence of intractability: under the widely believed $P \neq NP$, finding a polynomial-time algorithm for any NP-complete problem is impossible. So a proof of NP-completeness is a warning not to keep hunting for one.

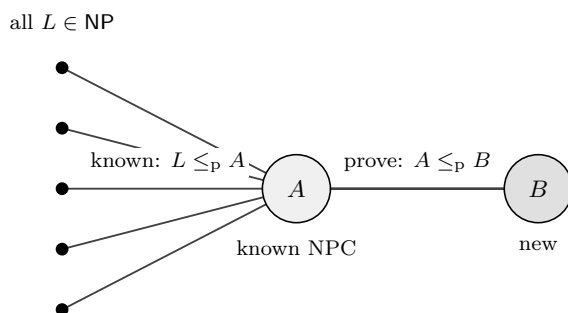
The engine: proving a new problem NP-complete

The definition of NP-hardness quantifies over *all* of NP, an infinite family of languages – hopeless to check one reduction at a time. The next theorem replaces that infinite task by a single reduction from one already-known NP-complete problem.

Theorem 13.1.11 (Reduction propagates completeness). *Suppose A is NP-complete, $A \leq_p B$, and $B \in NP$. Then B is NP-complete.*

Proof. Condition (i) of Definition 13.1.8 is given: $B \in NP$. For condition (ii), take any $L \in NP$. Since A is NP-complete it is NP-hard, so $L \leq_p A$. By hypothesis $A \leq_p B$. Transitivity of $\leq_p$ (Proposition 13.1.3) gives $L \leq_p B$. As L was an arbitrary language of NP, every language of NP reduces to B , so B is NP-hard. With (i) and (ii), B is NP-complete. $\square$

This is the working recipe for the rest of the chapter. Once *one* NP-complete problem is in hand (the Cook–Levin theorem will supply *SAT* as the seed), proving a new B complete reduces to two finite, concrete obligations – membership and a single reduction – shown schematically below.



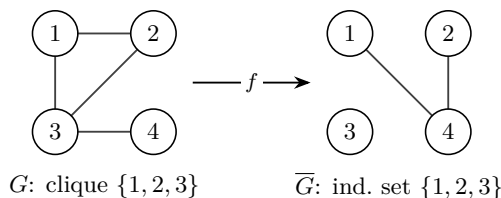
Method (Proving a language B is NP-complete).

1. **Show $B \in NP$.** Exhibit a polynomial-length certificate and a polynomial-time verifier (or a polynomial-time nondeterministic decider that guesses the certificate). This is obligation (i).
2. **Pick a known NP-complete problem A .** Choose the A whose structure most resembles B 's (formulas, graphs, sets, numbers) so the gadget design is natural.
3. **Define the reduction f from A to B .** Map an arbitrary instance x of A to an instance $f(x)$ of B . Direction is fixed: *known-hard A goes TO the new B .*
4. **Prove f is polynomial-time computable.** Bound the size of $f(x)$ and the time to build it, both polynomial in $|x|$.
5. **Prove correctness:** $x \in A \iff f(x) \in B$. Argue both implications – a satisfying object for x yields one for $f(x)$, and conversely. This is the heart; the gadgets are designed to make exactly this biconditional hold.
6. **Conclude.** Steps 3–5 give $A \leq_p B$; with step 1 and Theorem 13.1.11, B is NP-complete.

Worked Example 13.1.12 (Tracing the engine on a tiny graph reduction). Suppose it is already known that *CLIQUE* is NP-complete. A new problem *IS* (independent set, decision form) asks: given $\langle H, k \rangle$, does the undirected graph H contain k mutually *non*-adjacent vertices? The textbook reduction f sends $\langle G, k \rangle$ to $\langle \bar{G}, k \rangle$, where $\bar{G}$ is the complement graph (same vertices; an edge exactly where G has none). Trace f on a concrete G and confirm the membership equivalence on that instance.

Recognition. The trigger is “apply the engine”: we are handed $CLIQUE \leq_p IS$ via complement and asked to see, on one graph, why a k -clique of G becomes a k -independent-set of $\bar{G}$.

Solution. Take G on vertices $\{1, 2, 3, 4\}$ with edges $\{12, 13, 23, 34\}$, and $k = 3$. The triangle $\{1, 2, 3\}$ is a 3-clique, so $\langle G, 3 \rangle \in \text{CLIQUE}$. Form $\bar{G}$: it has every *non*-edge of G , namely $\{14, 24\}$.



In $\bar{G}$ the set $\{1, 2, 3\}$ has *no* internal edges (all three pairs were edges of G , hence non-edges of $\bar{G}$), so it is a 3-element independent set: $\langle \bar{G}, 3 \rangle \in \text{IS}$. The equivalence is structural – a clique in G is precisely a set with all internal pairs adjacent, which in $\bar{G}$ becomes a set with all internal pairs non-adjacent, an independent set, and vice versa. The map is poly-time: complementing the adjacency relation of an m -vertex graph touches $O(m^2)$ pairs.

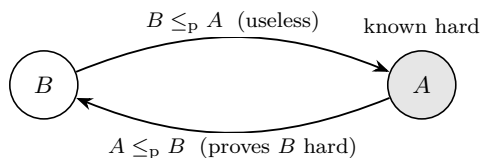
Which idea, and why? Theorem 13.1.11: a single poly-time reduction from the known-complete *CLIQUE* to *IS*, plus $\text{IS} \in \text{NP}$ (the set is the certificate), makes *IS* NP-complete – no need to revisit all of NP.

Check. Reverse it: $\bar{G}$ has independent set $\{1, 2, 3\}$; restored to G those three pairs are all edges, recovering the clique. The biconditional holds in both directions on this instance.

Worked Example 13.1.13 (Why direction is not optional). Argue that reducing the *new* problem B to the *known-hard* problem A (that is, proving $B \leq_p A$) does nothing to establish that B is hard.

Recognition. The trigger is “which way does the reduction go?” The hardness statement we want is “ B at least as hard as A ,” and Corollary 13.1.5 fixes which arrow carries that.

Solution. Suppose we only had $B \leq_p A$ with A known NP-hard. Theorem 13.1.4 reads: if the *right* side A were in P, then B would be in P. But A is hard, so this conditional is vacuous – it tells us nothing about B . In fact $B \leq_p A$ is *automatic* whenever A is NP-hard and $B \in \text{NP}$, so it carries no information about B ’s difficulty at all.



The useful arrow runs $A \rightarrow B$: by Corollary 13.1.5, A hard and $A \leq_p B$ force B hard.

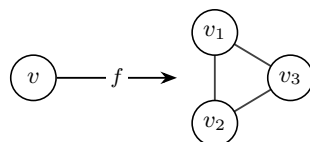
Which idea, and why? Hardness flows along the reduction arrow from source to target (Corollary 13.1.5), so to push hardness *onto* B the known-hard A must be the *source*, never the target.

Check. Consistency test: $A \leq_p B$ with A hard makes B hard; the reverse $B \leq_p A$ with A hard holds automatically whenever $B \in \text{NP}$, and indeed implies nothing – exactly as the asymmetry of Theorem 13.1.4 predicts.

Worked Example 13.1.14 (Checking a reduction’s running time by output size). A proposed reduction f maps an undirected graph G on m vertices to a graph $f(G)$ built by making three private copies of every vertex and joining copies according to a fixed local rule. Decide whether f can be polynomial-time computable, using only a size estimate.

Recognition. The trigger is “is this reduction efficient?” A necessary condition is that the output has polynomial size, since the machine needs at least $|f(G)|$ steps merely to write it.

Solution. Tripling vertices gives $3m$ vertices, so $f(G)$ has at most $\binom{3m}{2} = O(m^2)$ possible edges – polynomial in the $\Theta(m^2)$ -bit adjacency encoding of G . A local rule that inspects each original vertex or edge a constant number of times to decide which copy-edges to add runs in $O(m^2)$ time. Below, one vertex v and its triple of copies illustrate the blow-up staying linear per vertex.



one gadget: 3 vertices, ≤ 3 edges

Since both the size of $f(G)$ and the time to build it are $O(m^2)$, the output-size test is passed and (with a constant-work local rule) f is polynomial-time computable.

Which idea, and why? Output size is a fast necessary check on Definition 13.1.1: polynomial blow-up of the instance is required, since writing $f(x)$ alone costs $|f(x)|$ steps; an exponential blow-up would rule efficiency out immediately.

Check. For $m = 10$, $f(G)$ has 30 vertices and at most 435 edges – a modest polynomial jump, not the 2^{10} explosion a bad gadget could cause.

Caution. Reduction direction. To prove a target B hard, reduce the *known*-hard problem A to B : write $A \leq_p B$, i.e. translate instances of A into instances of B . Students routinely build $B \leq_p A$ instead (“I’ll solve my problem using the famous one”), which is the arrow for showing B is *easy*, not hard. The mnemonic: the arrow points from what you *know* is hard to what you *want* to prove hard, and hardness rides along it (Corollary 13.1.5).

Caution. NP-hard versus NP-complete. NP-hardness is only the lower-bound half: it says every NP language reduces to B , but B itself need not be in NP – some NP-hard languages are far harder (even undecidable). NP-complete adds the membership half $B \in \text{NP}$, pinning B as a hardest problem *within* the class. So always discharge *both* obligations; an NP-hardness proof alone does not earn the word “complete.”

Exercises.

- Let $A = \{w \in \{0, 1\}^* \mid w \text{ contains at least two 1s}\}$ and $B = \{w \in \{0, 1\}^* \mid w \text{ ends in } 1\}$. Give an explicit function f witnessing $A \leq_p B$, describe a machine computing it, argue the machine runs in time $O(n)$, and verify the biconditional $w \in A \iff f(w) \in B$ on the two strings 010 and 0110.
- Directly from Definition 13.1.2, prove that $\leq_p$ is reflexive: every language L satisfies $L \leq_p L$. Which polynomial-time function did you use, and why is it valid?
- Prove that if $A \leq_p B$ and $B \leq_p A$ both hold, it does *not* follow that $A = B$. Give a concrete pair $A \neq B$ over $\{0, 1\}^*$ with $A \leq_p B$ and $B \leq_p A$, exhibiting both reductions.
- Suppose $A \leq_p B$ via a reduction f computable in time $O(n^2)$, and B is decided by a machine running in time $O(n^3)$. Following the proof of Theorem 13.1.4, write down the machine M_A deciding A and give the tightest polynomial time bound you can for M_A as a function of the input length n . Identify the exact step where you use the bound $|f(w)| \leq O(n^2)$.
- For each statement decide true or false and give a one-line justification. (a) If $A \leq_p B$ and A is undecidable, then B is undecidable. (b) If B is NP-complete and $A \leq_p B$ with $A \in \text{NP}$, then A is NP-complete. (c) If B is NP-hard, then $B \in \text{NP}$. (d) If $A \leq_p B$ and $B \leq_p C$ and $C \in \text{P}$, then $A \in \text{P}$.
- Draw the reduction gadget. Consider the known-NP-complete problem *CLIQUE* and the target *VERTEX-COVER* (vertex cover): given $\langle H, k \rangle$, does H have a set of k vertices touching every edge? The standard reduction sends $\langle G, j \rangle$ to $\langle \overline{G}, m - j \rangle$, where G has m vertices and $\overline{G}$ is its complement. Take G on vertices $\{1, 2, 3, 4, 5\}$ with edges $\{12, 23, 34, 45, 51\}$ (a 5-cycle) and $j = 2$. Draw both G and $\overline{G}$ as labelled graphs (use **gvlab** vertices, **gedge** edges), find a 2-clique in G , and exhibit the corresponding vertex cover of size $m - j = 3$ in $\overline{G}$.
- Prove carefully that NP-hardness is preserved under $\leq_p$ in the following sense: if A is NP-hard and $A \leq_p B$, then B is NP-hard. State precisely which earlier result(s) you invoke and where transitivity is used.
- A language B satisfies $\text{NP} \subseteq \{L \mid L \leq_p B\}$ (every NP language reduces to it) and also $B \in \text{PSPACE}$ but $B \notin \text{NP}$. Is B NP-hard? Is B NP-complete? Justify both answers from Definition 13.1.8, and explain in one sentence what extra fact would be needed to upgrade B to NP-complete.

9. Find the bug. A student writes: “To prove the new problem *HAMPATH* is NP-complete, I show (i) $HAMPATH \in \text{NP}$, and (ii) $HAMPATH \leq_p 3SAT$, where $3SAT$ is known NP-complete. Hence *HAMPATH* is NP-complete.” Identify the flaw, state precisely what (ii) actually establishes, and give the corrected obligation that would complete the proof.
10. Find the bug. A reduction f is claimed to show $SAT \leq_p B$: “Given a Boolean formula φ with v variables, f enumerates all 2^v truth assignments, tests each, and outputs a fixed yes-instance of B if any assignment satisfies φ , else a fixed no-instance.” Explain why this f is correct as a function but is *not* a valid polynomial-time reduction, and name precisely which clause of Definition 13.1.1 it violates.
11. Suppose $A_1 \leq_p A_2 \leq_p A_3 \leq_p \dots \leq_p A_k$ is a chain of $k - 1$ poly-time reductions, each computable in time $O(n^2)$. Prove that $A_1 \leq_p A_k$ and give an explicit polynomial bounding the running time of the composed reduction as a function of n and k . (Hint: track how the output length can grow at each link.)
12. Let B be NP-complete and let C be any language with $\emptyset \neq C \neq \Sigma^*$ and $C \in \text{P}$. Prove that $C \leq_p B$. Conclude that $\leq_p$ does not respect the P-versus-NP-complete distinction by itself, i.e. a reduction *into* an NP-complete problem says nothing about the source’s hardness. (Hint: a poly-time decider for C lets f output one of two fixed strings.)

13.2 The Cook–Levin Theorem

The previous section gave us polynomial-time mapping reducibility $\leq_p$ and the definition of NP-completeness; what was missing was a *first* complete problem to seed every later reduction. This section supplies it: the Cook–Levin theorem says that the Boolean satisfiability problem *SAT* is NP-complete. The proof has two halves – the easy half puts *SAT* in NP by exhibiting a short, fast-to-check certificate, and the hard half encodes the *entire computation* of an arbitrary nondeterministic machine as one Boolean formula, so that satisfying assignments are exactly accepting computations. The encoding device is the *tableau*, a grid that lays out the machine’s configurations one per row, and the whole argument turns on a single local fact: a 2×3 window of cells is enough to police one step of the machine.

The satisfiability problem

Definition 13.2.1 (Boolean formula, satisfiability). A *Boolean formula* φ over variables $x_1, \dots, x_m$ is an expression built from those variables, the constants 0, 1, and the connectives $\wedge$ (and), $\vee$ (or), and $\neg$ (not; we also write $\bar{x}$ for $\neg x$). An *assignment* is a map $\alpha: \{x_1, \dots, x_m\} \rightarrow \{0, 1\}$; it *satisfies* φ if φ evaluates to 1 under α . The formula is *satisfiable* if some assignment satisfies it. Define

$$SAT = \{\langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula}\}.$$

A *literal* is a variable x or its negation $\bar{x}$; a *clause* is an $\vee$ of literals; a formula is in *conjunctive normal form* (cnf) if it is an $\wedge$ of clauses.

Theorem 13.2.2 (Cook–Levin). *SAT is NP-complete. That is, $SAT \in \text{NP}$, and $\mathcal{L} \leq_p SAT$ for every $\mathcal{L} \in \text{NP}$.*

We prove the two requirements in turn. Membership in NP is short; NP-hardness occupies the rest of the section.

SAT is in NP

Proposition 13.2.3. $SAT \in \text{NP}$.

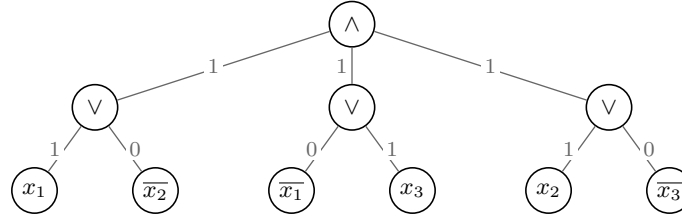
Proof. By the verifier characterization of NP, it suffices to give a polynomial-time verifier for *SAT* whose certificate has length polynomial in $|\langle \varphi \rangle|$. The certificate is a satisfying assignment α , written as one bit per variable. If φ mentions m variables then $|\alpha| = m \leq |\langle \varphi \rangle|$, so the certificate is *shorter* than the input – certainly polynomial. The verifier V on $\langle \langle \varphi \rangle, \alpha \rangle$ substitutes each bit of α for its variable and evaluates φ bottom up over its parse tree; this is a single pass that costs $O(|\langle \varphi \rangle|)$ steps. V accepts iff the formula

evaluates to 1. If φ is satisfiable, some α makes V accept; if not, no α does. Hence V is a polynomial-time verifier and $SAT \in NP$. $\square$

Worked Example 13.2.4 (A satisfying assignment is a poly-checkable certificate). Take $\varphi = (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_3) \wedge (x_2 \vee \overline{x_3})$. Exhibit a certificate showing $\langle \varphi \rangle \in SAT$ and trace the verifier on it.

Recognition. The trigger is “show membership in a language defined by an existential.” For NP the membership witness *is* the certificate; here the existential “some assignment” names the certificate directly – a row of bits, one per variable.

Solution. Try $\alpha = (x_1, x_2, x_3) = (1, 1, 1)$. The verifier walks the parse tree of φ , substituting and evaluating each clause:



root evaluates to $1 \wedge 1 \wedge 1 = 1$

Each clause evaluates to 1 (one true literal apiece), so the root $\wedge$ is 1 and V accepts. The certificate is three bits; evaluation visited a constant number of nodes per symbol of φ , hence ran in linear time.

Which idea, and why? Membership in an NP language is witnessed by a polynomial-size object that a deterministic machine checks fast; for SAT that object is a satisfying assignment and the check is one bottom-up formula evaluation.

Check. Drop the witness and the problem is hard again: deciding satisfiability without a supplied α would, by brute force, scan all $2^3 = 8$ assignments. The certificate collapses that search to one evaluation.

The tableau and the four clause groups

For the hard direction, fix any $\mathcal{L} \in NP$. By the nondeterministic characterization of NP there is a nondeterministic Turing machine N deciding $\mathcal{L}$ in time n^k for some constant k , with state set Q , tape alphabet Γ , and transition $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$. We build a polynomial-time function f sending an input w (of length n) to a formula φ with

$$w \in \mathcal{L} \iff N \text{ accepts } w \iff \varphi \text{ is satisfiable.}$$

Definition 13.2.5 (Tableau). A *tableau* for N on w is an $n^k \times n^k$ grid of cells. Row i holds the i -th configuration of one computation branch of N on w , written as a string over $C = Q \cup \Gamma \cup \{\#\}$, framed by $\#$ in the first and last columns. Row 1 is the start configuration; each row legally follows the one above it under δ , except that an accepting configuration, which has no successor, is repeated in every row below it (so a branch that accepts early still fills the grid). The tableau is *accepting* if some row contains the accept state q_{acc} .

Because N runs in n^k steps it makes at most n^k moves and visits at most $n^k + 1$ cells, so $n^k \times n^k$ is large enough to hold an entire accepting branch – up to a few extra rows and columns (for the final configuration, the state symbol, and the two $\#$'s), which change none of the bounds below and which we suppress. Accepting tableaux for N on w correspond exactly to accepting branches; so N accepts w iff an accepting tableau exists. The plan is to make a Boolean formula whose satisfying assignments are precisely the accepting tableaux.

The picture below shows the tableau as a grid: column headers j index the cells of a row, the left-hand labels mark which configuration each row holds, and the bold box is one 2×3 window.

		n^k columns, framed by #					
		$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	n^k
row 1: start		#	q_0	a	b	$\sqcup$	#
row 2		#	c	q_1	b	$\sqcup$	#
row 3		#	c	a	q_2	$\sqcup$	#
row 4		#	c	a	d	q_{acc}	#
$\vdots$							

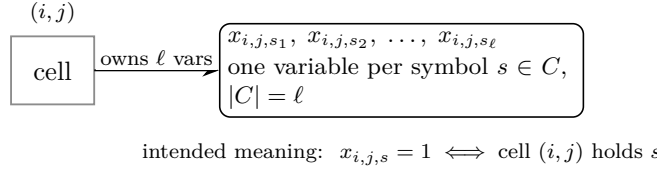
bold box: a 2×3 window

Variables. For each row i , column j (both in $\{1, \dots, n^k\}$), and each symbol $s \in C$, introduce a variable $x_{i,j,s}$, intended to be 1 exactly when cell (i, j) holds s . The formula is

$$\varphi = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}},$$

one $\wedge$ -clump per requirement. We describe each.

The variables are the bridge from grid to logic. Each single cell (i, j) does not store a symbol directly; instead it owns one Boolean variable $x_{i,j,s}$ for *every* candidate symbol $s \in C$, and “cell (i, j) holds s ” is encoded as $x_{i,j,s} = 1$.



Exactly one of a cell’s ℓ variables should be 1; the clause group φ_{cell} below enforces precisely that “exactly one” on every cell, so a satisfying assignment reads back as a well-formed grid of symbols.

- φ_{cell} – *every cell holds exactly one symbol*:

$$\varphi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \bigwedge_{\substack{s, t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right].$$

The left disjunction forces *at least one* symbol; the right pairwise clauses forbid *two* symbols at once. Together: exactly one.

- φ_{start} – *row 1 is the start configuration* $\# q_0 w_1 \dots w_n \sqcup \dots \sqcup \#$: a conjunction of single literals $x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge \dots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\sqcup} \wedge \dots \wedge x_{1,n^k,\#}$ nailing each cell of the top row.
- φ_{accept} – *the accept state appears somewhere*: $\varphi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{acc}}}$.
- φ_{move} – *every row follows the previous one*: the local condition of the next subsection.

φ_{move} and the 2×3 window

A configuration changes only near the head, so a transition rewrites just a constant-width patch. We police the whole transition by checking every 2×3 block of cells – two rows tall (the “before” row i and “after” row $i + 1$) and three columns wide (centered at j).

Definition 13.2.6 (Legal window). A 2×3 *window* is the six cells $(i, j-1), (i, j), (i, j+1)$ over $(i+1, j-1), (i+1, j), (i+1, j+1)$. The window is *legal* if its contents could appear in some correct one-step transition of N : either the head is far from these columns and the bottom row copies the top, or the head sits inside or just beside them and the change agrees with some rule of δ . A window whose top row contains q_{acc} is also legal when its bottom row copies the top, so that an accepting configuration may repeat.

Lemma 13.2.7 (Windows certify legal moves). *If row 1 of a tableau is the start configuration and every 2×3 window is legal, then each row legally follows the previous one, or repeats it if the previous one is accepting.*

Proof. Fix adjacent rows i (upper) and $i+1$ (lower). A tape symbol in column j of the upper row that is not adjacent to the state symbol sits as the top-center of a window whose top row carries no state; legality of that window forces the bottom-center to equal it, so the symbol is unchanged in the lower row – exactly as a one-step move leaves non-head cells alone. The single window whose top row contains the state symbol (with its two neighbors) must match a transition of δ (or, if the state is q_{acc} , copy its top row), so the three affected positions are rewritten consistently with some rule. Hence the lower row is a configuration that N can reach from the upper one in one step, or a repeat of an accepting one. The choice of *two* rows and *three* columns is exactly what is needed: one step rewrites only the state’s cell and its two neighbors (a left move changes all three), so the window centered on the state sees the whole change. $\square$

Encoding “the (i, j) -window is legal” as Boolean logic is a finite disjunction: enumerate every six-tuple $(a_1, \dots, a_6) \in C^6$ that forms a legal window, and assert that the six cells take one of these tuples.

$$\varphi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k \\ 1 \leq j < n^k}} \bigvee_{(a_1, \dots, a_6) \text{ legal}} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6}).$$

The set of legal tuples depends only on N (on δ and C), not on n , so each window’s disjunction has *constant* size.

Worked Example 13.2.8 (Writing the legality of one window as a clause). Let N have the rule $(q_1, \mathbf{d}, R) \in \delta(q_0, \mathbf{a})$: in state q_0 reading $\mathbf{a}$, N may write $\mathbf{d}$, move right, and enter q_1 . Write a Boolean subformula asserting that the specific window below is present, and confirm the window is legal.

Solution. The window has top row $\mathbf{c}, q_0, \mathbf{a}$ and bottom row $\mathbf{c}, \mathbf{d}, q_1$:

row i	$\mathbf{c}$	q_0	$\mathbf{a}$
row $i+1$	$\mathbf{c}$	$\mathbf{d}$	q_1
	$j-1$	j	$j+1$

As one disjunct of the window- (i, j) formula, this configuration of six cells is the conjunction

$$x_{i,j-1,\mathbf{c}} \wedge x_{i,j,q_0} \wedge x_{i,j+1,\mathbf{a}} \wedge x_{i+1,j-1,\mathbf{c}} \wedge x_{i+1,j,\mathbf{d}} \wedge x_{i+1,j+1,q_1}.$$

Recognition. The trigger is “this six-cell pattern matches a rule.” Reading the rule $(q_1, \mathbf{d}, R) \in \delta(q_0, \mathbf{a})$: the cell under the head (q_0 at column j , symbol $\mathbf{a}$ at $j+1$) becomes $\mathbf{d}$ and the head moves right to land as q_1 at column $j+1$; the off-head cell at $j-1$ is copied. That is exactly the bottom row, so the window is legal.

Which idea, and why? Each legal six-tuple becomes a six-literal conjunction; the window subformula ORs over all legal tuples, so a satisfying assignment must realise one of them at every window.

Check. The cell $(i, j-1) = \mathbf{c}$ is not adjacent to a head on its left, and indeed it is unchanged below ($\mathbf{c}$ again) – consistent with Lemma 13.2.7.

Worked Example 13.2.9 (An illegal window, and why φ_{move} excludes it). With the same N , decide whether the window with top row $\mathbf{c}, q_0, \mathbf{a}$ and bottom row $\mathbf{c}, \mathbf{d}, q_2$ is legal, where $q_2 \neq q_1$ and no rule of N sends $(q_0, \mathbf{a})$ to q_2 .

Solution. Picture both rows; the offending cell is the bottom-right state.

c	q_0	a	no rule yields q_2
c	d	q_2	

The transition $\delta(q_0, a)$ never enters q_2 , so no legal six-tuple has this top row paired with q_2 at the bottom-right. The window is therefore *not* among the disjuncts, and any assignment placing these symbols falsifies the window- (i, j) formula, hence falsifies φ_{move} and all of φ .

Recognition. The trigger is “the bottom row claims a transition the machine cannot make.” δ is the sole authority; a state the rule never produces cannot appear in the head’s landing cell.

Which idea, and why? φ_{move} is a whitelist: only tuples consistent with δ are disjuncts, so any inconsistent local change makes the conjunction over windows false.

Check. Swapping q_2 back to q_1 restores a legal tuple (the previous example), and the window formula becomes satisfiable again – confirming the formula tracks δ precisely.

Correctness and size of the reduction

Proof of the hard direction of Theorem 13.2.2. Set $f(w) = \varphi = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}$. A satisfying assignment of φ assigns exactly one symbol to every cell (by φ_{cell}), makes row 1 the start configuration (by φ_{start}), makes every row follow its predecessor or repeat an accepting one (by φ_{move} and Lemma 13.2.7), and forces q_{acc} to appear (by φ_{accept}) – i.e. it is an accepting tableau, so N accepts w . Conversely an accepting branch of N on w , with its accepting configuration repeated down to the last row, fills the tableau and the assignment $x_{i,j,s} = 1 \iff \text{cell } (i, j) \text{ holds } s$ satisfies all four parts. Thus φ is satisfiable iff N accepts w iff $w \in \mathcal{L}$. Finally f runs in polynomial time: the size bound below shows $|\varphi| = O(n^{2k})$, and the formula is so repetitive (identical fragments differing only in indices) that a machine emits it in time polynomial in n . Hence $\mathcal{L} \leq_p \text{SAT}$, and since $\mathcal{L} \in \text{NP}$ was arbitrary, SAT is NP-hard; with Proposition 13.2.3 it is NP-complete. $\square$

Worked Example 13.2.10 (Counting the formula size: why it is $O(n^{2k})$). Show that φ has $O(n^{2k})$ variables and total size $O(n^{2k})$, treating $|C| = \ell$ as a constant.

Recognition. The trigger is “bound the output of the reduction.” Reduction size = (number of cells) $\times$ (per-cell cost), summed over the four parts; the tableau is the size driver.

Solution. The tableau is $n^k \times n^k$, so it has n^{2k} cells, each carrying ℓ variables; variable count = $\ell n^{2k} = O(n^{2k})$ since ℓ depends only on N . Tally the parts:

part	fragments	size
φ_{cell}	one per cell	$O(n^{2k})$
φ_{start}	one per top-row cell	$O(n^k)$
φ_{move}	one per interior window	$O(n^{2k})$
φ_{accept}	one per cell	$O(n^{2k})$

Each fragment has *constant* size: a cell fragment uses $\binom{\ell}{2} + 1$ clauses over ℓ symbols, and a window fragment ORs over a constant number of legal C^6 -tuples. Summing, $|\varphi| = O(n^{2k}) + O(n^k) + O(n^{2k}) + O(n^{2k}) = O(n^{2k})$.

Which idea, and why? The grid has n^{2k} cells and the costliest parts pay constant cost per cell, so the formula is quadratic in the tableau side – polynomial in n , which is what makes f a valid $\leq_p$ reduction.

Check. A super-polynomial φ could never be *written* in polynomial time, so any correct tableau reduction must land at a fixed power of n ; $O(n^{2k})$ is exactly that. (Up to an $O(\log n)$ factor for writing the indices, which does not change polynomiality.)

Worked Example 13.2.11 (Encoding φ_{start} for a tiny input). Let N have start state q_0 , blank $\sqcup$, and suppose $n^k = 6$ (so each row has 6 cells). Write φ_{start} for input $w = 10$ ($n = 2$).

Recognition. The trigger is “pin down the first row.” φ_{start} is a pure conjunction of single literals, one per top-row cell, reading off $\# q_0 w_1 w_2 \sqcup \dots \#$.

Solution. Lay out row 1 in its six cells: # frames column 1, then q_0 , then the two input symbols 1, 0, then blanks fill to the closing # at column 6.

#	q_0	1	0	$\sqcup$	#
$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$

Reading column by column, each cell becomes one “on” literal:

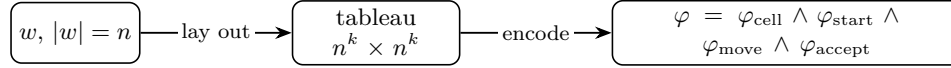
$$\varphi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,1} \wedge x_{1,4,0} \wedge x_{1,5,\sqcup} \wedge x_{1,6,\#}.$$

Which idea, and why? φ_{start} hard-codes the unique legal first row by asserting each of its cells is a fixed symbol – six conjoined literals here, $O(n^k)$ in general.

Check. Exactly 6 literals for 6 cells; combined with φ_{cell} (which forbids a second symbol per cell) this leaves the top row with no freedom, as a start configuration should have none.

Worked Example 13.2.12 (The NTM-to-formula pipeline at a glance). Summarise, as a flow, how the reduction f turns w into φ and what the equivalences guarantee.

Solution. The reduction is a fixed pipeline; the tableau is the bridge between machine and formula.



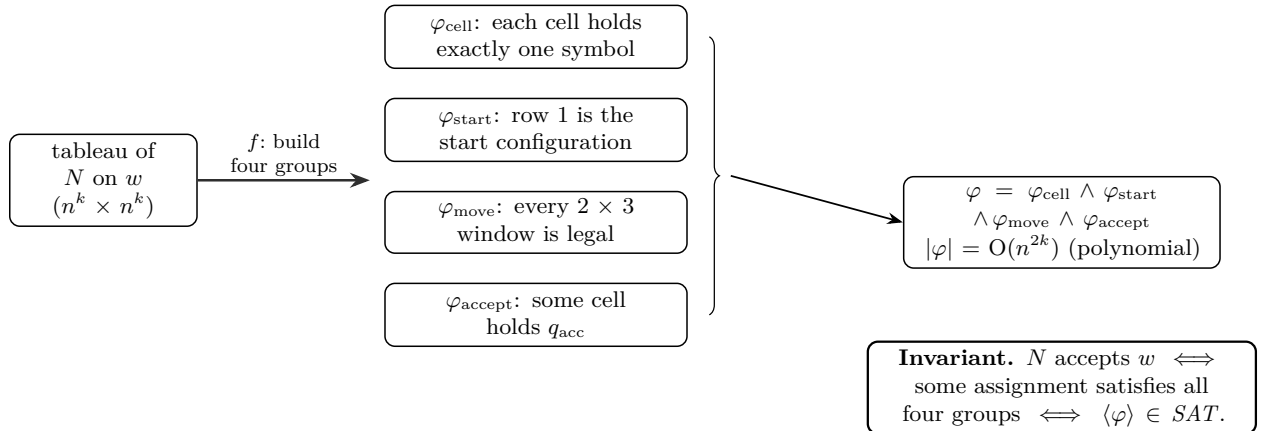
$$w \in \mathcal{L} \iff N \text{ accepts } w \iff \text{accepting tableau exists} \iff \varphi \in SAT$$

Recognition. The trigger is “a mapping reduction is a single computable function.” Here f never asks whether N accepts; it merely *writes down* the constraints an accepting computation would satisfy, and hands the search for an assignment to *SAT*.

Which idea, and why? f converts the existential “some accepting branch” into “some satisfying assignment” by making branches and assignments the same objects – the defining trick of the tableau reduction.

Check. Each arrow is polynomial-time: laying out the grid is $O(n^{2k})$ and emitting the four formula parts is $O(n^{2k})$, so f is a legitimate $\leq_p$ reduction end to end.

The whole reduction can now be seen at one level up: the construction map sends the tableau to a conjunction of four clause groups, each policing one feature of an accepting computation. The arrow $\text{tableau} \mapsto \varphi$ is the reduction f ; the box on the right is the invariant it guarantees.



Reading top to bottom, φ_{cell} makes the assignment describe a genuine grid, φ_{start} fixes its first row, φ_{move} makes each row follow the last, and φ_{accept} demands the computation halt accepting – so the satisfying assignments of φ are *exactly* the accepting tableaux of N on w . *Check.* Drop any one group and the equivalence breaks (e.g. without φ_{move} an “assignment” could jump to an accepting row that no transition reaches), which is why all four are conjoined.

Takeaway. Cook–Levin turns an NTM computation into a tableau, the tableau into Boolean variables $x_{i,j,s}$ (“cell (i,j) holds s ”), and legality into four clause groups; N accepts w iff $\varphi = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}$ is satisfiable, with $|\varphi|$ polynomial in n . The satisfying assignments of φ correspond exactly to the accepting tableaux of N on w , and $|\varphi| = O(n^{2k})$ is polynomial; hence *SAT* is NP-hard.

Method (Building the tableau formula for an NP language).

1. **Get the machine and clock.** From $\mathcal{L} \in \text{NP}$ extract an NTM N deciding $\mathcal{L}$ in time n^k ; record Q, Γ , and $C = Q \cup \Gamma \cup \{\#\}$.
2. **Lay the grid.** Build the $n^k \times n^k$ tableau; introduce a variable $x_{i,j,s}$ for every cell (i,j) and symbol $s \in C$ (intended meaning: cell (i,j) holds s).
3. **Write the four parts.** φ_{cell} (exactly one symbol per cell); φ_{start} (literals fixing row 1 to $\#q_0 w \sqcup \dots \#$); φ_{accept} (an $\vee$ asserting q_{acc} occurs); φ_{move} (an $\wedge$ over interior windows, each an $\vee$ over legal C^6 -tuples).
4. **Bound the size.** Count cells n^{2k} , multiply by the constant per-cell cost; confirm $|\varphi| = O(n^{2k})$, hence f runs in polynomial time.
5. **State the equivalence.** $w \in \mathcal{L} \iff \varphi \in \text{SAT}$, via accepting tableau $\leftrightarrow$ satisfying assignment. Conclude $\mathcal{L} \leq_p \text{SAT}$.

Caution. The window must be 2×3 , not 2×1 or 1×3 . A single column cannot tell whether a non-head cell is allowed to change, because the head might be sitting just to its left or right and about to move in; you need the two horizontal neighbors to see the head coming. Lemma 13.2.7 fails for narrower windows – a 2×1 window would wrongly accept a cell changing with no adjacent state. Three columns is the minimum width that makes “every window legal” equivalent to “every row legally follows.”

Caution. The size bound $O(n^{2k})$ is not a nicety – it is the whole reason this is a $\leq_p$ reduction. If any part of φ were exponential (say one giant clause enumerating all 2^m assignments), the function f could not even *print* its output in polynomial time, and the reduction would be worthless no matter how correct the logic. Always confirm the formula is a fixed power of n , with only constant-size fragments per cell and per window. The constant $\ell = |C|$ and the legal tuple count depend on N alone, never on n – that is what keeps each fragment constant-size.

Exercises.

1. Give a satisfying assignment for $\varphi = (\overline{x_1} \vee x_2) \wedge (x_1 \vee x_3) \wedge (\overline{x_2} \vee \overline{x_3})$, and draw its parse tree with each node labelled by the value it takes under your assignment, exactly as in the worked verifier trace. Then state, in one sentence, why this assignment is a valid NP certificate for $\langle \varphi \rangle \in \text{SAT}$.
2. Build a polynomial-time verifier V for *SAT* in numbered-stage form (input $\langle \langle \varphi \rangle, \alpha \rangle$). State the length of the certificate as a function of $|\langle \varphi \rangle|$ and the running time of V , and justify each bound in one line.
3. Consider an NTM N that runs in time n^2 on inputs of length n . For $n = 3$, state the exact dimensions of the tableau, the number of cells, and – if $|C| = 7$ – the number of variables $x_{i,j,s}$ the reduction introduces. Show your arithmetic.
4. For the same machine and the input $w = 011$ with start state q_0 , blank $\sqcup$, and $n^k = 9$, write the formula φ_{start} in full (all nine literals), and draw the top row of the tableau as a labelled strip of nine cells.
5. A classmate proposes shrinking the window to 2×2 (two rows, two columns) to “save half the cells.” Exhibit a concrete two-row pattern over a tiny alphabet that every 2×2 window in it would declare legal, yet which does *not* correspond to any single step of a machine. Conclude that Lemma 13.2.7 can fail at width 2.

6. Let N have the rule $(q_3, y, L) \in \delta(q_2, x)$. Write, as a six-literal conjunction over variables $x_{i,j,s}$, the disjunct of φ_{move} corresponding to a legal window that realises this leftward move, and draw the 2×3 window (top and bottom rows) it encodes. Indicate which cell is copied unchanged and why.
7. Find the bug. A student writes φ_{cell} using only the left half, $\bigwedge_{i,j} \bigvee_{s \in C} x_{i,j,s}$, dropping the pairwise “at most one” clauses. Give a satisfying assignment of this weakened φ_{cell} that places *two* symbols in some cell, and explain how such an assignment could let the rest of φ be satisfied by a “tableau” that does not correspond to any real computation.
8. Find the bug. A draft replaces $\varphi_{\text{accept}} = \bigvee_{i,j} x_{i,j,q_{\text{acc}}}$ with the single literal $x_{n^k,1,q_{\text{acc}}}$ (“the accept state must be in the bottom-left cell”). Explain why this makes the reduction *incorrect* – give a machine and input for which N accepts but the new formula is unsatisfiable.
9. Argue carefully that φ_{move} has size $O(n^{2k})$: count the interior windows, and bound the size of each window’s subformula by a constant that depends only on N . State precisely which quantity the constant depends on and which it does not.
10. Using Theorem 13.1.11 (if B is NP-complete and $B \leq_p C$ for $C \in \text{NP}$, then C is NP-complete), explain in three sentences why, once Cook–Levin gives that SAT is NP-complete, proving a new problem $C \in \text{NP}$ to be NP-complete only requires a single reduction $SAT \leq_p C$ (or from any known complete problem), not a fresh tableau argument.
11. Suppose an NTM N never moves its head left (every transition moves right). Sketch how the window-legality analysis simplifies: which family of windows can now be dropped from the legal set, and does the 2×3 size still remain necessary? Justify in two or three sentences.
12. True or false, with a one-line reason each. (a) Every cnf-formula is satisfiable. (b) The tableau reduction f decides whether N accepts w before producing φ . (c) The number of legal 2×3 windows depends on the input length n . (d) If $SAT \in P$ then $P = \text{NP}$.

13.3 3SAT and Reductions to Graph Problems

The Cook–Levin theorem of the previous section gave us a single seed of hardness, SAT . This section turns that seed into a forest. We first cut SAT down to the rigid normal form $3SAT$ (conjunctive normal form, exactly three literals per clause), which is still NP-complete but far easier to reduce *from*; then we use it to plant hardness in three graph problems by explicit *gadget reductions*. The one idea running through all of it is that a reduction $3SAT \leq_p B$ must build, from a formula alone and without ever solving it, an instance of B whose “yes” answer is equivalent – in both directions – to the formula being satisfiable.

Polynomial-time reducibility and $3SAT$

Definition 13.3.1 (Polynomial-time mapping reduction). A function $f: \Sigma^* \rightarrow \Sigma^*$ is *polynomial-time computable* if some deterministic Turing machine, on every input w , halts in time $O(|w|^c)$ for a constant c with $f(w)$ alone on its tape. A language A is *polynomial-time (mapping) reducible* to a language B , written $A \leq_p B$, if there is a polynomial-time computable f with

$$w \in A \iff f(w) \in B \quad \text{for every } w \in \Sigma^*.$$

We call f a *polynomial-time reduction* of A to B .

The biconditional is the whole content: f must send *every* yes-instance of A to a yes-instance of B and every no-instance to a no-instance. A function that only preserves “yes” is worthless, because then $f(w) \in B$ tells us nothing about w .

Proposition 13.3.2 (Reducibility transfers hardness). *If $A \leq_p B$ and $B \in P$, then $A \in P$. Consequently, if B is NP-complete and $B \leq_p C$ with $C \in \text{NP}$, then C is NP-complete.*

Proof. Let f reduce A to B in time $O(|w|^c)$ and let D decide B in time $O(m^d)$. To decide A on w : compute $f(w)$, then run D on it. Since f runs in time $O(|w|^c)$ its output has length $|f(w)| = O(|w|^c)$, so D costs $O((|w|^c)^d) = O(|w|^{cd})$; the total is polynomial and correct because $w \in A \iff f(w) \in B$. For the second claim, $C \in \text{NP}$ by hypothesis; every $A' \in \text{NP}$ satisfies $A' \leq_p B$ (as B is NP-complete) and $B \leq_p C$, and polynomial-time reductions compose (substitute one polynomial-time machine's output into the other), so $A' \leq_p C$. Thus C is NP-complete. $\square$

Definition 13.3.3 (3SAT). A *literal* is a variable x_i or its negation $\bar{x}_i$. A *clause* is a disjunction (OR) of literals. A formula is in *3cnf* if it is a conjunction (AND) of clauses each having exactly three literals. Let

$$3\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}.$$

A truth assignment *satisfies* ϕ iff every clause contains at least one true literal.

Theorem 13.3.4 (3SAT is NP-complete). *3SAT is NP-complete.*

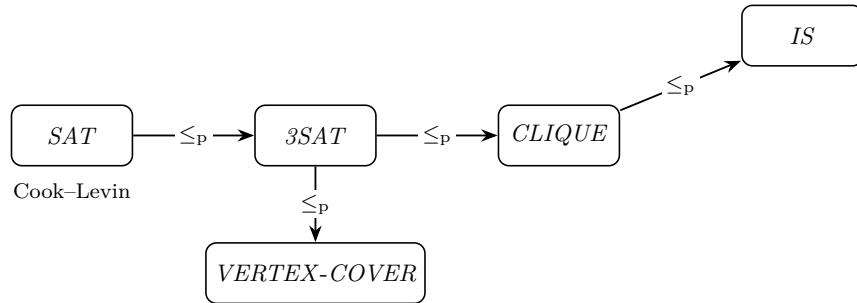
Proof. $3\text{SAT} \in \text{NP}$: a satisfying assignment is a certificate of length $O(\text{\#variables})$, and one pass evaluates every clause, so verification is polynomial. For hardness, take any $\mathcal{L} \in \text{NP}$ and the Cook–Levin formula φ for an input w (Theorem 13.2.2). It is already in cnf except for φ_{move} , an $\wedge$ over windows of $\vee$'s of six-literal $\wedge$'s; the distributive law rewrites each window's subformula as an equivalent cnf whose size depends only on the machine, so this conversion costs only polynomial time. (Multiplying out an *arbitrary* formula this way can blow it up exponentially, which is why we convert the Cook–Levin formula rather than an arbitrary SAT instance.) Then fix the clause width at three. A clause with one literal (a) becomes $(a \vee a \vee a)$; a two-literal clause $(a \vee b)$ becomes $(a \vee b \vee b)$; a long clause $(a_1 \vee \dots \vee a_\ell)$ with $\ell \geq 4$ splits, using fresh variables $z_1, \dots, z_{\ell-3}$, into

$$(a_1 \vee a_2 \vee z_1) \wedge (\bar{z}_1 \vee a_3 \vee z_2) \wedge \dots \wedge (\bar{z}_{\ell-3} \vee a_{\ell-1} \vee a_\ell).$$

If some a_i is true, set the z_j so the chain is satisfied; conversely if every a_i is false, no setting of the z_j can satisfy all $\ell - 2$ new clauses. Hence the new 3cnf-formula is satisfiable iff the original was, and the rewrite is polynomial. So $\mathcal{L} \leq_p 3\text{SAT}$ for every $\mathcal{L} \in \text{NP}$; in particular $\text{SAT} \leq_p 3\text{SAT}$. $\square$

Remark. From now on 3SAT is our preferred source of hardness. Its rigidity – a fixed conjunction of fixed-width disjunctions – is exactly what makes the gadget reductions below mechanical: one gadget per variable or per clause, glued by edges that encode “true literal” and “no contradiction.”

The three reductions of this section chain off 3SAT in the direction that spreads hardness: each arrow points *from* a problem already known NP-complete *to* a new target, so the target inherits completeness by Proposition 13.3.2.



From 3SAT to CLIQUE

Recall $\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ has } k \text{ pairwise-adjacent vertices}\}$. The reduction turns each clause into a column of three literal-vertices and joins literals across columns unless they clash.

Theorem 13.3.5 ($3\text{SAT} \leq_p \text{CLIQUE}$). *3SAT is polynomial-time reducible to CLIQUE ; hence CLIQUE is NP-complete.*

Proof. Let $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ be a 3cnf-formula with k clauses, where $C_r = (\ell_{r,1} \vee \ell_{r,2} \vee \ell_{r,3})$. Build $\langle G, k \rangle$ as follows. For each clause C_r create a *triple* of three vertices, one per literal, labelled by that literal. Place an edge between two vertices u (from clause C_r) and v (from clause C_s) iff

$$r \neq s \quad \text{and} \quad \text{their labels are not contradictory,}$$

where contradictory means one is x_i and the other is $\overline{x_i}$. Thus there are *no* edges inside a triple, and no edge between a variable and its negation. The map outputs $\langle G, k \rangle$ with the same k . It is polynomial: G has $3k$ vertices and at most $\binom{3k}{2}$ edges, each decidable by comparing two labels.

We prove ϕ is satisfiable $\iff G$ has a k -clique.

($\Rightarrow$) Let τ satisfy ϕ . Each clause C_r has some true literal; pick one such literal's vertex v_r (any one if several are true). The set $S = \{v_1, \dots, v_k\}$ has k vertices, one per triple. Any two v_r, v_s with $r \neq s$ are adjacent: they lie in different triples, and they cannot be contradictory because both literals are true under the single assignment τ (a variable and its negation are never simultaneously true). So S is a k -clique.

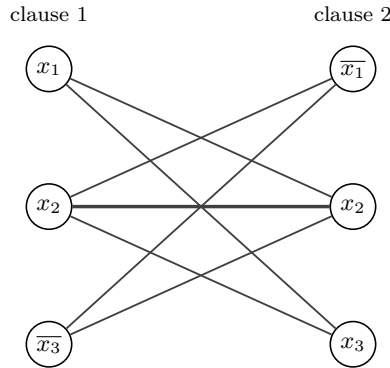
($\Leftarrow$) Let S be a k -clique in G . Since there are no edges inside a triple, no two vertices of S share a triple; as there are k triples and $|S| = k$, exactly one vertex of S lies in each triple. Define τ by setting each literal labelling a vertex of S to true. This is consistent: two contradictory literals are non-adjacent, so they cannot both be in the clique, hence we never demand x_i and $\overline{x_i}$ both true; extend τ arbitrarily on unconstrained variables. Every clause C_r contributes its S -vertex, whose literal is true, so every clause is satisfied and τ satisfies ϕ .

Finally $CLIQUE \in \text{NP}$ (the clique is the certificate), so by Proposition 13.3.2 it is NP-complete. $\square$

Worked Example 13.3.6 (Building the *CLIQUE* graph for a concrete two-clause formula). Carry out Theorem 13.3.5 on $\phi = (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3)$, draw G , and find a 2-clique.

Recognition. The trigger is “reduce a 3cnf-formula to *CLIQUE*”: make one triple per clause, then join cross-clause non-contradictory literals. Here $k = 2$, so we hunt for a 2-clique, i.e. a single edge.

Solution. The two triples are $\{x_1, x_2, \overline{x_3}\}$ (clause 1) and $\{\overline{x_1}, x_2, x_3\}$ (clause 2). Draw an edge between a vertex of triple 1 and a vertex of triple 2 unless the labels contradict. The only contradictory cross pairs are $x_1 \sim \overline{x_1}$ and $\overline{x_3} \sim x_3$; all seven other cross pairs get edges. No edges go inside a triple.



The bold edge x_2 - x_2 is a 2-clique. Reading off: put the chosen literals true, i.e. $x_2 = 1$. That single setting makes both clauses true (each contains x_2), so any extension, say $x_1 = 0, x_3 = 0$, satisfies ϕ .

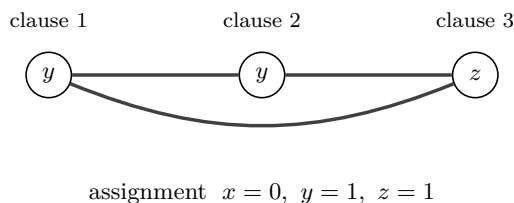
Which idea, and why? A k -clique forces one true literal per clause with no variable forced both ways; the edge x_2 - x_2 realizes “ x_2 true satisfies both clauses,” which is a genuine satisfying assignment.

Check. Under $x_2 = 1$: clause 1 = $(* \vee 1 \vee *) = 1$ and clause 2 = $(* \vee 1 \vee *) = 1$, so $\phi = 1$. Conversely the non-edges x_1 - $\overline{x_1}$ and $\overline{x_3}$ - x_3 correctly forbid the impossible demands “ x_1 true and false” or “ x_3 true and false.”

Worked Example 13.3.7 (Reading a satisfying assignment off a clique in a three-clause graph). The reduction applied to $\psi = (x \vee y \vee z) \wedge (\overline{x} \vee y \vee \overline{z}) \wedge (x \vee \overline{y} \vee z)$ yields a 9-vertex graph; suppose a search returns the 3-clique $\{y \text{ (clause 1), } y \text{ (clause 2), } z \text{ (clause 3)}\}$. Recover an assignment and verify it.

Recognition. “Given a clique, produce the assignment” is the ($\Leftarrow$) direction: the clique has exactly one vertex per triple, and we set each chosen literal true.

Solution. The chosen literals are y, y, z – no contradiction appears, so the clique is consistent. Set $y = 1$ and $z = 1$; the variable x is unconstrained, take $x = 0$.



Which idea, and why? Each triple yields exactly one true literal because exactly one of its vertices is in the clique; collecting those literals gives a clause-by-clause satisfying assignment, and the absence of contradictory edges guarantees the collection is consistent.

Check. Under $x = 0, y = 1, z = 1$: clause 1 = $(0 \vee 1 \vee 1) = 1$; clause 2 = $(1 \vee 1 \vee 0) = 1$; clause 3 = $(0 \vee 0 \vee 1) = 1$. All true, so ψ is satisfied – exactly as the clique predicted.

From *CLIQUE* to *INDEPENDENT-SET* (the complement graph)

A *clique* is a set of pairwise-adjacent vertices; an *independent set* is a set of pairwise-*non*-adjacent vertices. They are mirror images under graph complementation. Let $IS = \{\langle G, k \rangle \mid G \text{ has an independent set of size } k\}$ and let $\overline{G}$ be the *complement* graph on the same vertices, with $\{u, v\}$ an edge of $\overline{G}$ iff it is *not* an edge of G .

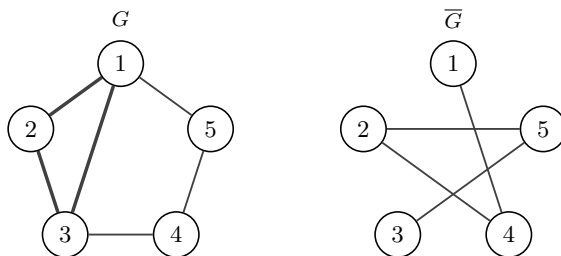
Theorem 13.3.8 ($CLIQUE \leq_p IS$). *The map $\langle G, k \rangle \mapsto \langle \overline{G}, k \rangle$ is a polynomial-time reduction from *CLIQUE* to *IS*.*

Proof. Computing $\overline{G}$ flips each of the $\binom{n}{2}$ possible adjacencies once, so the map is polynomial. For any vertex set S : S is a clique in G iff every pair in S is adjacent in G iff every pair in S is *non*-adjacent in $\overline{G}$ iff S is an independent set in $\overline{G}$. Taking $|S| = k$, G has a k -clique iff $\overline{G}$ has an independent set of size k , i.e. $\langle G, k \rangle \in CLIQUE \iff \langle \overline{G}, k \rangle \in IS$. $\square$

Worked Example 13.3.9 (The clique/independent-set correspondence on a five-vertex graph). Let G be the 5-cycle $1-2-3-4-5-1$ with the extra chord $\{1, 3\}$. Find a clique in G and the matching independent set in $\overline{G}$.

Recognition. “Clique here equals independent set in the complement” is Theorem 13.3.8: build $\overline{G}$ by drawing exactly the non-edges of G .

Solution. G has edges $12, 23, 34, 45, 51, 13$. The triangle $\{1, 2, 3\}$ is a clique ($12, 23, 13$ all present). The complement $\overline{G}$ on $\{1, \dots, 5\}$ has the remaining pairs as edges: $14, 24, 25, 35$. In $\overline{G}$ the set $\{1, 2, 3\}$ is independent – none of $12, 13, 23$ is an edge of $\overline{G}$.



Which idea, and why? Complementation swaps “all pairs adjacent” for “no pair adjacent,” so the same vertex set that is a clique in G is automatically an independent set in $\overline{G}$.

Check. In G the bold triangle $\{1, 2, 3\}$ uses edges $12, 23, 13$ (a clique); in $\overline{G}$ those three pairs are all missing edges, so $\{1, 2, 3\}$ is independent. The reduction would map $\langle G, 3 \rangle$ to $\langle \overline{G}, 3 \rangle$, and both are “yes” instances.

From 3SAT to VERTEX-COVER

A *vertex cover* of G is a set of vertices touching every edge; let

$$\text{VERTEX-COVER} = \{\langle G, k \rangle \mid G \text{ has a vertex cover of size } k\}.$$

We reduce directly from 3SAT, using a *variable gadget* (one edge per variable) and a *clause gadget* (one triangle per clause), linked by edges from each literal-vertex of a triangle to the matching variable-vertex.

Theorem 13.3.10 ($3\text{SAT} \leq_p \text{VERTEX-COVER}$). *3SAT is polynomial-time reducible to VERTEX-COVER; hence VERTEX-COVER is NP-complete.*

Proof. Let ϕ have m variables and ℓ clauses. Build G : (*variable gadget*) for each variable x_i , two vertices $x_i, \bar{x}_i$ joined by an edge; (*clause gadget*) for each clause C_r , a triangle on three vertices labelled by its three literals; (*links*) connect each clause-triangle vertex to the like-labelled variable-gadget vertex. Set $k = m + 2\ell$. The graph has $2m + 3\ell$ vertices and is built in polynomial time. We show ϕ satisfiable $\iff G$ has a vertex cover of size k .

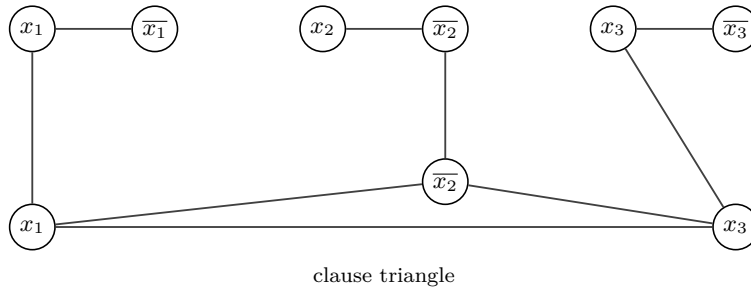
($\Rightarrow$) Given a satisfying τ : from each variable gadget take the *true* literal's vertex (m vertices, covering every variable-gadget edge). In each clause pick one true literal; take the *other two* triangle vertices (2ℓ vertices, covering all three triangle edges since two of three vertices always cover a triangle). Total $m + 2\ell = k$. Every link edge is covered: a link from the unpicked triangle vertex (the true literal) is covered at its variable-gadget end, which holds the true literal; the other two links are covered at their triangle ends. So this is a k -cover.

($\Leftarrow$) Given a k -cover S . Each variable-gadget edge needs ≥ 1 vertex of S , and each clause triangle needs ≥ 2 vertices of S (two are required to cover three triangle edges). That already forces $\geq m + 2\ell = k$ vertices, so S has *exactly* one vertex per variable gadget and exactly two per triangle. Set each variable-gadget vertex in S true (this is consistent, one per variable). For each clause, the one triangle vertex *not* in S has its link edge uncovered at the triangle end, so it must be covered at the variable-gadget end – meaning that literal's variable-vertex is in S , i.e. that literal is true. Hence each clause has a true literal and τ satisfies ϕ . Since $\text{VERTEX-COVER} \in \text{NP}$, Proposition 13.3.2 gives NP-completeness. $\square$

Worked Example 13.3.11 (The VERTEX-COVER gadget graph for a one-clause formula). Apply Theorem 13.3.10 to $\phi = (x_1 \vee \bar{x}_2 \vee x_3)$, draw G , state k , and exhibit a cover of that size from τ : $x_1 = 1, x_2 = 0, x_3 = 0$.

Recognition. “Reduce 3cnf to VERTEX-COVER”: one edge per variable, one triangle per clause, links by label; here $m = 3, \ell = 1$ so $k = m + 2\ell = 5$.

Solution. Variable gadgets: edges $x_1\text{--}\bar{x}_1, x_2\text{--}\bar{x}_2, x_3\text{--}\bar{x}_3$. One clause triangle on $x_1, \bar{x}_2, x_3$, with links to the equal-labelled variable vertices.

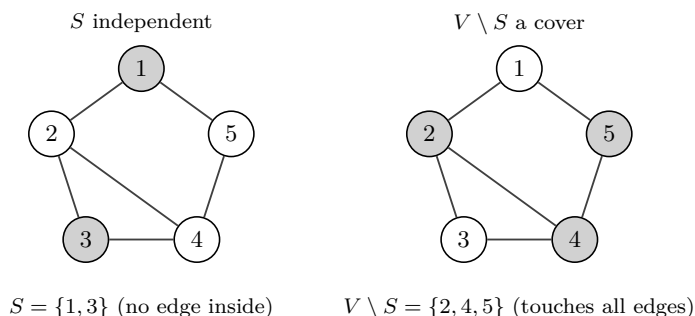


Cover of size $k = 5$: from the variable gadgets take the true literals $\{x_1, \bar{x}_2, \bar{x}_3\}$ (3 vertices). In the single clause pick the true literal x_1 and take the other two triangle vertices $\{\bar{x}_2, x_3\}$ (2 vertices). Total $3 + 2 = 5$.

Which idea, and why? One variable-vertex per gadget covers the gadget edge and encodes a truth value; two of three triangle vertices cover the triangle, leaving the third link to be covered by a *true* variable-vertex – which is exactly the clause being satisfied.

Check. Edges $x_1\text{--}\bar{x}_1, x_2\text{--}\bar{x}_2, x_3\text{--}\bar{x}_3$ are each covered (by $x_1, \bar{x}_2, \bar{x}_3$). The triangle's three edges are covered by its two chosen vertices. The link $x_1\text{--}x_1$ is covered at x_1 (true). All edges covered with $5 = m + 2\ell$ vertices, matching τ .

The three targets of this section are not really three problems but one, seen from three angles. The cleanest statement is the complement pairing of *IS* and *VERTEX-COVER*: on a fixed graph, a set S is an independent set exactly when the remaining vertices $V \setminus S$ form a vertex cover. The same five-vertex graph is drawn twice below – left with S (filled) marked independent, right with $V \setminus S$ (filled) marked as the cover.



Check. The graph has edges 12, 23, 34, 45, 51, 24. With $S = \{1, 3\}$: no edge has both ends in S (independent), and every edge has an endpoint in $V \setminus S = \{2, 4, 5\}$ (a cover). The two conditions are literally the same statement read from opposite sides: “no edge inside S ” is “every edge meets $V \setminus S$.” The clique angle is one complementation away – S is a clique in $\overline{G}$ iff S is independent in G .

Takeaway. In any graph, S is an independent set iff $V \setminus S$ is a vertex cover; *CLIQUE*, *IS*, and *VERTEX-COVER* are thus the same problem viewed three ways. Concretely: no edge has both endpoints in S iff every edge has an endpoint in $V \setminus S$; and S is a clique in G iff S is independent in the complement graph $\overline{G}$.

Method (The gadget-reduction recipe ($3SAT \leq_p B$)).

1. **Pick the source.** Reduce *from* a known NP-complete problem, almost always *3SAT*, *to* the target B . Never the other way.
2. **Design a variable gadget.** Find a structure of B with two stable configurations, one per truth value (an edge with two endpoints).
3. **Design a clause gadget.** Find a structure forcing “at least one literal true” (a vertex triple with only cross-edges that admits a k -clique only with one node per clause; a triangle that needs three cover-vertices unless a true literal helps).
4. **Glue with edges.** Connect gadgets so consistency (no x_i both true and false) and satisfaction (each clause served) are enforced – e.g. omit edges between contradictory literals, or link like-labelled vertices.
5. **Set the budget.** Choose k from the gadget counts (k clauses; $m + 2\ell$).
6. **Prove both directions and polynomiality.** Show satisfying assignment $\Rightarrow$ target object, target object $\Rightarrow$ satisfying assignment, and that the map runs in polynomial time.

Caution. In the $3SAT \leq_p \text{CLIQUE}$ graph, do *not* draw edges *within* a clause triple. Those omitted intra-triple edges are precisely what forces a k -clique to take one vertex from each of the k triples; add them and a clique could sit inside a single clause, destroying the correspondence with “one true literal per clause.”

Caution. A reduction is only valid if it is *polynomial-time* and the equivalence is an *iff*. Two failure modes: (i) the construction is correct but the map is too slow (e.g. it secretly tries assignments) – then it is not a reduction at all; (ii) the map is fast but only one implication holds (every satisfiable ϕ gives a k -clique, but some k -clique arises from an unsatisfiable ϕ) – then $f(w) \in B$ does not certify $w \in A$. Always verify both the clock and both arrows of Definition 13.1.2.

Exercises.

1. Apply the $3SAT \leq_p \text{CLIQUE}$ construction to $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$. Draw the resulting graph G (two triples, cross-clause non-contradictory edges only), state the target k , exhibit a k -clique, and read off a satisfying assignment of ϕ from it.

2. Carry out the same *CLIQUE* reduction on the three-clause formula $\phi = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z) \wedge (x \vee y \vee \bar{z})$. You need not draw every edge: instead list, for the candidate 3-clique $\{z(C_1), z(C_2), \bar{z}(C_3)\}$, whether each required cross-edge is present, and decide whether this set is a valid clique.
3. Let $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$. Build the *CLIQUE* graph and argue, by inspecting its edges, that ϕ is satisfiable. Then give the complement graph $\bar{G}$ and the independent set that Theorem 13.3.8's reduction associates with your clique.
4. Draw the full *VERTEX-COVER* gadget graph produced by the $3SAT \leq_p VERTEX-COVER$ reduction from $\psi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$. State the values of m , ℓ , and $k = m + 2\ell$, and from the assignment $x_1 = 1, x_2 = 1, x_3 = 0$ list explicitly which k vertices form the corresponding vertex cover.
5. For the graph G on vertices $\{1, 2, 3, 4, 5\}$ with edges $\{12, 13, 23, 34, 45\}$, find the largest clique and the largest independent set. Then draw $\bar{G}$ and confirm directly that a clique in G of size s corresponds to an independent set of size s in $\bar{G}$.
6. Prove that $\leq_p$ is transitive: if $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$. State clearly where you use that the composition of two polynomials is a polynomial, and where you use the two biconditionals $w \in A \iff f(w) \in B$ and $u \in B \iff g(u) \in C$.
7. In the $3SAT \leq_p VERTEX-COVER$ reduction, the clause gadget is a triangle and the budget allots exactly two of its three vertices to the cover. Prove that *any* vertex cover of a triangle must contain at least two of its three vertices, and explain in one sentence why this is what forces the third (the “true literal”) link to be covered from the variable side.
8. Show that *INDEPENDENT-SET* $\in$ NP by giving a certificate and a polynomial-time verifier. Then, combining this with Theorems 13.3.5 and 13.3.8, conclude that *INDEPENDENT-SET* is NP-complete, naming each theorem you invoke.
9. Find the bug. A student reduces $3SAT$ to *CLIQUE* but, “to be safe,” also adds an edge between the three vertices *inside* each clause triple. They then claim ϕ is satisfiable iff the new graph has a k -clique. Give a specific small ϕ (with its graph) on which their claim fails, and identify exactly which direction of the iff breaks.
10. Find the bug. A proposed reduction f from $3SAT$ to *CLIQUE* runs as follows: “On input ϕ , try all 2^m assignments; if one satisfies ϕ output a fixed 1-vertex graph with $k = 1$, otherwise output a single vertex with $k = 2$.” Verify that this f does satisfy $w \in 3SAT \iff f(w) \in CLIQUE$, yet is *not* a valid polynomial-time reduction. State precisely which clause of Definition 13.1.2 it violates.
11. Let ϕ be a 3cnf-formula in which some clause repeats a literal, e.g. $(x \vee x \vee \bar{y})$. Explain what the $3SAT \leq_p CLIQUE$ construction does with the two x -vertices of that triple (are they joined? to what are they each connected?), and argue that the duplication does not affect whether G has a k -clique.
12. Consider the variant reduction that maps a 3cnf-formula ϕ with k clauses to $\langle \bar{G}, 3k - k \rangle$ where G is the $3SAT \leq_p CLIQUE$ graph. Determine whether this is a correct reduction from $3SAT$ to *INDEPENDENT-SET*: state what an independent set of size $3k - k = 2k$ in $\bar{G}$ would mean back in G , and decide whether “ G has a k -clique” is equivalent to “ $\bar{G}$ has an independent set of size $2k$.” Justify your answer.

13.4 More NP-Complete Problems

The previous section drove $3SAT$ into three graph problems by reading a clause as a local constraint on a small gadget. Here we push the technique further, into problems whose gadgets are not static subgraphs but *routes*: a Hamiltonian path must thread every node exactly once, so a variable becomes a structure the path may traverse in two directions, and a clause becomes a node the path must remember to visit. We prove four problems NP-complete – directed *HAMPATH*, its undirected cousin *UHAMPATH*, the travelling-salesman decision problem *TSP*, and the purely arithmetic *SUBSET-SUM* – and along the way we exhibit the one tool that makes the whole zoo grow from a single root: *composing* polynomial-time reductions, so a new hardness proof may start from whichever known-hard problem is most convenient.

Reducing through a chain

Recall (Section 13.1) that B is NP-complete when $B \in \text{NP}$ and every $A \in \text{NP}$ satisfies $A \leq_p B$. To prove a *new* problem C complete we never revisit “every $A \in \text{NP}$ ” by hand; we lean on one known-complete B and the fact that $\leq_p$ is transitive.

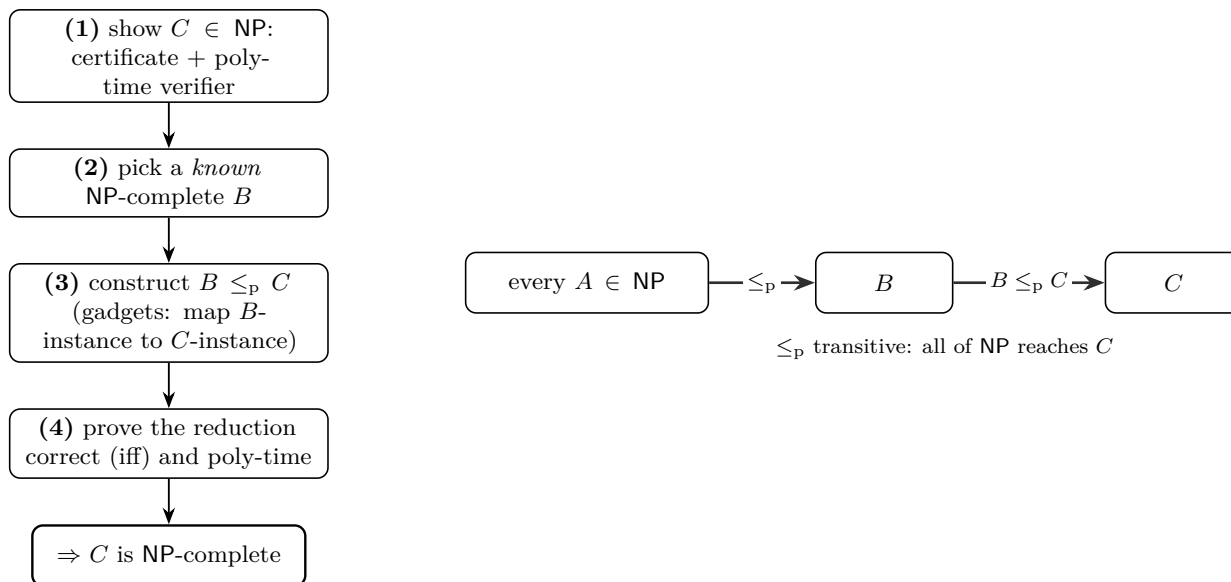
Lemma 13.4.1 (Composition of reductions). *If $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.*

Proof. Let f reduce A to B in time $O(n^a)$ and g reduce B to C in time $O(n^b)$. Define $h(w) = g(f(w))$. Since f is polynomial-time, $|f(w)| \leq c|w|^a$ for some constant c , so g runs on $f(w)$ in time $O(|f(w)|^b) = O(|w|^{ab})$, and computing f first adds $O(|w|^a)$; the total is polynomial. Correctness is immediate: $w \in A \iff f(w) \in B \iff g(f(w)) \in C$, i.e. $w \in A \iff h(w) \in C$. $\square$

Theorem 13.4.2 (Spreading completeness). *If B is NP-complete, $C \in \text{NP}$, and $B \leq_p C$, then C is NP-complete.*

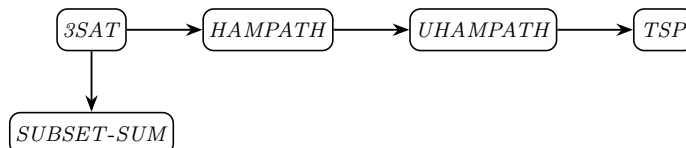
Proof. $C \in \text{NP}$ by hypothesis. For completeness take any $A \in \text{NP}$. Since B is complete, $A \leq_p B$; with $B \leq_p C$, Lemma 13.4.1 gives $A \leq_p C$. Thus every $A \in \text{NP}$ reduces to C . $\square$

The skeleton of every such proof is fixed; it is worth seeing the four obligations laid out as a checklist, with the composition fact that justifies step (3).



The right-hand strip is the composition note behind step (2)–(3): because B is NP-complete, *every* $A \in \text{NP}$ already reduces to B ; chaining $B \leq_p C$ then sends all of NP into C , so C inherits hardness. The $\leq_p$ arrows point *from* the known-hard problem *into* the target. *Check.* If the arrow in step (3) pointed the other way ($C \leq_p B$) it would only show C is no harder than B – the opposite of what an NP-completeness proof needs. *Takeaway.* To prove C NP-complete: show $C \in \text{NP}$, then reduce a *known* NP-complete B to C (that is $B \leq_p C$, the arrow pointing from B into C); since every NP problem $\leq_p B \leq_p C$, all of NP reduces to C . With $C \in \text{NP}$ and B NP-complete, $B \leq_p C$ gives C NP-complete because $\leq_p$ is transitive.

So every proof below has the same skeleton: show membership in NP (a certificate plus a polynomial-time check), then reduce a problem already known complete – 3SAT for HAMPATH and for SUBSET-SUM, HAMPATH for UHAMPATH, and UHAMPATH for TSP. The roots are fixed; only the edges of the chain change.



The directed Hamiltonian path problem

Definition 13.4.3 (*HAMPATH*). A *Hamiltonian path* in a directed graph $G = (V, E)$ is a directed path that visits every node of G exactly once. Write

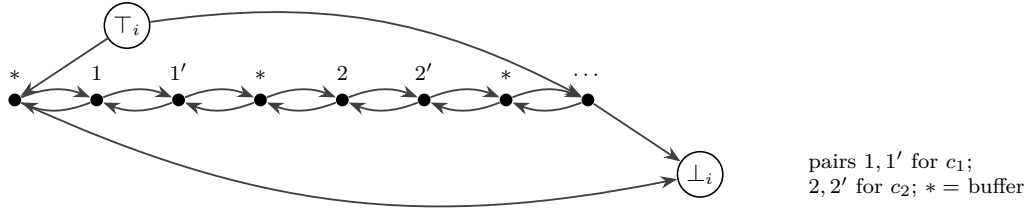
$$HAMPATH = \{\langle G, s, t \rangle \mid G \text{ is a digraph with a Hamiltonian path from } s \text{ to } t\}.$$

Theorem 13.4.4. *HAMPATH is NP-complete.*

Proof. Membership. A certificate is an ordering $s = v_1, v_2, \dots, v_m = t$ of all m nodes. A verifier checks in polynomial time that the list is a permutation of V (no repeats, all present), that it starts at s and ends at t , and that each consecutive pair (v_i, v_{i+1}) is an edge. So $HAMPATH \in \text{NP}$.

Hardness. We show $3SAT \leq_p HAMPATH$. Let $\varphi = (a_1 \vee b_1 \vee c_1) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$ have variables $x_1, \dots, x_l$ and clauses $c_1, \dots, c_k$ (each a, b, c a literal). We build a digraph G with designated s, t so that G has a Hamiltonian s - t path iff φ is satisfiable.

Variable gadget (a diamond). Each variable x_i becomes a “diamond”: a top node and a bottom node joined by a *horizontal row* of inner nodes, with the row edges running in *both* directions. The row holds $3k + 1$ inner nodes, grouped as k adjacent *pairs* (one pair per clause) separated by buffer nodes. The diamonds are stacked $s \rightarrow \text{diamond}_1 \rightarrow \dots \rightarrow \text{diamond}_l \rightarrow t$, each diamond’s bottom feeding the next diamond’s top. A Hamiltonian path is forced to enter each diamond at the top and leave at the bottom, sweeping the row either left-to-right (call it *zig*, encoding $x_i = \text{TRUE}$) or right-to-left (*zag*, encoding $x_i = \text{FALSE}$).



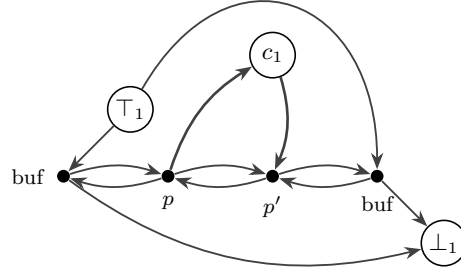
Clause gadget (a node). Each clause c_j becomes a single node c_j . If literal x_i occurs in c_j , add edges from the *left* member of the j th pair of diamond i to c_j and from c_j back to the *right* member, so a left-to-right (zig) sweep can *detour* out to c_j and return. If $\overline{x_i}$ occurs in c_j , add the two edges the other way around, so only a right-to-left (zag) sweep can detour. Thus the path can collect c_j exactly when some literal of c_j is satisfied by the chosen zig/zag pattern.

Correctness. If φ is satisfiable, sweep diamond i zig when $x_i = \text{TRUE}$ and zag otherwise; each clause has a true literal, so detour once into each c_j from a pair whose orientation matches. This visits every node once: a Hamiltonian s - t path. Conversely a Hamiltonian path must be *normal* – it can never enter c_j from one diamond and exit into another, because a row node next to where it left (the pair partner or a buffer) would then be stranded – so each detour returns to its own pair, the zig/zag pattern reads off a truth assignment, and the visit to each c_j certifies a true literal in c_j . Hence φ is satisfiable iff G has the path. The graph has $O(kl)$ nodes and is built in polynomial time. $\square$

Worked Example 13.4.5 (The two ways through a diamond). For the single clause $c_1 = (x_1 \vee \overline{x_2} \vee x_3)$, draw diamond 1 (variable x_1) with its one clause-pair and the detour edges to the clause node c_1 , and trace the detour the path takes when $x_1 = \text{TRUE}$.

Recognition. This is the variable-gadget rule: a diamond is swept *zig* (left to right) for TRUE. The trigger “ x_1 occurs positively in c_1 ” tells us the detour edges run left-member $\rightarrow c_1 \rightarrow$ right-member, so a zig sweep can take them.

Solution. With $k = 1$ the row of diamond 1 has $3k + 1 = 4$ inner nodes: a pair p, p' for c_1 plus two buffers. A TRUE (zig) sweep enters at p , and because x_1 is positive in c_1 the edges $p \rightarrow c_1 \rightarrow p'$ exist, so the path detours $p \rightarrow c_1 \rightarrow p'$ and continues.



The thick edges $p \rightarrow c_1 \rightarrow p'$ are the detour; the path is $\top_1 \rightarrow \text{buf} \rightarrow p \rightarrow c_1 \rightarrow p' \rightarrow \text{buf} \rightarrow \perp_1$, hitting every node of the gadget once and collecting c_1 .

Which idea, and why? The diamond is the variable gadget: zig encodes TRUE, and a positive occurrence of x_1 in c_1 installs detour edges that only a zig sweep can use – so a true literal is exactly what lets the path reach the clause node.

Check. Had we set $x_1 = \text{FALSE}$ (zag), the path would run $p' \rightarrow p$, but the detour edges point $p \rightarrow c_1 \rightarrow p'$, the wrong way; the path could not collect c_1 via x_1 – correct, since a false x_1 does not satisfy the positive literal.

The undirected Hamiltonian path problem

Definition 13.4.6 (*UHAMPATH*). A Hamiltonian path of an *undirected* graph visits every node exactly once. Write

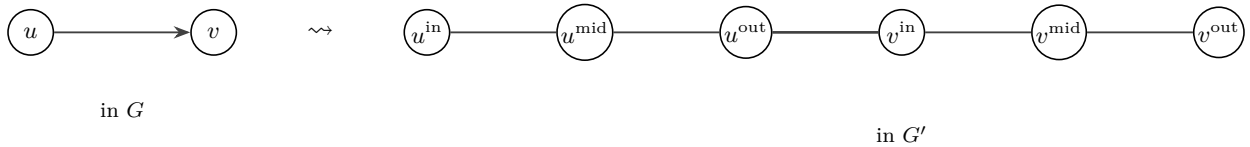
$$\text{UHAMPATH} = \{\langle G, s, t \rangle \mid G \text{ is undirected with a Hamiltonian path from } s \text{ to } t\}.$$

We cannot just erase edge directions: an undirected edge could be traversed either way, destroying the one-way logic the diamonds relied on. The fix is a gadget that re-installs direction *locally*, by node-splitting.

Theorem 13.4.7. *UHAMPATH is NP-complete.*

Proof. Membership. As for *HAMPATH*: the node ordering is the certificate, checked in polynomial time (now each consecutive pair must be an undirected edge). So *UHAMPATH* $\in$ NP.

Hardness. We show *HAMPATH* $\leq_p$ *UHAMPATH*; completeness then follows from Theorem 13.4.2 since *HAMPATH* is complete (Theorem 13.4.4). Given a digraph G with nodes s, t , build an undirected G' as follows. Split every node $u \notin \{s, t\}$ into a triple $u^{\text{in}}, u^{\text{mid}}, u^{\text{out}}$ with undirected edges $u^{\text{in}} - u^{\text{mid}} - u^{\text{out}}$. Set $s' = s^{\text{out}}$ and $t' = t^{\text{in}}$ (so s, t contribute only their out/in halves). For each directed edge $u \rightarrow v$ of G , add the undirected edge $u^{\text{out}} - v^{\text{in}}$.



Correctness. A Hamiltonian path $s, u_1, \dots, u_k, t$ in G becomes the path

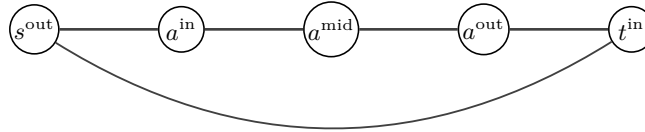
$$s^{\text{out}}, u_1^{\text{in}}, u_1^{\text{mid}}, u_1^{\text{out}}, u_2^{\text{in}}, u_2^{\text{mid}}, u_2^{\text{out}}, \dots, t^{\text{in}}$$

in G' , which is Hamiltonian. Conversely, any Hamiltonian path of G' from s^{out} to t^{in} must move triple by triple: from u^{out} the only neighbours besides u^{mid} are nodes v^{in} , and the only way to include u^{mid} (degree 2) is the run $u^{\text{in}} - u^{\text{mid}} - u^{\text{out}}$. So the visited triples, read as their original nodes, give a Hamiltonian s – t path of G . The construction adds at most two nodes per original node and runs in polynomial time. $\square$

Worked Example 13.4.8 (Converting a tiny directed instance to undirected). Let G be the digraph on $\{s, a, t\}$ with edges $s \rightarrow a$, $a \rightarrow t$, and $s \rightarrow t$. It has the Hamiltonian path s, a, t . Apply node-splitting to produce G' and exhibit its Hamiltonian path from s^{out} to t^{in} .

Recognition. The trigger is “make a directed Hamiltonian instance undirected,” which is exactly the node-splitting reduction: split only interior nodes (here just a) into in/mid/out, keep s ’s out and t ’s in, and turn each directed edge $u \rightarrow v$ into $u^{\text{out}} - v^{\text{in}}$.

Solution. Only a is interior, so it splits into $a^{\text{in}}, a^{\text{mid}}, a^{\text{out}}$; we keep $s' = s^{\text{out}}$ and $t' = t^{\text{in}}$. The three directed edges become $s^{\text{out}} - a^{\text{in}}$ (from $s \rightarrow a$), $a^{\text{out}} - t^{\text{in}}$ (from $a \rightarrow t$), and $s^{\text{out}} - t^{\text{in}}$ (from $s \rightarrow t$).



The thick edges trace the Hamiltonian path $s^{\text{out}} - a^{\text{in}} - a^{\text{mid}} - a^{\text{out}} - t^{\text{in}}$, which corresponds to s, a, t in G . The extra undirected edge $s^{\text{out}} - t^{\text{in}}$ (from the shortcut $s \rightarrow t$) is simply not used, because using it would leave a^{mid} unvisited.

Which idea, and why? Node-splitting forces direction back into an undirected graph: a^{mid} has degree 2, so any Hamiltonian path must traverse $a^{\text{in}} - a^{\text{mid}} - a^{\text{out}}$ in that order, which is exactly the orientation of the original directed edges.

Check. G' has 5 nodes and the listed path visits all 5 once, starting at $s^{\text{out}} = s'$ and ending at $t^{\text{in}} = t'$ – a valid Hamiltonian path, mirroring s, a, t .

Travelling salesman

Definition 13.4.9 (*TSP*). An instance is a complete undirected graph on m nodes with a nonnegative integer weight on each edge, plus a bound B . Let

$$TSP = \{\langle G, B \rangle \mid G \text{ has a tour visiting every node once and returning, of total weight } \leq B\}.$$

Theorem 13.4.10. *TSP is NP-complete.*

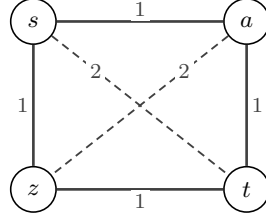
Proof. Membership. The certificate is the cyclic order of the m nodes; a verifier checks it is a permutation and sums the m edge weights against B in polynomial time.

Hardness. We show $UHAMPATH \leq_p TSP$ (Theorem 13.4.2 then applies). Given $\langle G, s, t \rangle$ with G on nodes V , $m = |V|$, add one new node z and build a complete graph G^* on the $m + 1$ nodes $V \cup \{z\}$. Weight an edge 1 if it is an edge of G and 2 otherwise. To turn the *path* requirement into a *tour*, give the two edges $\{t, z\}, \{z, s\}$ weight 1 as well (so a tour may close cheaply through z), and set the bound $B = m + 1$. Then a Hamiltonian s – t path of G (using $m - 1$ real edges, each weight 1) closes via t, z, s into a tour of weight $(m - 1) + 2 = m + 1 = B$. Conversely any tour of weight $\leq B = m + 1$ on $m + 1$ edges must use $m + 1$ edges each of weight exactly 1; the only weight-1 edges at z are $\{s, z\}$ and $\{z, t\}$, so the tour passes s, z, t , and deleting z leaves a weight-1 Hamiltonian path, i.e. a path using only edges of G , between s and t . So G has a Hamiltonian s – t path iff G^* has a tour of weight $\leq m + 1$. The construction is polynomial. $\square$

Worked Example 13.4.11 (A weighted instance from a path instance). Let G be the undirected graph on $\{s, a, t\}$ with edges $\{s, a\}, \{a, t\}$ (a path $s - a - t$). Build the *TSP* instance and read off the cheap tour.

Recognition. The trigger “decide a Hamiltonian path by weight” is the $UHAMPATH \leq_p TSP$ rule: add a node z , complete the graph, weight present edges 1 and absent edges 2, give the closing edges $\{t, z\}, \{z, s\}$ weight 1, and set $B = m + 1$.

Solution. Here $m = 3$, so $B = 4$. The complete graph on $\{s, a, t, z\}$ has six edges: $\{s, a\}, \{a, t\}$ are real (weight 1), the closing edges $\{t, z\}, \{z, s\}$ get weight 1, and $\{s, t\}, \{a, z\}$ get weight 2.



The tour $s \rightarrow a \rightarrow t \rightarrow z \rightarrow s$ uses four weight-1 edges, total $4 = B$: within budget. It corresponds to the Hamiltonian path $s-a-t$ (drop z and its two closing edges).

Which idea, and why? TSP packages a Hamiltonian *path* as a cheap *tour*: present edges cost 1, the rest cost 2, and $B = m + 1$ forces a tour to use only weight-1 edges – which is exactly a Hamiltonian path closed through the added node z .

Check. Any tour using a weight-2 edge has total $\geq 3 \cdot 1 + 2 = 5 > B$; so a tour meets the budget iff a Hamiltonian path exists – consistent with the path $s-a-t$ that G actually has.

Subset sum

Definition 13.4.12 (*SUBSET-SUM*). Given a multiset $S = \{x_1, \dots, x_n\}$ of positive integers and a target t ,

$$SUBSET-SUM = \{\langle S, t \rangle \mid \text{some sub-multiset } Y \subseteq S \text{ has } \sum_{y \in Y} y = t\}.$$

Theorem 13.4.13. *SUBSET-SUM is NP-complete.*

Proof. Membership. The sub-multiset Y is the certificate; a verifier adds its elements and compares to t in polynomial time (numbers are in binary/decimal, so addition is polynomial in the input length). So $SUBSET-SUM \in \text{NP}$.

Hardness. We show $3SAT \leq_p SUBSET-SUM$. Let φ have variables $x_1, \dots, x_l$ and clauses $c_1, \dots, c_k$. We build base-10 numbers with $l + k$ digit columns: one column per variable (left block) and one per clause (right block).

Variable numbers. For each x_i create y_i and z_i . Both have a 1 in the x_i column and 0 in the other variable columns. In clause column c_j : y_i has a 1 iff x_i occurs positively in c_j ; z_i has a 1 iff $\bar{x}_i$ occurs in c_j . (Choosing y_i will mean “ $x_i = \text{TRUE}$ ”.)

Slack numbers. For each clause c_j create two equal numbers g_j, h_j , each with a single 1 in the c_j column and 0 elsewhere.

Target. t has a 1 in every variable column and a 3 in every clause column.

Why no carries. Each column is summed over a subset of these numbers; the entries are all 0 or 1, and any column has at most five 1’s available (at most one y_i or z_i per literal, three literals per clause, plus the two slacks). Five < 10 , so column sums never carry – each target digit is matched *independently*.

Correctness. If φ is satisfiable, pick y_i when $x_i = \text{TRUE}$ and z_i when $x_i = \text{FALSE}$; this puts a single 1 in each variable column (matching the target’s 1). In clause column c_j the chosen variable numbers contribute between 1 and 3 (the number of distinct true literals in c_j , which is ≥ 1 since c_j is satisfied); choose enough of g_j, h_j to top the column up to 3. The subset sums to t . Conversely, if a subset sums to t , then – no carries – to get a 1 in variable column i it must take exactly one of y_i, z_i , defining a truth assignment. In clause column c_j the slacks $g_j + h_j$ supply at most 2, so reaching 3 forces at least one chosen variable number to contribute a 1 there – i.e. at least one literal of c_j is true. Hence φ is satisfied. The numbers have $l + k$ digits and there are $2l + 2k$ of them, so the instance is built in polynomial time. $\square$

Worked Example 13.4.14 (The number table for a 1-variable, 1-clause formula). Take $\varphi = (x_1 \vee x_1 \vee x_1)$ (one variable x_1 , one clause c_1 in which x_1 occurs positively). Build the *SUBSET-SUM* table and exhibit a subset summing to the target.

Recognition. The trigger “turn a 3cnf-formula into numbers” is the $3SAT \leq_p SUBSET-SUM$ construction: one variable column, one clause column; rows y_1, z_1 for the variable and g_1, h_1 for the clause; target = 1 in the variable column, 3 in the clause column.

Solution. With $l = 1$ and $k = 1$ each number has 2 digits: column x_1 then column c_1 . Since x_1 appears *positively* in c_1 , y_1 gets a 1 in column c_1 and z_1 gets a 0 there. The slacks $g_1 = h_1 = 01$ each carry a 1 in column c_1 . The target is 13.

	x_1	c_1
y_1	1	1
z_1	1	0
g_1	0	1
h_1	0	1
t	1	3

Read as decimals: $y_1 = 11$, $z_1 = 10$, $g_1 = h_1 = 1$, $t = 13$. Choose $Y = \{y_1, g_1, h_1\}$: $11 + 1 + 1 = 13 = t$. The choice of y_1 (over z_1) encodes $x_1 = \text{TRUE}$, which indeed satisfies φ .

Which idea, and why? Variables become y/z pairs (one 1 in the variable column to force an exclusive choice), clauses become columns whose target digit 3 is reached only with help from a true literal, and the slacks g, h absorb the surplus – all with no carries because every column has < 10 ones.

Check. The rival choice $Y' = \{z_1, g_1, h_1\}$ gives $10 + 1 + 1 = 12 \neq 13$: with z_1 (meaning $x_1 = \text{FALSE}$) the clause column reaches only 2, never 3 – exactly because a false x_1 leaves c_1 unsatisfied.

Method (How to add a new problem to the NP-complete family).

1. **Show $C \in \text{NP}$.** Name a polynomial-size certificate (a path, a tour, a subset, an assignment) and a polynomial-time verifier that checks it.
2. **Pick the source.** Choose a *known* NP-complete B that is structurally close to $C - 3\text{SAT}$ when you can build variable/clause gadgets; an already-reduced graph problem (here *HAMPATH*) when the target is also about paths/tours.
3. **Design gadgets.** Build a piece of a C -instance that simulates a variable (two mutually exclusive options) and a piece that simulates a clause (forces “at least one true literal”). Wire them so legal C -solutions correspond exactly to satisfying assignments.
4. **Prove both directions.** ($\Rightarrow$) From a solution of B build a solution of C ; ($\Leftarrow$) from any solution of C recover a solution of B . Both must hold for the same instance.
5. **Check polynomial time & invoke composition.** The map is computable in polynomial time; by Theorem 13.4.2, C is NP-complete. Future reductions may start from C .

Caution. Do not confuse the *directed* and *undirected* Hamiltonian problems. A directed edge $u \rightarrow v$ permits travel one way only; dropping its direction would let a path slip $v \rightarrow u$ and break the diamond’s zig/zag logic. That is why $\text{HAMPATH} \leq_p \text{UHAMPATH}$ *splits* each node into $u^{\text{in}} - u^{\text{mid}} - u^{\text{out}}$ – the degree-2 middle node forces any Hamiltonian path to pass through the triple in the fixed order, re-encoding direction with undirected edges. Reduce *from* the directed problem *to* the undirected one, never the reverse: you are proving the undirected problem is at least as hard.

Caution. The *SUBSET-SUM* numbers are engineered so that *no column ever carries*. If you wrote the gadget in binary, or packed more than nine 1’s into a column, a carry would let one column “borrow” from another and the clean “one 1 per variable column, exactly 3 per clause column” bookkeeping would collapse – the proof’s independence of columns would fail. Keep the base large enough (decimal is comfortable) that the largest possible column sum stays below the base, so each target digit is matched on its own.

Exercises.

1. Let G be the digraph on $\{s, u, v, t\}$ with directed edges $s \rightarrow u$, $u \rightarrow v$, $v \rightarrow t$, $s \rightarrow v$, $u \rightarrow t$. Decide by hand whether $G \in \text{HAMPATH}$ with the given s, t ; if it is, exhibit the Hamiltonian path, and if not, explain which node forces failure.
2. Apply the node-splitting reduction to the digraph on $\{s, a, b, t\}$ with edges $s \rightarrow a$, $a \rightarrow b$, $b \rightarrow t$, $a \rightarrow t$. Draw the undirected graph G' (label every $\cdot^{\text{in}}, \cdot^{\text{mid}}, \cdot^{\text{out}}$ node and every edge), then trace the Hamiltonian path of G' from s^{out} to t^{in} that corresponds to the directed path s, a, b, t .

3. In the node-splitting gadget, the middle node u^{mid} has degree exactly 2. Prove that any Hamiltonian path of G' from s^{out} to t^{in} must traverse $u^{\text{in}}, u^{\text{mid}}, u^{\text{out}}$ consecutively (in one of the two orders), and explain why this is what re-encodes edge direction in the undirected graph.
4. For the 3cnf-formula $\varphi = (x_1 \vee \overline{x_2} \vee x_3)$ (one clause), describe the directed graph that the $3SAT \leq_p HAMPATH$ reduction produces: how many diamonds, how many inner nodes per diamond's horizontal row, and which detour edges connect the clause node c_1 to which pairs. You need not draw the whole graph – give the node/edge counts and the three detour attachments.
5. Build the *SUBSET-SUM* number table for $\varphi = (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2)$. List the decimal values of all rows $y_1, z_1, y_2, z_2, g_1, h_1, g_2, h_2$ and the target t , then find a subset summing to t and read off the satisfying assignment it encodes.
6. Using the $UHAMPATH \leq_p TSP$ reduction, convert the undirected path-instance G on $\{s, a, b, t\}$ with edges $\{s, a\}, \{a, b\}, \{b, t\}$ into a *TSP* instance. State the weight of every edge of the complete graph G^* , the bound B , and the tour that meets the budget.
7. Prove that $TSP \in \text{NP}$ directly from the definition of **NP**: state the certificate, the verifier's checks, and why each check runs in time polynomial in the length of $\langle G, B \rangle$ (mind that the weights are written in binary).
8. Suppose someone hands you a polynomial-time algorithm that decides *UHAMPATH*. Using *only* the reductions of this section, explain step by step how to use it to decide *HAMPATH*, and name the lemma that guarantees the combined procedure still runs in polynomial time.
9. Find the bug. A student claims: “To prove *UHAMPATH* is NP-complete, reduce *UHAMPATH* to *HAMPATH* by orienting each undirected edge $\{u, v\}$ as the two directed edges $u \rightarrow v$ and $v \rightarrow u$.” Identify what is wrong with this argument: examine the *direction* of the reduction, and decide whether the proposed map even preserves the Hamiltonian-path property.
10. Find the bug. In a $3SAT \leq_p SUBSET-SUM$ reduction a student uses *binary* digits instead of decimal for the columns. Give a concrete tiny formula and target where a carry between columns makes a non-satisfying subset hit the target, so the reduction is no longer correct. Then state the single design rule that prevents carries.
11. Let φ be a 3cnf-formula with l variables and k clauses. In the $3SAT \leq_p HAMPATH$ reduction, the horizontal row of each diamond holds $3k + 1$ inner nodes. Explain why the count is $3k + 1$ rather than, say, $2k$: what are the buffer (separator) nodes for, and what would go wrong with a Hamiltonian path if adjacent clause-pairs were not separated?
12. Define the longest-path decision problem

$$LPATH = \{\langle G, s, t, k \rangle \mid G \text{ is a digraph with a simple } s\text{-}t \text{ path of length } \geq k\}.$$

Show $HAMPATH \leq_p LPATH$, and conclude that *LPATH* is **NP**-complete (you may assume $LPATH \in \text{NP}$). State the value of k your reduction uses in terms of the number of nodes of G .

13.5 Coping with NP-Completeness

By now a verdict of “NP-complete” may feel like the end of the road, but in practice it is the beginning of a second, more interesting question: *given* that a fast exact universal algorithm almost certainly does not exist, what do you actually run? This section surveys the standard responses – exact exponential search, approximation, heuristics and local search, parameterized algorithms, restriction to tame instance families, and average-case relaxation – and works two of them in detail (an FPT branching algorithm for *VERTEX-COVER* and the sharp split between $2SAT \in \text{P}$ and $3SAT$ hard). Throughout we lean on the polynomial-time reductions of the previous sections: the same arrows that prove hardness also tell us which coping tricks transfer from one problem to another.

The web of reductions

Every NP-completeness proof in this chapter has the same shape: take a problem already known hard, and reduce it *to* the target. Stringing those individual reductions together produces a directed graph whose arrows are the relation $\leq_p$. Reading it is a skill in its own right, so we draw it first.

Definition 13.5.1 (Polynomial-time mapping reducibility). A function $f: \Sigma^* \rightarrow \Sigma^*$ is *polynomial-time computable* if some deterministic Turing machine, running in time $n^{O(1)}$, halts on every input w with exactly $f(w)$ written on its tape. A language A is *polynomial-time mapping reducible* to a language B , written $A \leq_p B$, if there is a polynomial-time computable f with

$$w \in A \iff f(w) \in B \quad \text{for every } w \in \Sigma^*.$$

We call f a *polynomial-time reduction* of A to B .

Definition 13.5.2 (NP-completeness). A language B is *NP-complete* if (1) $B \in \text{NP}$, and (2) every $A \in \text{NP}$ satisfies $A \leq_p B$. A language satisfying only (2) is called *NP-hard*.

Proposition 13.5.3 (Reductions compose and transport hardness). *If $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$. Consequently, if B is NP-complete, $C \in \text{NP}$, and $B \leq_p C$, then C is NP-complete.*

Proof. Let f reduce A to B in time n^a and g reduce B to C in time n^b . The composite $g \circ f$ satisfies $w \in A \iff f(w) \in B \iff g(f(w)) \in C$. To compute $g(f(w))$, first compute $f(w)$ in time $O(|w|^a)$; since a machine cannot write more symbols than steps, $|f(w)| = O(|w|^a)$, so running g costs $O(|f(w)|^b) = O(|w|^{ab})$, a polynomial. Thus $A \leq_p C$. For the second claim: given any $A' \in \text{NP}$ we have $A' \leq_p B$ (as B is NP-complete) and $B \leq_p C$, so $A' \leq_p C$ by composition; with $C \in \text{NP}$ this is exactly Definition 13.1.8. $\square$

The whole point of Proposition 13.5.3 is that NP-completeness spreads along the arrows: once *SAT* is pinned down by the Cook–Levin theorem, every problem you can reach from it by a chain of $\leq_p$ arrows (while staying in NP) is NP-complete too. The classic chain is the following.

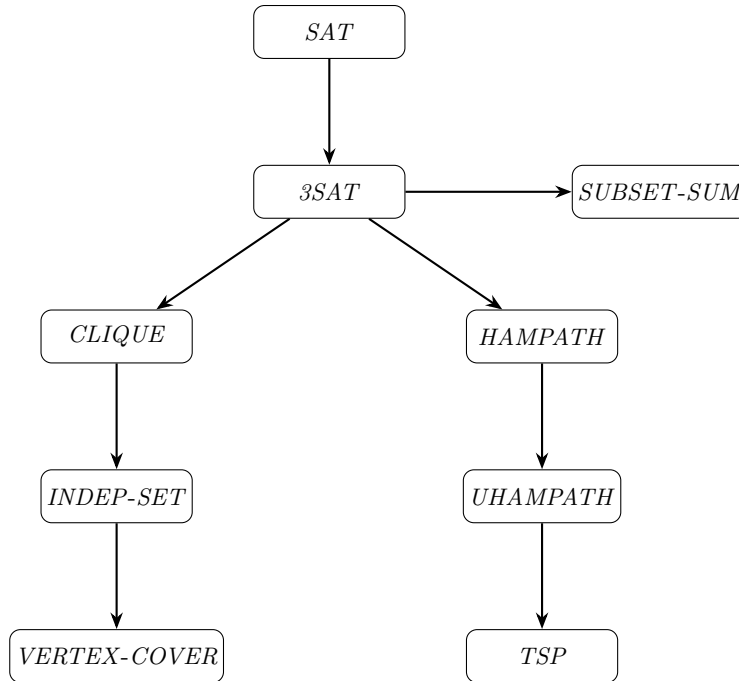
Worked Example 13.5.4 (The map of the classic NP-complete problems). Draw the directed graph whose nodes are

$$\text{SAT}, 3\text{SAT}, \text{CLIQUE}, \text{INDEP-SET}, \text{VERTEX-COVER}, \\ \text{HAMPATH}, \text{UHAMPATH}, \text{TSP}, \text{SUBSET-SUM}$$

and whose edges are the standard $\leq_p$ reductions, with *SAT* at the root.

Recognition. The trigger is “organize the known reductions.” Each textbook reduction is one arrow $X \leq_p Y$ (read “ X reduces to Y ”); chaining arrows out of *SAT* certifies that every reachable node is NP-complete by Proposition 13.5.3.

Solution. Cook–Levin gives *SAT*; restricting clause width gives $\text{SAT} \leq_p 3\text{SAT}$; the clause-triple gadget gives $3\text{SAT} \leq_p \text{CLIQUE}$; complementing the graph gives $\text{CLIQUE} \leq_p \text{INDEP-SET}$; “ S is an independent set iff $V \setminus S$ is a vertex cover” gives $\text{INDEP-SET} \leq_p \text{VERTEX-COVER}$; the diamond/clause construction gives $3\text{SAT} \leq_p \text{HAMPATH}$; forgetting edge directions (with a 3-node split) gives $\text{HAMPATH} \leq_p \text{UHAMPATH}$; weighting edges $1/2$ gives $\text{UHAMPATH} \leq_p \text{TSP}$; and an arithmetic gadget gives $3\text{SAT} \leq_p \text{SUBSET-SUM}$.



Which idea, and why? Each arrow is a single $\leq_p$; reachability from *SAT* plus membership in **NP** equals **NP-completeness**, so the entire reachable set is **NP-complete** at once.

Check. Every node other than *SAT* has at least one incoming arrow traceable back to *SAT*, so all nine problems are **NP-complete**; the arrows show only the reductions we proved – reverse arrows exist too, since any two **NP-complete** problems reduce to each other.

Caution. The arrows point *toward* the problem you are proving hard. “To show *TSP* is hard, reduce *TSP* to a known-hard problem” is backwards and proves nothing. You must reduce a *known-hard* problem to *TSP* ($UHAMPATH \leq_p TSP$): a solver for the target would then solve the known-hard source. Always reduce *from* hard to your target.

Five ways to cope

None of the strategies below repeals **NP-completeness**; each trades away some part of the “exact, fast, all instances, all the time” wish list.

1. Exact exponential algorithms. Sometimes you genuinely need the optimum and the inputs are small. A smarter-than-brute-force exponential algorithm can push the feasible size up by a real margin. Brute force for *VERTEX-COVER* tries all $\binom{n}{k}$ subsets; the branching algorithm below runs in $O(2^k n)$, which for fixed small k is *polynomial* in n even though it is exponential in k .

2. Approximation. Relax “find the optimum” to “find a provably near-optimal solution in polynomial time.” A ρ -approximation algorithm returns a solution within a factor ρ of optimal on *every* instance. We preview the classic 2-approximation for *VERTEX-COVER* here and develop the theory in Chapter 15.

3. Heuristics and local search. Drop the guarantee entirely: run greedy choices, simulated annealing, or local moves that usually do well but carry no worst-case promise. Fast and often excellent in practice – and silent about the worst case.

4. Parameterized / FPT. Isolate a secondary parameter k (a solution size, a treewidth, a number of clauses) and seek running time $f(k) \cdot n^{O(1)}$. The exponential cost is quarantined inside $f(k)$; if k stays small the algorithm is fast on large n . Such a problem is *fixed-parameter tractable* (FPT).

5. Restricted instances and average case. Many NP-complete problems become easy on structured inputs: planar graphs, bounded-treewidth graphs, or the width-2 special case $2SAT \in P$. Alternatively, abandon the worst case and ask for an algorithm that is fast (or correct) on *most* inputs drawn from some distribution.

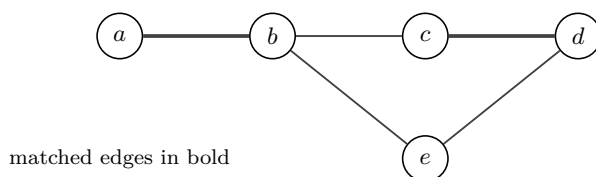
Theorem 13.5.5 (A 2-approximation for *VERTEX-COVER*). *There is a polynomial-time algorithm that, on any undirected graph G , outputs a vertex cover of size at most twice the size of a minimum vertex cover.*

Proof. Run the *maximal matching* algorithm: repeatedly pick any uncovered edge, add *both* its endpoints to the output set C , and delete all edges now touching either endpoint; stop when no edges remain. This is polynomial (each step removes at least one edge). The chosen edges form a matching M – they are pairwise disjoint, since adding both endpoints deletes every edge meeting them – so $|C| = 2|M|$. The set C covers every edge of G : an uncovered edge would still be in the graph, contradicting the stopping rule. Now any vertex cover, in particular a minimum cover C^* , must contain at least one endpoint of each of the $|M|$ disjoint matching edges, so $|C^*| \geq |M|$. Combining, $|C| = 2|M| \leq 2|C^*|$. $\square$

Worked Example 13.5.6 (Running the 2-approximation on a concrete graph). Apply the matching-based *VERTEX-COVER* approximation to the graph with vertices $\{a, b, c, d, e\}$ and edges ab, bc, cd, de, be . Report the cover it returns and compare to the optimum.

Recognition. The trigger is “cover within a factor of 2.” The algorithm of Theorem 13.5.5 grabs both endpoints of a maximal set of disjoint edges; the disjoint edges lower-bound the optimum.

Solution. Pick edge ab , add a, b , and delete ab, bc, be (all touch a or b). Remaining edges: cd, de . Pick cd , add c, d , delete cd, de . No edges remain. Output $C = \{a, b, c, d\}$, size 4.



The two bold edges ab, cd are the matching M , so $|M| = 2$ and the algorithm returns $2|M| = 4$ vertices. A minimum cover is $\{b, d\}$ of size 2 (check: b covers ab, bc, be ; d covers cd, de), and none is smaller, since the disjoint edges ab, cd need two distinct endpoints. So the algorithm’s 4 is exactly $2 \cdot 2$.

Which idea, and why? Both endpoints of each disjoint matched edge enter the cover, giving a factor of exactly 2 against a lower bound ($|M|$) that any cover must already pay.

Check. Output $4 \leq 2 \cdot 2 = 4$: the guarantee holds, here with equality (4 vs. optimum 2) – the factor 2 is a *worst-case* ratio, and this instance attains it.

Parameterized algorithms: FPT for *VERTEX-COVER*

The flagship coping win is for *VERTEX-COVER*. Instead of measuring everything against n , add a second knob k = the cover size we are allowed, and branch.

Theorem 13.5.7 (*VERTEX-COVER* is fixed-parameter tractable). *The problem *VERTEX-COVER* – does G have a vertex cover of size $\leq k$? – is decidable in time $O(2^k n)$, where $n = |V|$.*

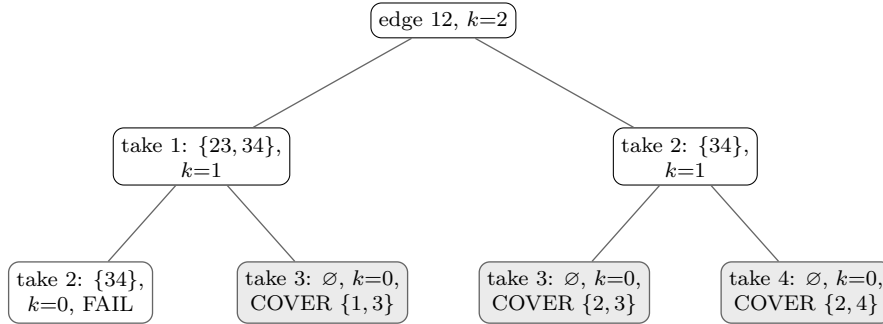
Proof. Use the branching rule: if G has no edges, accept (the empty cover works, and $0 \leq k$). If $k = 0$ but an edge uv remains, reject (a cover of size 0 cannot cover uv). Otherwise pick any edge uv . *Every* vertex cover must contain u or v – the edge uv has to be covered. So branch into two subproblems: “put u in the cover, recurse on $G - u$ with budget $k - 1$,” and “put v in the cover, recurse on $G - v$ with budget $k - 1$.” Accept iff either branch accepts.

Each node of the recursion has ≤ 2 children and the budget drops by 1 at each level, so the recursion tree has depth $\leq k$ and at most 2^k leaves, hence $O(2^k)$ nodes. At each node we do $O(n)$ work (find an uncovered edge, delete a vertex). Total: $O(2^k n)$. Correctness is exactly the branching rule – one of u, v is forced into every cover – applied recursively. $\square$

Worked Example 13.5.8 (The FPT branching tree for *VERTEX-COVER* with $k = 2$). Run the branching algorithm of Theorem 13.5.7 on the path graph P_4 with vertices $1-2-3-4$ (edges $12, 23, 34$) and budget $k = 2$. Draw the recursion tree.

Recognition. The trigger is “small target cover size.” Each remaining edge forces one of its two endpoints into the cover; we branch on that binary choice, decrementing k .

Solution. Start at the root ($\{12, 23, 34\}, k=2$). Pick edge 12 and branch on 1 vs. 2.



The left branch takes vertex 1 (cover so far $\{1\}$, budget 1, remaining edges $23, 34$), then branches on edge 23: taking 2 leaves edge 34 with budget 0 – FAIL; taking 3 deletes both 23 and 34, leaving no edges with budget 0 – accept with cover $\{1, 3\}$. The right branch (take 2) similarly yields covers $\{2, 3\}$ and $\{2, 4\}$. So P_4 has a size-2 cover; the tree has $\leq 2^2 = 4$ leaves, as the bound predicts.

Which idea, and why? Branching on the forced choice $u \in C$ or $v \in C$ per edge spends the budget k along a depth- k binary tree, isolating the exponential blow-up in 2^k rather than 2^n .

Check. Leaves = $4 = 2^k$; each root-to-leaf path uses $k = 2$ vertices; the accepting leaves $\{1, 3\}, \{2, 3\}, \{2, 4\}$ are all genuine vertex covers of P_4 .

Restricted instances: why *2SAT* is easy but *3SAT* is hard

Restricting the clause width from 3 to 2 collapses satisfiability from NP-complete to polynomial. The reason is structural, and worth seeing.

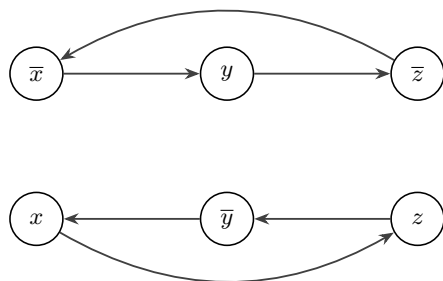
Theorem 13.5.9 ($2SAT \in P$). $2SAT = \{\langle \varphi \rangle \mid \varphi \text{ is a satisfiable 2cnf-formula}\}$ is in P.

Proof. A clause $(\ell_1 \vee \ell_2)$ is equivalent to the two implications $\overline{\ell_1} \Rightarrow \ell_2$ and $\overline{\ell_2} \Rightarrow \ell_1$. Build the *implication graph* D : one node per literal x and $\overline{x}$, one directed edge per implication. Then φ is unsatisfiable iff some variable x has both x and $\overline{x}$ in the same strongly connected component of D (a cycle forcing $x \Rightarrow \overline{x} \Rightarrow x$, so neither value of x works). Conversely, if no variable has this property, list the components in a topological order (every edge of D stays inside a component or goes to a later one), write $p(\ell)$ for the position of the component of ℓ , and make a literal ℓ true iff $p(\ell) > p(\overline{\ell})$; this sets each variable x consistently (as $p(x) \neq p(\overline{x})$), and a clause $(\ell_1 \vee \ell_2)$ with both literals false would give $p(\ell_1) < p(\overline{\ell_1}) \leq p(\ell_2) < p(\overline{\ell_2}) \leq p(\ell_1)$ (the two $\leq$ by its edges $\overline{\ell_1} \rightarrow \ell_2$ and $\overline{\ell_2} \rightarrow \ell_1$), which is impossible. Strongly connected components are computable in linear time, so $2SAT$ is decidable in polynomial time. $\square$

Worked Example 13.5.10 (Implication graph for a 2cnf formula vs. the 3-clause obstruction). Decide $\varphi = (x \vee y) \wedge (\overline{x} \vee z) \wedge (\overline{y} \vee \overline{z})$ by its implication graph, and explain in one line why the same idea fails for a 3-literal clause.

Recognition. The trigger is “width-2 clauses.” Each 2-clause is two implications; satisfiability reduces to a reachability/cycle question (Theorem 13.5.9). A 3-clause does not split into implications between single literals, so the trick has no analogue.

Solution. The clauses give edges $\overline{x} \rightarrow y, \overline{y} \rightarrow x, x \rightarrow z, \overline{z} \rightarrow \overline{x}, y \rightarrow \overline{z}, z \rightarrow \overline{y}$.



No variable has x and $\bar{x}$ in one strongly connected component (the only cycles, if any, stay among consistent literals), so φ is satisfiable – e.g. $x = 1, y = 0, z = 1$ gives clauses 1, 1, 1. For a 3-clause $(x \vee y \vee z)$, falsifying one literal does not *force* either other one (either of the remaining two may carry the clause), so there is no clean implication between single literals – which is precisely the freedom *3SAT* exploits to encode an entire NP computation.

Which idea, and why? Width 2 makes “not this literal” *force* “that literal,” turning satisfiability into a graph-reachability problem; width 3 removes the forcing, restoring the combinatorial search that Cook–Levin shows is NP-complete.

Check. The assignment $x = 1, y = 0, z = 1$ satisfies all three 2-clauses, matching the “satisfiable” verdict from the absence of a contradictory component.

Method (How to decide how to cope with an NP-complete problem).

1. **Confirm the hardness.** Exhibit a reduction $B \leq_p$ (your problem) from a known NP-complete B (Proposition 13.5.3). If you cannot, look harder for a polynomial algorithm before coping.
2. **Inspect the instances you actually face.** Are they planar, bounded-treewidth, width-2, or otherwise special? A restricted family may live in P outright (e.g. *2SAT*, Theorem 13.5.9). If so, stop – you do not need to cope.
3. **Find a small secondary parameter k .** Solution size, number of clauses, treewidth. If running time $f(k) n^{O(1)}$ suffices for your k , build an FPT algorithm (branching, kernelization), as in Theorem 13.5.7.
4. **Decide whether approximate is good enough.** If a provable factor ρ is acceptable, use an approximation algorithm (Theorem 13.5.5, Chapter 15). State the ratio and remember it is worst-case.
5. **Else: exact-exponential or heuristic.** Small inputs $\Rightarrow$ a tuned exponential exact solver. Large inputs with no guarantee needed $\Rightarrow$ heuristics / local search, validated empirically. Optionally lean on average-case behaviour.

Caution. An approximation or heuristic does *not* put the problem in P. “*VERTEX-COVER* has a polynomial 2-approximation” is a statement about an *easier, approximate* task; the exact decision problem *VERTEX-COVER* remains NP-complete, hence outside P unless $P = NP$. Likewise an FPT algorithm running in $O(2^k n)$ is polynomial only when k is treated as a constant – let k grow linearly with n and it is exponential again. And every ratio guarantee like “factor 2” is *worst-case*: it bounds the gap on the nastiest instance, not the typical one, and it certainly does not claim the optimum was found.

Exercises.

1. Draw the directed graph of $\leq_p$ arrows for the problems

3SAT, CLIQUE, INDEP-SET, VERTEX-COVER, HAMPATH, UHAMPATH, TSP,

putting exactly one arrow per standard textbook reduction. For each arrow write a one-phrase label naming the gadget or trick (e.g. “complement the graph”). Then circle every node reachable from *3SAT* and state in one sentence why each circled node is NP-complete.

2. For the graph G with vertices $\{1, 2, 3, 4, 5\}$ and edges 12, 23, 34, 45, 51, 13, run the maximal-matching 2-approximation for *VERTEX-COVER*. Show the matching it builds, the cover it outputs, and a minimum cover. Verify the inequality $|C| \leq 2|C^*|$ on this instance and report whether the bound is tight here.

3. Run the FPT branching algorithm of the section on the cycle C_4 (vertices 1–2–3–4–1) with budget $k = 2$. Draw the full recursion tree, mark each leaf as FAIL or as a cover, and confirm the number of leaves is at most 2^k . Does C_4 have a size-2 vertex cover?
4. Build the implication graph for the 2cnf formula $\varphi = (x \vee y) \wedge (\bar{x} \vee \bar{y}) \wedge (x \vee \bar{y})$. Draw it with one labelled node per literal and one directed edge per implication, then decide whether φ is satisfiable by inspecting the strongly connected components. If satisfiable, exhibit a satisfying assignment.
5. Prove the identity used to wire the reduction $INDEP-SET \leq_p VERTEX-COVER$: in any undirected graph $G = (V, E)$, a set $S \subseteq V$ is an independent set if and only if $V \setminus S$ is a vertex cover. Deduce that G has an independent set of size $\geq k$ iff G has a vertex cover of size $\leq |V| - k$.
6. Let f reduce A to B in time $O(n^3)$ and g reduce B to C in time $O(n^2)$. Give an explicit polynomial bound on the time to compute $g(f(w))$ as a function of $|w|$, and identify precisely the step in your argument that uses $|f(w)| = O(|w|^3)$.
7. A classmate writes: “ $VERTEX-COVER$ is fixed-parameter tractable in time $O(2^k n)$, therefore $VERTEX-COVER \in P$.” Find the bug. State exactly which quantity must be held constant for $O(2^k n)$ to be polynomial, and exhibit a family of instances on which k grows so that the running time becomes exponential in n .
8. Find the bug. To prove TSP is NP-complete, a student reduces TSP to $UHAMPATH$, arguing “ $UHAMPATH$ is known hard, so TSP inherits its hardness.” Explain in two sentences why the direction is wrong, and state the reduction that *would* correctly establish TSP ’s NP-hardness.
9. Consider $INDEP-SET$ restricted to graphs that are a disjoint union of triangles (K_3 ’s). Describe a polynomial-time exact algorithm for this restricted family, and explain why this does not contradict the NP-completeness of $INDEP-SET$ on general graphs.
10. The greedy heuristic for $VERTEX-COVER$ repeatedly adds a *maximum-degree* vertex and deletes its incident edges. Construct a small graph (at most 8 vertices, with a diagram) on which this greedy heuristic returns a cover strictly larger than the minimum cover, and report both sizes. (Larger, specially built graphs push this ratio beyond any constant, so the heuristic has no constant-factor guarantee, unlike Theorem 13.5.5.)
11. True or false, with a one-line reason each. (a) If $A \leq_p B$ and $B \in P$ then $A \in P$. (b) A polynomial-time 2-approximation for an NP-complete optimization problem implies $P = NP$. (c) The implication-graph argument that puts $2SAT$ in P also puts $3SAT$ in P . (d) Running time $f(k) \cdot n^c$ with c constant is polynomial in n for each fixed k .
12. Suppose someone exhibits a polynomial-time algorithm that always outputs a vertex cover within a factor 1.5 of optimal, but you are told the exact decision problem $VERTEX-COVER$ is NP-complete. Explain carefully why the existence of the 1.5-approximation does *not* contradict the NP-completeness of $VERTEX-COVER$, distinguishing the optimization task the approximation solves from the decision task that is NP-complete.

Chapter 14

Space Complexity

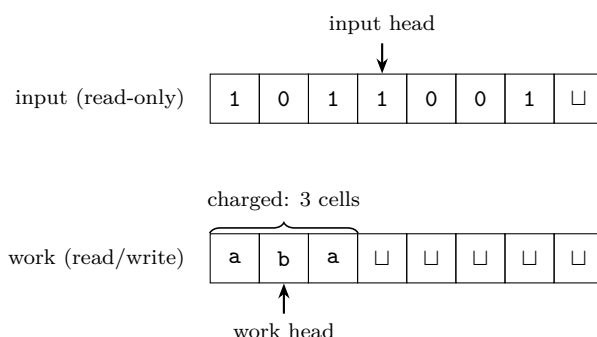
14.1 Space, PSPACE, and Savitch's Theorem

Time measured how *long* a decider runs; this chapter measures how much *memory* it needs. The one key idea is that a Turing machine has a read-only input tape plus a read/write work tape, and *only the work-tape cells are charged* – space, unlike time, can be *reused*, so a problem that is hopeless in polynomial time may still be cheap in polynomial space. We make space precise, collect the bounded languages into SPACE and NSPACE classes and the polynomial layer PSPACE, and then prove Savitch's theorem: a nondeterministic machine that uses $f(n)$ space can be simulated deterministically in only $f(n)^2$ space, which collapses PSPACE and NSPACE into one class.

Space and the space classes

Definition 14.1.1 (Space complexity). Fix a Turing machine with a read-only *input tape* and a read/write *work tape*. Let M be deterministic and halt on every input. The *space complexity* of M is the function $f: \mathbb{N} \rightarrow \mathbb{N}$ where $f(n)$ is the *maximum* number of work-tape cells M scans over all inputs of length n ; the input tape is not counted. We say M *runs in space* $f(n)$. If instead M is nondeterministic and every branch halts on every input, its space complexity $f(n)$ is the maximum number of work-tape cells scanned on *any branch* for any input of length n .

The picture below is the machine model the whole chapter assumes: one head that may only *read* the input, and a separate scratch tape that may be read and written and whose used length is the quantity we charge.



Definition 14.1.2 (Space complexity classes). For $f: \mathbb{N} \rightarrow \mathbb{R}^+$,

$$\text{SPACE}(f(n)) = \{\mathcal{L} \mid \mathcal{L} \text{ is decided by some } O(f(n)) \text{ space deterministic TM}\},$$

$$\text{NSPACE}(f(n)) = \{\mathcal{L} \mid \mathcal{L} \text{ is decided by some } O(f(n)) \text{ space nondeterministic TM}\}.$$

Definition 14.1.3 (PSPACE and NPSPACE).

$$\text{PSPACE} = \bigcup_{k \geq 1} \text{SPACE}(n^k), \quad \text{NPSPACE} = \bigcup_{k \geq 1} \text{NPSPACE}(n^k).$$

These are the languages decidable in *polynomial* space, deterministically and nondeterministically.

Two further bookkeeping facts make space usable. First, on a work tape of length $f(n)$ a machine with c states and g work-tape symbols has only finitely many distinct snapshots; second, counting them bounds how long it can run before a configuration must repeat.

Proposition 14.1.4 (Configuration count). *Let M run in space $f(n) \geq \log n$ with c control states and g work-tape symbols. The number of distinct configurations of M on an input of length n is at most*

$$c \cdot n \cdot f(n) \cdot g^{f(n)} = 2^{O(f(n))}.$$

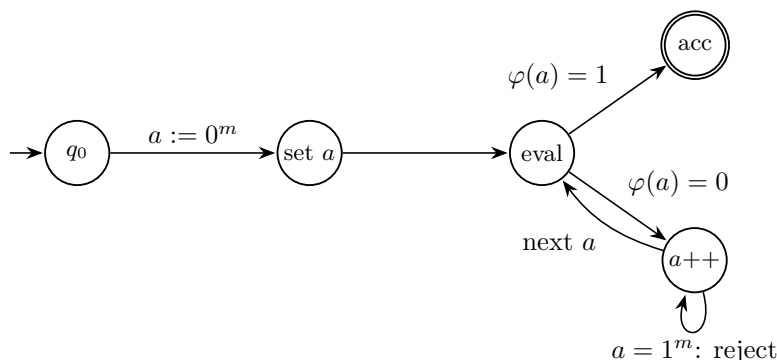
Consequently every halting branch of M runs for at most $2^{O(f(n))}$ steps.

Proof. A configuration (state + work-tape contents + both head positions, but *not* the fixed input) is determined by choosing the state (c ways), the work-tape string ($g^{f(n)}$ ways), the input-head position (n ways), and the work-head position ($f(n)$ ways). Multiplying gives $c n f(n) g^{f(n)}$. Taking logs base 2, this is $\log c + \log n + \log f(n) + f(n) \log g = O(f(n))$ once $f(n) \geq \log n$ (so $\log n = O(f(n))$), hence the count is $2^{O(f(n))}$. A halting computation can never repeat a configuration – a repeat would loop forever – so it makes fewer steps than there are configurations. $\square$

Worked Example 14.1.5 ($\text{SAT} \in \text{SPACE}(n)$ by reusing the assignment tape). Show that $\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula}\}$ lies in $\text{SPACE}(n)$, even though it is NP-complete and believed to need exponential time.

Recognition. The trigger is “try all of an exponential family of objects, but keep only *one* at a time.” Space is reusable, so the cost is the size of a single trial assignment, not the number of assignments.

Solution. Let φ have variables $x_1, \dots, x_m$ with $m \leq n = |\langle \varphi \rangle|$. The decider M_{SAT} keeps a single m -bit assignment on the work tape and sweeps through all 2^m of them like a binary odometer, evaluating φ after each:



Storing a takes $m \leq n$ cells. Evaluating $\varphi(a)$ walks the formula reading bits of a and constants, reusing $O(n)$ scratch cells that are wiped before the next assignment. At no instant are two assignments stored, so the simultaneous work-tape usage is $O(n)$, giving $\text{SAT} \in \text{SPACE}(n)$.

Which idea, and why? Space is the *maximum simultaneous* usage, not the sum over iterations: an exponential search reusing one $O(n)$ -cell slot costs $O(n)$ space.

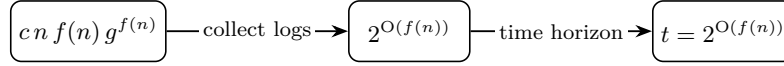
Check. For $\varphi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3)$, the odometer runs $a = 000, 001, \dots, 111$; it stops and accepts at $a = 010$ ($x_1=0, x_2=1, x_3=0$), having ever stored just one 3-bit string.

Worked Example 14.1.6 (Counting configurations to bound the time horizon). A nondeterministic machine N runs in space $f(n) = O(\log n)$ with $c = 8$ states and $g = 4$ work-tape symbols. Give an explicit bound on

the number of steps any accepting branch needs, and identify the value of t that a yieldability search must use.

Recognition. The trigger “how long before a configuration repeats” is exactly Proposition 14.1.4: the configuration count caps the running time.

Solution. With $f(n) = d \log_2 n$ for some constant d , the count is $c n f(n) g^{f(n)} = 8 \cdot n \cdot (d \log_2 n) \cdot 4^{d \log_2 n} = 8 d n \log_2 n \cdot n^{2d} = n^{O(1)}$, a polynomial. So every accepting branch needs at most $t = n^{O(1)}$ steps, and a search testing “does c_{start} reach c_{accept} within t steps” may take $t = 2^{d' f(n)} = n^{O(1)}$.



Which idea, and why? The configuration count $2^{O(f(n))}$ is simultaneously the running-time bound and the step budget t fed to a reachability test – there is no point searching past the last new configuration.

Check. At $f = \log_2 n$ ($d = 1$): count $= 8n \log_2 n \cdot n^2 = O(n^3 \log n)$, polynomial, matching the known fact that log-space machines run in polynomial time.

Savitch’s theorem

The naive way to simulate a nondeterministic $f(n)$ -space machine deterministically is to try its branches one at a time; but recording which branch you are on costs $2^{O(f(n))}$ space, blowing the budget. Savitch’s insight is to ask the *yieldability* question – can configuration c_1 reach c_2 within t steps? – and answer it by splitting at the *midpoint*, which trades a long path for a short recursion stack.

Theorem 14.1.7 (Savitch). *For any $f(n) \geq \log n$,*

$$\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2).$$

Proof. Let N decide $\mathcal{L}$ in space $f(n)$. Modify N so that on accepting it erases the work tape and parks both heads at the left end, giving a unique configuration c_{accept} ; let c_{start} be the start configuration on input w . By Proposition 14.1.4 pick a constant d with at most $2^{df(n)}$ configurations, so $t_0 = 2^{df(n)}$ upper-bounds the steps of any branch. We solve yieldability recursively.

CANYIELD(c_1, c_2, t) – “can N go from c_1 to c_2 in $\leq t$ steps?”

1. If $t = 1$: accept if $c_1 = c_2$ or c_1 yields c_2 in one step of N ; else reject.
2. If $t > 1$: for each configuration c_m on space $f(n)$: run CANYIELD($c_1, c_m, \lceil t/2 \rceil$) and CANYIELD($c_m, c_2, \lceil t/2 \rceil$); if both accept, accept.
3. If no c_m worked, reject.

The simulator M outputs CANYIELD($c_{\text{start}}, c_{\text{accept}}, t_0$).

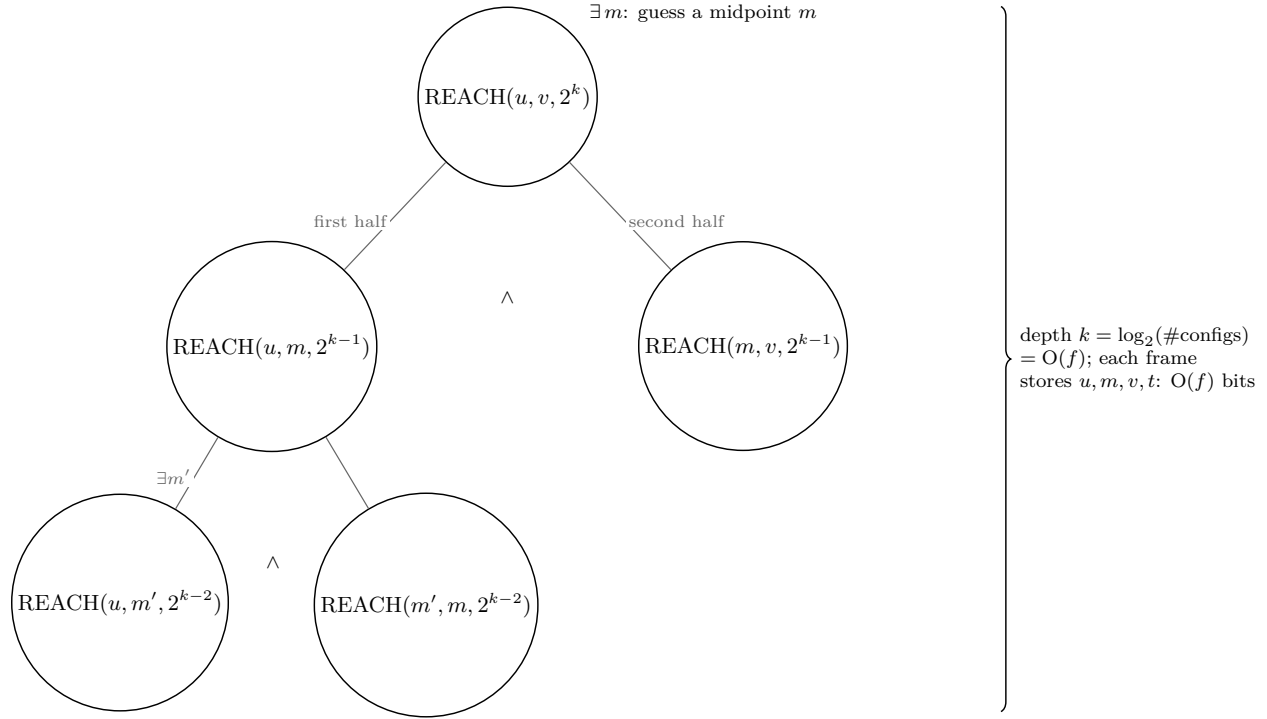
Correctness. N accepts w iff some branch reaches c_{accept} from c_{start} in $\leq t_0$ steps. Taking t_0 to be a power of 2 (round $df(n)$ up to an integer), every $t > 1$ that arises is a power of 2, so $\lceil t/2 \rceil = t/2$, and a path of length $\leq t$ from c_1 to c_2 exists iff there is a midpoint c_m with paths of length $\leq t/2$ on each side; the base case $t = 1$ is decided directly. So CANYIELD returns exactly the yieldability answer, and M accepts iff N does.

Space. Each invocation stores c_1, c_2, t and a loop pointer, all $O(f(n))$ bits ($t \leq t_0$ needs $\log t_0 = O(f(n))$ bits). The two recursive calls run *one at a time* on the same scratch region, so the only growing cost is the stack of suspended frames. Each call halves t , so the depth is $\log_2 t_0 = O(f(n))$. Total space $=$ (depth) $\times$ (frame size) $= O(f(n)) \cdot O(f(n)) = O(f(n)^2)$. Hence $\mathcal{L} \in \text{SPACE}(f(n)^2)$.

Knowing $f(n)$. M uses $f(n)$ to list configurations and to set t_0 . If $f(n)$ cannot be computed in $O(f(n))$ space, M tries $i = \lceil \log_2 n \rceil, \lceil \log_2 n \rceil + 1, \dots$ in its place: it accepts if c_{accept} is reachable through configurations with at most i work cells; it rejects if CANYIELD shows that no configuration with $i + 1$ work cells is reachable from c_{start} (then N never uses more than i cells, so the search was complete); otherwise it moves on to $i + 1$. It stops no later than the first $i \geq f(n)$, so the space stays $O(f(n)^2)$. $\square$

The divide-and-conquer recursion is the heart of the proof. The decisive identity is the *midpoint split*: a path from u to v of length $\leq 2^k$ exists iff some intermediate configuration m is reachable from u and

reaches v , each within half the budget 2^{k-1} . The picture below shows two levels of this binary recursion; the configurations u, v and the guessed midpoint m are the only data a frame holds.

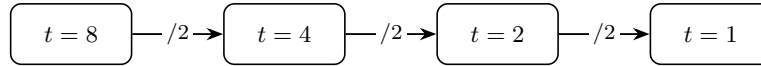


Takeaway. Savitch halves the step budget by guessing a MIDPOINT m : $\text{REACH}(u, v, 2^k)$ holds iff some m has $\text{REACH}(u, m, 2^{k-1})$ and $\text{REACH}(m, v, 2^{k-1})$; the depth- k recursion reuses $O(s)$ bits per frame, so the invariant “reachability within a step budget on $O(s)$ reused space” compounds to $\text{NSPACE}(s) \subseteq \text{SPACE}(s^2)$. *Check.* The recursion has depth $k = O(s)$ (since $\# \text{configs} = 2^{O(s)}$) and each frame stores $O(s)$ bits (u, m, v , and a counter), so total deterministic space is $O(s) \cdot O(s) = O(s^2)$; hence $\text{NSPACE}(s) \subseteq \text{SPACE}(s^2)$.

Worked Example 14.1.8 (Tracing the recursion depth for a concrete t). Take $t_0 = 8$ and trace the sequence of spans CANYIELD pushes onto the stack along one leftmost branch. How many frames are simultaneously live, and how does that scale with $f(n)$?

Recognition. The trigger is “how tall does the stack get”: each call halves t until $t = 1$, so the live-frame count is $\log_2 t_0 + 1$.

Solution. Starting at $t = 8$, the leftmost chain of first-half calls is $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$:



$$4 = \log_2 8 + 1 \text{ live frames, each } O(f) \text{ bits}$$

At the deepest point four frames sit on the stack: spans 8, 4, 2, 1. In general $t_0 = 2^{df(n)}$ gives $\log_2 t_0 + 1 = df(n) + 1 = O(f(n))$ live frames, each $O(f(n))$ bits, for $O(f(n)^2)$ total – exactly the theorem’s bound.

Which idea, and why? Halving t at each level makes the stack depth $\log t$, and $\log t = O(f)$ because t is the exponential configuration count; depth times frame size is the squared bound.

Check. If instead the recursion split off a *single* step at a time, the depth would be $t_0 = 2^{O(f)}$ and the space exponential – the midpoint split is what buys the square.

Corollary 14.1.9 ($\text{PSPACE} = \text{NSPACE}$). $\text{PSPACE} = \text{NSPACE}$.

Proof. $\text{PSPACE} \subseteq \text{NPSpace}$ since a deterministic machine is a nondeterministic one. Conversely let $\mathcal{L} \in \text{NPSpace}$, say $\mathcal{L} \in \text{NPSpace}(n^k)$. By Savitch (Theorem 14.1.7, with $f(n) = n^k \geq \log n$), $\mathcal{L} \in \text{SPACE}((n^k)^2) = \text{SPACE}(n^{2k}) \subseteq \text{PSPACE}$. The square of a polynomial is a polynomial, so nondeterminism buys nothing at the polynomial-space level. $\square$

Worked Example 14.1.10 ($\text{NPSpace}(n)$ lands inside $\text{SPACE}(n^2)$). A language $\mathcal{L}$ is decided by a nondeterministic machine in $O(n)$ space – for instance the complement of “an NFA accepts every string,” which guesses a rejected string while tracking the active-state set. What deterministic space bound does Savitch guarantee, and why does this put $\mathcal{L}$ in PSPACE ?

Recognition. The trigger “convert nondeterministic space to deterministic space” is precisely Theorem 14.1.7: feed $f(n)$ in, read $f(n)^2$ out.

Solution. With $f(n) = n$, Savitch gives $\mathcal{L} \in \text{SPACE}(n^2)$. Tracking which subset of a q -state NFA is active needs $q \leq n$ marker bits, and the guessed symbol plus a step counter add $O(n)$ more – nondeterministic linear space. Determinizing by midpoint recursion squares it to $O(n^2)$:



Since n^2 is a polynomial, $\text{SPACE}(n^2) \subseteq \text{PSPACE}$, so $\mathcal{L} \in \text{PSPACE}$.

Which idea, and why? Savitch squares the space bound; squaring a polynomial keeps it polynomial, so the whole nondeterministic-polynomial-space world collapses into PSPACE .

Check. The same machine run as time would be exponential ($2^{O(n)}$ steps), yet in space it is merely quadratic – the qualitative difference space-reuse makes.

Method (How to bound a problem’s space by configuration-graph reachability).

1. **Name the configurations.** A configuration is state + work-tape contents + both head positions (the read-only input is *not* included). Its size is $O(f(n))$ bits.
2. **Bound how many there are.** By Proposition 14.1.4 the count is $2^{O(f(n))}$; this is also the step budget t , since no halting branch repeats a configuration.
3. **Phrase acceptance as reachability.** “ N accepts w ” becomes “ c_{accept} is reachable from c_{start} within t steps” in the configuration graph.
4. **Search space-efficiently.** Do not list the graph (that is exponential). Use the midpoint recursion CANYIELD: depth $\log t = O(f(n))$, each frame $O(f(n))$ bits.
5. **Read off the bound.** Depth $\times$ frame size = $O(f(n)^2)$ deterministic space; conclude $\mathcal{L} \in \text{SPACE}(f(n)^2)$.

Caution. Charge *only the work tape*. The input occupies n cells but lives on a read-only tape, so a machine can be in $\text{SPACE}(\log n)$ while the input is linear – never count input cells toward the space bound. (This is why sublinear space is meaningful at all.)

Caution. Space is *reused*, so the cost is the *maximum simultaneous* work-tape length, not the *sum* over the run. The *SAT* machine visits 2^m assignments but stores one at a time, so it is $\text{SPACE}(n)$, not $\text{SPACE}(2^m)$. In Savitch the two recursive calls share one scratch region precisely so their spaces are taken as a maximum, not added; only the *stack* of suspended frames grows, and that is what the $\log t$ depth controls.

Exercises.

1. Using Definition 14.1.1 directly, explain why the language $D = \{0^k 1^k \mid k \geq 0\}$ is in $\text{SPACE}(\log n)$: describe a two-tape decider that keeps two binary counters on the work tape instead of crossing symbols off the (read-only) input, and state the work-tape length as a function of n .
2. Let $\mathcal{L}$ be decided by a deterministic machine that, on every input of length n , scans at most $5 \log_2 n + 12$ work-tape cells. Name the smallest standard class among L , NL , PSPACE (Definition 14.4.2) containing $\mathcal{L}$, and justify the choice in one sentence using Definition 14.1.2.

3. Consider $MAJ = \{\langle \varphi \rangle \mid \varphi \text{ is true for a strict majority of its } 2^m \text{ assignments}\}$, where φ has m variables. Describe a deterministic machine deciding MAJ in $SPACE(n)$ by sweeping all assignments while keeping just an assignment register and a binary tally counter. Explain precisely which two quantities the work tape holds and why their total length is $O(n)$.
4. Draw the configuration graph (as a directed graph, using labelled vertices and edges) of a tiny machine with exactly four configurations $c_{\text{start}}, a, b, c_{\text{accept}}$ where c_{start} may step to a or b , a steps to c_{accept} , and b steps back to c_{start} . Mark the start and accept nodes and state whether the machine accepts (is c_{accept} reachable?).
5. Apply Savitch's theorem (Theorem 14.1.7) to each bound below, writing the deterministic space guarantee and naming the value of $f(n)$ you used: (a) $NSPACE(n^3)$; (b) $NSPACE(n \log n)$; (c) $NSPACE(\log n)$. For (c), state the usual name of the class $NSPACE(\log n)$ (Definition 14.4.2).
6. Trace CANYIELD from the proof of Theorem 14.1.7 with step budget $t_0 = 4$. List, in the order they are pushed and popped, the sequence of (c_1, c_2, t) spans along the *first* complete root-to-leaf-to-return path, and state the maximum number of frames simultaneously on the stack.
7. For a nondeterministic machine running in space $f(n) = 2 \log_2 n$ with $c = 6$ states and $g = 3$ work-tape symbols, use Proposition 14.1.4 to give an explicit polynomial upper bound on the number of configurations, and hence on the step budget t that CANYIELD should be called with.
8. Prove that $NSPACE(\log n) \subseteq SPACE(\log^2 n)$ as a special case of Savitch's theorem, and verify the hypothesis $f(n) \geq \log n$ is met. Then explain in one sentence why this does *not* immediately tell us whether $L = NL$.
9. Let $\mathcal{L}$ be decided in $O(\sqrt{n})$ nondeterministic space. Give the best deterministic space bound Savitch provides, and decide whether $\mathcal{L} \in PSPACE$; justify with the polynomial-closure argument used in Corollary 14.1.9.
10. Find the bug. A student argues: "A machine that loops through all 2^n binary strings of length n , writing each in turn onto its work tape and checking a property, uses $SPACE(2^n)$ because it processes 2^n strings." Identify the error in this space claim, state the correct space bound, and name the principle from the chapter that resolves it.
11. Find the bug. A proof of Savitch's theorem replaces the midpoint recursion with: "test each successor c' of c_1 and recurse on $CANYIELD(c', c_2, t - 1)$." Explain why this version still computes yieldability correctly but *fails* the space bound, and give the recursion depth it actually achieves in terms of $t_0 = 2^{O(f(n))}$.
12. Using $P \subseteq PSPACE$ (a machine running in time $t(n)$ scans at most $t(n)$ cells) and Corollary 14.1.9, arrange the classes $P, NP, PSPACE, NSPACE$ into the known chain of containments, marking which containments are equalities and which are not known to be proper. Add one sentence on why $NP \subseteq PSPACE$ holds.

14.2 PSPACE-Completeness and TQBF

Savitch's theorem collapsed $NSPACE$ into $PSPACE$; this section asks the opposite kind of question. Among all problems solvable in polynomial space, which are the *hardest* – the ones that capture the full power of the class? Exactly as NP has its complete problems, $PSPACE$ has its own, and the prototype is $TQBF$, the truth problem for fully quantified Boolean formulas. The single idea that makes the chapter's theorem work is the Savitch-style *midpoint* trick: a quantifier that names the middle configuration of a computation and a universal quantifier that folds two recursive halves into one, keeping the formula polynomial instead of exponential.

Poly-time reducibility and PSPACE-completeness

We measure hardness inside $PSPACE$ with the *poly-time* mapping reduction $\leq_p$ from the time-complexity chapter: $A \leq_p B$ means there is a polynomial-time computable f with $w \in A \iff f(w) \in B$ for every string w . A reduction we are allowed to call "easy" must be weaker than the class it sorts; since $PSPACE$

already contains everything decidable in polynomial *space*, the reduction must be limited in *time*, hence $\leq_p$ rather than a space-bounded notion.

Definition 14.2.1 (PSPACE-hard, PSPACE-complete). A language B is *PSPACE-hard* if every $A \in \text{PSPACE}$ satisfies $A \leq_p B$. It is *PSPACE-complete* if in addition $B \in \text{PSPACE}$.

A PSPACE-complete B is a single problem into which all of PSPACE collapses under $\leq_p$: a polynomial-time – indeed a polynomial-space – algorithm for B would give one for every language in the class. The reason $\leq_p$ is the right tool is below.

Proposition 14.2.2 (Why $\leq_p$, not space-reducibility). *If $B \in \text{PSPACE}$ and $A \leq_p B$, then $A \in \text{PSPACE}$. Consequently a PSPACE-complete language lies in P iff $\text{P} = \text{PSPACE}$.*

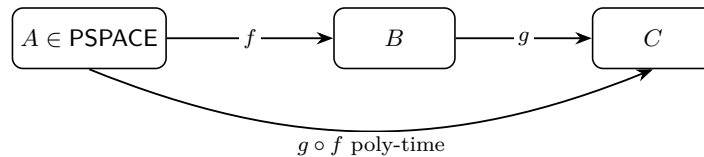
Proof. Let f be a poly-time reduction from A to B , computable in time n^a , so $|f(w)| \leq n^a$, and let B be decided in space m^b . On input w of length n , compute $f(w)$ (this uses at most n^a tape cells, since the machine writes one cell per step) and run the B -decider on it, using space $(n^a)^b = n^{ab}$. The total is polynomial, so $A \in \text{PSPACE}$. For the second claim: if a PSPACE-complete B were in P, then every $A \in \text{PSPACE}$ would reduce to B and be decided by “compute $f(w)$, run the poly-time B -decider,” a polynomial-time algorithm, giving $\text{PSPACE} \subseteq \text{P}$; the reverse inclusion $\text{P} \subseteq \text{PSPACE}$ always holds. $\square$

Remark. A *space*-bounded reduction, by contrast, would be useless here: a polynomial-space reduction can decide A outright and then output a fixed member or a fixed non-member of B , so *every* nontrivial language in PSPACE would reduce to every other, and “complete” would lose all meaning. The reduction must be strictly weaker than the class.

Worked Example 14.2.3 (Reductions chain: hardness propagates). Suppose B is PSPACE-complete, $B \leq_p C$, and $C \in \text{PSPACE}$. Show C is PSPACE-complete.

Recognition. The trigger is “a known-hard problem reduces *into* a new one.” Hardness flows along $\leq_p$ exactly as for NP-completeness, because $\leq_p$ is transitive.

Solution. Take any $A \in \text{PSPACE}$. Since B is PSPACE-hard, $A \leq_p B$; combine with the given $B \leq_p C$. Composing the two poly-time maps f (for $A \leq_p B$) and g (for $B \leq_p C$) gives $g \circ f$, computable in polynomial time (a polynomial of a polynomial is a polynomial), with $w \in A \iff g(f(w)) \in C$. Hence $A \leq_p C$. As A was arbitrary, C is PSPACE-hard, and $C \in \text{PSPACE}$ is given, so C is PSPACE-complete.



Which idea, and why? $\leq_p$ is transitive and closed under composition, so a single known-hard seed B stamps hardness onto every problem it reduces into – the engine behind every completeness proof.

Check. If f runs in time n^a and g in time n^c , then on w of length n the string $f(w)$ has length $\leq n^a$, so $g(f(w))$ costs $\leq (n^a)^c = n^{ac}$: still polynomial.

Quantified Boolean formulas and TQBF

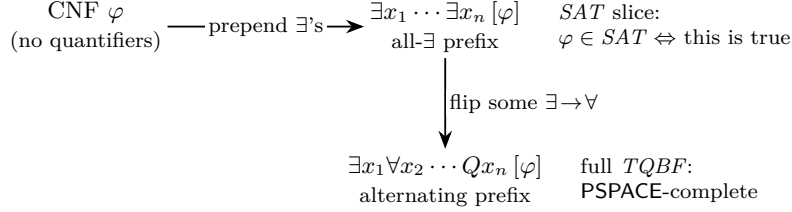
A *Boolean formula* is built from variables $x_1, x_2, \dots$, the constants 0, 1, and $\wedge, \vee, \neg$. A *quantified Boolean formula* prepends quantifiers $\exists x$ and $\forall x$, ranging over the universe $\{0, 1\}$: the statement $\exists x \psi$ is true iff ψ is true for $x = 0$ or $x = 1$, and $\forall x \psi$ is true iff ψ is true for $x = 0$ and $x = 1$. A formula is *fully quantified* (a *sentence*) when every variable lies in the scope of a quantifier; such a formula has no free variables and is simply true or false. We assume *prenex* form: all quantifiers up front, $Q_1 x_1 Q_2 x_2 \dots Q_k x_k [\psi]$ with ψ quantifier-free.

Definition 14.2.4 (TQBF).

$$\text{TQBF} = \{ \langle \phi \rangle \mid \phi \text{ is a true fully quantified Boolean formula} \}.$$

Ordinary satisfiability is the special case where every quantifier is $\exists$: $\langle \exists x_1 \cdots \exists x_m [\psi] \rangle \in TQBF$ iff $\psi \in SAT$. The universal quantifier and the *alternation* of the two are what give *TQBF* its extra muscle.

The structure diagram below makes the embedding explicit: a bare CNF φ acquires an all- $\exists$ prefix to become a *TQBF* sentence – the special case in which no $\forall$ appears – and switching one or more $\exists$'s to $\forall$ (genuine *alternation*) is the move that turns the NP-complete problem *SAT* into the PSPACE-complete problem *TQBF*.

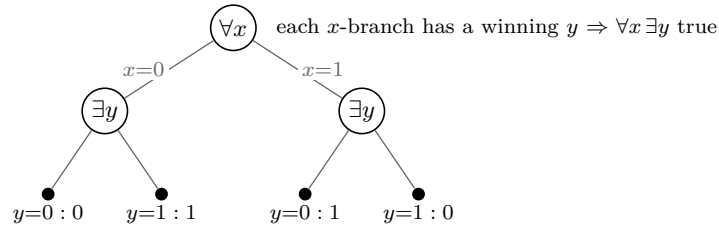


Takeaway. *TQBF* generalizes *SAT* by allowing quantifier alternation: a CNF φ is satisfiable iff the *TQBF* sentence $\exists x_1 \cdots \exists x_n [\varphi]$ is true, so *SAT* is the all- $\exists$ special case; adding $\forall$ quantifiers lifts the complete problem from NP (*SAT*) up to PSPACE (*TQBF*). The invariant preserved by the top map is truth/satisfiability itself. *Check.* $\varphi \in SAT$ iff $\exists x_1 \cdots \exists x_n [\varphi]$ is a true (alternation-free) *TQBF*; allowing $\forall$ yields the full, PSPACE-complete *TQBF* – these being the complete problems for the two classes.

Worked Example 14.2.5 (Order of quantifiers changes the truth value). Decide the truth of $\phi = \forall x \exists y [(x \vee y) \wedge (\neg x \vee \neg y)]$ and of the swapped sentence $\phi' = \exists y \forall x [(x \vee y) \wedge (\neg x \vee \neg y)]$.

Recognition. The matrix $(x \vee y) \wedge (\neg x \vee \neg y)$ is exactly “ $x \neq y$ ” (XOR). The trigger is “ $\forall\exists$ vs. $\exists\forall$ ”: in the first the responder y sees x first; in the second y is fixed before x moves.

Solution. Expand the matrix as a tiny truth table: it is 1 on $(x, y) \in \{(0, 1), (1, 0)\}$ and 0 on $(0, 0), (1, 1)$. For ϕ : given any x , pick $y = \neg x$; then $x \neq y$ and the matrix is 1. Both $x = 0$ and $x = 1$ are answerable, so ϕ is **true**. For ϕ' : now y is chosen *first*, then x ranges over both values. Whatever single y we commit to, the adversary sets $x = y$, forcing the matrix to 0. So no fixed y works and ϕ' is **false**.



Which idea, and why? A quantifier read left to right is a turn in a game: $\exists$ is “our” move, $\forall$ is “their” move. Swapping $\forall\exists$ to $\exists\forall$ forces our move *earlier*, stripping away the information that made the formula true.

Check. XOR has exactly one good y for each x – enough for $\forall x \exists y$, not enough for a single y to beat both x , matching “true” then “false.”

The theorem

Theorem 14.2.6 (*TQBF* is PSPACE-complete). *TQBF* is PSPACE-complete.

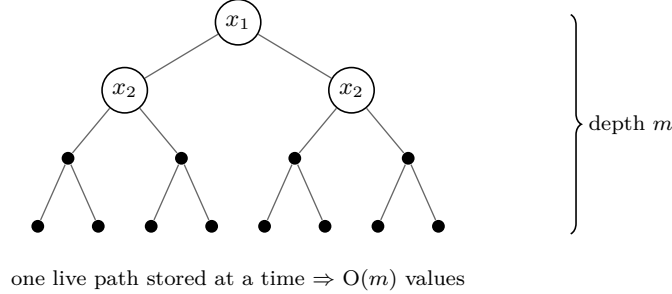
The proof has two halves: membership (an algorithm that decides *TQBF* in polynomial space) and hardness (a poly-time reduction from an arbitrary poly-space machine). We do membership first because its recursion is the same shape as the one that controls the size of the hard-reduction formula.

Membership: $TQBF \in PSPACE$. Strip the outer quantifier and recurse on the two Boolean values, reusing space. Define a recursive evaluator T :

$T(\phi)$:

- if ϕ has no quantifier, evaluate the constant expression and return its value;
- if $\phi = \exists x \psi$, return $T(\psi[x:=0]) \vee T(\psi[x:=1])$;
- if $\phi = \forall x \psi$, return $T(\psi[x:=0]) \wedge T(\psi[x:=1])$.

Correctness is the definition of the quantifiers. For space: the recursion depth equals the number m of variables, and each frame stores just one substituted value (a single bit), while evaluating the quantifier-free matrix at a leaf takes $O(|\langle\phi\rangle|)$ further cells. Crucially, the two recursive calls run *sequentially* and reuse the same work cells, so the total space is depth $\times$ frame + evaluation = $O(m + |\langle\phi\rangle|)$, polynomial (indeed linear) in the input. The picture is a depth-first walk of the quantifier tree, never holding more than one root-to-leaf path:



Hence $TQBF \in \text{SPACE}(n) \subseteq \text{PSPACE}$. □

Hardness: every $A \in \text{PSPACE}$ reduces to $TQBF$. Let A be decided by a deterministic TM M in space n^k . On input w with $|w| = n$ we build, in polynomial time, a sentence ϕ with $\phi \in TQBF \iff M$ accepts w . A configuration of M on w uses $O(n^k)$ cells, so it is encoded by a block of $\ell = O(n^k)$ Boolean variables (one group of variables per cell, recording its symbol and any head/state marker), exactly as in the Cook–Levin tableau.

The naive tableau fails. Following Cook–Levin directly, one writes a formula asserting the existence of a valid accepting tableau: rows are successive configurations, and adjacency constraints tie each cell to the three cells above it. The width is $O(n^k)$, but M may run for $2^{O(n^k)}$ steps, so the tableau has *exponentially many rows*. A direct formula would have size $2^{O(n^k)}$, and a poly-time reduction cannot emit an exponentially long string. We need a formula whose size tracks the *logarithm* of the running time.

The midpoint recursion. For configurations c_1, c_2 (each a block of ℓ variables) and a step bound t , define a formula $\phi_{c_1, c_2, t}$ that is true exactly when M can go from c_1 to c_2 in at most t steps. The base case $t = 1$ is a fixed, polynomial-size Boolean expression saying “ $c_1 = c_2$, or c_2 follows from c_1 in one M -step” (the local window constraints of Cook–Levin); call its size $s_0 = O(n^k)$.

A first, *broken*, recursion uses only $\exists$ on a midpoint m :

$$\phi_{c_1, c_2, t} = \exists m \left[\phi_{c_1, m, t/2} \wedge \phi_{m, c_2, t/2} \right].$$

It is correct – some midpoint splits a $\leq t$ -step run into two $\leq t/2$ runs – but each level *doubles* the formula because it writes *two* copies of the recursive subformula. After $\log t$ levels the size is $\sim 2^{\log t} = t = 2^{O(n^k)}$: exponential again.

The fix – a universal quantifier that folds the two halves. Introduce fresh configuration blocks c_3, c_4 and quantify universally over the two pairs (c_1, m) and (m, c_2) :

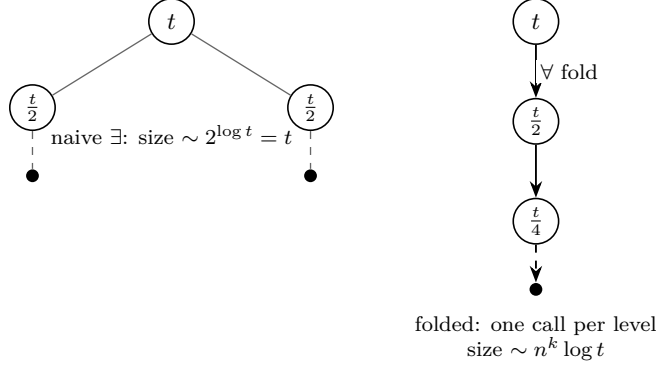
$$\phi_{c_1, c_2, t} = \exists m \forall (c_3, c_4) \in \{(c_1, m), (m, c_2)\} \left[\phi_{c_3, c_4, t/2} \right].$$

Now the recursive subformula $\phi_{c_3, c_4, t/2}$ appears *once*. The macro $\forall (c_3, c_4) \in \{(c_1, m), (m, c_2)\} [\dots]$ desugars to the legal quantified formula $\forall c_3 \forall c_4 \left[((c_3, c_4) = (c_1, m) \vee (c_3, c_4) = (m, c_2)) \rightarrow \dots \right]$, where Boolean equality and $\rightarrow$ expand into $\wedge, \vee, \neg$. Each level therefore adds only a configuration-sized chunk – $O(n^k)$ new variables and clauses – on top of *one* recursive call.

Size count. Let $S(t)$ be the size of $\phi_{c_1, c_2, t}$. The folded recursion gives

$$S(t) = S(t/2) + O(n^k), \quad S(1) = s_0 = O(n^k),$$

so $S(t) = O(n^k) \cdot \log t$. The depth-of-recursion is $\log t$, and the picture below contrasts the two recurrences:



Assembling ϕ . Modify M so its accepting configuration is unique (clear the tape, park the head left): call it c_{acc} , and let c_{start} be the start configuration on w . Choose d with 2^{dn^k} an upper bound on the number of configurations, hence on the running time, and set $h = 2^{dn^k}$ (a power of 2). Take

$$\phi = \phi_{c_{\text{start}}, c_{\text{acc}}, h}.$$

Then ϕ is true iff M reaches c_{acc} from c_{start} in $\leq h$ steps, i.e. iff M accepts w . Its size is $S(h) = O(n^k) \cdot \log h = O(n^k) \cdot O(n^k) = O(n^{2k})$: polynomial. The text of ϕ is produced by unrolling the recursion $\log h = O(n^k)$ levels, each emitting an $O(n^k)$ chunk, all in polynomial time. Hence $A \leq_p TQBF$, and since $A \in \text{PSPACE}$ was arbitrary, $TQBF$ is PSPACE-hard. With membership, it is PSPACE-complete. $\square$

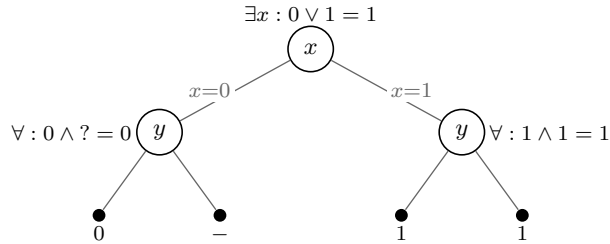
Worked Example 14.2.7 (Evaluate a small $TQBF$ by the recursion). Run the membership algorithm T on $\phi = \exists x \forall y [(x \vee y) \wedge (x \vee \neg y)]$ and report true/false, tracking the space used.

Recognition. The trigger is “a fully quantified sentence to decide”: strip the leftmost quantifier, recurse on 0 and 1, combine with $\vee$ for $\exists$ and $\wedge$ for $\forall$. The matrix simplifies to $x \vee (y \wedge \neg y) = x$.

Solution. Outer is $\exists x$, so we need at least one of $x = 0, x = 1$ to make the $\forall y$ body true.

- $x = 0$: body is $\forall y [(0 \vee y) \wedge (0 \vee \neg y)] = \forall y [y \wedge \neg y]$. At $y = 0$: $0 \wedge 1 = 0$. The $\forall$ already fails, return 0.
- $x = 1$: body is $\forall y [(1) \wedge (1)] = \forall y [1]$. At $y = 0$: 1; at $y = 1$: 1. Both hold, $\forall$ returns 1.

The $\exists$ combines $0 \vee 1 = 1$, so ϕ is **true**.



The $-$ leaf is never visited: once $y = 0$ falsifies the $\forall$, the algorithm short-circuits and abandons the $x = 0$ branch, illustrating the depth-first reuse of space.

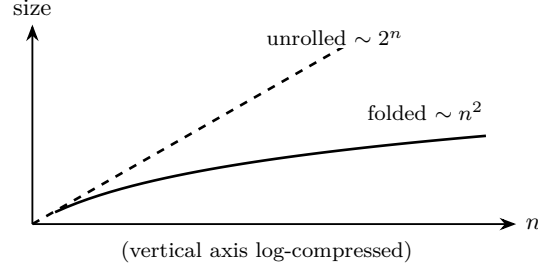
Which idea, and why? T is the quantifier game played out: $\exists$ wins if *some* child wins ($\vee$), $\forall$ wins only if *all* children win ($\wedge$); a single live root-to-leaf path is all the memory required.

Check. The matrix is $x \vee (y \wedge \neg y) = x$, so $\phi = \exists x \forall y [x] = \exists x [x]$, true at $x = 1$ – agreeing with the trace.

Worked Example 14.2.8 (Why the unfolded formula is exponential, and the midpoint fix). A machine M on input of length n runs for up to $h = 2^n$ steps using $\ell = O(n)$ configuration variables. Compare the size of the reachability formula built *without* the midpoint trick (unroll every step) against the folded $\phi_{c_1, c_2, t}$.

Recognition. The trigger is “express a length- h computation as a formula.” Writing the whole trajectory $c_{\text{start}} \rightarrow c_1 \rightarrow \dots \rightarrow c_h$ needs one configuration block per step; the midpoint recursion replaces “one block per *step*” with “one block per *level*,” and there are only $\log h$ levels.

Solution. Unrolled. Quantify all $h + 1$ intermediate configurations and chain single-step checks: $\exists c_0 \exists c_1 \dots \exists c_h [c_0 = c_{\text{start}} \wedge \bigwedge_i \text{Step}(c_i, c_{i+1}) \wedge c_h = c_{\text{acc}}]$. Size = $\Theta(h \cdot \ell) = \Theta(2^n \cdot n)$ – exponential, impossible for a poly-time reduction to print. *Folded.* The recurrence $S(t) = S(t/2) + O(\ell)$ from $t = h$ down to $t = 1$ has $\log h = n$ levels, each adding $O(\ell) = O(n)$, so $S(h) = O(n \cdot n) = O(n^2)$ – polynomial.



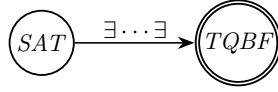
Which idea, and why? One configuration block *per step* makes size scale with $h = 2^n$; the midpoint $\exists m$ with the universal fold makes size scale with $\log h = n$, the depth of the divide-and-conquer – the same idea that turns Savitch’s $2^{O(f)}$ time into $O(f^2)$ space.

Check. At $n = 20$: the unrolled formula has size $\sim 2^{20} \cdot 20 \approx 2 \cdot 10^7$; the folded one $\sim 20^2 = 400$. The reduction must run in poly time, so only the second is admissible.

Worked Example 14.2.9 (*SAT* and *TQBF*: the all- $\exists$ slice). Show that $\langle \psi \rangle \in \text{SAT}$ iff $\langle \exists x_1 \dots \exists x_m [\psi] \rangle \in \text{TQBF}$, and explain why this does *not* make *TQBF* no harder than *SAT*.

Recognition. The trigger is “satisfiability is existence of an assignment,” which is precisely a block of $\exists$ quantifiers. The catch: *TQBF* also allows $\forall$ and alternation, which *SAT* has no way to express.

Solution. The fully existential sentence $\exists x_1 \dots \exists x_m [\psi]$ is true iff some assignment to $x_1, \dots, x_m$ makes ψ true, which is the definition of $\psi \in \text{SAT}$. So $\text{SAT} \leq_p \text{TQBF}$ by the identity-style map $\psi \mapsto \exists x_1 \dots \exists x_m [\psi]$.



only $\exists$ $\exists$ and $\forall$, alternating

But the reduction goes *one way*. *SAT* is NP-complete and *TQBF* is PSPACE-complete; a reduction $\text{TQBF} \leq_p \text{SAT}$ would put $\text{PSPACE} \subseteq \text{NP}$, which is not believed. The expressive jump is the $\forall$ quantifier and the alternation of Worked Example 14.2.5 (ϕ vs. ϕ'): those cannot be flattened into a single existential search.

Which idea, and why? *SAT* is the one-quantifier-type fragment of *TQBF*; the leap to full PSPACE-hardness comes precisely from *alternating* $\exists$ and $\forall$, which encode the adversarial back-and-forth of an exponentially long computation.

Check. On $\psi = (x_1 \vee \neg x_2)$: $\exists x_1 \exists x_2 [\psi]$ is true (take $x_1 = 1$), matching $\psi \in \text{SAT}$. Prefixing $\forall x_1$ instead gives $\forall x_1 \exists x_2 [x_1 \vee \neg x_2]$, true via $x_2 = 0$ – a different question that *SAT* cannot pose.

Method (Build $\phi_{c_1, c_2, t}$ by recursion on t).

1. **Fix the encoding.** A configuration of the n^k -space machine M is a block of $\ell = O(n^k)$ Boolean variables (symbol/state/head per cell). Pick $h = 2^{dn^k}$, a power of 2 bounding M ’s running time.
2. **Base $t = 1$.** Write $\phi_{c_1, c_2, 1} = “c_1 = c_2 \text{ or } c_2 \text{ follows from } c_1 \text{ in one step}”$ via the Cook–Levin local windows; size $O(n^k)$.
3. **Recurse with a midpoint and a fold.** For $t > 1$, introduce a fresh midpoint block m and fresh pair (c_3, c_4) , and write $\phi_{c_1, c_2, t} = \exists m \forall (c_3, c_4) \in \{(c_1, m), (m, c_2)\} [\phi_{c_3, c_4, t/2}]$. The $\forall$ makes the recursive subformula appear *once*.
4. **Desugar.** Replace $\forall (c_3, c_4) \in \{(y, z), (u, v)\} [\Phi]$ by $\forall c_3 \forall c_4 [((c_3, c_4) = (y, z) \vee (c_3, c_4) = (u, v)) \rightarrow \Phi]$, then expand $=, \rightarrow$ into $\wedge, \vee, \neg$.

5. **Top-level.** Output $\phi = \phi_{c_{\text{start}}, c_{\text{acc}}, h}$. Size $S(h) = O(n^k) \cdot \log h = O(n^{2k})$; the unrolling runs in polynomial time.

Caution. Using only the existential midpoint, $\phi_{c_1, c_2, t} = \exists m [\phi_{c_1, m, t/2} \wedge \phi_{m, c_2, t/2}]$, is *correct but fatal*: writing both halves explicitly doubles the formula at every level, so after $\log h$ levels the size is $\sim h = 2^{O(n^k)}$ and the “reduction” cannot finish in polynomial time. The universal quantifier is not a cosmetic rewrite – it is the device that collapses the two halves into one copy, turning a doubling recurrence $S(t) = 2S(t/2) + O(n^k)$ into an additive one $S(t) = S(t/2) + O(n^k)$.

Caution. Alternation is the source of PSPACE-hardness, so do not casually reorder or merge quantifiers. As the swapped-XOR example shows, $\forall x \exists y$ and $\exists y \forall x$ can disagree; a single block of $\exists$ ’s (as in *SAT*) sits inside NP, and only when $\exists$ and $\forall$ genuinely alternate do you climb to the full class. When you encode a game or a computation, keep each player’s moves in their own quantifier and in the right *order*.

Exercises.

- Evaluate each fully quantified Boolean sentence as true or false, and draw the quantifier tree (root-to-leaf, $\exists$ as “some child,” $\forall$ as “all children”): (a) $\forall x \exists y [(x \vee y) \wedge (\neg x \vee \neg y)]$; (b) $\exists x \forall y [(x \vee y) \wedge (\neg x \vee \neg y)]$; (c) $\forall x \forall y \exists z [(x \vee y \vee z) \wedge (\neg x \vee \neg y \vee \neg z)]$.
- Run the membership algorithm T from this section on $\phi = \exists x \forall y [(\neg x \vee y) \wedge (x \vee \neg y)]$. Show the four leaf evaluations, state which branch short-circuits, and give the final value. Then simplify the matrix algebraically and confirm your answer.
- For each formula below, swap the first two quantifiers and decide whether the truth value changes; explain in one sentence using the “move first vs. move second” reading. (a) $\exists x \forall y [x \vee \neg y]$; (b) $\forall x \exists y [x = y]$ (with $=$ meaning Boolean equality).
- Let $\psi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$. Write the fully existential sentence $\Phi = \exists x_1 \exists x_2 \exists x_3 [\psi]$, decide whether $\langle \Phi \rangle \in TQBF$, and state precisely what this says about ψ and *SAT*.
- Prove that PSPACE is closed under $\leq_p$: if $A \leq_p B$ and $B \in \text{PSPACE}$, then $A \in \text{PSPACE}$. Be explicit about the length bound $|f(w)| \leq \text{poly}(|w|)$ and where you use it in the space accounting.
- Suppose B is PSPACE-complete and $C \in \text{PSPACE}$ with $B \leq_p C$. Show C is PSPACE-complete. Draw the reduction-composition diagram (a box for an arbitrary $A \in \text{PSPACE}$, arrows $A \rightarrow B \rightarrow C$, and the composite $A \rightarrow C$).
- Let $h = 2^m$. Define $S(t)$ to be the size of the folded reachability formula $\phi_{c_1, c_2, t}$, with $S(1) = cn^k$ and $S(t) = S(t/2) + cn^k$. Solve the recurrence in closed form, then do the same for the broken existential-only recurrence $S'(t) = 2S'(t/2) + cn^k$, $S'(1) = cn^k$. State both $S(h)$ and $S'(h)$ as functions of m, n, k , and explain which one a poly-time reduction may produce.
- Find the bug. A student proposes this reduction showing *TQBF* is PSPACE-hard: “Given a poly-space machine M and input w , build the Cook–Levin tableau formula whose rows are the successive configurations of M on w , with one existential configuration block per row, and output it.” Identify exactly where the construction violates the requirements of a poly-time reduction, and name the device that repairs it.
- Find the bug. A student writes the recursive reachability formula as $\phi_{c_1, c_2, t} = \exists m [\phi_{c_1, m, t/2} \wedge \phi_{m, c_2, t/2}]$ and claims “this is polynomial size because each level only halves t .” Explain why the size is in fact exponential, write the correct recurrence for this construction, and give the one-line edit (involving a $\forall$) that fixes it.
- Consider the macro $\forall(c_3, c_4) \in \{(c_1, m), (m, c_2)\}[\Phi]$. Expand it into a syntactically legal quantified Boolean formula using $\forall c_3 \forall c_4$, Boolean equality, and implication, then rewrite $\rightarrow$ and $=$ purely in terms of $\wedge, \vee, \neg$.
- Let M be a machine that runs in space n^2 , so its running-time bound is $h = 2^{dn^2}$ for some constant d . Using $S(h) = O(n^2) \cdot \log h$, give the size of the formula $\phi = \phi_{c_{\text{start}}, c_{\text{acc}}, h}$ as a function of n , and state the matching general bound $O(f(n)^2)$ with $f(n) = n^2$.

12. Define $\text{Step}(c, c')$, the quantifier-free predicate asserting “ c' follows from c in one step of M ,” to be expressed by Cook–Levin local windows. For a toy machine with configuration encoded by three cells, sketch (in words and a small 3×2 tape diagram) which cell-triples the predicate must constrain, and argue its size is $O(n^k)$ rather than constant.
13. True, false, or open, with a one-line reason each. (a) Every language in **NP** is poly-time reducible to $TQBF$. (b) If $TQBF \in \mathbf{P}$ then $\mathbf{P} = \mathbf{PSPACE}$. (c) A **PSPACE**-complete problem cannot be in **NP**. (d) Reversing all the quantifiers of a true sentence always yields a false sentence.

14.3 Games and Generalized Geography

The previous section proved $TQBF$ is **PSPACE**-complete by reading a quantified formula as a recursive reachability test. This section reads the *same* formula a second way – as a two-player game – and the one key idea is that an alternation of quantifiers $\exists \forall \exists \dots$ is literally a game in which two players move in turn. Made precise, the truth of a fully quantified Boolean formula becomes the question “does the $\exists$ -player have a winning strategy?” We then carry that correspondence into a board game, Generalized Geography, whose graph-based moves encode an arbitrary formula game, giving a second clean **PSPACE**-complete problem and a template for proving that many games are hard.

The formula game

Fix a fully quantified Boolean formula in prenex form

$$\varphi = Q_1 x_1 Q_2 x_2 \dots Q_k x_k [\psi],$$

where each $Q_i \in \{\exists, \forall\}$ and ψ is a quantifier-free Boolean formula in the variables $x_1, \dots, x_k$. The universe of each variable is $\{0, 1\}$.

Definition 14.3.1 (The formula game). The *formula game* on φ is played by two players, *Player E* (the existential player) and *Player A* (the universal player). The variables are assigned values in the order $x_1, x_2, \dots, x_k$. On turn i , the player who owns quantifier Q_i picks the value of x_i : Player E picks x_i when $Q_i = \exists$, and Player A picks x_i when $Q_i = \forall$. After all k values are chosen, *Player E wins* if ψ evaluates to 1 under the assignment, and *Player A wins* if ψ evaluates to 0. A player has a *winning strategy* if that player can force a win no matter how the opponent plays.

Theorem 14.3.2 (Truth equals a winning strategy). *Player E has a winning strategy in the formula game on φ if and only if φ is true. Consequently the language*

$$FGAME = \{\langle \varphi \rangle \mid \text{Player E has a winning strategy on } \varphi\}$$

*equals $TQBF$, and is therefore **PSPACE**-complete.*

Proof. We induct on the number of leading quantifiers k . If $k = 0$ then $\varphi = \psi$ is a constant: it is true iff ψ evaluates to 1, which is exactly the winning condition with no moves to make, so Player E “wins” iff φ is true.

For the inductive step, write $\varphi = Q_1 x_1 [\varphi']$ where $\varphi' = Q_2 x_2 \dots Q_k x_k [\psi]$ has one fewer leading quantifier. Setting $x_1 = b$ turns φ' into a fully quantified formula φ'_b with $k - 1$ quantifiers, and the rest of the game on φ is exactly the formula game on φ'_b . By the induction hypothesis, Player E wins the game on φ'_b iff φ'_b is true.

If $Q_1 = \exists$, Player E moves first and wins iff she can pick some $b \in \{0, 1\}$ making the remaining game a win, i.e. iff φ'_0 or φ'_1 is true – which is exactly the meaning of $\exists x_1 \varphi'$ being true. If $Q_1 = \forall$, Player A moves first and Player E wins iff *every* choice b leaves a winning game, i.e. iff φ'_0 and φ'_1 are both true – exactly $\forall x_1 \varphi'$ being true. In both cases Player E has a winning strategy iff φ is true. Since the winning condition of the game is word-for-word the truth condition of the formula, $FGAME = TQBF$, which is **PSPACE**-complete. $\square$

The induction in the proof is exactly a depth-first walk of the *game tree*: the root is the empty assignment, each level branches on the two values of one variable, the branching node is labelled by whoever owns that level’s quantifier, and the leaves are labelled by the value of ψ . “ $\exists$ owns a node” means *take the best child* (1 if any child is 1); “ $\forall$ owns a node” means *the opponent picks the worst child* (1 only if all children are 1). Evaluating the root is minimax with $\exists = \max$ and $\forall = \min$.

and give Player E's strategy if she has one.

Recognition. The trigger is the alternation $\exists\forall\exists$: by Theorem 14.3.2 this is a game, and we evaluate it by minimax on the game tree with $\exists = \max$ (Player E maximizes the final ψ) and $\forall = \min$ (Player A minimizes it).

A game tree diagram illustrating a minimax problem. The root node is labeled $\exists (E): \max$. It has two children: the left child is labeled $\forall: \min$ and the right child is also labeled $\forall: \min$. The left $\forall: \min$ node has two children, both labeled 0 . The right $\forall: \min$ node has two children, labeled 0 and 1 . The right $\forall: \min$ node also has two children, labeled 1 and 0 . The right $\forall: \min$ node also has two children, labeled 0 and 0 . The right $\forall: \min$ node also has two children, labeled 0 and 0 .

Read the leaves left to right as (x_1, x_2, x_3) . Under $x_1 = 0$: the $x_2 = 0$ subtree has leaves $\psi(0, 0, 0) = 0$, $\psi(0, 0, 1) = 0$ (max = 0), and the $x_2 = 1$ subtree has $\psi(0, 1, 0) = 0$, $\psi(0, 1, 1) = 1$ (max = 1); Player A then takes the min over x_2 , getting $\min(0, 1) = 0$. Under $x_1 = 1$: the $x_2 = 0$ subtree has $\psi(1, 0, 0) = 1$, $\psi(1, 0, 1) = 0$ (max = 1), and the $x_2 = 1$ subtree has $\psi(1, 1, 0) = 0$, $\psi(1, 1, 1) = 0$ (max = 0); Player A's min is $\min(1, 0) = 0$. Finally Player E takes $\max(0, 0) = 0$, so Player E has no winning strategy: *Player A wins*. His strategy: whatever Player E opens with, answer $x_2 = x_1$.

Check. If $x_1 = 0$, the clause $(x_1 \vee x_2)$ needs $x_2 = 1$, so Player A answers $x_2 = 0$ and wins. If $x_1 = 1$, the clause $(\overline{x_1} \vee \overline{x_3})$ forces $x_3 = 0$, and then $(\overline{x_2} \vee x_3)$ needs $x_2 = 0$, so Player A answers $x_2 = 1$ and wins. Either way φ is false, confirming root value 0.

Recognition. The trigger is “same matrix, swapped quantifier order.” Truth of a quantified formula is order-sensitive, so the game it encodes can flip its winner; we evaluate each by its own little game tree.

```

graph TD
    Root["forall (A): min"] -- "x=0" --> Node0["exists y: max"]
    Root -- "x=1" --> Node1["exists y: max"]
    Node0 -- "y=0" --> L1["1"]
    Node0 -- "y=1" --> L2["0"]
    Node1 -- "y=0" --> L3["0"]
    Node1 -- "y=1" --> L4["1"]

```

401

(an E-node) is $\max(0, 0) = 0$, Player A wins. Swapping the two quantifiers turned a Player-E win into a Player-A win.

Which idea, and why? Whose turn it is encodes the quantifier, and the move order is the quantifier order; moving *second* on the matched variable is a decisive advantage, which is why $\forall\exists$ (E second) differs from $\exists\forall$ (A second).

Check. Numerically $\forall x \exists y [x = y]$ is true (pick $y = x$) and $\exists y \forall x [x = y]$ is false (no single y equals both 0 and 1), matching the two game outcomes.

Generalized Geography

In the children's game *geography*, players alternately name cities, each new city beginning with the letter that ended the previous one, no repeats; you lose when you cannot continue. Model cities as nodes of a directed graph with an edge $u \rightarrow v$ when u 's last letter is v 's first. A play is then a path that may not revisit a node. *Generalized Geography* keeps only the graph.

Definition 14.3.5 (Generalized Geography). An instance is a pair $\langle G, b \rangle$ where G is a directed graph and b a designated start node. Two players alternate; together they trace a *simple* directed path beginning at b . On each turn the mover must extend the current path along an out-edge to a node *not already on the path*. The first player who cannot move *loses*. Define

$$GG = \{ \langle G, b \rangle \mid \text{Player I (the first mover) has a winning strategy on } G \text{ from } b \}.$$

Theorem 14.3.6 (GG is in PSPACE). $GG \in \text{PSPACE}$.

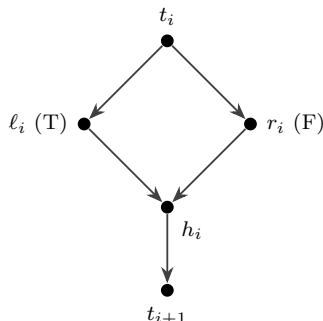
Proof. The recursive procedure $W(G, b)$ returns *true* iff the player to move at b on graph G wins. If b has out-degree 0, the mover cannot move, so return *false*. Otherwise, for each out-neighbor b_i of b , delete b (and its incident edges) to get G' – enforcing the simple-path rule, since b can never be reused – and recurse on $W(G', b_i)$. Return *true* iff some $W(G', b_i)$ is *false* (i.e. the mover has a move handing the opponent a loss). Each recursive level stores one node of the current path, the recursion depth is at most the number of nodes m , so the stack uses $O(m \log m)$ bits, which is polynomial. Hence $GG \in \text{PSPACE}$. $\square$

The reduction proving hardness turns each variable of a formula into a *diamond* gadget and each clause into a check node, so that play down the diamonds is the formula game and the check structure rewards the right player. We describe the two gadget types, then assemble a tiny example.

Theorem 14.3.7 (GG is PSPACE-complete). GG is PSPACE-complete.

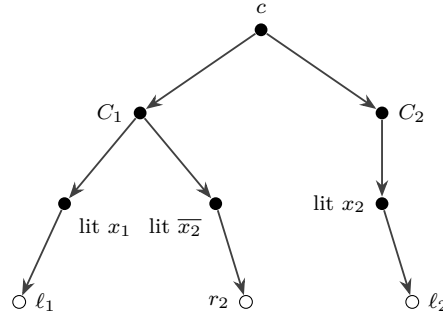
Proof. Membership is Theorem 14.3.6. For hardness we give a polynomial-time reduction $FGAME \leq_p GG$; since $FGAME = TQBF$ is PSPACE-complete (Theorem 14.3.2), this makes GG PSPACE-hard, hence PSPACE-complete. Take a formula $\varphi = \exists x_1 \forall x_2 \exists x_3 \cdots Qx_k[\psi]$ with ψ in conjunctive normal form and quantifiers strictly alternating, starting and ending with $\exists$ (pad with dummy variables otherwise; a matrix not in CNF is first put in CNF by naming each of its subformulas with a fresh innermost $\exists$ -variable, which keeps the truth value and the size linear). Player I plays Player E, Player II plays Player A.

Variable (diamond) gadgets. For each x_i build a diamond: a top node t_i , a left node ℓ_i and a right node r_i , and a bottom node h_i , with edges $t_i \rightarrow \ell_i$, $t_i \rightarrow r_i$, $\ell_i \rightarrow h_i$, $r_i \rightarrow h_i$, and $h_i \rightarrow t_{i+1}$ linking diamonds in a chain. The left branch encodes $x_i = \text{TRUE}$, the right branch $x_i = \text{FALSE}$.



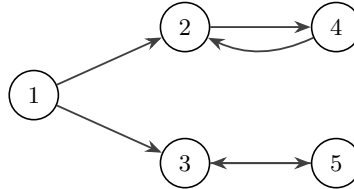
Whoever moves at t_i chooses left or right – i.e. assigns x_i . Because the start node $b = t_1$ and quantifiers alternate, the owner of t_i 's choice is Player I when $Q_i = \exists$ and Player II when $Q_i = \forall$, matching the formula game. The forced step $h_i \rightarrow t_{i+1}$ hands the next choice to the other player, preserving alternation.

Clause-check gadget. After the last diamond, a forced step reaches a hub node c , where it is Player II's turn. From c , Player II may move to any clause node $C_1, \dots, C_m$. From a clause node C_j , Player I may move to any of its literal nodes. Each literal node for the unnegated literal x_i has an edge into ℓ_i (the TRUE side of diamond i); each negated literal $\bar{x}_i$ has an edge into r_i (the FALSE side). Crucially these target the side *already traversed* when the literal is true under the chosen assignment.



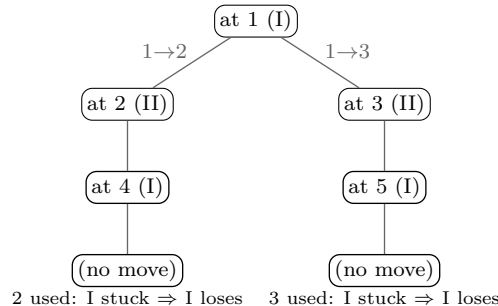
Correctness. Suppose ψ is false under the chosen assignment. Player II picks an unsatisfied clause C_j . Every literal of C_j is false, so each literal node Player I picks points to the side of its diamond *not yet visited*; Player II steps into that fresh node, and now Player I is stuck (the only out-edge of that side, $\rightarrow h_i$, leads to an already-used node), so Player I loses – correct, since a false ψ means Player A/II should win. If ψ is true, whichever clause C_j Player II picks contains a true literal; Player I picks that literal's node, which points to the *already-visited* side of its diamond, so Player II cannot move there and loses. Thus Player I wins iff ψ is true iff Player E wins the formula game. The graph has $4k + 1 + m$ nodes plus one per literal occurrence, $O(|\langle \varphi \rangle|)$ in all, and is built in polynomial time, completing the reduction. $\square$

Worked Example 14.3.8 (Deciding a small Generalized Geography board by minimax). On the directed graph below with start node $b = 1$, decide whether Player I has a winning strategy.



Recognition. The trigger is “who wins a GG board.” Use the recursion $W(G, v)$ of Theorem 14.3.6: a position is a win for the mover iff some out-edge leads to an unvisited node that is a loss for the opponent. Remember the path must stay *simple*.

Solution. Expand the game tree from $b = 1$, deleting visited nodes as we go. Player I owns even-depth nodes (root is I's move), Player II owns odd-depth.



Branch $1 \rightarrow 2$: Player II is forced $2 \rightarrow 4$, and Player I's only edge is $4 \rightarrow 2$, but 2 is already on the path, so Player I cannot move and loses. Branch $1 \rightarrow 3$: Player II forced $3 \rightarrow 5$, Player I's only edge is $5 \rightarrow 3$, but 3 is used, so Player I is stuck and loses. *Both* of Player I's opening moves end in Player I being stuck, so Player I has no winning move: *Player II has a winning strategy.*

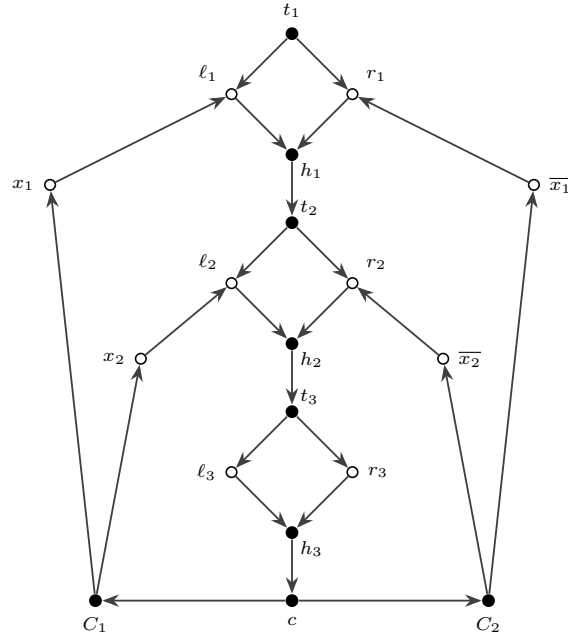
Which idea, and why? $W(G, b)$ is true iff some child position is a loss for the opponent; here every child forces a short cycle back onto a used node, stranding Player I, so the root is a loss for the mover – a Player II win.

Check. The only edges out of $\{4, 5\}$ point back to $\{2, 3\}$, which are always already visited by the time Player I must move there; with no escape the second mover wins, as the tree shows on both branches.

Worked Example 14.3.9 (Mapping a tiny *TQBF* to its Generalized Geography graph). Build the *GG* gadget graph for $\varphi = \exists x_1 \forall x_2 [(x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2})]$, identify who owns each diamond, and confirm the winner agrees with the formula's truth value.

Recognition. The trigger is “turn a formula game into a geography graph,” i.e. the reduction of Theorem 14.3.7: one diamond per variable chained $t_1 \rightarrow h_1 \rightarrow t_2$, then a hub c to clause nodes to literal nodes wired into diamond sides.

Solution. The prefix $\exists\forall$ ends with $\forall$, so the construction first pads it with a dummy $\exists x_3$ (x_3 occurs in no clause). Three diamonds (for x_1, x_2, x_3) feed a hub c ; the two clauses $C_1 = (x_1 \vee x_2)$ and $C_2 = (\overline{x_1} \vee \overline{x_2})$ become clause nodes; literal nodes wire $x_1 \rightarrow \ell_1$, $x_2 \rightarrow \ell_2$, $\overline{x_1} \rightarrow r_1$, $\overline{x_2} \rightarrow r_2$.



Start $b = t_1$. Diamond 1 ($Q_1 = \exists$) is owned by Player I; the forced $h_1 \rightarrow t_2$ hands diamond 2 ($Q_2 = \forall$) to Player II, and the forced $h_2 \rightarrow t_3$ hands the dummy diamond 3 ($Q_3 = \exists$) back to Player I. After it, $h_3 \rightarrow c$ is forced and it is Player II's move at c . Is φ true? It asks for a value of x_1 such that for both values of x_2 the matrix holds. Try x_1 : $x_1 = 0$ forces $C_1 = (0 \vee x_2) = x_2$ to need $x_2 = 1$, but then $C_2 = (1 \vee 0) = 1$ ok and C_1 ok – yet $x_2 = 0$ makes $C_1 = 0$, failing $\forall$. Try $x_1 = 1$: $C_2 = (\overline{1} \vee \overline{x_2}) = \overline{x_2}$ fails at $x_2 = 1$. So no x_1 works: φ is *false*, so Player II should win, and indeed Player II picks the clause left unsatisfied to strand Player I.

Which idea, and why? The reduction is mechanical: one diamond per variable (owned by E or A as the quantifier dictates), one hub-clause-literal tree for the CNF matrix, with literal nodes wired to the diamond side that the literal makes true.

Check. Node count $4k + 1 + m$ plus one per literal occurrence: 3 diamonds (12 nodes), hub c , 2 clause nodes, 4 literal nodes = 19 nodes, built directly from the 2 variables (plus the dummy) and 2 clauses.

Method (Turning alternating quantifiers into a two-player game).

1. **Name the players by quantifier.** $\exists$ is the maximizer (Player E / Player I); $\forall$ is the minimizer (Player A / Player II). The move order is the quantifier order.

2. **Build the game tree.** Branch on each variable's two values in turn; label each internal level by its quantifier's owner and each leaf by the value of the matrix ψ .
3. **Minimax from the leaves up.** Replace an $\exists$ -node by the max of its children, a $\forall$ -node by the min. The maximizing child at an E-node is E's move; the minimizing child at an A-node is A's threat.
4. **Read the root.** Root value 1 $\Leftrightarrow$ the formula is true $\Leftrightarrow$ Player E/I has a winning strategy.
5. **For a graph game, recurse.** Position = (current node, set of used nodes). The mover wins iff some legal out-edge reaches an unused node that is a *loss* for the opponent; delete the current node before recursing to keep the path simple.

Caution. The path in Generalized Geography must be *simple*: a node already on the path is off-limits, and the whole reduction's correctness rests on this. If you forget it, the diamond gadget breaks – the entire point is that a literal node pointing to an *already-visited* side leaves the opponent with no move. Equally, do not confuse the two “stuck” verdicts: the player who *cannot move loses*, so reaching a dead end is bad for the mover, not the opponent.

Caution. Whose turn it is *is* the quantifier. When you read or build a game tree, an $\exists$ at level i does not mean “some leaf below is 1”; it means “the player who controls level i can *steer* to a 1.” A node can have a 1-leaf in its subtree and still evaluate to 0 if the opponent controls an intervening level and routes away from it. Always propagate max/min strictly by *who owns each level*, never by “does a winning leaf exist somewhere below.”

Exercises.

1. Consider the formula game on $\varphi = \exists x_1 \forall x_2 \exists x_3 [(x_1 \vee \overline{x_2}) \wedge (x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_3})]$. Draw the full depth-3 game tree (levels x_1, x_2, x_3), label each leaf with the value of the matrix, and run minimax ($\exists = \max$, $\forall = \min$) to decide the winner. If Player E wins, state her strategy as a rule for x_3 in terms of x_2 .
2. Determine the winner of each game and give a one-line reason. (a) $\forall x \exists y [x \vee y]$; (b) $\exists y \forall x [x \vee y]$; (c) $\forall x \forall y [x \vee y]$; (d) $\exists x \exists y [x \wedge \overline{y}]$.
3. Prove that for *every* fully quantified Boolean formula φ , exactly one of the two players has a winning strategy in the formula game on φ (the game is *determined*). Use Theorem 14.3.2 and the fact that a sentence is either true or false.
4. Construct a fully quantified Boolean formula φ on three variables for which Player A has a winning strategy but for which *some* sequence of moves still ends with the matrix ψ equal to 1. Explain why this does not contradict “Player A has a winning strategy.”
5. On the directed graph with nodes $\{1, 2, 3, 4, 5, 6\}$ and edges $1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 4, 4 \rightarrow 5, 4 \rightarrow 6, 5 \rightarrow 3$, start node $b = 1$: draw the graph, expand the Generalized Geography game tree, and decide whether Player I has a winning strategy. Mark on the tree where the simple-path rule first forbids an edge.
6. Draw the variable (diamond) gadget for a single variable x_i and annotate which directed edge corresponds to setting $x_i = \text{TRUE}$ and which to $x_i = \text{FALSE}$. Then explain in two sentences why the forced edge $h_i \rightarrow t_{i+1}$ is what guarantees the two players alternate ownership of consecutive diamonds.
7. Carry out the full reduction of Theorem 14.3.7 on $\varphi = \exists x_1 \forall x_2 \exists x_3 [(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2})]$: draw the three diamonds, the hub c , the two clause nodes, and all literal nodes with their edges into the correct diamond sides. State who owns each diamond.
8. Show directly that $GG \in \text{PSPACE}$ by analyzing the space used by the recursive procedure $W(G, b)$ of Theorem 14.3.6: bound the recursion depth, bound the bits stored per level, and conclude a polynomial space bound. Why does storing the *entire* path explicitly still fit in polynomial space, and why is the recursion-stack bound the natural one?
9. In the clause-check gadget, suppose the chosen assignment makes ψ *true*. Player II must move from the hub c to some clause node C_j . Argue that Player I then has a forced winning reply, naming exactly which literal node Player I selects and why Player II is left unable to move.

10. Find the bug. A student claims: “In Generalized Geography the player who reaches a node of out-degree 0 *wins*, because they completed the longest path.” Identify the error and state the correct rule, then decide whether the student’s rule would flip the winner on the board of Exercise 5.
11. Find the bug. A proposed reduction from *FGAME* to *GG* omits the simple-path requirement and instead allows nodes to be revisited. Give a concrete two-variable formula and explain how a revisit lets the losing player escape a strand in the clause-check gadget, so the reduction no longer preserves the winner.
12. Generalized Geography comes in two flavors: in *vertex* geography (the version of this section) a *node* may not be revisited; in *edge* geography an *edge* may not be reused but nodes may repeat. Construct a small directed graph and start node where the two rules give *different* winners, and exhibit the winning strategy under each rule.

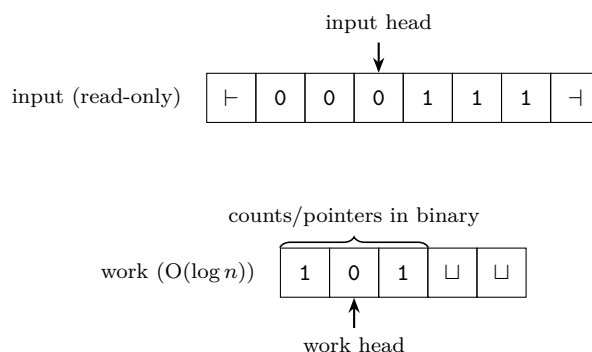
14.4 The Classes L and NL

Apart from a few sublinear examples, every space bound so far has been at least linear, large enough to hold a copy of the input. This section drops below that line. The one key idea is the split-tape model of Definition 14.1.1, now essential: a *read-only* input tape that we are allowed to scan but never write, and a separate *read/write* work tape whose cells are the only ones charged to the space budget. With only $O(\log n)$ work cells – just enough to hold a handful of pointers into the input – we get the classes L and NL, the sublinear-space analogues of P and NP, and we meet the problem *PATH* that sits at the heart of NL.

The log-space machine model

A single tape cannot separate “the input I may not alter” from “the scratch memory I am charged for,” so for sublinear space we use a machine with two tapes.

Definition 14.4.1 (Two-tape space-bounded machine). A *log-space* (more generally, sublinear-space) Turing machine has a *read-only input tape* holding w between two endmarkers, and a separate *read/write work tape*. The input head may move both directions and read the symbol under it but may never write; the work head reads and writes normally. The *space* used on input w is the number of distinct work-tape cells the machine scans – the input tape contributes nothing. For $s: \mathbb{N} \rightarrow \mathbb{R}^+$ the classes $\text{SPACE}(s(n))$ and $\text{NSPACE}(s(n))$ collect the languages decided by deterministic, respectively nondeterministic, $O(s(n))$ -space machines of this kind.



The picture is the whole point. The input is long but untouchable; the work tape is short and ours. Because a pointer to a position in an n -symbol input is just a number $\leq n$, and $\lceil \log_2 n \rceil + 1$ bits suffice to write it in binary, $O(\log n)$ work cells hold a *fixed number of pointers and counters* into the input. That single sentence – “log space = a constant number of indices into the input” – is the intuition to carry through the section.

Definition 14.4.2 (The classes L and NL).

$$\text{L} = \text{SPACE}(\log n), \quad \text{NL} = \text{NSPACE}(\log n).$$

A language is in L if some deterministic two-tape machine decides it using $O(\log n)$ work cells, and in NL if some nondeterministic such machine does.

We single out $\log n$ rather than, say, $\sqrt{n}$ or $\log^2 n$ for the same reasons we singled out polynomials for time: $\log n$ is the smallest bound that still lets the machine finger any position of the input, and the resulting classes are robust under changes of tape alphabet and reasonable changes of input encoding.

Configurations and the time-from-space bound

The earlier bound “ $s(n)$ space forces $\leq 2^{O(s(n))}$ time” (Proposition 14.1.4) absorbed one factor into the exponent using $s(n) \geq \log n$: with a read-only input we must also remember where the *input* head sits, which adds a factor of n .

Definition 14.4.3 (Configuration on w). For a two-tape machine M and input w of length n , a *configuration of M on w* records the state, the work-tape contents, the work-head position, and the input-head position. It does *not* include w itself (that is fixed and read-only).

Lemma 14.4.4 (Few configurations, hence polynomial time). *A two-tape machine M running in space $s(n) \geq \log n$ has at most $n 2^{O(s(n))}$ configurations on any input of length n . In particular, if $s(n) = O(\log n)$ the number of configurations is $2^{O(\log n)} = n^{O(1)}$, a polynomial in n .*

Proof. Let M have c states and work alphabet of size g . A configuration is a choice of: the state (c options), the work-tape contents on its $s(n)$ used cells ($g^{s(n)}$ strings), the work-head position ($s(n)$ options), and the input-head position ($n + 2$ options, counting the endmarkers). Multiplying,

$$\#\text{configs} \leq c \cdot g^{s(n)} \cdot s(n) \cdot (n + 2) = n \cdot 2^{O(s(n))}.$$

When $s(n) = O(\log n)$, $2^{O(\log n)} = n^{O(1)}$, and the extra factor n is absorbed, leaving a polynomial bound. $\square$

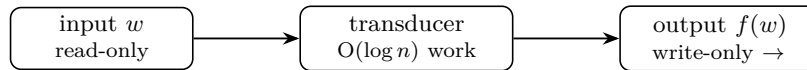
Corollary 14.4.5. $L \subseteq NL \subseteq P$.

Proof. $L \subseteq NL$ is immediate: a deterministic machine is a nondeterministic one that never branches. For $NL \subseteq P$, let M be a nondeterministic $O(\log n)$ -space decider for $\mathcal{L}$. A halting computation never repeats a configuration, so each branch runs for at most the number of configurations, which is polynomial by Lemma 14.4.4. Build the directed *configuration graph* on input w : its vertices are the polynomially many configurations, with an edge $c_1 \rightarrow c_2$ when M can step from c_1 to c_2 . Then M accepts w iff this graph has a path from the start configuration to the (unique) accept configuration – a question a deterministic polynomial-time machine settles by breadth-first search over the polynomially many vertices and edges. Hence $\mathcal{L} \in P$. $\square$

Log-space transducers

To compare problems inside NL we will eventually need reductions computed in log space; and a reduction must *output* a possibly long string while still being charged only $O(\log n)$ work cells. The fix is a third tape that is write-only.

Definition 14.4.6 (Log-space transducer). A *log-space transducer* is a machine with a read-only input tape, a read/write work tape limited to $O(\log n)$ cells, and a *write-only output tape* whose head moves only rightward (so the machine can never re-read what it has emitted). On input w it halts with some string on the output tape; that string is $f(w)$, and f is the *log-space computable function* computed by the transducer.



only the middle box is charged for space;
 $|f(w)|$ may far exceed $\log n$

The output tape is *not* counted against the space budget – the transducer may print a string of length n^{10} while keeping its work tape down to $O(\log n)$ cells. This is the same accounting principle as the input tape: only the work tape costs. We will use such transducers heavily in Section 14.5; here they justify calling functions like “increment a binary counter” or “output every position i with $w_i = 1$ ” log-space computable.

A deterministic log-space decider: counting in binary

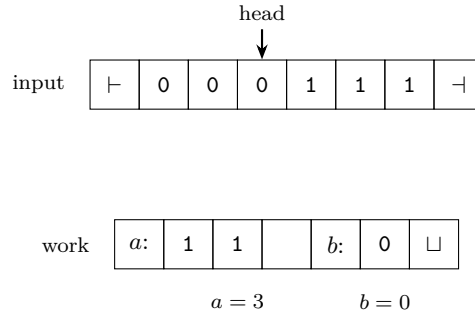
Worked Example 14.4.7 ($\{0^k 1^k\}$ is in L by binary counting). Show that $A = \{0^k 1^k \mid k \geq 0\}$ is in L. The familiar zig-zag decider crosses off matched symbols, but that needs to *write* on the input – forbidden here – or it needs linear scratch to remember which cells are crossed. Find a genuinely $O(\log n)$ -space method.

Recognition. The trigger is “decide a counting condition without storing the input.” When the test is “two quantities are equal,” keep the quantities themselves as *binary counters* on the work tape; a count up to n fits in $\lceil \log_2 n \rceil + 1$ cells.

Solution. The machine M_A never marks the input. It makes one left-to-right pass over the read-only tape:

1. Check the *shape*: verify the input matches $0^* 1^*$ (reject the instant a 0 follows a 1). This needs only the finite control, $O(1)$ space.
2. While in the 0-block, increment a binary counter a on the work tape once per 0; while in the 1-block, increment a second binary counter b once per 1.
3. At the right endmarker, accept iff $a = b$ (compare the two binary strings cell by cell).

A snapshot just after reading the third 0 of 000111, with $a = 3 = 11_2$ and $b = 0$ still on the work tape:



Each counter never exceeds $k \leq n$, so it occupies $\leq \lceil \log_2 n \rceil + 1$ work cells; two such counters are still $O(\log n)$. Incrementing a binary counter touches only those cells. Hence M_A runs in $O(\log n)$ space, so $A \in \text{L}$.

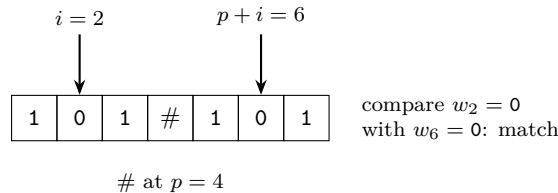
Which idea, and why? Log space cannot store the data, only *summaries* of it. Equality of two blocks is summarized by their two lengths, and a length up to n is exactly $O(\log n)$ bits – so counting in binary, not crossing off on the input, is the log-space move.

Check. On 000111: the shape passes, a counts up 1, 2, 3, then b counts up 1, 2, 3, and $3 = 3$ accepts. On 00011: $a = 3, b = 2$, and $3 \neq 2$ rejects – using never more than 2 bits per counter.

Worked Example 14.4.8 (Why a third pointer decides $\{w\#w\}$). Sketch a log-space decider for $E = \{w\#w \mid w \in \{0, 1\}^*\}$ over alphabet $\{0, 1, \#\}$, and say exactly what each log-space register holds.

Recognition. “Compare position i of the left copy with position i of the right copy, for all i .” That is an indexed comparison; the index i and the offset to the matching cell are numbers $\leq n$, hence $O(\log n)$ bits each.

Solution. First locate the single $\#$ and record its position p in a binary register (reject if the number of $\#$ is not exactly one). Then for $i = 1, 2, \dots, p - 1$: read the input symbol at position i (left copy) and the symbol at position $p + i$ (right copy), and reject on any mismatch; also reject unless the right copy has exactly $p - 1$ symbols. Each step holds three binary numbers – p , the loop index i , and the derived address $p + i$ – and repositions the *input* head by counting cells, never copying them.



Three $O(\log n)$ registers, no copying, so $E \in L$. Two registers would not suffice for the bookkeeping shown, but a constant number certainly is $O(\log n)$.

Which idea, and why? The same principle as $\{0^k 1^k\}$: store *indices into* the input, not the input. A constant number of $\log n$ -bit pointers is the entire log-space toolkit.

Check. On $101\#101$: $p = 4$; comparing $(1, 5), (2, 6), (3, 7)$ gives $1 = 1, 0 = 0, 1 = 1$, accept. On $101\#100$: position $(3, 7)$ gives $1 \neq 0$, reject.

PATH and nondeterministic log space

The flagship NL problem is directed reachability.

Definition 14.4.9 (*PATH*).

$$PATH = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a directed path from } s \text{ to } t\}.$$

Theorem 14.4.10. $PATH \in NL$.

Proof. We give a nondeterministic $O(\log n)$ -space decider N . Let m be the number of vertices of G (read off the encoding). N keeps two binary registers: *cur*, the label of the vertex it currently occupies, and *cnt*, a step counter. It runs:

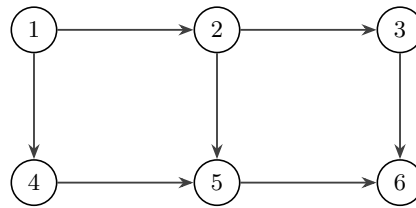
1. Set $cur \leftarrow s$ and $cnt \leftarrow 0$.
2. Repeat: if $cur = t$, accept. If $cnt = m$, reject. Otherwise nondeterministically pick a vertex v , verify on the read-only input that (cur, v) is an edge of G (reject this branch if not), set $cur \leftarrow v$ and $cnt \leftarrow cnt + 1$.

A genuine s - t path, if one exists, is witnessed by some branch that guesses its vertices in order; since a shortest path visits no vertex twice it has length $< m$, so the step counter never forces a premature reject. Conversely any accepting branch exhibits a real walk from s to t . The only memory used is *cur* and *cnt*, each a number $\leq m \leq n$, hence $O(\log n)$ bits. So N decides *PATH* in nondeterministic log space. $\square$

Worked Example 14.4.11 (Tracing the *PATH* walk on a concrete graph). Run the procedure of Theorem 14.4.10 on the graph below with $s = 1$, $t = 6$, and exhibit one accepting branch together with the contents of the two registers.

Recognition. “Reachability by a remembered current vertex” is exactly *PATH* in NL. The trigger: we are allowed to forget the path so far and remember only *where we are* plus *how many steps* we have taken.

Solution. Here is G ($m = 6$ vertices):



The accepting branch guesses $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$. The registers evolve as:

step	cur	cnt	action
0	1	0	guess 2, check $(1, 2) \in E$
1	2	1	guess 5, check $(2, 5) \in E$
2	5	2	guess 6, check $(5, 6) \in E$
3	6	3	$cur = t \Rightarrow$ accept

Throughout, only two numbers in $\{0, \dots, 6\}$ are stored, never the list $1, 2, 5, 6$ as a whole. A branch that instead guessed $1 \rightarrow 4 \rightarrow 5$ then tried a non-edge $(5, 3)$ would reject on that branch – harmless, since acceptance needs only *one* good branch.

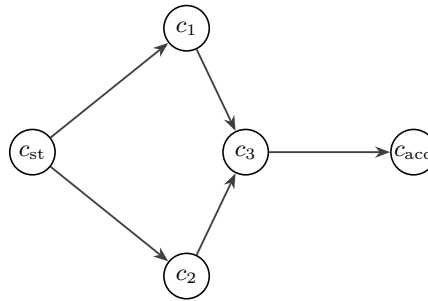
Which idea, and why? Nondeterminism supplies the next vertex for free; log space then only has to *check* the edge and overwrite the current vertex. Remembering the whole path would cost $\Omega(m \log m)$ space and is exactly what we avoid.

Check. The guessed walk has length $3 < m = 6$, so the counter never hits its cap; and each edge $(1, 2), (2, 5), (5, 6)$ is present, so every check passes. With $t = 3$ instead, the branch $1 \rightarrow 2 \rightarrow 3$ accepts in 2 steps.

Worked Example 14.4.12 (The configuration graph turns NL acceptance into a *PATH* instance). Explain, for a fixed nondeterministic log-space machine N and input w , why “ N accepts w ” is itself a directed-reachability question, and count the vertices.

Recognition. This is the engine behind Corollary 14.4.5. The trigger: a nondeterministic computation is a walk through the space of configurations, so acceptance is reachability in the configuration graph.

Solution. Form the directed graph $G_{N,w}$ whose vertices are the configurations of N on w (Definition 14.4.3) and whose edges are the legal single steps $c_1 \rightarrow c_2$. Designate $s = c_{\text{start}}$ and (after merging accept states) a single $t = c_{\text{acc}}$.



$$N \text{ accepts } w \iff c_{\text{st}} \rightsquigarrow c_{\text{acc}}$$

N accepts w exactly when some computation branch reaches c_{acc} , i.e. when $G_{N,w}$ has a path from s to t . By Lemma 14.4.4 there are $n^{O(1)}$ configurations, so $G_{N,w}$ has polynomially many vertices and a deterministic machine can solve the reachability by breadth-first search in polynomial time.

Which idea, and why? The configuration graph is the universal translation from “nondeterministic computation” to “*PATH*.” Its polynomial size (log space $\Rightarrow$ poly configurations) is precisely what proves $\text{NL} \subseteq \text{P}$.

Check. If N is in fact deterministic, every vertex has out-degree ≤ 1 , the part reachable from c_{st} is a simple chain, and reachability is just “follow the unique successor” – recovering the ordinary deterministic run.

Method (Designing a log-space algorithm).

1. **Refuse to copy.** You may scan the input freely but never store a copy of it. Decide what *summary* of the input suffices: a few counts, a few positions.
2. **Budget your registers.** Each quantity you keep must be a number $\leq \text{poly}(n)$, so it fits in $O(\log n)$ bits. Use a *constant* number of such registers (counters, indices, the current vertex, a loop bound).
3. **Recompute instead of remember.** To revisit input position i , move the input head there by counting from an endmarker – do not cache the symbols in between.
4. **For NL, guess then verify.** Let nondeterminism produce the next object (a vertex, a symbol); store only the *current* object and *check* the local condition on the read-only input. Cap the number of guesses with a binary step counter.
5. **Confirm the bound.** Add up the registers: a constant number, each $O(\log n)$ bits, is $O(\log n)$ space. The output tape of a transducer is exempt.

Caution. You may *not* copy the input onto the work tape. Writing all n input symbols to scratch is $O(n)$ space – linear, not logarithmic – and silently destroys the whole point of the model. Likewise you may not record an entire s - t path (that is $\Omega(m \log m)$ space); *PATH* stays in NL only because the walk keeps just the *current* vertex.

Caution. For a transducer, the *output* tape is free but the *work* tape is not. Producing a long string $f(w)$ is fine; staging that string on the work tape before emitting it is not. The output head moves one way only, so design f to print left to right and never look back – any re-reading of the output must be paid for in $O(\log n)$ work space, not on the output tape.

Exercises.

1. Describe a deterministic log-space machine deciding $\mathcal{L}_1 = \{0^i 1^j \mid i = 2j, i, j \geq 0\}$. Say exactly which binary registers your machine keeps and why the total work space is $O(\log n)$. (You may scan the read-only input as many times as you like.)
2. Let $D = \{w \in \{0, 1\}^* \mid w \text{ has equally many 0s and 1s}\}$. Give a log-space decider for D and explain, in one sentence each, why (a) a single binary counter holding the running difference ($\#0 - \#1$) suffices and (b) that counter never exceeds $O(\log n)$ cells.
3. Consider the input layout for a two-tape log-space machine. Draw the machine on input $w = 0011$: show the read-only input tape between endmarkers with the input head over the third cell, and a short work tape holding the binary number 10 with its own head. Label which tape is charged for space.
4. A directed graph G on vertices $\{1, \dots, 7\}$ has edges $1 \rightarrow 2, 2 \rightarrow 4, 4 \rightarrow 7, 1 \rightarrow 3, 3 \rightarrow 5, 5 \rightarrow 6, 6 \rightarrow 3$. Draw G (use labelled vertices and directed edges). Then run the NL procedure for *PATH* with $s = 1, t = 7$: list one accepting branch as a table of (cur, cnt) values, and identify one branch that rejects without reaching t .
5. For the same graph G of the previous exercise, argue that the procedure correctly *rejects* $\langle G, 3, t \rangle$ when t is chosen to be a vertex unreachable from 3 – name such a t and explain why every branch eventually rejects (via the step counter, unless an edge check fails first).
6. Prove that $\text{NL} \subseteq \text{P}$ by the configuration-graph argument: state the bound on the number of configurations of an $O(\log n)$ -space nondeterministic machine on input of length n , and explain why a deterministic breadth-first search over that graph runs in polynomial time.
7. Show that L is closed under complement. (Hint: a deterministic decider that always halts can have its accept/reject outcome flipped; be sure to explain why “always halts” holds for a log-space *decider*, using the configuration count.)
8. Let $f: \Sigma^* \rightarrow \Sigma^*$ send a string w to the string listing the positions i (in binary, separated by commas) at which $w_i = 1$. Argue that f is log-space computable by describing a transducer for it, and explain why the (possibly long) comma-list on the output tape does *not* violate the $O(\log n)$ work-space bound.
9. *Find the bug.* A student claims $\text{PATH} \in \text{L}$ as follows: “Do a deterministic depth-first search from s ; keep a list of the vertices on the current DFS stack so you never revisit one; accept if you ever reach t . The stack is the only memory, so it is small.” Identify precisely why this does *not* fit the $O(\log n)$ work-space budget, and state what is and is not currently known about whether $\text{PATH} \in \text{L}$.
10. *Find the bug.* Another student claims a log-space decider for $A = \{0^k 1^k\}$ that “copies the block of 0s onto the work tape, then deletes one work-tape 0 for each 1 read on the input.” Explain why this machine is correct but is *not* a log-space machine, and contrast it with the binary-counting decider that *is* in L .
11. A two-tape machine M uses work space $s(n) = 3 \log_2 n + 5$ on inputs of length n . Using the configuration count $n 2^{O(s(n))}$, give an explicit polynomial upper bound on the number of configurations of M on a length- n input, and conclude that $L(M) \in \text{P}$.
12. Define $2\text{-COL} = \{\langle G \rangle \mid G \text{ is 2-colorable (bipartite)}\}$ for undirected G . Sketch why the complement of 2-COL is decidable in nondeterministic log space by reducing the test to short reachability/parity checks on the read-only adjacency input (guess a closed walk of odd length), and state which registers your NL machine keeps. (You need only describe the registers and the local check, not a full correctness proof. 2-COL itself is then in NL by $\text{NL} = \text{coNL}$, Section 14.5.)

14.5 NL-Completeness and $\text{NL} = \text{coNL}$

Section 14.4 placed *PATH* inside NL but left two questions hanging: which problems are the *hardest* in NL , and whether nondeterministic log space is any stronger than its complement-closed shadow. Both are settled here. The key idea is that a log-space machine has so little memory that the right notion of reduction must itself fit in log space – it cannot even afford to write down its own output – and once we adopt that notion, the configuration graph of a log-space machine turns *every* NL problem into a single reachability query. That same configuration-graph picture, counted carefully, then yields the Immerman–Szelepcsényi theorem $\text{NL} = \text{coNL}$, one of the most counterintuitive equalities in complexity theory. We continue to count *only work-tape cells*; the read-only input tape is free.

Throughout, fix the canonical decision problem

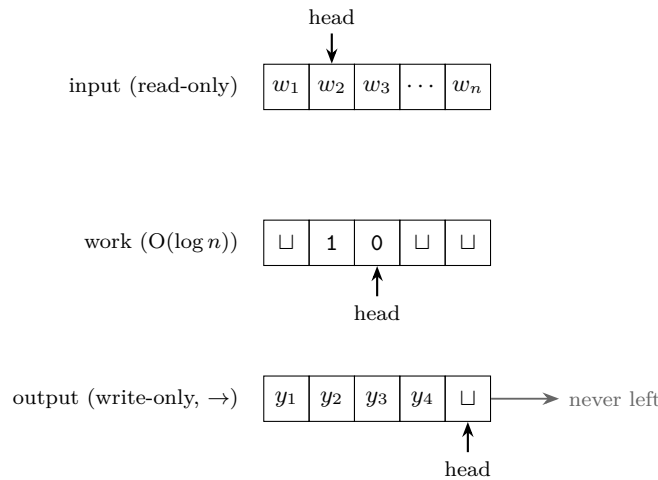
$$\text{PATH} = \{\langle G, s, t \rangle \mid G \text{ is a directed graph with a directed path from } s \text{ to } t\}.$$

Its complement is $\overline{\text{PATH}} = \{\langle G, s, t \rangle \mid G \text{ has no directed path from } s \text{ to } t\}$ (plus the malformed strings, which we ignore).

Log-space reductions

Definition 14.5.1 (Log-space transducer and $\leq_L$). A *log-space transducer* is a Turing machine with three tapes: a read-only input tape, a read/write work tape holding $O(\log n)$ cells, and a write-only output tape whose head never moves left (so the machine cannot reread its own output). On input w it halts with some string on the output tape; that string is the value $f(w)$, and f is then a *log-space computable function*. A language A is *log-space reducible* to B , written $A \leq_L B$, if there is a log-space computable f with $w \in A \iff f(w) \in B$ for all w .

The output tape is generous – $f(w)$ may be polynomially long – but the work tape is not, and that asymmetry forces a particular programming discipline: the transducer must *stream* its output, emitting each symbol and forgetting it, because it can never store $f(w)$ in its own working memory. The layout is the picture to keep in mind.



Two structural facts make $\leq_L$ the correct reduction for NL : the classes NL and L are *closed* under it, and it *composes*. Both are proved by the same trick of recomputing output symbols on demand instead of storing them.

Theorem 14.5.2 (Closure under $\leq_L$). *If $A \leq_L B$ and $B \in \text{NL}$, then $A \in \text{NL}$. The same statement holds with NL replaced by L .*

Proof. Let f be a log-space reduction of A to B computed by transducer T , and let N_B decide B in space $O(\log |f(w)|) = O(\log n)$ (polynomial output has logarithmic length-logarithm). The naive plan – compute

$f(w)$, store it, run N_B – fails, because $f(w)$ can be polynomially long and will not fit on a log-space work tape. Instead we run N_B on the *virtual* input $f(w)$, never materializing it. We keep a binary counter p holding the position of N_B 's input head within $f(w)$; whenever N_B wants the symbol at position p , we run T on w from scratch, discard every output symbol except the p -th, hand that one symbol to N_B , and resume. The stored data is: N_B 's work tape ($O(\log n)$), the pointer p ($O(\log n)$ bits since $f(w)$ is polynomial), and T 's own work tape during each recomputation ($O(\log n)$). The total is $O(\log n)$, and the simulation is nondeterministic exactly when N_B is, so $A \in \text{NL}$; if N_B is deterministic the whole construction is deterministic, giving the L statement. $\square$

Theorem 14.5.3 ($\leq_L$ composes). *If $A \leq_L B$ and $B \leq_L C$, then $A \leq_L C$.*

Proof. Let f (transducer T_f) reduce A to B and g (transducer T_g) reduce B to C , i.e. $x \in B \iff g(x) \in C$. We want a single log-space transducer for $g \circ f$. The obstacle is the same as before: T_g expects $f(w)$ on its input tape, but $f(w)$ is too long to store. We run T_g on the virtual input $f(w)$. A pointer p tracks T_g 's input-head position in $f(w)$; each time T_g reads, we restart T_f on w and stream out symbols of $f(w)$, counting until we reach p , feed that one symbol to T_g , and continue. Whatever T_g writes to its output tape is the real output $g(f(w))$, emitted left to right and never reread. Space in use: T_g 's work tape ($O(\log n)$), T_f 's work tape during a recomputation ($O(\log n)$), and the pointer p ($O(\log n)$). Hence $g \circ f$ is log-space computable, and $w \in A \iff f(w) \in B \iff g(f(w)) \in C$. $\square$

Caution. In both proofs the recomputation is not an optimization you may skip – it is the whole point. If you try to *store* the intermediate string $f(w)$ to avoid rerunning T_f , you blow the $O(\log n)$ work-tape budget the instant $f(w)$ exceeds logarithmic length, which it routinely does (a graph reduction outputs a polynomial-size graph). Log-space reductions trade time for space: you may recompute $f(w)$ a polynomial number of times, but you may never write it down all at once.

Definition 14.5.4 (NL-completeness). A language B is *NL-complete* if (i) $B \in \text{NL}$ and (ii) every $A \in \text{NL}$ satisfies $A \leq_L B$. If only (ii) holds, B is *NL-hard*.

Theorem 14.5.2 is what makes this definition useful: if any NL-complete B were in L , then every $A \in \text{NL}$ would satisfy $A \leq_L B \in \text{L}$, forcing $A \in \text{L}$ and hence $\text{L} = \text{NL}$. So an NL-complete problem is a single target whose membership in L would collapse the whole class.

PATH is NL-complete

Theorem 14.5.5 (*PATH* is NL-complete). *PATH is NL-complete.*

Proof. Membership $\text{PATH} \in \text{NL}$ was shown in Section 14.4: guess the path one node at a time, storing only the current node and a step counter, in $O(\log m)$ space on an m -node graph. For hardness, let $A \in \text{NL}$ be decided by a nondeterministic machine N running in space $O(\log n)$. We build a log-space transducer R that maps w to $\langle G, s, t \rangle$ where G is the *configuration graph* of N on w .

A configuration of N on w records the state, the work-tape contents, and the two head positions (but not the fixed input w); by the count of Section 14.4 there are $n 2^{O(\log n)} = n^{O(1)}$ of them, each describable by a string of $c \log n$ bits. The nodes of G are these configuration strings; we put an edge $c_1 \rightarrow c_2$ exactly when N 's transition function lets c_1 yield c_2 in one step. Set s to the start configuration and – after modifying N to wipe its work tape and park its heads on accepting, giving a *unique* accept configuration – set t to that configuration. Then N accepts w iff some computation branch reaches accept iff G has an $s \rightsquigarrow t$ path, i.e. $w \in A \iff \langle G, s, t \rangle \in \text{PATH}$.

R runs in log space: to list nodes, cycle through all $c \log n$ -bit strings (a binary counter, $O(\log n)$ bits) and output those that are legal configurations. To list edges, cycle through all pairs (c_1, c_2) and output the pair when c_1 yields c_2 – a check that inspects only the input symbol and work symbol named in c_1 , again $O(\log n)$ space. No configuration list is ever stored; each is generated, tested, emitted, and forgotten. $\square$

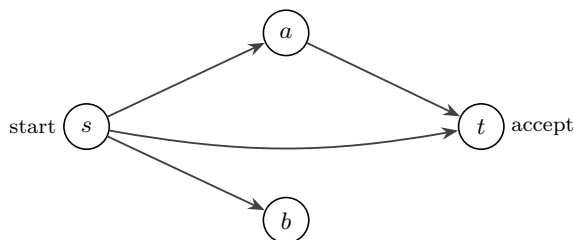
Corollary 14.5.6. $\text{NL} \subseteq \text{P}$.

Proof. A log-space machine runs in time $n2^{O(\log n)} = n^{O(1)}$, so a log-space reduction is in particular a polynomial-time reduction. By Theorem 14.5.5 every $A \in \text{NL}$ satisfies $A \leq_L \text{PATH}$, hence $A \leq_P \text{PATH}$; and $\text{PATH} \in \text{P}$ by ordinary breadth-first search. A language polynomial-time reducible to a language in P is in P , so $A \in \text{P}$. $\square$

Worked Example 14.5.7 (Building the configuration graph for a tiny NL machine). Let N be a nondeterministic log-space machine with three relevant configurations on a fixed input w : a start configuration s , an intermediate a , and the unique accept configuration t , where N may step $s \rightarrow a$, may step $s \rightarrow t$ directly, and may step $a \rightarrow t$, with a dead branch $s \rightarrow b$ that goes nowhere. Draw the configuration graph and read off whether N accepts.

Recognition. This is the PATH -hardness reduction applied by hand: nodes = configurations, edges = one-step yields, acceptance = existence of an $s \rightsquigarrow t$ path. The trigger is “decide acceptance of a nondeterministic machine,” which the reduction converts into a reachability query.

Solution. List N ’s one-step moves and draw one edge per move; mark s and t .



There is a path $s \rightarrow a \rightarrow t$ (and even a direct edge $s \rightarrow t$), so t is reachable from s and N accepts w . The dead branch $s \rightarrow b$ is irrelevant: PATH asks only whether *some* path reaches t .

Which idea, and why? Every NL acceptance question is a reachability question on the configuration graph, because a nondeterministic branch is exactly a directed walk from the start configuration.

Check. Removing both edges into t (deleting moves $s \rightarrow t$ and $a \rightarrow t$) would strand t , making $\langle G, s, t \rangle \in \overline{\text{PATH}}$ and N reject – the reduction tracks acceptance faithfully.

Worked Example 14.5.8 (PATH -hardness, encoding a 2-bit-counter machine). A machine N keeps a single counter $k \in \{0, 1, 2, 3\}$ in binary on its work tape (two bits, $O(\log n)$ space here trivially) and, ignoring its input for simplicity, may at each step either increment k or leave it; it accepts the instant $k = 3$. Take the start configuration $k = 0$. Exhibit the configuration graph and the resulting PATH instance.

Recognition. Same reduction, now with concrete configurations: a configuration is the counter value, an edge is a legal one-step update, accept is $k = 3$. The trigger is “turn the machine’s moves into a graph.”

Solution. The four configurations are $k = 0, 1, 2, 3$; the moves give a self-loop (“leave k ”) and an up-edge (“increment”) at each value below 3; t is the $k = 3$ node.



The PATH instance is $\langle G, s, t \rangle$ with s the node 0 and t the node 3. A path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ exists, so N accepts. The transducer emitting this graph stores only the current two-bit counter value and a loop index – log space, as required.

Which idea, and why? The reduction is purely local: each edge is decided by looking at one configuration and applying N ’s transition rule, so a transducer with $O(\log n)$ memory can emit the entire (polynomially many) edges one at a time without ever holding the graph.

Check. If N could only *leave* k (delete the up-edges), every node would have just a self-loop, 3 would be unreachable from 0, and N would never accept – consistent with the graph.

The Immerman–Szelepcsényi theorem: $\text{NL} = \text{coNL}$

By analogy with coNP , define $\text{coNL} = \{\bar{A} \mid A \in \text{NL}\}$ – the languages whose *complements* are in NL . Since PATH is NL -complete, $\overline{\text{PATH}}$ is the canonical coNL -complete problem, and the entire question $\text{NL} \stackrel{?}{=} \text{coNL}$ reduces to one challenge: *can a nondeterministic log-space machine certify that t is NOT reachable from s ?* Guessing a “non-path” makes no sense, so the certificate must be indirect. The Immerman–Szelepcsényi idea is to compute the *exact count* of reachable nodes and use it as the certificate.

Theorem 14.5.9 (Immerman–Szelepcsényi). $\text{NL} = \text{coNL}$. *Equivalently, $\overline{\text{PATH}} \in \text{NL}$.*

Fix G with m nodes and source s . For $i \geq 0$ let

$$R_i = \{v \mid G \text{ has a directed path of length } \leq i \text{ from } s \text{ to } v\}, \quad r_i = |R_i|.$$

Then $R_0 = \{s\}$, $r_0 = 1$, the sets nest $R_0 \subseteq R_1 \subseteq \dots$, and R_m is exactly the set of nodes reachable from s (no shortest path needs length $> m - 1$). The argument has two parts: (A) given the true count r_i we can verify non-membership in R_i , and (B) we can compute r_{i+1} from r_i . Iterating (B) from $r_0 = 1$ yields $r = r_m$, after which (A) at level m certifies $t \notin R_m$, i.e. non-reachability.

Lemma 14.5.10 (Certify membership and – given the count – non-membership). *There is an $O(\log m)$ -space nondeterministic procedure that, given G, s, i , a node v , and the correct value r_i , has an accepting branch iff $v \in R_i$, and a separate such procedure that, given the correct r_i , has an accepting branch iff $v \notin R_i$. Every wrong-count or wrong-guess branch rejects.*

Proof. Membership is direct: nondeterministically walk from s , at most i edges, storing only the current node and a step count; accept iff the walk lands on v . This needs $O(\log m)$ space and has an accepting branch iff a path of length $\leq i$ reaches v , i.e. iff $v \in R_i$.

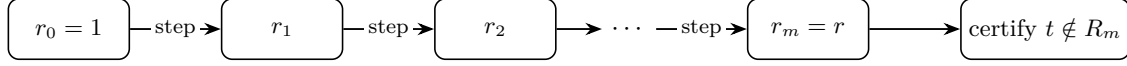
Non-membership uses the count. Go through all m nodes u in turn; for each, nondeterministically *guess* whether $u \in R_i$. On a “yes” guess, verify it by the membership walk above (reject the branch if the walk fails), increment a counter d of confirmed members, and – crucially – reject immediately if this confirmed u equals v . After all m nodes, reject unless $d = r_i$. The point: a branch reaches the end only if it correctly identified *all* r_i members of R_i (any shortfall leaves $d < r_i$); and since it rejected the moment it confirmed v as a member, the surviving branch has witnessed every member of R_i and found v among the non-members. Such a surviving branch exists iff $v \notin R_i$. Storage is the loop node u , the counter d , the target v , and one walk pointer – all $O(\log m)$. $\square$

Lemma 14.5.11 (Inductive counting step). *Given the correct r_i , an $O(\log m)$ -space nondeterministic procedure has at least one branch that outputs r_{i+1} , and every branch either outputs r_{i+1} or rejects.*

Proof. A node v lies in R_{i+1} iff $v \in R_i$ or some $u \in R_i$ has an edge $u \rightarrow v$. We recount R_{i+1} directly. Initialize $r_{i+1} \leftarrow 0$. For each node v (outer loop): run an inner loop over all nodes u , guessing for each whether $u \in R_i$; confirm every “yes” by the membership walk (reject on failure) and tally confirmed members in d . While scanning, if a confirmed member u satisfies $u = v$ or $(u, v) \in E$, mark v as belonging to R_{i+1} . After the inner loop, reject unless $d = r_i$ (which guarantees the branch saw *all* of R_i , so its verdict on v is trustworthy); if v was marked, increment r_{i+1} . After the outer loop, output r_{i+1} . Only branches that correctly enumerated all of R_i at every v survive, and on those branches the emitted count is exactly $|R_{i+1}| = r_{i+1}$. Space: the two loop variables u, v , the counters d, r_{i+1} , and a walk pointer, all $O(\log m)$. $\square$

Proof of Theorem 14.5.9. We give a nondeterministic $O(\log m)$ -space machine M for $\overline{\text{PATH}}$. On $\langle G, s, t \rangle$: set $r_0 = 1$; for $i = 0, \dots, m - 1$ apply Lemma 14.5.11 to replace r_i by r_{i+1} (rejecting on any failing branch), arriving at $r = r_m$, the number of reachable nodes. Finally invoke the non-membership procedure of Lemma 14.5.10 with $v = t$, level m , and the count r : M accepts iff that procedure confirms $t \notin R_m$, i.e. iff t is unreachable from s . At every stage M stores only i , the current count, one or two loop nodes, and a walk pointer – $O(\log m)$ in total. Thus $\overline{\text{PATH}} \in \text{NL}$. Since PATH is NL -complete and NL is closed under $\leq_L$ (Theorem 14.5.2), $\overline{\text{PATH}} \in \text{NL}$ forces $\bar{A} \in \text{NL}$ for every $A \in \text{NL}$, i.e. $\text{coNL} \subseteq \text{NL}$; complementing gives $\text{NL} \subseteq \text{coNL}$, so $\text{NL} = \text{coNL}$. $\square$

The pipeline of counts is the structural heart of the proof, and is worth drawing once.



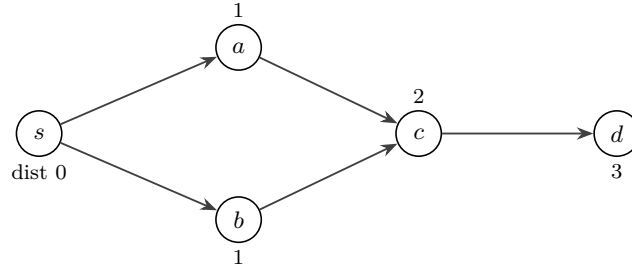
Lemma 14.5.11, applied m times

Lemma 14.5.10

Worked Example 14.5.12 (One inductive-counting step: deriving r_{i+1} from r_i on a concrete graph). Take the 5-node graph with edges $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow c$, $b \rightarrow c$, and one more edge $c \rightarrow d$, source s . Suppose the algorithm has correctly reached level $i = 1$ with $r_1 = |R_1|$. Compute R_1 , then run the counting step to obtain $r_2 = |R_2|$, exhibiting the per-node verdicts.

Recognition. This is Lemma 14.5.11 on actual numbers: R_{i+1} adds the nodes one edge beyond R_i . The trigger is “advance the reachable-within- i frontier by one layer.”

Solution. First draw G and the distance layers from s .



The nodes within distance 1 of s are $R_1 = \{s, a, b\}$, so $r_1 = 3$. Now the step recounts R_2 : for each node v , scan all nodes u , confirm the members of R_1 (here s, a, b , giving a tally of $3 = r_1$, so the branch survives), and mark v into R_2 if $v \in R_1$ or some confirmed $u \in R_1$ has $u \rightarrow v$.

v	$v \in R_1?$	$\exists u \in R_1 : u \rightarrow v?$	$v \in R_2?$
s	yes	—	yes
a	yes	—	yes
b	yes	—	yes
c	no	$a \rightarrow c$ ($a \in R_1$)	yes
d	no	only $c \rightarrow d$, $c \notin R_1$	no

So $R_2 = \{s, a, b, c\}$ and $r_2 = 4$. The verdict for d is the delicate one: d is excluded *only because* the branch first proved it had seen all $r_1 = 3$ members of R_1 and none of them points to d – exactly the “certify a negative by counting” move.

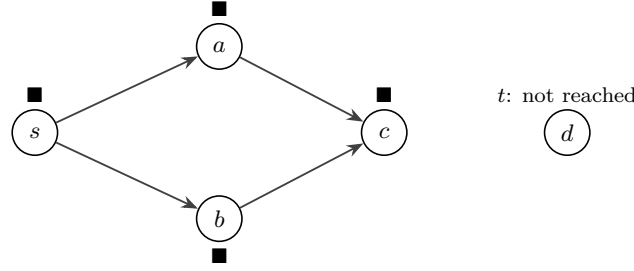
Which idea, and why? R_{i+1} is computed, not guessed: a node enters iff it is already in R_i or is one confirmed edge past R_i , and the count check (tally = r_i) is what licenses trusting a “no.”

Check. Continuing, the step from $r_2 = 4$ adds d (since $c \in R_2$ and $c \rightarrow d$), giving $r_3 = 5 = m$; all five nodes are reachable, and indeed $t = d$ would be reachable – so on this graph the $\overline{PATH}$ machine would correctly *reject* the query $\langle G, s, d \rangle$.

Worked Example 14.5.13 (Certifying non-reachability from the final count). On the previous graph delete the edge $c \rightarrow d$, so d becomes isolated. The reachable set is $R_m = \{s, a, b, c\}$ with $r = 4$. Show how the non-membership procedure certifies $t = d \notin R_m$, producing an accepting branch of the $\overline{PATH}$ machine.

Recognition. Lemma 14.5.10, non-membership half: walk through all nodes, confirm exactly $r = 4$ reachable ones by guessing-and-verifying paths, and conclude the rest – including d – are unreachable. The trigger is “prove t is NOT reachable,” answerable only via the exact count.

Solution. The surviving branch guesses “reachable” precisely for s, a, b, c and verifies each by a short path from s ; it guesses “not reachable” for d and never tries to confirm it.



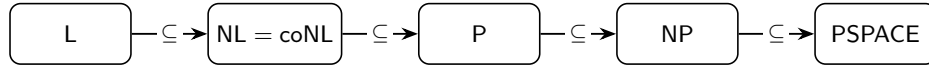
The branch confirms $r = 4$ members (s, a, b, c) , never confirms $t = d$ as a member, and finishes with its tally equal to the count, so it accepts: $t \notin R_m$ is certified, and the machine declares $\langle G, s, d \rangle \in \overline{PATH}$. Any *wrong* branch – one that under-counts, or one that tries to verify d as reachable – rejects, because no length- m path to d exists and the final tally $= r$ test fails.

Which idea, and why? A negative (“ t unreachable”) becomes nondeterministically verifiable once you know how *many* nodes are reachable: confirm that many, and everything left over is provably out.

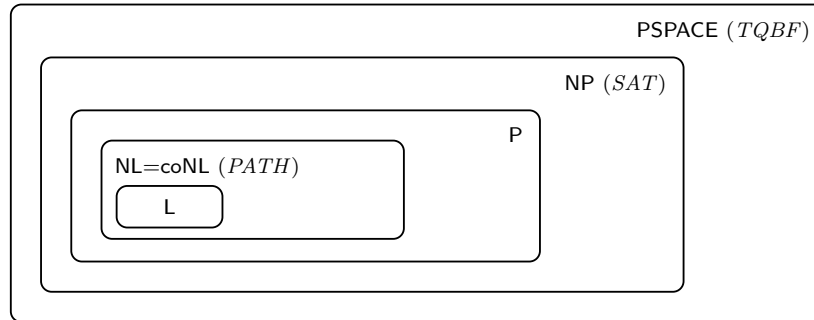
Check. If we instead asked about $\langle G, s, c \rangle$, the same count $r = 4$ would force the procedure to confirm c as a member (it is one of the four), so the “confirm v as member $\Rightarrow$ reject” rule fires and *no* branch accepts – correctly reporting $c \in R_m$, i.e. $\langle G, s, c \rangle \notin \overline{PATH}$.

The class map

Collecting Corollary 14.5.6, Theorem 14.5.9, and the inclusions from earlier sections ($L \subseteq NL$ trivially, $P \subseteq NP \subseteq PSPACE$ from Chapters 12–13 and Section 14.1) gives the canonical chain. Every inclusion shown is known; none is known to be proper.



A nested-region view of the same containments, with the representative complete or canonical problem sitting in each band, makes the layering concrete.



Method (How to certify a negative in nondeterministic log space by counting).

1. **Name the positive object you can already verify.** Here “ v is reachable within i steps” is checkable by guessing and walking a path – a guessable, log-space certificate.
2. **Get the exact count of positives.** Compute $r_i = |R_i|$, the number of objects that pass the positive test. Use induction if you cannot count in one shot: derive r_{i+1} from r_i (Lemma 14.5.11).
3. **Confirm exactly that many.** Loop over all candidates, guess which are positive, verify each guess, and tally; *reject any branch whose tally $\neq r_i$* . Only a branch that has exhibited *every* positive survives.
4. **Declare the leftovers negative.** On the surviving branch, any candidate never confirmed as positive is certifiably negative – because all positives have been accounted for. Accept iff your target v is among those leftovers.

Caution. The counting must be *exact*, not an upper bound. If a branch is allowed to survive with $d \leq r_i$ (“found no more than the claimed number of members”), it could quietly miss a genuine member u of R_i , fail to follow $u \rightarrow v$, and wrongly brand a reachable v as unreachable. The test is therefore strict equality $d = r_i$: a branch that under-counts has *not* seen all of R_i and must reject, while a branch that over-counts is impossible because every confirmed member is verified by an actual path. Equality is exactly the condition that the branch’s census of R_i is complete and correct.

Caution. Do not confuse **coNL** with the complement of the class. $\text{coNL} = \{\overline{A} \mid A \in \text{NL}\}$ is a class of languages (those whose complement is in **NL**); it is not “everything outside **NL**.” The theorem $\text{NL} = \text{coNL}$ says the class is closed under complement – if a language is decidable in nondeterministic log space, so is its complement – which is precisely the surprise, since the analogous $\text{NP} = \text{coNP}$ is widely believed false.

Exercises.

1. Let N be a nondeterministic log-space machine whose configurations on a fixed input are $\{s, a, b, c, t\}$, with one-step moves $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow c$, $b \rightarrow t$, $c \rightarrow c$ (a self-loop), and t accepting. Draw the configuration graph using labelled vertices and directed edges, then decide whether N accepts by reading off reachability of t from s . Identify which branch (if any) loops forever and explain why *PATH* membership is nonetheless decided.
2. A directed graph G has nodes $\{1, 2, 3, 4, 5\}$ and edges $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$, $4 \rightarrow 5$. Taking $s = 1$, list R_0, R_1, R_2, R_3 and the counts r_0, r_1, r_2, r_3 explicitly. Then carry out the inductive counting step that produces r_3 from r_2 , writing the per-node verdict table ($v \in R_2?$ / some confirmed $u \in R_2$ with $u \rightarrow v?$ / $v \in R_3?$).
3. Show that the log-space transducer in the *PATH*-hardness reduction (Theorem 14.5.5) can output the edge list of the configuration graph while storing only $O(\log n)$ bits. Specify exactly what is on the work tape at the moment the transducer is testing whether the pair (c_1, c_2) is an edge, and why no list of previously emitted edges is needed.
4. Prove directly that **NL** is closed under union: given $A_1, A_2 \in \text{NL}$ via machines N_1, N_2 , describe a nondeterministic log-space machine for $A_1 \cup A_2$ and argue its space bound. State where, if anywhere, your construction uses the read-only input tape rather than the work tape.
5. Let $\text{CYCLE} = \{\langle G \rangle \mid G \text{ is a directed graph containing a directed cycle}\}$. Give a log-space reduction from *PATH* to *CYCLE*. (Hint: from $\langle G, s, t \rangle$ build a graph in which a cycle exists iff G has an $s \rightsquigarrow t$ path; you may first make G acyclic by leveling it into m copies before adding a back edge.) Argue the transducer is log space.
6. Using $\text{NL} = \text{coNL}$, show that $\overline{\text{CYCLE}} \in \text{NL}$. Then state, in one sentence each, what $\overline{\text{CYCLE}} \in \text{NL}$ means operationally and why the same fact was *not* obvious before the Immerman–Szelepcsényi theorem.
7. In the inductive counting step (Lemma 14.5.11), suppose the strict test “reject unless $d = r_i$ ” is weakened to “reject unless $d \leq r_i$.” Construct a small directed graph and a nondeterministic branch on which the weakened algorithm outputs the *wrong* value of r_{i+1} . Explain precisely which reachable node the branch misses.
8. *Find the bug.* A student proposes this “simpler” **NL** algorithm for $\overline{\text{PATH}}$: “On $\langle G, s, t \rangle$, nondeterministically guess, for each node, whether it is reachable from s ; verify each guessed-reachable node by guessing a path; if t was guessed unreachable and no contradiction arose, accept.” Exhibit a graph and a guess pattern on which this machine *wrongly* accepts an instance where t is reachable, and pinpoint the missing ingredient (compare with Lemma 14.5.10).
9. Prove that if $A \leq_L B$ and $B \leq_L A$ both hold, and $B \in \text{L}$, then $A \in \text{L}$. Cite exactly which of Theorems 14.5.2 and 14.5.3 you use and where.
10. *Find the bug.* To show $\leq_L$ composes, a student writes: “Run T_f on w , store $f(w)$ on the work tape, then run T_g on $f(w)$.” Identify the resource that this violates, give a concrete family of inputs on which $f(w)$ has length exceeding any $O(\log n)$ bound, and state the recomputation fix in one sentence.

11. Let $STCON_k$ be $PATH$ restricted to graphs of out-degree at most k . Show $STCON_2$ is still NL-complete by giving a log-space reduction from general $PATH$ that replaces each high-out-degree node by a small gadget of out-degree ≤ 2 nodes. Draw the gadget that splits a node with out-edges to v_1, v_2, v_3 .
12. Argue carefully that $NL \subseteq P$ by combining the $PATH$ -hardness reduction with the running-time bound of a log-space machine. State the exact polynomial-time procedure that decides $PATH$ and explain why a log-space reduction is automatically a polynomial-time one.
13. Define $coNL$ as a class and explain, in two or three sentences, the precise difference between the assertion “ $NL = coNL$ ” and the (false-looking) assertion “every language is in NL .” Then state what the analogous open question $NP \stackrel{?}{=} coNP$ asks and why $NL = coNL$ does not settle it.

Chapter 15

Intractability and Advanced Topics

15.1 The Hierarchy Theorems

Common sense says that a Turing machine handed a bigger time or space budget ought to be able to decide strictly more languages, and the hierarchy theorems make that intuition into a proof. The one key idea is *diagonalization with a budget*: we build a single machine D that, inside the larger bound $f(n)$, simulates every machine living in the smaller bound on its own description and then flips the answer, so D 's language cannot coincide with any of theirs. The only catch – and it is what the whole section turns on – is that D must be able to *measure* its own budget before it spends it, which forces the technical hypothesis of *constructibility*. From the resulting strict containments we will manufacture genuinely intractable problems: languages that are decidable in principle yet provably outside P or PSPACE.

Constructible functions

For the simulator to “mark off $f(n)$ cells” or “count down $t(n)$ steps,” it has to compute the bound from the input length without already exceeding it. That is exactly what constructibility guarantees.

Definition 15.1.1 (Space constructible). A function $f: \mathbb{N} \rightarrow \mathbb{N}$ with $f(n) \geq O(\log n)$ is *space constructible* if the map $1^n \mapsto$ (binary representation of $f(n)$) is computable by some Turing machine using $O(f(n))$ space. Fractional values such as $\sqrt{n}$ or $n \log n$ are rounded down to the nearest integer.

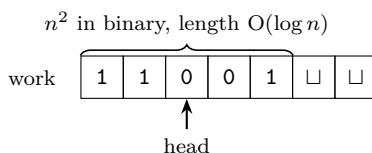
Definition 15.1.2 (Time constructible). A function $t: \mathbb{N} \rightarrow \mathbb{N}$ with $t(n) \geq O(n \log n)$ is *time constructible* if the map $1^n \mapsto$ (binary representation of $t(n)$) is computable by some Turing machine in $O(t(n))$ time.

Every function you are likely to write down – $\log n$, n , $n \log n$, n^2 , n^3 , 2^n – is constructible in both senses (above their respective thresholds). The point of the definitions is not to exclude anything natural; it is to exclude pathological bounds that are too erratic to be measured, which is the loophole the second pitfall below exploits.

Worked Example 15.1.3 (n^2 is space constructible). Exhibit a machine showing that $f(n) = n^2$ is space constructible, and confirm the space it uses is $O(n^2)$.

Recognition. The trigger is “produce $f(n)$ in binary from 1^n within $O(f(n))$ space.” The recipe is: count the input length into a binary counter, then arithmetic on that counter.

Solution. On input 1^n the machine sweeps the input once, incrementing a binary counter, ending with n in binary on the work tape. It then squares that number by any schoolbook method, leaving n^2 in binary. The work tape never holds more than the counter, the multiplicand, and the product.



The binary numbers involved have length $O(\log n)$, so the working space for the arithmetic is $O(\log n)$, vastly inside the $O(n^2)$ budget the definition allows (indeed inside $O(n)$).

Which idea, and why? Constructibility only asks that f be computable *within space* $O(f)$; here the count-then-square machine uses merely $O(\log n)$ space, comfortably under n^2 , so n^2 qualifies.

Check. At $n = 5$ the machine writes $25 = 11001$ – five bits – using a handful of work cells, nowhere near 25.

The space hierarchy theorem

Theorem 15.1.4 (Space hierarchy). *For any space-constructible $f: \mathbb{N} \rightarrow \mathbb{N}$ there is a language A decidable in $O(f(n))$ space but not in $o(f(n))$ space.*

Proof. We build a decider D for the witness language A ; running in $O(f(n))$ space gives one half, and diagonalization against every $o(f)$ -space machine gives the other. Define

$D = \text{“On input } w\text{:”}$

1. Let $n = |w|$. Using space constructibility, compute $f(n)$ and mark off exactly $f(n)$ tape cells; if any later step tries to leave this marked zone, **reject**.
2. If w is not of the form $\langle M \rangle 10^*$ for a TM M , **reject**.
3. Simulate M on w , counting simulated steps; if the count exceeds $n 2^{f(n)}$, **reject**.
4. If M accepts, **reject**; if M rejects, **accept**.

The step counter of stage 3 fits in $O(f(n))$ space and kills any infinite loop that stays inside the marked zone (a machine confined to $g(n)$ space has at most $n 2^{O(g(n))}$ distinct configurations on an input of length n , so running longer means looping). Hence D always halts and uses $O(f(n))$ space, so $A = L(D) \in \text{SPACE}(f(n))$.

Now suppose, for contradiction, some machine M decides A in space $g(n) = o(f(n))$. Because D 's fixed tape alphabet must encode M 's arbitrary alphabet, D simulates M with a constant blow-up: it uses $d \cdot g(n)$ space for a constant d depending only on M . Since $g = o(f)$, there is an n_0 such that for all $n \geq n_0$ both $d \cdot g(n) < f(n)$ and M 's running time, at most $n 2^{O(g(n))}$ by the configuration count, is below the clock $n 2^{f(n)}$. The padded inputs $\langle M \rangle 10^k$ make $|w|$ as large as we like while still naming M ; choose k so that $w = \langle M \rangle 10^k$ has length $\geq n_0$. On that input D 's simulation fits inside the marked zone and runs to completion, so by stage 4, D outputs the *opposite* of M . Thus $w \in A \iff w \notin L(M)$, contradicting $L(M) = A$. Therefore no $o(f)$ -space machine decides A . $\square$

The padding 10^* is the quiet hero: each pad gives D a fresh, longer input that still describes M , so even an M whose asymptotics “kick in” late is eventually caught on a long enough pad. The next diagram is the picture behind the whole argument.

Worked Example 15.1.5 (The diagonalization table for the space hierarchy). Lay out the table whose diagonal D contradicts, and read off why A escapes every $o(f)$ -space machine.

Recognition. This is the same device that defeated A_{TM} : index the rows by machines, the columns by their (padded) descriptions, and have D disagree with M_i in the cell (i, i) .

Solution. List the $o(f)$ -space machines $M_1, M_2, \dots$ down the side and the inputs $\langle M_1 \rangle 10^*, \langle M_2 \rangle 10^*, \dots$ across the top. Cell (i, j) holds M_i 's verdict on input j (A = accept, R = reject). D is engineered so that its row *flips the boxed diagonal entry*: where M_i says A, D says R, and vice versa.

	$\langle M_1 \rangle 10^*$	$\langle M_2 \rangle 10^*$	$\langle M_3 \rangle 10^*$	$\dots$
M_1	A	R	A	$\dots$
M_2	R	R	A	$\dots$
M_3	A	A	R	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
D	R	A	A	$\dots$ D flips each boxed entry

Reading column i : D 's entry differs from M_i 's boxed diagonal entry, so D and M_i disagree on at least the input $\langle M_i \rangle 10^*$. Hence $L(D) \neq L(M_i)$ for every i , which is precisely “ A is not in $o(f)$ space.”

Which idea, and why? Diagonalization guarantees difference by *construction*: forcing D to disagree with M_i on input i makes D 's language unequal to every row's language at once.

Check. The boxed entries read A, R, R; D 's diagonal reads R, A, A – the exact complement, so D matches no row.

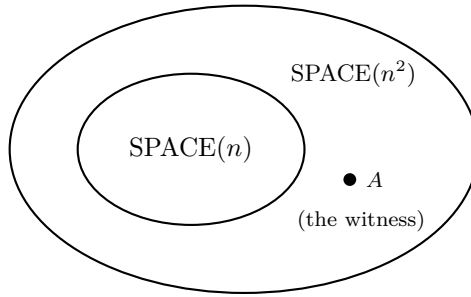
Corollary 15.1.6. *If $f_1(n) = o(f_2(n))$ and f_2 is space constructible, then $\text{SPACE}(f_1) \subsetneq \text{SPACE}(f_2)$. In particular $\text{SPACE}(n) \subsetneq \text{SPACE}(n^2)$, and more generally $\text{SPACE}(n^a) \subsetneq \text{SPACE}(n^b)$ whenever $0 \leq a < b$.*

Proof. $\text{SPACE}(f_1) \subseteq \text{SPACE}(f_2)$ since $f_1 = o(f_2)$ gives $f_1(n) \leq f_2(n)$ eventually. Theorem 15.1.4 supplies $A \in \text{SPACE}(f_2)$ not decidable in $o(f_2(n))$ space; as $f_1 = o(f_2)$, $A \notin \text{SPACE}(f_1)$, so the inclusion is proper. $\square$

Worked Example 15.1.7 (The strict containment $\text{SPACE}(n) \subsetneq \text{SPACE}(n^2)$). Prove $\text{SPACE}(n) \subsetneq \text{SPACE}(n^2)$, naming the constructible function and the small- o fact that drive it.

Recognition. The trigger is “separate two space classes,” which is exactly Corollary 15.1.6: pick the larger bound, check it is constructible, check the smaller is o of it.

Solution. Take $f_2(n) = n^2$ (space constructible by Worked Example 15.1.3) and $f_1(n) = n$. Since $n/n^2 = 1/n \rightarrow 0$, we have $n = o(n^2)$. Corollary 15.1.6 then yields the witness language $A \in \text{SPACE}(n^2) \setminus \text{SPACE}(n)$, so the containment of $\text{SPACE}(n)$ inside $\text{SPACE}(n^2)$ is strict. Pictorially the larger class strictly engulfs the smaller, with A living in the gap.



Which idea, and why? A constructible upper bound plus a strictly-smaller lower bound is the exact hypothesis of the hierarchy corollary, and $n = o(n^2)$ is the strict-smaller fact; together they force the proper containment.

Check. The same template gives $\text{SPACE}(n^2) \subsetneq \text{SPACE}(n^3)$ via $n^2 = o(n^3)$, so the powers really do form an infinite strictly increasing tower inside PSPACE.

Corollary 15.1.8. $\text{NL} \subsetneq \text{PSPACE}$, and similarly $\text{PSPACE} \subsetneq \text{EXPSPACE}$, where $\text{EXPSPACE} = \bigcup_k \text{SPACE}(2^{n^k})$.

Proof. Savitch's theorem gives $NL \subseteq SPACE(\log^2 n)$, and Theorem 15.1.4 gives $SPACE(\log^2 n) \subsetneq SPACE(n) \subseteq PSPACE$, whence $NL \subsetneq PSPACE$. For the second part, every $SPACE(n^k) \subseteq SPACE(2^n)$, and $SPACE(2^n) \subsetneq SPACE(2^{2n}) \subseteq EXSPACE$ by the hierarchy theorem (with the constructible bound 2^{2n}), so $PSPACE \neq EXSPACE$. $\square$

The time hierarchy theorem

For time the same plan works, but the bookkeeping costs more. While simulating one of M 's steps, D must also decrement a clock; keeping that clock beside the moving head costs a factor of $\log t(n)$ per step. That logarithmic overhead is exactly why the lower bound in the theorem reads $o(t/\log t)$ rather than $o(t)$.

Theorem 15.1.9 (Time hierarchy). *For any time-constructible $t: \mathbb{N} \rightarrow \mathbb{N}$ there is a language A decidable in $O(t(n))$ time but not in time $o(t(n)/\log t(n))$.*

Proof. The decider mirrors the space proof. On input w of length n : compute $t(n)$ by time constructibility and load $\lceil t(n)/\log t(n) \rceil$ into a binary counter; reject if w is not $\langle M \rangle 10^*$; otherwise simulate M on w , decrementing the counter before each step D itself takes in the simulation and rejecting if it hits 0; finally output the opposite of M 's verdict. The simulation is run on a three-track tape: track 1 carries M 's tape, track 2 carries M 's state and a copy of its transition function kept beside the head, track 3 carries the counter, also dragged along beside the head. Because the second track travels with the head, looking up M 's next move costs $O(1)$ per step (a constant depending on M), so a $g(n)$ -time M is simulated in $O(g(n))$ time *before* the clock. The counter has magnitude about $t/\log t$, hence length $O(\log t)$, and moving it each step multiplies the cost by $O(\log t)$. The total is $O(t/\log t) \cdot O(\log t) = O(t)$, so $A = L(D) \in TIME(t(n))$.

For the lower bound, suppose M decides A in time $g(n) = o(t/\log t)$. Then D simulates M in time $d \cdot g(n)$ for a constant d , and for $n \geq n_0$ we have $d \cdot g(n) < t(n)/\log t(n)$, so on a long enough pad $\langle M \rangle 10^k$ the clock does not expire and D runs M to completion – then flips its answer. Hence $L(M) \neq A$, a contradiction. $\square$

Corollary 15.1.10. *If $t_1(n) = o(t_2(n)/\log t_2(n))$ and t_2 is time constructible, then $TIME(t_1) \subsetneq TIME(t_2)$. Consequently $TIME(n^a) \subsetneq TIME(n^b)$ for $1 \leq a < b$, and $P \subsetneq EXPTIME$.*

Proof. Properness is the hierarchy theorem as in Corollary 15.1.6; the log factor only sharpens the gap needed between t_1 and t_2 , and $n^a = o(n^b/\log n^b)$ whenever $a < b$. For $P \subsetneq EXPTIME$: every $TIME(n^k) \subseteq TIME(2^n)$, while $TIME(2^n) \subsetneq TIME(2^{2n}) \subseteq EXPTIME$ by the theorem, so no polynomial bound exhausts $EXPTIME$. $\square$

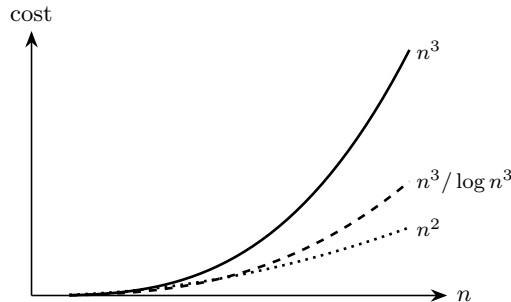
Worked Example 15.1.11 (Why $TIME(n^2) \subsetneq TIME(n^3)$ needs the log gap). Use Corollary 15.1.10 to separate $TIME(n^2)$ from $TIME(n^3)$, and check that the log-factor slack is available.

Recognition. “Separate two time classes” triggers Corollary 15.1.10; the extra obligation versus space is to verify $t_1 = o(t_2/\log t_2)$, not merely $t_1 = o(t_2)$.

Solution. Put $t_2 = n^3$ (time constructible) and $t_1 = n^2$. We need $n^2 = o(n^3/\log n^3)$. Indeed

$$\frac{n^2}{n^3/\log(n^3)} = \frac{n^2 \cdot 3 \log n}{n^3} = \frac{3 \log n}{n} \rightarrow 0,$$

so the hypothesis holds with room to spare and $TIME(n^2) \subsetneq TIME(n^3)$ follows. The slack between the bare n^3 and the diminished $n^3/\log n^3$ is the price of single-tape clock bookkeeping.



Which idea, and why? The time corollary's hypothesis is $t_1 = o(t_2/\log t_2)$, so we must beat the *shrunk* bound $n^3/\log n^3$; the ratio $3\log n/n \rightarrow 0$ certifies that n^2 does, which is what licenses the separation.

Check. The curve n^2 (dotted) stays below $n^3/\log n^3$ (dashed) for large n , exactly as the vanishing ratio predicts; the bare n^3 (solid) sits above both.

From hierarchy to genuine intractability

The witness languages above are artificial – their only job is to occupy the gap between two classes. Their value is leverage: a single *complete* problem for the larger class inherits the separation and becomes provably intractable for the smaller one.

Definition 15.1.12 (EXPSPACE-complete). A language B is *EXPSPACE-complete* if $B \in \text{EXPSPACE}$ and every $A \in \text{EXPSPACE}$ satisfies $A \leq_p B$ (polynomial-time many-one reducible).

Theorem 15.1.13. *Let*

$$EQREX\uparrow = \{\langle Q, R \rangle \mid Q, R \text{ are equivalent regular expressions with the exponentiation operator}\},$$

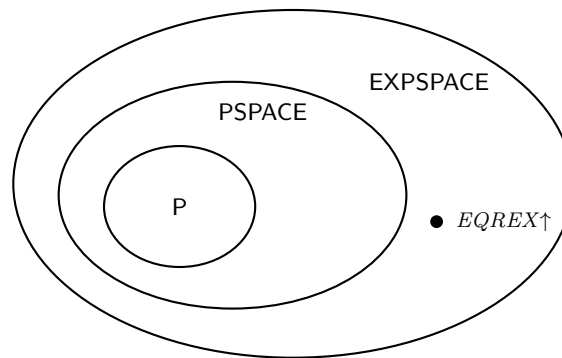
where $R\uparrow k$ abbreviates R concatenated with itself k times and exponents are written in binary. Then $EQREX\uparrow$ is EXPSPACE-complete, and therefore $EQREX\uparrow \notin \text{PSPACE}$ – in particular $EQREX\uparrow \notin \text{P}$.

Proof sketch. Membership. Expand each $R\uparrow k$ by repetition; a binary exponent of value up to 2^ℓ inflates the expression by a factor up to 2^ℓ , so the de-exponentiated expressions, and the NFAs built from them, have size $O(n2^n)$. Testing two NFAs for equivalence in nondeterministic linear space, then applying Savitch, decides equivalence in space $O((n2^n)^2) = 2^{O(n)}$, so $EQREX\uparrow \in \text{EXPSPACE}$. *Hardness.* Given an EXPSPACE machine M running in space 2^{n^k} and an input w , a polynomial-time reduction outputs $R_1 = \Delta^*$ (all strings over the history alphabet) and an R_2 generating every string that is *not* a rejecting computation history of M on w ; then $R_1 \equiv R_2$ iff M accepts w . The exponentiation operator is what keeps R_2 polynomial-sized despite the 2^{n^k} -length configurations: a gap of that width is written $\Delta\uparrow(2^{n^k})$ with the exponent in binary, length $O(n^k)$. Now if $EQREX\uparrow \in \text{PSPACE}$, then by the reduction every EXPSPACE language would be in PSPACE, giving $\text{EXPSPACE} = \text{PSPACE}$ and contradicting Corollary 15.1.8. Hence $EQREX\uparrow \notin \text{PSPACE} \supseteq \text{P}$. $\square$

Worked Example 15.1.14 (Locating $EQREX\uparrow$ on the class map). Place $EQREX\uparrow$ among P , PSPACE , EXPSPACE and state, honestly, what is proved versus assumed.

Recognition. The trigger is “a complete problem for the strictly larger class is outside the smaller one.” Completeness transports the proven separation $\text{PSPACE} \subsetneq \text{EXPSPACE}$ down onto the concrete language.

Solution. Corollary 15.1.8 *proves* $\text{PSPACE} \subsetneq \text{EXPSPACE}$ with no unproven assumption. Theorem 15.1.13 puts $EQREX\uparrow$ at the top of EXPSPACE via completeness, so it cannot drop into PSPACE without collapsing the two classes. The containments $\text{P} \subseteq \text{PSPACE} \subseteq \text{EXPSPACE}$ are known; the bottom is conjectured strict (P vs PSPACE is open), but the *top* step is a theorem, and that is all we need.



top step proven strict; bottom step open

Which idea, and why? An EXPSPACE-complete language sits provably outside PSPACE precisely because $\text{PSPACE} \subsetneq \text{EXPSPACE}$ is a theorem; completeness is the lever that turns a class separation into a statement about one explicit problem.

Check. The reasoning is honest: it never claims $\text{P} \neq \text{PSPACE}$ (unknown), only that $\text{EQREX}\uparrow$ escapes PSPACE, hence *a fortiori* escapes P.

Method (How to separate two complexity classes by a hierarchy theorem).

1. **Pick the larger bound as f_2 (or t_2) and verify it is constructible.** Every natural function above the threshold qualifies; cite the count-then-compute machine.
2. **Confirm the smaller bound is o of it** – for space, $f_1 = o(f_2)$; for time, the stricter $t_1 = o(t_2/\log t_2)$. Compute the ratio limit explicitly.
3. **Invoke the hierarchy corollary** to get a witness language in the larger class but not the smaller, hence proper containment.
4. **(For a named problem)** Show your target is *complete* for the larger class under reductions that the smaller class respects; the separation then forces the target out of the smaller class, or else the two classes collapse.

Caution. The two theorems are not symmetric: space allows $o(f)$, but time only allows $o(t/\log t)$. The missing log factor is real – it is the cost of dragging a step counter alongside the head on a single tape, and no constant-overhead single-tape clock is known. Writing “ $\text{TIME}(o(t)) \subsetneq \text{TIME}(t)$ ” as the time analogue of the space statement claims more than the diagonal argument above delivers.

Caution. Constructibility cannot be dropped. If f is too erratic to compute within $O(f)$ space, the simulator D cannot even mark off its own budget, so stage 1 fails before any diagonalization begins – and indeed there exist (artificially built) non-constructible gaps $f_1 = o(f_2)$ for which $\text{SPACE}(f_1) = \text{SPACE}(f_2)$ (the Borodin–Trakhtenbrot gap phenomenon). The hypothesis is load-bearing, not decorative.

Exercises.

1. Show directly from Definition 15.1.1 that $f(n) = 2n + 5$ is space constructible. Describe a machine that, on input 1^n , leaves the binary representation of $2n + 5$ on its tape, and state the space it uses in terms of n .
2. Prove from Definition 15.1.2 that $t(n) = n^2$ is time constructible. Sketch a machine that writes n^2 in binary from 1^n and argue its running time is $O(n^2)$ (count the cost of the counter increments and of the squaring step separately).
3. Using the space hierarchy corollary, prove each strict containment and name the constructible larger bound and the small- o fact you use:

- (a) $\text{SPACE}(n^3) \subsetneq \text{SPACE}(n^4)$; (b) $\text{SPACE}(\log n) \subsetneq \text{SPACE}(\log^2 n)$;
(c) $\text{SPACE}(n) \subsetneq \text{SPACE}(n \log n)$.

4. Decide whether each separation can be obtained from the *time* hierarchy theorem as stated, and give a one-line reason. (a) $\text{TIME}(n) \subsetneq \text{TIME}(n^2)$. (b) $\text{TIME}(n) \subsetneq \text{TIME}(n \log \log n)$. (c) $\text{TIME}(2^n) \subsetneq \text{TIME}(2^{2n})$. For any you reject, identify exactly which hypothesis of Corollary 15.1.10 fails.
5. Draw the diagonalization table for the *time* hierarchy theorem: rows indexed by $o(t/\log t)$ -time machines M_1, M_2, M_3 , columns by the padded inputs $\langle M_i \rangle 10^*$, with each M_i 's diagonal verdict boxed and the D -row showing the flipped entries. Then state in one sentence which single input witnesses $L(D) \neq L(M_2)$.
6. A classmate proposes to prove $\text{TIME}(n^2) \subsetneq \text{TIME}(n^2 \log \log n)$ “by the same template as $\text{TIME}(n^2) \subsetneq \text{TIME}(n^3)$.” Check whether

$$n^2 = o(n^2 \log \log n / \log(n^2 \log \log n))$$

by computing the ratio limit, and conclude whether the proposed separation follows from the theorem as stated.

7. Find the bug. A student writes: “By the space hierarchy theorem, $\text{SPACE}(f_1) \subsetneq \text{SPACE}(f_2)$ whenever $f_1(n) = o(f_2(n))$, no other hypothesis needed.” Give the missing hypothesis, and describe in one sentence the failure in the diagonalizing machine D when that hypothesis is dropped.
8. Find the bug. A student claims the time hierarchy theorem gives $\text{TIME}(o(t)) \subsetneq \text{TIME}(t)$ “because diagonalization always flips one bit and a flipped bit is a different language.” Explain precisely where the single-tape simulation forces a weaker conclusion than $o(t)$, and state the strongest lower bound the theorem actually delivers.
9. Let D be the space-hierarchy diagonalizer for a space-constructible f . Explain why D runs M on every input of the form $\langle M \rangle 10^*$ rather than only on the single input $\langle M \rangle$. Construct a scenario (an M whose $o(f)$ asymptotics “kick in” only for large n) in which testing only $\langle M \rangle$ would let M escape diagonalization, and show how a sufficiently long pad repairs it.
10. Using Savitch’s theorem and the space hierarchy theorem, give a complete proof that $\text{NL} \subsetneq \text{PSPACE}$. State which intermediate space class the hierarchy theorem is applied to and why the bound there is space constructible.
11. Let $\text{EQREX}\uparrow$ be the equivalence problem for regular expressions with the exponentiation operator (binary exponents). Explain why ordinary regular-expression equivalence (no exponentiation) lies in PSPACE , whereas the exponentiated version is EXPSPACE -complete; identify the single feature of the encoding that causes the exponential jump, and state what EXPSPACE -completeness lets you conclude about $\text{EQREX}\uparrow$ versus P .
12. Define $\text{EXPSPACE} = \bigcup_k \text{SPACE}(2^{n^k})$. Prove $\text{PSPACE} \subsetneq \text{EXPSPACE}$ by exhibiting a single constructible bound to which the space hierarchy theorem applies, and explain why *every* PSPACE class $\text{SPACE}(n^k)$ is contained in the class you separate against.

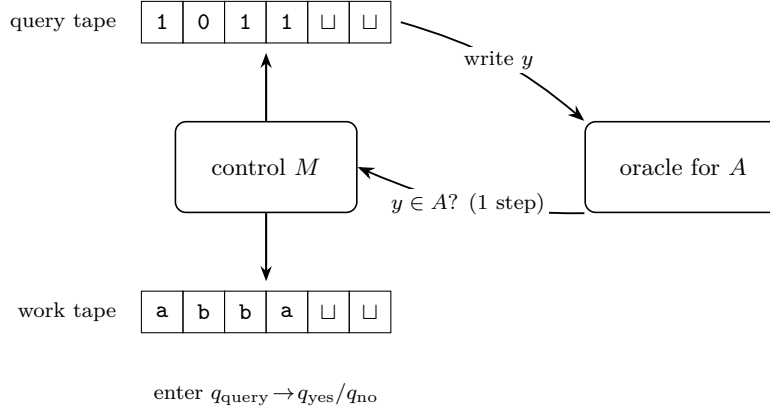
15.2 Relativization and the Limits of Diagonalization

The hierarchy theorems of the previous section separated complexity classes by *diagonalization*: build a machine D that simulates each bounded machine M_i in turn and deliberately disagrees with it. It is tempting to try the same trick on P versus NP – run every polynomial-time machine and beat it with a nondeterministic one. This section explains why that hope is misplaced. We give a machine an *oracle* (a free membership test for some fixed language), study the relativized classes P^A and NP^A , and prove the Baker–Gill–Solovay theorem: one oracle makes the two classes equal, another makes them unequal. Because plain diagonalization survives the addition of *any* oracle, no such proof can decide a question whose answer flips between oracles. The moral is sharp: settling P versus NP requires techniques that do *not* relativize.

Oracle machines and relativized classes

Definition 15.2.1 (Oracle, oracle Turing machine). An *oracle* for a language $A \subseteq \Sigma^*$ is a black box that, given any string y , reports in one step whether $y \in A$. An *oracle Turing machine* M^A is an ordinary Turing machine augmented with a write-only *query tape* and three distinguished states q_{query} , q_{yes} , q_{no} . When M^A enters q_{query} with a string y written on the query tape, in the *next* step the control jumps to q_{yes} if $y \in A$ and to q_{no} if $y \notin A$; the query tape is then erased. The machine M is fixed, but its behavior M^A depends on the oracle A plugged in.

A single query costs one step no matter how long y is or how hard membership in A would be to decide unaided. That is the whole point: the oracle hands over answers “for free,” so a polynomial-time machine may consult A on polynomially many strings, each of polynomial length.



Definition 15.2.2 (Relativized classes). Let A be a language. Define

$$P^A = \{\mathcal{L} \mid \mathcal{L} \text{ is decided by some polynomial-time deterministic } M^A\},$$

and define NP^A the same way with a polynomial-time *nondeterministic* oracle machine. We read P^A as “ P relative to A ” and call A the oracle. With the empty oracle (no useful queries) we recover $P^\emptyset = P$ and $NP^\emptyset = NP$.

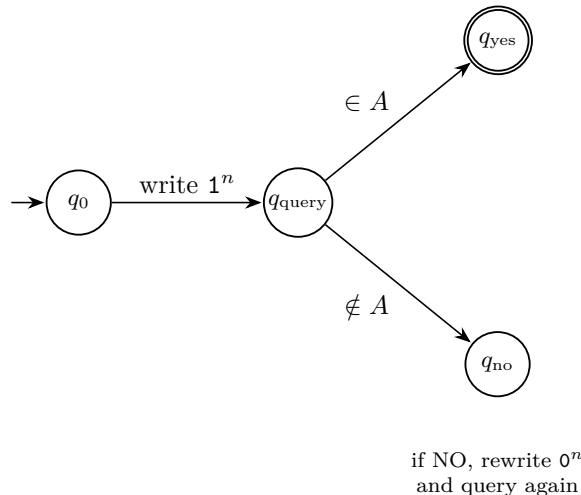
Proposition 15.2.3 (A strong oracle collapses NP and $coNP$ into P^A). $NP \subseteq P^{SAT}$ and $coNP \subseteq P^{SAT}$.

Proof. Let $\mathcal{L} \in NP$. Since SAT is NP -complete, there is a polynomial-time computable reduction f with $w \in \mathcal{L} \iff f(w) \in SAT$. The oracle machine M^{SAT} that, on input w , computes $f(w)$, writes it on the query tape, and accepts iff the oracle says $f(w) \in SAT$, runs in polynomial time and decides $\mathcal{L}$. Hence $\mathcal{L} \in P^{SAT}$. For $coNP$: if $\mathcal{L} \in coNP$ then $\overline{\mathcal{L}} \in NP \subseteq P^{SAT}$, and P^{SAT} is closed under complement (a deterministic decider can swap its accept and reject answers), so $\mathcal{L} \in P^{SAT}$. $\square$

Worked Example 15.2.4 (Reading an oracle machine M^A run by run). Fix $A = \{11, 0000\}$. Trace the oracle machine that, on input 1^n , queries 1^n and then queries 0^n , accepting iff at least one query returns YES. What language does it decide?

Recognition. An oracle machine is an ordinary machine plus a query tape whose contents are judged by one membership test (Definition 15.2.1). The trigger “what does M^A decide” means: pin down A , then resolve each query as a single YES/NO step.

Solution. Two queries fire per input; draw the control as a tiny automaton whose two branch states are the oracle’s verdicts.



On $1^2 = 11$: query $11 \in A$? YES, so accept. On 1^4 : query $1111 \in A$? NO; then query $0000 \in A$? YES, so accept. On $1^1 = 1$: query $1 \in A$? NO; query $0 \in A$? NO, so reject. On 1^3 : both 111 and 000 are absent, reject. So the machine accepts exactly 1^n with $1^n \in A$ or $0^n \in A$, i.e. $n \in \{2, 4\}$, giving the finite language $\{11, 1111\}$.

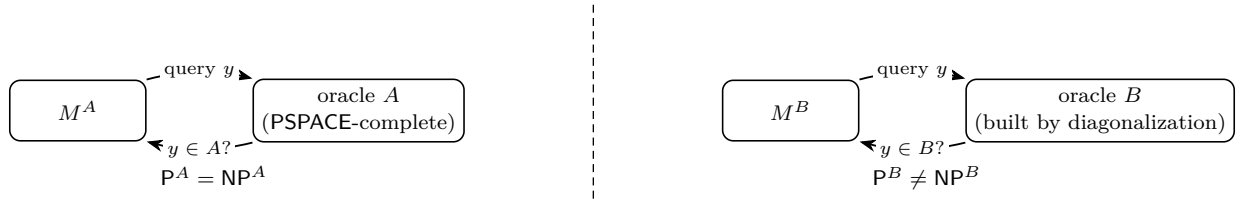
Which idea, and why? Each query is one step that branches the computation on membership in the *fixed* oracle A ; tracing M^A is just resolving those branches with the given A .

Check. Only two strings of A have the all-1 or all-0 shape the machine probes (11 directly, 0000 via the second query at $n = 4$), matching the two accepted inputs.

The Baker–Gill–Solovay theorem

Theorem 15.2.5 (Baker–Gill–Solovay). *There is an oracle A with $P^A = NP^A$, and there is an oracle B with $P^B \neq NP^B$.*

The theorem is best read as two parallel “worlds,” each a machine plugged into the *same kind* of query box – an oracle – yet answering the P vs NP question oppositely.



Takeaway. There are oracles A with $P^A = NP^A$ and B with $P^B \neq NP^B$ (Baker–Gill–Solovay): any proof that “relativizes” holds for *all* oracles, so it cannot settle P vs NP – which is why diagonalization alone is not enough. This is an oracle-relative statement and makes *no* claim that $P \neq NP$ absolutely.

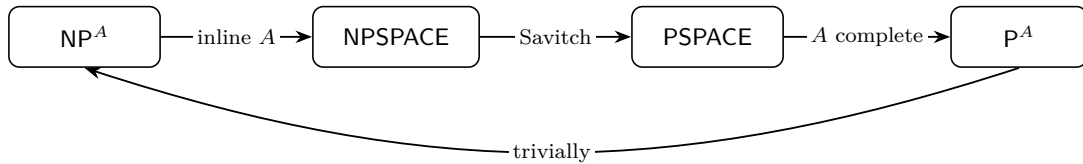
The two halves are proved by completely different means. For the collapsing oracle we borrow a *powerful* language so that both classes balloon up to PSPACE and meet. For the separating oracle we *build* the language by diagonalization, planting a string the polynomial-time machine cannot find in time.

Proof of the collapsing oracle A. Let A be any PSPACE-complete language (for instance $TQBF$, the true quantified Boolean formulas). We show the chain

$$NP^A \subseteq NPSpace \subseteq PSPACE \subseteq P^A,$$

which forces $P^A = NP^A = PSPACE$ (the reverse inclusion $P^A \subseteq NP^A$ is automatic).

First inclusion. Let $\mathcal{L} \in NP^A$ via a nondeterministic polynomial-time oracle machine N^A . Replace each oracle call by an in-line computation: since $A \in PSPACE$, a query “ $y \in A$?” can be answered in space polynomial in $|y|$, and $|y|$ is polynomial in the input length because N runs in polynomial time. The resulting machine is a nondeterministic polynomial-space decider for $\mathcal{L}$, so $\mathcal{L} \in NPSpace$. *Second inclusion.* Savitch’s theorem gives $NPSpace \subseteq PSPACE$. *Third inclusion.* If $\mathcal{L} \in PSPACE$, then since A is PSPACE-complete there is a polynomial-time reduction f with $w \in \mathcal{L} \iff f(w) \in A$; the oracle machine that computes $f(w)$ and asks the A -oracle decides $\mathcal{L}$ in polynomial time, so $\mathcal{L} \in P^A$. Stringing the inclusions together, $NP^A \subseteq P^A$, hence equality. $\square$



Proof of the separating oracle B. Define, for any oracle B , the unary “does B have a length- n string” language

$$L_B = \{1^n \mid \exists x \in B \text{ with } |x| = n\}.$$

$L_B \in \mathbf{NP}^B$ for every B . On input 1^n a nondeterministic oracle machine guesses a string x of length n (in n steps), queries $x \in B$, and accepts iff the answer is YES. Some branch accepts iff B holds a string of length n , so this machine decides L_B in nondeterministic polynomial time.

We build B in stages so that $L_B \notin \mathbf{P}^B$. List all polynomial-time *deterministic* oracle machines $M_1, M_2, \dots$; we may assume M_i runs in time n^i on inputs of length n . We decide membership in B string by string, keeping a finite set of commitments. Stage i defeats M_i .

Stage i . Only finitely many strings have been committed so far. Choose a length n larger than every committed string and large enough that $2^n > n^i$. Run M_i on input 1^n . Whenever M_i queries a string y :

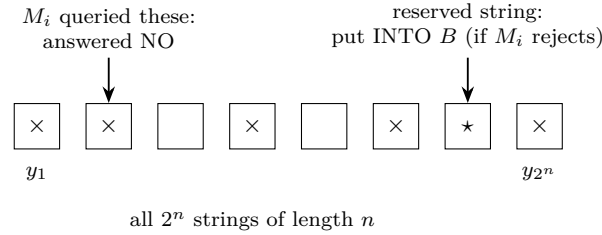
- if y 's membership is already committed, answer consistently;
- otherwise answer NO and commit $y \notin B$.

M_i runs for at most $n^i < 2^n$ steps, so it queries fewer than 2^n of the 2^n strings of length n ; *at least one* length- n string is left untouched. When M_i^B halts, flip its answer:

- if M_i *accepts* 1^n , commit *all* length- n strings to be *out* of B , making $1^n \notin L_B$;
- if M_i *rejects* 1^n , take an unqueried length- n string and commit it *into* B , making $1^n \in L_B$.

Either way $M_i^B(1^n)$ disagrees with L_B on 1^n . Crucially, the flip touches only length- n strings whose status M_i never inspected, so it is consistent with every answer already given. Commit all remaining undecided strings of length $\leq n$ to be out of B , and proceed to stage $i + 1$.

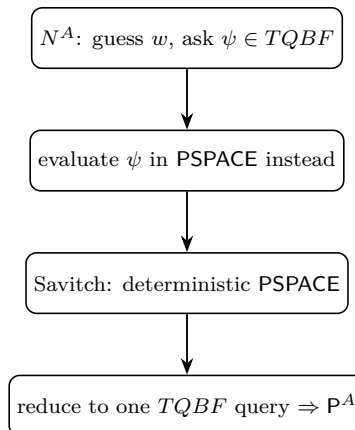
Each M_i^B fails to decide L_B , so $L_B \notin \mathbf{P}^B$. Since $L_B \in \mathbf{NP}^B$, we conclude $\mathbf{P}^B \neq \mathbf{NP}^B$. $\square$



Worked Example 15.2.6 (Verifying the collapse $\mathbf{P}^A = \mathbf{NP}^A$ on a concrete query). With $A = \mathbf{TQBF}$, suppose a nondeterministic oracle machine N^A decides a language by guessing a witness of length $\leq n^2$ and making a single oracle call on a quantified formula ψ of length $\leq n^3$. Argue concretely that the language it decides lies in $\mathbf{P}^A$, illustrating the three-step chain.

Recognition. The collapse proof (Theorem 15.2.5) turns oracle calls to a $\mathbf{PSPACE}$ -complete A into in-line $\mathbf{PSPACE}$ work, then deterministic via Savitch, then back to a single A -query. The trigger is “ A is $\mathbf{PSPACE}$ -complete,” which is exactly the hypothesis that makes the chain run.

Solution. Follow the formula ψ through the chain.



First, replace the oracle call: evaluating whether the length- n^3 formula ψ is true is a PSPACE computation (alternately try the two truth values of each quantified variable using a stack of depth n^3), so the whole machine becomes nondeterministic polynomial *space* – it is in NPSPACE. Second, Savitch’s theorem squares the space, $\text{NPSPACE} \subseteq \text{PSPACE}$, giving a *deterministic* polynomial-space decider. Third, that decider’s language is in PSPACE, and $TQBF$ is PSPACE-complete, so a single polynomial-time reduction maps each input to one $TQBF$ instance answered by one oracle call. The net machine is a deterministic polynomial-time oracle machine, witnessing membership in P^A .

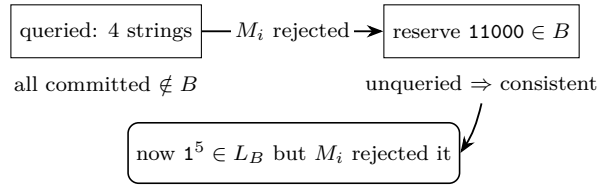
Which idea, and why? When the oracle is as powerful as the surrounding space class, the nondeterminism buys nothing the space could not already buy – the oracle, not the guessing, dominates – so NP^A deflates to P^A .

Check. The polynomial bounds survive each step: length n^3 in, PSPACE work, (n^3) -space squared by Savitch is still polynomial, one final query – nowhere does anything become exponential.

Worked Example 15.2.7 (Running one stage of the construction of B). Carry out stage i for a machine M_i with time bound $n^i = n^2$, taking $n = 3$ (so $2^3 = 8 > 9$ fails – pick $n = 4$ instead, where $2^4 = 16 > 16$ also fails – so pick $n = 5$, where $2^5 = 32 > 25$). Suppose at $n = 5$ the machine M_i on input 1^5 queries the four strings 00000, 11111, 10101, 01010, all answered NO, and then *rejects*. Show how to extend B to defeat it.

Recognition. The separating construction (Theorem 15.2.5) reserves an unqueried string of the chosen length and flips the machine’s verdict. The trigger is “ M_i rejects after too few queries,” which means a length- n string is still free to be planted in B .

Solution. At $n = 5$ there are $2^5 = 32$ binary strings of length 5, and M_i inspected only $4 < 32$ of them, all set to NO. Because M_i rejected 1^5 , it “believes” no length-5 string is in B , i.e. $1^5 \notin L_B$. We make that belief false: pick any unqueried length-5 string, say 11000, and commit $11000 \in B$.



Now B has a length-5 string, so $1^5 \in L_B$, yet $M_i^B(1^5)$ rejected – a disagreement. Since 11000 was never queried, every answer already given to M_i remains valid, so M_i would indeed have rejected. Finish the stage by committing the other 31 length-5 strings (and any shorter undecided strings) to be out of B , then move to stage $i + 1$ with a fresh, larger length.

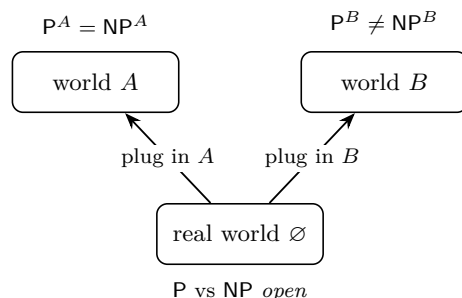
Which idea, and why? The counting inequality $n^i < 2^n$ guarantees an unqueried string; planting it (or withholding all of them) reverses the machine’s verdict on 1^n without contradicting any answer it already saw.

Check. M_i made 4 queries, well under its $n^i = 25$ step budget, which is itself under $2^n = 32$ – so a free length-5 string is guaranteed, exactly as the inequality promises.

Worked Example 15.2.8 (Why the two oracles do not contradict each other). A student protests: “You proved $\text{P}^A = \text{NP}^A$ and $\text{P}^B \neq \text{NP}^B$. Doesn’t substituting both into the ‘same’ statement give a contradiction?” Resolve the worry.

Recognition. The objects P^A and P^B are *different* classes, indexed by different oracles. The trigger “contradiction between two oracle results” is the classic error of treating a relativized claim as if it were the unrelativized one.

Solution. The unrelativized question asks about $\text{P} = \text{P}^\emptyset$ versus $\text{NP} = \text{NP}^\emptyset$ – one fixed pair of classes. Each Baker–Gill–Solovay oracle replaces the empty oracle by something else, producing a *new* universe of computation.



There is no single proposition into which both A and B are substituted; the two theorems live in separate worlds and never meet. What they jointly imply is about *proof techniques*, not about P versus NP itself: see the corollary below.

Which idea, and why? A relativized equation is a statement *within* the world of one oracle; P^A and P^B are not the same class, so disagreeing verdicts in different worlds is no paradox.

Check. Neither result mentions the empty oracle, so neither claims anything about the actual classes P and NP – consistent with that question still being open.

What “relativizes,” and the moral

Definition 15.2.9 (A proof relativizes). A proof of an inclusion or separation between complexity classes *relativizes* if it remains valid when every machine in it is handed the same oracle O – that is, the same argument proves the corresponding statement about the relativized classes, for every oracle O .

Pure diagonalization relativizes. Its engine is one machine simulating another and then disagreeing; if both machines carry the same oracle O , the simulator simply forwards each query to O and reads back the same answer, so the simulation – and the disagreement – proceed exactly as before. Nothing in the argument inspected the *internal structure* of the simulated computation; it treated the machine as a black box, which is precisely why an oracle does not disturb it.

Corollary 15.2.10 (Diagonalization alone cannot settle P vs NP). *No proof that relativizes can establish $P = NP$, and none can establish $P \neq NP$.*

Proof. Suppose a relativizing proof showed $P = NP$. Relativizing, it would show $P^O = NP^O$ for *every* oracle O , in particular for $O = B$ from Theorem 15.2.5, contradicting $P^B \neq NP^B$. Symmetrically, a relativizing proof of $P \neq NP$ would force $P^A \neq NP^A$, contradicting $P^A = NP^A$. So no relativizing proof can land on either answer. $\square$

Because plain diagonalization relativizes, it is one of the techniques ruled out by Corollary 15.2.10. Settling P versus NP therefore demands an argument that *looks inside* computations – something that fails to relativize, such as the arithmetization behind $IP = PSPACE$ or the combinatorics of circuit lower bounds – rather than merely simulating machines from the outside.

Method (How to run an oracle construction by stages).

1. **Fix the target language.** Define the language L_B you want to land in the easy upper class (here NP^B) for *every* oracle, and verify that membership is immediate from the oracle’s bits.
2. **Enumerate the adversaries.** List the machines $M_1, M_2, \dots$ of the class you want to escape (here polynomial-time deterministic oracle machines), with an explicit time bound n^i for M_i .
3. **One stage per adversary, on a fresh scale.** At stage i pick a length n exceeding all committed strings and satisfying the counting margin ($2^n > n^i$), so the adversary cannot probe everything.
4. **Simulate, answering minimally.** Run M_i on the diagonal input; answer each new query NO (the cheapest commitment), respecting prior commitments.
5. **Flip the verdict using a reserved string.** Use an unqueried string (guaranteed by the margin) to make L_B disagree with the adversary’s output on that input.

6. **Freeze and advance.** Commit all remaining short strings, then go to stage $i + 1$. The union of all stages defines B ; each M_i^B fails, so L_B escapes the class.

Caution. “Relativizes” is a property of a *proof*, not of a theorem. A theorem can be true while every *known* proof of it relativizes; the barrier says only that a relativizing proof cannot decide a question whose relativized answer flips, like P vs NP. It does not say the question is undecidable or unprovable – only that one family of methods is too coarse.

Caution. The two oracles A and B are not in tension. $P^A = NP^A$ and $P^B \neq NP^B$ are statements about *different* machine universes; there is no single equation accepting both substitutions. Treating P^A and P^B as “the same P” is exactly the slip that makes the result look contradictory. Likewise, neither result says anything about the unrelativized P and NP, which remain open.

Exercises.

- Fix the oracle $A = \{0, 010, 0101010\}$ (the strings of the form $(01)^*0$ of odd length 1, 3, 7). Trace the oracle machine that, on input 1^n , writes the string $(01)^k0$ where $n = 2k + 1$ and queries it (rejecting at once if n is even). For which $n \leq 9$ does it accept? State the language it decides.
- Draw the schematic of an oracle Turing machine: a control box, a work tape, a write-only query tape, and the oracle box, with arrows for the query state q_{query} and the two answer states $q_{\text{yes}}, q_{\text{no}}$. Mark which single step the membership test occupies.
- Show that P^A is closed under complement for every oracle A : given a polynomial-time deterministic oracle machine M^A deciding $\mathcal{L}$, describe a machine deciding $\bar{\mathcal{L}}$ with the same oracle and time bound. Where exactly does determinism get used?
- Prove that for every oracle A the chain $P^A \subseteq NP^A$ holds, and give a one-sentence reason why the reverse inclusion is the open question in the relativized world (for a generic A).
- Let A be a PSPACE-complete language. In the collapse proof we used the inclusions $NP^A \subseteq NPSpace \subseteq PSPACE \subseteq P^A$. State precisely which theorem justifies the *middle* inclusion, and explain in one sentence why the *first* inclusion needs A to be in PSPACE (not just PSPACE-hard).
- In the separating construction, stage i requires a length n with $2^n > n^i$. For $i = 4$ (time bound n^4), find the smallest integer $n \geq 2$ satisfying $2^n > n^4$, and explain why *some* such n exists for every fixed i .
- Run one stage of the construction of B by hand. Take M_i with bound $n^i = n^3$ and choose $n = 10$. Suppose M_i on 1^{10} queries the strings 0^{10} , 1^{10} , and 10^9 (all answered NO) and then *accepts*. Decide whether to put a length-10 string into B or to keep all of them out, justify the choice so that M_i^B is wrong on 1^{10} , and state the resulting membership of 1^{10} in L_B .
- Prove carefully that $L_B = \{1^n \mid \exists x \in B, |x| = n\}$ is in NP^B for *every* oracle B . Your machine must run in nondeterministic polynomial time; state the guess, the single oracle query, and the acceptance condition.
- Explain why the two Baker–Gill–Solovay oracles do *not* contradict each other. Then state, in one precise sentence, the *only* thing the pair lets us conclude about the unrelativized P versus NP question.
- Find the bug. A classmate argues: “Diagonalization proved the time hierarchy theorem $P \subsetneq EXPTIME$, a genuine separation. So apply the same diagonalization to separate P from NP and we are done.” Identify the false step using the notion of relativization, and name which Baker–Gill–Solovay oracle blocks the conclusion.
- Find the bug. A proof sketch claims to settle the relativized world for *all* oracles at once: “Take any oracle O . By Savitch, $NP^O \subseteq NPSpace \subseteq PSPACE \subseteq P^O$, hence $P^O = NP^O$ for every O .” This contradicts the existence of the separating oracle B . Pinpoint exactly which inclusion in the chain fails for a general oracle O and why (consider what answering a query to O costs in space when $O \notin PSPACE$).
- Define an oracle machine to be *benign* if on every input it makes at most one oracle query. Show that for the SAT oracle, $NP \subseteq P^{SAT}$ is already witnessed by benign machines, and explain why $coNP \subseteq P^{SAT}$ likewise needs only one query plus a swap of the accept/reject answers.

13. Let $U = \{1^n \mid n \text{ is even}\}$, a fixed unary language not depending on any oracle. Show $U \in P^A$ for every oracle A (the machine may ignore the oracle entirely), and contrast this with L_B , whose membership genuinely depends on the oracle's bits. What does this contrast say about why L_B , not U , is the right language to diagonalize against?

15.3 Approximation Algorithms

When a problem is NP-hard we cannot, unless $P = NP$, compute an exact optimum in polynomial time. The escape route this section explores is to lower our sights: aim for a solution *provably close* to optimal, in polynomial time. The one idea that makes the analysis work is to bound our algorithm's output not against the (hard-to-compute) optimum directly, but against a *computable lower or upper bound* on the optimum – a quantity we can hold in our hand. We develop this for three landmark cases (vertex cover, max-cut, max-3sat), and close with an honest word on the limits: for some problems even a modest approximation ratio is itself NP-hard.

Optimization problems and the approximation ratio

The decision problems *VERTEX-COVER*, *CLIQUE*, and so on each have an *optimization* twin. Instead of asking “does G have a vertex cover of size $\leq k$?” we ask “what is the *smallest* vertex cover of G ?” and want the cover itself, not a yes/no.

Definition 15.3.1 (Optimization problem). An *optimization problem* Π assigns to each instance x a set of *feasible solutions* $\text{Sol}(x)$ and a *cost* (or *value*) $\text{cost}(x, s) \in \mathbb{R}^+$ for each $s \in \text{Sol}(x)$. In a *minimization* problem we seek $\text{OPT}(x) = \min_s \text{cost}(x, s)$; in a *maximization* problem, $\text{OPT}(x) = \max_s \text{cost}(x, s)$. An algorithm *solves* Π if on input x it outputs a feasible s with $\text{cost}(x, s) = \text{OPT}(x)$.

Definition 15.3.2 (Approximation ratio). Let A be a polynomial-time algorithm that, on instance x , returns a feasible solution of cost $A(x)$. For $\rho \geq 1$ we say A is a ρ -*approximation* if for every x

$$(\text{minimization}) \quad A(x) \leq \rho \cdot \text{OPT}(x), \quad (\text{maximization}) \quad A(x) \geq \frac{1}{\rho} \cdot \text{OPT}(x).$$

The number ρ is the *approximation ratio* (or *performance guarantee*). A ratio of 1 would be exact; the closer ρ is to 1, the better.

The convention $\rho \geq 1$ unifies both directions: a minimizer's output is at most ρ times too *large*, a maximizer's at least $1/\rho$ times the best – in each case the gap from optimum is the factor ρ . We sometimes call a minimization algorithm with ratio k a k -*optimal* or *factor- k* approximation. For maximization problems it is also customary to quote the fraction $1/\rho \leq 1$ instead; in that form the MAX-CUT and MAX-3SAT algorithms below are called $\frac{1}{2}$ - and $\frac{7}{8}$ -approximations (that is, $\rho = 2$ and $\rho = \frac{8}{7}$).

Method (The one move behind every guarantee in this section). You cannot compare $A(x)$ to $\text{OPT}(x)$ directly – $\text{OPT}(x)$ is what you could not compute. Instead:

1. **Find a computable bound on OPT.** For a minimizer, exhibit a quantity L you *can* compute with $L \leq \text{OPT}(x)$. For a maximizer, a quantity U with $\text{OPT}(x) \leq U$ (often U = total number of constraints/edges).
2. **Bound your output by the same quantity.** Show $A(x) \leq \rho \cdot L$ (minimizer) or $A(x) \geq \frac{1}{\rho} \cdot U$ (maximizer).
3. **Chain.** Then $A(x) \leq \rho L \leq \rho \text{OPT}(x)$, or $A(x) \geq \frac{1}{\rho} U \geq \frac{1}{\rho} \text{OPT}(x)$. The unknown OPT never appears in the computation, only in the inequality.

A 2-approximation for minimum vertex cover

Recall a *vertex cover* of an undirected graph $G = (V, E)$ is a set $C \subseteq V$ such that every edge has at least one endpoint in C . *MIN-VC* asks for a smallest such C . Finding it exactly is NP-hard, yet the following two-line algorithm comes within a factor of 2.

$A =$ “On input $\langle G \rangle$ with $G = (V, E)$ undirected:

1. Set $C \leftarrow \emptyset$ and let $M \leftarrow \emptyset$ (the edges we pick).
2. While some edge $\{u, v\}$ has *both* endpoints outside C : add $\{u, v\}$ to M , and put *both* u and v into C .
3. Output C .”

The picked edges M form a *matching*: no two of them share an endpoint, because once an edge goes into M both its endpoints enter C and are thereafter ineligible. And M is *maximal* – when the loop stops, every remaining edge has an endpoint in C , hence touches a matched edge.

Theorem 15.3.3 (Vertex-cover 2-approximation). *A runs in polynomial time and returns a vertex cover C with $|C| \leq 2 \cdot \text{OPT}$, where OPT is the size of a minimum vertex cover of G .*

Proof. C is a cover. Each iteration inspects every edge once, so the algorithm is polynomial. When the loop halts, no edge has both endpoints outside C ; equivalently every edge has an endpoint in C . So C covers E .

The lower bound on OPT . Let Y be any minimum vertex cover, $|Y| = \text{OPT}$. The matching M consists of pairwise vertex-disjoint edges. Y must cover each edge of M , so for every $e \in M$ at least one endpoint of e lies in Y . Because the edges of M share no endpoints, these chosen endpoints are *distinct*, one per matched edge. Hence

$$\text{OPT} = |Y| \geq |M|.$$

Here $|M|$ is the computable lower bound on OPT that the Method calls for.

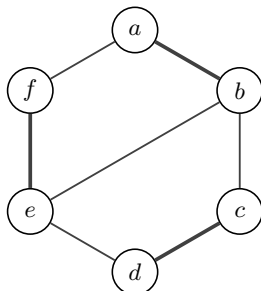
The upper bound on A . Each matched edge contributed exactly two new vertices to C , so $|C| = 2|M|$. Chaining,

$$|C| = 2|M| \leq 2 \cdot \text{OPT}. \quad \square$$

Worked Example 15.3.4 (Running the matching 2-approximation on a concrete graph). Run A on the graph G with vertices $\{a, b, c, d, e, f\}$ and edges $\{a, b\}, \{b, c\}, \{c, d\}, \{d, e\}, \{e, f\}, \{a, f\}$ (a 6-cycle) plus the chord $\{b, e\}$. Compare the cover it returns to a true minimum cover.

Recognition. The trigger is “vertex cover, guarantee wanted, no time for exact.” Apply Theorem 15.3.3: greedily pick disjoint edges, take both endpoints, and read the factor-2 bound off $|M|$.

Solution. Suppose the loop first picks $\{a, b\}$, then – among edges still uncovered – $\{c, d\}$, then $\{e, f\}$. After those three picks every edge has an endpoint in C , so the loop halts.



bold = matching M ; all six matched endpoints form C

The matching is $M = \{\{a, b\}, \{c, d\}, \{e, f\}\}$, so $|M| = 3$ and the algorithm outputs $C = \{a, b, c, d, e, f\}$ with $|C| = 6 = 2|M|$. The guarantee says $\text{OPT} \geq |M| = 3$. In fact a minimum cover here is $Y = \{b, d, f\}$ (check: b kills $\{a, b\}, \{b, c\}, \{b, e\}$; d kills $\{c, d\}, \{d, e\}$; f kills $\{e, f\}, \{a, f\}$), so $\text{OPT} = 3$ and A returned $6 = 2 \cdot 3$.

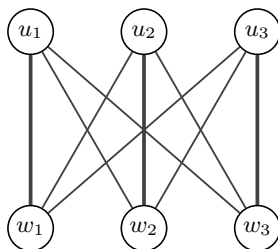
Which idea, and why? The matching size is a lower bound on every cover, and the algorithm’s output is exactly twice the matching; the two facts pinch the ratio to 2.

Check. $|C|/\text{OPT} = 6/3 = 2 \leq 2$, consistent with Theorem 15.3.3; and C really is a cover (every one of the seven edges touches $\{a, \dots, f\}$, trivially).

Worked Example 15.3.5 (A tight example forcing exactly $2 \cdot \text{OPT}$). Exhibit an infinite family on which A may output a cover of size exactly $2 \cdot \text{OPT}$, so the factor 2 in Theorem 15.3.3 cannot be lowered for this algorithm.

Recognition. The trigger is “show a bound is tight”: build an instance where the matching M is forced to be large while a tiny cover exists. A complete bipartite graph $K_{m,m}$ does this.

Solution. Take $K_{m,m}$ with left part $\{u_1, \dots, u_m\}$ and right part $\{w_1, \dots, w_m\}$; every u_i joins every w_j . A perfect matching $\{u_1, w_1\}, \dots, \{u_m, w_m\}$ is maximal, so A can pick all m of those edges and output all $2m$ vertices. But either entire side – say $\{u_1, \dots, u_m\}$ – already covers every edge, so $\text{OPT} = m$.



$K_{3,3}$: bold perfect matching forces $C = \text{all 6 vertices}$, yet top row alone covers

Thus $|C| = 2m = 2 \text{OPT}$ exactly. As $m \rightarrow \infty$ the ratio stays pinned at 2, so no sharper constant works for this algorithm.

Which idea, and why? Tightness needs an instance where the maximal matching is as big as it can be relative to OPT ; $K_{m,m}$ has a size- m matching but a size- m cover, and A pays for both endpoints of all m matched edges.

Check. For $m = 3$: A outputs 6 vertices, $\text{OPT} = 3$, ratio $6/3 = 2$ – the worst the theorem permits, attained.

Caution. The ratio must point the right way. For a *minimization* problem the guarantee is $A(x) \leq \rho \cdot \text{OPT}$ with $\rho \geq 1$: the output is an *over-estimate*. Writing “ $A(x) \geq \frac{1}{2} \text{OPT}$ ” for vertex cover says nothing useful – a cover is always at least as large as the minimum. Likewise for *maximization* the bound is $A(x) \geq \frac{1}{\rho} \text{OPT}$ (an *under-estimate*), never $A(x) \leq \rho \text{OPT}$, which holds trivially. Fix the direction by asking: “which way can my heuristic err?”

A $\frac{1}{2}$ -approximation for MAX-CUT by local search

A *cut* of $G = (V, E)$ splits V into two parts (S, T) ; an edge is *cut* if it crosses, with one endpoint in S and one in T . *MAX-CUT* asks for a cut with the most crossing edges; it too is **NP-hard**. The local-search heuristic is: start anywhere, and flip a vertex to the other side whenever doing so creates more crossing edges.

$B =$ “On input $\langle G \rangle$ with $G = (V, E)$:

1. Start with any cut (S, T) , e.g. $S = \emptyset, T = V$.
2. While some vertex, moved to the opposite side, *increases* the number of cut edges: move it; repeat.
3. Output the current cut.”

Theorem 15.3.6 (MAX-CUT local-search $\frac{1}{2}$ -approximation). B runs in polynomial time and outputs a cut whose size is at least $\frac{1}{2} |E|$, hence at least $\frac{1}{2} \text{OPT}$.

Proof. Each move strictly increases the cut size, an integer bounded above by $|E|$, so at most $|E|$ moves occur and B halts in polynomial time. Now consider any vertex v at the moment B stops, and let $c(v)$ be the number of cut edges at v and $u(v)$ the number of uncut (same-side) edges at v . If $u(v) > c(v)$, moving v would convert its $u(v)$ uncut edges to cut and its $c(v)$ cut edges to uncut, a net gain of $u(v) - c(v) > 0$ –

contradicting that B halted. So at *every* vertex $c(v) \geq u(v)$. Sum over all vertices. Each cut edge is counted twice in $\sum_v c(v)$ (once per endpoint) and likewise each uncut edge twice in $\sum_v u(v)$, so

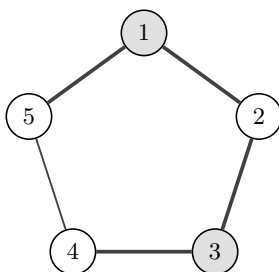
$$2(\# \text{ cut edges}) = \sum_v c(v) \geq \sum_v u(v) = 2(\# \text{ uncut edges}).$$

Thus the cut has at least as many edges as it leaves uncut, so it contains at least half of all $|E|$ edges. Since $\text{OPT} \leq |E|$ (a cut cannot beat every edge), the output is $\geq \frac{1}{2}|E| \geq \frac{1}{2}\text{OPT}$. $\square$

Worked Example 15.3.7 (Local search on a 5-cycle). Run B on the cycle C_5 with vertices 1, 2, 3, 4, 5 in a ring. Track the cut size as vertices flip, and compare to OPT .

Recognition. “Improve a cut by single flips” is the local-move trigger of Theorem 15.3.6. Start from the worst cut and let each flip pay off until none does.

Solution. Begin $S = \emptyset$, $T = \{1, 2, 3, 4, 5\}$: zero cut edges. Flip 1 into S : its two edges $\{1, 2\}$ and $\{1, 5\}$ now cross, cut size 2. Flip 3 into S : edges $\{2, 3\}$ and $\{3, 4\}$ cross, cut size 4. Now every vertex has $c(v) \geq u(v)$, so B stops with $S = \{1, 3\}$, $T = \{2, 4, 5\}$.



shaded = $S = \{1, 3\}$; bold = cut edges (4 of 5)

The output cut has 4 crossing edges out of 5. The theorem only promised $\geq \frac{1}{2} \cdot 5 = 2.5$, i.e. at least 3; we got 4. The true optimum for an odd cycle is $\text{OPT} = 4$ (one edge must stay uncut), so here local search actually hit the optimum.

Which idea, and why? At a local maximum each vertex has at least as many crossing as same-side edges; summing that node inequality gives “cut $\geq$ half the edges,” the engine of the $\frac{1}{2}$ guarantee.

Check. Every vertex: vertex 4 has edges to 3 (cut) and 5 (uncut), so $c = 1 = u$, no improving flip; the same balance or better holds at all five, confirming a local maximum.

A $\frac{7}{8}$ -approximation for MAX-3SAT by random assignment

In *MAX-3SAT* we are given a 3CNF formula – a conjunction of clauses, each an OR of exactly three literals over distinct variables – and we want an assignment satisfying the *most* clauses. The brute-force-free heuristic is almost embarrassingly simple: assign each variable true/false by an independent fair coin.

Theorem 15.3.8 (Random assignment satisfies $\frac{7}{8}$ in expectation). *Let φ be a 3CNF formula with m clauses, each having three distinct variables. If each variable is set true or false independently with probability $\frac{1}{2}$, the expected number of satisfied clauses is $\frac{7}{8}m$. Consequently some assignment satisfies at least $\frac{7}{8}m$ clauses, and since $\text{OPT} \leq m$, this is a $\frac{7}{8}$ -approximation in expectation.*

Proof. Fix a clause $C_j = (\ell_1 \vee \ell_2 \vee \ell_3)$ on three distinct variables. C_j is *unsatisfied* only when all three literals are false, and each literal is false with probability $\frac{1}{2}$ independently, so

$$\Pr[C_j \text{ unsatisfied}] = \left(\frac{1}{2}\right)^3 = \frac{1}{8}, \quad \Pr[C_j \text{ satisfied}] = 1 - \frac{1}{8} = \frac{7}{8}.$$

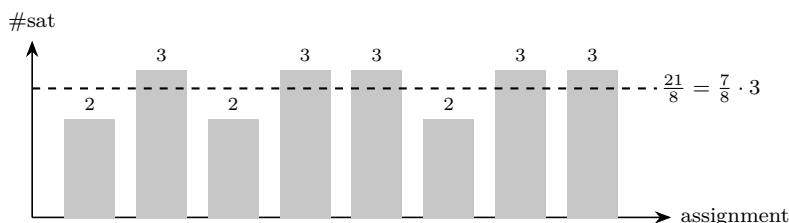
Let X_j be the indicator that C_j is satisfied, so $\mathbb{E}[X_j] = \frac{7}{8}$. By linearity of expectation (which needs *no* independence between clauses), the total $X = \sum_{j=1}^m X_j$ satisfies $\mathbb{E}[X] = \sum_j \mathbb{E}[X_j] = \frac{7}{8}m$. A random variable cannot always fall below its mean, so *some* outcome has $X \geq \mathbb{E}[X] = \frac{7}{8}m$ clauses satisfied. As $\text{OPT} \leq m$, that assignment satisfies at least $\frac{7}{8}m \geq \frac{7}{8}\text{OPT}$ clauses. $\square$

Worked Example 15.3.9 (Counting satisfied clauses over all assignments). Take $\varphi = (x \vee y \vee z) \wedge (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z)$, three clauses on variables x, y, z . Verify the $\frac{7}{8}$ figure by averaging over the $2^3 = 8$ assignments, and find an assignment matching the expectation bound.

Recognition. The trigger is “MAX-3SAT, expected coverage”: Theorem 15.3.8 says each clause fails on exactly one of its 8 local patterns, so the average over all assignments is $\frac{7}{8}$ per clause.

Solution. Each clause is falsified by exactly one of the 8 assignments (the one making all three of its literals false). Tabulate which assignment kills which clause, then count satisfied clauses per row:

x	y	z	C_1	C_2	C_3	#sat
0	0	0	0	1	1	2
0	0	1	1	1	1	3
0	1	0	1	1	0	2
0	1	1	1	1	1	3
1	0	0	1	1	1	3
1	0	1	1	0	1	2
1	1	0	1	1	1	3
1	1	1	1	1	1	3



The column totals: C_1 is satisfied by 7 of 8 assignments (it fails only at $x=y=z=0$), and likewise C_2 and C_3 each by 7. Summing satisfied clauses across the 8 rows gives $2 + 3 + 2 + 3 + 3 + 2 + 3 + 3 = 21$, so the average is $21/8 = \frac{7}{8} \cdot 3$, exactly the expectation. Any of the five rows with “#sat = 3” (e.g. $x=0, y=1, z=1$) beats the $\frac{7}{8}m = 2.625$ bound.

Which idea, and why? Each clause fails on exactly one of the 8 patterns of its three variables, so it is satisfied on 7/8 of them; linearity averages these per-clause 7/8’s into $\frac{7}{8}m$ overall.

Check. $21/8 = 2.625 = \frac{7}{8} \cdot 3$; and the maximum row value 3 means φ is in fact fully satisfiable, so here $\text{OPT} = 3$ and the random method’s 2.625 is $\frac{7}{8}$ of it.

An honest word on the limits: the PCP theorem

The three guarantees above raise a question they cannot answer: how good a ratio is *possible*? For some problems the answer is sobering. The **PCP theorem** (Arora–Safra and Arora–Lund–Motwani–Sudan–Szegedy, 1992) recasts NP as the languages whose membership proofs can be checked by a verifier that reads only a *constant* number of bits of the proof using $O(\log n)$ random bits. A celebrated consequence, due to Håstad, is that $\frac{7}{8}$ for MAX-3SAT is the *end of the road*: for every $\varepsilon > 0$, achieving ratio $\frac{7}{8} + \varepsilon$ in polynomial time would force $\text{P} = \text{NP}$. So the trivial random algorithm of Theorem 15.3.8 is, astonishingly, optimal among polynomial-time algorithms (assuming $\text{P} \neq \text{NP}$). Similar inapproximability is known for many problems: *CLIQUE* cannot be approximated within $n^{1-\varepsilon}$, and vertex cover cannot be approximated within $\sqrt{2} - \varepsilon$ (and, under a stronger conjecture, within $2 - \varepsilon$ – which would make our simple algorithm essentially best possible).

We are careful about what is *proved* versus *stated* here. We proved Theorems 15.3.3, 15.3.6, and 15.3.8 in full. The PCP theorem and the hardness-of-approximation results we only *state*: their proofs are deep and well beyond this book. The honest takeaway is that approximability is itself a rich complexity classification – some ratios are easy, others provably as hard as NP itself.

Caution. A good approximation does *not* transfer through a reduction. If $\Pi \leq_p \Pi'$ and you have a 2-approximation for Π' , you generally get *nothing* for Π : an ordinary polynomial-time reduction preserves

yes/no answers but mangles costs, so a near-optimal Π' solution can pull back to a terrible Π solution. Transferring guarantees needs a special *approximation-preserving* (e.g. L- or AP-) reduction that controls how the cost distorts. Never assume the standard NP-completeness reduction does this for you.

Exercises.

1. Let G be the “bowtie”: two triangles $\{a, b, c\}$ and $\{c, d, e\}$ sharing the vertex c (edges $\{a, b\}, \{b, c\}, \{a, c\}, \{c, d\}, \{d, e\}, \{c, e\}$). Run the matching 2-approximation A for vertex cover, drawing the graph and shading the matching M it picks and the cover C it outputs. Then find a minimum cover Y by hand and report $|C|$, $|M|$, and OPT . Is $|M| \leq \text{OPT} \leq |C| = 2|M|$?
2. State, for each problem, whether it is a minimization or a maximization problem, and write the defining inequality a ρ -approximation A must satisfy (using OPT , ρ , and $A(x)$): (a) MIN-VERTEX-COVER; (b) MAX-CUT; (c) shortest path between two fixed nodes; (d) MAX-INDEPENDENT-SET. For one of them, explain in a sentence why a bound written in the *opposite* direction would be vacuous.
3. Prove that the matching M built by algorithm A in Theorem 15.3.3 is always a *maximal* matching, and conversely that taking both endpoints of *any* maximal matching yields a vertex cover of size at most 2OPT . (So the guarantee depends only on maximality, not on which maximal matching is chosen.)
4. Construct a single graph on which two different runs of A – differing only in the order the “while” loop picks edges – output covers of *different* sizes. Draw both runs’ matchings, and confirm both outputs still satisfy $|C| \leq 2 \text{OPT}$. (Conclude that A is not deterministic in its output unless an edge order is fixed.)
5. Run the MAX-CUT local-search algorithm B on the complete graph K_4 starting from $S = \emptyset$, $T = \{1, 2, 3, 4\}$. Show a sequence of improving single-vertex flips, draw the final cut (shade S , bold the cut edges), and report the final cut size. Compare it to OPT for K_4 and to the guarantee $\frac{1}{2}|E|$.
6. Find the bug. A student argues: “Algorithm B for MAX-CUT keeps strictly increasing the cut by single flips, so when it halts the cut must be *maximum*.” Give a concrete graph and a starting cut where B halts at a cut that is *not* maximum, draw both the local optimum and a strictly larger cut, and state precisely the weaker property B actually guarantees.
7. Let φ be a 3CNF formula with m clauses, each on three distinct variables. Using the random-assignment analysis, prove that *if* $m \geq 1$, then some assignment satisfies at least one clause; more sharply, prove that every such φ has an assignment satisfying $\geq \lceil \frac{7}{8}m \rceil$ clauses. Where does linearity of expectation enter, and why is independence *across* clauses not needed?
8. (Derandomizing by conditional expectations.) For the formula $\varphi = (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee \bar{y} \vee \bar{z})$, find a single assignment satisfying $\geq \frac{7}{8} \cdot 3$ clauses *deterministically* as follows: set x to whichever of 0, 1 gives the larger conditional expected number of satisfied clauses, then y , then z , recomputing the conditional expectation at each step. Show your three conditional-expectation comparisons and the final assignment.
9. Suppose a clause is allowed to repeat a variable, e.g. $(x \vee x \vee y)$. Recompute the probability that a uniformly random assignment satisfies such a clause, and explain why the clean $\frac{7}{8}$ bound of Theorem 15.3.8 relied on the three literals being on *distinct* variables. What ratio do you get for 2-literal clauses $(x \vee y)$?
10. Carefully distinguish, in one paragraph, what the PCP theorem *proves* from what is merely *stated* in this section. In particular, is the claim “ $\frac{7}{8}$ is optimal for MAX-3SAT unless $\text{P} = \text{NP}$ ” a theorem proved here, a theorem proved elsewhere, or a conjecture? State the logical form of the inapproximability claim for *CLIQUE* (“cannot be approximated within $n^{1-\epsilon}$ unless ...”).
11. Find the bug. A student writes: “*VERTEX-COVER* $\leq_p$ *CLIQUE*, and there is a 2-approximation for *VERTEX-COVER*, so by following the reduction we get a 2-approximation for the maximum-clique optimization problem.” Identify the two distinct errors (one about direction of the reduction, one about what ordinary reductions preserve), and state what kind of reduction would be needed to transfer an approximation guarantee.
12. Give a 2-approximation algorithm for the following *minimization* problem and prove its ratio by the Method of this section. MIN-EDGE-DOMINATING-SET: given a graph G , choose a smallest set of edges F such that every *edge* of G is either in F or shares an endpoint with an edge in F (a dominating-style cover).

Hint: you need a computable lower bound on OPT; relate it to a maximal set of pairwise non-adjacent edges or vertices, mirroring the vertex-cover argument, and state clearly which quantity plays the role of $|M|$.

15.4 Probabilistic Computation

Every machine so far has been forced to commit to a single next move. A *probabilistic* machine is allowed, at certain steps, to flip a fair coin and let the outcome decide where to go – so its answer on a fixed input is now a random variable, and “deciding a language” must be replaced by “deciding it with high probability.” The one key idea of this section is that a tiny, controllable chance of error buys real algorithmic power: problems with no known deterministic algorithm nearly as simple (testing primality, checking a polynomial identity, finding a small cut) fall to short randomized procedures, and the error can be driven below any threshold by the cheap trick of repeating and voting. We make “coin-flip machine” precise, name the randomized classes BPP, RP, coRP, ZPP that sit around P and NP, prove the amplification lemma, and run the canonical examples end to end.

Probabilistic Turing machines

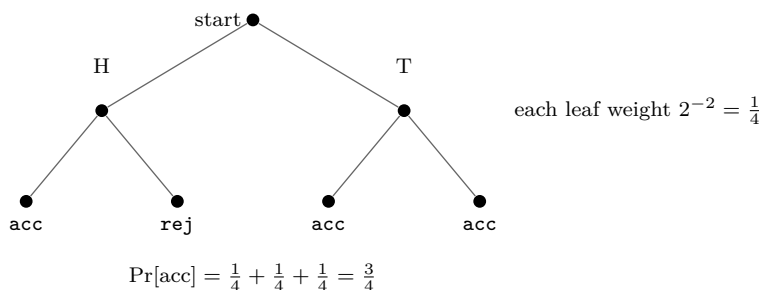
Definition 15.4.1 (Probabilistic Turing machine). A *probabilistic Turing machine* M is a nondeterministic Turing machine in which every nondeterministic step is a *coin-flip step* with exactly two legal next moves, “heads” and “tails.” A computation *branch* b on input w is one root-to-leaf path in the computation tree; if b makes k coin-flip steps we set

$$\Pr[b] = 2^{-k},$$

the probability of tossing exactly that sequence of k fair coins. The probability that M accepts w is the total weight of its accepting branches,

$$\Pr[M \text{ accepts } w] = \sum_{b \text{ accepting}} \Pr[b], \quad \Pr[M \text{ rejects } w] = 1 - \Pr[M \text{ accepts } w].$$

The picture to keep in mind is a binary tree whose every internal node is a coin flip; each leaf is reached with probability $2^{-(\text{depth})}$, and acceptance probability is the fraction of leaf-weight that lands on **accept**.



Definition 15.4.2 (Deciding with bounded error). For $0 \leq \epsilon < \frac{1}{2}$, a probabilistic machine M *decides* a language A with *error probability* ϵ if for every input w ,

$$w \in A \implies \Pr[M \text{ accepts } w] \geq 1 - \epsilon, \quad w \notin A \implies \Pr[M \text{ rejects } w] \geq 1 - \epsilon.$$

Time and space are measured, as for a nondeterministic machine, by the worst branch.

Definition 15.4.3 (The randomized classes). Let all machines below run in probabilistic polynomial time.

- BPP (*two-sided bounded error*): some machine decides the language with error probability $\frac{1}{3}$.
- RP (*one-sided*): $w \in A \implies \Pr[\text{accept}] \geq \frac{1}{2}$, and $w \notin A \implies \Pr[\text{accept}] = 0$ (never a false “yes”).
- coRP: the mirror image – $w \notin A \implies \Pr[\text{reject}] \geq \frac{1}{2}$, and $w \in A \implies \Pr[\text{reject}] = 0$ (never a false “no”).

- $ZPP = RP \cap \text{coRP}$ (*zero error*, Las Vegas): the machine may output **acc**, **rej**, or “?” (don’t know); it is never wrong, and outputs “?” with probability $\leq \frac{1}{2}$.

The containments $P \subseteq ZPP \subseteq RP \subseteq BPP$ and $RP \subseteq NP$ are immediate from the definitions (a deterministic decider is a machine that flips no coins; an RP accepting branch is an NP certificate). The two constants $\frac{1}{3}$ and $\frac{1}{2}$ look arbitrary; the next theorem (with the remark after it) shows the choice does not matter – *any* gap from $\frac{1}{2}$ is as good as any other.

Amplification

Theorem 15.4.4 (Amplification for two-sided error). *Fix a constant ϵ with $0 < \epsilon < \frac{1}{2}$. If a language A is decided by a probabilistic polynomial-time machine M_1 with error ϵ , then for every polynomial $p(n)$ there is a probabilistic polynomial-time machine M_2 deciding A with error at most $2^{-p(n)}$.*

Proof. M_2 on input x runs $t = 2k$ independent copies of $M_1(x)$ and outputs the *majority* answer; we choose k below. Each copy is correct with probability $\geq 1 - \epsilon$ independently, so M_2 errs only if at least half of the $2k$ copies err. Fix the input and let $\delta \leq \epsilon$ be the true error of M_1 on it. A particular outcome sequence with w wrong and $c = 2k - w$ correct copies has probability $\delta^w(1 - \delta)^c$. For a *bad* sequence ($w \geq c$, i.e. $w \geq k$) we bound, using $\delta(1 - \delta) \leq \epsilon(1 - \epsilon)$ and $\epsilon < 1 - \epsilon$,

$$\delta^w(1 - \delta)^c \leq \epsilon^w(1 - \epsilon)^c \leq (\epsilon(1 - \epsilon))^k.$$

There are at most 2^{2k} outcome sequences in all, so summing over bad ones,

$$\Pr[M_2 \text{ errs}] \leq 2^{2k} (\epsilon(1 - \epsilon))^k = (4\epsilon(1 - \epsilon))^k.$$

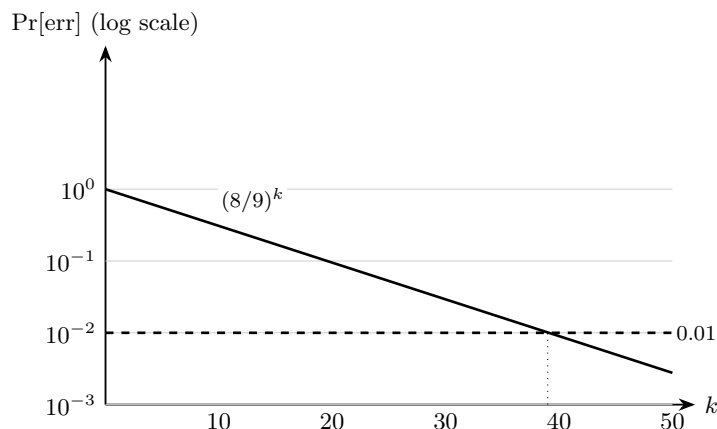
Since $\epsilon < \frac{1}{2}$, the base $\beta := 4\epsilon(1 - \epsilon) < 1$. Put $\alpha = -\log_2 \beta > 0$ and choose $k \geq p(n)/\alpha$; then $\beta^k = 2^{-\alpha k} \leq 2^{-p(n)}$. As $k = O(p(n))$ and each run is polynomial, M_2 runs in polynomial time. $\square$

Remark. This is the elementary “Chernoff/Markov” calculation in disguise: the chance that an average of $2k$ independent $\{0, 1\}$ votes lands on the wrong side of $\frac{1}{2}$ falls *geometrically* in k , with ratio $\beta = 4\epsilon(1 - \epsilon)$. One-sided error amplifies even more cheaply: an RP machine errs only by saying “reject” when it should accept, so running it t times and accepting if *any* run accepts cannot create a false positive, and the false-negative probability drops from $\leq \frac{1}{2}$ to $\leq 2^{-t}$ – no majority vote needed.

Worked Example 15.4.5 (From error $1/3$ to error ≤ 0.01 : read off the repeat count). A BPP machine M_1 has error exactly $\epsilon = \frac{1}{3}$. How many independent runs $t = 2k$, voting by majority, guarantee error ≤ 0.01 ?

Recognition. The trigger is “bounded two-sided error, drive it down”: apply the bound $\Pr[\text{err}] \leq \beta^k$ from Theorem 15.4.4 with $\beta = 4\epsilon(1 - \epsilon)$ and solve for k .

Solution. With $\epsilon = \frac{1}{3}$, $\beta = 4 \cdot \frac{1}{3} \cdot \frac{2}{3} = \frac{8}{9}$. We need $(\frac{8}{9})^k \leq 0.01$, i.e. $k \geq \frac{\ln 100}{\ln(9/8)} = \frac{4.6052}{0.11778} \approx 39.1$, so $k = 40$ and $t = 2k = 80$ runs suffice by the *provable* bound. Plotting β^k shows the geometric decay crossing the 0.01 line near $k = 40$:



Which idea, and why? Two-sided error needs the *majority-vote* bound β^k with $\beta = 4\epsilon(1 - \epsilon)$; one solves $\beta^k \leq$ target by taking logs. The slow base $\frac{8}{9}$ (because $\frac{1}{3}$ is close to $\frac{1}{2}$) is why dozens of runs are needed, not a handful.

Check. $(\frac{8}{9})^{40} = 2^{40 \log_2(8/9)} = 2^{-6.79} \approx 0.0090 \leq 0.01$, while $(\frac{8}{9})^{39} \approx 0.0101 > 0.01$ – so $k = 40$ is exactly the first integer that works. (An exact binomial count gives a smaller honest number, $t = 47$; our self-contained bound is deliberately loose.)

Randomized primality testing

We now turn to the most famous probabilistic algorithm. Recall Fermat’s little theorem: if p is prime and $1 \leq a \leq p - 1$, then $a^{p-1} \equiv 1 \pmod{p}$. Read contrapositively, finding one a with $a^{p-1} \not\equiv 1 \pmod{p}$ *certifies* that p is composite – without exhibiting a factor. Call such an a a *Fermat witness*.

Worked Example 15.4.6 (Fermat witnesses: 15 is composite, 13 is prime-like). Use base $a = 2$ to test $n = 15$, then $n = 13$, by computing $a^{n-1} \bmod n$.

Recognition. Trigger: “decide prime vs. composite cheaply.” One Fermat test is a single modular exponentiation, polynomial in $\log n$ by repeated squaring.

Solution. For $n = 15$: $2^{14} \bmod 15$. Squaring, $2^4 = 16 \equiv 1$, so $2^{14} = 2^{12} \cdot 2^2 \equiv 1 \cdot 4 = 4 \not\equiv 1 \pmod{15}$. Thus $a = 2$ is a Fermat witness and 15 is *provably composite*. For $n = 13$: $2^{12} \bmod 13$; $2^4 = 16 \equiv 3$, $2^8 \equiv 3^2 = 9$, $2^{12} = 2^8 \cdot 2^4 \equiv 9 \cdot 3 = 27 \equiv 1 \pmod{13}$ – 13 passes, consistent with primality.

Which idea, and why? Fermat’s theorem makes “ $a^{n-1} \bmod n \neq 1$ ” a one-shot certificate of compositeness; passing the test is only suggestive, since composites can pass.

Check. $15 = 3 \cdot 5$ really is composite, caught here without factoring; 13 is prime, so by the theorem it passes for *every* base, including $a = 2$.

The plain Fermat test fails on *Carmichael numbers* (e.g. $561 = 3 \cdot 11 \cdot 17$), which are composite yet pass for every base coprime to them. The Miller–Rabin test repairs this by also looking for a nontrivial square root of 1: modulo a prime, the only square roots of 1 are ± 1 , so a square root other than ± 1 is a second kind of witness.

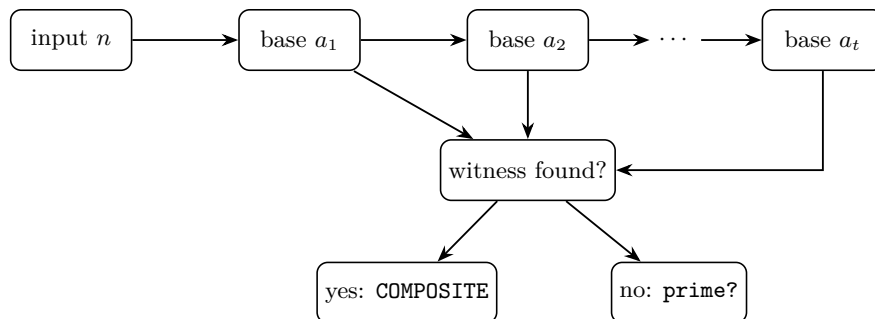
Theorem 15.4.7 (Miller–Rabin one-sided error). Write $n - 1 = s \cdot 2^\ell$ with s odd. For odd n and random $a \in \{1, \dots, n - 1\}$, declare a a witness if $a^{n-1} \not\equiv 1 \pmod{n}$, or if the sequence $a^s, a^{2s}, \dots, a^{2^{\ell-1}s} \pmod{n}$ contains a 1 whose immediate predecessor is neither 1 nor -1 . If n is prime, no witness exists; if n is odd composite, at least half the bases in $\{1, \dots, n - 1\}$ are witnesses.

Proof sketch. If n is prime, $a^{n-1} \equiv 1$ by Fermat, and the only square roots of 1 modulo a prime are ± 1 (a quadratic $x^2 - 1 = (x - 1)(x + 1)$ has at most two roots in the field $\mathbb{Z}_n$); so the doubling sequence can only reach 1 through -1 , leaving no witness. If n is odd composite, one constructs a single witness t (using the Chinese remainder theorem to split n , or the binomial expansion for a prime power) and then shows $d \mapsto dt \bmod n$ injects nonwitnesses into witnesses, so witnesses are at least as numerous as nonwitnesses; hence at least half the bases are witnesses. $\square$

Worked Example 15.4.8 (Why Miller–Rabin puts COMPOSITES in RP). Package the test as a probabilistic algorithm for $\text{COMPOSITES} = \{n : n \text{ is composite}\}$ with one-sided error, and identify which direction it can err.

Recognition. Trigger: “each random base is a Fermat/Miller–Rabin test that catches a composite with probability $\geq \frac{1}{2}$ and never slanders a prime” – the exact shape of RP (Definition 15.4.3).

Solution. The machine: pick $a \in \{1, \dots, n - 1\}$ uniformly; **accept** (declare “composite”) if a is a witness, else **reject**. If n is prime, no witness exists, so $\Pr[\text{accept}] = 0$ – never a false “composite.” If n is composite, $\Pr[a \text{ is a witness}] \geq \frac{1}{2}$, so $\Pr[\text{accept}] \geq \frac{1}{2}$. Both run in time polynomial in $\log n$ (modular exponentiation by repeated squaring). The one-sided error is exactly the RP shape; amplify by trying t independent bases and accepting if any is a witness:



After t bases, a composite escapes detection with probability $\leq 2^{-t}$, while a prime is *never* flagged. Equivalently $\text{PRIMES} \in \text{coRP}$ (the complement), and so $\text{PRIMES} \in \text{BPP}$.

Which idea, and why? One-sided error lives in RP: the algorithm can only be wrong by *missing* a composite (false “prime”), never by falsely calling a prime composite; so accepting-if-any-witness amplifies with no majority vote.

Check. On the Carmichael number 561 the plain Fermat test would say “prime” for every base coprime to 561, but Miller–Rabin finds a nontrivial square root of 1 for at least half the bases, so a few random tries expose it – exactly the gap Theorem 15.4.7 closes.

Remark (PRIMES is in P). In 2002 Agrawal, Kayal, and Saxena gave a *deterministic* polynomial-time primality test (the AKS algorithm), proving $\text{PRIMES} \in \text{P}$ outright. We state this; we do not prove it. The randomized Miller–Rabin test remains the one used in practice because it is far faster, with error driven to, say, 2^{-128} in a few dozen rounds.

A second concrete algorithm: polynomial-identity testing

Here randomness shines where no efficient deterministic method is known. Two polynomials are equal iff their difference is the zero polynomial; the trick is to test that by evaluating at a *random* point.

Lemma 15.4.9 (Schwartz–Zippel). *Let $q(x_1, \dots, x_m)$ be a nonzero polynomial over a field $\mathbb{F}$, of total degree at most d . If $a_1, \dots, a_m$ are chosen independently and uniformly from a finite set $S \subseteq \mathbb{F}$, then*

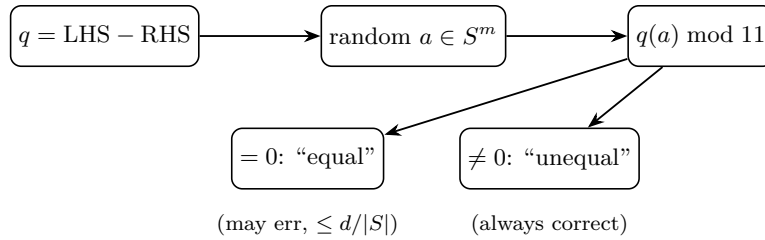
$$\Pr[q(a_1, \dots, a_m) = 0] \leq \frac{d}{|S|}.$$

Proof. Induct on m . For $m = 1$, a nonzero univariate degree- d polynomial has at most d roots, so a uniform $a_1 \in S$ is a root with probability $\leq d/|S|$. For the step, write $q = \sum_{i=0}^d x_1^i q_i(x_2, \dots, x_m)$ and let j be the largest index with $q_j \neq 0$; then q_j has total degree $\leq d - j$. If $q(a_1, \dots, a_m) = 0$ then either $q_j(a_2, \dots, a_m) = 0$ – probability $\leq (d - j)/|S|$ by induction – or $q_j(a_2, \dots, a_m) \neq 0$, in which case $q(x_1, a_2, \dots, a_m)$ is a nonzero degree- j univariate polynomial and a_1 is one of its $\leq j$ roots, probability $\leq j/|S|$. Summing, the total is at most $(d - j)/|S| + j/|S| = d/|S|$. $\square$

Worked Example 15.4.10 (Testing a polynomial identity with one random point). Decide whether $(x + y)(x - y) \stackrel{?}{=} x^2 - y^2 + xy$ as formal polynomials over $\mathbb{Z}_{11}$, by the Schwartz–Zippel method, using the sample point $(x, y) = (3, 5)$.

Recognition. Trigger: “are two polynomials identical?” Form $q = \text{LHS} - \text{RHS}$; $q \equiv 0$ iff the identity holds. Test q at a random point: if $q \neq 0$ but we sampled a root, we err – with probability $\leq d/|S|$ by Lemma 15.4.9.

Solution. Here $q = (x + y)(x - y) - (x^2 - y^2 + xy) = (x^2 - y^2) - (x^2 - y^2 + xy) = -xy$, a nonzero polynomial of total degree $d = 2$. Evaluate at $(3, 5)$ over $\mathbb{Z}_{11}$: $q(3, 5) = -3 \cdot 5 = -15 \equiv -15 + 22 = 7 \not\equiv 0 \pmod{11}$. Since the value is nonzero, q is not the zero polynomial and the identity is *false*. Choosing the sample coordinates from $S = \mathbb{Z}_{11}$ ($|S| = 11$) bounds the error of a single point at $d/|S| = 2/11 \approx 0.18$ when the identity actually *fails*:



Which idea, and why? Equality of polynomials is a root question on their difference; a random evaluation catches a nonzero q unless we land on one of its roots, an event of probability $\leq d/|S|$ – so a larger field S shrinks the error for free.

Check. The honest answer is “unequal” (the difference is $-xy \neq 0$), and our single point $(3, 5)$ already exposed it; a “ $\neq 0$ ” verdict is *never* wrong, so this side has zero error – the routine is one-sided.

One-way functions and an honest crypto teaser

Modern cryptography rests on functions that are easy to compute but hard to invert.

Definition 15.4.11 (One-way function, informal). A polynomial-time computable $f: \Sigma^* \rightarrow \Sigma^*$ is *one-way* if no probabilistic polynomial-time algorithm, given $f(x)$ for a random x , can output some preimage x' with $f(x') = f(x)$ except with negligible probability.

The honest caveat: *no one has proved a one-way function exists*. Such a proof would imply $P \neq NP$ (inverting f is an NP search problem), so it is at least as hard as the central open question. What we have is *circumstantial* evidence: integer multiplication $f(p, q) = pq$ is easy, while factoring its output appears hard on average, and public-key systems like RSA stake their security on exactly that asymmetry. Randomized primality testing is what lets one *generate* the large primes p, q in the first place – the practical link back to this section.

Where BPP sits

Theorem 15.4.12 ($BPP \subseteq PSPACE$). *Every language in BPP is in PSPACE.*

Proof. Let M decide $A \in BPP$ in time $p(n)$ with error $\leq \frac{1}{3}$, using at most $p(n)$ coin flips on any branch. A deterministic machine S on input w loops over all $2^{p(n)}$ coin-flip strings $r \in \{0, 1\}^{p(n)}$, simulates the single deterministic branch of M that those coins select (reusing the same polynomial space each time), and keeps a running count of accepting branches. It accepts iff more than half of the $2^{p(n)}$ branches accept. Each simulation needs only $O(p(n))$ space and the counter needs $O(p(n))$ bits, all reused across the exponentially many iterations, so S runs in polynomial *space*. Correctness is Definition 15.4.2: the accept fraction is $\Pr[M \text{ accepts } w]$, which is $\geq \frac{2}{3}$ on members and $\leq \frac{1}{3}$ on non-members. $\square$

Theorem 15.4.13 (Adleman: $BPP \subseteq P/poly$). *Every language in BPP is decided by a family of polynomial-size Boolean circuits (one circuit per input length).*

Proof. Let $A \in BPP$. By Theorem 15.4.4 amplify to error $< 2^{-n}$ at input length n , with $r(n)$ random bits per run. Call a coin string $\rho \in \{0, 1\}^{r(n)}$ *bad* for w if running M with ρ gives the wrong answer on w . For each fixed w of length n , $\Pr_\rho[\rho \text{ bad for } w] < 2^{-n}$. Summing over the 2^n inputs w of length n ,

$$\Pr_\rho[\rho \text{ bad for some } w] < 2^n \cdot 2^{-n} = 1,$$

so *some* single ρ^* is good for *all* 2^n inputs of length n at once. Hard-wire that ρ^* into M : the result is a deterministic polynomial-time computation, which (via its tableau as in Section 13.2, each cell a constant-size circuit of the three cells above it) compiles to a polynomial-size circuit C_n . The family (C_n) decides A . $\square$

This is a probabilistic *union bound*: amplify until each input's failure probability is below 2^{-n} , then the failures of all 2^n inputs together cannot cover every coin string, so a universally good coin string survives to be baked in. Combined with Theorem 15.4.12, randomness keeps us inside the deterministic world (P/poly and PSPACE); whether $\text{BPP} = \text{P}$ – the conjecture that randomness adds *no* power – is open but widely believed.

Method (Design and analyze a one-sided-error algorithm, then amplify).

1. **Find a witness notion.** Identify an object (a base a , an evaluation point) that, when it “fires,” *proves* the answer with certainty on one side.
2. **Bound the witness density.** Show a random draw fires with probability $\geq \frac{1}{2}$ on inputs of one type, and *never* fires on the other type. This is the RP shape: one-sided, error only on yes-instances missing a witness.
3. **Check the per-trial cost.** Confirm one trial runs in polynomial time (modular exponentiation, one polynomial evaluation, one random cut).
4. **Amplify.** For RP, run t trials and accept if *any* fires: error $\leq 2^{-t}$. For two-sided BPP, run $2k$ trials and take the *majority*: error $\leq (4\epsilon(1-\epsilon))^k$; solve for k to hit a target.
5. **Name the class.** One-sided no-false-yes $\Rightarrow$ RP; no-false-no $\Rightarrow$ coRP; two-sided $\Rightarrow$ BPP; never wrong but sometimes “?” $\Rightarrow$ ZPP.

Caution. Do not confuse *Monte Carlo* with *Las Vegas*. A Monte Carlo algorithm (the BPP/RP kind) always runs in polynomial time but may give a wrong answer with small probability. A Las Vegas algorithm (the ZPP kind) is *never* wrong but has a random *running time*; you trade error for time. “Repeat until sure” converts a one-sided Monte Carlo procedure into a Las Vegas one only when a fired witness is verifiable – otherwise you can never become sure.

Caution. Keep straight *which way* each class can err. RP for COMPOSITES can only output a wrong “prime” (a false *reject* on a composite); it never calls a prime composite. A two-sided BPP machine may err in *both* directions, so you must use the *majority* vote, not “accept if any run accepts” – the latter amplifies RP correctly but *ruins* a BPP machine, because a single spurious accept among many runs would then be enough to flip a true “no” to “yes.”

Exercises.

1. A probabilistic Turing machine makes exactly three coin-flip steps on input w , producing eight equally likely branches; the accepting branches are those whose flip sequence (reading H = 1, T = 0) is one of 111, 110, 101, 100, 011. Draw the depth-3 binary computation tree, mark the accepting leaves, and compute $\Pr[M \text{ accepts } w]$. Does this w satisfy the membership condition of Definition 15.4.2 for error $\frac{1}{3}$? Justify.
2. A BPP machine has error $\epsilon = \frac{1}{5}$. Using the self-contained bound $\Pr[\text{err}] \leq (4\epsilon(1-\epsilon))^k$ for $2k$ majority-voted runs, find the smallest k that guarantees error $\leq 10^{-6}$. Show the logarithm computation, and state the total number of runs $t = 2k$.
3. Prove directly from Definition 15.4.3 that $\text{RP} \subseteq \text{BPP}$ and that $\text{RP} \subseteq \text{NP}$. For the second inclusion, explain what plays the role of the NP verifier's certificate.
4. Run the Miller–Rabin witness test on $n = 25$ with base $a = 7$. First write $n - 1 = 24 = s \cdot 2^\ell$ with s odd, then compute the sequence $7^s, 7^{2s}, \dots, 7^{2^\ell s} \pmod{25}$, and decide whether 7 is a witness. Conclude whether the test has (correctly) certified 25 as composite.
5. Let $C = \{\langle p \rangle : p \text{ is composite}\}$. Describe a probabilistic polynomial-time machine showing $C \in \text{RP}$, then describe the amplified machine that achieves error $\leq 2^{-50}$, and state exactly how many independent bases it tries. Which kind of input (prime or composite) can the unamplified machine answer incorrectly, and in which direction?
6. Over the field $\mathbb{Z}_{13}$, use the Schwartz–Zippel method to test whether $(x + 2y)^2 \stackrel{?}{=} x^2 + 4xy + 4y^2$ as formal polynomials. Form the difference q , give a bound d on its total degree, evaluate q at the point $(x, y) = (4, 1)$, and report the verdict. State the single-trial error bound $d/|S|$ and say which verdict ($= 0$ or $\neq 0$) can never be wrong.

7. Prove the $m = 1$ base case of the Schwartz–Zippel lemma in full: show that a nonzero univariate polynomial of degree d over a field has at most d roots, and conclude the probability bound $d/|S|$ for a uniform sample from S . (Use the factor theorem: $x - a$ divides q exactly when $q(a) = 0$.)
8. Here is the *randomized min-cut* idea. Given a connected multigraph G , repeatedly pick a uniformly random edge and *contract* it (merge its endpoints, keeping parallel edges, deleting self-loops) until two vertices remain; output the edges between them as a cut. Carry out one full run by hand on the 4-cycle with vertices 1, 2, 3, 4 and edges 12, 23, 34, 41: draw the graph, contract the edges in the order 12, 34, and report the cut you obtain and its size. (You need not prove the success probability.)
9. Show $P \subseteq ZPP$ and $ZPP \subseteq RP$ directly from the definitions. For the first, explain why a deterministic decider meets the zero-error requirement; for the second, describe how a Las Vegas machine that may answer “?” is turned into an RP machine.
10. In the proof that $BPP \subseteq PSPACE$ we loop over all $2^{p(n)}$ coin strings. Explain in two sentences why the simulating machine still uses only polynomial space despite performing exponentially many simulations, and identify the one piece of state that must persist across iterations.
11. *Find the bug.* A student claims: “Since $COMPOSITES \in RP$ with error $\leq \frac{1}{2}$, I can boost confidence by running the one-base test t times and taking the *majority* vote, accepting ‘composite’ iff most runs say composite.” Explain why majority voting is the wrong amplifier for this one-sided algorithm, what can go wrong on a *composite* input, and state the correct RP amplification rule.
12. *Find the bug.* A note states: “Theorem 15.4.13 shows $BPP \subseteq P/poly$ by fixing one good random string ρ^* ; the same argument with error only $\leq \frac{1}{3}$ (no amplification) already finds a ρ^* good for all inputs of length n .” Identify the flaw: redo the union-bound count with per-input error $\frac{1}{3}$ and show the bound becomes vacuous, then state how much amplification is actually required for the argument to go through.

15.5 Interactive Proofs and the Big Picture

An ordinary proof of membership in NP is a single static certificate: write it down once, and a polynomial-time verifier checks it. This section relaxes that picture in two ways at once – the verifier may *toss coins* and may carry on a *dialogue* with an all-powerful but untrusted prover – and the payoff is enormous: the resulting class IP turns out to be exactly PSPACE. The one idea to hold onto is the split of responsibilities: the prover is unbounded and possibly lying, the verifier is the only party that is poly-time and randomized, and *soundness must defeat every cheating prover* while *completeness* only needs the honest one to succeed. We then locate IP on the grand map of classes from L up to EXPTIME.

The model: an honest prover, a skeptical randomized verifier

Definition 15.5.1 (Interactive proof system). An *interactive proof system* for a language A consists of two parties that exchange a polynomial number of messages on a common input w .

- The *Verifier* V is a probabilistic polynomial-time machine: it reads w , draws a random string r , and computes each outgoing message (or a final **accept/reject**) from w , r , and the history of messages so far.
- The *Prover* P is a function of w and the message history with *no computational bound whatsoever*.

Write $\Pr[V \leftrightarrow P \text{ accepts } w]$ for the probability, over V ’s coins, that the interaction ends in **accept**. We say $A \in IP$ if there is a poly-time V such that

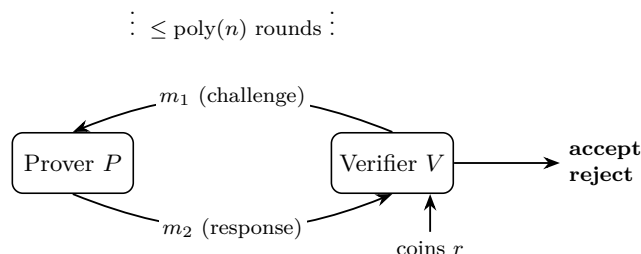
$$w \in A \implies \exists P : \Pr[V \leftrightarrow P \text{ accepts } w] \geq \frac{2}{3} \quad (\text{completeness}),$$

$$w \notin A \implies \forall \tilde{P} : \Pr[V \leftrightarrow \tilde{P} \text{ accepts } w] \leq \frac{1}{3} \quad (\text{soundness}).$$

The quantifiers are the crux: completeness needs *some* prover to convince V of a true statement; soundness must hold against *every* prover $\tilde{P}$ for a false one.

Remark. The constants $\frac{2}{3}$ and $\frac{1}{3}$ are not sacred. Running the whole protocol t times independently and taking a majority vote drives the error to $2^{-\Omega(t)}$, exactly the amplification argument used for BPP. So “high probability” may be read as “all but an exponentially small chance.”

Two sanity checks anchor the definition. If we forbid randomness and allow a single prover message, V just checks a certificate, so $\text{NP} \subseteq \text{IP}$. If we forbid the prover entirely, V is a randomized poly-time decider, so $\text{BPP} \subseteq \text{IP}$. The interesting languages are those, like graph non-isomorphism below, that seem to lie in neither.



Graph non-isomorphism has a two-message proof

Two graphs are *isomorphic* if one is a relabeling of the other; call the relation $G \cong H$. The language $\text{ISO} = \{\langle G, H \rangle \mid G \cong H\}$ is in NP (the isomorphism itself is the certificate), but its complement

$$\text{NONISO} = \{\langle G, H \rangle \mid G \not\cong H\}$$

is not known to be in NP: there is no obvious short certificate of *non*-isomorphism. Yet a prover can convince a skeptical verifier, by the following idea – the verifier secretly scrambles one graph and dares the prover to say which one it scrambled.

Theorem 15.5.2 ($\text{NONISO} \in \text{IP}$). *The protocol below, run twice in parallel with independent coins (V accepting only if both guesses are right), is a complete and sound interactive proof system for NONISO.*

Protocol. On input $\langle G_1, G_2 \rangle$ (same vertex set $\{1, \dots, n\}$):

1. V flips a fair coin to pick $b \in \{1, 2\}$, picks a uniformly random permutation π of $\{1, \dots, n\}$, forms $H = \pi(G_b)$, and sends H to P .
2. P replies with a guess $b' \in \{1, 2\}$.
3. V accepts iff $b' = b$.

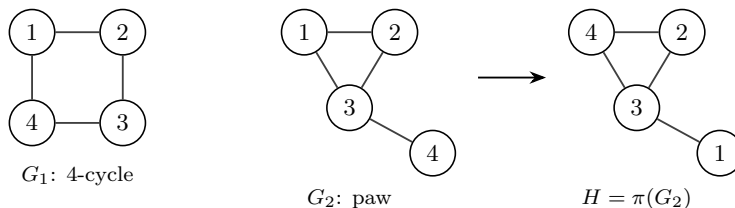
Proof. Completeness. If $G_1 \not\cong G_2$, then H is isomorphic to exactly one of G_1, G_2 – namely G_b . An unbounded honest prover examines H , determines which of the two it is isomorphic to (testing isomorphism is something an unbounded party can do), sets $b' = b$, and is always right. Hence V accepts with probability $1 \geq \frac{2}{3}$.

Soundness. If $G_1 \cong G_2$, then a random relabeling of G_1 and a random relabeling of G_2 have the *same* distribution: H is a uniformly random graph isomorphic to G_1 (equally to G_2), and its distribution does *not* depend on the secret bit b . So H carries zero information about b . Any prover $\tilde{P}$, however powerful, is reduced to guessing a bit it cannot see, and $\Pr[b' = b] = \frac{1}{2}$. Repeating the protocol independently k times and accepting only if the prover is right every time gives acceptance probability $2^{-k} \leq \frac{1}{3}$ already at $k = 2$. Thus false instances are rejected with high probability. $\square$

Worked Example 15.5.3 (Running the non-isomorphism challenge on a concrete pair). Let G_1 be the 4-cycle $1-2-3-4-1$ and let G_2 be the “paw”: a triangle on $\{1, 2, 3\}$ with a pendant edge $3-4$. These are non-isomorphic (G_1 is 2-regular, G_2 has degree sequence $1, 2, 2, 3$). Trace one round in which V secretly chooses $b = 2$ and the permutation $\pi = (1\ 4)$ (swap labels 1 and 4), and explain why an unbounded prover answers $b' = 2$ with certainty.

Recognition. The trigger is “convince a verifier of a *negative* fact with no short certificate.” The interactive move is: hide one graph under a random relabeling and let the prover’s unbounded power identify the original. Recognition here means matching H back to whichever G_b it is isomorphic to.

Solution. Apply $\pi = (1\ 4)$ to G_2 . The paw's edges $\{12, 13, 23, 34\}$ become $\{42, 43, 23, 31\} = \{24, 34, 23, 13\}$, i.e. a triangle on $\{2, 3, 4\}$ with a pendant edge to 1. The picture: G_1 , G_2 , and the relabeled challenge $H = \pi(G_2)$.



H is a triangle plus a pendant, hence $H \cong G_2$ and $H \not\cong G_1$ (the 4-cycle has no triangle). The prover sees H , recognizes the triangle, and answers $b' = 2$. Since $b = 2$, V accepts this round.

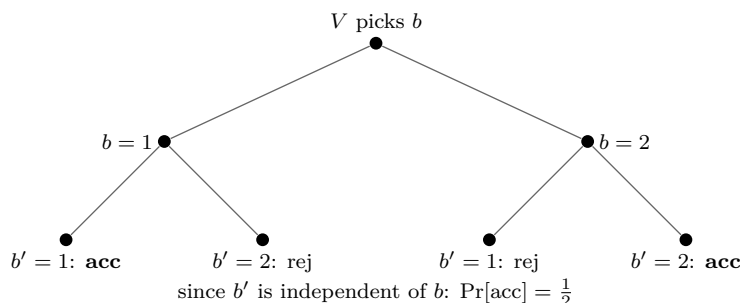
Which idea, and why? The protocol works because for non-isomorphic graphs the challenge H is correlated with the secret bit b (it is isomorphic to G_b and only G_b); the prover's unbounded power converts that correlation into a certain answer.

Check. If instead the two inputs had been isomorphic, H would be a triangle-plus-pendant reachable from *both*, and the triangle would tell the prover nothing about b – the guess would be a coin flip, exactly as soundness predicts.

Worked Example 15.5.4 (Why a cheating prover is caught with probability at least $1/2$). Suppose $G_1 \cong G_2$ (a *false* instance of *NONISO*) and a cheating prover $\tilde{P}$ wants V to accept. Show $\Pr[V \text{ accepts one round}] = \frac{1}{2}$ no matter what strategy $\tilde{P}$ uses, and that k rounds cut this to 2^{-k} .

Recognition. The trigger is “soundness against an arbitrary prover.” Recognition: pin down the *distribution* of the verifier’s message and argue it is independent of the secret the prover must guess.

Solution. Because $G_1 \cong G_2$, fix an isomorphism σ with $\sigma(G_1) = G_2$. When $b = 1$ the challenge is $\pi(G_1)$ for uniform π ; when $b = 2$ it is $\rho(G_2) = \rho\sigma(G_1)$ for uniform ρ . As π and $\rho\sigma$ are both uniform over all $n!$ permutations, the two challenge distributions are *identical*. The view of $\tilde{P}$ is therefore statistically independent of b , so its output b' satisfies $\Pr[b' = b] = \frac{1}{2}$. The decision tree shows the four possible root-to-leaf outcomes of one round:



With k independent rounds, V accepts only if $\tilde{P}$ wins all k , probability $(\frac{1}{2})^k = 2^{-k}$. Taking $k = 2$ already gives $2^{-2} = \frac{1}{4} \leq \frac{1}{3}$.

Which idea, and why? Soundness is an *indistinguishability* argument: when the statement is false the verifier’s challenge leaks nothing about its hidden coin, so even an omniscient prover does no better than chance, and independent repetition makes “chance” exponentially small.

Check. The bound is tight per round – $\tilde{P}$ can guess right half the time – so $\frac{1}{2}$ is exactly the per-round soundness error, and amplification, not a smarter single round, is what buys confidence.

The headline theorem: $\text{IP} = \text{PSPACE}$

Theorem 15.5.5 (Shamir). $\text{IP} = \text{PSPACE}$.

We do not give the full proof, but both directions have a clean shape, and the harder one introduces the technique that dominates modern complexity.

The easy inclusion $\text{IP} \subseteq \text{PSPACE}$. Given a verifier V , the best a prover can do against it is a quantity

$$\Pr[V \text{ accepts } w] = \max_P \Pr[V \leftrightarrow P \text{ accepts } w],$$

the value of a game tree whose levels alternate between the prover's *maximizing* choice of the next message and the verifier's *averaging* over its random coins. This tree has polynomial depth (poly many rounds) and can be evaluated by a depth-first recursion that stores only one message per level – polynomial space. If the value is $\geq \frac{2}{3}$ accept, else reject.

The hard inclusion $\text{PSPACE} \subseteq \text{IP}$: *arithmetization and sum-check*. It suffices to give an interactive proof for a PSPACE -complete problem; the model protocol is the counting problem

$$\#SAT = \{\langle \phi, k \rangle \mid \phi \text{ has exactly } k \text{ satisfying assignments}\},$$

whose technique then extends to a fully quantified formula. The four moves of the idea are:

1. **Arithmetize.** Replace a Boolean formula $\phi(x_1, \dots, x_m)$ by a low-degree polynomial p over a finite field $\mathbb{F}_q$, using $\neg \alpha \mapsto 1 - \alpha$, $\alpha \wedge \beta \mapsto \alpha\beta$, and $\alpha \vee \beta \mapsto 1 - (1 - \alpha)(1 - \beta)$. On $\{0, 1\}$ -inputs p agrees with ϕ .
2. **Count by summation.** The number of satisfying assignments is the field sum

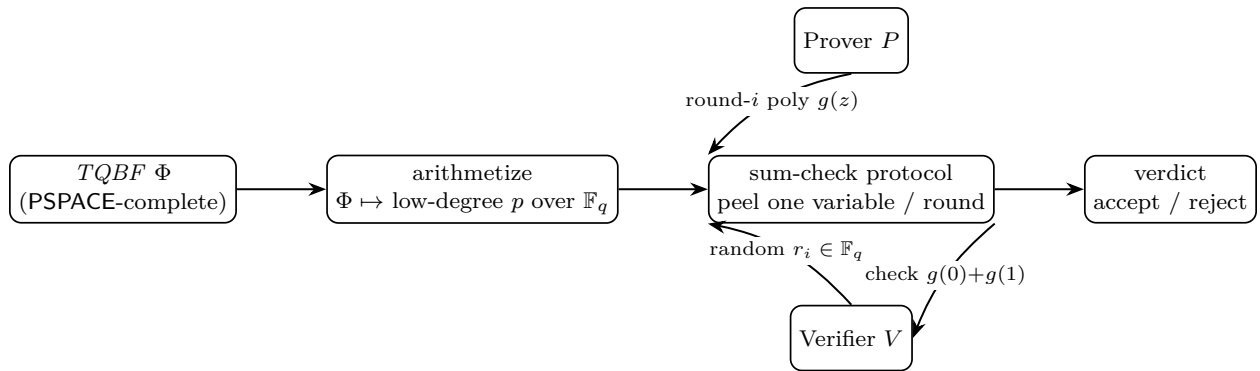
$$\sum_{x_1, \dots, x_m \in \{0, 1\}} p(x_1, \dots, x_m),$$

an exponential sum the prover claims equals k .

3. **Peel one variable per round.** Define $f_i(a_1, \dots, a_i) = \sum_{x_{i+1}, \dots, x_m \in \{0, 1\}} p(a_1, \dots, a_i, x_{i+1}, \dots, x_m)$. In round i the prover sends the univariate polynomial $g(z) = f_i(r_1, \dots, r_{i-1}, z)$; the verifier checks the *consistency equation* $f_{i-1}(r_1, \dots, r_{i-1}) = g(0) + g(1)$, then sends back a random field point r_i and recurses on $f_i(r_1, \dots, r_i)$.
4. **Close at a random point.** After m rounds the verifier checks the single value $f_m(r_1, \dots, r_m) = p(r_1, \dots, r_m)$ directly – one polynomial evaluation.

Soundness rests on one fact: a nonzero polynomial of degree d over $\mathbb{F}_q$ has at most d roots, so two distinct low-degree polynomials agree at a random point with probability $\leq d/q$, negligible when q is large. A lying prover who misstates the top sum is forced to lie about *some* univariate polynomial each round; at the random challenge r_i its lie almost surely fails to match the truth, and the final direct check catches it.

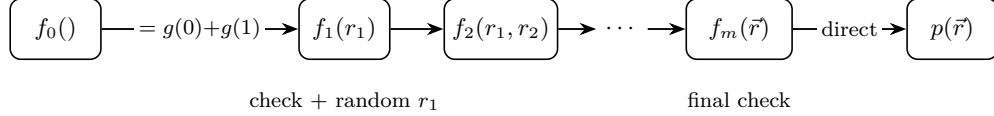
Zooming out from the per-round bookkeeping, the whole result is a single pipeline. The same arithmetization that turns a count into a polynomial turns a *fully quantified* formula Φ into a low-degree polynomial, and the sum-check protocol below carries the prover's claim, one randomized round at a time, down to a single field-point check the verifier can perform unaided.



invariant: $\Phi \text{ true} \iff \text{the polynomial identity } V \text{ checks holds}$

Takeaway. Arithmetization turns a quantified Boolean formula into a low-degree polynomial whose value the verifier checks via the round-by-round sum-check protocol; this puts the PSPACE -complete $TQBF$ in IP , and since $\text{IP} \subseteq \text{PSPACE}$, $\text{IP} = \text{PSPACE}$ (Shamir). The protocol idea is sketched, not fully proved.

The next picture opens up the sum-check box itself: the chain of partial sums $f_0, f_1, \dots, f_m$ the two parties drive down to a single evaluation.



Worked Example 15.5.6 (Arithmetizing a tiny formula and reading off $f_0()$). Let $\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$. Arithmetize it to a polynomial p , compute the satisfying count $f_0() = \sum_{x_1, x_2 \in \{0,1\}} p$, and identify $g(z) = f_1(z)$.

Recognition. The trigger is “turn a Boolean count into a polynomial sum.” Recognition: apply the three replacement rules termwise, then sum over the Boolean cube.

Solution. Replace $\vee$ by $a * b = 1 - (1 - a)(1 - b)$ and $\neg x_1$ by $1 - x_1$:

$$p(x_1, x_2) = (1 - (1 - x_1)(1 - x_2)) \cdot (1 - x_1(1 - x_2)).$$

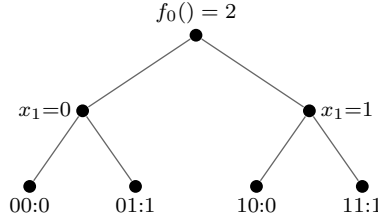
Evaluate on the four Boolean points (where p matches ϕ):

(x_1, x_2)	$x_1 \vee x_2$	$\neg x_1 \vee x_2$	$\phi = p$
(0, 0)	0	1	0
(0, 1)	1	1	1
(1, 0)	1	0	0
(1, 1)	1	1	1

So $f_0() = \sum p = 0 + 1 + 0 + 1 = 2$: the formula has two satisfying assignments. Peeling x_1 ,

$$f_1(z) = p(z, 0) + p(z, 1), \quad p(z, 0) = (1 - (1 - z))(1 - z) = z(1 - z), \quad p(z, 1) = 1 \cdot 1 = 1,$$

hence $f_1(z) = z(1 - z) + 1 = 1 + z - z^2$. Its degree-2 shape is what the prover would transmit in round 1.



Which idea, and why? Arithmetization replaces a logical count by a polynomial sum so that algebra – the fact that low-degree polynomials are rigid – can be used; here $f_0() = 2$ is read straight off the leaves, and $f_1(z)$ packages the same count as a univariate polynomial the verifier can spot-check.

Check. Consistency holds: $g(0) + g(1) = f_1(0) + f_1(1) = (1) + (1) = 2 = f_0()$, exactly the equation the verifier tests in round 1.

Zero knowledge, in one breath

Interactive proofs can reveal that a statement is true while leaking *nothing else* – a *zero-knowledge* proof. For graph 3-coloring (an NP-complete problem), the prover commits to a proper 3-coloring by locking each vertex’s color in a sealed “box,” randomly permuting the three color names first. The verifier picks one random edge and asks for just those two boxes opened. If the coloring is proper they differ; a cheating prover with no valid coloring has some monochromatic edge and is caught with probability $\geq 1/|E|$ per round, amplified by repetition. Crucially the verifier learns only that the two endpoints of a random edge got different (re-randomized) colors – which it could have generated itself – so it gains *conviction without the coloring*. We state this only as a teaser: the formal definition (a simulator reproducing the verifier’s view) is beyond our scope.

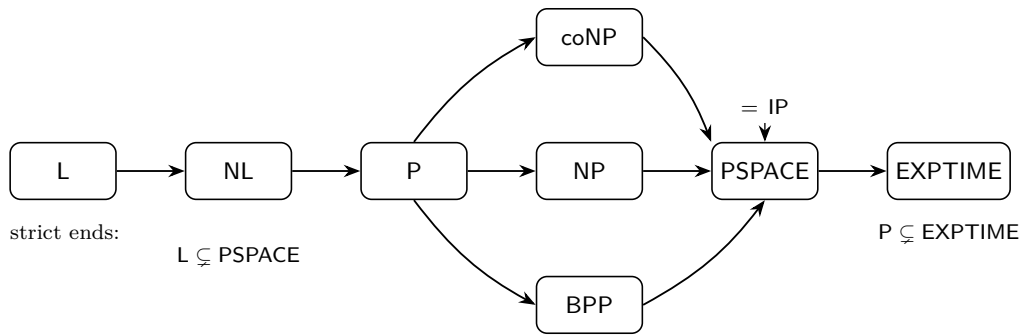
Method (How to prove a protocol is an interactive proof for A).

1. **State the protocol precisely.** Who sends what, in what order; mark every place the verifier uses a coin and every place it could reject.

2. **Completeness.** Assuming $w \in A$, exhibit an *honest* prover strategy (it may be unbounded) and show V accepts with probability $\geq \frac{2}{3}$ – often probability 1.
3. **Soundness.** Assuming $w \notin A$, take an *arbitrary* prover $\tilde{P}$ and bound $\Pr[V \text{ accepts}] \leq \frac{1}{3}$. The standard tool is to show the verifier’s challenge is (nearly) independent of the secret $\tilde{P}$ would need, so its best response is essentially a guess.
4. **Amplify.** If one round only gives error $\leq \frac{1}{2}$ (and completeness is perfect), repeat independently and accept on unanimous success; k rounds give error 2^{-k} .

The grand map

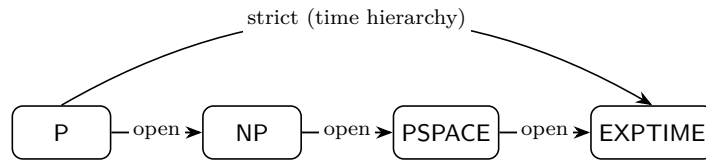
We close by placing the classes of this chapter on one diagram. Solid containments are theorems; no single containment shown is known to be *strict*, and the separations that the hierarchy theorems do give join classes farther apart, such as $L \subsetneq PSPACE$ and $P \subsetneq EXPTIME$.



Worked Example 15.5.7 (Reading the map: which inclusions are known strict, which are open). Decide for each pair whether the inclusion is *known strict*, *known equal*, or *open*: (a) P vs NP ; (b) $PSPACE$ vs IP ; (c) L vs $PSPACE$; (d) NP vs $PSPACE$; (e) P vs $EXPTIME$.

Recognition. The trigger is “classify a containment.” Recognition: separations come almost only from the *hierarchy theorems* (diagonalization across a resource gap); everything else in this range is famously open.

Solution. (a) *Open*: $P \subseteq NP$ is trivial, but $P \neq NP$ is the central unsolved problem. (b) *Known equal*: Theorem 15.5.5, $IP = PSPACE$. (c) *Known strict*: the space hierarchy theorem gives $L \subsetneq PSPACE$ (logarithmic versus polynomial space is a genuine gap). (d) *Open*: $NP \subseteq PSPACE$ holds, but no separation is known. (e) *Known strict*: the time hierarchy theorem gives $P \subsetneq EXPTIME$.



Which idea, and why? A containment is provably strict only when a hierarchy theorem separates the two resource bounds; whether chains like $P \subseteq NP \subseteq PSPACE$ collapse is unknown because no diagonalization crosses a single such gap.

Check. The two known strict pairs, $L \subsetneq PSPACE$ and $P \subsetneq EXPTIME$, straddle the long open middle; if any open inclusion were proved equal it would not contradict these endpoints, which is why the picture is consistent.

Caution. The asymmetry between completeness and soundness is the single most common error. Completeness only asks that the *honest* prover succeed; soundness must hold against *every* prover, including an unbounded, adversarial, infinitely clever one. A protocol that “works if the prover follows the rules” is worthless – a cheating prover never follows the rules. Always test soundness by handing the prover the worst-case strategy and showing the *verifier’s randomness* still defeats it.

Caution. It is the *verifier*, not the prover, that is poly-time and randomized. Bounding the prover, or making the verifier deterministic, collapses the model: a deterministic poly-time verifier that needs no prover-secret reduces IP to NP, losing exactly the power (*NONISO*, *#SAT*, all of PSPACE) that interaction-plus-randomness buys.

Exercises.

1. In the graph non-isomorphism protocol of Theorem 15.5.2, the verifier sends $H = \pi(G_b)$ for a random bit b and a random permutation π . State precisely *which* quantity must be independent of b for soundness to hold, and explain in one sentence why that independence fails when $G_1 \not\cong G_2$ but holds when $G_1 \cong G_2$.
2. Let G_1 be the path 1–2–3 and let G_2 be the triangle 1–2–3–1. Draw both graphs, decide whether $\langle G_1, G_2 \rangle \in \text{NONISO}$, and – if so – carry out one round of the protocol for the secret choice $b = 1$ and $\pi = (23)$: draw the challenge graph H and give the honest prover’s answer b' .
3. A student proposes the following “proof” that $\text{NONISO} \in \text{NP}$: “the certificate is the verifier’s random challenge H together with the prover’s correct answer b' .” Find the bug: explain why this is not a valid NP certificate, referring to who is allowed to generate the randomness.
4. Suppose a verifier runs the non-isomorphism protocol for k independent rounds and accepts iff the prover answers correctly in *at least* $k - 1$ of them (allowing one mistake). On a false instance ($G_1 \cong G_2$), give a formula for the acceptance probability as a function of k , and find the smallest k that makes it $\leq \frac{1}{3}$.
5. Arithmetize the formula $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_2 \vee x_3)$ into a polynomial $p(x_1, x_2, x_3)$ using the three replacement rules. Then compute the satisfying count $f_0() = \sum_{x_1, x_2, x_3 \in \{0,1\}} p$ directly, and state the upper bound on the degree of any single variable in p .
6. For the formula ϕ of the previous exercise, write the univariate polynomial $f_1(z)$ that the prover transmits in round 1 of the *#SAT* protocol (i.e. peel x_1 , summing x_2, x_3 over $\{0,1\}$). Verify the consistency equation $f_1(0) + f_1(1) = f_0()$.
7. The *#SAT* soundness argument uses the fact that two distinct degree- d univariate polynomials over a field $\mathbb{F}_q$ agree at a uniformly random point with probability at most d/q . Prove this fact from the statement “a nonzero degree- d polynomial has at most d roots,” and explain why the protocol chooses q much larger than the formula length n .
8. Place each of the following languages as high or as low on the grand map as the section’s results allow, and justify: (a) *SAT*; (b) *NONISO*; (c) a PSPACE-complete language such as *TQBF* (true quantified Boolean formulas); (d) any language in BPP.
9. Decide whether each statement is true, false, or open; give a one-line reason. (a) $\text{NP} \subseteq \text{IP}$. (b) $\text{IP} \subseteq \text{EXPTIME}$. (c) $\text{IP} = \text{NP}$. (d) Every language in PSPACE has an interactive proof in which the verifier never tosses a coin.
10. Find the bug. A classmate argues: “Soundness of the non-isomorphism protocol only needs to hold against the *honest* prover strategy, since that is the prover we designed the protocol with.” Identify exactly what is wrong with this claim, and describe a concrete way a cheating prover could exploit the mistaken reading on a false instance.
11. Consider modifying the non-isomorphism protocol so the verifier sends $H = \pi(G_1)$ *always* (never using G_2 , and never flipping the bit b). Show that this broken protocol is no longer sound for *NONISO* by exhibiting a cheating prover and a false instance on which the verifier always accepts. What essential feature of the original protocol did the modification destroy?
12. Explain why the inclusion $\text{IP} \subseteq \text{PSPACE}$ can be proved by evaluating a game tree in polynomial space, and identify which tree levels correspond to a *maximum* over prover messages and which correspond to a *weighted average* over verifier coins. Why is the depth of this tree polynomial rather than exponential?

Chapter 16

Algebraic and Combinatorial Methods in Computation

16.1 Monoids and Machines: Recognition by Finite Monoids

A deterministic finite automaton reads a string letter by letter and slides between states; what survives that reading is not the string but its *effect* on the state set. This section makes that effect into an algebraic object. Each string induces a function on states, concatenation of strings becomes composition of functions, and the collection of all such functions is a finite *monoid* – a set with an associative product and an identity (the empty string ε). Reading a language through a homomorphism into a finite monoid turns out to be exactly as powerful as a DFA, and the *smallest* such monoid – the syntactic monoid – is the algebraic twin of the Myhill–Nerode equivalence from regular-language theory. We close with the dictionary entry that makes the whole subject worth the algebra: a regular language is star-free iff its syntactic monoid contains no nontrivial group.

Monoids and homomorphisms

Definition 16.1.1 (Monoid). A *monoid* is a triple $(M, \cdot, 1)$ where M is a set, $\cdot: M \times M \rightarrow M$ is an associative binary operation ($a(bc) = (ab)c$ for all $a, b, c \in M$), and $1 \in M$ is a two-sided *identity* ($1a = a1 = a$ for all a). A monoid is *finite* if M is finite. A *submonoid* is a subset closed under $\cdot$ and containing 1.

A group is a monoid in which every element has an inverse; a monoid need not be a group. The motivating example is purely combinatorial: for any alphabet Σ , the set Σ^* of all finite strings, with concatenation as the product and the empty string ε as identity, is a monoid – the *free monoid* on Σ . It is the universal host: every monoid we meet will receive strings from Σ^* through a structure-preserving map.

Definition 16.1.2 (Monoid homomorphism). A *homomorphism* of monoids $\phi: M \rightarrow N$ is a map with $\phi(ab) = \phi(a)\phi(b)$ for all $a, b \in M$ and $\phi(1_M) = 1_N$. A homomorphism out of a free monoid Σ^* is determined by its values on the single letters, since $\phi(a_1a_2 \cdots a_k) = \phi(a_1)\phi(a_2) \cdots \phi(a_k)$ and $\phi(\varepsilon) = 1_N$.

This last remark is the engine of the whole section: *to specify a homomorphism out of Σ^* , you only name where each letter goes*, and the empty string is forced to the identity. (This is the monoid analogue of the Dummit–Foote fact that a group homomorphism out of a free group is fixed by its values on generators.)

The transition monoid of a DFA

Fix a DFA $M = (Q, \Sigma, \delta, q_0, F)$. A single letter $a \in \Sigma$ moves each state q to $\delta(q, a)$, so a names a function $Q \rightarrow Q$. A whole string w names the composite function $q \mapsto \hat{\delta}(q, w)$. Composing the function for u with the function for v gives the function for uv : reading u then v . The set of all functions that arise, under composition, is a finite monoid.

Definition 16.1.3 (Transition monoid). For a DFA M with state set Q , let $\rho_w: Q \rightarrow Q$ be the map $\rho_w(q) = \widehat{\delta}(q, w)$ for $w \in \Sigma^*$. The *transition monoid* of M is

$$T(M) = \{\rho_w \mid w \in \Sigma^*\} \subseteq Q^Q,$$

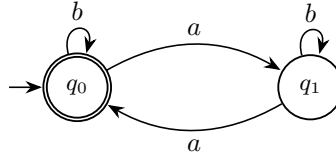
with product $\rho_u \rho_v = \rho_{uv}$ (apply u first, then v) and identity $\rho_\varepsilon = \text{id}_Q$.

Because Q^Q is finite ($|Q|^{|Q|}$ functions), $T(M)$ is finite, and $w \mapsto \rho_w$ is a surjective homomorphism $\Sigma^* \rightarrow T(M)$. The product is honest function composition, so associativity comes for free; the only modelling decision is the order of composition, which we fix as “read left to right.”

Worked Example 16.1.4 (Compute the transition monoid of a 2-state parity DFA). Let M_1 over $\Sigma = \{a, b\}$ have states $\{q_0, q_1\}$ with a *flipping* the state and b *fixing* it; start q_0 , accept q_0 . Compute $T(M_1)$ and its multiplication table.

Recognition. The trigger is “transition monoid of a small DFA”: enumerate the functions $Q \rightarrow Q$ that strings induce, stopping when no new function appears. Each letter is a function; multiply by composing.

Solution. The machine and the two single-letter functions:



Write a function on Q as the pair (image of q_0 , image of q_1). Then $\rho_b = (q_0, q_1) = \text{id}$ (b fixes both states) and $\rho_a = (q_1, q_0)$ (the swap). Composing: $\rho_{aa} = \rho_a \rho_a = \text{id} = \rho_\varepsilon$, and every ρ_w depends only on the parity of the number of a 's. So

$$T(M_1) = \{\text{id}, s\} \quad \text{where } s = \rho_a = (q_1, q_0), \quad s^2 = \text{id}.$$

This is the two-element group $\mathbb{Z}/2\mathbb{Z}$. Its table:

$\cdot$	id	s
id	id	s
s	s	id

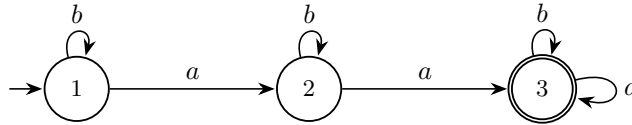
Which idea, and why? The transition monoid is the image of Σ^* under $w \mapsto \rho_w$; here only two functions on two states ever occur, the identity and the swap, and they form a group because the swap is its own inverse.

Check. $\widehat{\delta}(q_0, abab)$: $abab$ has two a 's, even parity, so it lands back at q_0 and is accepted – consistent with $\rho_{abab} = s^2 \cdot \text{id} \cdots = \text{id}$, fixing q_0 .

Worked Example 16.1.5 (A transition monoid that is not a group). Over $\Sigma = \{a, b\}$ let M_2 have states $\{1, 2, 3\}$, with a acting as $1 \mapsto 2 \mapsto 3 \mapsto 3$ and b acting as the identity on states. Compute $T(M_2)$ and observe it has no inverses for the non-identity elements.

Recognition. “Transition monoid”: list the induced functions $Q \rightarrow Q$. The trigger that this will *not* be a group is that a collapses two states onto 3 – a non-injective function has no inverse in Q^Q .

Solution. The diagram, with a pushing every state one step toward the sink 3:



Write a function as the triple $(1 \mapsto ?, 2 \mapsto ?, 3 \mapsto ?)$. Then $\rho_b = \text{id} = (1, 2, 3)$, $\rho_a = (2, 3, 3)$, $\rho_{aa} = (3, 3, 3)$, and $\rho_{aaa} = (3, 3, 3) = \rho_{aa}$. No further functions appear, so

$$T(M_2) = \{\text{id}, \rho_a = (2, 3, 3), \rho_{a^2} = (3, 3, 3)\}.$$

The element ρ_{a^2} is a *constant* map (everything to 3); composing it with anything on the left or right still sends everything to 3, so it can never be undone – there is no x with $\rho_{a^2} x = \text{id}$. Table (rows = first factor):

·	id	ρ_a	ρ_{a^2}
id	id	ρ_a	ρ_{a^2}
ρ_a	ρ_a	ρ_{a^2}	ρ_{a^2}
ρ_{a^2}	ρ_{a^2}	ρ_{a^2}	ρ_{a^2}

Which idea, and why? A monoid is a group only if every element is invertible; the moment a string induces a non-injective (here constant) function on states, that element has no inverse, so the transition monoid is a genuine monoid and not a group.

Check. $\rho_a^2 = \rho_{a^2}$ and $\rho_a^3 = \rho_{a^2}$: once you reach the constant map you stay there. The element ρ_a satisfies $\rho_a^2 = \rho_a^3$, the algebraic trace of a sink state.

Recognition by a finite monoid

The transition monoid suggests a machine-free notion of accepting a language: push the string through a homomorphism into *any* finite monoid, then check membership of the result in a fixed subset.

Definition 16.1.6 (Recognition). A homomorphism $\phi: \Sigma^* \rightarrow M$ into a finite monoid M *recognizes* a language $L \subseteq \Sigma^*$ if there is a subset $S \subseteq M$ with $L = \phi^{-1}(S) = \{w \in \Sigma^* \mid \phi(w) \in S\}$. We say M recognizes L if some such ϕ and S exist.

Theorem 16.1.7 (Monoid recognition = regularity). *A language $L \subseteq \Sigma^*$ is regular if and only if some finite monoid recognizes it.*

Proof. ($\Rightarrow$) Let M be a DFA with $L = L(M)$. Take the transition monoid $T(M)$ as the finite monoid, and $\phi(w) = \rho_w$, a homomorphism by Definition 16.1.3. Put $S = \{\rho_w \mid \hat{\delta}(q_0, w) \in F\}$, the functions that carry the start state into the accept set. A string w is accepted iff $\rho_w(q_0) \in F$ iff $\rho_w \in S$, so $L = \phi^{-1}(S)$.

($\Leftarrow$) Suppose $\phi: \Sigma^* \rightarrow M$ recognizes $L = \phi^{-1}(S)$ with M finite. Build a DFA with state set M , start state 1_M , accept set S , and transitions $\delta(m, a) = m \cdot \phi(a)$. By induction on $|w|$, $\hat{\delta}(1_M, w) = \phi(w)$, so the machine accepts w iff $\phi(w) \in S$ iff $w \in L$. Thus L is regular. $\square$

Recognition is one square. The free monoid sits on the left, the finite monoid on the right, ϕ carries strings to elements, and the accepting subset $S \subseteq M$ pulls back along ϕ to the language L on the left – reading w is computing $\phi(w)$, and acceptance is the single test $\phi(w) \in S$.

$$\begin{array}{ccc}
 \Sigma^* & \xrightarrow{\phi} & M \\
 \uparrow & & \uparrow \\
 L = \phi^{-1}(S) & \xrightarrow{\phi} & S
 \end{array}$$

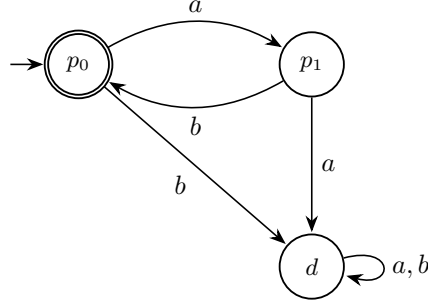
Check. The square commutes by construction: $w \in L \iff \phi(w) \in S$, so the bottom row is exactly the restriction of ϕ to L , landing in S ; the vertical hooks are the inclusions $L \hookrightarrow \Sigma^*$ and $S \hookrightarrow M$.

Takeaway. A finite monoid M *recognizes* L through a homomorphism $\phi: \Sigma^* \rightarrow M$ together with an accepting set $S \subseteq M$ for which $L = \phi^{-1}(S)$: read w to the element $\phi(w)$, and accept exactly when it lands in S . By Theorem 16.1.7 a language is regular iff some finite monoid recognizes it, and the transition monoid $T(M)$ (or the syntactic monoid $M(L)$) of a DFA furnishes such an M – the invariant displayed is the recognized language L itself, independent of which recognizer M one chooses.

Worked Example 16.1.8 (A six-element monoid recognizes $(ab)^*$). Show that the language $L = (ab)^* = \{\varepsilon, ab, abab, \dots\}$ over $\Sigma = \{a, b\}$ is recognized by a small finite monoid, and exhibit ϕ and S .

Recognition. “Recognized by a monoid”: by Theorem 16.1.7 build a DFA, then take its transition monoid. The trigger “ $(ab)^*$ ” says we need to track “properly alternating, starting and ending correctly,” which a 3-state DFA with a dead state does.

Solution. A DFA for L : p_0 (start, accept) expects an a ; p_1 expects a b ; d is the dead state for any violation.



Write functions as triples $(p_0 \mapsto?, p_1 \mapsto?, d \mapsto?)$. Then $\rho_a = (p_1, d, d)$, $\rho_b = (d, p_0, d)$, $\rho_{ab} = (p_0, d, d)$ (do a then b : $p_0 \rightarrow p_1 \rightarrow p_0$, $p_1 \rightarrow d$, $d \rightarrow d$), $\rho_{ba} = (d, p_1, d)$, and $\rho_{aa} = \rho_{bb} = (d, d, d)$ the constant sink. Computing all products closes up at six elements:

$$M = \{ 1 = \rho_\varepsilon, \alpha = \rho_a, \beta = \rho_b, \alpha\beta = \rho_{ab}, \beta\alpha = \rho_{ba}, z = \rho_{aa} = (d, d, d) \},$$

where z is a *zero* ($zx = xz = z$ for all x), $\alpha^2 = \beta^2 = z$, $\alpha\beta\alpha = \alpha$, and $\beta\alpha\beta = \beta$. Define ϕ by $\phi(a) = \alpha$, $\phi(b) = \beta$ and set $S = \{1, \alpha\beta\}$ – the identity (for ε) and the element (p_0, d, d) that fixes the start state. Then $w \in L$ iff $\rho_w(p_0) = p_0$ iff $\rho_w \in S$, so $L = \phi^{-1}(S)$.

Which idea, and why? Recognition by a monoid is Theorem 16.1.7 in action: the transition monoid of any DFA recognizes the DFA’s language, with S the functions sending start to an accept state. Here that monoid has just six elements.

Check. **abab**: $\phi(abab) = \alpha\beta\alpha\beta = (\alpha\beta)(\alpha\beta)$; since $\alpha\beta = (p_0, d, d)$ fixes p_0 , its square fixes p_0 , lands in S , accepted. And **aba**: $\phi(aba) = \alpha\beta\alpha$; $\alpha\beta$ fixes p_0 then α sends $p_0 \rightarrow p_1 \neq p_0$, not in S , rejected – correct.

The syntactic congruence and the syntactic monoid

Among all monoids recognizing L there is a smallest one, obtained by collapsing Σ^* exactly as much as L permits. The right notion of “collapse” is a congruence: an equivalence relation compatible with the product, so that the quotient is again a monoid (the monoid analogue of a normal subgroup giving a quotient group in Dummit–Foote).

Definition 16.1.9 (Congruence; syntactic congruence). An equivalence $\sim$ on Σ^* is a *congruence* if $u \sim u'$ and $v \sim v'$ imply $uv \sim u'v'$; then $\Sigma^*/\sim$ is a monoid under $[u][v] = [uv]$. For a language L , the *syntactic congruence* $\equiv_L$ is defined by

$$u \equiv_L v \iff (\forall x, y \in \Sigma^* : xuy \in L \iff xvy \in L),$$

“ u and v are interchangeable in every two-sided context.”

Proposition 16.1.10 ($\equiv_L$ is the coarsest congruence saturating L). *For every language L , $\equiv_L$ is a congruence, L is a union of $\equiv_L$ -classes, and every congruence $\sim$ with L a union of $\sim$ -classes refines $\equiv_L$ (i.e. $u \sim v \Rightarrow u \equiv_L v$).*

Proof. Congruence. Suppose $u \equiv_L u'$ and $v \equiv_L v'$; we show $uv \equiv_L u'v'$. For any contexts x, y , replacing u by u' in the context (x, vy) gives $xu'vy \in L \iff xu'vy \in L$; then replacing v by v' in the context (xu', y) gives $xu'v'y \in L \iff xu'v'y \in L$. Chaining, $xuvy \in L \iff xu'v'y \in L$ for all x, y , i.e. $uv \equiv_L u'v'$. *Saturation.* Taking $x = y = \varepsilon$ in the definition, $u \equiv_L v$ forces $u \in L \iff v \in L$, so each class lies wholly inside or outside L . *Coarsest.* If $\sim$ is a congruence saturating L and $u \sim v$, then for any x, y we have $xuy \sim xvy$ (congruence), so xuy and xvy lie in the same $\sim$ -class, hence both in L or both out (saturation). That is exactly $u \equiv_L v$. $\square$

Definition 16.1.11 (Syntactic monoid). The *syntactic monoid* of L is the quotient $M(L) = \Sigma^*/\equiv_L$, with the *syntactic homomorphism* $\eta: \Sigma^* \rightarrow M(L)$, $\eta(w) = [w]$. It recognizes L via $S = \eta(L) = \{[w] \mid w \in L\}$.

Theorem 16.1.12 (Myhill–Nerode, algebraic form). *A language L is regular iff $M(L)$ is finite, in which case $M(L)$ is the smallest monoid (fewest elements) recognizing L , and it divides every monoid that recognizes L .*

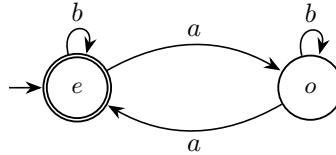
Proof sketch. The classical Myhill–Nerode relation $u \sim_L v$ (“ $uz \in L \Leftrightarrow vz \in L$ for all z ,” a *right* congruence) has finitely many classes iff L is regular, and those classes are the states of the minimal DFA. The two-sided $\equiv_L$ refines $\sim_L$ (take $x = \varepsilon$) but is itself refined by the relation $\rho_u = \rho_v$ on the states of the minimal DFA; so $\equiv_L$ has finitely many classes iff $\sim_L$ does, and finiteness of $M(L)$ is equivalent to regularity. Minimality and division: if $\phi: \Sigma^* \rightarrow N$ recognizes $L = \phi^{-1}(S)$, then $\phi(u) = \phi(v)$ implies $xuy \in L \Leftrightarrow xvy \in L$ for all x, y (apply ϕ to both and use $\phi^{-1}(S) = L$), so η factors through ϕ ; hence $M(L)$ is a quotient of a submonoid of N – it “divides” N – and is therefore no larger than any recognizer. $\square$

So the syntactic monoid is the Myhill–Nerode minimal DFA, re-read as algebra: states are right-contexts, but the full two-sided structure assembles them into a *monoid*, and that monoid is a canonical invariant of L – it does not depend on any chosen machine.

Worked Example 16.1.13 (The syntactic monoid of “even number of a ’s”). Over $\Sigma = \{a, b\}$ let $L = \{w \mid |w|_a \equiv 0 \pmod{2}\}$, strings with an even count of a . Build $M(L)$ from a DFA and identify it as a known group.

Recognition. “Syntactic monoid”: take a DFA, form its transition monoid; for a minimal DFA the transition monoid *is* the syntactic monoid. Trigger: “count mod 2” is a parity, so two states suffice.

Solution. The minimal DFA tracks parity of a ’s; b is a self-loop everywhere.



As functions on $\{e, o\}$: $\rho_b = \text{id}$, $\rho_a = \text{swap}$, and $\rho_a^2 = \text{id}$. The transition monoid is $\{\text{id}, s\}$ with $s^2 = \text{id}$, i.e. $\mathbb{Z}/2\mathbb{Z}$. Since the DFA is minimal and the two single-letter functions already distinguish all contexts, this two-element group is exactly $M(L)$, with $\eta(w) = s^{|w|_a \bmod 2}$ and $S = \{\text{id}\}$. The syntactic monoid *is* a group.

Which idea, and why? For a minimal DFA the transition monoid equals the syntactic monoid; a mod- k counting language has a cyclic group $\mathbb{Z}/k\mathbb{Z}$ as its syntactic monoid, the algebraic signature of genuine counting.

Check. $\eta(abba) = s^2 = \text{id} \in S$ (two a ’s, accepted); $\eta(ababa) = s^3 = s \notin S$ (three a ’s, rejected). The group element is literally the parity.

Aperiodic monoids and star-free languages

The contrast between the two-state parity machine (whose monoid was the *group* $\mathbb{Z}/2\mathbb{Z}$) and the sink machine (whose monoid had a constant “zero”) is the central dichotomy of the theory. Counting modulo k needs a cyclic group; languages that avoid all such counting have monoids with no nontrivial group inside.

Definition 16.1.14 (Aperiodic monoid). A finite monoid M is *aperiodic* (or *group-free*) if there is an integer $n \geq 1$ with $x^n = x^{n+1}$ for every $x \in M$. Equivalently, M contains no subset that is a nontrivial group under M ’s product.

The equivalence is the reason for the name: $x^n = x^{n+1}$ says the cyclic submonoid generated by x eventually becomes constant rather than cycling, so it cannot host a nontrivial cyclic group. The group $\mathbb{Z}/2\mathbb{Z}$ fails this ($s^n = s^{n+1}$ would force $s = \text{id}$); the sink monoid of Worked Example 16.1.5 passes it ($\rho_a^2 = \rho_a^3$, and every element satisfies $x^2 = x^3$).

Definition 16.1.15 (Star-free language). The *star-free* languages over Σ are the smallest class containing $\emptyset$, $\{a\}$ for each letter a , and the whole Σ^* , and closed under finite union, concatenation, and *complement* – but *not* Kleene star. (Star is recovered only in the restricted form Σ^* via complement; e.g. a^* is star-free because it is the complement of “contains a non- a letter,” but “even length” is not.)

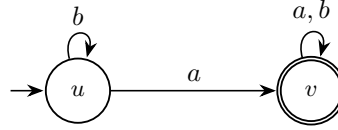
Theorem 16.1.16 (Schützenberger). A regular language L is star-free if and only if its syntactic monoid $M(L)$ is aperiodic.

We state this landmark without proof: the forward direction shows that the star-free operations can only build aperiodic monoids, and the converse decomposes an aperiodic monoid into star-free pieces. The theorem turns a question about *syntax* (can L be written without star?) into a finite *algebraic* test (does $M(L)$ have a nontrivial group?), which is decidable: compute $M(L)$, check $x^n = x^{n+1}$.

Worked Example 16.1.17 (Aperiodic vs. non-aperiodic: a star-free language and a counting language). Decide which of $L_1 = \Sigma^* a \Sigma^*$ (“contains at least one a ”) and $L_2 = \{w \mid |w|_a \equiv 0 \pmod{2}\}$ is star-free, using Schützenberger’s criterion.

Recognition. “Star-free or not”: by Theorem 16.1.16 compute each syntactic monoid and test aperiodicity $x^n = x^{n+1}$. Trigger: “contains an a ” smells star-free (a pattern), “even number of a ’s” smells like counting (a group).

Solution. L_1 . A two-state DFA: u (no a yet) and v (seen an a , accept).



Functions on $\{u, v\}$: $\rho_\varepsilon = \text{id} = (u, v)$, $\rho_a = (v, v)$, $\rho_b = (u, v) = \text{id}$. So $M(L_1) = \{\text{id}, c\}$ with $c = (v, v)$ the constant map; $c^2 = c$, hence $x^1 = x^2$ for every element. *Aperiodic* ($n = 1$), so L_1 is star-free – indeed $L_1 = \Sigma^* \cdot \{a\} \cdot \Sigma^*$, built using $\Sigma^* = \overline{\emptyset}$, no star. L_2 . By Worked Example 16.1.13 its syntactic monoid is $\mathbb{Z}/2\mathbb{Z} = \{\text{id}, s\}$ with $s^2 = \text{id}$. Then $s^1 = s \neq \text{id} = s^2$, and no n gives $s^n = s^{n+1}$ (the powers cycle $s, \text{id}, s, \text{id}, \dots$). *Not aperiodic*, so L_2 is *not* star-free.

language	syntactic monoid	$x^n = x^{n+1}$?	star-free?
$L_1 = \Sigma^* a \Sigma^*$	$\{\text{id}, c\}$, $c^2 = c$	yes ($n=1$)	yes
$L_2 = \text{even } \#a$	$\mathbb{Z}/2\mathbb{Z}$, $s^2 = \text{id}$	no	no

Which idea, and why? Schützenberger’s theorem reduces “star-free?” to “aperiodic syntactic monoid?”. A constant-collapsing monoid is aperiodic (pattern-matching, no counting); a cyclic group inside the monoid is the obstruction, and modular counting puts one there.

Check. Directly from Definition 16.1.9: for every n , exactly one of a^n and a^{n+1} has an even number of a ’s, so the empty context separates them and $a^n \not\equiv_{L_2} a^{n+1}$ – no power of $[a]$ ever stabilizes, exactly what the live group $\mathbb{Z}/2\mathbb{Z}$ detects.

Method (How to build the syntactic monoid from a DFA).

1. **Minimize the DFA.** Run state minimization so distinct states mean distinct right-contexts; then the transition monoid will be the syntactic monoid, not a larger recognizer.
2. **Write each letter as a function on states.** For each $a \in \Sigma$, record the tuple $(\delta(q, a))_{q \in Q}$ – the image of every state. These generate the monoid.
3. **Close under composition.** Repeatedly compose known functions ($\rho_u \rho_v = \rho_{uv}$, u then v) until no new function appears. Always include $\rho_\varepsilon = \text{id}$. The result is a finite list of functions $Q \rightarrow Q$.
4. **Fill the multiplication table** and mark the accepting subset $S = \{\rho_w \mid \rho_w(q_0) \in F\}$.
5. **Read off structure.** Identify the identity, any zero (constant map), and test aperiodicity by checking $x^n = x^{n+1}$ for each element. A nontrivial cyclic block means a group inside, hence (by Schützenberger) the language is not star-free.

Caution. A monoid is *not* a group: do not assume elements have inverses. The transition monoid of any DFA with a sink or a merge contains non-injective functions, which are never invertible. Writing “ x^{-1} ” or cancelling $xa = xb \Rightarrow a = b$ inside a syntactic monoid is illegitimate unless you have checked the element is a unit.

Caution. The identity element is the image of ε , *not* of any letter, and it must be included even when no single letter induces id. In Worked Example 16.1.8 neither ρ_a nor ρ_b equals id, yet $\rho_\varepsilon = \text{id}$ is a genuine element of the monoid and lies in S (because $\varepsilon \in L$). Forgetting $\varepsilon \mapsto 1$ both miscounts the monoid and, when $\varepsilon \in L$, breaks recognition.

Caution. Star-free does not mean “no star in the given expression”: it means *some* expression avoids star. And “aperiodic” is about the *syntactic* monoid specifically – the transition monoid of a non-minimal DFA can be larger and even contain spurious group elements, so always minimize (or pass to $M(L)$) before applying Schützenberger’s test.

Exercises.

- Over $\Sigma = \{a, b\}$ let M have states $\{0, 1, 2\}$, start 0, with a acting as $0 \mapsto 1 \mapsto 2 \mapsto 0$ (a 3-cycle) and b acting as the identity on states. Draw the state diagram, write ρ_a and ρ_b as functions on $\{0, 1, 2\}$, and compute the full transition monoid $T(M)$. Which familiar group is it? Is it aperiodic?
- Let M over $\Sigma = \{a, b\}$ have states $\{1, 2\}$ with a the swap and b the constant map sending both states to 1. List every function on $\{1, 2\}$ that some string induces, give the multiplication table of $T(M)$, and decide whether $T(M)$ is a group, a monoid that is not a group, or aperiodic (it may be more than one of these).
- Specify the monoid homomorphism $\phi: \{a, b\}^* \rightarrow M$ where $M = \mathbb{Z}/3\mathbb{Z}$ (written additively, identity 0) by $\phi(a) = 1$ and $\phi(b) = 0$. Describe in words the language $L = \phi^{-1}(\{0\})$, and write down a subset $S \subseteq M$ so that $\phi^{-1}(S)$ is instead the language “the number of a ’s is 1 or 2 mod 3.”
- Prove directly from Definition 16.1.2 that if $\phi: \Sigma^* \rightarrow M$ is any monoid homomorphism, then $\phi(\varepsilon) = 1_M$, and that ϕ is determined by the values $\phi(a)$ for $a \in \Sigma$. (Hint: induct on $|w|$.)
- For the language $L = (\mathbf{ba})^*$ over $\{a, b\}$, draw a DFA, compute its transition monoid as functions on the state set, and exhibit a homomorphism ϕ and a subset S with $L = \phi^{-1}(S)$. How many elements does your monoid have?
- Let $L = \Sigma^* \mathbf{ab} \Sigma^*$ (“contains ab as a substring”) over $\{a, b\}$. Build a minimal DFA, read off the syntactic monoid as the transition monoid, and verify $x^2 = x^3$ holds for every element (so L is aperiodic, hence star-free by Schützenberger). Identify the zero element.
- Compute the syntactic congruence classes of $L = \{w \in \{a\}^* \mid |w| \equiv 0 \pmod{3}\}$ directly from Definition 16.1.9 (only the letter a occurs, so contexts are powers of a). Show there are exactly three classes and that the syntactic monoid is $\mathbb{Z}/3\mathbb{Z}$. Is L star-free? Justify with Schützenberger’s theorem.
- Prove that the syntactic monoid $M(L)$ recognizes L via $S = \eta(L)$; that is, show $\eta^{-1}(\eta(L)) = L$ using only the saturation property of $\equiv_L$ from Proposition 16.1.10.
- Find the bug. A student claims: “Every transition monoid is a group, because each string w has the inverse string $w^{\mathcal{R}}$ (its reversal), and $\rho_w \rho_{w^{\mathcal{R}}} = \rho_\varepsilon = \text{id}$.” Give a concrete two-state DFA and a string w for which $\rho_w \rho_{w^{\mathcal{R}}} \neq \text{id}$, and state the actual reason a transition monoid can fail to be a group.
- Find the bug. A student computes the transition monoid of the DFA in Worked Example 16.1.8 and reports $M = \{\rho_a, \rho_b, \rho_{ab}, \rho_{ba}, \rho_{aa}\}$ with five elements, concluding it recognizes $(\mathbf{ab})^*$ with $S = \{\rho_{ab}\}$. Identify the element they dropped, explain why it must be in the monoid, and say why omitting it makes the recognition of ε wrong.
- Let L be a regular language whose syntactic monoid is the constant-collapsing monoid $\{1, c\}$ with $c^2 = c$ (so c is a zero). Argue that L is aperiodic, and give an example of such an L over $\{a, b\}$ together with a star-free expression for it (no Kleene star, using complement and Σ^*).
- Suppose finite monoids M_1 and M_2 recognize languages L_1 and L_2 over the same alphabet via ϕ_i and S_i . Show that the product monoid $M_1 \times M_2$ (with componentwise product and identity $(1, 1)$) recognizes $L_1 \cap L_2$, by giving the homomorphism into $M_1 \times M_2$ and the accepting subset. What subset gives $L_1 \cup L_2$? Relate this to closure of the regular languages under intersection and union.

16.2 Group Actions and Counting: Burnside and Pólya

Many counting problems in computation ask not “how many objects” but “how many objects *up to a symmetry*” – how many distinct circuits ignoring a relabeling of wires, how many states of a board ignoring rotation, how many colorings of a ring ignoring the rotation that makes a necklace a necklace. The one key idea of this section is that a symmetry is a *group acting on a set*, that “the same up to symmetry” is exactly “in the same orbit,” and that the number of orbits has a closed formula – the Burnside (Cauchy–Frobenius) lemma – which replaces a hopeless enumeration of objects by a short average over the group. We build the machinery (action, orbit, stabilizer, the orbit–stabilizer theorem), prove Burnside’s lemma by a clean double count, package the bookkeeping as the cycle index and Pólya’s theorem, and run the method on necklaces and on colorings of a square, tying the counts back to the regular-language and complexity threads of the chapter.

Group actions, orbits, stabilizers

Definition 16.2.1 (Group action). Let $(G, \cdot, e)$ be a group and X a set. An *action* of G on X is a map $G \times X \rightarrow X$, written $(g, x) \mapsto g \cdot x$ (or just gx), satisfying

$$e \cdot x = x \quad \text{and} \quad g \cdot (h \cdot x) = (gh) \cdot x \quad \text{for all } g, h \in G, x \in X.$$

We say G *acts* on X , and call X a G -set.

The two axioms say that the identity does nothing and that acting by h then by g is the same as acting once by the product gh . A useful first consequence: each group element acts as a *permutation* of X .

Proposition 16.2.2 (Each g permutes X). *If G acts on X , then for every $g \in G$ the map $\sigma_g: X \rightarrow X$, $\sigma_g(x) = gx$, is a bijection, with inverse $\sigma_{g^{-1}}$. Hence an action is the same data as a homomorphism $G \rightarrow \text{Sym}(X)$ into the symmetric group on X .*

Proof. Using both axioms, $\sigma_{g^{-1}}(\sigma_g(x)) = g^{-1}(gx) = (g^{-1}g)x = ex = x$, and symmetrically $\sigma_g(\sigma_{g^{-1}}(x)) = x$. So $\sigma_{g^{-1}}$ is a two-sided inverse of σ_g , which is therefore a bijection. The assignment $g \mapsto \sigma_g$ sends products to composites, $\sigma_{gh} = \sigma_g \circ \sigma_h$, and e to id_X , i.e. it is a homomorphism $G \rightarrow \text{Sym}(X)$; conversely any such homomorphism defines an action by $gx := \sigma_g(x)$. $\square$

Definition 16.2.3 (Orbit, stabilizer, fixed set). Let G act on X and fix $x \in X$, $g \in G$.

- The *orbit* of x is $\text{Orb}(x) = \{gx \mid g \in G\} \subseteq X$.
- The *stabilizer* of x is $\text{Stab}(x) = \{g \in G \mid gx = x\} \subseteq G$.
- The *fixed set* of g is $\text{Fix}(g) = \{x \in X \mid gx = x\} \subseteq X$.

Mind the homes of these objects: an orbit and a fixed set are *subsets of X* , while a stabilizer is a *subset of G* . The orbits are the equivalence classes of “same up to symmetry,” so counting them is the whole game.

Proposition 16.2.4 (Orbits partition; stabilizers are subgroups). *Let G act on X . Then: (a) the relation $x \sim y \iff y = gx$ for some $g \in G$ is an equivalence relation, whose classes are exactly the orbits; in particular the orbits partition X . (b) $\text{Stab}(x)$ is a subgroup of G for every x .*

Proof. (a) Reflexive: $x = ex$. Symmetric: if $y = gx$ then $x = g^{-1}y$ by Proposition 16.2.2. Transitive: if $y = gx$ and $z = hy$ then $z = h(gx) = (hg)x$. An equivalence relation partitions its underlying set, and the class of x is $\{y \mid x \sim y\} = \{gx \mid g \in G\} = \text{Orb}(x)$. (b) $e \in \text{Stab}(x)$ since $ex = x$. If $g, h \in \text{Stab}(x)$ then $(gh)x = g(hx) = gx = x$, so $gh \in \text{Stab}(x)$. If $gx = x$ then applying g^{-1} gives $x = g^{-1}x$, so $g^{-1} \in \text{Stab}(x)$. Thus $\text{Stab}(x) \leq G$. $\square$

Theorem 16.2.5 (Orbit–Stabilizer). *Let a finite group G act on a set X , and let $x \in X$. Then*

$$|G| = |\text{Orb}(x)| \cdot |\text{Stab}(x)|, \quad \text{equivalently} \quad |\text{Orb}(x)| = \frac{|G|}{|\text{Stab}(x)|}.$$

Proof. Write $H = \text{Stab}(x) \leq G$. Define Φ from the set G/H of left cosets to the orbit by $\Phi(gH) = gx$. It is well defined and injective at once: for $g, g' \in G$,

$$gH = g'H \iff g^{-1}g' \in H \iff (g^{-1}g')x = x \iff g'x = gx,$$

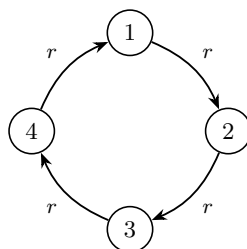
where the middle equivalence is the definition of H and the last applies g to both sides (legal by Proposition 16.2.2). And Φ is surjective because every orbit element is $gx = \Phi(gH)$ for some g . Hence Φ is a bijection $G/H \rightarrow \text{Orb}(x)$, so $|\text{Orb}(x)| = |G/H| = |G|/|H|$ by Lagrange's theorem (the number of left cosets of H is $|G|/|H|$). Rearranging gives the product form. $\square$

The picture worth carrying is this: the orbit of x is a single “ring” of states the group shuffles among themselves, and its size is forced to divide $|G|$.

Worked Example 16.2.6 (An action on the corners of a square, by hand). Let $G = \langle r \rangle = \{e, r, r^2, r^3\} \cong \mathbb{Z}/4\mathbb{Z}$ be the cyclic group of order 4, acting on the corner set $X = \{1, 2, 3, 4\}$ of a square by the quarter-turn $r: 1 \mapsto 2 \mapsto 3 \mapsto 4 \mapsto 1$. Compute $\text{Orb}(1)$ and $\text{Stab}(1)$ and check orbit–stabilizer.

Recognition. The trigger is “a group of rotations moving labeled positions.” Orbit = everywhere x can land; stabilizer = the rotations that pin x ; then Theorem 16.2.5 must balance the two against $|G| = 4$.

Solution. The four powers of r send corner 1 to 1, 2, 3, 4 respectively, so $\text{Orb}(1) = \{1, 2, 3, 4\} = X$: the action is *transitive*. Only $g = e$ fixes corner 1 (any nontrivial rotation moves it), so $\text{Stab}(1) = \{e\}$. The orbit is the whole 4-cycle:



Orbit–stabilizer reads $|G| = |\text{Orb}(1)| \cdot |\text{Stab}(1)| = 4 \cdot 1 = 4$, as required.

Which idea, and why? Orbit–Stabilizer (Theorem 16.2.5): the orbit size times the stabilizer size equals $|G|$, so a transitive action of $\mathbb{Z}/4\mathbb{Z}$ on 4 points forces trivial stabilizers.

Check. Every corner has the same trivial stabilizer (conjugate stabilizers within one orbit), and the single orbit of size 4 accounts for all of X , consistent with Proposition 16.2.4(a).

The Burnside (Cauchy–Frobenius) orbit-counting lemma

Theorem 16.2.7 (Burnside’s lemma). *Let a finite group G act on a finite set X . The number N of orbits is the average number of fixed points:*

$$N = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

Proof. Double-count the set of *incident pairs* $P = \{(g, x) \in G \times X \mid gx = x\}$. Grouping the pairs by their first coordinate, the pairs with a fixed g are exactly (g, x) with $x \in \text{Fix}(g)$, so $|P| = \sum_{g \in G} |\text{Fix}(g)|$. Grouping instead by the second coordinate, the pairs with a fixed x are (g, x) with $g \in \text{Stab}(x)$, so $|P| = \sum_{x \in X} |\text{Stab}(x)|$. Equate the two counts:

$$\sum_{g \in G} |\text{Fix}(g)| = \sum_{x \in X} |\text{Stab}(x)|.$$

By Theorem 16.2.5, $|\text{Stab}(x)| = |G|/|\text{Orb}(x)|$, so the right side is $|G| \sum_{x \in X} 1/|\text{Orb}(x)|$. Now split the sum over X by orbits (Proposition 16.2.4(a)): within a single orbit $\mathcal{O}$, every term is $1/|\mathcal{O}|$ and there are $|\mathcal{O}|$ of them, contributing exactly 1. With N orbits the sum is N . Hence $\sum_{g \in G} |\text{Fix}(g)| = |G| \cdot N$, which rearranges to the claim. $\square$

The proof is the whole method in miniature: a 0/1 incidence matrix with rows indexed by G and columns by X , summed two ways.

	columns = X				
	1	$M_{g,x} = [gx = x]$			
rows = G			1		$\sum_g \text{Fix}(g) $
				1	
	column x : $\sum_x \text{Stab}(x) $				

Counting colorings: actions on functions

To count *colorings* of a structured set we let G act on the positions and ride that up to an action on functions. Write Y^X for the set of all functions $f: X \rightarrow Y$ (a *coloring* assigns each position $x \in X$ a color $f(x) \in Y$); there are $|Y|^{|X|}$ of them.

Definition 16.2.8 (Induced action on colorings). If G acts on X , the *induced action* on Y^X is $(gf)(x) = f(g^{-1}x)$.

The g^{-1} is exactly what is needed for gh to act as g after h : $(g(hf))(x) = (hf)(g^{-1}x) = f(h^{-1}g^{-1}x) = f((gh)^{-1}x) = ((gh)f)(x)$, and $ef = f$. Two colorings lie in the same orbit iff one is carried to the other by a symmetry – “the same coloring up to symmetry.” The fixed colorings of g have a clean description.

Lemma 16.2.9 (Fixed colorings are constant on cycles). *Let G act on a finite X and $g \in G$. Under the induced action on Y^X , a coloring f satisfies $gf = f$ iff f is constant on each cycle of the permutation σ_g of X . Consequently, if $c(g)$ is the number of cycles of σ_g (counting fixed points as 1-cycles), then*

$$|\text{Fix}(g)| = |Y|^{c(g)}.$$

Proof. $gf = f$ means $f(g^{-1}x) = f(x)$ for all x , i.e. f is invariant under applying g^{-1} (equivalently g). Iterating, f is constant along every g -orbit of a point, which is precisely a cycle of σ_g . Conversely, a coloring constant on each cycle is unchanged by g . To build such an f we freely choose one color in Y for each of the $c(g)$ cycles, so there are $|Y|^{c(g)}$ of them. $\square$

Combining Lemma 16.2.9 with Burnside’s lemma turns coloring counts into a sum of $|Y|^{c(g)}$ over the group – a calculation driven entirely by the *cycle structure* of each group element.

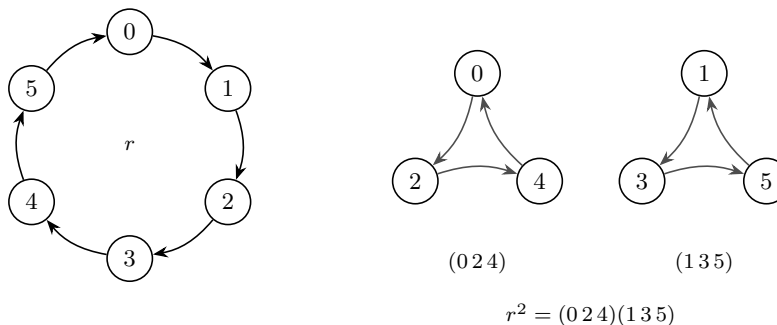
Worked Example 16.2.10 (Two-color necklaces of length 6 under rotation). A bead-6 necklace is a function $f: \mathbb{Z}/6\mathbb{Z} \rightarrow \{B, W\}$, two of them identified when one is a rotation of the other. Count the distinct necklaces with Burnside.

Recognition. The trigger is “arrangements on a ring, equal up to rotation.” The acting group is the cyclic group $C_6 = \langle r \rangle$ of the 6 rotations; by Lemma 16.2.9 each $|\text{Fix}(r^k)| = 2^{c(r^k)}$, and $c(r^k) = \gcd(k, 6)$ for a single 6-cycle rotated by k .

Solution. The rotation r acts on the 6 positions as one 6-cycle; its power r^k splits into $\gcd(k, 6)$ cycles each of length $6/\gcd(k, 6)$. Tabulate:

$g = r^k$	$c(g) = \gcd(k, 6)$	$ \text{Fix}(g) = 2^{c(g)}$
$r^0 = e$	6	64
r^1	1	2
r^2	2	4
r^3	3	8
r^4	2	4
r^5	1	2

The cyclic action on the labeled ring, and the way r^2 (which fixes the coloring iff all three “thirds” agree) breaks the ring into two 3-cycles:



By Burnside, the number of necklaces is

$$N = \frac{1}{6}(64 + 2 + 4 + 8 + 4 + 2) = \frac{84}{6} = 14.$$

Which idea, and why? Burnside’s lemma (Theorem 16.2.7) plus the cycle count (Lemma 16.2.9): average $2^{\gcd(k,n)}$ over the n rotations.

Check. The identity alone contributes $64 = 2^6$, the raw number of labeled colorings; dividing the corrected sum 84 by 6 gives an *integer*, 14, as any true orbit count must.

Worked Example 16.2.11 (The general necklace formula and its number-theoretic shadow). Derive the count of q -color necklaces of length n under the cyclic group C_n , and read off a divisibility fact.

Recognition. “Rotations of a ring of n beads, q colors.” Same engine as before but symbolic: $c(r^k) = \gcd(k, n)$ for the single n -cycle, so $|\text{Fix}(r^k)| = q^{\gcd(k,n)}$.

Solution. Burnside gives $N = \frac{1}{n} \sum_{k=0}^{n-1} q^{\gcd(k,n)}$. Group the k ’s by $d = \gcd(k, n)$: the number of $k \in \{0, \dots, n-1\}$ with $\gcd(k, n) = d$ equals $\varphi(n/d)$ (Euler’s totient), and as d ranges over divisors of n so does n/d . Hence

$$N = \frac{1}{n} \sum_{d|n} \varphi(n/d) q^d = \frac{1}{n} \sum_{d|n} \varphi(d) q^{n/d}.$$

The grouping of residues by their gcd with n :

$k :$	0	1	2	3	4	5
$\gcd(k, 6) :$	6	1	2	3	2	1

For $n = 6$ this is $\frac{1}{6}(\varphi(1)q^6 + \varphi(2)q^3 + \varphi(3)q^2 + \varphi(6)q^1) = \frac{1}{6}(q^6 + q^3 + 2q^2 + 2q)$; at $q = 2$ it returns 14.

Which idea, and why? Burnside with the totient bookkeeping: residues sharing a gcd with n contribute identical fixed counts, and there are $\varphi(n/d)$ of them.

Check. Since $N \in \mathbb{Z}$ for every q , the polynomial $\sum_{d|n} \varphi(d)q^{n/d}$ is divisible by n for all integers q – a nontrivial congruence (for $n = 6$, $6 \mid q^6 + q^3 + 2q^2 + 2q$) handed to us free by the orbit interpretation.

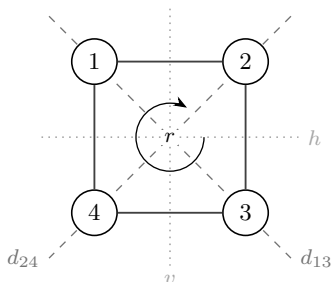
Colorings of a square under the dihedral group

The full symmetry group of a square is the dihedral group D_4 of order 8: the four rotations e, r, r^2, r^3 and four reflections. We count colorings of its *four corners*; the same template handles edges or faces.

Worked Example 16.2.12 (Colorings of a square’s corners under D_4). Count the q -colorings of the corners of a square, identifying colorings related by any symmetry in D_4 .

Recognition. “A square, up to rotation and reflection” is the signature of D_4 acting on the 4 corners. By Lemma 16.2.9 we need $c(g)$ for each of the 8 symmetries.

Solution. Label corners 1, 2, 3, 4 clockwise. List the eight symmetries as permutations of corners and read off their cycle counts (a diagonal reflection fixes two corners and swaps the other two; an edge-midpoint reflection swaps both pairs):



symmetry g	permutation	$c(g)$	$ \text{Fix}(g) = q^{c(g)}$
e	$(1)(2)(3)(4)$	4	q^4
r (90°)	$(1\ 2\ 3\ 4)$	1	q
r^2 (180°)	$(1\ 3)(2\ 4)$	2	q^2
r^3 (270°)	$(1\ 4\ 3\ 2)$	1	q
d_{13}	$(1)(3)(2\ 4)$	3	q^3
d_{24}	$(2)(4)(1\ 3)$	3	q^3
h	$(1\ 4)(2\ 3)$	2	q^2
v	$(1\ 2)(3\ 4)$	2	q^2

Averaging over the 8 elements,

$$N = \frac{1}{8}(q^4 + 2q + 3q^2 + 2q^3) = \frac{q^4 + 2q^3 + 3q^2 + 2q}{8}.$$

At $q = 2$: $N = \frac{16+16+12+4}{8} = \frac{48}{8} = 6$ distinct two-colorings of the corners.

Which idea, and why? Burnside over D_4 : each symmetry contributes $q^{(\#\text{cycles})}$, and the two diagonal reflections (each with two fixed corners) are what separate the dihedral count from the purely rotational one.

Check. The two-color orbits are easy to list directly – $\{0, 4\}$ black corners give $1 + 1$, $\{1, 3\}$ give $1 + 1$, and 2 black corners give 2 (adjacent vs. opposite) – totalling $1 + 1 + 1 + 1 + 2 = 6$, matching the formula.

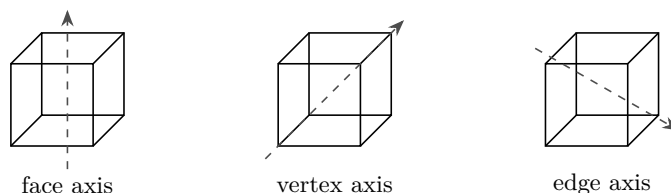
Worked Example 16.2.13 (Face-colorings of a cube under its rotation group). Count q -colorings of the 6 faces of a cube, up to the 24 rotations of the cube.

Recognition. “A cube up to rotation” means the rotation group of order 24 acting on 6 faces. Sort the rotations by axis type and count cycles on faces with Lemma 16.2.9.

Solution. Group the 24 rotations by conjugacy class and record their action on the face set:

rotation type	$\#g$	$c(g)$ on faces	contribution
identity	1	6	q^6
face axis $\pm 90^\circ$	6	3	$6q^3$
face axis 180°	3	4	$3q^4$
vertex axis $\pm 120^\circ$	8	2	$8q^2$
edge axis 180°	6	3	$6q^3$

A face-axis quarter turn cycles the four side faces and fixes the two on the axis, giving $c = 1 + 1 + 1 = 3$ (two fixed faces plus one 4-cycle); a vertex 120° turn makes two 3-cycles of faces, $c = 2$. The three axis families:



Burnside gives

$$N = \frac{1}{24} (q^6 + 6q^3 + 3q^4 + 8q^2 + 6q^3) = \frac{q^6 + 3q^4 + 12q^3 + 8q^2}{24}.$$

At $q = 2$: $N = \frac{64+48+96+32}{24} = \frac{240}{24} = 10$ distinct 2-colorings of a cube's faces.

Which idea, and why? Burnside over the order-24 rotation group: the count is a single average once each rotation's face-cycle count is known.

Check. The coefficients $1 + 6 + 3 + 8 + 6 = 24 = |G|$ account for every rotation, and $N = 10$ for $q = 2$ matches the classic hand count (e.g. 2 colorings with 3 black faces: the three meeting at a corner, or three forming a band).

Pólya enumeration: the cycle index

When we want a coloring count for *every* number of colors at once – or refined by how many beads get each color – we package all the cycle data of G into one polynomial.

Definition 16.2.14 (Cycle index). Let G act on X with $|X| = n$. For $g \in G$ let $c_i(g)$ be the number of i -cycles of σ_g . The *cycle index* of the action is the polynomial in variables $t_1, \dots, t_n$

$$Z_G(t_1, \dots, t_n) = \frac{1}{|G|} \sum_{g \in G} \prod_{i=1}^n t_i^{c_i(g)}.$$

Theorem 16.2.15 (Pólya enumeration, unweighted form). *If G acts on X with $|X| = n$ and Y is a set of q colors, the number of orbits of G on Y^X is $Z_G(q, q, \dots, q)$. More generally, assigning each color $y \in Y$ a variable and substituting $t_i \mapsto \sum_{y \in Y} y^i$ yields a generating function whose monomial $\prod_y y^{m_y}$ has coefficient equal to the number of orbits using each color y exactly m_y times.*

Proof sketch. Setting every $t_i = q$ makes $\prod_i t_i^{c_i(g)} = q^{\sum_i c_i(g)} = q^{c(g)} = |\text{Fix}(g)|$ by Lemma 16.2.9, so $Z_G(q, \dots, q) = \frac{1}{|G|} \sum_g |\text{Fix}(g)|$ is exactly Burnside's average – the orbit count. The weighted statement is a refinement of the same double count, replacing the count 1 per fixed coloring by its color-content monomial; because a fixed coloring of g is constant on cycles, the i -cycles each contribute $\sum_y y^i$, which is the substitution claimed. $\square$

Worked Example 16.2.16 (Cycle index of C_4 , and three color counts in one stroke). Compute the cycle index of C_4 acting on 4 positions, and use it to read off the number of 2-, 3-, and q -color necklaces of length 4.

Recognition. “One polynomial, all color counts” is the cue for the cycle index (Definition 16.2.14) and Theorem 16.2.15.

Solution. The four rotations have cycle types: e is four 1-cycles (t_1^4); r and r^3 are single 4-cycles (t_4 each); r^2 is two 2-cycles (t_2^2). Hence

$$Z_{C_4}(t_1, t_2, t_3, t_4) = \frac{1}{4} (t_1^4 + t_2^2 + 2t_4).$$

Substituting $t_i = q$ gives the orbit count $\frac{1}{4}(q^4 + q^2 + 2q)$. Evaluating:

q	$\frac{1}{4}(q^4 + q^2 + 2q)$	necklaces
2	$\frac{1}{4}(16 + 4 + 4)$	6
3	$\frac{1}{4}(81 + 9 + 6)$	24
q	$\frac{1}{4}(q^4 + q^2 + 2q)$	(polynomial)

For the *weighted* count with $Y = \{B, W\}$, substitute $t_i \mapsto B^i + W^i$:

$$Z_{C_4} = \frac{1}{4}[(B + W)^4 + (B^2 + W^2)^2 + 2(B^4 + W^4)] = B^4 + B^3W + 2B^2W^2 + BW^3 + W^4,$$

so there are exactly two necklaces with two black and two white beads (adjacent vs. opposite), the lone coefficient 2.

Which idea, and why? The cycle index (Definition 16.2.14) with Pólya's substitution (Theorem 16.2.15): one polynomial encodes the orbit count for all q and, when colors become variables, the count refined by color content.

Check. The plain count at $q = 2$ is 6, agreeing with the length-4 specialization of the $\frac{1}{n} \sum_{d|n} \varphi(d) q^{n/d}$ formula and with summing the weighted coefficients $1 + 1 + 2 + 1 + 1 = 6$.

Method (How to count objects up to symmetry with Burnside / Pólya).

1. **Name the set X and the group G .** X is the set of *positions* being colored or arranged (corners, beads, faces, wires); G is the symmetry group declared to identify objects (rotations C_n , dihedral D_n , the full symmetric group S_n for “relabeling”). The objects you count are the colorings Y^X or the structures, and “the same” means “same orbit.”
2. **Enumerate G by cycle type.** For each $g \in G$ write σ_g as a product of disjoint cycles on X and record $c(g)$ = the number of cycles. Group conjugate elements: they share a cycle type, so you only analyze one representative and multiply by the class size.
3. **Count fixed objects.** For colorings, $|\text{Fix}(g)| = q^{c(g)}$ (Lemma 16.2.9); for other structures, count directly the objects g leaves unchanged. *The identity always fixes all $|X|$ -many positions freely, contributing $q^{|X|} = |Y^X|$ – never omit it.*
4. **Average.** $N = \frac{1}{|G|} \sum_g |\text{Fix}(g)|$ (Theorem 16.2.7). The result is an integer; if it is not, recheck a cycle count.
5. **(Optional) refine with the cycle index.** For all color counts at once or for counts by color content, form Z_G (Definition 16.2.14) and substitute $t_i \mapsto q$ or $t_i \mapsto \sum_y y^i$ (Theorem 16.2.15).

Caution. The identity element fixes *every* coloring, so its term is the full $q^{|X|} = |Y^X|$ – the largest term in the Burnside sum. Forgetting it (or writing q^1 for it out of habit) wrecks the average. In the necklace-6 count the identity alone contributes 64 of the 84 total; drop it and you get 20/6, not even an integer.

Caution. Orbits are equivalence classes, *not* group elements. The answer to “how many distinct objects” is the number of orbits in X (or in Y^X), which by Burnside is the *average* of $|\text{Fix}(g)|$ over G – it is generally neither $|X|$ nor $|G|$ nor $|X|/|G|$. Dividing the raw object count $|Y^X|$ by $|G|$ is only correct when the action is *free* (every nonidentity g fixes nothing), which fails for necklaces the moment a coloring has a rotational symmetry of its own.

Caution. Choosing the wrong group changes the answer, and it is silent. Necklaces that may be *flipped over* are counted under the dihedral group D_n (order $2n$), not the cyclic C_n (order n); counting structures “up to relabeling of all n items” uses the full symmetric group S_n . State which symmetries you are quotienting by *before* you build the cycle table, since the corners of a square give 6 orbits under D_4 but $\frac{1}{4}(2^4 + 2^2 + 2 \cdot 2) = 6$ under C_4 only by coincidence at $q = 2$ – the polynomials differ.

These counting tools sit squarely on the chapter's algebra-combinatorics seam. The orbit count is, by Burnside, a *trace-like average* over the group – the same shape of computation that the permutation representation $g \mapsto \sigma_g$ of Proposition 16.2.2 makes literal: $|\text{Fix}(g)|$ is the trace of the permutation matrix of σ_g , so the orbit count is the average trace, the multiplicity of the trivial representation. Computationally, Burnside converts an enumeration whose naive cost is $|Y|^{|X|}$ (exponential in the number of positions) into a sum of $|G|$ cycle counts; when $|G|$ is small this is an exponential speedup, the algebraic cousin of the divide-by-symmetry savings that collapse a search space. The same orbit-counting principle reappears when we count distinct automata, distinct circuits up to wire-relabeling, or distinct graphs up to isomorphism (the S_n action on edge sets) – the last being exactly the symmetry that makes graph isomorphism a nontrivial problem rather than a syntactic check.

Exercises.

- Let the cyclic group $C_5 = \langle r \rangle$ act on the five corners of a regular pentagon by rotation. For the corner labeled 1, compute $\text{Orb}(1)$ and $\text{Stab}(1)$ explicitly, draw the orbit as a labeled 5-cycle, and verify the orbit-stabilizer identity $|C_5| = |\text{Orb}(1)| \cdot |\text{Stab}(1)|$.
- Consider $G = \mathbb{Z}/4\mathbb{Z}$ acting on $X = \{0, 1, 2, 3\}$ by $g \cdot x = (x + g) \bmod 4$. List $\text{Fix}(g)$ for each $g \in G$, then apply Burnside's lemma to compute the number of orbits. Confirm your answer by exhibiting the orbits directly.
- Use Burnside's lemma to count the distinct 3-color necklaces of length 4 under the rotation group C_4 . Build the full table of g , $c(g)$, and $|\text{Fix}(g)| = 3^{c(g)}$, then average. Check the result against the formula $\frac{1}{4}(q^4 + q^2 + 2q)$ at $q = 3$.
- Count the distinct 2-color necklaces of length 6 under the *dihedral* group D_6 (order 12), which adds the six reflections to the six rotations of Worked Example 16.2.10's setting. Tabulate the cycle count of each reflection (distinguish the reflections through two opposite beads from those through two opposite edge-midpoints), and compare your orbit count to the cyclic answer 14.
- Let S_3 act on the three vertices of a triangle (the full symmetry group $D_3 = S_3$). Write each of the six permutations in cycle notation, compute the cycle index $Z_{S_3}(t_1, t_2, t_3)$, and use it to find the number of q -colorings of the vertices up to symmetry. Evaluate at $q = 2$ and $q = 3$.
- Draw the regular hexagon with its six axes of reflection marked, labeling which three axes pass through opposite *vertices* and which three pass through opposite *edges*. For each kind of axis, give the cycle decomposition of the corresponding reflection acting on the six vertices, and state the resulting $c(g)$.
- Prove that for any finite group G acting on a finite set X , the quantity $\frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|$ is a nonnegative integer. (Hint: it counts something. Use the partition-into-orbits result and orbit-stabilizer; do not just cite Burnside as a black box – reproduce the double-counting step.)
- A circular arrangement of n beads in q colors is called *aperiodic* if no nontrivial rotation fixes it. Show that the number of aperiodic colorings (as labeled strings, before quotienting) is $\sum_{d|n} \mu(n/d) q^d$, where μ is the Möbius function, and explain how this counts *primitive necklaces* once divided by n . Relate this to the $\frac{1}{n} \sum_{d|n} \varphi(d) q^{n/d}$ formula by indicating where the two differ.
- Count the distinct ways to color the four *edges* of a square with q colors up to the action of D_4 . Set up the action of D_4 on the edge set (note the permutation a reflection induces on edges differs from its action on corners), build the cycle table, and give a closed formula in q .
- Find the bug. A student counts the 2-colorings of the corners of a square under D_4 as follows: "There are $2^4 = 16$ colorings and $|D_4| = 8$ symmetries, so by symmetry there are $16/8 = 2$ distinct colorings." Identify precisely why the division is wrong, name the property of the action that would have made it correct, and give the right answer with a one-line Burnside computation.
- Find the bug. To count 2-color necklaces of length 4 under C_4 , a student writes the Burnside sum as $\frac{1}{4}(2^1 + 2^4 + 2^2 + 2^4)$, reasoning " r is a 4-cycle so 2^1 , the identity and r^2 each fix everything so 2^4 , and r^3 is two 2-cycles so 2^2 ." Two of the four cycle counts are wrong. Correct the table (give the true cycle type of each of e, r, r^2, r^3) and recompute the orbit count.
- Let G be a finite group acting on a finite set X , and suppose the action is *transitive* (a single orbit). Using Burnside's lemma, prove that the average number of fixed points is exactly 1: $\sum_{g \in G} |\text{Fix}(g)| = |G|$. Then deduce that some nonidentity $g \in G$ has $\text{Fix}(g) = \emptyset$ whenever $|X| \geq 2$ (Jordan's theorem), by comparing the identity's contribution $|X|$ to the total $|G|$.
- Two finite-state machines over the same alphabet are considered "the same" if one is obtained from the other by renaming its k states. Model this as the symmetric group S_k acting on the set of transition tables, and explain in two or three sentences how Burnside's lemma would, in principle, count the distinct machines up to renaming. Identify the set X , the group G , and what plays the role of a "coloring" here. (You need not evaluate the sum.)

16.3 Generating Functions for Computation

A recurrence tells you how to compute the next value, but rarely tells you, at a glance, how the values *grow* – and growth is exactly what an algorithm’s running time is. A generating function packages an entire sequence $a_0, a_1, a_2, \dots$ into a single algebraic object, an element of the ring $\mathbb{C}[[x]]$ of formal power series, so that the recurrence becomes one equation you can *solve* for that object and then read the n th term back off as a coefficient. The one idea of this section is the pipeline *recurrence* $\rightarrow$ *generating function* $\rightarrow$ *closed form/coefficients*: it converts the counting and timing recurrences that govern recursive algorithms (Fibonacci-style splits, $T(n) = 2T(n-1) + 1$, the Catalan recursion behind every binary-tree parse) into formulas, and it ties the rate of growth of those coefficients directly to cost. Because the series are *formal*, no question of convergence ever arises: x is a placeholder, not a number.

Two kinds of generating function

Definition 16.3.1 (OGF and EGF). Let $a_0, a_1, a_2, \dots$ be a sequence of complex numbers. Its *ordinary generating function* (OGF) is the formal power series

$$A(x) = \sum_{n \geq 0} a_n x^n \in \mathbb{C}[[x]],$$

and its *exponential generating function* (EGF) is

$$\hat{A}(x) = \sum_{n \geq 0} a_n \frac{x^n}{n!} \in \mathbb{C}[[x]].$$

We write $[x^n]A(x) = a_n$ for the operation “extract the coefficient of x^n ,” so $[x^n]\hat{A}(x) = a_n/n!$.

The series live in $\mathbb{C}[[x]]$, where addition is termwise and multiplication is the Cauchy product $(\sum a_n x^n)(\sum b_n x^n) = \sum_n c_n x^n$ with $c_n = \sum_{k=0}^n a_k b_{n-k}$. Two standard facts, both proved by formal manipulation (never by substituting a value for x), do almost all the work below.

Proposition 16.3.2 (Geometric series and inversion). *In $\mathbb{C}[[x]]$ we have $(1-x)\sum_{n \geq 0} x^n = 1$, hence $\sum_{n \geq 0} x^n = \frac{1}{1-x}$, and more generally $\sum_{n \geq 0} r^n x^n = \frac{1}{1-rx}$ for any constant r . A series $A(x) = \sum a_n x^n$ has a multiplicative inverse in $\mathbb{C}[[x]]$ if and only if $a_0 \neq 0$.*

Proof. Multiplying out, $(1-x)\sum_{n \geq 0} x^n = \sum_{n \geq 0} x^n - \sum_{n \geq 0} x^{n+1} = 1$, since every power x^m with $m \geq 1$ cancels. Replacing x by rx gives the weighted form. For the inverse, if $A(x)B(x) = 1$ then comparing constant terms gives $a_0 b_0 = 1$, so $a_0 \neq 0$. Conversely if $a_0 \neq 0$ set $b_0 = 1/a_0$ and, for $n \geq 1$, solve $\sum_{k=0}^n a_k b_{n-k} = 0$ for $b_n = -a_0^{-1} \sum_{k=1}^n a_k b_{n-k}$ recursively; this defines a genuine $B(x) \in \mathbb{C}[[x]]$ with $A(x)B(x) = 1$. $\square$

Definition 16.3.3 (Generalized binomial coefficient). For $\alpha \in \mathbb{C}$ and $n \geq 0$,

$$\binom{\alpha}{n} = \frac{\alpha(\alpha-1) \cdots (\alpha-n+1)}{n!},$$

with $\binom{\alpha}{0} = 1$. Newton’s series $(1+x)^\alpha = \sum_{n \geq 0} \binom{\alpha}{n} x^n$ holds formally in $\mathbb{C}[[x]]$, where $(1+x)^\alpha$ is *defined* to be that series.

The four-step method for a recurrence

The whole technique is a fixed recipe; we state it once and then run it repeatedly.

Method (Solving a recurrence with a generating function).

1. **Multiply and sum.** Multiply the recurrence by x^n and sum over every n for which it is valid. Choose n to be the largest index appearing.

2. **Name the series.** Put $A(x) = \sum_{n \geq 0} a_n x^n$ and rewrite each shifted sum $\sum a_{n-k} x^n = x^k A(x)$, folding the boundary values $a_0, \dots$ into explicit low-order terms.
3. **Solve algebraically.** You now have one equation in $A(x)$. Solve for $A(x)$; if the recurrence is nonlinear (e.g. a convolution) you may get a quadratic in $A(x)$ and use the quadratic formula.
4. **Extract coefficients.** Expand $A(x)$ – by geometric series, by *partial fractions*, or by Newton’s series – and read off $a_n = [x^n]A(x)$.

Worked Example 16.3.4 (The runtime recurrence $T(n) = 2T(n-1) + 1$ by its OGF). A recursive routine on input size n does one unit of its own work and two recursive calls of size $n-1$; its step count satisfies $T(0) = 0$ and $T(n) = 2T(n-1) + 1$ for $n \geq 1$. Solve for $T(n)$ in closed form using the OGF, and confirm the cost is exponential.

Recognition. This is a first-order *linear* recurrence with a constant inhomogeneous term; the trigger “ a_n in terms of a_{n-1} plus a constant” says step 1 will produce a single rational equation in $A(x)$, solvable directly.

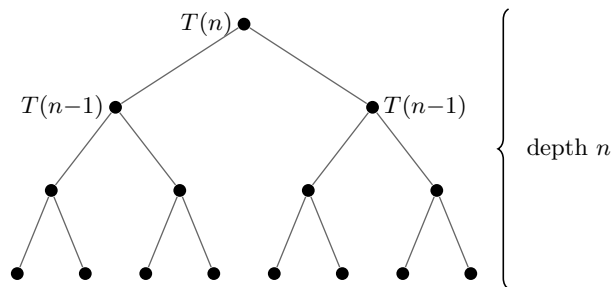
Solution. Let $A(x) = \sum_{n \geq 0} T(n)x^n$. Multiply $T(n) = 2T(n-1) + 1$ by x^n and sum over $n \geq 1$. The left side is $\sum_{n \geq 1} T(n)x^n = A(x) - T(0) = A(x)$. On the right, $\sum_{n \geq 1} 2T(n-1)x^n = 2x \sum_{n \geq 1} T(n-1)x^{n-1} = 2xA(x)$, and the constant term contributes $\sum_{n \geq 1} x^n = x/(1-x)$. Thus

$$A(x) = 2xA(x) + \frac{x}{1-x} \implies A(x)(1-2x) = \frac{x}{1-x} \implies A(x) = \frac{x}{(1-x)(1-2x)}.$$

Partial fractions: write $\frac{x}{(1-x)(1-2x)} = \frac{B}{1-x} + \frac{C}{1-2x}$, so $x = B(1-2x) + C(1-x)$. Setting $x = 1$ gives $1 = -B$, i.e. $B = -1$; setting $x = \frac{1}{2}$ gives $\frac{1}{2} = \frac{1}{2}C$, i.e. $C = 1$. Hence, by the geometric series (Proposition 16.3.2),

$$A(x) = \frac{1}{1-2x} - \frac{1}{1-x} = \sum_{n \geq 0} 2^n x^n - \sum_{n \geq 0} x^n = \sum_{n \geq 0} (2^n - 1)x^n,$$

so $T(n) = 2^n - 1$. The denominator’s reciprocal roots 1 and 2 became the bases 1^n and 2^n ; the larger one, 2, sets the growth. The recursion-tree picture explains the 2^n directly – the call tree is a complete binary tree of depth n , so it has $2^n - 1$ internal nodes, one “+1” charged at each:



Which idea, and why? A first-order linear recurrence with constant coefficients always yields a *rational* OGF whose denominator factors into $(1 - r_i x)$; partial fractions then turn each factor $1/(1 - r_i x)$ into the geometric term r_i^n , so a_n is a combination of the r_i^n (with polynomial coefficients when a factor repeats), dominated by the largest $|r_i|$.

Check. $T(0) = 2^0 - 1 = 0$, $T(1) = 1$, $T(2) = 3$, $T(3) = 7$ – and indeed $2 \cdot 7 + 1 = 15 = 2^4 - 1 = T(4)$. The leading 2^n flags the algorithm as exponential, the hallmark of unmemoized doubling recursion.

Worked Example 16.3.5 (Fibonacci: a two-term recurrence and partial fractions). The recurrence $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ counts (among much else) the number of leaf-calls in the naive recursive Fibonacci routine. Derive a closed form for F_n from its OGF.

Recognition. A second-order linear recurrence; the trigger “ a_n depends on the two previous terms” means step 2 introduces both $xA(x)$ and $x^2A(x)$, and the denominator will be a *quadratic* to be factored for partial fractions.

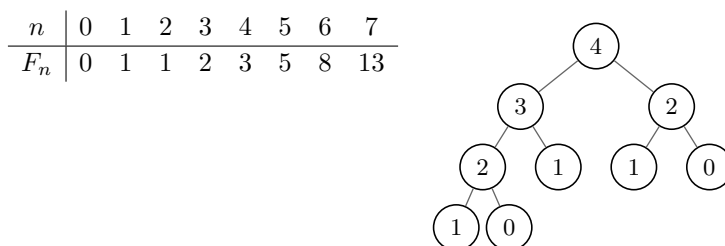
Solution. Let $A(x) = \sum_{n \geq 0} F_n x^n$. Summing $F_n x^n = (F_{n-1} + F_{n-2})x^n$ over $n \geq 2$: the left side is $A(x) - F_0 - F_1 x = A(x) - x$; the right side is $x(A(x) - F_0) + x^2 A(x) = (x + x^2)A(x)$. So $A(x) - x = (x + x^2)A(x)$, giving

$$A(x) = \frac{x}{1 - x - x^2}.$$

Factor the denominator: its reciprocal roots are $\varphi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$ (the roots of $t^2 - t - 1 = 0$), so $1 - x - x^2 = (1 - \varphi x)(1 - \psi x)$. Partial fractions $\frac{x}{(1 - \varphi x)(1 - \psi x)} = \frac{B}{1 - \varphi x} + \frac{C}{1 - \psi x}$ give $B = \frac{1}{\sqrt{5}}$, $C = -\frac{1}{\sqrt{5}}$ (solve $x = B(1 - \psi x) + C(1 - \varphi x)$ at $x = 1/\varphi$ and $x = 1/\psi$). Expanding both geometric series,

$$F_n = [x^n]A(x) = \frac{1}{\sqrt{5}}(\varphi^n - \psi^n),$$

Binet's formula. We display the first coefficients beside the call tree of the naive routine, whose number of leaves is exactly F_{n+1} :



Which idea, and why? The OGF method handles a multi-term linear recurrence mechanically: the characteristic polynomial $t^2 - t - 1$ appears as the *reversed* denominator $1 - x - x^2$, and its roots' reciprocals are the growth bases. Guessing Binet's formula from the integers 0, 1, 1, 2, 3, 5 would be hopeless; the algebra delivers it.

Check. $\varphi\psi = -1$ and $\varphi + \psi = 1$, so $\varphi^2 = \varphi + 1$; then $F_2 = \frac{1}{\sqrt{5}}(\varphi^2 - \psi^2) = \frac{1}{\sqrt{5}}(\varphi - \psi) = 1$. The ratio $F_{n+1}/F_n \rightarrow \varphi \approx 1.618$, the exponential blow-up that forces memoization.

The Catalan thread: a quadratic generating function

Linear recurrences gave rational OGFs. The moment a recurrence *convolves* the sequence with itself – which is what happens whenever a structure is built from an ordered pair of smaller copies of itself – the OGF becomes *algebraic*. The canonical case is the Catalan numbers, and they sit at the heart of parsing and recursion.

Definition 16.3.6 (Catalan numbers). The Catalan numbers $C_0, C_1, C_2, \dots$ are defined by $C_0 = 1$ and the convolution recurrence

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i} \quad (n \geq 1).$$

The recurrence is the algebra of *splitting at the root*: a nonempty binary tree is a root with a left subtree of some size i and a right subtree of size $n - 1 - i$, and summing over the split point i gives the recurrence. The same split counts balanced parenthesizations, triangulations of a polygon, and – the computational payoff – *distinct parse trees*, so C_n measures the inherent branching a recursive-descent or shift-reduce parser must confront.

Theorem 16.3.7 (Catalan generating function and closed form). *The OGF $C(x) = \sum_{n \geq 0} C_n x^n$ satisfies $C(x) = 1 + x C(x)^2$, and therefore*

$$C(x) = \frac{1 - \sqrt{1 - 4x}}{2x}, \quad C_n = \frac{1}{n+1} \binom{2n}{n}.$$

Proof. Multiply the recurrence by x^n and sum over $n \geq 1$. The left side is $C(x) - C_0 = C(x) - 1$. The right side is $\sum_{n \geq 1} (\sum_{i=0}^{n-1} C_i C_{n-1-i}) x^n = x \sum_{m \geq 0} (\sum_{i=0}^m C_i C_{m-i}) x^m = x C(x)^2$, since the inner sum is exactly the Cauchy product coefficient of $C(x)^2$. Hence $C(x) - 1 = x C(x)^2$, i.e.

$$x C(x)^2 - C(x) + 1 = 0.$$

Solving this quadratic in $C(x)$ by the quadratic formula, $C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}$. We choose the sign that keeps $C(x)$ a genuine power series: by Definition 16.3.3, $\sqrt{1-4x} = (1-4x)^{1/2} = \sum_{n \geq 0} \binom{1/2}{n} (-4x)^n = 1 - 2x - 2x^2 - \dots$ has constant term 1. With the $+$ sign the numerator $1 + \sqrt{1-4x}$ has constant term 2, so dividing by $2x$ would force a $1/x$ term – impossible in $\mathbb{C}[[x]]$. With the $-$ sign the numerator $1 - \sqrt{1-4x}$ has *zero* constant term, so dividing by $2x$ merely lowers each exponent by one and lands back in $\mathbb{C}[[x]]$. Thus $C(x) = \frac{1 - \sqrt{1-4x}}{2x}$. Finally, expanding with Definition 16.3.3 and simplifying $\binom{1/2}{n+1}$ gives the standard identity $C_n = [x^n]C(x) = \frac{1}{n+1} \binom{2n}{n}$. $\square$

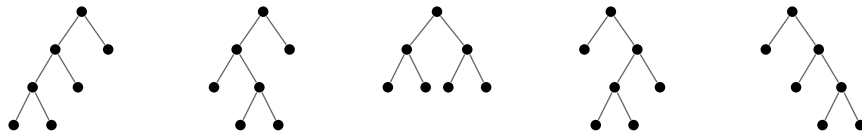
Worked Example 16.3.8 (Verifying C_0 through C_4 two ways). Compute $C_0, \dots, C_4$ from the convolution recurrence and from the closed form $C_n = \frac{1}{n+1} \binom{2n}{n}$, and exhibit the $C_3 = 5$ binary trees.

Recognition. The trigger is “check a closed form against its recurrence”: evaluate both for small n and confirm agreement, then draw the objects the number counts.

Solution. From $C_0 = 1$ and $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$:

$$C_1 = C_0^2 = 1, \quad C_2 = C_0 C_1 + C_1 C_0 = 2, \quad C_3 = C_0 C_2 + C_1^2 + C_2 C_0 = 2 + 1 + 2 = 5,$$

and $C_4 = C_0 C_3 + C_1 C_2 + C_2 C_1 + C_3 C_0 = 5 + 2 + 2 + 5 = 14$. The closed form agrees: $\frac{1}{1} \binom{0}{0} = 1$, $\frac{1}{2} \binom{2}{1} = 1$, $\frac{1}{3} \binom{4}{2} = 2$, $\frac{1}{4} \binom{6}{3} = 5$, $\frac{1}{5} \binom{8}{4} = 14$. The five shapes counted by C_3 – full binary trees with three internal nodes, equivalently the five parse trees of a length-four associative product – are:



Which idea, and why? The convolution recurrence is the generating-function fingerprint of a structure that recursively splits into an ordered pair of smaller copies; its square $C(x)^2$ in $C(x) = 1 + xC(x)^2$ is that split, and the closed form follows from one quadratic.

Check. The sequence 1, 1, 2, 5, 14 matches OEIS A000108; and $C_4 = \frac{1}{5} \binom{8}{4} = \frac{70}{5} = 14$, consistent with the recurrence sum $5 + 2 + 2 + 5$.

EGFs for labelled structures

OGFs count *unlabelled* shapes, where positions are interchangeable. When the objects carry distinct labels $1, 2, \dots, n$ – permutations, labelled trees, set partitions – the natural bookkeeping divides by $n!$, because the Cauchy product of two EGFs distributes the labels: $\hat{A}(x)\hat{B}(x) = \sum_n (\sum_k \binom{n}{k} a_k b_{n-k}) x^n / n!$, and the $\binom{n}{k}$ is exactly “choose which labels go to the first part.”

Proposition 16.3.9 (Two basic EGFs). *The number of permutations of an n -set is $n!$, with EGF $\sum_{n \geq 0} n! x^n / n! = \frac{1}{1-x}$. The Bell number B_n , counting set partitions of an n -set, has EGF $\sum_{n \geq 0} B_n x^n / n! = e^{e^x - 1}$.*

Proof. The first is immediate: $a_n = n!$ gives $\sum_n n! x^n / n! = \sum_n x^n = 1/(1-x)$. For the second, a set partition is an unordered collection of nonempty *blocks*; a single nonempty block on a labelled set has EGF $\sum_{k \geq 1} x^k / k! = e^x - 1$ (one block of each size $k \geq 1$), and the exponential formula – forming an unordered set of labelled components exponentiates the component EGF – gives $\hat{B}(x) = e^{e^x - 1}$. $\square$

Worked Example 16.3.10 (An EGF example: derangements). A *derangement* of $\{1, \dots, n\}$ is a permutation with no fixed point. Let D_n be their number ($D_0 = 1$). Using the labelled split “every permutation chooses a set of fixed points and deranges the rest,” find the EGF of D_n and extract D_n .

Recognition. The objects are *labelled* (the points are distinct), so the trigger “count labelled permutations” points at an EGF, not an OGF; the phrase “choose fixed points, derange the rest” is a labelled product, which multiplies EGFs.

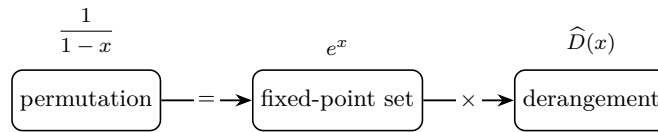
Solution. Every permutation of $[n]$ is determined by choosing its set of fixed points and a derangement of the complement. In EGF terms, the class “all permutations” is the labelled product of “a set of fixed points” (EGF e^x , since the number of ways to have all k points fixed is 1 for each k) and “a derangement” (EGF $\hat{D}(x) = \sum D_n x^n / n!$). The product rule for EGFs gives

$$\frac{1}{1-x} = e^x \cdot \hat{D}(x) \implies \hat{D}(x) = \frac{e^{-x}}{1-x}.$$

Now $e^{-x} = \sum_{k \geq 0} (-1)^k x^k / k!$ and $1/(1-x) = \sum_{j \geq 0} x^j$, so their Cauchy product has $[x^n] \hat{D}(x) = \sum_{k=0}^n (-1)^k / k!$. Multiplying by $n!$,

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}.$$

The labelled split that drives the algebra, and the resulting coefficient table, are:



A coefficient table makes the convolution-by- $\binom{n}{k}$ concrete:

n	0	1	2	3	4	5
$n!$	1	1	2	6	24	120
D_n	1	0	1	2	9	44

Which idea, and why? Labelled structures multiply through their EGFs, so the set identity “permutation = (fixed points) $\sqcup$ (derangement)” becomes the product $\frac{1}{1-x} = e^x \hat{D}(x)$; dividing isolates $\hat{D}(x)$, and one Cauchy product reads off D_n .

Check. $D_2 = 2(1 - 1 + \frac{1}{2}) = 1$ (only 21), $D_3 = 6(1 - 1 + \frac{1}{2} - \frac{1}{6}) = 2$ (231, 312). And $D_n/n! \rightarrow e^{-1} \approx 0.368$, the famous “one in e ” fraction of fixed-point-free shuffles.

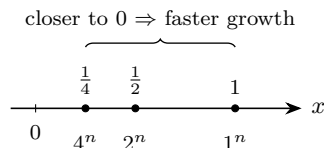
Coefficient asymptotics and algorithm cost

The reason the method matters for computation is that the *location and order of the singularity* of an OGF control the growth of its coefficients, and that growth is the cost. A simple pole at $x = 1/r$ gives $a_n \sim c r^n$ (geometric growth, base r); the algebraic singularity $\sqrt{1-4x}$ of the Catalan OGF, at $x = 1/4$, gives $C_n \sim 4^n / (n^{3/2} \sqrt{\pi})$ – the 4^n explains why exhaustively enumerating parse trees is hopeless, while the $n^{-3/2}$ polynomial correction is invisible to a crude exponential bound but real.

Worked Example 16.3.11 (Reading cost off the dominant singularity). For each OGF below, name the dominant singularity and the resulting growth rate of a_n : (a) $\frac{1}{1-2x}$, (b) $\frac{x}{(1-x)(1-2x)}$, (c) $\frac{1-\sqrt{1-4x}}{2x}$.

Recognition. The trigger is “estimate a_n without a closed form.” The singularity nearest the origin dominates; a pole at $x = 1/r$ contributes r^n (times a polynomial set by its multiplicity), a $\sqrt{}$ branch point at $x = 1/r$ contributes $r^n n^{-3/2}$.

Solution. (a) The only singularity is the pole at $x = \frac{1}{2}$, so $a_n = 2^n$ exactly – pure geometric growth. (b) Poles at $x = 1$ and $x = \frac{1}{2}$; the one nearer 0 is $x = \frac{1}{2}$, so $a_n \sim 2^n$ (here exactly $2^n - 1$, as in the $T(n)$ example). (c) A square-root branch point at $x = \frac{1}{4}$, so $a_n = C_n \sim 4^n n^{-3/2} / \sqrt{\pi}$. The reciprocal of the dominant singularity is the exponential base of the cost:



Which idea, and why? Coefficient growth is governed by the dominant (nearest-the-origin) singularity: its reciprocal is the exponential base, its type (pole vs. branch point) sets the polynomial factor. This is why a generating function predicts runtime without ever solving for a_n .

Check. For (b), $x = \frac{1}{2}$ sits closer to 0 than $x = 1$, and indeed $2^n - 1$ is governed by 2^n , not 1^n – the farther pole only shifts the constant.

Caution. OGF or EGF – match the structure, not your mood. Use an OGF for *unlabelled* objects, where the n slots are interchangeable (parenthesizations, integer compositions, binary-tree shapes); use an EGF for *labelled* objects, where the n items are distinct (permutations, set partitions, labelled graphs). Counting set partitions with an OGF, or binary-tree shapes with an EGF, multiplies in the wrong $\binom{n}{k}$ factors and gives the wrong sequence. The tell: if relabelling the items produces a genuinely different object, you are in the labelled world and want the EGF.

Caution. These are formal series; do not “plug in” x . Writing $\sum_n x^n = 1/(1-x)$ is an identity in $\mathbb{C}[[x]]$ proved by multiplying by $1-x$, *not* a claim about a numeric sum that needs $|x| < 1$. The manipulations – inverting $1-2x$, taking $\sqrt{1-4x}$, dividing the Catalan numerator by x – are all legal precisely because convergence is irrelevant; the only real constraint is the one from Proposition 16.3.2, that you may invert a series exactly when its constant term is nonzero (and may divide by x^k exactly when the first k coefficients vanish, as the Catalan numerator’s zero constant term allowed).

Exercises.

- Let $a_0 = 3$ and $a_n = 5a_{n-1}$ for $n \geq 1$. Following the four-step method, form the OGF $A(x) = \sum_{n \geq 0} a_n x^n$, solve for $A(x)$ as a rational function, and extract a_n by a geometric expansion. State the dominant singularity of $A(x)$ and read off the growth rate.
- A divide-and-conquer routine has step count $T(0) = 0$ and $T(n) = 2T(n-1) + 3$ for $n \geq 1$. Derive $A(x) = \sum_{n \geq 0} T(n)x^n$ in closed form, split it by partial fractions over the factors $(1-x)$ and $(1-2x)$, and conclude an exact closed form for $T(n)$. Verify it against $T(1), T(2), T(3)$ computed directly.
- Consider $a_0 = 1$, $a_1 = 5$, and $a_n = 5a_{n-1} - 6a_{n-2}$ for $n \geq 2$. Show that the OGF is $A(x) = \frac{1}{1-5x+6x^2}$, factor the denominator as $(1-2x)(1-3x)$, and use partial fractions to prove $a_n = A \cdot 2^n + B \cdot 3^n$ for constants you determine. Which term dominates the growth, and why?
- The denominator of an OGF need not have distinct roots. For $a_0 = 1, a_1 = -4$ and $a_n = 4a_{n-1} - 4a_{n-2}$ ($n \geq 2$), show $A(x) = \frac{1-8x}{(1-2x)^2}$, and using the expansion $\frac{1}{(1-2x)^2} = \sum_{n \geq 0} (n+1)2^n x^n$ derive a closed form $a_n = (\alpha + \beta n)2^n$. (A repeated root produces a *polynomial* coefficient in front of r^n .)
- Starting only from $C(x) = 1 + xC(x)^2$, prove the convolution recurrence $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$ by comparing the coefficient of x^n on both sides for $n \geq 1$. Then read off C_0 from the coefficient of x^0 .
- Draw all $C_3 = 5$ ways to fully parenthesize the product $w \cdot x \cdot y \cdot z$ of four symbols, and pair each parenthesization with the corresponding binary parse tree. Then draw the $C_4 = 14$ triangulations of a convex hexagon for one of the four “fans” to convince yourself the count is 14 (you need not draw all 14; explain the bijection to binary trees instead).
- Let b_n count binary strings of length n over $\{0, 1\}$ that contain no two consecutive 1s. Show $b_n = b_{n-1} + b_{n-2}$ with $b_0 = 1, b_1 = 2$, derive the OGF $B(x) = \frac{1+x}{1-x-x^2}$, and explain in one sentence why $b_n = F_{n+2}$. (Tie this back to the regular language $L = \{0, 1\}^*$ minus those containing the substring 11 and sketch a small DFA recognizing L .)

8. Using the EGF $\frac{1}{1-x}$ for permutations and the EGF $e^x - 1$ for a single nonempty block, explain why the EGF of set partitions is e^{e^x-1} . Then compute the Bell numbers $B_0, \dots, B_4$ from the recurrence $B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$ and check $B_4 = 15$.
9. Let f_n count the permutations of $[n]$ all of whose cycles have length at least 2 (equivalently, permutations with no fixed point). Argue $f_n = D_n$, the derangement number, and using the EGF $\widehat{D}(x) = \frac{e^{-x}}{1-x}$ derived in this section, extract the formula $D_n = n! \sum_{k=0}^n (-1)^k / k!$ by a Cauchy product. Verify $D_4 = 9$ by listing the nine derangements of $\{1, 2, 3, 4\}$ in two-line notation.
10. Find the bug. A student writes: “The Catalan recurrence is linear, so its OGF is rational; hence C_n grows like a constant times r^n for a single base r .” Identify *two* false claims in this sentence (one about the form of the recurrence, one about the shape of the asymptotics), and give the correct growth $C_n \sim 4^n n^{-3/2} / \sqrt{\pi}$ with a one-line justification from the singularity at $x = \frac{1}{4}$.
11. Find the bug. To solve $a_0 = 1$, $a_n = a_{n-1} + 1$, a student multiplies by x^n , sums over *all* $n \geq 0$ (not $n \geq 1$), and writes $A(x) = xA(x) + \frac{1}{1-x}$, then solves to get $A(x) = \frac{1}{(1-x)^2}$ and concludes $a_n = n + 1$. The final answer happens to be correct, but one step is illegitimate. Identify which summation index is wrong and why, redo the derivation summing over $n \geq 1$ with the boundary term a_0 handled separately, and confirm you again reach $A(x) = \frac{1}{(1-x)^2}$.
12. Determine, for each sequence, whether you would model it with an OGF or an EGF, and give the generating function in closed form: (a) the number of compositions of n into positive parts; (b) the number of ways to seat n distinct people in a row; (c) the number of unlabelled full binary trees with n internal nodes; (d) the number of functions from $[n]$ to $[3]$. For each, justify the labelled-vs-unlabelled choice in one phrase.

16.4 Expander and Cayley Graphs

A graph can be sparse – only a constant number of edges at each vertex – and yet so richly interconnected that no small set of vertices is bottlenecked off from the rest. Such *expanders* are the workhorses of derandomization: a random walk on one mixes to near-uniform in a logarithmic number of steps, which is exactly what a randomized algorithm needs to recycle a few random bits into many. This section measures interconnection two ways – combinatorially, through the *isoperimetric ratio*, and algebraically, through the second eigenvalue λ_2 of the adjacency matrix – and states the Cheeger-type inequality that ties the two together. The cleanest sparse graphs with provable expansion come from algebra: *Cayley graphs* $\text{Cay}(G, S)$ built from a finite group G and a symmetric generating set S . We draw a few, read their degree and connectivity straight off the group, and close on why $\max_{i \geq 2} |\lambda_i|$ controls how fast a walk forgets where it started – the engine behind error reduction and Reingold’s $L = \text{SL}$.

Expansion: the combinatorial side

Throughout, $G = (V, E)$ is a finite d -regular graph: every vertex has exactly d neighbors. We allow multi-edges and self-loops where convenient (each self-loop contributing 1 to the degree).

Definition 16.4.1 (Edge boundary and edge expansion). For $S \subseteq V$ let ∂S be the set of edges with exactly one endpoint in S (the *edge boundary*). The *edge expansion* (or isoperimetric ratio, Cheeger constant) of G is

$$h(G) = \min_{\substack{S \subseteq V \\ 0 < |S| \leq |V|/2}} \frac{|\partial S|}{|S|}.$$

The restriction $|S| \leq |V|/2$ rules out the trivial choice $S = V$ (whose boundary is empty) and pins the bottleneck to the *smaller* side of any cut.

Read $h(G)$ as the worst case of “edges leaving per vertex inside.” Large h means *every* set of at most half the vertices leaks many edges to the outside: there is no sparse cut, no bottleneck. There is a vertex-flavored cousin.

Definition 16.4.2 (Vertex expansion). The *vertex boundary* $N(S) \setminus S$ is the set of vertices outside S with a neighbor in S . The *vertex expansion* is

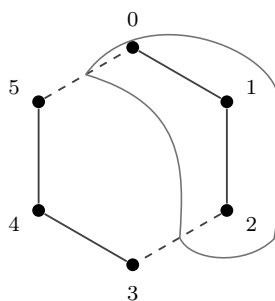
$$h_V(G) = \min_{0 < |S| \leq |V|/2} \frac{|N(S) \setminus S|}{|S|}.$$

In a d -regular graph the two are comparable: every boundary edge lands on a boundary vertex and each boundary vertex absorbs at most d boundary edges, so $h(G)/d \leq h_V(G) \leq h(G)$. We work mostly with the edge version, which the spectrum controls.

Worked Example 16.4.3 (Edge expansion of the 6-cycle versus the complete graph). Compute $h(C_6)$ for the cycle C_6 and $h(K_4)$ for the complete graph K_4 , and say which is the better expander.

Recognition. The trigger is “edge expansion of a small graph”: minimize $|\partial S|/|S|$ over sets of size up to half. On a cycle the cheapest cut is a contiguous arc; on a clique every cut is expensive.

Solution. For C_6 (vertices $0, \dots, 5$ in a ring) take the arc $S = \{0, 1, 2\}$, the largest allowed set. A contiguous arc has exactly 2 boundary edges (one at each end), so $|\partial S|/|S| = 2/3$. No allowed set does better, hence $h(C_6) = 2/3$.



arc $S = \{0, 1, 2\}$, ∂S dashed

For K_4 we minimize over the two allowed shapes. A single vertex $S = \{v\}$ gives $|\partial S| = 3$ and ratio $3/1 = 3$; a balanced cut $|S| = 2$ gives $|\partial S| = 4$ and ratio $4/2 = 2$. The minimum is the smaller, so $h(K_4) = 2$. The clique’s $h = 2$ on 4 vertices dwarfs the cycle’s $2/3$.

Which idea, and why? Edge expansion is a worst-case ratio over sparse cuts. A cycle has a cheap length-2 cut no matter how long it is, so its expansion shrinks toward 0; a clique has no cheap cut, so its expansion is large – but it is dense, with degree $n - 1$, not a constant.

Check. C_6 has the cheapest cut a human would draw – snip the ring in two places. K_4 forces you to sever half of all edges to split it; that is the signature of an expander, paid for here by high degree.

The spectral gap

The adjacency matrix $A = A(G)$ is the $V \times V$ matrix with $A_{uv} = 1$ when $uv \in E$ (counting multiplicities), 0 otherwise. It is symmetric, so it has $n = |V|$ real eigenvalues, which we list in decreasing order

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n.$$

Proposition 16.4.4 (Top of the spectrum of a d -regular graph). *If G is d -regular then $\lambda_1 = d$, with the all-ones vector $\mathbf{1}$ as eigenvector. Moreover $|\lambda_i| \leq d$ for all i , and G is connected iff $\lambda_2 < d$ (strict).*

Proof. Each row of A sums to d (the degree), so $A\mathbf{1} = d\mathbf{1}$ and d is an eigenvalue with eigenvector $\mathbf{1}$. For any eigenvector x with eigenvalue λ , pick a coordinate v of largest absolute value $|x_v|$. Then $|\lambda| |x_v| =$

$|(Ax)_v| = |\sum_{u \sim v} x_u| \leq \sum_{u \sim v} |x_u| \leq d |x_v|$, so $|\lambda| \leq d$; thus $\lambda_1 = d$. If G is disconnected, the indicator of one component is an eigenvector of eigenvalue d independent of $\mathbf{1}$, so $\lambda_2 = d$. Conversely if G is connected the above inequality is tight only when all neighbors of v share x_v ; propagating connectivity forces x constant, so the eigenspace of d is one-dimensional and $\lambda_2 < d$. $\square$

Definition 16.4.5 (Spectral gap). The *spectral gap* of a connected d -regular graph is $d - \lambda_2 > 0$. A large gap (λ_2 well below d) is the spectral signature of an expander.

The combinatorial bottleneck $h(G)$ and the spectral gap measure the same thing from two sides, and a Cheeger-type inequality makes the link quantitative. We state it; the proof is a standard but lengthy variational argument.

Theorem 16.4.6 (Discrete Cheeger inequality, stated). *For a connected d -regular graph G with second eigenvalue λ_2 ,*

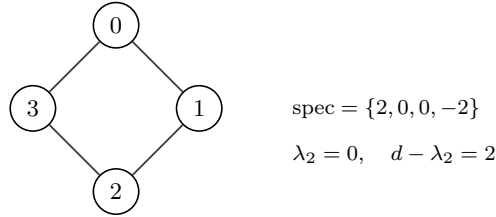
$$\frac{d - \lambda_2}{2} \leq h(G) \leq \sqrt{2d(d - \lambda_2)}.$$

The moral: $h(G)$ is bounded *below* by half the gap and *above* by (a constant times) its square root, so $h(G)$ is large *iff* the gap $d - \lambda_2$ is large. Expansion and the spectral gap are equivalent up to this quadratic loss; one can estimate the elusive combinatorial h by computing the single number λ_2 .

Worked Example 16.4.7 (Spectrum and gap of the 4-cycle). Compute the adjacency spectrum of C_4 , read off λ_2 and the gap, and reconcile with its expansion.

Recognition. “Spectrum of a cycle”: C_n is a circulant graph, so its eigenvalues are $2 \cos(2\pi k/n)$ for $k = 0, \dots, n-1$, with $\lambda_1 = 2$ at $k = 0$.

Solution. Here $d = 2$ and $n = 4$. The eigenvalues are $2 \cos(2\pi k/4) = 2, 0, -2, 0$ for $k = 0, 1, 2, 3$. Sorted: $\lambda_1 = 2, \lambda_2 = 0, \lambda_3 = 0, \lambda_4 = -2$. So the gap is $d - \lambda_2 = 2 - 0 = 2$.



Cheeger predicts $\frac{d - \lambda_2}{2} = 1 \leq h(C_4) \leq \sqrt{2 \cdot 2 \cdot 2} = 2\sqrt{2}$. The true value $h(C_4) = 2/2 = 1$ (cut $S = \{0, 1\}$: two boundary edges over two vertices) sits exactly at the lower bound.

Which idea, and why? For circulant graphs the eigenvalues are a clean trig formula, so λ_2 and the gap drop out by arithmetic; Cheeger then brackets the combinatorial h without searching over cuts.

Check. $\lambda_1 = 2 = d$ as Proposition 16.4.4 demands, and $\lambda_2 = 0 < 2$ confirms C_4 is connected.

Definition 16.4.8 (Expander family). A sequence $\{G_n\}_{n \rightarrow \infty}$ of d -regular graphs with $|V(G_n)| \rightarrow \infty$ is an *expander family* (with parameters d, ϵ) if the degree d is a fixed constant and there is a fixed $\epsilon > 0$ with $h(G_n) \geq \epsilon$ for all n . Equivalently, by Cheeger, the spectral gaps are bounded below: $d - \lambda_2(G_n) \geq \delta > 0$ for all n .

Both conditions – *constant* degree and expansion *bounded below* as $n \rightarrow \infty$ – are essential, and both are what make expanders a miracle: a graph stays sparse (constant edges per vertex) yet refuses to develop any sparse cut however large it grows. That such families exist is nontrivial; the cleanest constructions are algebraic.

Cayley graphs

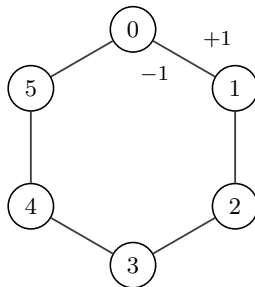
Definition 16.4.9 (Cayley graph). Let G be a finite group and $S \subseteq G$ a generating set that is *symmetric* ($s \in S \Rightarrow s^{-1} \in S$) and does not contain the identity. The *Cayley graph* $\text{Cay}(G, S)$ has vertex set G , with an edge $\{g, gs\}$ for every $g \in G$ and $s \in S$. It is $|S|$ -regular and connected (since S generates G). Symmetry of S is what makes the graph undirected: the edge from g via s is the edge from gs via s^{-1} .

The group's multiplication *is* the graph: walking an edge labelled s is right-multiplication by s , and a walk spells a word in the generators. Two staple examples turn familiar algebra into familiar graphs.

Worked Example 16.4.10 (The cycle as a Cayley graph: $\text{Cay}(\mathbb{Z}_6, \{\pm 1\})$). Build $\text{Cay}(\mathbb{Z}_6, S)$ with $S = \{+1, -1\}$ (here $-1 = 5$ in $\mathbb{Z}_6$), and read off its degree and connectivity.

Recognition. “Cayley graph of $\mathbb{Z}_n$ with ± 1 ”: the generators add and subtract 1, so each vertex g joins $g + 1$ and $g - 1$ – the ring structure of a cycle.

Solution. $G = \mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$ under addition mod 6. The set $S = \{1, 5\}$ is symmetric ($5 = -1 = 1^{-1}$) and generates $\mathbb{Z}_6$, and $|S| = 2$, so the graph is 2-regular. Vertex g has neighbors $g + 1$ and $g + 5 = g - 1$, which lays the six elements out in a ring:



This is precisely the 6-cycle C_6 . Connectivity is automatic: 1 alone generates $\mathbb{Z}_6$, so repeated $+1$ steps reach every vertex.

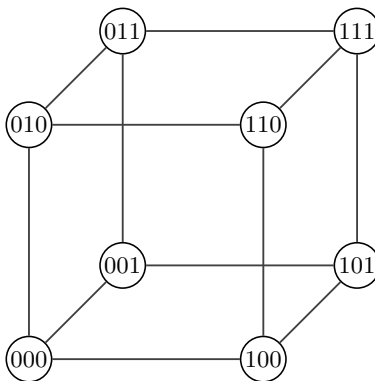
Which idea, and why? $\text{Cay}(\mathbb{Z}_n, \{\pm 1\}) = C_n$ because the single generator 1 has order n ; its powers trace out one cycle through all of G . Degree = $|S| = 2$.

Check. Walking $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 0$ spells the word $(+1)^6 = 0$ in $\mathbb{Z}_6$, a closed walk of length 6 – the cycle closes exactly when the generator's order is reached.

Worked Example 16.4.11 (The hypercube as a Cayley graph: $\text{Cay}(\mathbb{Z}_2^3, \{e_1, e_2, e_3\})$). Build the Cayley graph of $\mathbb{Z}_2^3$ with the standard basis vectors as generators, and identify it.

Recognition. “Cayley graph of $\mathbb{Z}_2^n$ with the basis”: each generator e_i flips coordinate i (since $-e_i = e_i$ in characteristic 2), so neighbors differ in exactly one bit – the Hamming structure of a cube.

Solution. $G = \mathbb{Z}_2^3 = \{000, 001, \dots, 111\}$ under coordinatewise XOR. The generators $e_1 = 100$, $e_2 = 010$, $e_3 = 001$ are each their own inverse, so S is symmetric with $|S| = 3$: the graph is 3-regular. Adding e_i to g flips bit i , so g is joined to the three strings at Hamming distance 1. That is the 3-cube Q_3 :



Every vertex shows 3 edges, one per flipped bit; the 8 vertices and $8 \cdot 3/2 = 12$ edges are exactly Q_3 .

Which idea, and why? $\text{Cay}(\mathbb{Z}_2^n, \{e_1, \dots, e_n\}) = Q_n$, the n -cube: generator e_i is the “flip bit i ” move, so adjacency is Hamming distance 1. Degree = $|S| = n$.

Check. 000 neighbors 100, 010, 001 – the three weight-one strings – and the opposite corner 111 is three flips away, the diameter of Q_3 .

The cube and the cycle are *not* expanders: Q_n has degree n growing with the graph (not constant), and C_n has expansion $h(C_n) \rightarrow 0$. They are warm-ups. The theorem that powers derandomization is that the *right* generators on the right groups do give bounded-degree expansion.

Theorem 16.4.12 (Algebraic expanders exist, stated). *There are explicit constant-size symmetric generating sets making families of Cayley graphs into expanders. For instance, on $G_p = \text{SL}_2(\mathbb{Z}_p)$ (invertible 2×2 matrices of determinant 1 over $\mathbb{Z}_p$, as p ranges over primes) the two matrices*

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \quad \text{and their inverses}$$

generate, and $\{\text{Cay}(G_p, S)\}_p$ is a family of constant-degree expanders. More elementary still, choosing $O(\log |G|)$ random generators in any group makes $\text{Cay}(G, S)$ an expander with high probability.

Method (How to form $\text{Cay}(G, S)$ and read off its degree and connectivity).

1. **Name the group G and its operation.** List the elements; these are the vertices. (E.g. $\mathbb{Z}_n$ under $+$, or $\mathbb{Z}_2^n$ under XOR.)
2. **Symmetrize the generators.** Take your intended generators and throw in every inverse, so $S = S^{-1}$; discard the identity. This guarantees an undirected graph.
3. **Read the degree.** It is $|S|$ exactly – one edge per generator at each vertex. (Self-inverse generators like e_i in $\mathbb{Z}_2^n$ still count once.)
4. **Draw edges by right multiplication.** Join g to gs for each $s \in S$. Equivalently, label each edge by the generator that traverses it.
5. **Check connectivity.** The graph is connected iff S generates G ; if you can write every element as a word in S , you are done.

Why expanders matter for computation

The payoff is that a random walk on an expander mixes fast. Run a walk by, from the current vertex, stepping to a uniformly random neighbor; after t steps the distribution is A^t/d^t applied to the start. Decomposing the start distribution along the eigenbasis, the uniform component (eigenvalue d) is preserved while every other component shrinks by a factor $|\lambda_i|/d \leq \lambda/d$ per step, where $\lambda = \max_i |\lambda_i|$ over $i \geq 2$. So the distance to uniform decays like $(\lambda/d)^t$: a constant gap $d - \lambda$ forces mixing in $O(\log n)$ steps.

Theorem 16.4.13 (Expander mixing lemma, stated). *Let G be d -regular on n vertices with $\lambda = \max(|\lambda_2|, |\lambda_n|)$. For all $S, T \subseteq V$,*

$$\left| e(S, T) - \frac{d |S| |T|}{n} \right| \leq \lambda \sqrt{|S| |T|},$$

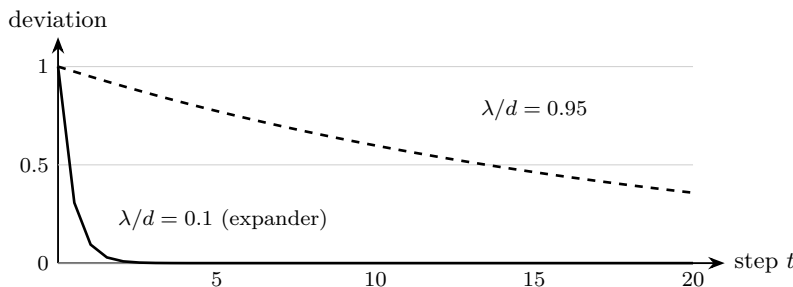
where $e(S, T) = \sum_{u \in S, v \in T} A_{uv}$ counts edges with one endpoint in S and one in T (an edge with both endpoints in $S \cap T$ counted twice).

The lemma says an expander *looks random*: the number of edges between any two sets is within $\lambda \sqrt{|S| |T|}$ of the $d |S| |T| / n$ a random d -regular graph would have. Small λ (large gap) makes the error term small – pseudorandomness made quantitative.

Worked Example 16.4.14 (Random walk mixing, qualitatively, on a high-gap versus low-gap graph). Compare how fast a random walk forgets its start on a graph with $\lambda/d = 0.1$ versus one with $\lambda/d = 0.95$.

Recognition. “Mixing rate from the spectrum”: the non-uniform part of the distribution decays by λ/d each step, so t steps multiply the deviation by $(\lambda/d)^t$. The trigger is “how many steps to mix” = “solve $(\lambda/d)^t \leq \text{target}$.”

Solution. To drive the deviation below ϵ we need $(\lambda/d)^t \leq \epsilon$, i.e. $t \geq \log(1/\epsilon) / \log(d/\lambda)$. With $\lambda/d = 0.1$, one step kills 90% of the deviation, so $t = \lceil \log_{10}(1/\epsilon) \rceil$ steps suffice – about 6 for $\epsilon = 10^{-6}$. With $\lambda/d = 0.95$, each step removes only 5%, and $t \approx \log(10^6) / \log(1/0.95) \approx 269$ steps. A diagram of the two decay curves:



The high-gap (small λ/d) curve plunges to the axis within a handful of steps; the low-gap curve crawls. Only the first is useful for derandomization.

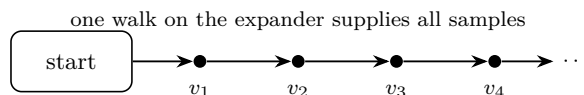
Which idea, and why? Mixing time is governed by the *ratio* λ/d : the deviation from uniform contracts geometrically with that ratio, so a constant gap (ratio bounded below 1) yields logarithmic mixing, independent of the graph's size.

Check. At $\lambda/d = 0.1$, $t = 6$ gives $0.1^6 = 10^{-6}$, matching the target; at 0.95, $0.95^{269} \approx e^{-269 \cdot 0.051} \approx e^{-13.8} \approx 10^{-6}$, matching the slow estimate.

Worked Example 16.4.15 (Error reduction by a walk instead of independent samples). A randomized algorithm errs with probability $\leq 1/4$ using r random bits. Contrast the random bits needed to push error to $2^{-\Omega(k)}$ by (a) independent repetition and (b) an expander walk.

Recognition. “Amplify a one-sided/bounded-error algorithm”: independent trials cost fresh randomness each time; an expander walk reuses one long string of correlated samples that, by the mixing lemma, behave almost independently.

Solution. (a) *Independent repetition.* Running the algorithm k times and taking a majority vote drops the error to $2^{-\Omega(k)}$, but each run draws a *fresh* block of r bits, for $k \cdot r$ random bits total.



(b) *Expander walk.* Build an expander on the 2^r random strings (vertices). Pick one start with r bits, then take $k - 1$ steps; each step needs only $\log d = O(1)$ bits to name a neighbor. The k vertices $v_1, \dots, v_k$ visited are the samples. By the expander Chernoff bound (a consequence of the mixing lemma), the fraction of “bad” samples concentrates just as for independent draws, so the error is $2^{-\Omega(k)}$ – while the randomness used is only $r + O(k)$ bits, versus $k \cdot r$.

Which idea, and why? An expander walk is a randomness-efficient sampler: it spends r bits to enter the graph and $O(1)$ bits per additional sample, yet the mixing lemma makes the correlated samples concentrate like independent ones. That $r + O(k)$ versus kr saving is the heart of derandomization.

Check. For $r = 100$, $k = 50$: independent repetition burns 5000 bits; the walk burns $100 + O(50) \approx 150$ – a 30× saving, with the same exponential error decay.

Remark. The deepest computational payoff is Reingold’s theorem that undirected s – t connectivity is in L (logarithmic space), settling $SL = L$. The proof turns any graph into an expander by an iterated “zig-zag” product, on which a walk of *logarithmically* many steps – trackable with a counter in $O(\log n)$ space – is guaranteed to find a path if one exists. Expansion is exactly the property that converts a space-bounded walk into a connectivity test.

Caution. Expansion is a property of a *family*, not of any one graph. Asking “is this single 8-vertex graph an expander?” is a category error: $h(G) > 0$ holds for *every* connected graph, so the bare inequality says nothing. The content of Definition 16.4.8 is *asymptotic* – a constant degree d and a *uniform* lower bound ϵ on $h(G_n)$ as $n \rightarrow \infty$. A lone graph is at most “a member of a hoped-for family.”

Caution. Combinatorial expansion (h , about cuts and boundaries) and spectral expansion (the gap $d - \lambda_2$) are equivalent only *up to the quadratic Cheeger loss* of Theorem 16.4.6, not equal. A graph can have a moderate combinatorial h but, through the $\sqrt{\cdot}$ upper bound, only a modest gap; do not quote a spectral bound as if

it were the exact isoperimetric ratio. And do not confuse λ_2 (second *largest*) with λ_n (most negative): the mixing lemma needs $\lambda = \max(|\lambda_2|, |\lambda_n|)$, so a graph with λ_n near $-d$ (a near-bipartite graph) mixes badly even if λ_2 is small.

Exercises.

1. Draw $\text{Cay}(\mathbb{Z}_5, \{\pm 1\})$ and $\text{Cay}(\mathbb{Z}_5, \{\pm 2\})$ as labelled graphs. Check that both are 2-regular and connected, and explain why they are isomorphic as graphs even though the generating sets differ. What does this say about “the” Cayley graph of a group depending on S ?
2. Let $G = \mathbb{Z}_2^2 = \{00, 01, 10, 11\}$ under XOR with generators $S = \{10, 01\}$. Draw $\text{Cay}(G, S)$, give its degree, and identify the resulting graph by name. Then add 11 to S and redraw; what is the new degree and what graph results?
3. For the cycle C_8 , compute the edge expansion $h(C_8)$ by exhibiting the cheapest cut, and compute the spectral gap $d - \lambda_2$ from the circulant eigenvalue formula $2\cos(2\pi k/8)$, $k = 0, \dots, 7$. Verify that your numbers obey the Cheeger inequality $\frac{d-\lambda_2}{2} \leq h \leq \sqrt{2d(d-\lambda_2)}$.
4. Let $S = \{v_1, \dots, v_k\}$ be a vertex set in a d -regular graph with edge boundary ∂S and vertex boundary $N(S) \setminus S$. Prove the two inequalities $|N(S) \setminus S| \leq |\partial S|$ and $|\partial S| \leq d |N(S) \setminus S|$, and deduce $h(G)/d \leq h_V(G) \leq h(G)$ for the expansion ratios.
5. The complete bipartite graph $K_{3,3}$ is 3-regular on 6 vertices. Its adjacency spectrum is $\{3, 0, 0, 0, 0, -3\}$. Compute λ_2 , λ_n , and the gap $d - \lambda_2$, then use the expander mixing lemma’s parameter $\lambda = \max(|\lambda_2|, |\lambda_n|)$ to explain why a random walk on $K_{3,3}$ does *not* mix to uniform. (Hint: what is λ_n , and what does a step of the walk do to the bipartition?)
6. A d -regular graph G on n vertices has $\lambda = \max_{i \geq 2} |\lambda_i| \leq d/3$. Using the rule that the deviation of a random walk from uniform shrinks by λ/d per step, estimate the number of steps needed to bring the deviation below 2^{-20} . Express your answer as a function of n and argue it is $O(1)$, hence $O(\log n)$.
7. Take $G = S_3$, the symmetric group on $\{1, 2, 3\}$, with $S = \{(12), (23)\}$ (both transpositions are self-inverse). List the 6 elements, draw $\text{Cay}(S_3, S)$, and give its degree. Is the graph connected? (Justify by showing the two transpositions generate S_3 .)
8. Prove directly from Definition 16.4.9 that $\text{Cay}(G, S)$ is regular of degree $|S|$, being careful about the case where some generator s satisfies $s = s^{-1}$ (an involution). Show that involutions still contribute exactly one edge per vertex, not a double edge.
9. Using the expander mixing lemma, show that in a d -regular graph on n vertices with $\lambda < d/2$, every vertex subset S with $|S| \leq n/2$ has at least $\frac{d}{2}|S|(1 - \frac{2\lambda}{d})$ edges leaving it. (Apply the lemma with $T = S$, where $e(S, S)$ counts each edge inside S twice, bound $e(S, S)$ from above, and use $|\partial S| = d|S| - e(S, S)$.) Conclude a lower bound on $h(G)$ in terms of d and λ .
10. Find the bug. A student writes: “The single graph $\text{Cay}(\mathbb{Z}_2^{10}, \{e_1, \dots, e_{10}\})$, the 10-cube, is an expander because every connected graph has $h(G) > 0$.” Identify *two* distinct reasons this argument fails (one about what “expander” means, one about the degree of the cube), citing the relevant definition.
11. Find the bug. A claim reads: “Since the Cheeger inequality gives $h(G) \geq (d - \lambda_2)/2$, a graph with spectral gap $d - \lambda_2 = 1.8$ has edge expansion exactly 0.9.” Explain precisely what is wrong, and state the correct conclusion the inequality supports.
12. Let G be a finite group and S a symmetric generating set. Prove that $\text{Cay}(G, S)$ is *vertex-transitive*: for any two vertices g, h there is a graph automorphism sending g to h . (Hint: consider left multiplication $x \mapsto hg^{-1}x$ and check it preserves edges.) Why does vertex-transitivity make Cayley graphs especially clean expander candidates?

16.5 Synthesis: Where Algebra, Combinatorics, and Graphs Meet Computation

The four preceding sections each pulled one thread – monoids in §16.1, group actions and counting in §16.2, generating functions in §16.3, Cayley and expander graphs in §16.4 – out of the same fabric. This closing section ties the knot. The single idea is that *algebra, combinatorics, and graph theory are three lenses on one phenomenon*: a finite object, finitely described, processed by a machine, and counted. We make the dictionary precise (regular languages $\leftrightarrow$ finite monoids; existence proofs $\leftrightarrow$ counting; reductions $\leftrightarrow$ graph gadgets; derandomization $\leftrightarrow$ expanders; recurrences $\leftrightarrow$ generating functions), draw the bridges in one diagram, then take *one* concrete set of strings and recognize/count it three ways. We close with a probabilistic-method existence proof – a lower bound on the Ramsey number $R(k, k)$ – where pure counting forces a combinatorial object to exist that no construction has ever matched.

The unifying slogan

Definition 16.5.1 (The four pillars). We organize the synthesis around four properties an object can have. *Finite structure*: the object lives in a finite or finitely-generated algebraic frame (a monoid, a group, a graph). *Finite description*: it is named by a bounded gadget (a DFA, a multiplication table, a generating function, an adjacency list). *Computation*: a machine reads the description and decides or transforms. *Counting*: the number of such objects – or of accepting paths, or of colorings – is itself the quantity of interest. The slogan of the chapter is the conjunction

FINITE STRUCTURE + FINITE DESCRIPTION + COMPUTATION + COUNTING.

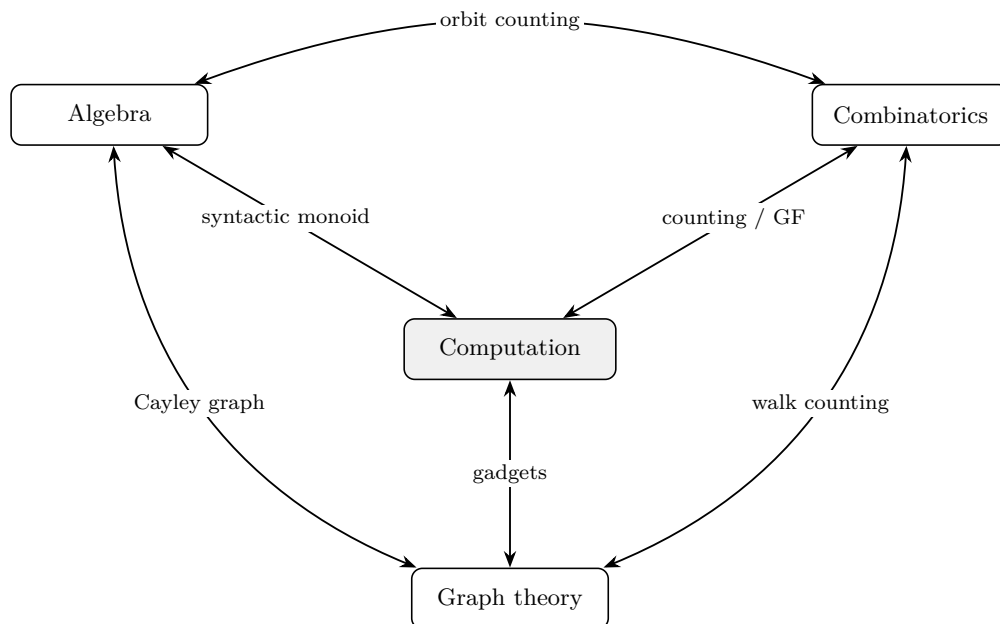
Each pillar is the home turf of one field. Finite structure is *algebra* (Dummit & Foote: monoids, groups, quotients). Counting is *combinatorics* (Sagan: bijections, generating functions; Erdős: the probabilistic method). Finite description-as-a-network is *graph theory* (vertices, edges, gadgets). Computation is the spine that runs through all three. The bridges between the fields are not analogies; they are equalities, and we name them now.

Theorem 16.5.2 (The synthesis dictionary). *The following correspondences hold, each proved in this chapter or in Ch. 1–13 (except the expander facts behind (4), which §16.4 states without proof).*

1. (*Algebra $\leftrightarrow$ computation*) A language $L \subseteq \Sigma^*$ is regular iff some finite monoid recognizes it, namely the syntactic monoid $M(L)$ (Theorem 16.1.12). The DFA is the monoid, re-drawn as a graph on its states.
2. (*Counting $\leftrightarrow$ existence*) If the number of objects with a forbidden property is strictly less than the total number of objects, an object without the property exists (the probabilistic method, §16.5). Counting yields existence with no construction.
3. (*Graphs $\leftrightarrow$ reductions*) An NP problem reduces to a graph problem by a gadget: a fixed finite subgraph encoding one clause or variable, glued into a network whose global structure mirrors the formula (Ch. 13).
4. (*Expanders $\leftrightarrow$ randomness*) A constant-degree expander lets a random walk of length $O(\log n)$ mix to near-uniform, so it stretches few random bits into many near-independent samples (§16.4): the engine of derandomization.
5. (*Generating functions $\leftrightarrow$ analysis*) A counting or cost recurrence $a_n = \sum_k c_k a_{n-k}$ becomes a rational generating function $A(x) = P(x)/Q(x)$, whose pole of smallest modulus fixes the exponential growth rate of a_n (§16.3).

Proof. Items (1)–(5) are results from the cited sections; here we only record that each is a genuine biconditional or transfer, not a loose parallel. (1) is the algebraic Myhill–Nerode theorem, an “iff”. (2) is the contrapositive of a union bound: $\#\{\text{bad}\} < \#\{\text{all}\}$ leaves a good object, since $\#\{\text{good}\} = \#\{\text{all}\} - \#\{\text{bad}\} > 0$. (3) is the definition of a many-one reduction realized in graph form; correctness is the gadget-gluing argument of Ch. 13. (4) is the expander mixing/hitting property. (5) is the exponential growth formula of analytic combinatorics: a rational $A(x)$ has $\limsup_n |[x^n]A(x)|^{1/n} = 1/r$ where r is the smallest modulus of a pole. We assemble, not reprove. $\square$

The next diagram is the chapter in one picture: the three fields as nodes, computation at the center, and the six labelled bridges as edges.



Remark. Read the inner three edges (field-to-center) as “what each field contributes to a computation,” and the outer three (field-to-field) as “how the fields translate into each other.” A Cayley graph turns a *group* into a *graph* (§16.4); orbit counting turns a group *action* into a *count* (§16.2); walk counting turns an automaton’s *graph* into a *count* (Worked Example 16.5.5). The triangle commutes: every path from one corner to another lands on the same computational object.

One object, three lenses

We now demonstrate the slogan on a single concrete language and recognize/count it three distinct ways – as a monoid, as a generating function, and as an automaton/graph. Fix the alphabet $\Sigma = \{a, b\}$ and the language

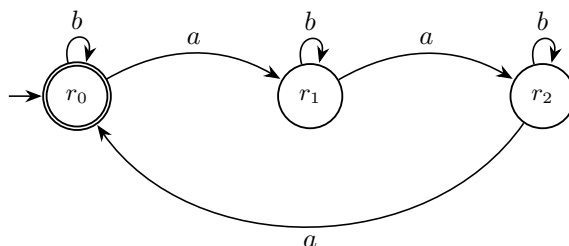
$$L = \{w \in \Sigma^* \mid |w|_a \equiv 0 \pmod{3}\},$$

the strings whose number of a ’s is a multiple of 3 (the b ’s are irrelevant). This one object will be handled algebraically, combinatorially, and graph-theoretically, and the three answers will agree.

Worked Example 16.5.3 (Lens 1 (algebra): the syntactic monoid of L is the cyclic group $\mathbb{Z}/3\mathbb{Z}$). Compute the syntactic monoid of $L = \{w \mid |w|_a \equiv 0 \pmod{3}\}$ and identify it as a group.

Recognition. The trigger “count mod k ” is the algebraic signature of genuine counting; by the §16.1 method, the syntactic monoid of a mod- k language is the cyclic group $\mathbb{Z}/k\mathbb{Z}$. Build the minimal DFA, then read off its transition monoid.

Solution. The minimal DFA tracks $|w|_a \pmod{3}$ in three states; b loops at each, a advances cyclically. Only r_0 accepts.



Write a function on $\{r_0, r_1, r_2\}$ as a triple of images. Then $\rho_b = \text{id} = (r_0, r_1, r_2)$ and $\rho_a = (r_1, r_2, r_0)$ is the 3-cycle. Composition gives $\rho_{a^2} = (r_2, r_0, r_1)$ and $\rho_{a^3} = \text{id} = \rho_\varepsilon$; no new function appears. So

$$M(L) = \{\text{id}, \rho_a, \rho_a^2\}, \quad \rho_a^3 = \text{id},$$

the cyclic group $\mathbb{Z}/3\mathbb{Z}$, with $\eta(w) = \rho_a^{|w|_a \bmod 3}$ and accepting set $S = \{\text{id}\}$. The multiplication table is addition mod 3:

$\cdot$	id	ρ_a	ρ_a^2
id	id	ρ_a	ρ_a^2
ρ_a	ρ_a	ρ_a^2	id
ρ_a^2	ρ_a^2	id	ρ_a

Which idea, and why? For a minimal DFA the transition monoid is the syntactic monoid; a pure mod- k count produces no merging, so every induced function is a bijection and the monoid is the group $\mathbb{Z}/k\mathbb{Z}$ – the algebra detects exactly the counting.

Check. $\eta(\text{aabaa})$: four a 's, $4 \bmod 3 = 1$, so $\eta = \rho_a \notin S$, rejected. $\eta(\text{ababa})$: three a 's, $\rho_a^3 = \text{id} \in S$, accepted. The group element is the residue.

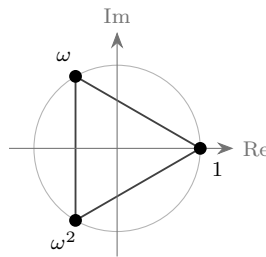
Worked Example 16.5.4 (Lens 2 (combinatorics): a generating function counts the strings of L). Let $\ell_n = |L \cap \Sigma^n|$ be the number of length- n strings in L . Find a closed form for ℓ_n and its ordinary generating function $\mathcal{L}(x) = \sum_{n \geq 0} \ell_n x^n$.

Recognition. “Count the members of a language by length” is the §16.3 trigger: a length- n word is a sequence of n letters, each a or b , so this is a coefficient-extraction. The mod-3 constraint is handled by a root-of-unity filter, the combinatorial twin of the group $\mathbb{Z}/3\mathbb{Z}$ from Lens 1.

Solution. A length- n word has $\binom{n}{j}$ ways to place exactly j letters a (the rest b). So $\ell_n = \sum_{j \equiv 0(3)} \binom{n}{j}$. The root-of-unity filter with $\omega = e^{2\pi i/3}$ extracts the residue-0 part of $(1+t)^n = \sum_j \binom{n}{j} t^j$:

$$\ell_n = \frac{1}{3} \left((1+1)^n + (1+\omega)^n + (1+\omega^2)^n \right) = \frac{1}{3} \left(2^n + 2 \cos \frac{n\pi}{3} \right),$$

using $1+\omega = -\omega^2$, $|1+\omega| = 1$, and $\arg(1+\omega) = \pi/3$. The filter is the geometry of the cube roots: summing $(1+\zeta)^n$ over the three roots $\zeta \in \{1, \omega, \omega^2\}$ and dividing by 3 kills every binomial term whose index is not $\equiv 0 \pmod{3}$.



three cube roots; their orbit under $\times \omega$ is the group $\mathbb{Z}/3\mathbb{Z}$

For the generating function, a string is a sequence of letters and we weight each a by a marker; collapsing to lengths and keeping only $|w|_a \equiv 0$ gives a rational function. Directly: since L is recognized by the 3-state DFA above, transfer-matrix counting (Lens 3 below) yields

$$\mathcal{L}(x) = \sum_{n \geq 0} \ell_n x^n = \frac{(1-x)^2}{(1-2x)(1-x+x^2)},$$

whose poles are $x = \frac{1}{2}$ (giving the dominant 2^n term) and the primitive sixth roots of unity, the roots of $1-x+x^2$ (giving the bounded oscillation $2 \cos(n\pi/3)$). A short table:

n	0	1	2	3	4	5
2^n	1	2	4	8	16	32
ℓ_n	1	1	1	2	5	11

Which idea, and why? Counting a regular language by length is coefficient extraction from a rational generating function whose denominator is read off the DFA's transition structure; the modular constraint is exactly a root-of-unity filter, which is the analytic shadow of the group $\mathbb{Z}/3\mathbb{Z}$ that Lens 1 found.

Check. $n = 3$: the length-3 words with $|w|_a \equiv 0 \pmod{3}$ are those with zero a 's (bbb) or three a 's (aaa), so $\ell_3 = \binom{3}{0} + \binom{3}{3} = 1 + 1 = 2$. The closed form agrees: $\frac{1}{3}(2^3 + 2 \cos \pi) = \frac{1}{3}(8 - 2) = 2$. At $n = 4$, $\frac{1}{3}(16 + 2 \cos \frac{4\pi}{3}) = \frac{1}{3}(16 - 1) = 5$, matching the table.

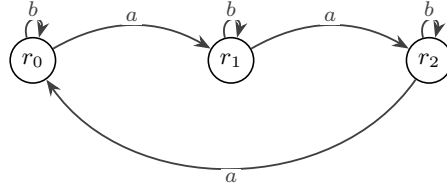
Worked Example 16.5.5 (Lens 3 (graph/automaton): L as a walk-count on a labelled digraph). Re-read the same L as a directed graph and express ℓ_n as a count of walks, recovering the cube-root-of-unity spectrum from the graph's eigenvalues.

Recognition. The trigger is “count accepted strings” = “count length- n walks from start to an accept state.” This is the transfer-matrix method viewed graph-theoretically (§16.4): the automaton is a digraph, and $\ell_n = (A^n)_{\text{start,accept-sum}}$ for the labelled adjacency matrix A .

Solution. Treat the 3-state DFA as a digraph on $\{r_0, r_1, r_2\}$; each state has a b -self-loop and an a -edge to the next residue. The walk-counting (transfer) matrix, with $r_i \rightarrow r_j$ entry = number of letters taking r_i to r_j , is

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} = I + C, \quad C = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \text{ (the 3-cycle),}$$

where the “1” on the diagonal is the b -loop and C is the a -shift. The labelled digraph:



Then ℓ_n is the number of length- n walks from r_0 back to r_0 , i.e. $\ell_n = (A^n)_{00} = e_0^\top A^n e_0$. Since $A = I + C$ and C is the cyclic shift with eigenvalues $1, \omega, \omega^2$, the eigenvalues of A are $2, 1 + \omega, 1 + \omega^2$ – the *same* numbers $\{2, -\omega^2, -\omega\}$ whose n th powers appeared in Lens 2. Spectral decomposition gives back

$$\ell_n = \frac{1}{3}(2^n + (1 + \omega)^n + (1 + \omega^2)^n) = \frac{1}{3}(2^n + 2 \cos \frac{n\pi}{3}),$$

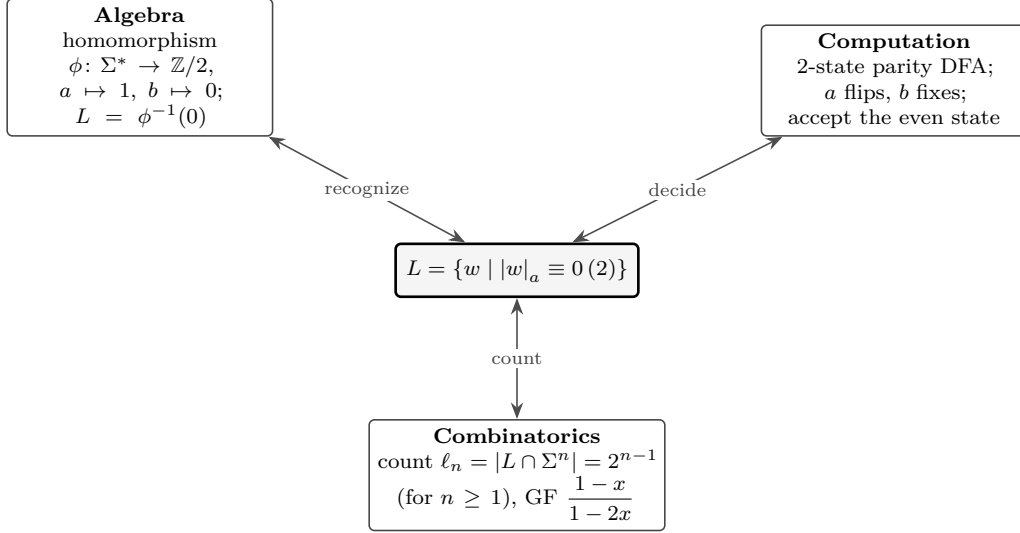
identical to Lens 2's closed form.

Which idea, and why? An automaton is a labelled digraph; counting accepted strings is counting walks, which is powering the adjacency matrix, whose *eigenvalues* are exactly the poles' reciprocals of the generating function and, shifted by 1, the eigenvalues of the group element from Lens 1. The same $\mathbb{Z}/3\mathbb{Z}$ shows up as a cyclic permutation matrix.

Check. A^1 has $(A)_{00} = 1$ (the single b -loop word of length 1 that stays at r_0), matching $\ell_1 = 1$. A^3 : direct multiplication gives $(A^3)_{00} = 2$, matching $\ell_3 = 2$ from Lens 2's count.

The three lenses are not three theories; they are one. The group $\mathbb{Z}/3\mathbb{Z}$ (algebra), the root-of-unity filter and the rational generating function (combinatorics), and the cyclic permutation matrix C with its cube-root spectrum (graph) are the same structure photographed from three angles. The slogan of Definition 16.5.1 is exactly this coincidence: finite structure ($\mathbb{Z}/3\mathbb{Z}$), finite description (a 3-state machine / a rational GF / a 3×3 matrix), computation (run the DFA / extract a coefficient / power the matrix), and counting (ℓ_n).

The whole chapter in miniature: take the one language $L = \{w \mid |w|_a \equiv 0 \pmod{2}\}$ of strings with an even number of a 's, and watch the three fields each name the *same* object. Algebra hands you a homomorphism into $\mathbb{Z}/2$, computation a two-state parity machine, combinatorics a generating function for the counts – one L at the hub, three lenses on the rim.



Check. All three agree on small cases. Length $n = 2$ strings in L are those with an even count of a 's: **bb**, **aa**, so $\ell_2 = 2 = 2^1$, matching 2^{n-1} and the coefficient $[x^2] \frac{1-x}{1-2x}$. The DFA accepts each of **bb**, **aa** (back at the even state) and the homomorphism sends both to $\phi = 0 \in \mathbb{Z}/2$, the accepting class.

Takeaway. One language, three lenses: $L = \{\text{even number of } a\text{'s}\}$ is the homomorphism $\Sigma^* \rightarrow \mathbb{Z}/2$ with $a \mapsto 1, b \mapsto 0$ and accepting class 0 (algebra), the two-state parity DFA (computation), and the generating-function count $\ell_n = 2^{n-1}$ ($n \geq 1$), $\mathcal{L}(x) = (1-x)/(1-2x)$ (combinatorics) – finite structure + finite description + counting, the same regular object photographed three ways. The invariant is L at the hub; the three rim descriptions are interchangeable presentations of it.

Counting forces existence: a Ramsey lower bound

The second face of the synthesis is that *counting alone can prove a combinatorial object exists*, even one no known algorithm can build efficiently. We prove the classical probabilistic lower bound on the diagonal Ramsey number, an existence theorem with no construction.

Definition 16.5.6 (Ramsey number). $R(k, k)$ is the least N such that *every* red/blue coloring of the edges of the complete graph K_N contains a monochromatic clique on k vertices (all $\binom{k}{2}$ edges the same color). Equivalently, every graph on $R(k, k)$ vertices contains a clique *or* an independent set of size k .

Theorem 16.5.7 (Erdős: $R(k, k) > 2^{k/2}$). *For every $k \geq 3$, $R(k, k) > 2^{k/2}$. That is, there exists a 2-coloring of the edges of K_N with $N = \lfloor 2^{k/2} \rfloor$ having no monochromatic K_k .*

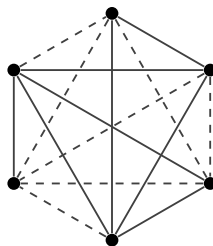
Proof. Color each edge of K_N red or blue independently with probability $\frac{1}{2}$. Fix a set T of k vertices. The probability that T is monochromatic is the probability all $\binom{k}{2}$ of its edges share a color: $2 \cdot 2^{-\binom{k}{2}} = 2^{1-\binom{k}{2}}$. By the union bound over all $\binom{N}{k}$ choices of T ,

$$\Pr[\text{some monochromatic } K_k] \leq \binom{N}{k} 2^{1-\binom{k}{2}}.$$

We show this is < 1 for $N = \lfloor 2^{k/2} \rfloor$. Using $\binom{N}{k} \leq \frac{N^k}{k!}$ and $\binom{k}{2} = \frac{k(k-1)}{2}$,

$$\binom{N}{k} 2^{1-\binom{k}{2}} \leq \frac{N^k}{k!} 2^{1-k(k-1)/2} \leq \frac{2^{k^2/2}}{k!} 2^{1-k^2/2+k/2} = \frac{2^{1+k/2}}{k!}.$$

For $k \geq 3$ the right side is < 1 : at $k = 3$ it is $2^{2.5}/6 \approx 0.94 < 1$, and the factor $k!$ in the denominator grows far faster than $2^{k/2}$ in the numerator, so the bound only shrinks as k increases. Hence $\Pr[\text{some monochromatic } K_k] < 1$, so with positive probability the random coloring has *none*. A coloring with no monochromatic K_k therefore exists, and $N = \lfloor 2^{k/2} \rfloor < R(k, k)$. $\square$



solid = red, dashed = blue

Remark (Why this is a computational impossibility). The proof never exhibits a coloring; it shows the bad colorings are too few to fill the space. This is the second dictionary bridge (Theorem 16.5.2(2)) at full strength: a *count* of failures, compared against a *count* of all objects, forces a good object into existence. Computationally it is a lower bound on a combinatorial *guarantee*: no adversary can force a monochromatic K_k until N exceeds $2^{k/2}$. And it is famously non-constructive – after seventy years no explicit coloring matches the bound, so counting proves the existence of an object that *computation* has not caught up to. The very gap between the random object and the best constructible one is what expanders (§16.4) and derandomization try to close.

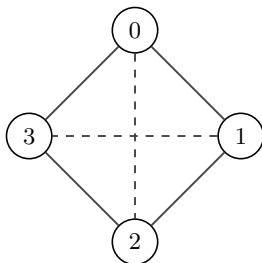
Worked Example 16.5.8 (Reading the Ramsey bound at $k = 4$). Use Theorem 16.5.7 to certify that some 2-coloring of K_4 's edges avoids a monochromatic K_4 , and compare with the elementary upper bound $R(4, 4) \leq \binom{6}{3} = 20$.

Recognition. The trigger is “does an object with no bad substructure exist?”. The probabilistic method (Theorem 16.5.7) answers via the inequality $\binom{N}{k} 2^{1-\binom{k}{2}} < 1$; just plug in.

Solution. With $k = 4$ the lower-bound construction uses $N = \lfloor 2^{4/2} \rfloor = \lfloor 4 \rfloor = 4$. The expected number of monochromatic K_4 's in a random coloring of K_4 is

$$\binom{4}{4} \cdot 2^{1-\binom{4}{2}} = 1 \cdot 2^{1-6} = 2^{-5} = \frac{1}{32} < 1,$$

so the random coloring is monochromatic- K_4 -free with probability at least $1 - \frac{1}{32}$; in particular such a coloring exists, giving $R(4, 4) > 4$. (The true value is $R(4, 4) = 18$; the probabilistic bound is weak here but becomes the best known method as $k \rightarrow \infty$.) The shape of the argument – expected count < 1 implies a zero-count instance exists – is the whole method on a single line. Here is one explicit witness: color the outer 4-cycle red and the two diagonals blue. No color owns all $\binom{4}{2} = 6$ edges, so neither is a monochromatic K_4 .



solid = red (4-cycle), dashed = blue (diagonals)

Which idea, and why? First-moment / union bound: if the expected number of bad substructures is below 1, an instance with zero of them exists, since a nonnegative integer random variable with mean < 1 must take the value 0 somewhere.

Check. Expected count $\frac{1}{32}$ is the mean of a nonnegative integer; if it were always ≥ 1 the mean would be ≥ 1 . Contradiction, so some coloring has count 0.

Method (Choosing the right lens for a finite-structure problem).

1. **Is it about recognizing or transforming a set of strings?** Use the *algebra* lens: build the DFA, pass to the transition/syntactic monoid, and read structure (group = counting, aperiodic = pattern) off the multiplication table (§16.1).

2. **Is it about how many objects there are, or about a runtime recurrence?** Use the *counting* lens: set up a generating function, extract the coefficient, and locate the dominant pole/eigenvalue for the growth rate (§16.3). For symmetry, divide by a group action with Burnside (§16.2).
3. **Is it “does a good object exist?” with no obvious construction?** Use the *probabilistic* lens: put a random object in the space, bound the expected number of defects, and conclude existence when the expectation is < 1 (the first-moment method).
4. **Is it a hardness or reduction question?** Use the *graph* lens: design a gadget per variable/clause, glue into a network, and argue the global property holds iff the formula is satisfiable (Ch. 13).
5. **Need to derandomize a probabilistic algorithm?** Use the *expander* lens: replace independent samples by an expander walk to recycle randomness (§16.4).
6. **Cross-check.** A good answer survives translation: the same growth rate should appear as a reciprocal pole and as an eigenvalue. Disagreement signals an error in one lens.

Caution. These are *complementary views of one phenomenon*, not separate theories. The group $\mathbb{Z}/3\mathbb{Z}$, the rational generating function, and the cyclic permutation matrix in the three lenses above are the same object; do not treat “the algebraic answer” and “the combinatorial answer” as independent results that happen to agree. The number 2 that governs $\ell_n \sim 2^n/3$ is at once the dominant eigenvalue of A , the reciprocal of the dominant pole $x = \frac{1}{2}$, and the size of the alphabet acting on the group $\mathbb{Z}/3\mathbb{Z}$. If the three lenses ever seem to disagree, the resolution is an arithmetic slip in one of them, never a genuine conflict between the fields.

Caution. The probabilistic method proves *existence*, not *construction*. A bound like $R(k, k) > 2^{k/2}$ does not hand you a coloring; it only certifies that one is hiding in the space. Reading a non-constructive existence proof as an algorithm – “so just output the random coloring” – confuses “exists with positive probability” with “computable efficiently and verifiably.” The two are different pillars (Definition 16.5.1): counting guarantees existence, but only computation gives you the object in hand.

Caution. Do not over-read the dictionary’s bridges as *isomorphisms of everything*. A Cayley graph encodes a group, but two non-isomorphic groups can have isomorphic Cayley graphs for bad generator choices; a generating function encodes a count, but loses the bijection that produced it. The bridges of Theorem 16.5.2 transport the *quantity* of interest (regularity, a count, a growth rate, an existence guarantee) faithfully; they do not promise that every feature of one side survives to the other.

Exercises.

1. Fix $\Sigma = \{a, b\}$ and $L = \{w \mid |w|_a \equiv 0 \pmod{4}\}$. Carry L through all three lenses of this section: (a) draw the minimal DFA and identify its syntactic monoid as a known group; (b) write the closed form for $\ell_n = |L \cap \Sigma^n|$ using a root-of-unity filter with a fourth root of unity; (c) write the labelled transfer matrix A , state its eigenvalues, and confirm that the nonzero ones are the reciprocals of the poles of your generating function.
2. For the language $K = \{w \in \{a, b\}^* \mid |w|_b \equiv 1 \pmod{2}\}$ (odd number of b ’s), build the syntactic monoid and decide, using Schützenberger’s criterion from §16.1, whether K is star-free. Then give the rational generating function for $|K \cap \Sigma^n|$ and check the two answers describe the same growth.
3. Use the first-moment method to prove: for n large enough, there is a tournament (a complete graph with each edge oriented one way) on n vertices in which no set of $\lceil 2 \log_2 n \rceil + 1$ vertices is *transitive* (ordered so that each vertex beats every later one). Set up the random orientation, name the bad event, bound its expected count, and conclude existence. State clearly where the bound $\Pr[\text{bad}] < 1$ is used.
4. Draw the synthesis triangle of this section for a *specific* computation: counting the binary necklaces of length 6 up to rotation. Label the three corners (which group acts, which count you extract, which graph/orbit structure appears) and the bridge you cross from “group action” to “count” (name the lemma from §16.2).
5. Let A be the 3×3 transfer matrix of Worked Example 16.5.5 (Lens 3). Compute A^2 and A^3 by hand, read off $(A^2)_{00}$ and $(A^3)_{00}$, and verify they equal ℓ_2 and ℓ_3 from the table. Then explain in one sentence why $(A^n)_{00}$ counts exactly the length- n words of L .

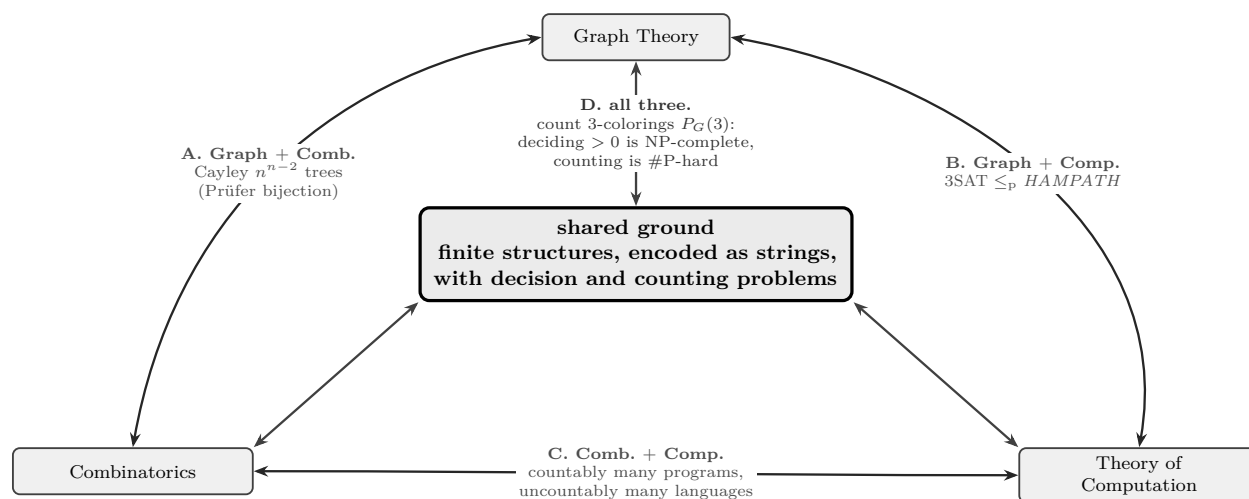
6. Prove the contrapositive form of the probabilistic method used in Theorem 16.5.2(2): if X is a nonnegative integer-valued random variable with $\mathbb{E}[X] < 1$, then $\Pr[X = 0] > 0$. Use only the definition of expectation for a nonnegative integer variable. Then state how this single line powers the Ramsey lower bound.
7. Build a reduction gadget. Draw a small graph gadget that encodes one clause $(x \vee \bar{y} \vee z)$ of a *3SAT* instance for a reduction to *CLIQUE*, marking which vertices represent which literal and stating which vertices may be chosen together. (You may model it on the Ch. 13 clause-gadget pattern; the point is to exhibit the gadget, not to reprove the reduction.)
8. Two languages over $\{a, b\}$ have syntactic monoids $M_1 = \mathbb{Z}/2\mathbb{Z}$ and $M_2 = \{1, c\}$ with $c^2 = c$. For each, decide (i) whether the language is star-free, and (ii) whether the length-counting sequence ℓ_n is determined by the monoid alone. (Compare $\{w \mid |w| \text{ even}\}$ with $\{w \mid |w|_a \text{ even}\}$, which both have syntactic monoid $\mathbb{Z}/2\mathbb{Z}$.) Is the presence of a nontrivial group inside the monoid always visible in the *counting* lens?
9. Find the bug. A student argues: “Theorem 16.5.7 gives a two-coloring of K_N with no monochromatic K_k for $N = \lfloor 2^{k/2} \rfloor$, so the algorithm ‘output a uniformly random coloring’ produces such a coloring.” Identify precisely what is wrong with calling this an algorithm that *produces* the object, and state what the theorem actually guarantees.
10. Find the bug. A classmate claims the generating function for $L = \{w \mid |w|_a \equiv 0 \pmod{3}\}$ over $\{a, b\}$ is $\mathcal{L}(x) = \frac{1}{3(1-2x)}$ “because there are 2^n strings of length n and a third of them have $|w|_a \equiv 0$.” Show by computing ℓ_0 and ℓ_1 from the claimed series that it is wrong, and explain in one sentence why the “exactly a third” reasoning fails for small n (relate it to the oscillating $2\cos(n\pi/3)$ term).
11. State the dictionary of Theorem 16.5.2 as five one-line “ X on the left equals Y on the right” slogans, then for each slogan name the single object from the “one object, three lenses” running example (or from the Ramsey proof) that instantiates it. Which of the five bridges does the Ramsey lower bound exercise?
12. Cayley-graph crossing. Take the group $\mathbb{Z}/3\mathbb{Z}$ that appeared as the syntactic monoid of L . Draw its Cayley graph with generator $\{1\}$ (a directed triangle), and explain how this *same* triangle is the a -edge structure of the DFA for L with the b -self-loops removed. Which two corners of the synthesis triangle does this identification connect?

Appendix A

An Integrated Framework: Graph Theory, Computation, and Combinatorics

A.1 The Integrated Concept Map

This framework connects three fields. The map shows only what the bridge problems of this appendix actually establish: the four field-pairings A–D into which those problems are grouped, each with one example quoted from a specific problem.



Each labelled example is one of the Twenty Bridge Problems below: **A** is Cayley’s formula proved via the Prüfer correspondence; **B** is the reduction $3SAT \leq_p HAMPATH$; **C** is the diagonalization counting argument that most languages are undecidable; **D** is the chromatic-polynomial problem $P_G(3)$, where *deciding* 3-colorability is NP-complete while *counting* colorings is #P-hard. The map asserts nothing those problems do not establish.

A.2 A Ten-Week Integrated Study Plan

This plan runs ten weeks at an upper-division pace, interleaving Graph Theory, Theory of Computability, and Combinatorics so that each subject continually supplies the objects, counting tools, and lower-bound

arguments the others need. Four standing bridges (thematic ones, not the field-pairings A–D of the concept map) recur throughout: **Bridge A** (graphs as binary relations and automata/Turing tableaux as labelled directed graphs), **Bridge B** (counting and generating functions as the bookkeeping for structures, runs, and state spaces), **Bridge C** (pigeonhole/Ramsey/extremal arguments as the engine behind both combinatorial existence proofs and computational lower bounds), and **Bridge D** (reductions and diagonalization as structure-preserving maps between graph problems, languages, and counting problems).

Week 1 – Foundations: sets, relations, induction, and graphs as relations

- *Combinatorics/foundations*: sets, functions, equivalence and order relations, well-ordering, weak/strong induction, structural induction.
- *Graph theory*: a (di)graph as a relation $E \subseteq V \times V$; symmetry = undirected, reflexivity = loops, transitive closure = reachability; adjacency matrix.
- *Computability (preview)*: a deterministic transition function δ is just a relation from (state, symbol) pairs to states — foreshadow Bridge A.
- *Bridge worked*: **Bridge A** introduced — prove reachability = transitive closure of E , and read A^k entries as walk counts.
- *Deliverables*: (1) prove by induction $|2^S| = 2^{|S|}$; (2) show a relation is an equivalence relation iff its graph is a disjoint union of cliques with a loop at every vertex.
- *Why together*: every later object (graph, automaton, tableau) is a relation, so relational induction is the shared substrate.

Week 2 – Counting and pigeonhole: the bookkeeping layer

- *Combinatorics*: sum/product rules, permutations, combinations, binomial theorem, inclusion–exclusion, basic and generalized pigeonhole.
- *Graph theory*: handshake lemma ($\sum \deg = 2|E|$) as a double-count; counting walks via A^k ; degree sequences.
- *Computability (preview)*: pigeonhole as the kernel of the pumping lemma — a long accepting run must repeat a state.
- *Bridge worked*: **Bridge B** (counting) opened and **Bridge C** (pigeonhole) seeded; double-counting links combinatorial identities to degree sums.
- *Deliverables*: (1) prove handshake lemma two ways (double count and induction on edges); (2) inclusion–exclusion count of surjections $[n] \rightarrow [k]$.
- *Why together*: counting is the common currency — edges, subsets, and machine runs are all enumerated by the same rules.

Week 3 – Trees, recurrences, and counting structures

- *Graph theory*: trees, spanning trees, Cayley’s formula n^{n-2} , Prüfer sequences as a bijection, matrix-tree theorem (statement).
- *Combinatorics*: recurrences (Fibonacci, Catalan), bijective proofs, the bijection method via Prüfer codes.
- *Computability (preview)*: parse trees and computation trees of nondeterministic machines as the trees we are counting.
- *Bridge worked*: **Bridge B** deepened — a labelled-tree count is a closed form for a structure that later indexes derivations and runs.

- *Deliverables:* (1) prove Cayley’s formula via Prüfer bijection; (2) set up and solve the Catalan recurrence for binary trees.
- *Why together:* trees are simultaneously graph objects, combinatorial counts, and the shape of parses/computations.

Week 4 – Generating functions and graph enumeration

- *Combinatorics:* ordinary and exponential generating functions, convolution, solving recurrences, compositions and partitions.
- *Graph theory:* counting labelled graphs, connected vs. all graphs via the exponential formula; subgraph and walk counts via series.
- *Computability (preview):* the generating function of a regular language (number of accepted strings of length n) is rational — foreshadow Week 7.
- *Bridge worked:* **Bridge B** at full strength — generating functions unify recurrence-solving across graphs, structures, and (soon) automata.
- *Deliverables:* (1) derive the EGF relating all graphs to connected graphs; (2) use an OGF to count binary strings avoiding 11 (preview of regular counting).
- *Why together:* one analytic tool counts graphs now and language strings later, exposing the regular/rational link early.

Week 5 – Colorings, chromatic structure, and the Ramsey/extremal engine

- *Graph theory:* proper colorings, chromatic number, chromatic polynomial, bipartite = 2-colorable = no odd cycle.
- *Combinatorics:* Ramsey numbers $R(s, t)$, $R(3, 3) = 6$, probabilistic lower bound; extremal flavor via Turán’s theorem.
- *Computability (preview):* k -coloring and clique as decision problems we will prove hard in Week 10.
- *Bridge worked:* **Bridge C** fully developed — pigeonhole generalizes to Ramsey; the same counting bound that proves $R(s, t)$ exists later forces computational structure.
- *Deliverables:* (1) prove $R(3, 3) = 6$; (2) compute a chromatic polynomial and verify it counts proper colorings.
- *Why together:* coloring is a graph property, a counting object (chromatic polynomial), and a future NP-complete problem.

Week 6 – Matchings, Hall’s theorem, and connectivity

- *Graph theory:* bipartite matchings, Hall’s marriage theorem, König’s theorem, augmenting paths, vertex/edge connectivity, Menger’s theorem.
- *Combinatorics:* systems of distinct representatives, Latin-rectangle completion as a matching problem.
- *Computability (preview):* matching is in P via augmenting paths — a tractable contrast to the hard problems of Week 10.
- *Bridge worked:* **Bridge A** reused — alternating/augmenting paths are walks in a derived graph; min–max duality (Hall/König/Menger) previews reduction thinking (Bridge D).
- *Deliverables:* (1) prove Hall’s theorem; (2) reduce an SDR existence question to a bipartite matching and solve it.

- *Why together:* matchings show an efficiently solvable graph problem and a min–max duality that reductions later exploit.

Week 7 – Automata as labelled graphs; regular and context-free languages

- *Computability:* DFA/NFA, subset construction, regular expressions, pumping lemma for regular languages; CFGs, parse trees, pushdown automata, CF pumping lemma.
- *Graph theory:* a DFA is a labelled digraph; reachable/dead states via graph reachability; strongly connected components and language properties.
- *Combinatorics:* counting accepted strings of length n via A^n (transfer matrix) confirms the rational generating function from Week 4.
- *Bridge worked:* **Bridge A** made explicit (automaton = graph) and **Bridge C** reused — pumping lemma = pigeonhole on a long path.
- *Deliverables:* (1) NFA-to-DFA on a concrete example, then minimize; (2) prove $\{0^n 1^n\}$ non-regular by pumping and count strings of a regular language via transfer matrix.
- *Why together:* automata literally are the graphs of Weeks 1–3 with labels, and their string counts are the generating functions of Week 4.

Week 8 – Turing machines, countability, and diagonalization

- *Computability:* Turing machines, configurations and the computation as a tableau, Church–Turing thesis, universal TM, countable vs. uncountable sets.
- *Combinatorics/foundations:* countability arguments, Cantor diagonalization, the counting gap (countably many programs, uncountably many languages).
- *Graph theory:* the TM computation graph (configuration digraph); acceptance = reachability of an accepting configuration.
- *Bridge worked:* **Bridge D** opened — diagonalization is the counting/Bridge-C argument turned into a non-existence (undecidability) proof; **Bridge A** via the configuration graph.
- *Deliverables:* (1) prove the set of languages is uncountable while programs are countable; (2) draw the configuration digraph of a small TM and trace acceptance as reachability.
- *Why together:* the same countability counting that powers combinatorial existence proofs, run in reverse, proves undecidability.

Week 9 – Decidability, reductions, and the halting problem

- *Computability:* decidable vs. recognizable languages, the halting problem via diagonalization, mapping reductions $\leq_m$, Rice’s theorem.
- *Graph theory:* graph decision problems (s – t reachability decidable; emptiness/equivalence of automata as graph questions).
- *Combinatorics:* reductions as structure-preserving maps — the same bijective discipline from Week 3, now between problems.
- *Bridge worked:* **Bridge D** at full strength — a reduction is a map that preserves yes-instances and no-instances, the computational analogue of a bijection.
- *Deliverables:* (1) prove A_{TM} undecidable, then reduce it to show $HALT_{TM}$ undecidable; (2) classify two automaton properties via Rice’s theorem.
- *Why together:* reductions are the unifying morphism — bijections count, reductions transfer (un)solvability, and both preserve structure.

Week 10 – The complexity bridge: NP-completeness, SAT-to-graph, counting complexity (synthesis/capstone prep)

- *Complexity*: P vs. NP, polynomial-time reductions, Cook–Levin (SAT NP-complete), NP-complete graph problems (CLIQUE, INDEPENDENT-SET, VERTEX-COVER, k -COLORING, HAMILTONIAN).
- *Graph theory*: SAT-to-graph reductions ($3\text{-SAT} \leq_p \text{CLIQUE}$, $\leq_p 3\text{-COLOR}$) realized as explicit gadget graphs.
- *Combinatorics*: counting complexity — #P, permanent vs. determinant, #counting perfect matchings as the hard sibling of Week 6.
- *Bridge worked*: all four converge — **A** (gadgets are graphs), **B** (#P counts the structures of Weeks 3–5), **C** (Ramsey-in-computation: unavoidable substructure forcing hardness), **D** (poly-time reductions complete the reduction story).
- *Deliverables*: (1) carry out the $3\text{-SAT} \leq_p \text{CLIQUE}$ reduction on a concrete formula and prove correctness; (2) capstone proposal — pick one NP-complete graph problem and write its reduction, its #P counting version, and its Ramsey/extremal flavor.
- *Why together*: the capstone shows a single problem wearing all three hats — a graph, a count, and a computation — which is the entire point of interleaving the subjects.

A.3 Twenty Bridge Problems

Bridge Problem 1. [Graph + Combinatorics] Prove Cayley’s formula: the number of labeled spanning trees on the vertex set $\{1, 2, \dots, n\}$ is n^{n-2} . Do this by exhibiting the *Prüfer correspondence*, a bijection between labeled trees on n vertices and sequences in $\{1, \dots, n\}^{n-2}$. Carefully describe both the encoding (repeatedly delete the smallest leaf and record its neighbor) and the decoding map, and prove they are mutual inverses. Then explain how Kruskal’s or Prim’s algorithm finds *one* spanning tree of a connected weighted graph in near-linear time, and contrast “counting all spanning trees” (Matrix-Tree theorem, a determinant) with “constructing one” (a greedy algorithm). *Hint: a vertex labeled v appears in the Prüfer sequence exactly $\deg(v) - 1$ times.*

Bridge Problem 2. [Graph + Combinatorics] Let G be a graph and let $P_G(k)$ denote the number of proper k -colorings of G . Using inclusion–exclusion over the edge set, prove that

$$P_G(k) = \sum_{S \subseteq E(G)} (-1)^{|S|} k^{c(S)},$$

where $c(S)$ is the number of connected components of the spanning subgraph $(V(G), S)$. Conclude that P_G is a polynomial in k of degree $|V(G)|$ (the chromatic polynomial). Compute $P_G(k)$ explicitly for the cycle C_4 and for K_4 . Finally, explain the computational-complexity intuition: evaluating P_G at a fixed integer $k \geq 3$ is #P-hard in general, even though the formula above is a finite sum. *Hint: a coloring is proper iff it violates the constraint on $S = \emptyset$ and no nonempty S ; expand $\prod_e (1 - [e \text{ monochromatic}])$.*

Bridge Problem 3. [Graph + Combinatorics] Use the probabilistic method to prove the Ramsey lower bound $R(k, k) > 2^{k/2}$ for all $k \geq 3$: show that if $n \leq 2^{k/2}$ then a uniformly random 2-coloring of the edges of K_n has positive probability of containing no monochromatic K_k , hence such a coloring exists. Carry out the union-bound computation showing $\binom{n}{k} 2^{1 - \binom{k}{2}} < 1$ for $n = \lfloor 2^{k/2} \rfloor$. Then discuss why this argument is *non-constructive*: it proves a good coloring exists without exhibiting one, and no polynomial-time algorithm is known to produce a K_n coloring witnessing this bound. *Hint: there are $\binom{n}{k}$ candidate k -subsets and each is monochromatic with probability $2 \cdot 2^{-\binom{k}{2}}$.*

Bridge Problem 4. [Graph + Combinatorics] The complete bipartite graph $K_{m,n}$ has $m+n$ vertices. By a Prüfer-style / Matrix-Tree argument, prove that the number of labeled spanning trees of $K_{m,n}$ is $m^{n-1} n^{m-1}$. As a warm-up, verify the formula directly for $K_{2,2}$ and $K_{1,n}$ (a star). Then count, by a double-counting

argument, the number of *perfect matchings* of $K_{n,n}$ and explain why this equals $n!$, connecting it to the permanent of the all-ones $n \times n$ matrix. *Hint: apply the Matrix-Tree theorem to the Laplacian of $K_{m,n}$, whose nonzero eigenvalues are m (mult. $n - 1$), n (mult. $m - 1$), and $m + n$.*

Bridge Problem 5. [Graph + Combinatorics] A tournament is an orientation of K_n (every pair of players has a unique winner). Prove by the probabilistic method (or by averaging) that there exists a tournament on n players with at least $n! 2^{-(n-1)}$ Hamiltonian paths, so the number of such paths can be exponentially large. Separately, count the number of distinct labeled tournaments on n vertices and the number that are *transitive* (acyclic). Discuss how “does this tournament have a Hamiltonian path” is trivial (every tournament has one, by induction) whereas in a general digraph the Hamiltonian-path question is hard. *Hint: consider the expected number of Hamiltonian paths over a uniformly random orientation, where each of the $n!$ vertex orderings is consistent with probability $2^{-(n-1)}$.*

Bridge Problem 6. [Graph + Computability] Give a polynomial-time reduction $3SAT \leq_p 3\text{-COLOR}$, where 3-COLOR asks whether a given graph is properly 3-colorable. Describe the standard gadget construction: a “palette triangle” with nodes True/False/Base, a variable gadget for each variable, and an OR-gadget (the 5-node gadget) for each clause. For a formula with n variables and m clauses, count exactly how many vertices and edges your reduced graph has as functions of n and m , and verify the count is polynomial. Prove correctness in both directions (satisfying assignment $\Leftrightarrow$ proper 3-coloring). *Hint: the clause OR-gadget can be properly 3-colored with its output node colored “True” iff at least one input literal is colored “True”.*

Bridge Problem 7. [Graph + Computability] Consider the language

$$\text{CONN} = \{ \langle G \rangle : G \text{ is a connected graph} \}.$$

First prove CONN is decidable and in fact in P by describing a BFS/DFS-based decider and bounding its running time in terms of $|V|$ and $|E|$. Next, prove $\text{CONN} \in \text{L}$ (logarithmic space) follows from Reingold’s theorem $\text{SL} = \text{L}$, and at minimum show $\text{CONN} \in \text{NL}$ by giving a nondeterministic logspace algorithm that guesses a path. Contrast the *decidability* of graph connectivity with the *undecidability* of asking whether the configuration graph of an arbitrary Turing machine on input w ever reaches a halting configuration. *Hint: the nondeterministic logspace machine stores only a current vertex and a step counter, repeatedly guessing the next vertex.*

Bridge Problem 8. [Graph + Computability] Define

$$\text{HAMPATH} = \{ \langle G, s, t \rangle : G \text{ has a Hamiltonian path from } s \text{ to } t \}.$$

(a) For a specific small graph G_0 (the 3×3 king graph, say, with s and t opposite corners) count by hand the number of Hamiltonian s – t paths and exhibit one. (b) Prove $\text{HAMPATH} \in \text{NP}$ by describing the certificate and the polynomial-time verifier. (c) Sketch the reduction $3SAT \leq_p \text{HAMPATH}$ using the “diamond/zig-zag” variable gadgets, and explain why the hand-count in (a) does *not* contradict NP-completeness (small instances are easy; hardness is asymptotic). *Hint: a Hamiltonian-path certificate is a permutation of the vertices; verifying it visits each once and uses only existing edges is linear time.*

Bridge Problem 9. [Graph + Computability] Let $\text{ISO} = \{ \langle G, H \rangle : G \cong H \}$ be the graph isomorphism language. (a) Prove $\text{ISO} \in \text{NP}$ by giving the certificate (a bijection $\phi : V(G) \rightarrow V(H)$) and verifier. (b) Show that ISO is closed under complement-style symmetry and explain why it is not known to be NP-complete, situating it between P and NP-complete. (c) Combinatorial angle: for two graphs on n labeled vertices, how many candidate bijections must a brute-force verifier consider, and how does the orbit-counting (Burnside) viewpoint relate the number of isomorphism classes of n -vertex graphs to $2^{\binom{n}{2}}/n!$ asymptotically? *Hint: by Burnside, the number of isomorphism classes of n -vertex graphs is $\frac{1}{n!} \sum_{\pi \in S_n} 2^{c(\pi)}$ where $c(\pi)$ counts cycles of π acting on pairs.*

Bridge Problem 10. [Graph + Computability] Give a polynomial-time reduction showing $\text{INDEPENDENTSET} \leq_p \text{CLIQUE}$ and back, via the complement graph $\overline{G}$: prove G has an independent set of size k iff $\overline{G}$ has a clique of size k . Count the edges of $\overline{G}$ in terms of those of G to confirm the reduction is polynomial. Then, assuming CLIQUE is NP-complete, deduce that INDEPENDENTSET and

VERTEXCOVER are NP-complete as well (use the identity: S is independent iff $V \setminus S$ is a vertex cover). State each reduction as a mapping reduction and verify it is computable in logspace. *Hint: a single graph transformation $G \mapsto \overline{G}$ serves the whole chain; the parameter k either stays fixed or maps to $n - k$.*

Bridge Problem 11. [Combinatorics + Computability] Prove that undecidable languages over $\{0, 1\}$ exist by a counting argument. (a) Show that the set of all Turing machines (and hence all decidable languages) is countable, by encoding each machine as a finite binary string and listing Σ^* in length-lexicographic order. (b) Show that the set of all languages 2^{Σ^*} is uncountable by a Cantor diagonalization. (c) Conclude that “most” languages are undecidable. Make the cardinality contrast precise: there are $\aleph_0$ deciders but $2^{\aleph_0}$ languages. *Hint: a function $\Sigma^* \rightarrow \{0, 1\}$ is exactly an element of 2^{Σ^*} ; diagonalize against any claimed enumeration $L_1, L_2, \dots$ by flipping the i -th bit on the i -th string.*

Bridge Problem 12. [Combinatorics + Computability] Let N be an NFA without ε -transitions, with state set Q , $|Q| = k$, over alphabet Σ . (a) For a fixed input string w of length n , set up a transfer-matrix (generating-function) method: let A_a be the $k \times k$ Boolean (or integer) transition matrix for symbol a . Show that the number of distinct accepting computation paths of N on $w = w_1 w_2 \cdots w_n$ equals $\mathbf{s}^\top A_{w_1} A_{w_2} \cdots A_{w_n} \mathbf{f}$, where $\mathbf{s}, \mathbf{f}$ are the start/accept indicator vectors over $\mathbb{Z}$. (b) Deduce that counting accepting paths is in polynomial time even though the number of paths can be exponential in n . (c) Use this to count the number of length- n binary strings with no two consecutive 1s, recovering the Fibonacci numbers from a 2-state automaton. *Hint: the entry (i, j) of the product matrix counts paths from state i to state j spelling w ; matrix multiplication does the path-counting bookkeeping.*

Bridge Problem 13. [Combinatorics + Computability] Design a DFA M recognizing $L = \{w \in \{a, b\}^* : |w|_a \equiv 0 \pmod{3} \text{ and } |w|_b \text{ is even}\}$, where $|w|_x$ counts occurrences of x . (a) Give the state diagram and argue it is minimal (Myhill–Nerode: count the equivalence classes of $\equiv_L$). (b) Compute the *syntactic monoid* of L and report its size; describe its elements as transformations of the state set. (c) Combinatorially, count how many of the 2^n strings of length n lie in L , using a transfer-matrix or roots-of-unity filter argument. *Hint: the syntactic monoid is the transition monoid of the minimal DFA, i.e. the set of distinct state-to-state maps induced by input words; for this L it is a quotient of $(\mathbb{Z}/3) \times (\mathbb{Z}/2)$.*

Bridge Problem 14. [Combinatorics + Computability] Recall #SAT asks for the number of satisfying assignments of a Boolean formula. (a) By inclusion–exclusion over the clauses of a CNF formula φ with m clauses and n variables, derive a formula for the number of *falsifying* assignments and hence for $\#\varphi$. (b) Explain why this inclusion–exclusion sum has 2^m terms in the worst case, matching the intuition that #SAT is #P-complete. (c) Contrast with the case of a single read-once / disjoint-clause formula, where the count factors and is computable in polynomial time. State precisely the relationship between $\text{SAT} \in \text{NP}$ and $\#\text{SAT} \in \#\text{P}$. *Hint: let B_i be the set of assignments violating clause C_i (all literals false); then $\#\text{falsifying} = |\bigcup_i B_i|$ by inclusion–exclusion, and each $|B_{i_1} \cap \cdots \cap B_{i_t}|$ is 0 or a power of two.*

Bridge Problem 15. [Combinatorics + Computability] The busy-beaver-flavored counting question: let $T(n)$ be the number of distinct n -state, 2-symbol Turing machines (with the usual finite-control encoding). (a) Give an exact combinatorial formula for $T(n)$ by counting the choices of (write symbol, move direction, next state) for each of the $2n$ (state, symbol) pairs, including halting. (b) Conclude $T(n)$ is finite for each n , hence the busy-beaver value $\Sigma(n)$ is well-defined as a maximum over a finite set. (c) Explain the apparent paradox: Σ is a perfectly well-defined total function $\mathbb{N} \rightarrow \mathbb{N}$ defined by a finite max at each input, yet Σ is *uncomputable* and grows faster than every computable function. Reconcile “finite at each n ” with “uncomputable overall.” *Hint: each (state, symbol) pair has $4n + 1$ possible actions (2 symbols $\times$ 2 directions $\times$ n next-states, plus halt), so $T(n) = (4n + 1)^{2n}$; finiteness per n does not give a uniform halting decider.*

Bridge Problem 16. [Graph + Combinatorics + Computability] Consider counting proper 3-colorings of a graph G , i.e. evaluating the chromatic polynomial $P_G(3)$. (a) Combinatorics: using deletion–contraction $P_G = P_{G-e} + P_{G/e}$, compute $P_G(3)$ for the cycle C_5 and the wheel W_4 by hand. (b) Graph theory: relate $P_G(3) > 0$ to 3-colorability and exhibit a recursion tree of the deletion–contraction expansion, bounding its size. (c) Computability: explain why *deciding* $P_G(3) > 0$ is NP-complete (it is 3-COLOR) while *computing* the exact value $P_G(3)$ is #P-hard, and why the $2^{|E|}$ -term inclusion–exclusion formula gives only an exponential-time exact algorithm. *Hint: deletion–contraction makes one edge disappear per step; the leaves of the recursion are edgeless graphs with $P(k) = k^{\#\text{vertices}}$.*

Bridge Problem 17. [Graph + Combinatorics + Computability] Let $\text{HAMCOUNT}(G)$ be the number of Hamiltonian cycles of a graph G . (a) Combinatorics: for K_n prove $\text{HAMCOUNT}(K_n) = (n-1)!/2$ by a circular-permutation argument, and compute it for $n = 5$. (b) Graph/algorithms: present the Bellman–Held–Karp dynamic program that decides *HAMPATH* in time $O(2^n \cdot n^2)$ by indexing subproblems on (visited vertex subset, current endpoint); count the number of DP states. (c) Computability: explain why *HAMPATH* is NP-complete and *HAMCOUNT* is #P-complete, and why the Held–Karp $\Theta(2^n n)$ state count is exponential yet far better than the $n!$ brute-force permutation count. *Hint: the DP table has one entry per (subset $S \ni s$, endpoint $v \in S$) pair, giving $\Theta(2^n n)$ states, each filled by a min/OR over predecessors.*

Bridge Problem 18. [Graph + Combinatorics + Computability] Random graphs and the first-moment method. Let $G(n, 1/2)$ be the Erdős–Rényi random graph. (a) Combinatorics/probabilistic method: compute the expected number of cliques of size k as $\binom{n}{k} 2^{-\binom{k}{2}}$ and find the threshold $k^*(n) \approx 2 \log_2 n$ at which this expectation drops below 1, giving a high-probability upper bound on the clique number. (b) Graph theory: relate this to *CLIQUE* as a graph property. (c) Computability/complexity: discuss the (conjectured) hardness of *finding* a clique of size $(1 + \epsilon) \log_2 n$ in $G(n, 1/2)$ (an open problem posed by Karp in 1976, related to the planted-clique problem) versus the fact that cliques of size nearly $2 \log_2 n$ *exist* w.h.p. (part (a) gives the matching upper bound; existence needs a second-moment argument), and connect this gap to the broader non-constructiveness theme. *Hint: $\mathbb{E}[\#k\text{-cliques}] = \binom{n}{k} 2^{-\binom{k}{2}}$; when this is $o(1)$, Markov’s inequality says no k -clique exists w.h.p.*

Bridge Problem 19. [Graph + Combinatorics + Computability] A regular language’s “graph” is its minimal DFA. Let $L = \{w \in \{0,1\}^* : w \text{ read as a binary number is } \equiv 0 \pmod{5}\}$. (a) Computability/automata: construct the DFA (a 5-state transition graph) and prove minimality via Myhill–Nerode. (b) Graph theory: view the DFA as a labeled digraph and observe it is strongly connected; compute its adjacency structure. (c) Combinatorics: using the transfer matrix $A = A_0 + A_1$ of this digraph, count the number of length- n binary strings in L via $\mathbf{s}^\top A^n \mathbf{f}$, and find the eigenvalue that governs the asymptotic density (it should approach $2^n/5$). Discuss how strong connectivity of the DFA digraph relates to Perron–Frobenius. *Hint: transitions are $q \mapsto (2q + b) \pmod{5}$ on input bit b ; the count of accepted length- n strings is the (q_0, q_{acc}) entry of $(A_0 + A_1)^n$.*

Bridge Problem 20. [Graph + Combinatorics + Computability] Tilings, the Domino problem, and matchings. (a) Graph + combinatorics: prove that the number of domino tilings (perfect matchings) of the $2 \times n$ grid graph is the Fibonacci number F_{n+1} , by a transfer-matrix recurrence on a 2-state “column profile.” (b) Combinatorics: state Kasteleyn’s theorem that perfect matchings of a planar graph can be counted in polynomial time via a Pfaffian/determinant, contrasting with the #P-hardness of the permanent (general bipartite matching count). (c) Computability: state Berger’s theorem that the *unrestricted* Wang-tile / plane-tiling problem is *undecidable*, and explain the conceptual jump from “count tilings of a finite grid (easy, transfer matrix)” to “decide tileability of the infinite plane (undecidable).” *Hint: the $2 \times n$ profile transfer matrix is $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, whose powers give Fibonacci numbers; undecidability appears only when the board is infinite and the tile set is arbitrary.*

A.4 Ten Capstone Projects

Capstone 1. SAT-to-Graph Reductions: Measuring Gadget Blow-up. The central question is how much a problem grows when it is translated through the classic NP-completeness reductions $3\text{SAT} \leq_p \text{CLIQUE} \leq_p \text{VERTEX-COVER}$ and $3\text{SAT} \leq_p \text{HAMPATH}$: theory promises polynomial size, but what is the actual constant and exponent on real instances? Theory of computability supplies the many-one reductions $\leq_p$ and the meaning of NP-completeness; graph theory supplies the target objects (clique, vertex-cover, and Hamiltonian-path gadgets) and the structural invariants of the graphs produced; combinatorics supplies the counting of vertices and edges each gadget contributes as a function of clauses and variables. Implement each reduction as an explicit gadget compiler that emits the graph in DIMACS/edge-list form, then feed both the original CNF (to a SAT solver) and the reduced graph (to a generic brute-force or ILP checker) and verify answers agree on thousands of random 3-CNF instances at varied clause-to-variable ratios. The stretch goal is to fit the empirical blow-up curves, locate the phase-transition ratio where instances are hardest, and contrast clause-driven against variable-driven growth across these reductions.

Capstone 2. Enumerating Small Graphs with Burnside and Pólya, and the Complexity of Their Properties. How many non-isomorphic graphs on n vertices have a given property, and is deciding that property easy or hard once the graph is in hand? Combinatorics enters through the Burnside lemma and Pólya enumeration applied to the action of S_n on the $\binom{n}{2}$ possible edges; graph theory enters through the properties tested (connectivity, planarity, k -colorability, Hamiltonicity); theory of computability enters by classifying each property's decision problem in P versus NP and contrasting that with the cost of the enumeration itself. Implement an orderly-generation or canonical-augmentation enumerator that lists all graphs on $n \leq 8$ vertices up to isomorphism, cross-check the totals against the Pólya cycle-index prediction, and tabulate how many realize each property. The stretch goal is to push to $n = 10$ with a nauty-style canonical labeling and empirically estimate the proportion of graphs that are Hamiltonian, comparing the trend to known asymptotic results.

Capstone 3. A Regular-Expression to Syntactic-Monoid Pipeline: Star-Free versus Counting Languages. Which regular languages can be defined without counting modulo anything, and how does that algebraic boundary show up in the structure of their automata? Theory of computability provides the regular-expression $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ syntactic-monoid pipeline and the Schützenberger characterization of star-free languages by aperiodic monoids; graph theory views each automaton as a labeled directed graph whose strongly connected components and cycle structure encode counting behavior; combinatorics counts the monoid elements, the distinct Myhill–Nerode classes, and the growth of accepted strings by length. Implement the full pipeline with Thompson construction, subset-construction determinization, Hopcroft minimization, and transition-monoid computation, then test each syntactic monoid for aperiodicity to decide star-freeness. The stretch goal is to generate random regular expressions, measure how often the language is star-free, and chart NFA-versus-DFA state blow-up against syntactic-monoid size.

Capstone 4. Estimating Ramsey Numbers by the Probabilistic Method, and the Complexity of the Search. For which n does every red/blue edge-coloring of K_n contain a monochromatic K_k , and how hard is it to find good colorings that push the lower bound? Combinatorics enters through the probabilistic method, the union bound, and the Lovász Local Lemma that together yield existence proofs and lower bounds; graph theory enters through complete graphs, their colorings, and monochromatic-clique detection; theory of computability enters through the complexity of verifying a coloring and the exponential blow-up of exhaustive search. Implement a randomized coloring generator plus a local-search/simulated-annealing solver that minimizes monochromatic K_k copies, and use it to certify lower bounds for small diagonal Ramsey numbers such as $R(3, 3)$, $R(4, 4)$, and attempts at $R(5, 5)$. The stretch goal is to derandomize one construction via the method of conditional expectations and compare its guaranteed bound against the best random trial.

Capstone 5. Generating-Function Analysis of an Algorithm, Verified Empirically. Can the running time of a recursive graph algorithm be predicted exactly from a generating function and then confirmed by measurement? Combinatorics supplies ordinary and exponential generating functions and singularity analysis to solve the recurrence that counts the algorithm's basic operations; theory of computability supplies the cost model and the distinction between worst-case and average-case complexity; graph theory supplies the input family, for instance random recursive trees or spanning-tree enumeration where the recurrence naturally lives. Implement the algorithm with an operation counter, derive the generating function for its expected cost, extract the asymptotics, and overlay the closed-form curve on measured operation counts across input sizes. The stretch goal is to repeat the analysis for a second input distribution and show how a change in the generating function's dominant singularity shifts the predicted growth from $\Theta(n \log n)$ to $\Theta(n^{3/2})$ or similar.

Capstone 6. Expander and Cayley-Graph Construction with a Derandomization Demo. How well can explicitly constructed expander graphs replace true randomness in a simple randomized algorithm? Combinatorics and algebra enter through Cayley graphs of groups such as $\mathbb{Z}/p\mathbb{Z}$ and $\text{SL}_2(\mathbb{F}_p)$ and the spectral gap that certifies expansion; graph theory enters through eigenvalues of the adjacency matrix, edge expansion, and random-walk mixing time; theory of computability enters through the role of expanders in derandomization and randomness-efficient sampling. Implement a Cayley-graph generator, compute its spectrum and second eigenvalue, and empirically measure mixing time of a random walk against the spectral-gap prediction. The stretch goal is to use a short expander walk to amplify the success probability of a randomized primality or polynomial-identity test using far fewer random bits than independent sampling, and to chart the error decay.

Capstone 7. Counting Perfect Matchings: A #P Exploration via the Permanent. Counting can be far harder than deciding: a perfect matching is found in polynomial time, yet counting all of them is NP-hard and indeed #P-complete. Graph theory enters through bipartite graphs, matchings, and Hall’s theorem linking the existence of a perfect matching to a combinatorial neighborhood condition; combinatorics enters through the permanent of the biadjacency matrix as the exact count and through inclusion–exclusion formulas; theory of computability enters through counting complexity and the contrast between the easy decision problem and the hard counting problem. Implement Hopcroft–Karp for existence, exact permanent computation by Ryser’s inclusion–exclusion formula for the count, and a Markov-chain or Godsil–Gutman estimator for approximation. The stretch goal is to benchmark the exact-versus-approximate gap on random bipartite graphs and visualize how Ryser’s $\Theta(2^n)$ cost makes the counting problem intractable well before the matching decision problem strains.

Capstone 8. A Planarity and Coloring Toolkit with Complexity Notes. Every planar graph is 4-colorable and easily 5-colorable, but 3-colorability is NP-complete even for planar graphs: how do these facts coexist in one piece of software? Graph theory enters through planarity, Kuratowski/Wagner forbidden minors, and the chromatic number; theory of computability enters through the P planarity test, the linear-time 5-coloring, and the NP-completeness of planar 3-coloring via the crossing gadget; combinatorics enters through the chromatic polynomial and the deletion–contraction recurrence that counts colorings. Implement a planarity tester, a constructive 5-coloring driven by repeatedly removing a degree- ≤ 5 vertex, and a chromatic-polynomial evaluator by deletion–contraction. The stretch goal is to add a planar 3-coloring SAT-solver front end, generate the crossing gadgets automatically, and contrast solver runtime on planar versus general instances to make the complexity gap tangible.

Capstone 9. Tree-Decompositions, Treewidth, and Fixed-Parameter Tractability. Many graph problems that are NP-hard in general become tractable when the graph is “treelike”: how does bounded treewidth turn intractability into efficient dynamic programming? Graph theory enters through tree-decompositions, the treewidth parameter, and bramble/separator duality; theory of computability enters through fixed-parameter tractability and Courcelle-style algorithms whose runtime is $O(f(k) \cdot n)$ rather than $O(n^k)$; combinatorics enters through counting independent sets or colorings by aggregating partial solutions across the bags of the decomposition. Implement a heuristic tree-decomposition builder plus a dynamic program that solves maximum independent set and counts q -colorings over the decomposition. The stretch goal is to empirically plot runtime against measured treewidth on random and structured graphs, confirming the exponential-in- k but linear-in- n behavior and locating where the heuristic decomposition degrades.

Capstone 10. Automatic Sequences, Finite Automata, and Walks on Graphs. Which integer sequences are exactly those readable off a finite automaton fed the base- b digits of n , and how do their counting statistics behave? Theory of computability enters through deterministic finite automata with output and the Cobham characterization of automatic sequences; graph theory enters by viewing the automaton as a labeled digraph and reading sequence values as outputs of walks indexed by digit strings; combinatorics enters through the transfer-matrix method, where powers of the transition matrix count walks and yield the generating function of digit-sum and pattern statistics. Implement an automaton-driven generator for sequences such as Thue–Morse and Rudin–Shapiro, build the transfer matrix, and compute occurrence counts of factors via matrix powers. The stretch goal is to test, on a long prefix, whether a candidate sequence is automatic by minimizing the conjectured automaton and checking the finiteness of its kernel, then compare the predicted and measured asymptotic density of a chosen pattern.

A.5 The Concept Map in Mermaid

The following is the Mermaid source for the integrated concept map, renderable at mermaid.live or with `mermaid-cli`.

```
%%{init: {"theme": "neutral"}}%
flowchart TB
    GT["Graph Theory"]
    SG["shared ground"]
```

```

finite structures, encoded as strings,
with decision and counting problems"]
    CB["Combinatorics"]
    TC["Theory of Computation"]

    GT <== "A. Graph + Comb.
Cayley  $n^{(n-2)}$  trees
(Prüfer bijection)" ==> CB
    GT <== "B. Graph + Comp.
3SAT  $\leq_p$  HAMPATH" ==> TC
    CB <== "C. Comb. + Comp.
countably many programs,
uncountably many languages" ==> TC
    GT <== "D. all three.
count 3-colorings  $P_G(3)$ :
deciding  $> 0$  is NP-complete,
counting is #P-hard" ==> SG
    SG <== "D" ==> CB
    SG <== "D" ==> TC

classDef field fill:#f0f0f0,stroke:#404040,color:#000;
classDef core fill:#ebebcb,stroke:#000,stroke-width:2px,color:#000,font-weight:bold;

class GT,CB,TC field;
class SG core;

```